

Building Microservices

Bản dịch song ngữ Anh-Việt · Designing Fine-Grained Systems, Second Edition

Sam Newman (nguyên tác)

2026

Mục lục

Preface — Lời nói đầu	12
Who Should Read This Book — Ai nên đọc cuốn sách này	13
Why I Wrote This Book — Vì sao tôi viết cuốn sách này	13
What's Changed Since the First Edition? — Có gì thay đổi kể từ lần xuất bản đầu tiên? . . .	14
Navigating This Book — Cách đọc cuốn sách này	15
Part I, Foundation — Phần I, Nền tảng (Foundation)	16
Part II, Implementation — Phần II, Triển khai (Implementation)	17
Part III, People — Phần III, Con người (People)	18
Conventions Used in This Book — Các quy ước được sử dụng trong cuốn sách này	18
O'Reilly Online Learning — O'Reilly Online Learning	23
How to Contact Us — Cách liên hệ với chúng tôi	23
Acknowledgments — Lời cảm ơn	24
CHAPTER 1 What Are Microservices? — Chương 1: Microservices là gì?	27
Microservices at a Glance — Tổng quan nhanh về Microservices	27
Are Service-Oriented Architecture and Microservices Different Things? — Kiến trúc hướng dịch vụ và Microservices có phải là hai thứ khác nhau?	30
Key Concepts of Microservices — Các khái niệm cốt lõi của Microservices	32
Independent Deployability — Khả năng triển khai độc lập	32
Modeled Around a Business Domain — Mô hình hóa xoay quanh một miền nghiệp vụ . . .	34
Owning Their Own State — Tự sở hữu trạng thái của riêng mình	36
Size — Kích thước	39
Flexibility — Tính linh hoạt	41
Alignment of Architecture and Organization — Sự đồng bộ giữa Kiến trúc và Tổ chức . . .	41
A Note on “Fake” Companies — Một lưu ý về các công ty “giả”	46
The Monolith — Monolith	47
The Single-Process Monolith — Single-Process Monolith	47
The Modular Monolith — Modular Monolith	49
The Distributed Monolith — Distributed Monolith	52
Monoliths and Delivery Contention — Monolith và sự tranh chấp trong chuyển giao	52
Advantages of Monoliths — Ưu điểm của Monolith	53
Enabling Technology — Công nghệ hỗ trợ	54
Log Aggregation and Distributed Tracing — Gom log và Distributed Tracing	55
Containers and Kubernetes — Container và Kubernetes	58
Streaming — Streaming	59
Public Cloud and Serverless — Đám mây công cộng và Serverless	60
Advantages of Microservices — Ưu điểm của Microservices	61
Technology Heterogeneity — Tính đa dạng công nghệ	61
Robustness — Độ vững chắc	63

Scaling — Mở rộng	64
Ease of Deployment — Dễ dàng triển khai	65
Organizational Alignment — Sự đồng bộ với tổ chức	66
Composability — Tính khả ghép	66
Microservice Pain Points — Những điểm đau của Microservice	67
Developer Experience — Trải nghiệm của lập trình viên	67
Technology Overload — Quá tải công nghệ	68
Cost — Chi phí	69
Reporting — Báo cáo	70
Monitoring and Troubleshooting — Giám sát và gỡ rối	71
Security — Bảo mật	72
Testing — Kiểm thử	72
Latency — Độ trễ	73
Data Consistency — Tính nhất quán dữ liệu	74
Should I Use Microservices? — Tôi có nên dùng Microservices không?	74
Whom They Might Not Work For — Đối tượng mà chúng có thể không phù hợp	75
Where They Work Well — Nơi chúng hoạt động tốt	77
Summary — Tóm tắt	78
CHAPTER 2 How to Model Microservices — Chương 2: Cách mô hình hóa Microservice	80
Introducing MusicCorp — Giới thiệu về MusicCorp	80
What Makes a Good Microservice Boundary? — Điều gì tạo nên một ranh giới microservice tốt?	81
Information Hiding — Che giấu thông tin (Information Hiding)	82
Cohesion — Cohesion (Tính gắn kết)	84
Coupling — Coupling (Sự phụ thuộc)	85
The Interplay of Coupling and Cohesion — Sự tương tác giữa Coupling và Cohesion	85
Types of Coupling — Các loại Coupling	87
Prior Art in Structured Programming — Thành tựu đi trước trong Lập trình có cấu trúc	87
Domain Coupling — Domain Coupling	89
A Brief Note on Temporal Coupling — Một ghi chú ngắn về Temporal Coupling	91
Pass-Through Coupling — Pass-Through Coupling	92
Common Coupling — Common Coupling	96
Content Coupling — Content Coupling	104
Just Enough Domain-Driven Design — Domain-Driven Design vừa đủ	107
Ubiquitous Language — Ubiquitous Language	108
Aggregate — Aggregate	109
Bounded Context — Bounded Context	113
Mapping Aggregates and Bounded Contexts to Microservices — Ánh xạ Aggregate và Bounded Context sang Microservice	117
Event Storming — Event Storming	119
The Case for Domain-Driven Design for Microservices — Lập luận cho Domain-Driven Design dành cho Microservice	122
Alternatives to Business Domain Boundaries — Các phương án thay thế cho ranh giới theo miền nghiệp vụ	123
Volatility — Tính dễ biến động (Volatility)	123
Data — Dữ liệu	125
Technology — Công nghệ	127
Organizational — Tổ chức	129
Layering Inside Versus Layering Outside — Phân lớp bên trong so với Phân lớp bên ngoài	132

Mixing Models and Exceptions — Pha trộn các mô hình và những ngoại lệ	133
Summary — Tóm tắt	134
CHAPTER 3 Splitting the Monolith — Chương 3: Tách rời khối Monolith	136
Have a Goal — Hãy có một mục tiêu	136
Incremental Migration — Di trú từng bước	139
The Monolith Is Rarely the Enemy — Khối monolith hiếm khi là kẻ thù	141
The Dangers of Premature Decomposition — Những hiểm họa của việc phân rã quá sớm	142
What to Split First? — Tách cái gì trước?	143
Decomposition by Layer — Phân rã theo tầng	145
Code First — Mã trước	148
Data First — Dữ liệu trước	149
Useful Decompositional Patterns — Những pattern phân rã hữu ích	151
Strangler Fig Pattern — Pattern Strangler Fig (cây đa búp nghệt)	151
Parallel Run — Chạy song song (Parallel Run)	152
Feature Toggle — Feature Toggle	153
Data Decomposition Concerns — Những mối quan ngại khi phân rã dữ liệu	153
Performance — Hiệu năng	154
Data Integrity — Toàn vẹn dữ liệu	157
Transactions — Giao dịch	158
Tooling — Công cụ (Tooling)	159
Reporting Database — Cơ sở dữ liệu báo cáo (Reporting Database)	159
Summary — Tóm tắt	161
CHAPTER 4 Microservice Communication Styles — Chương 4: Các phong cách giao tiếp giữa Microservice	162
From In-Process to Inter-Process — Từ trong tiến trình đến giữa các tiến trình	162
Performance — Hiệu năng	163
Changing Interfaces — Thay đổi giao diện (Interface)	164
Error Handling — Xử lý lỗi (Error Handling)	165
Technology for Inter-Process Communication: So Many Choices — Công nghệ cho giao tiếp giữa các tiến trình: Quá nhiều lựa chọn	167
Styles of Microservice Communication — Các phong cách giao tiếp giữa Microservice	168
Mix and Match — Pha trộn và kết hợp	170
Pattern: Synchronous Blocking — Pattern: Đồng bộ có chặn	171
Advantages — Ưu điểm	171
Disadvantages — Nhược điểm	172
Where to Use It — Khi nào nên dùng	173
Pattern: Asynchronous Nonblocking — Pattern: Bất đồng bộ không chặn	176
Advantages — Ưu điểm	176
Disadvantages — Nhược điểm	178
Async/Await and When Asynchronous Is Still Blocking — Async/Await và khi nào Bất đồng bộ vẫn là Có chặn	178
Where to Use It — Khi nào nên dùng	180
Pattern: Communication Through Common Data — Pattern: Giao tiếp qua dữ liệu dùng chung	180
Implementation — Cách hiện thực	181
Advantages — Ưu điểm	183
Disadvantages — Nhược điểm	183
Where to Use It — Khi nào nên dùng	184
Pattern: Request-Response Communication — Pattern: Giao tiếp Request-Response	185
Commands Versus Requests — Lệnh (Command) so với Yêu cầu (Request)	186

Implementation: Synchronous Versus Asynchronous — Cách hiện thực: Đồng bộ so với Bất đồng bộ	187
Parallel Versus Sequential Calls — Lời gọi Song song so với Tuần tự	189
Where to Use It — Khi nào nên dùng	190
Pattern: Event-Driven Communication — Pattern: Giao tiếp Hướng sự kiện	190
Events and Messages — Sự kiện và Thông điệp	193
Implementation — Cách hiện thực	194
What's in an Event? — Bên trong một Sự kiện có gì?	196
Where to Use It — Khi nào nên dùng	201
Proceed with Caution — Hãy thận trọng khi tiến hành	202
Summary — Tóm tắt	204

CHAPTER 5 Implementing Microservice Communication — Chương 5: Hiện thực giao tiếp giữa các Microservice

	205
Looking for the Ideal Technology — Đi tìm công nghệ lý tưởng	205
Make Backward Compatibility Easy — Làm cho khả năng tương thích ngược trở nên dễ dàng	205
Make Your Interface Explicit — Làm cho interface của bạn tường minh	206
Keep Your APIs Technology Agnostic — Giữ cho API của bạn không phụ thuộc công nghệ . .	206
Make Your Service Simple for Consumers — Làm cho dịch vụ của bạn đơn giản với consumer	207
Hide Internal Implementation Detail — Ẩn đi chi tiết hiện thực nội bộ	207
Technology Choices — Các lựa chọn công nghệ	208
Remote Procedure Calls — Lời gọi thủ tục từ xa (Remote Procedure Calls)	208
REST — REST	214
GraphQL — GraphQL	222
Message Brokers — Message Broker	225
Serialization Formats — Các định dạng tuần tự hóa	232
Textual Formats — Các định dạng văn bản	233
Binary Formats — Các định dạng nhị phân	234
Schemas — Schema	235
Structural Versus Semantic Contract Breakages — Phá vỡ hợp đồng kiểu cấu trúc và kiểu ngữ nghĩa	236
Should You Use Schemas? — Bạn có nên dùng Schema không?	237
Handling Change Between Microservices — Xử lý thay đổi giữa các Microservice	238
Avoiding Breaking Changes — Tránh các thay đổi gây phá vỡ	239
Expansion Changes — Các thay đổi mở rộng	240
Tolerant Reader — Tolerant Reader	240
Right Technology — Công nghệ phù hợp	241
Explicit Interface — Interface tường minh	242
Semantic Versioning — Đánh phiên bản theo ngữ nghĩa (Semantic Versioning)	243
Catch Accidental Breaking Changes Early — Bắt sớm các thay đổi gây phá vỡ vô tình	244
Managing Breaking Changes — Quản lý các thay đổi gây phá vỡ	245
Lockstep Deployment — Triển khai đồng bộ bước	246
Coexist Incompatible Microservice Versions — Cho các phiên bản microservice không tương thích cùng tồn tại	246
Emulate the Old Interface — Mô phỏng interface cũ	249
Which Approach Do I Prefer? — Tôi thích cách tiếp cận nào?	251
The Social Contract — Hợp đồng xã hội	252
Tracking Usage — Theo dõi việc sử dụng	253
Extreme Measures — Các biện pháp cực đoan	254

DRY and the Perils of Code Reuse in a Microservice World — DRY và những hiểm họa của tái sử dụng mã trong thế giới Microservice	255
Sharing Code via Libraries — Chia sẻ mã qua thư viện	256
Service Discovery — Service Discovery	258
Domain Name System (DNS) — Domain Name System (DNS)	259
Dynamic Service Registries — Các sổ đăng ký dịch vụ động (Dynamic Service Registries)	262
Don't Forget the Humans! — Đừng quên con người!	266
Service Meshes and API Gateways — Service Mesh và API Gateway	266
API Gateways — API Gateway	268
Service Meshes — Service Mesh	272
What About Other Protocols? — Còn các giao thức khác thì sao?	278
Documenting Services — Lập tài liệu cho dịch vụ	278
Explicit Schemas — Schema tường minh	279
The Self-Describing System — Hệ thống tự mô tả	280
Summary — Tóm tắt	283
CHAPTER 6 Workflow — Chương 6: Workflow	286
Database Transactions — Giao dịch cơ sở dữ liệu	286
ACID Transactions — Giao dịch ACID	287
Still ACID, but Lacking Atomicity? — Vẫn là ACID, nhưng thiếu tính nguyên tử?	288
Distributed Transactions—Two-Phase Commits — Giao dịch phân tán—Two-Phase Commit	291
Distributed Transactions—Just Say No — Giao dịch phân tán—Hãy cứ nói không	294
Database Distributed Transactions — Giao dịch phân tán cơ sở dữ liệu	295
Sagas — Saga	296
Saga Failure Modes — Các chế độ lỗi của Saga	301
Implementing Sagas — Triển khai Saga	308
BPM Tools — Công cụ BPM	312
Tracing Calls — Truy vết các lời gọi	317
Sagas Versus Distributed Transactions — Saga so với Giao dịch phân tán	318
Summary — Tóm tắt	319
CHAPTER 7 Build — Chương 7: Build	321
A Brief Introduction to Continuous Integration — Giới thiệu ngắn gọn về Tích hợp liên tục (Continuous Integration)	321
Are You Really Doing CI? — Bạn có thực sự đang làm CI không?	323
Branching Models — Các mô hình phân nhánh (Branching Models)	324
Build Pipelines and Continuous Delivery — Build Pipeline và Continuous Delivery	327
Continuous Delivery Versus Continuous Deployment — Continuous Delivery so với Continuous Deployment	330
Tooling — Công cụ	330
Trade-Offs and Environments — Đánh đổi và Môi trường	331
Artifact Creation — Tạo Artifact	332
Mapping Source Code and Builds to Microservices — Ánh xạ Mã nguồn và các bản Build với Microservices	336
One Giant Repo, One Giant Build — Một Repo khổng lồ, một Build khổng lồ	336
Pattern: One Repository per Microservice (aka Multirepo) — Mẫu: Mỗi Microservice một Repository (còn gọi là Multirepo)	338
Pattern: Monorepo — Mẫu: Monorepo	345
In this example, @doctocat owns any files in the build/logs # directory at the root of the repository and any of its # subdirectories. /build/logs/ @doctocat —	352

In this example, @octocat owns any file in an apps directory # anywhere in your repository. apps/ @octocat — 353

In this example, @doctocat owns any file in the /docs # directory in the root of your repository. /docs/ @doctocat — In this example, @doctocat owns any file in the /docs # directory in the root of your repository. /docs/ @doctocat 354

Which Approach Would I Use? — Tôi sẽ dùng cách tiếp cận nào? 358
Summary — Tóm tắt 359

CHAPTER 8 Deployment — Chương 8: Triển khai 360

From Logical to Physical — Từ Logic đến Vật lý	360
Multiple Instances — Nhiều Instance	361
The Database — Cơ sở Dữ liệu	363
Environments — Môi trường	370
Principles of Microservice Deployment — Nguyên tắc Triển khai Microservice	374
Isolated Execution — Thực thi Cô lập	375
Focus on Automation — Tập trung vào Tự động hóa	380
Infrastructure as Code (IAC) — Infrastructure as Code (IAC)	382
Zero-Downtime Deployment — Triển khai Không Gián đoạn	383
Desired State Management — Quản lý Trạng thái Mong muốn	385
Deployment Options — Các Lựa chọn Triển khai	389
Physical Machines — Máy Vật lý	391
Virtual Machines — Máy Ảo	392
Containers — Container	395
Blurred Lines — Ranh giới Mờ nhạt	402
Application Containers — Application Container	404
Platform as a Service (PaaS) — Platform as a Service (PaaS)	407
Function as a Service (FaaS) — Function as a Service (FaaS)	408
WebAssembly (Wasm) — WebAssembly (Wasm)	411
Which Deployment Option Is Right for You? — Lựa chọn Triển khai Nào Phù hợp với Bạn?	420
Role for Puppet, Chef, and Other Tools? — Vai trò của Puppet, Chef và Các Công cụ Khác?	423
Kubernetes and Container Orchestration — Kubernetes và Điều phối Container	424
The Case for Container Orchestration — Lý do cho Điều phối Container	425
A Simplified View of Kubernetes Concepts — Một Góc nhìn Đơn giản hóa về Các Khái niệm Kubernetes	426
Multitenancy and Federation — Đa người thuê (Multitenancy) và Liên kết (Federation)	429
The Background Behind Kubernetes — Bối cảnh Đằng sau Kubernetes	432
The Cloud Native Computing Federation — Cloud Native Computing Federation	433
Platforms and Portability — Nền tảng và Tính Di động	435
Helm, Operators, and CRDs, Oh My! — Helm, Operator và CRD, Ôi Trời!	436
And Knative — Và Knative	437
The Future — Tương lai	438
Should You Use It? — Bạn Có Nên Dùng Nó?	439
Progressive Delivery — Chuyển giao Tăng tiến (Progressive Delivery)	440
Separating Deployment from Release — Tách Triển khai khỏi Phát hành	441
On to Progressive Delivery — Tiến tới Chuyển giao Tăng tiến	442
Feature Toggles — Feature Toggle	443
Canary Release — Canary Release	444
Parallel Run — Parallel Run	445
Summary — Tóm tắt	448

CHAPTER 9 Testing — Chương 9: Kiểm thử	450
Types of Tests — Các loại kiểm thử	451
Manual Exploratory Testing — Kiểm thử khám phá thủ công (Manual Exploratory Testing) .	453
Test Scope — Phạm vi kiểm thử (Test Scope)	454
Unit Tests — Unit Test	456
Service Tests — Service Test	458
End-to-End Tests — End-to-End Test	459
What About Integration Tests? — Còn kiểm thử tích hợp (Integration Tests) thì sao?	460
Trade-Offs — Đánh đổi (Trade-Offs)	460
Implementing Service Tests — Triển khai Service Test	462
Mocking or Stubbing — Mocking hay Stubbing	463
A Smarter Stub Service — Một dịch vụ Stub thông minh hơn	464
Implementing (Those Tricky) End-to-End Tests — Triển khai (Những) End-to-End Test (gai góc đó)	465
Flaky and Brittle Tests — Kiểm thử chập chờn và giòn (Flaky and Brittle Tests)	467
Who Writes These End-to-End Tests? — Ai viết những End-to-End Test này?	469
How Long Should End-to-End Tests Run? — End-to-End Test nên chạy trong bao lâu?	472
The Great Pile-Up — Sự dồn ứ lớn (The Great Pile-Up)	473
The Metaversion — Siêu phiên bản (The Metaversion)	474
Lack of Independent Testability — Thiếu khả năng kiểm thử độc lập (Lack of Independent Testability)	475
Should You Avoid End-to-End Tests? — Bạn có nên tránh End-to-End Test không?	475
Contract Tests and Consumer-Driven Contracts (CDCs) — Contract Test và Consumer-Driven Contracts (CDCs)	476
The Final Word — Lời cuối (The Final Word)	481
Developer Experience — Trải nghiệm của lập trình viên (Developer Experience)	482
From Preproduction to In-Production Testing — Từ kiểm thử tiên-production đến kiểm thử trên production	484
Types of In-Production Testing — Các loại kiểm thử trên production	485
Making Testing in Production Safe — Làm cho kiểm thử trên production trở nên an toàn . .	486
Mean Time to Repair over Mean Time Between Failures? — Thời gian trung bình để sửa chữa hơn là Thời gian trung bình giữa các lần thất bại?	487
Cross-Functional Testing — Kiểm thử liên-chức năng (Cross-Functional Testing)	488
Performance Tests — Kiểm thử hiệu năng (Performance Tests)	490
Robustness Tests — Kiểm thử độ vững chắc (Robustness Tests)	492
Summary — Tóm tắt	493
CHAPTER 10 From Monitoring to Observability — Chương 10: Từ Giám sát đến Khả năng quan sát (From Monitoring to Observability)	495
Disruption, Panic, and Confusion — Gián đoạn, Hoảng loạn và Bối rối	496
Single Microservice, Single Server — Một Microservice, Một Máy chủ	498
Single Microservice, Multiple Servers — Một Microservice, Nhiều Máy chủ	500
Multiple Services, Multiple Servers — Nhiều Dịch vụ, Nhiều Máy chủ	502
Observability Versus Monitoring — Khả năng quan sát so với Giám sát (Observability Versus Monitoring)	504
The Pillars of Observability? Not So Fast — Các Trụ cột của Khả năng quan sát? Khoan đã .	505
Building Blocks for Observability — Các Khối xây dựng cho Khả năng quan sát	507
Log Aggregation — Gom log (Log Aggregation)	509
Metrics Aggregation — Tổng hợp Metric (Metrics Aggregation)	524

Monitoring and Observability Systems Are Production Systems — Hệ thống Giám sát và Khả năng quan sát là các Hệ thống Production	528
Distributed Tracing — Distributed Tracing	528
Are We Doing OK? — Chúng ta Đang Ổn chứ? (Are We Doing OK?)	532
Alerting — Cảnh báo (Alerting)	536
Alarm Versus Alert — Báo động so với Cảnh báo (Alarm Versus Alert)	539
Semantic Monitoring — Semantic Monitoring	541
Testing in Production — Kiểm thử trong Production (Testing in Production)	544
Standardization — Chuẩn hóa (Standardization)	548
Selecting Tools — Lựa chọn Công cụ (Selecting Tools)	549
Democratic — Mang tính Dân chủ (Democratic)	549
Easy to Integrate — Dễ Tích hợp (Easy to Integrate)	550
Provide Context — Cung cấp Bối cảnh (Provide Context)	550
Real-Time — Thời gian thực (Real-Time)	551
Suitable for Your Scale — Phù hợp với Quy mô của Bạn (Suitable for Your Scale)	551
The Expert in the Machine — Chuyên gia trong Cổ máy (The Expert in the Machine)	552
Getting Started — Bắt đầu (Getting Started)	554
Summary — Tóm tắt (Summary)	555
CHAPTER 11 Security — Chương 11: Bảo mật	557
Core Principles — Các nguyên tắc cốt lõi	559
Principle of Least Privilege — Nguyên tắc Đặc quyền Tối thiểu	559
Defense in Depth — Phòng thủ nhiều lớp	560
Types of Security Controls — Các loại kiểm soát bảo mật	561
Automation — Tự động hóa	562
Build Security into the Delivery Process — Đưa bảo mật vào quy trình chuyển giao	563
The Five Functions of Cybersecurity — Năm chức năng của an ninh mạng	564
Identify — Nhận diện	565
Protect — Bảo vệ	568
Detect — Phát hiện	568
Respond — Ứng phó	568
Recover — Phục hồi	569
Foundations of Application Security — Nền tảng của bảo mật ứng dụng	570
Credentials — Credential	570
Scanning for Keys — Quét tìm Khóa	576
Patching — Vá lỗi	580
Backups — Sao lưu	583
Rebuild — Dựng lại	586
Implicit Trust Versus Zero Trust — Tin cậy ngầm định và zero trust	590
Implicit Trust — Tin cậy ngầm định	590
Zero Trust — Zero trust	591
It's a Spectrum — Đó là một dải quang phổ	592
Securing Data — Bảo vệ dữ liệu	595
Data in Transit — Dữ liệu khi truyền	595
Data at Rest — Dữ liệu khi nghỉ	599
Authentication and Authorization — Xác thực và phân quyền	605
Service-to-Service Authentication — Xác thực giữa dịch vụ với dịch vụ	606
Human Authentication — Xác thực con người	606
Common Single Sign-On Implementations — Các cách triển khai Single Sign-On phổ biến	607
Single Sign-On Gateway — Cổng Single Sign-On	608

Fine-Grained Authorization — Phân quyền chi tiết	611
The Confused Deputy Problem — Vấn đề Phó tướng Bối rối	613
Centralized, Upstream Authorization — Phân quyền tập trung, thượng nguồn	615
Decentralizing Authorization — Phi tập trung hóa phân quyền	616
JSON Web Tokens — JSON Web Token	616
Summary — Tóm tắt	622
CHAPTER 12 Resiliency — Chương 12: Khả năng phục hồi (Resiliency)	623
What Is Resiliency? — Khả năng phục hồi là gì?	624
Robustness — Độ vững chắc (Robustness)	625
Rebound — Bật lại (Rebound)	626
Graceful Extensibility — Khả năng mở rộng có kiểm soát (Graceful Extensibility)	627
Sustained Adaptability — Khả năng thích ứng bền vững (Sustained Adaptability)	628
And Microservice Architecture — Và kiến trúc microservice	629
Failure Is Everywhere — Sự cố ở khắp mọi nơi	630
How Much Is Too Much? — Bao nhiêu là quá nhiều?	631
Degrading Functionality — Suy giảm chức năng	633
Stability Patterns — Các pattern ổn định	635
Time-Outs — Time-out	639
Retries — Retry	644
Bulkheads — Bulkhead	645
Circuit Breakers — Circuit breaker	647
Isolation — Cô lập (Isolation)	651
Redundancy — Dự phòng (Redundancy)	653
Middleware — Middleware	653
Idempotency — Tính idempotent (Idempotency)	654
Spreading Your Risk — Phân tán rủi ro của bạn	656
CAP Theorem — Định lý CAP	658
Sacrificing Consistency — Hy sinh tính nhất quán	660
Sacrificing Availability — Hy sinh tính sẵn sàng	660
Sacrificing Partition Tolerance? — Hy sinh khả năng chịu phân vùng?	662
AP or CP? — AP hay CP?	662
It's Not All or Nothing — Không phải tất-cả-hoặc-không-có-gì	663
And the Real World — Và thế giới thực	664
Antifragility — Tính kháng vỡ (Antifragility)	665
Chaos Engineering — Chaos engineering	665
Game Days — Game Days	666
Production Experiments — Thực nghiệm trên môi trường sản xuất	667
From Robustness to Beyond — Từ độ vững chắc đến xa hơn	668
Blame — Đổ lỗi	669
Summary — Tóm tắt	671
CHAPTER 13 Scaling — Chương 13: Mở rộng	673
The Four Axes of Scaling — Bốn trục của việc mở rộng	673
Vertical Scaling — Mở rộng dọc	675
Horizontal Duplication — Nhân bản theo chiều ngang	678
Data Partitioning — Phân vùng dữ liệu	683
Functional Decomposition — Phân rã theo chức năng	690
Combining Models — Kết hợp các mô hình	693
Start Small — Bắt đầu từ nhỏ	696
CQRS and Event Sourcing — CQRS và Event Sourcing	697

Caching — Caching	698
For Performance — Vì hiệu năng	699
For Scale — Vì mở rộng	700
For Robustness — Vì độ vững chắc	700
Where to Cache — Cache ở đâu	701
Invalidation — Vô hiệu hóa	708
The Golden Rule of Caching — Quy tắc vàng của caching	715
Freshness Versus Optimization — Độ tươi mới so với Tối ưu	718
Cache Poisoning: A Cautionary Tale — Cache Poisoning: Một câu chuyện cảnh tỉnh	718
Autoscaling — Autoscaling	720
Starting Again — Bắt đầu lại từ đầu	722
Summary — Tóm tắt	723
CHAPTER 14 User Interfaces — Chương 14: Giao diện người dùng	724
Toward Digital — Hướng tới kỹ thuật số (Digital)	725
Ownership Models — Các mô hình quyền sở hữu (Ownership Models)	726
Drivers for Dedicated Frontend Teams — Các động lực dẫn tới đội frontend chuyên trách	728
Toward Stream-Aligned Teams — Hướng tới các đội stream-aligned	729
Sharing Specialists — Chia sẻ chuyên gia	731
Ensuring Consistency — Đảm bảo tính nhất quán	733
Working Through Technical Challenges — Vượt qua các thách thức kỹ thuật	735
Pattern: Monolithic Frontend — Pattern: Frontend nguyên khối (Monolithic Frontend)	736
When to Use It — Khi nào nên dùng	737
Pattern: Micro Frontends — Pattern: Micro Frontend	738
Implementation — Hiện thực	739
When to Use It — Khi nào nên dùng	739
Self-Contained Systems — Self-Contained Systems	739
Pattern: Page-Based Decomposition — Pattern: Phân rã theo trang (Page-Based Decomposition)	741
The Web — Web	743
Where to Use It — Khi nào nên dùng	743
Pattern: Widget-Based Decomposition — Pattern: Phân rã theo widget (Widget-Based Decomposition)	744
Implementation — Hiện thực	747
Web Components — Web Components	750
When to Use It — Khi nào nên dùng	750
Constraints — Các ràng buộc (Constraints)	751
Pattern: Central Aggregating Gateway — Pattern: Gateway gộp trung tâm (Central Aggregating Gateway)	753
Ownership — Quyền sở hữu	755
Different Types of User Interfaces — Các loại giao diện người dùng khác nhau	756
Multiple Concerns — Nhiều mối quan tâm (Multiple Concerns)	759
When to Use It — Khi nào nên dùng	760
Pattern: Backend for Frontend (BFF) — Pattern: Backend for Frontend (BFF)	761
How Many BFFs? — Bao nhiêu BFF?	762
Reuse and BFFs — Tái sử dụng và BFF	765
BFFs for Desktop Web and Beyond — BFF cho web desktop và hơn thế nữa	770
When to Use — Khi nào nên dùng	771
GraphQL — GraphQL	772
A Hybrid Approach — Một cách tiếp cận lai (Hybrid)	774
Summary — Tóm tắt	774

CHAPTER 15 Organizational Structures — Chương 15: Cấu trúc tổ chức	776
Loosely Coupled Organizations — Tổ chức có coupling lỏng	776
Conway's Law — Định luật Conway	778
Evidence — Bằng chứng	779
Team Size — Quy mô nhóm	781
Understanding Conway's Law — Hiểu Định luật Conway	783
Small Teams, Large Organization — Nhóm nhỏ, tổ chức lớn	784
On Autonomy — Về tính tự chủ	786
Strong Versus Collective Ownership — Quyền sở hữu mạnh so với quyền sở hữu tập thể	788
Strong Ownership — Quyền sở hữu mạnh	789
Collective Ownership — Quyền sở hữu tập thể	791
At a Team Level Versus an Organizational Level — Ở cấp độ nhóm so với ở cấp độ tổ chức	792
Balancing Models — Cân bằng các mô hình	793
Enabling Teams — Các nhóm hỗ trợ (Enabling Teams)	794
Communities of Practice — Cộng đồng thực hành (Communities of Practice)	797
The Platform — Nền tảng (The Platform)	798
Shared Microservices — Microservice dùng chung	802
Too Hard to Split — Quá khó để tách	803
Cross-Cutting Changes — Các thay đổi cắt ngang	803
Delivery Bottlenecks — Nút thắt cổ chai trong chuyển giao	804
Internal Open Source — Mã nguồn mở nội bộ	806
Role of the Core Committers — Vai trò của các core committer	806
Maturity — Độ trưởng thành	807
Tooling — Công cụ	808
Pluggable, Modular Microservices — Microservice có thể cắm vào, mô-đun hóa	808
Change Reviews — Review các thay đổi	813
The Orphaned Service — Dịch vụ mồ côi (The Orphaned Service)	818
Case Study: realestate.com.au — Nghiên cứu tình huống: realestate.com.au	820
Geographical Distribution — Phân tán địa lý	822
Conway's Law in Reverse — Định luật Conway theo chiều ngược lại	824
People — Con người	826
Summary — Tóm tắt	827
CHAPTER 16 The Evolutionary Architect — Chương 16: Kiến trúc sư tiến hóa	829
What's in a Name? — Tên gọi có ý nghĩa gì?	829
What Is Software Architecture? — Kiến trúc phần mềm là gì?	832
Making Change Possible — Làm cho thay đổi trở nên khả thi	834
An Evolutionary Vision for the Architect — Một tầm nhìn tiến hóa cho kiến trúc sư	836
Defining System Boundaries — Định nghĩa ranh giới hệ thống	837
A Social Construct — Một kiến tạo xã hội	841
Habitability — Khả năng cư ngụ	843
A Principled Approach — Một cách tiếp cận dựa trên nguyên tắc	845
Strategic Goals — Mục tiêu chiến lược	845
Principles — Nguyên tắc	846
Practices — Tập quán	847
Combining Principles and Practices — Kết hợp nguyên tắc và tập quán	847
A Real-World Example — Một ví dụ thực tế	848
Guiding an Evolutionary Architecture — Dẫn dắt một kiến trúc tiến hóa	849
Architecture in a Stream-Aligned Organization — Kiến trúc trong một tổ chức theo hướng dòng chảy	851

Building a Team — Xây dựng một đội ngũ	855
The Required Standard — Tiêu chuẩn bắt buộc	855
Monitoring — Giám sát	856
Interfaces — Interface	857
Architectural Safety — An toàn kiến trúc	857
Governance and the Paved Road — Governance và con đường đã lát	858
Exemplars — Mẫu hình tham chiếu	859
Tailored Microservice Template — Mẫu microservice may đo	860
The Paved Road at Scale — Con đường đã lát ở quy mô lớn	862
Technical Debt — Nợ kỹ thuật	863
Exception Handling — Xử lý ngoại lệ	864
Summary — Tóm tắt	864

Afterword: Bringing It All Together — Lời kết: Kết nối tất cả lại với nhau	866
What Are Microservices? — Microservice là gì?	866
Moving to Microservices — Chuyển sang Microservice	868
Communication Styles — Các Phong cách Giao tiếp	868
Workflow — Workflow	871
Build — Build	872
Deployment — Triển khai	872
Testing — Kiểm thử	873
Monitoring and Observability — Giám sát và Khả năng quan sát (Monitoring and Observability)	874
Security — Bảo mật	875
Resiliency — Khả năng phục hồi (Resiliency)	876
Scaling — Mở rộng (Scaling)	877
User Interfaces — Giao diện người dùng	877
Organization — Tổ chức	878
Architecture — Kiến trúc	879
Further Reading	880
Looking Forward	880
Final Words	881

Preface — Lời nói đầu

Microservices are an approach to distributed systems that promote the use of finely grained services that can be changed, deployed, and released independently. For organizations that are moving toward more loosely coupled systems, with autonomous teams delivering user-facing functionality, microservices work incredibly well. Beyond this, microservices provide us with a huge number of options for building out systems, giving us a lot of flexibility to ensure our system can change to meet the needs of the users.

Microservices là một cách tiếp cận đối với hệ thống phân tán (distributed system), khuyến khích việc sử dụng các service (dịch vụ) chia nhỏ tinh vi (finely grained) có thể được thay đổi, triển khai (deploy) và phát hành (release) một cách độc lập. Đối với những tổ chức đang chuyển dịch sang các hệ thống có coupling lỏng hơn, với những đội nhóm tự chủ tạo ra chức năng hướng tới người dùng, microservices vận hành cực kỳ hiệu quả. Hơn thế nữa, microservices mang đến cho chúng ta vô số lựa chọn để xây dựng hệ thống, cho ta rất nhiều sự linh hoạt để bảo đảm rằng hệ thống có thể thay đổi nhằm đáp ứng nhu cầu của người dùng.

Microservices aren't without significant downsides, though. As a **distributed system**, they bring a host of complexity, much of which may be new even to experienced developers.

Tuy nhiên, microservices không phải không có những nhược điểm đáng kể. Là một hệ thống phân tán, chúng kéo theo hàng loạt sự phức tạp, phần lớn trong số đó có thể còn mới mẻ ngay cả với những lập trình viên dày dạn kinh nghiệm.

The experiences of people all over the world, along with the emergence of new technologies, are having a profound effect on how microservices are used. This book brings these ideas together, along with concrete, real-world examples, to help you understand whether microservices are right for you.

Trải nghiệm của mọi người trên khắp thế giới, cùng với sự xuất hiện của các công nghệ mới, đang tạo ra ảnh hưởng sâu sắc đến cách microservices được sử dụng. Cuốn sách này tập hợp những ý tưởng đó lại với nhau, kèm theo các ví dụ cụ thể từ thực tế, để giúp bạn hiểu liệu microservices có phù hợp với mình hay không.

Who Should Read This Book — Ai nên đọc cuốn sách này

The scope of Building Microservices is broad, as the implications of **microservice** architectures are also broad. As such, this book should appeal to people interested in aspects of design, development, **deployment**, **testing**, and maintenance of systems. Those of you who have already embarked on the journey toward finer-grained architectures, whether for a greenfield application or as part of decomposing an existing, more monolithic system, will find plenty of practical advice to help you. This book will also help those of you who want to know what all the fuss is about, so that you can determine whether microservices are right for you.

Phạm vi của Building Microservices rất rộng, bởi vì những hệ quả của kiến trúc microservice (microservice architecture) cũng rất rộng. Do đó, cuốn sách này nên hấp dẫn những ai quan tâm đến các khía cạnh thiết kế, phát triển, triển khai (deployment), kiểm thử (testing) và bảo trì hệ thống. Những bạn đã bắt đầu hành trình hướng tới các kiến trúc chia nhỏ tinh vi hơn—dù là cho một ứng dụng greenfield (xây mới từ đầu) hay như một phần của việc phân rã một hệ thống monolith hiện hữu—sẽ tìm thấy rất nhiều lời khuyên thực tế hữu ích. Cuốn sách này cũng sẽ giúp những bạn muốn biết tất cả những ồn ào xung quanh microservices rốt cuộc là về cái gì, để bạn có thể tự xác định liệu microservices có phù hợp với mình hay không.

Why I Wrote This Book — Vì sao tôi viết cuốn sách này

At one level, I wrote this book because I wanted to make sure that the information in the first edition remained up to date, accurate, and useful. I wrote the first edition because there were really interesting ideas that I wanted to share. I was fortunate that I was in a place where I had the time and support to write the first edition, and that I could do so from a fairly unbiased viewpoint because I didn't work for a big technology vendor. I wasn't selling a solution, and I hoped I wasn't selling microservices either—I just found the ideas fascinating, and I loved unpacking the concept of microservices and finding ways of sharing it more broadly.

Ở một mức độ, tôi viết cuốn sách này vì tôi muốn bảo đảm rằng thông tin trong lần xuất bản đầu tiên vẫn cập nhật, chính xác và hữu ích. Tôi viết phiên bản đầu tiên vì có những ý tưởng thực sự thú vị mà tôi muốn chia sẻ. Tôi may mắn ở vào một vị trí mà tôi có thời gian và sự hỗ trợ để viết phiên bản đầu, và tôi có thể viết từ một góc nhìn khá khách quan vì tôi không làm việc cho một hãng công nghệ lớn nào. Tôi không bán một giải pháp, và

tôi hy vọng mình cũng không bán cả microservices—tôi chỉ thấy những ý tưởng này hấp dẫn, và tôi thích mổ xẻ khái niệm microservices rồi tìm cách chia sẻ nó rộng rãi hơn.

When I really break it down, I wrote a second edition for two main reasons. Firstly, I felt I could do a better job this time. I've learned more, and I am hopefully a bit better as a writer. But I also wrote this second edition because I played a small part in helping these ideas hit the mainstream, and as such I have a duty of care to make sure they are presented in a sensible, balanced way. Microservices have become, for many, the default architectural choice. This is something that I think is hard to justify, and I wanted a chance to share why.

Khi thực sự phân tích kỹ, tôi viết phiên bản thứ hai vì hai lý do chính. Thứ nhất, tôi cảm thấy lần này mình có thể làm tốt hơn. Tôi đã học được nhiều hơn, và hy vọng tôi cũng đã khá hơn một chút với tư cách người viết. Nhưng tôi còn viết phiên bản thứ hai này vì tôi đã góp một phần nhỏ trong việc giúp những ý tưởng này trở nên phổ biến rộng rãi, và vì thế tôi có trách nhiệm phải bảo đảm rằng chúng được trình bày một cách hợp lý, cân bằng. Với nhiều người, microservices đã trở thành lựa chọn kiến trúc mặc định. Đây là điều mà tôi cho rằng khó biện minh, và tôi muốn có cơ hội chia sẻ vì sao.

This book isn't pro microservices, nor is it anti microservices. I just want to make sure that I've properly explored the context in which these ideas work well and shared the problems they can cause.

Cuốn sách này không cổ vũ microservices, cũng không chống lại microservices. Tôi chỉ muốn bảo đảm rằng mình đã khám phá thấu đáo bối cảnh mà những ý tưởng này phát huy tốt, đồng thời chia sẻ những vấn đề mà chúng có thể gây ra.

What's Changed Since the First Edition? — Có gì thay đổi kể từ lần xuất bản đầu tiên?

I wrote the first edition of Building Microservices in around a year, starting at the beginning of 2014, with the book being released in February 2015. This was early in the microservices story, at least in terms of the wider industry's awareness of the term. Since then, microservices have gone mainstream in a way that I couldn't have predicted. With that growth has come a much wider set of experiences to draw on, and more technology to explore.

Tôi viết phiên bản đầu của Building Microservices trong khoảng một năm, bắt đầu từ đầu năm 2014, và cuốn sách được phát hành vào tháng 2 năm 2015. Đó là giai đoạn sớm trong câu chuyện về microservices, ít nhất là xét theo mức độ nhận biết của ngành công nghiệp rộng lớn về thuật ngữ này. Kể từ đó, microservices đã trở nên phổ biến theo một cách mà tôi không thể nào dự đoán được. Cùng với sự phát triển ấy là một tập hợp trải nghiệm rộng lớn hơn nhiều để rút ra, và nhiều công nghệ hơn để khám phá.

As I worked with more teams in the wake of the first edition, I started to refine my thinking around some of the ideas associated with microservices. In some cases, this meant that ideas that were only at the periphery of my thinking, like **information hiding**, started to become more clear as foundational concepts that needed to be better highlighted. In other areas, new technology presented both new solutions and new problems for our systems. Seeing so many people flock to **Kubernetes** in the hope that it would solve all their issues with microservice architectures certainly gave me pause for thought.

Khi tôi làm việc với nhiều đội nhóm hơn sau khi phiên bản đầu ra đời, tôi bắt đầu tinh chỉnh tư duy của mình quanh một số ý tưởng gắn với microservices. Trong một số trường hợp, điều này có nghĩa là những ý tưởng vốn chỉ nằm ở rìa suy nghĩ của tôi, như che giấu

thông tin (information hiding), bắt đầu trở nên rõ ràng hơn như những khái niệm nền tảng cần được làm nổi bật hơn. Ở những mảng khác, công nghệ mới mang đến cả những giải pháp mới lẫn những vấn đề mới cho hệ thống của chúng ta. Việc chứng kiến quá nhiều người đổ xô sang Kubernetes với hy vọng nó sẽ giải quyết mọi vấn đề của họ với kiến trúc microservice chắc chắn đã khiến tôi phải dừng lại suy nghĩ.

Additionally, I'd written the first edition of Building Microservices to provide not just an explanation of microservices, but also a broad overview of how this architectural approach changes aspects of software development. So as I looked more deeply at aspects around **security** and **resiliency**, I found myself wanting to go back and expand on those topics that are increasingly important to modern software development.

Ngoài ra, tôi đã viết phiên bản đầu của Building Microservices không chỉ để giải thích microservices, mà còn để cung cấp một cái nhìn tổng quan rộng về cách cách tiếp cận kiến trúc này làm thay đổi các khía cạnh của phát triển phần mềm. Vì vậy, khi nhìn sâu hơn vào các khía cạnh quanh bảo mật (security) và khả năng phục hồi (resiliency), tôi thấy mình muốn quay lại và mở rộng những chủ đề ngày càng quan trọng đối với phát triển phần mềm hiện đại.

Thus in this second edition, I spend more time sharing explicit examples to better explain the ideas. Every chapter has been reexamined, and every sentence reviewed. Not a lot of the first edition remains in terms of concrete prose, but the ideas are all still here. I've tried to be clearer in my own opinions, while still recognizing that there are often multiple ways of solving a problem. This has meant expanding the discussion of inter-process communication, which is now split across three chapters. I also spend more time looking at the implications of technologies such as containers, Kubernetes, and **serverless**; as a result, there are now separate **build** and deployment chapters.

Do đó, trong phiên bản thứ hai này, tôi dành nhiều thời gian hơn để chia sẻ các ví dụ cụ thể nhằm giải thích các ý tưởng tốt hơn. Mọi chương đều đã được xem xét lại, và mọi câu đều đã được rà soát. Không còn nhiều nội dung của phiên bản đầu còn lại xét về câu chữ cụ thể, nhưng các ý tưởng thì vẫn còn nguyên ở đây. Tôi đã cố gắng trình bày rõ ràng hơn quan điểm của riêng mình, đồng thời vẫn thừa nhận rằng thường có nhiều cách để giải quyết một vấn đề. Điều này đồng nghĩa với việc mở rộng phần thảo luận về giao tiếp liên tiến trình (inter-process communication), nay được chia thành ba chương. Tôi cũng dành nhiều thời gian hơn để xem xét những hệ quả của các công nghệ như container, Kubernetes và serverless; kết quả là giờ đây có các chương riêng về build và triển khai (deployment).

My hope had been to create a book around the same size as the first edition, while finding a way to pack in more ideas. As you can see, I've failed to achieve my goal—this edition is bigger! But I think I've succeeded in being clearer in how I articulate the ideas.

Hy vọng của tôi là tạo ra một cuốn sách có cùng kích cỡ với phiên bản đầu, đồng thời tìm cách nhồi nhét thêm nhiều ý tưởng hơn. Như bạn có thể thấy, tôi đã thất bại trong mục tiêu này—phiên bản này dày hơn! Nhưng tôi nghĩ mình đã thành công trong việc diễn đạt các ý tưởng rõ ràng hơn.

Navigating This Book — Cách đọc cuốn sách này

This book is primarily organized in a topic-based format. I have structured and written the book as though you will read it from beginning to end, but of course you may want to jump to the specific topics that most interest you. If you do decide to dive right into a specific chapter, you may find the

Glossary toward the end of the book helpful in explaining new or unfamiliar terms. On the topic of terminology, I use *microservice* and **service** interchangeably throughout the book; you can assume that the two terms refer to the same thing unless I explicitly state otherwise. I also summarize some of the book's key advice in the Bibliography if you really just want to jump to the end—just remember you'll be missing a lot of detail if you do that!

Cuốn sách này chủ yếu được tổ chức theo định dạng dựa trên chủ đề. Tôi đã cấu trúc và viết cuốn sách như thể bạn sẽ đọc nó từ đầu đến cuối, nhưng tất nhiên bạn có thể muốn nhảy đến những chủ đề cụ thể khiến bạn quan tâm nhất. Nếu bạn quyết định lao thẳng vào một chương cụ thể, bạn có thể thấy phần Bảng thuật ngữ (Glossary) ở gần cuối sách hữu ích trong việc giải thích các thuật ngữ mới hoặc xa lạ. Về chủ đề thuật ngữ, tôi dùng *microservice* và *service* (dịch vụ) thay thế cho nhau xuyên suốt cuốn sách; bạn có thể giả định rằng hai thuật ngữ này chỉ cùng một thứ trừ khi tôi nói rõ điều ngược lại. Tôi cũng tóm tắt một số lời khuyên then chốt của cuốn sách trong phần Bibliography nếu bạn thực sự chỉ muốn nhảy đến cuối—nhưng hãy nhớ rằng bạn sẽ bỏ lỡ rất nhiều chi tiết nếu làm vậy!

The main body of the book is broken into three separate parts—Foundation, Implementation, and People. Let's look at what each part covers.

Phần thân chính của cuốn sách được chia thành ba phần riêng biệt—Foundation (Nền tảng), Implementation (Triển khai) và People (Con người). Hãy cùng xem mỗi phần đề cập đến điều gì.

Part I, Foundation — Phần I, Nền tảng (Foundation)

In this part, I detail some of the core ideas behind microservices.

Trong phần này, tôi trình bày chi tiết một số ý tưởng cốt lõi đằng sau microservices.

Chapter 1, What Are Microservices? This is a general introduction to microservices in which I briefly discuss a number of topics that I'll expand on later in the book.

Chương 1, Microservices là gì? Đây là phần giới thiệu tổng quát về microservices, trong đó tôi điểm qua ngắn gọn một số chủ đề mà tôi sẽ mở rộng sau này trong cuốn sách.

Chapter 2, How to Model Microservices This chapter examines the importance of concepts like information hiding, **coupling**, **cohesion**, and **domain-driven design** in helping find the right boundaries for your microservices.

Chương 2, Cách mô hình hóa Microservices Chương này xem xét tầm quan trọng của các khái niệm như che giấu thông tin (information hiding), coupling, cohesion (tính gắn kết) và domain-driven design (DDD) trong việc giúp tìm ra ranh giới phù hợp cho các microservice của bạn.

Chapter 3, Splitting the **Monolith** This chapter provides some guidance on how to take an existing monolithic application and break it apart into microservices.

Chương 3, Tách rời Monolith Chương này đưa ra một số hướng dẫn về cách lấy một ứng dụng monolith hiện hữu và phân rã nó thành các microservice.

Chapter 4, Microservice Communication Styles The last chapter in this part offers a discussion of the different types of microservice communication, including **asynchronous** versus **synchronous** calls and **request-response** and **event-driven** collaboration styles.

Chương 4, Các kiểu giao tiếp giữa Microservice Chương cuối cùng trong phần này thảo luận về các kiểu giao tiếp microservice khác nhau, bao gồm lời gọi bất đồng bộ (asynchronous) so với đồng bộ (synchronous), cũng như các phong cách cộng tác request-response và hướng sự kiện (event-driven).

Part II, Implementation — Phần II, Triển khai (Implementation)

Moving from higher-level concepts to implementation detail, in this part we look at techniques and technologies that can help you get the most out of microservices.

Chuyển từ các khái niệm ở tầng cao hơn sang chi tiết triển khai, trong phần này chúng ta xem xét các kỹ thuật và công nghệ có thể giúp bạn khai thác tối đa microservices.

Chapter 5, Implementing Microservice Communication In this chapter we take a deeper look at the specific technologies used to implement inter-microservice communication.

Chương 5, Hiện thực hóa giao tiếp giữa Microservice Trong chương này, chúng ta xem xét sâu hơn các công nghệ cụ thể được dùng để hiện thực hóa giao tiếp giữa các microservice.

Chapter 6, **Workflow** This chapter offers a comparison of sagas and distributed transactions and discusses their usefulness in modeling business processes involving multiple microservices.

Chương 6, Workflow Chương này đưa ra một sự so sánh giữa saga và giao dịch phân tán, đồng thời thảo luận về tính hữu dụng của chúng trong việc mô hình hóa các quy trình nghiệp vụ liên quan đến nhiều microservice.

Chapter 7, Build This chapter explains mapping from a microservice to repositories and builds.

Chương 7, Build Chương này giải thích việc ánh xạ từ một microservice sang các repository và các bản build.

Chapter 8, Deployment In this chapter you'll find discussion of the myriad options around deploying a microservice, including a look at containers, Kubernetes, and FaaS.

Chương 8, Triển khai (Deployment) Trong chương này, bạn sẽ thấy phần thảo luận về vô số lựa chọn xoay quanh việc triển khai một microservice, bao gồm cái nhìn về container, Kubernetes và FaaS.

Chapter 9, Testing Here I discuss the challenges of testing microservices, including the issues caused by end-to-end tests, and how consumer-driven contracts and in-production testing can help.

Chương 9, Kiểm thử Ở đây, tôi thảo luận về những thách thức trong việc kiểm thử (testing) microservices, bao gồm các vấn đề do end-to-end test gây ra, và cách consumer-driven contract cùng kiểm thử trên môi trường production có thể giúp ích.

Chapter 10, From **Monitoring** to **Observability** This chapter covers the shift from focusing on static monitoring activities to thinking more broadly about how to improve the observability of microservice architectures, along with some specific recommendations regarding tooling.

Chương 10, Từ Monitoring đến Observability Chương này bàn về sự chuyển dịch từ chỗ tập trung vào các hoạt động monitoring (giám sát) tĩnh sang việc suy nghĩ rộng hơn về cách cải thiện observability (khả năng quan sát) của kiến trúc microservice, cùng với một số khuyến nghị cụ thể về công cụ.

Chapter 11, Security Microservice architectures can increase the surface area of attack but also give us more opportunity to defend in depth. In this chapter, we explore this balance.

Chương 11, Bảo mật Kiến trúc microservice có thể làm tăng bề mặt tấn công nhưng cũng cho chúng ta nhiều cơ hội hơn để phòng thủ nhiều lớp (defense in depth). Trong chương này, chúng ta khám phá sự cân bằng đó.

Chapter 12, Resiliency This chapter offers a wider look at what resiliency is and at the part that microservices can play in improving the resiliency of your applications.

Chương 12, Khả năng phục hồi Chương này đưa ra cái nhìn rộng hơn về khả năng phục hồi (resiliency) là gì và về vai trò mà microservices có thể đóng góp trong việc cải thiện khả năng phục hồi của các ứng dụng của bạn.

Chapter 13, **Scaling** In this chapter I outline the four axes of scaling and show how they can be used in combination to scale a microservice **architecture**.

Chương 13, Mở rộng quy mô Trong chương này, tôi phác thảo bốn trục của việc mở rộng (scaling) và chỉ ra cách chúng có thể được kết hợp để mở rộng một kiến trúc microservice.

Part III, People — Phần III, Con người (People)

Ideas and technology don't mean anything without the people and organization to back them up.

Ý tưởng và công nghệ chẳng có ý nghĩa gì nếu thiếu con người và tổ chức để hậu thuẫn cho chúng.

Chapter 14, User Interfaces From moving away from dedicated **frontend** teams to the use of BFFs and **GraphQL**, this chapter explores how microservices and user interfaces can work together.

Chương 14, Giao diện người dùng Từ việc rời xa các đội frontend chuyên trách cho đến việc sử dụng BFF và GraphQL, chương này khám phá cách microservices và giao diện người dùng (UI) có thể phối hợp với nhau.

Chapter 15, Organizational Structures This penultimate chapter focuses on how stream-aligned and enabling teams can work in the context of microservice architectures.

Chương 15, Cấu trúc tổ chức Chương áp chót này tập trung vào cách các stream-aligned team và enabling team có thể vận hành trong bối cảnh kiến trúc microservice.

Chapter 16, The **Evolutionary Architect** Microservice architectures aren't static, so your view of **system architecture** may need to change—a topic that this chapter explores in depth.

Chương 16, Kiến trúc sư tiến hóa Kiến trúc microservice không tĩnh tại, vì vậy góc nhìn của bạn về kiến trúc hệ thống có thể cần thay đổi—một chủ đề mà chương này khám phá một cách sâu sắc.

Conventions Used in This Book — Các quy ước được sử dụng trong cuốn sách này

The following typographical conventions are used in this book:

Cuốn sách này sử dụng các quy ước trình bày sau đây:

Italic Indicates new terms, URLs, email addresses, filenames, and file extensions.

In nghiêng Cho biết các thuật ngữ mới, URL, địa chỉ email, tên tệp và phần mở rộng tệp.

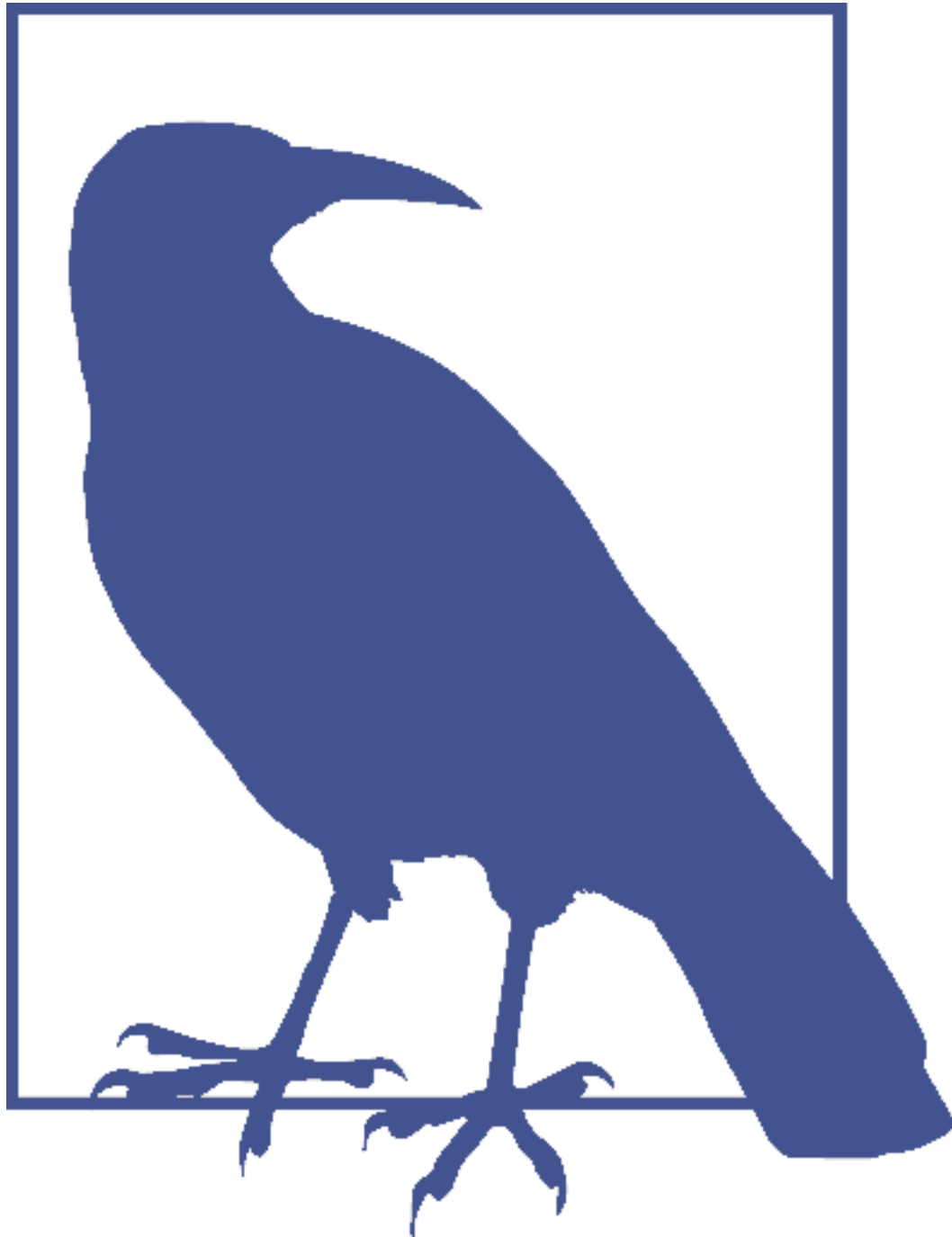
Constant width Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Độ rộng cố định (Constant width) Dùng cho các đoạn liệt kê chương trình, cũng như trong các đoạn văn để chỉ các phần tử của chương trình như tên biến hoặc tên hàm, database, kiểu dữ liệu, biến môi trường, câu lệnh và từ khóa.



This element signifies a tip or suggestion.

Phần tử này biểu thị một mẹo hoặc gợi ý.



This element signifies a general note.

Phần tử này biểu thị một ghi chú chung.



This element indicates a warning or caution.

Phần tử này biểu thị một cảnh báo hoặc lưu ý.

O'Reilly Online Learning — O'Reilly Online Learning



For more than 40 years, O'Reilly Media has provided technology and business training, knowledge, and insight to help companies succeed.

Hơn 40 năm qua, O'Reilly Media đã cung cấp các khóa đào tạo, kiến thức và hiểu biết sâu sắc về công nghệ và kinh doanh nhằm giúp các công ty thành công.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <http://oreilly.com>.

Mạng lưới chuyên gia và nhà đổi mới độc đáo của chúng tôi chia sẻ kiến thức và chuyên môn của họ thông qua sách, bài viết và nền tảng học trực tuyến của chúng tôi. Nền tảng học trực tuyến của O'Reilly cho bạn quyền truy cập theo yêu cầu đến các khóa đào tạo trực tiếp, các lộ trình học chuyên sâu, các môi trường lập trình tương tác, cùng một bộ sưu tập khổng lồ gồm văn bản và video từ O'Reilly và hơn 200 nhà xuất bản khác. Để biết thêm thông tin, hãy truy cập <http://oreilly.com>.

How to Contact Us — Cách liên hệ với chúng tôi

Please address comments and questions concerning this book to the publisher:

Vui lòng gửi các nhận xét và câu hỏi liên quan đến cuốn sách này tới nhà xuất bản:

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (in the United States or Canada) 707-829-0515 (international or local) 707-829-0104 (fax)

O'Reilly Media, Inc. 1005 Gravenstein Highway North Sebastopol, CA 95472 800-998-9938 (tại Hoa Kỳ hoặc Canada) 707-829-0515 (quốc tế hoặc địa phương) 707-829-0104 (fax)

We have a web page for this book, where we list errata, examples, and any additional information. You can access this page at https://oreil.ly/Building_Microservices2.

Chúng tôi có một trang web cho cuốn sách này, nơi chúng tôi liệt kê đính chính, ví dụ và mọi thông tin bổ sung. Bạn có thể truy cập trang này tại https://oreil.ly/Building_Microservices2.

Email bookquestions@oreilly.com to comment or ask technical questions about this book.

Gửi email tới bookquestions@oreilly.com để nhận xét hoặc đặt câu hỏi kỹ thuật về cuốn sách này.

For news and information about our books and courses, visit <http://oreilly.com>.

Để biết tin tức và thông tin về sách và khóa học của chúng tôi, hãy truy cập <http://oreilly.com>.

Find us on Facebook: <http://facebook.com/oreilly>

Tìm chúng tôi trên Facebook: <http://facebook.com/oreilly>

Follow us on Twitter: <http://twitter.com/oreillymedia>

Theo dõi chúng tôi trên Twitter: <http://twitter.com/oreillymedia>

Watch us on YouTube: <http://youtube.com/oreillymedia>

Xem chúng tôi trên YouTube: <http://youtube.com/oreillymedia>

Acknowledgments — Lời cảm ơn

I am continually amazed by the support I get from my family, especially that of my wife, Lindy Stephens, who puts up with so much with amazing good grace. To say this book wouldn't exist without her is an understatement, even if she doesn't always believe me when I tell her that. This is for her. It's also for my dad, Jack, Josie, Kane, and the wider Gilmanco Staynes clan.

Tôi không ngừng kinh ngạc trước sự hỗ trợ mà tôi nhận được từ gia đình, đặc biệt là từ vợ tôi, Lindy Stephens, người chịu đựng rất nhiều với sự độ lượng đáng kinh ngạc. Nói rằng cuốn sách này sẽ không tồn tại nếu không có cô ấy vẫn còn là một sự nói giảm, dù cô ấy không phải lúc nào cũng tin tôi khi tôi nói điều đó với cô. Cuốn sách này dành cho cô. Nó cũng dành cho cha tôi, Jack, Josie, Kane, và cả đại gia đình Gilmanco Staynes.

Much of this book was written during a global pandemic, which as I write these acknowledgments is still ongoing. It may not mean much, but I want to extend my thanks to the NHS here in the UK, and to all the people worldwide who are keeping us safe by working on vaccines, treating the sick, delivering our food, and helping in a thousand other ways I wouldn't understand. This is also for all of you.

Phần lớn cuốn sách này được viết trong một đại dịch toàn cầu, mà ngay khi tôi viết những lời cảm ơn này vẫn đang tiếp diễn. Có thể điều này không có nhiều ý nghĩa, nhưng tôi muốn gửi lời cảm ơn đến NHS ở Anh Quốc, và đến tất cả những người trên khắp thế giới đang giữ cho chúng ta an toàn bằng cách nghiên cứu vaccine, chữa trị cho người bệnh, vận chuyển thực phẩm cho chúng ta, và giúp đỡ theo hàng nghìn cách khác mà tôi không thể nào hiểu hết. Cuốn sách này cũng dành cho tất cả các bạn.

This second edition wouldn't exist without the first, so I'd like to reiterate my thanks to everyone who helped me during the challenging process of writing my first book, include the technical reviewers Ben Christensen, Martin Fowler, and Venkat Subramaniam; James Lewis for our many enlightening conversations; the O'Reilly team of Brian MacDonald, Rachel Monaghan, Kristen Brown, and Betsy Waliszewski; and excellent review feedback from readers Anand Krishnaswamy, Kent McNeil, Charles Haynes, Chris Ford, Aidy Lewis, Will Thames, Jon Eaves, Rolf Russell, Badrinath Janakiraman, Daniel Bryant, Ian Robinson, Jim Webber, Stewart Gleadow, Evan Bottcher, Eric Sword, and Olivia Leonard. And thank to Mike Loukides, I guess, for getting me into this mess in the first place!

Phiên bản thứ hai này sẽ không tồn tại nếu không có phiên bản đầu, nên tôi muốn nhắc lại lời cảm ơn đến tất cả những người đã giúp tôi trong quá trình đầy thử thách khi viết cuốn sách đầu tiên, bao gồm các nhà phản biện kỹ thuật Ben Christensen, Martin Fowler và Venkat Subramaniam; James Lewis vì nhiều cuộc trò chuyện khai sáng của chúng tôi; đội ngũ O'Reilly gồm Brian MacDonald, Rachel Monaghan, Kristen Brown và Betsy Waliszewski; và những phản hồi phản biện tuyệt vời từ các độc giả Anand Krishnaswamy, Kent McNeil, Charles Haynes, Chris Ford, Aidy Lewis, Will Thames, Jon Eaves, Rolf Russell, Badrinath Janakiraman, Daniel Bryant, Ian Robinson, Jim Webber, Stewart Gleadow, Evan Bottcher, Eric Sword và Olivia Leonard. Và cảm ơn Mike Loukides, tôi đoán vậy, vì đã đẩy tôi vào mớ rắc rối này ngay từ đầu!

For the second edition, Martin Fowler once again returned as technical reviewer and was joined by Daniel Bryant and Sarah Wells, who were generous with their time and feedback. I'd also like to thank Nicky Wrightson and Alexander von Zitzewitz for helping push the technical review over the finishing line. On the O'Reilly front, the whole process was overseen by my awesome editor, Nicole Taché, without whom I would certainly have gone insane; Melissa Duffield, who seems to do a better job of managing my workload than I do; Deb Baker, Arthur Johnson, and the **rest** of the production team (I'm sorry I don't know all your names, but thank you!); and Mary Treseler, for steering the ship through some difficult times.

Với phiên bản thứ hai, Martin Fowler một lần nữa trở lại với vai trò nhà phản biện kỹ thuật và được tiếp sức bởi Daniel Bryant và Sarah Wells, những người đã hào phóng dành thời gian và phản hồi. Tôi cũng muốn cảm ơn Nicky Wrightson và Alexander von Zitzewitz vì đã giúp đưa quá trình phản biện kỹ thuật về đích. Về phía O'Reilly, toàn bộ quá trình được giám sát bởi biên tập viên tuyệt vời của tôi, Nicole Taché, người mà nếu không có thì chắc chắn tôi đã phát điên; Melissa Duffield, người dường như quản lý khối lượng công việc của tôi tốt hơn cả chính tôi; Deb Baker, Arthur Johnson và phần còn lại của đội sản xuất (tôi xin lỗi vì không biết hết tên các bạn, nhưng cảm ơn các bạn!); và Mary Treseler, vì đã lèo lái con thuyền vượt qua những thời điểm khó khăn.

In addition, the second edition has benefited hugely from help and insights from a number of people, including (in no particular order) Dave Coombes and the team at Tyro, Dave Halsey and the team at Money Supermarket, Tom Kerkhove, Erik Doernenburg, Graham Tackley, Kent Beck, Kevlin Henney, Laura Bell, Adrian Mouat, Sarah Taraporewalla, Uwe Friedrichse, Liz Fong-Jones, Kane Stephens, Gilmanco Staynes, Adam Tornhill, Venkat Subramaniam, Susanne Kaiser, Jan Schaumann, Grady Booch, Pini Reznik, Nicole Forsgren, Jez Humble, Gene Kim, Manuel Pais, Matthew Skelton, and the South Sydney Rabbitohs. Finally, I'd like to thank the awesome readers of the early access version of the book, who provided invaluable feedback; they include Felipe de Moraes, Mark Gardner, David Lauzon, Assam Zafar, Michael Bleterman, Nicola Musatti, Eleonora Lester, Felipe de Moraes, Nathan DiMauro, Daniel Lemke, Soner Eker, Ripple Shah, Joel Lim, and Himanshu Pant. And finally, hello to Jason Isaacs.

Ngoài ra, phiên bản thứ hai đã được hưởng lợi rất nhiều từ sự giúp đỡ và những hiểu biết sâu sắc của nhiều người, bao gồm (không theo thứ tự cụ thể nào) Dave Coombes và đội ngũ ở Tyro, Dave Halsey và đội ngũ ở Money Supermarket, Tom Kerkhove, Erik Doernenburg, Graham Tackley, Kent Beck, Kevlin Henney, Laura Bell, Adrian Mouat, Sarah Taraporewalla, Uwe Friedrichse, Liz Fong-Jones, Kane Stephens, Gilmanco Staynes, Adam Tornhill, Venkat Subramaniam, Susanne Kaiser, Jan Schaumann, Grady Booch, Pini Reznik, Nicole Forsgren, Jez Humble, Gene Kim, Manuel Pais, Matthew Skelton, và đội South Sydney Rabbitohs. Cuối cùng, tôi muốn cảm ơn những độc giả tuyệt vời của phiên bản early access của cuốn sách, những người đã đưa ra phản hồi vô giá; họ bao gồm Felipe de Moraes, Mark Gardner, David Lauzon, Assam Zafar, Michael Bleterman, Nicola

Musatti, Eleonora Lester, Felipe de Moraes, Nathan DiMauro, Daniel Lemke, Soner Eker, Ripple Shah, Joel Lim, và Himanshu Pant. Và cuối cùng, xin chào Jason Isaacs.

I wrote the bulk of this book throughout the whole of 2020 and in the first half of

Tôi viết phần lớn cuốn sách này trong suốt năm 2020 và nửa đầu năm

2021. I used Visual Studio Code on macOS for most of the writing, although on rare occasions I also used Working Copy on iOS. OmniGraffle was used to create all the diagrams in the book. AsciiDoc was used to format the book and on the whole was excellent, with O'Reilly's Atlas toolchain doing the magic of making a book appear on the other side.

2021. Tôi dùng Visual Studio Code trên macOS cho phần lớn việc viết, mặc dù trong những dịp hiếm hoi tôi cũng dùng Working Copy trên iOS. OmniGraffle được dùng để tạo tất cả các sơ đồ trong cuốn sách. AsciiDoc được dùng để định dạng cuốn sách và nhìn chung là tuyệt vời, với bộ công cụ Atlas của O'Reilly làm phép màu biến một cuốn sách thành hình hài ở phía bên kia.

CHAPTER 1 What Are Microservices? — Chương 1: Microservices là gì?

Microservices have become an increasingly popular **architecture** choice in the half decade or more since I wrote the first edition of this book. I can't claim credit for the subsequent explosion in popularity, but the rush to make use of **microservice** architectures means that while many of the ideas I captured previously are now tried and tested, new ideas have come into the mix at the same time that earlier practices have fallen out of favor. So it's once again time to distill the essence of microservice architecture while highlighting the core concepts that make microservices work.

Microservices đã trở thành một lựa chọn kiến trúc (architecture) ngày càng phổ biến trong khoảng nửa thập kỷ hoặc hơn kể từ khi tôi viết ấn bản đầu tiên của cuốn sách này. Tôi không thể nhận công cho sự bùng nổ về mức độ phổ biến sau đó, nhưng việc đổ xô áp dụng kiến trúc microservice có nghĩa là trong khi nhiều ý tưởng tôi đã ghi lại trước đây giờ đã được kiểm chứng và thử nghiệm, thì những ý tưởng mới cũng xuất hiện trong bức tranh chung, đồng thời một số thực hành trước kia lại không còn được ưa chuộng. Vì vậy, một lần nữa đã đến lúc chắt lọc tinh túy của kiến trúc microservice trong khi làm nổi bật những khái niệm cốt lõi khiến microservices vận hành được.

This book as a whole is designed to give a broad overview of the impact that microservices have on various aspects of software delivery. To start us off, this chapter will take a look at the core ideas behind microservices, the prior art that brought us here, and some reasons why these architectures are used so widely.

Toàn bộ cuốn sách này được thiết kế để mang lại một cái nhìn tổng quan rộng về tác động mà microservices tạo ra lên nhiều khía cạnh khác nhau của việc chuyển giao phần mềm. Để bắt đầu, chương này sẽ xem xét những ý tưởng cốt lõi đằng sau microservices, các công trình tiền nhiệm (prior art) đã đưa chúng ta đến đây, và một số lý do vì sao những kiến trúc này lại được sử dụng rộng rãi đến vậy.

Microservices at a Glance — Tổng quan nhanh về Microservices

Microservices are independently releasable services that are modeled around a **business domain**. A **service** encapsulates functionality and makes it accessible to other services via networks—you construct a more complex system from these building blocks. One microservice might represent inventory, another order management, and yet another shipping, but together they might constitute an entire ecommerce system. Microservices are an architecture choice that is focused on giving you

many options for solving the problems you might face.

Microservices là các service có thể phát hành độc lập (independently releasable) và được mô hình hóa xoay quanh một miền nghiệp vụ (business domain). Một service đóng gói chức năng và làm cho nó có thể truy cập được bởi các service khác qua mạng—bạn xây dựng một hệ thống phức tạp hơn từ những khối xây dựng này. Một microservice có thể đại diện cho kho hàng (inventory), một cái khác cho quản lý đơn hàng, và một cái khác nữa cho vận chuyển, nhưng khi kết hợp lại chúng có thể tạo thành toàn bộ một hệ thống thương mại điện tử. Microservices là một lựa chọn kiến trúc tập trung vào việc trao cho bạn nhiều phương án để giải quyết những vấn đề bạn có thể gặp phải.

They are a type of service-oriented architecture, albeit one that is opinionated about how service boundaries should be drawn, and one in which **independent deployability** is key. They are technology agnostic, which is one of the advantages they offer.

Chúng là một dạng kiến trúc hướng dịch vụ (service-oriented architecture), tuy là một dạng có quan điểm rõ ràng về cách vạch ra ranh giới service, và là một dạng trong đó khả năng triển khai độc lập (independent deployability) đóng vai trò then chốt. Chúng độc lập với công nghệ (technology agnostic), đây là một trong những lợi thế mà chúng mang lại.

From the outside, a single microservice is treated as a black box. It hosts business functionality on one or more network endpoints (for example, a queue or a **REST API**, as shown in Figure 1-1), over whatever protocols are most appropriate. Consumers, whether they're other microservices or other sorts of programs, access this functionality via these networked endpoints. Internal implementation details (such as the technology the service is written in or the way data is stored) are entirely hidden from the outside world. This means microservice architectures avoid the use of shared databases in most circumstances; instead, each microservice encapsulates its own **database** where required.

Nhìn từ bên ngoài, một microservice đơn lẻ được xem như một hộp đen (black box). Nó lưu trữ (host) chức năng nghiệp vụ trên một hoặc nhiều endpoint mạng (ví dụ một hàng đợi hoặc một REST API, như minh họa trong Hình 1-1), trên bất kỳ giao thức nào phù hợp nhất. Bên tiêu thụ (consumer), dù là các microservice khác hay các loại chương trình khác, đều truy cập chức năng này thông qua các endpoint mạng đó. Các chi tiết triển khai bên trong (như công nghệ mà service được viết bằng, hay cách dữ liệu được lưu trữ) hoàn toàn được ẩn đi khỏi thế giới bên ngoài. Điều này có nghĩa là kiến trúc microservice tránh việc sử dụng cơ sở dữ liệu dùng chung trong hầu hết các trường hợp; thay vào đó, mỗi microservice đóng gói cơ sở dữ liệu của riêng mình khi cần.

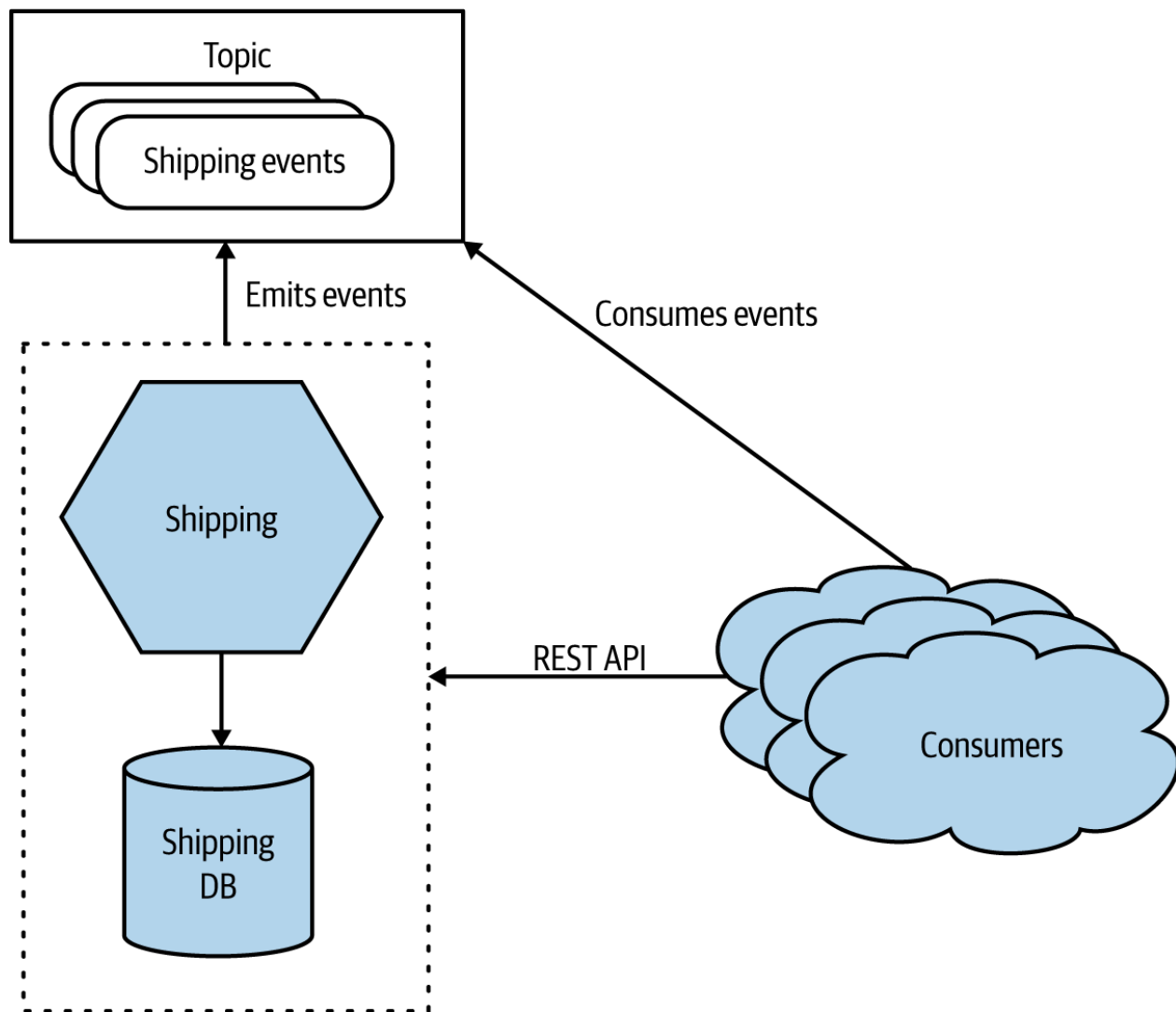


Figure 1-1. A microservice exposing its functionality over a REST API and a topic

Hình 1-1. Một microservice phơi bày chức năng của nó qua một REST API và một topic

Microservices embrace the concept of **information hiding**. 1 Information hiding means hiding as much information as possible inside a component and exposing as little as possible via external interfaces. This allows for clear separation between what can change easily and what is more difficult to change. Implementation that is hidden from external parties can be changed freely as long as the networked interfaces the microservice exposes don't change in a backward-incompatible fashion. Changes inside a microservice boundary (as shown in Figure 1-1) shouldn't affect an upstream

Microservices đón nhận khái niệm che giấu thông tin (information hiding). 1 Che giấu thông tin nghĩa là ẩn đi càng nhiều thông tin càng tốt bên trong một thành phần (component) và phơi bày càng ít càng tốt qua các interface bên ngoài. Điều này cho phép phân tách rõ ràng giữa những gì có thể thay đổi dễ dàng và những gì khó thay đổi hơn. Phần triển khai bị ẩn khỏi các bên bên ngoài có thể được thay đổi tự do miễn là các interface mạng mà microservice phơi bày không thay đổi theo cách phá vỡ tương thích ngược (backward-incompatible). Những thay đổi bên trong ranh giới một microservice (như minh họa trong Hình 1-1) không nên ảnh hưởng đến một

1 This concept was first outlined by David Parnas in “Information Distribution Aspects of Design Methodology”, Information Processing: Proceedings of the IFIP Congress 1971 (Amsterdam: North-Holland, 1972), 1:339–44.

1 Khái niệm này lần đầu được David Parnas trình bày trong “Information Distribution Aspects of Design Methodology”, Information Processing: Proceedings of the IFIP Congress 1971 (Amsterdam: North-Holland, 1972), 1:339–44.

consumer, enabling independent releasability of functionality. This is essential in allowing our microservices to be worked on in isolation and released on demand. Having clear, stable service boundaries that don’t change when the internal implementation changes results in systems that have looser **coupling** and stronger **cohesion**.

bên tiêu thụ ở phía trên (upstream consumer), nhờ đó cho phép phát hành chức năng một cách độc lập. Điều này là thiết yếu để cho phép các microservice của chúng ta được làm việc một cách tách biệt và được phát hành theo nhu cầu. Việc có những ranh giới service rõ ràng, ổn định, không thay đổi khi phần triển khai bên trong thay đổi sẽ tạo ra những hệ thống có coupling lỏng hơn và cohesion mạnh hơn.

While we’re talking about hiding internal implementation detail, it would be remiss of me not to mention the Hexagonal Architecture pattern, first detailed by Alistair Cockburn. 2 This pattern describes the importance of keeping the internal implementation separate from its external interfaces, with the idea that you might want to interact with the same functionality over different types of interfaces. I draw my microservices as hexagons partly to differentiate them from “normal” services, but also as an homage to this piece of prior art.

Trong khi đang nói về việc ẩn chi tiết triển khai bên trong, sẽ là thiếu sót nếu tôi không nhắc đến pattern Hexagonal Architecture, lần đầu được Alistair Cockburn trình bày chi tiết. 2 Pattern này mô tả tầm quan trọng của việc giữ phần triển khai bên trong tách biệt khỏi các interface bên ngoài của nó, với ý tưởng rằng bạn có thể muốn tương tác với cùng một chức năng qua nhiều loại interface khác nhau. Tôi vẽ các microservice của mình thành hình lục giác một phần để phân biệt chúng với các service “thông thường”, nhưng cũng như một sự tri ân với công trình tiền nhiệm này.

Are Service-Oriented Architecture and Microservices Different Things? — Kiến trúc hướng dịch vụ và Microservices có phải là hai thứ khác nhau?

Service-oriented architecture (SOA) is a design approach in which multiple services collaborate to provide a certain end set of capabilities. (A service here typically means a completely separate operating system process.) Communication between these services occurs via calls across a network rather than method calls within a process boundary.

Kiến trúc hướng dịch vụ (service-oriented architecture, SOA) là một cách tiếp cận thiết kế trong đó nhiều service cộng tác với nhau để cung cấp một tập năng lực đầu cuối nhất định. (Một service ở đây thường có nghĩa là một tiến trình hệ điều hành hoàn toàn riêng biệt.) Giao tiếp giữa các service này diễn ra thông qua các lời gọi qua mạng thay vì các lời gọi phương thức (method call) trong phạm vi một tiến trình.

SOA emerged as an approach to combat the challenges of large, monolithic applications. This approach aims to promote the reusability of software; two or more end-user applications, for example, could use the same services. SOA aims to make it easier to maintain or rewrite software,

as theoretically we can replace one service with another without anyone knowing, as long as the semantics of the service don't change too much.

SOA xuất hiện như một cách tiếp cận để chống lại các thách thức của những ứng dụng monolith lớn. Cách tiếp cận này hướng tới việc thúc đẩy khả năng tái sử dụng phần mềm; chẳng hạn, hai hoặc nhiều ứng dụng người dùng cuối có thể sử dụng cùng các service. SOA hướng tới việc làm cho việc bảo trì hoặc viết lại phần mềm dễ dàng hơn, vì về lý thuyết chúng ta có thể thay thế một service này bằng một service khác mà không ai hay biết, miễn là ngữ nghĩa của service không thay đổi quá nhiều.

SOA at its heart is a sensible idea. However, despite many efforts, there is a lack of good consensus on how to do SOA well. In my opinion, much of the industry failed to look holistically enough at the problem and present a compelling alternative to the narrative set out by various vendors in this space.

SOA về bản chất là một ý tưởng hợp lý. Tuy nhiên, bất chấp nhiều nỗ lực, vẫn thiếu một sự đồng thuận tốt về cách làm SOA cho hiệu quả. Theo ý kiến của tôi, phần lớn ngành công nghiệp đã không nhìn nhận vấn đề một cách đủ toàn diện và không đưa ra được một phương án thay thế thuyết phục cho lối kể chuyện mà các nhà cung cấp (vendor) khác nhau trong lĩnh vực này đã đặt ra.

Many of the problems laid at the door of SOA are actually problems with things like communication protocols (e.g., SOAP), vendor middleware, a lack of guidance about service granularity, or the wrong guidance on picking places to split your system. A cynic might suggest that vendors co-opted (and in some cases drove) the SOA movement as a way to sell more products, and those selfsame products in the end undermined the goal of SOA.

Nhiều vấn đề bị quy cho SOA thực chất lại là vấn đề của những thứ như giao thức giao tiếp (ví dụ SOAP), middleware của vendor, sự thiếu hướng dẫn về độ chi tiết (granularity) của service, hoặc hướng dẫn sai về việc chọn nơi chia tách hệ thống. Một người hoài nghi có thể cho rằng các vendor đã lợi dụng (và trong một số trường hợp đã thúc đẩy) phong trào SOA như một cách để bán thêm sản phẩm, và chính những sản phẩm đó rốt cuộc lại làm suy yếu mục tiêu của SOA.

2 Alistair Cockburn, "Hexagonal Architecture," January 4, 2005, <https://oreil.ly/NfvTP>.

2 Alistair Cockburn, "Hexagonal Architecture," 4 tháng 1, 2005, <https://oreil.ly/NfvTP>.

I've seen plenty of examples of SOA in which teams were striving to make the services smaller, but they still had everything coupled to a database and had to **deploy** everything together. Service oriented? Yes. But it's not microservices.

Tôi đã thấy vô số ví dụ về SOA trong đó các nhóm cố gắng làm cho các service nhỏ hơn, nhưng họ vẫn để mọi thứ phụ thuộc vào một cơ sở dữ liệu và phải triển khai tất cả cùng nhau. Hướng dịch vụ ư? Đúng vậy. Nhưng đó không phải là microservices.

The microservice approach has emerged from real-world use, taking our better understanding of systems and architecture to do SOA well. You should think of microservices as being a specific approach for SOA in the same way that Extreme Programming (XP) or Scrum is a specific approach for Agile software development.

Cách tiếp cận microservice đã nổi lên từ việc sử dụng thực tế, vận dụng sự hiểu biết tốt hơn của chúng ta về hệ thống và kiến trúc để làm SOA cho đúng. Bạn nên nghĩ về microservices như một cách tiếp cận cụ thể cho SOA, giống như cách Extreme Programming (XP) hay Scrum là một cách tiếp cận cụ thể cho phát triển phần mềm Agile.

Key Concepts of Microservices — Các khái niệm cốt lõi của Microservices

A few core ideas must be understood when you are exploring microservices. Given that some aspects are often overlooked, it's vital to explore these concepts further to help ensure that you understand just what it is that makes microservices work.

Có một vài ý tưởng cốt lõi cần được hiểu khi bạn khám phá microservices. Vì một số khía cạnh thường bị bỏ qua, việc khám phá sâu hơn những khái niệm này là rất quan trọng để giúp đảm bảo rằng bạn hiểu chính xác điều gì khiến microservices vận hành được.

Independent Deployability — Khả năng triển khai độc lập

Independent deployability is the idea that we can make a change to a microservice, deploy it, and **release** that change to our users, without having to deploy any other microservices. More important, it's not just the fact that we can do this; it's that this is actually how you manage deployments in your system. It's a discipline you adopt as your default release approach. This is a simple idea that is nonetheless complex in execution.

Khả năng triển khai độc lập (independent deployability) là ý tưởng rằng chúng ta có thể thực hiện một thay đổi đối với một microservice, triển khai nó, và phát hành thay đổi đó tới người dùng của mình, mà không phải triển khai bất kỳ microservice nào khác. Quan trọng hơn, vấn đề không chỉ là việc chúng ta có thể làm điều này; mà là việc đây thực sự là cách bạn quản lý các đợt triển khai trong hệ thống của mình. Đó là một kỷ luật mà bạn áp dụng làm cách tiếp cận phát hành mặc định. Đây là một ý tưởng đơn giản nhưng lại phức tạp khi thực thi.



If you take only one thing from this book and from the concept of microservices in general, it should be this: ensure that you embrace the concept of independent deployability of your microservices. Get into the habit of deploying and releasing changes to a single microservice into production without having to deploy anything else. From this, many good things will follow.

Nếu bạn chỉ rút ra được một điều từ cuốn sách này và từ khái niệm microservices nói

chung, thì điều đó nên là: hãy đảm bảo rằng bạn đón nhận khái niệm khả năng triển khai độc lập của các microservice của mình. Hãy tạo thói quen triển khai và phát hành các thay đổi cho một microservice duy nhất lên môi trường production mà không phải triển khai bất kỳ thứ gì khác. Từ điều này, nhiều điều tốt đẹp sẽ theo sau.

To ensure independent deployability, we need to make sure our microservices are loosely coupled : we must be able to change one service without having to change anything else. This means we need explicit, well-defined, and stable contracts between services. Some implementation choices make this difficult—the sharing of databases, for example, is especially problematic.

Để đảm bảo khả năng triển khai độc lập, chúng ta cần đảm bảo rằng các microservice của mình có coupling lỏng (loosely coupled): chúng ta phải có khả năng thay đổi một service mà không phải thay đổi bất kỳ thứ gì khác. Điều này có nghĩa là chúng ta cần các hợp đồng (contract) rõ ràng, được định nghĩa tốt và ổn định giữa các service. Một số lựa chọn triển khai khiến điều này trở nên khó khăn—chẳng hạn việc dùng chung cơ sở dữ liệu đặc biệt gây rắc rối.

Independent deployability in and of itself is clearly incredibly valuable. But to achieve independent deployability, there are so many other things you have to get right that in turn have their own benefits. So you can also see the focus on independent deployability as a forcing function—by focusing on this as an outcome, you'll achieve a number of ancillary benefits.

Bản thân khả năng triển khai độc lập rõ ràng là vô cùng giá trị. Nhưng để đạt được khả năng triển khai độc lập, có rất nhiều thứ khác bạn phải làm cho đúng, và những thứ đó đến lượt mình lại mang lại lợi ích riêng. Vì vậy bạn cũng có thể xem việc tập trung vào khả năng triển khai độc lập như một hàm cưỡng bức (forcing function)—bằng cách tập trung vào nó như một kết quả cần đạt, bạn sẽ thu được một số lợi ích phụ trợ.

The desire for loosely coupled services with stable interfaces guides our thinking about how we find our microservice boundaries in the first place.

Mong muốn có những service với coupling lỏng và interface ổn định sẽ dẫn dắt tư duy của chúng ta về cách xác định ranh giới microservice ngay từ đầu.

Modeled Around a Business Domain — Mô hình hóa xoay quanh một miền nghiệp vụ

Techniques like **domain-driven design** can allow you to structure your code to better represent the real-world domain that the software operates in. 3 With microservice architectures, we use this same idea to define our service boundaries. By modeling services around business domains, we can make it easier to roll out new functionality and to recombine microservices in different ways to deliver new functionality to our users.

Các kỹ thuật như domain-driven design (DDD) có thể cho phép bạn cấu trúc mã của mình để biểu diễn tốt hơn miền nghiệp vụ thực tế mà phần mềm vận hành trong đó. 3 Với kiến trúc microservice, chúng ta dùng chính ý tưởng này để định nghĩa ranh giới service. Bằng cách mô hình hóa các service xoay quanh các miền nghiệp vụ, chúng ta có thể làm cho việc triển khai chức năng mới và việc tái kết hợp các microservice theo những cách khác nhau để mang lại chức năng mới cho người dùng trở nên dễ dàng hơn.

Rolling out a feature that requires changes to more than one microservice is expensive. You need to coordinate the work across each service (and potentially across separate teams) and carefully manage the order in which the new versions of these services are deployed. That takes a lot more

work than making the same change inside a single service (or inside a **monolith**, for that matter). It therefore follows that we want to find ways to make cross-service changes as infrequent as possible.

Triển khai một tính năng đòi hỏi phải thay đổi nhiều hơn một microservice là việc tốn kém. Bạn cần phối hợp công việc giữa từng service (và có thể giữa các nhóm riêng biệt) và quản lý cẩn thận thứ tự triển khai các phiên bản mới của những service này. Điều đó tốn nhiều công sức hơn nhiều so với việc thực hiện cùng một thay đổi bên trong một service đơn lẻ (hoặc bên trong một monolith). Từ đó suy ra rằng chúng ta muốn tìm cách làm cho các thay đổi xuyên service xảy ra càng ít càng tốt.

I often see layered architectures, as typified by the three-tiered architecture in Figure 1-2 . Here, each layer in the architecture represents a different service boundary, with each service boundary based on related technical functionality. If I need to make a change to just the presentation layer in this example, that would be fairly efficient. However, experience has shown that changes in functionality typically span multiple layers in these types of architectures—requiring changes in presentation, application, and data tiers. This problem is exacerbated if the architecture is even more layered than the simple example in Figure 1-2 ; often each tier is split into further layers.

Tôi thường thấy các kiến trúc phân tầng (layered architecture), điển hình là kiến trúc ba tầng (three-tiered) trong Hình 1-2. Ở đây, mỗi tầng trong kiến trúc đại diện cho một ranh giới service khác nhau, với mỗi ranh giới service dựa trên chức năng kỹ thuật liên quan. Nếu tôi cần chỉ thay đổi tầng trình bày (presentation layer) trong ví dụ này, điều đó sẽ khá hiệu quả. Tuy nhiên, kinh nghiệm cho thấy rằng những thay đổi về chức năng thường trải dài qua nhiều tầng trong các loại kiến trúc này—đòi hỏi thay đổi ở các tầng trình bày, ứng dụng và dữ liệu. Vấn đề này càng trầm trọng hơn nếu kiến trúc còn nhiều tầng hơn cả ví dụ đơn giản trong Hình 1-2; thường thì mỗi tầng lại được chia nhỏ thành các tầng con.

By making our services end-to-end slices of business functionality, we ensure that our architecture is arranged to make changes to business functionality as efficient as possible. Arguably, with microservices we have made a decision to prioritize high cohesion of business functionality over high cohesion of technical functionality.

Bằng cách làm cho các service của mình trở thành những lát cắt đầu-cuối (end-to-end) của chức năng nghiệp vụ, chúng ta đảm bảo rằng kiến trúc của mình được sắp xếp sao cho việc thay đổi chức năng nghiệp vụ hiệu quả nhất có thể. Có thể nói, với microservices chúng ta đã đưa ra quyết định ưu tiên cohesion cao của chức năng nghiệp vụ hơn là cohesion cao của chức năng kỹ thuật.

3 For an in-depth introduction to domain-driven design, see *Domain-Driven Design* by Eric Evans (Addison- Wesley)—or for a more condensed overview, see *Domain-Driven Design Distilled* by Vaughn Vernon (Addison-Wesley).

3 Để có một phần giới thiệu chuyên sâu về domain-driven design, xem *Domain-Driven Design* của Eric Evans (Addison-Wesley)—hoặc để có một cái nhìn tổng quan cô đọng hơn, xem *Domain-Driven Design Distilled* của Vaughn Vernon (Addison-Wesley).

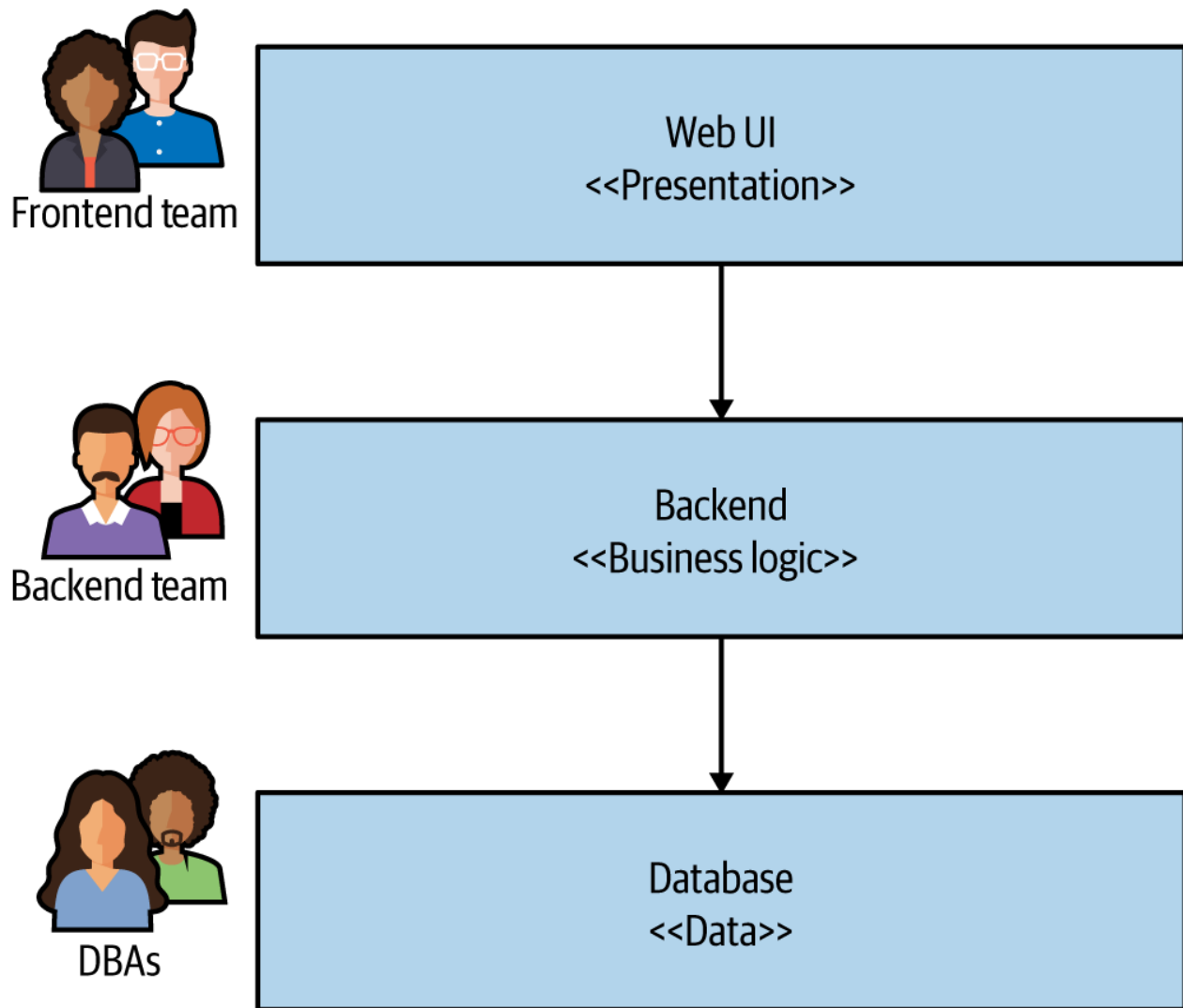


Figure 1-2. A traditional three-tiered architecture

Hình 1-2. Một kiến trúc ba tầng truyền thống

We will come back to the interplay of domain-driven design and how it interacts with organizational design later in this chapter.

Chúng ta sẽ quay lại sự tương tác giữa domain-driven design và cách nó liên hệ với thiết kế tổ chức ở phần sau của chương này.

Owning Their Own State — Tự sở hữu trạng thái của riêng mình

One of the things I see people having the hardest time with is the idea that microservices should avoid the use of shared databases. If a microservice wants to access data held by another microservice, it should go and ask that second microservice for the data. This gives the microservices the ability to decide what is shared and what is hidden, which allows us to clearly separate functionality that can change freely (our internal implementation) from the functionality that we want to change infrequently (the external contract that the consumers use).

Một trong những điều mà tôi thấy mọi người khó tiếp nhận nhất là ý tưởng rằng

microservices nên tránh việc sử dụng cơ sở dữ liệu dùng chung. Nếu một microservice muốn truy cập dữ liệu do một microservice khác nắm giữ, nó nên đi hỏi microservice thứ hai đó để lấy dữ liệu. Điều này trao cho các microservice khả năng quyết định cái gì được chia sẻ và cái gì được ẩn đi, cho phép chúng ta phân tách rõ ràng chức năng có thể thay đổi tự do (phần triển khai bên trong của chúng ta) khỏi chức năng mà chúng ta muốn thay đổi không thường xuyên (hợp đồng bên ngoài mà các consumer sử dụng).

If we want to make independent deployability a reality, we need to ensure that we limit backward-incompatible changes to our microservices. If we break compatibility with upstream consumers, we will force them to change as well. Having a clean delineation between internal implementation detail and an external contract for a microservice can help reduce the need for backward-incompatible changes.

Nếu chúng ta muốn biến khả năng triển khai độc lập thành hiện thực, chúng ta cần đảm bảo rằng mình hạn chế các thay đổi phá vỡ tương thích ngược đối với các microservice. Nếu chúng ta phá vỡ tương thích với các consumer phía trên, chúng ta sẽ buộc họ cũng phải thay đổi theo. Việc phân định rạch ròi giữa chi tiết triển khai bên trong và hợp đồng bên ngoài của một microservice có thể giúp giảm nhu cầu phải thực hiện các thay đổi phá vỡ tương thích ngược.

Hiding internal state in a microservice is analogous to the practice of **encapsulation** in object-oriented (OO) programming. Encapsulation of data in OO systems is an example of information hiding in action.

Việc ẩn trạng thái bên trong của một microservice tương tự như thực hành đóng gói (encapsulation) trong lập trình hướng đối tượng (object-oriented, OO). Việc đóng gói dữ liệu trong các hệ thống OO là một ví dụ về che giấu thông tin trong thực tế.



Don't share databases unless you really need to. And even then do everything you can to avoid it. In my opinion, sharing databases is one of the worst things you can do if you're trying to achieve independent deployability.

Đừng dùng chung cơ sở dữ liệu trừ khi bạn thực sự cần. Và ngay cả khi đó, hãy làm mọi thứ có thể để tránh điều này. Theo ý kiến của tôi, việc dùng chung cơ sở dữ liệu là một

trong những điều tồi tệ nhất bạn có thể làm nếu đang cố gắng đạt được khả năng triển khai độc lập.

As discussed in the previous section, we want to think of our services as end-to-end slices of business functionality that, where appropriate, encapsulate **user interface** (UI), business logic, and data. This is because we want to reduce the effort needed to change business-related functionality. The encapsulation of data and behavior in this way gives us high cohesion of business functionality. By hiding the database that backs our service, we also ensure that we reduce coupling. We come back to coupling and cohesion in Chapter 2.

Như đã bàn ở phần trước, chúng ta muốn nghĩ về các service của mình như những lát cắt đầu-cuối của chức năng nghiệp vụ mà, ở những nơi phù hợp, đóng gói giao diện người dùng (UI), logic nghiệp vụ và dữ liệu. Đây là vì chúng ta muốn giảm công sức cần thiết để thay đổi chức năng liên quan đến nghiệp vụ. Việc đóng gói dữ liệu và hành vi theo cách này mang lại cho chúng ta cohesion cao của chức năng nghiệp vụ. Bằng cách ẩn đi cơ sở dữ liệu hậu thuẫn cho service của mình, chúng ta cũng đảm bảo rằng mình giảm được coupling. Chúng ta sẽ quay lại với coupling và cohesion trong Chương 2.

Size — Kích thước

“How big should a microservice be?” is one of the most common questions I hear. Considering the word “micro” is right there in the name, this comes as no surprise. However, when you get into what makes microservices work as a type of architecture, the concept of size is actually one of the least interesting aspects.

“Một microservice nên lớn cỡ nào?” là một trong những câu hỏi phổ biến nhất mà tôi nghe được. Xét rằng từ “micro” ngay trong cái tên, điều này chẳng có gì đáng ngạc nhiên. Tuy nhiên, khi bạn đi sâu vào điều gì khiến microservices vận hành được như một loại kiến trúc, thì khái niệm kích thước thực ra lại là một trong những khía cạnh ít thú vị nhất.

How do you measure size? By counting lines of code? That doesn’t make much sense to me. Something that might require 25 lines of code in Java could be written in 10 lines of Clojure. That’s not to say Clojure is better or worse than Java; some languages are simply more expressive than others.

Bạn đo kích thước bằng cách nào? Bằng cách đếm số dòng mã (lines of code)? Với tôi điều đó chẳng mấy ý nghĩa. Một thứ có thể cần 25 dòng mã trong Java lại có thể được viết trong 10 dòng Clojure. Điều này không có nghĩa Clojure tốt hơn hay tệ hơn Java; một số ngôn ngữ đơn giản là biểu cảm hơn những ngôn ngữ khác.

James Lewis, technical director at Thoughtworks, has been known to say that “a microservice should be as big as my head.” At first glance, this doesn’t seem terribly helpful. After all, how big is James’s head exactly? The rationale behind this statement is that a microservice should be kept to the size at which it can be easily understood. The challenge, of course, is that different people’s ability to understand something isn’t always the same, and as such you’ll need to make your own judgment regarding what size works for you. An experienced team may be able to better manage a larger codebase than another team could. So perhaps it would be better to read James’s quote here as “a microservice should be as big as your head.”

James Lewis, giám đốc kỹ thuật tại Thoughtworks, từng có câu nói rằng “một microservice nên lớn bằng cái đầu của tôi”. Thoạt nhìn, điều này có vẻ không hữu ích cho lắm. Suy cho cùng, đầu của James lớn cỡ nào? Lý lẽ đằng sau câu nói này là một microservice nên được giữ ở kích thước mà nó có thể được hiểu một cách dễ dàng. Tất nhiên, thách thức là khả năng hiểu một thứ gì đó của mỗi người không phải lúc nào cũng như nhau, và do đó bạn

sẽ cần tự đưa ra phán đoán về kích thước nào phù hợp với mình. Một nhóm giàu kinh nghiệm có thể quản lý một codebase lớn hơn tốt hơn so với một nhóm khác. Vì vậy có lẽ nên đọc câu nói của James ở đây là “một microservice nên lớn bằng cái đầu của bạn”.

I think the closest I get to “size” having any meaning in terms of microservices is something that Chris Richardson, the author of *Microservice Patterns* (Manning Publications), once said—the goal of microservices is to have “as small an **interface** as possible.” That aligns with the concept of information hiding again, but it does represent an attempt to find meaning in the term “microservices” that wasn’t there initially. When the term was first used to define these architectures, the focus, at least initially, was not specifically on the size of the interfaces.

Tôi nghĩ điều gần nhất mà tôi tiến tới việc cho “kích thước” có một ý nghĩa nào đó về mặt microservices là điều mà Chris Richardson, tác giả của *Microservice Patterns* (Manning Publications), từng nói—mục tiêu của microservices là có “một interface nhỏ nhất có thể”. Điều đó lại một lần nữa phù hợp với khái niệm che giấu thông tin, nhưng nó đại diện cho một nỗ lực tìm ý nghĩa trong thuật ngữ “microservices” mà ban đầu vốn không có. Khi thuật ngữ này lần đầu được dùng để định nghĩa các kiến trúc này, sự tập trung, ít nhất ở thời điểm ban đầu, không nhằm cụ thể vào kích thước của các interface.

Ultimately, the concept of size is highly contextual. Speak to a person who has worked on a system for 15 years, and they’ll feel that their system with 100,000 lines of code is really easy to understand. Ask the opinion of someone brand-new to the project, and they’ll feel it’s much too big. Likewise, ask a company that has just embarked on its microservice transition and has perhaps 10 or fewer microservices, and you’ll get a different answer than you would from a similar-sized company for which microservices have been the norm for many years and that now has hundreds.

Rốt cuộc, khái niệm kích thước phụ thuộc rất nhiều vào ngữ cảnh. Hãy nói chuyện với một người đã làm việc trên một hệ thống suốt 15 năm, và họ sẽ cảm thấy rằng hệ thống của mình với 100.000 dòng mã thực sự rất dễ hiểu. Hãy hỏi ý kiến của một người hoàn toàn mới với dự án, và họ sẽ cảm thấy nó quá lớn. Tương tự, hãy hỏi một công ty vừa mới bắt đầu hành trình chuyển đổi sang microservice và có lẽ có 10 microservice trở xuống, và bạn sẽ nhận được câu trả lời khác với câu trả lời từ một công ty có quy mô tương tự nhưng đối với họ microservices đã là chuẩn mực trong nhiều năm và giờ có hàng trăm cái.

I urge people not to worry about size. When you are first starting out, it’s much more important that you focus on two key things. First, how many microservices can you handle? As you have more services, the complexity of your system will increase, and you’ll need to learn new skills (and perhaps adopt new technology) to cope with this. A move to microservices will introduce new sources of complexity, with all the challenges this can bring. It’s for this reason that I am a strong advocate for incremental migration to a microservice architecture. Second, how do you define microservice boundaries to get the most out of them, without everything becoming a horribly coupled mess? These topics are much more important to focus on when you start your journey.

Tôi khuyên mọi người đừng lo lắng về kích thước. Khi mới bắt đầu, điều quan trọng hơn nhiều là bạn tập trung vào hai điều then chốt. Thứ nhất, bạn có thể xử lý được bao nhiêu microservice? Khi bạn có nhiều service hơn, độ phức tạp của hệ thống sẽ tăng lên, và bạn sẽ cần học các kỹ năng mới (và có lẽ áp dụng công nghệ mới) để đối phó với điều này. Một bước chuyển sang microservices sẽ đưa vào những nguồn phức tạp mới, với tất cả những thách thức mà nó có thể mang lại. Chính vì lý do này mà tôi là một người ủng hộ mạnh mẽ việc di trú dần dần (incremental migration) sang kiến trúc microservice. Thứ hai, bạn định nghĩa ranh giới microservice như thế nào để tận dụng tối đa chúng, mà không khiến mọi thứ trở thành một mớ bòng bong coupling khủng khiếp? Những chủ đề này quan trọng hơn nhiều để tập trung vào khi bạn bắt đầu hành trình của mình.

Flexibility — Tính linh hoạt

Another quote from James Lewis is that “microservices buy you options.” Lewis was being deliberate with his words—they buy you options. They have a cost, and you must decide whether the cost is worth the options you want to take up. The resulting flexibility on a number of axes—organizational, technical, scale, **robustness**—can be incredibly appealing.

Một câu nói khác của James Lewis là “microservices mua cho bạn các lựa chọn”. Lewis đã cố ý chọn từ ngữ—chúng mua cho bạn các lựa chọn. Chúng có một cái giá, và bạn phải quyết định liệu cái giá đó có xứng đáng với những lựa chọn mà bạn muốn tận dụng hay không. Tính linh hoạt thu được trên nhiều trục—tổ chức, kỹ thuật, quy mô, độ vững chắc (robustness)—có thể vô cùng hấp dẫn.

We don’t know what the future holds, so we’d like an architecture that can theoretically help us solve whatever problems we might face down the road. Finding a balance between keeping your options open and bearing the cost of architectures like this can be a real art.

Chúng ta không biết tương lai sẽ ra sao, vì vậy chúng ta muốn một kiến trúc mà về lý thuyết có thể giúp chúng ta giải quyết bất kỳ vấn đề nào có thể đối mặt trên chặng đường phía trước. Tìm được sự cân bằng giữa việc giữ cho các lựa chọn của mình rộng mở và việc gánh chịu chi phí của những kiến trúc như thế này có thể là một nghệ thuật thực thụ.

Think of adopting microservices as less like flipping a switch, and more like turning a dial. As you turn up the dial, and you have more microservices, you have increased flexibility. But you likely ramp up the pain points too. This is yet another reason I strongly advocate incremental adoption of microservices. By turning up the dial gradually, you are better able to assess the impact as you go, and to stop if required.

Hãy nghĩ về việc áp dụng microservices không giống như bật một công tắc, mà giống như vặn một núm xoay. Khi bạn vặn núm lên, và bạn có nhiều microservice hơn, bạn có được tính linh hoạt tăng lên. Nhưng bạn cũng có khả năng làm tăng các điểm đau (pain point). Đây lại là một lý do nữa khiến tôi mạnh mẽ ủng hộ việc áp dụng microservices dần dần. Bằng cách vặn núm lên từ từ, bạn có thể đánh giá tác động tốt hơn khi tiến hành, và dừng lại nếu cần.

Alignment of Architecture and Organization — Sự đồng bộ giữa Kiến trúc và Tổ chức

MusicCorp, an ecommerce company that sells CDs online, uses the simple three-tiered architecture shown earlier in Figure 1-2. We’ve decided to move MusicCorp kicking and screaming into the 21st century, and as part of that, we’re assessing the existing **system architecture**. We have a web-based UI, a business logic layer in the form of a monolithic **backend**, and data storage in a traditional database. These layers, as is common, are owned by different teams. We’ll be coming back to the trials and tribulations of MusicCorp throughout the book.

MusicCorp, một công ty thương mại điện tử bán CD trực tuyến, sử dụng kiến trúc ba tầng đơn giản đã thấy trước đó trong Hình 1-2. Chúng ta đã quyết định kéo MusicCorp—dù nó có giãy giụa và la hét—vào thế kỷ 21, và như một phần của việc đó, chúng ta đang đánh giá kiến trúc hệ thống hiện có. Chúng ta có một UI dựa trên web, một tầng logic nghiệp vụ dưới dạng một backend monolith, và lưu trữ dữ liệu trong một cơ sở dữ liệu truyền thống. Các tầng này, như thường thấy, được sở hữu bởi các nhóm khác nhau. Chúng ta sẽ quay lại với những thử thách và gian truân của MusicCorp xuyên suốt cuốn sách.

We want to make a simple update to our functionality: we want to allow our customers to specify their favorite genre of music. This update requires us to change the UI to show the genre choice UI, the backend service to allow for the genre to be surfaced to the UI and for the value to be changed, and the database to accept this change. These changes will need to be managed by each team and deployed in the correct order, as outlined in Figure 1-3 .

Chúng ta muốn thực hiện một cập nhật đơn giản cho chức năng của mình: chúng ta muốn cho phép khách hàng chỉ định thể loại âm nhạc yêu thích của họ. Cập nhật này đòi hỏi chúng ta thay đổi UI để hiển thị phần giao diện chọn thể loại, thay đổi service backend để cho phép thể loại được hiển thị lên UI và cho phép thay đổi giá trị, và thay đổi cơ sở dữ liệu để chấp nhận thay đổi này. Những thay đổi này sẽ cần được quản lý bởi từng nhóm và triển khai theo đúng thứ tự, như được phác họa trong Hình 1-3.

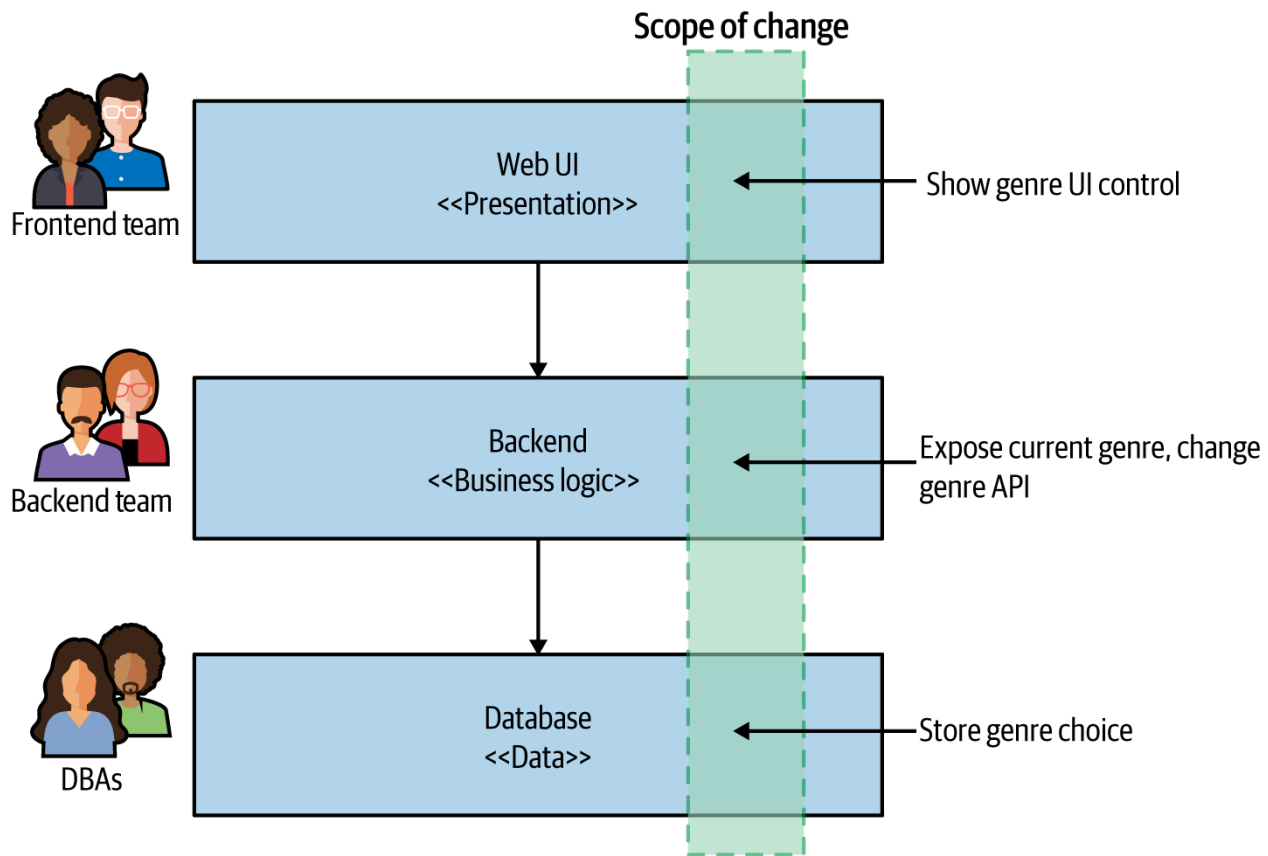


Figure 1-3. Making a change across all three tiers is more involved

Hình 1-3. Thực hiện một thay đổi xuyên suốt cả ba tầng thì phức tạp hơn

Now this architecture isn't bad. All architecture ends up getting optimized around a set of goals. Three-tiered architecture is so common partly because it is universal— everyone has heard about it. So the tendency to pick a common architecture that you might have seen elsewhere is often one reason we keep seeing this pattern. But I think the biggest reason we see this architecture again and again is because it is based on how we organize our teams.

Giờ thì kiến trúc này không hề tệ. Mọi kiến trúc tốt cuộc đều được tối ưu xoay quanh một tập mục tiêu. Kiến trúc ba tầng phổ biến đến vậy một phần vì nó mang tính phổ quát—ai cũng đã nghe nói về nó. Vì vậy khuynh hướng chọn một kiến trúc phổ biến mà bạn có thể đã thấy ở đâu đó thường là một lý do khiến chúng ta cứ thấy đi thấy lại pattern này. Nhưng

tôi nghĩ lý do lớn nhất khiến chúng ta thấy kiến trúc này lặp đi lặp lại là vì nó dựa trên cách chúng ta tổ chức các nhóm của mình.

The now famous Conway's law states the following:

Định luật Conway nay đã nổi tiếng phát biểu như sau:

Organizations which design systems...are constrained to produce designs which are copies of the communication structures of these organizations. —Melvin Conway, “How Do Committees Invent?”

Các tổ chức thiết kế hệ thống...bị ràng buộc phải tạo ra những thiết kế là bản sao của các cấu trúc giao tiếp của chính các tổ chức đó. —Melvin Conway, “How Do Committees Invent?”

The three-tiered architecture is a good example of this law in action. In the past, the primary way IT organizations grouped people was in terms of their core competency:

Kiến trúc ba tầng là một ví dụ điển hình về định luật này trong thực tế. Trong quá khứ, cách chính mà các tổ chức IT nhóm con người lại với nhau là dựa trên năng lực cốt lõi của họ:

database admins were in a team with other database admins; Java developers were in a team with other Java developers; and **frontend** developers (who nowadays know exotic things like JavaScript and native mobile application development) were in yet another team. We group people based on their core competency, so we create IT assets that can be aligned to those teams.

quản trị viên cơ sở dữ liệu (database admin) ở chung một nhóm với các quản trị viên cơ sở dữ liệu khác; lập trình viên Java ở chung một nhóm với các lập trình viên Java khác; và lập trình viên frontend (những người ngày nay biết những thứ kỳ lạ như JavaScript và phát triển ứng dụng di động native) lại ở một nhóm khác. Chúng ta nhóm con người dựa trên năng lực cốt lõi của họ, nên chúng ta tạo ra các tài sản IT có thể đồng bộ với những nhóm đó.

That explains why this architecture is so common. It's not bad; it's just optimized around one set of forces—how we traditionally grouped people, around familiarity. But the forces have changed. Our aspirations around our software have changed. We now group people in poly-skilled teams to reduce handoffs and silos. We want to ship software much more quickly than ever before. That is driving us to make different choices about the way we organize our teams, so that we organize them in terms of the way we break our systems apart.

Điều đó giải thích vì sao kiến trúc này lại phổ biến đến vậy. Nó không tệ; nó chỉ được tối ưu xoay quanh một tập lực—cách chúng ta nhóm con người theo truyền thống, dựa trên sự quen thuộc. Nhưng các lực đã thay đổi. Khát vọng của chúng ta về phần mềm đã thay đổi. Giờ đây chúng ta nhóm con người vào các nhóm đa kỹ năng (poly-skilled) để giảm việc bàn giao (handoff) và các ốc đảo (silo). Chúng ta muốn chuyển giao phần mềm nhanh hơn nhiều so với trước đây. Điều đó đang thúc đẩy chúng ta đưa ra những lựa chọn khác về cách tổ chức các nhóm, sao cho chúng ta tổ chức chúng theo cách mà chúng ta chia tách các hệ thống của mình.

Most changes that we are asked to make to our system relate to changes in business functionality. But in Figure 1-3, our business functionality is in effect spread across all three tiers, increasing the chance that a change in functionality will cross layers. This is an architecture that has high cohesion of related technology but low cohesion of business functionality. If we want to make it easier to make changes, instead we need to change how we group code, choosing cohesion of business functionality rather than technology. Each service may or may not end up containing a mix of these three layers, but that is a local service implementation concern.

Hầu hết các thay đổi mà chúng ta được yêu cầu thực hiện đối với hệ thống đều liên quan đến những thay đổi trong chức năng nghiệp vụ. Nhưng trong Hình 1-3, chức năng nghiệp vụ của chúng ta thực chất bị trải dài khắp cả ba tầng, làm tăng khả năng một thay đổi về chức năng sẽ cắt ngang qua các tầng. Đây là một kiến trúc có cohesion cao về công nghệ liên quan nhưng cohesion thấp về chức năng nghiệp vụ. Nếu chúng ta muốn làm cho việc thực hiện các thay đổi dễ dàng hơn, thay vào đó chúng ta cần thay đổi cách nhóm mã, chọn cohesion của chức năng nghiệp vụ thay vì của công nghệ. Mỗi service có thể có hoặc không cuối cùng chứa một sự pha trộn của ba tầng này, nhưng đó là một mối quan tâm triển khai cục bộ của service.

Let's compare this with a potential alternative architecture, illustrated in Figure 1-4 . Rather than a horizontally layered architecture and organization, we instead break down our organization and architecture along vertical business lines. Here we see a dedicated team that has full-end-to-end responsibility for making changes to aspects of the customer profile, which ensures that the scope of change in this example is limited to one team.

Hãy so sánh điều này với một kiến trúc thay thế tiềm năng, được minh họa trong Hình 1-4. Thay vì một kiến trúc và tổ chức phân tầng theo chiều ngang, thay vào đó chúng ta chia nhỏ tổ chức và kiến trúc của mình dọc theo các tuyến nghiệp vụ theo chiều dọc. Ở đây chúng ta thấy một nhóm chuyên trách có trách nhiệm đầu-cuối hoàn toàn cho việc thực hiện các thay đổi đối với các khía cạnh của hồ sơ khách hàng (customer profile), điều này đảm bảo rằng phạm vi thay đổi trong ví dụ này được giới hạn trong một nhóm.

As an implementation, this could be achieved through a single microservice owned by the profile team that exposes a UI to allow customers to update their information, with state of the customer also stored within this microservice. The choice of a favorite genre is associated with a given customer, so this change is much more localized. In Figure 1-5 , we also show the list of available genres being fetched from a Catalog microservice, something that would likely already be in place. We also see a new Recommendation microservice accessing our favorite genre information, something that could easily follow in a subsequent release.

Về mặt triển khai, điều này có thể đạt được thông qua một microservice duy nhất do nhóm hồ sơ sở hữu, phơi bày một UI để cho phép khách hàng cập nhật thông tin của họ, với trạng thái của khách hàng cũng được lưu trữ bên trong microservice này. Lựa chọn thể loại yêu thích gắn liền với một khách hàng cụ thể, nên thay đổi này mang tính cục bộ hơn nhiều. Trong Hình 1-5, chúng ta cũng cho thấy danh sách các thể loại có sẵn được lấy về từ một microservice Catalog, thứ mà có lẽ đã sẵn có. Chúng ta cũng thấy một microservice Recommendation mới truy cập thông tin thể loại yêu thích của chúng ta, thứ có thể dễ dàng được đưa vào trong một đợt phát hành sau đó.

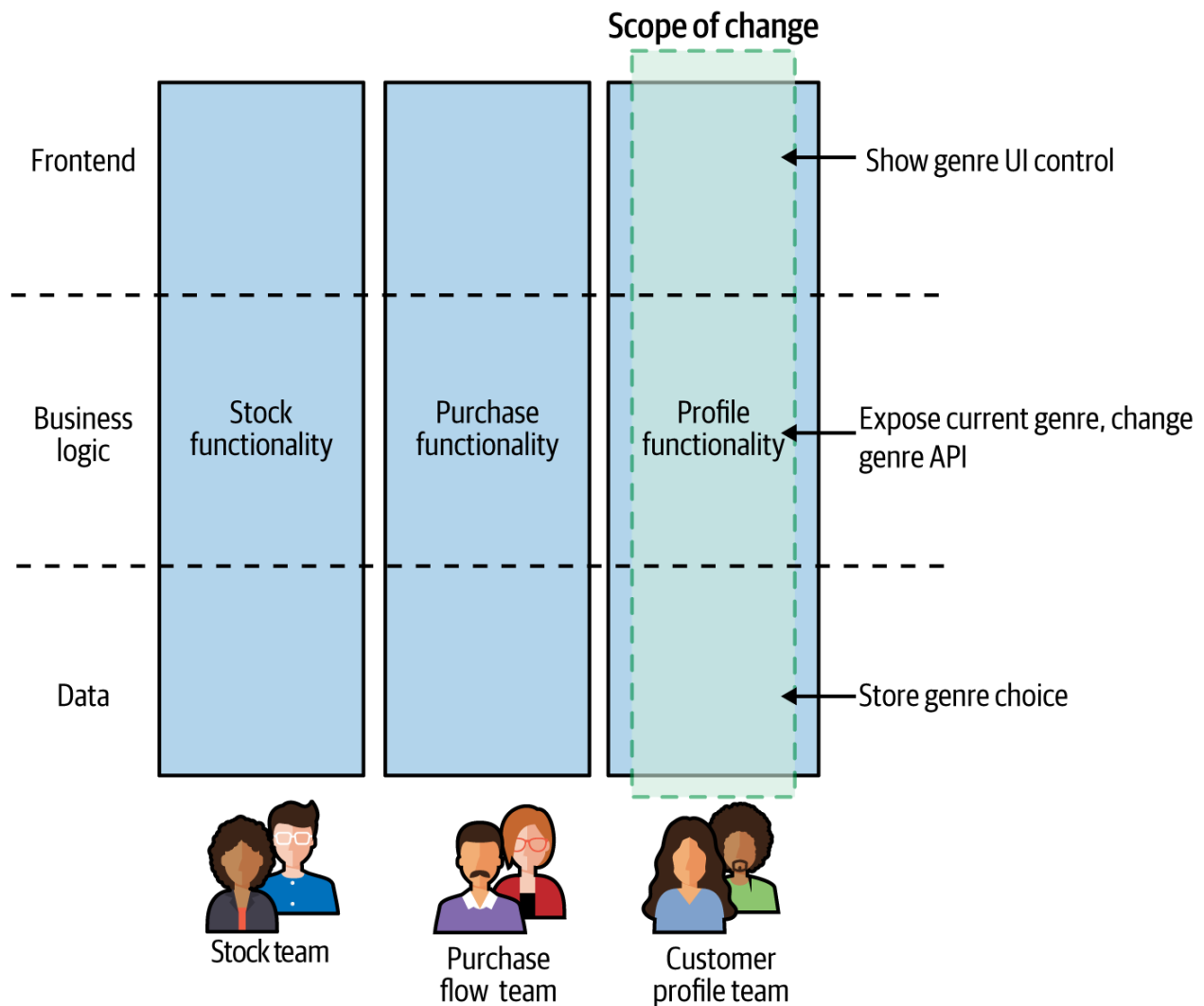


Figure 1-4. The UI is broken apart and is owned by a team that also manages the serverside functionality that supports the UI

Hình 1-4. UI được tách ra và được sở hữu bởi một nhóm cũng quản lý chức năng phía máy chủ hỗ trợ cho UI

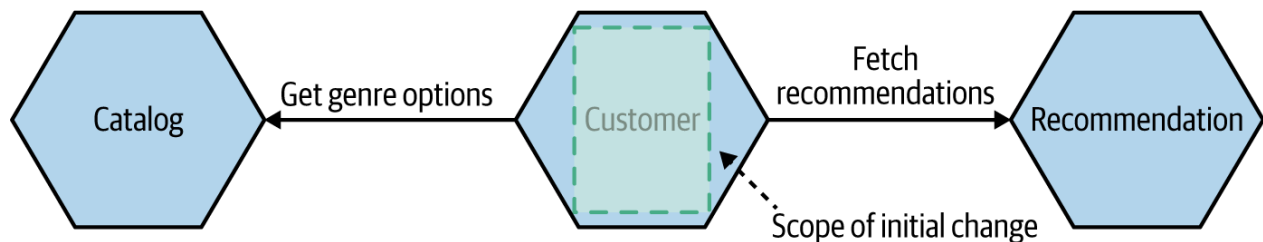


Figure 1-5. A dedicated Customer microservice can make it much easier to record the favorite musical genre for a customer

Hình 1-5. Một microservice Customer chuyên trách có thể làm cho việc ghi nhận thể loại âm nhạc yêu thích của một khách hàng dễ dàng hơn nhiều

In such a situation, our Customer microservice encapsulates a thin slice of each of the three tiers—it

has a bit of UI, a bit of application logic, and a bit of data storage. Our business domain becomes the primary force driving our system architecture, hopefully making it easier to make changes, as well as making it easier for us to align our teams to lines of business within the organization.

Trong tình huống như vậy, microservice Customer của chúng ta đóng gói một lát cắt mỏng của mỗi tầng trong ba tầng—nó có một chút UI, một chút logic ứng dụng, và một chút lưu trữ dữ liệu. Miền nghiệp vụ của chúng ta trở thành lực chính dẫn dắt kiến trúc hệ thống, hy vọng làm cho việc thực hiện các thay đổi dễ dàng hơn, cũng như giúp chúng ta dễ dàng đồng bộ các nhóm với các tuyến nghiệp vụ trong tổ chức hơn.

Often, the UI is not provided directly by the microservice, but even if this is the case, we would expect the portion of the UI related to this functionality to still be owned by the Customer Profile Team, as Figure 1-4 indicates. This concept of a team owning an end-to-end slice of user-facing functionality is gaining traction. The book *Team Topologies 4* introduces the idea of a **stream-aligned team**, which embodies this concept:

Thông thường, UI không được cung cấp trực tiếp bởi microservice, nhưng ngay cả khi đúng là như vậy, chúng ta vẫn mong đợi rằng phần UI liên quan đến chức năng này vẫn được sở hữu bởi Nhóm Hồ sơ Khách hàng (Customer Profile Team), như Hình 1-4 chỉ ra. Khái niệm một nhóm sở hữu một lát cắt đầu-cuối của chức năng hướng tới người dùng đang ngày càng được chú ý. Cuốn sách *Team Topologies 4* giới thiệu ý tưởng về một stream-aligned team, vốn thể hiện rõ khái niệm này:

A stream-aligned team is a team aligned to a single, valuable stream of work...[T]he team is empowered to **build** and deliver customer or user value as quickly, safely, and independently as possible, without requiring hand-offs to other teams to perform parts of the work. The teams shown in Figure 1-4 would be stream-aligned teams, a concept we'll explore in more depth in Chapters 14 and 15, including how these types of organizational structures work in practice, and how they align with microservices.

Một stream-aligned team là một nhóm được đồng bộ với một luồng công việc giá trị, duy nhất...[N]hóm được trao quyền để xây dựng và chuyển giao giá trị cho khách hàng hoặc người dùng nhanh nhất, an toàn nhất và độc lập nhất có thể, mà không cần bàn giao cho các nhóm khác để thực hiện các phần của công việc. Các nhóm được thể hiện trong Hình 1-4 sẽ là các stream-aligned team, một khái niệm mà chúng ta sẽ khám phá sâu hơn trong Chương 14 và 15, bao gồm cả việc các loại cấu trúc tổ chức này hoạt động ra sao trong thực tế, và chúng đồng bộ với microservices như thế nào.

A Note on “Fake” Companies — Một lưu ý về các công ty “giả”

Throughout the book, at different stages, we will meet MusicCorp, FinanceCo, FoodCo, AdvertCo, and PaymentCo.

Xuyên suốt cuốn sách, ở các giai đoạn khác nhau, chúng ta sẽ gặp MusicCorp, FinanceCo, FoodCo, AdvertCo, và PaymentCo.

FoodCo, AdvertCo, and PaymentCo are real companies whose names I have changed for confidentiality reasons. In addition, when sharing information about these companies, I have often omitted certain details to provide more clarity. The real world is often messy. I've always strived, though, to remove only extraneous detail that wouldn't be helpful, while still ensuring that the underlying reality of the situation remains.

FoodCo, AdvertCo, và PaymentCo là các công ty có thật mà tôi đã đổi tên vì lý do bảo mật. Ngoài ra, khi chia sẻ thông tin về các công ty này, tôi thường lược bỏ một số chi tiết nhất

định để mang lại sự rõ ràng hơn. Thế giới thực thường lộn xộn. Tuy vậy, tôi luôn cố gắng chỉ loại bỏ những chi tiết không liên quan, không hữu ích, đồng thời vẫn đảm bảo rằng thực tế nền tảng của tình huống vẫn được giữ nguyên.

MusicCorp, on the other hand, is a fake company that is a composite of many organizations I have worked with. The stories I share around MusicCorp are reflections of real things I have seen, but they haven't all happened to the same company!

MusicCorp, mặt khác, là một công ty giả, một sự tổng hợp của nhiều tổ chức mà tôi đã từng làm việc cùng. Những câu chuyện tôi chia sẻ xoay quanh MusicCorp là sự phản ánh của những điều có thật mà tôi đã chứng kiến, nhưng không phải tất cả chúng đều xảy ra với cùng một công ty!

The Monolith — Monolith

We've spoken about microservices, but microservices are most often discussed as an architectural approach that is an alternative to monolithic architecture. To more clearly distinguish the microservice architecture, and to help you better understand whether microservices are worth considering, I should also discuss what exactly I mean by monoliths .

Chúng ta đã nói về microservices, nhưng microservices thường được bàn đến nhất như một cách tiếp cận kiến trúc thay thế cho kiến trúc monolith. Để phân biệt rõ ràng hơn kiến trúc microservice, và để giúp bạn hiểu rõ hơn liệu microservices có đáng cân nhắc hay không, tôi cũng nên thảo luận về việc chính xác thì tôi muốn nói gì khi nói về monolith.

When I talk about monoliths throughout this book, I am primarily referring to a unit of **deployment**. When all functionality in a system must be deployed together, I

Khi tôi nói về monolith xuyên suốt cuốn sách này, tôi chủ yếu đang nói đến một đơn vị triển khai (unit of deployment). Khi tất cả chức năng trong một hệ thống phải được triển khai cùng nhau, tôi

4 Matthew Skelton and Manuel Pais, Team Topologies (Portland, OR: IT Revolution, 2019).

4 Matthew Skelton và Manuel Pais, Team Topologies (Portland, OR: IT Revolution, 2019).

consider it a monolith. Arguably, multiple architectures fit this definition, but I'm going to discuss those I see most often: the single-process monolith, the **modular monolith**, and the **distributed monolith**.

coi nó là một monolith. Có thể nói nhiều kiến trúc phù hợp với định nghĩa này, nhưng tôi sẽ bàn đến những kiến trúc mà tôi thấy thường gặp nhất: single-process monolith, modular monolith, và distributed monolith.

The Single-Process Monolith — Single-Process Monolith

The most common example that comes to mind when discussing monoliths is a system in which all of the code is deployed as a single process , as in Figure 1-6 . You may have multiple instances of this process for robustness or **scaling** reasons, but fundamentally all the code is packed into a single process. In reality, these single-process systems can be simple distributed systems in their own right because they nearly always end up reading data from or storing data into a database, or presenting information to web or mobile applications.

Ví dụ phổ biến nhất nảy ra trong đầu khi bàn về monolith là một hệ thống trong đó toàn bộ mã được triển khai như một tiến trình duy nhất (single process), như trong Hình 1-6. Bạn có thể có nhiều thể hiện (instance) của tiến trình này vì lý do độ vững chắc hoặc mở rộng (scaling), nhưng về cơ bản tất cả mã đều được đóng gói vào một tiến trình duy nhất. Trên thực tế, các hệ thống single-process này bản thân chúng có thể là những hệ thống phân tán đơn giản, vì chúng gần như luôn rất cuộc đọc dữ liệu từ hoặc lưu dữ liệu vào một cơ sở dữ liệu, hoặc trình bày thông tin cho các ứng dụng web hay di động.

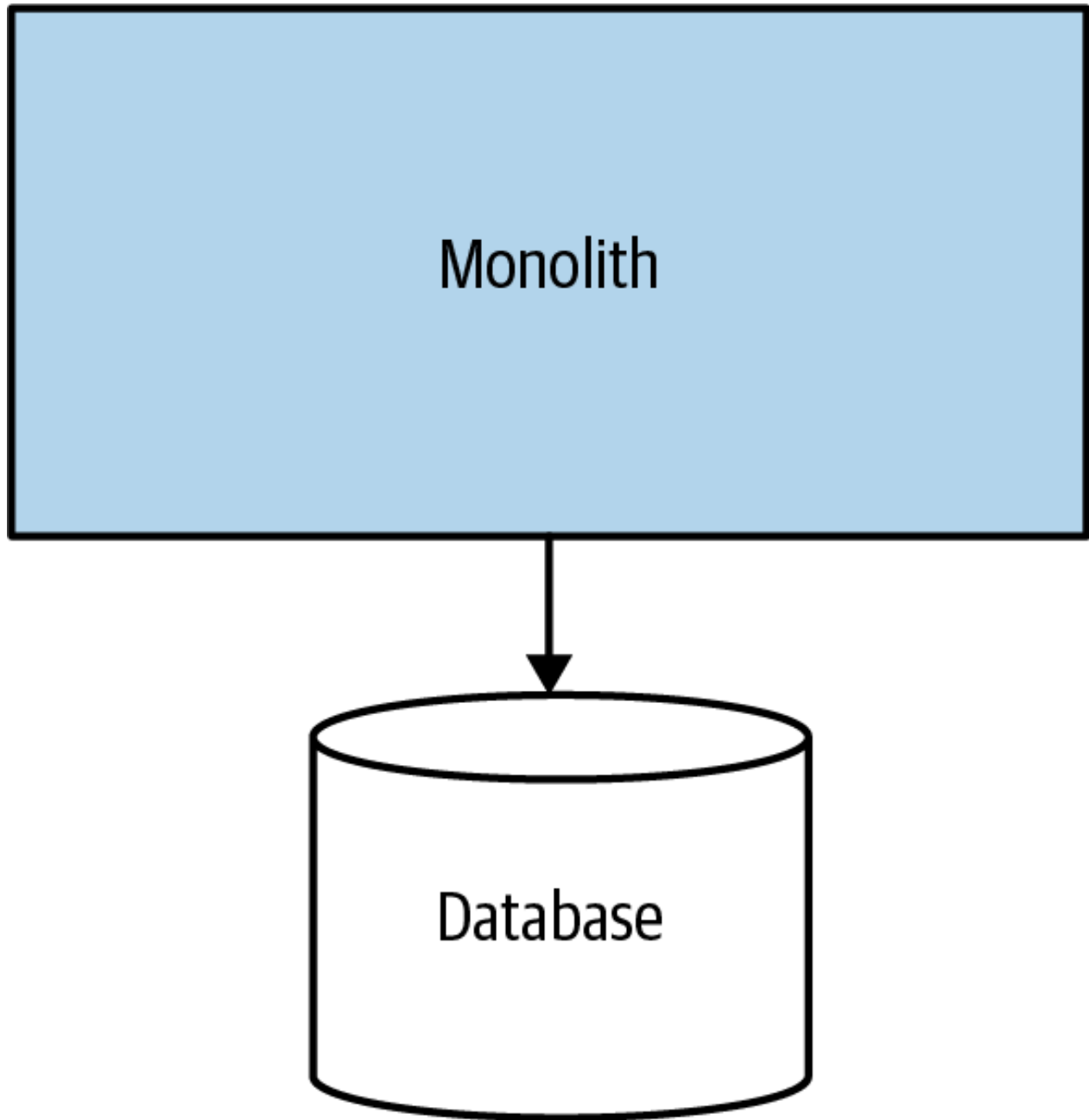


Figure 1-6. In a single-process monolith, all code is packaged into a single process

Hình 1-6. Trong một single-process monolith, tất cả mã được đóng gói vào một tiến trình duy nhất

Although this fits most people’s understanding of a classic monolith, most systems I encounter are somewhat more complex than this. You may have two or more monoliths that are tightly coupled to one another, potentially with some vendor software in the mix.

Mặc dù điều này phù hợp với cách hiểu của hầu hết mọi người về một monolith cổ điển, hầu hết các hệ thống tôi gặp đều phức tạp hơn thế đôi chút. Bạn có thể có hai hoặc nhiều monolith có coupling chặt với nhau, có thể có một số phần mềm của vendor lẫn vào trong đó.

A classic single-process monolithic deployment can make sense for many organizations. David Heinemeier Hansson, the creator of Ruby on Rails, has made the case effectively that such an architecture makes sense for smaller organizations.⁵ Even as the organization grows, however, the monolith can potentially grow with it, which brings us to the modular monolith.

Một đợt triển khai monolith single-process cổ điển có thể hợp lý đối với nhiều tổ chức. David Heinemeier Hansson, người tạo ra Ruby on Rails, đã lập luận một cách thuyết phục rằng một kiến trúc như vậy là hợp lý đối với các tổ chức nhỏ hơn.⁵ Tuy nhiên, ngay cả khi tổ chức phát triển lớn lên, monolith vẫn có thể phát triển cùng với nó, điều này đưa chúng ta đến với modular monolith.

⁵ David Heinemeier Hansson, “The Majestic Monolith,” Signal v. Noise, February 29, 2016, <https://oreil.ly/WwG1C>.

⁵ David Heinemeier Hansson, “The Majestic Monolith,” Signal v. Noise, 29 tháng 2, 2016, <https://oreil.ly/WwG1C>.

The Modular Monolith — Modular Monolith

As a subset of the single-process monolith, the modular monolith is a variation in which the single process consists of separate modules. Each module can be worked on independently, but all still need to be combined together for deployment, as shown in Figure 1-7. The concept of breaking software into modules is nothing new; modular software has its roots in work done around structured programming in the 1970s, and even further back than that. Nonetheless, this is an approach that I still don’t see enough organizations properly engage with.

Là một tập con của single-process monolith, modular monolith là một biến thể trong đó tiến trình duy nhất bao gồm các module riêng biệt. Mỗi module có thể được làm việc một cách độc lập, nhưng tất cả vẫn cần được kết hợp lại với nhau để triển khai, như được thể hiện trong Hình 1-7. Khái niệm chia phần mềm thành các module chẳng có gì mới; phần mềm dạng module có nguồn gốc từ công trình về lập trình có cấu trúc (structured programming) vào thập niên 1970, và thậm chí còn xa hơn thế nữa. Tuy vậy, đây vẫn là một cách tiếp cận mà tôi thấy chưa đủ nhiều tổ chức tiếp cận một cách đúng đắn.

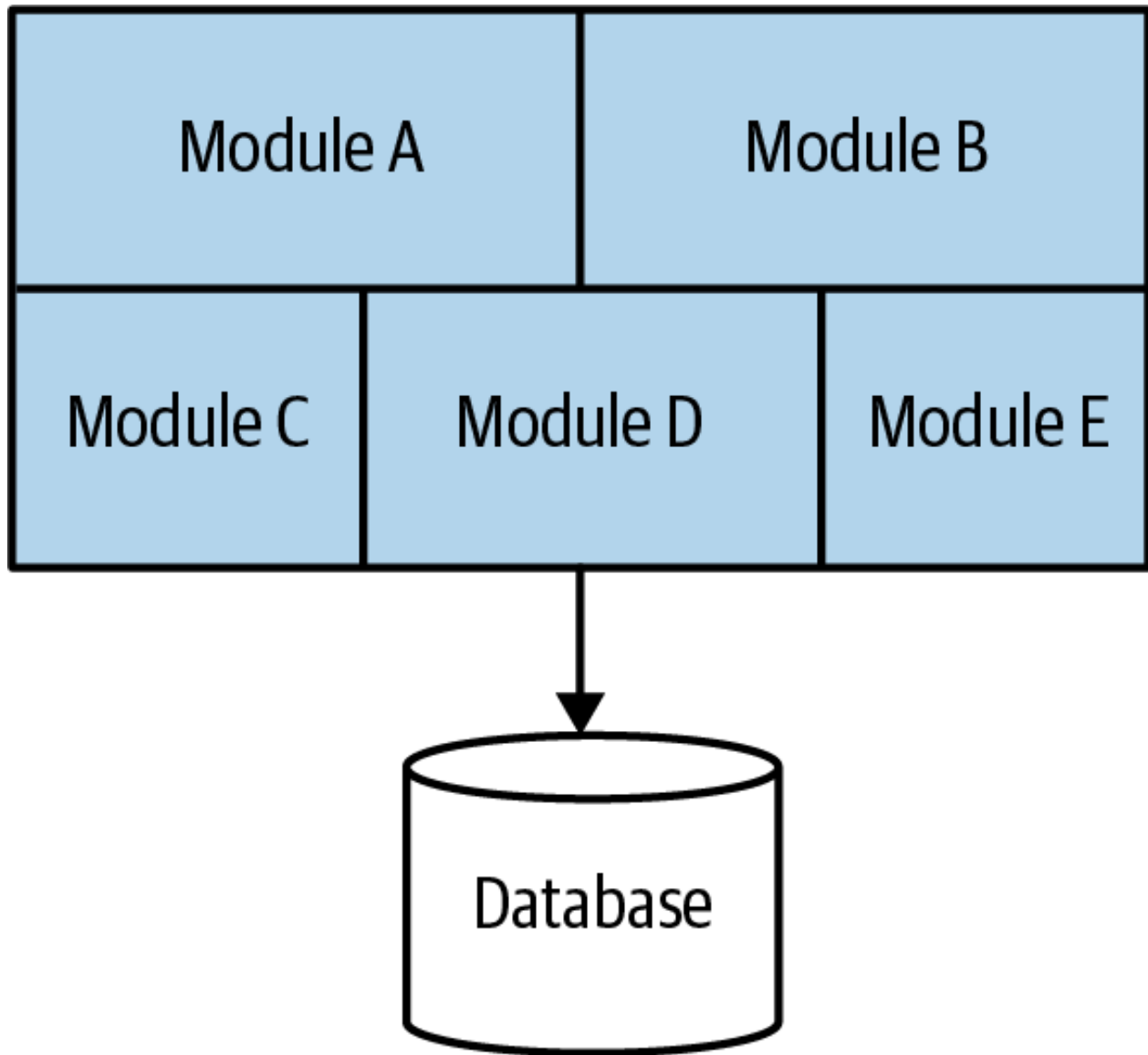


Figure 1-7. In a modular monolith, the code inside the process is divided into modules

Hình 1-7. Trong một modular monolith, mã bên trong tiến trình được chia thành các module

For many organizations, the modular monolith can be an excellent choice. If the module boundaries are well defined, it can allow for a high degree of parallel work, while avoiding the challenges of the more distributed microservice architecture by having a much simpler deployment topology. Shopify is a great example of an organization that has used this technique as an alternative to microservice decomposition, and it seems to work really well for that company. 6

Đối với nhiều tổ chức, modular monolith có thể là một lựa chọn tuyệt vời. Nếu các ranh giới module được định nghĩa tốt, nó có thể cho phép một mức độ làm việc song song cao, đồng thời tránh được các thách thức của kiến trúc microservice phân tán hơn nhờ có một cấu trúc triển khai (deployment topology) đơn giản hơn nhiều. Shopify là một ví dụ tuyệt vời về một tổ chức đã sử dụng kỹ thuật này như một phương án thay thế cho việc phân rã thành microservice, và nó dường như hoạt động rất tốt với công ty đó. 6

One of the challenges of a modular monolith is that the database tends to lack the decomposition we find in the code level, leading to significant challenges if you want to pull apart the monolith in the future. I have seen some teams attempt to push the idea of the modular monolith further by having the database decomposed along the same lines as the modules, as shown in Figure 1-8 .

Một trong những thách thức của modular monolith là cơ sở dữ liệu có xu hướng thiếu sự phân rã mà chúng ta thấy ở cấp độ mã, dẫn đến những thách thức đáng kể nếu sau này bạn muốn tách rời monolith. Tôi đã thấy một số nhóm cố gắng đẩy ý tưởng modular monolith đi xa hơn bằng cách cho phân rã cơ sở dữ liệu theo cùng các đường ranh giới như các module, như được thể hiện trong Hình 1-8.

6 For some useful insights into the thinking behind Shopify’s use of a modular monolith rather than microservices, watch “Deconstructing the Monolith” by Kirsten Westeinde.

6 Để có một số góc nhìn hữu ích về tư duy đằng sau việc Shopify sử dụng modular monolith thay vì microservices, hãy xem “Deconstructing the Monolith” của Kirsten Westeinde.

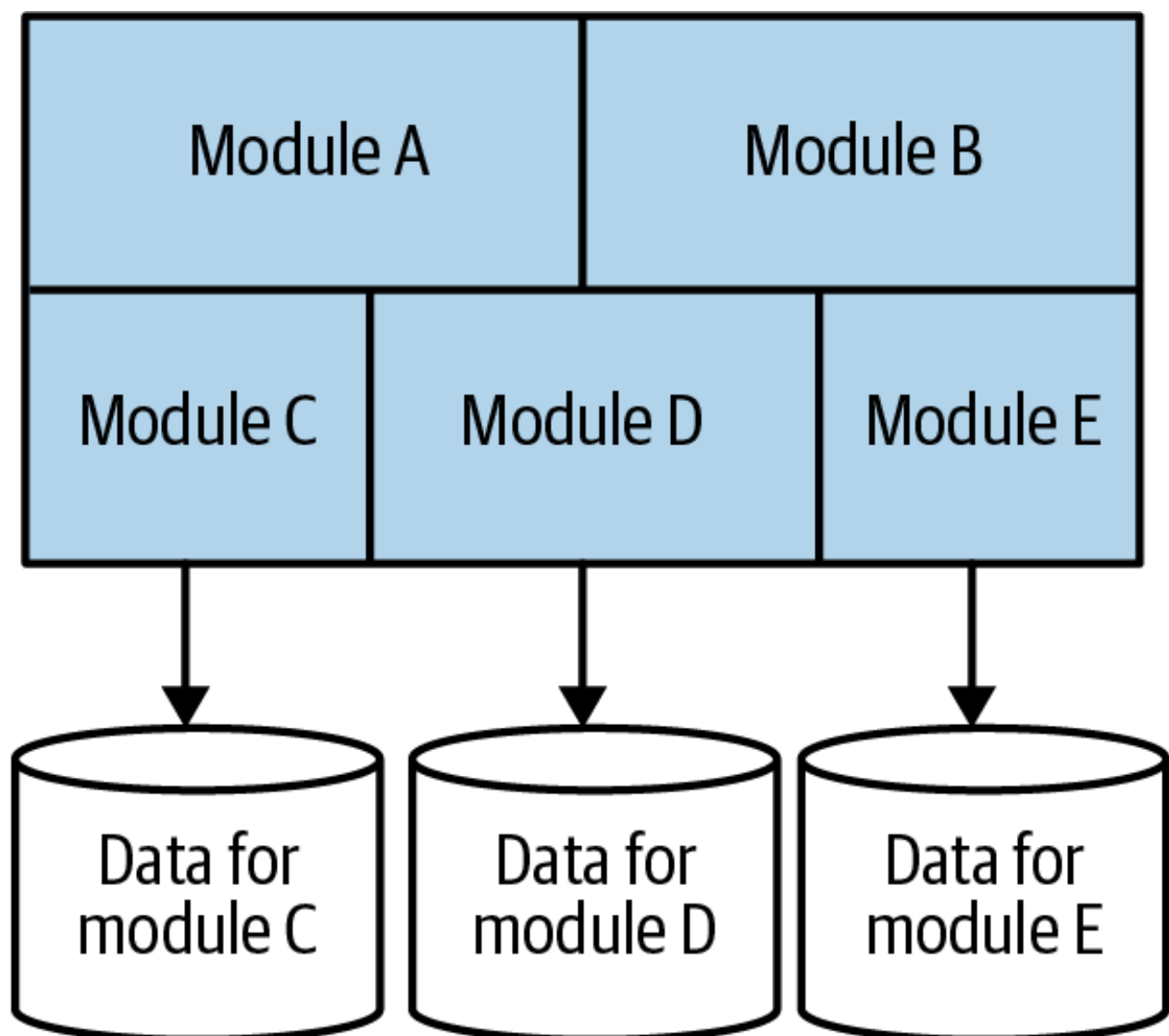


Figure 1-8. A modular monolith with a decomposed database

Hình 1-8. Một modular monolith với cơ sở dữ liệu đã được phân rã

The Distributed Monolith — Distributed Monolith

A **distributed system** is one in which the failure of a computer you didn't even know existed can render your own computer unusable. 7 —Leslie Lamport

Một hệ thống phân tán là hệ thống mà trong đó sự cố của một máy tính mà bạn thậm chí không biết là nó tồn tại có thể khiến chính máy tính của bạn không dùng được nữa. 7
—Leslie Lamport

A distributed monolith is a system that consists of multiple services, but for whatever reason, the entire system must be deployed together. A distributed monolith might well meet the definition of an SOA, but all too often, it fails to deliver on the promises of SOA. In my experience, a distributed monolith has all the disadvantages of a distributed system, and the disadvantages of a single-process monolith, without having enough of the upsides of either. Encountering a number of distributed monoliths in my work has in large part influenced my own interest in microservice architecture.

Một distributed monolith là một hệ thống bao gồm nhiều service, nhưng vì lý do nào đó, toàn bộ hệ thống phải được triển khai cùng nhau. Một distributed monolith có thể đáp ứng định nghĩa của một SOA, nhưng quá thường xuyên, nó lại không thực hiện được những lời hứa của SOA. Theo kinh nghiệm của tôi, một distributed monolith có tất cả những nhược điểm của một hệ thống phân tán, và những nhược điểm của một single-process monolith, mà lại không có đủ những mặt tích cực của cả hai. Việc gặp phải một số distributed monolith trong công việc đã ảnh hưởng phần lớn đến sự quan tâm của chính tôi đối với kiến trúc microservice.

Distributed monoliths typically emerge in an environment in which not enough focus was placed on concepts like information hiding and cohesion of business functionality. Instead, highly coupled architectures cause changes to ripple across service boundaries, and seemingly innocent changes that appear to be local in scope break other parts of the system.

Distributed monolith thường nảy sinh trong một môi trường mà sự tập trung không đủ được đặt vào các khái niệm như che giấu thông tin và cohesion của chức năng nghiệp vụ. Thay vào đó, những kiến trúc có coupling cao khiến các thay đổi lan tỏa qua các ranh giới service, và những thay đổi tưởng chừng vô hại có vẻ chỉ mang phạm vi cục bộ lại phá vỡ các phần khác của hệ thống.

Monoliths and Delivery Contention — Monolith và sự tranh chấp trong chuyển giao

As more and more people work in the same place, they get in one another's way—for example, different developers wanting to change the same piece of code, different teams wanting to push functionality live at different times (or to delay deployments), and confusion around who owns what and who makes decisions. A multitude of

Khi ngày càng nhiều người làm việc ở cùng một chỗ, họ cản trở lẫn nhau—chẳng hạn, các lập trình viên khác nhau muốn thay đổi cùng một đoạn mã, các nhóm khác nhau muốn đưa chức năng lên môi trường thật vào những thời điểm khác nhau (hoặc muốn hoãn các đợt triển khai), và sự nhầm lẫn về việc ai sở hữu cái gì và ai ra quyết định. Vô số

7 Leslie Lamport, email message to a DEC SRC bulletin board at 12:23:29 PDT on May 28, 1987.

7 Leslie Lamport, thư điện tử gửi đến một bảng tin DEC SRC lúc 12:23:29 PDT ngày 28 tháng 5, 1987.

studies have shown the challenges of confused lines of **ownership**.⁸ I refer to this problem as delivery contention.

ngiên cứu đã cho thấy các thách thức của những ranh giới sở hữu mập mờ.⁸ Tôi gọi vấn đề này là sự tranh chấp trong chuyển giao (delivery contention).

Having a monolith doesn't mean you will definitely face the challenges of delivery contention any more than having a microservice architecture means that you won't ever face the problem. But a microservice architecture does give you more concrete boundaries around which ownership lines can be drawn in a system, giving you much more flexibility when it comes to reducing this problem.

Việc có một monolith không có nghĩa là bạn chắc chắn sẽ đối mặt với các thách thức của sự tranh chấp trong chuyển giao, cũng như việc có một kiến trúc microservice không có nghĩa là bạn sẽ không bao giờ gặp vấn đề đó. Nhưng một kiến trúc microservice quả thực trao cho bạn những ranh giới cụ thể hơn để vạch ra các đường ranh giới sở hữu trong một hệ thống, mang lại cho bạn nhiều linh hoạt hơn nhiều khi nói đến việc giảm thiểu vấn đề này.

Advantages of Monoliths — Ưu điểm của Monolith

Some monoliths, such as the single-process or modular monoliths, have a whole host of advantages too. Their much simpler deployment topology can avoid many of the pitfalls associated with distributed systems. This can result in much simpler developer workflows, and **monitoring**, troubleshooting, and activities like end-to-end **testing** can be greatly simplified as well.

Một số monolith, chẳng hạn như single-process hay modular monolith, cũng có cả một loạt ưu điểm. Cấu trúc triển khai đơn giản hơn nhiều của chúng có thể tránh được nhiều cạm bẫy liên quan đến các hệ thống phân tán. Điều này có thể dẫn đến các quy trình làm việc của lập trình viên đơn giản hơn nhiều, và việc giám sát, gỡ rối (troubleshooting), cùng các hoạt động như kiểm thử đầu-cuối cũng có thể được đơn giản hóa đi rất nhiều.

Monoliths can also simplify code reuse within the monolith itself. If we want to reuse code within a distributed system, we need to decide whether we want to copy code, break out libraries, or push the shared functionality into a service. With a monolith, our choices are much simpler, and many people like that simplicity—all the code is there; just use it!

Monolith cũng có thể đơn giản hóa việc tái sử dụng mã bên trong chính monolith. Nếu chúng ta muốn tái sử dụng mã bên trong một hệ thống phân tán, chúng ta cần quyết định liệu mình muốn sao chép mã, tách ra thành các thư viện, hay đẩy chức năng dùng chung vào một service. Với một monolith, các lựa chọn của chúng ta đơn giản hơn nhiều, và nhiều người thích sự đơn giản đó—tất cả mã đều ở đó; cứ dùng nó thôi!

Unfortunately, people have come to view the monolith as something to be avoided— as something inherently problematic. I've met multiple people for whom the term monolith is synonymous with legacy. This is a problem. A monolithic architecture is a choice, and a valid one at that. I'd go further and say that in my opinion it is the sensible default choice as an architectural style. In other words, I am looking for a reason to be convinced to use microservices, rather than looking for a reason not to use them.

Thật không may, mọi người đã đi đến chỗ xem monolith như một thứ cần tránh né—như một thứ vốn dĩ đầy vấn đề. Tôi đã gặp nhiều người mà với họ thuật ngữ monolith đồng nghĩa với legacy (di sản cũ). Đây là một vấn đề. Một kiến trúc monolith là một lựa chọn, và là một lựa chọn hợp lệ. Tôi xin đi xa hơn và nói rằng theo ý kiến của tôi nó là lựa chọn

mặc định hợp lý cho một phong cách kiến trúc. Nói cách khác, tôi đang tìm một lý do để bị thuyết phục sử dụng microservices, thay vì tìm một lý do để không sử dụng chúng.

If we fall into the trap of systematically undermining the monolith as a viable option for delivering our software, we're at risk of not doing right by ourselves or by the users of our software.

Nếu chúng ta rơi vào cái bẫy liên tục làm suy yếu monolith như một phương án khả thi để chuyển giao phần mềm của mình, chúng ta có nguy cơ không làm điều đúng đắn cho bản thân hoặc cho những người dùng phần mềm của mình.

Enabling Technology — Công nghệ hỗ trợ

As I touched on earlier, I don't think you need to adopt lots of new technology when you first start using microservices. In fact, that can be counterproductive. Instead, as you ramp up your microservice architecture, you should constantly be looking for

Như tôi đã đề cập trước đó, tôi không nghĩ bạn cần áp dụng nhiều công nghệ mới khi bạn mới bắt đầu sử dụng microservices. Thực tế, điều đó có thể phản tác dụng. Thay vào đó, khi bạn tăng tốc kiến trúc microservice của mình, bạn nên liên tục tìm kiếm

8 Microsoft Research has carried out studies in this space, and I recommend all of them, but as a starting point, I suggest “Don't Touch My Code! Examining the Effects of Ownership on Software Quality” by Christian Bird et al.

8 Microsoft Research đã thực hiện các nghiên cứu trong lĩnh vực này, và tôi đề xuất tất cả chúng, nhưng như một điểm khởi đầu, tôi gợi ý “Don't Touch My Code! Examining the Effects of Ownership on Software Quality” của Christian Bird và cộng sự.

issues caused by your increasingly distributed system, and then for technology that might help.

những vấn đề gây ra bởi hệ thống ngày càng phân tán của mình, rồi sau đó tìm công nghệ có thể giúp ích.

That said, technology has played a large part in the adoption of microservices as a concept. Understanding the tools that are available to help you get the most out of this architecture is going to be a key part of making any implementation of microservices a success. In fact, I would go as far to say that microservices require an understanding of the supporting technology to such a degree that previous distinctions between logical and physical architecture can be problematic—if you are involved in helping shape a microservice architecture, you'll need a breadth of understanding of these two worlds.

Dù vậy, công nghệ đã đóng một vai trò lớn trong việc áp dụng microservices như một khái niệm. Việc hiểu các công cụ sẵn có để giúp bạn tận dụng tối đa kiến trúc này sẽ là một phần then chốt trong việc làm cho bất kỳ triển khai microservice nào trở nên thành công. Thực tế, tôi xin đi xa đến mức nói rằng microservices đòi hỏi sự hiểu biết về công nghệ hỗ trợ đến mức mà những phân biệt trước kia giữa kiến trúc logic và kiến trúc vật lý có thể trở nên rắc rối—if bạn tham gia vào việc giúp định hình một kiến trúc microservice, bạn sẽ cần một sự hiểu biết rộng về cả hai thế giới này.

We'll be exploring a lot of this technology in detail in subsequent chapters, but before that, let's briefly introduce some of the enabling technology that might help you if you decide to make use of microservices.

Chúng ta sẽ khám phá rất nhiều công nghệ này một cách chi tiết trong các chương sau, nhưng trước đó, hãy giới thiệu ngắn gọn một số công nghệ hỗ trợ có thể giúp ích cho bạn

nếu bạn quyết định tận dụng microservices.

Log Aggregation and Distributed Tracing — Gom log và Distributed Tracing

With the increasing number of processes you are managing, it can be difficult to understand how your system is behaving in a production setting. This can in turn make troubleshooting much more difficult. We'll be exploring these ideas in more depth in Chapter 10 , but at a bare minimum, I strongly advocate the implementation of a **log aggregation** system as a prerequisite for adopting a microservice architecture.

Với số lượng tiến trình mà bạn đang quản lý ngày càng tăng, có thể khó hiểu được hệ thống của bạn đang hành xử ra sao trong môi trường production. Điều này đến lượt nó có thể làm cho việc gỡ rối khó khăn hơn nhiều. Chúng ta sẽ khám phá những ý tưởng này sâu hơn trong Chương 10, nhưng ở mức tối thiểu nhất, tôi mạnh mẽ ủng hộ việc triển khai một hệ thống gom log (log aggregation) như một điều kiện tiên quyết để áp dụng kiến trúc microservice.



Be cautious about taking on too much new technology when you start off with microservices. That said, a log aggregation tool is so essential that you should consider it a prerequisite for adopting microservices.

Hãy thận trọng với việc gánh quá nhiều công nghệ mới khi bạn mới bắt đầu với microservices. Dù vậy, một công cụ gom log thiết yếu đến mức bạn nên coi nó là một điều

kiện tiên quyết để áp dụng microservices.

These systems allow you to collect and **aggregate** logs from across all your services, providing you a central place from which logs can be analyzed, and even made part of an active **alerting** mechanism. Many options in this space cater to numerous situations. I'm a big fan of Humio for several reasons, but the simple logging services provided by the main public cloud vendors might be good enough to get you started.

Các hệ thống này cho phép bạn thu thập và gom log từ khắp tất cả các service của mình, mang lại cho bạn một nơi tập trung mà từ đó log có thể được phân tích, và thậm chí trở thành một phần của một cơ chế cảnh báo chủ động. Nhiều phương án trong lĩnh vực này đáp ứng vô số tình huống. Tôi là một người hâm mộ lớn của Humio vì một số lý do, nhưng các dịch vụ ghi log đơn giản do các nhà cung cấp đám mây công cộng (public cloud) chính cung cấp có thể đủ tốt để giúp bạn bắt đầu.

You can make these log aggregation tools even more useful by implementing correlation IDs, in which a single ID is used for a related set of service calls—for example, the chain of calls that might be triggered due to user interaction. By logging this ID as part of each log entry, isolating the logs associated with a given flow of calls becomes much easier, which in turn makes troubleshooting much easier.

Bạn có thể làm cho các công cụ gom log này còn hữu ích hơn nữa bằng cách triển khai các ID tương quan (correlation ID), trong đó một ID duy nhất được dùng cho một tập các lời gọi service liên quan—chẳng hạn, chuỗi các lời gọi có thể được kích hoạt do một tương tác của người dùng. Bằng cách ghi log ID này như một phần của mỗi mục log, việc cô lập các log liên quan đến một luồng lời gọi nhất định trở nên dễ dàng hơn nhiều, điều này đến lượt nó làm cho việc gỡ rối dễ dàng hơn nhiều.

As your system grows in complexity, it becomes essential to consider tools that allow you to better explore what your system is doing, providing the ability to analyze traces across multiple services, detect bottlenecks, and ask questions of your system that you didn't know you would want to ask in the first place. Open source tools can provide some of these features. One example is Jaeger, which focuses on the **distributed tracing** side of the equation.

Khi hệ thống của bạn tăng độ phức tạp, việc cân nhắc các công cụ cho phép bạn khám phá tốt hơn những gì hệ thống đang làm trở nên thiết yếu, mang lại khả năng phân tích các trace (vết) qua nhiều service, phát hiện các nút thắt cổ chai (bottleneck), và đặt ra cho hệ thống những câu hỏi mà ngay từ đầu bạn còn không biết là mình sẽ muốn hỏi. Các công cụ mã nguồn mở có thể cung cấp một số tính năng này. Một ví dụ là Jaeger, vốn tập trung vào khía cạnh distributed tracing của vấn đề.

But products like Lightstep and Honeycomb (shown in Figure 1-9) take these ideas further. They represent a new generation of tools that move beyond traditional monitoring approaches, making it much easier to explore the state of your running system. You might already have more conventional tools in place, but you really should look at the capabilities these products provide. They've been built from the ground up to solve the sorts of problems that operators of microservice architectures have to deal with.

Nhưng các sản phẩm như Lightstep và Honeycomb (được hiển thị trong Hình 1-9) đẩy những ý tưởng này đi xa hơn. Chúng đại diện cho một thế hệ công cụ mới vượt ra ngoài các cách tiếp cận giám sát truyền thống, làm cho việc khám phá trạng thái của hệ thống đang chạy của bạn dễ dàng hơn nhiều. Bạn có thể đã có sẵn các công cụ thông thường hơn, nhưng bạn thực sự nên xem xét những năng lực mà các sản phẩm này mang lại. Chúng được xây dựng từ đầu để giải quyết những loại vấn đề mà người vận hành các kiến trúc

microservice phải đối phó.

Query at 5/31 3:35PM > Trace e6ee35b206e1c9e5

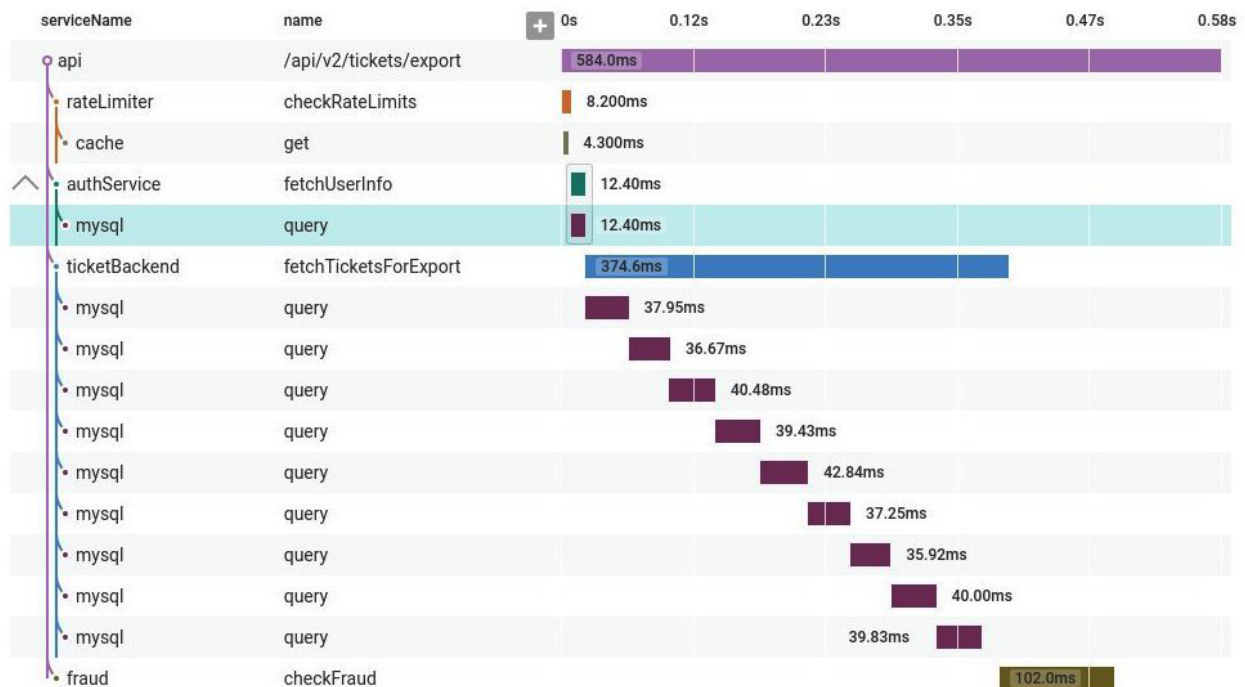


Figure 1-9. A distributed trace shown in Honeycomb, allowing you to identify where time is being spent for operations that can span multiple microservices

Hình 1-9. Một distributed trace được hiển thị trong Honeycomb, cho phép bạn xác định thời gian đang được tiêu tốn ở đâu cho các thao tác có thể trải dài qua nhiều microservice

Containers and Kubernetes — Container và Kubernetes

Ideally, you want to run each microservice instance in isolation. This ensures that issues in one microservice can't affect another microservice—for example, by gobbling up all the CPU. Virtualization is one way to create isolated execution environments on existing hardware, but normal virtualization techniques can be quite heavy when we consider the size of our microservices. Containers, on the other hand, provide a much more lightweight way to provision isolated execution for service instances, resulting in faster spin-up times for new **container** instances, along with being much more cost effective for many architectures.

Lý tưởng nhất, bạn muốn chạy mỗi thể hiện microservice một cách tách biệt. Điều này đảm bảo rằng các vấn đề ở một microservice không thể ảnh hưởng đến một microservice khác—chẳng hạn, bằng cách ngốn hết toàn bộ CPU. Ảo hóa (virtualization) là một cách để tạo ra các môi trường thực thi tách biệt trên phần cứng hiện có, nhưng các kỹ thuật ảo hóa thông thường có thể khá nặng nề khi chúng ta xét đến kích thước của các microservice của mình. Container, mặt khác, cung cấp một cách nhẹ nhàng hơn nhiều để cấp phát sự thực thi tách biệt cho các thể hiện service, dẫn đến thời gian khởi động nhanh hơn cho các thể hiện container mới, đồng thời tiết kiệm chi phí hơn nhiều đối với nhiều kiến trúc.

After you begin playing around with containers, you'll also realize that you need something to allow you to manage these containers across lots of underlying machines. Container orchestration

platforms like **Kubernetes** do exactly that, allowing you to distribute container instances in such a way as to provide the robustness and throughput your service needs, while allowing you to make efficient use of the underlying machines. In Chapter 8 we'll explore the concepts of operational isolation, containers, and Kubernetes.

Sau khi bạn bắt đầu nghịch ngợm với container, bạn cũng sẽ nhận ra rằng mình cần một thứ gì đó cho phép bạn quản lý các container này trên nhiều máy nền tảng (underlying machine). Các nền tảng điều phối container (container orchestration platform) như Kubernetes làm chính xác điều đó, cho phép bạn phân phối các thể hiện container theo cách sao cho cung cấp được độ vững chắc và thông lượng (throughput) mà service của bạn cần, đồng thời cho phép bạn sử dụng hiệu quả các máy nền tảng. Trong Chương 8 chúng ta sẽ khám phá các khái niệm về cô lập vận hành, container, và Kubernetes.

Don't feel the need to rush to adopt Kubernetes, or even containers for that matter. They absolutely offer significant advantages over more traditional deployment techniques, but their adoption is difficult to justify if you have only a few microservices. After the overhead of managing deployment begins to become a significant headache, start considering containerization of your service and the use of Kubernetes. But if you do end up doing that, do your best to ensure that someone else is running the Kubernetes cluster for you, perhaps by making use of a managed service on a public cloud provider. Running your own Kubernetes cluster can be a significant amount of work!

Đừng cảm thấy cần phải vội vàng áp dụng Kubernetes, hay thậm chí là container đi chăng nữa. Chúng chắc chắn mang lại những lợi thế đáng kể so với các kỹ thuật triển khai truyền thống hơn, nhưng việc áp dụng chúng khó mà biện minh được nếu bạn chỉ có một vài microservice. Sau khi gánh nặng của việc quản lý triển khai bắt đầu trở thành một cơn đau đầu đáng kể, hãy bắt đầu cân nhắc việc đóng container cho service của bạn và sử dụng Kubernetes. Nhưng nếu bạn rốt cuộc làm điều đó, hãy cố gắng hết sức để đảm bảo rằng có một người khác đang vận hành cụm Kubernetes (Kubernetes cluster) cho bạn, có lẽ bằng cách tận dụng một dịch vụ được quản lý trên một nhà cung cấp đám mây công cộng. Tự vận hành cụm Kubernetes của riêng bạn có thể là một khối lượng công việc đáng kể!

Streaming — Streaming

Although with microservices we are moving away from monolithic databases, we still need to find ways to share data between microservices. This is happening at the same time that organizations are wanting to move away from batch reporting operations and toward more real-time feedback, allowing them to react more quickly. Products that allow for the easy streaming and processing of what can often be large volumes of data have therefore become popular with people using microservice architectures.

Mặc dù với microservices chúng ta đang dịch chuyển khỏi các cơ sở dữ liệu monolith, chúng ta vẫn cần tìm cách chia sẻ dữ liệu giữa các microservice. Điều này đang diễn ra cùng lúc với việc các tổ chức muốn dịch chuyển khỏi các thao tác báo cáo theo lô (batch reporting) và hướng tới phản hồi mang tính thời gian thực hơn, cho phép họ phản ứng nhanh hơn. Các sản phẩm cho phép dễ dàng streaming và xử lý cái mà thường có thể là những khối lượng lớn dữ liệu vì thế đã trở nên phổ biến với những người sử dụng kiến trúc microservice.

For many people, Apache Kafka has become the de facto choice for streaming data in a microservice environment, and for good reason. Capabilities such as message permanence, compaction, and the ability to scale to handle large volumes of messages can be incredibly useful. Kafka has started adding stream-processing capabilities in the form of KSQLDB, but you can also use it with dedicated stream-

processing solutions like Apache Flink . Debezium is an open source tool developed to help stream data from existing datasources over Kafka, helping ensure that traditional datasources can become part of a stream-based architecture. In Chapter 4 we'll look at how streaming technology can play a part in microservice integration.

Đối với nhiều người, Apache Kafka đã trở thành lựa chọn mặc định trên thực tế để streaming dữ liệu trong một môi trường microservice, và vì lý do chính đáng. Các năng lực như tính bền vững của thông điệp (message permanence), nén (compaction), và khả năng mở rộng để xử lý những khối lượng lớn thông điệp có thể vô cùng hữu ích. Kafka đã bắt đầu bổ sung các năng lực xử lý luồng (stream-processing) dưới dạng KSQLDB, nhưng bạn cũng có thể dùng nó với các giải pháp xử lý luồng chuyên dụng như Apache Flink. Debezium là một công cụ mã nguồn mở được phát triển để giúp streaming dữ liệu từ các nguồn dữ liệu hiện có qua Kafka, giúp đảm bảo rằng các nguồn dữ liệu truyền thống có thể trở thành một phần của một kiến trúc dựa trên luồng. Trong Chương 4 chúng ta sẽ xem công nghệ streaming có thể đóng vai trò như thế nào trong việc tích hợp microservice.

Public Cloud and Serverless — Đám mây công cộng và Serverless

Public cloud providers, or more specifically the main three providers—Google Cloud, Microsoft Azure, and Amazon Web Services (AWS)—offer a huge array of managed services and deployment options for managing your application. As your microservice architecture grows, more and more work will be pushed into the operational space. Public cloud providers offer a host of managed services, from managed database instances or Kubernetes clusters to message brokers or distributed filesystems. By making use of these managed services, you are offloading a large amount of this work to a third party that is arguably better able to deal with these tasks.

Các nhà cung cấp đám mây công cộng, hay cụ thể hơn là ba nhà cung cấp chính—Google Cloud, Microsoft Azure, và Amazon Web Services (AWS)—cung cấp một mảng khổng lồ các dịch vụ được quản lý và các phương án triển khai để quản lý ứng dụng của bạn. Khi kiến trúc microservice của bạn lớn lên, ngày càng nhiều công việc sẽ bị đẩy vào không gian vận hành. Các nhà cung cấp đám mây công cộng cung cấp một loạt các dịch vụ được quản lý, từ các thể hiện cơ sở dữ liệu được quản lý hay các cụm Kubernetes cho đến các message broker hay các hệ thống tệp phân tán. Bằng cách tận dụng các dịch vụ được quản lý này, bạn đang chuyển giao một lượng lớn công việc này cho một bên thứ ba mà có thể nói là có khả năng xử lý các tác vụ này tốt hơn.

Of particular interest among the public cloud offerings are the products that sit under the banner of **serverless** . These products hide the underlying machines, allowing you to work at a higher level of abstraction. Examples of serverless products include message brokers, storage solutions, and databases. **Function as a Service** (FaaS) platforms are of special interest because they provide a nice abstraction around the deployment of code. Rather than worrying about how many servers you need to run your service, you just deploy your code and let the underlying platform handle spinning up instances of your code on demand. We'll look at serverless in more detail in Chapter 8 .

Đặc biệt đáng quan tâm trong số các dịch vụ đám mây công cộng là các sản phẩm nằm dưới ngọn cờ serverless. Các sản phẩm này ẩn đi các máy nền tảng, cho phép bạn làm việc ở một mức trừu tượng (abstraction) cao hơn. Các ví dụ về sản phẩm serverless bao gồm message broker, các giải pháp lưu trữ, và cơ sở dữ liệu. Các nền tảng Function as a Service (FaaS) đặc biệt đáng quan tâm vì chúng cung cấp một sự trừu tượng đẹp đẽ xung quanh việc triển khai mã. Thay vì lo lắng về việc bạn cần bao nhiêu máy chủ để chạy service của

mình, bạn chỉ cần triển khai mã của mình và để nền tảng nền tảng xử lý việc khởi động các thể hiện mã của bạn theo nhu cầu. Chúng ta sẽ xem xét serverless chi tiết hơn trong Chương 8.

Advantages of Microservices — Ưu điểm của Microservices

The advantages of microservices are many and varied. Many of these benefits can be laid at the door of any distributed system. Microservices, however, tend to achieve these benefits to a greater degree primarily because they take a more opinionated stance in the way service boundaries are defined. By combining the concepts of information hiding and domain-driven design with the power of distributed systems, microservices can help deliver significant gains over other forms of distributed architectures.

Ưu điểm của microservices thì nhiều và đa dạng. Nhiều trong số những lợi ích này có thể được quy cho bất kỳ hệ thống phân tán nào. Tuy nhiên, microservices có xu hướng đạt được những lợi ích này ở một mức độ cao hơn chủ yếu vì chúng có một lập trường mang tính quan điểm hơn trong cách định nghĩa ranh giới service. Bằng cách kết hợp các khái niệm che giấu thông tin và domain-driven design với sức mạnh của các hệ thống phân tán, microservices có thể giúp mang lại những bước tiến đáng kể so với các dạng kiến trúc phân tán khác.

Technology Heterogeneity — Tính đa dạng công nghệ

With a system composed of multiple, collaborating microservices, we can decide to use different technologies inside each one. This allows us to pick the right tool for each job rather than having to select a more standardized, one-size-fits-all approach that often ends up being the lowest common denominator.

Với một hệ thống bao gồm nhiều microservice cộng tác với nhau, chúng ta có thể quyết định sử dụng các công nghệ khác nhau bên trong mỗi cái. Điều này cho phép chúng ta chọn đúng công cụ cho từng công việc thay vì phải chọn một cách tiếp cận chuẩn hóa hơn, kiểu một-cỡ-cho-tất-cả mà thường rất cuộc trở thành mẫu số chung nhỏ nhất.

If one part of our system needs to improve its performance, we might decide to use a different technology stack that is better able to achieve the required performance levels. We might also decide that the way we store our data needs to change for different parts of our system. For example, for a social network, we might store our users' interactions in a graph-oriented database to reflect the highly interconnected nature of a social graph, but perhaps the posts the users make could be stored in a document-oriented data store, giving rise to a heterogeneous architecture like the one shown in Figure 1-10.

Nếu một phần của hệ thống cần cải thiện hiệu năng, chúng ta có thể quyết định sử dụng một ngăn xếp công nghệ (technology stack) khác có khả năng đạt được mức hiệu năng yêu cầu tốt hơn. Chúng ta cũng có thể quyết định rằng cách chúng ta lưu trữ dữ liệu cần thay đổi đối với các phần khác nhau của hệ thống. Chẳng hạn, đối với một mạng xã hội, chúng ta có thể lưu trữ các tương tác của người dùng trong một cơ sở dữ liệu hướng đồ thị (graph-oriented) để phản ánh bản chất kết nối cao của một đồ thị xã hội, nhưng có lẽ các bài đăng mà người dùng tạo ra lại có thể được lưu trữ trong một kho dữ liệu hướng tài liệu (document-oriented), tạo nên một kiến trúc không đồng nhất (heterogeneous) như cái được thể hiện trong Hình 1-10.

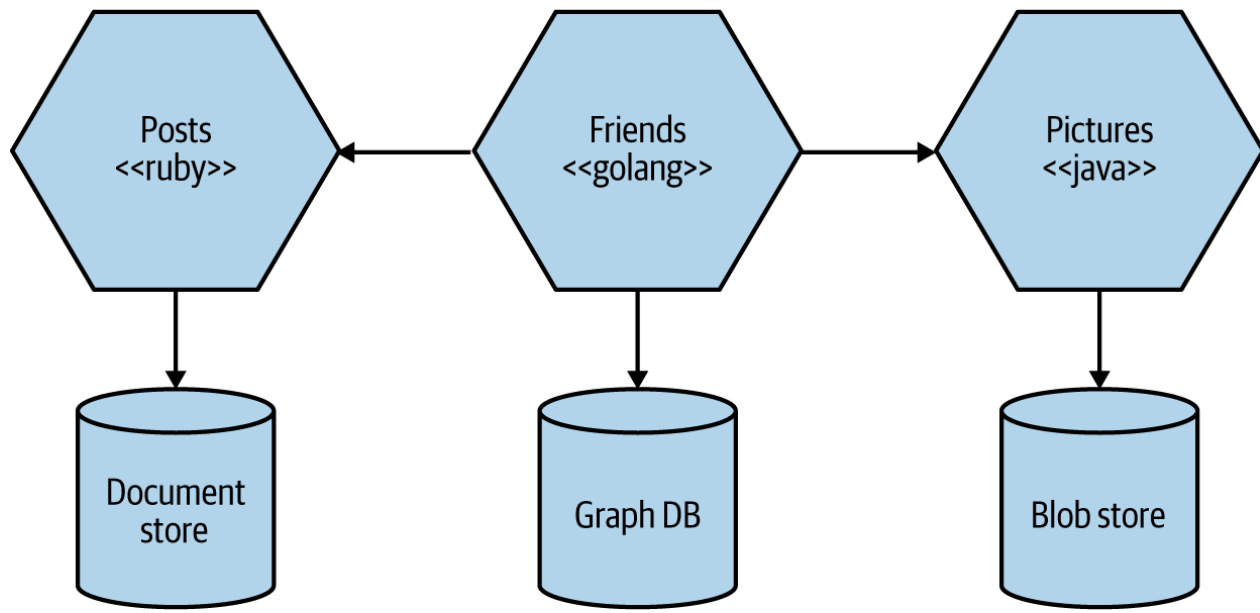


Figure 1-10. Microservices can allow you to more easily embrace different technologies

Hình 1-10. Microservices có thể cho phép bạn dễ dàng đón nhận các công nghệ khác nhau hơn

With microservices, we are also able to more quickly adopt technologies and to understand how new advancements might help us. One of the biggest barriers to trying out and adopting a new technology is the risks associated with it. With a monolithic application, if I want to try a new programming language, database, or framework, any change will affect much of my system. With a system consisting of multiple services, I have multiple new places to try out a new piece of technology. I can pick a microservice with perhaps the lowest risk and use the technology there, knowing that I can limit any potential negative impact. Many organizations find this ability to more quickly absorb new technologies to be a real advantage.

Với microservices, chúng ta cũng có khả năng áp dụng các công nghệ nhanh hơn và hiểu được những tiến bộ mới có thể giúp ích cho chúng ta như thế nào. Một trong những rào cản lớn nhất để thử nghiệm và áp dụng một công nghệ mới là những rủi ro liên quan đến nó. Với một ứng dụng monolith, nếu tôi muốn thử một ngôn ngữ lập trình, cơ sở dữ liệu hay framework mới, bất kỳ thay đổi nào cũng sẽ ảnh hưởng đến phần lớn hệ thống của tôi. Với một hệ thống bao gồm nhiều service, tôi có nhiều nơi mới để thử nghiệm một mẫu công nghệ mới. Tôi có thể chọn một microservice có lẽ ít rủi ro nhất và sử dụng công nghệ ở đó, biết rằng tôi có thể giới hạn bất kỳ tác động tiêu cực tiềm tàng nào. Nhiều tổ chức nhận thấy khả năng tiếp thu các công nghệ mới nhanh hơn này là một lợi thế thực sự.

Embracing multiple technologies doesn't come without overhead, of course. Some organizations choose to place some constraints on language choices. Netflix and Twitter, for example, mostly use the Java Virtual Machine (JVM) as a platform because those companies have a very good understanding of the reliability and performance of that system. They also develop libraries and tooling for the JVM that make operating at scale much easier, but the reliance on JVM-specific libraries makes things more difficult for non-Java-based services or clients. But neither Twitter nor Netflix uses only one technology stack for all jobs.

Tất nhiên, việc đón nhận nhiều công nghệ không phải là không kèm theo chi phí phụ. Một số tổ chức chọn cách đặt ra một số ràng buộc đối với các lựa chọn ngôn ngữ. Chẳng hạn, Netflix và Twitter chủ yếu sử dụng Java Virtual Machine (JVM) làm nền tảng vì các công

ty đó có một sự hiểu biết rất tốt về độ tin cậy và hiệu năng của hệ thống đó. Họ cũng phát triển các thư viện và công cụ cho JVM giúp việc vận hành ở quy mô lớn dễ dàng hơn nhiều, nhưng sự phụ thuộc vào các thư viện đặc thù của JVM lại khiến mọi thứ khó khăn hơn đối với các service hoặc client không dựa trên Java. Nhưng cả Twitter lẫn Netflix đều không chỉ dùng một ngăn xếp công nghệ duy nhất cho mọi công việc.

The fact that internal technology implementation is hidden from consumers can also make upgrading technologies easier. Your entire microservice architecture might be based on Spring Boot, for example, but you could change JVM version or framework versions for just one microservice, making it easier to manage the risk of upgrades.

Việc phần triển khai công nghệ bên trong được ẩn khỏi các consumer cũng có thể làm cho việc nâng cấp công nghệ dễ dàng hơn. Chẳng hạn, toàn bộ kiến trúc microservice của bạn có thể dựa trên Spring Boot, nhưng bạn có thể thay đổi phiên bản JVM hoặc phiên bản framework chỉ cho một microservice, làm cho việc quản lý rủi ro của các đợt nâng cấp dễ dàng hơn.

Robustness — Độ vững chắc

A key concept in improving the robustness of your application is the **bulkhead**. A component of a system may fail, but as long as that failure doesn't cascade, you can isolate the problem, and the rest of the system can carry on working. Service boundaries become your obvious bulkheads. In a monolithic service, if the service fails, everything stops working. With a monolithic system, we can run on multiple machines to reduce our chance of failure, but with microservices, we can build systems that handle the total failure of some of the constituent services and degrade functionality accordingly.

Một khái niệm then chốt trong việc cải thiện độ vững chắc của ứng dụng của bạn là bulkhead. Một thành phần của một hệ thống có thể hỏng, nhưng miễn là sự cố đó không lan tỏa, bạn có thể cô lập vấn đề, và phần còn lại của hệ thống có thể tiếp tục hoạt động. Các ranh giới service trở thành những bulkhead hiển nhiên của bạn. Trong một service monolith, nếu service hỏng, mọi thứ ngừng hoạt động. Với một hệ thống monolith, chúng ta có thể chạy trên nhiều máy để giảm khả năng xảy ra sự cố, nhưng với microservices, chúng ta có thể xây dựng những hệ thống xử lý được sự cố hoàn toàn của một số service cấu thành và suy giảm chức năng một cách tương ứng.

We do need to be careful, however. To ensure that our microservice systems can properly embrace this improved robustness, we need to understand the new sources of failure that distributed systems have to deal with. Networks can and will fail, as will machines. We need to know how to handle such failures and the impact (if any) those failures will have on the end users of our software. I have certainly worked with teams who have ended up with a less robust system after their migration to microservices due to their not taking these concerns seriously enough.

Tuy nhiên, chúng ta cần phải cẩn thận. Để đảm bảo rằng các hệ thống microservice của mình có thể đón nhận đúng cách độ vững chắc được cải thiện này, chúng ta cần hiểu các nguồn sự cố mới mà các hệ thống phân tán phải đối phó. Mạng có thể và sẽ hỏng, máy móc cũng vậy. Chúng ta cần biết cách xử lý những sự cố như vậy và tác động (nếu có) mà những sự cố đó sẽ gây ra cho người dùng cuối của phần mềm của chúng ta. Tôi chắc chắn đã từng làm việc với các nhóm rất cuộc có một hệ thống kém vững chắc hơn sau khi di trú sang microservices do họ không xem xét những mối quan tâm này một cách nghiêm túc đủ mức.

Scaling — Mở rộng

With a large, monolithic service, we need to scale everything together. Perhaps one small part of our overall system is constrained in performance, but if that behavior is locked up in a giant monolithic application, we need to handle scaling everything as a piece. With smaller services, we can scale just those services that need scaling, allowing us to run other parts of the system on smaller, less powerful hardware, as illustrated in Figure 1-11.

Với một service monolith lớn, chúng ta cần mở rộng mọi thứ cùng nhau. Có lẽ một phần nhỏ trong toàn bộ hệ thống của chúng ta bị hạn chế về hiệu năng, nhưng nếu hành vi đó bị khóa chặt bên trong một ứng dụng monolith khổng lồ, chúng ta cần xử lý việc mở rộng mọi thứ như một khối. Với các service nhỏ hơn, chúng ta có thể chỉ mở rộng những service cần được mở rộng, cho phép chúng ta chạy các phần khác của hệ thống trên phần cứng nhỏ hơn, kém mạnh mẽ hơn, như được minh họa trong Hình 1-11.

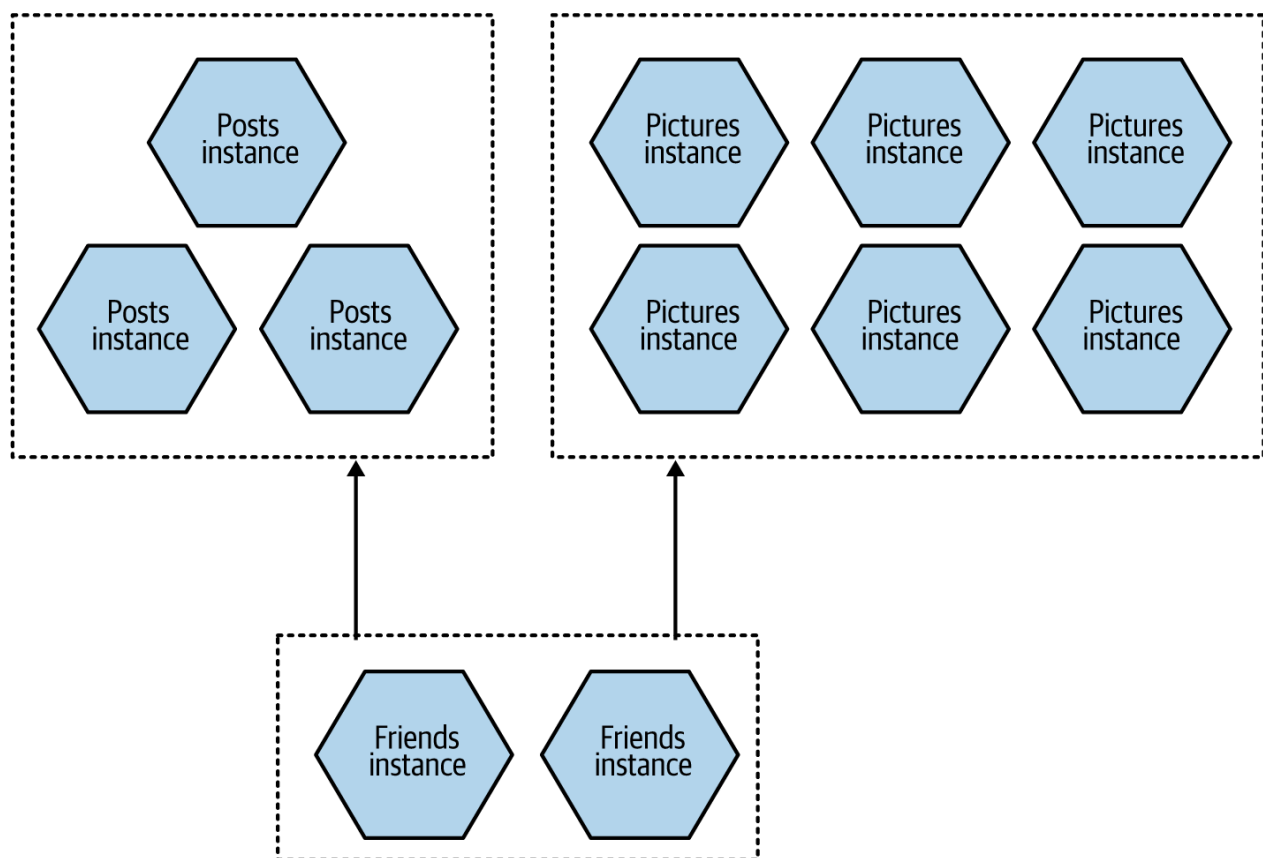


Figure 1-11. You can target scaling at just the microservices that need it. Gilt, an online fashion retailer, adopted microservices for this exact reason. Having started in 2007 with a monolithic Rails application, by 2009 Gilt's system was unable to cope with the load being placed on it. By splitting out core parts of its system, Gilt was better able to deal with its traffic spikes, and today it has more than 450 microservices, each one running on multiple separate machines.

Hình 1-11. Bạn có thể nhắm việc mở rộng vào chỉ những microservice cần đến nó. Gilt, một nhà bán lẻ thời trang trực tuyến, đã áp dụng microservices vì chính lý do này. Khởi đầu vào năm 2007 với một ứng dụng Rails monolith, đến năm 2009 hệ thống của Gilt không thể đối phó với tải đặt lên nó. Bằng cách tách ra các phần cốt lõi của hệ thống, Gilt đã có thể đối phó với các đợt tăng vọt lưu lượng của mình tốt hơn, và ngày nay nó có hơn 450

microservice, mỗi cái chạy trên nhiều máy riêng biệt.

When embracing on-demand provisioning systems like those provided by AWS, we can even apply this scaling on demand for those pieces that need it. This allows us to control our costs more effectively. It's not often that an architectural approach can be so closely correlated to an almost immediate cost savings.

Khi đón nhận các hệ thống cấp phát theo nhu cầu (on-demand provisioning) như những hệ thống do AWS cung cấp, chúng ta thậm chí có thể áp dụng việc mở rộng theo nhu cầu này cho những phần cần đến nó. Điều này cho phép chúng ta kiểm soát chi phí của mình hiệu quả hơn. Không phải thường xuyên mà một cách tiếp cận kiến trúc lại có thể tương quan chặt chẽ đến vậy với một sự tiết kiệm chi phí gần như tức thì.

Ultimately, we can scale our applications in a multitude of ways, and microservices can be an effective part of this. We'll look at the scaling of microservices in more detail in Chapter 13 .

Rất cuộc, chúng ta có thể mở rộng các ứng dụng của mình theo vô số cách, và microservices có thể là một phần hiệu quả của việc này. Chúng ta sẽ xem xét việc mở rộng microservice chi tiết hơn trong Chương 13.

Ease of Deployment — Dễ dàng triển khai

A one-line change to a million-line monolithic application requires the entire application to be deployed in order to release the change. That could be a large-impact, high-risk deployment. In practice, deployments such as these end up happening infrequently because of understandable fear. Unfortunately, this means that our changes continue to build up between releases, until the new version of our application entering production has masses of changes. And the bigger the delta between releases, the higher the risk that we'll get something wrong!

Một thay đổi một dòng đối với một ứng dụng monolith một triệu dòng đòi hỏi toàn bộ ứng dụng phải được triển khai để phát hành thay đổi đó. Đó có thể là một đợt triển khai tác động lớn, rủi ro cao. Trong thực tế, những đợt triển khai như thế này rất cuộc xảy ra không thường xuyên vì một nỗi sợ dễ hiểu. Thật không may, điều này có nghĩa là các thay đổi của chúng ta cứ tiếp tục tích tụ giữa các đợt phát hành, cho đến khi phiên bản mới của ứng dụng đi vào production có một khối lượng khổng lồ các thay đổi. Và độ chênh lệch giữa các đợt phát hành càng lớn, thì rủi ro chúng ta làm sai điều gì đó càng cao!

With microservices, we can make a change to a single service and deploy it independently of the rest of the system. This allows us to get our code deployed more quickly. If a problem does occur, it can be quickly isolated to an individual service, making fast rollback easy to achieve. It also means that we can get our new functionality out to customers more quickly. This is one of the main reasons organizations like Amazon and Netflix use these architectures—to ensure that they remove as many impediments as possible to getting software out the door.

Với microservices, chúng ta có thể thực hiện một thay đổi đối với một service duy nhất và triển khai nó một cách độc lập với phần còn lại của hệ thống. Điều này cho phép chúng ta đưa mã của mình được triển khai nhanh hơn. Nếu một vấn đề có xảy ra, nó có thể được nhanh chóng cô lập về một service riêng lẻ, làm cho việc quay lui (rollback) nhanh để đạt được. Nó cũng có nghĩa là chúng ta có thể đưa chức năng mới của mình đến khách hàng nhanh hơn. Đây là một trong những lý do chính khiến các tổ chức như Amazon và Netflix sử dụng những kiến trúc này—để đảm bảo rằng họ loại bỏ càng nhiều trở ngại càng tốt cho việc đưa phần mềm ra cửa.

Organizational Alignment — Sự đồng bộ với tổ chức

Many of us have experienced the problems associated with large teams and large codebases. These problems can be exacerbated when the team is distributed. We also know that smaller teams working on smaller codebases tend to be more productive.

Nhiều người trong chúng ta đã trải nghiệm các vấn đề liên quan đến những nhóm lớn và những codebase lớn. Những vấn đề này có thể trầm trọng hơn khi nhóm bị phân tán. Chúng ta cũng biết rằng những nhóm nhỏ hơn làm việc trên những codebase nhỏ hơn có xu hướng năng suất hơn.

Microservices allow us to better align our architecture to our organization, helping us minimize the number of people working on any one codebase to hit the sweet spot of team size and productivity. Microservices also allow us to change ownership of services as the organization changes—enabling us to maintain the alignment between architecture and organization in the future.

Microservices cho phép chúng ta đồng bộ kiến trúc của mình với tổ chức tốt hơn, giúp chúng ta giảm thiểu số người làm việc trên bất kỳ một codebase nào để đạt được điểm ngọt giữa kích thước nhóm và năng suất. Microservices cũng cho phép chúng ta thay đổi quyền sở hữu các service khi tổ chức thay đổi—giúp chúng ta duy trì sự đồng bộ giữa kiến trúc và tổ chức trong tương lai.

Composability — Tính khả ghép

One of the key promises of distributed systems and service-oriented architectures is that we open up opportunities for reuse of functionality. With microservices, we allow for our functionality to be consumed in different ways for different purposes. This can be especially important when we think about how our consumers use our software.

Một trong những lời hứa then chốt của các hệ thống phân tán và kiến trúc hướng dịch vụ là chúng ta mở ra các cơ hội để tái sử dụng chức năng. Với microservices, chúng ta cho phép chức năng của mình được tiêu thụ theo những cách khác nhau cho những mục đích khác nhau. Điều này có thể đặc biệt quan trọng khi chúng ta nghĩ về cách các consumer của mình sử dụng phần mềm của chúng ta.

Gone is the time when we could think narrowly about either our desktop website or our mobile application. Now we need to think of the myriad ways that we might want to weave together capabilities for the web, native application, mobile web, tablet app, or wearable device. As organizations move away from thinking in terms of narrow channels to embracing more holistic concepts of customer engagement, we need architectures that can keep up.

Đã qua rồi cái thời mà chúng ta có thể nghĩ một cách hạn hẹp về hoặc trang web desktop hoặc ứng dụng di động của mình. Giờ đây chúng ta cần nghĩ về vô số cách mà chúng ta có thể muốn đan dệt các năng lực lại với nhau cho web, ứng dụng native, web di động, ứng dụng máy tính bảng, hay thiết bị đeo được (wearable). Khi các tổ chức dịch chuyển khỏi việc suy nghĩ theo các kênh hạn hẹp để đón nhận các khái niệm toàn diện hơn về sự gắn kết khách hàng, chúng ta cần những kiến trúc có thể theo kịp.

With microservices, think of us opening up seams in our system that are addressable by outside parties. As circumstances change, we can build applications in different ways. With a monolithic application, I often have one coarse-grained seam that can be used from the outside. If I want to break that up to get something more useful, I'll need a hammer!

Với microservices, hãy nghĩ về việc chúng ta mở ra những đường nối (seam) trong hệ thống của mình mà các bên bên ngoài có thể định địa chỉ tới. Khi hoàn cảnh thay đổi, chúng ta có thể xây dựng các ứng dụng theo những cách khác nhau. Với một ứng dụng monolith, tôi thường chỉ có một đường nối thô (coarse-grained) duy nhất có thể được dùng từ bên ngoài. Nếu tôi muốn chia tách nó ra để có được thứ gì đó hữu ích hơn, tôi sẽ cần một cái búa!

Microservice Pain Points — Những điểm đau của Microservice

Microservice architectures bring a host of benefits, as we’ve already seen. But they also bring a host of complexity. If you are considering adopting a microservice architecture, it’s important that you be able to compare the good with the bad. In reality, most microservice points can be laid at the door of distributed systems and thus would just as likely be evident in a distributed monolith as in a microservice architecture.

Kiến trúc microservice mang lại cả một loạt lợi ích, như chúng ta đã thấy. Nhưng chúng cũng mang lại cả một loạt sự phức tạp. Nếu bạn đang cân nhắc áp dụng một kiến trúc microservice, điều quan trọng là bạn có thể so sánh cái tốt với cái xấu. Trên thực tế, hầu hết các điểm đau của microservice có thể được quy cho các hệ thống phân tán và do đó cũng sẽ dễ dàng hiện diện trong một distributed monolith như trong một kiến trúc microservice.

We’ll be covering many of these issues in depth throughout the rest of the book—in fact, I’d argue that the bulk of this book is about dealing with the pain, suffering, and horror of owning a microservice architecture.

Chúng ta sẽ đề cập đến nhiều trong số những vấn đề này một cách chuyên sâu xuyên suốt phần còn lại của cuốn sách—thực tế, tôi xin tranh luận rằng phần lớn cuốn sách này nói về việc đối phó với nỗi đau, sự khổ sở, và nỗi kinh hoàng của việc sở hữu một kiến trúc microservice.

Developer Experience — Trải nghiệm của lập trình viên

As you have more and more services, the developer experience can begin to suffer. More resource-intensive runtimes like the JVM can limit the number of microservices that can be run on a single developer machine. I could probably run four or five JVM-based microservices as separate processes on my laptop, but could I run 10 or 20? Most likely not. Even with less taxing runtimes, there is a limit to the number of things you can run locally, which inevitably will start conversations about what to do when you can’t run the entire system on one machine. This can become even more complicated if you are using cloud services that you cannot run locally.

Khi bạn có ngày càng nhiều service, trải nghiệm của lập trình viên có thể bắt đầu sa sút. Các runtime tốn tài nguyên hơn như JVM có thể giới hạn số lượng microservice có thể chạy trên một máy của lập trình viên. Tôi có lẽ có thể chạy bốn hoặc năm microservice dựa trên JVM như các tiến trình riêng biệt trên laptop của mình, nhưng liệu tôi có thể chạy 10 hay 20 cái không? Nhiều khả năng là không. Ngay cả với các runtime ít nặng nề hơn, vẫn có một giới hạn cho số lượng thứ bạn có thể chạy cục bộ, điều này chắc chắn sẽ bắt đầu những cuộc trao đổi về việc phải làm gì khi bạn không thể chạy toàn bộ hệ thống trên một máy. Điều này thậm chí có thể trở nên phức tạp hơn nếu bạn đang sử dụng các dịch vụ đám mây mà bạn không thể chạy cục bộ.

Extreme solutions can involve “developing in the cloud,” where developers move away from being able to develop locally anymore. I’m not a fan of this, because feedback cycles can suffer greatly. Instead, I think limiting the scope of which parts of a system a developer needs to work on is likely to be a much more straightforward approach. However, this might be problematic if you want to embrace more of a “collective ownership” model in which any developer is expected to work on any part of the system.

Các giải pháp cực đoan có thể bao gồm “phát triển trên đám mây”, nơi các lập trình viên dịch chuyển khỏi việc còn có thể phát triển cục bộ nữa. Tôi không phải là người ủng hộ điều này, vì các chu kỳ phản hồi có thể sa sút rất nhiều. Thay vào đó, tôi nghĩ việc giới hạn phạm vi những phần nào của một hệ thống mà một lập trình viên cần làm việc có khả năng là một cách tiếp cận đơn giản hơn nhiều. Tuy nhiên, điều này có thể gây rắc rối nếu bạn muốn đón nhận nhiều hơn một mô hình “sở hữu tập thể” (collective ownership) trong đó bất kỳ lập trình viên nào cũng được kỳ vọng có thể làm việc trên bất kỳ phần nào của hệ thống.

Technology Overload — Quá tải công nghệ

The sheer weight of new technology that has sprung up to enable the adoption of microservice architectures can be overwhelming. I’ll be honest and say that a lot of this technology has just been rebranded as “microservice friendly,” but some advances have legitimately helped in dealing with the complexity of these sorts of architectures. There is a danger, though, that this wealth of new toys can lead to a form of technology fetishism. I’ve seen so many companies adopting microservice architecture who decided that it was also the best time to introduce vast arrays of new and often alien technology.

Sức nặng tuyệt đối của các công nghệ mới đã nảy sinh để cho phép việc áp dụng kiến trúc microservice có thể gây choáng ngợp. Tôi sẽ thành thật mà nói rằng rất nhiều công nghệ trong số này chỉ đơn thuần được gắn nhãn lại là “thân thiện với microservice”, nhưng một số tiến bộ thực sự đã giúp ích trong việc đối phó với sự phức tạp của những loại kiến trúc này. Tuy nhiên, có một mối nguy là sự dồi dào của những món đồ chơi mới này có thể dẫn đến một dạng tôn sùng công nghệ. Tôi đã thấy rất nhiều công ty áp dụng kiến trúc microservice quyết định rằng đó cũng là thời điểm tốt nhất để giới thiệu vô vàn các công nghệ mới và thường là xa lạ.

Microservices may well give you the option for each microservice to be written in a different programming language, to run on a different runtime, or to use a different database—but these are options, not requirements. You have to carefully balance the breadth and complexity of the technology you use against the costs that a diverse array of technology can bring.

Microservices quả thực có thể trao cho bạn lựa chọn để mỗi microservice được viết bằng một ngôn ngữ lập trình khác nhau, chạy trên một runtime khác nhau, hoặc sử dụng một cơ sở dữ liệu khác nhau—nhưng đây là các lựa chọn, không phải các yêu cầu. Bạn phải cân bằng cẩn thận bề rộng và độ phức tạp của công nghệ bạn sử dụng với các chi phí mà một mảng công nghệ đa dạng có thể mang lại.

When you start adopting microservices, some fundamental challenges are inescapable: you’ll need to spend a lot of time understanding issues around **data consistency**, latency, service modeling, and the like. If you’re trying to understand how these ideas change the way you think about software development at the same time that you’re embracing a huge amount of new technology, you’ll have a hard time of it. It’s also worth pointing out that the bandwidth taken up by trying to understand all of this new technology will reduce the time you have for actually shipping features to your users.

Khi bạn bắt đầu áp dụng microservices, một số thách thức cơ bản là không thể tránh khỏi: bạn sẽ cần dành nhiều thời gian để hiểu các vấn đề xoay quanh tính nhất quán dữ liệu, độ trễ (latency), việc mô hình hóa service, và những thứ tương tự. Nếu bạn đang cố hiểu xem những ý tưởng này thay đổi cách bạn nghĩ về phát triển phần mềm ra sao trong khi cùng lúc đang đón nhận một lượng khổng lồ công nghệ mới, bạn sẽ gặp khó khăn với nó. Cũng đáng chỉ ra rằng bằng thông bị chiếm dụng bởi việc cố gắng hiểu tất cả công nghệ mới này sẽ làm giảm thời gian bạn có để thực sự đưa các tính năng đến người dùng của mình.

As you (gradually) increase the complexity of your microservice architecture, look to introduce new technology as you need it. You don't need a Kubernetes cluster when you have three services! In addition to ensuring that you're not overloaded with the complexity of these new tools, this gradual increase has the added benefit of allowing you to gain new and better ways of doing things that will no doubt emerge over time.

Khi bạn (dần dần) tăng độ phức tạp của kiến trúc microservice của mình, hãy hướng tới việc giới thiệu công nghệ mới khi bạn cần đến nó. Bạn không cần một cụm Kubernetes khi bạn có ba service! Ngoài việc đảm bảo rằng bạn không bị quá tải với độ phức tạp của những công cụ mới này, sự gia tăng dần dần này còn có thêm lợi ích là cho phép bạn có được những cách làm mới và tốt hơn mà chắc chắn sẽ xuất hiện theo thời gian.

Cost — Chi phí

It's highly likely that in the short term at least you'll see an increase in costs from a number of factors. Firstly, you'll likely need to run more things—more processes, more computers, more network, more storage, and more supporting software (which will incur additional license fees).

Rất có khả năng rằng ít nhất trong ngắn hạn bạn sẽ thấy chi phí tăng lên từ một số yếu tố. Thứ nhất, bạn có thể sẽ cần chạy nhiều thứ hơn—nhiều tiến trình hơn, nhiều máy tính hơn, nhiều mạng hơn, nhiều lưu trữ hơn, và nhiều phần mềm hỗ trợ hơn (vốn sẽ phát sinh thêm phí bản quyền).

Secondly, any change you introduce into a team or an organization will slow you down in the short term. It takes time to learn new ideas, and to work out how to use them effectively. While this is going on, other activities will be impacted. This will result in either a direct slowdown in delivery of new functionality or the need to add more people to offset this cost.

Thứ hai, bất kỳ thay đổi nào bạn đưa vào một nhóm hay một tổ chức cũng sẽ làm bạn chậm lại trong ngắn hạn. Phải mất thời gian để học các ý tưởng mới, và để tìm ra cách sử dụng chúng một cách hiệu quả. Trong khi điều này đang diễn ra, các hoạt động khác sẽ bị ảnh hưởng. Điều này sẽ dẫn đến hoặc một sự chậm lại trực tiếp trong việc chuyển giao chức năng mới hoặc nhu cầu phải thêm người để bù đắp cho chi phí này.

In my experience, microservices are a poor choice for an organization primarily concerned with reducing costs, as a cost-cutting mentality—where IT is seen as a cost center rather than a profit center—will constantly be a drag on getting the most out of this architecture. On the other hand, microservices can help you make more money if you can use these architectures to reach more customers or develop more functionality in parallel. So are microservices a way to drive more profits? Perhaps. Are microservices a way to reduce costs? Not so much.

Theo kinh nghiệm của tôi, microservices là một lựa chọn tồi đối với một tổ chức chủ yếu quan tâm đến việc giảm chi phí, vì một tư duy cắt giảm chi phí—nơi IT được xem như một trung tâm chi phí (cost center) thay vì một trung tâm lợi nhuận (profit center)—sẽ liên

tục là một lực cản đối với việc tận dụng tối đa kiến trúc này. Mặt khác, microservices có thể giúp bạn kiếm được nhiều tiền hơn nếu bạn có thể sử dụng những kiến trúc này để tiếp cận nhiều khách hàng hơn hoặc phát triển nhiều chức năng hơn một cách song song. Vậy microservices có phải là một cách để thúc đẩy nhiều lợi nhuận hơn không? Có lẽ vậy. Microservices có phải là một cách để giảm chi phí không? Không hẳn.

Reporting — Báo cáo

With a monolithic system, you typically have a monolithic database. This means that stakeholders who want to analyze all the data together, often involving large join operations across data, have a ready-made schema against which to run their reports. They can just run them directly against the monolithic database, perhaps against a read replica, as shown in Figure 1-12 .

Với một hệ thống monolith, bạn thường có một cơ sở dữ liệu monolith. Điều này có nghĩa là các bên liên quan muốn phân tích toàn bộ dữ liệu cùng nhau, thường liên quan đến các thao tác join lớn xuyên dữ liệu, có sẵn một schema làm sẵn để chạy các báo cáo của họ. Họ có thể chỉ cần chạy chúng trực tiếp trên cơ sở dữ liệu monolith, có lẽ trên một bản sao đọc (read replica), như được thể hiện trong Hình 1-12.

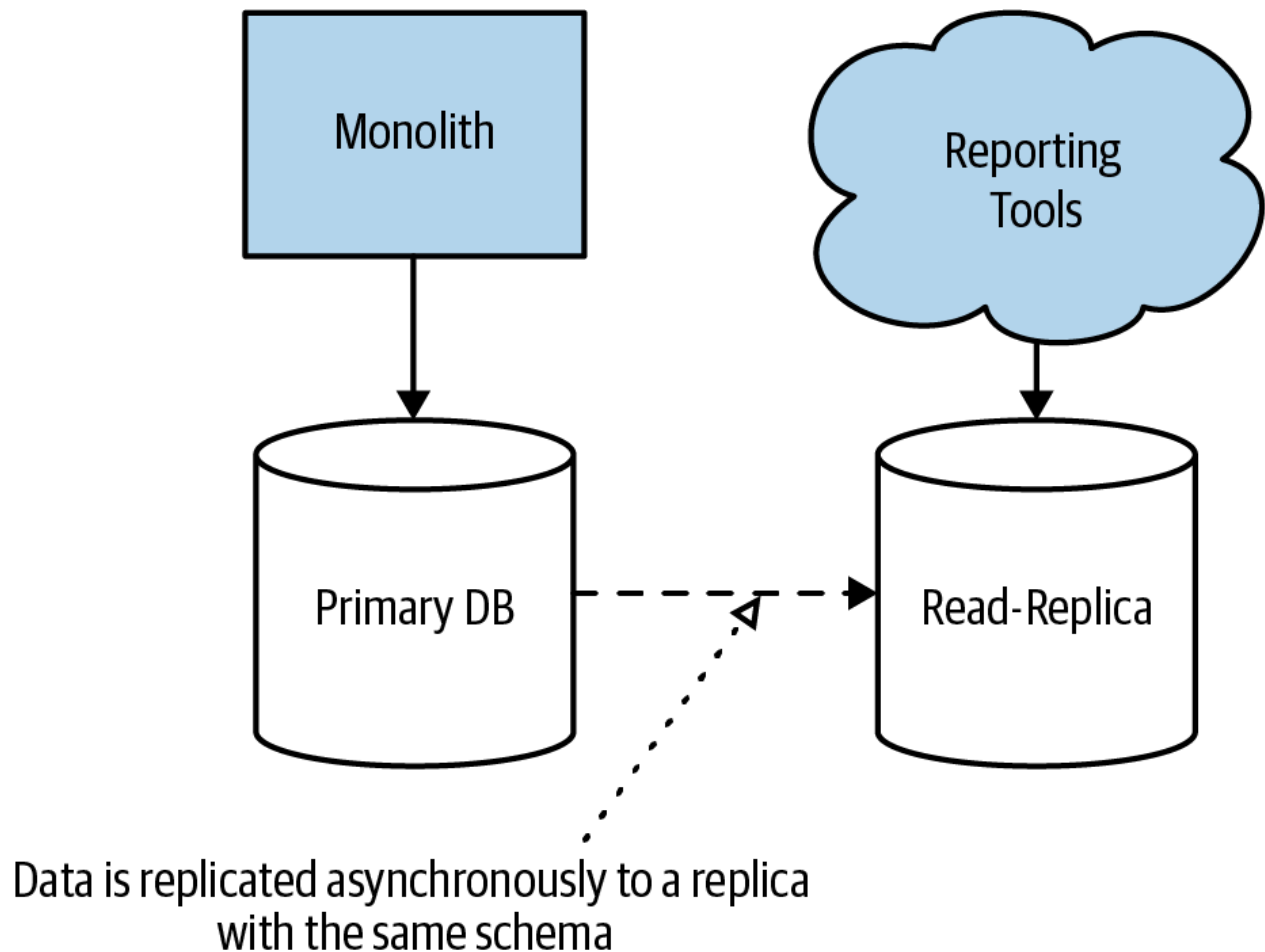


Figure 1-12. Reporting carried out directly on the database of a monolith With a microservice architecture, we have broken up this monolithic schema. That doesn't mean that the need for reporting across all our data has gone away; we've just made it much more difficult, because now

our data is scattered across multiple logically isolated schemas.

Hình 1-12. Báo cáo được thực hiện trực tiếp trên cơ sở dữ liệu của một monolith. Với một kiến trúc microservice, chúng ta đã chia nhỏ schema monolith này. Điều đó không có nghĩa là nhu cầu báo cáo xuyên toàn bộ dữ liệu của chúng ta đã biến mất; chúng ta chỉ làm cho nó khó khăn hơn nhiều, vì giờ đây dữ liệu của chúng ta bị rải rác qua nhiều schema được cô lập về mặt logic.

More modern approaches to reporting, such as using streaming to allow for real-time reporting on large volumes of data, can work well with a microservice architecture but typically require the adoption of new ideas and associated technology. Alternatively, you might simply need to publish data from your microservices into central reporting databases (or perhaps less structured data lakes) to allow for reporting use cases.

Các cách tiếp cận báo cáo hiện đại hơn, chẳng hạn như sử dụng streaming để cho phép báo cáo thời gian thực trên những khối lượng lớn dữ liệu, có thể hoạt động tốt với một kiến trúc microservice nhưng thường đòi hỏi việc áp dụng các ý tưởng mới và công nghệ liên quan. Một cách khác, bạn có thể chỉ đơn giản cần xuất bản dữ liệu từ các microservice của mình vào các cơ sở dữ liệu báo cáo tập trung (hoặc có lẽ các hồ dữ liệu (data lake) ít cấu trúc hơn) để cho phép các trường hợp sử dụng báo cáo.

Monitoring and Troubleshooting — Giám sát và gỡ rối

With a standard monolithic application, we can have a fairly simplistic approach to monitoring. We have a small number of machines to worry about, and the failure mode of the application is somewhat binary—the application is often either all up or all down. With a microservice architecture, do we understand the impact if just a single instance of a service goes down?

Với một ứng dụng monolith tiêu chuẩn, chúng ta có thể có một cách tiếp cận khá đơn giản đối với việc giám sát. Chúng ta có một số lượng nhỏ máy móc cần lo lắng, và chế độ sự cố của ứng dụng có phân nhị phân—ứng dụng thường hoặc tất cả đều chạy hoặc tất cả đều ngừng. Với một kiến trúc microservice, liệu chúng ta có hiểu được tác động nếu chỉ một thể hiện duy nhất của một service ngừng hoạt động?

With a monolithic system, if our CPU is stuck at 100% for a long time, we know it's a big problem. With a microservice architecture with tens or hundreds of processes, can we say the same thing? Do we need to wake someone up at 3 a.m. when just one process is stuck at 100% CPU?

Với một hệ thống monolith, nếu CPU của chúng ta bị kẹt ở 100% trong một thời gian dài, chúng ta biết đó là một vấn đề lớn. Với một kiến trúc microservice có hàng chục hoặc hàng trăm tiến trình, liệu chúng ta có thể nói điều tương tự không? Liệu chúng ta có cần đánh thức ai đó dậy lúc 3 giờ sáng khi chỉ một tiến trình bị kẹt ở 100% CPU?

Luckily, there are a whole host of ideas in this space that can help. If you'd like to explore this concept in more detail, I recommend Distributed Systems **Observability** by Cindy Sridharan (O'Reilly) as an excellent starting point, although we'll also be taking our own look at monitoring and observability in Chapter 10.

May mắn thay, có cả một loạt ý tưởng trong lĩnh vực này có thể giúp ích. Nếu bạn muốn khám phá khái niệm này chi tiết hơn, tôi đề xuất Distributed Systems Observability của Cindy Sridharan (O'Reilly) như một điểm khởi đầu xuất sắc, mặc dù chúng ta cũng sẽ tự mình xem xét việc giám sát và observability trong Chương 10.

Security — Bảo mật

With a single-process monolithic system, much of our information flowed within that process. Now, more information flows over networks between our services. This can make our data more vulnerable to being observed in transit and also to potentially being manipulated as part of man-in-the-middle attacks. This means that you might need to direct more care to protecting data in transit and to ensuring that your microservice endpoints are protected so that only authorized parties are able to make use of them. Chapter 11 is dedicated entirely to looking at the challenges in this space.

Với một hệ thống monolith single-process, phần lớn thông tin của chúng ta lưu chuyển bên trong tiến trình đó. Giờ đây, nhiều thông tin lưu chuyển qua mạng giữa các service của chúng ta. Điều này có thể làm cho dữ liệu của chúng ta dễ bị quan sát trong khi truyền và cũng có khả năng bị thao túng như một phần của các cuộc tấn công người-đứng-giữa (man-in-the-middle). Điều này có nghĩa là bạn có thể cần hướng nhiều sự chăm chú hơn vào việc bảo vệ dữ liệu trong khi truyền và vào việc đảm bảo rằng các endpoint microservice của bạn được bảo vệ sao cho chỉ các bên được phép mới có thể tận dụng chúng. Chương 11 dành trọn vẹn cho việc xem xét các thách thức trong lĩnh vực này.

Testing — Kiểm thử

With any type of automated functional test, you have a delicate balancing act. The more functionality a test executes—i.e., the broader the scope of the test—the more confidence you have in your application. On the other hand, the larger the scope of the test, the harder it is to set up test data and supporting fixtures, the longer the test can take to run, and the harder it can be to work out what is broken when it fails. In Chapter 9 I'll share a number of techniques for making testing work in this more challenging environment.

Với bất kỳ loại kiểm thử chức năng tự động nào, bạn đều có một sự cân bằng tinh tế. Một bài kiểm thử thực thi càng nhiều chức năng—tức là phạm vi của bài kiểm thử càng rộng—bạn càng có nhiều niềm tin vào ứng dụng của mình. Mặt khác, phạm vi của bài kiểm thử càng lớn, thì càng khó thiết lập dữ liệu kiểm thử và các fixture hỗ trợ, bài kiểm thử có thể mất càng nhiều thời gian để chạy, và càng khó tìm ra cái gì bị hỏng khi nó thất bại. Trong Chương 9 tôi sẽ chia sẻ một số kỹ thuật để làm cho việc kiểm thử hoạt động trong môi trường đầy thách thức hơn này.

End-to-end tests for any type of system are at the extreme end of the scale in terms of the functionality they cover, and we are used to them being more problematic to write and maintain than smaller-scoped unit tests. Often this is worth it, though, because we want the confidence that comes from having an **end-to-end test** use our systems in the same way a user might.

Các bài kiểm thử đầu-cuối cho bất kỳ loại hệ thống nào đều nằm ở đầu cực trị của thang đo về mặt chức năng mà chúng bao phủ, và chúng ta đã quen với việc chúng rắc rối hơn để viết và bảo trì so với các unit test có phạm vi nhỏ hơn. Tuy nhiên, điều này thường đáng giá, vì chúng ta muốn niềm tin có được từ việc có một bài kiểm thử đầu-cuối sử dụng các hệ thống của chúng ta theo cùng cách mà một người dùng có thể làm.

But with a microservice architecture, the scope of our end-to-end tests becomes very large. We would now need to run tests across multiple processes, all of which need to be deployed and appropriately configured for the test scenarios. We also need to be prepared for the false negatives that occur when environmental issues, such as service instances dying or network time-outs or failed deployments, cause our tests to fail.

Nhưng với một kiến trúc microservice, phạm vi của các bài kiểm thử đầu-cuối của chúng ta trở nên rất lớn. Giờ đây chúng ta sẽ cần chạy các bài kiểm thử xuyên nhiều tiến trình, tất cả trong số đó cần được triển khai và cấu hình phù hợp cho các kịch bản kiểm thử. Chúng ta cũng cần chuẩn bị cho các âm tính giả (false negative) xảy ra khi các vấn đề về môi trường, chẳng hạn như các thể hiện service chết hoặc các time-out mạng của những đợt triển khai thất bại, khiến các bài kiểm thử của chúng ta thất bại.

These forces mean that as your microservice architecture grows, you will get a diminishing return on investment when it comes to end-to-end testing. The testing will cost more but won't manage to give you the same level of confidence that it did in the past. This will drive you toward new forms of testing, such as contract-driven testing or testing in production, as well as the exploration of progressive delivery techniques such as parallel runs or canary releases, which we'll look at in Chapter 8 .

Những lực này có nghĩa là khi kiến trúc microservice của bạn lớn lên, bạn sẽ nhận được lợi suất giảm dần trên khoản đầu tư khi nói đến kiểm thử đầu-cuối. Việc kiểm thử sẽ tốn kém hơn nhưng sẽ không thể mang lại cho bạn cùng mức độ niềm tin như nó từng làm trong quá khứ. Điều này sẽ thúc đẩy bạn hướng tới các dạng kiểm thử mới, chẳng hạn như kiểm thử dựa trên hợp đồng (contract-driven testing) hoặc kiểm thử trong production, cũng như việc khám phá các kỹ thuật chuyển giao tiệm tiến (progressive delivery) như các lần chạy song song (parallel run) hay các đợt phát hành kiểu canary (canary release), thứ mà chúng ta sẽ xem xét trong Chương 8.

Latency — Độ trễ

With a microservice architecture, processing that might previously have been done locally on one processor can now end up being split across multiple separate microservices. Information that previously flowed within only a single process now needs to be serialized, transmitted, and deserialized over networks that you might be exercising more than ever before. All of this can result in worsening latency of your system.

Với một kiến trúc microservice, việc xử lý trước kia có thể đã được thực hiện cục bộ trên một bộ xử lý giờ đây có thể rất cuộc bị chia tách qua nhiều microservice riêng biệt. Thông tin trước kia chỉ lưu chuyển bên trong một tiến trình duy nhất giờ đây cần được tuần tự hóa (serialize), truyền đi, và giải tuần tự hóa (deserialize) qua các mạng mà bạn có thể đang sử dụng nhiều hơn bao giờ hết. Tất cả những điều này có thể dẫn đến độ trễ của hệ thống của bạn tệ đi.

Although it can be difficult to measure the exact impact on latency of operations at the design or coding phase, this is another reason it's important to undertake any microservice migration in an incremental fashion. Make a small change and then measure the impact. This assumes that you have some way of measuring the end-to-end latency for the operations you care about—distributed tracing tools like Jaeger can help here. But you also need to have an understanding of what latency is acceptable for these operations. Sometimes making an operation slower is perfectly acceptable, as long as it is still fast enough!

Mặc dù có thể khó đo lường tác động chính xác lên độ trễ của các thao tác ở giai đoạn thiết kế hay viết mã, đây là một lý do nữa khiến việc tiến hành bất kỳ cuộc di trú microservice nào theo cách tiệm tiến là quan trọng. Hãy thực hiện một thay đổi nhỏ rồi đo lường tác động. Điều này giả định rằng bạn có một cách nào đó để đo lường độ trễ đầu-cuối cho các thao tác mà bạn quan tâm—các công cụ distributed tracing như Jaeger có thể giúp ích ở đây. Nhưng bạn cũng cần có một sự hiểu biết về độ trễ nào là chấp nhận được đối với các

thao tác này. Đôi khi việc làm một thao tác chậm hơn là hoàn toàn chấp nhận được, miễn là nó vẫn đủ nhanh!

Data Consistency — Tính nhất quán dữ liệu

Shifting from a monolithic system, in which data is stored and managed in a single database, to a much more distributed system, in which multiple processes manage state in different databases, causes potential challenges with respect to consistency of data. Whereas in the past you might have relied on database transactions to manage state changes, you'll need to understand that similar safety cannot easily be provided in a distributed system. The use of distributed transactions in most cases proves to be highly problematic in coordinating state changes.

Việc dịch chuyển từ một hệ thống monolith, nơi dữ liệu được lưu trữ và quản lý trong một cơ sở dữ liệu duy nhất, sang một hệ thống phân tán hơn nhiều, nơi nhiều tiến trình quản lý trạng thái trong các cơ sở dữ liệu khác nhau, gây ra những thách thức tiềm tàng đối với tính nhất quán của dữ liệu. Trong khi trước kia bạn có thể đã dựa vào các giao dịch cơ sở dữ liệu để quản lý các thay đổi trạng thái, bạn sẽ cần hiểu rằng sự an toàn tương tự không thể dễ dàng được cung cấp trong một hệ thống phân tán. Việc sử dụng các giao dịch phân tán trong hầu hết các trường hợp tỏ ra rất rắc rối trong việc điều phối các thay đổi trạng thái.

Instead, you might need to start using concepts like sagas (something I'll detail at length in Chapter 6) and **eventual consistency** to manage and reason about state in your system. These ideas can require fundamental changes in the way you think about data in your systems, something that can be quite daunting when migrating existing systems. Yet again, this is another good reason to be cautious in how quickly you decompose your application. Adopting an incremental approach to decomposition, so that you are able to assess the impact of changes to your architecture in production, is really important.

Thay vào đó, bạn có thể cần bắt đầu sử dụng các khái niệm như saga (thứ mà tôi sẽ trình bày chi tiết ở Chương 6) và nhất quán cuối cùng (eventual consistency) để quản lý và lập luận về trạng thái trong hệ thống của bạn. Những ý tưởng này có thể đòi hỏi những thay đổi cơ bản trong cách bạn nghĩ về dữ liệu trong các hệ thống của mình, một điều có thể khá đáng sợ khi di trú các hệ thống hiện có. Một lần nữa, đây là một lý do tốt nữa để thận trọng trong việc bạn phân rã ứng dụng của mình nhanh đến đâu. Áp dụng một cách tiếp cận tiệm tiến đối với việc phân rã, sao cho bạn có thể đánh giá tác động của các thay đổi đối với kiến trúc của mình trong production, thực sự là quan trọng.

Should I Use Microservices? — Tôi có nên dùng Microservices không?

Despite the drive in some quarters to make microservice architectures the default approach for software, I feel that because of the numerous challenges I've outlined, adopting them still requires careful thought. You need to assess your own problem space, skills, and technology landscape and understand what you are trying to achieve before deciding whether microservices are right for you. They are an architectural approach, not the architectural approach. Your own context should play a huge part in your decision whether to go down that path.

Bất chấp sự thúc đẩy ở một số nơi nhằm biến kiến trúc microservice thành cách tiếp cận mặc định cho phần mềm, tôi cảm thấy rằng vì vô số thách thức mà tôi đã phác họa, việc

áp dụng chúng vẫn đòi hỏi suy nghĩ cẩn thận. Bạn cần đánh giá không gian vấn đề, kỹ năng và bối cảnh công nghệ của riêng mình và hiểu được bạn đang cố gắng đạt được điều gì trước khi quyết định liệu microservices có phù hợp với bạn không. Chúng là một cách tiếp cận kiến trúc, không phải là cách tiếp cận kiến trúc. Bối cảnh của riêng bạn nên đóng một vai trò to lớn trong quyết định của bạn có đi theo con đường đó hay không.

That said, I want to outline a few situations that would typically tip me away from— or toward—picking microservices.

Dù vậy, tôi muốn phác họa một vài tình huống mà thường sẽ đẩy tôi ra xa—hoặc kéo tôi lại gần—việc chọn microservices.

Whom They Might Not Work For — Đối tượng mà chúng có thể không phù hợp

Given the importance of defining stable service boundaries, I feel that microservice architectures are often a bad choice for brand-new products or startups. In either case, the domain that you are working with is typically undergoing significant change as you iterate on the fundamentals of what you are trying to build. This shift in domain models will, in turn, result in more changes being made to service boundaries, and coordinating changes across service boundaries is an expensive undertaking. In general, I feel it's more appropriate to wait until enough of the domain model has stabilized before looking to define service boundaries.

Xét tầm quan trọng của việc định nghĩa các ranh giới service ổn định, tôi cảm thấy rằng kiến trúc microservice thường là một lựa chọn tồi đối với các sản phẩm hoàn toàn mới hoặc các startup. Trong cả hai trường hợp, miền mà bạn đang làm việc với thường đang trải qua sự thay đổi đáng kể khi bạn lặp lại trên những điều căn bản của thứ bạn đang cố xây dựng. Sự dịch chuyển trong các mô hình miền này đến lượt nó sẽ dẫn đến nhiều thay đổi hơn được thực hiện đối với các ranh giới service, và việc điều phối các thay đổi xuyên các ranh giới service là một công việc tốn kém. Nói chung, tôi cảm thấy thích hợp hơn là chờ đến khi đủ phần mô hình miền đã ổn định trước khi tìm cách định nghĩa các ranh giới service.

I do see a temptation for startups to go microservice first, the reasoning being, “If we’re really successful, we’ll need to scale!” The problem is that you don’t necessarily know if anyone is even going to want to use your new product. And even if you do become successful enough to require a highly scalable architecture, the thing you end up delivering to your users might be very different from what you started building in the first place. Uber initially focused on limos, and Flickr spun out of attempts to create a multiplayer online game. The process of finding product market fit means that you might end up with a very different product at the end than the one you thought you’d build when you started.

Tôi quả thực thấy một sự cám dỗ đối với các startup là đi theo microservice trước tiên, với lý lẽ là, “Nếu chúng ta thực sự thành công, chúng ta sẽ cần mở rộng!” Vấn đề là bạn không nhất thiết biết liệu có ai sẽ thậm chí muốn dùng sản phẩm mới của bạn hay không. Và ngay cả khi bạn có trở nên thành công đủ để cần một kiến trúc có khả năng mở rộng cao, thì thứ mà bạn rốt cuộc chuyển giao cho người dùng có thể rất khác với thứ bạn bắt đầu xây dựng ban đầu. Uber ban đầu tập trung vào xe limo, và Flickr tách ra từ những nỗ lực tạo ra một trò chơi trực tuyến nhiều người chơi. Quá trình tìm kiếm sự phù hợp giữa sản phẩm và thị trường (product market fit) có nghĩa là bạn có thể rốt cuộc có một sản phẩm rất khác ở cuối so với sản phẩm bạn nghĩ mình sẽ xây dựng khi bắt đầu.

Startups also typically have fewer people available to build the system, which creates more challenges with respect to microservices. Microservices bring with them sources of new work and complexity, and this can tie up valuable bandwidth. The smaller the team, the more pronounced this cost will be. When working with smaller teams with just a handful of developers, I'm very hesitant to suggest microservices for this reason.

Các startup cũng thường có ít người hơn để xây dựng hệ thống, điều này tạo ra nhiều thách thức hơn đối với microservices. Microservices mang theo các nguồn công việc và sự phức tạp mới, và điều này có thể chiếm dụng băng thông quý giá. Nhóm càng nhỏ, chi phí này sẽ càng rõ rệt. Khi làm việc với các nhóm nhỏ hơn chỉ với một dúm lập trình viên, tôi rất do dự khi gợi ý microservices vì lý do này.

The challenge of microservices for startups is compounded by the fact that normally your biggest constraint is people. For a small team, a microservice architecture can be difficult to justify because there is work required just to handle the deployment and management of the microservices themselves. Some people have described this as the “microservice tax.” When that investment benefits lots of people, it's easier to justify. But if one person out of your five-person team is spending their time on these issues, that's a lot of valuable time not being spent building your product. It's much easier to move to microservices later, after you understand where the constraints are in your architecture and what your pain points are—then you can focus your energy on using microservices in the most sensible places.

Thách thức của microservices đối với các startup càng tệ thêm bởi thực tế rằng thường thì ràng buộc lớn nhất của bạn là con người. Đối với một nhóm nhỏ, một kiến trúc microservice có thể khó biện minh được vì có công việc cần thực hiện chỉ để xử lý việc triển khai và quản lý chính các microservice. Một số người đã mô tả điều này như “thuế microservice” (microservice tax). Khi khoản đầu tư đó mang lại lợi ích cho nhiều người, thì nó dễ biện minh hơn. Nhưng nếu một người trong nhóm năm người của bạn dành thời gian của họ cho những vấn đề này, thì đó là rất nhiều thời gian quý giá không được dùng để xây dựng sản phẩm của bạn. Việc chuyển sang microservices về sau dễ hơn nhiều, sau khi bạn hiểu được những ràng buộc nằm ở đâu trong kiến trúc của mình và các điểm đau của bạn là gì—khi đó bạn có thể tập trung năng lượng của mình vào việc sử dụng microservices ở những nơi hợp lý nhất.

Finally, organizations creating software that will be deployed and managed by their customers may struggle with microservices. As we've already covered, microservice architectures can push a lot of complexity into the deployment and operational domain. If you are running the software yourself, you are able to offset this new complexity by adopting new technology, developing new skills, and changing working practices. This isn't something you can expect your customers to do. If they are used to receiving your software as a Windows installer, it's going to come as an awful shock to them when you send out the next version of your software and say, “Just put these 20 pods on your Kubernetes cluster!” In all likelihood, they will have no idea what a pod, Kubernetes, or a cluster even is.

Cuối cùng, các tổ chức tạo ra phần mềm sẽ được triển khai và quản lý bởi khách hàng của họ có thể chật vật với microservices. Như chúng ta đã đề cập, các kiến trúc microservice có thể đẩy rất nhiều sự phức tạp vào miền triển khai và vận hành. Nếu bạn tự vận hành phần mềm, bạn có thể bù đắp sự phức tạp mới này bằng cách áp dụng công nghệ mới, phát triển các kỹ năng mới, và thay đổi các thực hành làm việc. Đây không phải là thứ bạn có thể kỳ vọng khách hàng của mình làm. Nếu họ đã quen với việc nhận phần mềm của bạn dưới dạng một trình cài đặt Windows, thì sẽ là một cú sốc khủng khiếp đối với họ khi bạn gửi ra phiên bản tiếp theo của phần mềm và nói, “Chỉ cần đặt 20 pod này lên cụm Kubernetes của bạn thôi!” Nhiều khả năng là họ sẽ chẳng có ý niệm gì về một pod,

Kubernetes, hay một cụm thậm chí là gì.

Where They Work Well — Nơi chúng hoạt động tốt

In my experience, probably the single biggest reason that organizations adopt microservices is to allow for more developers to work on the same system without getting in each other's way. Get your architecture and organizational boundaries right, and you allow more people to work independently of each other, reducing delivery contention. A five-person startup is likely to find a microservice architecture a drag. A hundred-person scale-up that is growing rapidly is likely to find that its growth is much easier to accommodate with a microservice architecture properly aligned around its product development efforts.

Theo kinh nghiệm của tôi, có lẽ lý do đơn lẻ lớn nhất khiến các tổ chức áp dụng microservices là để cho phép nhiều lập trình viên hơn làm việc trên cùng một hệ thống mà không cản trở lẫn nhau. Hãy làm cho đúng các ranh giới kiến trúc và tổ chức của bạn, và bạn cho phép nhiều người hơn làm việc độc lập với nhau, giảm sự tranh chấp trong chuyển giao. Một startup năm người có khả năng sẽ thấy một kiến trúc microservice là một lực cản. Một công ty đang mở rộng quy mô (scale-up) với một trăm người đang tăng trưởng nhanh chóng có khả năng sẽ thấy rằng sự tăng trưởng của mình dễ dàng được đáp ứng hơn nhiều với một kiến trúc microservice được đồng bộ đúng cách xoay quanh các nỗ lực phát triển sản phẩm của nó.

Software as a Service (SaaS) applications are, in general, also a good fit for a microservice architecture. These products are typically expected to operate 24-7, which creates challenges when it comes to rolling out changes. The independent releasability of microservice architectures is a huge boon in this area. Furthermore, the microservices can be scaled up or down as required. This means that as you establish a sensible baseline for your system's load characteristics, you get more control over ensuring that you can scale your system in the most cost-effective way possible.

Các ứng dụng phần mềm dạng dịch vụ (Software as a Service, SaaS) nói chung cũng là một sự phù hợp tốt cho một kiến trúc microservice. Các sản phẩm này thường được kỳ vọng vận hành 24-7, điều này tạo ra các thách thức khi nói đến việc triển khai các thay đổi. Khả năng phát hành độc lập của các kiến trúc microservice là một lợi ích to lớn trong lĩnh vực này. Hơn nữa, các microservice có thể được mở rộng lên hoặc xuống khi cần. Điều này có nghĩa là khi bạn thiết lập một mức cơ sở hợp lý cho các đặc tính tải của hệ thống của mình, bạn có được nhiều quyền kiểm soát hơn đối với việc đảm bảo rằng bạn có thể mở rộng hệ thống của mình theo cách tiết kiệm chi phí nhất có thể.

The technology-agnostic nature of microservices ensures that you can get the most out of cloud platforms. Public cloud vendors provide a wide array of services and deployment mechanisms for your code. You can much more easily match the requirements of specific services to the cloud services that will best help you implement them. For example, you might decide to deploy one service as a set of functions, another as a managed virtual machine (VM), and another on a managed **Platform as a Service** (PaaS) platform.

Bản chất độc lập với công nghệ của microservices đảm bảo rằng bạn có thể tận dụng tối đa các nền tảng đám mây. Các nhà cung cấp đám mây công cộng cung cấp một mảng rộng các dịch vụ và cơ chế triển khai cho mã của bạn. Bạn có thể dễ dàng hơn nhiều trong việc khớp các yêu cầu của các service cụ thể với các dịch vụ đám mây sẽ giúp bạn triển khai chúng tốt nhất. Chẳng hạn, bạn có thể quyết định triển khai một service như một tập các function, một service khác như một máy ảo (virtual machine, VM) được quản lý, và một service khác trên một nền tảng Platform as a Service (PaaS) được quản lý.

Although it's worth noting that adopting a wide range of technology can often be a problem, being able to try out new technology easily is a good way to rapidly identify new approaches that might yield benefits. The growing popularity of FaaS platforms is one such example. For the appropriate workloads, an FaaS platform can drastically reduce the amount of operational overhead, but at present, it's not a deployment mechanism that would be suitable in all cases.

Mặc dù đáng lưu ý rằng việc áp dụng một dải rộng các công nghệ thường có thể là một vấn đề, việc có khả năng thử nghiệm công nghệ mới một cách dễ dàng là một cách tốt để nhanh chóng nhận diện các cách tiếp cận mới có thể mang lại lợi ích. Sự phổ biến ngày càng tăng của các nền tảng FaaS là một ví dụ như vậy. Đối với các khối lượng công việc phù hợp, một nền tảng FaaS có thể giảm đáng kể lượng phụ phí vận hành, nhưng ở hiện tại, đó không phải là một cơ chế triển khai phù hợp trong mọi trường hợp.

Microservices also present clear benefits for organizations looking to provide services to their customers over a variety of new channels. A lot of digital transformation efforts seem to involve trying to unlock functionality hidden away in existing systems. The desire is to create new customer experiences that can support the needs of users via whatever interaction mechanism makes the most sense.

Microservices cũng mang lại những lợi ích rõ ràng cho các tổ chức đang tìm cách cung cấp dịch vụ cho khách hàng của họ qua một loạt các kênh mới. Rất nhiều nỗ lực chuyển đổi số (digital transformation) dường như liên quan đến việc cố mở khóa chức năng đang bị giấu kín trong các hệ thống hiện có. Mong muốn là tạo ra những trải nghiệm khách hàng mới có thể hỗ trợ nhu cầu của người dùng thông qua bất kỳ cơ chế tương tác nào hợp lý nhất.

Above all, a microservice architecture is one that can give you a lot of flexibility as you continue to evolve your system. That flexibility has a cost, of course, but if you want to keep your options open regarding changes you might want to make in the future, it could be a price worth paying.

Trên hết, một kiến trúc microservice là một kiến trúc có thể trao cho bạn rất nhiều linh hoạt khi bạn tiếp tục tiến hóa hệ thống của mình. Tất nhiên, sự linh hoạt đó có một cái giá, nhưng nếu bạn muốn giữ cho các lựa chọn của mình rộng mở liên quan đến những thay đổi bạn có thể muốn thực hiện trong tương lai, thì đó có thể là một cái giá đáng trả.

Summary — Tóm tắt

Microservice architectures can give you a huge degree of flexibility in choosing technology, handling robustness and scaling, organizing teams, and more. This flexibility is in part why many people are embracing microservice architectures. But microservices bring with them a significant degree of complexity, and you need to ensure that this complexity is warranted. For many, they have become a default system architecture, to be used in virtually all situations. However, I still think that they are an architectural choice whose use must be justified by the problems you are trying to solve; often, simpler approaches can deliver much more easily.

Các kiến trúc microservice có thể trao cho bạn một mức độ linh hoạt to lớn trong việc chọn công nghệ, xử lý độ vững chắc và mở rộng, tổ chức các nhóm, và hơn thế nữa. Sự linh hoạt này một phần là lý do vì sao nhiều người đang đón nhận các kiến trúc microservice. Nhưng microservices mang theo một mức độ phức tạp đáng kể, và bạn cần đảm bảo rằng sự phức tạp này là chính đáng. Đối với nhiều người, chúng đã trở thành một kiến trúc hệ thống mặc định, để được sử dụng trong hầu như mọi tình huống. Tuy nhiên, tôi vẫn nghĩ rằng chúng là một lựa chọn kiến trúc mà việc sử dụng phải được biện minh bởi những vấn đề

bạn đang cố giải quyết; thường thì những cách tiếp cận đơn giản hơn có thể chuyển giao dễ dàng hơn nhiều.

Nonetheless, many organizations, especially larger ones, have shown how effective microservices can be. When the core concepts of microservices are properly understood and implemented, they can help create empowering, productive architectures that can help systems become more than the sum of their parts.

Tuy nhiên, nhiều tổ chức, đặc biệt là các tổ chức lớn hơn, đã cho thấy microservices có thể hiệu quả đến mức nào. Khi các khái niệm cốt lõi của microservices được hiểu và triển khai đúng cách, chúng có thể giúp tạo ra những kiến trúc trao quyền, năng suất, có thể giúp các hệ thống trở thành nhiều hơn tổng các phần của chúng.

I hope this chapter has served as a good introduction to these topics. Next, we're going to look at how we define microservice boundaries, exploring the topics of structured programming and domain-driven design along the way.

Tôi hy vọng chương này đã đóng vai trò như một phần giới thiệu tốt cho những chủ đề này. Tiếp theo, chúng ta sẽ xem xét cách chúng ta định nghĩa các ranh giới microservice, khám phá các chủ đề về lập trình có cấu trúc và domain-driven design trên hành trình đó.

CHAPTER 2 How to Model Microservices — Chương 2: Cách mô hình hóa Microservice

My opponent’s reasoning reminds me of the heathen, who, being asked on what the world stood, replied, “On a tortoise.” But on what does the tortoise stand? “On another tortoise.” —Rev. Joseph Frederick Berg (1854)

Lập luận của đối thủ làm tôi nhớ đến gã ngoại đạo, khi được hỏi thế giới đứng trên cái gì, đã trả lời: “Trên một con rùa.” Nhưng con rùa ấy đứng trên cái gì? “Trên một con rùa khác.” —Mục sư Joseph Frederick Berg (1854)

So you know what **microservices** are and, I hope, have a sense of their key benefits. You’re probably eager now to go and start making them, right? But where to start? In this chapter, we’ll look at some foundational concepts such as **information hiding**, **coupling**, and **cohesion** and understand how they’ll shift our thinking about drawing boundaries around our microservices. We’ll also look at different forms of decomposition you might use, as well as focusing more deeply on **domain-driven design** being a hugely useful technique in this space.

Vậy là bạn đã biết microservice là gì và, tôi hy vọng, có cảm nhận về những lợi ích chính của chúng. Có lẽ bây giờ bạn đang nóng lòng muốn bắt tay vào xây dựng chúng, đúng không? Nhưng bắt đầu từ đâu? Trong chương này, chúng ta sẽ xem xét một số khái niệm nền tảng như che giấu thông tin (information hiding), coupling (sự phụ thuộc) và cohesion (tính gắn kết), và hiểu xem chúng sẽ làm thay đổi cách tư duy của ta thế nào khi vẽ ranh giới quanh các microservice. Chúng ta cũng sẽ xem xét những hình thức phân rã khác nhau mà bạn có thể dùng, đồng thời đi sâu hơn vào domain-driven design (DDD) như một kỹ thuật cực kỳ hữu ích trong lĩnh vực này.

We’ll look at how to think about the boundaries of your microservices so as to maximize the upsides and avoid some of the potential downsides. But first, we need something to work with.

Chúng ta sẽ xem xét cách tư duy về ranh giới của các microservice sao cho tối đa hóa mặt lợi và tránh một số mặt hại tiềm tàng. Nhưng trước hết, chúng ta cần một thứ gì đó để làm việc cùng.

Introducing MusicCorp — Giới thiệu về MusicCorp

Books about ideas work better with examples. Where possible, I’ll be sharing stories from real-world situations, but I’ve found it’s also useful to have a fictional scenario to work with. Throughout the

book, we'll be returning to this scenario, seeing how the concept of microservices works within this world.

Sách về các ý tưởng sẽ hiệu quả hơn khi có ví dụ. Bất cứ khi nào có thể, tôi sẽ chia sẻ những câu chuyện từ các tình huống thực tế, nhưng tôi nhận thấy việc có một kịch bản hư cấu để làm việc cùng cũng rất hữu ích. Xuyên suốt cuốn sách, chúng ta sẽ quay lại với kịch bản này, để xem khái niệm microservice vận hành ra sao trong thế giới đó.

So let's turn our attention to the cutting-edge online retailer MusicCorp. MusicCorp was until recently just a brick-and-mortar retailer, but after the bottom dropped out of the vinyl record business, it focused more and more of its efforts online. The company has a website but feels that now is the time to double-down on the online world. After all, those smartphones for music are just a passing fad (Zunes are way better, obviously), and music fans are quite happy to wait for CDs to arrive at their doorsteps. Quality over convenience, right? And while it may have just learned that Spotify is in fact a digital music **service** rather than some sort of skin treatment for teenagers, MusicCorp is pretty happy with its own focus and is sure all of this streaming business will blow over soon.

Vậy hãy chuyển sự chú ý sang nhà bán lẻ trực tuyến tân tiến MusicCorp. Cho đến gần đây, MusicCorp vẫn chỉ là một nhà bán lẻ truyền thống có cửa hàng vật lý, nhưng sau khi ngành kinh doanh đĩa than sụp đổ, công ty dồn ngày càng nhiều nỗ lực vào hoạt động trực tuyến. Công ty có một trang web nhưng cảm thấy rằng giờ là lúc phải dốc sức gấp đôi cho thế giới trực tuyến. Suy cho cùng, mấy cái smartphone để nghe nhạc chỉ là một trào lưu nhất thời (hiển nhiên là Zune xịn hơn nhiều), và các fan âm nhạc hoàn toàn vui lòng chờ đĩa CD được giao tận cửa. Chất lượng quan trọng hơn sự tiện lợi, đúng không? Và dù vừa mới biết rằng Spotify thực ra là một dịch vụ nhạc số chứ không phải một loại kem trị mụn nào đó cho tuổi teen, MusicCorp vẫn khá hài lòng với hướng đi của riêng mình và tin chắc rằng cái trò streaming này rồi sẽ sớm chìm vào quên lãng.

Despite being a little behind the curve, MusicCorp has grand ambitions. Luckily, it has decided that its best chance of taking over the world is to make sure it can make changes as easily as possible. Microservices for the win!

Dù có hơi chậm chân, MusicCorp vẫn ôm những tham vọng lớn lao. May thay, công ty đã xác định rằng cơ hội tốt nhất để thống trị thế giới là đảm bảo mình có thể thực hiện các thay đổi dễ dàng nhất có thể. Microservice lên ngôi!

What Makes a Good Microservice Boundary? — Điều gì tạo nên một ranh giới microservice tốt?

Before the team from MusicCorp tears off into the distance, creating service after service in an attempt to deliver eight-track tapes to all and sundry, let's put the brakes on and talk a bit about the most important underlying idea we need to keep in mind. We want our microservices to be able to be changed and deployed, and their functionality released to our users, in an independent fashion. The ability to change one **microservice** in isolation from another is vital. So what things do we need to bear in mind when we think about how we draw the boundaries around them?

Trước khi đội ngũ của MusicCorp lao đi vun vút, tạo hết service này đến service khác trong nỗ lực giao băng tám rãnh (eight-track tape) cho khắp thiên hạ, hãy đạp phanh lại và bàn một chút về ý tưởng nền tảng quan trọng nhất mà ta cần ghi nhớ. Chúng ta muốn các microservice của mình có thể được thay đổi và triển khai, và chức năng của chúng được phát hành tới người dùng, theo một cách độc lập. Khả năng thay đổi một microservice một

cách tách biệt với những microservice khác là cực kỳ thiết yếu. Vậy ta cần lưu ý những điều gì khi nghĩ về cách vẽ ranh giới quanh chúng?

In essence, microservices are just another form of modular decomposition, albeit one that has network-based interaction between the models and all the associated challenges that brings. Luckily, this means we can rely on a lot of prior art in the space of modular software and structured programming to help guide us in terms of working out how to define our boundaries. With that in mind, let's look more deeply at three key concepts that we touched on briefly in Chapter 1 and that are vital to grasp when it comes to working out what makes for a good microservice boundary—information hiding, cohesion, and coupling.

Về bản chất, microservice chỉ là một dạng phân rã theo module khác, dù đó là dạng có sự tương tác qua mạng giữa các mô-đun cùng tất cả những thách thức đi kèm. May thay, điều này nghĩa là ta có thể dựa vào rất nhiều thành tựu đi trước trong lĩnh vực phần mềm module và lập trình có cấu trúc để định hướng cho việc xác định ranh giới của mình. Với ý đó trong đầu, hãy đi sâu hơn vào ba khái niệm cốt lõi mà ta đã chạm tới sơ qua ở Chương 1 và là những điều thiết yếu phải nắm khi cần xác định điều gì tạo nên một ranh giới microservice tốt—đó là che giấu thông tin (information hiding), cohesion và coupling.

Information Hiding — Che giấu thông tin (Information Hiding)

Information hiding is a concept developed by David Parnas to look at the most effective way to define module boundaries. 1 Information hiding describes a desire to hide as many details as possible behind a module (or, in our case, microservice) boundary. Parnas looked at the benefits that modules should theoretically give us, namely:

Che giấu thông tin là một khái niệm do David Parnas phát triển nhằm tìm ra cách hiệu quả nhất để định nghĩa ranh giới module. 1 Che giấu thông tin mô tả mong muốn giấu càng nhiều chi tiết càng tốt phía sau ranh giới của một module (hay, trong trường hợp của ta, một microservice). Parnas đã xem xét những lợi ích mà về mặt lý thuyết các module nên mang lại cho chúng ta, cụ thể là:

Improved development time By allowing modules to be developed independently, we can allow for more work to be done in parallel and reduce the impact of adding more developers to a project.

Rút ngắn thời gian phát triển Bằng cách cho phép các module được phát triển độc lập, ta có thể tiến hành nhiều công việc song song hơn và giảm tác động tiêu cực của việc bổ sung thêm lập trình viên vào một dự án.

1 David, Parnas, “On the Criteria to Be Used in Decomposing Systems into Modules,” (journal contribution, Carnegie Mellon University, 1971), <https://oreil.ly/BnVVg>.

1 David, Parnas, “On the Criteria to Be Used in Decomposing Systems into Modules,” (journal contribution, Carnegie Mellon University, 1971), <https://oreil.ly/BnVVg>.

Comprehensibility Each module can be looked at in isolation and understood in isolation. This in turn makes it easier to understand what the system as a whole does.

Khả năng dễ hiểu Mỗi module có thể được xem xét tách biệt và hiểu một cách tách biệt. Điều này đến lượt nó giúp dễ hiểu hơn về việc toàn bộ hệ thống làm gì.

Flexibility Modules can be changed independently from one another, allowing for changes to be made to the functionality of the system without requiring other modules to change. In addition, modules can be combined in different ways to deliver new functionality.

Tính linh hoạt Các module có thể được thay đổi độc lập với nhau, cho phép thực hiện các thay đổi đối với chức năng của hệ thống mà không đòi hỏi các module khác phải thay đổi. Ngoài ra, các module có thể được kết hợp theo những cách khác nhau để cung cấp chức năng mới.

This list of desirable characteristics nicely complements what we are trying to achieve with microservice architectures—and indeed I now see microservices as just another form of modular **architecture**. Adrian Colyer has actually looked back at a number of David Parnas's papers from this period and examined them with respect to microservices, and his summaries are well worth reading. 2

Danh sách những đặc tính đáng mong muốn này bổ trợ rất đẹp cho điều mà ta đang cố đạt được với các kiến trúc microservice—và thực ra giờ đây tôi xem microservice chỉ là một dạng kiến trúc module khác mà thôi. Adrian Colyer thực sự đã nhìn lại một số bài viết của David Parnas từ thời kỳ này và xem xét chúng dưới góc độ microservice, và các bản tóm tắt của ông rất đáng đọc. 2

The reality, as Parnas explored throughout much of his work, is that having modules doesn't result in your actually achieving these outcomes. A lot depends on how the module boundaries are formed. From his own research, information hiding was a key technique to help get the most out of modular architectures, and with a modern eye, the same applies to microservices too.

Thực tế, như Parnas đã khám phá xuyên suốt phần lớn công trình của mình, là việc có module không tự động dẫn đến việc bạn thực sự đạt được những kết quả này. Phần lớn phụ thuộc vào cách các ranh giới module được hình thành. Từ nghiên cứu của chính ông, che giấu thông tin là một kỹ thuật then chốt giúp tận dụng tối đa các kiến trúc module, và với con mắt hiện đại, điều này cũng áp dụng cho microservice.

From another of Parnas's papers, 3 we have this gem:

Từ một bài viết khác của Parnas, 3 ta có viên ngọc này:

The connections between modules are the assumptions which the modules make about each other. By reducing the number of assumptions that one module (or microservice) makes about another, we directly impact the connections between them. By keeping the number of assumptions small, it is easier to ensure that we can change one module without impacting others. If a developer changing a module has a clear understanding as to how the module is used by others, it will be easier for the developer to make changes safely in such a way that upstream callers won't also have to change.

Mối liên kết giữa các module chính là những giả định mà các module đặt ra về nhau. Bằng cách giảm số lượng giả định mà một module (hay microservice) đặt ra về module khác, ta tác động trực tiếp đến mối liên kết giữa chúng. Bằng cách giữ số lượng giả định nhỏ, ta dễ dàng đảm bảo có thể thay đổi một module mà không ảnh hưởng đến những module khác. Nếu một lập trình viên đang thay đổi một module hiểu rõ cách module đó được những bên khác sử dụng, thì lập trình viên ấy sẽ dễ thực hiện các thay đổi một cách an toàn theo hướng mà các bên gọi phía trên (upstream caller) cũng không phải thay đổi theo.

This applies with microservices, as well, except that we also have the opportunity to **deploy** that changed microservice without having to deploy anything else, arguably amplifying the three desirable characteristics that Parnas describes of improved development time, comprehensibility, and flexibility.

Điều này cũng áp dụng với microservice, ngoại trừ việc ta còn có thêm cơ hội triển khai microservice đã thay đổi đó mà không phải triển khai bất cứ thứ gì khác, có thể nói là khuếch đại thêm ba đặc tính đáng mong muốn mà Parnas mô tả là rút ngắn thời gian phát

triển, khả năng dễ hiểu và tính linh hoạt.

The implications of information hiding play out in so many ways, and I'll pick up this theme throughout the book.

Các hệ quả của che giấu thông tin biểu hiện theo rất nhiều cách, và tôi sẽ trở lại với chủ đề này xuyên suốt cuốn sách.

2 The obvious starting point is Adrian's summary of "On the Criteria..." , but Adrian's coverage of Parnas's earlier work, "Information Distribution Aspects of Design Methodology" , contains some great insights along with commentary from Parnas himself.

2 Điểm khởi đầu hiển nhiên là bản tóm tắt của Adrian về "On the Criteria..." , nhưng phần Adrian viết về công trình trước đó của Parnas, "Information Distribution Aspects of Design Methodology" , chứa nhiều nhận định sâu sắc cùng với bình luận từ chính Parnas.

3 Parnas, "Information Distribution Aspects."

3 Parnas, "Information Distribution Aspects."

Cohesion — Cohesion (Tính gắn kết)

One of the most succinct definitions I've heard for describing cohesion is this: "the code that changes together, stays together." 4 For our purposes, this is a pretty good definition. As we've already discussed, we're optimizing our microservice architecture around ease of making changes in business functionality—so we want the functionality grouped in such a way that we can make changes in as few places as possible.

Một trong những định nghĩa súc tích nhất mà tôi từng nghe để mô tả cohesion là: "đoạn mã thay đổi cùng nhau thì ở cùng nhau." 4 Với mục đích của chúng ta, đây là một định nghĩa khá tốt. Như đã bàn, ta đang tối ưu kiến trúc microservice của mình quanh việc dễ dàng thực hiện các thay đổi về chức năng nghiệp vụ—nên ta muốn chức năng được nhóm lại theo cách cho phép thực hiện các thay đổi ở càng ít nơi càng tốt.

We want related behavior to sit together, and unrelated behavior to sit elsewhere. Why? Well, if we want to change behavior, we want to be able to change it in one place, and to **release** that change as soon as possible. If we have to change that behavior in lots of different places, we'll have to release lots of different services (perhaps at the same time) to deliver that change. Making changes in lots of different places is slower, and deploying lots of services at once is risky—thus we want to avoid both.

Chúng ta muốn các hành vi có liên quan đến nhau nằm cùng nhau, và các hành vi không liên quan nằm ở nơi khác. Vì sao? À, nếu ta muốn thay đổi một hành vi, ta muốn có thể thay đổi nó ở một nơi, và phát hành thay đổi đó càng sớm càng tốt. Nếu phải thay đổi hành vi đó ở nhiều nơi khác nhau, ta sẽ phải phát hành nhiều service khác nhau (có lẽ là cùng lúc) để mang lại thay đổi ấy. Thực hiện thay đổi ở nhiều nơi khác nhau thì chậm hơn, và triển khai nhiều service cùng lúc thì rủi ro—do đó ta muốn tránh cả hai.

So we want to find boundaries within our problem domain that help ensure related behavior is in one place and that communicate with other boundaries as loosely as possible. If the related functionality is spread across the system, we say that cohesion is weak—whereas for our microservice architectures we're aiming for strong cohesion.

Vậy nên ta muốn tìm ra những ranh giới trong miền vấn đề của mình giúp đảm bảo các hành vi liên quan nằm ở một nơi và giao tiếp với các ranh giới khác một cách lỏng lẻo nhất

có thể. Nếu chức năng liên quan bị rải khắp hệ thống, ta nói rằng cohesion là yếu—trong khi với các kiến trúc microservice của mình, ta hướng tới cohesion mạnh.

Coupling — Coupling (Sự phụ thuộc)

When services are loosely coupled, a change to one service should not require a change to another. The whole point of a microservice is being able to make a change to one service and deploy it without needing to change any other part of the system. This is really quite important.

Khi các service có coupling lỏng (loosely coupled), một thay đổi ở một service không nên đòi hỏi phải thay đổi service khác. Toàn bộ điểm mấu chốt của một microservice là có thể thực hiện một thay đổi ở một service và triển khai nó mà không cần thay đổi bất kỳ phần nào khác của hệ thống. Điều này thực sự rất quan trọng.

What sorts of things cause **tight coupling**? A classic mistake is to pick an integration style that tightly binds one service to another, causing changes inside the service to require a change to consumers.

Những thứ kiểu gì gây ra coupling chặt? Một sai lầm kinh điển là chọn một kiểu tích hợp ràng buộc chặt một service với service khác, khiến những thay đổi bên trong service đòi hỏi phía tiêu thụ (consumer) cũng phải thay đổi.

A loosely coupled service knows as little as it needs to about the services with which it collaborates. This also means we probably want to limit the number of different types of calls from one service to another, because beyond the potential performance problem, chatty communication can lead to tight coupling.

Một service có coupling lỏng biết về các service mà nó cộng tác cùng ở mức tối thiểu cần thiết. Điều này cũng nghĩa là ta có lẽ muốn giới hạn số lượng các loại lời gọi khác nhau từ service này sang service khác, bởi ngoài vấn đề hiệu năng tiềm tàng, giao tiếp lắm lời (chatty) có thể dẫn đến coupling chặt.

Coupling, though, comes in many forms, and I've seen a number of misunderstandings about the nature of coupling as it pertains to a service-based architecture. With that in mind, I think it's important that we explore this topic in more detail, something we'll do shortly.

Tuy nhiên, coupling tồn tại dưới nhiều hình thức, và tôi đã thấy không ít sự hiểu lầm về bản chất của coupling khi nó liên quan đến một kiến trúc dựa trên service. Với ý đó trong đầu, tôi nghĩ điều quan trọng là ta phải khám phá chủ đề này chi tiết hơn, một việc ta sẽ làm ngay sau đây.

4 Annoyingly, I cannot find the original source of this definition.

4 Phiền nỗi, tôi không tìm được nguồn gốc của định nghĩa này.

The Interplay of Coupling and Cohesion — Sự tương tác giữa Coupling và Cohesion

As we've already touched on, the concepts of coupling and cohesion are obviously related. Logically, if related functionality is spread across our system, changes to this functionality will ripple across those boundaries, implying tighter coupling. Constantine's law, named for structured design pioneer Larry Constantine, sums this up neatly:

Như ta đã chạm tới, các khái niệm coupling và cohesion rõ ràng có liên quan đến nhau. Một cách logic, nếu chức năng liên quan bị rải khắp hệ thống, những thay đổi đối với chức năng này sẽ lan tỏa qua các ranh giới đó, ngụ ý coupling chặt hơn. Định luật Constantine, đặt theo tên Larry Constantine, nhà tiên phong về thiết kế có cấu trúc, tóm gọn điều này một cách gọn ghẽ:

A structure is stable if cohesion is strong and coupling is low. 5 The concept here of stability is important to us. For our microservice boundaries to deliver on the promise of **independent deployability**, allowing us to work on microservices in parallel and reduce the amount of coordination between teams working on these services, we need some degree of stability in the boundaries themselves. If the contract that a microservice exposes is constantly changing in a backward-incompatible fashion, then this will cause upstream consumers to constantly have to change too.

Một cấu trúc là ổn định nếu cohesion mạnh và coupling thấp. 5 Khái niệm về sự ổn định ở đây quan trọng với chúng ta. Để các ranh giới microservice của ta thực hiện được lời hứa về khả năng triển khai độc lập, cho phép ta làm việc trên các microservice song song và giảm lượng phối hợp cần thiết giữa các đội làm việc trên những service này, ta cần một mức độ ổn định nhất định ở chính các ranh giới ấy. Nếu hợp đồng (contract) mà một microservice phơi bày liên tục thay đổi theo cách không tương thích ngược, thì điều này sẽ khiến các bên tiêu thụ phía trên cũng liên tục phải thay đổi theo.

Coupling and cohesion are strongly related and, at some level at least, are arguably the same in that both concepts describe the relationship between things. Cohesion applies to the relationship between things inside a boundary (a microservice in our context), whereas coupling describes the relationship between things across a boundary. There is no absolute best way to organize our code; coupling and cohesion are just one way to articulate the various trade-offs we make around where we group code, and why. All we can strive to do is to find the right balance between these two ideas, one that makes the most sense for your given context and the problems you are currently facing.

Coupling và cohesion liên hệ chặt chẽ với nhau và, ít nhất ở một mức độ nào đó, có thể nói là giống nhau ở chỗ cả hai khái niệm đều mô tả mối quan hệ giữa các sự vật. Cohesion áp dụng cho mối quan hệ giữa các sự vật bên trong một ranh giới (trong ngữ cảnh của ta là một microservice), trong khi coupling mô tả mối quan hệ giữa các sự vật xuyên qua một ranh giới. Không có cách tổ chức mã tốt nhất tuyệt đối; coupling và cohesion chỉ là một cách để diễn đạt những đánh đổi khác nhau mà ta thực hiện quanh việc nhóm mã ở đâu, và vì sao. Tất cả những gì ta có thể cố gắng làm là tìm ra sự cân bằng đúng đắn giữa hai ý tưởng này, một sự cân bằng hợp lý nhất với ngữ cảnh cụ thể của bạn và những vấn đề bạn đang đối mặt.

Remember, the world isn't static—it's possible that as your system requirements change, you'll find reasons to revisit your decisions. Sometimes parts of your system may be going through so much change that stability might be impossible. We'll look at an example of this in Chapter 3 when I share the experiences of the product development team behind Snap CI.

Hãy nhớ, thế giới không tĩnh tại—rất có thể khi các yêu cầu hệ thống của bạn thay đổi, bạn sẽ thấy có lý do để xem lại các quyết định của mình. Đôi khi các phần trong hệ thống của bạn có thể đang trải qua nhiều thay đổi đến mức không thể có sự ổn định. Chúng ta sẽ xem một ví dụ về điều này ở Chương 3 khi tôi chia sẻ trải nghiệm của đội phát triển sản phẩm đứng sau Snap CI.

Types of Coupling — Các loại Coupling

You could infer from the preceding overview above that all coupling is bad. That isn't strictly true. Ultimately, some coupling in our system will be unavoidable. What we want to do is reduce how much coupling we have.

Từ phần tổng quan ở trên, bạn có thể suy ra rằng mọi coupling đều xấu. Điều đó không hẳn đúng. Suy cho cùng, một mức coupling nào đó trong hệ thống của ta là không thể tránh khỏi. Điều ta muốn làm là giảm bớt lượng coupling mà ta có.

5 In my book **Monolith** to Microservices (O'Reilly) I attributed this to Larry Constantine himself. While the statement neatly sums up much of Constantine's work in this space, the quote should really be attributed to Albert Endres and Dieter Rombach, from their 2003 book *A Handbook of Software and Systems Engineering* (Addison-Wesley).

5 Trong cuốn *Monolith to Microservices* (O'Reilly) của mình, tôi đã gán câu này cho chính Larry Constantine. Dù phát biểu ấy tóm gọn rất đẹp phần lớn công trình của Constantine trong lĩnh vực này, lời trích thực ra nên được gán cho Albert Endres và Dieter Rombach, trong cuốn *A Handbook of Software and Systems Engineering* (Addison-Wesley) năm 2003 của họ.

A lot of work has been done to look at the different forms of coupling in the context of structured programming, which was largely considering modular (non-distributed, monolithic) software. Many of these different models for assessing coupling overlap or clash, and in any case they speak primarily about things at the code level, rather than considering service-based interactions. As microservices are a style of modular architecture (albeit with the added complexity of distributed systems), we can use a lot of these original concepts and apply them in the context of our microservice-based systems.

Rất nhiều công sức đã được bỏ ra để xem xét các hình thức coupling khác nhau trong ngữ cảnh lập trình có cấu trúc, vốn chủ yếu cân nhắc phần mềm dạng module (không phân tán, nguyên khối). Nhiều mô hình khác nhau để đánh giá coupling chồng chéo hoặc xung đột với nhau, và dù sao thì chúng cũng chủ yếu nói về những thứ ở cấp độ mã, hơn là cân nhắc các tương tác dựa trên service. Vì microservice là một phong cách kiến trúc module (dù có thêm độ phức tạp của hệ thống phân tán), ta có thể dùng nhiều khái niệm gốc này và áp dụng chúng trong ngữ cảnh các hệ thống dựa trên microservice của mình.

Prior Art in Structured Programming — Thành tựu đi trước trong Lập trình có cấu trúc

Much of our work in computing involves building on the work that came before. It is sometimes impossible to recognize everything that came before, but with this second edition I have aimed to highlight prior art where I can—partly to give credit where credit is due, partly as a way of ensuring that I lay down some breadcrumbs for those readers who want to explore certain topics in more detail, but also to show that many of these ideas are tried and tested.

Phần lớn công việc của chúng ta trong lĩnh vực điện toán liên quan đến việc xây dựng dựa trên những thành tựu đã có trước đó. Đôi khi không thể nào nhận diện được hết mọi thứ đã đi trước, nhưng với ấn bản thứ hai này tôi đã cố gắng làm nổi bật những thành tựu đi trước ở những nơi tôi có thể—một phần để ghi công cho những người xứng đáng, một phần như một cách để đảm bảo tôi rải lại vài mẩu bánh mì dẫn đường cho những độc giả muốn khám phá một số chủ đề chi tiết hơn, nhưng cũng để cho thấy rằng nhiều ý tưởng trong số này đã được thử nghiệm và kiểm chứng.

When it comes to building on the work that came before, there are few subject areas in this book that have quite as much prior art as structured programming. I've already mentioned Larry Constantine; his book with Edward Yourdon, *Structured Design*, 6 is considered one of the most important texts in this area. Meilir Page-Jones's *The Practical Guide to Structured Systems Design* 7 is also useful. Unfortunately, one thing these books have in common is how hard they can be to get hold of, as they are out of print and aren't available in ebook format. Yet another reason to support your local library!

Khi nói đến việc xây dựng dựa trên thành tựu đi trước, có rất ít lĩnh vực chủ đề trong cuốn sách này có nhiều thành tựu đi trước như lập trình có cấu trúc. Tôi đã nhắc đến Larry Constantine; cuốn sách của ông cùng Edward Yourdon, *Structured Design*, 6 được coi là một trong những tài liệu quan trọng nhất trong lĩnh vực này. Cuốn *The Practical Guide to Structured Systems Design* 7 của Meilir Page-Jones cũng hữu ích. Đáng tiếc, một điểm chung của những cuốn sách này là chúng khó tìm đến mức nào, vì đã ngừng in và không có bản ebook. Lại thêm một lý do nữa để ủng hộ thư viện địa phương của bạn!

Not all the ideas map cleanly, so I have done my best to synthesize a working model for the different types of coupling for microservices. Where these ideas map cleanly to previous definitions, I've stuck with those terms. In other places I have had to come up with new terms or blend in ideas from elsewhere. So please consider what follows to be built on top of a lot of prior art in this space, which I am attempting to give more meaning in the context of microservices.

Không phải mọi ý tưởng đều ánh xạ một cách gọn gàng, nên tôi đã cố hết sức tổng hợp một mô hình khả dụng cho các loại coupling khác nhau dành cho microservice. Ở những chỗ mà các ý tưởng này ánh xạ gọn gàng vào các định nghĩa trước đây, tôi giữ nguyên những thuật ngữ đó. Ở những chỗ khác, tôi phải nghĩ ra thuật ngữ mới hoặc pha trộn ý tưởng từ nơi khác vào. Vậy nên xin hãy xem những gì sau đây là được xây dựng dựa trên rất nhiều thành tựu đi trước trong lĩnh vực này, mà tôi đang cố gắng mang lại nhiều ý nghĩa hơn trong ngữ cảnh microservice.

In Figure 2-1 we see a brief overview of the different types of coupling, organized from low (desirable) to high (undesirable).

Ở Hình 2-1, ta thấy một bản tổng quan ngắn gọn về các loại coupling khác nhau, được sắp xếp từ thấp (đáng mong muốn) đến cao (không mong muốn).

Next, we'll examine each form of coupling in turn, and look at examples that show how these forms may manifest themselves in our microservice architecture.

Tiếp theo, ta sẽ xem xét lần lượt từng hình thức coupling, và xem các ví dụ cho thấy những hình thức này có thể biểu hiện ra sao trong kiến trúc microservice của ta.

6 Edward Yourdon and Larry L. Constantine, *Structured Design* (New York: Yourdon Press, 1976).

6 Edward Yourdon and Larry L. Constantine, *Structured Design* (New York: Yourdon Press, 1976).

7 Meilir Page-Jones, *The Practical Guide to Structured Systems Design* (New York: Yourdon Press Computing, 1980).

7 Meilir Page-Jones, *The Practical Guide to Structured Systems Design* (New York: Yourdon Press Computing, 1980).

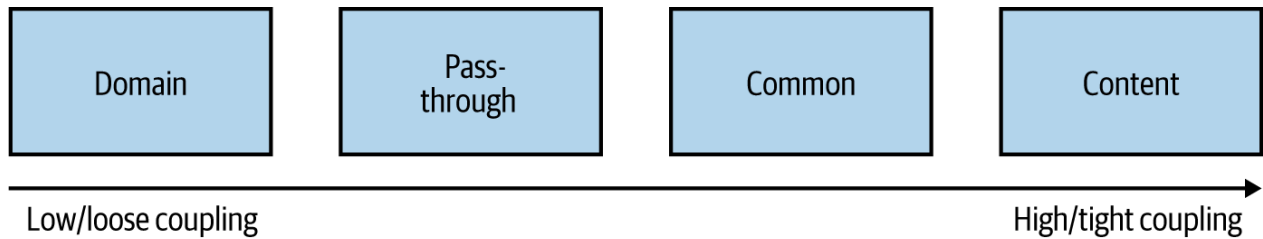


Figure 2-1. The different types of coupling, from loose (low) to tight (high)

Hình 2-1. Các loại coupling khác nhau, từ lỏng (thấp) đến chặt (cao)

Domain Coupling — Domain Coupling

Domain coupling describes a situation in which one microservice needs to interact with another microservice, because the first microservice needs to make use of the functionality that the other microservice provides. 8

Domain coupling mô tả tình huống trong đó một microservice cần tương tác với một microservice khác, bởi microservice thứ nhất cần sử dụng chức năng mà microservice kia cung cấp. 8

In Figure 2-2 , we see part of how orders for CDs are managed inside MusicCorp. In this example, Order Processor calls the Warehouse microservice to reserve stock, and the Payment microservice to take payment. The Order Processor is therefore dependent on, and coupled to, the Warehouse and Payment microservices for this operation. We see no such coupling between Warehouse and Payment , though, as they don't interact.

Ở Hình 2-2, ta thấy một phần của cách quản lý các đơn hàng đĩa CD bên trong MusicCorp. Trong ví dụ này, Order Processor gọi microservice Warehouse để giữ chỗ hàng tồn kho, và gọi microservice Payment để nhận thanh toán. Do đó Order Processor phụ thuộc vào, và bị coupling với, các microservice Warehouse và Payment cho thao tác này. Tuy vậy, ta không thấy có coupling tương tự giữa Warehouse và Payment , bởi chúng không tương tác với nhau.

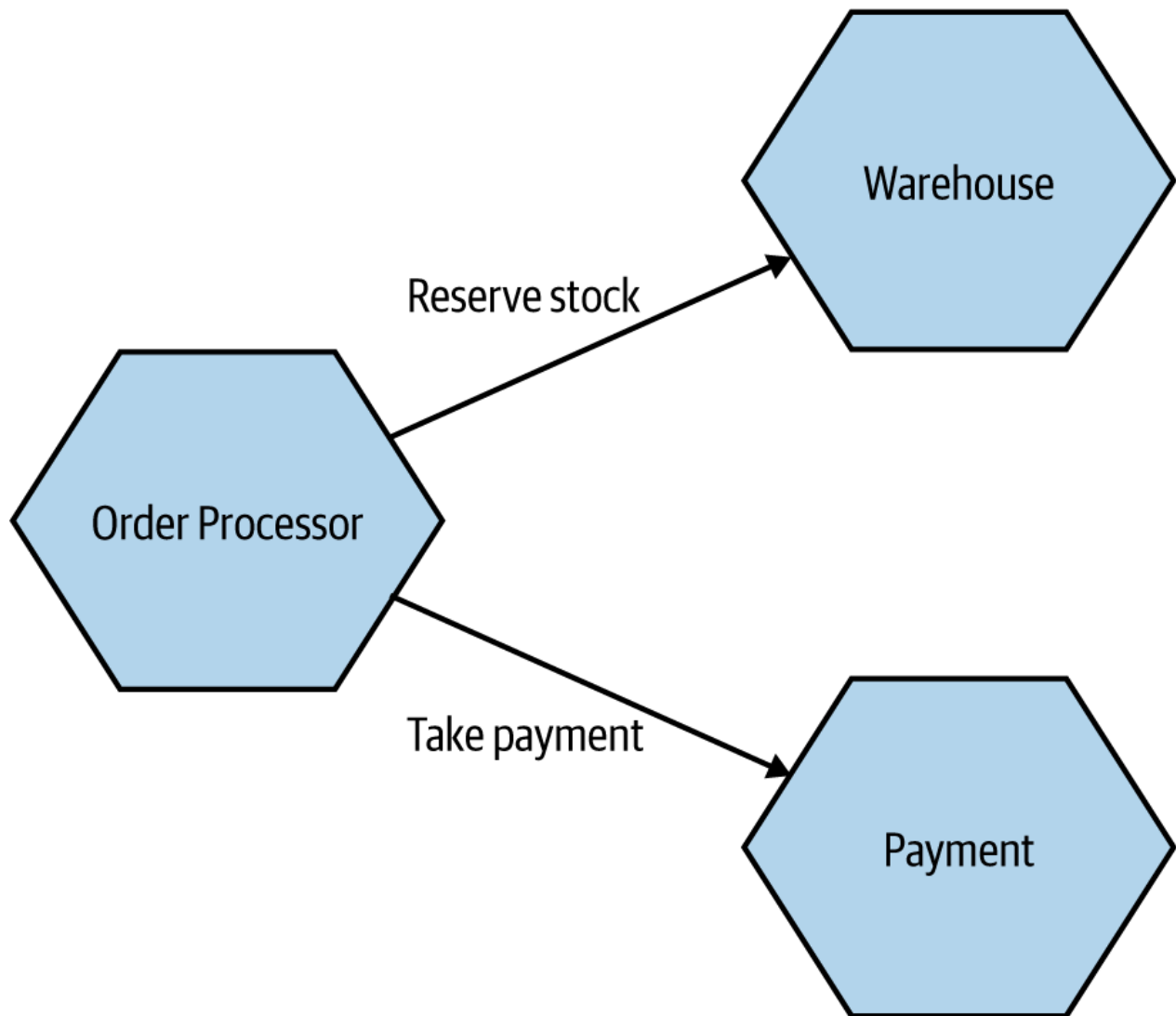


Figure 2-2. An example of domain coupling, where Order Processor needs to make use of the functionality provided by other microservices

Hình 2-2. Một ví dụ về domain coupling, nơi Order Processor cần sử dụng chức năng do các microservice khác cung cấp

8 This concept is similar to the domain application protocol, which defines the rules by which components interact in a **REST**-based system.

8 Khái niệm này tương tự domain application protocol, vốn định nghĩa các quy tắc mà theo đó các thành phần tương tác trong một hệ thống dựa trên REST.

In a microservice architecture, this type of interaction is largely unavoidable. A microservice-based system relies on multiple microservices collaborating in order for it to do its work. We still want to keep this to a minimum, though; whenever you see a single microservice depending on multiple downstream services in this way, it can be a cause for concern—it might imply a microservice that is doing too much.

Trong một kiến trúc microservice, kiểu tương tác này phần lớn là không thể tránh khỏi. Một hệ thống dựa trên microservice dựa vào nhiều microservice cộng tác với nhau để hoàn thành công việc của nó. Tuy nhiên, ta vẫn muốn giữ điều này ở mức tối thiểu; mỗi khi bạn

thấy một microservice đơn lẻ phụ thuộc vào nhiều service phía dưới (downstream) theo cách này, nó có thể là một nguyên nhân đáng lo ngại—nó có thể ngụ ý một microservice đang làm quá nhiều việc.

As a general rule, domain coupling is considered to be a loose form of coupling, although even here we can hit problems. A microservice that needs to talk to lots of downstream microservices might point to a situation in which too much logic has been centralized. Domain coupling can also become problematic as more complex sets of data are sent between services—this can often point to the more problematic forms of coupling we'll explore shortly.

Theo quy tắc chung, domain coupling được xem là một hình thức coupling lỏng, mặc dù ngay cả ở đây ta cũng có thể gặp vấn đề. Một microservice cần nói chuyện với rất nhiều microservice phía dưới có thể chỉ ra một tình huống trong đó quá nhiều logic đã bị tập trung hóa. Domain coupling cũng có thể trở nên rắc rối khi những tập dữ liệu phức tạp hơn được gửi giữa các service—điều này thường có thể chỉ ra những hình thức coupling rắc rối hơn mà ta sẽ khám phá ngay sau đây.

Just remember the importance of information hiding. Share only what you absolutely have to, and send only the absolute minimum amount of data that you need.

Hãy nhớ tầm quan trọng của che giấu thông tin. Chỉ chia sẻ những gì bạn thực sự bắt buộc phải chia sẻ, và chỉ gửi đi lượng dữ liệu tối thiểu tuyệt đối mà bạn cần.

A Brief Note on Temporal Coupling — Một ghi chú ngắn về Temporal Coupling

Another form of coupling you may have heard of is temporal coupling . From a code-centric view of coupling, temporal coupling refers to a situation in which concepts are bundled together purely because they happen at the same time. Temporal coupling has a subtly different meaning in the context of a **distributed system**, where it refers to a situation in which one microservice needs another microservice to do something at the same time for the operation to complete.

Một hình thức coupling khác mà bạn có thể đã nghe đến là temporal coupling . Từ góc nhìn coupling lấy mã làm trung tâm, temporal coupling chỉ tình huống trong đó các khái niệm bị gói chung lại với nhau thuần túy vì chúng xảy ra cùng một thời điểm. Temporal coupling có một ý nghĩa khác đi đôi chút trong ngữ cảnh hệ thống phân tán, nơi nó chỉ tình huống trong đó một microservice cần một microservice khác làm việc gì đó vào cùng một thời điểm để thao tác hoàn tất.

Both microservices need to be up and available to communicate with each other at the same time for the operation to complete. So in Figure 2-3 , where MusicCorp's Order Processor is making a **synchronous** HTTP call to the Warehouse service, Warehouse needs to be up and available at the same time the call is made.

Cả hai microservice đều cần phải hoạt động và sẵn sàng để giao tiếp với nhau vào cùng một thời điểm thì thao tác mới hoàn tất. Vậy nên ở Hình 2-3, nơi Order Processor của MusicCorp đang thực hiện một lời gọi HTTP đồng bộ tới service Warehouse , Warehouse cần phải hoạt động và sẵn sàng vào đúng lúc lời gọi được thực hiện.

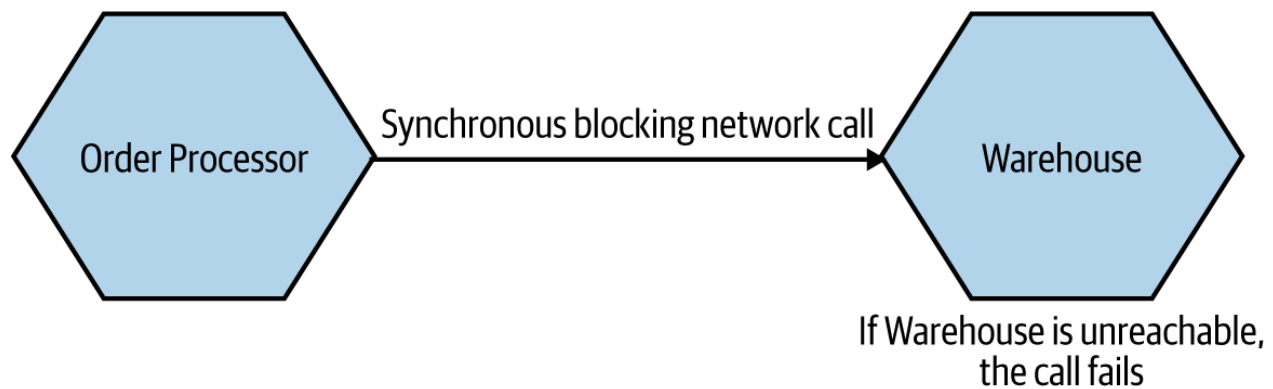


Figure 2-3. An example of temporal coupling, in which Order Processor makes a synchronous HTTP call to the Warehouse microservice

Hình 2-3. Một ví dụ về temporal coupling, trong đó Order Processor thực hiện một lời gọi HTTP đồng bộ tới microservice Warehouse

If for some reason Warehouse can't be reached by the Order Processor, then the operation fails, as we can't reserve the CDs to be sent out. Order Processor will also have to block and wait for a response from Warehouse, potentially causing issues in terms of resource contention.

Nếu vì lý do nào đó Order Processor không thể tiếp cận được Warehouse, thì thao tác thất bại, vì ta không thể giữ chỗ những đĩa CD để gửi đi. Order Processor cũng sẽ phải chặn (block) và chờ phản hồi từ Warehouse, có khả năng gây ra vấn đề về tranh chấp tài nguyên (resource contention).

Temporal coupling isn't always bad; it's just something to be aware of. As you have more microservices, with more complex interactions between them, the challenges of temporal coupling can increase to such a point that it becomes more difficult to scale your system and keep it working. One of the ways to avoid temporal coupling is to use some form of **asynchronous** communication, such as a **message broker**.

Temporal coupling không phải lúc nào cũng xấu; nó chỉ là điều cần lưu tâm. Khi bạn có nhiều microservice hơn, với những tương tác phức tạp hơn giữa chúng, các thách thức của temporal coupling có thể gia tăng đến mức việc mở rộng hệ thống và giữ cho nó hoạt động trở nên khó khăn hơn. Một trong những cách để tránh temporal coupling là sử dụng một dạng giao tiếp bất đồng bộ nào đó, chẳng hạn như message broker.

Pass-Through Coupling — Pass-Through Coupling

"Pass-through coupling" 9 describes a situation in which one microservice passes data to another microservice purely because the data is needed by some other microservice further downstream. In many ways it's one of the most problematic forms of implementation coupling, as it implies not only that the caller knows not just that the microservice it is invoking calls yet another microservice, but also that it potentially needs to know how that one-step-removed microservice works.

"Pass-through coupling" 9 mô tả tình huống trong đó một microservice chuyển dữ liệu cho một microservice khác thuần túy vì dữ liệu đó cần thiết cho một microservice nào đó nằm xa hơn về phía dưới. Theo nhiều cách, nó là một trong những hình thức implementation coupling rắc rối nhất, vì nó ngụ ý không chỉ rằng bên gọi biết rằng microservice mà nó gọi lại gọi tiếp một microservice khác, mà còn rằng nó có khả năng cần biết microservice cách-một-bước đó hoạt động ra sao.

As an example of pass-through coupling, let's look more closely now at part of how MusicCorp's order processing works. In Figure 2-4, we have an Order Processor, which is sending a request to Warehouse to prepare an order for dispatch. As part of the request payload, we send along a Shipping Manifest. This Shipping Manifest contains not only the address of the customer but also the shipping type. The Warehouse just passes this manifest on to the downstream Shipping microservice.

Như một ví dụ về pass-through coupling, giờ hãy xem xét kỹ hơn một phần trong cách xử lý đơn hàng của MusicCorp hoạt động. Ở Hình 2-4, ta có một Order Processor, đang gửi một yêu cầu tới Warehouse để chuẩn bị một đơn hàng cho việc gửi đi. Là một phần của payload yêu cầu, ta gửi kèm một Shipping Manifest. Shipping Manifest này không chỉ chứa địa chỉ của khách hàng mà còn cả loại hình vận chuyển. Warehouse chỉ đơn giản chuyển tiếp manifest này cho microservice Shipping ở phía dưới.

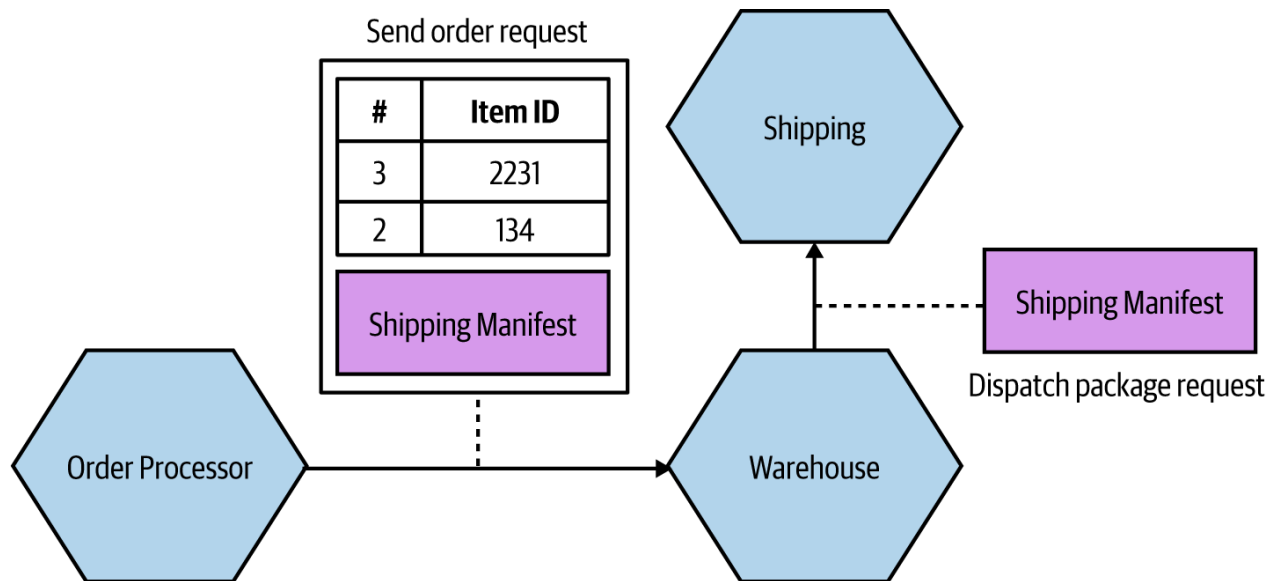


Figure 2-4. Pass-through coupling, in which data is passed to a microservice purely because another downstream service needs it

Hình 2-4. Pass-through coupling, trong đó dữ liệu được chuyển cho một microservice thuần túy vì một service khác ở phía dưới cần nó

9 Pass-through coupling is my name for what was originally described as “tramp coupling” by Meilir Page-Jones in The Practical Guide to Structured Systems Design. I chose to use a different term here due to the fact that I found the original term to be somewhat problematic and not terribly meaningful to a wider audience.

9 Pass-through coupling là tên gọi tôi đặt cho thứ ban đầu được Meilir Page-Jones mô tả là “tramp coupling” trong The Practical Guide to Structured Systems Design. Tôi chọn dùng một thuật ngữ khác ở đây do tôi thấy thuật ngữ gốc có phần rắc rối và không thật sự có ý nghĩa với một lượng độc giả rộng hơn.

The major issue with pass-through coupling is that a change to the required data downstream can cause a more significant upstream change. In our example, if Shipping now needs the format or content of the data to be changed, then both Warehouse and Order Processor would likely need to change.

Vấn đề lớn của pass-through coupling là một thay đổi đối với dữ liệu cần thiết ở phía dưới có thể gây ra một thay đổi đáng kể hơn ở phía trên. Trong ví dụ của ta, nếu giờ Shipping

cần thay đổi định dạng hoặc nội dung của dữ liệu, thì cả Warehouse lẫn Order Processor đều có khả năng phải thay đổi theo.

There are a few ways this can be fixed. The first is to consider whether it makes sense for the calling microservice to just bypass the intermediary. In our example, this might mean Order Processor speaks directly to Shipping, as in Figure 2-5. However, this causes some other headaches. Our Order Processor is increasing its domain coupling, as Shipping is yet another microservice it needs to know about—if that was the only issue, this might still be fine, as domain coupling is of course a looser form of coupling. This solution gets more complex here, though, as stock has to be reserved with Warehouse before we dispatch the package using Shipping, and after the shipping has been done we need to update the stock accordingly. This pushes more complexity and logic into Order Processor that was previously hidden inside Warehouse.

Có một vài cách để khắc phục điều này. Cách thứ nhất là cân nhắc xem có hợp lý không khi microservice gọi cứ thế bỏ qua bên trung gian. Trong ví dụ của ta, điều này có thể nghĩa là Order Processor nói chuyện trực tiếp với Shipping, như ở Hình 2-5. Tuy nhiên, điều này gây ra một số rắc rối khác. Order Processor của ta đang tăng domain coupling của nó, vì Shipping là thêm một microservice nữa mà nó cần biết tới—nếu đó là vấn đề duy nhất, điều này có lẽ vẫn ổn, vì domain coupling tất nhiên là một hình thức coupling lỏng hơn. Tuy nhiên, giải pháp này phức tạp hơn ở chỗ hàng phải được giữ chỗ với Warehouse trước khi ta gửi gói hàng đi bằng Shipping, và sau khi việc vận chuyển đã xong ta cần cập nhật hàng tồn kho cho phù hợp. Điều này đẩy thêm độ phức tạp và logic vào Order Processor mà trước đây vốn được giấu kín bên trong Warehouse.

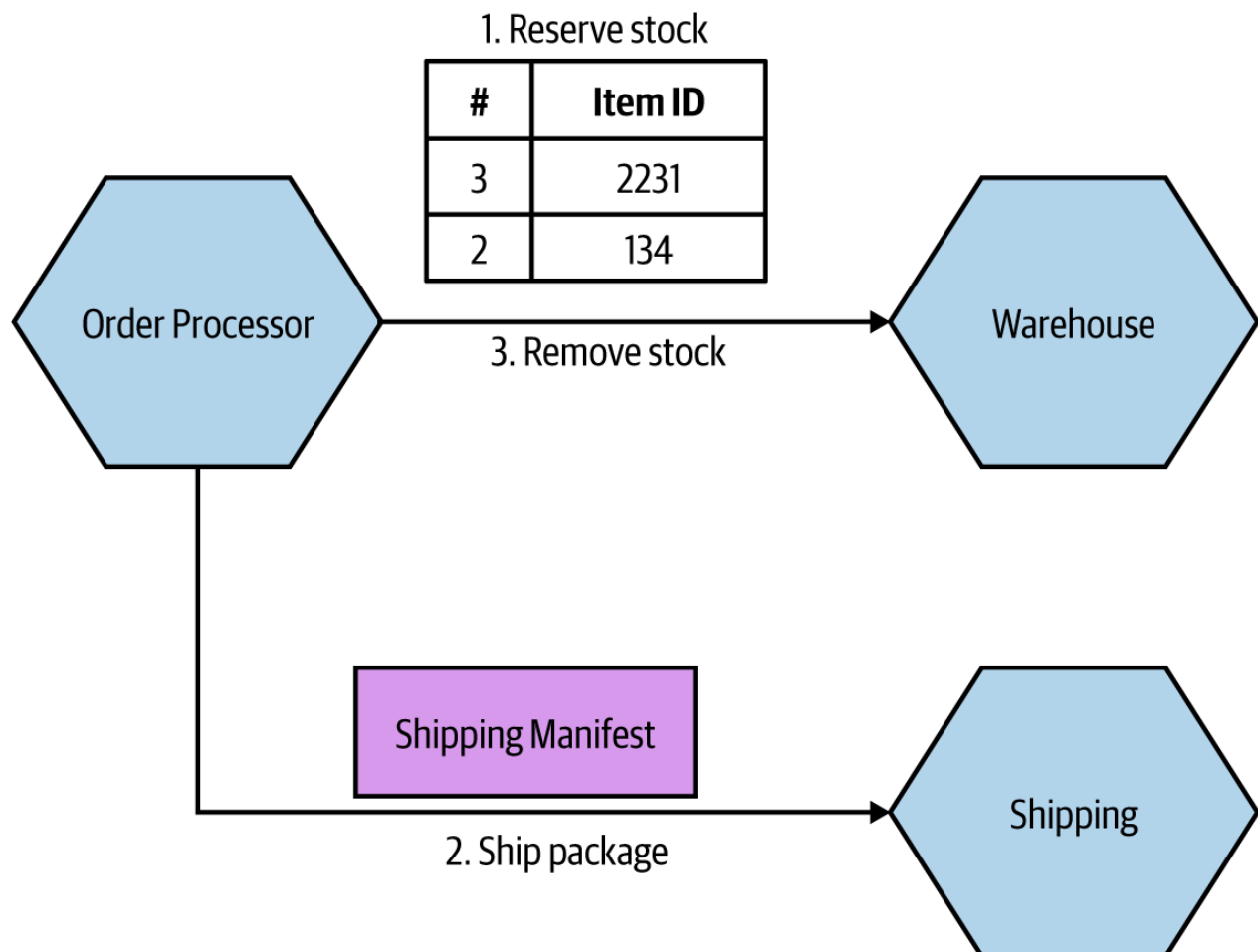


Figure 2-5. One way to work around pass-through coupling involves communicating directly with the downstream service

Hình 2-5. Một cách để khắc phục pass-through coupling liên quan đến việc giao tiếp trực tiếp với service ở phía dưới

For this specific example, I might consider a simpler (albeit more nuanced) change— namely, to totally hide the requirement for a Shipping Manifest from Order Processor . The idea of delegating the work of both managing stock and arranging for dispatch of the package to our Warehouse service makes sense, but we don't like the fact that we have leaked some lower-level implementation— namely, the fact that the Shipping microservice wants a Shipping Manifest . One way to hide this detail would be to have Warehouse take in the required information as part of its contract, and then have it construct the Shipping Manifest locally, as we see in Figure 2-6 . This means that if the Shipping service changes its service contract, this change will be invisible from the viewpoint of Order Processor , as long as Warehouse collects the required data.

Với ví dụ cụ thể này, tôi có thể cân nhắc một thay đổi đơn giản hơn (dù tinh tế hơn)—cụ thể là, giấu hoàn toàn yêu cầu về một Shipping Manifest khỏi Order Processor . Ý tưởng giao việc quản lý hàng tồn kho lẫn việc sắp xếp gửi gói hàng cho service Warehouse của ta là hợp lý, nhưng ta không thích cái sự thật rằng ta đã để rò rỉ một phần triển khai ở cấp thấp hơn—cụ thể là, sự thật rằng microservice Shipping muốn có một Shipping Manifest . Một cách để giấu chi tiết này là để Warehouse nhận vào thông tin cần thiết như một phần hợp đồng của nó, rồi để nó tự dựng Shipping Manifest tại chỗ, như ta thấy ở Hình 2-6. Điều này nghĩa là nếu service Shipping thay đổi hợp đồng của nó, thay đổi đó sẽ vô hình từ góc nhìn của Order Processor , miễn là Warehouse thu thập được dữ liệu cần thiết.

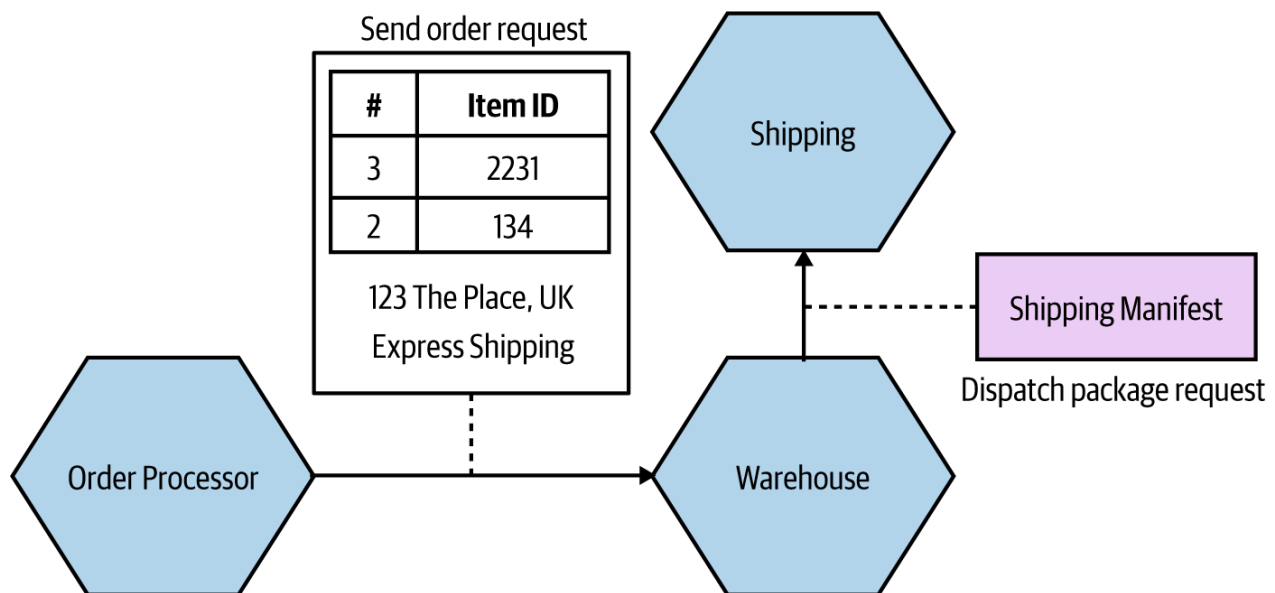


Figure 2-6. Hiding the need for a Shipping Manifest from the Order Processor

Hình 2-6. Giấu nhu cầu về một Shipping Manifest khỏi Order Processor

While this will help protect the Warehouse microservice from some changes to Shipping , there are some things that would still require all parties to change. Let's consider the idea that we want to start shipping internationally. As part of this, the Shipping service needs a Customs Declaration to be included in the Shipping Manifest . If this is an optional parameter, then we could deploy a new version of the Shipping microservice without issue. If this is a required parameter, however, then

Warehouse would need to create one. It might be able to do this with existing information that it has (or is given), or it might require that additional information be passed to it by the Order Processor .

Dù điều này sẽ giúp bảo vệ microservice Warehouse khỏi một số thay đổi đối với Shipping , vẫn có những thứ đòi hỏi tất cả các bên phải thay đổi. Hãy xét ý tưởng rằng ta muốn bắt đầu vận chuyển quốc tế. Là một phần của việc này, service Shipping cần một Customs Declaration được bao gồm trong Shipping Manifest . Nếu đây là một tham số tùy chọn, thì ta có thể triển khai một phiên bản mới của microservice Shipping mà không gặp vấn đề gì. Tuy nhiên, nếu đây là một tham số bắt buộc, thì Warehouse sẽ cần tạo ra một cái. Nó có thể làm được điều này với thông tin sẵn có (hoặc được cung cấp), hoặc nó có thể đòi hỏi Order Processor phải chuyển thêm thông tin cho nó.

Although in this case we haven't eliminated the need for changes to be made across all three microservices, we have been given much more power over when and how these changes could be made. If we had the tight (pass-through) coupling of the initial example, adding this new Customs Declaration might require a lockstep rollout of all three microservices. At least by hiding this detail we could much more easily phase **deployment**.

Mặc dù trong trường hợp này ta chưa loại bỏ được nhu cầu thực hiện thay đổi trên cả ba microservice, ta đã được trao nhiều quyền chủ động hơn về thời điểm và cách thức thực hiện những thay đổi này. Nếu ta có kiểu coupling chặt (pass-through) như trong ví dụ ban đầu, việc thêm Customs Declaration mới này có thể đòi hỏi một đợt triển khai đồng loạt (lockstep) cả ba microservice. Ít nhất bằng cách giấu chi tiết này đi, ta có thể phân kỳ việc triển khai theo từng giai đoạn dễ dàng hơn nhiều.

One final approach that could help reduce the pass-through coupling would be for the Order Processor to still send the Shipping Manifest to the Shipping microservice via the Warehouse , but to have the Warehouse be totally unaware of the structure of the Shipping Manifest itself. The Order Processor sends the manifest as part of the order request, but the Warehouse makes no attempt to look at or process the field—it just treats it like a blob of data and doesn't care about the contents. Instead, it just sends it along. A change in the format of the the Shipping Manifest would still require a change to both the Order Processor and the Shipping microservice, but as the Warehouse doesn't care about what is actually in the manifest, it doesn't need to change.

Một cách tiếp cận cuối cùng có thể giúp giảm pass-through coupling là để Order Processor vẫn gửi Shipping Manifest tới microservice Shipping thông qua Warehouse , nhưng để Warehouse hoàn toàn không biết gì về cấu trúc của chính Shipping Manifest . Order Processor gửi manifest như một phần của yêu cầu đơn hàng, nhưng Warehouse không hề cố nhìn vào hay xử lý trường dữ liệu đó—nó chỉ coi đó như một khối dữ liệu (blob) và không quan tâm đến nội dung. Thay vào đó, nó chỉ chuyển tiếp đi. Một thay đổi về định dạng của Shipping Manifest vẫn sẽ đòi hỏi thay đổi cả Order Processor lẫn microservice Shipping , nhưng vì Warehouse không quan tâm đến những gì thực sự nằm trong manifest, nên nó không cần thay đổi.

Common Coupling — Common Coupling

Common coupling occurs when two or more microservices make use of a common set of data. A simple and common example of this form of coupling would be multiple microservices making use of the same **shared database**, but it could also manifest itself in the use of shared memory or a shared filesystem.

Common coupling xảy ra khi hai hoặc nhiều microservice sử dụng một tập dữ liệu chung.

Một ví dụ đơn giản và phổ biến về hình thức coupling này là nhiều microservice cùng sử dụng một cơ sở dữ liệu dùng chung, nhưng nó cũng có thể biểu hiện ở việc sử dụng bộ nhớ dùng chung hay một hệ thống tệp dùng chung.

The main issue with common coupling is that changes to the structure of the data can impact multiple microservices at once. Consider the example of some of MusicCorp's services in Figure 2-7 . As we discussed earlier, MusicCorp operates around the world, so it needs various bits of information about the countries in which it operates. Here, multiple services are all reading static reference data from a shared **database**. If the schema of this database changed in a backward-incompatible way, it would require changes to each consumer of the database. In practice, shared data like this tends to be very difficult to change as a result.

Vấn đề chính của common coupling là những thay đổi đối với cấu trúc của dữ liệu có thể ảnh hưởng đến nhiều microservice cùng lúc. Hãy xét ví dụ về một số service của MusicCorp ở Hình 2-7. Như đã bàn ở trên, MusicCorp hoạt động trên toàn thế giới, nên nó cần nhiều mẫu thông tin khác nhau về các quốc gia nơi nó hoạt động. Ở đây, nhiều service đều đang đọc dữ liệu tham chiếu tĩnh từ một cơ sở dữ liệu dùng chung. Nếu schema của cơ sở dữ liệu này thay đổi theo cách không tương thích ngược, nó sẽ đòi hỏi thay đổi mỗi bên tiêu thụ của cơ sở dữ liệu. Trong thực tế, dữ liệu dùng chung kiểu này thường rất khó thay đổi.

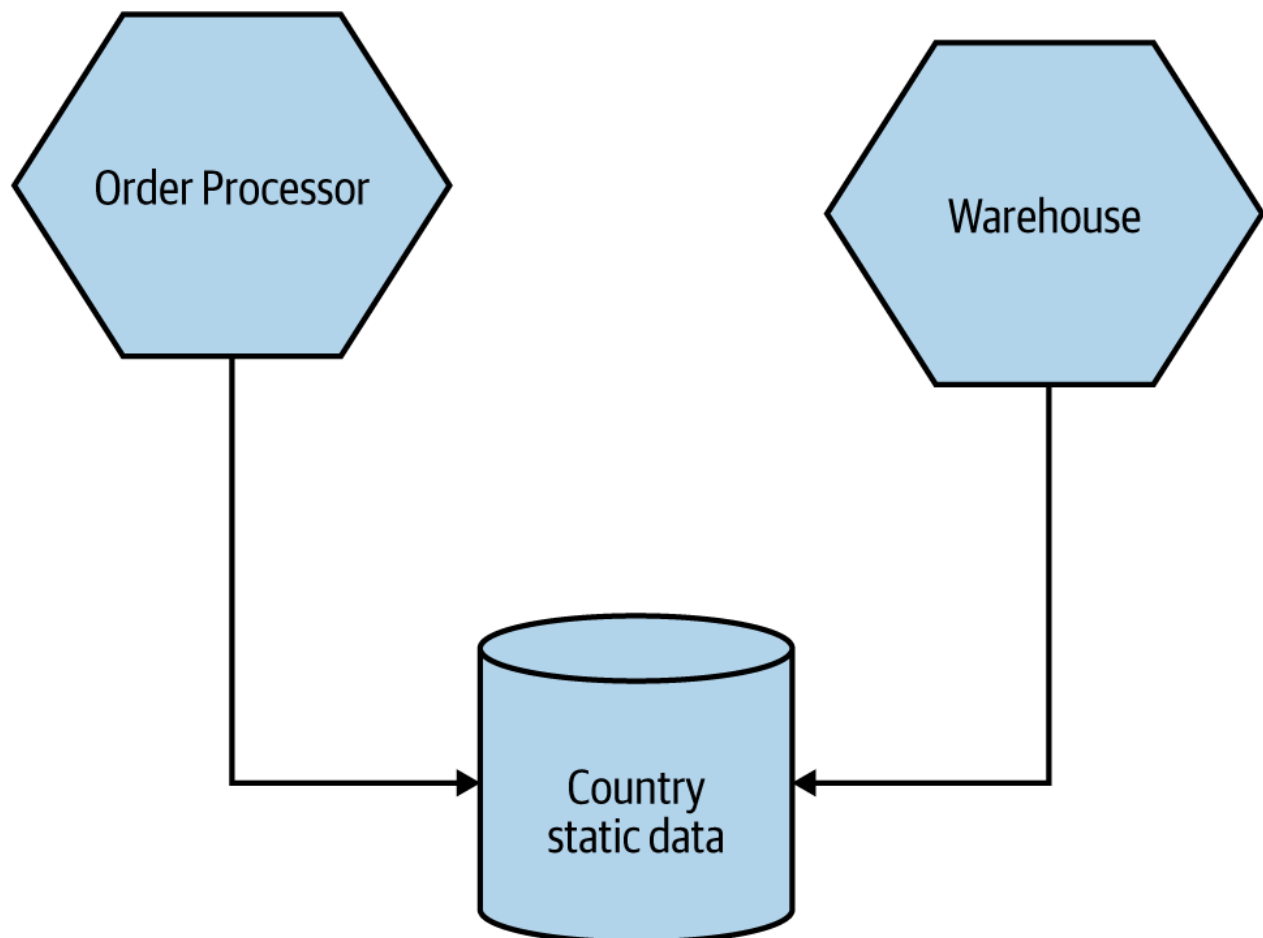


Figure 2-7. Multiple services accessing shared static reference data related to countries from the same database

Hình 2-7. Nhiều service truy cập dữ liệu tham chiếu tĩnh dùng chung liên quan đến các

quốc gia từ cùng một cơ sở dữ liệu

The example in Figure 2-7 is, relatively speaking, fairly benign. This is because by its very nature static reference data doesn't tend to change often, and also because this data is read-only—as a result I tend to be relaxed about sharing static reference data in this way. Common coupling becomes more problematic, though, if the structure of the common data changes more frequently, or if multiple microservices are reading and writing to the same data.

Ví dụ ở Hình 2-7, tương đối mà nói, khá lành tính. Đó là vì về bản chất, dữ liệu tham chiếu tĩnh không hay thay đổi thường xuyên, và cũng vì dữ liệu này chỉ-đọc—kết quả là tôi thường khá thoải mái về việc chia sẻ dữ liệu tham chiếu tĩnh theo cách này. Tuy nhiên, common coupling trở nên rắc rối hơn nếu cấu trúc của dữ liệu chung thay đổi thường xuyên hơn, hoặc nếu nhiều microservice cùng đọc và ghi vào cùng một dữ liệu.

Figure 2-8 shows us a situation in which the Order Processor and Warehouse service are both reading and writing from a shared Order table to help manage the process of dispatching CDs to MusicCorp's customers. Both microservices are updating the Status column. The Order Processor can set the PLACED, PAID, and COMPLETED statuses, whereas the Warehouse will apply PICKING or SHIPPED statuses.

Hình 2-8 cho ta thấy một tình huống trong đó service Order Processor và Warehouse đều đang đọc và ghi từ một bảng Order dùng chung để giúp quản lý quá trình gửi đĩa CD cho khách hàng của MusicCorp. Cả hai microservice đều đang cập nhật cột Status. Order Processor có thể đặt các trạng thái PLACED, PAID, và COMPLETED, trong khi Warehouse sẽ áp dụng các trạng thái PICKING hoặc SHIPPED.

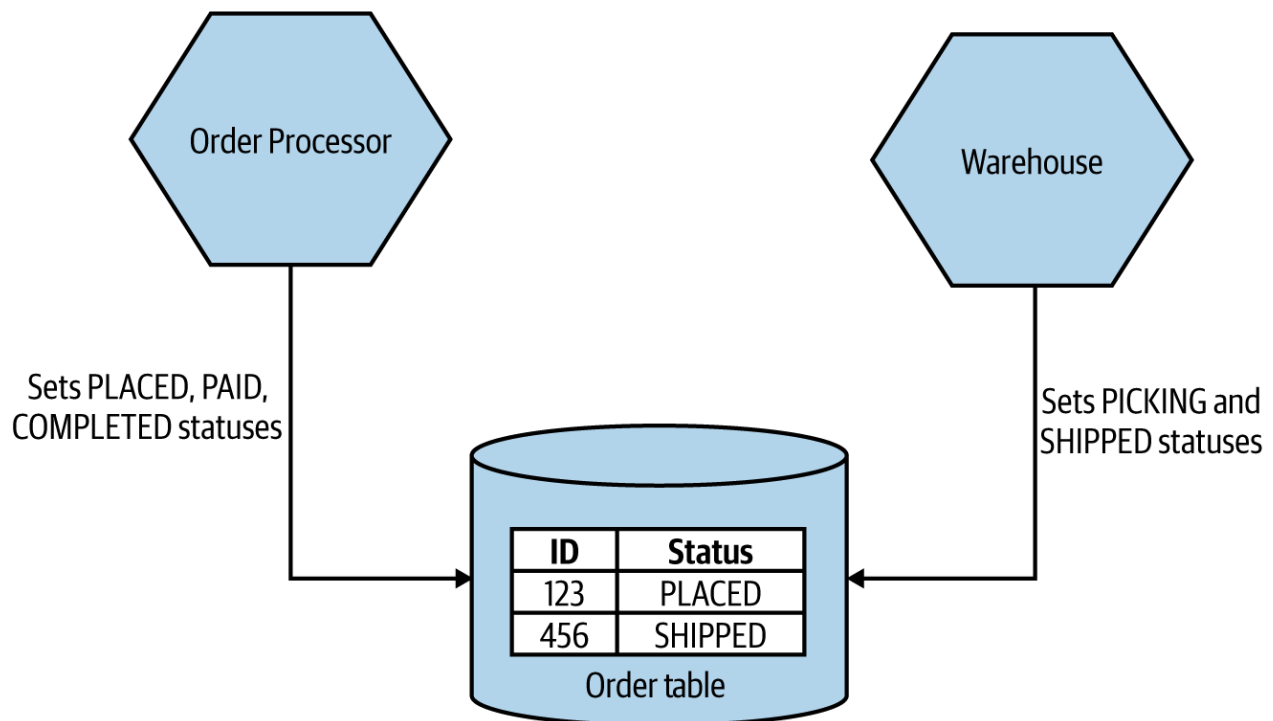


Figure 2-8. An example of common coupling in which both Order Processor and Warehouse are updating the same order record

Hình 2-8. Một ví dụ về common coupling trong đó cả Order Processor lẫn Warehouse đều đang cập nhật cùng một bản ghi đơn hàng

Although you might consider Figure 2-8 to be somewhat contrived, this nonetheless straightforward example of common coupling helps illustrate a core problem. Conceptually, we have both the Order Processor and Warehouse microservices managing different aspects of the life cycle of an order. When making changes in Order Processor, can I be sure that I am not changing the order data in such a way that it breaks Warehouse's view of the world, or vice versa?

Mặc dù bạn có thể cho rằng Hình 2-8 hơi gượng ép, ví dụ đơn giản về common coupling này dù sao cũng giúp minh họa một vấn đề cốt lõi. Về mặt khái niệm, ta có cả hai microservice Order Processor và Warehouse cùng quản lý những khía cạnh khác nhau trong vòng đời của một đơn hàng. Khi thực hiện các thay đổi trong Order Processor, liệu tôi có thể chắc chắn rằng mình không thay đổi dữ liệu đơn hàng theo cách phá vỡ cái nhìn về thế giới của Warehouse, hay ngược lại?

One way to ensure that the state of something is changed in a correct fashion would be to create a finite state machine. A state machine can be used to manage the transition of some entity from one state to another, ensuring invalid state transitions are prohibited. In Figure 2-9, you can see the allowed transitions of state for an order in MusicCorp. An order can go directly from PLACED to PAID, but not straight from PLACED to PICKING (this state machine likely wouldn't be sufficient for the real-world business processes involved in full end-to-end buying and shipping of goods, but I wanted to give a simple example to illustrate the idea).

Một cách để đảm bảo rằng trạng thái của thứ gì đó được thay đổi theo cách đúng đắn là tạo ra một máy trạng thái hữu hạn (finite state machine). Một máy trạng thái có thể được dùng để quản lý sự chuyển đổi của một thực thể nào đó từ trạng thái này sang trạng thái khác, đảm bảo những chuyển đổi trạng thái không hợp lệ bị cấm. Ở Hình 2-9, bạn có thể thấy các chuyển đổi trạng thái được phép của một đơn hàng trong MusicCorp. Một đơn hàng có thể đi thẳng từ PLACED sang PAID, nhưng không thể đi thẳng từ PLACED sang PICKING (máy trạng thái này có lẽ sẽ không đủ cho các quy trình nghiệp vụ thực tế liên quan đến việc mua và vận chuyển hàng hóa đầy đủ từ đầu đến cuối, nhưng tôi muốn đưa ra một ví dụ đơn giản để minh họa ý tưởng).

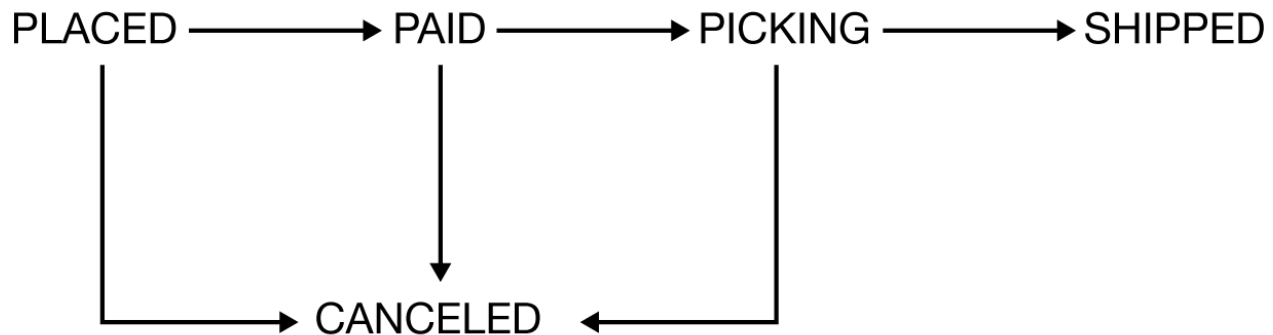


Figure 2-9. An overview of the allowable state transitions for an order in MusicCorp

Hình 2-9. Một bản tổng quan về các chuyển đổi trạng thái được phép của một đơn hàng trong MusicCorp

The problem in this specific example is that both Warehouse and Order Processor share responsibilities for managing this state machine. How do we ensure that they are in agreement as to what transitions are allowed? There are ways to manage processes like this across microservice boundaries; we will return to this topic when we discuss sagas in Chapter 6.

Vấn đề trong ví dụ cụ thể này là cả Warehouse lẫn Order Processor đều chia sẻ trách nhiệm quản lý máy trạng thái này. Làm thế nào để ta đảm bảo rằng chúng nhất trí với nhau về

những chuyển đổi nào được phép? Có những cách để quản lý các quy trình như thế này xuyên qua các ranh giới microservice; ta sẽ quay lại chủ đề này khi bàn về saga ở Chương 6.

A potential solution here would be to ensure that a single microservice manages the order state. In Figure 2-10, either Warehouse or Order Processor can send status update requests to the Order service. Here, the Order microservice is the source of truth for any given order. In this situation, it is really important that we see the requests from Warehouse and Order Processor as just that—requests. In this scenario, it is the job of the Order service to manage the acceptable state transitions associated with an order **aggregate**. As such, if the Order service received a request from Order Processor to move a status straight from PLACED to COMPLETED, it is free to reject that request if that is an invalid change.

Một giải pháp tiềm năng ở đây là đảm bảo một microservice đơn lẻ quản lý trạng thái đơn hàng. Ở Hình 2-10, cả Warehouse hoặc Order Processor đều có thể gửi yêu cầu cập nhật trạng thái tới service Order. Ở đây, microservice Order là nguồn chân lý (source of truth) cho bất kỳ đơn hàng nào. Trong tình huống này, điều thực sự quan trọng là ta phải nhìn nhận các yêu cầu từ Warehouse và Order Processor đúng như bản chất của chúng—các yêu cầu (requests). Trong kịch bản này, nhiệm vụ của service Order là quản lý các chuyển đổi trạng thái được chấp nhận gắn với một aggregate đơn hàng. Như vậy, nếu service Order nhận được một yêu cầu từ Order Processor để chuyển một trạng thái thẳng từ PLACED sang COMPLETED, nó hoàn toàn được phép từ chối yêu cầu đó nếu đó là một thay đổi không hợp lệ.



Make sure you see a request that is sent to a microservice as something that the downstream microservice can reject if it is invalid.

Hãy đảm bảo bạn nhìn nhận một yêu cầu được gửi tới một microservice như một thứ mà microservice ở phía dưới có thể từ chối nếu nó không hợp lệ.

An alternative approach I see in such cases is to implement the Order service as little more than a wrapper around database CRUD operations, where requests just map directly to database updates. This is akin to an object having private fields but public getters and setters—the behavior has leaked from the microservice to upstream consumers (reducing cohesion), and we’re back in the world of managing acceptable state transitions across multiple different services.

Một cách tiếp cận thay thế mà tôi thấy trong những trường hợp như vậy là triển khai service Order chẳng khác gì một lớp bọc (wrapper) mỏng quanh các thao tác CRUD của cơ sở dữ liệu, nơi các yêu cầu chỉ ánh xạ trực tiếp tới các cập nhật cơ sở dữ liệu. Điều này giống như một đối tượng có các trường riêng tư nhưng lại có getter và setter công khai—hành vi đã rò rỉ từ microservice ra các bên tiêu thụ phía trên (giảm cohesion), và ta lại quay về với thế giới quản lý các chuyển đổi trạng thái được chấp nhận xuyên qua nhiều service khác nhau.



If you see a microservice that just looks like a thin wrapper around database CRUD operations, that is a sign that you may have weak cohesion and tighter coupling, as logic that should be in that service

to manage the data is instead spread elsewhere in your system.

Nếu bạn thấy một microservice trông chẳng khác gì một lớp bọc mỏng quanh các thao tác CRUD của cơ sở dữ liệu, đó là dấu hiệu cho thấy bạn có thể đang có cohesion yếu và coupling chặt hơn, vì logic lẽ ra phải nằm trong service đó để quản lý dữ liệu thì lại bị rải rác ở nơi khác trong hệ thống của bạn.

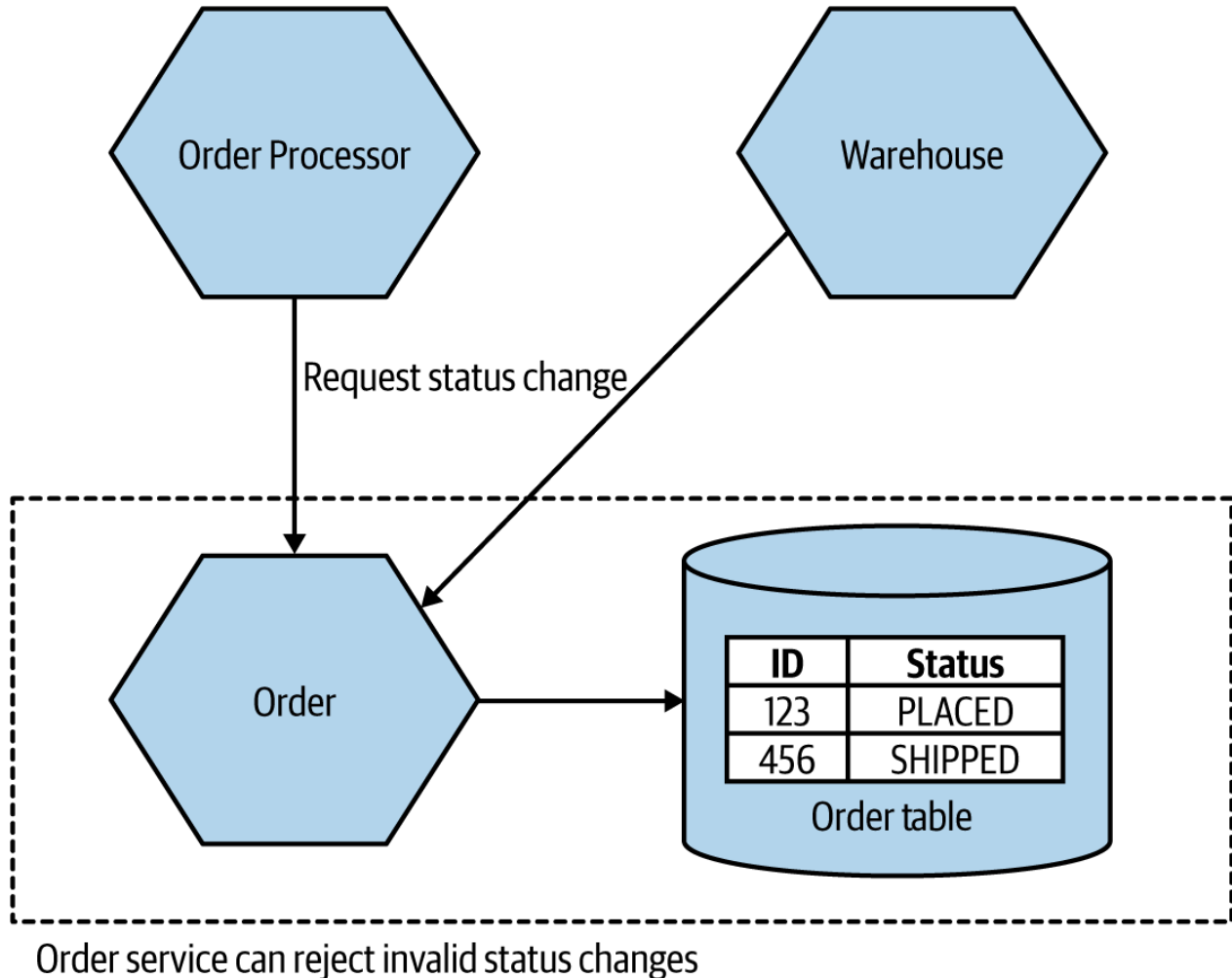


Figure 2-10. Both Order Processor and Warehouse can request that changes be made to an order, but the Order microservice decides which requests are acceptable

Hình 2-10. Cả Order Processor lẫn Warehouse đều có thể yêu cầu thực hiện thay đổi đối với một đơn hàng, nhưng microservice Order mới là bên quyết định những yêu cầu nào được chấp nhận

Sources of common coupling are also potential sources of resource contention. Multiple microservices making use of the same filesystem or database could overload that shared resource, potentially causing significant problems if the shared resource becomes slow or even entirely unavailable. A shared database is especially prone to this problem, as multiple consumers can run arbitrary queries against the database itself, which in turn can have wildly different performance characteristics. I've seen more than one database brought to its knees by an expensive SQL query—I may have even been the culprit once or twice. 10

Các nguồn của common coupling cũng là những nguồn tiềm tàng của tranh chấp tài

nguyên. Nhiều microservice cùng sử dụng một hệ thống tệp hay một cơ sở dữ liệu có thể làm quá tải tài nguyên dùng chung đó, có khả năng gây ra những vấn đề nghiêm trọng nếu tài nguyên dùng chung trở nên chậm hoặc thậm chí hoàn toàn không khả dụng. Một cơ sở dữ liệu dùng chung đặc biệt dễ gặp vấn đề này, vì nhiều bên tiêu thụ có thể chạy các truy vấn tùy ý lên chính cơ sở dữ liệu, mà những truy vấn đó đến lượt nó lại có thể có những đặc tính hiệu năng khác nhau một trời một vực. Tôi đã thấy không chỉ một cơ sở dữ liệu bị quật ngã bởi một câu SQL tốn kém—và có lẽ chính tôi đôi khi cũng là thủ phạm.

10

So common coupling is sometimes OK, but often it's not. Even when it's benign, it means that we are limited in what changes can be made to the shared data, but it often speaks to a lack of cohesion in our code. It can also cause us problems in terms of operational contention. It's for those reasons that we consider common coupling to be one of the least desirable forms of coupling—but it can get worse.

Vậy nên common coupling đôi khi cũng ổn, nhưng thường thì không. Ngay cả khi nó lành tính, nó nghĩa là ta bị giới hạn về những thay đổi có thể thực hiện đối với dữ liệu dùng chung, nhưng nó thường nói lên sự thiếu cohesion trong mã của ta. Nó cũng có thể gây ra vấn đề về tranh chấp ở mức vận hành. Chính vì những lý do đó mà ta xem common coupling là một trong những hình thức coupling ít đáng mong muốn nhất—nhưng nó còn có thể tệ hơn nữa.

Content Coupling — Content Coupling

Content coupling describes a situation in which an upstream service reaches into the internals of a downstream service and changes its internal state. The most common

Content coupling mô tả tình huống trong đó một service phía trên thò tay vào phần ruột bên trong của một service phía dưới và thay đổi trạng thái nội bộ của nó. Biểu hiện phổ biến nhất

10 OK, more than once or twice. A lot more than once or twice... manifestation of this is an external service accessing another microservice's database and changing it directly. The differences between content coupling and common coupling are subtle. In both cases, two or more microservices are reading and writing to the same set of data. With common coupling, you understand that you are making use of a shared, external dependency. You know it's not under your control. With content coupling, the lines of **ownership** become less clear, and it becomes more difficult for developers to change a system.

10 Được rồi, không chỉ đôi khi. Nhiều hơn hẳn đôi khi... của điều này là một service bên ngoài truy cập cơ sở dữ liệu của một microservice khác và thay đổi nó trực tiếp. Khác biệt giữa content coupling và common coupling rất tinh tế. Trong cả hai trường hợp, hai hoặc nhiều microservice đang đọc và ghi vào cùng một tập dữ liệu. Với common coupling, bạn hiểu rằng bạn đang sử dụng một phụ thuộc bên ngoài, dùng chung. Bạn biết nó không nằm dưới sự kiểm soát của bạn. Với content coupling, lằn ranh sở hữu trở nên kém rõ ràng hơn, và việc lập trình viên thay đổi một hệ thống trở nên khó khăn hơn.

Let's revisit our earlier example from MusicCorp. In Figure 2-11, we have an Order service that is supposed to manage the allowable state changes to orders in our system. The Order Processor is sending requests to the Order service, delegating not just the exact change in state that will be made but also responsibility for deciding what state transitions are allowable. On the other hand, the Warehouse service is directly updating the table in which order data is stored, bypassing any

functionality in the Order service that might check for allowable changes. We have to hope that the Warehouse service has a consistent set of logic to ensure that only valid changes are made. At best, this represents a duplication of logic. In the worst case, the checking around allowable changes in Warehouse is different from that in the Order service, and as a result we could end up with orders in very odd, confusing states.

Hãy xem lại ví dụ trước đó từ MusicCorp. Ở Hình 2-11, ta có một service Order lẽ ra phải quản lý những thay đổi trạng thái được phép đối với các đơn hàng trong hệ thống của ta. Order Processor đang gửi các yêu cầu tới service Order, giao phó không chỉ thay đổi trạng thái cụ thể sẽ được thực hiện mà còn cả trách nhiệm quyết định những chuyển đổi trạng thái nào là được phép. Mặt khác, service Warehouse lại đang cập nhật trực tiếp bảng chứa dữ liệu đơn hàng, bỏ qua mọi chức năng trong service Order vốn có thể kiểm tra những thay đổi được phép. Ta phải hy vọng rằng service Warehouse có một tập logic nhất quán để đảm bảo chỉ những thay đổi hợp lệ mới được thực hiện. Tốt nhất thì đây là một sự trùng lặp logic. Trong trường hợp xấu nhất, việc kiểm tra quanh các thay đổi được phép trong Warehouse khác với trong service Order, và kết quả là ta có thể có những đơn hàng ở những trạng thái rất kỳ quặc, rối rắm.

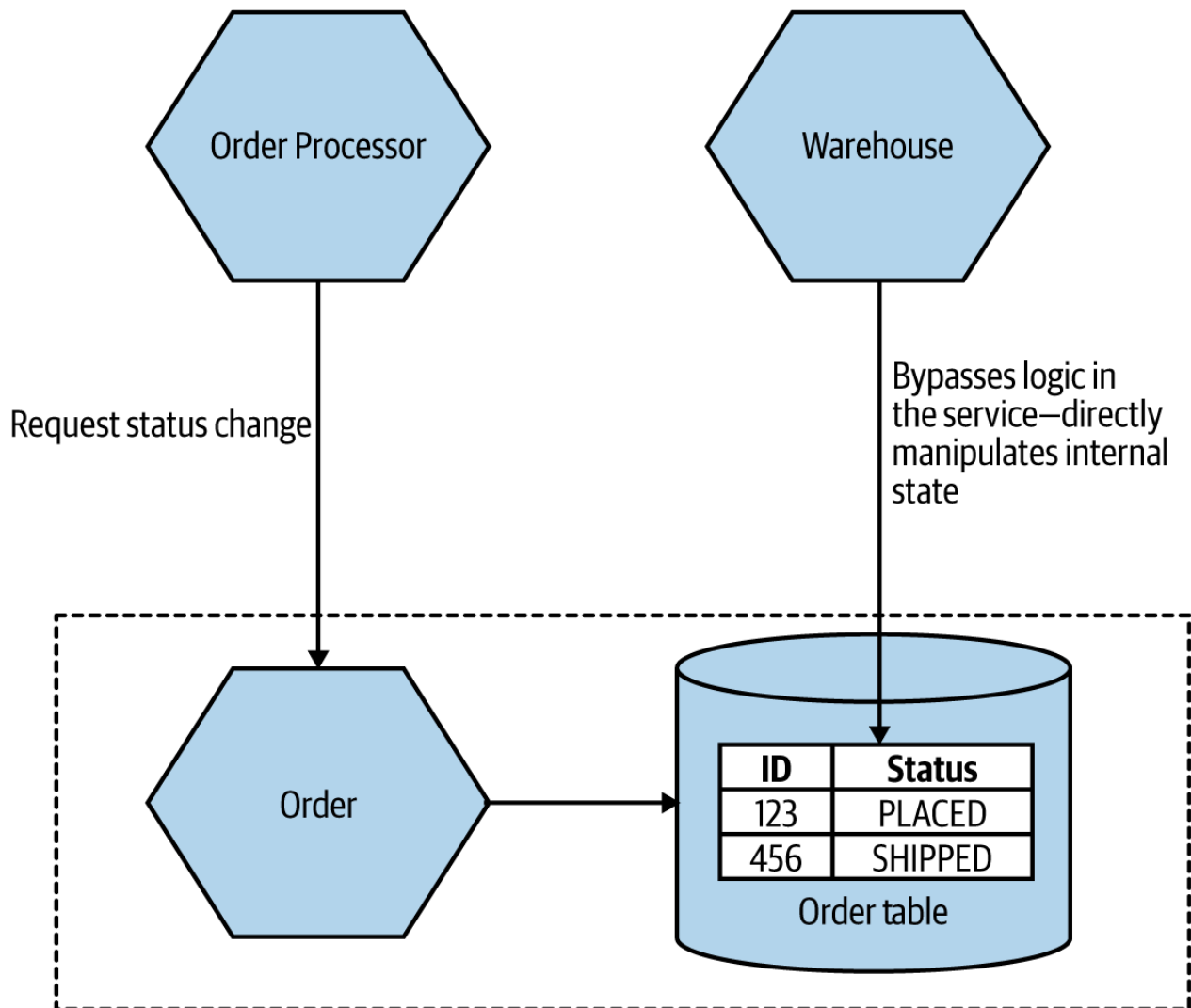


Figure 2-11. An example of content coupling in which the Warehouse is directly accessing the internal data of the Order service. In this situation, we also have the issue that the internal data structure of

our order table is exposed to an outside party. When changing the Order service, we now have to be extremely careful about making changes to that particular table—that’s even assuming it’s obvious to us that this table is being directly accessed by an outside party. The easy fix here is to have the Warehouse send requests to the Order service itself, where we can vet the request but also hide the internal detail, making subsequent changes to the Order service much easier.

Hình 2-11. Một ví dụ về content coupling trong đó Warehouse đang truy cập trực tiếp dữ liệu nội bộ của service Order Trong tình huống này, ta còn có vấn đề là cấu trúc dữ liệu nội bộ của bảng đơn hàng của ta bị phơi bày cho một bên ngoài. Khi thay đổi service Order , giờ đây ta phải cực kỳ cẩn trọng về việc thực hiện các thay đổi đối với cái bảng cụ thể đó—đó là còn giả định rằng ta nhận ra rõ ràng bảng này đang bị một bên ngoài truy cập trực tiếp. Cách khắc phục dễ dàng ở đây là để Warehouse gửi yêu cầu tới chính service Order , nơi ta có thể thẩm tra yêu cầu nhưng cũng giấu được chi tiết nội bộ, khiến những thay đổi sau này đối với service Order dễ dàng hơn nhiều.

If you are working on a microservice, it’s vital that you have a clear separation between what can be changed freely and what cannot. To be explicit, as a developer you need to know when you are changing functionality that is part of the contract your service exposes to the outside world. You need to ensure that if you make changes, that you will not break upstream consumers. Functionality that doesn’t impact the contract your microservice exposes can be changed without concern.

Nếu bạn đang làm việc trên một microservice, điều thiết yếu là bạn phải có một sự phân tách rõ ràng giữa cái gì có thể thay đổi tự do và cái gì thì không. Nói rõ ra, với tư cách một lập trình viên, bạn cần biết khi nào mình đang thay đổi chức năng vốn là một phần của hợp đồng mà service của bạn phơi bày ra thế giới bên ngoài. Bạn cần đảm bảo rằng nếu bạn thực hiện các thay đổi, bạn sẽ không phá vỡ các bên tiêu thụ phía trên. Chức năng không tác động đến hợp đồng mà microservice của bạn phơi bày thì có thể được thay đổi mà không phải bận tâm.

It’s certainly the case that the problems that occur with common coupling also apply with content coupling, but content coupling has some additional headaches that make it problematic enough that some people refer to it as pathological coupling .

Chắc chắn là những vấn đề xảy ra với common coupling cũng áp dụng với content coupling, nhưng content coupling còn có thêm một số nhược điểm khiến nó rắc rối đến mức một số người gọi nó là pathological coupling (coupling bệnh lý).

When you allow an outside party to directly access your database, the database in effect becomes part of that external contract, albeit one where you cannot easily reason about what can or cannot be changed. You’ve lost the ability to define what is shared (and therefore cannot be changed easily) and what is hidden. Information hiding has gone out of the window.

Khi bạn cho phép một bên ngoài truy cập trực tiếp vào cơ sở dữ liệu của mình, cơ sở dữ liệu trên thực tế trở thành một phần của hợp đồng bên ngoài đó, dù là một hợp đồng mà ở đó bạn không thể dễ dàng lý giải được cái gì có thể hay không thể thay đổi. Bạn đã mất đi khả năng định nghĩa cái gì được chia sẻ (và do đó không thể thay đổi dễ dàng) và cái gì được giấu kín. Che giấu thông tin đã bay ra ngoài cửa sổ.

In short, avoid content coupling.

Tóm lại, hãy tránh content coupling.

Just Enough Domain-Driven Design — Domain-Driven Design vừa đủ

As I introduced in Chapter 1, the primary mechanism we use for finding microservice boundaries is around the domain itself, making use of domain-driven design (DDD) to help create a model of our domain. Let's now extend our understanding of how DDD works in the context of microservices.

Như tôi đã giới thiệu ở Chương 1, cơ chế chính mà ta dùng để tìm ranh giới microservice là quanh chính miền nghiệp vụ, tận dụng domain-driven design (DDD) để giúp tạo ra một mô hình của miền nghiệp vụ của ta. Giờ hãy mở rộng hiểu biết của ta về cách DDD hoạt động trong ngữ cảnh microservice.

The desire to have our programs better represent the real world in which they will operate is not new. Object-oriented programming languages like Simula were developed to allow us to model real domains. But it takes more than program language capabilities for this idea to really take shape.

Mong muốn để các chương trình của ta đại diện tốt hơn cho thế giới thực mà chúng sẽ vận hành trong đó không phải là điều mới. Các ngôn ngữ lập trình hướng đối tượng như Simula được phát triển để cho phép ta mô hình hóa các miền nghiệp vụ thực. Nhưng cần nhiều hơn là chỉ những khả năng của ngôn ngữ lập trình thì ý tưởng này mới thực sự thành hình.

Eric Evans's Domain-Driven Design 11 presented a series of important ideas that helped us better represent the problem domain in our programs. A full exploration of

Cuốn Domain-Driven Design 11 của Eric Evans đã trình bày một loạt ý tưởng quan trọng giúp ta đại diện tốt hơn cho miền vấn đề trong các chương trình của mình. Một khám phá đầy đủ về

11 Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software (Boston: Addison-Wesley, 2004).

11 Eric Evans, Domain-Driven Design: Tackling Complexity in the Heart of Software (Boston: Addison-Wesley, 2004).

these ideas is outside the scope of this book, but there are some core concepts of DDD that are worth highlighting, including:

những ý tưởng này nằm ngoài phạm vi cuốn sách này, nhưng có một số khái niệm cốt lõi của DDD đáng được làm nổi bật, bao gồm:

Ubiquitous language Defining and adopting a common language to be used in code and in describing the domain, to aid communication.

Ubiquitous language Định nghĩa và áp dụng một ngôn ngữ chung được dùng trong mã và trong việc mô tả miền nghiệp vụ, nhằm hỗ trợ giao tiếp.

Aggregate A collection of objects that are managed as a single entity, typically referring to real-world concepts.

Aggregate Một tập hợp các đối tượng được quản lý như một thực thể đơn nhất, thường tham chiếu đến các khái niệm trong thế giới thực.

Bounded context An explicit boundary within a **business domain** that provides functionality to the wider system but that also hides complexity.

Bounded context Một ranh giới rõ ràng bên trong một miền nghiệp vụ, cung cấp chức năng cho hệ thống rộng hơn nhưng đồng thời cũng giấu đi độ phức tạp.

Ubiquitous Language — Ubiquitous Language

Ubiquitous language refers to the idea that we should strive to use the same terms in our code as the users use. The idea is that having a common language between the delivery team and the actual people will make it easier to model the real-world domain and also should improve communication.

Ubiquitous language nói đến ý tưởng rằng ta nên cố gắng dùng cùng những thuật ngữ trong mã của mình như những thuật ngữ mà người dùng dùng. Ý tưởng là việc có một ngôn ngữ chung giữa đội phát triển và những con người thực tế sẽ giúp dễ dàng hơn trong việc mô hình hóa miền nghiệp vụ thực, và cũng nên cải thiện giao tiếp.

As a counterexample, I recall a situation when working at a large, global bank. We were working in the area of corporate liquidity, a fancy term that basically refers to the ability to move cash among different accounts held by the same corporate entity. The product owner was really great to work with, and she had a fantastically deep understanding of the various products that she wanted to bring to market. When working with her, we'd have discussions about things like haircuts and end-of-day sweeps, all things that made a lot of sense in her world and that had meaning to her customers.

Như một phản ví dụ, tôi nhớ một tình huống khi làm việc tại một ngân hàng lớn, toàn cầu. Chúng tôi làm việc trong lĩnh vực thanh khoản doanh nghiệp (corporate liquidity), một thuật ngữ hoa mỹ về cơ bản chỉ khả năng dịch chuyển tiền mặt giữa các tài khoản khác nhau do cùng một pháp nhân doanh nghiệp nắm giữ. Người phụ trách sản phẩm (product owner) thực sự tuyệt vời để cộng tác, và cô ấy có một hiểu biết sâu sắc đến kinh ngạc về các sản phẩm khác nhau mà cô muốn đưa ra thị trường. Khi làm việc với cô, chúng tôi sẽ có những cuộc thảo luận về những thứ như haircut và end-of-day sweep, tất cả những thứ rất có ý nghĩa trong thế giới của cô và có nghĩa với khách hàng của cô.

The code, on the other hand, had none of this language in there. At some point previously, a decision had been made to use a standard data model for the database. It was widely referred to as “the IBM banking model,” but I came to grips with whether this was a standard IBM product or just the creation of a consultant from IBM. By defining the loose concept of an “arrangement,” the theory went that any banking operation could be modeled. Taking out a loan? That was an arrangement. Buying a share? That’s an arrangement! Applying for a credit card? Guess what—that’s an arrangement too!

Mặt khác, đoạn mã thì chẳng có chút ngôn ngữ nào trong đó. Ở một thời điểm nào đó trước kia, người ta đã quyết định dùng một mô hình dữ liệu chuẩn cho cơ sở dữ liệu. Nó được gọi rộng rãi là “mô hình ngân hàng IBM,” nhưng tôi vẫn không nắm được liệu đây là một sản phẩm chuẩn của IBM hay chỉ là tác phẩm của một chuyên gia tư vấn nào đó từ IBM. Bằng cách định nghĩa khái niệm lỏng lẻo về một “arrangement” (thỏa thuận), lý thuyết cho rằng bất kỳ thao tác ngân hàng nào cũng có thể được mô hình hóa. Vay một khoản? Đó là một arrangement. Mua một cổ phiếu? Đó là một arrangement! Đăng ký một thẻ tín dụng? Đoán xem—đó cũng là một arrangement luôn!

The data model had polluted the code to such an extent that the codebase was shorn of all real understanding of the system we were building. We weren’t building a generic banking application. We were building a system specifically to manage corporate liquidity. The problem was that we had to map the rich domain language of the product owner to the generic code concepts—meaning a lot of work in helping translate. Our business analysts were often just spending their time explaining the same concepts over and over again as a result.

Mô hình dữ liệu đã làm ô nhiễm đoạn mã đến mức codebase bị tước sạch mọi hiểu biết thực sự về hệ thống mà chúng tôi đang xây dựng. Chúng tôi không xây dựng một ứng dụng ngân hàng chung chung. Chúng tôi đang xây dựng một hệ thống chuyên biệt để quản lý thanh khoản doanh nghiệp. Vấn đề là chúng tôi phải ánh xạ ngôn ngữ miền nghiệp vụ phong

phú của product owner sang những khái niệm mã chung chung—nghĩa là rất nhiều công sức trong việc hỗ trợ phiên dịch. Kết quả là các nhà phân tích nghiệp vụ của chúng tôi thường chỉ dành thời gian giải thích đi giải thích lại cùng những khái niệm hết lần này đến lần khác.

By working the real-world language into the code, things became much easier. A developer picking up a story written using the terms that had come straight from the product owner was much more likely to understand their meaning and work out what needed to be done.

Bằng cách đưa ngôn ngữ của thế giới thực vào trong mã, mọi thứ trở nên dễ dàng hơn nhiều. Một lập trình viên nhận một story được viết bằng những thuật ngữ đến thẳng từ product owner sẽ nhiều khả năng hiểu được ý nghĩa của chúng và xác định được cần làm gì.

Aggregate — Aggregate

In DDD, an aggregate is a somewhat confusing concept, with many different definitions out there. Is it just an arbitrary collection of objects? The smallest unit that should be taken out of a database? The model that has always worked for me is to first consider an aggregate as a representation of a real domain concept—think of something like an Order, an Invoice, a Stock Item, and so on. Aggregates typically have a life cycle around them, which opens them up to being implemented as a state machine.

Trong DDD, aggregate là một khái niệm có phần khó hiểu, với rất nhiều định nghĩa khác nhau ngoài kia. Liệu nó có phải chỉ là một tập hợp đối tượng tùy ý? Đơn vị nhỏ nhất nên được lấy ra khỏi một cơ sở dữ liệu? Mô hình luôn hiệu quả với tôi là trước hết hãy xem một aggregate như một biểu diễn của một khái niệm miền nghiệp vụ thực—hãy nghĩ đến những thứ như một Order (Đơn hàng), một Invoice (Hóa đơn), một Stock Item (Mặt hàng tồn kho), v.v. Aggregate thường có một vòng đời gắn với chúng, điều này mở ra khả năng triển khai chúng dưới dạng một máy trạng thái.

As an example in the MusicCorp domain, an Order aggregate might contain multiple line items that represent the items in the order. Those line items have meaning only as part of the overall Order aggregate.

Như một ví dụ trong miền nghiệp vụ của MusicCorp, một aggregate Order có thể chứa nhiều dòng mặt hàng (line item) đại diện cho các mặt hàng trong đơn. Những dòng mặt hàng đó chỉ có ý nghĩa khi là một phần của aggregate Order tổng thể.

We want to treat aggregates as self-contained units; we want to ensure that the code that handles the state transitions of an aggregate are grouped together, along with the state itself. So one aggregate should be managed by one microservice, although a single microservice might own management of multiple aggregates.

Chúng ta muốn xử lý các aggregate như những đơn vị độc lập, khép kín; ta muốn đảm bảo rằng đoạn mã xử lý các chuyển đổi trạng thái của một aggregate được nhóm lại với nhau, cùng với chính trạng thái đó. Vậy nên một aggregate nên được quản lý bởi một microservice, mặc dù một microservice đơn lẻ có thể sở hữu việc quản lý nhiều aggregate.

In general, though, you should think of an aggregate as something that has state, identity, a life cycle that will be managed as part of the system. Aggregates typically refer to real-world concepts.

Tuy nhiên, nhìn chung, bạn nên nghĩ về một aggregate như một thứ có trạng thái, danh tính, một vòng đời sẽ được quản lý như một phần của hệ thống. Aggregate thường tham

chiếu đến các khái niệm trong thế giới thực.

A single microservice will handle the life cycle and data storage of one or more different types of aggregates. If functionality in another service wants to change one of these aggregates, it needs to either directly request a change in that aggregate or else have the aggregate itself react to other things in the system to initiate its own state transitions, perhaps by subscribing to events issued by other microservices.

Một microservice đơn lẻ sẽ xử lý vòng đời và việc lưu trữ dữ liệu của một hoặc nhiều loại aggregate khác nhau. Nếu chức năng trong một service khác muốn thay đổi một trong những aggregate này, nó cần hoặc trực tiếp yêu cầu một thay đổi trong aggregate đó, hoặc để chính aggregate phản ứng với những thứ khác trong hệ thống nhằm khởi xướng các chuyển đổi trạng thái của riêng nó, có lẽ bằng cách đăng ký nhận (subscribe) các sự kiện do các microservice khác phát ra.

The key thing to understand here is that if an outside party requests a state transition in an aggregate, the aggregate can say no. You ideally want to implement your aggregates in such a way that illegal state transitions are impossible.

Điều then chốt cần hiểu ở đây là nếu một bên ngoài yêu cầu một chuyển đổi trạng thái trong một aggregate, thì aggregate có thể nói không. Lý tưởng nhất là bạn muốn triển khai các aggregate của mình theo cách khiến những chuyển đổi trạng thái bất hợp pháp là bất khả thi.

Aggregates can have relationships with other aggregates. In Figure 2-12, we have a Customer aggregate that is associated with one or more Orders and one or more Wishlists. These aggregates could be managed by the same microservice or by different microservices.

Các aggregate có thể có mối quan hệ với các aggregate khác. Ở Hình 2-12, ta có một aggregate Customer được liên kết với một hoặc nhiều Order và một hoặc nhiều Wishlist. Những aggregate này có thể được quản lý bởi cùng một microservice hoặc bởi các microservice khác nhau.

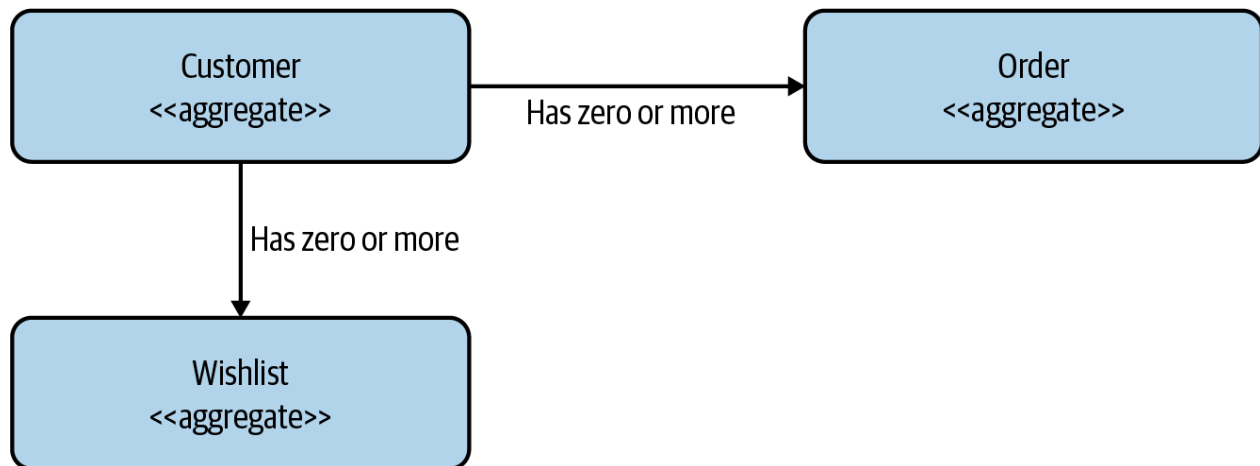


Figure 2-12. One Customer aggregate may be associated with one or more Order or Wishlist aggregates

Hình 2-12. Một aggregate Customer có thể được liên kết với một hoặc nhiều aggregate Order hoặc Wishlist

If these relationships between aggregates exist inside the scope of a single microservice, they could easily be stored using something like a foreign key relationship if using a relational database. If the

relationships between these aggregates span microservice boundaries, though, we need some way to model the relationships.

Nếu những mối quan hệ giữa các aggregate này tồn tại bên trong phạm vi của một microservice đơn lẻ, chúng có thể dễ dàng được lưu trữ bằng một thứ như quan hệ khóa ngoại (foreign key) nếu dùng cơ sở dữ liệu quan hệ. Tuy nhiên, nếu các mối quan hệ giữa những aggregate này trải dài qua các ranh giới microservice, ta cần một cách nào đó để mô hình hóa các mối quan hệ.

Now, we could simply store the ID of the aggregate directly in our local database. For example, consider a Finance microservice that manages a financial ledger, which stores transactions against a customer. Locally, within the Finance microservice's database we could have a CustID column that contains the ID of that customer. If we wanted to get more information about that customer, we'd have to do a lookup against the Customer microservice using that ID.

Giờ đây, ta có thể đơn giản lưu trữ ID của aggregate trực tiếp trong cơ sở dữ liệu cục bộ của mình. Ví dụ, hãy xét một microservice Finance quản lý một sổ cái tài chính, lưu trữ các giao dịch gắn với một khách hàng. Cục bộ, bên trong cơ sở dữ liệu của microservice Finance, ta có thể có một cột CustID chứa ID của khách hàng đó. Nếu muốn lấy thêm thông tin về khách hàng đó, ta sẽ phải thực hiện một lần tra cứu lên microservice Customer bằng ID đó.

The problem with this concept is that it isn't very explicit—in fact, the relationship between the CustID column and the remote customer is entirely implicit. To know how that ID was being used, we'd have to look at the code of the Finance microservice itself. It would be nice if we could store a reference to a foreign aggregate in a way that is more obvious.

Vấn đề với khái niệm này là nó không thật sự tường minh—thực ra, mối quan hệ giữa cột CustID và khách hàng từ xa hoàn toàn là ngầm định. Để biết ID đó được dùng ra sao, ta sẽ phải nhìn vào mã của chính microservice Finance. Sẽ thật tốt nếu ta có thể lưu trữ một tham chiếu tới một aggregate ngoại theo cách hiển nhiên hơn.

In Figure 2-13, we have changed things to make the relationship explicit. Rather than a vanilla ID for the customer reference, we instead store a URI, which we might use if building a REST-based system.

Ở Hình 2-13, ta đã thay đổi mọi thứ để khiến mối quan hệ trở nên tường minh. Thay vì một ID trơn cho tham chiếu khách hàng, ta thay vào đó lưu trữ một URI, thứ mà ta có thể dùng nếu xây dựng một hệ thống dựa trên REST.

I know some people object to the use of templated URIs in REST systems, and I understand why—I just want to keep things simple for this example.

Tôi biết một số người phản đối việc dùng URI có khuôn mẫu (templated URI) trong các hệ thống REST, và tôi hiểu vì sao—chỉ là tôi muốn giữ mọi thứ đơn giản cho ví dụ này.

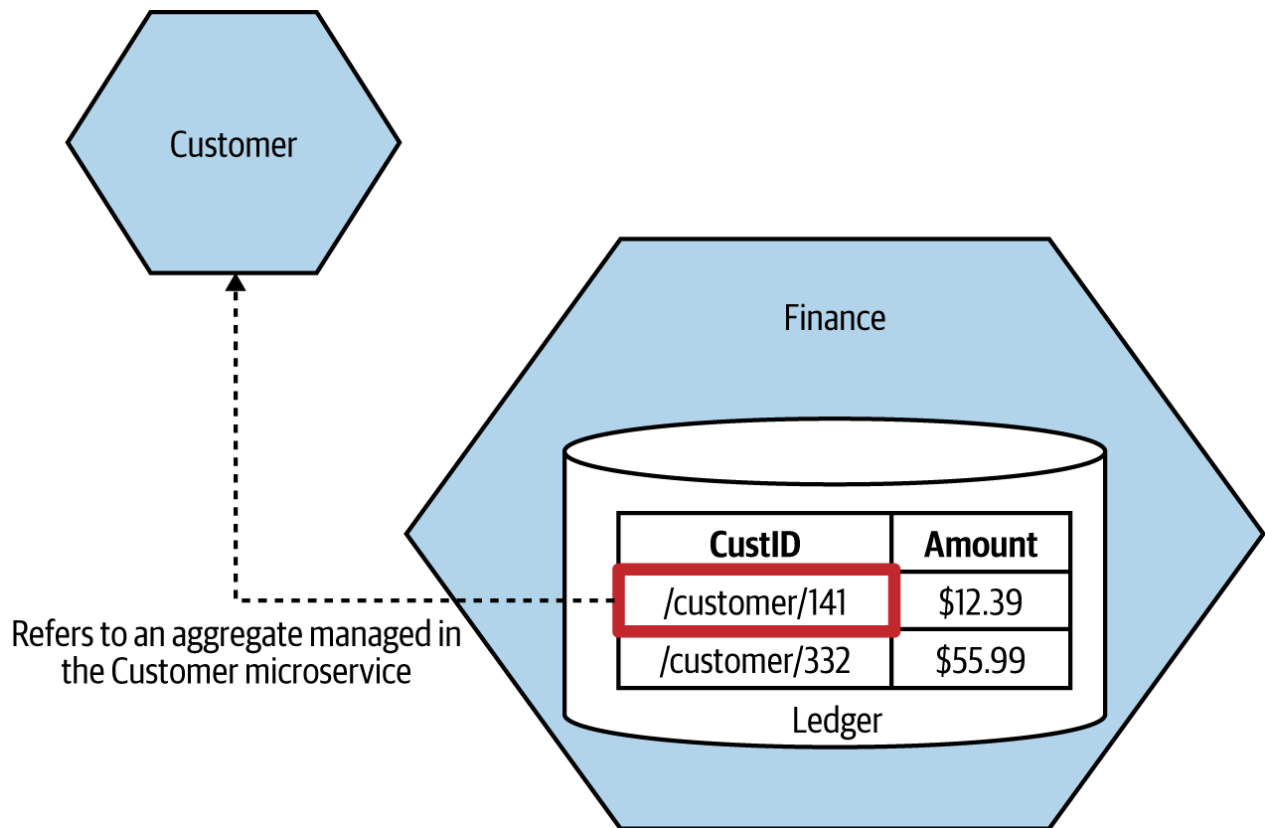


Figure 2-13. An example of how a relationship between two aggregates in different microservices can be implemented

Hình 2-13. Một ví dụ về cách một mối quan hệ giữa hai aggregate ở các microservice khác nhau có thể được triển khai

The benefits of this approach are twofold. The nature of the relationship is explicit, and in a REST system we could directly dereference this URI to look up the associated resource. But what if you aren't building a REST system? Phil Calçado describes a variation of this approach in use at SoundCloud, 13 where they developed a pseudo- URI scheme for cross-service references. For example, `soundcloud:tracks:123` would be a reference to a track with the ID of 123 . This is much more explicit for a human looking at this identifier, but it is also a useful enough scheme that it would be easy to imagine creating code that could ease cross-microservice aggregate lookups if needed.

Lợi ích của cách tiếp cận này có hai mặt. Bản chất của mối quan hệ là tường minh, và trong một hệ thống REST ta có thể trực tiếp giải tham chiếu (dereference) URI này để tra cứu tài nguyên liên quan. Nhưng nếu bạn không xây dựng một hệ thống REST thì sao? Phil Calçado mô tả một biến thể của cách tiếp cận này đang được dùng tại SoundCloud, 13 nơi họ phát triển một lược đồ pseudo-URI cho các tham chiếu liên service. Ví dụ, `soundcloud:tracks:123` sẽ là một tham chiếu tới một track có ID là 123 . Điều này tường minh hơn nhiều đối với một con người nhìn vào định danh này, nhưng nó cũng là một lược đồ đủ hữu ích đến mức dễ dàng hình dung việc tạo ra đoạn mã có thể giúp đơn giản hóa việc tra cứu aggregate liên microservice nếu cần.

There are lots of ways to break a system into aggregates, with some choices being highly subjective. You may decide, for performance reasons or for ease of implementation, to reshape aggregates over time. I consider implementation concerns to be secondary, however; I begin by letting the mental

model of the system users be my guiding light on initial design until other factors come into play.

Có rất nhiều cách để chia một hệ thống thành các aggregate, với một số lựa chọn mang tính chủ quan cao. Bạn có thể quyết định, vì lý do hiệu năng hoặc để dễ triển khai, định hình lại các aggregate theo thời gian. Tuy nhiên, tôi xem các mối bận tâm về triển khai là thứ yếu; tôi bắt đầu bằng cách để mô hình tư duy của những người dùng hệ thống làm kim chỉ nam cho thiết kế ban đầu cho đến khi các yếu tố khác bắt đầu xen vào.

13 Phil Calçado, “Pattern: Using Pseudo-URIs with Microservices,” <https://oreil.ly/xOYMr> .

13 Phil Calçado, “Pattern: Using Pseudo-URIs with Microservices,” <https://oreil.ly/xOYMr> .

Bounded Context — Bounded Context

A bounded context typically represents a larger organizational boundary. Within the scope of that boundary, explicit responsibilities need to be carried out. That’s all a bit woolly, so let’s look at another specific example.

Một bounded context thường đại diện cho một ranh giới tổ chức lớn hơn. Trong phạm vi của ranh giới đó, những trách nhiệm rõ ràng cần được thực thi. Nói thế thì hơi mơ hồ, vậy hãy xem một ví dụ cụ thể khác.

At MusicCorp, our warehouse is a hive of activity—managing orders being shipped out (and the odd return), taking delivery of new stock, having forklift truck races, and so on. Elsewhere, the finance department is perhaps less fun-loving but still has an important function inside our organization, handling payroll, paying for shipments, and the like.

Tại MusicCorp, nhà kho của chúng ta là một tổ ong đầy hoạt động—quản lý việc gửi các đơn hàng đi (và đôi khi có hàng trả lại), nhận hàng tồn kho mới về, đưa xe nâng, v.v. Ở nơi khác, phòng tài chính có lẽ ít vui nhộn hơn nhưng vẫn có một chức năng quan trọng bên trong tổ chức của ta, xử lý bảng lương, thanh toán cho các lô hàng, và những thứ tương tự.

Bounded contexts hide implementation detail. There are internal concerns—for example, the types of forklift trucks used are of little interest to anyone other than the folks in the warehouse. These internal concerns should be hidden from the outside world, which doesn’t need to know, nor should it care.

Các bounded context giấu chi tiết triển khai. Có những mối bận tâm nội bộ—ví dụ, các loại xe nâng được dùng thì chẳng mấy ai ngoài những người trong nhà kho quan tâm. Những mối bận tâm nội bộ này nên được giấu khỏi thế giới bên ngoài, vốn không cần biết, cũng không nên bận tâm.

From an implementation point of view, bounded contexts contain one or more aggregates. Some aggregates may be exposed outside the bounded context; others may be hidden internally. As with aggregates, bounded contexts may have relationships with other bounded contexts—when mapped to services, these dependencies become inter-service dependencies.

Từ góc độ triển khai, các bounded context chứa một hoặc nhiều aggregate. Một số aggregate có thể được phơi bày ra bên ngoài bounded context; số khác có thể được giấu ở bên trong. Cũng như với aggregate, các bounded context có thể có mối quan hệ với các bounded context khác—khi được ánh xạ sang service, những phụ thuộc này trở thành phụ thuộc liên service.

Let's return for a moment to the MusicCorp business. Our domain is the whole business in which we are operating. It covers everything from the warehouse to the reception desk, from finance to ordering. We may or may not model all of that in our software, but that is nonetheless the domain in which we are operating. Let's think about parts of that domain that look like the bounded contexts to which Eric Evans refers.

Hãy quay lại một chút với hoạt động kinh doanh của MusicCorp. Miền nghiệp vụ của ta là toàn bộ hoạt động kinh doanh mà ta đang vận hành trong đó. Nó bao trùm mọi thứ từ nhà kho đến quầy lễ tân, từ tài chính đến đặt hàng. Ta có thể mô hình hóa hoặc không mô hình hóa tất cả những thứ đó trong phần mềm của mình, nhưng dù sao đó vẫn là miền nghiệp vụ mà ta đang vận hành trong đó. Hãy nghĩ về những phần của miền nghiệp vụ đó trông giống các bounded context mà Eric Evans nhắc đến.

Hidden models

Các mô hình bị giấu

For MusicCorp, we can consider the finance department and the warehouse to be two separate bounded contexts. They both have an explicit **interface** to the outside world (in terms of inventory reports, pay slips, etc.), and they have details that only they need to know about (forklift trucks, calculators).

Với MusicCorp, ta có thể xem phòng tài chính và nhà kho là hai bounded context riêng biệt. Cả hai đều có một giao diện rõ ràng với thế giới bên ngoài (về mặt báo cáo tồn kho, phiếu lương, v.v.), và chúng có những chi tiết mà chỉ riêng chúng cần biết (xe nâng, máy tính).

The finance department doesn't need to know about the detailed inner workings of the warehouse. It does need to know some things, however; for example, it needs to know about stock levels to keep the accounts up to date. Figure 2-14 shows an example context diagram. We see concepts that are internal to the warehouse, like a picker (someone who picks orders), shelves that represent stock locations, and so on. Likewise, entries in the general ledger are integral to finance but are not shared externally here.

Phòng tài chính không cần biết về cách vận hành nội bộ chi tiết của nhà kho. Tuy nhiên, nó có cần biết một số thứ; ví dụ, nó cần biết về mức tồn kho để giữ cho sổ sách được cập nhật. Hình 2-14 cho thấy một ví dụ về sơ đồ ngữ cảnh. Ta thấy những khái niệm nội bộ của nhà kho, như một picker (người gom hàng theo đơn), các kệ đại diện cho vị trí lưu hàng, v.v. Tương tự, các bút toán trong sổ cái tổng hợp là không thể thiếu với tài chính nhưng không được chia sẻ ra bên ngoài ở đây.

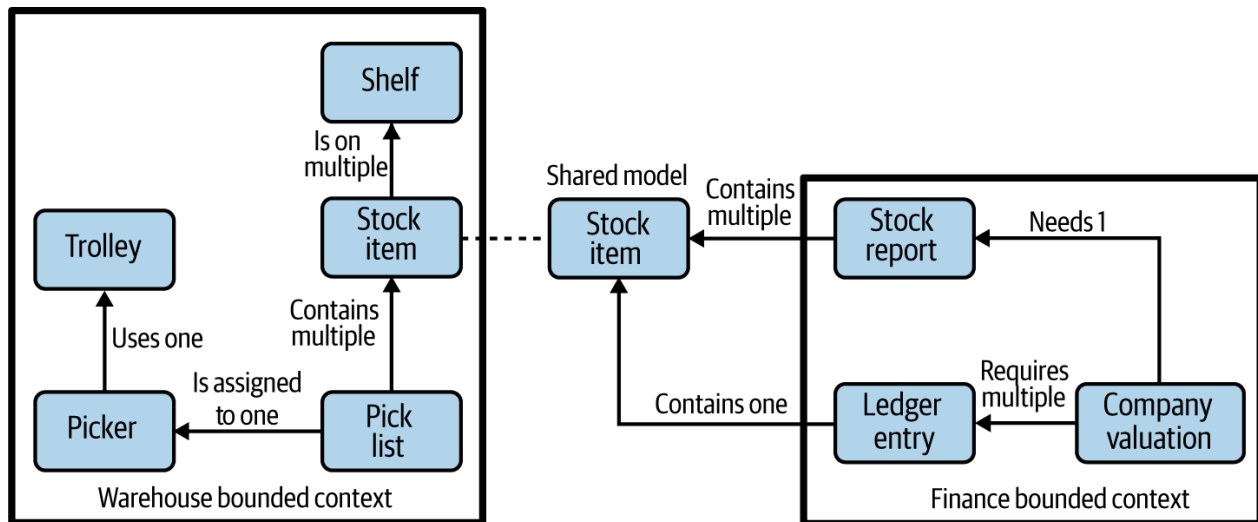


Figure 2-14. A shared model between the finance department and the warehouse

Hình 2-14. Một mô hình được chia sẻ giữa phòng tài chính và nhà kho

To be able to work out the valuation of the company, though, the finance employees need information about the stock we hold. The stock item then becomes a shared model between the two contexts. However, note that we don't need to blindly expose everything about the stock item from the warehouse context. In Figure 2-15, we see how Stock Item inside the warehouse bounded context contains references to the shelf locations, but the shared representation contains only a count. So there is the internal-only representation and the external representation we expose. Often, when you have different internal and external representations, it may be beneficial to name them differently to avoid confusion—in this situation, one approach could be to call the shared Stock Item a Stock Count instead.

Tuy nhiên, để có thể tính được giá trị định giá của công ty, các nhân viên tài chính cần thông tin về hàng tồn kho mà ta đang giữ. Khi đó, mặt hàng tồn kho trở thành một mô hình được chia sẻ giữa hai ngữ cảnh. Tuy nhiên, lưu ý rằng ta không cần phơi bày một cách mù quáng mọi thứ về mặt hàng tồn kho từ ngữ cảnh nhà kho. Ở Hình 2-15, ta thấy cách Stock Item bên trong bounded context nhà kho chứa các tham chiếu đến vị trí kệ, nhưng biểu diễn được chia sẻ chỉ chứa một con số đếm. Vậy là có biểu diễn chỉ-nội-bộ và biểu diễn bên ngoài mà ta phơi bày. Thường thì, khi bạn có các biểu diễn nội bộ và bên ngoài khác nhau, đặt tên cho chúng khác nhau có thể có lợi để tránh nhầm lẫn—trong tình huống này, một cách tiếp cận có thể là gọi Stock Item được chia sẻ là Stock Count.

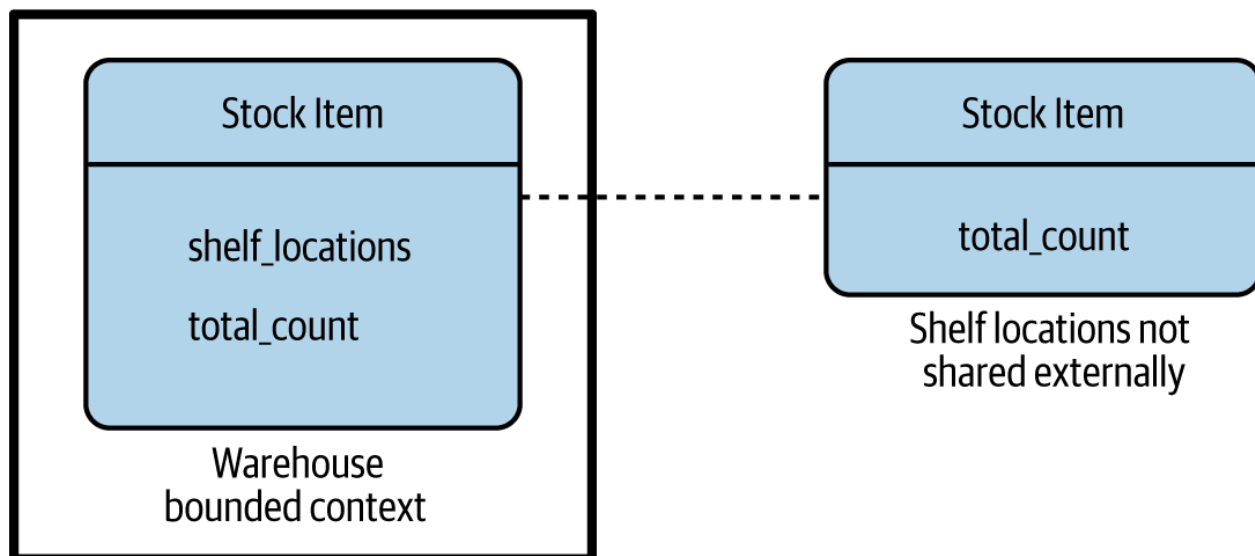


Figure 2-15. A model that is shared can decide to hide information that should not be shared externally

Hình 2-15. Một mô hình được chia sẻ có thể quyết định giấu đi thông tin không nên được chia sẻ ra bên ngoài

Shared models

Các mô hình được chia sẻ

We can also have concepts that appear in more than one bounded context. In Figure 2-14 we saw that a customer exists in both locations. What does this mean? Is the customer copied? The way to think about this is that conceptually, both finance and the warehouse need to know something about our customer. Finance needs to know about the financial payments made to a customer, whereas the warehouse needs to know about the customer to the extent that it knows what packages have been sent so as to allow for deliveries to be traced.

Ta cũng có thể có những khái niệm xuất hiện trong nhiều hơn một bounded context. Ở Hình 2-14, ta thấy một khách hàng tồn tại ở cả hai nơi. Điều này nghĩa là gì? Khách hàng có bị sao chép không? Cách để nghĩ về điều này là, về mặt khái niệm, cả tài chính lẫn nhà kho đều cần biết điều gì đó về khách hàng của ta. Tài chính cần biết về các khoản thanh toán tài chính đã thực hiện cho một khách hàng, trong khi nhà kho cần biết về khách hàng ở mức độ nó biết những gói hàng nào đã được gửi đi để cho phép truy vết việc giao hàng.

When you have a situation like this, a shared model like customer can have different meanings in the different bounded contexts and therefore might be called different things. We might be happy to keep the name “customer” in finance, but in the warehouse we might call them a “recipient,” as that is the role they play in that context. We store information about the customer in both locations, but the information is different. Finance stores information about the customer’s financial payments (or refunds); the warehouse stores information related to the goods shipped. We still may need to link both local concepts to a global customer, and we may want to look up common, shared information about that customer like their name or email address—we could use a technique like that shown in Figure 2-13 to achieve this.

Khi bạn có một tình huống như thế này, một mô hình được chia sẻ như khách hàng có thể có những ý nghĩa khác nhau trong các bounded context khác nhau và do đó có thể được gọi bằng những cái tên khác nhau. Ta có thể vui lòng giữ tên “customer” trong tài chính,

nhưng trong nhà kho ta có thể gọi họ là “recipient” (người nhận), vì đó là vai trò họ đóng trong ngữ cảnh đó. Ta lưu thông tin về khách hàng ở cả hai nơi, nhưng thông tin là khác nhau. Tài chính lưu thông tin về các khoản thanh toán tài chính của khách hàng (hoặc hoàn tiền); nhà kho lưu thông tin liên quan đến hàng hóa đã gửi đi. Ta vẫn có thể cần liên kết cả hai khái niệm cục bộ với một khách hàng toàn cục, và ta có thể muốn tra cứu thông tin chung, được chia sẻ về khách hàng đó như tên hay địa chỉ email của họ—ta có thể dùng một kỹ thuật như thứ được trình bày ở Hình 2-13 để đạt được điều này.

Mapping Aggregates and Bounded Contexts to Microservices — Ánh xạ Aggregate và Bounded Context sang Microservice

Both the aggregate and the bounded context give us units of cohesion with well-defined interfaces with the wider system. The aggregate is a self-contained state machine that focuses on a single domain concept in our system, with the bounded context representing a collection of associated aggregates, again with an explicit interface to the wider world.

Cả aggregate lẫn bounded context đều cho ta những đơn vị cohesion với các giao diện được định nghĩa rõ ràng với hệ thống rộng hơn. Aggregate là một máy trạng thái khép kín tập trung vào một khái niệm miền nghiệp vụ đơn lẻ trong hệ thống của ta, còn bounded context đại diện cho một tập hợp các aggregate liên quan, cũng với một giao diện rõ ràng với thế giới rộng hơn.

Both can therefore work well as service boundaries. When starting out, as I’ve already mentioned, you want to reduce the number of services you work with. As a result, you should probably target services that encompass entire bounded contexts. As you find your feet and decide to break these services into smaller services, you need to remember that aggregates themselves don’t want to be split apart—one microservice can manage one or more aggregates, but we don’t want one aggregate to be managed by more than one microservice.

Do đó cả hai đều có thể vận hành tốt như những ranh giới service. Khi mới bắt đầu, như tôi đã nhắc, bạn muốn giảm số lượng service mà bạn làm việc cùng. Kết quả là, bạn có lẽ nên nhắm tới những service bao trùm trọn vẹn các bounded context. Khi bạn đã quen tay và quyết định chia những service này thành các service nhỏ hơn, bạn cần nhớ rằng bản thân các aggregate không muốn bị xé lẻ—một microservice có thể quản lý một hoặc nhiều aggregate, nhưng ta không muốn một aggregate bị quản lý bởi nhiều hơn một microservice.

Turtles all the way down

Rùa nổi tiếp rùa, mãi không hết

At the start, you will probably identify a number of coarse-grained bounded contexts. But these bounded contexts can in turn contain further bounded contexts. For example, you could decompose the warehouse into capabilities associated with order fulfillment, inventory management, or goods receiving. When considering the boundaries of your microservices, first think in terms of the larger, coarser-grained contexts, and then subdivide along these nested contexts when you’re looking for the benefits of splitting out these seams.

Lúc bắt đầu, bạn có lẽ sẽ nhận diện ra một số bounded context thô (coarse-grained). Nhưng những bounded context này đến lượt chúng lại có thể chứa các bounded context nhỏ hơn nữa. Ví dụ, bạn có thể phân rã nhà kho thành các năng lực gắn với việc hoàn thành đơn hàng (order fulfillment), quản lý tồn kho, hoặc nhận hàng. Khi cân nhắc ranh giới của các microservice, trước hết hãy tư duy theo các ngữ cảnh lớn hơn, thô hơn, rồi chia nhỏ dọc

theo các ngữ cảnh lồng nhau này khi bạn tìm kiếm lợi ích của việc tách ra những đường nối này.

A trick here is that even if you decide to split a service that models an entire bounded context into smaller services later on, you can still hide this decision from the outside world—perhaps by presenting a coarser-grained **API** to consumers. The decision to decompose a service into smaller parts is arguably an implementation decision, so we might as well hide it if we can. In Figure 2-16 we see an example of this. We’ve split Warehouse down into Inventory and Shipping . As far as the outside world is concerned, there is still just the Warehouse microservice. Internally though, we’ve further decomposed things to allow Inventory to manage Stock Items and have Shipping manage Shipments . Remember, we want to keep the ownership of a single aggregate inside a single microservice.

Một mẹo ở đây là ngay cả khi bạn quyết định chia một service mô hình hóa trọn vẹn một bounded context thành các service nhỏ hơn về sau, bạn vẫn có thể giấu quyết định này khỏi thế giới bên ngoài—có lẽ bằng cách trình ra một API thô hơn cho các bên tiêu thụ. Quyết định phân rã một service thành các phần nhỏ hơn có thể nói là một quyết định triển khai, nên ta cũng nên giấu nó đi nếu có thể. Ở Hình 2-16, ta thấy một ví dụ về điều này. Ta đã chia Warehouse thành Inventory và Shipping . Đối với thế giới bên ngoài, vẫn chỉ có một microservice Warehouse. Tuy nhiên, ở bên trong, ta đã phân rã mọi thứ thêm nữa để cho phép Inventory quản lý các Stock Item và để Shipping quản lý các Shipment . Hãy nhớ, ta muốn giữ quyền sở hữu một aggregate đơn lẻ bên trong một microservice đơn lẻ.

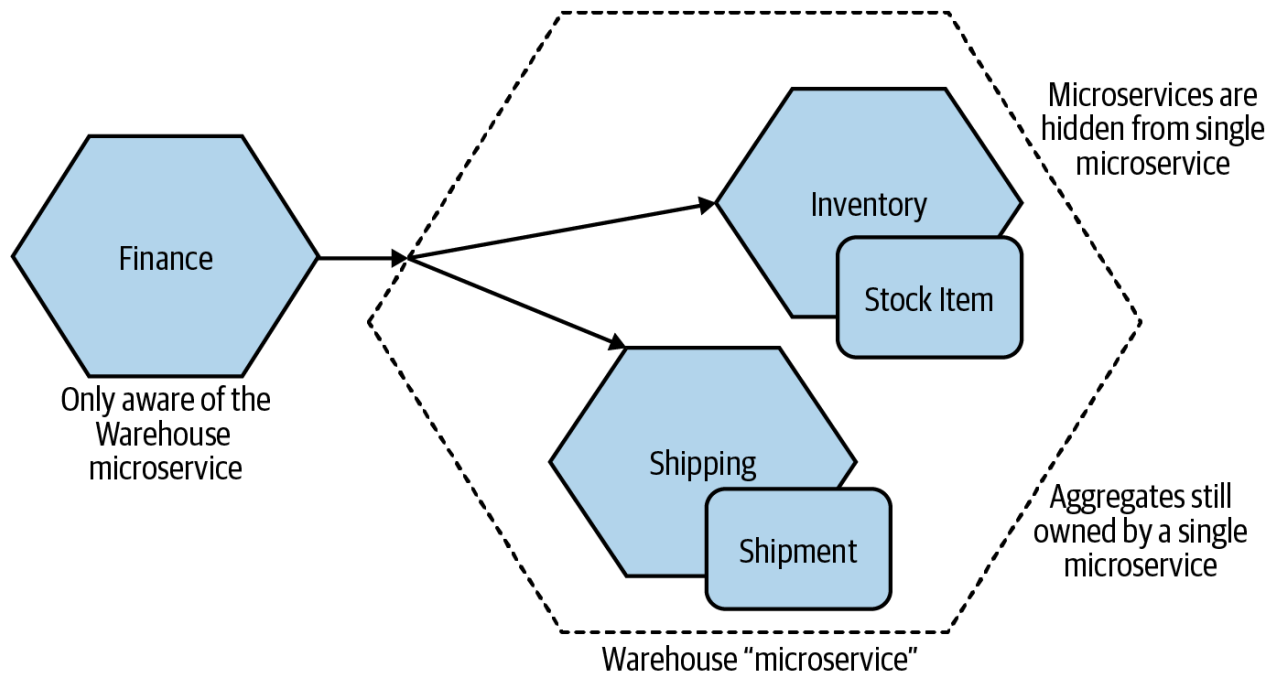


Figure 2-16. The Warehouse service internally has been split into Inventory and Shipping microservices

Hình 2-16. Service Warehouse ở bên trong đã được chia thành các microservice Inventory và Shipping

This is another form of information hiding—we’ve hidden a decision about internal implementation in such a way that if this implementation detail changes again in the future, our consumers will be unaware.

Đây là một hình thức che giấu thông tin khác—ta đã giấu một quyết định về triển khai nội bộ theo cách mà nếu chi tiết triển khai này lại thay đổi trong tương lai, các bên tiêu thụ của ta sẽ không hề hay biết.

Another reason to prefer the nested approach could be to chunk up your architecture to simplify **testing**. For example, when testing services that consume the warehouse, I don't have to stub each service inside the warehouse context—just the more coarse-grained API. This can also give you a unit of isolation when considering larger-scoped tests. I may, for example, decide to have end-to-end tests in which I launch all services inside the warehouse context, but for all other collaborators I might stub them out. We'll explore more about testing and isolation in Chapter 9 .

Một lý do khác để ưu tiên cách tiếp cận lồng nhau có thể là để chia nhỏ kiến trúc của bạn nhằm đơn giản hóa việc kiểm thử. Ví dụ, khi kiểm thử các service tiêu thụ nhà kho, tôi không phải stub từng service bên trong ngữ cảnh nhà kho—chỉ cần API thô hơn. Điều này cũng có thể cho bạn một đơn vị cô lập khi cân nhắc các bài kiểm thử có phạm vi lớn hơn. Chẳng hạn, tôi có thể quyết định có những bài kiểm thử end-to-end trong đó tôi khởi chạy tất cả các service bên trong ngữ cảnh nhà kho, nhưng với tất cả các bên cộng tác khác tôi có thể stub chúng đi. Chúng ta sẽ khám phá thêm về kiểm thử và sự cô lập ở Chương 9.

Event Storming — Event Storming

Event storming , a technique developed by Alberto Brandolini, is a collaborative brainstorming exercise designed to help surface a domain model. Rather than having an architect sit in a corner and come up with their own representation of what the domain model is, 14 event storming brings together technical and nontechnical stakeholders in a joint exercise. The idea is that, by making the development of the domain model a joint activity, you end up with a shared, joined-up view of the world.

Event storming , một kỹ thuật do Alberto Brandolini phát triển, là một bài tập động não cộng tác được thiết kế để giúp làm lộ ra một mô hình miền nghiệp vụ. Thay vì để một kiến trúc sư ngồi một góc và tự nghĩ ra biểu diễn riêng của họ về mô hình miền nghiệp vụ, 14 event storming quy tụ các bên liên quan kỹ thuật và phi kỹ thuật trong một bài tập chung. Ý tưởng là, bằng cách biến việc phát triển mô hình miền nghiệp vụ thành một hoạt động chung, bạn rất cuộc có được một cái nhìn chung, thống nhất về thế giới.

It's worth mentioning at this point that while the domain models defined through event storming can be used to implement **event-driven** systems—and indeed, the mapping is very straightforward—you can also use such a domain model to **build** a more request/response-oriented system.

Ở điểm này đáng nhắc rằng dù các mô hình miền nghiệp vụ được định nghĩa qua event storming có thể được dùng để triển khai các hệ thống hướng sự kiện—và quả thật, việc ánh xạ rất trực tiếp—bạn cũng có thể dùng một mô hình miền nghiệp vụ như vậy để xây dựng một hệ thống thiên về request/response hơn.

Logistics

Khâu hậu cần

Alberto has some very specific views as to how event storming should be run, and on some of these points I am very much in agreement. Firstly, get everyone in a room together. This is often the most difficult step—getting people's calendars to line up can be a problem, as can finding a big enough room. Those issues were all true in a pre-COVID world, but as I write this during the virus-related lockdown in the UK, I'm aware that this step might be even more problematic in the future. The key, though, is to have all stakeholders present at the same time. You want representatives of all parts of

the domain that you plan to model: users, subject matter experts, product owners—whoever is best placed to help represent each part of the domain.

Alberto có một số quan điểm rất cụ thể về cách event storming nên được tiến hành, và ở một số điểm này tôi rất đồng tình. Thứ nhất, hãy đưa tất cả mọi người vào cùng một phòng. Đây thường là bước khó khăn nhất—việc khớp lịch của mọi người có thể là một vấn đề, cũng như việc tìm một căn phòng đủ lớn. Những vấn đề đó đều đúng trong một thế giới tiền COVID, nhưng khi tôi viết những dòng này giữa thời kỳ phong tỏa do virus ở Anh, tôi nhận thức được rằng bước này có thể còn rắc rối hơn nữa trong tương lai. Tuy nhiên, điều then chốt là phải có tất cả các bên liên quan hiện diện cùng một lúc. Bạn muốn có đại diện của tất cả các phần của miền nghiệp vụ mà bạn dự định mô hình hóa: người dùng, các chuyên gia lĩnh vực (subject matter expert), product owner—bất cứ ai phù hợp nhất để giúp đại diện cho từng phần của miền nghiệp vụ.

Once everyone is in a room together, Alberto suggests the removal of all chairs to make sure everyone gets up and is involved. As someone with a bad back, while this strategy is something I understand, I recognize that it may not work for everyone. One thing I do agree with Alberto about is the need to have a large space where the modeling can be done. A common solution is to pin large rolls of brown paper to the walls of the room, allowing for all the walls to be used for capturing information.

Một khi tất cả mọi người đã ở cùng một phòng, Alberto đề xuất dẹp hết ghế đi để đảm bảo mọi người đều đứng dậy và tham gia. Là một người bị đau lưng, dù tôi hiểu chiến lược này, tôi nhận ra rằng nó có thể không hợp với tất cả mọi người. Một điều tôi đồng ý với Alberto là nhu cầu có một không gian rộng nơi việc mô hình hóa có thể được thực hiện. Một giải pháp phổ biến là ghim những cuộn giấy nâu lớn lên các bức tường của phòng, cho phép dùng tất cả các bức tường để ghi lại thông tin.

The main modeling tool is sticky notes to capture the various concepts, with differently colored notes representing different concepts.

Công cụ mô hình hóa chính là các tờ giấy nhớ (sticky note) để ghi lại các khái niệm khác nhau, với những tờ giấy màu khác nhau đại diện cho các khái niệm khác nhau.

The process

Quy trình

The exercise starts with the participants identifying the domain events. These represent things that happen in the system—they are the facts that you care about. “Order Placed” would be an event that we would care about in the context of MusicCorp, as would “Payment Received.” These are captured on orange sticky notes. It is at this point that I have another disagreement with Alberto’s structure, as the events are far and away the most numerous things you’ll be capturing, and orange sticky notes are surprisingly hard to get hold of. 15

Bài tập bắt đầu với việc các thành viên nhận diện các sự kiện miền nghiệp vụ (domain event). Những thứ này đại diện cho những điều xảy ra trong hệ thống—chúng là những sự kiện thực tế mà bạn quan tâm. “Order Placed” (Đơn hàng đã đặt) sẽ là một sự kiện mà ta quan tâm trong ngữ cảnh MusicCorp, cũng như “Payment Received” (Đã nhận thanh toán). Những thứ này được ghi lên các tờ giấy nhớ màu cam. Chính ở điểm này tôi lại có một bất đồng nữa với cấu trúc của Alberto, vì các sự kiện cho đến nay là những thứ nhiều nhất bạn sẽ ghi lại, và giấy nhớ màu cam thì khó kiếm đến bất ngờ. 15

14 I mean no disrespect if this is you—I’ve done this myself more than once.

14 Tôi không có ý xúc phạm nếu đây chính là bạn—tôi đã tự làm điều này không chỉ một lần.

15 I mean, why not yellow? It's the most common color!

15 Ý tôi là, sao không dùng màu vàng? Đó là màu phổ biến nhất mà!

Next, participants identify the commands that cause these events to happen. A command is a decision made by a human (a user of the software) to do something. Here you are trying to understand the system's boundary, and identify the key human actors in the system. Commands are captured on blue sticky notes.

Tiếp theo, các thành viên nhận diện các lệnh (command) gây ra những sự kiện này xảy ra. Một command là một quyết định do một con người (một người dùng của phần mềm) đưa ra để làm việc gì đó. Ở đây bạn đang cố hiểu ranh giới của hệ thống, và nhận diện các tác nhân con người then chốt trong hệ thống. Các command được ghi lên các tờ giấy nhớ màu xanh dương.

The techies in the event storming session should be listening to what their nontechnical colleagues come up with here. A key part of this exercise is not to let any current implementation warp the perception of what the domain is (that comes later). At this stage you want to create a space in which you can get the concepts out of the heads of the key stakeholders and out in the open.

Những người làm kỹ thuật trong buổi event storming nên lắng nghe những gì các đồng nghiệp phi kỹ thuật của họ nghĩ ra ở đây. Một phần then chốt của bài tập này là không để bất kỳ triển khai hiện tại nào bóp méo nhận thức về việc miền nghiệp vụ thực sự là gì (điều đó đến sau). Ở giai đoạn này bạn muốn tạo ra một không gian nơi bạn có thể lôi các khái niệm ra khỏi đầu của các bên liên quan then chốt và đưa chúng ra ngoài cho mọi người thấy.

With events and commands captured, aggregates come next. The events you have at this stage not only are useful for sharing not just what happens in the system, but they also start to highlight what the potential aggregates might be. Think of the aforementioned domain event “Order Placed.” The noun here—“Order”—could well be a potential aggregate. And “Placed” describes something that can happen to an order, so this may well be part of the life cycle of the aggregate. Aggregates are represented by yellow sticky notes, and the commands and events associated with that aggregate are moved and clustered around the aggregate. This also helps you understand how aggregates are related to each other—events from one aggregate might trigger behavior in another.

Với các sự kiện và command đã được ghi lại, các aggregate đến tiếp theo. Các sự kiện mà bạn có ở giai đoạn này không chỉ hữu ích cho việc chia sẻ điều gì xảy ra trong hệ thống, mà chúng còn bắt đầu làm nổi bật những aggregate tiềm năng có thể là gì. Hãy nghĩ về sự kiện miền nghiệp vụ “Order Placed” đã nhắc đến. Danh từ ở đây—“Order”—rất có thể là một aggregate tiềm năng. Và “Placed” mô tả một điều có thể xảy ra với một đơn hàng, nên đây rất có thể là một phần trong vòng đời của aggregate. Các aggregate được đại diện bởi các tờ giấy nhớ màu vàng, và các command cùng sự kiện gắn với aggregate đó được di chuyển và gom cụm quanh aggregate. Điều này cũng giúp bạn hiểu các aggregate liên quan với nhau ra sao—các sự kiện từ một aggregate có thể kích hoạt hành vi ở một aggregate khác.

With the aggregates identified, they are grouped into bounded contexts. Bounded contexts most commonly follow a company's organizational structure, and the participants of the exercise are well placed to understand what aggregates are used by which parts of the organization.

Với các aggregate đã được nhận diện, chúng được nhóm thành các bounded context. Các bounded context phổ biến nhất thì đi theo cấu trúc tổ chức của công ty, và các thành viên của bài tập ở vị trí thích hợp để hiểu những aggregate nào được dùng bởi những phần nào của tổ chức.

There is more to event storming than what I've just described—this was just meant as a brief overview. For a more detailed look at event storming, I'd suggest you read the (currently in progress) book *EventStorming* by Alberto Brandolini (Leanpub). 16

Event storming còn nhiều thứ hơn những gì tôi vừa mô tả—đây chỉ là một bản tổng quan ngắn gọn. Để có cái nhìn chi tiết hơn về event storming, tôi khuyên bạn nên đọc cuốn (hiện đang viết dở) *EventStorming* của Alberto Brandolini (Leanpub). 16

The Case for Domain-Driven Design for Microservices — Lập luận cho Domain-Driven Design dành cho Microservice

We've explored how DDD can work in the context of microservices, so let's summarize how this approach is useful to us.

Ta đã khám phá cách DDD có thể vận hành trong ngữ cảnh microservice, vậy hãy tóm tắt cách tiếp cận này hữu ích với ta ra sao.

Firstly, a big part of what makes DDD so powerful is that bounded contexts, which are so important to DDD, are explicitly about hiding information—presenting a clear boundary to the wider system while hiding internal complexity that is able to change without impacting other parts of the system. This means that when we follow a DDD approach, whether we realize it or not, we are also adopting information hiding—and as we've seen, this is vital in helping to find stable microservice boundaries.

Thứ nhất, một phần lớn điều khiến DDD mạnh mẽ đến vậy là các bounded context, vốn rất quan trọng đối với DDD, một cách tường minh nói về việc giấu thông tin—trình ra một ranh giới rõ ràng cho hệ thống rộng hơn trong khi giấu đi độ phức tạp nội bộ vốn có thể thay đổi mà không ảnh hưởng đến các phần khác của hệ thống. Điều này nghĩa là khi ta theo một cách tiếp cận DDD, dù ta có nhận ra hay không, ta cũng đang áp dụng che giấu thông tin—và như ta đã thấy, điều này là thiết yếu trong việc giúp tìm ra những ranh giới microservice ổn định.

16 Alberto Brandolini, *EventStorming* (Victoria, BC: Leanpub, forthcoming).

16 Alberto Brandolini, *EventStorming* (Victoria, BC: Leanpub, forthcoming).

Secondly, the focus on defining a common, ubiquitous language helps greatly when it comes to defining microservice endpoints. It neatly gives us a shared vocabulary to draw on when coming up with APIs, event formats, and the like. It also helps solve the problem of how far the standardization of APIs needs to go in terms of allowing language to change within bounded contexts—change inside a boundary impacting that boundary itself.

Thứ hai, việc tập trung vào định nghĩa một ngôn ngữ chung, ubiquitous giúp ích rất nhiều khi cần định nghĩa các endpoint của microservice. Nó cho ta một vốn từ vựng chung gọn ghẽ để dựa vào khi nghĩ ra các API, định dạng sự kiện, và những thứ tương tự. Nó cũng giúp giải quyết vấn đề chuẩn hóa API cần đi xa đến đâu xét về mặt cho phép ngôn ngữ thay đổi bên trong các bounded context—thay đổi bên trong một ranh giới chỉ tác động đến chính ranh giới đó.

The changes we implement to our system are often about changes the business wants to make in how the system behaves. We are changing functionality—capabilities—that are exposed to our customers. If our systems are decomposed along the bounded contexts that represent our domain, any changes we want to make are more likely to be isolated to a single microservice boundary. This reduces the number of places we need to make a change and allows us to deploy that change quickly.

Những thay đổi mà ta triển khai cho hệ thống của mình thường là về những thay đổi mà doanh nghiệp muốn thực hiện trong cách hệ thống hành xử. Ta đang thay đổi chức năng—các năng lực—được phơi bày cho khách hàng của ta. Nếu các hệ thống của ta được phân rã dọc theo các bounded context đại diện cho miền nghiệp vụ của ta, bất kỳ thay đổi nào ta muốn thực hiện đều có nhiều khả năng được cô lập trong một ranh giới microservice đơn lẻ. Điều này giảm số lượng nơi ta cần thực hiện thay đổi và cho phép ta triển khai thay đổi đó nhanh chóng.

Fundamentally, DDD puts the business domain at the heart of the software we are building. The encouragement that it gives us to pull the language of the business into our code and service design helps improve domain expertise among the people who build the software. This in turn helps build understanding and empathy for the users of our software and builds greater communication among technical delivery, product development, and the end users. If you are interested in moving toward stream-aligned teams, DDD fits in neatly as a mechanism to help align the technical architecture with the wider organizational structure. In a world in which we are increasingly trying to tear down the silos between IT and “the business,” this is no bad thing.

Về cơ bản, DDD đặt miền nghiệp vụ vào trung tâm của phần mềm mà ta đang xây dựng. Sự khích lệ mà nó dành cho ta để kéo ngôn ngữ của doanh nghiệp vào trong mã và thiết kế service của ta giúp cải thiện chuyên môn về miền nghiệp vụ trong những người xây dựng phần mềm. Điều này đến lượt nó giúp xây dựng sự thấu hiểu và đồng cảm với người dùng phần mềm của ta và xây dựng sự giao tiếp tốt hơn giữa đội triển khai kỹ thuật, đội phát triển sản phẩm, và người dùng cuối. Nếu bạn quan tâm đến việc tiến tới các stream-aligned team, DDD ăn khớp gọn ghẽ như một cơ chế giúp căn chỉnh kiến trúc kỹ thuật với cấu trúc tổ chức rộng hơn. Trong một thế giới mà ta ngày càng cố phá bỏ những bức tường ngăn cách giữa IT và “doanh nghiệp,” đây không phải điều gì xấu.

Alternatives to Business Domain Boundaries — Các phương án thay thế cho ranh giới theo miền nghiệp vụ

As I’ve outlined, DDD can be incredibly useful when building microservice architectures, but it would be a mistake to think that this is the only technique you should consider when finding microservice boundaries. In fact, I often use multiple methods in conjunction with DDD to help identify how (and if) a system should be split. Let’s look at some of the other factors we might consider when finding boundaries.

Như tôi đã trình bày, DDD có thể cực kỳ hữu ích khi xây dựng các kiến trúc microservice, nhưng sẽ là một sai lầm nếu nghĩ rằng đây là kỹ thuật duy nhất bạn nên cân nhắc khi tìm ranh giới microservice. Thực tế, tôi thường dùng nhiều phương pháp kết hợp với DDD để giúp nhận diện một hệ thống nên (và có nên hay không) được chia ra sao. Hãy xem một số yếu tố khác mà ta có thể cân nhắc khi tìm ranh giới.

Volatility — Tính dễ biến động (Volatility)

I’ve increasingly heard of pushback against domain-oriented decomposition, often by advocates for volatility being the primary driver for decomposition. Volatility-based decomposition has you identify the parts of your system going through more frequent change and then extract that functionality into their own services, where they can be more effectively worked on. Conceptually, I don’t have a problem with this, but promoting it as the only way to do things isn’t helpful, especially when we consider the different drivers that might be pushing us toward microservices. If my biggest

issue is related to the need to scale my application, for example, a volatility-based decomposition is unlikely to deliver much of a benefit.

Tôi ngày càng nghe nhiều sự phản đối đối với việc phân rã theo định hướng miền nghiệp vụ, thường là từ những người ủng hộ tính dễ biến động (volatility) như yếu tố thúc đẩy chính cho việc phân rã. Phân rã dựa trên tính dễ biến động khiến bạn nhận diện những phần trong hệ thống đang trải qua thay đổi thường xuyên hơn rồi trích xuất chức năng đó ra thành các service riêng, nơi chúng có thể được làm việc hiệu quả hơn. Về mặt khái niệm, tôi không có vấn đề gì với điều này, nhưng việc cố xúi nó như cách duy nhất để làm mọi thứ thì không có ích, đặc biệt khi ta cân nhắc những động lực khác nhau có thể đang đẩy ta về phía microservice. Ví dụ, nếu vấn đề lớn nhất của tôi liên quan đến nhu cầu mở rộng ứng dụng, thì một cách phân rã dựa trên tính dễ biến động khó lòng mang lại nhiều lợi ích.

The mindset behind volatility-based decomposition is also evident in approaches like bimodal IT. A concept put forward by Gartner, bimodal IT neatly breaks the world down into the snappily named “Mode 1” (aka Systems of Record) and “Mode 2” (aka Systems of Innovation) categories based on how fast (or slow) different systems need to go. Mode 1 systems, we are told, don’t change much and don’t need much business involvement. Mode 2 is where the action is, with systems that need to change quickly and that require close involvement from the business. Putting aside for one moment the drastic oversimplification inherent in such a categorization scheme, it also implies a very fixed view of the world, and belies the sorts of transformations that are evident across industry as companies look to “go digital.” Parts of companies’ systems that didn’t need to change much in the past suddenly do, in order to open up new market opportunities and provide services to their customers in ways that they previously didn’t imagine.

Tư duy đằng sau việc phân rã dựa trên tính dễ biến động cũng thể hiện rõ trong những cách tiếp cận như bimodal IT. Một khái niệm do Gartner đưa ra, bimodal IT chia gọn thế giới thành hai phạm trù với cái tên giật gân “Mode 1” (còn gọi là Systems of Record, Hệ thống lưu trữ hồ sơ) và “Mode 2” (còn gọi là Systems of Innovation, Hệ thống đổi mới) dựa trên việc các hệ thống khác nhau cần đi nhanh (hay chậm) ra sao. Các hệ thống Mode 1, ta được bảo, không thay đổi nhiều và không cần nhiều sự tham gia của doanh nghiệp. Mode 2 là nơi có hành động, với những hệ thống cần thay đổi nhanh và đòi hỏi sự tham gia sát sao của doanh nghiệp. Tạm gác lại trong giây lát sự đơn giản hóa đến mức thô bạo vốn có trong một sơ đồ phân loại như vậy, nó cũng ngụ ý một cái nhìn rất cứng nhắc về thế giới, và che lấp đi những kiểu chuyển đổi đang diễn ra rõ rệt khắp các ngành khi các công ty tìm cách “go digital” (chuyển đổi số). Những phần trong hệ thống của các công ty mà trước kia không cần thay đổi nhiều bỗng nhiên lại cần, để mở ra những cơ hội thị trường mới và cung cấp dịch vụ cho khách hàng của họ theo những cách mà trước đây họ chưa từng hình dung.

Let’s come back to MusicCorp. Its first foray into what we now call digital was just having a web page; all it offered back in the mid-nineties was a listing of what was for sale, but you had to phone MusicCorp to place the order. It was little more than an advert in a newspaper. Then online ordering became a thing, and the entire warehouse, which up until that point had just been handled with paper, had to be digitized. Who knows—perhaps MusicCorp will at some stage have to consider making music available digitally! Although you might consider that MusicCorp is behind the times, you can still appreciate the amount of upheaval that companies have been going through as they understand how changing technology and customer behavior can require significant changes in parts of a business that couldn’t easily be foreseen.

Hãy quay lại với MusicCorp. Bước thử nghiệm đầu tiên của nó vào cái mà giờ ta gọi là kỹ thuật số chỉ đơn giản là có một trang web; tất cả những gì nó cung cấp hồi giữa thập

niên chín mươi là một danh mục những thứ đang được bán, nhưng bạn phải gọi điện cho MusicCorp để đặt hàng. Nó chẳng khác gì một mẫu quảng cáo trên báo. Rồi đặt hàng trực tuyến trở thành một thứ phổ biến, và toàn bộ nhà kho, vốn cho đến thời điểm đó chỉ được xử lý bằng giấy tờ, phải được số hóa. Ai mà biết được—có lẽ đến một lúc nào đó MusicCorp sẽ phải cân nhắc cung cấp âm nhạc dưới dạng số! Mặc dù bạn có thể cho rằng MusicCorp đang lạc hậu so với thời đại, bạn vẫn có thể đánh giá đúng mức độ xáo trộn mà các công ty đã trải qua khi họ nhận ra công nghệ thay đổi và hành vi khách hàng có thể đòi hỏi những thay đổi đáng kể ở những phần của một doanh nghiệp mà không dễ gì lường trước được.

I dislike bimodal IT as a concept, as it becomes a way for people to dump stuff that is hard to change into a nice neat box and say “we don’t need to deal with the issues in there—that’s Mode 1.” It’s yet another model that a company can adopt to ensure that nothing actually has to change. It also avoids the fact that quite often changes in functionality require changes in “Systems of Record” (Mode 1) to allow for changes in “Systems of Innovation” (Mode 2). In my experience, organizations adopting bimodal IT do end up having two speeds—slow and slower.

Tôi không thích bimodal IT như một khái niệm, vì nó trở thành một cách để người ta dồn những thứ khó thay đổi vào một cái hộp gọn gàng ngăn nắp rồi nói “chúng ta không cần xử lý các vấn đề trong đó—đó là Mode 1.” Nó lại là thêm một mô hình mà một công ty có thể áp dụng để đảm bảo rằng thực sự chẳng có gì phải thay đổi cả. Nó cũng né tránh cái thực tế rằng khá thường xuyên những thay đổi về chức năng đòi hỏi những thay đổi trong “Systems of Record” (Mode 1) để cho phép những thay đổi trong “Systems of Innovation” (Mode 2). Theo kinh nghiệm của tôi, các tổ chức áp dụng bimodal IT rất cuộc có hai tốc độ—chậm và chậm hơn.

To be fair to proponents of volatility-based decomposition, many of them aren’t necessarily recommending such simplistic models as bimodal IT. In fact, I find this technique to be highly useful in helping to determine boundaries if the main driver is about fast time to market—extracting functionality that is changing or needs to change frequently makes perfect sense in such a situation. But again, the goal determines the most appropriate mechanism.

Để công bằng với những người ủng hộ phân rã dựa trên tính dễ biến động, nhiều người trong số họ không nhất thiết khuyến nghị những mô hình đơn giản hóa kiểu như bimodal IT. Thực tế, tôi thấy kỹ thuật này cực kỳ hữu ích trong việc giúp xác định ranh giới nếu động lực chính là về thời gian đưa ra thị trường nhanh—trích xuất chức năng đang thay đổi hoặc cần thay đổi thường xuyên là hoàn toàn hợp lý trong một tình huống như vậy. Nhưng một lần nữa, mục tiêu quyết định cơ chế phù hợp nhất.

Data — Dữ liệu

The nature of the data you hold and manage can drive you toward different forms of decomposition. For example, you might want to limit which services handle personally identifiable information (PII), both to reduce your risk of data breaches and to simplify oversight and implementation of things like GDPR.

Bản chất của dữ liệu mà bạn nắm giữ và quản lý có thể đẩy bạn về những hình thức phân rã khác nhau. Ví dụ, bạn có thể muốn giới hạn những service nào xử lý thông tin định danh cá nhân (personally identifiable information, PII), vừa để giảm rủi ro rò rỉ dữ liệu vừa để đơn giản hóa việc giám sát và triển khai những thứ như GDPR.

For one of my recent clients, a payment company we’ll call PaymentCo, the use of certain types of data directly influenced the decisions we made about system decomposition. PaymentCo handles

credit card data, which means that its system needs to comply with various requirements set down by Payment Card Industry (PCI) standards for how this data needs to be managed. As part of this compliance, the company's system and processes needed to be audited. PaymentCo had a need to handle the full credit card data, and at a volume that meant its system had to comply with PCI Level 1, which is the most stringent level and which requires quarterly external assessment of the systems and practices related to how the data is managed.

Với một trong những khách hàng gần đây của tôi, một công ty thanh toán mà ta sẽ gọi là PaymentCo, việc sử dụng một số loại dữ liệu nhất định đã trực tiếp ảnh hưởng đến các quyết định mà chúng tôi đưa ra về việc phân rã hệ thống. PaymentCo xử lý dữ liệu thẻ tín dụng, nghĩa là hệ thống của nó cần tuân thủ nhiều yêu cầu khác nhau do các chuẩn của Payment Card Industry (PCI) đặt ra về cách dữ liệu này cần được quản lý. Là một phần của việc tuân thủ này, hệ thống và các quy trình của công ty cần được kiểm toán. PaymentCo có nhu cầu xử lý đầy đủ dữ liệu thẻ tín dụng, và ở một mức khối lượng khiến hệ thống của nó phải tuân thủ PCI Level 1, vốn là mức nghiêm ngặt nhất và đòi hỏi đánh giá độc lập bên ngoài hàng quý đối với các hệ thống và thực hành liên quan đến cách dữ liệu được quản lý.

Many of the PCI requirements are common sense, but ensuring that the whole system complied with these requirements, not least the need for the system to be audited by an external party, was proving to be quite onerous. As a result, the company wanted to split out the part of the system that handled the full credit card data— meaning that only a subset of the system required this additional level of oversight. In Figure 2-17, we see a simplified form of the design we came up with. Services operating in the green zone (enclosed by a dotted green line) never see any credit card information—that data is limited to processes (and networks) in the red zone (surrounded by red dashes). The gateway diverts calls to the appropriate services (and the appropriate zone); as the credit card information passes through this gateway, it is in effect also in the red zone.

Nhiều yêu cầu PCI mang tính lẽ thường, nhưng việc đảm bảo toàn bộ hệ thống tuân thủ những yêu cầu này, đặc biệt là nhu cầu hệ thống phải được một bên ngoài kiểm toán, hóa ra là khá nặng nề. Kết quả là, công ty muốn tách riêng phần hệ thống xử lý đầy đủ dữ liệu thẻ tín dụng ra—nghĩa là chỉ một tập con của hệ thống cần mức giám sát bổ sung này. Ở Hình 2-17, ta thấy một dạng đơn giản hóa của thiết kế mà chúng tôi đã nghĩ ra. Các service vận hành trong vùng xanh lá (được bao bởi một đường chấm xanh lá) không bao giờ nhìn thấy bất kỳ thông tin thẻ tín dụng nào—dữ liệu đó bị giới hạn trong các tiến trình (và mạng) ở vùng đỏ (được bao quanh bởi các vạch đứt màu đỏ). Gateway chuyển hướng các lời gọi tới các service phù hợp (và vùng phù hợp); khi thông tin thẻ tín dụng đi qua gateway này, trên thực tế nó cũng nằm trong vùng đỏ.

As credit card information never flows into the green zone, all services in this area can be exempted from a full PCI audit. Services in the red zone are in scope for such oversight. When working through the design, we did everything we could to limit what has to be in this red zone. It's key to note that we had to make sure that the credit card information never flows to the green zone at all—if a microservice in the green zone could request this information, or if that information could be sent back to the green zone by a microservice in the red zone, then the clear lines of separation would break down.

Vì thông tin thẻ tín dụng không bao giờ chảy vào vùng xanh lá, tất cả các service trong vùng này có thể được miễn khỏi một cuộc kiểm toán PCI đầy đủ. Các service ở vùng đỏ nằm trong phạm vi của sự giám sát như vậy. Khi làm việc qua thiết kế này, chúng tôi đã làm mọi thứ có thể để giới hạn những gì phải nằm trong vùng đỏ này. Điều then chốt cần lưu ý là chúng tôi phải đảm bảo rằng thông tin thẻ tín dụng không bao giờ chảy sang vùng xanh lá—nếu một microservice ở vùng xanh lá có thể yêu cầu thông tin này, hoặc nếu

thông tin đó có thể được gửi ngược lại vùng xanh lá bởi một microservice ở vùng đỏ, thì các lần ranh phân tách rõ ràng sẽ sụp đổ.

Segregation of data is often driven by a variety of privacy and **security** concerns; we'll come back to this topic and the example of PaymentCo in Chapter 11 .

Việc phân tách dữ liệu thường được thúc đẩy bởi nhiều mối quan tâm khác nhau về quyền riêng tư và bảo mật; ta sẽ quay lại chủ đề này và ví dụ về PaymentCo ở Chương 11.

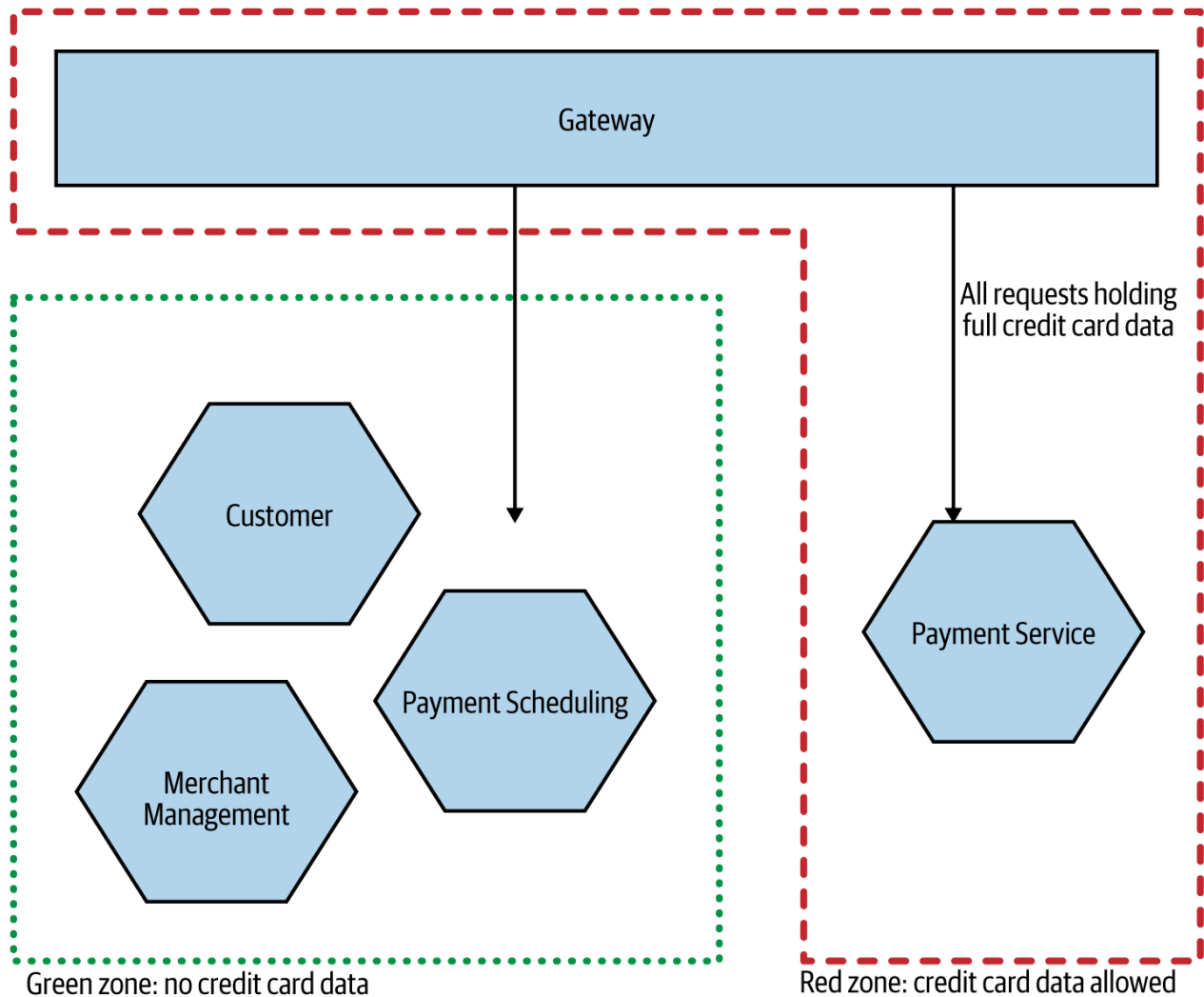


Figure 2-17. PaymentCo, which segregates processes based on its use of credit card information to limit the scope of PCI requirements

Hình 2-17. PaymentCo, công ty phân tách các tiến trình dựa trên việc sử dụng thông tin thẻ tín dụng để giới hạn phạm vi của các yêu cầu PCI

Technology — Công nghệ

The need to make use of different technology can also be a factor in terms of finding a boundary. You can accommodate different databases in a single running microservice, but if you want to mix different runtime models, you may face a challenge. If you determine that part of your functionality

needs to be implemented in a runtime like Rust, which enables you to eke out additional performance improvements, that ends up being a major forcing factor.

Nhu cầu sử dụng công nghệ khác nhau cũng có thể là một yếu tố xét về mặt tìm một ranh giới. Bạn có thể chứa các cơ sở dữ liệu khác nhau trong một microservice đang chạy duy nhất, nhưng nếu bạn muốn pha trộn các mô hình runtime khác nhau, bạn có thể đối mặt với một thách thức. Nếu bạn xác định rằng một phần chức năng của mình cần được triển khai trong một runtime như Rust, vốn cho phép bạn vắt ra thêm các cải thiện hiệu năng, thì điều đó rất cuộc trở thành một yếu tố thúc đẩy chủ đạo.

Of course, we have to be aware of where this can drive us if adopted as a general means of decomposition. The classic three-tiered architecture that we discussed in the opening chapter, and that we show again in Figure 2-18 , is an example of related technology being grouped together. As we’ve already explored, this is often a less than ideal architecture.

Tất nhiên, ta phải nhận thức được điều này có thể đẩy ta tới đâu nếu được áp dụng như một phương tiện phân rã chung. Kiến trúc ba tầng kinh điển mà ta đã bàn ở chương mở đầu, và mà ta lại trình bày ở Hình 2-18, là một ví dụ về công nghệ liên quan được nhóm lại với nhau. Như ta đã khám phá, đây thường là một kiến trúc kém lý tưởng.

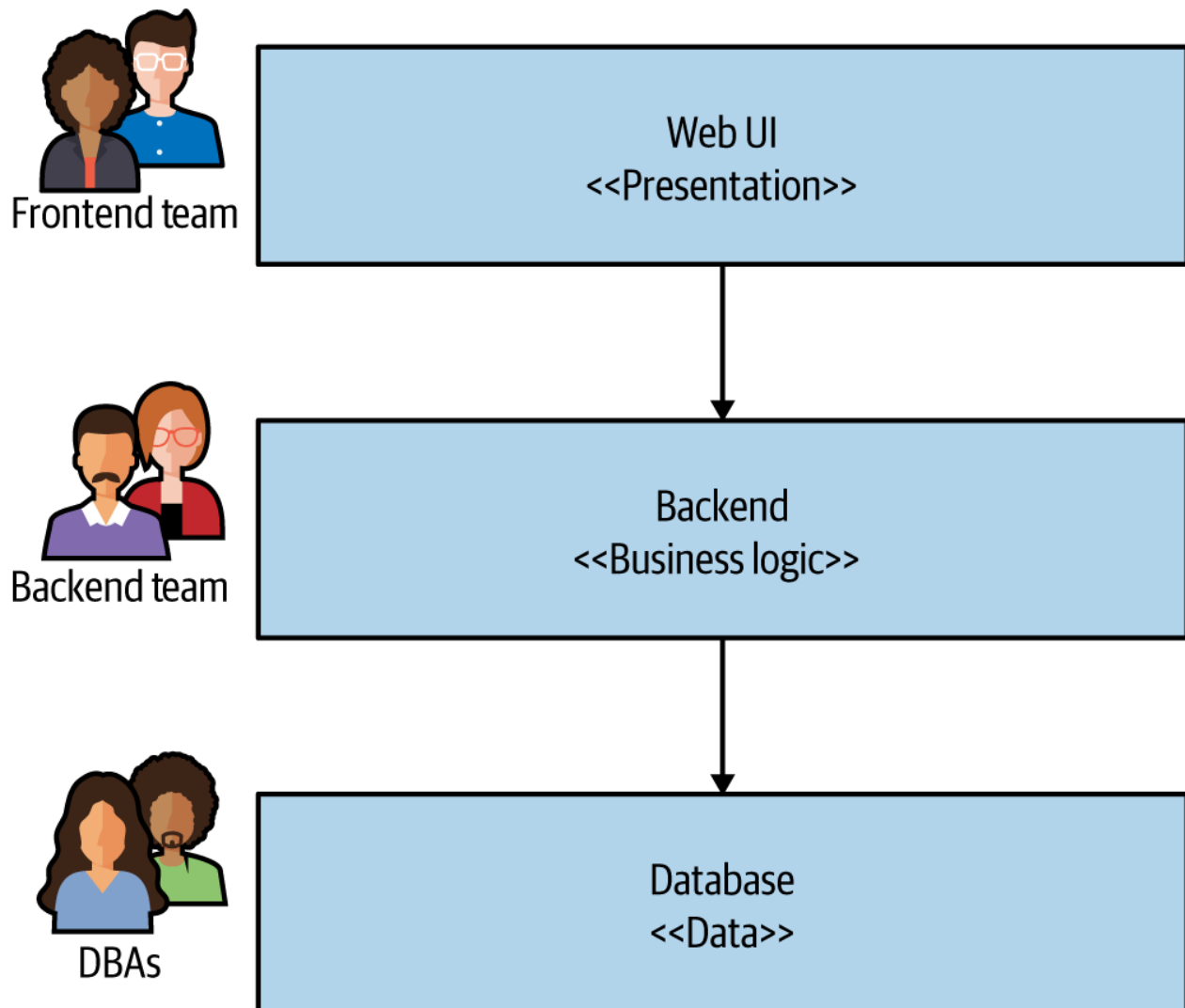


Figure 2-18. A traditional three-tiered architecture is often driven by technological boundaries

Hình 2-18. Một kiến trúc ba tầng truyền thống thường bị thúc đẩy bởi các ranh giới công nghệ

Organizational — Tổ chức

As we established when I introduced Conway's law back in Chapter 1 , there is an inherent interplay between organizational structure and the **system architecture** you end up with. Quite aside from the studies that have shown this link, in my own anecdotal experience I have seen this play out time and time again. How you organize yourself ends up driving your systems architecture, for good or for ill. When it comes to helping us define our service boundaries, we have to consider this as a key part of our decision making.

Như ta đã xác lập khi tôi giới thiệu Định luật Conway ở Chương 1, có một sự tương tác cố hữu giữa cấu trúc tổ chức và kiến trúc hệ thống mà bạn rốt cuộc có được. Hoàn toàn tách biệt khỏi các nghiên cứu đã cho thấy mối liên hệ này, trong kinh nghiệm giai thoại của riêng tôi, tôi đã thấy điều này diễn ra hết lần này đến lần khác. Cách bạn tự tổ chức rốt cuộc lại thúc đẩy kiến trúc hệ thống của bạn, dù tốt hay xấu. Khi cần giúp ta định nghĩa các ranh giới service, ta phải cân nhắc điều này như một phần then chốt trong việc ra quyết định của mình.

Defining a service boundary whose ownership would cut across multiple different teams is unlikely to yield the outcomes we would desire—as we'll explore further in Chapter 15 , shared ownership of microservices is a fraught affair. It therefore follows that we must take into account the existing organizational structure when considering where and when to define boundaries, and in some situations we should perhaps even consider changing the organizational structure to support the architecture we want.

Việc định nghĩa một ranh giới service mà quyền sở hữu của nó sẽ cắt ngang qua nhiều đội khác nhau khó lòng mang lại những kết quả mà ta mong muốn—như ta sẽ khám phá thêm ở Chương 15, quyền sở hữu chung của các microservice là một chuyện đầy rủi ro. Do đó suy ra rằng ta phải tính đến cấu trúc tổ chức hiện hữu khi cân nhắc nên định nghĩa ranh giới ở đâu và khi nào, và trong một số tình huống ta có lẽ thậm chí nên cân nhắc thay đổi cấu trúc tổ chức để hỗ trợ kiến trúc mà ta mong muốn.

Of course, we also have to consider what happens if our organizational structure changes too. Does that mean we now have to rearchitect our software? Well, in the worst case, it might cause us to examine an existing microservice that now needs to be split, as it contains functionality that now may be owned by two separate teams, whereas before a single team was responsible for both parts. On the other hand, often organizational changes would just require that the owner of an existing microservice changes. Consider a situation in which the team in charge of warehousing operations previously also handled functionality around working out how many items should be ordered from suppliers. Let's say that we decide to move this responsibility to a dedicated forecasting team that wants to pull information from current sales and planned promotions to work out what needs to be ordered. If the warehousing team had a dedicated Supplier Ordering microservice, this could just be moved to the new forecasting team. On the other hand, if this functionality was previously integrated into a larger-scoped system owned by warehousing, then it might need to be split out.

Tất nhiên, ta cũng phải cân nhắc điều gì xảy ra nếu cấu trúc tổ chức của ta cũng thay đổi. Điều đó có nghĩa là giờ ta phải tái kiến trúc phần mềm của mình không? À, trong trường hợp xấu nhất, nó có thể khiến ta phải xem xét một microservice hiện hữu giờ cần được

chia ra, vì nó chứa chức năng mà giờ có thể được sở hữu bởi hai đội riêng biệt, trong khi trước đây một đội duy nhất chịu trách nhiệm cho cả hai phần. Mặt khác, thường thì những thay đổi về tổ chức chỉ đòi hỏi rằng chủ sở hữu của một microservice hiện hữu thay đổi. Hãy xét một tình huống trong đó đội phụ trách hoạt động kho bãi trước đây cũng xử lý chức năng quanh việc tính toán nên đặt bao nhiêu mặt hàng từ nhà cung cấp. Giả sử ta quyết định chuyển trách nhiệm này cho một đội dự báo chuyên trách, đội này muốn lấy thông tin từ doanh số hiện tại và các chương trình khuyến mãi đã lên kế hoạch để tính ra cái gì cần được đặt. Nếu đội kho bãi có một microservice Supplier Ordering chuyên trách, cái này có thể chỉ cần được chuyển sang đội dự báo mới. Mặt khác, nếu chức năng này trước đây được tích hợp vào một hệ thống có phạm vi lớn hơn do kho bãi sở hữu, thì nó có thể cần được tách ra.

Even when we work within an existing organizational structure, there is a danger that we won't get our boundaries in the right place. Many years ago, a few colleagues and I were working with a client in California, helping the company adopt some cleaner code practices and move more toward automated testing. We'd started with some of the low-hanging fruit, such as service decomposition, when we noticed something much more worrying. I can't go into too much detail as to what the application did, but it was a public-facing application with a large, global customer base.

Ngay cả khi ta làm việc trong một cấu trúc tổ chức hiện hữu, vẫn có nguy cơ ta không đặt ranh giới của mình đúng chỗ. Nhiều năm trước, một vài đồng nghiệp và tôi làm việc với một khách hàng ở California, giúp công ty áp dụng một số thực hành viết mã sạch hơn và tiến nhiều hơn về phía kiểm thử tự động. Chúng tôi đã bắt đầu với một số trái cây dễ hái, như phân rã service, thì chúng tôi nhận ra một thứ đáng lo ngại hơn nhiều. Tôi không thể đi vào quá nhiều chi tiết về việc ứng dụng đó làm gì, nhưng nó là một ứng dụng hướng ra công chúng với một lượng khách hàng lớn, toàn cầu.

The team, and system, had grown. Originally one person's vision, the system had taken on more and more features, and more and more users. Eventually, the organization decided to increase the capacity of the team by having a new group of developers based in Brazil take on some of the work. The system got split up, with the front half of the application being essentially stateless, implementing the public-facing website, as shown in Figure 2-19. The back half of the system was simply a **remote procedure call (RPC)** interface over a data store. Essentially, imagine you'd taken a **repository** layer in your codebase and made this a separate service.

Đội ngũ, và hệ thống, đã phát triển lên. Vốn ban đầu là tầm nhìn của một người, hệ thống đã nhận thêm ngày càng nhiều tính năng, và ngày càng nhiều người dùng. Cuối cùng, tổ chức quyết định tăng năng lực của đội bằng cách để một nhóm lập trình viên mới đặt tại Brazil đảm nhận một phần công việc. Hệ thống bị chia ra, với nửa trước của ứng dụng về cơ bản là không trạng thái (stateless), triển khai trang web hướng ra công chúng, như trình bày ở Hình 2-19. Nửa sau của hệ thống chỉ đơn giản là một giao diện lời gọi thủ tục từ xa (RPC) đặt trên một kho dữ liệu. Về cơ bản, hãy hình dung bạn đã lấy một lớp repository trong codebase của mình và biến nó thành một service riêng.

Changes frequently had to be made to both services. Both services spoke in terms of low-level, RPC-style method calls, which were overly brittle (we'll discuss this further in Chapter 4). The service interface was very chatty as well, resulting in performance issues. This led to the need for elaborate RPC-batching mechanisms. I called this "onion architecture," as it had lots of layers and made me cry when we had to cut through it.

Các thay đổi thường xuyên phải được thực hiện trên cả hai service. Cả hai service đều nói chuyện bằng những lời gọi phương thức ở cấp thấp, kiểu RPC, vốn quá mong manh dễ vỡ (ta sẽ bàn thêm về điều này ở Chương 4). Giao diện service cũng rất lảm nhảm (chatty),

dẫn đến các vấn đề về hiệu năng. Điều này dẫn đến nhu cầu có những cơ chế gom mẻ (batching) RPC phức tạp. Tôi gọi đây là “kiến trúc củ hành,” vì nó có rất nhiều lớp và làm tôi muốn khóc mỗi khi phải cắt xuyên qua nó.

Now on the face of it, the idea of splitting the previously monolithic system along geographical/organizational lines makes perfect sense, as we’ll expand on in Chapter 15 . Here, however, rather than taking a vertical, business-focused slice through the stack, the team picked what was previously an in-process API and made a horizontal slice. A better model would have been for the team in California to have one end-to-end vertical slice, consisting of the related parts of the **frontend** and data access functionality, with the team in Brazil taking another slice.

Giờ thoát nhìn, ý tưởng chia hệ thống vốn nguyên khối trước đây dọc theo các lần ranh địa lý/tổ chức nghe rất hợp lý, như ta sẽ mở rộng ở Chương 15. Tuy nhiên, ở đây, thay vì lấy một lát cắt dọc, hướng nghiệp vụ xuyên qua tầng (stack), đội đã chọn cái trước đây là một API trong-tiến-trình (in-process) và cắt một lát ngang. Một mô hình tốt hơn hẳn là để đội ở California có một lát cắt dọc end-to-end, gồm các phần liên quan của frontend và chức năng truy cập dữ liệu, với đội ở Brazil lấy một lát khác.

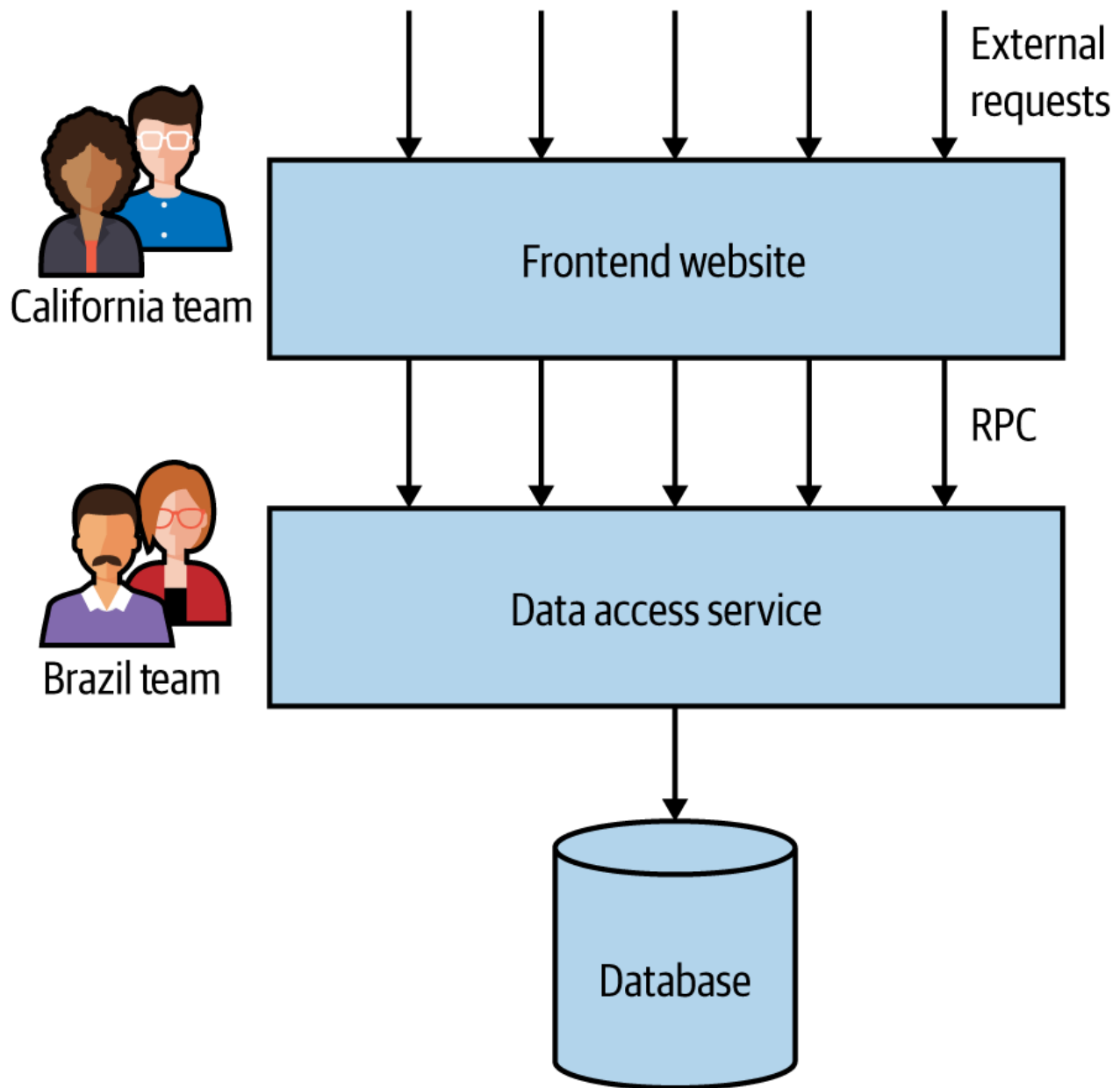


Figure 2-19. A service boundary split across technical seams

Hình 2-19. Một ranh giới service bị chia cắt dọc theo các đường nối kỹ thuật

Layering Inside Versus Layering Outside — Phân lớp bên trong so với Phân lớp bên ngoài

As I hope you can see by now, I'm not a fan of horizontally layered architecture. Layering can have its place, though. Within a microservice boundary, it can be totally sensible to delineate between different layers to make the code easier to manage. The problem occurs when this layering becomes the mechanism by which your microservice and ownership boundaries are drawn.

Như tôi hy vọng bạn có thể thấy lúc này, tôi không phải là người hâm mộ kiến trúc phân

lớp theo chiều ngang. Tuy nhiên, phân lớp vẫn có chỗ đứng của nó. Bên trong một ranh giới microservice, việc phân định giữa các lớp khác nhau để giúp mã dễ quản lý hơn có thể hoàn toàn hợp lý. Vấn đề xảy ra khi việc phân lớp này trở thành cơ chế mà theo đó các ranh giới microservice và ranh giới quyền sở hữu của bạn được vẽ ra.

Mixing Models and Exceptions — Pha trộn các mô hình và những ngoại lệ

As I hope is clear so far, I am not dogmatic in terms of how you find these boundaries. If you follow the guidelines of information hiding and appreciate the interplay of coupling and cohesion, then chances are you'll avoid some of the worst pitfalls of whatever mechanism you pick. I happen to think that by focusing on these ideas you are more likely to end up with a domain-oriented architecture, but that is by the by. The fact is, though, that there can often be reasons to mix models, even if “domain-oriented” is what you decide to pick as your main mechanism for defining microservice boundaries.

Như tôi hy vọng đã rõ cho đến giờ, tôi không giáo điều về cách bạn tìm ra những ranh giới này. Nếu bạn tuân theo các nguyên tắc che giấu thông tin và trân trọng sự tương tác giữa coupling và cohesion, thì nhiều khả năng bạn sẽ tránh được một số cạm bẫy tệ nhất của bất kỳ cơ chế nào bạn chọn. Tôi tình cờ nghĩ rằng bằng cách tập trung vào những ý tưởng này bạn nhiều khả năng rất cuộc sẽ có một kiến trúc định hướng theo miền nghiệp vụ, nhưng đó là chuyện bên lề. Tuy nhiên, sự thật là, thường có thể có những lý do để pha trộn các mô hình, ngay cả khi “định hướng theo miền nghiệp vụ” là thứ bạn quyết định chọn làm cơ chế chính để định nghĩa các ranh giới microservice.

The different mechanisms we've outlined so far also have a lot of potential interplay between them. Being too narrow in your choices here will cause you to follow the dogma rather than doing the right thing. Volatility-based decomposition can make a lot of sense if your focus is on improving the speed of delivery, but if this causes you to extract a service that crosses organizational boundaries, then expect your pace of change to suffer due to delivery contention.

Các cơ chế khác nhau mà ta đã trình bày cho đến nay cũng có rất nhiều sự tương tác tiềm tàng giữa chúng. Việc quá hạn hẹp trong các lựa chọn của bạn ở đây sẽ khiến bạn đi theo giáo điều thay vì làm điều đúng đắn. Phân rã dựa trên tính dễ biến động có thể rất hợp lý nếu trọng tâm của bạn là cải thiện tốc độ chuyển giao, nhưng nếu điều này khiến bạn trích xuất một service cắt ngang qua các ranh giới tổ chức, thì hãy chờ đợi nhịp độ thay đổi của bạn sẽ chịu thiệt do tranh chấp trong chuyển giao.

I might define a nice Warehouse service based on my understanding of the business domain, but if one part of that system needs to be implemented in C++ and another part in Kotlin, then you'll have to decompose further along those technical lines.

Tôi có thể định nghĩa một service Warehouse đẹp đẽ dựa trên hiểu biết của tôi về miền nghiệp vụ, nhưng nếu một phần của hệ thống đó cần được triển khai bằng C++ và một phần khác bằng Kotlin, thì bạn sẽ phải phân rã thêm dọc theo các lần ranh kỹ thuật đó.

Organizational and domain-driven service boundaries are my own starting point. But that's just my default approach. Typically, a number of the factors I've outlined here come into play, and which ones influence your own decisions will be based on the problems you are trying to solve. You need to look at your own specific circumstances to determine what works best for you—and hopefully I've given you a few different options to consider. Just remember, if someone says “The only way to do this is X!” they are likely just selling you more dogma. You can do better than that.

Các ranh giới service theo tổ chức và theo domain-driven là điểm khởi đầu của riêng tôi. Nhưng đó chỉ là cách tiếp cận mặc định của tôi. Thông thường, một số yếu tố mà tôi đã trình bày ở đây sẽ xen vào, và việc yếu tố nào ảnh hưởng đến quyết định của riêng bạn sẽ dựa trên những vấn đề bạn đang cố giải quyết. Bạn cần nhìn vào hoàn cảnh cụ thể của riêng mình để xác định điều gì hiệu quả nhất với bạn—và hy vọng tôi đã trao cho bạn một vài phương án khác nhau để cân nhắc. Chỉ cần nhớ, nếu ai đó nói “Cách duy nhất để làm việc này là X!” thì họ nhiều khả năng chỉ đang bán cho bạn thêm giáo điều mà thôi. Bạn có thể làm tốt hơn thế.

With all that said, let’s dive deeper into the topic of domain modeling by exploring domain-driven design in a little more detail.

Với tất cả những điều đã nói, hãy đi sâu hơn vào chủ đề mô hình hóa miền nghiệp vụ bằng cách khám phá domain-driven design chi tiết hơn một chút.

Summary — Tóm tắt

In this chapter, you’ve learned a bit about what makes a good microservice boundary, and how to find seams in our problem space that give us the dual benefits of both low coupling and strong cohesion. Having a detailed understanding of our domain can be a vital tool in helping us find these seams, and by aligning our microservices to these boundaries we ensure that the resulting system has every chance of keeping those virtues intact. We’ve also gotten a hint about how we can subdivide our microservices further.

Trong chương này, bạn đã học được một chút về điều gì tạo nên một ranh giới microservice tốt, và cách tìm ra những đường nối trong không gian vấn đề của ta mang lại lợi ích kép vừa coupling thấp vừa cohesion mạnh. Có một hiểu biết chi tiết về miền nghiệp vụ của ta có thể là một công cụ thiết yếu trong việc giúp ta tìm ra những đường nối này, và bằng cách căn chỉnh các microservice của mình theo những ranh giới này ta đảm bảo rằng hệ thống thu được có mọi cơ hội để giữ những phẩm chất đó nguyên vẹn. Ta cũng đã có một gợi ý về cách ta có thể chia nhỏ các microservice của mình thêm nữa.

The ideas presented in Eric Evans’s Domain-Driven Design are very useful to us in finding sensible boundaries for our services, and I’ve just scratched the surface here— Eric’s book goes into much more detail. If you want to go deeper, I can recommend Vaughn Vernon’s book *Implementing Domain-Driven Design* 17 to help you understand the practicalities of this approach, while Vernon’s *Domain-Driven Design Distilled* 18 is a great condensed overview if you’re looking for something more brief.

Những ý tưởng được trình bày trong cuốn *Domain-Driven Design* của Eric Evans rất hữu ích với ta trong việc tìm những ranh giới hợp lý cho các service của ta, và tôi mới chỉ chạm tới bề mặt ở đây—cuốn sách của Eric đi vào chi tiết nhiều hơn nữa. Nếu bạn muốn đi sâu hơn, tôi có thể giới thiệu cuốn *Implementing Domain-Driven Design* 17 của Vaughn Vernon để giúp bạn hiểu các khía cạnh thực tiễn của cách tiếp cận này, trong khi cuốn *Domain-Driven Design Distilled* 18 của Vernon là một bản tổng quan cô đọng tuyệt vời nếu bạn đang tìm kiếm thứ gì đó ngắn gọn hơn.

Much of this chapter has described how we’d find the boundary for our microservices. But what happens if you already have a monolithic application and are looking to migrate toward a microservice architecture? That’s something we’ll explore in more detail in the next chapter.

Phần lớn chương này đã mô tả cách ta tìm ranh giới cho các microservice của mình. Nhưng điều gì xảy ra nếu bạn đã có một ứng dụng nguyên khối và đang tìm cách di trú về phía một kiến trúc microservice? Đó là điều mà ta sẽ khám phá chi tiết hơn ở chương tiếp theo.

17 Vaughn Vernon, Implementing Domain-Driven Design (Upper Saddle River, NJ: Addison-Wesley, 2013). 18 Vaughn Vernon, Domain-Driven Design Distilled (Boston: Addison-Wesley, 2016).

17 Vaughn Vernon, Implementing Domain-Driven Design (Upper Saddle River, NJ: Addison-Wesley, 2013). 18 Vaughn Vernon, Domain-Driven Design Distilled (Boston: Addison-Wesley, 2016).

CHAPTER 3 Splitting the Monolith —

Chương 3: Tách rời khỏi Monolith

Many of you reading this book likely don't have a blank slate on which to design your system, and even if you did, starting with **microservices** might not be a great idea, for reasons we explored in Chapter 1. Many of you will already have an existing system, perhaps some form of monolithic **architecture**, which you are looking to migrate to a **microservice** architecture.

Nhiều bạn đang đọc cuốn sách này có lẽ không có một “tờ giấy trắng” để thiết kế hệ thống của mình, và kể cả khi có, việc khởi đầu bằng microservices cũng chưa chắc là một ý tưởng hay, vì những lý do mà chúng ta đã khám phá ở Chương 1. Nhiều bạn sẽ đã có sẵn một hệ thống đang tồn tại, có lẽ là một dạng kiến trúc monolith (khối nguyên khối) nào đó, mà bạn đang muốn di trú sang kiến trúc microservice.

In this chapter I'll outline some first steps, patterns, and general tips to help you navigate the transition to a microservice architecture.

Trong chương này, tôi sẽ phác thảo một vài bước đầu tiên, các pattern (mẫu thiết kế) và những lời khuyên chung để giúp bạn định hướng quá trình chuyển đổi sang kiến trúc microservice.

Have a Goal — Hãy có một mục tiêu

Microservices are not the goal. You don't “win” by having microservices. Adopting a microservice architecture should be a conscious decision, one based on rational decision making. You should be thinking of migrating to a microservice architecture only if you can't find any easier way to move toward your end goal with your current architecture.

Microservices không phải là mục tiêu. Bạn không “chiến thắng” chỉ vì sở hữu microservices. Việc áp dụng kiến trúc microservice nên là một quyết định có ý thức, dựa trên sự cân nhắc hợp lý. Bạn chỉ nên nghĩ tới việc di trú sang kiến trúc microservice nếu bạn không thể tìm ra cách nào dễ dàng hơn để tiến tới mục tiêu cuối cùng của mình với kiến trúc hiện tại.

Without a clear understanding as to what you are trying to achieve, you could fall into the trap of confusing activity with outcome. I've seen teams obsessed with creating microservices without ever asking why. This is problematic in the extreme given the new sources of complexity that microservices can introduce.

Khi không hiểu rõ điều bạn đang cố gắng đạt được, bạn có thể rơi vào cái bẫy nhầm lẫn giữa hoạt động và kết quả. Tôi đã chứng kiến nhiều đội nhóm bị ám ảnh với việc tạo ra

microservices mà chẳng bao giờ hỏi tại sao. Điều này cực kỳ nguy hiểm, xét đến những nguồn phức tạp mới mà microservices có thể đưa vào.

Fixating on microservices rather than on the end goal also means you will likely stop thinking of other ways in which you might bring about the change you are looking for. For example, microservices can help you scale your system, but there are often a number of alternative **scaling** techniques that should be looked at first. Spinning up a few more copies of your existing monolithic system behind a load balancer may well help you scale your system much more effectively than going through a complex and lengthy decomposition to microservices.

Việc chăm chăm vào microservices thay vì vào mục tiêu cuối cùng cũng có nghĩa là bạn nhiều khả năng sẽ ngừng nghĩ tới những cách khác để tạo ra sự thay đổi mà bạn đang tìm kiếm. Ví dụ, microservices có thể giúp bạn mở rộng (scale) hệ thống, nhưng thường có một số kỹ thuật mở rộng thay thế khác mà bạn nên xem xét trước. Việc khởi chạy thêm vài bản sao của hệ thống monolithic hiện tại phía sau một bộ cân bằng tải (load balancer) rất có thể giúp bạn mở rộng hệ thống hiệu quả hơn nhiều so với việc trải qua một quá trình phân rã sang microservices vừa phức tạp vừa kéo dài.



Microservices aren't easy. Try the simple stuff first.

Microservices không hề dễ. Hãy thử những cách đơn giản trước đã.

Finally, without a clear goal, it becomes difficult to know where to start. Which microservice should you create first? Without an overarching understanding of what you are trying to achieve, you're

flying blind.

Cuối cùng, khi không có một mục tiêu rõ ràng, sẽ rất khó để biết nên bắt đầu từ đâu. Microservice nào nên được tạo ra đầu tiên? Khi không có một sự thấu hiểu bao quát về điều bạn đang cố gắng đạt được, bạn đang bay trong vô định.

So be clear about what change you are trying to achieve, and consider easier ways to achieve that end goal before considering microservices. If microservices really are the best way to move forward, then track your progress against that end goal and change course as necessary.

Vì vậy, hãy rõ ràng về sự thay đổi mà bạn đang cố gắng đạt được, và cân nhắc những cách dễ dàng hơn để đạt mục tiêu cuối cùng đó trước khi nghĩ tới microservices. Nếu microservices thực sự là cách tốt nhất để tiến lên, thì hãy theo dõi tiến độ của bạn so với mục tiêu cuối cùng đó và điều chỉnh hướng đi khi cần thiết.

Incremental Migration — Di trú từng bước

If you do a big-bang rewrite, the only thing you're guaranteed of is a big bang. —Martin Fowler

Nếu bạn làm một cuộc viết lại kiểu “big-bang” (làm tất cả cùng lúc), thì điều duy nhất bạn chắc chắn nhận được là một tiếng “bang” thật to. —Martin Fowler

If you get to the point of deciding that breaking apart your existing monolithic system is the right thing to do, I strongly advise you to chip away at the **monolith**, extracting a bit at a time. An incremental approach will help you learn about microservices as you go and will also limit the impact of getting something wrong (and you will get things wrong!). Think of our monolith as a block of marble. We could blow the whole thing up, but that rarely ends well. It makes much more sense to just chip away at it incrementally.

Nếu bạn đi tới điểm quyết định rằng việc tách rời hệ thống monolithic hiện tại là việc đúng đắn nên làm, tôi mạnh mẽ khuyên bạn hãy gọt đẽo khối monolith từng chút một, mỗi lần trích xuất một phần nhỏ. Cách tiếp cận tăng dần (incremental) sẽ giúp bạn học hỏi về microservices trong quá trình thực hiện, và cũng giới hạn được tác động khi làm sai điều gì đó (và bạn chắc chắn sẽ làm sai!). Hãy hình dung khối monolith của chúng ta như một khối đá cẩm thạch. Chúng ta có thể cho nổ tung cả khối, nhưng việc đó hiếm khi có kết cục tốt đẹp. Việc gọt đẽo nó từng chút một sẽ hợp lý hơn nhiều.

Break the big journey into lots of little steps. Each step can be carried out and learned from. If it turns out to be a retrograde step, it was only a small one. Either way, you learn from it, and the next step you take will be informed by those steps that came before.

Hãy chia hành trình lớn thành thật nhiều bước nhỏ. Mỗi bước có thể được thực hiện và rút ra bài học. Nếu hóa ra đó là một bước thụt lùi, thì nó cũng chỉ là một bước nhỏ. Dù thế nào đi nữa, bạn cũng học được từ nó, và bước tiếp theo bạn thực hiện sẽ được định hình bởi những bước đã đi qua trước đó.

Breaking things into smaller pieces also allows you to identify quick wins and learn from them. This can help make the next step easier and can help **build** momentum. By splitting out microservices one at a time, you also get to unlock the value they bring incrementally, rather than having to wait for some big bang **deployment**.

Việc chia mọi thứ thành các mảnh nhỏ hơn cũng cho phép bạn nhận diện những thắng lợi nhanh (quick wins) và học hỏi từ chúng. Điều này có thể giúp bước tiếp theo dễ dàng hơn và giúp tạo đà. Bằng cách tách từng microservice một, bạn cũng được mở khóa dần dần giá trị mà chúng mang lại, thay vì phải chờ đợi một đợt triển khai big-bang nào đó.

All of this leads to what has become my stock advice for people looking at microservices: if you think microservices are a good idea, start somewhere small. Choose one or two areas of functionality, implement them as microservices, get them deployed into production, and then reflect on whether creating your new microservices helped you get closer to your end goal.

Tất cả những điều này dẫn tới lời khuyên đã trở thành “ruột” của tôi dành cho những ai đang nhắm tới microservices: nếu bạn cho rằng microservices là một ý tưởng hay, hãy bắt đầu từ một chỗ nào đó nhỏ thôi. Hãy chọn một hoặc hai mảng chức năng, hiện thực chúng dưới dạng microservices, đưa chúng vào triển khai (deploy) trên môi trường production, rồi sau đó suy ngẫm xem việc tạo ra những microservice mới này có giúp bạn tiến gần hơn tới mục tiêu cuối cùng hay không.



You won't appreciate the true horror, pain, and suffering that a microservice architecture can bring until you are running in production.

Bạn sẽ không thực sự thấm thía hết nỗi kinh hoàng, đau đớn và khổ sở mà một kiến trúc

microservice có thể mang lại cho tới khi bạn vận hành nó trên môi trường production.

The Monolith Is Rarely the Enemy — Khối monolith hiếm khi là kẻ thù

While I already made the case at the start of the book that some form of monolithic architecture can be a totally valid choice, it warrants repeating that a monolithic architecture isn't inherently bad and therefore shouldn't be viewed as the enemy. Don't focus on "not having the monolith"; focus instead on the benefits you expect your change in architecture to bring.

Mặc dù tôi đã lập luận ngay từ đầu cuốn sách rằng một dạng kiến trúc monolithic nào đó hoàn toàn có thể là một lựa chọn chính đáng, vẫn cần nhắc lại rằng kiến trúc monolithic về bản chất không hề xấu, và do đó không nên bị xem là kẻ thù. Đừng tập trung vào việc "không còn monolith"; thay vào đó hãy tập trung vào những lợi ích mà bạn kỳ vọng sự thay đổi kiến trúc của mình sẽ mang lại.

It is common for the existing monolithic architecture to remain after a shift toward microservices, albeit often in a diminished capacity. For example, a move to improve the ability of the application to handle more load might be satisfied by removing the 10% of functionality that is currently bottlenecked, leaving the remaining 90% in the monolithic system.

Việc kiến trúc monolithic hiện tại vẫn còn tồn tại sau khi chuyển dịch sang microservices là điều phổ biến, dù thường là ở một quy mô thu nhỏ hơn. Ví dụ, một nỗ lực nhằm cải thiện khả năng xử lý tải cao hơn của ứng dụng có thể được thỏa mãn bằng cách gỡ bỏ 10% chức năng hiện đang là nút thắt cổ chai, để lại 90% còn lại trong hệ thống monolithic.

Many people find the reality of a monolith and microservices coexisting to be "messy"—but the architecture of a real-world running system is never clean or pristine. If you want a "clean" architecture, by all means laminate a printout of an idealized version of the **system architecture** you might have had, if only you had perfect foresight and limitless funds. Real system architecture is a constantly evolving thing that must adapt as needs and knowledge change. The skill is in getting used to this idea, something I'll come back to in Chapter 16 .

Nhiều người thấy thực tế của việc monolith và microservices cùng tồn tại là "lộn xộn"—nhưng kiến trúc của một hệ thống đang vận hành trong thực tế thì chẳng bao giờ sạch sẽ hay tinh khôi cả. Nếu bạn muốn một kiến trúc "sạch", cứ việc in ra rồi ép nhựa một phiên bản lý tưởng hóa của kiến trúc hệ thống mà bạn lẽ ra đã có, giá như bạn có tầm nhìn hoàn hảo và nguồn kinh phí vô hạn. Kiến trúc hệ thống thực tế là một thứ liên tục tiến hóa, phải thích nghi khi nhu cầu và kiến thức thay đổi. Kỹ năng nằm ở chỗ làm quen với ý tưởng này, điều mà tôi sẽ quay lại ở Chương 16.

By making your migration to microservices an incremental journey, you are able to chip away at the existing monolithic architecture, delivering improvements along the way, while also, importantly, knowing when to stop.

Bằng cách biến quá trình di trú sang microservices thành một hành trình tăng dần, bạn có thể gọt đẽo kiến trúc monolithic hiện tại, mang lại những cải tiến trong suốt chặng đường, đồng thời, điều quan trọng là, biết được khi nào nên dừng lại.

In surprisingly rare circumstances, the demise of the monolith might be a hard requirement. In my experience, this is often limited to situations in which the existing monolith is based on dead or dying technology, is tied to infrastructure that needs to be retired, or is perhaps an expensive third-party

system that you want to ditch. Even in these situations, an incremental approach to decomposition is warranted for the reasons I've outlined.

Trong những trường hợp hiếm hoi một cách đáng ngạc nhiên, việc khai tử khối monolith có thể là một yêu cầu bắt buộc. Theo kinh nghiệm của tôi, điều này thường chỉ giới hạn ở những tình huống mà khối monolith hiện tại dựa trên công nghệ đã chết hoặc đang lụi tàn, bị gắn chặt với hạ tầng cần phải loại bỏ, hoặc có lẽ là một hệ thống bên thứ ba đắt đỏ mà bạn muốn vứt bỏ. Ngay cả trong những tình huống này, cách tiếp cận phân rã tăng dần vẫn là chính đáng vì những lý do mà tôi đã trình bày.

The Dangers of Premature Decomposition — Những hiểm họa của việc phân rã quá sớm

There is danger in creating microservices when you have an unclear understanding of the domain. An example of the problems this can cause comes from my previous company, Thoughtworks. One of its products was Snap CI, a hosted **continuous integration** and **continuous delivery** tool (we'll discuss those concepts in Chapter 7). The team had previously worked on a similar tool, GoCD, a now open source continuous delivery tool that can be deployed locally rather than being hosted in the cloud.

Có một mối nguy hiểm khi tạo ra microservices trong lúc bạn chưa hiểu rõ về miền nghiệp vụ (domain). Một ví dụ về những vấn đề mà điều này có thể gây ra đến từ công ty cũ của tôi, Thoughtworks. Một trong những sản phẩm của họ là Snap CI, một công cụ tích hợp liên tục (CI) và chuyển giao liên tục (CD) được lưu trữ trên cloud (chúng ta sẽ bàn về những khái niệm này ở Chương 7). Trước đó đội ngũ này đã từng làm việc trên một công cụ tương tự, GoCD, một công cụ chuyển giao liên tục nay đã thành mã nguồn mở, có thể được triển khai cục bộ thay vì được lưu trữ trên cloud.

Although there was some code reuse very early on between the Snap CI and GoCD projects, in the end Snap CI turned out to be a completely new codebase. Nonetheless, the previous experience of the team in the domain of CD tooling emboldened them to move more quickly in identifying boundaries and building their system as a set of microservices.

Mặc dù có một chút tái sử dụng mã giữa hai dự án Snap CI và GoCD ở giai đoạn rất sớm, nhưng cuối cùng Snap CI hóa ra lại là một codebase hoàn toàn mới. Tuy nhiên, kinh nghiệm trước đó của đội ngũ trong lĩnh vực công cụ CD đã khiến họ tự tin tiến nhanh hơn trong việc xác định các ranh giới và xây dựng hệ thống của mình thành một tập hợp các microservices.

After a few months, though, it became clear that the use cases of Snap CI were subtly different enough that the initial take on the **service** boundaries wasn't quite right. This led to lots of changes being made across services, and an associated high cost of change. Eventually, the team merged the services back into one monolithic system, giving the team members time to better understand where the boundaries should exist. A year later, the team was able to split the monolithic system into microservices, whose boundaries proved to be much more stable. This is far from the only example of this situation I have seen. Prematurely decomposing a system into microservices can be costly, especially if you are new to the domain. In many ways, having an existing codebase you want to decompose into microservices is much easier than trying to go to microservices from the beginning for this very reason.

Tuy nhiên, sau vài tháng, rõ ràng là các tình huống sử dụng (use case) của Snap CI khác biệt một cách tinh tế đủ để khiến cách phân định ranh giới service ban đầu không thật

sự đúng. Điều này dẫn đến rất nhiều thay đổi phải thực hiện xuyên suốt các service, kèm theo đó là chi phí thay đổi cao. Cuối cùng, đội ngũ đã gộp các service trở lại thành một hệ thống monolithic, để cho các thành viên có thời gian hiểu rõ hơn về việc các ranh giới nên nằm ở đâu. Một năm sau, đội ngũ đã có thể tách hệ thống monolithic thành các microservices, mà các ranh giới của chúng tỏ ra ổn định hơn nhiều. Đây hoàn toàn không phải là ví dụ duy nhất về tình huống này mà tôi đã chứng kiến. Việc phân rã một hệ thống thành microservices quá sớm có thể tốn kém, đặc biệt nếu bạn còn mới với miền nghiệp vụ đó. Xét theo nhiều khía cạnh, việc có sẵn một codebase mà bạn muốn phân rã thành microservices thì dễ dàng hơn nhiều so với việc cố gắng đi thẳng tới microservices ngay từ đầu, chính vì lý do này.

What to Split First? — Tách cái gì trước?

Once you have a firm grasp on why you think microservices are a good idea, you can use this understanding to help prioritize which microservices to create first. Want to scale the application? Functionality that currently constrains the system's ability to handle load is going to be high on the list. Want to improve time to market? Look at the system's volatility to identify those pieces of functionality that change most frequently, and see if they would work as microservices. You can use static analysis tools like CodeScene to quickly find volatile parts of your codebase. You can see an example of a view from CodeScene in Figure 3-1, where we see hotspots in the open source Apache Zookeeper project.

Một khi bạn đã nắm vững lý do vì sao bạn cho rằng microservices là một ý tưởng hay, bạn có thể dùng sự hiểu biết này để giúp ưu tiên việc tạo ra microservice nào trước. Muốn mở rộng ứng dụng? Những chức năng hiện đang kìm hãm khả năng xử lý tải của hệ thống sẽ nằm cao trong danh sách. Muốn cải thiện thời gian đưa sản phẩm ra thị trường (time to market)? Hãy xem xét tính biến động (volatility) của hệ thống để nhận diện những mảng chức năng thay đổi thường xuyên nhất, và xem liệu chúng có hoạt động tốt khi là microservices hay không. Bạn có thể dùng các công cụ phân tích tĩnh như CodeScene để nhanh chóng tìm ra những phần hay biến động trong codebase của mình. Bạn có thể thấy một ví dụ về góc nhìn từ CodeScene ở Hình 3-1, nơi chúng ta thấy các “điểm nóng” (hotspot) trong dự án mã nguồn mở Apache Zookeeper.

But you also have to consider what decompositions are viable. Some functionality can be so deeply baked into the existing monolithic application that it is impossible to see how it can be detangled. Or perhaps the functionality in question is so critical to the application that any changes are considered high risk. Alternatively, the functionality you want to migrate might already be somewhat self-contained, and so the extraction seems very straightforward.

Nhưng bạn cũng phải cân nhắc xem những cách phân rã nào là khả thi. Một số chức năng có thể được “nướng” sâu vào trong ứng dụng monolithic hiện tại đến mức không thể hình dung làm sao gỡ rối được nó. Hoặc có lẽ chức năng đang xét là quá quan trọng đối với ứng dụng đến mức bất kỳ thay đổi nào cũng bị coi là rủi ro cao. Ngược lại, chức năng bạn muốn di trú có thể đã phần nào tự khép kín sẵn, và do đó việc trích xuất có vẻ rất đơn giản.

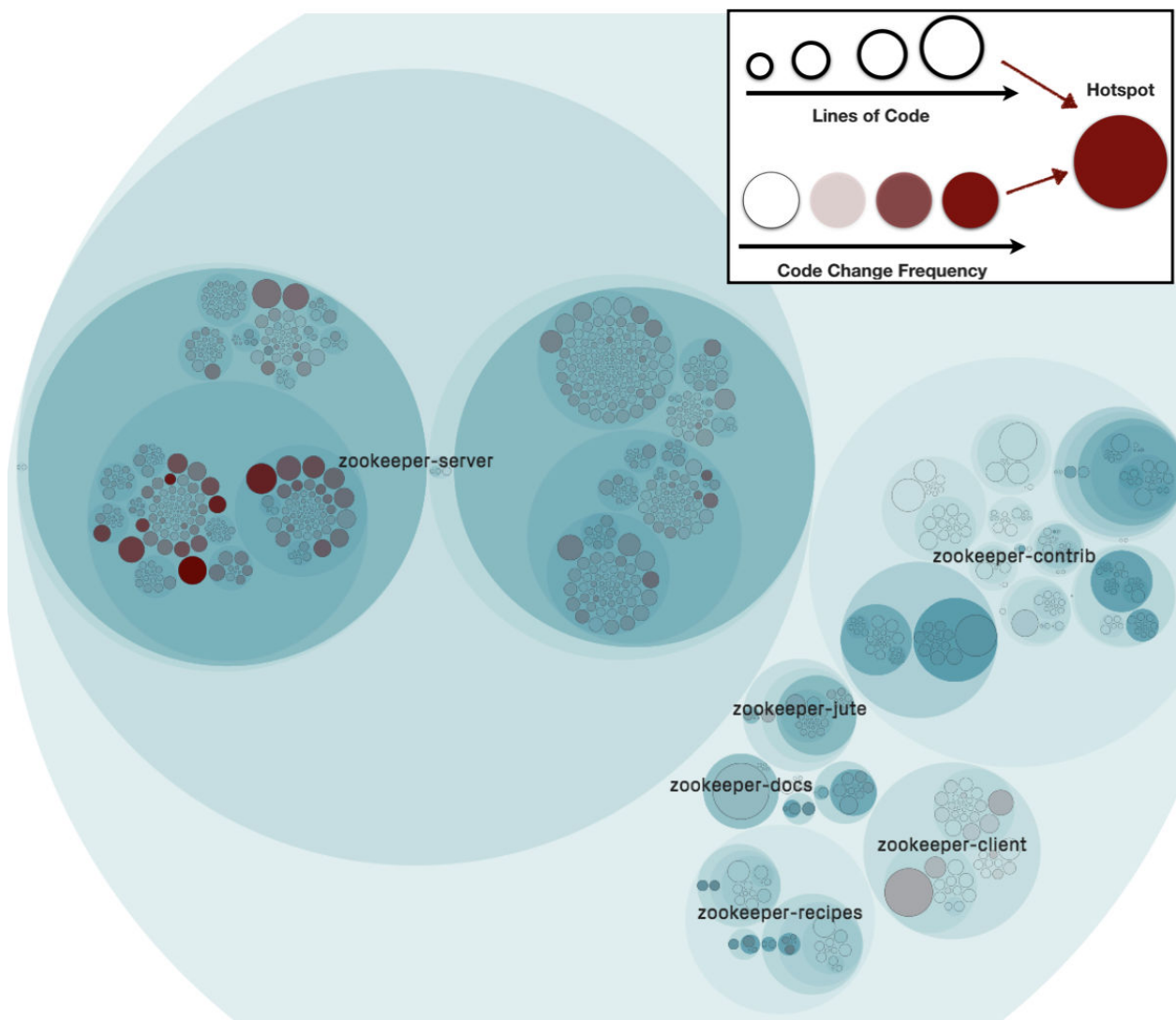


Figure 3-1. The hotspot view in CodeScene, helping identify parts of the codebase that change frequently

Hình 3-1. Góc nhìn điểm nóng trong CodeScene, giúp nhận diện những phần của codebase thay đổi thường xuyên

Fundamentally, the decision about which functionality to split into a microservice will end up being a balance between these two forces—how easy the extraction is versus the benefit of extracting the microservice in the first place.

Về cơ bản, quyết định về việc tách chức năng nào thành một microservice rất cuộc sẽ là sự cân bằng giữa hai lực này—việc trích xuất dễ dàng đến đâu so với lợi ích của việc trích xuất microservice đó ngay từ đầu.

My advice for the first couple of microservices would be to pick things that lean a bit more toward the “easy” end of the spectrum—a microservice that we think has some impact in terms of achieving our end-to-end goal, certainly, but something we’d consider to be low-hanging fruit. It is important with a transition like this, especially one that could take months or years, to gain a sense of momentum early on. So you need some quick wins under your belt.

Lời khuyên của tôi cho một vài microservice đầu tiên là chọn những thứ nghiêng nhiều

hơn về phía đầu “dễ” của quang phổ—một microservice mà chúng ta nghĩ có chút tác động trong việc đạt được mục tiêu xuyên suốt của mình, dĩ nhiên, nhưng cũng là thứ mà ta xem là “quả ngọt trong tầm với” (low-hanging fruit). Với một quá trình chuyển đổi như thế này, đặc biệt là quá trình có thể kéo dài hàng tháng hoặc hàng năm, điều quan trọng là phải tạo được cảm giác về đà tiến ngay từ sớm. Vì vậy bạn cần một vài thắng lợi nhanh giắt lưng.

On the other hand, if you try to extract what you consider to be the easiest microservice and aren't able to make it work, it might be worth reconsidering whether microservices are really right for you and your organization.

Mặt khác, nếu bạn cố trích xuất cái mà bạn cho là microservice dễ nhất mà vẫn không thể làm cho nó hoạt động, thì có lẽ đáng để cân nhắc lại xem liệu microservices có thực sự phù hợp với bạn và tổ chức của bạn hay không.

With a few successes and some lessons learned, you'll be much better placed to tackle more complex extractions, which may also be operating in more critical areas of functionality.

Với một vài thành công và một số bài học rút ra, bạn sẽ ở vào vị thế tốt hơn nhiều để giải quyết những cuộc trích xuất phức tạp hơn, mà cũng có thể đang vận hành ở những mảng chức năng quan trọng hơn.

Decomposition by Layer — Phân rã theo tầng

So you've identified your first microservice to extract; what next? Well, we can break that decomposition down into further, smaller steps.

Vậy là bạn đã xác định được microservice đầu tiên cần trích xuất; tiếp theo thì sao? Chà, chúng ta có thể chia nhỏ cuộc phân rã đó thành những bước nhỏ hơn nữa.

If we consider the traditional three tiers of a web-based services stack, then we can look at the functionality we want to extract in terms of its **user interface**, **backend** application code, and data.

Nếu chúng ta xét ba tầng (tier) truyền thống của một ngăn xếp dịch vụ trên nền web, thì chúng ta có thể nhìn vào chức năng cần trích xuất theo các khía cạnh: giao diện người dùng (UI), mã ứng dụng phía backend, và dữ liệu.

The mapping from a microservice to a user **interface** is often not 1:1 (this is a topic we explore in a lot more depth in Chapter 14). As such, extracting user interface functionality related to the microservice could be considered a separate step. I will sound a note of caution here about ignoring the user interface part of the equation. I've seen far too many organizations look only at the benefits of decomposing the backend functionality, which often results in an overly siloed approach to any architectural restructuring. Sometimes the biggest benefits can come from decomposition of the UI, so ignore this at your peril. Often decomposition of the UI tends to lag behind decomposition of the backend into microservices, since until the microservices are available, it's difficult to see the possibilities for UI decomposition; just make sure it doesn't lag too much.

Việc ánh xạ từ một microservice tới một giao diện người dùng thường không phải là 1:1 (đây là một chủ đề mà chúng ta sẽ khám phá sâu hơn nhiều ở Chương 14). Như vậy, việc trích xuất chức năng giao diện người dùng liên quan tới microservice có thể được coi là một bước riêng biệt. Tôi xin lưu ý một lời cảnh báo ở đây về việc bỏ qua phần giao diện người dùng trong phương trình. Tôi đã chứng kiến quá nhiều tổ chức chỉ nhìn vào những lợi ích của việc phân rã chức năng backend, điều thường dẫn tới một cách tiếp cận quá tách biệt thành các “silo” trong bất kỳ cuộc tái cấu trúc kiến trúc nào. Đôi khi những lợi ích lớn nhất lại có thể đến từ việc phân rã UI, vậy nên hãy coi chừng kéo bỏ qua điều này.

Việc phân rã UI thường có xu hướng đi sau việc phân rã backend thành microservices, bởi vì cho tới khi các microservice có sẵn, sẽ khó mà thấy được các khả năng cho việc phân rã UI; chỉ cần đảm bảo nó đừng đi sau quá nhiều là được.

If we then look at the backend code and related storage, it's vital for both to be in scope when extracting a microservice. Let's consider Figure 3-2, where we are looking to extract functionality related to managing a customer's wishlist. There is some application code that lives in the monolith, and some related data storage in the **database**. So which bit should we extract first?

Nếu sau đó chúng ta nhìn vào mã backend và phần lưu trữ liên quan, thì cả hai đều cần nằm trong phạm vi khi trích xuất một microservice. Hãy xét Hình 3-2, nơi chúng ta đang muốn trích xuất chức năng liên quan đến việc quản lý danh sách yêu thích (wishlist) của khách hàng. Có một phần mã ứng dụng nằm trong khối monolith, và một phần lưu trữ dữ liệu liên quan trong cơ sở dữ liệu. Vậy chúng ta nên trích xuất phần nào trước?

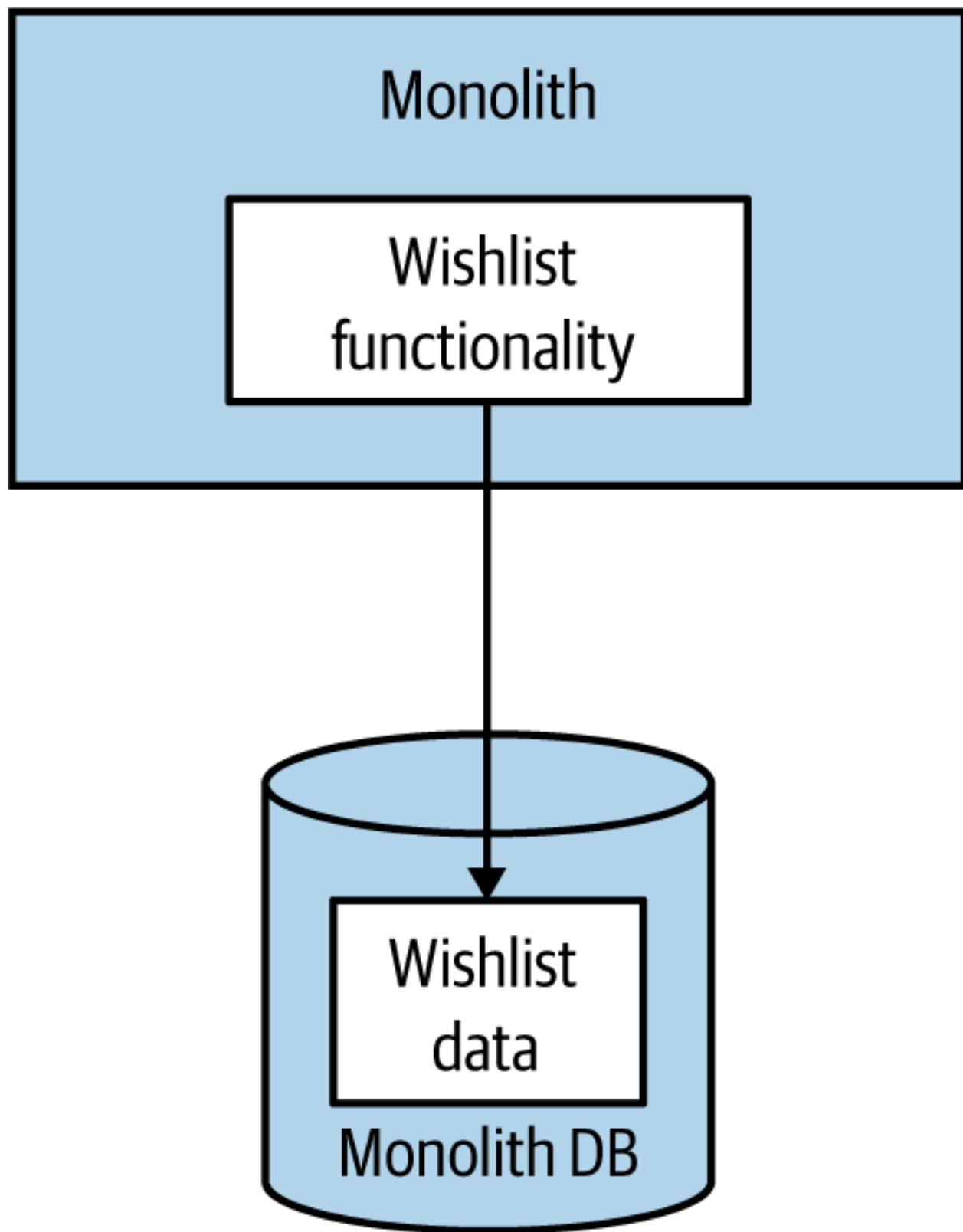


Figure 3-2. The wishlist code and data in the existing monolithic application

Hình 3-2. Mã và dữ liệu wishlist trong ứng dụng monolithic hiện tại

Code First — Mã trước

In Figure 3-3, we have extracted the code associated with the wishlist functionality into a new microservice. The data for the wishlist remains in the monolithic database at this stage—we haven't completed the decomposition until we've also moved out the data related to the new Wishlist microservice.

Trong Hình 3-3, chúng ta đã trích xuất phần mã gắn với chức năng wishlist vào một microservice mới. Ở giai đoạn này, dữ liệu cho wishlist vẫn còn nằm trong cơ sở dữ liệu monolithic—chúng ta chưa hoàn tất việc phân rã cho tới khi cũng đã chuyển ra ngoài cả phần dữ liệu liên quan tới microservice Wishlist mới.

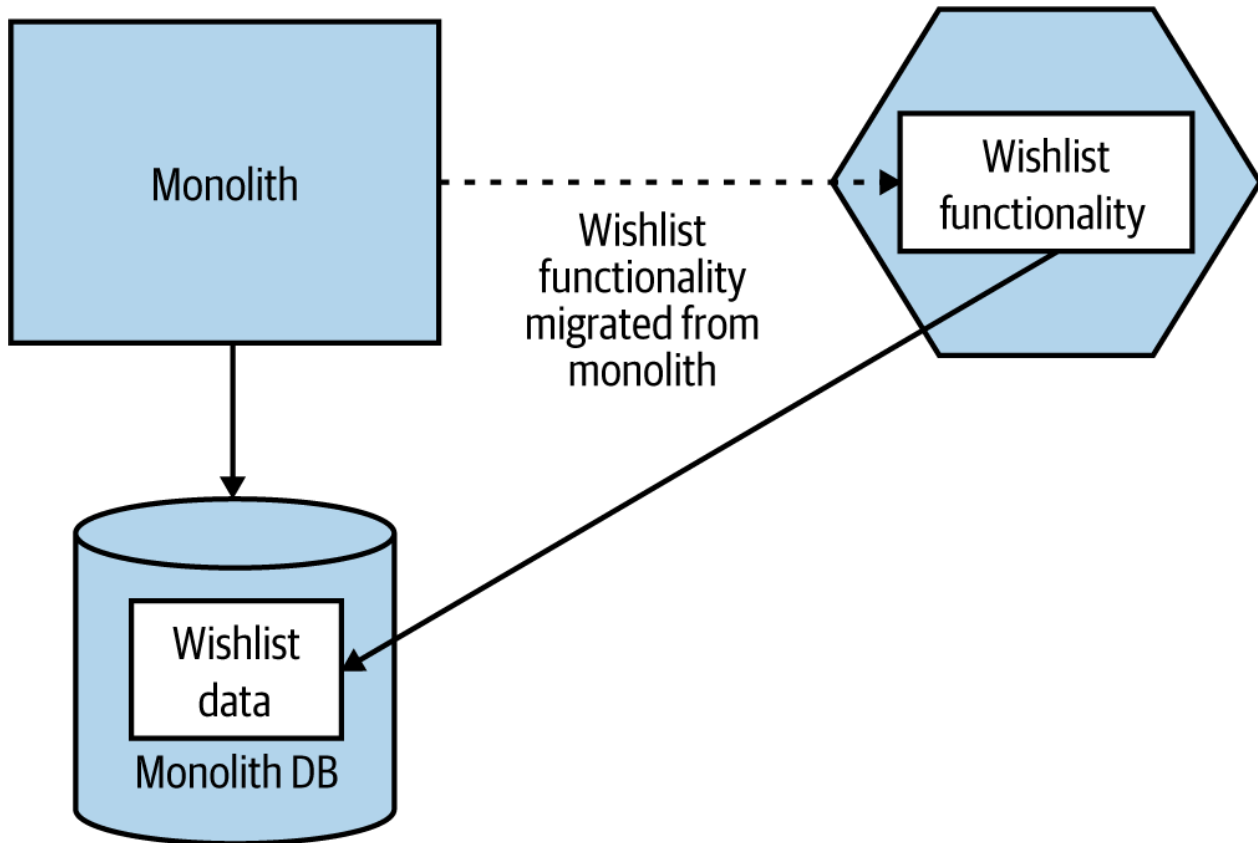


Figure 3-3. Moving the wishlist code into a new microservice first, leaving the data in the monolithic database In my experience, this tends to be the most common first step. The main reason for this is that it tends to deliver more short-term benefit. If we left the data in the monolithic database, we're storing up lots of pain for the future, so that does need to be addressed too, but we have gained a lot from our new microservice.

Hình 3-3. Chuyển mã wishlist vào một microservice mới trước, để lại dữ liệu trong cơ sở dữ liệu monolithic. Theo kinh nghiệm của tôi, đây có xu hướng là bước đầu tiên phổ biến nhất. Lý do chính cho điều này là vì nó có xu hướng mang lại nhiều lợi ích ngắn hạn hơn. Nếu chúng ta để dữ liệu lại trong cơ sở dữ liệu monolithic, chúng ta đang tích trữ rất nhiều nỗi đau cho tương lai, nên việc đó cũng cần được giải quyết, nhưng chúng ta đã đạt được rất nhiều từ microservice mới của mình.

Extracting the application code tends to be easier than extracting things from the database. If we found that it was impossible to extract the application code cleanly, we could abort any further work,

avoiding the need to detangle the database. If, however, the application code is cleanly extracted but extracting the data proves to be impossible, we could be in trouble—thus it's essential that even if you decide to extract the application code before the data, you need to have looked at the associated data storage and have some idea as to whether extraction is viable and how you will go about it. So do the legwork to sketch out how both application code and data will be extracted before you start.

Việc trích xuất mã ứng dụng có xu hướng dễ hơn so với việc trích xuất mọi thứ ra khỏi cơ sở dữ liệu. Nếu chúng ta phát hiện rằng không thể trích xuất mã ứng dụng một cách gọn gàng, chúng ta có thể hủy bỏ mọi công việc tiếp theo, tránh được việc phải gỡ rối cơ sở dữ liệu. Tuy nhiên, nếu mã ứng dụng được trích xuất gọn gàng nhưng việc trích xuất dữ liệu lại hóa ra bất khả thi, chúng ta có thể gặp rắc rối—do đó điều thiết yếu là kể cả khi bạn quyết định trích xuất mã ứng dụng trước phần dữ liệu, bạn vẫn cần phải đã xem xét phần lưu trữ dữ liệu liên quan và có một ý niệm nào đó về việc liệu việc trích xuất có khả thi không và bạn sẽ thực hiện nó như thế nào. Vậy nên hãy làm công việc nền tảng để phác thảo cách cả mã ứng dụng lẫn dữ liệu sẽ được trích xuất ra sao trước khi bạn bắt đầu.

Data First — Dữ liệu trước

In Figure 3-4 , we see the data being extracted first, before the application code. I see this approach less often, but it can be useful in situations in which you are unsure whether the data can be separated cleanly. Here, you prove that this can be done before moving on to the hopefully easier application code extraction.

Trong Hình 3-4, chúng ta thấy dữ liệu được trích xuất trước, trước cả mã ứng dụng. Tôi thấy cách tiếp cận này ít gặp hơn, nhưng nó có thể hữu ích trong những tình huống bạn không chắc liệu dữ liệu có thể được tách ra một cách gọn gàng hay không. Ở đây, bạn chứng minh rằng việc này có thể làm được trước khi chuyển sang việc trích xuất mã ứng dụng vốn hy vọng là dễ dàng hơn.

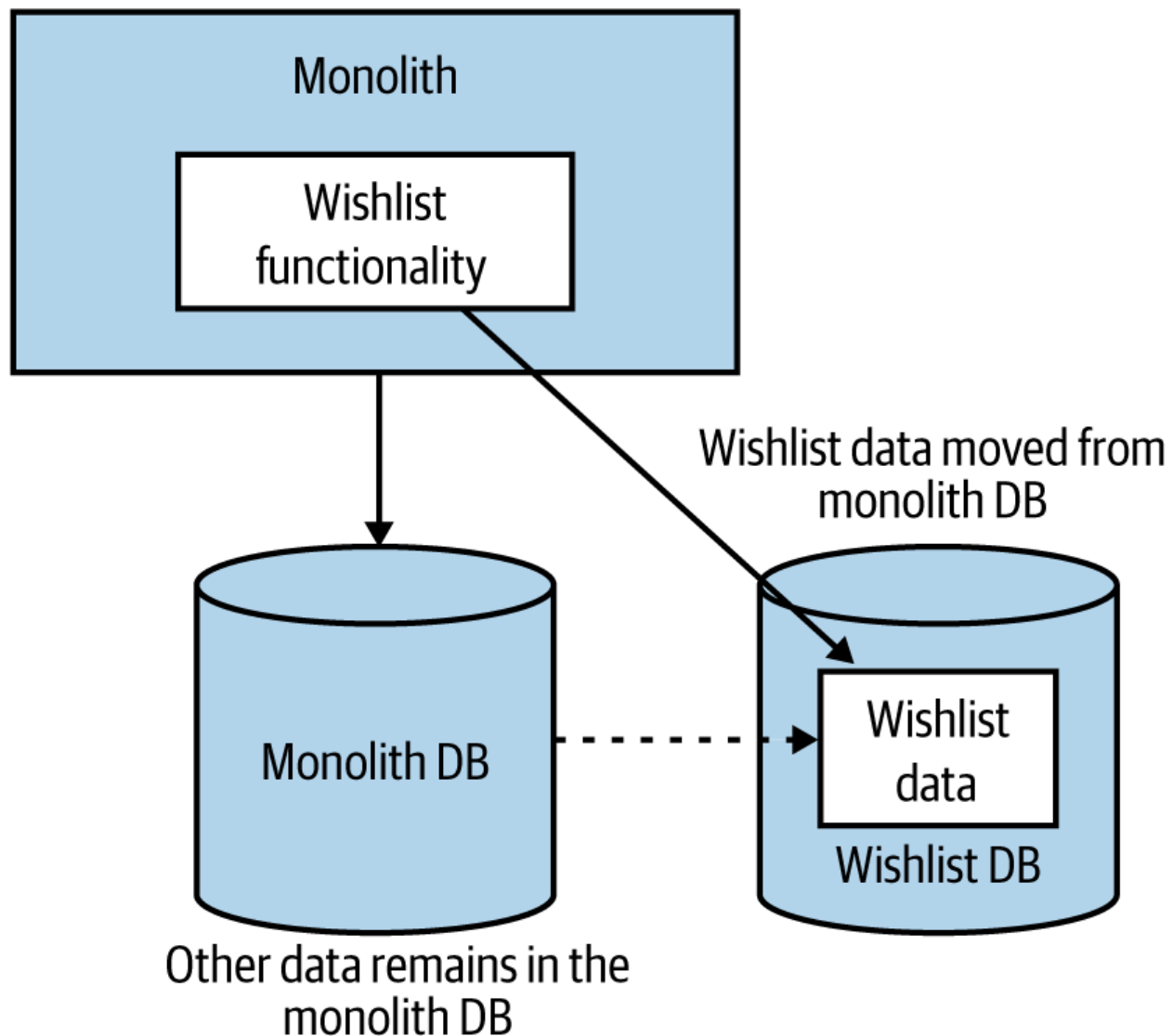


Figure 3-4. The tables associated with the wishlist functionality are extracted first

Hình 3-4. Các bảng gắn với chức năng wishlist được trích xuất trước

The main benefit of this approach in the short term is in derisking the full extraction of the microservice. It forces you to deal up front with issues like loss of enforced **data integrity** in your database or lack of transactional operations across both sets of data. We'll touch briefly on the implications of both issues later in this chapter.

Lợi ích chính của cách tiếp cận này trong ngắn hạn nằm ở việc giảm rủi ro cho toàn bộ quá trình trích xuất microservice. Nó buộc bạn phải xử lý ngay từ đầu những vấn đề như mất đi sự ràng buộc toàn vẹn dữ liệu được thực thi trong cơ sở dữ liệu, hay việc thiếu các thao tác giao dịch (transaction) xuyên suốt cả hai tập dữ liệu. Chúng ta sẽ điểm qua sơ lược về hệ quả của cả hai vấn đề này ở phần sau của chương.

Useful Decompositional Patterns — Những pattern phân rã hữu ích

A number of patterns can be useful in helping break apart an existing system. Many of these are explored in detail in my book *Monolith to Microservices* ; 1 rather than repeat them all here, I will share an overview of some of them to give you an idea of what is possible.

Một số pattern có thể hữu ích trong việc giúp tách rời một hệ thống đang tồn tại. Nhiều pattern trong số này được khám phá chi tiết trong cuốn sách *Monolith to Microservices* của tôi;¹ thay vì lặp lại tất cả ở đây, tôi sẽ chia sẻ một cái nhìn tổng quan về một vài trong số chúng để cho bạn một ý niệm về những gì là khả thi.

Strangler Fig Pattern — Pattern Strangler Fig (cây đa bóp nghẹt)

A technique that has seen frequent use during system rewrites is the strangler fig pattern , a term coined by Martin Fowler . Inspired by a type of plant, the pattern describes the process of wrapping an old system with the new system over time, allowing the new system to take over more and more features of the old system incrementally.

Một kỹ thuật đã được sử dụng thường xuyên trong các cuộc viết lại hệ thống là pattern strangler fig, một thuật ngữ do Martin Fowler đặt ra. Lấy cảm hứng từ một loài cây, pattern này mô tả quá trình bao bọc một hệ thống cũ bằng hệ thống mới theo thời gian, cho phép hệ thống mới dần dần tiếp quản ngày càng nhiều tính năng của hệ thống cũ.

The approach as shown in Figure 3-5 is straightforward. You intercept calls to the existing system—in our case the existing monolithic application. If the call to that piece of functionality is implemented in our new microservice architecture, it is redirected to the microservice. If the functionality is still provided by the monolith, the call is allowed to continue to the monolith itself.

Cách tiếp cận như được thể hiện trong Hình 3-5 khá đơn giản. Bạn chặn các lời gọi tới hệ thống hiện tại—trong trường hợp của chúng ta là ứng dụng monolithic hiện tại. Nếu lời gọi tới mảng chức năng đó được hiện thực trong kiến trúc microservice mới của chúng ta, nó sẽ được chuyển hướng tới microservice. Nếu chức năng đó vẫn còn do khối monolith cung cấp, lời gọi sẽ được cho phép tiếp tục đi tới chính khối monolith.

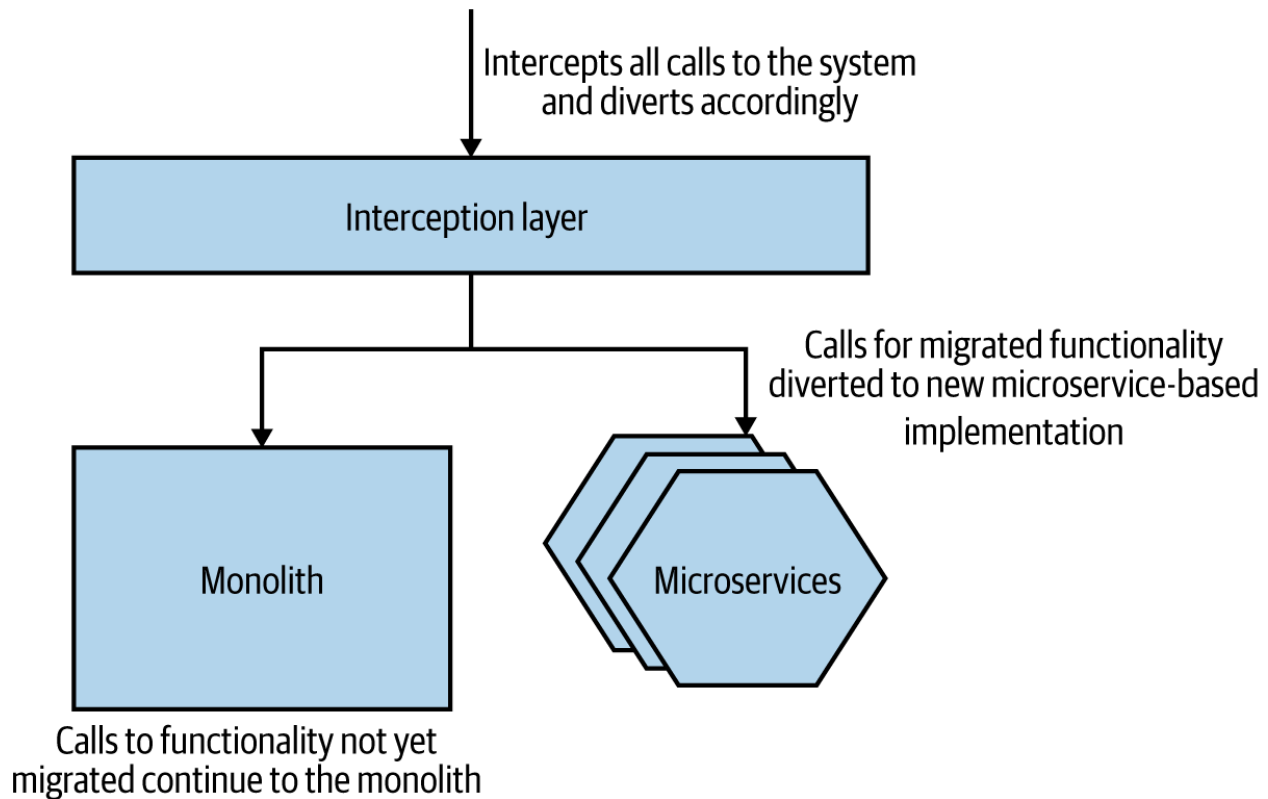


Figure 3-5. An overview of the strangler fig pattern

Hình 3-5. Tổng quan về pattern strangler fig

1 Sam Newman, Monolith to Microservices (Sebastopol: O'Reilly, 2019).

1 Sam Newman, Monolith to Microservices (Sebastopol: O'Reilly, 2019).

The beauty of this pattern is that it can often be done without making any changes to the underlying monolithic application. The monolith is unaware that it has even been “wrapped” with a newer system.

Cái hay của pattern này là nó thường có thể được thực hiện mà không cần thay đổi gì đối với ứng dụng monolithic bên dưới. Khối monolith thậm chí còn không hề biết rằng nó đã bị “bao bọc” bởi một hệ thống mới hơn.

Parallel Run — Chạy song song (Parallel Run)

When switching from functionality provided by an existing tried and tested application architecture to a fancy new microservice-based one, there may be some nervousness, especially if the functionality being migrated is critical to your organization.

Khi chuyển từ chức năng do một kiến trúc ứng dụng đã được kiểm chứng và tin cậy cung cấp sang một kiến trúc dựa trên microservice mới toanh, có thể sẽ có chút lo lắng, đặc biệt nếu chức năng đang được di trú là quan trọng đối với tổ chức của bạn.

One way to make sure the new functionality is working well without risking the existing system behavior is to make use of the parallel run pattern: running both your monolithic implementation of

the functionality and the new microservice implementation side by side, serving the same requests, and comparing the results. We'll explore this pattern in more detail in “Parallel Run” on page 272 .

Một cách để đảm bảo chức năng mới đang hoạt động tốt mà không gây rủi ro cho hành vi của hệ thống hiện tại là tận dụng pattern chạy song song (parallel run): chạy cả phần hiện thực monolithic của chức năng lẫn phần hiện thực microservice mới cạnh nhau, cùng phục vụ những yêu cầu giống nhau, và so sánh kết quả. Chúng ta sẽ khám phá pattern này chi tiết hơn ở mục “Parallel Run” trang 272.

Feature Toggle — Feature Toggle

A feature toggle is a mechanism that allows a feature to be switched off or on, or to switch between two different implementations of some functionality. The feature toggle is a pattern that has good general applicability, but it can be especially useful as part of a microservice migration.

Một feature toggle là một cơ chế cho phép một tính năng được bật hay tắt, hoặc chuyển đổi giữa hai cách hiện thực khác nhau của một chức năng nào đó. Feature toggle là một pattern có tính ứng dụng chung tốt, nhưng nó có thể đặc biệt hữu ích như một phần của quá trình di trú microservice.

As I outlined with the strangler fig application, we'll often leave the existing functionality in place in the monolith during the transition, and we'll want the ability to switch between versions of the functionality—the functionality in the monolith and that in the new microservice. With the strangler fig pattern example of using an HTTP proxy, we could implement the feature toggle in the proxy layer to allow for a simple control to switch between implementations.

Như tôi đã phác thảo với ứng dụng strangler fig, chúng ta thường sẽ để chức năng hiện tại nằm nguyên tại chỗ trong khối monolith trong suốt quá trình chuyển đổi, và chúng ta sẽ muốn có khả năng chuyển đổi giữa các phiên bản của chức năng—chức năng trong khối monolith và chức năng trong microservice mới. Với ví dụ về pattern strangler fig sử dụng một HTTP proxy, chúng ta có thể hiện thực feature toggle ở tầng proxy để cho phép một cơ chế điều khiển đơn giản nhằm chuyển đổi giữa các cách hiện thực.

For a broader introduction to feature toggles, I recommend Pete Hodgson's article “Feature Toggles (aka Feature Flags).”²

Để có một giới thiệu rộng hơn về feature toggle, tôi khuyên đọc bài viết “Feature Toggles (aka Feature Flags)” của Pete Hodgson.²

² Pete Hodgson, “Feature Toggles (aka Feature Flags),” [martinfowler.com](https://martinfowler.com/oreil.ly/XiU2t), October 9, 2017, <https://oreil.ly/XiU2t> .

² Pete Hodgson, “Feature Toggles (aka Feature Flags),” [martinfowler.com](https://martinfowler.com/oreil.ly/XiU2t), October 9, 2017, <https://oreil.ly/XiU2t> .

Data Decomposition Concerns — Những mối quan ngại khi phân rã dữ liệu

When we start breaking databases apart, we can cause a number of issues. Here are a few of the challenges you might face, and some tips to help.

Khi chúng ta bắt đầu tách rời các cơ sở dữ liệu, chúng ta có thể gây ra một số vấn đề. Dưới đây là một vài thử thách mà bạn có thể đối mặt, cùng một số lời khuyên để giúp bạn.

Performance — Hiệu năng

Databases, especially relational databases, are good at joining data across different tables. Very good. So good, in fact, that we take this for granted. Often, though, when we split databases apart in the name of microservices, we end up having to move join operations from the data tier up into the microservices themselves. And try as we might, it's unlikely to be as fast.

Các cơ sở dữ liệu, đặc biệt là cơ sở dữ liệu quan hệ, rất giỏi trong việc kết nối (join) dữ liệu giữa các bảng khác nhau. Rất giỏi. Giỏi đến mức, thực ra, chúng ta coi đó là chuyện hiển nhiên. Tuy nhiên, thường thì khi chúng ta tách rời các cơ sở dữ liệu nhân danh microservices, chúng ta rốt cuộc phải chuyển các thao tác join từ tầng dữ liệu lên chính các microservice. Và dù có cố gắng đến đâu, nó cũng khó mà nhanh được như trước.

Consider Figure 3-6, which illustrates a situation we find ourselves in regarding MusicCorp. We've decided to extract our catalog functionality—something that can manage and expose information about artists, tracks, and albums. Currently, our catalog-related code inside the monolith uses an Albums table to store information about the CDs that we might have available for sale. These albums end up getting referenced in our Ledger table, which is where we track all sales. The rows in the Ledger table record the date on which something is sold, along with an identifier that refers to the item sold; the identifier in our example is called a SKU (stock keeping unit), a common practice in retail systems. ³

Hãy xét Hình 3-6, minh họa một tình huống mà chúng ta gặp phải liên quan tới MusicCorp. Chúng ta đã quyết định trích xuất chức năng danh mục (catalog)—thứ có thể quản lý và phơi bày thông tin về nghệ sĩ, bài hát và album. Hiện tại, mã liên quan tới catalog bên trong khối monolith sử dụng một bảng Albums để lưu trữ thông tin về các đĩa CD mà chúng ta có thể có sẵn để bán. Những album này rốt cuộc được tham chiếu trong bảng Ledger của chúng ta, nơi chúng ta theo dõi tất cả các giao dịch bán hàng. Các hàng trong bảng Ledger ghi lại ngày mà một thứ gì đó được bán, kèm theo một định danh tham chiếu tới món hàng đã bán; trong ví dụ của chúng ta định danh này được gọi là SKU (stock keeping unit—đơn vị lưu kho), một thực hành phổ biến trong các hệ thống bán lẻ.³

At the end of each month, we need to generate a report outlining our best-selling CDs. The Ledger table helps us understand which SKU sold the most copies, but the information about that SKU is over in the Albums table. We want to make the reports nice and easy to read, so rather than saying, “We sold 400 copies of SKU 123 and made \$1,596,” we'd add more information about what was sold, saying instead, “We sold 400 copies of Now That's What I Call Death Polka and made \$1,596.” To do this, the database query triggered by our finance code needs to join information from the Ledger table to the Albums table, as Figure 3-6 shows.

Vào cuối mỗi tháng, chúng ta cần tạo ra một báo cáo phác thảo những đĩa CD bán chạy nhất của mình. Bảng Ledger giúp chúng ta hiểu được SKU nào đã bán được nhiều bản nhất, nhưng thông tin về SKU đó lại nằm bên kia trong bảng Albums. Chúng ta muốn làm cho báo cáo đẹp dễ và dễ đọc, nên thay vì nói “Chúng ta đã bán 400 bản SKU 123 và thu về 1.596 đô la”, chúng ta sẽ thêm nhiều thông tin hơn về món đã bán, thay vào đó nói rằng “Chúng ta đã bán 400 bản Now That's What I Call Death Polka và thu về 1.596 đô la”. Để làm được điều này, truy vấn cơ sở dữ liệu được kích hoạt bởi mã tài chính của chúng ta cần join thông tin từ bảng Ledger sang bảng Albums, như Hình 3-6 thể hiện.

3 Obviously, this is a simplification of what a real-world system would look like. It seems reasonable, for example, that we would record how much we sold an item for in a financial ledger!

3 Obviously, this is a simplification of what a real-world system would look like. It seems reasonable, for example, that we would record how much we sold an item for in a financial

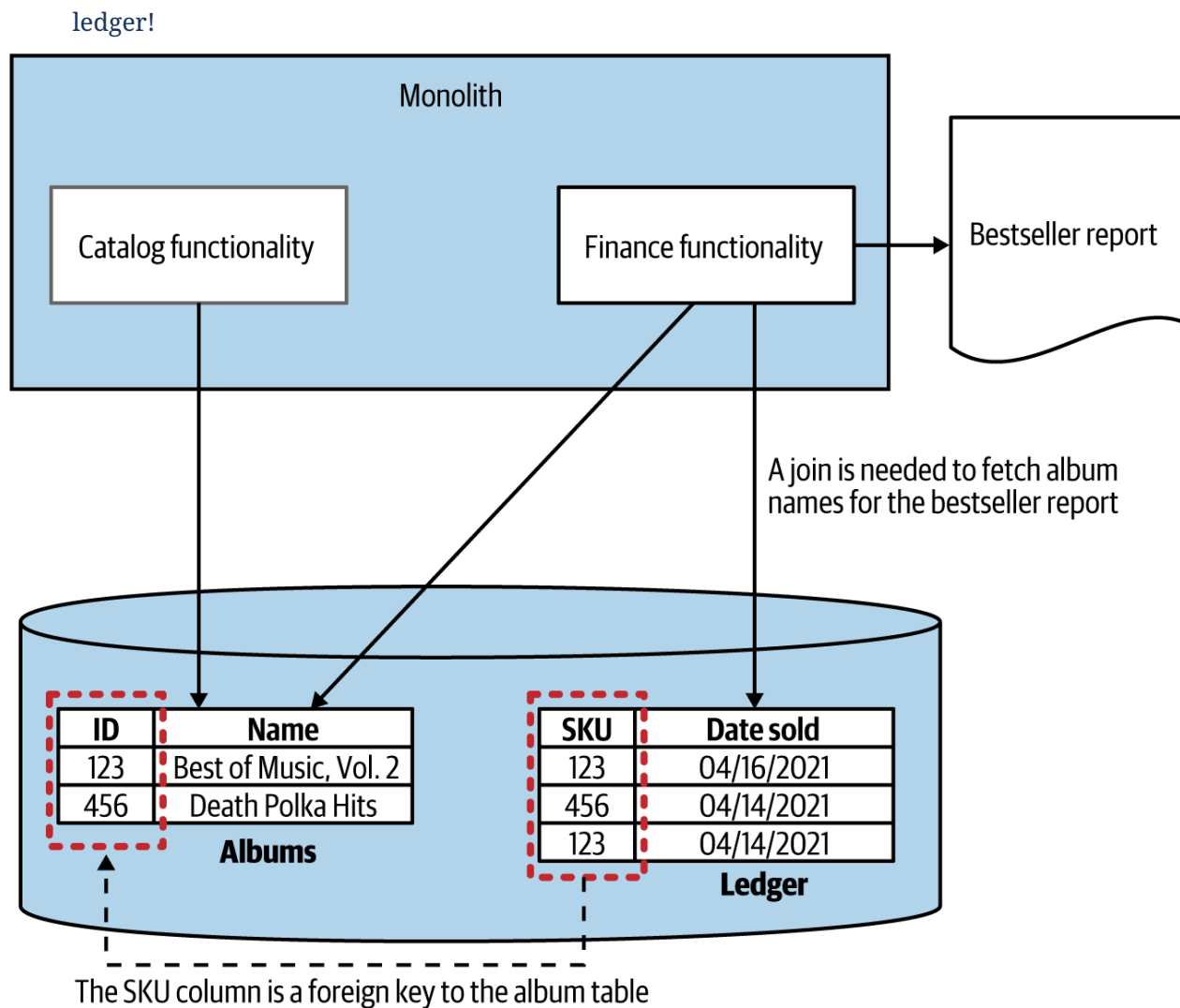
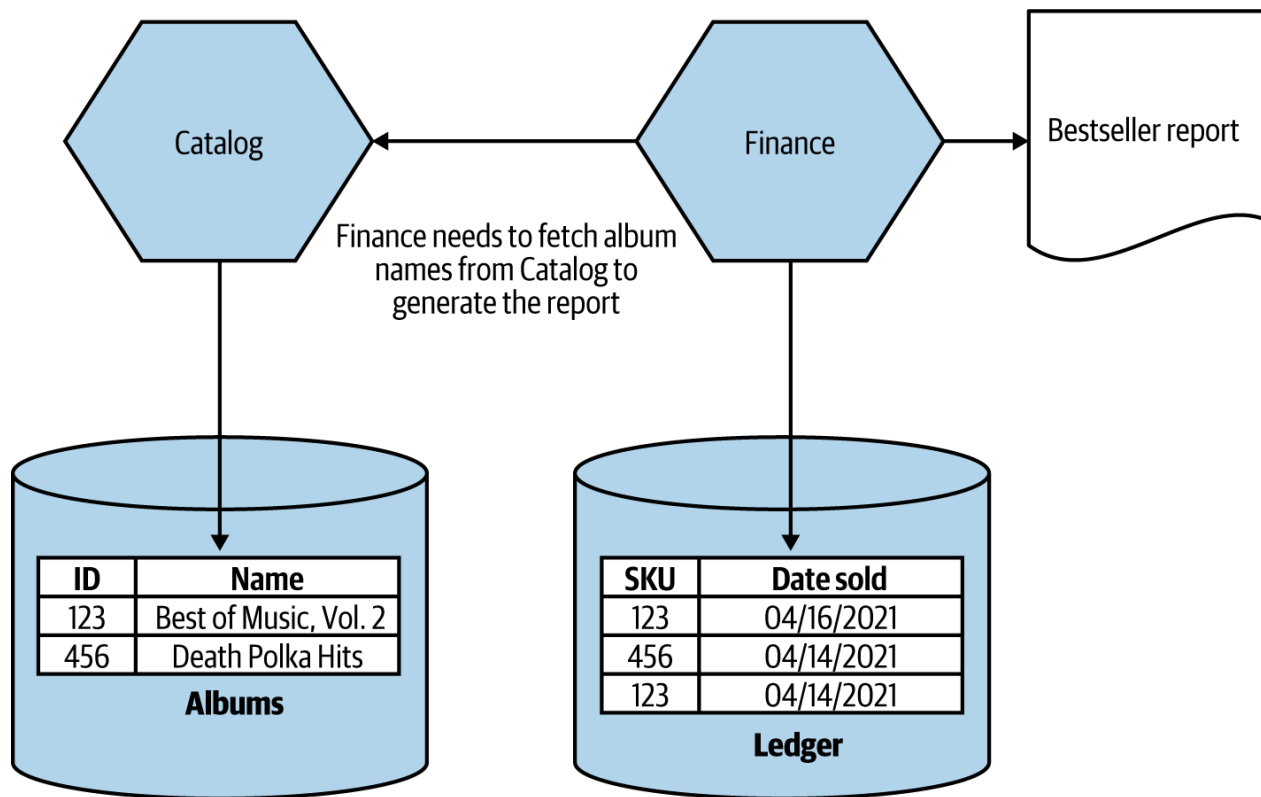


Figure 3-6. A join operation in the monolithic database

Hình 3-6. Một thao tác join trong cơ sở dữ liệu monolithic

In our new microservice-based world, our new Finance microservice has the responsibility of generating the bestsellers report but doesn't have the album data locally. So it will need to fetch this data from our new Catalog microservice, as shown in Figure 3-7. When generating the report, the Finance microservice first queries the Ledger table, extracting the list of best-selling SKUs for the last month. At this point, the only information we have locally is a list of SKUs and the number of copies sold for each SKU.

Trong thế giới dựa trên microservice mới của chúng ta, microservice Finance mới có trách nhiệm tạo ra báo cáo bán chạy nhất nhưng lại không có dữ liệu album một cách cục bộ. Vì vậy nó sẽ cần phải lấy dữ liệu này từ microservice Catalog mới của chúng ta, như được thể hiện trong Hình 3-7. Khi tạo báo cáo, microservice Finance đầu tiên truy vấn bảng Ledger, trích ra danh sách các SKU bán chạy nhất của tháng trước. Tại thời điểm này, thông tin duy nhất chúng ta có một cách cục bộ là một danh sách các SKU và số bản đã bán cho mỗi SKU.



Hình 3-7. Thay thế một thao tác join trong cơ sở dữ liệu bằng các lời gọi service

Next, we need to call the Catalog microservice, requesting information on each of these SKUs. This request in turn will cause the Catalog microservice to make its own local SELECT on its own database.

Tiếp theo, chúng ta cần gọi microservice Catalog, yêu cầu thông tin về mỗi SKU này. Yêu cầu này đến lượt nó sẽ khiến microservice Catalog thực hiện một câu lệnh SELECT cục bộ của riêng nó trên cơ sở dữ liệu của riêng nó.

Logically, the join operation is still happening, but it is now happening inside the Finance microservice rather than in the database. The join has gone from the data tier to the application code tier. Unfortunately, this operation isn't going to be anywhere near as efficient as it would have been had the join remained in the database. We've gone from a world in which we have a single SELECT statement, to a new world in which we have a SELECT query against the Ledger table, followed by a call to the Catalog microservice, which in turn triggers a SELECT statement against the Albums table, as we see in Figure 3-7.

Về mặt logic, thao tác join vẫn đang diễn ra, nhưng giờ đây nó đang diễn ra bên trong microservice Finance thay vì trong cơ sở dữ liệu. Thao tác join đã chuyển từ tầng dữ liệu sang tầng mã ứng dụng. Đáng tiếc là thao tác này sẽ không hề hiệu quả gần bằng khi thao tác join vẫn còn nằm trong cơ sở dữ liệu. Chúng ta đã đi từ một thế giới chỉ có một câu lệnh SELECT duy nhất, sang một thế giới mới trong đó chúng ta có một truy vấn SELECT trên bảng Ledger, theo sau là một lời gọi tới microservice Catalog, mà đến lượt nó lại kích hoạt một câu lệnh SELECT trên bảng Albums, như chúng ta thấy trong Hình 3-7.

In this situation, I'd be very surprised if the overall latency of this operation didn't increase. That may not be a significant problem in this particular case, as this report is generated monthly and could therefore be aggressively cached (we'll explore this topic in more detail in "**Caching**" on page

435). But if this is a frequent operation, that could be more problematic. We can mitigate the likely impact of this increase in latency by allowing for SKUs to be looked up in the Catalog microservice in bulk, or perhaps even by caching the required album information locally.

Trong tình huống này, tôi sẽ rất ngạc nhiên nếu độ trễ (latency) tổng thể của thao tác này không tăng lên. Điều đó có thể không phải là một vấn đề đáng kể trong trường hợp cụ thể này, vì báo cáo này được tạo ra hằng tháng và do đó có thể được cache một cách “hung hăng” (chúng ta sẽ khám phá chủ đề này chi tiết hơn ở mục “Caching” trang 435). Nhưng nếu đây là một thao tác diễn ra thường xuyên, thì điều đó có thể gây rắc rối hơn. Chúng ta có thể giảm thiểu tác động có khả năng xảy ra của việc tăng độ trễ này bằng cách cho phép tra cứu các SKU trong microservice Catalog theo lô (bulk), hoặc thậm chí có lẽ bằng cách cache thông tin album cần thiết một cách cục bộ.

Data Integrity — Toàn vẹn dữ liệu

Databases can be useful in ensuring integrity of our data. Coming back to Figure 3-6 , with both the Album and Ledger tables being in the same database, we could (and likely would) define a foreign key relationship between the rows in the Ledger table and the Album table. This would ensure that we’d always be able to navigate from a record in the Ledger table back to information about the album sold, as we wouldn’t be able to delete records from the Album table if they were referenced in Ledger .

Các cơ sở dữ liệu có thể hữu ích trong việc đảm bảo tính toàn vẹn dữ liệu (data integrity) của chúng ta. Quay lại Hình 3-6, với cả hai bảng Album và Ledger cùng nằm trong một cơ sở dữ liệu, chúng ta có thể (và nhiều khả năng sẽ) định nghĩa một quan hệ khóa ngoại (foreign key) giữa các hàng trong bảng Ledger và bảng Album. Điều này sẽ đảm bảo rằng chúng ta luôn có thể điều hướng từ một bản ghi trong bảng Ledger trở về thông tin về album đã bán, vì chúng ta sẽ không thể xóa các bản ghi khỏi bảng Album nếu chúng được tham chiếu trong Ledger.

With these tables now living in different databases, we no longer have enforcement of the integrity of our data model. There is nothing to stop us from deleting a row in the Album table, causing an issue when we try to work out exactly what item was sold.

Với các bảng này giờ đây sống trong những cơ sở dữ liệu khác nhau, chúng ta không còn sự thực thi tính toàn vẹn của mô hình dữ liệu nữa. Chẳng có gì ngăn cản chúng ta khỏi việc xóa một hàng trong bảng Album, gây ra một vấn đề khi chúng ta cố gắng tìm ra chính xác món hàng nào đã được bán.

To an extent, you’ll simply need to get used to the fact that you can no longer rely on your database to enforce the integrity of inter-entity relationships. Obviously, for data that remains inside a single database, this isn’t an issue.

Ở một mức độ nào đó, bạn sẽ đơn giản là phải làm quen với thực tế rằng bạn không còn có thể dựa vào cơ sở dữ liệu của mình để thực thi tính toàn vẹn của các mối quan hệ liên thực thể (inter-entity) nữa. Dĩ nhiên, đối với dữ liệu vẫn còn nằm bên trong một cơ sở dữ liệu duy nhất, đây không phải là vấn đề.

There are a number of work-arounds, although “coping patterns” would be a better term for ways we might deal with this problem. We could use a soft delete in the Album table so that we don’t actually remove a record but just mark it as deleted. Another option could be to copy the name of the album into the Ledger table when a sale is made, but we would have to resolve how we wanted to handle synchronizing changes in the album name.

Có một số cách giải quyết tạm thời, mặc dù “các pattern đối phó” (coping patterns) sẽ là một thuật ngữ phù hợp hơn để chỉ những cách mà chúng ta có thể xử lý vấn đề này. Chúng ta có thể dùng cơ chế xóa mềm (soft delete) trong bảng Album để không thực sự gỡ bỏ một bản ghi mà chỉ đánh dấu nó là đã bị xóa. Một lựa chọn khác có thể là sao chép tên album vào bảng Ledger khi một giao dịch bán hàng được thực hiện, nhưng khi đó chúng ta sẽ phải giải quyết cách mà chúng ta muốn xử lý việc đồng bộ các thay đổi trong tên album.

Transactions — Giao dịch

Many of us have come to rely on the guarantees we get from managing data in transactions. Based on that certainty, we’ve built applications in a certain way, knowing that we can rely on the database to handle a number of things for us. Once we start splitting data across multiple databases, though, we lose the safety of the **ACID** transactions we are used to. (I explain the acronym ACID and discuss ACID transactions in more depth in Chapter 6.)

Nhiều người trong chúng ta đã đi tới chỗ dựa dẫm vào những bảo đảm mà chúng ta nhận được từ việc quản lý dữ liệu trong các giao dịch. Dựa trên sự chắc chắn đó, chúng ta đã xây dựng các ứng dụng theo một cách nhất định, biết rằng chúng ta có thể dựa vào cơ sở dữ liệu để xử lý hộ chúng ta một số việc. Tuy nhiên, một khi chúng ta bắt đầu tách dữ liệu ra nhiều cơ sở dữ liệu, chúng ta sẽ mất đi sự an toàn của các giao dịch ACID mà chúng ta đã quen thuộc. (Tôi giải thích từ viết tắt ACID và bàn về các giao dịch ACID sâu hơn ở Chương 6.)

For people moving from a system in which all state changes could be managed in a single transactional boundary, the shift to distributed systems can be a shock, and often the reaction is to look to implement distributed transactions to regain the guarantees that ACID transactions gave us with simpler architectures. Unfortunately, as we’ll cover in depth in “Database Transactions” on page 175, distributed transactions are not only complex to implement, even when done well, but they also don’t actually give us the same guarantees that we came to expect with more narrowly scoped database transactions.

Đối với những người chuyển từ một hệ thống mà mọi thay đổi trạng thái đều có thể được quản lý trong một ranh giới giao dịch duy nhất, thì việc chuyển sang các hệ thống phân tán có thể là một cú sốc, và phản ứng thường thấy là tìm cách hiện thực các giao dịch phân tán (distributed transaction) để giành lại những bảo đảm mà các giao dịch ACID đã cho chúng ta với những kiến trúc đơn giản hơn. Đáng tiếc là, như chúng ta sẽ trình bày chuyên sâu ở mục “Database Transactions” trang 175, các giao dịch phân tán không chỉ phức tạp để hiện thực, kể cả khi được làm tốt, mà chúng còn thực sự không cho chúng ta cùng những bảo đảm mà chúng ta đã quen kỳ vọng với các giao dịch cơ sở dữ liệu có phạm vi hẹp hơn.

As we go on to explore in “Sagas” on page 182, there are alternative (and preferable) mechanisms to distributed transactions for managing state changes across multiple microservices, but they come with new sources of complexity. As with data integrity, we have to come to terms with the fact that by breaking apart our databases for what may be very good reasons, we will encounter a new set of problems.

Như chúng ta sẽ tiếp tục khám phá ở mục “Sagas” trang 182, có những cơ chế thay thế (và đáng ưa chuộng hơn) cho giao dịch phân tán để quản lý các thay đổi trạng thái xuyên suốt nhiều microservice, nhưng chúng đi kèm với những nguồn phức tạp mới. Cũng giống như với toàn vẹn dữ liệu, chúng ta phải chấp nhận thực tế rằng bằng cách tách rời các cơ sở dữ liệu của mình vì những lý do có thể rất chính đáng, chúng ta sẽ gặp phải một loạt vấn đề mới.

Tooling — Công cụ (Tooling)

Changing databases is difficult for many reasons, one of which is that limited tools remain available to allow us to make changes easily. With code, we have refactoring tooling built into our IDEs, and we have the added benefit that the systems we are changing are fundamentally stateless. With a database, the things we are changing have state, and we also lack good refactoring-type tooling.

Việc thay đổi các cơ sở dữ liệu khó khăn vì nhiều lý do, một trong số đó là có ít công cụ còn sẵn để cho phép chúng ta thực hiện các thay đổi một cách dễ dàng. Với mã nguồn, chúng ta có công cụ refactoring tích hợp sẵn trong các IDE, và chúng ta có thêm lợi thế là những hệ thống mà chúng ta đang thay đổi về cơ bản là phi trạng thái (stateless). Với một cơ sở dữ liệu, những thứ mà chúng ta đang thay đổi lại có trạng thái, và chúng ta cũng thiếu các công cụ kiểu refactoring tốt.

There are many tools out there to help you manage the process of changing the schema of a relational database, but most follow the same pattern. Each schema change is defined in a version-controlled delta script. These scripts are then run in strict order in an idempotent manner. Rails migrations work in this way, as did DBDeploy, a tool I helped create many years ago.

Có rất nhiều công cụ ngoài kia để giúp bạn quản lý quá trình thay đổi schema của một cơ sở dữ liệu quan hệ, nhưng hầu hết đều tuân theo cùng một pattern. Mỗi thay đổi schema được định nghĩa trong một script delta được quản lý phiên bản. Các script này sau đó được chạy theo thứ tự nghiêm ngặt theo cách idempotent. Rails migrations hoạt động theo cách này, DBDeploy cũng vậy—một công cụ mà tôi đã góp phần tạo ra nhiều năm trước.

Nowadays I point people to either Flyway or Liquibase to achieve the same outcome, if they don't already have a tool that works in this way.

Ngày nay tôi hướng mọi người đến với hoặc Flyway hoặc Liquibase để đạt được cùng kết quả đó, nếu họ chưa có sẵn một công cụ hoạt động theo cách này.

Reporting Database — Cơ sở dữ liệu báo cáo (Reporting Database)

As part of extracting microservices from our monolithic application, we also break apart our databases, as we want to hide access to our internal data storage. By hiding direct access to our databases, we are better able to create stable interfaces, which make **independent deployability** possible. Unfortunately, this causes us issues when we do have legitimate use cases for accessing data from more than one microservice, or when that data is better made available in a database, rather than via something like a **REST API**.

Như một phần của việc trích xuất các microservice ra khỏi ứng dụng monolithic của mình, chúng ta cũng tách rời các cơ sở dữ liệu, vì chúng ta muốn che giấu việc truy cập tới phần lưu trữ dữ liệu nội bộ của mình. Bằng cách che giấu việc truy cập trực tiếp tới các cơ sở dữ liệu của mình, chúng ta có khả năng tốt hơn để tạo ra các interface ổn định, vốn là thứ giúp cho khả năng triển khai độc lập trở nên khả thi. Đáng tiếc là, điều này gây ra cho chúng ta những vấn đề khi chúng ta thực sự có những tình huống sử dụng chính đáng cần truy cập dữ liệu từ nhiều hơn một microservice, hoặc khi dữ liệu đó tốt hơn nên được làm cho sẵn có trong một cơ sở dữ liệu, thay vì thông qua một thứ gì đó như REST API.

With a reporting database, we instead create a dedicated database that is designed for external access, and we make it the responsibility of the microservice to push data from internal storage

to the externally accessible reporting database, as seen in Figure 3-8 .

Với một cơ sở dữ liệu báo cáo, thay vào đó chúng ta tạo ra một cơ sở dữ liệu chuyên dụng được thiết kế cho việc truy cập từ bên ngoài, và chúng ta đặt trách nhiệm cho microservice là đẩy dữ liệu từ phần lưu trữ nội bộ sang cơ sở dữ liệu báo cáo có thể truy cập từ bên ngoài, như được thấy trong Hình 3-8.

The reporting database allows us to hide internal state management, while still presenting the data in a database—something which can be very useful. For example, you might want to allow people to run off ad hoc defined SQL queries, run large-scale joins, or make use of existing toolchains that expect to have access to a SQL endpoint. The reporting database is a nice solution to this problem.

Cơ sở dữ liệu báo cáo cho phép chúng ta che giấu việc quản lý trạng thái nội bộ, đồng thời vẫn trình bày dữ liệu trong một cơ sở dữ liệu—một điều có thể rất hữu ích. Ví dụ, bạn có thể muốn cho phép mọi người chạy các truy vấn SQL được định nghĩa tùy biến (ad hoc), chạy các phép join quy mô lớn, hoặc tận dụng các chuỗi công cụ (toolchain) hiện có vốn kỳ vọng được truy cập tới một điểm cuối (endpoint) SQL. Cơ sở dữ liệu báo cáo là một giải pháp hay cho vấn đề này.

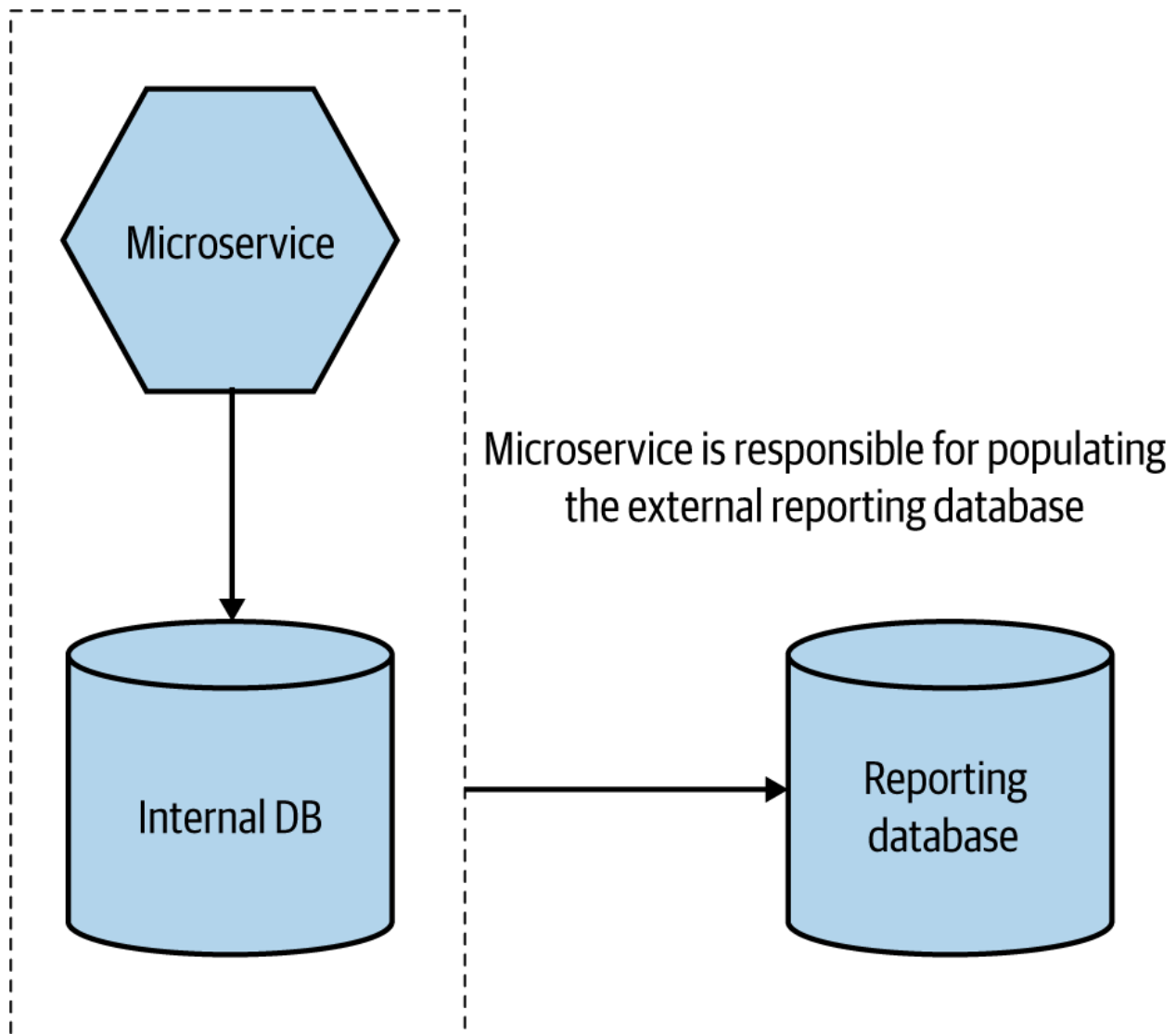


Figure 3-8. An overview of the reporting database pattern

Hình 3-8. Tổng quan về pattern cơ sở dữ liệu báo cáo

There are two key points to highlight here. Firstly, we still want to practice **information hiding**. So we should expose only the bare minimum of data in the reporting database. This means that what is in the reporting database may be only a subset of the data the microservice stores. However, as this is not a direct mapping, it creates the opportunity to come up with a schema design for the reporting database that is tailored exactly to the requirements of the consumers—this could involve using a radically different schema, or perhaps even a different type of database technology altogether.

Có hai điểm then chốt cần nhấn mạnh ở đây. Thứ nhất, chúng ta vẫn muốn thực hành việc che giấu thông tin. Vì vậy chúng ta nên chỉ phơi bày lượng dữ liệu tối thiểu nhất có thể trong cơ sở dữ liệu báo cáo. Điều này có nghĩa là những gì có trong cơ sở dữ liệu báo cáo có thể chỉ là một tập con của dữ liệu mà microservice lưu trữ. Tuy nhiên, vì đây không phải là một sự ánh xạ trực tiếp, nó tạo ra cơ hội để nghĩ ra một thiết kế schema cho cơ sở dữ liệu báo cáo được “may đo” chính xác theo các yêu cầu của bên tiêu thụ (consumer)—điều này có thể bao gồm việc sử dụng một schema khác biệt hoàn toàn, hoặc thậm chí có lẽ là một loại công nghệ cơ sở dữ liệu hoàn toàn khác.

The second key point is that the reporting database should be treated like any other microservice endpoint, and it is the job of the microservice maintainer to ensure that compatibility of this endpoint is maintained even if the microservice changes its internal implementation detail. The mapping from internal state to reporting database is the responsibility of the people who develop the microservice itself.

Điểm then chốt thứ hai là cơ sở dữ liệu báo cáo nên được đối xử giống như bất kỳ điểm cuối microservice nào khác, và việc đảm bảo rằng tính tương thích của điểm cuối này được duy trì kể cả khi microservice thay đổi chi tiết hiện thực nội bộ của nó là nhiệm vụ của người bảo trì microservice. Việc ánh xạ từ trạng thái nội bộ sang cơ sở dữ liệu báo cáo là trách nhiệm của những người phát triển chính microservice đó.

Summary — Tóm tắt

So, to distill things down, when embarking on work to migrate functionality from a monolithic architecture to a microservice architecture, you must have a clear understanding of what you expect to achieve. This goal will shape how you go about the work and will also help you understand whether you're moving in the right direction.

Vậy, để cô đọng mọi thứ lại, khi bắt tay vào công việc di trú chức năng từ một kiến trúc monolithic sang một kiến trúc microservice, bạn phải hiểu rõ ràng điều bạn kỳ vọng đạt được. Mục tiêu này sẽ định hình cách bạn tiến hành công việc và cũng sẽ giúp bạn hiểu liệu bạn có đang đi đúng hướng hay không.

Migration should be incremental. Make a change, roll that change out, assess it, and go again. Even the act of splitting out one microservice can itself be broken down into a series of small steps.

Việc di trú nên diễn ra theo từng bước. Thực hiện một thay đổi, triển khai thay đổi đó, đánh giá nó, rồi lại làm tiếp. Ngay cả hành động tách ra một microservice cũng tự nó có thể được chia nhỏ thành một loạt các bước nhỏ.

CHAPTER 4 Microservice

Communication Styles — Chương 4:

Các phong cách giao tiếp giữa Microservice

Getting communication between **microservices** right is problematic for many due in great part, I feel, to the fact that people gravitate toward a chosen technological approach without first considering the different types of communication they might want. In this chapter, I'll try and tease apart the different styles of communication to help you understand the pros and cons of each, as well as which approach will best fit your problem space.

Việc thiết lập giao tiếp giữa các microservice cho đúng là một bài toán nan giải với nhiều người, mà phần lớn nguyên nhân, theo tôi cảm nhận, đến từ việc người ta có xu hướng lao ngay vào một cách tiếp cận công nghệ đã chọn trước mà chưa cân nhắc đến các kiểu giao tiếp khác nhau mà họ có thể muốn dùng. Trong chương này, tôi sẽ cố gắng bóc tách các phong cách giao tiếp khác nhau để giúp bạn hiểu được ưu nhược điểm của từng phong cách, cũng như cách tiếp cận nào phù hợp nhất với không gian bài toán của bạn.

We'll be looking at **synchronous blocking** and **asynchronous nonblocking** communication mechanisms, as well as comparing **request-response** collaboration with **event-driven** collaboration.

Chúng ta sẽ xem xét các cơ chế giao tiếp đồng bộ có chặn (synchronous blocking) và bất đồng bộ không chặn (asynchronous nonblocking), cũng như so sánh kiểu cộng tác request-response với kiểu cộng tác hướng sự kiện (event-driven).

By the end of this chapter you should be much better prepared to understand the different options available to you and will have a foundational knowledge that will help when we start looking at more detailed implementation concerns in the following chapters.

Đến cuối chương này, bạn sẽ được chuẩn bị tốt hơn nhiều để hiểu các lựa chọn khác nhau sẵn có cho mình, và sẽ có một nền tảng kiến thức cơ bản hữu ích khi chúng ta bắt đầu đi sâu vào những vấn đề triển khai chi tiết hơn ở các chương sau.

From In-Process to Inter-Process — Từ trong tiến trình đến giữa các tiến trình

OK, let's get the easy stuff out of the way first—or at least what I hope is the easy stuff. Namely, calls between different processes across a network (inter-process) are very different from calls within

a single process (in-process). At one level, we can ignore this distinction. It's easy, for example, to think of one object making a method call on another object and then just map this interaction to two microservices communicating via a network. Putting aside the fact that microservices aren't just objects, this thinking can get us into a lot of trouble.

Được rồi, hãy giải quyết những thứ dễ trước—hoặc ít nhất là những gì tôi hy vọng là dễ. Cụ thể, các lời gọi giữa những tiến trình khác nhau qua mạng (giữa các tiến trình, inter-process) rất khác với các lời gọi bên trong một tiến trình đơn lẻ (trong tiến trình, in-process). Ở một mức độ nào đó, ta có thể bỏ qua sự khác biệt này. Chẳng hạn, thật dễ để hình dung một đối tượng gọi một phương thức trên một đối tượng khác rồi đơn giản ánh xạ tương tác đó thành hai microservice giao tiếp qua mạng. Gạt qua một bên thực tế rằng microservice không chỉ đơn thuần là đối tượng, lối suy nghĩ này có thể đẩy chúng ta vào rất nhiều rắc rối.

Let's look at some of these differences and how they might change how you think about the interactions between your microservices.

Hãy xem xét một số khác biệt này và cách chúng có thể thay đổi cách bạn suy nghĩ về các tương tác giữa các microservice của mình.

Performance — Hiệu năng

The performance of an in-process call is fundamentally different from that of an inter-process call. When I make an in-process call, the underlying compiler and runtime can carry out a whole host of optimizations to reduce the impact of the call, including inlining the invocation so it's as though there was never a call in the first place. No such optimizations are possible with inter-process calls. Packets have to be sent. Expect the overhead of an inter-process call to be significant compared to the overhead of an in-process call. The former is very measurable—just round-tripping a single packet in a data center is measured in milliseconds—whereas the overhead of making a method call is something you don't need to worry about.

Hiệu năng của một lời gọi trong tiến trình về cơ bản khác với hiệu năng của một lời gọi giữa các tiến trình. Khi tôi thực hiện một lời gọi trong tiến trình, trình biên dịch và runtime bên dưới có thể thực hiện hàng loạt tối ưu hóa để giảm thiểu chi phí của lời gọi, bao gồm cả việc nội tuyến (inlining) lời gọi sao cho như thể chưa từng có lời gọi nào ngay từ đầu. Không có tối ưu hóa nào như vậy khả thi với các lời gọi giữa các tiến trình. Các gói tin phải được gửi đi. Hãy kỳ vọng rằng chi phí phụ trội (overhead) của một lời gọi giữa các tiến trình là đáng kể so với chi phí của một lời gọi trong tiến trình. Cái trước rất dễ đo lường—chỉ riêng việc một gói tin đi một vòng khứ hồi trong một trung tâm dữ liệu đã được đo bằng mili-giây—trong khi chi phí của việc thực hiện một lời gọi phương thức là thứ bạn không cần phải bận tâm.

This can often lead you to want to rethink APIs. An **API** that makes sense in-process may not make sense in inter-process situations. I can make one thousand calls across an API boundary in-process without concern. Do I want to make one thousand network calls between two microservices? Perhaps not.

Điều này thường khiến bạn muốn suy nghĩ lại về các API. Một API hợp lý trong tiến trình có thể không hợp lý trong các tình huống giữa các tiến trình. Tôi có thể thực hiện một nghìn lời gọi qua một ranh giới API trong tiến trình mà không lo lắng gì. Liệu tôi có muốn thực hiện một nghìn lời gọi mạng giữa hai microservice không? Có lẽ là không.

When I pass a parameter into a method, the data structure I pass in typically doesn't move—what's

more likely is that I pass around a pointer to a memory location. Passing in an object or data structure to another method doesn't necessitate more memory being allocated in order to copy the data.

Khi tôi truyền một tham số vào một phương thức, cấu trúc dữ liệu mà tôi truyền vào thường không di chuyển—điều nhiều khả năng hơn là tôi truyền đi một con trỏ tới một vị trí trong bộ nhớ. Việc truyền một đối tượng hay cấu trúc dữ liệu vào một phương thức khác không đòi hỏi phải cấp phát thêm bộ nhớ để sao chép dữ liệu.

When making calls between microservices over a network, on the other hand, the data actually has to be serialized into some form that can be transmitted over a network. The data then needs to be sent and deserialized at the other end. We therefore may need to be more mindful about the size of payloads being sent between processes. When was the last time you were aware of the size of a data structure that you were passing around inside a process? The reality is that you likely didn't need to know; now you do. This might lead you to reduce the amount of data being sent or received (perhaps not a bad thing if we think about **information hiding**), pick more efficient serialization mechanisms, or even offload data to a filesystem and pass around a reference to that file location instead.

Ngược lại, khi thực hiện các lời gọi giữa các microservice qua mạng, dữ liệu thực sự phải được tuần tự hóa (serialize) thành một dạng nào đó có thể truyền qua mạng. Sau đó dữ liệu cần được gửi đi và giải tuần tự hóa (deserialize) ở đầu bên kia. Vì vậy chúng ta có thể cần để tâm hơn đến kích thước của các payload được gửi giữa các tiến trình. Lần gần nhất bạn ý thức được kích thước của một cấu trúc dữ liệu mà bạn đang truyền đi bên trong một tiến trình là khi nào? Thực tế là bạn có lẽ đã không cần biết; còn bây giờ thì có. Điều này có thể dẫn bạn đến việc giảm lượng dữ liệu được gửi đi hoặc nhận về (có lẽ không phải điều tệ nếu ta nghĩ đến việc che giấu thông tin—**information hiding**), chọn các cơ chế tuần tự hóa hiệu quả hơn, hoặc thậm chí đẩy dữ liệu ra một hệ thống tệp (filesystem) rồi truyền đi một tham chiếu tới vị trí của tệp đó thay vì truyền dữ liệu.

These differences may not cause you issues straight away, but you certainly need to be aware of them. I've seen a lot of attempts to hide from the developer the fact that a network call is even taking place. Our desire to create abstractions to hide detail is a big part of what allows us to do more things more efficiently, but sometimes we create abstractions that hide too much. A developer needs to be aware if they are doing something that will result in a network call; otherwise, you should not be surprised if you end up with some nasty performance bottlenecks further down the line caused by odd inter-**service** interactions that weren't visible to the developer writing the code.

Những khác biệt này có thể không gây cho bạn vấn đề ngay lập tức, nhưng chắc chắn bạn cần phải ý thức về chúng. Tôi đã chứng kiến rất nhiều nỗ lực nhằm giấu đi với lập trình viên cái sự thật rằng thậm chí một lời gọi mạng đang diễn ra. Khao khát tạo ra các lớp trừu tượng để giấu chi tiết là một phần lớn trong những gì cho phép chúng ta làm được nhiều việc hơn một cách hiệu quả hơn, nhưng đôi khi chúng ta tạo ra những lớp trừu tượng giấu đi quá nhiều. Một lập trình viên cần ý thức được liệu họ có đang làm điều gì đó sẽ dẫn đến một lời gọi mạng hay không; nếu không, bạn đừng nên ngạc nhiên nếu rốt cuộc gặp phải một số nút thắt cổ chai hiệu năng khó chịu về sau gây ra bởi những tương tác liên dịch vụ kỳ quặc mà lập trình viên viết code không hề nhìn thấy.

Changing Interfaces — Thay đổi giao diện (Interface)

When we consider changes to an **interface** inside a process, the act of rolling out the change is straightforward. The code implementing the interface and the code calling the interface are all packaged together in the same process. In fact, if I change a method signature using an IDE with

refactoring capability, often the IDE itself will automatically refactor calls to this changing method. Rolling out such a change can be done in an atomic fashion—both sides of the interface are packaged together in a single process.

Khi chúng ta cân nhắc các thay đổi đối với một giao diện bên trong một tiến trình, hành động triển khai thay đổi đó là chuyện đơn giản. Code hiện thực giao diện và code gọi giao diện đều được đóng gói cùng nhau trong cùng một tiến trình. Trên thực tế, nếu tôi thay đổi chữ ký của một phương thức bằng một IDE có khả năng refactor, thường thì chính IDE đó sẽ tự động refactor các lời gọi tới phương thức đang thay đổi này. Việc triển khai một thay đổi như vậy có thể được thực hiện theo cách nguyên tử (atomic)—cả hai phía của giao diện được đóng gói cùng nhau trong một tiến trình duy nhất.

With communication between microservices, however, the **microservice** exposing an interface and the consuming microservices using that interface are separately deployable microservices. When making a backward-incompatible change to a microservice interface, we either need to do a lockstep **deployment** with consumers, making sure they are updated to use the new interface, or else find some way to phase the rollout of the new microservice contract. We'll explore this concept in more detail later in this chapter.

Tuy nhiên, với giao tiếp giữa các microservice, microservice phơi bày một giao diện và các microservice tiêu thụ sử dụng giao diện đó là những microservice triển khai (deploy) độc lập với nhau. Khi thực hiện một thay đổi không tương thích ngược (backward-incompatible) đối với giao diện của một microservice, hoặc là chúng ta cần làm một đợt triển khai đồng loạt (lockstep deployment) với các bên tiêu thụ, đảm bảo chúng được cập nhật để dùng giao diện mới, hoặc là tìm cách nào đó để chia giai đoạn cho việc triển khai hợp đồng (contract) mới của microservice. Chúng ta sẽ khám phá khái niệm này chi tiết hơn ở phần sau của chương.

Error Handling — Xử lý lỗi (Error Handling)

Within a process, if I call a method, the nature of the errors tends to be pretty straightforward. Simplistically, the errors either are expected and easy to handle, or they are catastrophic to the point that we just propagate the error up the call stack. Errors, on the whole, are deterministic.

Bên trong một tiến trình, nếu tôi gọi một phương thức, bản chất của các lỗi có xu hướng khá đơn giản. Nói một cách giản lược, các lỗi hoặc là được lường trước và dễ xử lý, hoặc là thậm chí đến mức ta chỉ việc đẩy lỗi lên trên theo ngăn xếp lời gọi (call stack). Nhìn chung, các lỗi mang tính tất định (deterministic).

With a **distributed system**, the nature of errors can be different. You are vulnerable to a host of errors that are outside your control. Networks time out. Downstream microservices might be temporarily unavailable. Networks get disconnected, containers get killed due to consuming too much memory, and in extreme situations, bits of your data center can catch fire. 1

Với một hệ thống phân tán, bản chất của các lỗi có thể khác đi. Bạn dễ tổn thương trước hàng loạt lỗi nằm ngoài tầm kiểm soát của mình. Mạng bị quá hạn (time out). Các microservice ở hạ nguồn (downstream) có thể tạm thời không khả dụng. Mạng bị ngắt kết nối, container bị giết do tiêu thụ quá nhiều bộ nhớ, và trong những tình huống cực đoan, vài phần trong trung tâm dữ liệu của bạn có thể bốc cháy. 1

In their book *Distributed Systems*, 2 Andrew Tanenbaum and Maarten Steen break down the five types of failure modes you can see when looking at an inter-process communication. Here's a simplified version:

Trong cuốn sách Distributed Systems của họ, 2 Andrew Tanenbaum và Maarten Steen phân tích năm kiểu chế độ hỏng (failure mode) mà bạn có thể gặp khi xem xét giao tiếp giữa các tiến trình. Dưới đây là một phiên bản giản lược:

Crash failure Everything was fine till the server crashed. Reboot!

Hỏng do sập (Crash failure) Mọi thứ vẫn ổn cho đến khi máy chủ sập. Khởi động lại thôi!

1 True story.

1 Chuyện có thật.

2 Maarten van Steen and Andrew S. Tanenbaum, Distributed Systems , 3rd ed. (Scotts Valley, CA: CreateSpace Independent Publishing Platform, 2017).

2 Maarten van Steen and Andrew S. Tanenbaum, Distributed Systems , 3rd ed. (Scotts Valley, CA: CreateSpace Independent Publishing Platform, 2017).

Omission failure You sent something, but you didn't get a response. Also includes situations in which you expect a downstream microservice to be firing messages (perhaps including events), and it just stops.

Hỏng do thiếu sót (Omission failure) Bạn đã gửi đi cái gì đó, nhưng không nhận được phản hồi. Cũng bao gồm các tình huống trong đó bạn kỳ vọng một microservice hạ nguồn sẽ bắn ra các thông điệp (có thể gồm cả sự kiện), nhưng nó lại ngừng bật.

Timing failure Something happened too late (you didn't get it in time), or something happened too early!

Hỏng về thời gian (Timing failure) Có điều gì đó xảy ra quá muộn (bạn không nhận được nó kịp lúc), hoặc có điều gì đó xảy ra quá sớm!

Response failure You got a response, but it just seems wrong. For example, you asked for an order summary, but needed pieces of information are missing in the response.

Hỏng về phản hồi (Response failure) Bạn nhận được một phản hồi, nhưng nó có vẻ sai. Ví dụ, bạn yêu cầu một bản tóm tắt đơn hàng, nhưng những mẫu thông tin cần thiết lại thiếu trong phản hồi.

Arbitrary failure Otherwise known as Byzantine failure, this is when something has gone wrong, but participants are unable to agree if the failure has occurred (or why). As it sounds like, this is a bad time all around.

Hỏng tùy tiện (Arbitrary failure) Còn được gọi là hỏng kiểu Byzantine (Byzantine failure), đây là khi có điều gì đó đã trục trặc, nhưng các bên tham gia không thể thống nhất được liệu lỗi có thực sự xảy ra hay không (hoặc vì sao). Đúng như cái tên gợi ý, đây là một tình huống tồi tệ về mọi mặt.

Many of these errors are often transient in nature—they are short-lived problems that might go away. Consider the situation in which we send a request to a microservice but don't hear back (a type of omission failure). This could mean that the downstream microservice never got the request in the first place, so we need to send it again. Other problems can't be dealt with easily and might need a human operator to intervene. As a result, it can become important to have a richer set of semantics for returning errors in a way that can allow clients to take appropriate action.

Nhiều lỗi trong số này thường mang tính nhất thời (transient)—chúng là những vấn đề ngắn ngủi có thể tự biến mất. Hãy xét tình huống trong đó chúng ta gửi một yêu cầu đến một microservice nhưng không nhận được hồi đáp (một dạng hỏng do thiếu sót). Điều này có thể có nghĩa là microservice hạ nguồn ngay từ đầu chưa từng nhận được yêu cầu, nên

chúng ta cần gửi lại nó. Những vấn đề khác thì không thể xử lý dễ dàng và có thể cần một người vận hành can thiệp. Kết quả là, việc có một tập ngữ nghĩa phong phú hơn để trả về lỗi theo cách cho phép phía client thực hiện hành động phù hợp có thể trở nên quan trọng.

HTTP is an example of a protocol that understands the importance of this. Every HTTP response has a code, with the 400 and 500 series codes being reserved for errors. 400 series error codes are request errors—essentially, a downstream service is telling the client that there is something wrong with the original request. As such, it's probably something you should give up on—is there any point retrying a 404 Not Found, for example? The 500 series response codes relate to downstream issues, a subset of which indicate to the client that the issue might be temporary. A 503 Service Unavailable, for example, indicates that the downstream server is unable to handle the request, but it may be a temporary state, in which case an upstream client might decide to **retry** the request. On the other hand, if a client receives a 501 Not Implemented response, a retry is unlikely to help much.

HTTP là một ví dụ về một giao thức hiểu được tầm quan trọng của điều này. Mỗi phản hồi HTTP đều có một mã, với dải mã 400 và 500 được dành riêng cho các lỗi. Các mã lỗi dải 400 là lỗi yêu cầu (request error)—về cơ bản, một dịch vụ hạ nguồn đang nói với client rằng có điều gì đó sai với yêu cầu ban đầu. Như vậy, đó có lẽ là điều bạn nên từ bỏ—chẳng hạn, có ích gì khi thử lại một mã 404 Not Found? Các mã phản hồi dải 500 liên quan đến các vấn đề ở hạ nguồn, một tập con trong đó cho client biết rằng vấn đề có thể chỉ là tạm thời. Ví dụ, mã 503 Service Unavailable cho biết rằng máy chủ hạ nguồn không thể xử lý yêu cầu, nhưng đó có thể là một trạng thái tạm thời, trong trường hợp đó một client thượng nguồn (upstream) có thể quyết định thử lại yêu cầu. Mặt khác, nếu một client nhận được phản hồi 501 Not Implemented, thì việc thử lại nhiều khả năng cũng chẳng giúp được gì.

Whether or not you pick an HTTP-based protocol for communication between microservices, if you have a rich set of semantics around the nature of the error, you'll make it easier for clients to carry out compensating actions, which in turn should help you **build** more robust systems.

Dù bạn có chọn một giao thức dựa trên HTTP cho giao tiếp giữa các microservice hay không, nếu bạn có một tập ngữ nghĩa phong phú xoay quanh bản chất của lỗi, bạn sẽ giúp client thực hiện các hành động bù trừ (compensating action) dễ dàng hơn, điều này đến lượt nó sẽ giúp bạn xây dựng những hệ thống vững chắc hơn.

Technology for Inter-Process Communication: So Many Choices — Công nghệ cho giao tiếp giữa các tiến trình: Quá nhiều lựa chọn

And in a world where we have too many choices and too little time, the obvious thing to do is just ignore stuff. —Seth Godin

Và trong một thế giới mà chúng ta có quá nhiều lựa chọn và quá ít thời gian, điều hiển nhiên cần làm là cứ bỏ qua mọi thứ. —Seth Godin

The range of technology available to us for inter-process communication is vast. As a result, we can often be overburdened with choice. I often find that people gravitate to technology that is familiar to them, or perhaps just to the latest hot technology they learned about from a conference. The problem with this is that when you buy into a specific technology choice, you are often buying into a set of ideas and constraints that come along for the ride. These constraints might not be the right ones for you— and the mindset behind the technology may not actually line up with the problem you are

trying to solve.

Phạm vi công nghệ sẵn có cho chúng ta để giao tiếp giữa các tiến trình là khổng lồ. Kết quả là, chúng ta thường bị quá tải bởi lựa chọn. Tôi thường thấy người ta có xu hướng nghiêng về công nghệ quen thuộc với họ, hoặc có lẽ chỉ là công nghệ thời thượng mới nhất mà họ vừa biết được tại một hội thảo. Vấn đề của chuyện này là khi bạn đắm thân vào một lựa chọn công nghệ cụ thể, bạn thường đồng thời đắm thân vào một tập các ý tưởng và ràng buộc đi kèm theo. Những ràng buộc này có thể không phải là những ràng buộc đúng đắn cho bạn—và tư duy đằng sau công nghệ đó có thể thực ra không khớp với bài toán bạn đang cố giải.

If you're trying to build a website, single page app technology like Angular or React is a bad fit. Likewise, trying to use Kafka for request-response really isn't a good idea, as it was designed for more event-based interactions (topics we'll get to in just a moment). And yet I see technology used in the wrong place time and time again. People pick the shiny new tech (like microservices!) without considering whether it actually fits their problem.

Nếu bạn đang cố xây dựng một website, công nghệ ứng dụng trang đơn (single page app) như Angular hay React là một lựa chọn không phù hợp. Tương tự, cố dùng Kafka cho request-response thực sự không phải là một ý hay, bởi nó được thiết kế cho các tương tác hướng sự kiện hơn (những chủ đề mà chúng ta sẽ đề cập ngay sau đây). Vậy mà tôi vẫn thấy công nghệ được dùng sai chỗ hết lần này đến lần khác. Người ta chọn món công nghệ mới bóng bẩy (như microservice chẳng hạn!) mà không cân nhắc liệu nó có thực sự phù hợp với bài toán của họ hay không.

Thus when it comes to the bewildering array of technology available to us for communication between microservices, I think it's important to talk first about the style of communication you want, and only then look for the right technology to implement that style. With that in mind, let's take a look at a model I've been using for several years to help distinguish between the different approaches for microservice-to-microservice communication, which in turn can help you filter the technology options you'll want to look at.

Vì vậy khi nói đến cái mớ công nghệ rối rắm sẵn có cho chúng ta để giao tiếp giữa các microservice, tôi cho rằng điều quan trọng là trước hết phải bàn về phong cách giao tiếp mà bạn muốn, rồi mới đi tìm công nghệ phù hợp để hiện thực phong cách đó. Với suy nghĩ đó, hãy cùng xem một mô hình mà tôi đã sử dụng trong nhiều năm để giúp phân biệt các cách tiếp cận khác nhau cho giao tiếp giữa microservice với microservice, điều này đến lượt nó có thể giúp bạn lọc ra các lựa chọn công nghệ mà bạn sẽ muốn xem xét.

Styles of Microservice Communication — Các phong cách giao tiếp giữa Microservice

In Figure 4-1 we see an outline for the model I use for thinking about different styles of communication. This model is not meant to be entirely exhaustive (I'm not trying to present a grand unified theory of inter-process communication here), but it provides a good high-level overview for considering the different styles of communication that are most widely used for microservice architectures.

Trong Hình 4-1 chúng ta thấy một bản phác thảo cho mô hình mà tôi dùng để suy nghĩ về các phong cách giao tiếp khác nhau. Mô hình này không nhằm mục đích hoàn toàn đầy đủ (tôi không cố trình bày một lý thuyết thống nhất vĩ đại về giao tiếp giữa các tiến trình ở

đây đâu), nhưng nó cung cấp một cái nhìn tổng quan ở mức cao tốt để cân nhắc các phong cách giao tiếp khác nhau được sử dụng rộng rãi nhất cho các kiến trúc microservice.

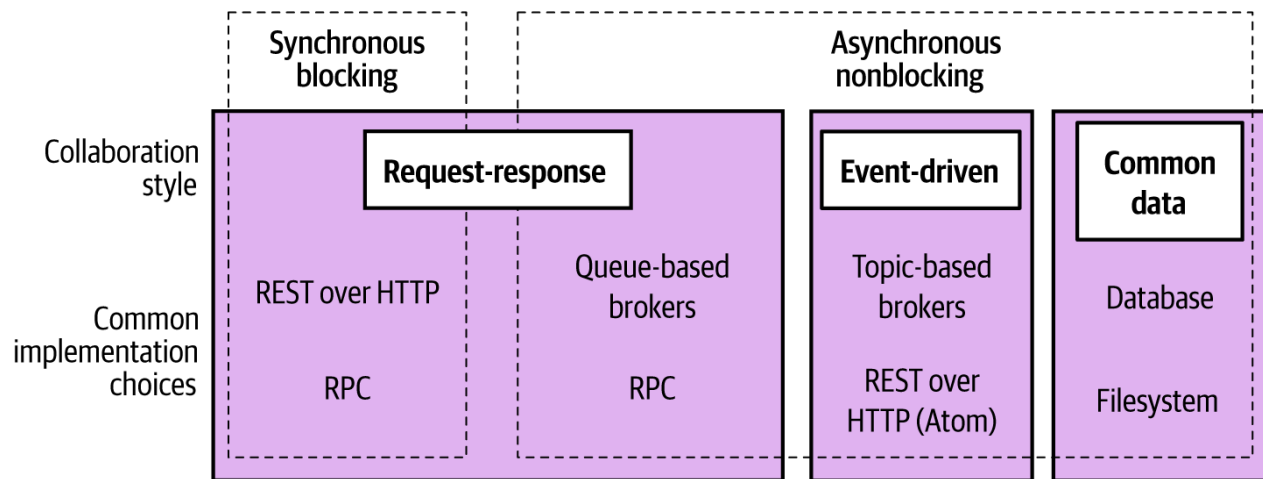


Figure 4-1. Different styles of inter-microservice communication along with example implementing technologies

Hình 4-1. Các phong cách giao tiếp khác nhau giữa các microservice cùng với các công nghệ hiện thực ví dụ

We'll look at the different elements of this model in more detail shortly, but first I'd like to briefly outline them:

Chúng ta sẽ xem xét các thành phần khác nhau của mô hình này chi tiết hơn ngay sau đây, nhưng trước tiên tôi muốn phác thảo sơ qua về chúng:

Synchronous blocking A microservice makes a call to another microservice and blocks operation waiting for the response.

Đồng bộ có chặn (Synchronous blocking) Một microservice thực hiện một lời gọi đến một microservice khác và chặn hoạt động lại để chờ phản hồi.

Asynchronous nonblocking The microservice emitting a call is able to carry on processing whether or not the call is received.

Bất đồng bộ không chặn (Asynchronous nonblocking) Microservice phát ra lời gọi có thể tiếp tục xử lý dù lời gọi có được nhận hay không.

Request-response A microservice sends a request to another microservice asking for something to be done. It expects to receive a response informing it of the result.

Request-response Một microservice gửi một yêu cầu đến một microservice khác để nghị làm một việc gì đó. Nó kỳ vọng nhận được một phản hồi thông báo cho nó về kết quả.

Event-driven Microservices emit events, which other microservices consume and react to accordingly. The microservice emitting the event is unaware of which microservices, if any, consume the events it emits.

Hướng sự kiện (Event-driven) Các microservice phát ra sự kiện, mà các microservice khác tiêu thụ và phản ứng tương ứng. Microservice phát ra sự kiện không hề biết microservice nào, nếu có, tiêu thụ những sự kiện mà nó phát ra.

Common data Not often seen as a communication style, microservices collaborate via some shared data source.

Dữ liệu dùng chung (Common data) Tuy không thường được xem là một phong cách giao tiếp, các microservice cộng tác thông qua một nguồn dữ liệu dùng chung nào đó.

When using this model to help teams decide on the right approach, I spend a lot of time understanding the context in which they are operating. Their needs in terms of reliable communication, acceptable latency, and volume of communication are all going to play a part in making a technology choice. But in general, I tend to start with deciding whether a request-response or an event-driven style of collaboration is more appropriate for the given situation. If I'm looking at request-response, then both synchronous and asynchronous implementations are still available to me, so I have a second choice to make. If picking an event-driven collaboration style, however, my implementation choices will be limited to nonblocking asynchronous choices.

Khi dùng mô hình này để giúp các đội nhóm quyết định cách tiếp cận đúng, tôi dành rất nhiều thời gian để hiểu bối cảnh mà họ đang vận hành. Nhu cầu của họ về mặt giao tiếp tin cậy, độ trễ chấp nhận được, và khối lượng giao tiếp đều sẽ đóng vai trò trong việc đưa ra lựa chọn công nghệ. Nhưng nhìn chung, tôi có xu hướng bắt đầu bằng việc quyết định xem một phong cách cộng tác request-response hay hướng sự kiện là phù hợp hơn cho tình huống cụ thể đó. Nếu tôi đang nghiêng về request-response, thì cả hai cách hiện thực đồng bộ và bất đồng bộ đều vẫn còn sẵn cho tôi, nên tôi có một lựa chọn thứ hai phải đưa ra. Tuy nhiên, nếu chọn một phong cách cộng tác hướng sự kiện, thì các lựa chọn hiện thực của tôi sẽ bị giới hạn ở các lựa chọn bất đồng bộ không chặn.

A host of other considerations come into play when picking the right technology that go beyond the style of communication—for example, the need for lower-latency communication, **security**-related aspects, or the ability to scale. It's unlikely that you can make a reasoned technology choice without taking into account the requirements (and constraints) of your specific problem space. When we look at technology options in Chapter 5, we'll discuss some of these issues.

Hàng loạt cân nhắc khác cũng tham gia khi chọn công nghệ đúng, vượt ra ngoài phong cách giao tiếp—ví dụ, nhu cầu giao tiếp độ trễ thấp hơn, các khía cạnh liên quan đến bảo mật, hay khả năng mở rộng (scale). Khó mà đưa ra được một lựa chọn công nghệ có lý lẽ mà không tính đến các yêu cầu (và ràng buộc) của không gian bài toán cụ thể của bạn. Khi chúng ta xem xét các lựa chọn công nghệ ở Chương 5, chúng ta sẽ thảo luận một vài trong số những vấn đề này.

Mix and Match — Pha trộn và kết hợp

It's important to note that a microservice **architecture** as a whole may have a mix of styles of collaboration, and this is typically the norm. Some interactions just make sense as request-response, while others make sense as event-driven. In fact, it's common for a single microservice to implement more than one form of collaboration. Consider an Order microservice that exposes a request-response API that allows for orders to be placed or changed and then fires events when these changes are made.

Điều quan trọng cần lưu ý là một kiến trúc microservice xét tổng thể có thể có một sự pha trộn các phong cách cộng tác, và đây thường là chuyện bình thường. Một số tương tác đơn giản là hợp lý theo kiểu request-response, trong khi một số khác lại hợp lý theo kiểu hướng sự kiện. Trên thực tế, một microservice đơn lẻ hiện thực nhiều hơn một hình thức cộng tác là chuyện phổ biến. Hãy xét một microservice Order với một API request-response cho phép đặt hoặc thay đổi đơn hàng rồi bắn ra các sự kiện khi những thay đổi này được thực hiện.

With that said, let's look at these different styles of communication in more detail.

Như vậy, hãy cùng xem xét các phong cách giao tiếp khác nhau này chi tiết hơn.

Pattern: Synchronous Blocking — Pattern: Đồng bộ có chặn

With a synchronous blocking call, a microservice sends a call of some kind to a downstream process (likely another microservice) and blocks until the call has completed, and potentially until a response has been received. In Figure 4-2, the Order Processor sends a call to the Loyalty microservice to inform it that some points should be added to a customer's account.

Với một lời gọi đồng bộ có chặn, một microservice gửi một lời gọi nào đó đến một tiến trình hạ nguồn (nhiều khả năng là một microservice khác) và chặn lại cho đến khi lời gọi hoàn tất, và có thể là cho đến khi nhận được phản hồi. Trong Hình 4-2, Order Processor gửi một lời gọi đến microservice Loyalty để thông báo rằng cần cộng một số điểm vào tài khoản của khách hàng.

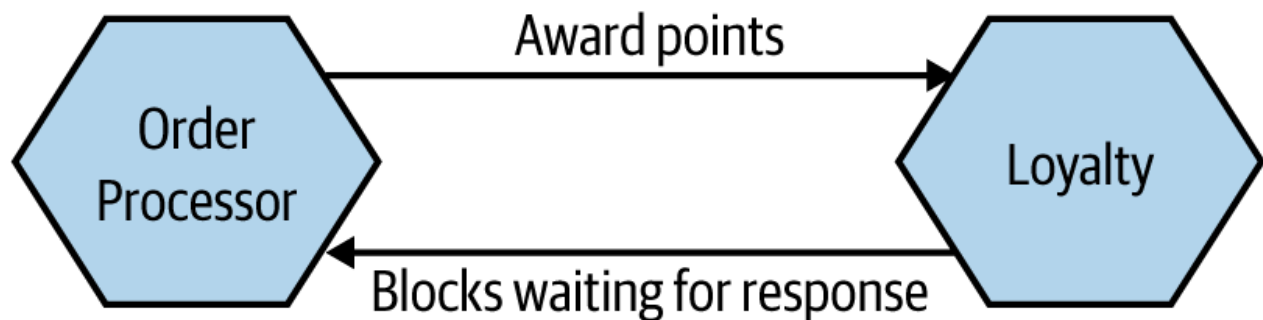


Figure 4-2. Order Processor sends a synchronous call to the Loyalty microservice, blocks, and waits for a response

Hình 4-2. Order Processor gửi một lời gọi đồng bộ đến microservice Loyalty, chặn lại, và chờ phản hồi

Typically, a synchronous blocking call is one that is waiting for a response from the downstream process. This may be because the result of the call is needed for some further operation, or just because it wants to make sure the call worked and to carry out some sort of retry if not. As a result, virtually every synchronous blocking call I see would also constitute being a request-response call, something we'll look at shortly.

Thông thường, một lời gọi đồng bộ có chặn là lời gọi đang chờ phản hồi từ tiến trình hạ nguồn. Điều này có thể là vì kết quả của lời gọi cần thiết cho một thao tác tiếp theo nào đó, hoặc đơn giản là vì nó muốn chắc chắn rằng lời gọi đã thành công và thực hiện thử lại theo kiểu nào đó nếu không. Kết quả là, hầu như mọi lời gọi đồng bộ có chặn mà tôi thấy cũng đều có thể được xem là một lời gọi request-response, điều mà chúng ta sẽ xem xét ngay sau đây.

Advantages — Ưu điểm

There is something simple and familiar about a blocking, synchronous call. Many of us learned to program in a fundamentally synchronous style—reading a piece of code like a script, with each line executing in turn, and with the next line of code waiting its turn to do something. Most of the situations in which you would have used inter-process calls were probably done in a synchronous,

blocking style—running a SQL query on a **database**, for example, or making an HTTP request of a downstream API.

Có điều gì đó đơn giản và quen thuộc về một lời gọi đồng bộ, có chặn. Nhiều người trong chúng ta học lập trình theo một phong cách về cơ bản là đồng bộ—đọc một đoạn code như đọc một kịch bản, với mỗi dòng thực thi lần lượt, và dòng code tiếp theo chờ đến lượt mình để làm việc gì đó. Hầu hết các tình huống trong đó bạn từng dùng các lời gọi giữa các tiến trình có lẽ đều được làm theo phong cách đồng bộ, có chặn—ví dụ, chạy một truy vấn SQL trên một cơ sở dữ liệu, hay thực hiện một yêu cầu HTTP tới một API hạ nguồn.

When moving from a less distributed architecture, like that of a single process **monolith**, sticking with those ideas that are familiar when there is so much else going on that is brand-new can make sense.

Khi chuyển từ một kiến trúc ít phân tán hơn, như kiến trúc của một monolith đơn tiến trình, việc bám lấy những ý tưởng quen thuộc trong khi có quá nhiều thứ khác đang diễn ra hoàn toàn mới mẻ là điều hợp lý.

Disadvantages — Nhược điểm

The main challenge with synchronous calls is the inherent temporal **coupling** that occurs, a topic we explored briefly in Chapter 2 . When the Order Processor makes a call to Loyalty in the preceding example, the Loyalty microservice needs to be reachable for the call to work. If the Loyalty microservice is unavailable, then the call will fail, and Order Processor needs to work out what kind of compensating action to carry out—this might involve an immediate retry, buffering the call to retry later, or perhaps giving up altogether.

Thách thức chính với các lời gọi đồng bộ là sự coupling theo thời gian (temporal coupling) cố hữu xuất hiện, một chủ đề chúng ta đã khám phá sơ qua ở Chương 2. Khi Order Processor thực hiện một lời gọi đến Loyalty trong ví dụ trước, microservice Loyalty cần phải tiếp cận được thì lời gọi mới hoạt động. Nếu microservice Loyalty không khả dụng, thì lời gọi sẽ thất bại, và Order Processor cần phải tính xem nên thực hiện loại hành động bù trừ nào—điều này có thể bao gồm thử lại ngay lập tức, đệm lời gọi lại để thử lại sau, hoặc có lẽ là bỏ cuộc hẳn.

This coupling is two-way. With this style of integration, the response is typically sent over the same inbound network connection to the upstream microservice. So if the Loyalty microservice wants to send a response back to Order Processor , but the upstream instance has subsequently died, the response will get lost. The temporal coupling here isn't just between two microservices; it's between two specific instances of these microservices.

Sự coupling này có tính hai chiều. Với phong cách tích hợp này, phản hồi thường được gửi qua cùng kết nối mạng đến (inbound) tới microservice thượng nguồn. Vậy nên nếu microservice Loyalty muốn gửi một phản hồi trở lại cho Order Processor, nhưng instance thượng nguồn đó sau đó đã chết, thì phản hồi sẽ bị mất. Coupling theo thời gian ở đây không chỉ là giữa hai microservice; nó là giữa hai instance cụ thể của các microservice này.

As the sender of the call is blocking and waiting for the downstream microservice to respond, it also follows that if the downstream microservice responds slowly, or if there is an issue with the latency of the network, then the sender of the call will be blocked for a prolonged period of time waiting for a response. If the Loyalty microservice is under significant load and is responding slowly to requests, this in turn will cause the Order Processor to respond slowly.

Vì bên gửi lời gọi đang chặn và chờ microservice hạ nguồn phản hồi, nên cũng kéo theo rằng nếu microservice hạ nguồn phản hồi chậm, hoặc nếu có vấn đề với độ trễ của mạng, thì bên gửi lời gọi sẽ bị chặn trong một khoảng thời gian kéo dài để chờ phản hồi. Nếu microservice Loyalty đang chịu tải lớn và phản hồi chậm cho các yêu cầu, điều này đến lượt nó sẽ khiến Order Processor phản hồi chậm.

Thus the use of synchronous calls can make a system vulnerable to cascading issues caused by downstream outages more readily than can the use of asynchronous calls.

Như vậy việc dùng các lời gọi đồng bộ có thể khiến một hệ thống dễ tổn thương trước các vấn đề lan truyền theo dây chuyền (cascading) do các sự cố gián đoạn ở hạ nguồn gây ra, dễ hơn so với việc dùng các lời gọi bất đồng bộ.

Where to Use It — Khi nào nên dùng

For simple microservice architectures, I don't have a massive problem with the use of synchronous, blocking calls. Their familiarity for many people is an advantage when coming to grips with distributed systems.

Với các kiến trúc microservice đơn giản, tôi không có vấn đề gì lớn với việc dùng các lời gọi đồng bộ, có chặn. Sự quen thuộc của chúng đối với nhiều người là một lợi thế khi mới bắt đầu làm quen với các hệ thống phân tán.

For me, these types of calls begin to be problematic when you start having more chains of calls—in Figure 4-3, for instance, we have an example flow from MusicCorp, where we are checking a payment for potentially fraudulent activity. The Order Processor calls the Payment service to take payment. The Payment service in turn wants to check with the Fraud Detection microservice as to whether or not this should be allowed. The Fraud Detection microservice in turn needs to get information from the Customer microservice.

Với tôi, những loại lời gọi này bắt đầu trở nên rắc rối khi bạn bắt đầu có nhiều chuỗi lời gọi hơn—chẳng hạn trong Hình 4-3, chúng ta có một luồng ví dụ từ MusicCorp, nơi chúng ta đang kiểm tra một khoản thanh toán xem có hoạt động gian lận tiềm ẩn hay không. Order Processor gọi dịch vụ Payment để thu tiền. Dịch vụ Payment đến lượt mình muốn kiểm tra với microservice Fraud Detection xem liệu điều này có nên được cho phép hay không. Microservice Fraud Detection đến lượt nó cần lấy thông tin từ microservice Customer.

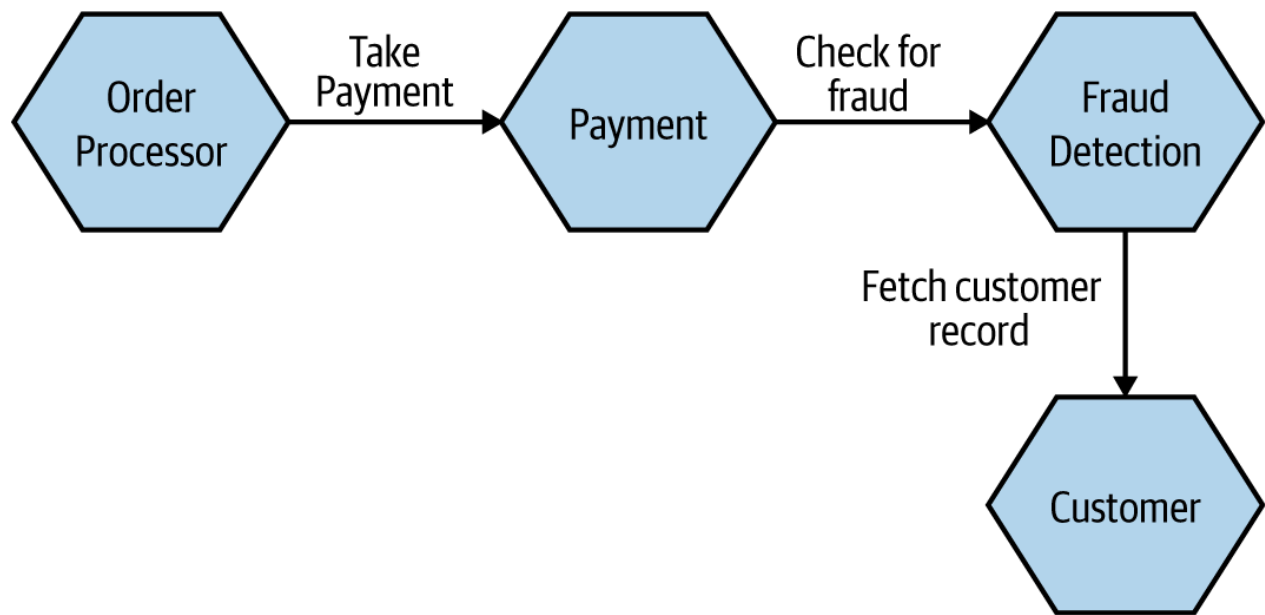


Figure 4-3. Checking for potentially fraudulent behavior as part of order processing flow

Hình 4-3. Kiểm tra hành vi gian lận tiềm ẩn như một phần của luồng xử lý đơn hàng

If all of these calls are synchronous and blocking, there are a number of issues we might face. An issue in any of the four involved microservices, or in the network calls between them, could cause the whole operation to fail. This is quite aside from the fact that these kinds of long chains can cause significant resource contention. Behind the scenes, the Order Processor likely has a network connection open waiting to hear back from Payment. Payment in turn has a network connection open waiting for a response from Fraud Detection, and so on. Having a lot of connections that need to be kept open can have an impact on the running system—you are much more likely to experience issues in which you run out of available connections or suffer from increased network congestion as a result.

Nếu tất cả các lời gọi này đều đồng bộ và có chặn, có một số vấn đề mà chúng ta có thể gặp phải. Một sự cố trong bất kỳ microservice nào trong bốn microservice liên quan, hoặc trong các lời gọi mạng giữa chúng, đều có thể khiến toàn bộ thao tác thất bại. Đó là chưa kể đến thực tế rằng những chuỗi dài kiểu này có thể gây ra tranh chấp tài nguyên (resource contention) đáng kể. Đằng sau hậu trường, Order Processor nhiều khả năng có một kết nối mạng mở đang chờ nghe phản hồi từ Payment. Payment đến lượt nó có một kết nối mạng mở đang chờ phản hồi từ Fraud Detection, và cứ thế. Việc có nhiều kết nối cần phải giữ mở có thể tác động đến hệ thống đang chạy—bạn dễ gặp phải các vấn đề trong đó bạn cạn kiệt các kết nối khả dụng hoặc hứng chịu tình trạng tắc nghẽn mạng gia tăng như một hệ quả.

To improve this situation, we could reexamine the interactions between the microservices in the first place. For example, maybe we take the use of Fraud Detection out of the main purchase flow, as shown in Figure 4-4, and instead have it run in the background. If it finds a problem with a specific customer, their records are updated accordingly, and this is something that could be checked earlier in the payment process. Effectively, this means we're doing some of this work in parallel. By reducing the length of the call chain, we'll see the overall latency of the operation improve, and we'll take one of our microservices (Fraud Detection) out of the critical path for the purchase flow, giving us one fewer dependency to worry about for what is a critical operation.

Để cải thiện tình hình này, chúng ta có thể xem xét lại các tương tác giữa các microservice

ngay từ đầu. Ví dụ, có lẽ chúng ta đưa việc dùng Fraud Detection ra khỏi luồng mua hàng chính, như được minh họa trong Hình 4-4, và thay vào đó cho nó chạy nền (background). Nếu nó phát hiện một vấn đề với một khách hàng cụ thể, hồ sơ của họ được cập nhật tương ứng, và đây là điều có thể được kiểm tra sớm hơn trong quy trình thanh toán. Về thực chất, điều này có nghĩa là chúng ta đang làm một phần của công việc này song song. Bằng cách giảm chiều dài của chuỗi lời gọi, chúng ta sẽ thấy độ trễ tổng thể của thao tác được cải thiện, và chúng ta sẽ đưa một trong các microservice của mình (Fraud Detection) ra khỏi đường găng (critical path) của luồng mua hàng, cho chúng ta bớt đi một phụ thuộc phải lo lắng cho một thao tác trọng yếu.

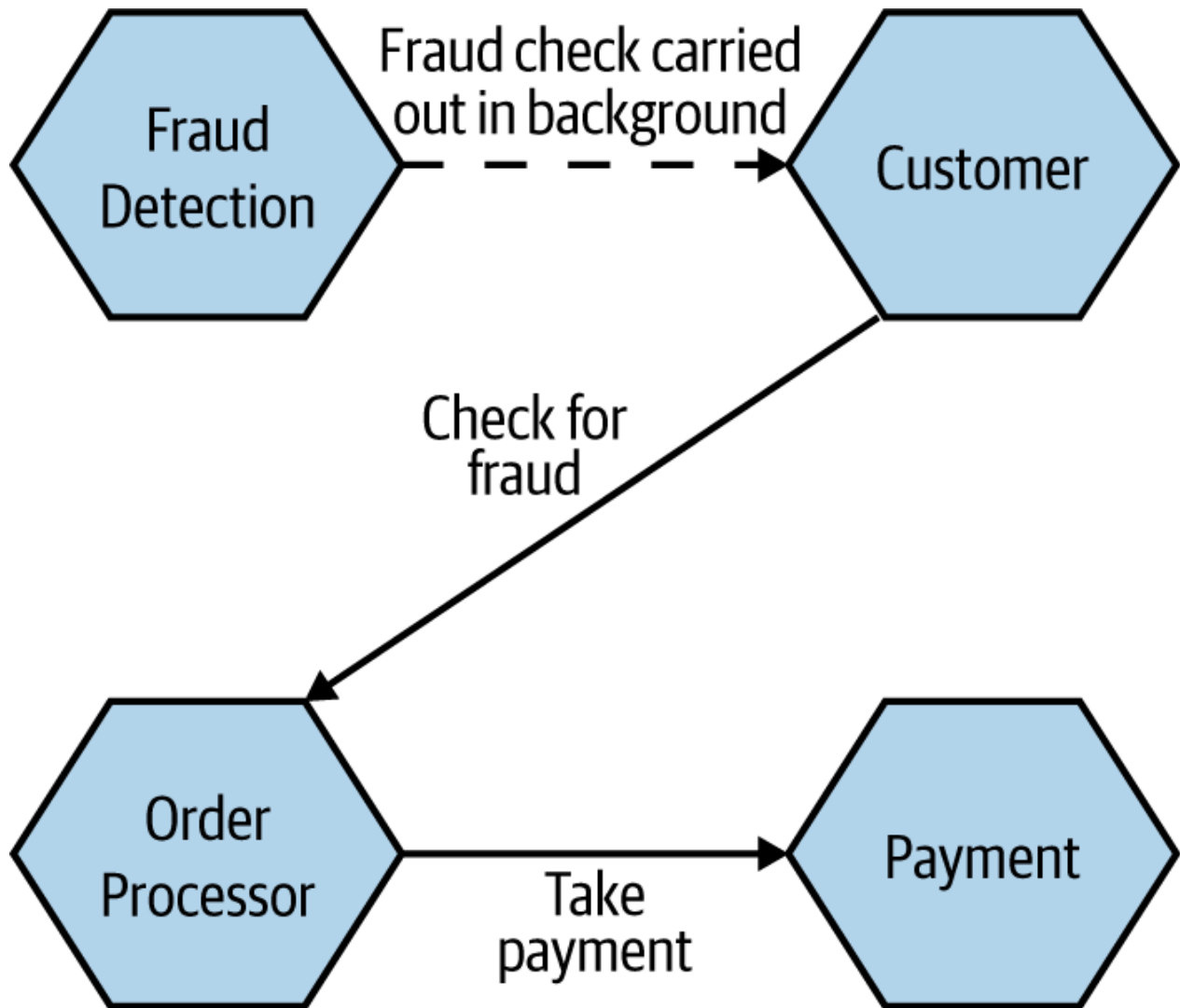


Figure 4-4. Moving Fraud Detection to a background process can reduce concerns around the length of the call chain

Hình 4-4. Chuyển Fraud Detection sang một tiến trình chạy nền có thể giảm bớt mối lo về chiều dài của chuỗi lời gọi

Of course, we could also replace the use of blocking calls with some style of nonblocking interaction without changing the **workflow** here, an approach we'll explore next.

Tất nhiên, chúng ta cũng có thể thay việc dùng các lời gọi có chặn bằng một phong cách

tương tác không chặn nào đó mà không cần thay đổi quy trình làm việc ở đây, một cách tiếp cận mà chúng ta sẽ khám phá tiếp theo.

Pattern: Asynchronous Nonblocking — Pattern: Bất đồng bộ không chặn

With asynchronous communication, the act of sending a call out over the network doesn't block the microservice issuing the call. It is able to carry on with any other processing without having to wait for a response. Nonblocking asynchronous communication comes in many forms, but we'll be looking in more detail at the three most common styles I see in microservice architecture. They are:

Với giao tiếp bất đồng bộ, hành động gửi một lời gọi ra ngoài qua mạng không chặn microservice phát ra lời gọi đó lại. Nó có thể tiếp tục với bất kỳ xử lý nào khác mà không phải chờ phản hồi. Giao tiếp bất đồng bộ không chặn có nhiều hình thức, nhưng chúng ta sẽ xem xét chi tiết hơn ba phong cách phổ biến nhất mà tôi thấy trong kiến trúc microservice. Đó là:

Communication through common data The upstream microservice changes some common data, which one or more microservices later make use of.

Giao tiếp qua dữ liệu dùng chung Microservice thượng nguồn thay đổi một số dữ liệu dùng chung, mà một hoặc nhiều microservice sau đó sử dụng.

Request-response A microservice sends a request to another microservice asking it to do something. When the requested operation completes, whether successfully or not, the upstream microservice receives the response. Specifically, any instance of the upstream microservice should be able to handle the response.

Request-response Một microservice gửi một yêu cầu đến một microservice khác để nghị nó làm một việc gì đó. Khi thao tác được yêu cầu hoàn tất, dù thành công hay không, microservice thượng nguồn nhận được phản hồi. Cụ thể, bất kỳ instance nào của microservice thượng nguồn cũng phải có khả năng xử lý được phản hồi.

Event-driven interaction A microservice broadcasts an event, which can be thought of as a factual statement about something that has happened. Other microservices can listen for the events they are interested in and react accordingly.

Tương tác hướng sự kiện Một microservice phát quảng bá (broadcast) một sự kiện, có thể được hiểu là một phát biểu mang tính sự thật về một điều gì đó đã xảy ra. Các microservice khác có thể lắng nghe các sự kiện mà chúng quan tâm và phản ứng tương ứng.

Advantages — Ưu điểm

With nonblocking asynchronous communication, the microservice making the initial call and the microservice (or microservices) receiving the call are decoupled temporarily. The microservices that receive the call do not need to be reachable at the same time the call is made. This means we avoid the concerns of temporal decoupling that we discussed in Chapter 2 (see “A Brief Note on Temporal Coupling” on page 42).

Với giao tiếp bất đồng bộ không chặn, microservice thực hiện lời gọi ban đầu và microservice (hoặc các microservice) nhận lời gọi được tách rời (decouple) về mặt thời gian. Các microservice nhận lời gọi không cần phải tiếp cận được vào đúng thời điểm

lời gọi được thực hiện. Điều này có nghĩa là chúng ta tránh được những mối lo về tách rời theo thời gian mà chúng ta đã thảo luận ở Chương 2 (xem “A Brief Note on Temporal Coupling” trên trang 42).

This style of communication is also beneficial if the functionality being triggered by a call will take a long time to process. Let’s come back to our example of MusicCorp, and specifically the process of sending out a package. In Figure 4-5 , the Order Processor has taken payment and has decided that it is time to dispatch the package, so it sends a call to the Warehouse microservice. The process of finding the CDs, taking them off the shelf, packaging them up, and having them picked up could take many hours, and potentially even days, depending on how the actual dispatch process works. It makes sense, therefore, for the Order Processor to issue a nonblocking asynchronous call to the Warehouse and have the Warehouse call back later to inform the Order Processor of its progress. This is a form of asynchronous request-response communication.

Phong cách giao tiếp này cũng có lợi nếu chức năng được kích hoạt bởi một lời gọi sẽ mất nhiều thời gian để xử lý. Hãy quay lại ví dụ về MusicCorp của chúng ta, và cụ thể là quy trình gửi đi một bưu kiện. Trong Hình 4-5, Order Processor đã thu tiền và đã quyết định rằng đã đến lúc gửi bưu kiện đi, nên nó gửi một lời gọi đến microservice Warehouse. Quy trình tìm các đĩa CD, lấy chúng ra khỏi kệ, đóng gói chúng lại, và cho người đến lấy có thể mất nhiều giờ, và thậm chí có thể nhiều ngày, tùy thuộc vào cách quy trình gửi hàng thực tế vận hành. Vì vậy, sẽ là hợp lý khi Order Processor phát ra một lời gọi bất đồng bộ không chặn đến Warehouse và để Warehouse gọi lại sau để thông báo cho Order Processor về tiến độ của nó. Đây là một dạng giao tiếp request-response bất đồng bộ.

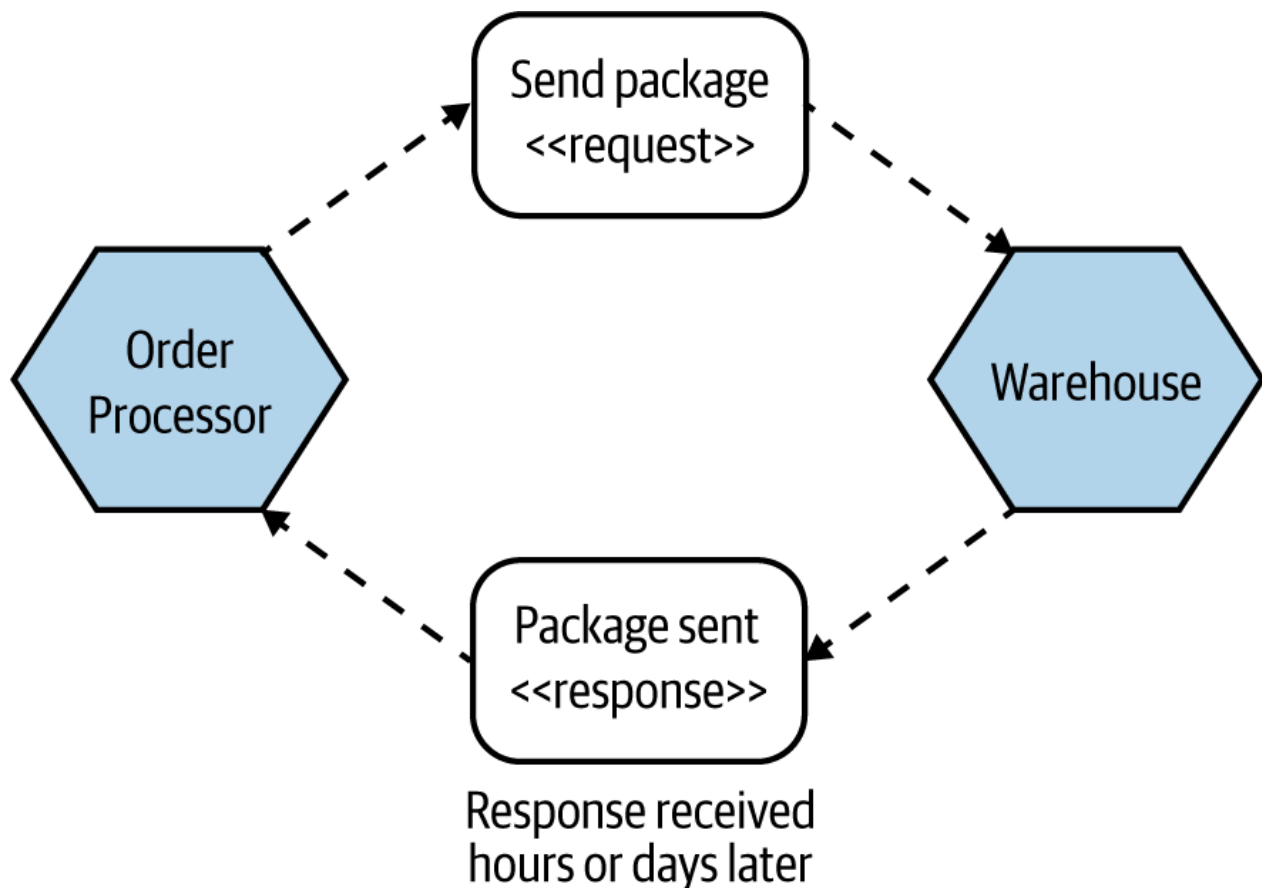


Figure 4-5. The Order Processor kicks off the process to package and ship an order, which is done in

an asynchronous fashion

Hình 4-5. Order Processor khởi động quy trình đóng gói và vận chuyển một đơn hàng, việc này được thực hiện theo kiểu bất đồng bộ

If we tried doing something similar with synchronous blocking calls, then we'd have to restructure the interactions between Order Processor and Warehouse—it wouldn't be feasible for Order Processor to open a connection, send a request, block any further operations in calling the thread, and wait for a response for what might be hours or days.

Nếu chúng ta thử làm điều gì đó tương tự với các lời gọi đồng bộ có chặn, thì chúng ta sẽ phải tái cấu trúc các tương tác giữa Order Processor và Warehouse—sẽ là bất khả thi khi Order Processor mở một kết nối, gửi một yêu cầu, chặn mọi thao tác tiếp theo trong luồng gọi, và chờ một phản hồi trong khoảng thời gian có thể là hàng giờ hoặc hàng ngày.

Disadvantages — Nhược điểm

The main downsides of nonblocking asynchronous communication, relative to blocking synchronous communication, are the level of complexity and the range of choice. As we've already outlined, there are different styles of asynchronous communication to choose from—which one is right for you? When we start digging into how these different styles of communication are implemented, there is a potentially bewildering list of technology we could look at.

Những nhược điểm chính của giao tiếp bất đồng bộ không chặn, so với giao tiếp đồng bộ có chặn, là mức độ phức tạp và phạm vi lựa chọn. Như chúng ta đã phác thảo, có những phong cách giao tiếp bất đồng bộ khác nhau để lựa chọn—cái nào là đúng cho bạn? Khi chúng ta bắt đầu đào sâu vào cách các phong cách giao tiếp khác nhau này được hiện thực, có một danh sách công nghệ có khả năng khiến ta hoa mắt mà ta có thể phải xem xét.

If asynchronous communication doesn't map to your mental models of computing, adopting an asynchronous style of communication will be challenging at first. And as we'll explore further when we look in detail at the various styles of asynchronous communication, there are a lot of different, interesting ways in which you can get yourself into a lot of trouble.

Nếu giao tiếp bất đồng bộ không khớp với các mô hình tư duy về điện toán của bạn, thì việc áp dụng một phong cách giao tiếp bất đồng bộ ban đầu sẽ là một thử thách. Và như chúng ta sẽ khám phá thêm khi xem xét chi tiết các phong cách giao tiếp bất đồng bộ khác nhau, có rất nhiều cách khác nhau, thú vị mà bạn có thể tự đẩy mình vào rất nhiều rắc rối.

Async/Await and When Asynchronous Is Still Blocking — Async/Await và khi nào Bất đồng bộ vẫn là Có chặn

As with many areas of computing, we can use the same term in different contexts to have very different meanings. A style of programming that appears to be especially popular is the use of constructs like `async/await` to work with a potentially asynchronous source of data but in a blocking, synchronous style.

Như với nhiều lĩnh vực trong điện toán, chúng ta có thể dùng cùng một thuật ngữ trong những bối cảnh khác nhau để mang những ý nghĩa rất khác nhau. Một phong cách lập trình có vẻ đặc biệt phổ biến là việc dùng các cấu trúc như `async/await` để làm việc với một nguồn dữ liệu có khả năng bất đồng bộ nhưng theo một phong cách có chặn, đồng bộ.

In Example 4-1 , we see a very simple JavaScript example of this in action. The currency exchange rates fluctuate frequently through the day, and we receive these via a **message broker**. We define a Promise . Generically, a promise is something that will resolve to a state at some point in the future. In our case, our eurToGbp will eventually resolve to being the next euro-to-GBP exchange rate.

Trong Ví dụ 4-1, chúng ta thấy một ví dụ JavaScript rất đơn giản về điều này trong thực tế. Tỷ giá hối đoái tiền tệ dao động thường xuyên suốt cả ngày, và chúng ta nhận các tỷ giá này qua một message broker. Chúng ta định nghĩa một Promise. Một cách tổng quát, một promise là một thứ sẽ phân giải (resolve) thành một trạng thái nào đó tại một thời điểm trong tương lai. Trong trường hợp của chúng ta, eurToGbp của chúng ta cuối cùng sẽ phân giải thành tỷ giá euro-sang-GBP tiếp theo.

Example 4-1. An example of working with a potentially asynchronous call in a blocking, synchronous fashion

Ví dụ 4-1. Một ví dụ về làm việc với một lời gọi có khả năng bất đồng bộ theo kiểu có chặn, đồng bộ

```
async function f() {  
  async function f() {  
let eurToGbp = new Promise((resolve, reject) => { //code to fetch latest exchange rate between EUR  
and GBP ... });  
  
    let eurToGbp = new Promise((resolve, reject) => { //code to fetch latest exchange rate  
    between EUR and GBP ... });  
  
var latestRate = await eurToGbp; process(latestRate); }  
    var latestRate = await eurToGbp; process(latestRate); }
```

Wait until the latest EUR-to-GBP exchange rate is fetched.

Chờ cho đến khi tỷ giá EUR-sang-GBP mới nhất được lấy về.

Won't run until the promise is fulfilled.

Sẽ không chạy cho đến khi promise được hoàn thành (fulfilled).

When we reference eurToGbp using await , we block until latestRate 's state is fulfilled— process isn't reached until we resolve the state of eurToGbp . 3

Khi chúng ta tham chiếu eurToGbp bằng cách dùng await, chúng ta chặn lại cho đến khi trạng thái của latestRate được hoàn thành—process không được tới cho đến khi chúng ta phân giải trạng thái của eurToGbp. 3

Even though our exchange rates are being received in an asynchronous fashion, the use of await in this context means we are blocking until the state of latestRate is resolved. So even if the underlying technology we are using to get the rate could be considered asynchronous in nature (for example, waiting for the rate), from the point of our code, this is inherently a synchronous, blocking interaction.

Mặc dù các tỷ giá hối đoái của chúng ta đang được nhận theo kiểu bất đồng bộ, việc dùng await trong bối cảnh này có nghĩa là chúng ta đang chặn lại cho đến khi trạng thái của latestRate được phân giải. Vậy nên dù công nghệ bên dưới mà chúng ta đang dùng để lấy tỷ giá có thể được xem là bất đồng bộ về bản chất (ví dụ, chờ tỷ giá), thì từ góc nhìn của code của chúng ta, đây về bản chất là một tương tác đồng bộ, có chặn.

Where to Use It — Khi nào nên dùng

Ultimately, when considering whether asynchronous communication is right for you, you also have to consider which type of asynchronous communication you want to pick, as each type has its own trade-offs. In general, though, there are some specific use cases that would have me reaching for some form of asynchronous communication. Long-running processes are an obvious candidate, as we explored in Figure 4-5. Also, situations in which you have long call chains that you can't easily restructure could be a good candidate. We'll dive deeper into this when we look at three of the most common forms of asynchronous communication—request-response calls, event-driven communication, and communication through common data.

Rốt cuộc, khi cân nhắc liệu giao tiếp bất đồng bộ có đúng cho bạn hay không, bạn cũng phải cân nhắc loại giao tiếp bất đồng bộ nào bạn muốn chọn, vì mỗi loại có những đánh đổi riêng. Tuy nhiên nhìn chung, có một số trường hợp sử dụng cụ thể khiến tôi nghĩ ngay đến một hình thức giao tiếp bất đồng bộ nào đó. Các tiến trình chạy lâu (long-running) là một ứng viên hiển nhiên, như chúng ta đã khám phá trong Hình 4-5. Ngoài ra, các tình huống trong đó bạn có những chuỗi lời gọi dài mà bạn không thể dễ dàng tái cấu trúc cũng có thể là một ứng viên tốt. Chúng ta sẽ đào sâu hơn vào điều này khi xem xét ba trong số các hình thức giao tiếp bất đồng bộ phổ biến nhất—các lời gọi request-response, giao tiếp hướng sự kiện, và giao tiếp qua dữ liệu dùng chung.

Pattern: Communication Through Common Data — Pattern: Giao tiếp qua dữ liệu dùng chung

A style of communication that spans a multitude of implementations is communication through common data. This pattern is used when one microservice puts data into a defined location, and another microservice (or potentially multiple microservices) then makes use of the data. It can be as simple as one microservice dropping a file in a location, and at some point later on another microservice picks up that file and does something with it. This integration style is fundamentally asynchronous in nature.

Một phong cách giao tiếp trải rộng trên vô số cách hiện thực là giao tiếp qua dữ liệu dùng chung. Pattern này được dùng khi một microservice đặt dữ liệu vào một vị trí đã định, và một microservice khác (hoặc có khả năng là nhiều microservice) sau đó sử dụng dữ liệu đó. Nó có thể đơn giản như một microservice thả một tệp vào một vị trí, và tại một thời điểm nào đó về sau một microservice khác lấy tệp đó lên và làm gì đó với nó. Phong cách tích hợp này về bản chất là bất đồng bộ.

An example of this style can be seen in Figure 4-6, where the New Product Importer creates a file that will then be read by the downstream Inventory and Catalog microservices.

Một ví dụ về phong cách này có thể thấy trong Hình 4-6, nơi New Product Importer tạo ra một tệp mà sau đó sẽ được đọc bởi các microservice hạ nguồn Inventory và Catalog.

3 Please note, this is very simplified—I've completely omitted error-handling code, for example. If you want to know more about `async/await`, specifically in JavaScript, the Modern JavaScript Tutorial is a great place to start.

3 Xin lưu ý, đây là một ví dụ rất giản lược—chẳng hạn tôi đã bỏ hoàn toàn code xử lý lỗi. Nếu bạn muốn biết thêm về `async/await`, cụ thể là trong JavaScript, thì Modern JavaScript Tutorial là một nơi tuyệt vời để bắt đầu.

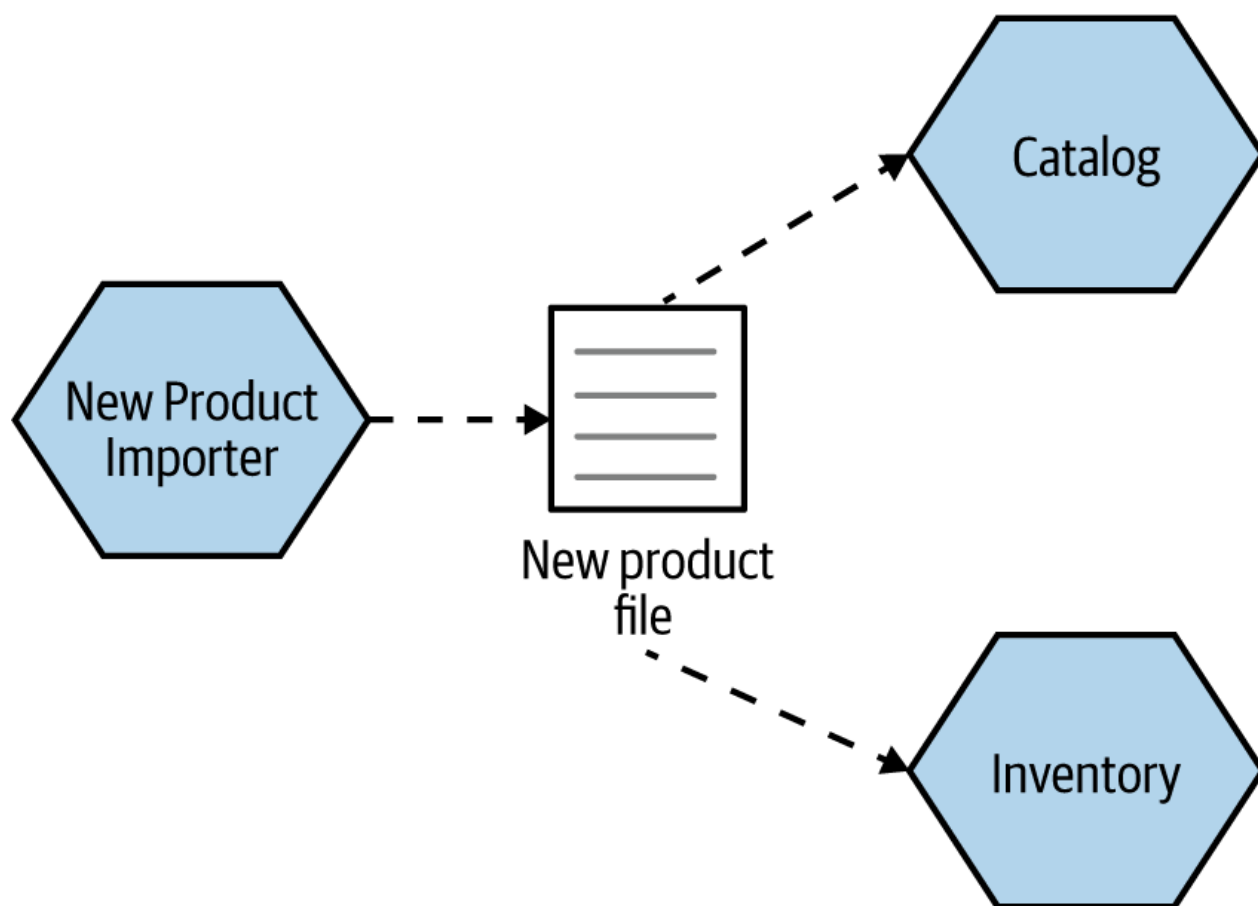


Figure 4-6. One microservice writes out a file that other microservices make use of

Hình 4-6. Một microservice ghi ra một tệp mà các microservice khác sử dụng

This pattern is in some ways the most common general inter-process communication pattern that you'll see, and yet we sometimes fail to see it as a communication pattern at all—I think largely because the communication between processes is often so indirect as to be hard to spot.

Pattern này theo một số khía cạnh là pattern giao tiếp giữa các tiến trình tổng quát phổ biến nhất mà bạn sẽ gặp, vậy mà đôi khi chúng ta không nhận ra nó là một pattern giao tiếp gì cả—tôi nghĩ phần lớn là vì giao tiếp giữa các tiến trình thường gián tiếp đến mức khó mà nhận ra.

Implementation — Cách hiện thực

To implement this pattern, you need some sort of persistent store for the data. A filesystem in many cases can be enough. I've built many systems that just periodically scan a filesystem, note the presence of a new file, and react on it accordingly. You could use some sort of robust distributed memory store as well, of course. It's worth noting that any downstream microservice that is going to act on this data will need its own mechanism to identify that new data is available—polling is a frequent solution to this problem.

Để hiện thực pattern này, bạn cần một dạng kho lưu trữ bền vững (persistent store) nào đó cho dữ liệu. Trong nhiều trường hợp một hệ thống tệp là đủ. Tôi đã xây dựng nhiều hệ thống chỉ định kỳ quét một hệ thống tệp, ghi nhận sự hiện diện của một tệp mới, và phản

ứng với nó tương ứng. Tất nhiên bạn cũng có thể dùng một dạng kho lưu trữ bộ nhớ phân tán vững chắc nào đó. Đáng lưu ý là bất kỳ microservice hạ nguồn nào sắp hành động dựa trên dữ liệu này sẽ cần cơ chế riêng để nhận biết rằng có dữ liệu mới khả dụng—polling (thăm dò định kỳ) là một giải pháp thường gặp cho vấn đề này.

Two common examples of this pattern are the data lake and the data warehouse. In both cases, these solutions are typically designed to help process large volumes of data, but arguably they exist at opposite ends of the spectrum regarding coupling. With a data lake, sources upload raw data in whatever format they see fit, and downstream consumers of this raw data are expected to know how to process the information. With a data warehouse, the warehouse itself is a structured data store. Microservices pushing data to the data warehouse need to know the structure of the data warehouse—if the structure changes in a backward-incompatible way, then these producers will need to be updated.

Hai ví dụ phổ biến của pattern này là data lake và data warehouse. Trong cả hai trường hợp, các giải pháp này thường được thiết kế để giúp xử lý khối lượng dữ liệu lớn, nhưng có thể nói chúng nằm ở hai đầu đối lập của quang phổ về mặt coupling. Với một data lake, các nguồn tải lên dữ liệu thô ở bất kỳ định dạng nào chúng thấy phù hợp, và các bên tiêu thụ hạ nguồn của dữ liệu thô này được kỳ vọng phải biết cách xử lý thông tin. Với một data warehouse, bản thân kho dữ liệu là một kho dữ liệu có cấu trúc. Các microservice đẩy dữ liệu vào data warehouse cần biết cấu trúc của data warehouse—nếu cấu trúc thay đổi theo cách không tương thích ngược, thì các bên sản xuất (producer) này sẽ cần được cập nhật.

With both the data warehouse and the data lake, the assumption is that the flow of information is in a single direction. One microservice publishes data to the common data store, and downstream consumers read that data and carry out appropriate actions. This unidirectional flow can make it easier to reason about the flow of information. A more problematic implementation would be the use of a **shared database** in which multiple microservices both read and write to the same data store, an example of which we discussed in Chapter 2 when we explored common coupling—Figure 4-7 shows both Order Processor and Warehouse updating the same record.

Với cả data warehouse lẫn data lake, giả định là dòng chảy thông tin đi theo một chiều duy nhất. Một microservice công bố dữ liệu vào kho dữ liệu dùng chung, và các bên tiêu thụ hạ nguồn đọc dữ liệu đó và thực hiện các hành động phù hợp. Dòng chảy một chiều này có thể giúp dễ suy luận về dòng chảy thông tin hơn. Một cách hiện thực rắc rối hơn sẽ là việc dùng một cơ sở dữ liệu dùng chung trong đó nhiều microservice cùng đọc và ghi vào cùng một kho dữ liệu, một ví dụ về điều này mà chúng ta đã thảo luận ở Chương 2 khi khám phá common coupling—Hình 4-7 cho thấy cả Order Processor và Warehouse đều đang cập nhật cùng một bản ghi.

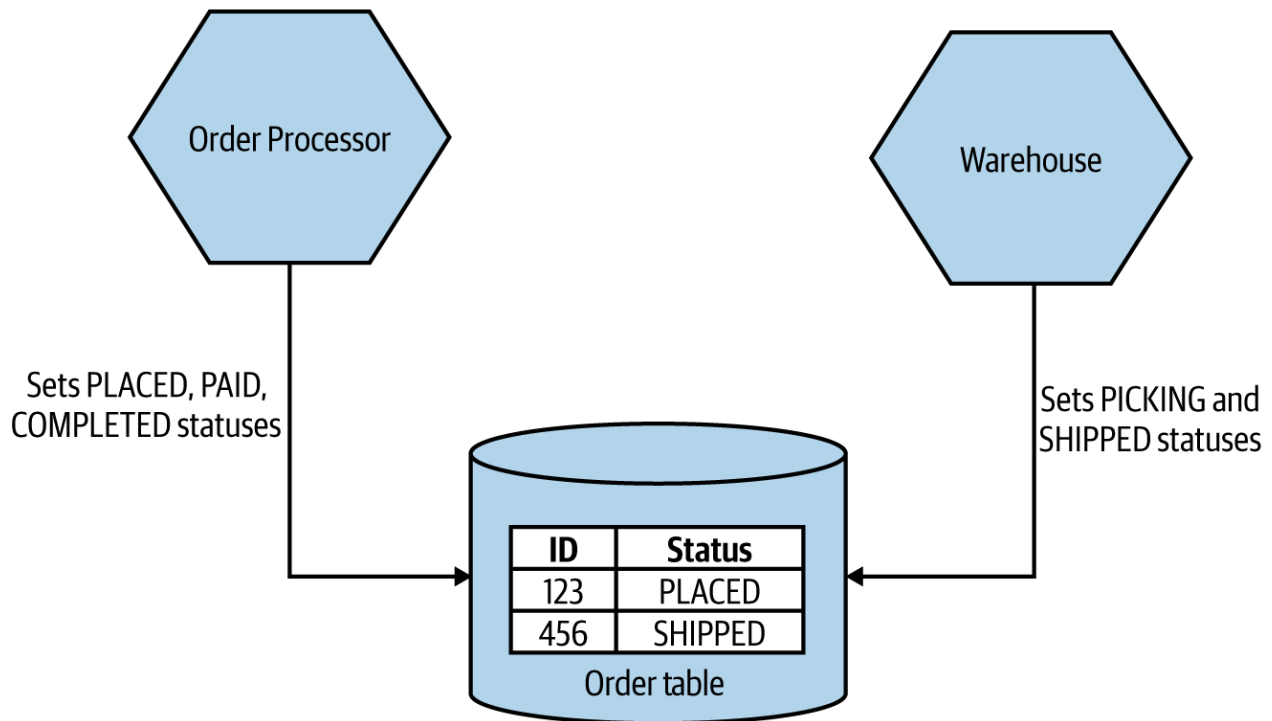


Figure 4-7. An example of common coupling in which both Order Processor and Warehouse are updating the same order record

Hình 4-7. Một ví dụ về common coupling trong đó cả Order Processor và Warehouse đều đang cập nhật cùng một bản ghi đơn hàng

Advantages — Ưu điểm

This pattern can be implemented very simply, using commonly understood technology. If you can read or write to a file or read and write to a database, you can use this pattern. The use of prevalent and well-understood technology also enables interoperability between different types of systems, including older mainframe applications or customizable off-the-shelf (COTS) software products. Data volumes are also less of a concern here—if you're sending lots of data in one big go, this pattern can work well.

Pattern này có thể được hiện thực rất đơn giản, sử dụng công nghệ được hiểu rộng rãi. Nếu bạn có thể đọc hoặc ghi vào một tệp hay đọc và ghi vào một cơ sở dữ liệu, thì bạn có thể dùng pattern này. Việc dùng công nghệ phổ biến và được hiểu rõ cũng tạo điều kiện cho khả năng tương tác (interoperability) giữa các loại hệ thống khác nhau, bao gồm cả các ứng dụng mainframe cũ hơn hay các sản phẩm phần mềm thương mại tùy biến sẵn (customizable off-the-shelf, COTS). Khối lượng dữ liệu cũng ít là mối lo ở đây—nếu bạn gửi nhiều dữ liệu trong một lần lớn, pattern này có thể hoạt động tốt.

Disadvantages — Nhược điểm

Downstream consuming microservices will typically be aware that there is new data to process via some sort of polling mechanism, or else perhaps through a periodically triggered timed job. That means that this mechanism is unlikely to be useful in low-latency situations. You can of course

combine this pattern with some other sort of call informing a downstream microservice that new data is available. For example, I could write a file to a shared filesystem and then send a call to the interested microservice informing it that there is new data that it may want. This can close the gap between data being published and data being processed. In general, though, if you're using this pattern for very large volumes of data, it's less likely that low latency is high on your list of requirements. If you are interested in sending larger volumes of data and having them processed more in "real time," then using some sort of streaming technology like Kafka would be a better fit.

Các microservice tiêu thụ hạ nguồn thường sẽ nhận biết rằng có dữ liệu mới để xử lý thông qua một dạng cơ chế polling nào đó, hoặc nếu không thì có lẽ thông qua một công việc hẹn giờ được kích hoạt định kỳ. Điều đó có nghĩa là cơ chế này nhiều khả năng không hữu ích trong các tình huống độ trễ thấp. Tất nhiên bạn có thể kết hợp pattern này với một dạng lời gọi khác nào đó để thông báo cho một microservice hạ nguồn rằng có dữ liệu mới khả dụng. Ví dụ, tôi có thể ghi một tệp vào một hệ thống tệp dùng chung rồi gửi một lời gọi đến microservice quan tâm để thông báo cho nó rằng có dữ liệu mới mà nó có thể muốn. Điều này có thể thu hẹp khoảng cách giữa lúc dữ liệu được công bố và lúc dữ liệu được xử lý. Tuy nhiên nhìn chung, nếu bạn đang dùng pattern này cho những khối lượng dữ liệu rất lớn, thì ít khả năng độ trễ thấp nằm cao trong danh sách yêu cầu của bạn. Nếu bạn quan tâm đến việc gửi những khối lượng dữ liệu lớn hơn và xử lý chúng "thời gian thực" hơn, thì việc dùng một dạng công nghệ luồng (streaming) nào đó như Kafka sẽ phù hợp hơn.

Another big disadvantage, and something that should be fairly obvious if you remember back to our exploration of common coupling in Figure 4-7, is that the common data store becomes a potential source of coupling. If that data store changes structure in some way, it can break communication between microservices.

Một nhược điểm lớn khác, và là điều khá hiển nhiên nếu bạn còn nhớ lại phần khám phá common coupling của chúng ta trong Hình 4-7, là kho dữ liệu dùng chung trở thành một nguồn coupling tiềm tàng. Nếu kho dữ liệu đó thay đổi cấu trúc theo cách nào đó, nó có thể phá vỡ giao tiếp giữa các microservice.

The **robustness** of the communication will also come down to the robustness of the underlying data store. This isn't a disadvantage strictly speaking, but it's something to be aware of. If you're dropping a file on a filesystem, you might want to make sure that the filesystem itself isn't going to fail in interesting ways.

Độ vững chắc của giao tiếp cũng sẽ phụ thuộc vào độ vững chắc của kho dữ liệu bên dưới. Nói cho chặt chẽ thì đây không phải là một nhược điểm, nhưng đó là điều cần lưu ý. Nếu bạn đang thả một tệp lên một hệ thống tệp, bạn có thể muốn đảm bảo rằng bản thân hệ thống tệp đó sẽ không hỏng theo những kiểu kỳ quặc.

Where to Use It — Khi nào nên dùng

Where this pattern really shines is in enabling interoperability between processes that might have restrictions on what technology they can use. Having an existing system talk to your microservice's GRPC interface or subscribe to its Kafka topic might well be more convenient from the point of view of the microservice, but not from the point of view of a consumer. Older systems may have limitations on what technology they can support and may have high costs of change. On the other hand, even old mainframe systems should be able to read data out of a file. This does of course all depend on using data store technology that is widely supported—I could also implement this pattern using something like a Redis **cache**. But can your old mainframe system talk to Redis?

Nơi pattern này thực sự tỏa sáng là trong việc cho phép khả năng tương tác giữa các tiến trình có thể có những hạn chế về công nghệ chúng có thể dùng. Việc khiến một hệ thống hiện có nói chuyện với giao diện gRPC của microservice của bạn hay đăng ký vào Kafka topic của nó có thể thuận tiện hơn từ góc nhìn của microservice, nhưng không phải từ góc nhìn của một bên tiêu thụ. Các hệ thống cũ hơn có thể có những giới hạn về công nghệ chúng có thể hỗ trợ và có thể có chi phí thay đổi cao. Mặt khác, ngay cả các hệ thống mainframe cũ cũng nên có khả năng đọc dữ liệu ra từ một tệp. Tất nhiên điều này phụ thuộc vào việc dùng công nghệ kho dữ liệu được hỗ trợ rộng rãi—tôi cũng có thể hiện thực pattern này bằng một thứ gì đó như Redis cache. Nhưng liệu hệ thống mainframe cũ của bạn có nói chuyện được với Redis không?

Another major sweet spot for this pattern is in sharing large volumes of data. If you need to send a multigigabyte file to a filesystem or load a few million rows into a database, then this pattern is the way to go.

Một điểm mạnh lớn khác cho pattern này là trong việc chia sẻ những khối lượng dữ liệu lớn. Nếu bạn cần gửi một tệp nhiều gigabyte tới một hệ thống tệp hoặc nạp vài triệu hàng vào một cơ sở dữ liệu, thì pattern này là cách nên đi.

Pattern: Request-Response Communication — Pattern: Giao tiếp Request-Response

With request-response, a microservice sends a request to a downstream service asking it to do something and expects to receive a response with the result of the request. This interaction can be undertaken via a synchronous blocking call, or it could be implemented in an asynchronous nonblocking fashion. A simple example of this interaction is shown in Figure 4-8, where the Chart microservice, which collates the best-selling CDs for different genres, sends a request to the Inventory service asking for the current stock levels for some CDs.

Với request-response, một microservice gửi một yêu cầu đến một dịch vụ hạ nguồn đề nghị nó làm một việc gì đó và kỳ vọng nhận được một phản hồi với kết quả của yêu cầu. Tương tác này có thể được thực hiện qua một lời gọi đồng bộ có chặn, hoặc nó có thể được hiện thực theo kiểu bất đồng bộ không chặn. Một ví dụ đơn giản về tương tác này được minh họa trong Hình 4-8, nơi microservice Chart, vốn tổng hợp các đĩa CD bán chạy nhất cho các thể loại khác nhau, gửi một yêu cầu đến dịch vụ Inventory đề nghị cung cấp mức tồn kho hiện tại cho một số đĩa CD.

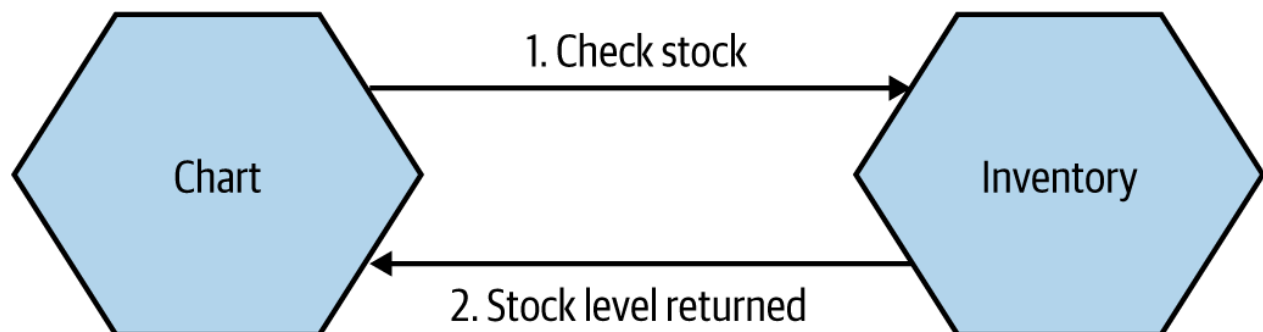


Figure 4-8. The Chart microservice sends a request to Inventory asking for stock levels

Hình 4-8. Microservice Chart gửi một yêu cầu đến Inventory đề nghị cung cấp mức tồn kho

Retrieving data from other microservices like this is a common use case for a request-response call. Sometimes, though, you just need to make sure something gets done. In Figure 4-9, the Warehouse microservice is sent a request from Order Processor asking it to reserve stock. The Order Processor just needs to know that stock has been successfully reserved before it can carry on with taking payment. If the stock can't be reserved—perhaps because an item is no longer available—then the payment can be cancelled. Using request-response calls in situations like this where calls need to be completed in a certain order is commonplace.

Việc lấy dữ liệu từ các microservice khác như thế này là một trường hợp sử dụng phổ biến cho một lời gọi request-response. Tuy nhiên đôi khi, bạn chỉ cần đảm bảo rằng một việc gì đó được hoàn thành. Trong Hình 4-9, microservice Warehouse được gửi một yêu cầu từ Order Processor đề nghị nó giữ chỗ (reserve) hàng tồn kho. Order Processor chỉ cần biết rằng hàng đã được giữ chỗ thành công trước khi nó có thể tiếp tục với việc thu tiền. Nếu không thể giữ chỗ hàng—có lẽ vì một mặt hàng không còn nữa—thì khoản thanh toán có thể bị hủy. Việc dùng các lời gọi request-response trong các tình huống như thế này khi các lời gọi cần được hoàn thành theo một thứ tự nhất định là chuyện thường gặp.

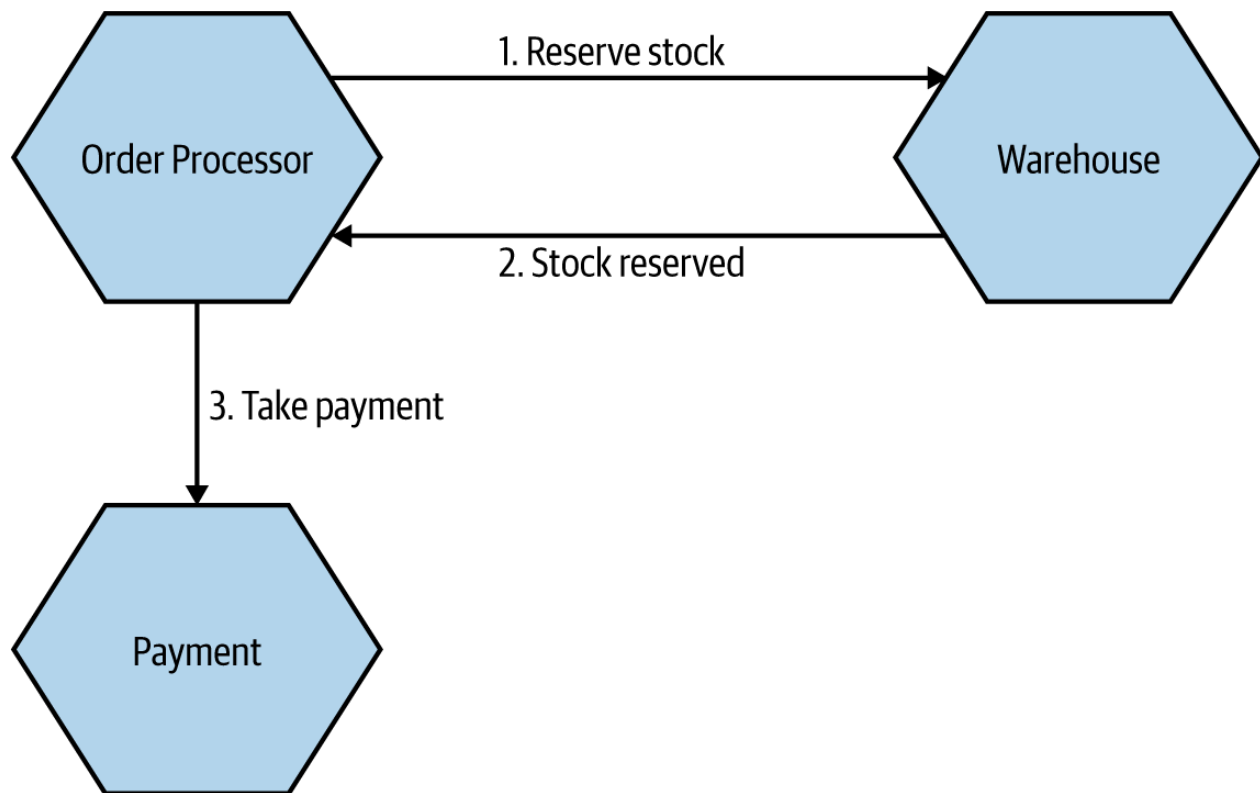


Figure 4-9. Order Processor needs to ensure stock can be reserved before payment can be taken

Hình 4-9. Order Processor cần đảm bảo có thể giữ chỗ hàng tồn kho trước khi có thể thu tiền

Commands Versus Requests — Lệnh (Command) so với Yêu cầu (Request)

I've heard some people talk about sending commands, rather than requests, specifically in the context of asynchronous request-response communication. The intent behind the term command is

arguably the same as that of request —namely, an upstream microservice is asking a downstream microservice to do something.

Tôi đã nghe một số người nói về việc gửi các lệnh (command), thay vì các yêu cầu (request), cụ thể là trong bối cảnh giao tiếp request-response bất đồng bộ. Ý định đằng sau thuật ngữ lệnh có thể nói là giống với ý định của yêu cầu—cụ thể, một microservice thượng nguồn đang đề nghị một microservice hạ nguồn làm một việc gì đó.

Personally speaking, though, I much prefer the term request . A command implies a directive that must be obeyed, and it can lead to a situation in which people feel that the command has to be acted on. A request implies something that can be rejected. It is right that a microservice examines each request on its merits and, based on its own internal logic, decides whether the request should be acted on. If the request it has been sent violates internal logic, the microservice should reject it. Although it's a subtle difference, I don't feel that the term command conveys the same meaning.

Tuy nhiên, nói riêng về cá nhân, tôi thích thuật ngữ yêu cầu hơn nhiều. Một lệnh ngụ ý một mệnh lệnh phải được tuân theo, và nó có thể dẫn đến một tình huống trong đó người ta cảm thấy rằng lệnh nhất định phải được thi hành. Một yêu cầu ngụ ý một thứ có thể bị từ chối. Đúng là một microservice nên xem xét từng yêu cầu theo giá trị của nó và, dựa trên logic nội bộ của chính mình, quyết định xem yêu cầu có nên được thi hành hay không. Nếu yêu cầu mà nó được gửi vi phạm logic nội bộ, microservice nên từ chối nó. Mặc dù đó là một khác biệt tinh tế, tôi không cảm thấy rằng thuật ngữ lệnh truyền tải được ý nghĩa tương tự.

I'll stick to using request over command , but whatever term you decide to use, just remember that a microservice gets to reject the request/command if appropriate.

Tôi sẽ vẫn dùng yêu cầu thay vì lệnh, nhưng bất kể bạn quyết định dùng thuật ngữ nào, chỉ cần nhớ rằng một microservice có quyền từ chối yêu cầu/lệnh nếu thích hợp.

Implementation: Synchronous Versus Asynchronous — Cách hiện thực: Đồng bộ so với Bất đồng bộ

Request-response calls like this can be implemented in either a blocking synchronous or a nonblocking asynchronous style. With a synchronous call, what you'd typically see is a network connection being opened with the downstream microservice, with the request being sent along this connection. The connection is kept open while the upstream microservice waits for the downstream microservice to respond. In this case, the microservice sending the response doesn't really need to know anything about the microservice that sent the request—it's just sending stuff back over an inbound connection. If that connection dies, perhaps because either the upstream or the downstream microservice instance dies, then we might have a problem.

Các lời gọi request-response như thế này có thể được hiện thực theo phong cách đồng bộ có chặn hoặc bất đồng bộ không chặn. Với một lời gọi đồng bộ, điều mà bạn thường thấy là một kết nối mạng được mở với microservice hạ nguồn, với yêu cầu được gửi đi dọc theo kết nối này. Kết nối được giữ mở trong khi microservice thượng nguồn chờ microservice hạ nguồn phản hồi. Trong trường hợp này, microservice gửi phản hồi thực ra không cần biết gì về microservice đã gửi yêu cầu—nó chỉ đang gửi thứ gì đó trở lại qua một kết nối đến. Nếu kết nối đó chết, có lẽ vì hoặc instance microservice thượng nguồn hoặc hạ nguồn chết, thì chúng ta có thể gặp vấn đề.

With an asynchronous request-response, things are less straightforward. Let's revisit the process associated with reserving stock. In Figure 4-10 , the request to reserve stock is sent as a message

over some sort of message broker (we'll explore message brokers later in this chapter). Rather than the message going directly to the Inventory microservice from Order Processor, it instead sits in a queue. The Inventory consumes messages from this queue when it is able. It reads the request, carries out the associated work of reserving the stock, and then needs to send the response back to a queue that Order Processor is reading from. The Inventory microservice needs to know where to route the response. In our example, it sends this response back over another queue that is in turn consumed by Order Processor.

Với một request-response bất đồng bộ, mọi thứ ít đơn giản hơn. Hãy xem lại quy trình liên quan đến việc giữ chỗ hàng tồn kho. Trong Hình 4-10, yêu cầu giữ chỗ hàng được gửi đi như một thông điệp qua một dạng message broker nào đó (chúng ta sẽ khám phá message broker ở phần sau của chương). Thay vì thông điệp đi thẳng đến microservice Inventory từ Order Processor, nó thay vào đó nằm trong một hàng đợi (queue). Inventory tiêu thụ các thông điệp từ hàng đợi này khi nó có thể. Nó đọc yêu cầu, thực hiện công việc liên quan là giữ chỗ hàng, và rồi cần gửi phản hồi trở lại một hàng đợi mà Order Processor đang đọc từ đó. Microservice Inventory cần biết định tuyến phản hồi đến đâu. Trong ví dụ của chúng ta, nó gửi phản hồi này trở lại qua một hàng đợi khác mà đến lượt nó được tiêu thụ bởi Order Processor.

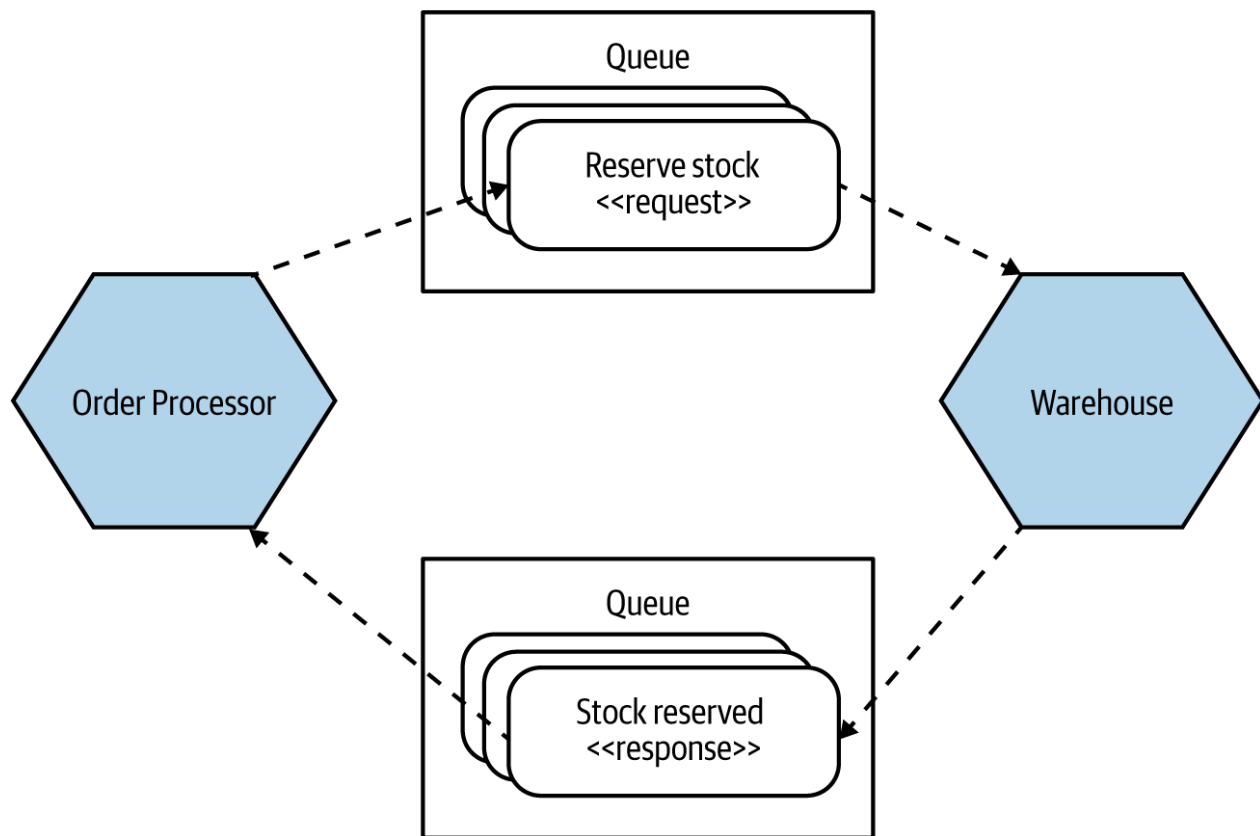


Figure 4-10. Using queues to send stock reservation requests

Hình 4-10. Dùng các hàng đợi để gửi các yêu cầu giữ chỗ hàng tồn kho

So with a nonblocking asynchronous interaction, the microservice that receives the request needs either to know implicitly where to route the response or else to be told where the response should go. When using a queue, we have the added benefit that multiple requests could be buffered up in the queue waiting to be handled. This can help in situations in which the requests can't be handled

quickly enough. The microservice can consume the next request when it is ready, rather than being overwhelmed by too many calls. Of course, a lot then depends on the queue absorbing these requests.

Vậy nên với một tương tác bất đồng bộ không chặn, microservice nhận yêu cầu cần hoặc là biết một cách ngầm định nên định tuyến phản hồi đến đâu, hoặc là được cho biết phản hồi nên đi đâu. Khi dùng một hàng đợi, chúng ta có thêm lợi ích là nhiều yêu cầu có thể được đệm lại trong hàng đợi để chờ được xử lý. Điều này có thể giúp ích trong các tình huống mà các yêu cầu không thể được xử lý đủ nhanh. Microservice có thể tiêu thụ yêu cầu tiếp theo khi nó sẵn sàng, thay vì bị quá tải bởi quá nhiều lời gọi. Tất nhiên, khi đó nhiều thứ phụ thuộc vào việc hàng đợi hấp thụ các yêu cầu này.

When a microservice receives a response in this way, it might need to relate the response to the original request. This can be challenging, as a lot of time may have passed, and depending on the nature of the protocol being used, the response may not come back to the same instance of the microservice that sent the request. In our example of reserving stock as part of placing an order, we'd need to know how to associate the “stock reserved” response with a given order so we can carry on processing that particular order. An easy way to handle this would be to store any state associated with the original request into a database, such that when the response comes in, the receiving instance can reload any associated state and act accordingly.

Khi một microservice nhận một phản hồi theo cách này, nó có thể cần liên hệ phản hồi với yêu cầu ban đầu. Điều này có thể là một thử thách, vì có thể đã trôi qua rất nhiều thời gian, và tùy thuộc vào bản chất của giao thức đang được dùng, phản hồi có thể không quay về cùng instance của microservice đã gửi yêu cầu. Trong ví dụ về việc giữ chỗ hàng tồn kho như một phần của việc đặt một đơn hàng, chúng ta sẽ cần biết cách liên kết phản hồi “đã giữ chỗ hàng” với một đơn hàng nhất định để chúng ta có thể tiếp tục xử lý đúng đơn hàng cụ thể đó. Một cách dễ dàng để xử lý điều này sẽ là lưu bất kỳ trạng thái nào liên quan đến yêu cầu ban đầu vào một cơ sở dữ liệu, sao cho khi phản hồi đến, instance nhận có thể nạp lại bất kỳ trạng thái liên quan nào và hành động tương ứng.

One last note: all forms of request-response interaction are likely going to require some form of **time-out** handling to avoid issues where the system gets blocked waiting for something that may never happen. How this time-out functionality is implemented can vary based on the implementation technology, but it will be needed. We'll look at time-outs in more detail in Chapter 12 .

Một lưu ý cuối: mọi hình thức tương tác request-response nhiều khả năng sẽ đòi hỏi một dạng xử lý time-out nào đó để tránh các vấn đề trong đó hệ thống bị chặn lại để chờ một thứ gì đó có thể không bao giờ xảy ra. Chức năng time-out này được hiện thực thế nào có thể khác nhau dựa trên công nghệ hiện thực, nhưng nó sẽ là cần thiết. Chúng ta sẽ xem xét time-out chi tiết hơn ở Chương 12.

Parallel Versus Sequential Calls — Lời gọi Song song so với Tuần tự

When working with request-response interactions, you'll often encounter a situation in which you will need to make multiple calls before you can continue with some processing.

Khi làm việc với các tương tác request-response, bạn sẽ thường gặp một tình huống trong đó bạn cần thực hiện nhiều lời gọi trước khi có thể tiếp tục với một xử lý nào đó.

Consider a situation in which MusicCorp needs to check on the price for a given item from three different stockists, which we do by issuing API calls. We want to get the prices back from all three stockists before deciding which one we want to order new stock from. We could decide to make the

three calls in sequence—waiting for each one to finish before proceeding with the next. In such a situation, we'd be waiting for the sum of latencies of each of the calls. If the API call to each provider took one second to return, we'd be waiting three seconds before we can decide who we should order from.

Hãy xét một tình huống trong đó MusicCorp cần kiểm tra giá cho một mặt hàng nhất định từ ba nhà cung cấp hàng (stockist) khác nhau, điều mà chúng ta làm bằng cách phát ra các lời gọi API. Chúng ta muốn lấy lại giá từ cả ba nhà cung cấp trước khi quyết định nhà nào trong số đó chúng ta muốn đặt hàng tồn kho mới. Chúng ta có thể quyết định thực hiện ba lời gọi theo trình tự—chờ mỗi lời gọi xong trước khi tiến hành với lời gọi tiếp theo. Trong một tình huống như vậy, chúng ta sẽ phải chờ bằng tổng độ trễ của từng lời gọi. Nếu lời gọi API đến mỗi nhà cung cấp mất một giây để trả về, chúng ta sẽ phải chờ ba giây trước khi có thể quyết định nên đặt hàng từ ai.

A better option would be to run these three requests in parallel; then the overall latency of the operation would be based on the slowest API call, rather than on the sum of latencies of each API call.

Một lựa chọn tốt hơn sẽ là chạy ba yêu cầu này song song; khi đó độ trễ tổng thể của thao tác sẽ dựa trên lời gọi API chậm nhất, thay vì dựa trên tổng độ trễ của từng lời gọi API.

Reactive extensions and mechanisms like `async/await` can be very useful in helping to run calls in parallel, and this can result in significant improvements in the latency of some operations.

Các phần mở rộng reactive (reactive extensions) và các cơ chế như `async/await` có thể rất hữu ích trong việc giúp chạy các lời gọi song song, và điều này có thể dẫn đến những cải thiện đáng kể về độ trễ của một số thao tác.

Where to Use It — Khi nào nên dùng

Request-response calls make perfect sense for any situation in which the result of a request is needed before further processing can take place. They also fit really well in situations where a microservice wants to know if a call didn't work so that it can carry out some sort of compensating action, like a retry. If either aligns with your situation, then request-response is a sensible approach; the only remaining question is to decide on a synchronous versus asynchronous implementation, with the same trade-offs we discussed earlier.

Các lời gọi request-response hoàn toàn hợp lý cho bất kỳ tình huống nào trong đó kết quả của một yêu cầu là cần thiết trước khi có thể tiến hành xử lý tiếp. Chúng cũng rất phù hợp trong các tình huống mà một microservice muốn biết liệu một lời gọi có thất bại hay không để nó có thể thực hiện một dạng hành động bù trừ nào đó, như thử lại. Nếu một trong hai điều này khớp với tình huống của bạn, thì request-response là một cách tiếp cận hợp lý; câu hỏi duy nhất còn lại là quyết định một cách hiện thực đồng bộ so với bất đồng bộ, với cùng những đánh đổi mà chúng ta đã thảo luận trước đó.

Pattern: Event-Driven Communication — Pattern: Giao tiếp Hướng sự kiện

Event-driven communication looks quite odd compared to request-response calls. Rather than a microservice asking some other microservice to do something, a microservice emits events that may

or may not be received by other microservices. It is an inherently asynchronous interaction, as the event listeners will be running on their own thread of execution.

Giao tiếp hướng sự kiện trông khá lạ so với các lời gọi request-response. Thay vì một microservice đề nghị một microservice khác làm một việc gì đó, một microservice phát ra các sự kiện mà có thể được hoặc không được các microservice khác nhận. Đây là một tương tác mang tính bất đồng bộ cố hữu, vì các bên lắng nghe sự kiện sẽ chạy trên luồng thực thi riêng của chúng.

An event is a statement about something that has occurred, nearly always something that has happened inside the world of the microservice that is emitting the event. The microservice emitting the event has no knowledge of the intent of other microservices to use the event, and indeed it may not even be aware that other microservices exist. It emits the event when required, and that is the end of its responsibilities.

Một sự kiện là một phát biểu về một điều gì đó đã xảy ra, gần như luôn là một điều gì đó đã xảy ra bên trong thế giới của microservice phát ra sự kiện đó. Microservice phát ra sự kiện không hề biết ý định của các microservice khác về việc dùng sự kiện, và thực ra nó thậm chí có thể không biết rằng các microservice khác tồn tại. Nó phát ra sự kiện khi cần, và đó là điểm kết thúc trách nhiệm của nó.

In Figure 4-11 , we see the Warehouse emitting events related to the process of packaging up an order. These events are received by two microservices, Notifications and Inventory , and they react accordingly. The Notifications microservice sends an email to update our customer about changes in order status, while the Inventory microservice can update stock levels as items are packaged into the customer's order.

Trong Hình 4-11, chúng ta thấy Warehouse phát ra các sự kiện liên quan đến quy trình đóng gói một đơn hàng. Những sự kiện này được nhận bởi hai microservice, Notifications và Inventory, và chúng phản ứng tương ứng. Microservice Notifications gửi một email để cập nhật cho khách hàng của chúng ta về những thay đổi trong trạng thái đơn hàng, trong khi microservice Inventory có thể cập nhật mức tồn kho khi các mặt hàng được đóng gói vào đơn hàng của khách.

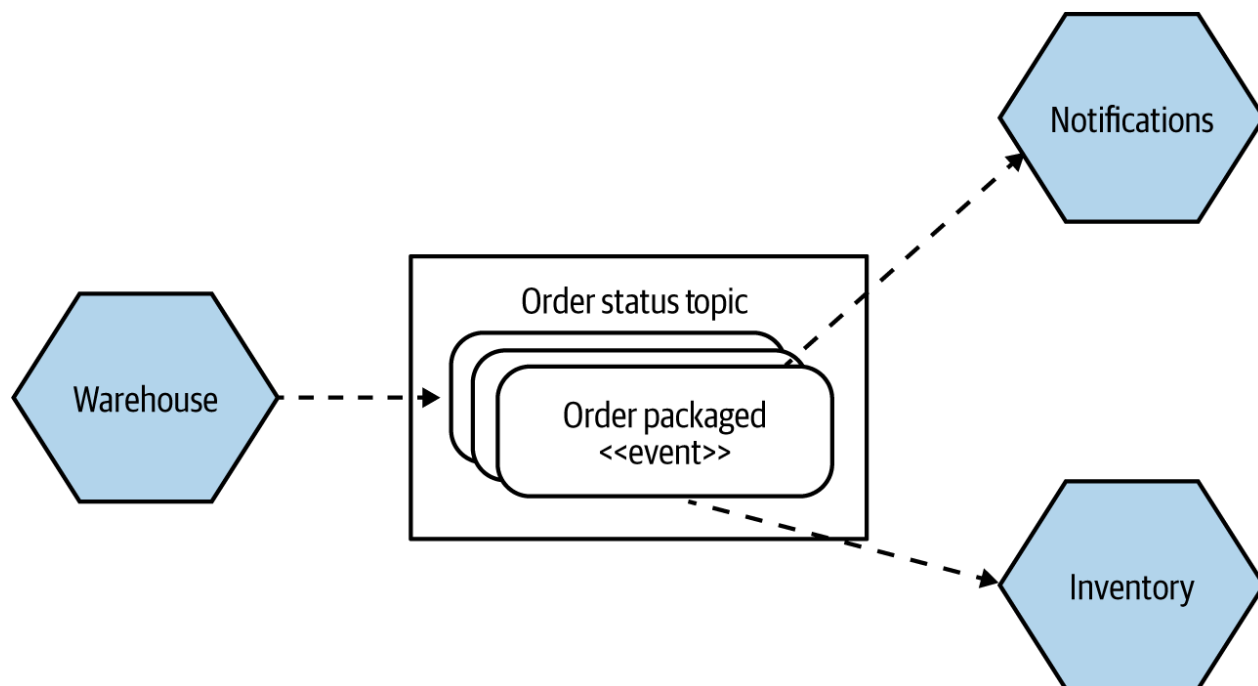


Figure 4-11. The Warehouse emits events that some downstream microservices subscribe to

Hình 4-11. Warehouse phát ra các sự kiện mà một số microservice hạ nguồn đăng ký nhận

The Warehouse is just broadcasting events, assuming that interested parties will react accordingly. It is unaware of who the recipients of the events are, making event-driven interactions much more loosely coupled in general. When you compare this to a request-response call, it may take you a while to get your head around the inversion of responsibility. With request-response, we might instead expect Warehouse to tell the Notifications microservice to send emails when appropriate. In such a model, Warehouse would need to know what events require customer notification. With an event-driven interaction, we are instead pushing that responsibility into the Notifications microservice.

Warehouse chỉ đang phát quảng bá các sự kiện, giả định rằng các bên quan tâm sẽ phản ứng tương ứng. Nó không biết người nhận của các sự kiện là ai, khiến các tương tác hướng sự kiện nhìn chung có coupling lỏng hơn nhiều. Khi bạn so sánh điều này với một lời gọi request-response, có thể bạn sẽ mất một lúc để thấm cái sự đảo ngược trách nhiệm này. Với request-response, thay vào đó chúng ta có thể kỳ vọng Warehouse bảo microservice Notifications gửi email khi thích hợp. Trong một mô hình như vậy, Warehouse sẽ cần biết những sự kiện nào đòi hỏi phải thông báo cho khách hàng. Với một tương tác hướng sự kiện, thay vào đó chúng ta đang đẩy trách nhiệm đó vào microservice Notifications.

The intent behind an event could be considered the opposite of a request. The event emitter is leaving it up to the recipients to decide what to do. With request-response, the microservice sending the request knows what should be done and is telling the other microservice what it thinks needs to happen next. This of course means that in request-response, the requester has to have knowledge of what the downstream recipient can do, implying a greater degree of domain coupling. With event-driven collaboration, the event emitter doesn't need to know what any downstream microservices are able to do, and in fact may not even know they exist—as a result, coupling is greatly reduced.

Ý định đằng sau một sự kiện có thể được xem là đối lập với một yêu cầu. Bên phát ra sự kiện để cho các bên nhận tự quyết định phải làm gì. Với request-response, microservice gửi yêu cầu biết điều gì nên được làm và đang bảo microservice kia điều mà nó cho rằng

cần xảy ra tiếp theo. Điều này tất nhiên có nghĩa là trong request-response, bên yêu cầu phải có hiểu biết về điều mà bên nhận hạ nguồn có thể làm, ngụ ý một mức độ domain coupling lớn hơn. Với cộng tác hướng sự kiện, bên phát ra sự kiện không cần biết bất kỳ microservice hạ nguồn nào có khả năng làm gì, và trên thực tế thậm chí có thể không biết chúng tồn tại—kết quả là, coupling được giảm đi đáng kể.

The distribution of responsibility we see with our event-driven interactions can mirror the distribution of responsibility we see with organizations trying to create more autonomous teams. Rather than holding all the responsibility centrally, we want to push it into the teams themselves to allow them to operate in a more autonomous fashion—a concept we will revisit in Chapter 15. Here, we are pushing responsibility from Warehouse into Notifications and Payment—this can help us reduce the complexity of microservices like Warehouse and lead to a more even distribution of “smarts” in our system. We’ll explore that idea in more detail when we compare choreography and orchestration in Chapter 6.

Sự phân bổ trách nhiệm mà chúng ta thấy với các tương tác hướng sự kiện của mình có thể phản chiếu sự phân bổ trách nhiệm mà chúng ta thấy ở các tổ chức đang cố tạo ra những đội nhóm tự chủ hơn. Thay vì giữ toàn bộ trách nhiệm tập trung ở một chỗ, chúng ta muốn đẩy nó vào chính các đội nhóm để cho phép họ vận hành theo kiểu tự chủ hơn—một khái niệm mà chúng ta sẽ trở lại ở Chương 15. Ở đây, chúng ta đang đẩy trách nhiệm từ Warehouse vào Notifications và Payment—điều này có thể giúp chúng ta giảm độ phức tạp của các microservice như Warehouse và dẫn đến một sự phân bổ “trí tuệ” đồng đều hơn trong hệ thống của chúng ta. Chúng ta sẽ khám phá ý tưởng đó chi tiết hơn khi so sánh choreography và orchestration ở Chương 6.

Events and Messages — Sự kiện và Thông điệp

On occasion I’ve seen the terms messages and events get confused. An event is a fact—a statement that something happened, along with some information about exactly what happened. A message is a thing we send over an asynchronous communication mechanism, like a message broker.

Đôi khi tôi đã thấy các thuật ngữ thông điệp (message) và sự kiện (event) bị nhầm lẫn. Một sự kiện là một sự thật—một phát biểu rằng có điều gì đó đã xảy ra, kèm theo một số thông tin về chính xác điều gì đã xảy ra. Một thông điệp là một thứ mà chúng ta gửi qua một cơ chế giao tiếp bất đồng bộ, như một message broker.

With event-driven collaboration, we want to broadcast that event, and a typical way to implement that broadcast mechanism would be to put the event into a message. The message is the medium; the event is the payload.

Với cộng tác hướng sự kiện, chúng ta muốn phát quảng bá sự kiện đó, và một cách điển hình để hiện thực cơ chế phát quảng bá đó sẽ là đặt sự kiện vào một thông điệp. Thông điệp là phương tiện; sự kiện là payload.

Likewise, we might want to send a request as the payload of a message—in which case we would be implementing a form of asynchronous request-response.

Tương tự, chúng ta có thể muốn gửi một yêu cầu như là payload của một thông điệp—trong trường hợp đó chúng ta sẽ đang hiện thực một dạng request-response bất đồng bộ.

Implementation — Cách hiện thực

There are two main aspects we need to consider here: a way for our microservices to emit events, and a way for our consumers to find out those events have happened.

Có hai khía cạnh chính mà chúng ta cần cân nhắc ở đây: một cách để các microservice của chúng ta phát ra sự kiện, và một cách để các bên tiêu thụ của chúng ta phát hiện ra rằng những sự kiện đó đã xảy ra.

Traditionally, message brokers like RabbitMQ try to handle both problems. Producers use an API to publish an event to the broker. The broker handles subscriptions, allowing consumers to be informed when an event arrives. These brokers can even handle the state of consumers—for example, by helping keep track of what messages they have seen before. These systems are normally designed to be scalable and resilient, but that doesn't come for free. It can add complexity to the development process, because it is another system you may need to run to develop and test your services. Additional machines and expertise may also be required to keep this infrastructure up and running. But once it is, it can be an incredibly effective way to implement loosely coupled, event-driven architectures. In general, I'm a fan.

Theo truyền thống, các message broker như RabbitMQ cố gắng xử lý cả hai vấn đề. Các bên sản xuất dùng một API để công bố một sự kiện đến broker. Broker xử lý các đăng ký (subscription), cho phép các bên tiêu thụ được thông báo khi một sự kiện đến. Các broker này thậm chí có thể xử lý trạng thái của các bên tiêu thụ—ví dụ, bằng cách giúp theo dõi những thông điệp nào chúng đã thấy trước đó. Các hệ thống này thường được thiết kế để có khả năng mở rộng và bền bỉ, nhưng điều đó không miễn phí. Nó có thể làm tăng độ phức tạp cho quy trình phát triển, vì nó là một hệ thống khác mà bạn có thể cần chạy để phát triển và kiểm thử các dịch vụ của mình. Có thể cũng cần thêm máy móc và chuyên môn để giữ cho hạ tầng này luôn vận hành. Nhưng một khi nó đã vận hành, nó có thể là một cách cực kỳ hiệu quả để hiện thực các kiến trúc hướng sự kiện, có coupling lỏng. Nhìn chung, tôi là một người hâm mộ.

Do be wary, though, about the world of middleware, of which the message broker is just a small part. Queues in and of themselves are perfectly sensible, useful things. However, vendors tend to want to package lots of software with them, which can lead to more and more smarts being pushed into the middleware, as evidenced by things like the enterprise service bus. Make sure you know what you're getting: keep your middleware dumb, and keep the smarts in the endpoints.

Tuy nhiên, hãy cảnh giác với cái thế giới của middleware, mà message broker chỉ là một phần nhỏ. Bản thân các hàng đợi là những thứ hoàn toàn hợp lý, hữu ích. Tuy nhiên, các nhà cung cấp có xu hướng muốn đóng gói rất nhiều phần mềm cùng với chúng, điều này có thể dẫn đến ngày càng nhiều “trí tuệ” bị đẩy vào middleware, như được minh chứng bởi những thứ như enterprise service bus. Hãy chắc chắn rằng bạn biết mình đang nhận về cái gì: giữ cho middleware của bạn ngu ngốc, và giữ “trí tuệ” ở các điểm cuối (endpoint).

Another approach is to try to use HTTP as a way of propagating events. Atom is a **REST**-compliant specification that defines semantics (among other things) for publishing feeds of resources. Many client libraries exist that allow us to create and consume these feeds. So our customer service could just publish an event to such a feed whenever our customer service changes. Our consumers simply poll the feed, looking for changes. On one hand, the fact that we can reuse the existing Atom specification and any associated libraries is useful, and we know that HTTP handles scale very well. However, this use of HTTP is not good at low latency (where some message brokers excel), and we still must deal with the fact that the consumers need to keep track of what messages they have seen and manage their own polling schedule.

Một cách tiếp cận khác là cố dùng HTTP như một cách để lan truyền các sự kiện. Atom là một đặc tả tuân thủ REST định nghĩa các ngữ nghĩa (cùng nhiều thứ khác) để công bố các luồng (feed) tài nguyên. Nhiều thư viện client tồn tại cho phép chúng ta tạo và tiêu thụ các luồng này. Vậy nên dịch vụ customer của chúng ta có thể chỉ việc công bố một sự kiện đến một luồng như vậy mỗi khi dịch vụ customer của chúng ta thay đổi. Các bên tiêu thụ của chúng ta đơn giản là polling luồng đó, tìm kiếm các thay đổi. Một mặt, việc chúng ta có thể tái sử dụng đặc tả Atom hiện có và bất kỳ thư viện liên quan nào là hữu ích, và chúng ta biết rằng HTTP xử lý quy mô rất tốt. Tuy nhiên, cách dùng HTTP này không tốt ở độ trễ thấp (nơi một số message broker tỏa sáng), và chúng ta vẫn phải đối mặt với thực tế rằng các bên tiêu thụ cần theo dõi những thông điệp nào chúng đã thấy và tự quản lý lịch trình polling của riêng mình.

I have seen people spend ages implementing more and more of the behaviors that you get out of the box with an appropriate message broker to make Atom work for some use cases. For example, the competing consumer pattern describes a method whereby you bring up multiple worker instances to compete for messages, which works well for **scaling** up the number of workers to handle a list of independent jobs (we'll come back to that in the next chapter). However, we want to avoid the case in which two or more workers see the same message, as we'll end up doing the same task more than we need to. With a message broker, a standard queue will handle this. With Atom, we now need to manage our own shared state among all the workers to try to reduce the chances of reproducing effort.

Tôi đã thấy người ta tốn cả đồng thời gian hiện thực ngày càng nhiều các hành vi mà bạn nhận được sẵn từ một message broker phù hợp để khiến Atom hoạt động cho một số trường hợp sử dụng. Ví dụ, competing consumer pattern mô tả một phương pháp theo đó bạn dựng lên nhiều instance worker để cạnh tranh giành các thông điệp, điều này hoạt động tốt cho việc mở rộng số lượng worker nhằm xử lý một danh sách các công việc độc lập (chúng ta sẽ trở lại điều đó ở chương tiếp theo). Tuy nhiên, chúng ta muốn tránh trường hợp trong đó hai hay nhiều worker cùng thấy một thông điệp, vì rồi cuộc chúng ta sẽ làm cùng một tác vụ nhiều hơn mức cần thiết. Với một message broker, một hàng đợi tiêu chuẩn sẽ xử lý điều này. Với Atom, giờ đây chúng ta cần quản lý trạng thái dùng chung của riêng mình giữa tất cả các worker để cố giảm khả năng tái lập công sức.

If you already have a good, resilient message broker available to you, consider using it to handle publishing and subscribing to events. If you don't already have one, give Atom a look, but be aware of the sunk cost fallacy. If you find yourself wanting more and more of the support that a message broker gives you, at a certain point you might want to change your approach.

Nếu bạn đã có sẵn một message broker tốt, bền bỉ trong tay, hãy cân nhắc dùng nó để xử lý việc công bố và đăng ký các sự kiện. Nếu bạn chưa có sẵn cái nào, hãy ngó qua Atom, nhưng hãy lưu ý đến nguy biện chi phí chìm (sunk cost fallacy). Nếu bạn thấy mình ngày càng muốn nhiều hơn sự hỗ trợ mà một message broker mang lại, thì đến một thời điểm nào đó bạn có thể muốn thay đổi cách tiếp cận của mình.

In terms of what we actually send over these asynchronous protocols, the same considerations apply as with synchronous communication. If you are currently happy with encoding requests and responses using JSON, stick with it.

Về phần những gì chúng ta thực sự gửi qua các giao thức bất đồng bộ này, vẫn áp dụng những cân nhắc tương tự như với giao tiếp đồng bộ. Nếu bạn hiện đang hài lòng với việc mã hóa các yêu cầu và phản hồi bằng JSON, hãy cứ giữ nó.

What's in an Event? — Bên trong một Sự kiện có gì?

In Figure 4-12 , we see an event being broadcast from the Customer microservice, informing interested parties that a new customer has registered with the system. Two of the downstream microservices, Loyalty and Notifications , care about this event. The Loyalty microservice reacts to receiving the event by setting up an account for the new customer so that they can start earning points, whereas the Notifications microservice sends an email to the newly registered customer welcoming them to the wondrous delights of MusicCorp.

Trong Hình 4-12, chúng ta thấy một sự kiện đang được phát quảng bá từ microservice Customer, thông báo cho các bên quan tâm rằng một khách hàng mới đã đăng ký với hệ thống. Hai trong số các microservice hạ nguồn, Loyalty và Notifications, quan tâm đến sự kiện này. Microservice Loyalty phản ứng với việc nhận sự kiện bằng cách thiết lập một tài khoản cho khách hàng mới để họ có thể bắt đầu tích điểm, trong khi microservice Notifications gửi một email đến khách hàng vừa đăng ký để chào mừng họ đến với những niềm vui kỳ diệu của MusicCorp.

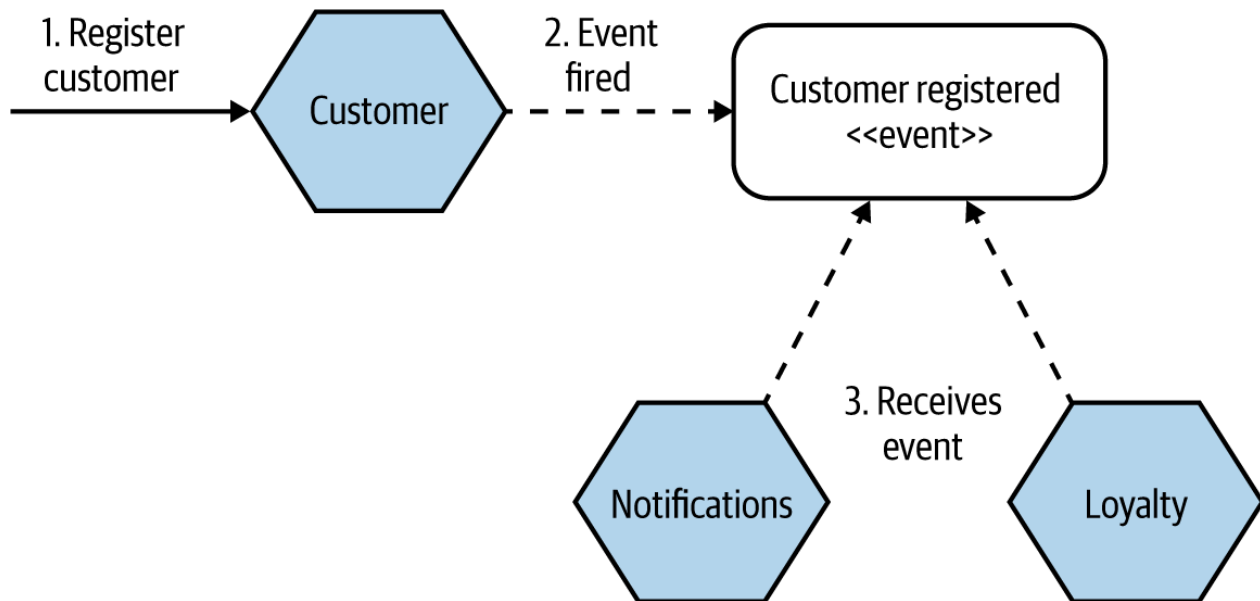


Figure 4-12. Notifications and Loyalty microservices receive an event when a new customer is registered

Hình 4-12. Các microservice Notifications và Loyalty nhận một sự kiện khi một khách hàng mới được đăng ký

With a request, we are asking a microservice to do something and providing the required information for the requested operation to be carried out. With an event, we are broadcasting a fact that other parties might be interested in, but as the microservice emitting an event can't and shouldn't know who receives the event, how do we know what information other parties might need from the event? What, exactly, should be inside the event?

Với một yêu cầu, chúng ta đang đề nghị một microservice làm một việc gì đó và cung cấp thông tin cần thiết để thao tác được yêu cầu được thực hiện. Với một sự kiện, chúng ta đang phát quảng bá một sự thật mà các bên khác có thể quan tâm, nhưng vì microservice phát ra một sự kiện không thể và không nên biết ai nhận sự kiện đó, làm thế nào chúng ta biết những thông tin nào mà các bên khác có thể cần từ sự kiện? Chính xác thì cái gì nên nằm bên trong sự kiện?

Just an ID

Chỉ một ID

One option is for the event to just contain an identifier for the newly registered customer, as shown in Figure 4-13 . The Loyalty microservice needs only this identifier to create the matching loyalty account, so it has all the information it needs. However, while the Notifications microservice knows that it needs to send a welcome email when this type of event is received, it will need additional information to do its job—at least an email address, and probably the name of the customer as well to give the email that personal touch. As this information isn't in the event that the Notifications microservice receives, it has no choice but to fetch this information from the Customer microservice, something we see in Figure 4-13 .

Một lựa chọn là sự kiện chỉ chứa một định danh (identifier) cho khách hàng vừa đăng ký, như được minh họa trong Hình 4-13. Microservice Loyalty chỉ cần định danh này để tạo tài khoản loyalty tương ứng, nên nó có tất cả thông tin nó cần. Tuy nhiên, trong khi microservice Notifications biết rằng nó cần gửi một email chào mừng khi loại sự kiện này được nhận, nó sẽ cần thêm thông tin để làm việc của mình—ít nhất là một địa chỉ email, và có lẽ cả tên của khách hàng nữa để cho email cái chạm cá nhân đó. Vì thông tin này không nằm trong sự kiện mà microservice Notifications nhận, nó không còn lựa chọn nào khác ngoài việc đi lấy thông tin này từ microservice Customer, điều mà chúng ta thấy trong Hình 4-13.

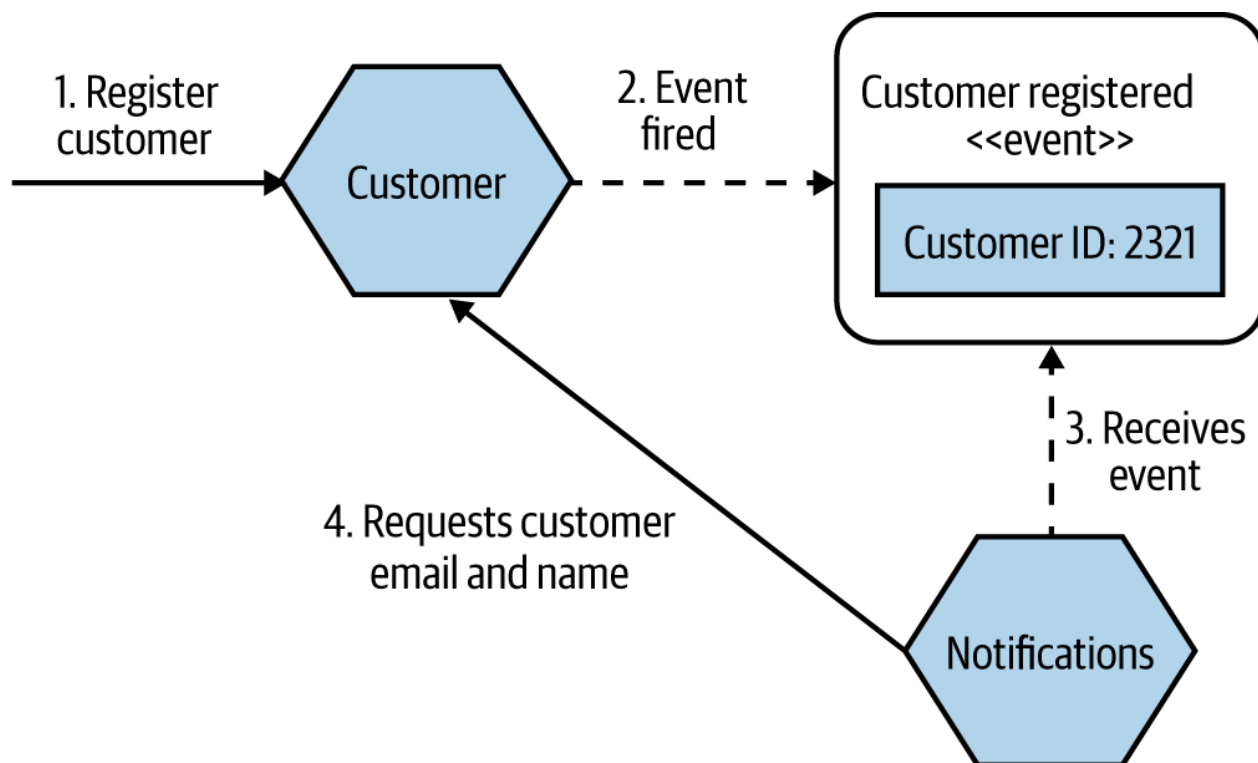


Figure 4-13. The Notifications microservice needs to request further details from the Customer microservice that aren't included in the event

Hình 4-13. Microservice Notifications cần yêu cầu thêm chi tiết từ microservice Customer mà không được bao gồm trong sự kiện

There are some downsides with this approach. Firstly, the Notifications microservice now has to know about the Customer microservice, adding additional domain coupling. While domain

coupling, as we discussed in Chapter 2, is on the looser end of the coupling spectrum, we'd still like to avoid it where possible. If the event that the Notifications microservice received contained all the information it needed, then this callback wouldn't be required. The callback from the receiving microservice can also lead to the other major downside—namely, that in a situation with a large number of receiving microservices, the microservice emitting the event might get a barrage of requests as a result. Imagine if five different microservices all received the same customer creation event, and all needed to request additional information—they'd all have to immediately send a request to the Customer microservice to get what they needed. As the number of microservices interested in a particular event increases, the impact of these calls could become significant.

Có một số nhược điểm với cách tiếp cận này. Thứ nhất, microservice Notifications giờ đây phải biết về microservice Customer, làm tăng thêm domain coupling. Mặc dù domain coupling, như chúng ta đã thảo luận ở Chương 2, nằm ở đầu lỏng hơn của quang phổ coupling, chúng ta vẫn muốn tránh nó nếu có thể. Nếu sự kiện mà microservice Notifications nhận được chứa tất cả thông tin nó cần, thì lời gọi ngược (callback) này sẽ không cần thiết. Lời gọi ngược từ microservice nhận cũng có thể dẫn đến nhược điểm lớn còn lại—cụ thể, trong một tình huống với một số lượng lớn các microservice nhận, microservice phát ra sự kiện có thể nhận được một loạt yêu cầu dồn dập như một hệ quả. Hãy tưởng tượng nếu năm microservice khác nhau đều nhận cùng một sự kiện tạo khách hàng, và tất cả đều cần yêu cầu thêm thông tin—tất cả chúng sẽ phải ngay lập tức gửi một yêu cầu đến microservice Customer để lấy thứ chúng cần. Khi số lượng microservice quan tâm đến một sự kiện cụ thể tăng lên, tác động của những lời gọi này có thể trở nên đáng kể.

Fully detailed events

Sự kiện chi tiết đầy đủ

The alternative, which I prefer, is to put everything into an event that you would be happy otherwise sharing via an API. If you'd let the Notifications microservice ask for the email address and name of a given customer, why not just put that information in the event in the first place? In Figure 4-14, we see this approach—Notifications is now more self-sufficient and is able to do its job without needing to communicate with the Customer microservice. In fact, it might never need to know the Customer microservice exists.

Lựa chọn thay thế, mà tôi ưa thích hơn, là đặt mọi thứ vào một sự kiện mà nếu không thì bạn cũng sẽ vui vẻ chia sẻ qua một API. Nếu bạn cho phép microservice Notifications hỏi về địa chỉ email và tên của một khách hàng nhất định, thì tại sao không đặt luôn thông tin đó vào sự kiện ngay từ đầu? Trong Hình 4-14, chúng ta thấy cách tiếp cận này—Notifications giờ đây tự túc hơn và có thể làm việc của mình mà không cần giao tiếp với microservice Customer. Trên thực tế, nó có thể không bao giờ cần biết microservice Customer tồn tại.

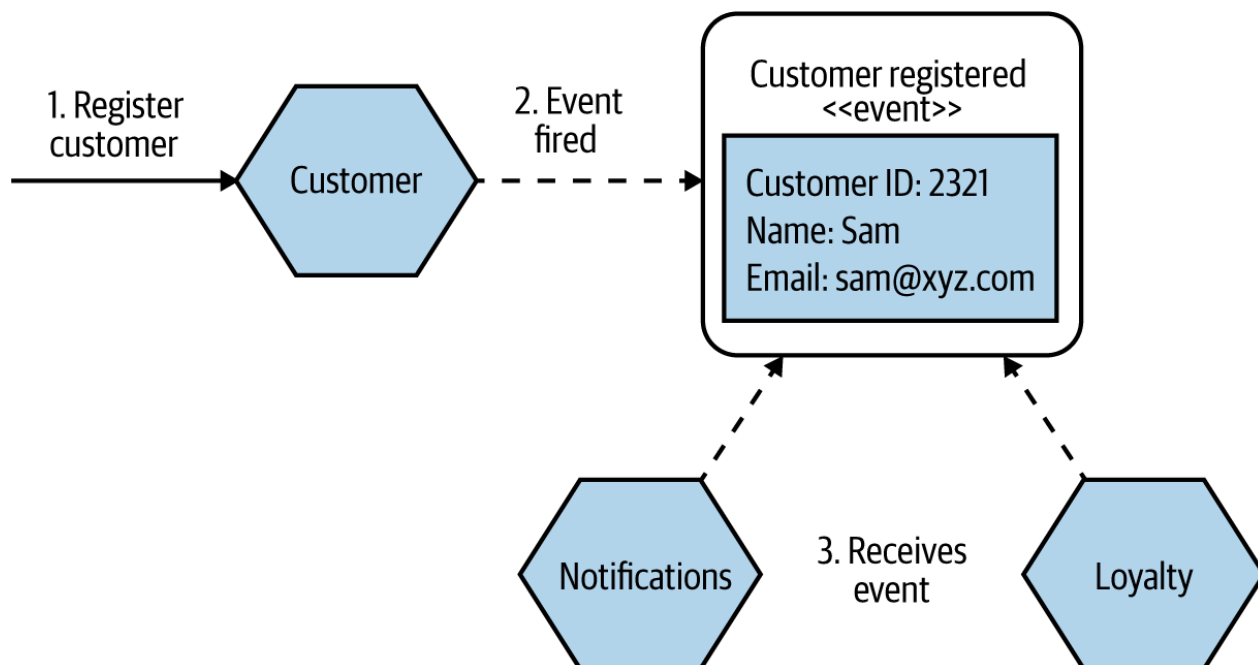


Figure 4-14. An event with more information in it can allow receiving microservices to act without requiring further calls to the source of the event

Hình 4-14. Một sự kiện với nhiều thông tin hơn bên trong nó có thể cho phép các microservice nhận hành động mà không đòi hỏi thêm các lời gọi đến nguồn của sự kiện

In addition to the fact that events with more information can allow for looser coupling, events with more information can also double as a historical record of what happened to a given entity. This could help you as part of implementing an auditing system, or perhaps even provide the ability to reconstitute an entity at given points in time—meaning that these events could be used as part of an event sourcing, a concept we’ll explore briefly in a moment.

Ngoài thực tế rằng các sự kiện với nhiều thông tin hơn có thể cho phép coupling lỏng hơn, các sự kiện với nhiều thông tin hơn cũng có thể đồng thời đóng vai trò như một bản ghi lịch sử về những gì đã xảy ra với một thực thể (entity) nhất định. Điều này có thể giúp bạn như một phần của việc hiện thực một hệ thống kiểm toán (auditing), hoặc thậm chí có lẽ cung cấp khả năng tái dựng một thực thể tại các thời điểm nhất định—nghĩa là những sự kiện này có thể được dùng như một phần của event sourcing, một khái niệm mà chúng ta sẽ khám phá sơ qua trong chốc lát.

While this approach is definitely my preference, it’s not without downsides. Firstly, if the data associated with an event is large, we might have concerns about the size of the event. Modern message brokers (assuming you’re using one to implement your event broadcast mechanism) have fairly generous limits for message size; the default maximum size for a message in Kafka is 1 MB, and the latest **release** of RabbitMQ has a theoretical upper limit of 512 MB for a single message (down from the previous limit of 2 GB!), even though one could expect there to be some interesting performance issues with large messages like this. But even the 1 MB afforded to us as the maximum size of a message on Kafka gives us a lot of scope to send quite a bit of data. Ultimately, if you’re venturing into a space in which you are starting to worry about the size of your events, then I’d recommend a hybrid approach in which some information is in the event but other (larger) data can be looked up if required.

Mặc dù cách tiếp cận này chắc chắn là cái tôi ưa thích, nó không phải là không có nhược

điểm. Thứ nhất, nếu dữ liệu liên quan đến một sự kiện là lớn, chúng ta có thể có những mối lo về kích thước của sự kiện. Các message broker hiện đại (giả định bạn đang dùng một cái để hiện thực cơ chế phát quảng bá sự kiện của mình) có những giới hạn khá rộng rãi cho kích thước thông điệp; kích thước tối đa mặc định cho một thông điệp trong Kafka là 1 MB, và bản phát hành mới nhất của RabbitMQ có một giới hạn trên lý thuyết là 512 MB cho một thông điệp đơn lẻ (giảm từ giới hạn trước đó là 2 GB!), mặc dù người ta có thể kỳ vọng sẽ có một số vấn đề hiệu năng thú vị với những thông điệp lớn như thế này. Nhưng ngay cả 1 MB được dành cho chúng ta như kích thước tối đa của một thông điệp trên Kafka cũng cho chúng ta rất nhiều dư địa để gửi kha khá dữ liệu. Rốt cuộc, nếu bạn đang mạo hiểm bước vào một không gian trong đó bạn bắt đầu lo lắng về kích thước của các sự kiện của mình, thì tôi sẽ khuyên một cách tiếp cận lai (hybrid) trong đó một số thông tin nằm trong sự kiện nhưng các dữ liệu khác (lớn hơn) có thể được tra cứu nếu cần.

In Figure 4-14, Loyalty doesn't need to know the email address or name of the customer, and yet it nonetheless receives it via the event. This could lead to concerns if we are trying to limit the scope of which microservices can see what kind of data—for example, I might want to limit which microservices can see personally identifiable information (or PII), payment card details, or similar sensitive data. A way to solve this could be to send two different types of events—one that contains PII and can be seen by some microservices, and another that excludes PII and can be broadcast more widely. This adds complexity in terms of managing visibility of different events and ensuring that both events actually get fired. What happens when a microservice sends the first type of event but dies before the second event can be sent?

Trong Hình 4-14, Loyalty không cần biết địa chỉ email hay tên của khách hàng, vậy mà nó vẫn nhận được chúng qua sự kiện. Điều này có thể dẫn đến những mối lo nếu chúng ta đang cố giới hạn phạm vi về việc microservice nào có thể thấy loại dữ liệu nào—ví dụ, tôi có thể muốn giới hạn microservice nào có thể thấy thông tin định danh cá nhân (personally identifiable information, hay PII), chi tiết thẻ thanh toán, hay dữ liệu nhạy cảm tương tự. Một cách để giải quyết điều này có thể là gửi hai loại sự kiện khác nhau—một loại chứa PII và có thể được thấy bởi một số microservice, và một loại khác loại trừ PII và có thể được phát quảng bá rộng rãi hơn. Điều này làm tăng độ phức tạp về mặt quản lý khả năng nhìn thấy của các sự kiện khác nhau và đảm bảo rằng cả hai sự kiện thực sự được bắn ra. Điều gì xảy ra khi một microservice gửi loại sự kiện thứ nhất nhưng chết trước khi sự kiện thứ hai có thể được gửi đi?

Another consideration is that once we put data into an event, it becomes part of our contract with the outside world. We have to be aware that if we remove a field from an event, we may break external parties. Information hiding is still an important concept in event-driven collaboration—the more data we put into an event, the more assumptions external parties will have about the event. My general rule is that I am OK putting information into an event if I'd be happy sharing the same data over a request-response API.

Một cân nhắc khác là một khi chúng ta đặt dữ liệu vào một sự kiện, nó trở thành một phần của hợp đồng của chúng ta với thế giới bên ngoài. Chúng ta phải ý thức rằng nếu chúng ta gỡ một trường khỏi một sự kiện, chúng ta có thể phá vỡ các bên bên ngoài. Che giấu thông tin vẫn là một khái niệm quan trọng trong cộng tác hướng sự kiện—càng nhiều dữ liệu chúng ta đặt vào một sự kiện, càng nhiều giả định các bên bên ngoài sẽ có về sự kiện đó. Nguyên tắc chung của tôi là tôi thấy ổn khi đặt thông tin vào một sự kiện nếu tôi sẽ vui vẻ chia sẻ cùng dữ liệu đó qua một API request-response.

Where to Use It — Khi nào nên dùng

Event-driven collaboration thrives in situations in which information wants to be broadcast, and in situations in which you are happy to invert intent. Moving away from a model of telling other things what to do and instead letting downstream microservices work this out for themselves has a great deal of attraction.

Cộng tác hướng sự kiện phát triển mạnh trong các tình huống mà thông tin muốn được phát quảng bá, và trong các tình huống mà bạn vui vẻ với việc đảo ngược ý định. Việc rời xa một mô hình bảo những thứ khác phải làm gì và thay vào đó để các microservice hạ nguồn tự mình tìm ra điều này có một sức hấp dẫn rất lớn.

In a situation in which you are focusing on **loose coupling** more than other factors, event-driven collaboration is going to have obvious appeal.

Trong một tình huống mà bạn đang tập trung vào coupling lỏng hơn các yếu tố khác, cộng tác hướng sự kiện sẽ có sức hấp dẫn rõ ràng.

The cautionary note is that there are often new sources of complexity that come to the fore with this style of collaboration, especially if you've had limited exposure to it. If you are unsure about this form of communication, remember that our microservice architecture can (and likely will) contain a mix of different styles of interaction. You don't have to go all in with event-driven collaboration; perhaps start with just one event and take it from there.

Lưu ý cảnh báo là thường có những nguồn phức tạp mới nổi lên hàng đầu với phong cách cộng tác này, đặc biệt nếu bạn có ít trải nghiệm với nó. Nếu bạn không chắc chắn về hình thức giao tiếp này, hãy nhớ rằng kiến trúc microservice của chúng ta có thể (và nhiều khả năng sẽ) chứa một sự pha trộn các phong cách tương tác khác nhau. Bạn không cần phải dồn tất cả vào cộng tác hướng sự kiện; có lẽ hãy bắt đầu chỉ với một sự kiện rồi đi tiếp từ đó.

Personally, I find myself gravitating toward event-driven collaboration almost as a default. My brain seems to have rewired itself in such a way that these types of communication just seem obvious to me. This isn't entirely helpful, as it can be tricky to try and explain why this is the case, other than to say it feels right. But that is just my own built-in bias—I naturally gravitate to what I know, based on my own experiences. There is a strong possibility that my attraction to this form of interaction is driven almost entirely by my previous bad experiences with overly coupled systems. I might just be the general fighting the last battle over and over again without considering that perhaps this time it really is different.

Nói riêng cá nhân, tôi thấy mình có xu hướng nghiêng về cộng tác hướng sự kiện gần như theo mặc định. Não tôi dường như đã tự đấu nối lại theo cách mà những loại giao tiếp này có vẻ hiển nhiên với tôi. Điều này không hoàn toàn hữu ích, vì có thể khá khó để cố giải thích vì sao lại như vậy, ngoài việc nói rằng nó cảm thấy đúng. Nhưng đó chỉ là thiên kiến cố hữu của riêng tôi—tôi tự nhiên nghiêng về thứ tôi biết, dựa trên trải nghiệm của chính mình. Có một khả năng lớn rằng sự hấp dẫn của tôi đối với hình thức tương tác này gần như hoàn toàn bị thúc đẩy bởi những trải nghiệm tồi tệ trước đây của tôi với các hệ thống bị coupling quá mức. Tôi có thể chỉ đang là viên tướng cứ đánh đi đánh lại trận chiến vừa qua mà không cân nhắc rằng có lẽ lần này nó thực sự khác.

What I will say, putting my own biases aside, is that I see far more teams replacing request-response interactions with event-driven interactions than the reverse.

Điều tôi sẽ nói, gạt thiên kiến của riêng mình sang một bên, là tôi thấy nhiều đội nhóm thay thế các tương tác request-response bằng các tương tác hướng sự kiện hơn là chiều

ngược lại rất nhiều.

Proceed with Caution — Hãy thận trọng khi tiến hành

Some of this asynchronous stuff seems fun, right? Event-driven architectures seem to lead to significantly more decoupled, scalable systems. And they can. But these communication styles do lead to an increase in complexity. This isn't just the complexity required to manage publishing and subscribing to messages, as we just discussed, but also complexity in the other problems we might face. For example, when considering long-running async request-response, we have to think about what to do when the response comes back. Does it come back to the same node that initiated the request? If so, what happens if that node is down? If not, do I need to store information somewhere so I can react accordingly? Short-lived async can be easier to manage if you've got the right APIs, but even so, it is a different way of thinking for programmers who are accustomed to intra-process synchronous message calls.

Một số thứ bất đồng bộ này có vẻ vui, phải không? Các kiến trúc hướng sự kiện dường như dẫn đến những hệ thống tách rời, có khả năng mở rộng cao hơn đáng kể. Và chúng có thể làm được điều đó. Nhưng những phong cách giao tiếp này quả thực dẫn đến một sự gia tăng về độ phức tạp. Đây không chỉ là độ phức tạp cần thiết để quản lý việc công bố và đăng ký các thông điệp, như chúng ta vừa thảo luận, mà còn là độ phức tạp trong những vấn đề khác mà chúng ta có thể gặp phải. Ví dụ, khi cân nhắc request-response bất đồng bộ chạy lâu, chúng ta phải nghĩ về việc phải làm gì khi phản hồi quay về. Liệu nó có quay về đúng node đã khởi tạo yêu cầu không? Nếu vậy, điều gì xảy ra nếu node đó bị sập? Nếu không, liệu tôi có cần lưu thông tin ở đâu đó để tôi có thể phản ứng tương ứng không? Bất đồng bộ ngắn hạn có thể dễ quản lý hơn nếu bạn có các API phù hợp, nhưng dù vậy, nó là một cách tư duy khác đối với các lập trình viên đã quen với các lời gọi thông điệp đồng bộ trong tiến trình.

It's time for a cautionary tale. Back in 2006, I was working on building a pricing system for a bank. We would look at market events and work out which items in a portfolio needed to be repriced. Once we determined the list of things to work through, we put these all onto a message queue. We were making use of a grid to create a pool of pricing workers, allowing us to scale up and down the pricing farm on request. These workers used the competing consumers pattern, each one gobbling messages as fast as possible until there was nothing left to process.

Đã đến lúc kể một câu chuyện cảnh báo. Hồi năm 2006, tôi đang làm việc xây dựng một hệ thống định giá cho một ngân hàng. Chúng tôi sẽ xem xét các sự kiện thị trường và tính toán xem những mặt hàng nào trong một danh mục đầu tư cần được định giá lại. Một khi chúng tôi xác định được danh sách những thứ cần xử lý, chúng tôi đặt tất cả những thứ này lên một hàng đợi thông điệp. Chúng tôi đang tận dụng một lưới (grid) để tạo ra một bể (pool) các worker định giá, cho phép chúng tôi mở rộng tăng và giảm trang trại định giá theo yêu cầu. Các worker này dùng competing consumers pattern, mỗi worker ngẫu nhiên các thông điệp nhanh hết mức có thể cho đến khi không còn gì để xử lý.

The system was up and running, and we were feeling rather smug. One day, though, just after we pushed a release out, we hit a nasty problem: our workers kept dying. And dying. And dying.

Hệ thống đã chạy và vận hành, và chúng tôi đang cảm thấy khá tự mãn. Tuy nhiên, một ngày nọ, ngay sau khi chúng tôi đẩy ra một bản phát hành, chúng tôi gặp phải một vấn đề khó chịu: các worker của chúng tôi cứ chết. Và chết. Và chết.

Eventually, we tracked down the problem. A bug had crept in whereby a certain type of pricing

request would cause a worker to crash. We were using a transacted queue: as the worker died, its lock on the request timed out, and the pricing request was put back on the queue—only for another worker to pick it up and die. This was a classic example of what Martin Fowler calls a catastrophic failover .

Cuối cùng, chúng tôi lần ra được vấn đề. Một con bug đã len lỏi vào theo đó một loại yêu cầu định giá nhất định sẽ khiến một worker sập. Chúng tôi đang dùng một hàng đợi có giao dịch (transacted queue): khi worker chết, khóa của nó trên yêu cầu hết hạn, và yêu cầu định giá được đặt trở lại lên hàng đợi—chỉ để một worker khác lấy nó lên và chết. Đây là một ví dụ kinh điển về cái mà Martin Fowler gọi là catastrophic failover (chuyển đổi dự phòng thảm họa).

Aside from the bug itself, we'd failed to specify a maximum retry limit for the job on the queue. So we fixed the bug, and configured a maximum retry. But we also realized we needed a way to view and potentially replay these bad messages. We ended up having to implement a message hospital (or dead letter queue), where messages got sent if they failed. We also created a UI to view those messages and retry them if needed. These sorts of problems aren't immediately obvious if you are familiar only with synchronous point-to-point communication.

Ngoài bản thân con bug, chúng tôi đã không chỉ định một giới hạn thử lại tối đa cho công việc trên hàng đợi. Vậy nên chúng tôi sửa con bug, và cấu hình một số lần thử lại tối đa. Nhưng chúng tôi cũng nhận ra rằng chúng tôi cần một cách để xem và có khả năng phát lại những thông điệp tồi tệ này. Rốt cuộc chúng tôi đã phải hiện thực một bệnh viện thông điệp (message hospital, hay dead letter queue), nơi các thông điệp được gửi đến nếu chúng thất bại. Chúng tôi cũng tạo một UI để xem những thông điệp đó và thử lại chúng nếu cần. Những loại vấn đề này không hiển hiện ngay lập tức nếu bạn chỉ quen thuộc với giao tiếp điểm-đến-điểm (point-to-point) đồng bộ.

The associated complexity with event-driven architectures and asynchronous programming in general leads me to believe that you should be cautious in how eagerly you start adopting these ideas. Ensure you have good **monitoring** in place, and strongly consider the use of correlation IDs, which allow you to trace requests across process boundaries, as we'll cover in depth in Chapter 10 .

Độ phức tạp đi kèm với các kiến trúc hướng sự kiện và lập trình bất đồng bộ nói chung khiến tôi tin rằng bạn nên thận trọng trong việc háo hức đến mức nào khi bắt đầu áp dụng những ý tưởng này. Hãy đảm bảo bạn có giám sát tốt được đặt sẵn, và mạnh mẽ cân nhắc việc dùng các correlation ID, vốn cho phép bạn truy vết các yêu cầu xuyên qua các ranh giới tiến trình, như chúng ta sẽ đề cập sâu ở Chương 10.

I also strongly recommend checking out Enterprise Integration Patterns by Gregor Hohpe and Bobby Woolf, 4 which contains a lot more detail on the different messaging patterns that you may want to consider in this space.

Tôi cũng mạnh mẽ khuyến nghị tìm đọc Enterprise Integration Patterns của Gregor Hohpe và Bobby Woolf, 4 cuốn sách chứa nhiều chi tiết hơn về các messaging pattern khác nhau mà bạn có thể muốn cân nhắc trong không gian này.

We also have to be honest, though, about the integration styles that we might consider “simpler”—the problems associated with knowing whether things worked or not is not limited to asynchronous forms of integration. With a synchronous, blocking call, if you get a time-out, did this happen because the request got lost and the downstream party didn't receive it? Or did the request get through, but the response got lost? What do you do in that situation? If you retry, but the original request did get through what then? (Well, this is where **idempotency** comes in, a topic we cover in Chapter 12 .)

Tuy nhiên, chúng ta cũng phải thành thật về các phong cách tích hợp mà chúng ta có thể

coi là “đơn giản hơn”—các vấn đề liên quan đến việc biết liệu mọi thứ có hoạt động hay không không chỉ giới hạn ở các hình thức tích hợp bất đồng bộ. Với một lời gọi đồng bộ, có chặn, nếu bạn gặp một time-out, điều này xảy ra vì yêu cầu bị mất và bên hạ nguồn không nhận được nó? Hay yêu cầu đã đến nơi, nhưng phản hồi bị mất? Bạn làm gì trong tình huống đó? Nếu bạn thử lại, nhưng yêu cầu ban đầu thực sự đã đến nơi thì sao? (Vâng, đây là chỗ tính idempotent phát huy tác dụng, một chủ đề mà chúng ta đề cập ở Chương 12.)

Arguably, with regard to failure handling, synchronous blocking calls can cause us just as many headaches when it comes to working out if things have happened (or not). It’s just that those headaches may be more familiar to us!

Có thể nói, về mặt xử lý lỗi, các lời gọi đồng bộ có chặn có thể gây cho chúng ta không ít đau đầu khi đến lúc phải tính xem liệu mọi thứ đã xảy ra (hay chưa). Chỉ là những cơn đau đầu đó có thể quen thuộc với chúng ta hơn mà thôi!

Summary — Tóm tắt

In this chapter, I broke down some of the key styles of microservice communication and discussed the various trade-offs. There isn’t always a single right option, but hopefully I’ve detailed enough information regarding synchronous and asynchronous calls and event-driven and request-response styles of communication to help you make the correct call for your given context. My own biases toward asynchronous, event-driven collaboration are a function not just of my experiences but also of my aversion to coupling in general. But this style of communication comes with significant complexity that cannot be ignored, and every situation is unique.

Trong chương này, tôi đã bóc tách một số phong cách chính của giao tiếp microservice và thảo luận về các đánh đổi khác nhau. Không phải lúc nào cũng có một lựa chọn đúng duy nhất, nhưng hy vọng tôi đã trình bày đủ thông tin về các lời gọi đồng bộ và bất đồng bộ và các phong cách giao tiếp hướng sự kiện và request-response để giúp bạn đưa ra lựa chọn đúng cho bối cảnh cụ thể của mình. Những thiên kiến của riêng tôi nghiêng về cộng tác bất đồng bộ, hướng sự kiện là một hàm số không chỉ của các trải nghiệm của tôi mà còn của sự ác cảm của tôi với coupling nói chung. Nhưng phong cách giao tiếp này đi kèm với độ phức tạp đáng kể không thể bị bỏ qua, và mỗi tình huống là độc nhất.

In this chapter, I briefly mentioned a few specific technologies that can be used to implement these interaction styles. We’re now ready to begin the second part of this book—implementation. In the next chapter, we’ll explore implementing microservice communication in more depth.

Trong chương này, tôi đã nhắc qua một vài công nghệ cụ thể có thể được dùng để hiện thực các phong cách tương tác này. Giờ chúng ta đã sẵn sàng bắt đầu phần thứ hai của cuốn sách—triển khai. Ở chương tiếp theo, chúng ta sẽ khám phá việc hiện thực giao tiếp microservice sâu hơn.

4 Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

4 Gregor Hohpe and Bobby Woolf, *Enterprise Integration Patterns* (Boston: Addison-Wesley, 2003).

CHAPTER 5 Implementing Microservice Communication — Chương 5: Hiện thực giao tiếp giữa các Microservice

As we discussed in the previous chapter, your choice of technology should be driven in large part by the style of communication you want. Deciding between **blocking synchronous** or **nonblocking asynchronous** calls, **request-response** or **event-driven** collaboration, will help you whittle down what might otherwise be a very long list of technology. In this chapter, we're going to look at some of the technology commonly used for **microservice** communication.

Như đã bàn ở chương trước, lựa chọn công nghệ của bạn nên được dẫn dắt phần lớn bởi kiểu giao tiếp mà bạn muốn. Việc quyết định giữa các lời gọi đồng bộ (synchronous) kiểu chặn (blocking) hay bất đồng bộ (asynchronous) kiểu không chặn (nonblocking), giữa cộng tác kiểu request-response hay hướng sự kiện (event-driven), sẽ giúp bạn thu gọn lại một danh sách công nghệ mà nếu không thì có thể rất dài. Trong chương này, chúng ta sẽ xem xét một số công nghệ thường được dùng cho giao tiếp giữa các microservice.

Looking for the Ideal Technology — Tìm công nghệ lý tưởng

There is a bewildering array of options for how one microservice can talk to another. But which is the right one—SOAP? XML-RPC? **REST**? gRPC? And new options are always coming out. So before we discuss specific technology, let's think about what we want out of which technology we pick.

Có vô số lựa chọn cho việc một microservice giao tiếp với một microservice khác như thế nào. Nhưng đâu mới là lựa chọn đúng—SOAP? XML-RPC? REST? gRPC? Và những lựa chọn mới luôn liên tục xuất hiện. Vậy nên, trước khi bàn về công nghệ cụ thể, hãy nghĩ xem chúng ta muốn gì ở công nghệ mà mình chọn.

Make Backward Compatibility Easy — Làm cho khả năng tương thích ngược trở nên dễ dàng

When making changes to our **microservices**, we need to make sure we don't break compatibility with any consuming microservices. As such, we want to ensure that whatever technology we pick makes it easy to make backward-compatible changes. Simple operations like adding new fields

shouldn't break clients. We also ideally want the ability to validate that the changes we have made are backward-compatible—and have a way to get that feedback before we **deploy** our microservice into production.

Khi thay đổi các microservice của mình, chúng ta cần đảm bảo không phá vỡ khả năng tương thích với bất kỳ microservice tiêu thụ (consumer) nào. Vì vậy, ta muốn chắc chắn rằng bất kỳ công nghệ nào ta chọn cũng giúp việc thực hiện những thay đổi tương thích ngược (backward-compatible) trở nên dễ dàng. Những thao tác đơn giản như thêm trường mới không nên làm hỏng client. Lý tưởng nhất, ta cũng muốn có khả năng kiểm chứng rằng những thay đổi ta đã thực hiện là tương thích ngược—và có cách để nhận phản hồi đó trước khi triển khai (deploy) microservice lên môi trường production.

Make Your Interface Explicit — Làm cho interface của bạn tường minh

It is important that the **interface** that a microservice exposes to the outside world is explicit. This means that it is clear to a consumer of a microservice as to what functionality that microservice exposes. But it also means that it is clear to a developer working on the microservice what functionality needs to remain intact for external parties—we want to avoid a situation in which a change to a microservice causes an accidental breakage in compatibility.

Điều quan trọng là interface (giao diện) mà một microservice phơi bày ra thế giới bên ngoài phải tường minh. Điều này nghĩa là một consumer của microservice phải thấy rõ microservice đó phơi bày những chức năng nào. Nhưng nó cũng có nghĩa là một lập trình viên làm việc trên microservice phải thấy rõ những chức năng nào cần được giữ nguyên vẹn cho các bên bên ngoài—chúng ta muốn tránh tình huống trong đó một thay đổi đối với microservice gây phá vỡ khả năng tương thích một cách vô tình.

Explicit schemas can go a long way in helping ensure that the interface a microservice exposes is explicit. Some of the technology we can look at requires the use of a schema; for other technology, the use of a schema is optional. Either way, I strongly encourage the use of an explicit schema, as well as there being enough supporting documentation to be clear about what functionality a consumer can expect a microservice to provide.

Schema tường minh có thể giúp ích rất nhiều trong việc đảm bảo rằng interface mà một microservice phơi bày là tường minh. Một số công nghệ ta sẽ xem xét đòi hỏi phải dùng schema; với một số công nghệ khác, việc dùng schema là tùy chọn. Dù thế nào đi nữa, tôi mạnh mẽ khuyến khích việc dùng schema tường minh, kèm theo đủ tài liệu hỗ trợ để làm rõ một consumer có thể kỳ vọng microservice cung cấp những chức năng gì.

Keep Your APIs Technology Agnostic — Giữ cho API của bạn không phụ thuộc công nghệ

If you have been in the IT industry for more than 15 minutes, you don't need me to tell you that we work in a space that is changing rapidly. The one certainty is change. New tools, frameworks, and languages are coming out all the time, implementing new ideas that can help us work faster and more effectively. Right now, you might be a .NET shop. But what about a year from now, or five years from now? What if you want to experiment with an alternative technology stack that might make you more productive?

Nếu bạn đã làm trong ngành IT hơn 15 phút, thì tôi không cần phải nói cho bạn biết rằng chúng ta đang làm việc trong một lĩnh vực thay đổi nhanh chóng. Điều chắc chắn duy nhất là sự thay đổi. Các công cụ, framework và ngôn ngữ mới liên tục ra đời, hiện thực những ý tưởng mới có thể giúp ta làm việc nhanh hơn và hiệu quả hơn. Ngay lúc này, có thể bạn đang là một cửa hàng .NET. Nhưng một năm nữa thì sao, hay năm năm nữa thì sao? Nếu bạn muốn thử nghiệm một technology stack thay thế có thể giúp bạn làm việc năng suất hơn thì sao?

I am a big fan of keeping my options open, which is why I am such a fan of microservices. It is also why I think it is very important to ensure that you keep the APIs used for communication between microservices technology agnostic. This means avoiding integration technology that dictates what technology stacks we can use to implement our microservices.

Tôi là một người rất thích giữ cho các lựa chọn của mình luôn rộng mở, đó là lý do tôi rất thích microservice. Đó cũng là lý do tôi cho rằng việc giữ cho các API dùng để giao tiếp giữa các microservice không phụ thuộc công nghệ là rất quan trọng. Điều này có nghĩa là tránh dùng công nghệ tích hợp nào áp đặt technology stack nào ta có thể dùng để hiện thực các microservice của mình.

Make Your Service Simple for Consumers — Làm cho dịch vụ của bạn đơn giản với consumer

We want to make it easy for consumers to use our microservice. Having a beautifully factored microservice doesn't count for much if the cost of using it as a consumer is sky high! So let's think about what makes it easy for consumers to use our wonderful new **service**. Ideally, we'd like to allow our clients full freedom in their technology choice; on the other hand, providing a client library can ease adoption. Often, however, such libraries are incompatible with other things we want to achieve. For example, we might use client libraries to make it easy for consumers, but this can come at the cost of increased **coupling**.

Chúng ta muốn việc sử dụng microservice của mình trở nên dễ dàng đối với consumer. Có một microservice được phân tách đẹp để chẳng đáng giá là bao nếu chi phí để dùng nó với tư cách consumer lại cao ngất ngưỡng! Vậy hãy nghĩ xem điều gì khiến consumer dễ dàng sử dụng dịch vụ mới tuyệt vời của chúng ta. Lý tưởng nhất, ta muốn cho client toàn quyền tự do trong lựa chọn công nghệ của họ; mặt khác, việc cung cấp một thư viện client (client library) có thể giúp việc áp dụng dễ dàng hơn. Tuy nhiên, các thư viện như vậy thường không tương thích với những điều khác mà ta muốn đạt được. Ví dụ, ta có thể dùng client library để làm cho consumer thấy dễ dàng, nhưng điều này có thể phải trả giá bằng coupling (sự phụ thuộc) gia tăng.

Hide Internal Implementation Detail — Ẩn đi chi tiết hiện thực nội bộ

We don't want our consumers to be bound to our internal implementation, as it leads to increased coupling; that in turn means that if we want to change something inside our microservice, we can break our consumers by requiring them to also change. That increases the cost of change—exactly what we are trying to avoid. It also means we are less likely to want to make a change for fear of having to upgrade our consumers, which can lead to increased **technical debt** within the service. So any technology that pushes us to expose internal representation detail should be avoided.

Chúng ta không muốn consumer của mình bị ràng buộc vào hiện thực nội bộ của ta, vì điều đó dẫn đến coupling gia tăng; điều đó lại có nghĩa là nếu ta muốn thay đổi điều gì đó bên trong microservice, ta có thể làm hỏng consumer khi buộc họ cũng phải thay đổi theo. Điều đó làm tăng chi phí của sự thay đổi—đúng cái mà chúng ta đang cố tránh. Nó cũng có nghĩa là ta sẽ ít có khả năng muốn thực hiện một thay đổi vì sợ phải nâng cấp consumer, điều này có thể dẫn đến nợ kỹ thuật (technical debt) gia tăng bên trong dịch vụ. Vậy nên bất kỳ công nghệ nào thúc ép ta phải phơi bày chi tiết biểu diễn nội bộ đều nên bị tránh.

Technology Choices — Các lựa chọn công nghệ

There is a whole host of technology we could look at, but rather than looking broadly at a long list of options, I will highlight some of the most popular and interesting choices. Here are the options we'll be looking at:

Có cả một loạt công nghệ mà ta có thể xem xét, nhưng thay vì nhìn rộng vào một danh sách dài các lựa chọn, tôi sẽ làm nổi bật một vài lựa chọn phổ biến và thú vị nhất. Dưới đây là những lựa chọn mà chúng ta sẽ xem xét:

Remote procedure calls Frameworks that allow for local method calls to be invoked on a remote process. Common options include SOAP and gRPC.

Lời gọi thủ tục từ xa (Remote procedure call) Các framework cho phép gọi các phương thức cục bộ để được thực thi trên một tiến trình từ xa. Các lựa chọn phổ biến gồm SOAP và gRPC.

REST An architectural style where you expose resources (Customer, Order, etc.) that can be accessed using a common set of verbs (GET, POST). There is a bit more to REST than this, but we'll get to that shortly.

REST Một phong cách kiến trúc trong đó bạn phơi bày các tài nguyên (Customer, Order, v.v.) có thể được truy cập bằng một tập động từ chung (GET, POST). REST còn nhiều thứ hơn thế một chút, nhưng ta sẽ bàn tới đó sớm thôi.

GraphQL A relatively new protocol that allows consumers to define custom queries that can fetch information from multiple downstream microservices, filtering the results to return only what is needed.

GraphQL Một giao thức tương đối mới cho phép consumer định nghĩa các truy vấn tùy biến để lấy thông tin từ nhiều microservice phía dưới, lọc kết quả để chỉ trả về những gì cần thiết.

Message brokers Middleware that allows for asynchronous communication via queues or topics.

Message broker Phần mềm trung gian (middleware) cho phép giao tiếp bất đồng bộ qua hàng đợi (queue) hoặc chủ đề (topic).

Remote Procedure Calls — Lời gọi thủ tục từ xa (Remote Procedure Calls)

Remote procedure call (RPC) refers to the technique of making a local call and having it execute on a remote service somewhere. There are a number of different RPC implementations in use. Most of the technology in this space requires an explicit schema, such as SOAP or gRPC. In the context of RPC, the schema is often referred to as an interface definition language (IDL), with SOAP referring to

its schema format as a web service definition language (WSDL). The use of a separate schema makes it easier to generate client and server stubs for different technology stacks—so, for example, I could have a Java server exposing a SOAP interface, and a .NET client generated from the same WSDL definition of the interface. Other technology, such as Java RMI, calls for a tighter coupling between the client and the server, requiring that both use the same underlying technology but avoid the need for an explicit service definition, as the service definition is implicitly provided by the Java type definitions. All these technologies, however, have the same core characteristic: they make a remote call look like a local call.

Lời gọi thủ tục từ xa (remote procedure call - RPC) chỉ kỹ thuật thực hiện một lời gọi cục bộ và để nó được thực thi trên một dịch vụ từ xa ở đâu đó. Có một số hiện thực RPC khác nhau đang được dùng. Hầu hết công nghệ trong không gian này đòi hỏi một schema tường minh, chẳng hạn như SOAP hoặc gRPC. Trong bối cảnh RPC, schema thường được gọi là ngôn ngữ định nghĩa interface (interface definition language - IDL), với SOAP gọi định dạng schema của nó là ngôn ngữ định nghĩa dịch vụ web (web service definition language - WSDL). Việc dùng một schema riêng biệt giúp việc sinh ra các đoạn mã stub cho client và server dễ dàng hơn cho các technology stack khác nhau—chẳng hạn, tôi có thể có một server Java phơi bày một interface SOAP, và một client .NET được sinh ra từ cùng định nghĩa WSDL của interface đó. Các công nghệ khác, như Java RMI, đòi hỏi coupling chặt hơn giữa client và server, yêu cầu cả hai dùng cùng một công nghệ nền tảng nhưng tránh được nhu cầu có một định nghĩa dịch vụ tường minh, vì định nghĩa dịch vụ được cung cấp một cách ngầm định thông qua các định nghĩa kiểu của Java. Tuy nhiên, tất cả những công nghệ này đều có chung một đặc điểm cốt lõi: chúng khiến một lời gọi từ xa trông giống như một lời gọi cục bộ.

Typically, using an RPC technology means you are buying into a serialization protocol. The RPC framework defines how data is serialized and deserialized. For instance, gRPC uses the protocol buffer serialization format for this purpose. Some implementations are tied to a specific networking protocol (like SOAP, which makes nominal use of HTTP), whereas others might allow you to use different types of networking protocols, which can provide additional features. For example, TCP offers guarantees about delivery, whereas UDP doesn't but has a much lower overhead. This can allow you to use different networking technology for different use cases.

Thông thường, việc dùng một công nghệ RPC đồng nghĩa với việc bạn đang chấp nhận một giao thức tuần tự hóa (serialization). Framework RPC định nghĩa cách dữ liệu được tuần tự hóa (serialize) và giải tuần tự hóa (deserialize). Chẳng hạn, gRPC dùng định dạng tuần tự hóa protocol buffer cho mục đích này. Một số hiện thực bị gắn chặt vào một giao thức mạng cụ thể (như SOAP, vốn dùng HTTP một cách hình thức), trong khi những hiện thực khác có thể cho phép bạn dùng các loại giao thức mạng khác nhau, điều này có thể mang lại các tính năng bổ sung. Ví dụ, TCP cung cấp các đảm bảo về việc phân phối, trong khi UDP thì không nhưng lại có chi phí thấp hơn nhiều. Điều này cho phép bạn dùng công nghệ mạng khác nhau cho các trường hợp sử dụng khác nhau.

RPC frameworks that have an explicit schema make it very easy to generate client code. This can avoid the need for client libraries, as any client can just generate their own code against this service specification. For client-side code generation to work, though, the client needs some way to get the schema out of band—in other words, the consumer needs to have access to the schema before it plans to make calls. Avro RPC is an interesting outlier here, as it has the option to send the full schema along with the payload, allowing clients to dynamically interpret the schema.

Các framework RPC có schema tường minh khiến việc sinh mã client trở nên rất dễ dàng. Điều này có thể tránh được nhu cầu có các client library, vì bất kỳ client nào cũng chỉ cần tự sinh mã của mình dựa trên đặc tả dịch vụ này. Tuy nhiên, để việc sinh mã phía client

hoạt động, client cần một cách nào đó để lấy được schema từ ngoài luồng (out of band)—nói cách khác, consumer cần có quyền truy cập schema trước khi nó định thực hiện các lời gọi. Avro RPC là một ngoại lệ thú vị ở đây, vì nó có tùy chọn gửi toàn bộ schema cùng với payload, cho phép client diễn giải schema một cách động.

The ease of generation of client-side code is one of the main selling points of RPC. The fact that I can just make a normal method call and theoretically ignore the rest is a huge boon.

Sự dễ dàng trong việc sinh mã phía client là một trong những điểm bán hàng chính của RPC. Việc tôi chỉ cần thực hiện một lời gọi phương thức bình thường và về lý thuyết có thể bỏ qua phần còn lại là một lợi ích to lớn.

Challenges

Các thách thức

As we've seen, RPC offers some great advantages, but it's not without its downsides—and some RPC implementations can be more problematic than others. Many of these issues can be dealt with, but they deserve further exploration.

Như ta đã thấy, RPC mang lại một số lợi thế tuyệt vời, nhưng nó không phải không có nhược điểm—và một số hiện thực RPC có thể gây nhiều rắc rối hơn những hiện thực khác. Nhiều vấn đề trong số này có thể được xử lý, nhưng chúng đáng được khám phá kỹ hơn.

Technology coupling. Some RPC mechanisms, like Java RMI, are heavily tied to a specific platform, which can limit which technology can be used in the client and server. Thrift and gRPC have an impressive amount of support for alternative languages, which can reduce this downside somewhat, but be aware that RPC technology sometimes comes with restrictions on interoperability.

Coupling công nghệ. Một số cơ chế RPC, như Java RMI, bị gắn chặt vào một nền tảng cụ thể, điều này có thể giới hạn công nghệ nào được dùng ở client và server. Thrift và gRPC có mức hỗ trợ ấn tượng cho các ngôn ngữ thay thế, điều này có thể giảm bớt nhược điểm này phần nào, nhưng hãy lưu ý rằng công nghệ RPC đôi khi đi kèm các hạn chế về khả năng tương tác (interoperability).

In a way, this technology coupling can be a form of exposing internal technical implementation details. For example, the use of RMI ties not only the client to the JVM but the server as well.

Theo một cách nào đó, coupling công nghệ này có thể là một dạng phơi bày chi tiết hiện thực kỹ thuật nội bộ. Ví dụ, việc dùng RMI không chỉ gắn client vào JVM mà còn gắn cả server vào JVM.

To be fair, there are a number of RPC implementations that don't have this restriction—gRPC, SOAP, and Thrift are all examples that allow for interoperability between different technology stacks.

Công bằng mà nói, có một số hiện thực RPC không có hạn chế này—gRPC, SOAP và Thrift đều là những ví dụ cho phép khả năng tương tác giữa các technology stack khác nhau.

Local calls are not like remote calls. The core idea of RPC is to hide the complexity of a remote call. However, this can lead to hiding too much. The drive in some forms of RPC to make remote method calls look like local method calls hides the fact that these two things are very different. I can make large numbers of local, in-process calls without worrying too much about the performance. With RPC, though, the cost of marshaling and unmarshaling payloads can be significant, not to mention the time taken to send things over the network. This means you need to think differently about **API** design for remote interfaces versus local interfaces. Just taking a local API and trying to make it a service boundary without any more thought is likely to get you in trouble. In some of the worst examples, developers may be using remote calls without knowing it, if the abstraction is overly opaque.

Lời gọi cục bộ không giống lời gọi từ xa. Ý tưởng cốt lõi của RPC là ẩn đi sự phức tạp của một lời gọi từ xa. Tuy nhiên, điều này có thể dẫn đến việc ẩn đi quá nhiều. Trong một số dạng RPC, nỗ lực khiến lời gọi phương thức từ xa trông giống như lời gọi phương thức cục bộ đã che giấu thực tế rằng hai thứ này rất khác nhau. Tôi có thể thực hiện một số lượng lớn lời gọi cục bộ, trong cùng tiến trình, mà không phải lo lắng quá nhiều về hiệu năng. Tuy nhiên với RPC, chi phí của việc đóng gói (marshaling) và giải đóng gói (unmarshaling) payload có thể đáng kể, chưa kể thời gian gửi mọi thứ qua mạng. Điều này có nghĩa là bạn cần suy nghĩ khác đi về thiết kế API cho các interface từ xa so với interface cục bộ. Việc chỉ lấy một API cục bộ và cố biến nó thành một ranh giới dịch vụ mà không suy nghĩ gì thêm thì rất có khả năng gây rắc rối cho bạn. Trong một số ví dụ tệ nhất, lập trình viên có thể đang dùng lời gọi từ xa mà không hề hay biết, nếu lớp trừu tượng quá đỗi mờ đục.

You need to think about the network itself. Famously, the first of the fallacies of distributed computing is “The network is reliable” . Networks aren’t reliable. They can and will fail, even if your client and the server you are speaking to are fine. They can fail fast, they can fail slow, and they can even mangle your packets. You should assume that your networks are plagued with malevolent entities ready to unleash their ire on a whim. Therefore, you can expect to encounter types of failure modes that you may never have had to deal with in simpler, monolithic software. A failure could be caused by the remote server returning an error, or by you making a bad call. Can you tell the difference, and if so, can you do anything about it? And what do you do when the remote server just starts responding slowly? We’ll cover this topic when we talk about **resiliency** in Chapter 12 .

Bạn cần nghĩ về chính mạng lưới. Như đã nổi tiếng, nguy hiểm đầu tiên trong các nguy hiểm về điện toán phân tán là “Mạng thì đáng tin cậy”. Mạng thì không đáng tin cậy. Chúng có thể và sẽ hỏng, ngay cả khi client của bạn và server bạn đang trò chuyện cùng đều ổn. Chúng có thể hỏng nhanh, có thể hỏng chậm, và thậm chí có thể làm méo mó các gói tin của bạn. Bạn nên giả định rằng mạng của mình bị quấy nhiễu bởi các thực thể độc ác sẵn sàng trút cơn thịnh nộ của chúng bất cứ lúc nào. Do đó, bạn có thể trông đợi sẽ gặp những dạng lỗi mà có lẽ bạn chưa bao giờ phải xử lý trong phần mềm monolith đơn giản hơn. Một lỗi có thể do server từ xa trả về lỗi, hoặc do bạn thực hiện một lời gọi sai. Bạn có thể phân biệt được không, và nếu có, bạn có thể làm gì với nó không? Và bạn sẽ làm gì khi server từ xa chỉ đơn giản là bắt đầu phản hồi chậm? Chúng ta sẽ đề cập đến chủ đề này khi bàn về khả năng phục hồi (resiliency) ở Chương 12.

Brittleness. Some of the most popular implementations of RPC can lead to some nasty forms of brittleness, Java RMI being a very good example. Let’s consider a very simple Java interface that we have decided to make a remote API for our Customer service. Example 5-1 declares the methods we are going to expose remotely. Java RMI then generates the client and server stubs for our method.

Tính giòn (Brittleness). Một số hiện thực phổ biến nhất của RPC có thể dẫn đến những dạng giòn khó chịu, Java RMI là một ví dụ rất điển hình. Hãy xem xét một interface Java rất đơn giản mà ta đã quyết định biến thành một API từ xa cho dịch vụ Customer của mình. Ví dụ 5-1 khai báo các phương thức mà ta sẽ phơi bày ra từ xa. Java RMI sau đó sinh ra các stub client và server cho phương thức của ta.

Example 5-1. Defining a service endpoint using Java RMI

Ví dụ 5-1. Định nghĩa một service endpoint bằng Java RMI

```
import java.rmi.Remote; import java.rmi.RemoteException;

import java.rmi.Remote; import java.rmi.RemoteException;

public interface CustomerRemote extends Remote { public Customer findCustomer(String id) throws
RemoteException;
```

```
public interface CustomerRemote extends Remote { public Customer findCustomer(String id) throws RemoteException;
```

`public Customer createCustomer( String firstname, String surname, String emailAddress) throws RemoteException; }` In this interface, `createCustomer` takes the first name, surname, and email address. What happens if we decide to allow the Customer object to also be created with just an email address? We could add a new method at this point pretty easily, like so:

```
public Customer createCustomer( String firstname, String surname, String emailAddress) throws RemoteException; }
```

Trong interface này, `createCustomer` nhận tên, họ và địa chỉ email. Điều gì xảy ra nếu ta quyết định cho phép đối tượng Customer cũng có thể được tạo chỉ với một địa chỉ email? Ta có thể thêm một phương thức mới tại thời điểm này khá dễ dàng, như sau:

`... public Customer createCustomer(String emailAddress) throws RemoteException; ...` The problem is that now we need to regenerate the client stubs too. Clients that want to consume the new method need the new stubs, and depending on the nature of the changes to the specification, consumers that don't need the new method may also need to have their stubs upgraded. This is manageable, of course, but only to a point. The reality is that changes like this are fairly common. RPC endpoints often end up having a large number of methods for different ways of creating or interacting with objects. This is due in part to the fact that we are still thinking of these remote calls as local ones.

`... public Customer createCustomer(String emailAddress) throws RemoteException; ...` Vấn đề là giờ ta cũng cần sinh lại các stub client. Những client muốn dùng phương thức mới cần các stub mới, và tùy vào bản chất của các thay đổi đối với đặc tả, ngay cả những consumer không cần phương thức mới cũng có thể cần phải nâng cấp stub của họ. Tất nhiên điều này quản lý được, nhưng chỉ đến một mức nào đó. Thực tế là những thay đổi như thế này khá phổ biến. Các endpoint RPC thường rất cuộc có một số lượng lớn các phương thức cho những cách khác nhau để tạo hoặc tương tác với đối tượng. Điều này một phần là do ta vẫn còn nghĩ về những lời gọi từ xa này như những lời gọi cục bộ.

There is another sort of brittleness, though. Let's take a look at what our Customer object looks like:

Tuy nhiên, còn một dạng giòn khác nữa. Hãy xem đối tượng Customer của ta trông như thế nào:

```
public class Customer implements Serializable { private String firstName; private String surname; private String emailAddress; private String age; }
```

What if it turns out that, although we expose the age field in our Customer objects, none of our consumers ever use it? We decide we want to remove this field. But if the server implementation removes age from its definition of this type, and we don't do the same to all the consumers, then even though they never used the field, the code associated with deserializing the Customer object on the consumer side will break. To roll out this change, we'd need to make changes to the client code to support the new definition and deploy these updated clients at the same time as we roll out the new version of the server. This is a key challenge with any RPC mechanism that promotes the use of binary stub generation: you don't get to separate client and server deployments. If you use this technology, lockstep releases may be in your future.

```
public class Customer implements Serializable { private String firstName; private String surname; private String emailAddress; private String age; }
```

Điều gì xảy ra nếu hóa ra, mặc dù ta phơi bày trường age trong các đối tượng Customer, nhưng không consumer nào của ta từng dùng nó? Ta quyết định muốn loại bỏ trường này. Nhưng nếu hiện thực phía server loại bỏ age khỏi định nghĩa của kiểu này, mà ta không làm điều tương tự với tất cả consumer, thì dù chúng chưa bao giờ dùng trường đó, đoạn mã liên quan đến việc giải tuần tự hóa đối tượng Customer ở phía consumer sẽ bị hỏng. Để triển khai thay đổi này,

ta sẽ phải thay đổi mã client để hỗ trợ định nghĩa mới và triển khai các client đã cập nhật này cùng lúc với khi ta triển khai phiên bản mới của server. Đây là một thách thức chủ chốt với bất kỳ cơ chế RPC nào cố vũ việc dùng sinh stub nhị phân: bạn không thể tách rời việc triển khai client và server. Nếu bạn dùng công nghệ này, các đợt phát hành đồng bộ bước (lockstep release) có thể đang chờ bạn ở tương lai.

Similar problems occur if we want to restructure the Customer object, even if we didn't remove fields—for example, if we wanted to encapsulate firstName and sur name into a new naming type to make it easier to manage. We could, of course, fix this by passing around dictionary types as the parameters of our calls, but at that point, we lose many of the benefits of the generated stubs because we'll still have to manually match and extract the fields we want.

Các vấn đề tương tự cũng xảy ra nếu ta muốn tái cấu trúc đối tượng Customer, ngay cả khi ta không loại bỏ trường nào—ví dụ, nếu ta muốn đóng gói firstName và surname vào một kiểu naming mới để dễ quản lý hơn. Tất nhiên ta có thể khắc phục điều này bằng cách truyền các kiểu từ điển (dictionary) làm tham số cho các lời gọi của mình, nhưng đến lúc đó ta đánh mất nhiều lợi ích của stub được sinh ra vì ta vẫn sẽ phải tự khớp và trích xuất các trường mình muốn theo cách thủ công.

In practice, objects used as part of binary serialization across the wire can be thought of as “expand-only” types. This brittleness results in the types being exposed over the wire and becoming a mass of fields, some of which are no longer used but can't be safely removed.

Trên thực tế, các đối tượng được dùng như một phần của tuần tự hóa nhị phân qua đường truyền có thể được coi là các kiểu “chỉ-mở-rộng” (expand-only). Tính giòn này dẫn đến việc các kiểu được phơi bày qua đường truyền trở thành một mớ các trường, một số trong đó không còn được dùng nhưng không thể loại bỏ an toàn.

Where to use it

Khi nào nên dùng

Despite its shortcomings, I actually quite like RPC, and the more modern implementations, such as gRPC, are excellent, whereas other implementations have significant issues that would cause me to give them a wide berth. Java RMI, for example, has a number of issues regarding brittleness and limited technology choices, and SOAP is pretty heavyweight from a developer perspective, especially when compared with more modern choices.

Bất chấp những thiếu sót của nó, thực ra tôi khá thích RPC, và những hiện thực hiện đại hơn, như gRPC, thì xuất sắc, trong khi các hiện thực khác có những vấn đề đáng kể khiến tôi phải tránh xa. Java RMI chẳng hạn, có một số vấn đề về tính giòn và lựa chọn công nghệ bị hạn chế, còn SOAP thì khá nặng nề từ góc nhìn của lập trình viên, đặc biệt khi so với những lựa chọn hiện đại hơn.

Just be aware of some of the potential pitfalls associated with RPC if you're going to pick this model. Don't abstract your remote calls to the point that the network is completely hidden, and ensure that you can evolve the server interface without having to insist on lockstep upgrades for clients. Finding the right balance for your client code is important, for example. Make sure your clients aren't oblivious to the fact that a network call is going to be made. Client libraries are often used in the context of RPC, and if not structured right they can be problematic. We'll talk more about them shortly.

Chỉ cần lưu ý một số cạm bẫy tiềm tàng liên quan đến RPC nếu bạn định chọn mô hình này. Đừng trừu tượng hóa các lời gọi từ xa của bạn đến mức mạng bị che giấu hoàn toàn, và hãy đảm bảo rằng bạn có thể tiến hóa interface của server mà không phải khăng khăng

đòi nâng cấp đồng bộ bước cho client. Tìm được sự cân bằng đúng đắn cho mã client của bạn là điều quan trọng, chẳng hạn vậy. Hãy đảm bảo client của bạn không vô tâm trước thực tế rằng một lời gọi mạng sắp được thực hiện. Client library thường được dùng trong bối cảnh RPC, và nếu không được cấu trúc đúng cách thì chúng có thể gây rắc rối. Ta sẽ bàn thêm về chúng sớm thôi.

If I was looking at options in this space, gRPC would be at the top of my list. Built to take advantage of HTTP/2, it has some impressive performance characteristics and good general ease of use. I also appreciate the ecosystem around gRPC, including tools like Protolock , which is something we'll discuss later in this chapter when we discuss schemas.

Nếu tôi đang xem xét các lựa chọn trong không gian này, gRPC sẽ đứng đầu danh sách của tôi. Được xây dựng để tận dụng HTTP/2, nó có những đặc tính hiệu năng ấn tượng và dễ dùng nói chung. Tôi cũng đánh giá cao hệ sinh thái xung quanh gRPC, bao gồm các công cụ như Protolock, thứ mà ta sẽ bàn tới sau trong chương này khi nói về schema.

gRPC fits a synchronous request-response model well but can also work in conjunction with reactive extensions. It's high on my list whenever I'm in situations where I have a good deal of control over both the client and server ends of the spectrum. If you're having to support a wide variety of other applications that might need to talk to your microservices, the need to compile client-side code against a server-side schema can be problematic. In that case, some form of REST over HTTP API would likely be a better fit.

gRPC phù hợp tốt với mô hình request-response đồng bộ nhưng cũng có thể hoạt động kết hợp với các reactive extension. Nó đứng cao trong danh sách của tôi mỗi khi tôi ở trong tình huống có nhiều quyền kiểm soát đối với cả đầu client lẫn đầu server. Nếu bạn phải hỗ trợ nhiều loại ứng dụng khác nhau có thể cần trò chuyện với các microservice của bạn, thì nhu cầu phải biên dịch mã phía client dựa trên một schema phía server có thể gây rắc rối. Trong trường hợp đó, một dạng API REST trên HTTP nào đó có lẽ sẽ phù hợp hơn.

REST — REST

Representational State Transfer (REST) is an architectural style inspired by the web. There are many principles and constraints behind the REST style, but we are going to focus on those that really help us when we face integration challenges in a microservices world, and when we're looking for an alternative to RPC for our service interfaces.

Representational State Transfer (REST) là một phong cách kiến trúc lấy cảm hứng từ web. Có nhiều nguyên tắc và ràng buộc đằng sau phong cách REST, nhưng chúng ta sẽ tập trung vào những điều thực sự giúp ích khi ta đối mặt với các thách thức tích hợp trong thế giới microservice, và khi ta tìm kiếm một lựa chọn thay thế cho RPC cho các interface dịch vụ của mình.

Most important when thinking about REST is the concept of resources. You can think of a resource as a thing that the service itself knows about, like a Customer . The server creates different representations of this Customer on request. How a resource is shown externally is completely decoupled from how it is stored internally. A client might ask for a JSON representation of a Customer , for example, even if it is stored in a completely different format. Once a client has a representation of this Customer , it can then make requests to change it, and the server may or may not comply with them.

Quan trọng nhất khi nghĩ về REST là khái niệm tài nguyên (resource). Bạn có thể coi một tài nguyên là một thứ mà bản thân dịch vụ biết về nó, như một Customer. Server tạo ra

các biểu diễn (representation) khác nhau của Customer này khi có yêu cầu. Cách một tài nguyên được thể hiện ra bên ngoài hoàn toàn tách rời khỏi cách nó được lưu trữ bên trong. Một client có thể yêu cầu biểu diễn JSON của một Customer chẳng hạn, ngay cả khi nó được lưu trữ ở một định dạng hoàn toàn khác. Một khi client đã có biểu diễn của Customer này, nó có thể tiếp tục gửi các yêu cầu để thay đổi nó, và server có thể tuân theo hoặc không.

There are many different styles of REST, and I touch only briefly on them here. I strongly recommend you take a look at the Richardson Maturity Model , where the different styles of REST are compared.

Có nhiều phong cách REST khác nhau, và ở đây tôi chỉ chạm tới chúng một cách sơ lược. Tôi mạnh mẽ khuyến nghị bạn xem qua Richardson Maturity Model, nơi các phong cách REST khác nhau được so sánh.

REST itself doesn't really talk about underlying protocols, although it is most commonly used over HTTP. I have seen implementations of REST using very different protocols before, although this can require a lot of work. Some of the features that HTTP gives us as part of the specification, such as verbs, make implementing REST over HTTP easier, whereas with other protocols you'll have to handle these features yourself.

Bản thân REST không thực sự nói về các giao thức nền tảng, mặc dù nó thường được dùng nhất trên HTTP. Tôi đã từng thấy các hiện thực REST dùng các giao thức rất khác nhau, mặc dù điều này có thể đòi hỏi rất nhiều công sức. Một số tính năng mà HTTP cho ta như một phần của đặc tả, chẳng hạn như các động từ (verb), khiến việc hiện thực REST trên HTTP dễ dàng hơn, trong khi với các giao thức khác bạn sẽ phải tự xử lý các tính năng này.

REST and HTTP

REST và HTTP

HTTP itself defines some useful capabilities that play very well with the REST style. For instance, the HTTP verbs (e.g., GET, POST, and PUT) already have well-understood meanings in the HTTP specification as to how they should work with resources. The REST architectural style actually tells us that these verbs should behave the same way on all resources, and the HTTP specification happens to define a bunch of verbs we can use. GET retrieves a resource in an idempotent way, for example, and POST creates a new resource. This means we can avoid lots of different createCustomer or editCustomer methods. Instead, we can simply POST a customer representation to request that the server create a new resource, and then we can initiate a GET request to retrieve a representation of a resource. Conceptually, there is one endpoint in the form of a Customer resource in these cases, and the operations we can carry out on it are baked into the HTTP protocol.

Bản thân HTTP định nghĩa một số khả năng hữu ích phối hợp rất tốt với phong cách REST. Chẳng hạn, các động từ HTTP (ví dụ GET, POST và PUT) đã có những ý nghĩa được hiểu rõ trong đặc tả HTTP về cách chúng nên hoạt động với tài nguyên. Phong cách kiến trúc REST thực ra bảo ta rằng các động từ này nên hoạt động giống nhau trên mọi tài nguyên, và đặc tả HTTP tình cờ định nghĩa một loạt động từ mà ta có thể dùng. GET truy xuất một tài nguyên theo cách idempotent chẳng hạn, còn POST tạo một tài nguyên mới. Điều này có nghĩa là ta có thể tránh được vô số phương thức createCustomer hay editCustomer khác nhau. Thay vào đó, ta chỉ cần POST một biểu diễn customer để yêu cầu server tạo một tài nguyên mới, rồi ta có thể khởi tạo một yêu cầu GET để truy xuất biểu diễn của một tài nguyên. Về mặt khái niệm, có một endpoint duy nhất dưới dạng một tài nguyên Customer trong các trường hợp này, và các thao tác ta có thể thực hiện trên nó được tích hợp sẵn vào giao thức HTTP.

HTTP also brings a large ecosystem of supporting tools and technology. We get to use HTTP **caching** proxies like Varnish and load balancers like mod_proxy , and many **monitoring** tools already have

lots of support for HTTP out of the box. These building blocks allow us to handle large volumes of HTTP traffic and route them smartly, and in a fairly transparent way. We also get to use all the available **security** controls with HTTP to secure our communications. From basic auth to client certs, the HTTP ecosystem gives us lots of tools to make the security process easier, and we'll explore that topic more in Chapter 11. That said, to get these benefits, you have to use HTTP well. Use it badly, and it can be as insecure and hard to scale as any other technology out there. Use it right, though, and you get a lot of help.

HTTP cũng mang đến một hệ sinh thái lớn các công cụ và công nghệ hỗ trợ. Ta có thể dùng các caching proxy của HTTP như Varnish và các bộ cân bằng tải (load balancer) như mod_proxy, và nhiều công cụ giám sát đã sẵn có rất nhiều hỗ trợ cho HTTP ngay khi cài đặt. Những khối xây dựng này cho phép ta xử lý khối lượng lớn lưu lượng HTTP và định tuyến chúng một cách thông minh, theo một cách khá trong suốt. Ta cũng có thể dùng tất cả các kiểm soát bảo mật sẵn có với HTTP để bảo vệ giao tiếp của mình. Từ basic auth đến client cert, hệ sinh thái HTTP cho ta nhiều công cụ làm cho quy trình bảo mật dễ dàng hơn, và ta sẽ khám phá chủ đề đó thêm ở Chương 11. Tuy nhiên, để có được những lợi ích này, bạn phải dùng HTTP đúng cách. Dùng nó tệ, và nó có thể kém an toàn và khó mở rộng chẳng kém gì bất kỳ công nghệ nào khác ngoài kia. Nhưng dùng nó đúng, và bạn sẽ nhận được rất nhiều trợ giúp.

Note that HTTP can be used to implement RPC too. SOAP, for example, gets routed over HTTP, but it unfortunately uses very little of the specification. Verbs are ignored, as are simple things like HTTP error codes. On the other hand, gRPC has been designed to take advantage of the capabilities of HTTP/2, such as the ability to send multiple request-response streams over a single connection. But of course, when using gRPC, you're not doing REST just because you're using HTTP!

Lưu ý rằng HTTP cũng có thể được dùng để hiện thực RPC. SOAP chẳng hạn, được định tuyến qua HTTP, nhưng tiếc thay nó dùng rất ít đặc tả này. Các động từ bị bỏ qua, cũng như những thứ đơn giản như mã lỗi HTTP. Mặt khác, gRPC được thiết kế để tận dụng các khả năng của HTTP/2, chẳng hạn như khả năng gửi nhiều luồng request-response qua một kết nối duy nhất. Nhưng tất nhiên, khi dùng gRPC, bạn không hề làm REST chỉ vì bạn đang dùng HTTP!

Hypermedia as the engine of application state

Hypermedia như động cơ của trạng thái ứng dụng

Another principle introduced in REST that can help us avoid the coupling between client and server is the concept of hypermedia as the engine of application state (often abbreviated as HATEOAS, and boy, did it need an abbreviation). This is fairly dense wording and a fairly interesting concept, so let's break it down a bit.

Một nguyên tắc khác được giới thiệu trong REST có thể giúp ta tránh coupling giữa client và server là khái niệm hypermedia như động cơ của trạng thái ứng dụng (hypermedia as the engine of application state, thường được viết tắt là HATEOAS, và trời ạ, nó đúng là cần một cách viết tắt). Đây là cách diễn đạt khá rối rắm và là một khái niệm khá thú vị, nên hãy phân tích nó ra một chút.

Hypermedia is a concept wherein a piece of content contains links to various other pieces of content in a variety of formats (e.g., text, images, sounds). This should be pretty familiar to you, as it's what happens in the average web page: you follow links, which are a form of hypermedia controls, to see related content. The idea behind HATEOAS is that clients should perform interactions with the server (potentially leading to state transitions) via these links to other resources. A client doesn't need to know where exactly customers live on the server by knowing which URI to hit; instead, the

client looks for and navigates links to find what it needs.

Hypermedia là một khái niệm trong đó một mẫu nội dung chứa các liên kết đến nhiều mẫu nội dung khác ở nhiều định dạng khác nhau (ví dụ văn bản, hình ảnh, âm thanh). Điều này hẳn khá quen thuộc với bạn, vì đó là điều xảy ra trong một trang web thông thường: bạn theo các liên kết, vốn là một dạng kiểm soát hypermedia (hypermedia control), để xem nội dung liên quan. Ý tưởng đằng sau HATEOAS là client nên thực hiện các tương tác với server (có khả năng dẫn đến chuyển đổi trạng thái) thông qua những liên kết này đến các tài nguyên khác. Một client không cần biết chính xác customer nằm ở đâu trên server bằng cách biết URI nào cần gọi; thay vào đó, client tìm kiếm và điều hướng theo các liên kết để tìm thứ nó cần.

This is a bit of an odd concept, so let's first step back and consider how people interact with a web page, which we have already established is rich with hypermedia controls.

Đây là một khái niệm hơi lạ, nên trước tiên hãy lùi lại một bước và xem xét cách con người tương tác với một trang web, thứ mà ta đã xác định là giàu các kiểm soát hypermedia.

Think of the Amazon.com shopping site. The location of the shopping cart has changed over time. The graphic has changed. The link has changed. But as humans we are smart enough to still see a shopping cart, know what it is, and interact with it. We have an understanding of what a shopping cart means, even if the exact form and underlying control used to represent it have changed. We know that if we want to view the cart, this is the control we want to interact with. This is how web pages can change incrementally over time. As long as these implicit contracts between the customer and the website are still met, changes don't need to be breaking changes.

Hãy nghĩ về trang mua sắm Amazon.com. Vị trí của giỏ hàng đã thay đổi theo thời gian. Hình ảnh đã thay đổi. Liên kết đã thay đổi. Nhưng với tư cách con người, chúng ta đủ thông minh để vẫn nhìn thấy một giỏ hàng, biết nó là gì, và tương tác với nó. Chúng ta có sự hiểu biết về ý nghĩa của một giỏ hàng, ngay cả khi hình thức chính xác và kiểm soát nền tảng được dùng để biểu diễn nó đã thay đổi. Chúng ta biết rằng nếu muốn xem giỏ hàng, đây là kiểm soát mà ta muốn tương tác. Đây là cách các trang web có thể thay đổi từng bước theo thời gian. Miễn là những hợp đồng ngầm này giữa khách hàng và website vẫn được đáp ứng, các thay đổi không cần phải là những thay đổi gây phá vỡ.

With hypermedia controls, we are trying to achieve the same level of “smarts” for our electronic consumers. Let's look at a hypermedia control that we might have for MusicCorp. We've accessed a resource representing a catalog entry for a given album in Example 5-2 . Along with information about the album, we see a number of hypermedia controls.

Với các kiểm soát hypermedia, chúng ta đang cố đạt được cùng mức độ “thông minh” cho các consumer điện tử của mình. Hãy xem một kiểm soát hypermedia mà ta có thể có cho MusicCorp. Ta đã truy cập một tài nguyên biểu diễn một mục catalog cho một album cho trước trong Ví dụ 5-2. Cùng với thông tin về album, ta thấy một số kiểm soát hypermedia.

Example 5-2. Hypermedia controls used on an album listing

Ví dụ 5-2. Các kiểm soát hypermedia được dùng trên một mục danh sách album

Give Blood Awesome, short, brutish, funny and loud. Must buy!

Give Blood Awesome, short, brutish, funny and loud. Must buy!

This hypermedia control shows us where to find information about the artist.

Kiểm soát hypermedia này cho ta biết tìm thông tin về nghệ sĩ ở đâu.

And if we want to purchase the album, we now know where to go.

Và nếu ta muốn mua album, giờ ta đã biết đi đâu.

In this document, we have two hypermedia controls. The client reading such a document needs to know that a control with a relation of artist is where it needs to navigate to get information about the artist, and that `instantpurchase` is part of the protocol used to purchase the album. The client has to understand the semantics of the API in much the same way that a human needs to understand that on a shopping website the cart is where the items to be purchased will be.

Trong tài liệu này, ta có hai kiểm soát hypermedia. Client đọc một tài liệu như vậy cần biết rằng một kiểm soát có quan hệ artist là nơi nó cần điều hướng tới để lấy thông tin về nghệ sĩ, và rằng `instantpurchase` là một phần của giao thức dùng để mua album. Client phải hiểu ngữ nghĩa của API theo cách rất giống cách một con người cần hiểu rằng trên một website mua sắm thì giỏ hàng là nơi chứa các món sẽ được mua.

As a client, I don't need to know which URI scheme to access to buy the album; I just need to access the resource, find the buy control, and navigate to that. The buy control could change location, the URI could change, or the site could even send me to another service altogether, and as a client I wouldn't care. This gives us a huge amount of decoupling between the client and the server.

Với tư cách một client, tôi không cần biết phải truy cập sơ đồ URI nào để mua album; tôi chỉ cần truy cập tài nguyên, tìm kiểm soát mua, và điều hướng tới đó. Kiểm soát mua có thể đổi vị trí, URI có thể thay đổi, hoặc website thậm chí có thể gửi tôi sang một dịch vụ hoàn toàn khác, và với tư cách một client tôi sẽ không bận tâm. Điều này cho ta một mức độ tách rời (decoupling) khổng lồ giữa client và server.

We are greatly abstracted from the underlying detail here. We could completely change the implementation of how the control is presented as long as the client can still find a control that matches its understanding of the protocol, in the same way that a shopping cart control might go from being a simple link to a more complex JavaScript control. We are also free to add new controls to the document, perhaps representing new state transitions that we can perform on the resource in question. We would end up breaking our consumers only if we fundamentally changed the semantics of one of the controls so it behaved very differently, or if we removed a control altogether.

Chúng ta được trừu tượng hóa rất nhiều khỏi chi tiết nền tảng ở đây. Ta có thể hoàn toàn thay đổi hiện thực của cách kiểm soát được trình bày miễn là client vẫn có thể tìm thấy một kiểm soát khớp với hiểu biết của nó về giao thức, giống như cách một kiểm soát giỏ hàng có thể chuyển từ một liên kết đơn giản sang một kiểm soát JavaScript phức tạp hơn. Chúng ta cũng tự do thêm các kiểm soát mới vào tài liệu, có lẽ biểu diễn các chuyển đổi trạng thái mới mà ta có thể thực hiện trên tài nguyên đang xét. Ta sẽ chỉ rối cuộc làm hỏng consumer của mình nếu ta thay đổi căn bản ngữ nghĩa của một trong các kiểm soát khiến nó hoạt động rất khác đi, hoặc nếu ta loại bỏ hoàn toàn một kiểm soát.

The theory is that, by using these controls to decouple the client and server, we gain significant benefits over time that hopefully offset the increase in the time it takes to get these protocols up and running. Unfortunately, although these ideas all seem sensible in theory, I've found that this form of REST is rarely practiced, for reasons I've not entirely come to grips with. This makes HATEOAS in particular a much harder concept for me to promote for those already committed to the use of REST. Fundamentally, many of the ideas in REST are predicated on creating distributed hypermedia systems, and this isn't what most people end up building.

Lý thuyết là, bằng cách dùng các kiểm soát này để tách rời client và server, ta thu được những lợi ích đáng kể theo thời gian mà hy vọng bù đắp cho sự gia tăng thời gian cần để đưa các giao thức này vào vận hành. Tiếc thay, mặc dù những ý tưởng này nghe đều có vẻ

hợp lý về mặt lý thuyết, tôi nhận thấy dạng REST này hiếm khi được thực hành, vì những lý do mà tôi vẫn chưa hoàn toàn nắm bắt được. Điều này khiến HATEOAS nói riêng là một khái niệm khó hơn nhiều để tôi quảng bá cho những người vốn đã cam kết dùng REST. Về căn bản, nhiều ý tưởng trong REST dựa trên tiền đề về việc tạo ra các hệ thống hypermedia phân tán, và đây không phải là thứ mà phần lớn mọi người rất cuộc xây dựng.

Challenges

Các thách thức

In terms of ease of consumption, historically you wouldn't be able to generate client-side code for your REST over HTTP application protocol like you can with RPC implementations. This has often lead to people creating REST APIs that provide client libraries for consumers to make use of. These client libraries give you a binding to the API to make client integration easier. The problem is that client libraries can cause some challenges with regards to coupling between the client and the server, something we'll discuss in "DRY and the Perils of Code Reuse in a Microservice World" on page 154 .

Về mặt dễ tiêu thụ, trong lịch sử bạn sẽ không thể sinh mã phía client cho giao thức ứng dụng REST trên HTTP của mình như có thể làm với các hiện thực RPC. Điều này thường dẫn đến việc người ta tạo các API REST có cung cấp client library để consumer sử dụng. Những client library này cho bạn một binding tới API để giúp việc tích hợp client dễ dàng hơn. Vấn đề là client library có thể gây ra một số thách thức về coupling giữa client và server; điều mà ta sẽ bàn trong mục "DRY và những hiểm họa của tái sử dụng mã trong thế giới Microservice" ở trang 154.

In recent years this problem has been somewhat alleviated. The OpenAPI specification that grew out of the Swagger project now provides you with the ability to define enough information on a REST endpoint to allow for the generation of client-side code in a variety of languages. In my experience, I haven't seen many teams actually making use of this functionality, even if they were already using Swagger for documentation. I have a suspicion that this may be due to the difficulties of retrofitting its use into current APIs. I do also have concerns about a specification that was previously used just for documentation now being used to define a more explicit contract. This can lead to a much more complex specification—comparing an OpenAPI schema with a protocol buffer schema, for example, is quite a stark contrast. Despite my reservations, though, it's good that this option now exists.

Trong những năm gần đây, vấn đề này đã được giảm bớt phần nào. Đặc tả OpenAPI vốn nảy ra từ dự án Swagger nay cho bạn khả năng định nghĩa đủ thông tin trên một endpoint REST để cho phép sinh mã phía client ở nhiều ngôn ngữ. Theo kinh nghiệm của tôi, tôi chưa thấy nhiều nhóm thực sự dùng chức năng này, ngay cả khi họ vốn đã dùng Swagger để làm tài liệu. Tôi ngờ rằng điều này có thể là do những khó khăn của việc cải tạo để dùng nó vào các API hiện có. Tôi cũng có những lo ngại về một đặc tả vốn trước đây chỉ dùng để làm tài liệu nay lại được dùng để định nghĩa một hợp đồng tương minh hơn. Điều này có thể dẫn đến một đặc tả phức tạp hơn nhiều—so sánh một schema OpenAPI với một schema protocol buffer chẳng hạn, là một sự tương phản khá rõ rệt. Tuy nhiên, bất chấp những e ngại của tôi, thật tốt khi lựa chọn này nay đã tồn tại.

Performance may also be an issue. REST over HTTP payloads can actually be more compact than SOAP because REST supports alternative formats like JSON or even binary, but it will still be nowhere near as lean a binary protocol as Thrift might be. The overhead of HTTP for each request may also be a concern for low-latency requirements. All mainstream HTTP protocols in current use require the use of the Transmission Control Protocol (TCP) under the hood, which has inefficiencies compared with other networking protocols, and some RPC implementations allow you to use alternative networking protocols to TCP such as the User Datagram Protocol (UDP).

Hiệu năng cũng có thể là một vấn đề. Payload của REST trên HTTP thực ra có thể gọn hơn SOAP vì REST hỗ trợ các định dạng thay thế như JSON hay thậm chí nhị phân, nhưng nó vẫn sẽ không thể tinh gọn bằng một giao thức nhị phân như Thrift. Chi phí phụ của HTTP cho mỗi request cũng có thể là một mối lo cho các yêu cầu độ trễ thấp. Tất cả các giao thức HTTP chính dùng hiện nay đều đòi hỏi dùng Transmission Control Protocol (TCP) ở bên dưới, vốn có những điểm kém hiệu quả so với các giao thức mạng khác, và một số hiện thực RPC cho phép bạn dùng các giao thức mạng thay thế cho TCP như User Datagram Protocol (UDP).

The limitations placed on HTTP due to the requirement to use TCP are being addressed. HTTP/3, which is currently in the process of being finalized, is looking to shift over to using the newer QUIC protocol. QUIC provides the same sorts of capabilities as TCP (such as improved guarantees over UDP) but with some significant enhancements that have been shown to deliver improvements in latency and reductions in bandwidth. It's likely to be several years before HTTP/3 has a widespread impact on the public internet, but it seems reasonable to assume that organizations can benefit earlier than this within their own networks.

Các giới hạn đặt lên HTTP do yêu cầu phải dùng TCP đang được giải quyết. HTTP/3, hiện đang trong quá trình được hoàn thiện, đang hướng tới chuyển sang dùng giao thức QUIC mới hơn. QUIC cung cấp những loại khả năng giống như TCP (chẳng hạn các đảm bảo cải tiến so với UDP) nhưng kèm theo một số cải tiến đáng kể đã được chứng minh là mang lại cải thiện về độ trễ và giảm băng thông. Có lẽ phải vài năm nữa HTTP/3 mới có tác động rộng rãi lên internet công cộng, nhưng có vẻ hợp lý khi cho rằng các tổ chức có thể hưởng lợi sớm hơn thế ngay trong mạng nội bộ của mình.

With respect to HATEOAS specifically, you can encounter additional performance issues. As clients need to navigate multiple controls to find the right endpoints for a given operation, this can lead to very chatty protocols—multiple round trips may be required for each operation. Ultimately, this is a trade-off. If you decide to adopt a HATEOAS-style of REST, I would suggest you start by having your clients navigate these controls first and then optimize later if necessary. Remember that using HTTP provides us with a large amount of help out of the box, which we discussed earlier. The evils of premature optimization have been well documented before, so I don't need to expand on them here. Also note that a lot of these approaches were developed to create distributed hypertext systems, and not all of them fit! Sometimes you'll find yourself just wanting good old-fashioned RPC.

Riêng với HATEOAS, bạn có thể gặp thêm các vấn đề hiệu năng. Vì client cần điều hướng qua nhiều kiểm soát để tìm đúng endpoint cho một thao tác cho trước, điều này có thể dẫn đến các giao thức rất “lắm lời” (chatty)—có thể cần nhiều vòng khứ hồi cho mỗi thao tác. Rốt cuộc, đây là một sự đánh đổi. Nếu bạn quyết định áp dụng phong cách REST kiểu HATEOAS, tôi đề nghị bạn bắt đầu bằng cách cho client điều hướng theo các kiểm soát này trước rồi tối ưu sau nếu cần. Hãy nhớ rằng dùng HTTP cho ta rất nhiều trợ giúp ngay từ đầu, như đã bàn trước đó. Những điều tệ hại của việc tối ưu quá sớm đã được ghi chép kỹ trước đây, nên tôi không cần phải nói thêm về chúng ở đây. Cũng lưu ý rằng nhiều cách tiếp cận này được phát triển để tạo ra các hệ thống siêu văn bản (hypertext) phân tán, và không phải tất cả chúng đều phù hợp! Đôi khi bạn sẽ thấy mình chỉ muốn RPC kiểu cổ điển tốt lành.

Despite these disadvantages, REST over HTTP is a sensible default choice for service-to-service interactions. If you want to know more, I recommend *REST in Practice: Hypermedia and Systems Architecture* (O'Reilly) by Jim Webber, Savas Parastatidis, and Ian Robinson, which covers the topic of REST over HTTP in depth.

Bất chấp những nhược điểm này, REST trên HTTP là một lựa chọn mặc định hợp lý cho các

tương tác dịch vụ-tới-dịch vụ. Nếu bạn muốn biết thêm, tôi giới thiệu cuốn REST in Practice: Hypermedia and Systems Architecture (O'Reilly) của Jim Webber, Savas Parastatidis và Ian Robinson, vốn bàn về chủ đề REST trên HTTP một cách sâu sắc.

Where to use it

Khi nào nên dùng

Due to its widespread use in the industry, a REST-over-HTTP-based API is an obvious choice for a synchronous request-response interface if you are looking to allow access from as wide a variety of clients as possible. It would be a mistake to think of a REST API as just being a “good enough for most things” choice, but there is something to that. It’s a widely understood style of interface that most people are familiar with, and it guarantees interoperability from a huge variety of technologies.

Do được dùng rộng rãi trong ngành, một API dựa trên REST-trên-HTTP là một lựa chọn hiển nhiên cho một interface request-response đồng bộ nếu bạn muốn cho phép truy cập từ càng nhiều loại client càng tốt. Sẽ là sai lầm nếu nghĩ về một API REST chỉ như một lựa chọn “đủ tốt cho hầu hết mọi thứ”, nhưng có chút gì đó đúng trong điều đó. Đó là một phong cách interface được hiểu rộng rãi mà phần lớn mọi người đều quen thuộc, và nó đảm bảo khả năng tương tác từ vô số công nghệ khác nhau.

Due in large part to the capabilities of HTTP and the extent to which REST builds on these capabilities (rather than hiding them), REST-based APIs excel in situations in which you want large-scale and effective caching of requests. It’s for this reason that they are the obvious choice for exposing APIs to external parties or client interfaces. They may well suffer, though, when compared to more efficient communication protocols, and although you can construct asynchronous interaction protocols over the top of REST-based APIs, that’s not really a great fit compared to the alternatives for general microservice-to-microservice communication.

Phần lớn nhờ vào các khả năng của HTTP và mức độ mà REST xây dựng dựa trên các khả năng này (thay vì che giấu chúng), các API dựa trên REST tỏa sáng trong những tình huống bạn muốn caching các request ở quy mô lớn và hiệu quả. Chính vì lý do này mà chúng là lựa chọn hiển nhiên để phơi bày API ra cho các bên bên ngoài hoặc cho các interface client. Tuy nhiên, chúng có thể chịu thiệt thòi khi so với các giao thức giao tiếp hiệu quả hơn, và mặc dù bạn có thể xây dựng các giao thức tương tác bất đồng bộ chồng lên trên các API dựa trên REST, điều đó không thực sự phù hợp tốt so với các lựa chọn thay thế cho giao tiếp microservice-tới-microservice nói chung.

Despite intellectually appreciating the goals behind HATEOAS, I haven’t seen much evidence that the additional work to implement this style of REST delivers worthwhile benefits in the long run, nor can I recall in the last few years talking to any teams implementing a microservice architecture that can speak to the value of using HATEOAS. My own experiences are obviously only one set of data points, and I don’t doubt that for some people HATEOAS may have worked well. But this concept does not seem to have caught on as much as I thought it would. It could be that the concepts behind HATEOAS are too alien for us to grasp, or it could be the lack of tools or standards in this space, or perhaps the model just doesn’t work for the sorts of systems we have ended up building. It’s also possible, of course, that the concepts behind HATEOAS don’t really mix well with how we **build** microservices.

Mặc dù về mặt trí tuệ tôi đánh giá cao các mục tiêu đằng sau HATEOAS, tôi chưa thấy nhiều bằng chứng cho thấy công sức bổ sung để hiện thực phong cách REST này mang lại những lợi ích xứng đáng về lâu dài, tôi cũng không thể nhớ trong vài năm qua có nói chuyện với nhóm nào hiện thực một kiến trúc microservice mà có thể nói về giá trị của việc dùng HATEOAS. Trải nghiệm của riêng tôi rõ ràng chỉ là một tập điểm dữ liệu, và tôi không nghi ngờ gì rằng với một số người, HATEOAS có thể đã hoạt động tốt. Nhưng khái niệm này có

về không bắt rễ nhiều như tôi nghĩ. Có thể là các khái niệm đằng sau HATEOAS quá xa lạ để ta nắm bắt, hoặc có thể là do thiếu công cụ hay tiêu chuẩn trong không gian này, hoặc có lẽ mô hình này chỉ đơn giản là không hợp với những loại hệ thống mà ta rốt cuộc đã xây dựng. Tất nhiên cũng có khả năng là các khái niệm đằng sau HATEOAS không thực sự hòa hợp tốt với cách ta xây dựng microservice.

So for use at the perimeter, it works fantastically well, and for synchronous request-response-based communication between microservices, it's great.

Vậy nên để dùng ở vành đai (perimeter), nó hoạt động cực kỳ tốt, và để giao tiếp dựa trên request-response đồng bộ giữa các microservice, nó tuyệt vời.

GraphQL — GraphQL

In recent years, GraphQL has gained more popularity, due in large part to the fact that it excels in one specific area. Namely, it makes it possible for a client-side device to define queries that can avoid the need to make multiple requests to retrieve the same information. This can offer significant improvements in terms of the performance of constrained client-side devices and can also avoid the need to implement bespoke server-side aggregation.

Trong những năm gần đây, GraphQL đã trở nên phổ biến hơn, phần lớn nhờ vào việc nó vượt trội trong một lĩnh vực cụ thể. Cụ thể, nó giúp một thiết bị phía client có thể định nghĩa các truy vấn tránh được nhu cầu thực hiện nhiều request để lấy cùng một thông tin. Điều này có thể mang lại những cải thiện đáng kể về hiệu năng cho các thiết bị phía client bị hạn chế tài nguyên, và cũng có thể tránh được nhu cầu hiện thực việc gộp (aggregation) tùy biến ở phía server.

To take a simple example, imagine a mobile device that wants to display a page showing an overview of a customer's latest orders. The page needs to contain some information about the customer, along with information about the customer's five most recent orders. The screen needs only a few fields from the customer record, and only the date, value, and shipped status of each order. The mobile device could issue calls to two downstream microservices to retrieve the required information, but this would involve making multiple calls, including pulling back information that isn't actually required. Especially with mobile devices, this can be wasteful—it uses up more of a mobile device's data plan than is needed, and it can take longer.

Lấy một ví dụ đơn giản, hãy hình dung một thiết bị di động muốn hiển thị một trang cho thấy tổng quan về các đơn hàng gần nhất của một khách hàng. Trang này cần chứa một số thông tin về khách hàng, cùng với thông tin về năm đơn hàng gần nhất của khách hàng đó. Màn hình chỉ cần vài trường từ bản ghi khách hàng, và chỉ cần ngày, giá trị và trạng thái giao hàng của mỗi đơn. Thiết bị di động có thể phát các lời gọi tới hai microservice phía dưới để lấy thông tin cần thiết, nhưng điều này sẽ liên quan đến việc thực hiện nhiều lời gọi, bao gồm cả việc kéo về thông tin không thực sự cần thiết. Đặc biệt với các thiết bị di động, điều này có thể lãng phí—nó dùng nhiều gói cước dữ liệu của thiết bị di động hơn mức cần thiết, và có thể mất nhiều thời gian hơn.

GraphQL allows the mobile device to issue a single query that can pull back all the required information. For this to work, you need a microservice that exposes a GraphQL endpoint to the client device. This GraphQL endpoint is the entry for all client queries and exposes a schema for the client devices to use. This schema exposes the types available to the client, and a nice graphical query builder is also available to make creating these queries easier. By reducing the number of calls and the amount of data retrieved by the client device, you can deal neatly with some of the

challenges that occur when building user interfaces with microservice architectures.

GraphQL cho phép thiết bị di động phát một truy vấn duy nhất có thể kéo về tất cả thông tin cần thiết. Để điều này hoạt động, bạn cần một microservice phơi bày một endpoint GraphQL cho thiết bị client. Endpoint GraphQL này là đầu vào cho tất cả các truy vấn của client và phơi bày một schema để các thiết bị client dùng. Schema này phơi bày các kiểu sẵn có cho client, và còn có sẵn một trình xây dựng truy vấn dạng đồ họa đẹp mắt để làm cho việc tạo các truy vấn này dễ dàng hơn. Bằng cách giảm số lượng lời gọi và lượng dữ liệu mà thiết bị client truy xuất, bạn có thể xử lý gọn gàng một số thách thức nảy sinh khi xây dựng giao diện người dùng với các kiến trúc microservice.

Challenges

Các thách thức

Early on, one challenge was lack of language support for the GraphQL specification, with JavaScript being your only choice initially. This has improved greatly, with all major technologies now having support for the specification. In fact, there have been significant improvements in GraphQL and the various implementations across the board, making GraphQL a much less risky prospect than it might have been a few years ago. That said, you might want to be aware of a few remaining challenges with the technology.

Thời gian đầu, một thách thức là thiếu sự hỗ trợ ngôn ngữ cho đặc tả GraphQL, với JavaScript ban đầu là lựa chọn duy nhất của bạn. Điều này đã cải thiện rất nhiều, với tất cả các công nghệ lớn nay đều có hỗ trợ cho đặc tả. Thực tế, đã có những cải tiến đáng kể trong GraphQL và các hiện thực khác nhau trên diện rộng, khiến GraphQL trở thành một lựa chọn ít rủi ro hơn nhiều so với vài năm trước. Tuy nhiên, bạn có thể muốn lưu ý một vài thách thức còn lại với công nghệ này.

For one, the client device can issue dynamically changing queries, and I've heard of teams who have had issues with GraphQL queries causing significant load on the server side as a result of this capability. When we compare GraphQL with something like SQL, we see a similar issue. An expensive SQL statement can cause significant problems for a **database** and potentially have a large impact on the wider system. The same problem applies with GraphQL. The difference is that with SQL we at least have tools like query planners for our databases, which can help us diagnose problematic queries, whereas a similar problem with GraphQL can be harder to track down. Server-side throttling of requests is one potential solution, but as the execution of the call may be spread across multiple microservices, this is far from straightforward.

Thứ nhất, thiết bị client có thể phát các truy vấn thay đổi một cách động, và tôi từng nghe về các nhóm gặp vấn đề với việc các truy vấn GraphQL gây tải đáng kể lên phía server do khả năng này. Khi so sánh GraphQL với một thứ như SQL, ta thấy một vấn đề tương tự. Một câu lệnh SQL tốn kém có thể gây ra những vấn đề đáng kể cho một cơ sở dữ liệu và có khả năng tác động lớn lên hệ thống rộng hơn. Cùng một vấn đề đó áp dụng với GraphQL. Khác biệt là với SQL ta ít ra cũng có các công cụ như query planner cho cơ sở dữ liệu, giúp ta chẩn đoán các truy vấn có vấn đề, trong khi một vấn đề tương tự với GraphQL có thể khó truy vết hơn. Việc giới hạn (throttle) request ở phía server là một giải pháp khả dĩ, nhưng vì việc thực thi lời gọi có thể trải ra trên nhiều microservice, điều này còn xa mới đơn giản.

Compared with normal REST-based HTTP APIs, caching is also more complex. With REST-based APIs, I can set one of many response headers to help client-side devices, or intermediate caches like content delivery networks (CDNs), **cache** responses so they don't need to be requested again. This isn't possible in the same way with GraphQL. The advice I've seen on this issue seems to revolve

around just associating an ID with every returned resource (and remember, a GraphQL query could contain multiple resources) and then having the client device cache the request against that ID. As far as I can tell, this makes using CDNs or caching reverse proxies incredibly difficult without additional work.

So với các API HTTP dựa trên REST thông thường, caching cũng phức tạp hơn. Với các API dựa trên REST, tôi có thể đặt một trong nhiều header phản hồi để giúp các thiết bị phía client, hoặc các cache trung gian như mạng phân phối nội dung (CDN), cache các phản hồi để chúng không cần được yêu cầu lại. Điều này không thể làm theo cùng cách với GraphQL. Lời khuyên tôi từng thấy về vấn đề này có vẻ xoay quanh việc chỉ gắn một ID với mỗi tài nguyên trả về (và hãy nhớ, một truy vấn GraphQL có thể chứa nhiều tài nguyên) rồi cho thiết bị client cache request dựa trên ID đó. Theo tôi thấy, điều này khiến việc dùng CDN hay các reverse proxy caching trở nên cực kỳ khó khăn nếu không có thêm công sức.

Although I've seen some implementation-specific solutions to this problem (such as those found in the JavaScript Apollo implementation), caching feels like it was either consciously or unconsciously ignored as part of the initial development of GraphQL. If the queries you are issuing are highly specific in nature to a particular user, then this lack of request-level caching may not be a deal breaker, of course, as your cache-hit ratio is likely to be low. I do wonder, though, if this limitation means that you'll still end up with a hybrid solution for client devices, with some (more generic) requests going over normal REST-based HTTP APIs and other requests going over GraphQL.

Mặc dù tôi đã thấy một số giải pháp đặc thù theo hiện thực cho vấn đề này (như những giải pháp tìm thấy trong hiện thực Apollo của JavaScript), caching có cảm giác như đã bị bỏ qua một cách có ý thức hoặc vô thức như một phần của giai đoạn phát triển ban đầu của GraphQL. Nếu các truy vấn bạn phát ra có tính chất rất đặc thù với một người dùng cụ thể, thì việc thiếu caching ở mức request này có thể không phải là điều quyết định, tất nhiên, vì tỷ lệ cache-hit của bạn có khả năng sẽ thấp. Tuy nhiên, tôi tự hỏi liệu hạn chế này có nghĩa là bạn vẫn sẽ rốt cuộc có một giải pháp lai cho các thiết bị client, với một số request (chung chung hơn) đi qua các API HTTP dựa trên REST thông thường và các request khác đi qua GraphQL.

Another issue is that while GraphQL theoretically can handle writes, it doesn't seem to fit as well as for reads. This leads to situations in which teams are using GraphQL for read but REST for writes.

Một vấn đề khác là tuy GraphQL về lý thuyết có thể xử lý các thao tác ghi, nó có vẻ không phù hợp tốt như với các thao tác đọc. Điều này dẫn đến tình huống các nhóm dùng GraphQL cho đọc nhưng dùng REST cho ghi.

The last issue is something that may be entirely subjective, but I still think it's worth raising. GraphQL makes it feel like you are just working with data, which can reinforce the idea that the microservices you are talking to are just wrappers over databases. In fact, I've seen multiple people compare GraphQL to OData, a technology that is designed as a generic API for accessing data from databases. As we've already discussed at length, the idea of treating microservices just as wrappers over databases can be very problematic. Microservices expose functionality over networked interfaces. Some of that functionality might require or result in data being exposed, but they should still have their own internal logic and behavior. Just because you are using GraphQL, don't slip into thinking of your microservices as little more than an API on a database—it's essential that your GraphQL API isn't coupled to the underlying datastores of your microservices.

Vấn đề cuối cùng là thứ có thể hoàn toàn mang tính chủ quan, nhưng tôi vẫn cho rằng đáng nêu ra. GraphQL khiến ta có cảm giác như chỉ đang làm việc với dữ liệu, điều này có thể củng cố ý niệm rằng các microservice bạn đang trò chuyện cùng chỉ là những lớp bọc quanh các cơ sở dữ liệu. Thực tế, tôi đã thấy nhiều người so sánh GraphQL với OData,

một công nghệ được thiết kế như một API chung để truy cập dữ liệu từ cơ sở dữ liệu. Như ta đã bàn dài dòng, ý niệm coi microservice chỉ như những lớp bọc quanh cơ sở dữ liệu có thể rất rắc rối. Microservice phơi bày chức năng qua các interface kết nối mạng. Một số chức năng đó có thể đòi hỏi hoặc dẫn đến việc dữ liệu bị phơi bày, nhưng chúng vẫn nên có logic và hành vi nội bộ riêng. Chỉ vì bạn đang dùng GraphQL, đừng trượt vào lối nghĩ về các microservice của mình như chẳng hơn gì một API trên một cơ sở dữ liệu—điều thiết yếu là API GraphQL của bạn không bị coupling vào các kho dữ liệu nền tảng của các microservice.

Where to use it

Khi nào nên dùng

GraphQL's sweet spot is for use at the perimeter of the system, exposing functionality to external clients. These clients are typically GUIs, and it's an obvious fit for mobile devices given their constraints in terms of their limited ability to surface data to the end user and the nature of mobile networks. But GraphQL has also seen use for external APIs, GitHub being an early adopter of GraphQL. If you have an external API that often requires external clients to make multiple calls to get the information they need, then GraphQL can help make the API much more efficient and friendly.

Điểm mạnh nhất của GraphQL là dùng ở vành đai của hệ thống, phơi bày chức năng ra cho các client bên ngoài. Những client này thường là các GUI, và nó là một lựa chọn hiển nhiên cho các thiết bị di động, xét đến các hạn chế của chúng về khả năng giới hạn trong việc hiển thị dữ liệu cho người dùng cuối và bản chất của mạng di động. Nhưng GraphQL cũng đã được dùng cho các API bên ngoài, GitHub là một bên áp dụng GraphQL từ sớm. Nếu bạn có một API bên ngoài thường đòi hỏi các client bên ngoài phải thực hiện nhiều lời gọi để lấy thông tin họ cần, thì GraphQL có thể giúp làm cho API hiệu quả và thân thiện hơn nhiều.

Fundamentally, GraphQL is a call aggregation and filtering mechanism, so in the context of a microservice architecture it would be used to **aggregate** calls over multiple downstream microservices. As such, it's not something that would replace general microservice-to-microservice communication.

Về căn bản, GraphQL là một cơ chế gộp và lọc lời gọi, nên trong bối cảnh một kiến trúc microservice nó sẽ được dùng để gộp các lời gọi qua nhiều microservice phía dưới. Như vậy, nó không phải là thứ thay thế cho giao tiếp microservice-tới-microservice nói chung.

An alternative to the use of GraphQL would be to consider an alternative pattern like the **backend for frontend** (BFF) pattern—we'll look at that and compare it with GraphQL and other aggregation techniques in Chapter 14.

Một lựa chọn thay thế cho việc dùng GraphQL sẽ là xem xét một pattern thay thế như pattern backend for frontend (BFF)—ta sẽ xem xét nó và so sánh nó với GraphQL cùng các kỹ thuật gộp khác ở Chương 14.

Message Brokers — Message Broker

Message brokers are intermediaries, often called middleware, that sit between processes to manage communication between them. They are a popular choice to help implement asynchronous communication between microservices, as they offer a variety of powerful capabilities.

Message broker là các trung gian, thường được gọi là middleware, nằm giữa các tiến trình để quản lý giao tiếp giữa chúng. Chúng là một lựa chọn phổ biến để giúp hiện thực giao tiếp bất đồng bộ giữa các microservice, vì chúng cung cấp nhiều khả năng mạnh mẽ khác nhau.

As we discussed earlier, a message is a generic concept that defines the thing that a **message broker** sends. A message could contain a request, a response, or an event. Rather than one microservice directly communicating with another microservice, the microservice instead gives a message to a message broker, with information about how the message should be sent.

Như ta đã bàn trước đó, một message (thông điệp) là một khái niệm chung định nghĩa cái thứ mà message broker gửi đi. Một message có thể chứa một request, một response, hoặc một event (sự kiện). Thay vì một microservice giao tiếp trực tiếp với một microservice khác, microservice đó thay vào đó đưa một message cho message broker, kèm thông tin về cách message nên được gửi.

Topics and queues

Topic và queue

Brokers tend to provide either queues or topics, or both. Queues are typically point to point. A sender puts a message on a queue, and a consumer reads from that queue. With a topic-based system, multiple consumers are able to subscribe to a topic, and each subscribed consumer will receive a copy of that message.

Các broker thường cung cấp hoặc là queue, hoặc là topic, hoặc cả hai. Queue thường là điểm-tới-điểm (point to point). Một bên gửi đặt một message vào một queue, và một consumer đọc từ queue đó. Với một hệ thống dựa trên topic, nhiều consumer có thể đăng ký (subscribe) một topic, và mỗi consumer đã đăng ký sẽ nhận một bản sao của message đó.

A consumer could represent one or more microservices—typically modeled as a consumer group. This would be useful when you have multiple instances of a microservice, and you want any one of them to be able to receive a message. In Figure 5-1, we see an example in which the Order Processor has three deployed instances, all as part of the same consumer group. When a message is put into the queue, only one member of the consumer group will receive that message; this means the queue works as a load distribution mechanism. This is an example of the competing consumers pattern we touched on briefly in Chapter 4.

Một consumer có thể đại diện cho một hoặc nhiều microservice—thường được mô hình hóa như một consumer group. Điều này sẽ hữu ích khi bạn có nhiều instance của một microservice, và bạn muốn bất kỳ instance nào trong số đó cũng có thể nhận một message. Trong Hình 5-1, ta thấy một ví dụ trong đó Order Processor có ba instance được triển khai, tất cả đều là một phần của cùng một consumer group. Khi một message được đưa vào queue, chỉ một thành viên của consumer group sẽ nhận message đó; điều này nghĩa là queue hoạt động như một cơ chế phân phối tải. Đây là một ví dụ của pattern competing consumers (consumer cạnh tranh) mà ta đã chạm tới sơ qua ở Chương 4.

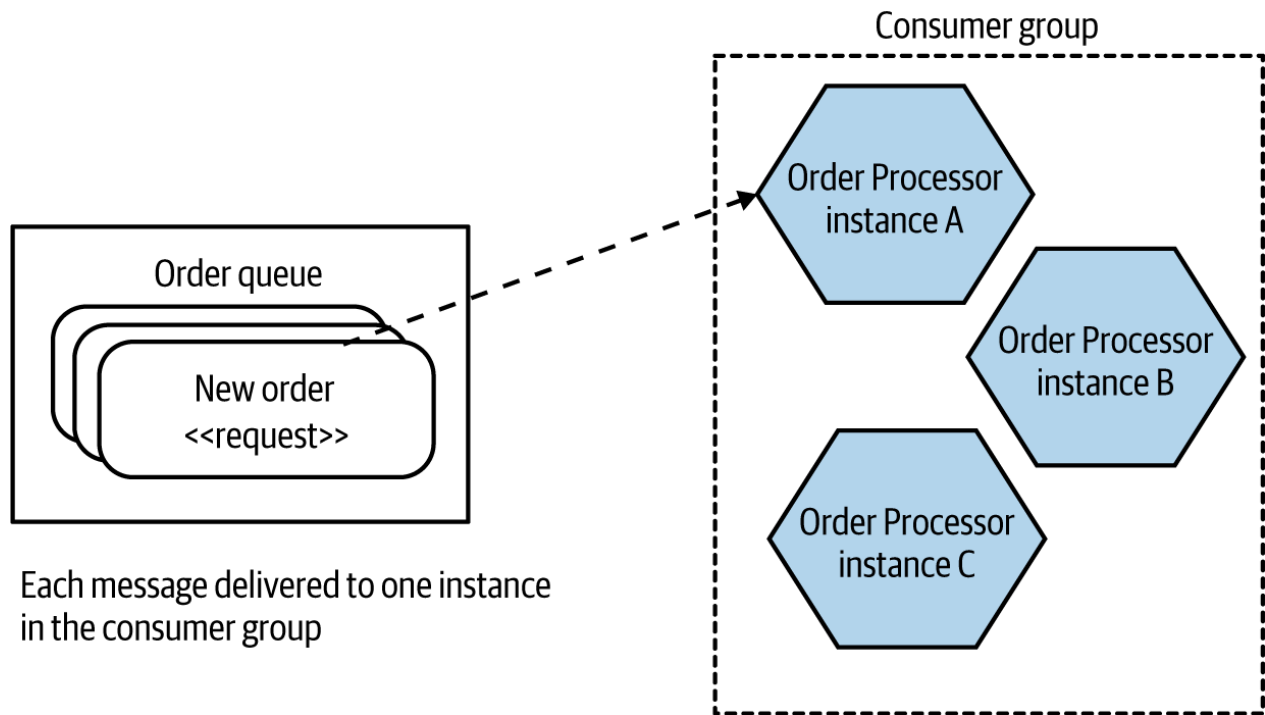


Figure 5-1. A queue allows for one consumer group

Hình 5-1. Một queue cho phép một consumer group

With topics, you can have multiple consumer groups. In Figure 5-2 , an event representing an order being paid for is put onto the Order Status topic. A copy of that event is received by both the Warehouse microservice and the Notifications microservice, which are in separate consumer groups. Only one instance of each consumer group will see that event.

Với topic, bạn có thể có nhiều consumer group. Trong Hình 5-2, một event biểu diễn việc một đơn hàng được thanh toán được đưa vào topic Order Status. Một bản sao của event đó được nhận bởi cả microservice Warehouse lẫn microservice Notifications, vốn thuộc các consumer group riêng biệt. Chỉ một instance của mỗi consumer group sẽ thấy event đó.

At first glance, a queue just looks like a topic with a single consumer group. A large part of the distinction between the two is that when a message is sent over a queue, there is knowledge of what the message is being sent to. With a topic, this information is hidden from the sender of the message—the sender is unaware of who (if anyone) will end up receiving the message.

Thoạt nhìn, một queue trông giống một topic chỉ với một consumer group. Một phần lớn của sự phân biệt giữa hai loại là khi một message được gửi qua một queue, có sự hiểu biết về việc message đang được gửi tới đâu. Với một topic, thông tin này bị ẩn khỏi bên gửi message—bên gửi không biết ai (nếu có) sẽ rốt cuộc nhận message.

Topics are a good fit for event-based collaboration, whereas queues would be more appropriate for request/response communication. This should be considered as general guidance rather than a strict rule, however.

Topic phù hợp tốt với cộng tác dựa trên event, trong khi queue sẽ phù hợp hơn với giao tiếp request/response. Tuy nhiên, điều này nên được coi là hướng dẫn chung chứ không phải một quy tắc cứng nhắc.

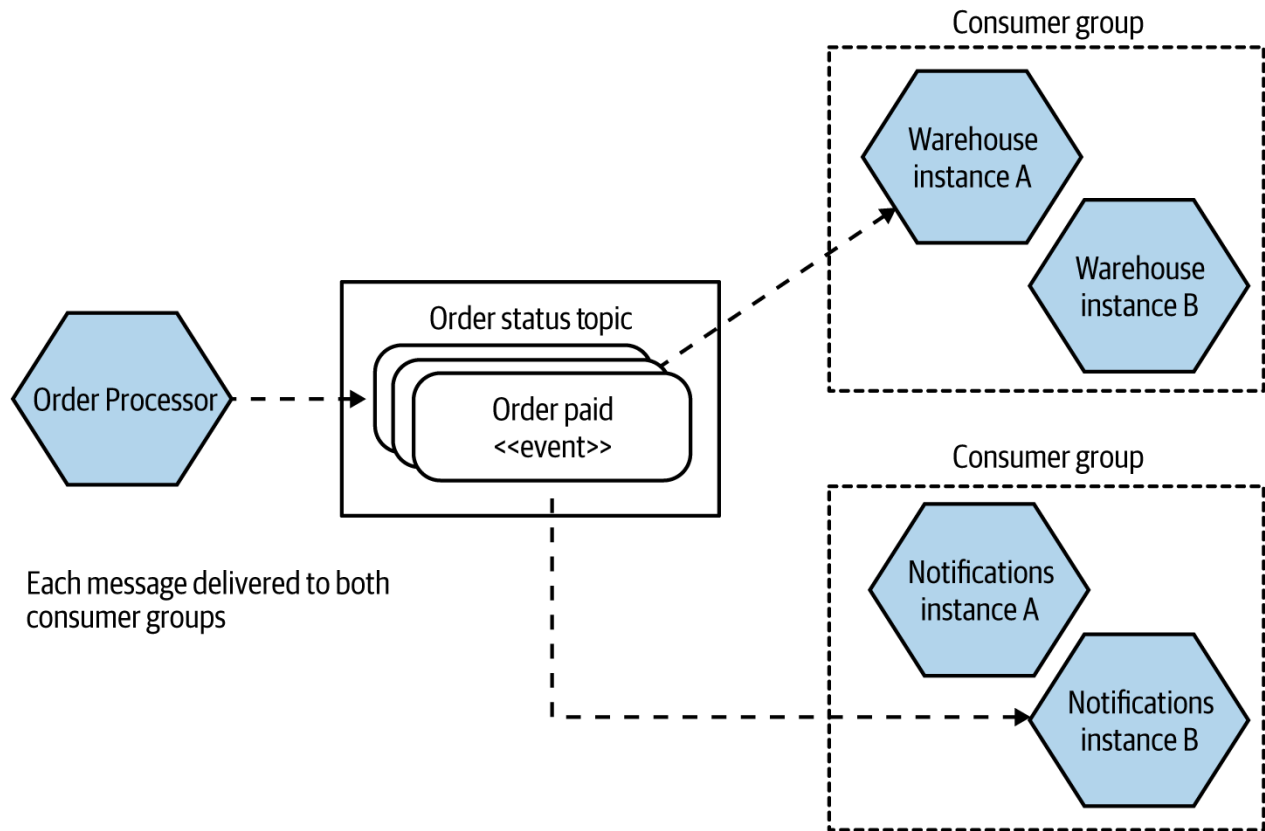


Figure 5-2. Topics allow for multiple subscribers to receive the same messages, which is useful for event broadcast

Hình 5-2. Topic cho phép nhiều bên đăng ký nhận cùng các message, điều hữu ích cho việc phát sóng (broadcast) event

Guaranteed delivery

Đảm bảo phân phối (Guaranteed delivery)

So why use a broker? Fundamentally, they provide some capabilities that can be very useful for asynchronous communication. The properties they provide vary, but the most interesting feature is that of guaranteed delivery, something that all widely used brokers support in some way. Guaranteed delivery describes a commitment by the broker to ensure that the message is delivered.

Vậy tại sao lại dùng một broker? Về căn bản, chúng cung cấp một số khả năng có thể rất hữu ích cho giao tiếp bất đồng bộ. Các đặc tính chúng cung cấp khác nhau, nhưng tính năng thú vị nhất là đảm bảo phân phối (guaranteed delivery), thứ mà tất cả các broker được dùng rộng rãi đều hỗ trợ theo một cách nào đó. Đảm bảo phân phối mô tả một cam kết của broker nhằm đảm bảo rằng message được phân phối.

From the point of view of the microservice sending the message, this can be very useful. It's not a problem if the downstream destination is unavailable—the broker will hold on to the message until it can be delivered. This can reduce the number of things an upstream microservice needs to worry about. Compare that to a synchronous direct call—for example, an HTTP request: if the downstream destination isn't reachable, the upstream microservice will need to work out what to do with the request; should it **retry** the call or give up?

Từ góc nhìn của microservice gửi message, điều này có thể rất hữu ích. Không thành vấn

đề nếu đích đến phía dưới không khả dụng—broker sẽ giữ lấy message cho đến khi nó có thể được phân phối. Điều này có thể giảm số thứ mà một microservice phía trên cần phải lo lắng. Hãy so sánh điều đó với một lời gọi trực tiếp đồng bộ—ví dụ, một request HTTP: nếu đích đến phía dưới không thể tiếp cận được, microservice phía trên sẽ cần tự tính xem phải làm gì với request; nó nên thử lại lời gọi hay từ bỏ?

For guaranteed delivery to work, a broker will need to ensure that any messages not yet delivered are going to be held in a durable fashion until they can be delivered. To deliver on this promise, a broker will normally run as some sort of cluster-based system, ensuring that the loss of a single machine doesn't cause a message to be lost. There is typically a lot involved in running a broker correctly, partly due to the challenges in managing cluster-based software. Often, the promise of guaranteed delivery can be undermined if the broker isn't set up correctly. As an example, RabbitMQ requires instances in a cluster to communicate over relatively low-latency networks; otherwise the instances can start to get confused about the current state of messages being handled, resulting in data loss. I'm not highlighting this particular limitation as a way of saying that RabbitMQ is in any way bad—all brokers have restrictions as to how they need to be run to deliver the promise of guaranteed delivery. If you plan to run your own broker, make sure you read the documentation carefully.

Để đảm bảo phân phối hoạt động, một broker sẽ cần đảm bảo rằng bất kỳ message nào chưa được phân phối sẽ được giữ ở dạng bền bỉ (durable) cho đến khi chúng có thể được phân phối. Để thực hiện được lời hứa này, một broker thường sẽ chạy như một dạng hệ thống dựa trên cụm (cluster), đảm bảo rằng việc mất một máy đơn lẻ không gây mất message. Thường có rất nhiều thứ liên quan đến việc vận hành một broker cho đúng, một phần do các thách thức trong việc quản lý phần mềm dựa trên cụm. Lời hứa đảm bảo phân phối thường có thể bị phá vỡ nếu broker không được thiết lập đúng. Lấy ví dụ, RabbitMQ đòi hỏi các instance trong một cụm phải giao tiếp qua các mạng có độ trễ tương đối thấp; nếu không các instance có thể bắt đầu bối rối về trạng thái hiện tại của các message đang được xử lý, dẫn đến mất dữ liệu. Tôi không làm nổi bật hạn chế cụ thể này như một cách để nói rằng RabbitMQ tệ ở bất cứ điểm nào—tất cả các broker đều có những hạn chế về cách chúng cần được vận hành để thực hiện lời hứa đảm bảo phân phối. Nếu bạn định tự vận hành broker của mình, hãy chắc chắn đọc tài liệu thật kỹ.

It's also worth noting that what any given broker means by guaranteed delivery can vary. Again, reading the documentation is a great start.

Cũng đáng lưu ý rằng điều mà một broker cho trước hiểu là đảm bảo phân phối có thể khác nhau. Một lần nữa, đọc tài liệu là một khởi đầu tuyệt vời.

Trust

Sự tin tưởng (Trust)

One of the big draws of a broker is the property of guaranteed delivery. But for this to work, you need to trust not only the people who created the broker but also the way that broker has operated. If you've built a system that is based on the assumption that delivery is guaranteed, and that turns out not to be the case due to an issue with the underlying broker, it can cause significant issues. The hope, of course, is that you are offloading that work to software created by people who can do that job better than you can. Ultimately, you have to decide how much you want to trust the broker you are making use of.

Một trong những điểm thu hút lớn của một broker là đặc tính đảm bảo phân phối. Nhưng để điều này hoạt động, bạn cần tin tưởng không chỉ những người đã tạo ra broker mà cả cách mà broker đó đã được vận hành. Nếu bạn đã xây dựng một hệ thống dựa trên giả định rằng việc phân phối được đảm bảo, và hóa ra điều đó lại không đúng do một vấn đề

với broker nền tảng, nó có thể gây ra những vấn đề đáng kể. Hy vọng, tất nhiên, là bạn đang giao việc đó cho phần mềm được tạo bởi những người có thể làm công việc đó tốt hơn bạn. Rốt cuộc, bạn phải quyết định bạn muốn tin tưởng đến mức nào vào broker mà bạn đang dùng.

Other characteristics

Các đặc tính khác

Besides guaranteed delivery, brokers can provide other characteristics that you may find useful.

Ngoài đảm bảo phân phối, các broker có thể cung cấp những đặc tính khác mà bạn có thể thấy hữu ích.

Most brokers can guarantee the order in which messages will be delivered, but this isn't universal, and even then the scope of this guarantee can be limited. With Kafka, for example, ordering is guaranteed only within a single partition. If you can't be certain that messages will be received in order, your consumer may need to compensate, perhaps by deferring processing of messages that are received out of order until any missing messages are received.

Hầu hết các broker có thể đảm bảo thứ tự mà các message sẽ được phân phối, nhưng điều này không phổ quát, và ngay cả khi vậy thì phạm vi của đảm bảo này có thể bị giới hạn. Với Kafka chẳng hạn, thứ tự chỉ được đảm bảo trong phạm vi một partition đơn lẻ. Nếu bạn không thể chắc chắn rằng các message sẽ được nhận đúng thứ tự, consumer của bạn có thể cần bù trừ, có lẽ bằng cách hoãn xử lý các message nhận được không đúng thứ tự cho đến khi bất kỳ message còn thiếu nào được nhận.

Some brokers provide transactions on write—for instance, Kafka allows you to write to multiple topics in a single **transaction**. Some brokers can also provide read transactionality, something I've taken advantage of when using a number of brokers via the Java Message Service (JMS) APIs. This can be useful if you want to ensure the message can be processed by the consumer before removing it from the broker.

Một số broker cung cấp giao dịch khi ghi—chẳng hạn, Kafka cho phép bạn ghi vào nhiều topic trong một giao dịch duy nhất. Một số broker cũng có thể cung cấp tính giao dịch khi đọc, điều mà tôi đã tận dụng khi dùng một số broker qua các API Java Message Service (JMS). Điều này có thể hữu ích nếu bạn muốn đảm bảo message có thể được consumer xử lý trước khi loại bỏ nó khỏi broker.

Another, somewhat controversial feature promised by some brokers is that of exactly once delivery. One of the easier ways to provide guaranteed delivery is allowing the message to be resent. This can result in a consumer seeing the same message more than once (even if this is a rare situation). Most brokers will do what they can to reduce the chance of this, or hide this fact from the consumer, but some brokers go further by guaranteeing exactly once delivery. This is a complex topic, as I've spoken to some experts who state that guaranteeing exactly once delivery in all cases is impossible, while other experts say you basically can do so with a few simple workarounds. Either way, if your broker of choice claims to implement this, then pay really careful attention to how it is implemented. Even better, build your consumers in such a way that they are prepared for the fact that they might receive a message more than once and can handle this situation. A very simple example would be for each message to have an ID, which a consumer can check each time a message is received. If a message with that ID has already been processed, the newer message can be ignored.

Một tính năng khác, có phần gây tranh cãi, được một số broker hứa hẹn là phân phối đúng một lần (exactly once delivery). Một trong những cách dễ hơn để cung cấp đảm bảo phân phối là cho phép gửi lại message. Điều này có thể dẫn đến việc một consumer thấy cùng

một message nhiều hơn một lần (ngay cả khi đây là một tình huống hiếm gặp). Hầu hết các broker sẽ làm những gì có thể để giảm khả năng này, hoặc che giấu sự thật này khỏi consumer, nhưng một số broker đi xa hơn bằng cách đảm bảo phân phối đúng một lần. Đây là một chủ đề phức tạp, vì tôi đã nói chuyện với một số chuyên gia khẳng định rằng việc đảm bảo phân phối đúng một lần trong mọi trường hợp là bất khả thi, trong khi các chuyên gia khác nói rằng về cơ bản bạn có thể làm điều đó với một vài cách giải quyết đơn giản. Dù sao đi nữa, nếu broker bạn chọn tuyên bố hiện thực điều này, thì hãy chú ý thật kỹ đến cách nó được hiện thực. Tốt hơn nữa, hãy xây dựng các consumer của bạn theo cách sẵn sàng cho thực tế rằng chúng có thể nhận một message nhiều hơn một lần và có thể xử lý tình huống này. Một ví dụ rất đơn giản sẽ là mỗi message có một ID, mà consumer có thể kiểm tra mỗi lần nhận một message. Nếu một message với ID đó đã được xử lý, thì message mới hơn có thể bị bỏ qua.

Choices

Các lựa chọn

A variety of message brokers exist. Popular examples include RabbitMQ, ActiveMQ, and Kafka (which we'll explore further shortly). The main public cloud vendors also provide a variety of products that play this role, from managed versions of those brokers that you can install on your own infrastructure to bespoke implementations that are specific to a given platform. AWS, for example, has Simple Queue Service (SQS), Simple Notification Service (SNS), and Kinesis, all of which provide different flavors of fully managed brokers. SQS was in fact the second-ever product released by AWS, having been launched back in 2006.

Có nhiều message broker khác nhau tồn tại. Các ví dụ phổ biến gồm RabbitMQ, ActiveMQ và Kafka (mà ta sẽ khám phá thêm sớm thôi). Các nhà cung cấp public cloud lớn cũng cung cấp nhiều sản phẩm đóng vai trò này, từ các phiên bản được quản lý của những broker đó mà bạn có thể cài đặt trên hạ tầng của riêng mình đến các hiện thực tùy biến đặc thù cho một nền tảng cho trước. AWS chẳng hạn, có Simple Queue Service (SQS), Simple Notification Service (SNS) và Kinesis, tất cả đều cung cấp các phiên bản broker được quản lý hoàn toàn ở những hương vị khác nhau. SQS thực ra là sản phẩm thứ hai từng được AWS phát hành, ra mắt từ tận năm 2006.

Kafka

Kafka

Kafka is worth highlighting as a specific broker, due in large part to its recent popularity. Part of that popularity is due to Kafka's use in helping move large volumes of data around as part of implementing stream processing pipelines. This can help move from batch-oriented processing to more real-time processing.

Kafka đáng được làm nổi bật như một broker cụ thể, phần lớn nhờ vào sự phổ biến gần đây của nó. Một phần của sự phổ biến đó là nhờ việc Kafka được dùng để giúp di chuyển khối lượng dữ liệu lớn như một phần của việc hiện thực các pipeline xử lý luồng (stream processing). Điều này có thể giúp chuyển từ xử lý theo lô (batch) sang xử lý gần thời gian thực hơn.

There are a few characteristics of Kafka that are worth highlighting. Firstly, it is designed for very large scale—it was built at LinkedIn to replace multiple existing message clusters with a single platform. Kafka is built to allow for multiple consumers and producers—I've spoken to one expert at a large technology company who had over fifty thousand producers and consumers working on the same cluster. To be fair, very few organizations have problems at that level of scale, but for some organizations, the ability to scale Kafka easily (relatively speaking) can be very useful.

Có một vài đặc điểm của Kafka đáng được làm nổi bật. Thứ nhất, nó được thiết kế cho quy mô rất lớn—nó được xây dựng tại LinkedIn để thay thế nhiều cụm message hiện có bằng một nền tảng duy nhất. Kafka được xây dựng để cho phép nhiều consumer và producer—tôi đã nói chuyện với một chuyên gia tại một công ty công nghệ lớn có hơn năm mươi ngàn producer và consumer làm việc trên cùng một cụm. Công bằng mà nói, rất ít tổ chức có vấn đề ở mức quy mô đó, nhưng với một số tổ chức, khả năng mở rộng Kafka dễ dàng (một cách tương đối) có thể rất hữu ích.

Another fairly unique feature of Kafka is message permanence. With a normal message broker, once the last consumer has received a message, the broker no longer needs to hold on to that message. With Kafka, messages can be stored for a configurable period. This means that messages can be stored forever. This can allow consumers to reingest messages that they had already processed, or allow newly deployed consumers to process messages that were sent previously.

Một tính năng khá độc đáo khác của Kafka là tính bền vững của message (message permanence). Với một message broker thông thường, một khi consumer cuối cùng đã nhận một message, broker không còn cần phải giữ message đó. Với Kafka, message có thể được lưu trữ trong một khoảng thời gian cấu hình được. Điều này nghĩa là message có thể được lưu trữ mãi mãi. Điều này có thể cho phép consumer nạp lại (reingest) các message mà chúng đã xử lý, hoặc cho phép các consumer mới được triển khai xử lý các message đã được gửi trước đó.

Finally, Kafka has been rolling out built-in support for stream processing. Rather than using Kafka to send messages to a dedicated stream processing tool like Apache Flink, some tasks can instead be done inside Kafka itself. Using KSQL, you can define SQL-like statements that can process one or more topics on the fly. This can give you something akin to a dynamically updating materialized database view, with the source of data being Kafka topics rather than a database. These capabilities open up some very interesting possibilities for how data is managed in distributed systems. If you'd like to explore these ideas in more detail, I can recommend *Designing Event-Driven Systems* (O'Reilly) by Ben Stopford. (I have to recommend Ben's book, as I wrote the foreword for it!) For a deeper dive into Kafka in general, I'd suggest *Kafka: The Definitive Guide* (O'Reilly) by Neha Narkhede, Gwen Shapira, and Todd Palino.

Cuối cùng, Kafka đã và đang triển khai hỗ trợ tích hợp sẵn cho xử lý luồng. Thay vì dùng Kafka để gửi message tới một công cụ xử lý luồng chuyên dụng như Apache Flink, một số tác vụ có thể được làm ngay bên trong chính Kafka. Dùng KSQL, bạn có thể định nghĩa các câu lệnh kiểu SQL có thể xử lý một hoặc nhiều topic ngay tại chỗ. Điều này có thể cho bạn một thứ gì đó tương tự một materialized view của cơ sở dữ liệu được cập nhật động, với nguồn dữ liệu là các topic Kafka thay vì một cơ sở dữ liệu. Những khả năng này mở ra một số khả năng rất thú vị cho cách dữ liệu được quản lý trong các hệ thống phân tán. Nếu bạn muốn khám phá các ý tưởng này chi tiết hơn, tôi có thể giới thiệu cuốn *Designing Event-Driven Systems* (O'Reilly) của Ben Stopford. (Tôi buộc phải giới thiệu cuốn sách của Ben, vì tôi đã viết lời tựa cho nó!) Để đi sâu hơn vào Kafka nói chung, tôi sẽ đề xuất cuốn *Kafka: The Definitive Guide* (O'Reilly) của Neha Narkhede, Gwen Shapira và Todd Palino.

Serialization Formats — Các định dạng tuần tự hóa

Some of the technology choices we've looked at—specifically, some of the RPC implementations—make choices for you regarding how data is serialized and deserialized. With gRPC, for example, any data sent will be converted into protocol buffer format. Many of the technology options, though, give us a lot of freedom in terms of how we covert data for network calls. Pick Kafka as your broker of

choice, and you can send messages in a variety of formats. So which format should you choose?

Một số lựa chọn công nghệ ta đã xem xét—cụ thể là một số hiện thực RPC—đưa ra các lựa chọn thay cho bạn về cách dữ liệu được tuần tự hóa và giải tuần tự hóa. Với gRPC chẳng hạn, bất kỳ dữ liệu nào được gửi sẽ được chuyển thành định dạng protocol buffer. Tuy nhiên, nhiều lựa chọn công nghệ cho ta rất nhiều tự do về cách ta chuyển đổi dữ liệu cho các lời gọi mạng. Chọn Kafka làm broker của bạn, và bạn có thể gửi message ở nhiều định dạng khác nhau. Vậy bạn nên chọn định dạng nào?

Textual Formats — Các định dạng văn bản

The use of standard textual formats gives clients a lot of flexibility in how they consume resources. REST APIs most often use a textual format for the request and response bodies, even if theoretically you can quite happily send binary data over HTTP. In fact, this is how gRPC works—using HTTP underneath but sending binary protocol buffers.

Việc dùng các định dạng văn bản chuẩn cho client rất nhiều sự linh hoạt trong cách họ tiêu thụ tài nguyên. Các API REST thường dùng một định dạng văn bản cho phần thân request và response, mặc dù về lý thuyết bạn hoàn toàn có thể vui vẻ gửi dữ liệu nhị phân qua HTTP. Thực tế, đây là cách gRPC hoạt động—dùng HTTP bên dưới nhưng gửi protocol buffer nhị phân.

JSON has usurped XML as the text serialization format of choice. You can point to a number of reasons why this has occurred, but the primary reason is that one of the main consumers of APIs is often a browser, where JSON is a great fit. JSON became popular partly as a result of the backlash against XML, and proponents cite its relative compactness and simplicity when compared to XML as another winning factor. The reality is that there's rarely a massive differential between the size of a JSON payload and that of an XML payload, especially as these payloads are typically compressed. It's also worth pointing out that some of the simplicity of JSON comes at a cost—in our rush to adopt simpler protocols, schemas went out of the window (more on that later).

JSON đã soán ngôi XML để trở thành định dạng tuần tự hóa văn bản được ưa chuộng. Bạn có thể chỉ ra một số lý do vì sao điều này xảy ra, nhưng lý do chính là một trong những bên tiêu thụ API chính thường là một trình duyệt, nơi JSON là một lựa chọn rất phù hợp. JSON trở nên phổ biến một phần là kết quả của làn sóng phản ứng chống lại XML, và những người ủng hộ nó viện dẫn tính tương đối gọn gàng và đơn giản của nó so với XML như một yếu tố thắng cuộc khác. Thực tế là hiếm khi có một sự chênh lệch lớn giữa kích thước của một payload JSON và một payload XML, đặc biệt khi các payload này thường được nén. Cũng đáng chỉ ra rằng một phần sự đơn giản của JSON phải trả giá—trong cơn vội vã áp dụng các giao thức đơn giản hơn của chúng ta, schema đã bị quăng ra ngoài cửa sổ (sẽ bàn thêm về điều này sau).

Avro is an interesting serialization format. It takes JSON as an underlying structure and uses it to define a schema-based format. Avro has found a lot of popularity as a format for message payloads, partly due to its ability to send the schema as part of the payload, which can make supporting multiple different messaging formats much easier.

Avro là một định dạng tuần tự hóa thú vị. Nó lấy JSON làm cấu trúc nền tảng và dùng nó để định nghĩa một định dạng dựa trên schema. Avro đã tìm thấy nhiều sự phổ biến như một định dạng cho payload message, một phần nhờ khả năng gửi schema như một phần của payload, điều có thể khiến việc hỗ trợ nhiều định dạng messaging khác nhau dễ dàng hơn nhiều.

Personally, though, I am still a fan of XML. Some of the tool support is better. For example, if I want to extract only certain parts of the payload (a technique we'll discuss more in "Handling Change Between Microservices" on page 144), I can use XPATH, which is a well-understood standard with lots of tool support, or even CSS selectors, which many find even easier. With JSON, I have JSONPath, but this is not as widely supported. I find it odd that people pick JSON because it is nice and lightweight but then try to push concepts into it like hypermedia controls that already exist in XML. I accept, though, that I am probably in the minority here and that JSON is the format of choice for many people!

Tuy nhiên, về phần cá nhân, tôi vẫn là một người hâm mộ XML. Một số hỗ trợ công cụ thì tốt hơn. Ví dụ, nếu tôi muốn chỉ trích xuất một số phần nhất định của payload (một kỹ thuật ta sẽ bàn thêm trong "Xử lý thay đổi giữa các Microservice" ở trang 144), tôi có thể dùng XPATH, vốn là một tiêu chuẩn được hiểu rõ với nhiều hỗ trợ công cụ, hoặc thậm chí CSS selector, thứ mà nhiều người thấy còn dễ hơn. Với JSON, tôi có JSONPath, nhưng nó không được hỗ trợ rộng rãi bằng. Tôi thấy lạ khi người ta chọn JSON vì nó hay ho và nhẹ nhàng nhưng rồi lại cố nhồi các khái niệm vào nó như các kiểm soát hypermedia vốn đã tồn tại sẵn trong XML. Tuy nhiên, tôi chấp nhận rằng tôi có lẽ thuộc thiểu số ở đây và rằng JSON là định dạng được nhiều người ưa chuộng!

Binary Formats — Các định dạng nhị phân

While textual formats have benefits such as making it easy for humans to read them and provide a lot of interoperability with different tools and technologies, the world of binary serialization protocols is where you want to be if you start getting worried about payload size or about the efficiencies of writing and reading the payloads. Protocol buffers have been around for a while and are often used outside the scope of gRPC—they probably represent the most popular binary serialization format for microservice-based communication.

Trong khi các định dạng văn bản có những lợi ích như giúp con người dễ đọc và cung cấp nhiều khả năng tương tác với các công cụ và công nghệ khác nhau, thì thế giới của các giao thức tuần tự hóa nhị phân mới là nơi bạn muốn đến nếu bạn bắt đầu lo lắng về kích thước payload hoặc về hiệu quả của việc viết và đọc các payload. Protocol buffer đã xuất hiện được một thời gian và thường được dùng bên ngoài phạm vi của gRPC—chúng có lẽ đại diện cho định dạng tuần tự hóa nhị phân phổ biến nhất cho giao tiếp dựa trên microservice.

This space is large, however, and a number of other formats have been developed with a variety of requirements in mind. Simple Binary Encoding, Cap'n Proto, and FlatBuffers all come to mind. Although benchmarks abound for each of these formats, highlighting their relevant benefits compared to protocol buffers, JSON, or other formats, benchmarks suffer from a fundamental problem in that they may not necessarily represent how you are going to use them. If you're looking to eke the last few bytes out of your serialization format, or to shave microseconds off the time taken to read or write these payloads, I strongly suggest you carry out your own comparison of these various formats. In my experience, the vast majority of systems rarely have to worry about such optimizations, as they can often achieve the improvements they are looking for by sending less data or by not making the call at all. However, if you are building an ultra-low-latency **distributed system**, make sure you're prepared to dive headfirst into the world of binary serialization formats.

Tuy nhiên, không gian này rất rộng, và một số định dạng khác đã được phát triển với nhiều yêu cầu khác nhau trong tâm trí. Simple Binary Encoding, Cap'n Proto và FlatBuffers đều xuất hiện trong đầu. Mặc dù có rất nhiều benchmark cho mỗi định dạng này, làm nổi bật

các lợi ích tương ứng của chúng so với protocol buffer, JSON hay các định dạng khác, các benchmark có một vấn đề căn bản là chúng có thể không nhất thiết đại diện cho cách bạn sẽ dùng chúng. Nếu bạn đang tìm cách vắt kiệt vài byte cuối cùng ra khỏi định dạng tuần tự hóa của mình, hoặc cạo bớt vài micro giây khỏi thời gian cần để đọc hoặc viết các payload này, tôi mạnh mẽ đề nghị bạn tự thực hiện so sánh của riêng mình giữa các định dạng khác nhau này. Theo kinh nghiệm của tôi, đại đa số các hệ thống hiếm khi phải lo lắng về những tối ưu như vậy, vì chúng thường có thể đạt được những cải thiện mong muốn bằng cách gửi ít dữ liệu hơn hoặc bằng cách không thực hiện lời gọi nào cả. Tuy nhiên, nếu bạn đang xây dựng một hệ thống phân tán độ trễ cực thấp, hãy chắc chắn rằng bạn sẵn sàng lao đầu vào thế giới của các định dạng tuần tự hóa nhị phân.

Schemas — Schema

A discussion that comes up time and again is whether we should use schemas to define what our endpoints expose and what they accept. Schemas can come in lots of different types, and picking a serialization format will typically define which schema technology you can use. If you're working with raw XML, you'd use XML Schema Definition (XSD); if you're working with raw JSON, you'd use JSON Schema. Some of the technology choices we've touched on (specifically, a sizable subset of the RPC options) require the use of explicit schemas, so if you picked those technologies you'd have to make use of schemas. SOAP works through the use of a WSDL, while gRPC requires the use of a protocol buffer specification. Other technology choices we've explored make the use of schemas optional, and this is where things get more interesting.

Một cuộc thảo luận cứ liên tục nổi lên là liệu ta có nên dùng schema để định nghĩa các endpoint của ta phơi bày cái gì và chấp nhận cái gì hay không. Schema có thể có nhiều loại khác nhau, và việc chọn một định dạng tuần tự hóa thường sẽ xác định công nghệ schema nào bạn có thể dùng. Nếu bạn làm việc với XML thô, bạn sẽ dùng XML Schema Definition (XSD); nếu bạn làm việc với JSON thô, bạn sẽ dùng JSON Schema. Một số lựa chọn công nghệ ta đã chạm tới (cụ thể là một tập con đáng kể các lựa chọn RPC) đòi hỏi việc dùng schema tường minh, nên nếu bạn chọn các công nghệ đó thì bạn sẽ phải dùng schema. SOAP hoạt động thông qua việc dùng một WSDL, trong khi gRPC đòi hỏi việc dùng một đặc tả protocol buffer. Các lựa chọn công nghệ khác ta đã khám phá thì để việc dùng schema là tùy chọn, và đây là nơi mọi thứ trở nên thú vị hơn.

As I've already discussed, I am in favor of having explicit schemas for microservice endpoints, for two key reasons. Firstly, they go a long way toward being an explicit representation of what a microservice endpoint exposes and what it can accept. This makes life easier both for developers working on the microservice and for their consumers. Schemas may not replace the need for good documentation, but they certainly can help reduce the amount of documentation required.

Như tôi đã bàn, tôi ủng hộ việc có schema tường minh cho các endpoint microservice, vì hai lý do chính. Thứ nhất, chúng đi một chặng đường dài hướng tới việc là một biểu diễn tường minh của những gì một endpoint microservice phơi bày và những gì nó có thể chấp nhận. Điều này làm cho cuộc sống dễ dàng hơn cho cả các lập trình viên làm việc trên microservice lẫn cho các consumer của họ. Schema có thể không thay thế được nhu cầu có tài liệu tốt, nhưng chúng chắc chắn có thể giúp giảm lượng tài liệu cần thiết.

The other reason I like explicit schemas, though, is how they help in terms of catching accidental breakages of microservice endpoints. We'll explore how to handle changes between microservices in a moment, but it's first worth exploring the different types of breakages and the role schemas can play.

Tuy nhiên, lý do khác mà tôi thích schema tường minh là cách chúng giúp ích trong việc bắt được các trường hợp phá vỡ vô tình các endpoint microservice. Ta sẽ khám phá cách xử lý các thay đổi giữa các microservice trong giây lát, nhưng trước tiên đáng để khám phá các loại phá vỡ khác nhau và vai trò mà schema có thể đóng.

Structural Versus Semantic Contract Breakages — Phá vỡ hợp đồng kiểu cấu trúc và kiểu ngữ nghĩa

Broadly speaking, we can break contract breakages down into two categories— structural breakages and semantic breakages. A structural breakage is a situation in which the structure of the endpoint changes in such a way that a consumer is now incompatible—this could represent fields or methods being removed, or new required fields being added. A semantic breakage refers to a situation in which the structure of the microservices endpoint remains the same but the behavior changes in such a way as to break consumers' expectations.

Nói rộng ra, ta có thể chia các trường hợp phá vỡ hợp đồng thành hai loại—phá vỡ cấu trúc (structural breakage) và phá vỡ ngữ nghĩa (semantic breakage). Một phá vỡ cấu trúc là tình huống trong đó cấu trúc của endpoint thay đổi theo cách khiến một consumer giờ không còn tương thích—điều này có thể biểu diễn việc các trường hoặc phương thức bị loại bỏ, hoặc các trường bắt buộc mới được thêm vào. Một phá vỡ ngữ nghĩa chỉ tình huống trong đó cấu trúc của endpoint microservice vẫn giữ nguyên nhưng hành vi thay đổi theo cách phá vỡ kỳ vọng của consumer.

Let's take a simple example. You have a highly complex Hard Calculations microservice that exposes a calculate method on its endpoint. This calculate method takes two integers, both of which are required fields. If you changed Hard Calculations such that the calculate method now takes only one integer, then consumers would break—they'd be sending requests with two integers that the Hard Calculations microservice would reject. This is an example of a structural change, and in general such changes can be easier to spot.

Hãy lấy một ví dụ đơn giản. Bạn có một microservice Hard Calculations cực kỳ phức tạp phơi bày một phương thức calculate trên endpoint của nó. Phương thức calculate này nhận hai số nguyên, cả hai đều là trường bắt buộc. Nếu bạn thay đổi Hard Calculations sao cho phương thức calculate giờ chỉ nhận một số nguyên, thì consumer sẽ bị phá vỡ—chúng sẽ gửi các request với hai số nguyên mà microservice Hard Calculations sẽ từ chối. Đây là một ví dụ của một thay đổi cấu trúc, và nhìn chung các thay đổi như vậy có thể dễ phát hiện hơn.

A semantic changes is more problematic. This is where the structure of the endpoint doesn't change but the behavior of the endpoint does. Coming back to our calculate method, imagine that in the first version, the two provided integers are added together and the results returned. So far, so good. Now we change Hard Calculations so that the calculate method multiplies the integers together and returns the result. The semantics of the calculate method have changed in a way that could break expectations of the consumers.

Một thay đổi ngữ nghĩa thì rắc rối hơn. Đây là khi cấu trúc của endpoint không thay đổi nhưng hành vi của endpoint thì có. Quay lại phương thức calculate của ta, hãy hình dung rằng ở phiên bản đầu tiên, hai số nguyên được cung cấp được cộng lại với nhau và kết quả được trả về. Cho tới đây thì ổn. Giờ ta thay đổi Hard Calculations sao cho phương thức calculate nhân các số nguyên lại với nhau và trả về kết quả. Ngữ nghĩa của phương thức calculate đã thay đổi theo cách có thể phá vỡ kỳ vọng của consumer.

Should You Use Schemas? — Bạn có nên dùng Schema không?

By using schemas and comparing different versions of schemas, we can catch structural breakages. Catching semantic breakages requires the use of **testing**. If you don't have schemas, or if you have schemas but decide not to compare schema changes for compatibility, then the burden of catching structural breakages before you get to production also falls on testing. Arguably, the situation is somewhat analogous with static versus dynamic typing in programming languages. With a statically typed language, the types are fixed at compile time—if your code does something with an instance of a type that isn't allowed (like calling a method that doesn't exist), then the compiler can catch that mistake. This can leave you to focus testing efforts on other sorts of problems. With a dynamically typed language, though, some of your testing will need to catch mistakes that a compiler picks up for statically typed languages.

Bằng cách dùng schema và so sánh các phiên bản schema khác nhau, ta có thể bắt được các phá vỡ cấu trúc. Bắt được các phá vỡ ngữ nghĩa đòi hỏi việc dùng kiểm thử. Nếu bạn không có schema, hoặc nếu bạn có schema nhưng quyết định không so sánh các thay đổi schema để kiểm tra tính tương thích, thì gánh nặng của việc bắt các phá vỡ cấu trúc trước khi lên production cũng đổ lên vai kiểm thử. Có thể lập luận rằng tình huống này có phần tương tự với kiểu tĩnh (static typing) so với kiểu động (dynamic typing) trong các ngôn ngữ lập trình. Với một ngôn ngữ kiểu tĩnh, các kiểu được cố định tại thời điểm biên dịch—nếu mã của bạn làm điều gì đó với một thể hiện của một kiểu mà không được phép (như gọi một phương thức không tồn tại), thì trình biên dịch có thể bắt được lỗi đó. Điều này có thể giúp bạn tập trung nỗ lực kiểm thử vào các loại vấn đề khác. Tuy nhiên, với một ngôn ngữ kiểu động, một phần kiểm thử của bạn sẽ cần bắt các lỗi mà trình biên dịch bắt được cho các ngôn ngữ kiểu tĩnh.

Now, I'm pretty relaxed about static versus dynamically typed languages, and I've found myself to be very productive (relatively speaking) in both. Certainly, dynamically typed languages give you some significant benefits that for many people justify giving up on compile-time safety. Personally speaking, though, if we bring the discussion back to microservice interactions, I haven't found that a similarly balanced trade-off exists when it comes to schema versus “schemaless” communication. Put simply, I think that having an explicit schema more than offsets any perceived benefit of having schemaless communication.

Giờ thì, tôi khá thoải mái về ngôn ngữ kiểu tĩnh so với kiểu động, và tôi thấy mình làm việc rất năng suất (một cách tương đối) ở cả hai. Chắc chắn, các ngôn ngữ kiểu động cho bạn một số lợi ích đáng kể mà với nhiều người là xứng đáng để từ bỏ sự an toàn ở thời điểm biên dịch. Tuy nhiên, về phần cá nhân, nếu ta đưa cuộc thảo luận trở lại các tương tác microservice, tôi không thấy có một sự đánh đổi cân bằng tương tự khi nói đến giao tiếp có schema so với “không schema” (schemaless). Nói đơn giản, tôi nghĩ rằng việc có một schema tường minh bù đắp dư thừa cho bất kỳ lợi ích nào được cho là của việc giao tiếp không schema.

Really, the question isn't actually whether you have a schema or not—it's whether or not that schema is explicit. If you are consuming data from a schemaless API, you still have expectations as to what data should be in there and how that data should be structured. Your code that will handle the data will be written with a set of assumptions in mind as to how that data is structured. In such a case, I'd argue that you do have a schema, but it's just totally implicit rather than explicit. 1 A lot of my desire for an explicit schema is driven by the fact that I think it's important to be as explicit as possible about what a microservice does (or doesn't) expose.

Thực ra, câu hỏi không thực sự là liệu bạn có schema hay không—mà là liệu schema đó có tường minh hay không. Nếu bạn đang tiêu thụ dữ liệu từ một API không schema, bạn

vẫn có những kỳ vọng về việc dữ liệu nào nên có trong đó và dữ liệu đó nên được cấu trúc như thế nào. Mã của bạn sẽ xử lý dữ liệu sẽ được viết với một tập giả định trong đầu về cách dữ liệu đó được cấu trúc. Trong trường hợp như vậy, tôi sẽ lập luận rằng bạn thực sự có một schema, nhưng nó chỉ là hoàn toàn ngầm định chứ không tường minh. 1 Phần lớn mong muốn của tôi về một schema tường minh được thúc đẩy bởi việc tôi nghĩ rằng quan trọng là phải tường minh nhất có thể về việc một microservice làm gì (hoặc không) phơi bày.

The main argument for schemaless endpoints seems to be that schemas need more work and don't give enough value. This, in my humble opinion, is partly a failure of imagination and partly a failure of good tooling to help schemas have more value when it comes to using them to catch structural breakages.

Lập luận chính cho các endpoint không schema có vẻ là schema cần nhiều công sức hơn và không mang lại đủ giá trị. Theo ý kiến khiêm tốn của tôi, điều này một phần là một sự thất bại của trí tưởng tượng và một phần là sự thất bại của các công cụ tốt trong việc giúp schema có nhiều giá trị hơn khi nói đến việc dùng chúng để bắt các phá vỡ cấu trúc.

Ultimately, a lot of what schemas provide is an explicit representation of part of the structure contract between a client and a server. They help make things explicit and can greatly aid communication between teams as well as work as a safety net. In situations in which the cost of change is reduced—for example, when both client and server are owned by the same team—I am more relaxed about not having schemas.

Rốt cuộc, phần lớn những gì schema cung cấp là một biểu diễn tường minh của một phần hợp đồng cấu trúc giữa một client và một server. Chúng giúp làm cho mọi thứ tường minh và có thể hỗ trợ rất nhiều cho giao tiếp giữa các nhóm cũng như hoạt động như một tấm lưới an toàn. Trong những tình huống mà chi phí của sự thay đổi được giảm bớt—ví dụ, khi cả client và server đều thuộc cùng một nhóm—tôi thoải mái hơn về việc không có schema.

1 Martin Fowler explores this in more detail in the context of schemaless data storage .

1 Martin Fowler khám phá điều này chi tiết hơn trong bối cảnh lưu trữ dữ liệu không schema.

Handling Change Between Microservices — Xử lý thay đổi giữa các Microservice

Probably the most common question I get about microservices, after “How big should they be?” is “How do you handle versioning?” When this question gets asked, it's rarely a query regarding what sort of numbering scheme you should use and more about how you handle changes in the contracts between microservices.

Có lẽ câu hỏi phổ biến nhất tôi nhận được về microservice, sau câu “Chúng nên to cỡ nào?”, là “Bạn xử lý việc đánh phiên bản (versioning) như thế nào?” Khi câu hỏi này được đặt ra, hiếm khi nó là một thắc mắc về loại sơ đồ đánh số bạn nên dùng mà nhiều hơn là về cách bạn xử lý các thay đổi trong hợp đồng giữa các microservice.

How you handle change really breaks down into two topics. In a moment, we'll look at what happens if you need to make a breaking change. But before that, let's look at what you can do to avoid making a breaking change in the first place.

Cách bạn xử lý thay đổi thực sự chia ra thành hai chủ đề. Trong giây lát, ta sẽ xem điều gì xảy ra nếu bạn cần thực hiện một thay đổi gây phá vỡ (breaking change). Nhưng trước đó, hãy xem bạn có thể làm gì để tránh thực hiện một thay đổi gây phá vỡ ngay từ đầu.

Avoiding Breaking Changes — Tránh các thay đổi gây phá vỡ

If you want to avoid making breaking changes, there are a few key ideas worth exploring, many of which we already touched on at the start of the chapter. If you can put these ideas into practice, you'll find it much easier to allow for microservices to be changed independently from one another.

Nếu bạn muốn tránh thực hiện các thay đổi gây phá vỡ, có một vài ý tưởng chủ chốt đáng để khám phá, nhiều trong số đó ta đã chạm tới ở đầu chương. Nếu bạn có thể đưa những ý tưởng này vào thực hành, bạn sẽ thấy việc cho phép các microservice được thay đổi độc lập với nhau dễ dàng hơn nhiều.

Expansion changes Add new things to a microservice interface; don't remove old things.

Các thay đổi mở rộng (Expansion changes) Thêm những thứ mới vào interface microservice; đừng loại bỏ những thứ cũ.

Tolerant reader When consuming a microservice interface, be flexible in what you expect.

Tolerant reader (bên đọc khoan dung) Khi tiêu thụ một interface microservice, hãy linh hoạt trong những gì bạn kỳ vọng.

Right technology Pick technology that makes it easier to make backward-compatible changes to the interface.

Công nghệ phù hợp (Right technology) Hãy chọn công nghệ giúp việc thực hiện các thay đổi tương thích ngược đối với interface dễ dàng hơn.

Explicit interface Be explicit about what a microservice exposes. This makes things easier for the client and easier for the maintainers of the microservice to understand what can be changed freely.

Interface tường minh (Explicit interface) Hãy tường minh về việc một microservice phơi bày cái gì. Điều này làm cho mọi thứ dễ dàng hơn cho client và dễ dàng hơn cho những người bảo trì microservice trong việc hiểu cái gì có thể được thay đổi một cách tự do.

Catch accidental breaking changes early Have mechanisms in place to catch interface changes that will break consumers in production before those changes are deployed.

Bắt sớm các thay đổi gây phá vỡ vô tình (Catch accidental breaking changes early) Hãy có sẵn các cơ chế để bắt các thay đổi interface sẽ phá vỡ consumer trên production trước khi những thay đổi đó được triển khai.

These ideas do reinforce each other, and many build on that key concept of **information hiding** that we've discussed frequently. Let's look at each idea in turn.

Những ý tưởng này quả thực củng cố lẫn nhau, và nhiều ý tưởng xây dựng dựa trên khái niệm chủ chốt về che giấu thông tin (information hiding) mà ta đã bàn thường xuyên. Hãy xem từng ý tưởng một.

Expansion Changes — Các thay đổi mở rộng

Probably the easiest place to start is by adding only new things to a microservice contract and not removing anything else. Consider the example of adding a new field to a payload—assuming the client is in some way tolerant of such changes, this shouldn't have a material impact. Adding a new `dateOfBirth` field to a customer record should be fine, for example.

Có lẽ nơi dễ nhất để bắt đầu là chỉ thêm những thứ mới vào hợp đồng microservice và không loại bỏ bất cứ thứ gì khác. Hãy xem xét ví dụ về việc thêm một trường mới vào một payload—giả sử client theo cách nào đó khoan dung với những thay đổi như vậy, thì điều này không nên có tác động đáng kể. Việc thêm một trường `dateOfBirth` mới vào một bản ghi customer chẳng hạn, sẽ không sao.

Tolerant Reader — Tolerant Reader

How the consumer of a microservice is implemented can bring a lot to bear on making backward-compatible changes easy. Specifically, we want to avoid client code binding too tightly to the interface of a microservice. Let's consider an Email microservice whose job it is to send out emails to our customers from time to time. It gets asked to send an “order shipped” email to a customer with the ID 1234; it goes off and retrieves the customer with that ID and gets back something like the response shown in Example 5-3.

Cách consumer của một microservice được hiện thực có thể ảnh hưởng rất nhiều đến việc làm cho các thay đổi tương thích ngược trở nên dễ dàng. Cụ thể, ta muốn tránh việc mã client bị gắn quá chặt vào interface của một microservice. Hãy xem xét một microservice Email có nhiệm vụ thỉnh thoảng gửi email cho khách hàng của ta. Nó được yêu cầu gửi một email “đơn hàng đã được giao” cho khách hàng có ID 1234; nó đi và truy xuất khách hàng có ID đó và nhận lại một thứ gì đó như phản hồi trong Ví dụ 5-3.

Example 5-3. Sample response from the Customer service

Ví dụ 5-3. Phản hồi mẫu từ dịch vụ Customer

Sam Newman sam@magpiebrain.com 555-1234-5678

Sam Newman sam@magpiebrain.com 555-1234-5678

Now, to send the email, the Email microservice needs only the `firstname`, `lastname`, and `email` fields. We don't need to know the `telephoneNumber`. We want to simply pull out those fields we care about and ignore the rest. Some binding technology, especially that used by strongly typed languages, can attempt to bind all fields whether the consumer wants them or not. What happens if we realize that no one is using the `telephoneNumber` and we decide to remove it? This could cause consumers to break needlessly.

Giờ, để gửi email, microservice Email chỉ cần các trường `firstname`, `lastname` và `email`. Ta không cần biết `telephoneNumber`. Ta muốn chỉ đơn giản rút ra những trường mình quan tâm và bỏ qua phần còn lại. Một số công nghệ binding, đặc biệt là loại được dùng bởi các ngôn ngữ kiểu mạnh (strongly typed), có thể cố gắng bind tất cả các trường dù consumer muốn chúng hay không. Điều gì xảy ra nếu ta nhận ra không ai dùng `telephoneNumber` và ta quyết định loại bỏ nó? Điều này có thể khiến consumer bị phá vỡ một cách không cần thiết.

Likewise, what if we wanted to restructure our Customer object to support more details, perhaps adding some further structure, as in Example 5-4? The data our Email service wants is still there,

with the same name, but if our code makes very explicit assumptions as to where the firstname and lastname fields will be stored, then it could break again. In this instance, we could instead use XPath to pull out the fields we care about, allowing us to be ambivalent about where the fields are as long as we can find them. This pattern—of implementing a reader able to ignore changes we don’t care about—is what Martin Fowler calls a tolerant reader .

Tương tự, điều gì xảy ra nếu ta muốn tái cấu trúc đối tượng Customer của mình để hỗ trợ thêm chi tiết, có lẽ thêm một số cấu trúc nữa, như trong Ví dụ 5-4? Dữ liệu mà dịch vụ Email của ta muốn vẫn còn đó, với cùng tên, nhưng nếu mã của ta đưa ra các giả định rất tường minh về việc các trường firstname và lastname sẽ được lưu ở đâu, thì nó lại có thể bị phá vỡ. Trong trường hợp này, thay vào đó ta có thể dùng XPath để rút ra các trường mình quan tâm, cho phép ta không cần bận tâm các trường nằm ở đâu miễn là ta có thể tìm thấy chúng. Pattern này—hiện thực một bên đọc có khả năng bỏ qua những thay đổi mà ta không quan tâm—là cái mà Martin Fowler gọi là tolerant reader.

Example 5-4. A restructured Customer resource: the data is all still there, but can our consumers find it?

Ví dụ 5-4. Một tài nguyên Customer được tái cấu trúc: dữ liệu vẫn còn đó cả, nhưng liệu consumer của ta có thể tìm thấy nó không?

Sam Newman Magpiebrain Sam “Magpiebrain” Newman sam@magpiebrain.com

Sam Newman Magpiebrain Sam “Magpiebrain” Newman sam@magpiebrain.com

The example of a client trying to be as flexible as possible in consuming a service demonstrates Postel’s law (otherwise known as the **robustness** principle), which states: “Be conservative in what you do, be liberal in what you accept from others.” The original context for this piece of wisdom was the interaction of devices over networks, where you should expect all sorts of odd things to happen. In the context of microservice-based interactions, it leads us to try and structure our client code to be tolerant of changes to payloads.

Ví dụ về một client cố gắng linh hoạt nhất có thể trong việc tiêu thụ một dịch vụ minh họa định luật Postel (còn được gọi là nguyên tắc bền vững - robustness principle), phát biểu rằng: “Hãy bảo thủ trong những gì bạn làm, hãy phóng khoáng trong những gì bạn chấp nhận từ người khác.” Bối cảnh gốc của mẫu trí tuệ này là sự tương tác của các thiết bị qua mạng, nơi bạn nên trông đợi đủ loại điều kỳ quặc xảy ra. Trong bối cảnh các tương tác dựa trên microservice, nó dẫn ta tới việc cố gắng cấu trúc mã client của mình để khoan dung với các thay đổi đối với payload.

Right Technology — Công nghệ phù hợp

As we’ve already explored, some technology can be more brittle when it comes to allowing us to change interfaces—I’ve already highlighted my own personal frustrations with Java RMI. On the other hand, some integration implementations go out of their way to make it as easy as possible for changes to be made without breaking clients. At the simple end of the spectrum, protocol buffers, the serialization format used as part of gRPC, have the concept of field number. Each entry in a protocol buffer has to define a field number, which client code expects to find. If new fields are added, the client doesn’t care. Avro allows for the schema to be sent along with the payload, allowing clients to potentially interpret a payload much like a dynamic type.

Như ta đã khám phá, một số công nghệ có thể giòn hơn khi nói đến việc cho phép ta thay đổi interface—tôi đã nói rõ những bức bối cá nhân của mình với Java RMI. Mặt khác, một số hiện thực tích hợp đi hết mình để làm cho việc thực hiện các thay đổi mà không phá vỡ

client dễ dàng nhất có thể. Ở đầu đơn giản của phổ này, protocol buffer, định dạng tuần tự hóa được dùng như một phần của gRPC, có khái niệm số trường (field number). Mỗi mục trong một protocol buffer phải định nghĩa một số trường, mà mã client kỳ vọng tìm thấy. Nếu các trường mới được thêm vào, client không bận tâm. Avro cho phép schema được gửi cùng với payload, cho phép client có khả năng diễn giải một payload giống như một kiểu động.

At the more extreme end of the spectrum, the REST concept of HATEOAS is largely all about enabling clients to make use of REST endpoints even when they change by making use of the previously discussed hypermedia links. This does call for you to buy into the entire HATEOAS mindset, of course.

Ở đầu cực đoan hơn của phổ, khái niệm HATEOAS của REST phần lớn là về việc cho phép client tận dụng các endpoint REST ngay cả khi chúng thay đổi bằng cách dùng các liên kết hypermedia đã bàn trước đó. Tất nhiên điều này đòi hỏi bạn phải chấp nhận toàn bộ tư duy HATEOAS.

Explicit Interface — Interface tường minh

I am a big fan of a microservice exposing an explicit schema that denotes what its endpoints do. Having an explicit schema makes it clear to consumers what they can expect, but it also makes it much clearer to a developer working on a microservice as to what things should remain untouched to ensure you don't break consumers. Put another way, an explicit schema goes a long way toward making the boundaries of information hiding more explicit—what's exposed in the schema is by definition not hidden.

Tôi là một người rất thích việc một microservice phơi bày một schema tường minh chỉ rõ các endpoint của nó làm gì. Việc có một schema tường minh làm cho consumer thấy rõ những gì họ có thể kỳ vọng, nhưng nó cũng làm cho một lập trình viên làm việc trên một microservice thấy rõ hơn nhiều những thứ nào nên được giữ nguyên để đảm bảo bạn không phá vỡ consumer. Nói cách khác, một schema tường minh đi một chặng đường dài hướng tới việc làm cho ranh giới của việc che giấu thông tin trở nên tường minh hơn—những gì được phơi bày trong schema thì theo định nghĩa là không bị che giấu.

Having an explicit schema for RPC is long established and is in fact a requirement for many RPC implementations. REST, on the other hand, has typically viewed the concept of a schema as optional, to the point that I find explicit schemas for REST endpoints to be vanishingly rare. This is changing, with things like the aforementioned OpenAPI specification gaining traction, and with the JSON Schema specification also gaining in maturity.

Việc có một schema tường minh cho RPC đã có từ lâu và thực ra là một yêu cầu đối với nhiều hiện thực RPC. Mặt khác, REST thường xem khái niệm schema là tùy chọn, đến mức tôi thấy schema tường minh cho các endpoint REST là cực kỳ hiếm. Điều này đang thay đổi, với những thứ như đặc tả OpenAPI đã đề cập đang dần được chấp nhận, và với đặc tả JSON Schema cũng đang trưởng thành hơn.

Asynchronous messaging protocols have struggled more in this space. You can have a schema for the payload of a message easily enough, and in fact this is an area in which Avro is frequently used. However, having an explicit interface needs to go further than this. If we consider a microservice that fires events, which events does it expose? A few attempts at making explicit schemas for event-based endpoints are now underway. One is AsyncAPI, which has picked up a number of big-name users, but the one gaining most traction seems to be CloudEvents, a specification which is backed by the Cloud Native Computing Foundation (CNCF). Azure's event grid product supports the CloudEvents

format, a sign of different vendors supporting this format, which should help with interoperability. This is still a fairly new space, so it will be interesting to see how things shake out over the next few years.

Các giao thức messaging bất đồng bộ đã gặp khó khăn hơn trong không gian này. Bạn có thể có một schema cho payload của một message khá dễ dàng, và thực tế đây là một lĩnh vực mà Avro thường được dùng. Tuy nhiên, việc có một interface tường minh cần đi xa hơn thế. Nếu ta xem xét một microservice phát ra các event, nó phơi bày những event nào? Một vài nỗ lực tạo schema tường minh cho các endpoint dựa trên event hiện đang được triển khai. Một là AsyncAPI, vốn đã thu hút một số người dùng tên tuổi lớn, nhưng cái có vẻ được chấp nhận nhiều nhất là CloudEvents, một đặc tả được hậu thuẫn bởi Cloud Native Computing Foundation (CNCF). Sản phẩm event grid của Azure hỗ trợ định dạng CloudEvents, một dấu hiệu cho thấy các nhà cung cấp khác nhau đang hỗ trợ định dạng này, điều sẽ giúp ích cho khả năng tương tác. Đây vẫn là một không gian khá mới, nên sẽ thú vị để xem mọi thứ ngã ngũ ra sao trong vài năm tới.

Semantic Versioning — Đánh phiên bản theo ngữ nghĩa (Semantic Versioning)

Wouldn't it be great if as a client you could look just at the version number of a service and know whether you can integrate with it? Semantic versioning is a specification that allows just that. With semantic versioning, each version number is in the form MAJOR.MINOR.PATCH . When the MAJOR number increments, it means that backward-incompatible changes have been made. When MINOR increments, new functionality has been added that should be backward compatible. Finally, a change to PATCH states that bug fixes have been made to existing functionality.

Chẳng phải sẽ tuyệt sao nếu với tư cách một client bạn có thể chỉ nhìn vào số phiên bản của một dịch vụ và biết liệu bạn có thể tích hợp với nó hay không? Đánh phiên bản theo ngữ nghĩa (semantic versioning) là một đặc tả cho phép đúng điều đó. Với đánh phiên bản theo ngữ nghĩa, mỗi số phiên bản có dạng MAJOR.MINOR.PATCH. Khi số MAJOR tăng, nó nghĩa là đã có các thay đổi không tương thích ngược. Khi MINOR tăng, chức năng mới đã được thêm vào mà nên tương thích ngược. Cuối cùng, một thay đổi ở PATCH nói rằng đã có các bản sửa lỗi cho chức năng hiện có.

To see how useful semantic versioning can be, let's look at a simple use case. Our helpdesk application is built to work against version 1.2.0 of the Customer service. If a new feature is added, causing the Customer service to change to 1.3.0, our helpdesk application should see no change in behavior and shouldn't be expected to make any changes. We couldn't guarantee that we could work against version 1.1.0 of the Customer service, though, as we may rely on functionality added in the 1.2.0 **release**. We could also expect to have to make changes to our application if a new 2.0.0 release of the Customer service comes out.

Để thấy đánh phiên bản theo ngữ nghĩa có thể hữu ích thế nào, hãy xem một trường hợp sử dụng đơn giản. Ứng dụng helpdesk của ta được xây dựng để hoạt động với phiên bản 1.2.0 của dịch vụ Customer. Nếu một tính năng mới được thêm vào, khiến dịch vụ Customer thay đổi sang 1.3.0, ứng dụng helpdesk của ta sẽ không thấy thay đổi gì về hành vi và không nên bị kỳ vọng phải thực hiện bất kỳ thay đổi nào. Tuy nhiên, ta không thể đảm bảo rằng ta có thể hoạt động với phiên bản 1.1.0 của dịch vụ Customer, vì ta có thể phụ thuộc vào chức năng được thêm trong bản phát hành 1.2.0. Ta cũng có thể trông đợi phải thực hiện các thay đổi cho ứng dụng của mình nếu một bản phát hành 2.0.0 mới của dịch vụ Customer ra mắt.

You may decide to have a semantic version for the service, or even for an individual endpoint on a service if you are coexisting them, as detailed in the next section.

Bạn có thể quyết định có một phiên bản ngữ nghĩa cho dịch vụ, hoặc thậm chí cho một endpoint riêng lẻ trên một dịch vụ nếu bạn đang cho chúng cùng tồn tại (coexist), như được nêu chi tiết trong mục tiếp theo.

This versioning scheme allows us to pack a lot of information and expectations into just three fields. The full specification outlines in very simple terms the expectations clients can have of changes to these numbers, and it can simplify the process of communicating about whether changes should impact consumers. Unfortunately, I haven't seen this approach used enough in distributed systems to understand its effectiveness in that context—something that hasn't really changed since the first edition of this book.

Sơ đồ đánh phiên bản này cho phép ta đóng gói rất nhiều thông tin và kỳ vọng vào chỉ ba trường. Đặc tả đầy đủ phác thảo bằng các thuật ngữ rất đơn giản những kỳ vọng mà client có thể có về các thay đổi đối với những con số này, và nó có thể đơn giản hóa quá trình trao đổi về việc liệu các thay đổi có nên tác động đến consumer hay không. Tiếc thay, tôi chưa thấy cách tiếp cận này được dùng đủ nhiều trong các hệ thống phân tán để hiểu được tính hiệu quả của nó trong bối cảnh đó—điều này thực sự chưa thay đổi kể từ ấn bản đầu tiên của cuốn sách này.

Catch Accidental Breaking Changes Early — Bắt sớm các thay đổi gây phá vỡ vô tình

It's crucial that we pick up changes that will break consumers as soon as possible, because even if we choose the best possible technology, an innocent change of a microservice could cause consumers to break. As we've already touched on, using schemas can help us pick up structural changes, assuming we use some sort of tooling to help compare schema versions. There is a wide range of tooling out there to do this for different schema types. We have Protolock for protocol buffers, json-schema-diff-validator for JSON Schema, and openapi-diff for the OpenAPI specification. 2 More tools seem to be cropping up all the time in this space. What you're looking for, though, is something that won't just report on the differences between two schemas but will pass or fail based on compatibility; this would allow you to fail a CI build if incompatible schemas are found, ensuring that your microservice won't get deployed.

Việc nắm bắt sớm nhất có thể các thay đổi sẽ phá vỡ consumer là cực kỳ quan trọng, vì ngay cả khi ta chọn công nghệ tốt nhất có thể, một thay đổi vô hại của một microservice cũng có thể khiến consumer bị phá vỡ. Như ta đã chạm tới, việc dùng schema có thể giúp ta nắm bắt các thay đổi cấu trúc, giả sử ta dùng một dạng công cụ nào đó để giúp so sánh các phiên bản schema. Có rất nhiều công cụ ngoài kia để làm điều này cho các loại schema khác nhau. Ta có Protolock cho protocol buffer, json-schema-diff-validator cho JSON Schema, và openapi-diff cho đặc tả OpenAPI. 2 Có vẻ như ngày càng có thêm nhiều công cụ xuất hiện trong không gian này. Tuy nhiên, cái bạn đang tìm là một thứ không chỉ báo cáo về các khác biệt giữa hai schema mà còn cho qua hoặc đánh trượt dựa trên tính tương thích; điều này sẽ cho phép bạn làm hỏng một bản build CI nếu các schema không tương thích được tìm thấy, đảm bảo rằng microservice của bạn sẽ không được triển khai.

The open source Confluent Schema Registry supports JSON Schema, Avro, and protocol buffers and is capable of comparing newly uploaded versions for backward compatibility. Although it was built to help as part of an ecosystem in which Kafka is being used, and needs Kafka to run, there is nothing to

stop you from using it to store and validate schemas being used for non-Kafka-based communication.

Confluent Schema Registry mã nguồn mở hỗ trợ JSON Schema, Avro và protocol buffer và có khả năng so sánh các phiên bản mới tải lên để kiểm tra tính tương thích ngược. Mặc dù nó được xây dựng để giúp ích như một phần của một hệ sinh thái trong đó Kafka đang được dùng, và cần Kafka để chạy, không có gì ngăn bạn dùng nó để lưu trữ và kiểm chứng các schema được dùng cho giao tiếp không dựa trên Kafka.

Schema comparison tools can help us catch structural breakages, but what about semantic breakages? Or what if you aren't making use of schemas in the first place? Then we're looking at testing. This is a topic we'll explore in more detail in "Contract Tests and Consumer-Driven Contracts (CDCs)" on page 292, but I wanted to highlight **consumer-driven contract** testing, which explicitly helps in this area—Pact being an excellent example of a tool aimed specifically at this problem. Just remember, if you don't have schemas, expect your testing to have to do more work to catch breaking changes.

Các công cụ so sánh schema có thể giúp ta bắt các phá vỡ cấu trúc, nhưng còn các phá vỡ ngữ nghĩa thì sao? Hoặc điều gì xảy ra nếu bạn không dùng schema ngay từ đầu? Khi đó ta đang nói đến kiểm thử. Đây là một chủ đề ta sẽ khám phá chi tiết hơn trong "Contract Test và Consumer-Driven Contract (CDC)" ở trang 292, nhưng tôi muốn làm nổi bật consumer-driven contract testing, thứ giúp ích một cách tường minh trong lĩnh vực này—Pact là một ví dụ tuyệt vời về một công cụ nhắm cụ thể vào vấn đề này. Chỉ cần nhớ, nếu bạn không có schema, hãy trông đợi việc kiểm thử của bạn sẽ phải làm nhiều việc hơn để bắt các thay đổi gây phá vỡ.

If you're supporting multiple different client libraries, running tests using each library you support against the latest service is another technique that can help. Once you realize you are going to break a consumer, you have the choice to either try to

Nếu bạn đang hỗ trợ nhiều client library khác nhau, việc chạy các bài kiểm thử dùng mỗi library bạn hỗ trợ với dịch vụ mới nhất là một kỹ thuật khác có thể giúp ích. Một khi bạn nhận ra mình sắp phá vỡ một consumer, bạn có lựa chọn hoặc cố

2 Note that there are actually three different tools in this space with the same name! The openapi-diff tool at <https://github.com/Azure/openapi-diff> seems to get closest to a tool that actually passes or fails compatibility.

2 Lưu ý rằng thực ra có tới ba công cụ khác nhau trong không gian này với cùng một cái tên! Công cụ openapi-diff tại <https://github.com/Azure/openapi-diff> có vẻ tiến gần nhất tới một công cụ thực sự cho hoặc đánh trượt tính tương thích.

avoid the break altogether or else embrace it and start having the right conversations with the people looking after the consuming services.

tránh hoàn toàn việc phá vỡ hoặc đón nhận nó và bắt đầu có những cuộc trò chuyện đúng đắn với những người chăm sóc các dịch vụ tiêu thụ.

Managing Breaking Changes — Quản lý các thay đổi gây phá vỡ

So you've gone as far as you can to ensure that the changes you're making to a microservice's interface are backward compatible, but you've realized that you just have to make a change that will constitute a breaking change. What can you do in such a situation? You've got three main options:

Vậy là bạn đã đi xa hết mức có thể để đảm bảo rằng các thay đổi bạn thực hiện đối với interface của một microservice là tương thích ngược, nhưng bạn nhận ra rằng bạn chỉ đơn

giản là phải thực hiện một thay đổi cấu thành một thay đổi gây phá vỡ. Bạn có thể làm gì trong tình huống như vậy? Bạn có ba lựa chọn chính:

Lockstep deployment Require that the microservice exposing the interface and all consumers of that interface are changed at the same time.

Triển khai đồng bộ bước (Lockstep deployment) Đòi hỏi rằng microservice phơi bày interface và tất cả các consumer của interface đó được thay đổi cùng một lúc.

Coexist incompatible microservice versions Run old and new versions of the microservice side by side.

Cho các phiên bản microservice không tương thích cùng tồn tại (Coexist incompatible microservice versions) Chạy phiên bản cũ và mới của microservice song song nhau.

Emulate the old interface Have your microservice expose the new interface and also emulate the old interface.

Mô phỏng interface cũ (Emulate the old interface) Để microservice của bạn phơi bày interface mới và cũng mô phỏng interface cũ.

Lockstep Deployment — Triển khai đồng bộ bước

Of course, lockstep deployment flies in the face of **independent deployability**. If we want to be able to deploy a new version of our microservice with a breaking change to its interface but still do this in an independent fashion, we need to give our consumers time to upgrade to the new interface. That leads us on to the next two options I'd consider.

Tất nhiên, triển khai đồng bộ bước đi ngược lại khả năng triển khai độc lập. Nếu ta muốn có khả năng triển khai một phiên bản mới của microservice với một thay đổi gây phá vỡ đối với interface của nó nhưng vẫn làm điều này theo cách độc lập, ta cần cho consumer thời gian để nâng cấp lên interface mới. Điều đó dẫn ta tới hai lựa chọn tiếp theo mà tôi sẽ cân nhắc.

Coexist Incompatible Microservice Versions — Cho các phiên bản microservice không tương thích cùng tồn tại

Another versioning solution often cited is to have different versions of the service live at once and for older consumers to route their traffic to the older version, with newer consumers seeing the new version, as shown in Figure 5-3 . This is the approach used sparingly by Netflix in situations in which the cost of changing older consumers is too high, especially in rare cases in which legacy devices are still tied to older versions of the API. I am not a fan of this idea personally, and understand why Netflix uses it rarely. First, if I need to fix an internal bug in my service, I now have to fix and deploy two different sets of services. This would probably mean I have to branch the codebase for my service, and that is always problematic. Second, it means I need smarts to handle directing consumers to the right microservice. This behavior inevitably ends up sitting in middleware somewhere, or in a bunch of nginx scripts, making it harder to reason about the behavior of the system. Finally, consider any persistent state our service might manage. Customers created by either version of the service need to be stored and made visible to all services, no matter which version was used to create the data in the first place. This can be an additional source of complexity.

Một giải pháp đánh phiên bản khác thường được viện dẫn là có các phiên bản khác nhau của dịch vụ tồn tại cùng lúc và để các consumer cũ định tuyến lưu lượng của chúng tới phiên bản cũ, trong khi các consumer mới hơn thấy phiên bản mới, như được thể hiện trong Hình 5-3. Đây là cách tiếp cận được Netflix dùng một cách dè dặt trong những tình huống mà chi phí thay đổi các consumer cũ quá cao, đặc biệt trong những trường hợp hiếm hoi khi các thiết bị legacy vẫn còn bị gắn vào các phiên bản API cũ. Về phần cá nhân tôi không thích ý tưởng này, và hiểu vì sao Netflix hiếm khi dùng nó. Thứ nhất, nếu tôi cần sửa một lỗi nội bộ trong dịch vụ của mình, giờ tôi phải sửa và triển khai hai tập dịch vụ khác nhau. Điều này có lẽ nghĩa là tôi phải tạo nhánh (branch) cho codebase của dịch vụ mình, và điều đó luôn rắc rối. Thứ hai, nó nghĩa là tôi cần sự thông minh để xử lý việc hướng các consumer tới đúng microservice. Hành vi này chắc chắn sẽ rớt cuộc nằm đâu đó trong middleware, hoặc trong một mớ script nginx, khiến việc suy luận về hành vi của hệ thống khó hơn. Cuối cùng, hãy xem xét bất kỳ trạng thái bền bỉ nào mà dịch vụ của ta có thể quản lý. Các customer được tạo bởi một trong hai phiên bản của dịch vụ cần được lưu trữ và làm cho hiển thị với tất cả các dịch vụ, bất kể phiên bản nào được dùng để tạo dữ liệu ngay từ đầu. Điều này có thể là một nguồn phức tạp bổ sung.

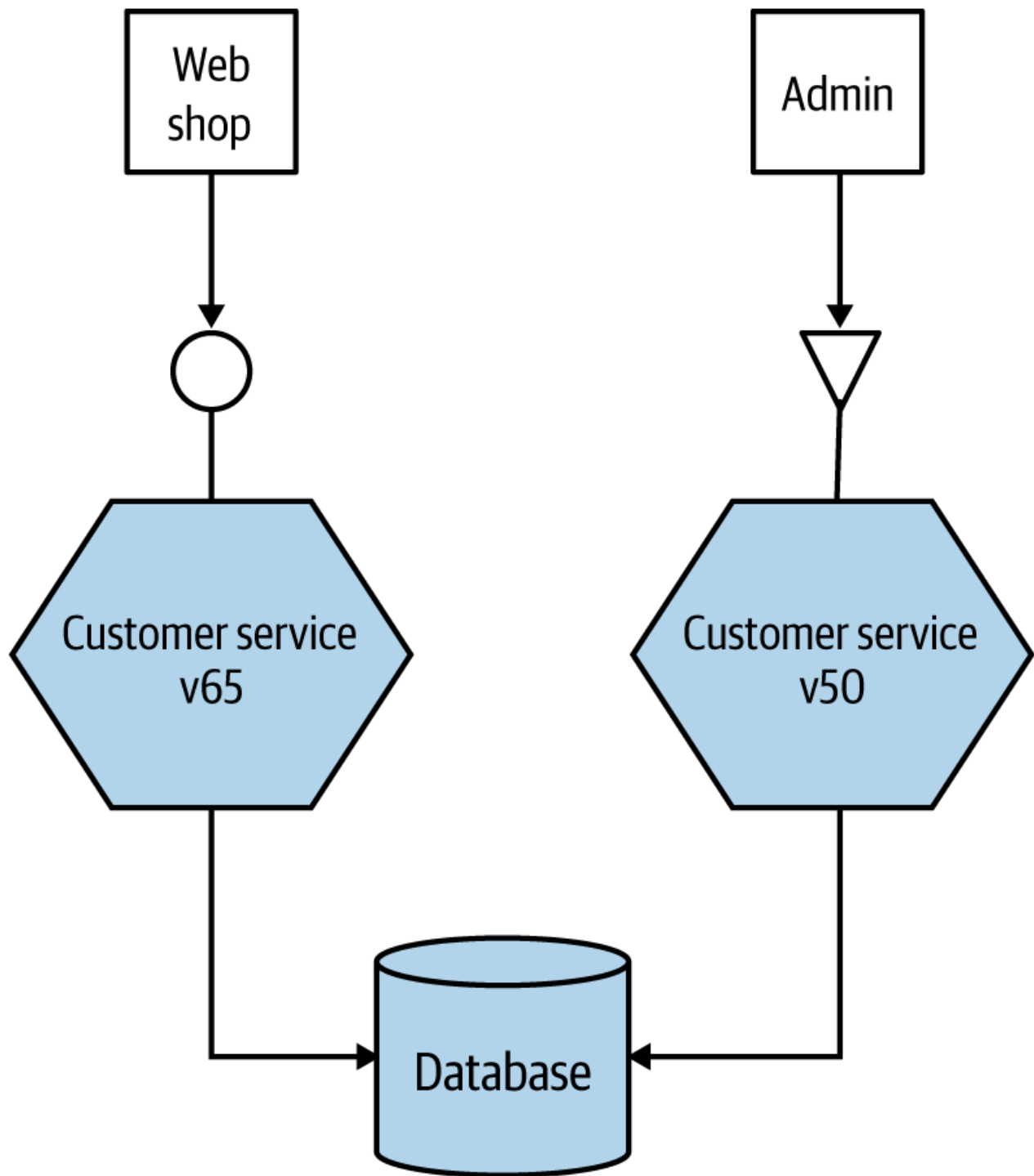


Figure 5-3. Running multiple versions of the same service to support old endpoints

Hình 5-3. Chạy nhiều phiên bản của cùng một dịch vụ để hỗ trợ các endpoint cũ

Coexisting concurrent service versions for a short period of time can make perfect sense, especially when you're doing something like a canary release (we'll be discussing this pattern more in "On to Progressive Delivery" on page 270). In these situations, we may be coexisting versions for only a few minutes or perhaps hours, and we normally will have only two different versions of the service present at the same time. The longer it takes for you to get consumers upgraded to the newer version

and released, the more you should look to coexist different endpoints in the same microservice rather than coexist entirely different versions. I remain unconvinced that this work is worthwhile for the average project.

Cho các phiên bản dịch vụ đồng thời cùng tồn tại trong một khoảng thời gian ngắn có thể hoàn toàn hợp lý, đặc biệt khi bạn đang làm một thứ như canary release (ta sẽ bàn về pattern này thêm trong “Tiến tới Progressive Delivery” ở trang 270). Trong những tình huống này, ta có thể chỉ cho các phiên bản cùng tồn tại trong vài phút hoặc có lẽ vài giờ, và ta thường sẽ chỉ có hai phiên bản khác nhau của dịch vụ hiện diện cùng một lúc. Việc đưa consumer nâng cấp lên phiên bản mới hơn và phát hành càng mất nhiều thời gian, thì bạn càng nên hướng tới việc cho các endpoint khác nhau cùng tồn tại trong cùng một microservice thay vì cho hoàn toàn các phiên bản khác nhau cùng tồn tại. Tôi vẫn chưa bị thuyết phục rằng công việc này là xứng đáng với một dự án trung bình.

Emulate the Old Interface — Mô phỏng interface cũ

If we’ve done all we can to avoid introducing a breaking interface change, our next job is to limit the impact. The thing we want to avoid is forcing consumers to upgrade in lockstep with us, as we always want to maintain the ability to release microservices independently of each other. One approach I have used successfully to handle this is to coexist both the old and new interfaces in the same running service. So if we want to release a breaking change, we deploy a new version of the service that exposes both the old and new versions of the endpoint.

Nếu ta đã làm tất cả những gì có thể để tránh đưa ra một thay đổi interface gây phá vỡ, công việc tiếp theo của ta là giới hạn tác động. Điều ta muốn tránh là buộc consumer phải nâng cấp đồng bộ bước cùng với ta, vì ta luôn muốn duy trì khả năng phát hành các microservice độc lập với nhau. Một cách tiếp cận mà tôi đã dùng thành công để xử lý điều này là cho cả interface cũ và mới cùng tồn tại trong cùng một dịch vụ đang chạy. Vậy nên nếu ta muốn phát hành một thay đổi gây phá vỡ, ta triển khai một phiên bản mới của dịch vụ phơi bày cả phiên bản cũ và mới của endpoint.

This allows us to get the new microservice out as soon as possible, along with the new interface, while giving time for consumers to move over. Once all the consumers are no longer using the old endpoint, you can remove it along with any associated code, as shown in Figure 5-4 .

Điều này cho phép ta đưa microservice mới ra càng sớm càng tốt, cùng với interface mới, đồng thời cho consumer thời gian để chuyển sang. Một khi tất cả consumer không còn dùng endpoint cũ, bạn có thể loại bỏ nó cùng với bất kỳ mã liên quan nào, như được thể hiện trong Hình 5-4.

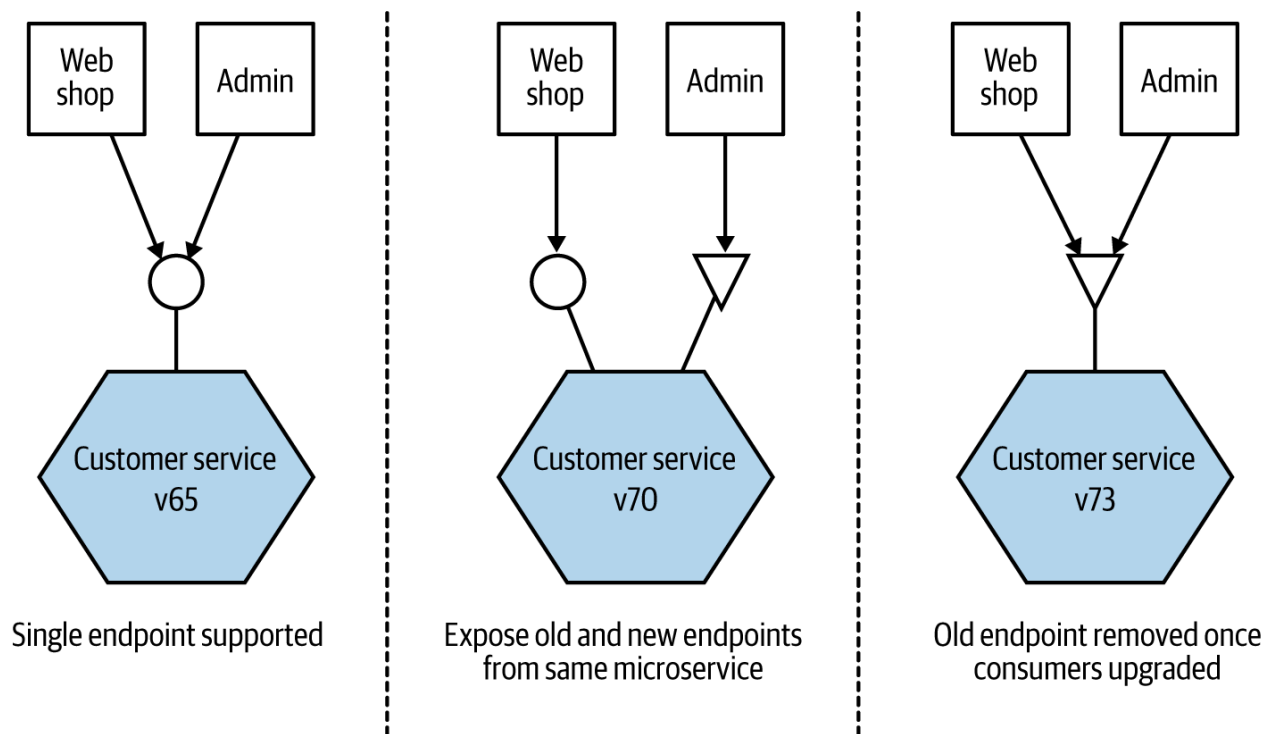


Figure 5-4. One microservice emulating the old endpoint and exposing the new backward-incompatible endpoint

Hình 5-4. Một microservice mô phỏng endpoint cũ và phơi bày endpoint mới không tương thích ngược

When I last used this approach, we had gotten ourselves into a bit of a mess with the number of consumers we had and the number of breaking changes we had made. This meant that we were actually coexisting three different versions of the endpoint. This is not something I'd recommend! Keeping all the code around and the associated testing required to ensure they all worked was absolutely an additional burden. To make this more manageable, we internally transformed all requests to the V1 endpoint to a V2 request, and then V2 requests to the V3 endpoint. This meant we could clearly delineate what code was going to be retired when the old endpoint(s) died.

Lần gần nhất tôi dùng cách tiếp cận này, chúng tôi đã tự đẩy mình vào một mớ hỗn độn với số lượng consumer mà chúng tôi có và số lượng thay đổi gây phá vỡ mà chúng tôi đã thực hiện. Điều này nghĩa là chúng tôi thực ra đang cho ba phiên bản khác nhau của endpoint cùng tồn tại. Đây không phải là điều tôi khuyến nghị! Việc giữ lại tất cả mã và việc kiểm thử liên quan cần thiết để đảm bảo tất cả chúng đều hoạt động đúng là tuyệt đối một gánh nặng bổ sung. Để làm cho điều này dễ quản lý hơn, chúng tôi đã biến đổi nội bộ tất cả các request tới endpoint V1 thành một request V2, rồi các request V2 thành endpoint V3. Điều này nghĩa là chúng tôi có thể phân định rõ ràng mã nào sẽ bị loại bỏ khi (các) endpoint cũ chết.

This is in effect an example of the expand and contract pattern, which allows us to phase in breaking changes. We expand the capabilities we offer, supporting both old and new ways of doing something. Once the old consumers do things in the new way, we contract our API, removing the old functionality.

Về bản chất, đây là một ví dụ của pattern mở rộng và thu hẹp (expand and contract), cho phép ta đưa các thay đổi gây phá vỡ vào dần dần. Ta mở rộng các khả năng ta cung cấp,

hỗ trợ cả cách cũ và cách mới để làm một việc gì đó. Một khi các consumer cũ làm việc theo cách mới, ta thu hẹp API của mình, loại bỏ chức năng cũ.

If you are going to coexist endpoints, you need a way for callers to route their requests accordingly. For systems making use of HTTP, I have seen this done with both version numbers in request headers and also in the URI itself—for example, `/v1/customer/` or `/v2/customer/`. I'm torn as to which approach makes the most sense. On the one hand, I like URIs being opaque to discourage clients from hardcoding URI templates, but on the other hand, this approach does make things very obvious and can simplify request routing.

Nếu bạn định cho các endpoint cùng tồn tại, bạn cần một cách để các bên gọi định tuyến request của họ một cách phù hợp. Đối với các hệ thống dùng HTTP, tôi đã thấy điều này được làm với cả số phiên bản trong header request và cả trong chính URI—ví dụ, `/v1/customer/` hoặc `/v2/customer/`. Tôi phân vân không biết cách tiếp cận nào hợp lý nhất. Một mặt, tôi thích các URI mờ đục (opaque) để ngăn client hardcode các mẫu URI, nhưng mặt khác, cách tiếp cận này quả thực làm cho mọi thứ rất rõ ràng và có thể đơn giản hóa việc định tuyến request.

For RPC, things can be a little trickier. I have handled this with protocol buffers by putting my methods in different namespaces—for example, `v1.createCustomer` and `v2.createCustomer`—but when you are trying to support different versions of the same types being sent over the network, this approach can become really painful.

Đối với RPC, mọi thứ có thể hơi khó nhằn hơn. Tôi đã xử lý điều này với protocol buffer bằng cách đặt các phương thức của mình vào các namespace khác nhau—ví dụ, `v1.createCustomer` và `v2.createCustomer`—nhưng khi bạn đang cố hỗ trợ các phiên bản khác nhau của cùng các kiểu được gửi qua mạng, cách tiếp cận này có thể trở nên thực sự đau đầu.

Which Approach Do I Prefer? — Tôi thích cách tiếp cận nào?

For situations in which the same team manages both the microservice and all consumers, I am somewhat relaxed about a lockstep release in limited situations. Assuming it really is a one-off situation, then doing this when the impact is limited to a single team can be justifiable. I am very cautious about this, though, as there is the danger that a one-off activity becomes business as usual, and there goes independent deployability. Use lockstep deployments too often, and you'll end up with a **distributed monolith** before long.

Đối với những tình huống mà cùng một nhóm quản lý cả microservice lẫn tất cả consumer, tôi có phần thoải mái về một bản phát hành đồng bộ bước trong những tình huống hạn chế. Giả sử nó thực sự là một tình huống xảy ra một lần, thì làm điều này khi tác động bị giới hạn trong một nhóm duy nhất có thể biện minh được. Tuy nhiên, tôi rất thận trọng về điều này, vì có nguy cơ một hoạt động một-lần trở thành chuyện thường ngày, và thế là khả năng triển khai độc lập đi tong. Dùng triển khai đồng bộ bước quá thường xuyên, và bạn sẽ chẳng mấy chốc rồi có một distributed monolith.

Coexisting different versions of the same microservice can be problematic, as we discussed. I'd consider doing this only in situations in which we planned to run the microservice versions side by side for only a short period of time. The reality is that when you need to give consumers time to upgrade, you could be looking at weeks or more. In other situations in which you might coexist microservice versions, perhaps as part of a blue-green deployment or canary release, the durations involved are much shorter, offsetting the downsides of this approach.

Cho các phiên bản khác nhau của cùng một microservice cùng tồn tại có thể gây rắc rối, như ta đã bàn. Tôi sẽ cân nhắc làm điều này chỉ trong những tình huống mà ta lên kế hoạch chạy các phiên bản microservice song song nhau chỉ trong một khoảng thời gian ngắn. Thực tế là khi bạn cần cho consumer thời gian để nâng cấp, bạn có thể đang nhìn vào hàng tuần hoặc hơn. Trong những tình huống khác mà bạn có thể cho các phiên bản microservice cùng tồn tại, có lẽ như một phần của một blue-green deployment hay canary release, thời lượng liên quan ngắn hơn nhiều, bù đắp cho các nhược điểm của cách tiếp cận này.

My general preference is to use emulation of old endpoints wherever possible. The challenges of implementing emulation are in my opinion much easier to deal with than those of coexisting microservice versions.

Sở thích chung của tôi là dùng mô phỏng các endpoint cũ bất cứ khi nào có thể. Các thách thức của việc hiện thực mô phỏng, theo ý kiến của tôi, dễ xử lý hơn nhiều so với các thách thức của việc cho các phiên bản microservice cùng tồn tại.

The Social Contract — Hợp đồng xã hội

Which approach you pick will be due in large part to the expectations consumers have of how these changes will be made. Keeping the old interface lying around can have a cost, and ideally you'd like to turn it off and remove associated code and infrastructure as soon as possible. On the other hand, you want to give consumers as much time as possible to make a change. And remember, in many cases the backward-incompatible changes you are making are often things that have been asked for by the consumers and/or will actually end up benefiting them. There is a balancing act, of course, between the needs of the microservice maintainers and those of the consumers, and this needs to be discussed.

Cách tiếp cận bạn chọn sẽ phần lớn phụ thuộc vào kỳ vọng mà consumer có về cách những thay đổi này sẽ được thực hiện. Việc giữ interface cũ tồn tại quanh quẩn có thể có một chi phí, và lý tưởng nhất bạn sẽ muốn tắt nó và loại bỏ mã cùng hạ tầng liên quan càng sớm càng tốt. Mặt khác, bạn muốn cho consumer càng nhiều thời gian càng tốt để thực hiện thay đổi. Và hãy nhớ, trong nhiều trường hợp, các thay đổi không tương thích ngược mà bạn đang thực hiện thường là những thứ đã được consumer yêu cầu và/hoặc thực sự sẽ rất cuộc mang lại lợi ích cho họ. Tất nhiên có một sự cân bằng giữa nhu cầu của những người bảo trì microservice và những người consumer, và điều này cần được thảo luận.

I've found that in many situations, how these changes will be handled has never been discussed, leading to all sorts of challenges. As with schemas, having some degree of explicitness in how backward-incompatible changes will be made can greatly simplify things.

Tôi nhận thấy rằng trong nhiều tình huống, việc các thay đổi này sẽ được xử lý như thế nào chưa bao giờ được thảo luận, dẫn đến đủ loại thách thức. Cũng như với schema, việc có một mức độ tường minh nào đó về cách các thay đổi không tương thích ngược sẽ được thực hiện có thể đơn giản hóa mọi thứ rất nhiều.

You don't necessarily need reams of paper and huge meetings to reach agreement on how changes will be handled. But assuming you aren't going down the route of lockstep releases, I'd suggest that both the owner and the consumer of a microservice need to be clear on a few things:

Bạn không nhất thiết cần hàng tập giấy và những cuộc họp khổng lồ để đạt được thỏa thuận về cách các thay đổi sẽ được xử lý. Nhưng giả sử bạn không đi theo con đường của

các bản phát hành đồng bộ bước, tôi đề nghị rằng cả chủ sở hữu lẫn consumer của một microservice cần rõ ràng về một vài điều:

- How will you raise the issue that the interface needs to change?
- How will the consumers and microservice teams collaborate to agree on what the change will look like?
- Who is expected to do the work to update the consumers?
- When the change is agreed on, how long will consumers have to shift over to the new interface before it is removed?
 - Bạn sẽ nêu vấn đề rằng interface cần thay đổi như thế nào?
 - Các nhóm consumer và microservice sẽ cộng tác để thống nhất về việc thay đổi sẽ trông như thế nào ra sao?
 - Ai được kỳ vọng sẽ làm công việc cập nhật các consumer?
 - Khi thay đổi đã được thống nhất, consumer sẽ có bao lâu để chuyển sang interface mới trước khi nó bị loại bỏ?

Remember, one of the secrets to an effective microservice architecture is to embrace a consumer-first approach. Your microservices exist to be called by other consumers. The consumers' needs are paramount, and if you are making changes to a microservice that are going to cause upstream consumers problems, this needs to be taken into account.

Hãy nhớ, một trong những bí quyết cho một kiến trúc microservice hiệu quả là đón nhận một cách tiếp cận lấy consumer làm đầu (consumer-first). Các microservice của bạn tồn tại để được gọi bởi các consumer khác. Nhu cầu của consumer là tối thượng, và nếu bạn đang thực hiện các thay đổi đối với một microservice sẽ gây rắc rối cho các consumer phía trên, điều này cần được tính đến.

In some situations, of course, it might not be possible to change the consumers. I've heard from Netflix that they had issues (at least historically) with old set-top boxes using older versions of the Netflix APIs. These set-top boxes cannot be upgraded easily, so the old endpoints have to remain available unless and until the number of older set-top boxes drops to a level at which they can have their support disabled. Decisions to stop old consumers from being able to access your endpoints can sometimes end up being financial ones—how much money does it cost you to support the old interface, balanced against how much money you make from those consumers.

Trong một số tình huống, tất nhiên, có thể không thể thay đổi các consumer. Tôi nghe từ Netflix rằng họ đã có vấn đề (ít ra là trong quá khứ) với các set-top box cũ dùng các phiên bản cũ hơn của các API Netflix. Những set-top box này không thể được nâng cấp dễ dàng, nên các endpoint cũ phải còn khả dụng trừ khi và cho đến khi số lượng set-top box cũ hơn giảm xuống một mức mà tại đó việc hỗ trợ chúng có thể bị vô hiệu hóa. Các quyết định ngừng cho các consumer cũ truy cập các endpoint của bạn đôi khi có thể rất cuộc là các quyết định tài chính—việc hỗ trợ interface cũ tốn của bạn bao nhiêu tiền, cân đối với việc bạn kiếm được bao nhiêu tiền từ những consumer đó.

Tracking Usage — Theo dõi việc sử dụng

Even if you do agree on a time by which consumers should stop using the old interface, would you know if they had actually stopped using it? Making sure you have logging in place for each endpoint your microservice exposes can help, as can ensuring that you have some sort of client identifier so you can chat to the team in question if you need to work with them to get them to migrate away from your old interface. This could be something as simple as asking consumers to put their identifier in

the user-agent header when making HTTP requests, or you could require that all calls go via some sort of **API gateway** where clients need keys to identify themselves.

Ngay cả khi bạn đã thống nhất về một thời điểm mà tại đó consumer nên ngừng dùng interface cũ, liệu bạn có biết được nếu họ đã thực sự ngừng dùng nó không? Việc đảm bảo bạn có sẵn logging cho mỗi endpoint mà microservice của bạn phơi bày có thể giúp ích, cũng như việc đảm bảo bạn có một dạng định danh client (client identifier) nào đó để bạn có thể trò chuyện với nhóm đang xét nếu bạn cần làm việc với họ để đưa họ chuyển khỏi interface cũ của bạn. Điều này có thể đơn giản như việc yêu cầu consumer đặt định danh của họ vào header user-agent khi thực hiện các request HTTP, hoặc bạn có thể đòi hỏi rằng tất cả các lời gọi đi qua một dạng API gateway nào đó nơi client cần khóa (key) để định danh chính mình.

Extreme Measures — Các biện pháp cực đoan

So assuming you know that a consumer is still using an old interface that you want to remove, and they are dragging their heels about moving to the new version, what can you do about it? Well, the first thing to do is talk to them. Perhaps you can lend them a hand to make the changes happen. If all else fails, and they still don't upgrade even after agreeing to do so, there are some extreme techniques I've seen used.

Vậy giả sử bạn biết rằng một consumer vẫn đang dùng một interface cũ mà bạn muốn loại bỏ, và họ cứ chần chừ trong việc chuyển sang phiên bản mới, bạn có thể làm gì về điều đó? Vâng, điều đầu tiên cần làm là nói chuyện với họ. Có lẽ bạn có thể giúp họ một tay để biến các thay đổi thành hiện thực. Nếu mọi nỗ lực khác đều thất bại, và họ vẫn không nâng cấp ngay cả sau khi đã đồng ý làm vậy, có một số kỹ thuật cực đoan mà tôi đã thấy được dùng.

At one large tech company, we discussed how it handled this issue. Internally, the company had a very generous period of one year before old interfaces would be retired. I asked how it knew if consumers were still using the old interfaces, and the company replied that it didn't really bother tracking that information; after one year it just turned the old interface off. It was recognized internally that if this caused a consumer to break, it was the fault of the consuming microservice's team—they'd had a year to make the change and hadn't done so. Of course, this approach won't work for many (I said it was extreme!). It also leads to a large degree of inefficiency. By not knowing if the old interface was used, the company denied itself the opportunity to remove it before the year had passed. Personally, even if I was to suggest just turning the endpoint off after a certain period of time, I'd still definitely want tracking of who was going to be impacted.

Tại một công ty công nghệ lớn, chúng tôi đã thảo luận về cách họ xử lý vấn đề này. Trong nội bộ, công ty có một khoảng thời gian rất hào phóng là một năm trước khi các interface cũ bị loại bỏ. Tôi hỏi làm sao họ biết liệu consumer có còn đang dùng các interface cũ hay không, và công ty trả lời rằng họ thực sự không bận tâm theo dõi thông tin đó; sau một năm họ chỉ tắt interface cũ. Trong nội bộ người ta thừa nhận rằng nếu điều này khiến một consumer bị phá vỡ, thì đó là lỗi của nhóm microservice tiêu thụ—they đã có một năm để thực hiện thay đổi và đã không làm. Tất nhiên, cách tiếp cận này sẽ không hiệu quả với nhiều người (tôi đã nói nó cực đoan mà!). Nó cũng dẫn đến một mức độ kém hiệu quả lớn. Bằng việc không biết liệu interface cũ có được dùng hay không, công ty đã tự tước đi cơ hội loại bỏ nó trước khi một năm trôi qua. Về phần cá nhân, ngay cả khi tôi đề nghị chỉ tắt endpoint sau một khoảng thời gian nhất định, tôi vẫn chắc chắn sẽ muốn theo dõi xem ai sẽ bị tác động.

Another extreme measure I saw was actually in the context of deprecating libraries, but it could

also theoretically be used for microservice endpoints. The example given was of an old library that people were trying to retire from use inside the organization in favor of a newer, better one. Despite lots of work to move code over to use the new library, some teams were still dragging their heels. The solution was to insert a sleep in the old library so that it responded more slowly to calls (with logging to show what was happening). Over time, the team driving the deprecation just kept increasing the duration of the sleep, until eventually the other teams got the message. You obviously have to be extremely sure that you've exhausted other reasonable efforts to get consumers to upgrade before considering something like this!

Một biện pháp cực đoan khác mà tôi thấy thực ra là trong bối cảnh khai tử (deprecate) các thư viện, nhưng về lý thuyết nó cũng có thể được dùng cho các endpoint microservice. Ví dụ được đưa ra là về một thư viện cũ mà người ta đang cố loại bỏ khỏi việc sử dụng trong tổ chức để ủng hộ một thư viện mới hơn, tốt hơn. Bất chấp rất nhiều công sức để chuyển mã sang dùng thư viện mới, một số nhóm vẫn cứ chần chừ. Giải pháp là chèn một lệnh sleep vào thư viện cũ để nó phản hồi chậm hơn với các lời gọi (kèm logging để cho thấy điều gì đang xảy ra). Theo thời gian, nhóm thúc đẩy việc khai tử cứ tiếp tục tăng thời lượng của lệnh sleep, cho đến khi cuối cùng các nhóm khác hiểu ra thông điệp. Rõ ràng bạn phải cực kỳ chắc chắn rằng bạn đã cạn kiệt các nỗ lực hợp lý khác để đưa consumer nâng cấp trước khi cân nhắc một thứ như thế này!

DRY and the Perils of Code Reuse in a Microservice World — DRY và những hiểm họa của tái sử dụng mã trong thế giới Microservice

One of the acronyms we developers hear a lot is DRY: don't repeat yourself. Though its definition is sometimes simplified as trying to avoid duplicating code, DRY more accurately means that we want to avoid duplicating our system behavior and knowledge. This is very sensible advice in general. Having lots of lines of code that do the same thing makes your codebase larger than needed and therefore harder to reason about. When you want to change behavior, and that behavior is duplicated in many parts of your system, it is easy to forget everywhere you need to make a change, which can lead to bugs. So using DRY as a mantra in general makes sense.

Một trong những từ viết tắt mà cánh lập trình viên chúng ta nghe rất nhiều là DRY: don't repeat yourself (đừng lặp lại chính mình). Mặc dù định nghĩa của nó đôi khi được đơn giản hóa thành cố gắng tránh lặp lại mã, DRY thực ra chính xác hơn có nghĩa là ta muốn tránh lặp lại hành vi và kiến thức của hệ thống. Đây là lời khuyên rất hợp lý nói chung. Việc có nhiều dòng mã làm cùng một việc khiến codebase của bạn lớn hơn cần thiết và do đó khó suy luận hơn. Khi bạn muốn thay đổi hành vi, và hành vi đó bị lặp lại ở nhiều phần của hệ thống, rất dễ quên đi mọi nơi bạn cần thực hiện thay đổi, điều này có thể dẫn đến lỗi. Vậy nên việc dùng DRY như một câu thần chú nói chung là hợp lý.

DRY is what leads us to create code that can be reused. We pull duplicated code into abstractions that we can then call from multiple places. Perhaps we go as far as making a shared library that we can use everywhere! It turns out, though, that sharing code in a microservice environment is a bit more involved than that. As always, we have more than one option to consider.

DRY là cái dẫn ta tới việc tạo ra mã có thể tái sử dụng. Ta rút mã bị lặp vào các lớp trừu tượng mà sau đó ta có thể gọi từ nhiều nơi. Có lẽ ta đi xa đến mức tạo một thư viện dùng chung mà ta có thể dùng ở khắp mọi nơi! Tuy nhiên, hóa ra việc chia sẻ mã trong một môi trường microservice lại phức tạp hơn thế một chút. Như mọi khi, ta có nhiều hơn một lựa

chọn để cân nhắc.

Sharing Code via Libraries — Chia sẻ mã qua thư viện

One thing we want to avoid at all costs is overly coupling a microservice and consumers such that any small change to the microservice itself can cause unnecessary changes to the consumer. Sometimes, however, the use of shared code can create this very coupling. For example, at one client we had a library of common domain objects that represented the core entities in use in our system. This library was used by all the services we had. But when a change was made to one of them, all services had to be updated. Our system communicated via message queues, which also had to be drained of their now invalid contents, and woe betide you if you forgot.

Một điều ta muốn tránh bằng mọi giá là việc coupling quá mức giữa một microservice và các consumer sao cho bất kỳ thay đổi nhỏ nào đối với bản thân microservice cũng có thể gây ra những thay đổi không cần thiết cho consumer. Tuy nhiên, đôi khi việc dùng mã dùng chung có thể tạo ra chính cái coupling này. Ví dụ, tại một khách hàng, chúng tôi có một thư viện các đối tượng miền (domain object) chung biểu diễn các thực thể cốt lõi đang được dùng trong hệ thống của chúng tôi. Thư viện này được dùng bởi tất cả các dịch vụ mà chúng tôi có. Nhưng khi một thay đổi được thực hiện đối với một trong số chúng, tất cả các dịch vụ đều phải được cập nhật. Hệ thống của chúng tôi giao tiếp qua các message queue, vốn cũng phải được xả sạch nội dung giờ đã không còn hợp lệ của chúng, và khôn khổ cho bạn nếu bạn quên.

If your use of shared code ever leaks outside your service boundary, you have introduced a potential form of coupling. Using common code like logging libraries is fine, as they are internal concepts that are invisible to the outside world. The website realestate.com.au makes use of a tailored service template to help bootstrap new service creation. Rather than make this code shared, the company copies it for every new service to ensure that coupling doesn't leak in.

Nếu việc dùng mã dùng chung của bạn từng rò rỉ ra ngoài ranh giới dịch vụ của bạn, thì bạn đã đưa vào một dạng coupling tiềm tàng. Việc dùng mã chung như các thư viện logging thì không sao, vì chúng là các khái niệm nội bộ vô hình với thế giới bên ngoài. Website realestate.com.au dùng một service template được điều chỉnh riêng để giúp khởi tạo việc tạo dịch vụ mới. Thay vì làm cho mã này dùng chung, công ty sao chép nó cho mỗi dịch vụ mới để đảm bảo rằng coupling không bị rò rỉ vào.

The really important point about sharing code via libraries is that you cannot update all uses of the library at once. Although multiple microservices might all use the same library, they do so typically by packaging that library into the microservice deployment. To upgrade the version of the library being used, you'd therefore need to redeploy the microservice. If you want to update the same library everywhere at exactly the same time, it could lead to a widespread deployment of multiple different microservices all at the same time, with all the associated headaches.

Điểm thực sự quan trọng về việc chia sẻ mã qua thư viện là bạn không thể cập nhật tất cả các nơi dùng thư viện cùng một lúc. Mặc dù nhiều microservice có thể đều dùng cùng một thư viện, chúng thường làm vậy bằng cách đóng gói thư viện đó vào bản triển khai microservice. Do đó, để nâng cấp phiên bản của thư viện đang dùng, bạn sẽ cần triển khai lại microservice. Nếu bạn muốn cập nhật cùng một thư viện ở mọi nơi vào đúng cùng một thời điểm, nó có thể dẫn đến một bản triển khai trên diện rộng của nhiều microservice khác nhau, tất cả cùng một lúc, với tất cả những phiền toái liên quan.

So if you are using libraries for code reuse across microservice boundaries, you have to accept that

multiple different versions of the same library might be out there at the same time. You can of course look to update all of these to the last version over time, but as long as you are OK with this fact, then by all means reuse code via libraries. If you really do need to update that code for all users at exactly the same time, then you'll actually want to look at reusing code via a dedicated microservice instead.

Vậy nên nếu bạn đang dùng thư viện để tái sử dụng mã qua các ranh giới microservice, bạn phải chấp nhận rằng nhiều phiên bản khác nhau của cùng một thư viện có thể đang tồn tại ngoài kia cùng một lúc. Tất nhiên bạn có thể hướng tới việc cập nhật tất cả chúng lên phiên bản cuối theo thời gian, nhưng miễn là bạn ổn với sự thật này, thì cứ thoải mái tái sử dụng mã qua thư viện. Nếu bạn thực sự cần cập nhật mã đó cho tất cả người dùng vào đúng cùng một thời điểm, thì bạn thực ra sẽ muốn xem xét việc tái sử dụng mã qua một microservice chuyên dụng thay vào đó.

There is one specific use case associated with reuse through libraries that is worth exploring further, though.

Tuy nhiên, có một trường hợp sử dụng cụ thể liên quan đến việc tái sử dụng qua thư viện đáng được khám phá thêm.

Client libraries

Client library

I've spoken to more than one team that has insisted that creating client libraries for your services is an essential part of creating services in the first place. The argument is that this makes it easy to use your service and avoids the duplication of code required to consume the service itself.

Tôi đã nói chuyện với hơn một nhóm khẳng định rằng việc tạo client library cho các dịch vụ của bạn là một phần thiết yếu của việc tạo dịch vụ ngay từ đầu. Lập luận là điều này làm cho việc dùng dịch vụ của bạn dễ dàng và tránh được sự lặp lại mã cần thiết để tiêu thụ chính dịch vụ đó.

The problem, of course, is that if the same people create both the server API and the client API, there is a danger that logic that should exist on the server will start leaking into the client. I should know: I've done this myself. The more logic that creeps into the client library, the more **cohesion** starts to break down, and you find yourself having to change multiple clients to roll out fixes to your server. You also limit technology choices, especially if you mandate that the client library has to be used.

Vấn đề, tất nhiên, là nếu cùng những người tạo ra cả API server lẫn API client, thì có nguy cơ logic vốn nên tồn tại trên server sẽ bắt đầu rò rỉ vào client. Tôi nên biết điều này: tôi đã tự mình làm vậy. Càng nhiều logic len lỏi vào client library, thì cohesion (tính gắn kết) càng bắt đầu sụp đổ, và bạn thấy mình phải thay đổi nhiều client để triển khai các bản sửa lỗi cho server của bạn. Bạn cũng giới hạn các lựa chọn công nghệ, đặc biệt nếu bạn bắt buộc rằng client library phải được dùng.

A model I like for client libraries is the one for Amazon Web Services (AWS). The underlying SOAP or REST web service calls can be made directly, but everyone ends up using just one of the various existing software development kits (SDKs), which provide abstractions over the underlying API. These SDKs, though, are written by the wider community, or else by people inside AWS other than those who work on the API itself. This degree of separation seems to work and avoids some of the pitfalls of client libraries. Part of the reason this works so well is that the client is in charge of when the upgrade happens. If you go down the path of client libraries yourself, make sure this is the case.

Một mô hình mà tôi thích cho client library là mô hình của Amazon Web Services (AWS). Các lời gọi web service SOAP hoặc REST nền tảng có thể được thực hiện trực tiếp, nhưng mọi người rốt cuộc chỉ dùng một trong nhiều bộ phát triển phần mềm (software

development kit - SDK) hiện có, vốn cung cấp các lớp trừu tượng trên API nền tảng. Tuy nhiên, các SDK này được viết bởi cộng đồng rộng hơn, hoặc bởi những người bên trong AWS khác với những người làm việc trên chính API. Mức độ tách biệt này có vẻ hiệu quả và tránh được một số cạm bẫy của client library. Một phần lý do điều này hiệu quả đến vậy là client là bên kiểm soát khi nào việc nâng cấp diễn ra. Nếu bạn tự đi theo con đường client library, hãy đảm bảo điều này đúng.

Netflix in particular places special emphasis on the client library, but I worry that people view that purely through the lens of avoiding code duplication. In fact, the client libraries used by Netflix are as much if not more about ensuring reliability and scalability of their systems. The Netflix client libraries handle **service discovery**, failure modes, logging, and other aspects that aren't actually about the nature of the service itself. Without these shared clients, it would be hard to ensure that each piece of client/server communications behaved well at the massive scale at which Netflix operates. Their use at Netflix has certainly made it easy to get up and running and increase productivity while also ensuring the system behaves well. However, according to at least one person at Netflix, over time this has led to a degree of coupling between client and server that has been problematic.

Netflix nói riêng đặt sự nhấn mạnh đặc biệt vào client library, nhưng tôi lo rằng người ta nhìn nhận điều đó hoàn toàn qua lăng kính tránh sự lặp lại mã. Thực tế, các client library được Netflix dùng cũng nhằm mục đích đảm bảo độ tin cậy và khả năng mở rộng của các hệ thống của họ không kém, nếu không muốn nói là nhiều hơn. Các client library của Netflix xử lý service discovery, các chế độ lỗi (failure mode), logging và các khía cạnh khác vốn thực ra không phải là về bản chất của chính dịch vụ. Nếu không có các client dùng chung này, sẽ khó để đảm bảo rằng mỗi mảnh giao tiếp client/server đều hoạt động tốt ở quy mô khổng lồ mà Netflix vận hành. Việc dùng chung tại Netflix chắc chắn đã giúp việc khởi động và chạy dễ dàng cũng như tăng năng suất, đồng thời đảm bảo hệ thống hoạt động tốt. Tuy nhiên, theo ít nhất một người tại Netflix, theo thời gian điều này đã dẫn đến một mức độ coupling giữa client và server đã trở nên rắc rối.

If the client library approach is something you're thinking about, it can be important to separate out client code to handle the underlying transport protocol, which can deal with things like service discovery and failure, from things related to the destination service itself. Decide whether or not you are going to insist on the client library being used, or if you'll allow people using different technology stacks to make calls to the underlying API. And finally, make sure that the clients are in charge of when to upgrade their client libraries: we need to ensure we maintain the ability to release our services independently of each other!

Nếu cách tiếp cận client library là điều bạn đang cân nhắc, có thể quan trọng là tách riêng mã client để xử lý giao thức truyền tải (transport) nền tảng, vốn có thể xử lý những thứ như service discovery và lỗi, khỏi những thứ liên quan đến chính dịch vụ đích. Hãy quyết định liệu bạn có khăng khăng đòi client library phải được dùng hay không, hoặc liệu bạn có cho phép những người dùng các technology stack khác nhau thực hiện các lời gọi tới API nền tảng. Và cuối cùng, hãy đảm bảo rằng các client là bên kiểm soát khi nào nâng cấp client library của họ: chúng ta cần đảm bảo duy trì khả năng phát hành các dịch vụ của mình độc lập với nhau!

Service Discovery — Service Discovery

Once you have more than a few microservices lying around, your attention inevitably turns to knowing where on earth everything is. Perhaps you want to know what is running in a given

environment so you know what you should be monitoring. Maybe it's as simple as knowing where your Accounts microservice is so that its consumers know where find it. Or perhaps you just want to make it easy for developers in your organization to know what APIs are available so they don't reinvent the wheel. Broadly speaking, all of these use cases fall under the banner of service discovery . And as always with microservices, we have quite a few different options at our disposal for dealing with it.

Một khi bạn có hơn vài microservice nằm rải rác xung quanh, sự chú ý của bạn chắc chắn sẽ chuyển sang việc biết mọi thứ rốt cuộc đang ở đâu. Có lẽ bạn muốn biết cái gì đang chạy trong một môi trường cho trước để bạn biết mình nên giám sát cái gì. Có thể đơn giản như việc biết microservice Accounts của bạn ở đâu để các consumer của nó biết tìm nó ở đâu. Hoặc có lẽ bạn chỉ muốn làm cho các lập trình viên trong tổ chức của bạn dễ dàng biết những API nào sẵn có để họ không phải tìm lại bánh xe. Nói rộng ra, tất cả những trường hợp sử dụng này đều nằm dưới ngọn cờ service discovery. Và như mọi khi với microservice, ta có khá nhiều lựa chọn khác nhau trong tầm tay để xử lý nó.

All of the solutions we'll look at handle things in two parts. First, they provide some mechanism for an instance to register itself and say, "I'm here!" Second, they provide a way to find the service once it's registered. Service discovery gets more complicated, though, when we are considering an environment in which we are constantly destroying and deploying new instances of services. Ideally, we'd want whatever solution we pick to cope with this.

Tất cả các giải pháp ta sẽ xem xét đều xử lý mọi thứ trong hai phần. Thứ nhất, chúng cung cấp một cơ chế nào đó để một instance tự đăng ký chính mình và nói, "Tôi ở đây!" Thứ hai, chúng cung cấp một cách để tìm dịch vụ một khi nó đã được đăng ký. Tuy nhiên, service discovery trở nên phức tạp hơn khi ta đang xét một môi trường trong đó ta liên tục hủy và triển khai các instance mới của các dịch vụ. Lý tưởng nhất, ta sẽ muốn bất kỳ giải pháp nào ta chọn cũng đối phó được với điều này.

Let's look at some of the most common solutions to service delivery and consider our options.

Hãy xem một số giải pháp phổ biến nhất cho service discovery và cân nhắc các lựa chọn của chúng ta.

Domain Name System (DNS) — Domain Name System (DNS)

It's nice to start simple. **DNS** lets us associate a name with the IP address of one or more machines. We could decide, for example, that our Accounts microservice is always found at `accounts.musiccorp.net` . We would then have that entry point to the IP address of the host running that microservice, or perhaps we'd have it resolve to a load balancer that is distributing load across a number of instances. This means we'd have to handle updating these entries as part of deploying our service.

Thật hay khi bắt đầu một cách đơn giản. DNS cho phép ta liên kết một tên với địa chỉ IP của một hoặc nhiều máy. Ví dụ, ta có thể quyết định rằng microservice Accounts của ta luôn được tìm thấy tại `accounts.musiccorp.net`. Sau đó ta sẽ trở mục đó tới địa chỉ IP của host chạy microservice đó, hoặc có lẽ ta sẽ cho nó phân giải (resolve) tới một load balancer đang phân phối tải trên một số instance. Điều này nghĩa là ta sẽ phải xử lý việc cập nhật các mục này như một phần của việc triển khai dịch vụ của mình.

When dealing with instances of a service in different environments, I have seen a convention-based domain template work well. For example, we might have a template defined as `-.musiccorp.net` , giving us entries like `accounts-uat.musiccorp.net` or `accounts-dev.musiccorp.net` .

Khi xử lý các instance của một dịch vụ trong các môi trường khác nhau, tôi đã thấy một mẫu tên miền dựa trên quy ước hoạt động tốt. Ví dụ, ta có thể có một mẫu được định nghĩa là `-.musiccorp.net`, cho ta các mục như `accounts-uat.musiccorp.net` hoặc `accounts-dev.musiccorp.net`.

A more advanced way of handling different environments is to have different domain name servers for these environments. So I could assume that `accounts.musiccorp.net` is where I always find the Accounts microservice, but it could resolve to different hosts depending on where I do the lookup. If you already have your environments sitting in different network segments and are comfortable with managing your own DNS servers and entries, this could be quite a neat solution, but it is a lot of work if you aren't getting other benefits from this setup.

Một cách xử lý các môi trường khác nhau tinh vi hơn là có các máy chủ tên miền (domain name server) khác nhau cho các môi trường này. Vậy nên tôi có thể giả định rằng `accounts.musiccorp.net` là nơi tôi luôn tìm thấy microservice Accounts, nhưng nó có thể phân giải tới các host khác nhau tùy vào nơi tôi thực hiện việc tra cứu. Nếu bạn đã có các môi trường của mình nằm trong các phân đoạn mạng khác nhau và thoải mái với việc quản lý các máy chủ DNS và các mục của riêng mình, thì đây có thể là một giải pháp khá gọn gàng, nhưng nó là rất nhiều công sức nếu bạn không thu được các lợi ích khác từ thiết lập này.

DNS has a host of advantages, the main one being it is such a well-understood and well-used standard that almost any technology stack will support it. Unfortunately, while a number of services exist for managing DNS inside an organization, few of them seem designed for an environment in which we are dealing with highly disposable hosts, making updating DNS entries somewhat painful. Amazon's Route 53 service does a pretty good job of this, but I haven't seen a self-hosted option that is as good yet, although (as we'll discuss shortly) some dedicated service discovery tools like Consul may help us here. Aside from the problems in updating DNS entries, the DNS specification itself can cause us some issues.

DNS có một loạt lợi thế, lợi thế chính là nó là một tiêu chuẩn được hiểu rõ và được dùng nhiều đến mức hầu như bất kỳ technology stack nào cũng sẽ hỗ trợ nó. Tiếc thay, mặc dù có một số dịch vụ tồn tại để quản lý DNS bên trong một tổ chức, ít trong số chúng có vẻ được thiết kế cho một môi trường trong đó ta đang đối phó với các host có tính dùng-rồi-bỏ (disposable) cao, khiến việc cập nhật các mục DNS có phần đau đầu. Dịch vụ Route 53 của Amazon làm khá tốt việc này, nhưng tôi chưa thấy một tùy chọn tự lưu trữ (self-hosted) nào tốt bằng, mặc dù (như ta sẽ bàn sớm) một số công cụ service discovery chuyên dụng như Consul có thể giúp ta ở đây. Ngoài các vấn đề trong việc cập nhật các mục DNS, bản thân đặc tả DNS cũng có thể gây cho ta một số vấn đề.

DNS entries for domain names have a time to live (TTL). This is how long a client can consider the entry fresh. When we want to change the host to which the domain name refers, we update that entry, but we have to assume that clients will be holding on to the old IP for at least as long as the TTL states. DNS entries can get cached in multiple places (even the JVM will cache DNS entries unless you tell it not to), and the more places they are cached in, the more stale the entry can be.

Các mục DNS cho tên miền có một thời gian sống (time to live - TTL). Đây là khoảng thời gian mà một client có thể coi mục là còn tươi mới (fresh). Khi ta muốn thay đổi host mà tên miền tham chiếu tới, ta cập nhật mục đó, nhưng ta phải giả định rằng client sẽ giữ lấy IP cũ ít nhất lâu bằng những gì TTL nói. Các mục DNS có thể bị cache ở nhiều nơi (ngay cả JVM cũng sẽ cache các mục DNS trừ khi bạn bảo nó đừng làm vậy), và càng nhiều nơi chúng bị cache, thì mục đó càng có thể cũ kỹ.

One way to work around this problem is to have the domain name entry for your service point to a

load balancer, which in turn points to the instances of your service, as shown in Figure 5-5 . When you deploy a new instance, you can take the old one out of the load-balancer entry and add the new one. Some people use DNS round-robin, where the DNS entries themselves refer to a group of machines. This technique is extremely problematic, as the client is hidden from the underlying host and therefore cannot easily stop routing traffic to one of the hosts should it become sick.

Một cách để khắc phục vấn đề này là cho mục tên miền của dịch vụ của bạn trỏ tới một load balancer, mà đến lượt nó trỏ tới các instance của dịch vụ của bạn, như được thể hiện trong Hình 5-5. Khi bạn triển khai một instance mới, bạn có thể đưa instance cũ ra khỏi mục của load balancer và thêm instance mới vào. Một số người dùng DNS round-robin, trong đó chính các mục DNS tham chiếu tới một nhóm máy. Kỹ thuật này cực kỳ rắc rối, vì client bị ẩn khỏi host nền tảng và do đó không thể dễ dàng ngừng định tuyến lưu lượng tới một trong các host nếu nó trở nên ốm yếu.

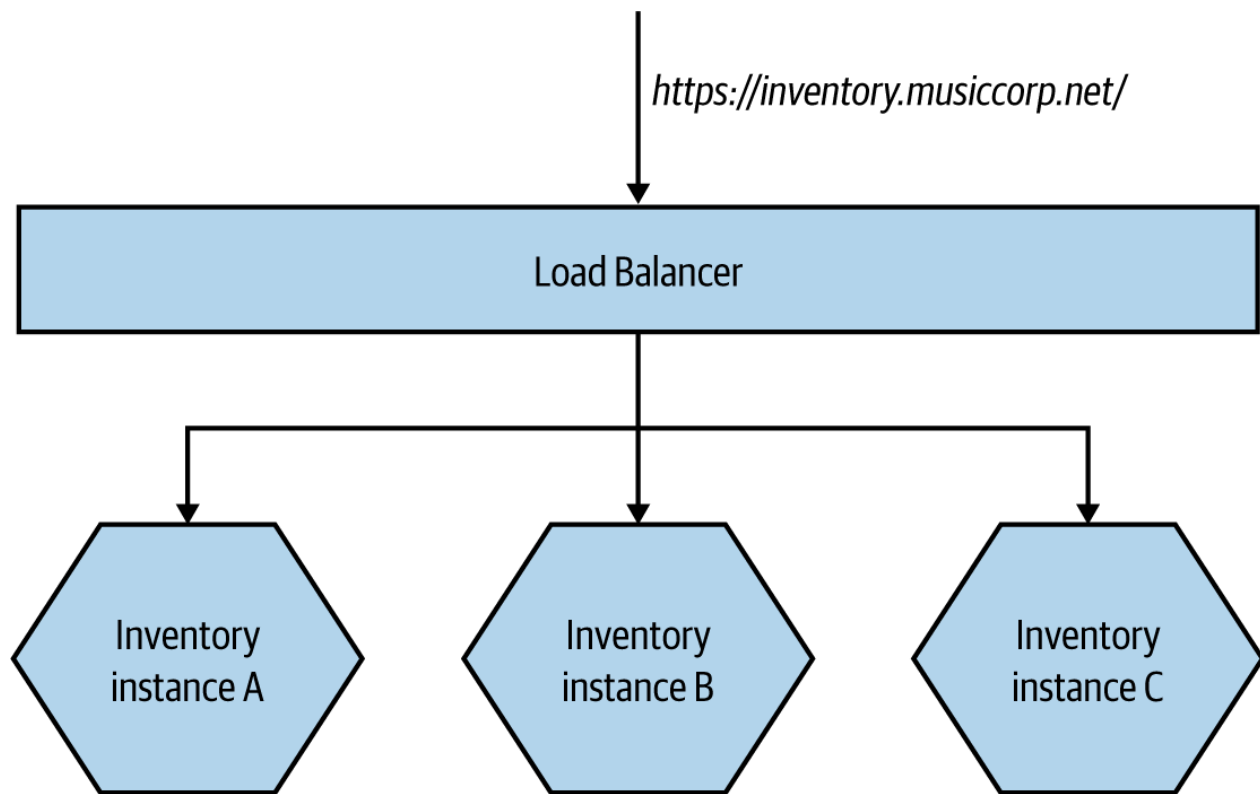


Figure 5-5. Using DNS to resolve to a load balancer to avoid stale DNS entries

Hình 5-5. Dùng DNS để phân giải tới một load balancer nhằm tránh các mục DNS cũ kỹ

As mentioned, DNS is well understood and widely supported. But it does have one or two downsides. I would suggest you investigate whether it is a good fit for you before picking something more complex. For a situation in which you have only single nodes, having DNS refer directly to hosts is probably fine. But for those situations in which you need more than one instance of a host, have DNS entries resolve to load balancers that can handle putting individual hosts into and out of service as appropriate.

Như đã đề cập, DNS được hiểu rõ và được hỗ trợ rộng rãi. Nhưng nó có một hoặc hai nhược điểm. Tôi sẽ đề nghị bạn điều tra xem liệu nó có phù hợp với bạn không trước khi chọn một thứ phức tạp hơn. Đối với tình huống mà bạn chỉ có các node đơn lẻ, việc cho DNS tham chiếu trực tiếp tới các host có lẽ là ổn. Nhưng đối với những tình huống mà bạn cần

hơn một instance của một host, hãy cho các mục DNS phân giải tới các load balancer có thể xử lý việc đưa các host riêng lẻ vào và ra khỏi dịch vụ một cách phù hợp.

Dynamic Service Registries — Các sổ đăng ký dịch vụ động (Dynamic Service Registries)

The downsides of DNS as a way of finding nodes in a highly dynamic environment have led to a number of alternative systems, most of which involve the service registering itself with some central registry, which in turn offers the ability to look up these services later on. Often, these systems do more than just providing service registration and discovery, which may or may not be a good thing. This is a crowded field, so we'll just look at a few options to give you a sense of what is available.

Các nhược điểm của DNS như một cách tìm node trong một môi trường có tính động cao đã dẫn đến một số hệ thống thay thế, hầu hết trong số đó liên quan đến việc dịch vụ tự đăng ký với một sổ đăng ký (registry) trung tâm nào đó, mà đến lượt nó cung cấp khả năng tra cứu các dịch vụ này về sau. Thường thì các hệ thống này làm nhiều hơn chỉ việc cung cấp đăng ký và discovery dịch vụ, điều có thể tốt hoặc không. Đây là một lĩnh vực đông đúc, nên ta sẽ chỉ xem một vài lựa chọn để cho bạn cảm nhận về những gì sẵn có.

ZooKeeper

ZooKeeper

ZooKeeper was originally developed as part of the Hadoop project. It is used for an almost bewildering array of use cases, including configuration management, synchronizing data between services, leader election, message queues, and (usefully for us) as a naming service.

ZooKeeper ban đầu được phát triển như một phần của dự án Hadoop. Nó được dùng cho một loạt trường hợp sử dụng gần như choáng ngợp, bao gồm quản lý cấu hình, đồng bộ hóa dữ liệu giữa các dịch vụ, bầu chọn lãnh đạo (leader election), message queue, và (hữu ích cho ta) như một dịch vụ đặt tên (naming service).

Like many similar types of systems, ZooKeeper relies on running a number of nodes in a cluster to provide various guarantees. This means you should expect to be running at least three Zookeeper nodes. Most of the smarts in ZooKeeper are around ensuring that data is replicated safely between these nodes, and that things remain consistent when nodes fail.

Như nhiều loại hệ thống tương tự, ZooKeeper dựa vào việc chạy một số node trong một cụm để cung cấp các đảm bảo khác nhau. Điều này nghĩa là bạn nên trông đợi phải chạy ít nhất ba node Zookeeper. Hầu hết sự thông minh trong ZooKeeper là xoay quanh việc đảm bảo rằng dữ liệu được sao chép an toàn giữa các node này, và rằng mọi thứ vẫn nhất quán khi các node bị lỗi.

At its heart, ZooKeeper provides a hierarchical namespace for storing information. Clients can insert new nodes in this hierarchy, change them, or query them. Furthermore, they can add watches to nodes to be told when they change. This means we could store the information about where our services are located in this structure and as a client be told when they change. ZooKeeper is often used as a general configuration store, so you could also store service-specific configuration in it, allowing you to do tasks like dynamically changing log levels or turning off features of a running system.

Về cốt lõi, ZooKeeper cung cấp một không gian tên (namespace) phân cấp để lưu trữ thông tin. Client có thể chèn các node mới vào hệ phân cấp này, thay đổi chúng, hoặc truy vấn chúng. Hơn nữa, chúng có thể thêm các “watch” vào các node để được báo khi chúng thay đổi. Điều này nghĩa là ta có thể lưu trữ thông tin về việc các dịch vụ của ta nằm ở

đầu trong cấu trúc này và với tư cách một client được báo khi chúng thay đổi. ZooKeeper thường được dùng như một kho cấu hình chung, nên bạn cũng có thể lưu trữ cấu hình đặc thù theo dịch vụ trong nó, cho phép bạn làm các tác vụ như thay đổi mức log một cách động hoặc tắt các tính năng của một hệ thống đang chạy.

In reality, better solutions exist for dynamic service registration, to the extent that I would actively avoid ZooKeeper for this use case nowadays.

Trên thực tế, có những giải pháp tốt hơn tồn tại cho việc đăng ký dịch vụ động, đến mức ngày nay tôi sẽ chủ động tránh ZooKeeper cho trường hợp sử dụng này.

Consul

Consul

Like ZooKeeper, Consul supports both configuration management and service discovery. But it goes further than ZooKeeper in providing more support for these key use cases. For example, it exposes an HTTP interface for service discovery, and one of Consul's killer features is that it actually provides a DNS server out of the box; specifically, it can serve SRV records, which give you both an IP and a port for a given name. This means if part of your system uses DNS already and can support SRV records, you can just drop in Consul and start using it without any changes to your existing system.

Như ZooKeeper, Consul hỗ trợ cả quản lý cấu hình lẫn service discovery. Nhưng nó đi xa hơn ZooKeeper trong việc cung cấp nhiều hỗ trợ hơn cho các trường hợp sử dụng chủ chốt này. Ví dụ, nó phơi bày một interface HTTP cho service discovery, và một trong những tính năng đỉnh cao của Consul là nó thực sự cung cấp một máy chủ DNS ngay khi cài đặt; cụ thể, nó có thể phục vụ các bản ghi SRV, vốn cho bạn cả một IP và một cổng cho một tên cho trước. Điều này nghĩa là nếu một phần hệ thống của bạn đã dùng DNS rồi và có thể hỗ trợ các bản ghi SRV, bạn có thể chỉ cần thả Consul vào và bắt đầu dùng nó mà không cần bất kỳ thay đổi nào đối với hệ thống hiện có của bạn.

Consul also builds in other capabilities that you might find useful, such as the ability to perform health checks on nodes. Thus Consul could well overlap the capabilities provided by other dedicated monitoring tools, although you would more likely use Consul as a source of this information and then pull it into a more comprehensive monitoring setup.

Consul cũng tích hợp sẵn các khả năng khác mà bạn có thể thấy hữu ích, chẳng hạn như khả năng thực hiện kiểm tra sức khỏe (health check) trên các node. Như vậy Consul có thể chồng lấn các khả năng được cung cấp bởi các công cụ giám sát chuyên dụng khác, mặc dù nhiều khả năng bạn sẽ dùng Consul như một nguồn của thông tin này rồi kéo nó vào một thiết lập giám sát toàn diện hơn.

Consul uses a RESTful HTTP interface for everything from registering a service to querying the key/value store or inserting health checks. This makes integration with different technology stacks very straightforward. Consul also has a suite of tools that work well with it, further improving its usefulness. One example is `consul-template`, which provides a way to update text files based on entries in Consul. At first glance, this doesn't seem that interesting, until you consider the fact that with `consul-template` you can now change a value in Consul—perhaps the location of a microservice, or a configuration value—and have configuration files all across your system dynamically update. Suddenly, any program that reads its configuration from a text file can have its text files updated dynamically without needing to know anything about Consul itself. A great use case for this would be dynamically adding or removing nodes to a load balancer pool using a software load balancer like HAProxy.

Consul dùng một interface HTTP kiểu RESTful cho mọi thứ từ việc đăng ký một dịch vụ

đến việc truy vấn kho key/value hay chèn các health check. Điều này khiến việc tích hợp với các technology stack khác nhau rất đơn giản. Consul cũng có một bộ công cụ hoạt động tốt với nó, càng cải thiện tính hữu ích của nó. Một ví dụ là consul-template, vốn cung cấp một cách để cập nhật các tệp văn bản dựa trên các mục trong Consul. Thoạt nhìn, điều này có vẻ không thú vị lắm, cho đến khi bạn xét đến sự thật rằng với consul-template giờ bạn có thể thay đổi một giá trị trong Consul—có lẽ là vị trí của một microservice, hoặc một giá trị cấu hình—và khiến các tệp cấu hình trên khắp hệ thống của bạn cập nhật một cách động. Đột nhiên, bất kỳ chương trình nào đọc cấu hình của nó từ một tệp văn bản đều có thể có các tệp văn bản của nó được cập nhật động mà không cần biết gì về chính Consul. Một trường hợp sử dụng tuyệt vời cho điều này sẽ là việc thêm hoặc xóa node một cách động vào một nhóm load balancer dùng một load balancer phần mềm như HAProxy.

Another tool that integrates well with Consul is Vault, a secrets management tool we'll revisit in “Secrets” on page 356. Secrets management can be painful, but the combination of Consul and Vault can certainly make life easier.

Một công cụ khác tích hợp tốt với Consul là Vault, một công cụ quản lý bí mật (secrets management) mà ta sẽ trở lại trong “Bí mật” ở trang 356. Quản lý bí mật có thể đau đầu, nhưng sự kết hợp của Consul và Vault chắc chắn có thể làm cho cuộc sống dễ dàng hơn.

etcd and **Kubernetes**

etcd và Kubernetes

If you're running on a platform that manages **container** workloads for you, chances are you already have a service discovery mechanism provided for you. Kubernetes is no different, and it comes partly from etcd, a configuration management store bundled with Kubernetes. etcd has capabilities similar to those of Consul, and Kubernetes uses it for managing a wide array of configuration information.

Nếu bạn đang chạy trên một nền tảng quản lý các tải công việc (workload) container giúp bạn, nhiều khả năng bạn đã có sẵn một cơ chế service discovery được cung cấp cho bạn. Kubernetes không khác, và nó đến một phần từ etcd, một kho quản lý cấu hình đi kèm với Kubernetes. etcd có các khả năng tương tự như của Consul, và Kubernetes dùng nó để quản lý một loạt thông tin cấu hình rộng lớn.

We'll explore Kubernetes in more detail in “Kubernetes and Container Orchestration” on page 259, but in a nutshell, the way service discovery works on Kubernetes is that you deploy a container in a pod, and then a service dynamically identifies which pods should be part of a service by pattern matching on metadata associated with the pod. It's a pretty elegant mechanism and can be very powerful. Requests to a service will then get routed to one of the pods that make up that service.

Ta sẽ khám phá Kubernetes chi tiết hơn trong “Kubernetes và Điều phối Container” ở trang 259, nhưng nói tóm gọn, cách service discovery hoạt động trên Kubernetes là bạn triển khai một container trong một pod, rồi một service xác định một cách động những pod nào nên là một phần của một service bằng cách so khớp mẫu (pattern matching) trên metadata gắn với pod. Đó là một cơ chế khá thanh lịch và có thể rất mạnh mẽ. Các request tới một service sau đó sẽ được định tuyến tới một trong các pod tạo nên service đó.

The capabilities that you get out of the box with Kubernetes may well result in you just wanting to make do with what comes with the core platform, eschewing the use of dedicated tools like Consul, and for many this makes a lot of sense, especially if the wider ecosystem of tooling around Consul isn't of interest to you. However, if you're running in a mixed environment, where you have workloads running on Kubernetes and elsewhere, then having a dedicated service discovery tool that could be used across both platforms may be the way to go.

Các khả năng mà bạn nhận được ngay khi cài đặt với Kubernetes hoàn toàn có thể dẫn đến việc bạn chỉ muốn xoay xở với những gì đi kèm nền tảng lõi, từ bỏ việc dùng các công cụ chuyên dụng như Consul, và với nhiều người điều này rất hợp lý, đặc biệt nếu hệ sinh thái công cụ rộng hơn quanh Consul không khiến bạn quan tâm. Tuy nhiên, nếu bạn đang chạy trong một môi trường hỗn hợp, nơi bạn có các tải công việc chạy trên Kubernetes và ở nơi khác, thì việc có một công cụ service discovery chuyên dụng có thể được dùng trên cả hai nền tảng có thể là hướng đi nên chọn.

Rolling your own

Tự xây dựng riêng

One approach I have used myself and seen used elsewhere is to roll your own system. On one project, we were making heavy use of AWS, which offers the ability to add tags to instances. When launching service instances, I would apply tags to help define what the instance was and what it was used for. These allowed for some rich metadata to be associated with a given host—for example:

Một cách tiếp cận mà tôi đã tự mình dùng và thấy được dùng ở nơi khác là tự xây dựng hệ thống của riêng bạn. Ở một dự án, chúng tôi đang dùng AWS rất nhiều, vốn cung cấp khả năng thêm các tag (thẻ) vào các instance. Khi khởi chạy các instance dịch vụ, tôi sẽ áp các tag để giúp định nghĩa instance là gì và nó được dùng để làm gì. Những thứ này cho phép một số metadata phong phú được gắn với một host cho trước—ví dụ:

- service = accounts
 - environment = production
 - version = 154
- service = accounts
 - environment = production
 - version = 154

I then used the AWS APIs to query all the instances associated with a given AWS account so I could find the machines I cared about. Here, AWS itself is handling the storing of the metadata associated with each instance and providing us with the ability to query it. I then built command-line tools for interacting with these instances and provided graphical interfaces to view instance status at a glance. All of this becomes quite a simple undertaking if you can programmatically gather information about the service interfaces.

Sau đó tôi dùng các API của AWS để truy vấn tất cả các instance liên kết với một tài khoản AWS cho trước để tôi có thể tìm các máy mà tôi quan tâm. Ở đây, bản thân AWS đang xử lý việc lưu trữ metadata liên kết với mỗi instance và cung cấp cho ta khả năng truy vấn nó. Sau đó tôi xây dựng các công cụ dòng lệnh để tương tác với các instance này và cung cấp các giao diện đồ họa để xem trạng thái instance trong nháy mắt. Tất cả những điều này trở thành một việc khá đơn giản nếu bạn có thể thu thập thông tin về các interface dịch vụ một cách lập trình được.

The last time I did this we didn't go as far as having services use the AWS APIs to find their service dependencies, but there is no reason why you couldn't. Obviously, if you want upstream services to be alerted when the location of a downstream service changes, you're on your own.

Lần gần nhất tôi làm điều này chúng tôi đã không đi xa tới mức cho các dịch vụ dùng các API của AWS để tìm các phụ thuộc dịch vụ của chúng, nhưng không có lý do gì bạn không thể làm vậy. Rõ ràng, nếu bạn muốn các dịch vụ phía trên được cảnh báo khi vị trí của một dịch vụ phía dưới thay đổi, bạn phải tự lo lấy.

Nowadays, this is not the route I would go. The crop of tooling in this space is mature enough that this would be a case of not just reinventing the wheel but recreating a much worse wheel.

Ngày nay, đây không phải là con đường tôi sẽ đi. Lứa công cụ trong không gian này đã đủ trưởng thành đến mức đây sẽ là một trường hợp không chỉ phát minh lại bánh xe mà còn tái tạo một bánh xe tồi hơn nhiều.

Don't Forget the Humans! — Đừng quên con người!

The systems we've looked at so far make it easy for a service instance to register itself and look up other services it needs to talk to. But as humans we sometimes want this information too. Making the information available in ways that allow for humans to consume it, perhaps by using APIs to pull this detail into humane registries (a topic we'll look at in a moment), can be vital.

Các hệ thống ta đã xem xét cho đến giờ giúp một instance dịch vụ dễ dàng tự đăng ký và tra cứu các dịch vụ khác mà nó cần trò chuyện cùng. Nhưng với tư cách con người, đôi khi ta cũng muốn thông tin này. Việc làm cho thông tin sẵn có theo những cách cho phép con người tiêu thụ nó, có lẽ bằng cách dùng các API để kéo chi tiết này vào các humane registry (sổ đăng ký nhân văn, một chủ đề ta sẽ xem xét trong giây lát), có thể là sống còn.

Service Meshes and API Gateways — Service Mesh và API Gateway

Few areas of technology associated with microservices have had as much attention, hype, and confusion around them as that of service meshes and API gateways. Both have their place, but confusingly, they can also overlap in responsibilities. The API gateway in particular is prone to misuse (and mis-selling), so it's important that we understand how these types of technology can fit into our microservice architecture. Rather than attempt to deliver a detailed view as to what you can do with these products, I instead want to provide an overview of where they fit in, how they can help, and some pitfalls to avoid.

Ít lĩnh vực công nghệ liên quan đến microservice nhận được nhiều sự chú ý, cường điệu và bối rối quanh chúng như service mesh và API gateway. Cả hai đều có chỗ đứng của mình, nhưng một cách gây bối rối, chúng cũng có thể chồng lấn về trách nhiệm. API gateway nói riêng dễ bị lạm dụng (và bán sai), nên điều quan trọng là ta hiểu các loại công nghệ này có thể phù hợp vào kiến trúc microservice của ta như thế nào. Thay vì cố gắng đưa ra một cái nhìn chi tiết về những gì bạn có thể làm với các sản phẩm này, tôi thay vào đó muốn cung cấp một cái nhìn tổng quan về việc chúng phù hợp ở đâu, chúng có thể giúp ích như thế nào, và một số cạm bẫy cần tránh.

In typical data center speak, we'd talk about "east-west" traffic as being inside a data center, with "north-south" traffic relating to interactions that enter or leave the data center from the outside world. From the point of view of networking, what a data center is has become a somewhat blurry concept, so for our purposes we'll talk more broadly about a networked perimeter. This could relate to an entire data center, a Kubernetes cluster, or perhaps just a virtual networking concept like a group of machines running on the same virtual LAN.

Theo cách nói điển hình của trung tâm dữ liệu, ta sẽ nói về lưu lượng "đông-tây" (east-west) là lưu lượng bên trong một trung tâm dữ liệu, với lưu lượng "bắc-nam" (north-south) liên quan đến các tương tác đi vào hoặc rời khỏi trung tâm dữ liệu từ thế giới bên ngoài. Từ góc nhìn của mạng, một trung tâm dữ liệu là gì đã trở thành một khái niệm có phần mờ

nhật, nên cho mục đích của ta, ta sẽ nói rộng hơn về một vành đai được kết nối mạng (networked perimeter). Điều này có thể liên quan đến cả một trung tâm dữ liệu, một cụm Kubernetes, hoặc có lẽ chỉ là một khái niệm mạng ảo như một nhóm máy chạy trên cùng một LAN ảo.

Speaking generally, an API gateway sits on the perimeter of your system and deals with north-south traffic. Its primary concerns are managing access from the outside world to your internal microservices. A **service mesh**, on the other hand, deals very narrowly with communication between microservices inside your perimeter—the east-west traffic—as Figure 5-6 shows.

Nói chung, một API gateway nằm trên vành đai của hệ thống của bạn và xử lý lưu lượng bắc-nam. Mỗi quan tâm chính của nó là quản lý truy cập từ thế giới bên ngoài tới các microservice nội bộ của bạn. Mặt khác, một service mesh xử lý rất hẹp việc giao tiếp giữa các microservice bên trong vành đai của bạn—lưu lượng đông-tây—như Hình 5-6 cho thấy.

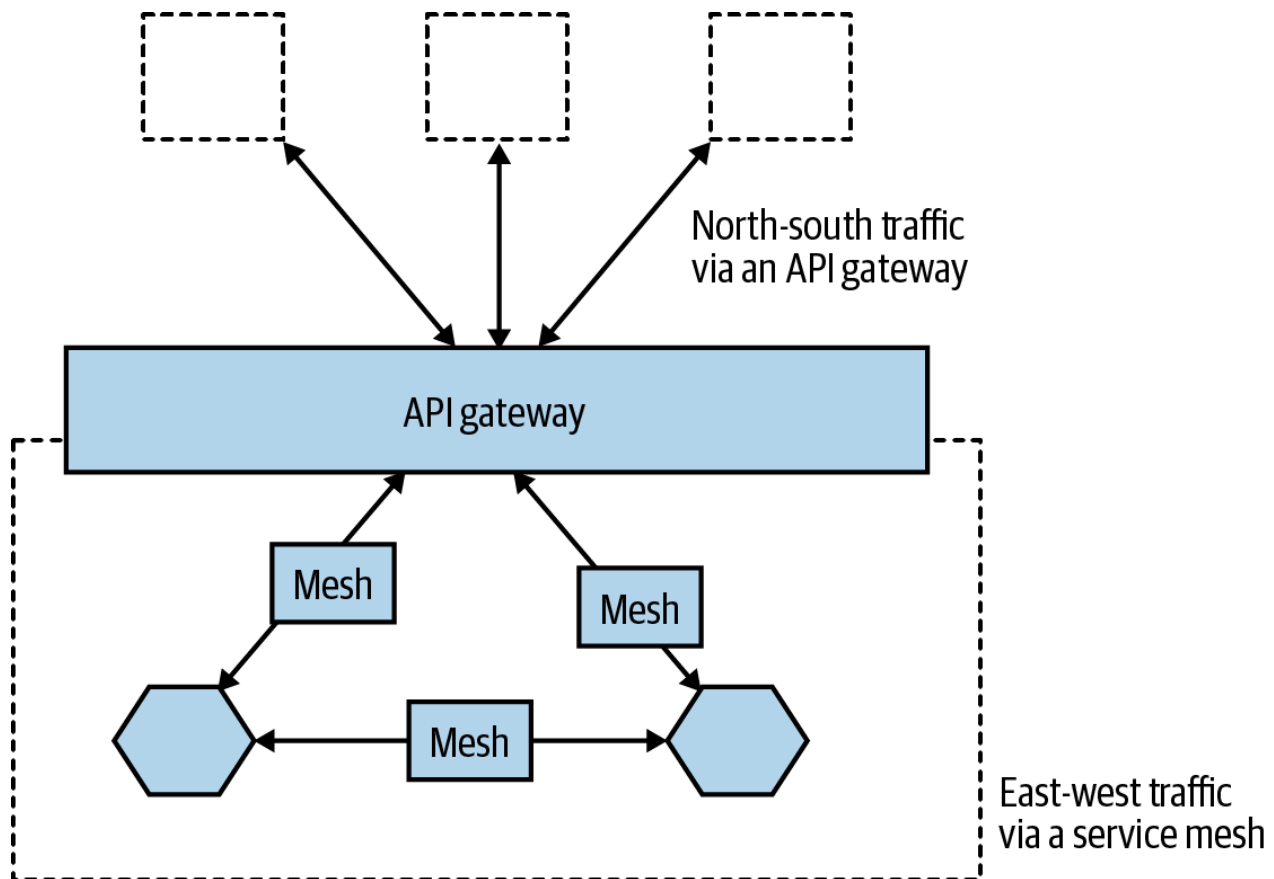


Figure 5-6. An overview of where API gateways and service meshes are used. Service meshes and API gateways can potentially allow microservices to share code without requiring the creation of new client libraries or new microservices. Put (very) simply, service meshes and API gateways can work as proxies between microservices. This can mean that they may be used to implement some microservice-agnostic behavior that might otherwise have to be done in code, such as service discovery or logging.

Hình 5-6. Một cái nhìn tổng quan về nơi API gateway và service mesh được dùng. Service mesh và API gateway có thể có khả năng cho phép các microservice chia sẻ mã mà không đòi hỏi việc tạo ra các client library mới hay các microservice mới. Nói (rất) đơn giản, service mesh và API gateway có thể hoạt động như các proxy giữa các microservice. Điều

này có thể nghĩa là chúng có thể được dùng để hiện thực một số hành vi không phụ thuộc-microservice (microservice-agnostic) mà nếu không thì có thể phải được làm trong mã, chẳng hạn như service discovery hay logging.

If you are using an API gateway or a service mesh to implement shared, common behavior for your microservices, it's essential that this behavior is totally generic—in other words, that the behavior in the proxy bears no relation to any specific behavior of an individual microservice.

Nếu bạn đang dùng một API gateway hay một service mesh để hiện thực hành vi chung, dùng chung cho các microservice của bạn, điều thiết yếu là hành vi này hoàn toàn chung chung—nói cách khác, rằng hành vi trong proxy không liên quan gì đến bất kỳ hành vi cụ thể nào của một microservice riêng lẻ.

Now, after explaining that, I also have to explain that the world isn't always as clear-cut as that. A number of API gateways attempt to provide capabilities for east-west traffic too, but that's something we'll discuss shortly. Firstly, let's look at API gateways and the sorts of things they can do.

Giờ, sau khi giải thích điều đó, tôi cũng phải giải thích rằng thế giới không phải lúc nào cũng rạch ròi như vậy. Một số API gateway cố gắng cung cấp các khả năng cho lưu lượng đông-tây nữa, nhưng đó là điều ta sẽ bàn sớm thôi. Trước tiên, hãy xem các API gateway và những loại việc chúng có thể làm.

API Gateways — API Gateway

Being focused more on north-south traffic, the API gateway's main concern in a microservices environment is mapping requests from external parties to internal microservices. This responsibility is akin to what you could achieve with a simple HTTP proxy, and in fact API gateways typically build more features on top of existing HTTP proxy products, and they largely function as reverse proxies. In addition, API gateways can be used to implement mechanisms like API keys for external parties, logging, rate limiting, and the like. Some API gateway products will also provide developer portals, often targeted at external consumers.

Vì tập trung nhiều hơn vào lưu lượng bắc-nam, mối quan tâm chính của API gateway trong một môi trường microservice là ánh xạ các request từ các bên bên ngoài tới các microservice nội bộ. Trách nhiệm này tương tự với những gì bạn có thể đạt được với một HTTP proxy đơn giản, và thực tế API gateway thường xây dựng thêm tính năng trên các sản phẩm HTTP proxy hiện có, và chúng phần lớn hoạt động như các reverse proxy. Ngoài ra, API gateway có thể được dùng để hiện thực các cơ chế như khóa API (API key) cho các bên bên ngoài, logging, giới hạn tốc độ (rate limiting), và những thứ tương tự. Một số sản phẩm API gateway cũng sẽ cung cấp các cổng thông tin dành cho lập trình viên (developer portal), thường nhắm vào các consumer bên ngoài.

Part of the confusion around the API gateway has to do with history. A while back, there was a huge amount of interest in what was called “the API economy.” The industry had started to understand the power of offering APIs to managed solutions, from SaaS products like Salesforce to platforms like AWS, as it was clear that an API gave customers a lot more flexibility in how their software was used. This caused a number of people to start looking at the software they already had and consider the benefits of exposing that functionality to their customers not just via a GUI but also via an API. The hope was that this would open up larger market opportunities, and, well, make more money. Amid this interest, a crop of API gateway products emerged to help make achieving those goals possible. Their featureset leaned heavily on managing API keys for third parties, enforcing rate limits, and tracking use for chargeback purposes. The reality is that while APIs have absolutely been shown to

be an excellent way to deliver services to some customers, the size of the API economy wasn't quite as big as some had hoped, and a lot of companies found that they had purchased API gateway products laden with features they never actually needed.

Một phần sự bối rối quanh API gateway liên quan đến lịch sử. Một thời gian trước, có một lượng quan tâm khổng lồ vào cái gọi là “nền kinh tế API” (the API economy). Ngành công nghiệp đã bắt đầu hiểu sức mạnh của việc cung cấp API cho các giải pháp được quản lý, từ các sản phẩm SaaS như Salesforce đến các nền tảng như AWS, vì rõ ràng một API cho khách hàng nhiều linh hoạt hơn nhiều trong cách phần mềm của họ được dùng. Điều này khiến một số người bắt đầu nhìn vào phần mềm mà họ đã có và cân nhắc các lợi ích của việc phơi bày chức năng đó ra cho khách hàng của họ không chỉ qua một GUI mà còn qua một API. Hy vọng là điều này sẽ mở ra các cơ hội thị trường lớn hơn, và, ờ thì, kiếm nhiều tiền hơn. Giữa sự quan tâm này, một lứa các sản phẩm API gateway đã xuất hiện để giúp làm cho việc đạt được những mục tiêu đó trở nên khả thi. Tập tính năng của chúng nghiêng nhiều về việc quản lý các khóa API cho bên thứ ba, thực thi các giới hạn tốc độ, và theo dõi việc sử dụng cho mục đích tính phí (chargeback). Thực tế là trong khi API tuyệt đối đã được chứng minh là một cách xuất sắc để cung cấp dịch vụ cho một số khách hàng, kích thước của nền kinh tế API không lớn lắm như một số người đã hy vọng, và rất nhiều công ty thấy rằng họ đã mua các sản phẩm API gateway chất đầy các tính năng mà họ thực ra chẳng bao giờ cần.

Much of the time, all an API gateway is actually being used for is to manage access to an organization's microservices from its own GUI clients (web pages, native mobile applications) via the public internet. There is no “third party” in the mix here. The need for some form of an API gateway for Kubernetes is essential, as Kubernetes natively handles networking only within the cluster and does nothing about handling communication to and from the cluster itself. But in such a use case, an API gateway designed for external third-party access is huge overkill.

Phần lớn thời gian, tất cả những gì một API gateway thực sự được dùng để làm là quản lý truy cập tới các microservice của một tổ chức từ chính các client GUI của tổ chức đó (các trang web, các ứng dụng di động native) qua internet công cộng. Không có “bên thứ ba” nào trong cuộc chơi ở đây. Nhu cầu có một dạng API gateway nào đó cho Kubernetes là thiết yếu, vì Kubernetes về bản chất chỉ xử lý mạng bên trong cụm và không làm gì về việc xử lý giao tiếp đi tới và đi từ chính cụm. Nhưng trong một trường hợp sử dụng như vậy, một API gateway được thiết kế cho truy cập của bên thứ ba bên ngoài là quá thừa thãi to lớn.

So if you want an API gateway, be really clear in what you expect from it. In fact, I'd go a bit further and say that you should probably avoid having an API gateway that does too much. But we'll get to that next.

Vậy nên nếu bạn muốn một API gateway, hãy thực sự rõ ràng về những gì bạn kỳ vọng từ nó. Thực tế, tôi sẽ đi xa hơn một chút và nói rằng bạn có lẽ nên tránh có một API gateway làm quá nhiều việc. Nhưng ta sẽ bàn tới đó tiếp theo.

Where to use them

Khi nào nên dùng chúng

Once you start to understand what sort of use cases you have, it becomes a bit easier to see what type of gateway you need. If it's just a case of exposing microservices running in Kubernetes, you could run your own reverse proxies—or better yet, you could look at a focused product like Ambassador, which was built from the ground up with that use case in mind. If you really find yourself needing to manage large numbers of third-party users accessing your API, then there are likely other products

to look at. It's possible, in fact, that you may end up with more than one gateway in the mix to better handle separation of concerns, and I can see that being sensible in many situations, although the usual caveats around increasing the overall system complexity and increasing network hops still apply.

Một khi bạn bắt đầu hiểu mình có loại trường hợp sử dụng nào, sẽ dễ hơn một chút để thấy bạn cần loại gateway nào. Nếu chỉ là trường hợp phơi bày các microservice chạy trong Kubernetes, bạn có thể chạy các reverse proxy của riêng mình—hoặc tốt hơn nữa, bạn có thể xem xét một sản phẩm tập trung như Ambassador, vốn được xây dựng từ đầu với trường hợp sử dụng đó trong tâm trí. Nếu bạn thực sự thấy mình cần quản lý số lượng lớn người dùng bên thứ ba truy cập API của bạn, thì có khả năng có những sản phẩm khác để xem xét. Thực tế, có thể bạn sẽ rất cuộc có hơn một gateway trong cuộc chơi để xử lý tốt hơn việc tách biệt các mối quan tâm, và tôi có thể thấy điều đó hợp lý trong nhiều tình huống, mặc dù các cảnh báo thông thường về việc tăng độ phức tạp tổng thể của hệ thống và tăng số bước nhảy mạng (network hop) vẫn áp dụng.

I have been involved from time to time in working directly with vendors to help with tool selection. I can say without any hesitation that I have experienced more mis-selling and poor or cutthroat behavior in the API gateway space than in any other— and as a result you won't find references to some vendor products in this chapter. I've put a lot of that down to VC-backed companies that had built a product for the boom times of the API economy, only to find that market doesn't exist, and so they are fighting on two fronts: they are fighting over the small number of users that actually need what the more complex gateways are offering, while also losing business to more focused API gateway products that are built for the vast majority of simpler needs.

Tôi đã thỉnh thoảng tham gia làm việc trực tiếp với các nhà cung cấp để giúp lựa chọn công cụ. Tôi có thể nói không chút do dự rằng tôi đã trải nghiệm nhiều hành vi bán sai và tôi tệ hoặc cạnh tranh tàn nhẫn trong không gian API gateway hơn bất kỳ không gian nào khác—và kết quả là bạn sẽ không tìm thấy các tham chiếu đến một số sản phẩm của nhà cung cấp nào đó trong chương này. Tôi quy phần lớn điều đó cho các công ty được hậu thuẫn bởi VC vốn đã xây dựng một sản phẩm cho thời kỳ bùng nổ của nền kinh tế API, chỉ để rồi thấy rằng thị trường đó không tồn tại, và thế là họ đang chiến đấu trên hai mặt trận: họ đang tranh giành số ít người dùng thực sự cần những gì các gateway phức tạp hơn cung cấp, đồng thời lại mất việc kinh doanh vào tay các sản phẩm API gateway tập trung hơn được xây dựng cho đại đa số các nhu cầu đơn giản hơn.

What to avoid

Cái cần tránh

Partly due to the apparent desperation of some API gateway vendors, all sorts of claims have been made for what these products can do. This has led to a lot of misuse of these products, and in turn to an unfortunate distrust of what is fundamentally quite a simple concept. Two key examples I've seen of misuse of API gateways is for call aggregation and protocol rewriting, but I've also seen a wider push to use API gateways for in-perimeter (east-west) calls too.

Một phần do sự tuyệt vọng rõ rệt của một số nhà cung cấp API gateway, đủ loại tuyên bố đã được đưa ra về việc các sản phẩm này có thể làm gì. Điều này đã dẫn đến rất nhiều việc lạm dụng các sản phẩm này, và đến lượt nó dẫn đến một sự ngờ vực đáng tiếc về cái mà về căn bản là một khái niệm khá đơn giản. Hai ví dụ chủ chốt mà tôi đã thấy về việc lạm dụng API gateway là cho việc gộp lời gọi (call aggregation) và viết lại giao thức (protocol rewriting), nhưng tôi cũng đã thấy một sự thúc đẩy rộng hơn để dùng API gateway cho các lời gọi trong-vành-đai (đông-tây) nữa.

In this chapter we’ve already briefly looked at the usefulness of a protocol like GraphQL to help us in a situation in which we need to make a number of calls and then aggregate and filter the results, but people are often tempted to solve this problem in API gateway layers too. It starts off innocently enough: you combine a couple of calls and return a single payload. Then you start making another downstream call as part of the same aggregated flow. Then you start wanting to add conditional logic, and before long you realize that you’ve baked core business processes into a third-party tool that is ill suited to the task.

Trong chương này ta đã xem xét sơ qua tính hữu ích của một giao thức như GraphQL để giúp ta trong tình huống ta cần thực hiện một số lời gọi rồi gộp và lọc kết quả, nhưng người ta thường bị cám dỗ để giải quyết vấn đề này trong các tầng API gateway nữa. Nó bắt đầu khá vô hại: bạn kết hợp một vài lời gọi và trả về một payload duy nhất. Rồi bạn bắt đầu thực hiện một lời gọi phía dưới khác như một phần của cùng một luồng được gộp. Rồi bạn bắt đầu muốn thêm logic điều kiện, và chẳng mấy chốc bạn nhận ra rằng bạn đã nướng các quy trình nghiệp vụ cốt lõi vào một công cụ bên thứ ba vốn không phù hợp với nhiệm vụ.

If you find yourself needing to do call aggregation and filtering, then look at the potential of GraphQL or the BFF pattern, which we’ll cover in Chapter 14 . If the call aggregation you are performing is fundamentally a business process, then this is better done through an explicitly modeled **saga**, which we’ll cover in Chapter 6 .

Nếu bạn thấy mình cần làm việc gộp và lọc lời gọi, thì hãy xem xét tiềm năng của GraphQL hoặc pattern BFF, mà ta sẽ đề cập ở Chương 14. Nếu việc gộp lời gọi bạn đang thực hiện về căn bản là một quy trình nghiệp vụ, thì điều này được làm tốt hơn thông qua một saga được mô hình hóa một cách tường minh, mà ta sẽ đề cập ở Chương 6.

Aside from the aggregation angle, protocol rewriting is also often pushed as something API gateways should be used for. I remember one unnamed vendor very aggressively promoting the idea that its product could “change any SOAP API into a REST API.” Firstly, REST is an entire architectural mindset that cannot simply be implemented in a proxy layer. Secondly, protocol rewriting, which is fundamentally what this is trying to do, shouldn’t be done in intermediate layers, as it’s pushing too much behavior to the wrong place.

Ngoài góc độ gộp, viết lại giao thức cũng thường được thúc đẩy như một thứ mà API gateway nên được dùng. Tôi nhớ một nhà cung cấp không nêu tên đã rất hung hăng quảng bá ý tưởng rằng sản phẩm của họ có thể “biến bất kỳ API SOAP nào thành một API REST.” Thứ nhất, REST là cả một tư duy kiến trúc không thể đơn giản được hiện thực trong một tầng proxy. Thứ hai, viết lại giao thức, vốn về căn bản là điều mà việc này đang cố làm, không nên được làm trong các tầng trung gian, vì nó đang đẩy quá nhiều hành vi vào sai chỗ.

The main problem with both the protocol rewriting capability and the implementation of call aggregation inside API gateways is that we are violating the rule of keeping the pipes dumb, and the endpoints smart. The “smarts” in our system want to live in our code, where we can have full control over them. The API gateway in this example is a pipe—we want it as simple as possible. With microservices, we are pushing for a model in which changes can be made and more easily released through independent deployability. Keeping smarts in our microservices helps this. If we now also have to make changes in intermediate layers, things become more problematic. Given the criticality of API gateways, changes to them are often tightly controlled. It seems unlikely that individual teams will be given free rein to self-service change these often centrally managed services. What does that mean? Tickets. To roll out a change to your software, you end up having the API gateway team make changes for you. The more behavior you leak into API gateways (or into enterprise service buses),

the more you run the risk of handoffs, increased coordination, and slowed delivery.

Vấn đề chính với cả khả năng viết lại giao thức lẫn việc hiện thực gộp lời gọi bên trong API gateway là chúng ta đang vi phạm quy tắc giữ cho các đường ống ngu (dumb pipes) và các endpoint thông minh (smart endpoints). Sự “thông minh” trong hệ thống của ta muốn sống trong mã của ta, nơi ta có thể có toàn quyền kiểm soát chúng. API gateway trong ví dụ này là một đường ống—ta muốn nó đơn giản nhất có thể. Với microservice, ta đang hướng tới một mô hình trong đó các thay đổi có thể được thực hiện và phát hành dễ dàng hơn thông qua khả năng triển khai độc lập. Việc giữ sự thông minh trong các microservice của ta giúp ích cho điều này. Nếu giờ ta cũng phải thực hiện các thay đổi trong các tầng trung gian, mọi thứ trở nên rắc rối hơn. Xét đến tầm quan trọng sống còn của API gateway, các thay đổi đối với chúng thường được kiểm soát chặt chẽ. Có vẻ không có khả năng các nhóm riêng lẻ sẽ được trao toàn quyền tự phục vụ (self-service) thay đổi các dịch vụ thường được quản lý tập trung này. Điều đó nghĩa là gì? Là các ticket (phiếu yêu cầu). Để triển khai một thay đổi cho phần mềm của bạn, bạn rất cuộc phải nhờ nhóm API gateway thực hiện các thay đổi cho bạn. Càng nhiều hành vi bạn rò rỉ vào API gateway (hoặc vào các enterprise service bus), bạn càng có nguy cơ về các lần bàn giao, sự phối hợp gia tăng, và việc chuyển giao bị chậm lại.

The last issue is the use of an API gateway as an intermediary for all inter-microservice calls. This can be extremely problematic. If we insert an API gateway or a normal network proxy between two microservices, then we have normally added at least a single network hop. A call from microservice A to microservice B first goes from A to the API gateway and then from the API gateway to B. We have to consider the latency impact of the additional network call and the overhead of whatever the proxy is doing. Service meshes, which we explore next, are much better placed to solve this problem.

Vấn đề cuối cùng là việc dùng một API gateway như một trung gian cho tất cả các lời gọi giữa các microservice. Điều này có thể cực kỳ rắc rối. Nếu ta chèn một API gateway hay một network proxy thông thường giữa hai microservice, thì ta thường đã thêm ít nhất một bước nhảy mạng. Một lời gọi từ microservice A tới microservice B trước tiên đi từ A tới API gateway rồi từ API gateway tới B. Ta phải xét đến tác động độ trễ của lời gọi mạng bổ sung và chi phí phụ của bất cứ điều gì proxy đang làm. Service mesh, mà ta khám phá tiếp theo, ở vị trí tốt hơn nhiều để giải quyết vấn đề này.

Service Meshes — Service Mesh

With a service mesh, common functionality associated with inter-microservice communication is pushed into the mesh. This reduces the functionality that a microservice needs to implement internally, while also providing consistency across how certain things are done.

Với một service mesh, chức năng chung liên quan đến giao tiếp giữa các microservice được đẩy vào mesh. Điều này giảm bớt chức năng mà một microservice cần hiện thực bên trong, đồng thời cũng cung cấp sự nhất quán trong cách một số thứ được làm.

Common features implemented by service meshes include mutual TLS, correlation IDs, service discovery and load balancing, and more. Often this type of functionality is fairly generic from one microservice to the next, so we'd end up making use of a shared library to handle it. But then you have to deal with what happens if different microservices have different versions of the libraries running, or what happens if you have microservices written in different runtimes.

Các tính năng chung được hiện thực bởi service mesh bao gồm mutual TLS, correlation ID, service discovery và cân bằng tải, và nhiều hơn nữa. Thường thì loại chức năng này khá

chung chung từ microservice này sang microservice khác, nên ta sẽ rất cuộc dùng một thư viện dùng chung để xử lý nó. Nhưng rồi bạn phải đối phó với điều gì xảy ra nếu các microservice khác nhau đang chạy các phiên bản khác nhau của các thư viện, hoặc điều gì xảy ra nếu bạn có các microservice được viết bằng các runtime khác nhau.

Historically at least, Netflix would mandate that all nonlocal network communication had to be done JVM to JVM. This was to ensure that the tried and tested common libraries that are a vital part of managing effective communication between microservices could be reused. With the use of a service mesh, though, we have the possibility of reusing common inter-microservice functionality across microservices written in different programming languages. Service meshes can also be incredibly useful in implementing standard behavior across microservices created by different teams— and the use of a service mesh, especially on Kubernetes, has increasingly become an assumed part of any given platform you might create for self-service deployment and management of microservices.

Trong lịch sử ít ra, Netflix sẽ bắt buộc rằng tất cả giao tiếp mạng không-cục-bộ phải được làm từ JVM tới JVM. Điều này nhằm đảm bảo rằng các thư viện chung đã được thử nghiệm và kiểm chứng, vốn là một phần sống còn của việc quản lý giao tiếp hiệu quả giữa các microservice, có thể được tái sử dụng. Tuy nhiên, với việc dùng một service mesh, ta có khả năng tái sử dụng chức năng chung giữa các microservice được viết bằng các ngôn ngữ lập trình khác nhau. Service mesh cũng có thể cực kỳ hữu ích trong việc hiện thực hành vi tiêu chuẩn trên các microservice được tạo bởi các nhóm khác nhau—và việc dùng một service mesh, đặc biệt trên Kubernetes, đã ngày càng trở thành một phần được mặc định của bất kỳ nền tảng nào bạn có thể tạo ra cho việc triển khai và quản lý microservice theo kiểu tự phục vụ.

Making it easy to implement common behavior across microservices is one of the big benefits of a service mesh. If this common functionality was implemented solely through shared libraries, changing this behavior would require every microservice to pull in a new version of said libraries and be deployed before that change is live. With a service mesh, you have much more flexibility in rolling out changes in terms of inter-microservice communication without requiring a rebuild and redeploy.

Việc làm cho việc hiện thực hành vi chung trên các microservice dễ dàng là một trong những lợi ích lớn của một service mesh. Nếu chức năng chung này được hiện thực hoàn toàn thông qua các thư viện dùng chung, việc thay đổi hành vi này sẽ đòi hỏi mỗi microservice phải kéo vào một phiên bản mới của các thư viện nói trên và được triển khai trước khi thay đổi đó có hiệu lực. Với một service mesh, bạn có nhiều linh hoạt hơn trong việc triển khai các thay đổi về giao tiếp giữa các microservice mà không đòi hỏi việc build lại và triển khai lại.

How they work

Chúng hoạt động như thế nào

In general, we'd expect to have less north-south traffic than east-west traffic with a microservice architecture. A single north-south call—placing an order, for example— could result in multiple east-west calls. This means that when considering any sort of proxy for in-perimeter calls, we have to be aware of the overhead these additional calls can cause, and this is a core consideration in terms of how service meshes are built.

Nói chung, ta sẽ trông đợi có ít lưu lượng bắc-nam hơn lưu lượng đông-tây với một kiến trúc microservice. Một lời gọi bắc-nam đơn lẻ—đặt một đơn hàng chẳng hạn—có thể dẫn đến nhiều lời gọi đông-tây. Điều này nghĩa là khi xét bất kỳ dạng proxy nào cho các lời gọi trong-vành-đai, ta phải nhận thức được chi phí phụ mà những lời gọi bổ sung này có thể

gây ra, và đây là một cân nhắc cốt lõi về cách các service mesh được xây dựng.

Service meshes come in different shapes and sizes, but what unites them is that their architecture is based on trying to limit the impact caused by calls to and from the proxy. This is achieved primarily by distributing the proxy processes to run on the same physical machines as the microservice instances, to ensure that the number of remote network calls is limited. In Figure 5-7 we see this in action—the Order Processor is sending a request to the Payment microservice. This call is first routed locally to a proxy instance running on the same machine as Order Processor, before continuing to the Payment microservice via its local proxy instance. The Order Processor thinks it's making a normal network call, unaware that the call is routed locally on the machine, which is significantly faster (and also less prone to partitions).

Service mesh đến trong các hình dạng và kích thước khác nhau, nhưng điều gắn kết chúng là kiến trúc của chúng dựa trên việc cố gắng giới hạn tác động gây ra bởi các lời gọi tới và từ proxy. Điều này đạt được chủ yếu bằng cách phân phối các tiến trình proxy để chạy trên cùng các máy vật lý với các instance microservice, nhằm đảm bảo rằng số lượng lời gọi mạng từ xa được giới hạn. Trong Hình 5-7 ta thấy điều này trong thực tế—Order Processor đang gửi một request tới microservice Payment. Lời gọi này trước tiên được định tuyến cục bộ tới một instance proxy chạy trên cùng máy với Order Processor, trước khi tiếp tục tới microservice Payment thông qua instance proxy cục bộ của nó. Order Processor nghĩ rằng nó đang thực hiện một lời gọi mạng bình thường, không biết rằng lời gọi được định tuyến cục bộ trên máy, điều này nhanh hơn đáng kể (và cũng ít bị phân vùng mạng - partition hơn).

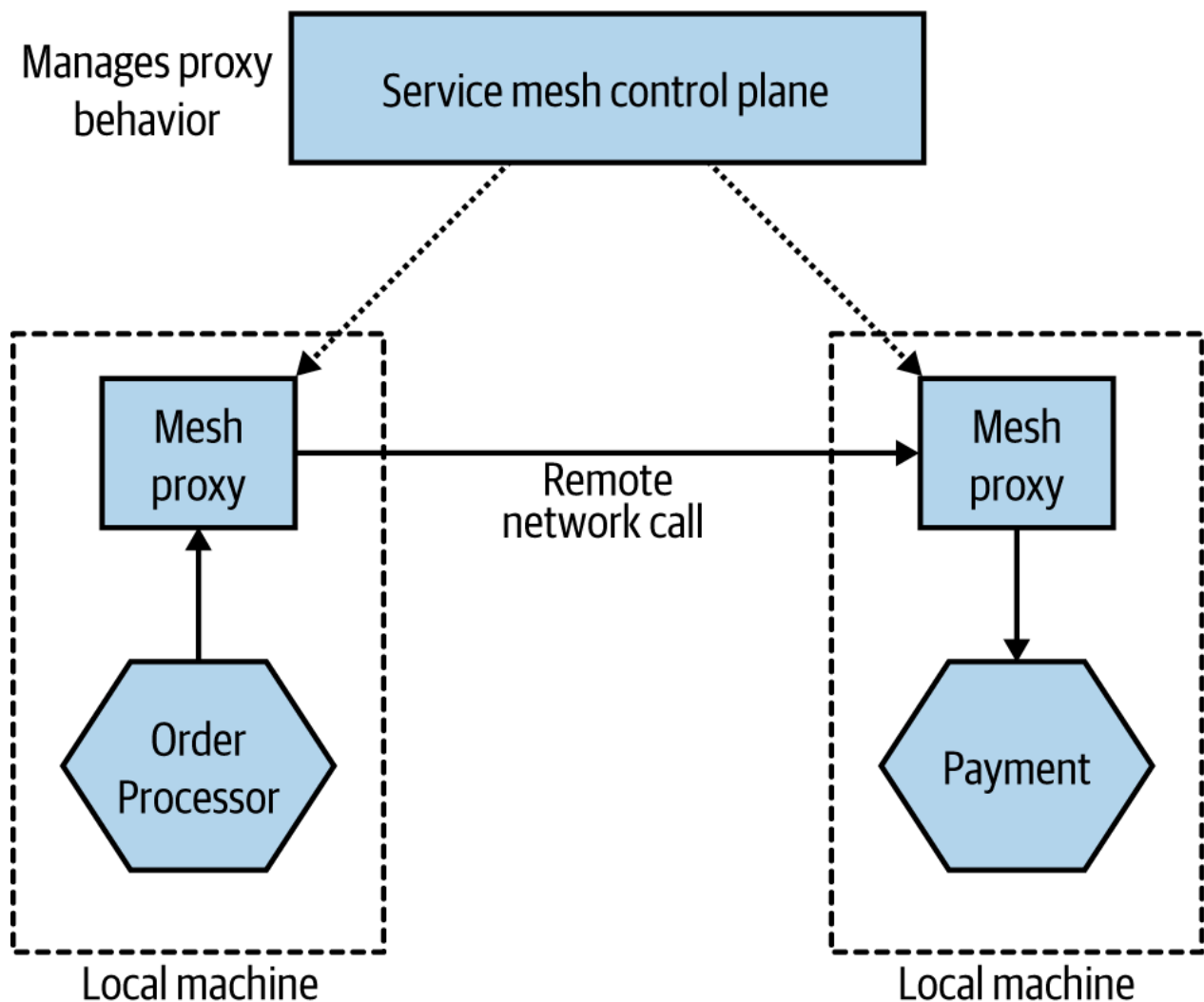


Figure 5-7. A service mesh is deployed to handle all direct inter-microservice communication

Hình 5-7. Một service mesh được triển khai để xử lý tất cả giao tiếp trực tiếp giữa các microservice

A control plane would sit on top of the local mesh proxies, acting as both a place in which the behavior of these proxies can be changed and a place in which you can collect information about what the proxies are doing.

Một control plane (mặt điều khiển) sẽ nằm trên các proxy mesh cục bộ, vừa đóng vai trò là nơi hành vi của các proxy này có thể được thay đổi vừa là nơi bạn có thể thu thập thông tin về những gì các proxy đang làm.

When deploying on Kubernetes, you would deploy each microservice instance in a pod with its own local proxy. A single pod is always deployed as a single unit, so you always know that you have a proxy available. Moreover, a single proxy dying would impact only that one pod. This setup also allows you to configure each proxy differently for different purposes. We'll look at these concepts in more detail in "Kubernetes and Container Orchestration" on page 259 .

Khi triển khai trên Kubernetes, bạn sẽ triển khai mỗi instance microservice trong một pod với proxy cục bộ riêng của nó. Một pod luôn được triển khai như một đơn vị duy nhất, nên bạn luôn biết rằng bạn có một proxy sẵn có. Hơn nữa, một proxy đơn lẻ chết sẽ chỉ

tác động đến một pod đó. Thiết lập này cũng cho phép bạn cấu hình mỗi proxy khác nhau cho các mục đích khác nhau. Ta sẽ xem các khái niệm này chi tiết hơn trong “Kubernetes và Điều phối Container” ở trang 259.

Many service mesh implementations use the Envoy proxy for the basis of these locally running processes. Envoy is a lightweight C++ proxy often used as the building block for service meshes and other types of proxy-based software—it is an important building block for Istio and Ambassador, for example.

Nhiều hiện thực service mesh dùng proxy Envoy làm cơ sở cho các tiến trình chạy cục bộ này. Envoy là một proxy C++ nhẹ thường được dùng làm khối xây dựng cho service mesh và các loại phần mềm dựa trên proxy khác—nó là một khối xây dựng quan trọng cho Istio và Ambassador chẳng hạn.

These proxies are in turn managed by a control plane. This will be a set of software that helps you see what is going on and control what is being done. When using a service mesh to implement mutual TLS, for example, the control plane would be used to distribute client and server certificates.

Các proxy này đến lượt chúng được quản lý bởi một control plane. Đây sẽ là một tập phần mềm giúp bạn thấy điều gì đang diễn ra và kiểm soát những gì đang được làm. Khi dùng một service mesh để hiện thực mutual TLS chẳng hạn, control plane sẽ được dùng để phân phối các chứng chỉ (certificate) client và server.

Aren't service meshes smart pipes?

Service mesh có phải là các đường ống thông minh không?

So all this talk of pushing common behavior into a service mesh might have alarm bells ringing for some of you. Isn't this approach open to the same sorts of problems as enterprise service buses or overly bloated API gateways? Aren't we at risk of pushing too many “smarts” into our service mesh?

Vậy tất cả những lời bàn về việc đẩy hành vi chung vào một service mesh này có thể đã làm chuông báo động reo lên với một số bạn. Chẳng phải cách tiếp cận này dễ gặp cùng những loại vấn đề như enterprise service bus hay các API gateway phình to quá mức sao? Chẳng phải ta đang có nguy cơ đẩy quá nhiều “sự thông minh” vào service mesh của mình sao?

The key thing to remember here is that the common behavior we are putting into the mesh is not specific to any one microservice. No business functionality has leaked to the outside. We're configuring generic things like how request time-outs are handled. In terms of common behavior that might want to be tweaked on a per-microservice basis, that's typically something that is well catered for, without the need for work to be done on a central platform. For example, with Istio, I can define my **time-out** requirements on a self-service basis just by changing my service definition.

Điều chủ chốt cần nhớ ở đây là hành vi chung mà ta đang đặt vào mesh không đặc thù cho bất kỳ một microservice nào. Không có chức năng nghiệp vụ nào rò rỉ ra bên ngoài. Ta đang cấu hình các thứ chung chung như cách các time-out request được xử lý. Về hành vi chung mà có thể muốn được tinh chỉnh trên cơ sở từng-microservice, đó thường là thứ được đáp ứng tốt, mà không cần phải làm việc trên một nền tảng trung tâm. Ví dụ, với Istio, tôi có thể định nghĩa các yêu cầu time-out của mình trên cơ sở tự phục vụ chỉ bằng cách thay đổi định nghĩa dịch vụ của tôi.

Do you need one?

Bạn có cần một cái không?

When the use of service meshes first started becoming popular, just after the release of the first edition of this book, I saw a lot of merit in the idea but also saw a lot of churn in the space. Different deployment models were suggested, built, and then dropped, and the number of companies offering solutions in this space increased drastically; but even for those tools that had been around for a long time, there was an apparent lack of stability. Linkerd, which arguably did as much as anyone to pioneer this space, totally rebuilt its product from scratch in the shift from v1 to v2. Istio, which was the Google-anointed service mesh, took years to get to an initial 1.0 release, and even still it had significant subsequent changes in its architecture (moving somewhat ironically, although sensibly, to a more monolithic deployment model for its control plane).

Khi việc dùng service mesh lần đầu bắt đầu trở nên phổ biến, ngay sau khi phát hành ấn bản đầu tiên của cuốn sách này, tôi thấy nhiều giá trị trong ý tưởng nhưng cũng thấy nhiều biến động (churn) trong không gian. Các mô hình triển khai khác nhau được đề xuất, xây dựng, rồi bị bỏ, và số lượng công ty cung cấp giải pháp trong không gian này tăng vọt; nhưng ngay cả với những công cụ đã tồn tại lâu, vẫn có một sự thiếu ổn định rõ rệt. Linkerd, vốn có thể nói là đã đóng góp nhiều như bất kỳ ai trong việc tiên phong không gian này, đã hoàn toàn xây dựng lại sản phẩm của mình từ đầu trong quá trình chuyển từ v1 sang v2. Istio, vốn là service mesh được Google tán phong, mất nhiều năm để đạt được một bản phát hành 1.0 ban đầu, và thậm chí nó vẫn có những thay đổi đáng kể về sau trong kiến trúc của mình (chuyển một cách hơi mỉa mai, dù hợp lý, sang một mô hình triển khai mang tính monolith hơn cho control plane của nó).

For much of the last five years, when I have been asked “Should we get a service mesh?” my advice has been “If you can afford to wait six months before making a choice, then wait six months.” I was sold on the idea but concerned about the stability. And something like a service mesh is not where I personally want to take a lot of risks—it’s so key, so essential to everything working well. You’re putting it on your critical path. It’s up there with selecting a message broker or cloud provider in terms of how seriously I’d take it.

Trong phần lớn năm năm qua, khi tôi được hỏi “Chúng tôi có nên mua một service mesh không?” lời khuyên của tôi là “Nếu bạn có thể đợi sáu tháng trước khi đưa ra lựa chọn, thì hãy đợi sáu tháng.” Tôi bị thuyết phục bởi ý tưởng nhưng lo ngại về sự ổn định. Và một thứ như service mesh không phải là nơi cá nhân tôi muốn chấp nhận nhiều rủi ro—nó quá chủ chốt, quá thiết yếu cho mọi thứ hoạt động tốt. Bạn đang đặt nó vào đường găng (critical path) của mình. Nó ngang hàng với việc chọn một message broker hay nhà cung cấp cloud về mức độ nghiêm túc mà tôi sẽ coi trọng nó.

Since then, I’m happy to say, this space has matured. The churn to some extent has slowed, but we still have a (healthy) plurality of vendors. That said, service meshes aren’t for everyone. Firstly, if you aren’t on Kubernetes, your options are limited. Secondly, they do add complexity. If you have five microservices, I don’t think you can easily justify a service mesh (it’s arguable as to whether you can justify Kubernetes if you only have five microservices!). For organizations that have more microservices, especially if they want the option for those microservices to be written in different programming languages, service meshes are well worth a look. Do your homework, though—switching between service meshes is painful!

Kể từ đó, tôi vui mừng nói rằng, không gian này đã trưởng thành. Sự biến động ở một mức độ nào đó đã chậm lại, nhưng ta vẫn có một sự đa dạng (lành mạnh) các nhà cung cấp. Tuy nhiên, service mesh không dành cho tất cả mọi người. Thứ nhất, nếu bạn không ở trên Kubernetes, các lựa chọn của bạn bị hạn chế. Thứ hai, chúng quả thực có thêm độ phức tạp. Nếu bạn có năm microservice, tôi không nghĩ bạn có thể dễ dàng biện minh cho một service mesh (còn tranh cãi được liệu bạn có thể biện minh cho Kubernetes nếu bạn chỉ có năm microservice hay không!). Đối với các tổ chức có nhiều microservice hơn,

đặc biệt nếu họ muốn lựa chọn cho các microservice đó được viết bằng các ngôn ngữ lập trình khác nhau, service mesh rất đáng để xem xét. Tuy nhiên, hãy làm bài tập về nhà của bạn—chuyển đổi giữa các service mesh thật đau đớn!

Monzo is one organization that has spoken openly about how its use of a service mesh was essential in allowing it to run its architecture at the scale it does. Its use of version 1 of Linkerd to help manage inter-microservice RPC calls proved hugely beneficial. Interestingly, Monzo had to handle the pain of a service mesh migration to help it achieve the scale it needed when the older architecture of Linkerd v1 no longer met its requirements. In the end it moved effectively to an in-house service mesh making use of the Envoy proxy.

Monzo là một tổ chức đã nói công khai về việc dùng một service mesh đã thiết yếu thế nào trong việc cho phép họ vận hành kiến trúc của mình ở quy mô như họ đang làm. Việc họ dùng phiên bản 1 của Linkerd để giúp quản lý các lời gọi RPC giữa các microservice đã chứng tỏ cực kỳ có lợi. Điều thú vị là, Monzo đã phải xử lý nỗi đau của một cuộc di trú service mesh để giúp họ đạt được quy mô họ cần khi kiến trúc cũ hơn của Linkerd v1 không còn đáp ứng các yêu cầu của họ. Cuối cùng họ đã chuyển một cách hiệu quả sang một service mesh tự xây dựng dùng proxy Envoy.

What About Other Protocols? — Còn các giao thức khác thì sao?

API gateways and service meshes are primarily used to handle HTTP-related calls. So REST, SOAP, gRPC, and the like can be managed via these products. Things get a bit more murky, though, when you start looking at communication via other protocols, like the use of message brokers such as Kafka. Typically, at this point the service mesh gets bypassed—communication is done directly with the broker itself. This means that you cannot assume your service mesh is able to work as an intermediary for all calls between microservices.

API gateway và service mesh chủ yếu được dùng để xử lý các lời gọi liên quan đến HTTP. Vậy nên REST, SOAP, gRPC và những thứ tương tự có thể được quản lý qua các sản phẩm này. Tuy nhiên, mọi thứ trở nên hơi mờ mịt hơn một chút khi bạn bắt đầu nhìn vào giao tiếp qua các giao thức khác, như việc dùng các message broker chẳng hạn như Kafka. Thường thì, tại điểm này service mesh bị bỏ qua—giao tiếp được làm trực tiếp với chính broker. Điều này nghĩa là bạn không thể giả định service mesh của bạn có khả năng hoạt động như một trung gian cho tất cả các lời gọi giữa các microservice.

Documenting Services — Lập tài liệu cho dịch vụ

By decomposing our systems into finer-grained microservices, we're hoping to expose lots of seams in the form of APIs that people can use to do many hopefully wonderful things. If you get our discovery right, we know where things are. But how do we know what those things do or how to use them? One option is obviously to have documentation about the APIs. Of course, documentation can often be out of date. Ideally, we'd ensure that our documentation is always up to date with the microservice API and make it easy to see this documentation when we know where a service endpoint is.

Bằng cách phân rã các hệ thống của ta thành các microservice mịn hơn, ta hy vọng phơi bày rất nhiều đường nối (seam) dưới dạng các API mà người ta có thể dùng để làm nhiều điều hy vọng là tuyệt vời. Nếu ta làm đúng phần discovery, ta biết mọi thứ ở đâu. Nhưng làm sao ta biết những thứ đó làm gì hay cách dùng chúng? Một lựa chọn rõ ràng là có tài liệu về các API. Tất nhiên, tài liệu thường có thể đã lỗi thời. Lý tưởng nhất, ta sẽ đảm bảo

rằng tài liệu của ta luôn được cập nhật với API microservice và làm cho việc xem tài liệu này dễ dàng khi ta biết một service endpoint ở đâu.

Explicit Schemas — Schema tường minh

Having explicit schemas does go a long way toward making it easier to understand what any given endpoint exposes, but by themselves they are often not enough. As we’ve already discussed, schemas help show the structure, but they don’t go very far in helping communicate the behavior of an endpoint, so good documentation could still be required to help consumers understand how to use an endpoint. It’s worth noting, of course, that if you decide not to use an explicit schema, your documentation will end up doing more work. You’ll need to explain what the endpoint does and also document the structure and detail of the interface. Moreover, without an explicit schema, detecting whether your documentation is up to date with the real endpoints is more difficult. Stale documentation is an ongoing problem, but at least an explicit schema gives you more chance of it being up to date.

Việc có schema tường minh quả thực đi một chặng đường dài hướng tới việc làm cho việc hiểu một endpoint cho trước phơi bày cái gì dễ dàng hơn, nhưng tự bản thân chúng thường là chưa đủ. Như ta đã bàn, schema giúp cho thấy cấu trúc, nhưng chúng không đi xa lắm trong việc giúp truyền đạt hành vi của một endpoint, nên tài liệu tốt vẫn có thể cần thiết để giúp consumer hiểu cách dùng một endpoint. Tất nhiên đáng lưu ý rằng nếu bạn quyết định không dùng một schema tường minh, tài liệu của bạn sẽ rất cuộc phải làm nhiều việc hơn. Bạn sẽ cần giải thích endpoint làm gì và cũng lập tài liệu cho cấu trúc và chi tiết của interface. Hơn nữa, không có một schema tường minh, việc phát hiện liệu tài liệu của bạn có được cập nhật với các endpoint thực hay không sẽ khó hơn. Tài liệu cũ kỹ là một vấn đề dai dẳng, nhưng ít ra một schema tường minh cho bạn nhiều cơ hội hơn để nó được cập nhật.

I’ve already introduced OpenAPI as a schema format, but it also is very effective in providing documentation, and a lot of open source and commercial tools now exist that can support consuming the OpenAPI descriptors to help create useful portals to allow developers to read the documentation. It’s worth noting that the open source portals for viewing OpenAPI seem somewhat basic—I struggled to find one that supported search functionality, for example. For those on Kubernetes, Ambassador’s developer portal is especially interesting. Ambassador is already a popular choice as an API gateway for Kubernetes, and its Developer Portal has the ability to autodiscover available OpenAPI endpoints. The idea of deploying a new microservice and having its documentation automatically available greatly appeals to me.

Tôi đã giới thiệu OpenAPI như một định dạng schema, nhưng nó cũng rất hiệu quả trong việc cung cấp tài liệu, và nhiều công cụ mã nguồn mở và thương mại nay tồn tại có thể hỗ trợ việc tiêu thụ các bộ mô tả OpenAPI để giúp tạo các cổng thông tin hữu ích cho phép lập trình viên đọc tài liệu. Đáng lưu ý rằng các cổng thông tin mã nguồn mở để xem OpenAPI có vẻ khá sơ sài—tôi đã chật vật tìm một cái hỗ trợ chức năng tìm kiếm chẳng hạn. Đối với những người trên Kubernetes, developer portal của Ambassador đặc biệt thú vị. Ambassador đã là một lựa chọn phổ biến như một API gateway cho Kubernetes, và Developer Portal của nó có khả năng tự động phát hiện (autodiscover) các endpoint OpenAPI sẵn có. Ý tưởng về việc triển khai một microservice mới và có tài liệu của nó tự động sẵn có rất hấp dẫn tôi.

In the past we’ve lacked good support for documenting event-based interfaces. Now at least we have options. The AsyncAPI format started off as an adaptation of Open- API, and we also now have

CloudEvents, which is a CNCF project. I've not used either in anger (that is, in a real setting), but I'm more drawn to CloudEvents purely because it seems to have a wealth of integration and support, due in large part to its association with the CNCF. Historically, at least, CloudEvents seemed to be more restrictive in terms of the event format compared to AsyncAPI, with only JSON being properly supported, until protocol buffer support was recently reintroduced after previously being removed; so that may be a consideration.

Trong quá khứ chúng ta đã thiếu hỗ trợ tốt cho việc lập tài liệu các interface dựa trên event. Giờ ít ra ta đã có các lựa chọn. Định dạng AsyncAPI khởi đầu như một sự điều chỉnh từ OpenAPI, và giờ ta cũng có CloudEvents, vốn là một dự án CNCF. Tôi chưa dùng cái nào trong thực chiến (tức là trong một bối cảnh thực), nhưng tôi bị thu hút hơn về CloudEvents thuần túy vì nó có vẻ có một sự dồi dào các tích hợp và hỗ trợ, phần lớn nhờ vào sự liên kết của nó với CNCF. Trong lịch sử ít ra, CloudEvents có vẻ hạn chế hơn về định dạng event so với AsyncAPI, với chỉ JSON được hỗ trợ đầy đủ, cho đến khi hỗ trợ protocol buffer gần đây được tái giới thiệu sau khi trước đó bị loại bỏ; nên đó có thể là một điều cần cân nhắc.

The Self-Describing System — Hệ thống tự mô tả

During the early evolution of SOA, standards like Universal Description, Discovery, and Integration (UDDI) emerged to help us make sense of what services were running. These approaches were fairly heavyweight, which led to alternative techniques to try to make sense of our systems. Martin Fowler has discussed the concept of the humane registry, a much more lightweight approach in which humans can record information about the services in the organization in something as basic as a wiki.

Trong giai đoạn tiến hóa sơ khai của SOA, các tiêu chuẩn như Universal Description, Discovery, and Integration (UDDI) đã xuất hiện để giúp ta hiểu được những dịch vụ nào đang chạy. Các cách tiếp cận này khá nặng nề, điều dẫn đến các kỹ thuật thay thế để cố gắng hiểu được các hệ thống của ta. Martin Fowler đã bàn về khái niệm humane registry, một cách tiếp cận nhẹ nhàng hơn nhiều trong đó con người có thể ghi lại thông tin về các dịch vụ trong tổ chức trong một thứ gì đó cơ bản như một wiki.

Getting a picture of our system and how it is behaving is important, especially when we're at scale. We've covered a number of different techniques that will help us gain understanding directly from our system. By tracking the health of our downstream services together with correlation IDs to help us see call chains, we can get real data in terms of how our services interrelate. Using **service discovery** systems like Consul, we can see where our **microservices** are running. Mechanisms like OpenAPI and CloudEvents can help us see what capabilities are being hosted on any given endpoint, while our health check pages and **monitoring** systems let us know the health of both the overall system and individual services.

Việc nắm được bức tranh về hệ thống của chúng ta và cách nó vận hành là điều quan trọng, đặc biệt khi chúng ta đang ở quy mô lớn. Chúng ta đã đề cập đến một số kỹ thuật khác nhau giúp ta hiểu hệ thống một cách trực tiếp. Bằng cách theo dõi tình trạng sức khỏe của các dịch vụ phía dưới (downstream) cùng với correlation ID để giúp ta thấy được chuỗi lời gọi (call chain), chúng ta có thể có được dữ liệu thực về cách các dịch vụ liên hệ với nhau. Bằng cách sử dụng các hệ thống service discovery như Consul, chúng ta có thể thấy microservices của mình đang chạy ở đâu. Các cơ chế như OpenAPI và CloudEvents có thể giúp ta thấy được những năng lực nào đang được phục vụ trên một endpoint bất kỳ, trong khi các trang health check và hệ thống monitoring cho ta biết tình trạng sức khỏe của cả hệ thống tổng thể lẫn từng dịch vụ riêng lẻ.

All of this information is available programmatically. All of this data allows us to make our humane registry more powerful than a simple wiki page that will no doubt get out of date. Instead, we should use it to harness and display all the information our system will be emitting. By creating custom dashboards, we can pull together the vast array of information that is available to help us make sense of our ecosystem.

Tất cả thông tin này đều có thể truy cập được bằng chương trình (programmatically). Tất cả dữ liệu này cho phép chúng ta làm cho humane registry của mình mạnh mẽ hơn so với một trang wiki đơn giản vốn chắc chắn sẽ trở nên lỗi thời. Thay vào đó, chúng ta nên dùng nó để khai thác và hiển thị tất cả thông tin mà hệ thống của ta phát ra. Bằng cách tạo ra các dashboard tùy chỉnh, chúng ta có thể tổng hợp lại kho thông tin khổng lồ đang có sẵn để giúp hiểu rõ hệ sinh thái của mình.

By all means, start with something as simple as a static web page or wiki that perhaps scrapes in a bit of data from the live system. But look to pull in more and more information over time. Making this information readily available is a key tool for managing the emerging complexity that will come from running these systems at scale.

Bằng mọi giá, hãy bắt đầu với một thứ đơn giản như một trang web tĩnh hoặc wiki mà có lẽ thu thập (scrape) một chút dữ liệu từ hệ thống đang chạy. Nhưng hãy hướng tới việc kéo về ngày càng nhiều thông tin hơn theo thời gian. Việc làm cho thông tin này luôn sẵn sàng là một công cụ then chốt để quản lý sự phức tạp đang nổi lên khi vận hành các hệ thống này ở quy mô lớn.

I've spoken with a number of companies that have had these issues and that ended up creating simple in-house registries to help collate metadata around services. Some of these registries simply crawl source code repositories, looking for metadata files to **build** up a list of services out there. This information can be merged with real data coming from **service** discovery systems like Consul or etcd to build up a richer picture of what is running and whom you could speak to about it.

Tôi đã trò chuyện với một số công ty từng gặp những vấn đề này và họ cuối cùng đã tạo ra các registry nội bộ đơn giản để giúp tập hợp metadata xoay quanh các dịch vụ. Một số registry kiểu này chỉ đơn giản là rà quét (crawl) các repository mã nguồn, tìm kiếm các tệp metadata để xây dựng nên danh sách các dịch vụ đang tồn tại. Thông tin này có thể được hợp nhất với dữ liệu thực đến từ các hệ thống service discovery như Consul hoặc etcd để dựng nên một bức tranh phong phú hơn về những gì đang chạy và bạn có thể trao đổi với ai về chúng.

The Financial Times created Biz Ops to help address this problem. The company has several hundred services developed by teams all over the world. The Biz Ops tool (Figure 5-8) gives the company a single place where you can find out lots of useful information about its microservices, in addition to information about other IT infrastructure services such as networks and file servers. Built on top of a graph **database**, Biz Ops has a lot of flexibility about what data it gathers and how the information can be modeled.

Financial Times đã tạo ra Biz Ops để giúp giải quyết vấn đề này. Công ty này có vài trăm dịch vụ được phát triển bởi các nhóm trên khắp thế giới. Công cụ Biz Ops (Hình 5-8) cho công ty một nơi duy nhất để bạn có thể tìm thấy rất nhiều thông tin hữu ích về microservices của họ, bên cạnh thông tin về các dịch vụ hạ tầng CNTT khác như mạng và file server. Được xây dựng trên nền một cơ sở dữ liệu đồ thị (graph database), Biz Ops có rất nhiều sự linh hoạt về việc thu thập dữ liệu nào và cách mô hình hóa thông tin.

FT

Biz Ops Admin

API explorer

Feedback/bugs

Looking for runbooks?

ABOUT

SEARCH

MY STUFF

ADD SOMETHING

REPORT (BETA)

System: FT.com Article Page

General information

Ownership & knowledge

Technical overview

Data governance

Related resources

Failover

Data recovery

Release

Key Management

Monitoring

Troubleshooting

More Information

Service Operability Review

Miscellaneous

System: FT.com Article Page

Show the definition of "System" ▾

View runbook

View Heimdall dashboard

General information ✎

Delete

Edit

Code ⓘ

next-article

Name ⓘ

FT.com Article Page

Description ⓘ

A service that provides the user facing layer of articles for [FT.com](#)

Primary URL ⓘ

<https://www.ft.com/content/00b2e3c8-e2b0-11e8-a6e5-792428919cee>

Service tier ⓘ

Platinum

Lifecycle stage ⓘ

Production

Ownership & knowledge ✎

Delivered by team ⓘ

[Next](#)

Supported by team ⓘ

[Next](#)

Stakeholders ⓘ

Known about by ⓘ

Technical overview ✎

Host platform ⓘ

Heroku

Architecture ⓘ

next-article serves the following routes: `^/content/([a-f0-9]+)-([a-f0-9]+)-([a-f0-9]+)-([a-f0-9]+)-([a-f0-9]+)`

Figure 5-8. The Financial Times Biz Ops tool, which collates information about its microservices

Hình 5-8. Công cụ Biz Ops của Financial Times, vốn tổng hợp thông tin về microservices của họ

The Biz Ops tool goes further than most of the similar tools I have seen, however. The tool calculates what it calls a System Operability Score, as shown in Figure 5-9 . The idea is that there are certain things that services and their teams should do to ensure the services can be easily operated. This can range from making sure the teams have provided the correct information in the registry to ensuring the services have proper health checks. The System Operability Score, once calculated, allows teams to see at a glance if there are things that need to be fixed.

Tuy nhiên, công cụ Biz Ops còn đi xa hơn hầu hết các công cụ tương tự mà tôi từng thấy. Công cụ này tính toán cái mà nó gọi là System Operability Score (Điểm vận hành hệ thống), như minh họa trong Hình 5-9 . Ý tưởng là có một số việc nhất định mà các dịch vụ và nhóm của chúng nên làm để đảm bảo dịch vụ có thể được vận hành dễ dàng. Điều này có thể từ việc đảm bảo các nhóm đã cung cấp thông tin chính xác trong registry cho đến việc đảm bảo các dịch vụ có health check phù hợp. Một khi được tính toán, System Operability Score cho phép các nhóm nhìn thoáng qua là biết liệu có những thứ cần phải sửa hay không.

This is a growing space. In the open source world, Spotify’s Backstage tool offers a mechanism for building a service catalog like Biz Ops, with a plug-in model to allow for sophisticated additions, such as being able to trigger the creation of a new **microservice** or pulling in live information from a **Kubernetes** cluster. Ambassador’s own Service Catalog is more narrowly focused on visibility of services in Kubernetes, which means it might not have as much general appeal as something like the FT’s Biz Ops, but it’s nonetheless good to see some new takes on this idea that are more generally available.

Đây là một lĩnh vực đang phát triển. Trong thế giới mã nguồn mở, công cụ Backstage của Spotify cung cấp một cơ chế để xây dựng một danh mục dịch vụ (service catalog) như Biz Ops, với một mô hình plug-in cho phép bổ sung các tính năng tinh vi, chẳng hạn như có thể kích hoạt việc tạo một microservice mới hoặc kéo về thông tin trực tiếp từ một cụm Kubernetes. Service Catalog riêng của Ambassador tập trung hẹp hơn vào khả năng thấy được các dịch vụ trong Kubernetes, nghĩa là nó có thể không có sức hấp dẫn chung như một thứ gì đó kiểu Biz Ops của FT, nhưng dù sao vẫn rất tốt khi thấy một vài cách tiếp cận mới cho ý tưởng này được phổ biến rộng rãi hơn.

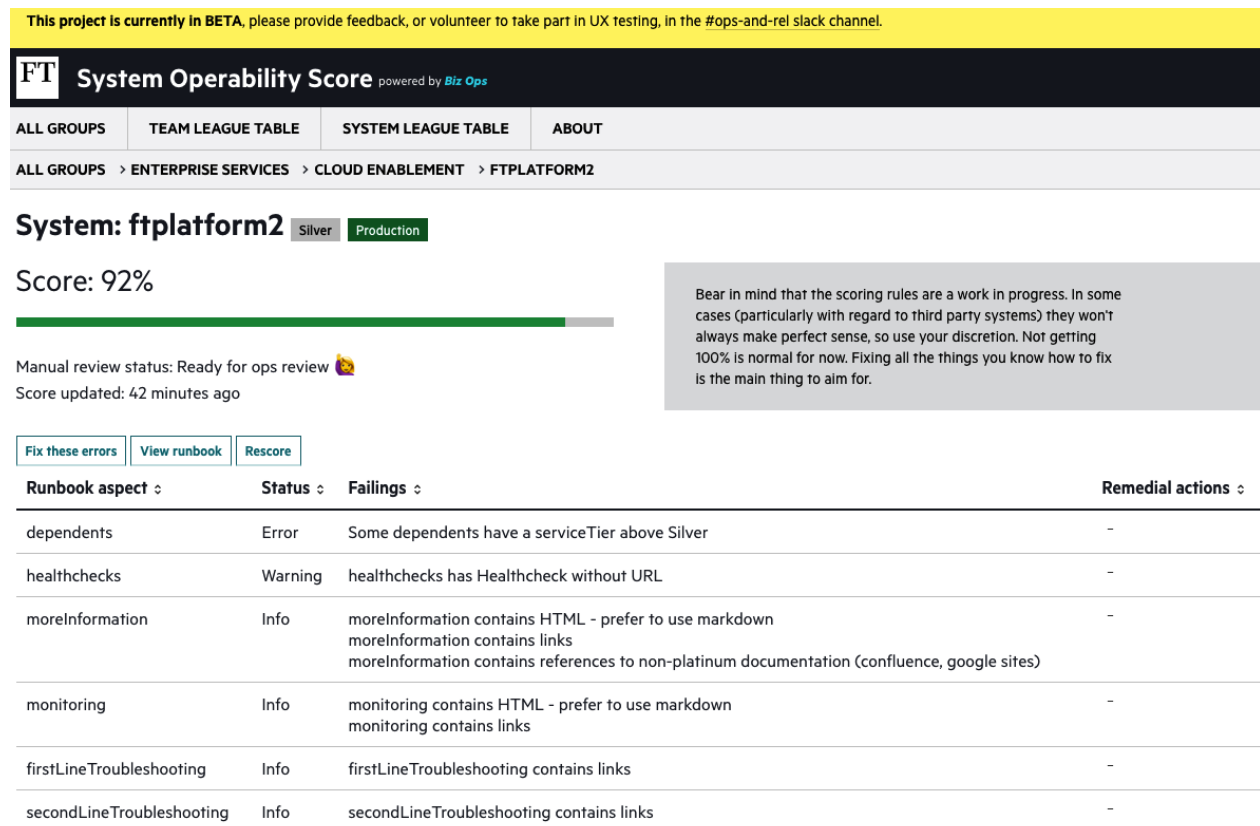


Figure 5-9. An example of the Service Operability Score for a microservice at the Financial Times

Hình 5-9. Một ví dụ về Service Operability Score cho một microservice tại Financial Times

Summary — Tóm tắt

So we’ve covered a lot of ground in this chapter—let’s break some of it down:

Vậy là chúng ta đã đề cập đến rất nhiều nội dung trong chương này—hãy điểm lại một số trong đó:

- To begin with, ensure that the problem you are trying to solve guides your technology choice. Based on your context and your preferred communication style, select the technology that is most appropriate for you—don't fall into the trap of picking the technology first. The summary of styles of inter-microservice communication, first introduced in Chapter 4 and shown again in Figure 5-10, can help guide your decision making, but just following this model isn't a replacement for sitting down and thinking about your own situation.

- Trước tiên, hãy đảm bảo rằng vấn đề bạn đang cố giải quyết dẫn dắt lựa chọn công nghệ của bạn. Dựa trên ngữ cảnh và phong cách giao tiếp ưa thích của mình, hãy chọn công nghệ phù hợp nhất với bạn—đừng rơi vào cái bẫy chọn công nghệ trước. Bản tóm tắt các phong cách giao tiếp giữa các microservice, lần đầu được giới thiệu ở Chương 4 và được thể hiện lại trong Hình 5-10, có thể giúp định hướng việc ra quyết định của bạn, nhưng việc chỉ làm theo mô hình này không thể thay thế cho việc ngồi xuống và suy nghĩ về tình huống của riêng bạn.

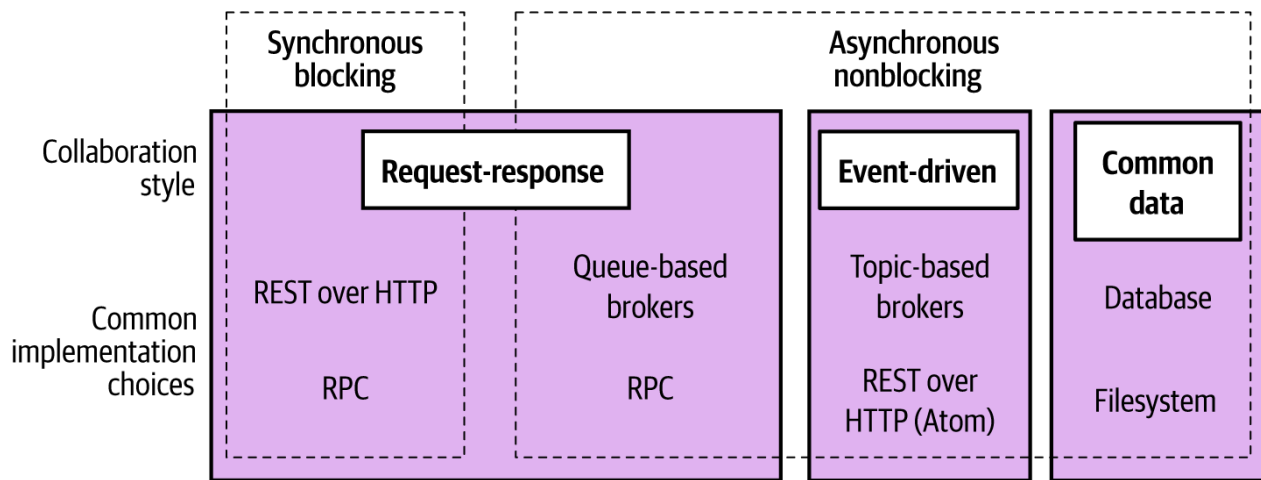


Figure 5-10. Different styles of inter-microservice communication, along with example implementing technologies

Hình 5-10. Các phong cách giao tiếp khác nhau giữa các microservice, kèm theo ví dụ về công nghệ triển khai

- Whatever choice you make, consider the use of schemas, in part to help make your contracts more explicit but also to help catch accidental breaking changes.
- Where possible, strive to make changes that are backward compatible to ensure that **independent deployability** remains a possibility.
- If you do have to make backward-incompatible changes, find a way to allow consumers time to upgrade to avoid lockstep deployments.
- Think about what you can do to help surface information about your endpoints to humans—consider the use of humane registries and the like to help make sense of the chaos.
 - Dù bạn chọn gì đi nữa, hãy cân nhắc việc sử dụng schema, một phần để giúp các contract của bạn rõ ràng hơn nhưng cũng để giúp phát hiện những thay đổi gây phá vỡ (breaking change) ngoài ý muốn.
 - Khi có thể, hãy nỗ lực thực hiện những thay đổi tương thích ngược (backward compatible) để đảm bảo khả năng triển khai độc lập vẫn còn là một khả năng.
 - Nếu bạn buộc phải thực hiện những thay đổi không tương thích ngược, hãy tìm cách cho phép bên tiêu dùng (consumer) có thời gian nâng cấp để tránh việc triển khai đồng loạt (lockstep deployment).

- Hãy nghĩ về những gì bạn có thể làm để giúp phơi bày thông tin về các endpoint của mình cho con người—cân nhắc việc sử dụng humane registry và những thứ tương tự để giúp hiểu được sự hỗn loạn.

We've looked at how we can implement a call between two microservices, but what happens when we need to coordinate operations between multiple microservices? That will be the focus of our next chapter.

Chúng ta đã xem cách triển khai một lời gọi giữa hai microservice, nhưng điều gì xảy ra khi chúng ta cần điều phối các thao tác giữa nhiều microservice? Đó sẽ là trọng tâm của chương tiếp theo.

CHAPTER 6 Workflow — Chương 6: Workflow

In the previous two chapters, we’ve looked at aspects of microservices related to how one microservice talks to another. But what happens when we want multiple microservices to collaborate, perhaps to implement a business process? How we model and implement these sorts of workflows in distributed systems can be a tricky thing to get right.

Trong hai chương trước, chúng ta đã xem xét các khía cạnh của microservice liên quan đến cách một microservice giao tiếp với một microservice khác. Nhưng điều gì xảy ra khi chúng ta muốn nhiều microservice cộng tác, có lẽ để triển khai một quy trình nghiệp vụ? Cách chúng ta mô hình hóa và triển khai những kiểu workflow (luồng công việc) này trong các hệ thống phân tán có thể là một việc khó làm cho đúng.

In this chapter, we’ll look at the pitfalls associated with using distributed transactions to solve this problem, and we’ll also look at sagas—a concept that can help us model our microservice workflows in a much more satisfactory manner.

Trong chương này, chúng ta sẽ xem xét những cạm bẫy liên quan đến việc sử dụng giao dịch phân tán (distributed transaction) để giải quyết vấn đề này, và chúng ta cũng sẽ xem xét saga—một khái niệm có thể giúp chúng ta mô hình hóa các workflow của microservice theo cách thỏa đáng hơn nhiều.

Database Transactions — Giao dịch cơ sở dữ liệu

Generically speaking, when we think of a **transaction** in the context of computing, we think of one or more actions that are going to occur that we want to treat as a single unit. When making multiple changes as part of the same overall operation, we want to confirm whether all the changes have been made. We also want a way to clean up after ourselves if an error occurs while these changes are happening. Typically, this results in us using something like a database transaction.

Nói một cách tổng quát, khi chúng ta nghĩ về một giao dịch (transaction) trong bối cảnh tính toán, chúng ta nghĩ đến một hoặc nhiều hành động sẽ diễn ra mà ta muốn xử lý như một đơn vị duy nhất. Khi thực hiện nhiều thay đổi như một phần của cùng một thao tác tổng thể, chúng ta muốn xác nhận liệu tất cả các thay đổi đã được thực hiện hay chưa. Chúng ta cũng muốn có một cách để dọn dẹp sau khi mình làm nếu có lỗi xảy ra trong lúc các thay đổi này đang diễn ra. Thông thường, điều này dẫn đến việc chúng ta sử dụng một thứ gì đó như một giao dịch cơ sở dữ liệu.

With a database, we use a transaction to ensure that one or more state changes have been made successfully. This could include data being removed, inserted, or changed. In a relational database,

this could involve multiple tables being updated within a single transaction.

Với một cơ sở dữ liệu, chúng ta sử dụng một giao dịch để đảm bảo rằng một hoặc nhiều thay đổi trạng thái đã được thực hiện thành công. Điều này có thể bao gồm việc dữ liệu bị xóa, được chèn vào, hoặc bị thay đổi. Trong một cơ sở dữ liệu quan hệ, điều này có thể liên quan đến việc nhiều bảng được cập nhật trong phạm vi một giao dịch duy nhất.

ACID Transactions — Giao dịch ACID

Typically when we talk about database transactions, we are talking about **ACID** transactions. ACID is an acronym outlining the key properties of database transactions that lead to a system we can rely on to ensure the durability and consistency of our data storage. ACID stands for atomicity, consistency, isolation, and durability, and here is what these properties give us:

Thông thường khi chúng ta nói về giao dịch cơ sở dữ liệu, ta đang nói về giao dịch ACID. ACID là một từ viết tắt phác thảo các thuộc tính chính của giao dịch cơ sở dữ liệu, vốn dẫn đến một hệ thống mà ta có thể tin cậy để đảm bảo tính bền vững (durability) và tính nhất quán (consistency) của việc lưu trữ dữ liệu. ACID là viết tắt của atomicity (tính nguyên tử), consistency (tính nhất quán), isolation (tính cô lập), và durability (tính bền vững), và đây là những gì các thuộc tính này mang lại cho chúng ta:

Atomicity Ensures that the operations attempted within the transaction either all complete or all fail. If any of the changes we're trying to make fail for some reason, then the whole operation is aborted, and it's as though no changes were ever made.

Atomicity (Tính nguyên tử) Đảm bảo rằng các thao tác được thử thực hiện trong giao dịch hoặc tất cả đều hoàn thành hoặc tất cả đều thất bại. Nếu bất kỳ thay đổi nào ta đang cố thực hiện thất bại vì một lý do nào đó, thì toàn bộ thao tác bị hủy bỏ, và mọi thứ như thể chưa có thay đổi nào từng được thực hiện.

Consistency When changes are made to our database, we ensure it is left in a valid, consistent state.

Consistency (Tính nhất quán) Khi các thay đổi được thực hiện đối với cơ sở dữ liệu của chúng ta, ta đảm bảo rằng nó được để lại ở một trạng thái hợp lệ, nhất quán.

Isolation Allows multiple transactions to operate at the same time without interfering. This is achieved by ensuring that any interim state changes made during one transaction are invisible to other transactions.

Isolation (Tính cô lập) Cho phép nhiều giao dịch hoạt động cùng lúc mà không can thiệp lẫn nhau. Điều này đạt được bằng cách đảm bảo rằng bất kỳ thay đổi trạng thái trung gian nào được thực hiện trong một giao dịch đều không thể thấy được bởi các giao dịch khác.

Durability Makes sure that once a transaction has been completed, we are confident the data won't get lost in the event of some system failure.

Durability (Tính bền vững) Đảm bảo rằng một khi một giao dịch đã được hoàn thành, chúng ta tin chắc rằng dữ liệu sẽ không bị mất trong trường hợp xảy ra một sự cố hệ thống nào đó.

It's worth noting that not all databases provide ACID transactions. All relational database systems I've ever used do, as do many of the newer NoSQL databases like Neo4j. MongoDB for many years supported ACID transactions only on changes being made to a single document, which could cause issues if you wanted to make an atomic update to more than one document. 1

Đáng lưu ý rằng không phải tất cả cơ sở dữ liệu đều cung cấp giao dịch ACID. Tất cả các hệ cơ sở dữ liệu quan hệ mà tôi từng dùng đều có, cũng như nhiều cơ sở dữ liệu NoSQL mới hơn như Neo4j. Trong nhiều năm, MongoDB chỉ hỗ trợ giao dịch ACID đối với các thay đổi được thực hiện trên một tài liệu (document) duy nhất, điều này có thể gây ra vấn đề nếu bạn muốn thực hiện một cập nhật nguyên tử trên nhiều hơn một tài liệu. 1

This isn't the book for a detailed exploration into these concepts; I've certainly simplified some of these descriptions for the sake of brevity. For those who would like to explore these concepts further, I recommend *Designing Data-Intensive Applications*. 2 We'll mostly concern ourselves with atomicity in what follows. That's not to say that the other properties aren't also important, but grappling with how to deal with the atomicity of database operations tends to be the first issue we hit when we start breaking apart functionality into microservices.

Đây không phải là cuốn sách dành cho việc khám phá chi tiết những khái niệm này; tôi chắc chắn đã đơn giản hóa một số mô tả vì lý do ngắn gọn. Đối với những ai muốn khám phá những khái niệm này sâu hơn, tôi khuyên đọc *Designing Data-Intensive Applications*. 2 Chúng ta sẽ chủ yếu quan tâm đến tính nguyên tử trong những phần tiếp theo. Điều đó không có nghĩa là các thuộc tính khác không quan trọng, nhưng việc vật lộn với cách xử lý tính nguyên tử của các thao tác cơ sở dữ liệu thường là vấn đề đầu tiên ta gặp phải khi bắt đầu chia tách chức năng thành các microservice.

1 This has changed, with support for multidocument ACID transactions being released as part of Mongo 4.0. I haven't used this feature of Mongo myself; I just know it exists!

1 Điều này đã thay đổi, với hỗ trợ cho giao dịch ACID đa tài liệu được phát hành như một phần của Mongo 4.0. Bản thân tôi chưa dùng tính năng này của Mongo; tôi chỉ biết là nó tồn tại!

2 Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol: O'Reilly, 2017).

2 Martin Kleppmann, *Designing Data-Intensive Applications* (Sebastopol: O'Reilly, 2017).

Still ACID, but Lacking Atomicity? — Vẫn là ACID, nhưng thiếu tính nguyên tử?

I want to be clear that we can still use ACID-style transactions when using microservices. A microservice is free to use an ACID transaction for operations to its own database, for example. It's just that the scope of these transactions is reduced to state change that happens locally within that single microservice. Consider Figure 6-1. Here, we are keeping track of the process involved in onboarding a new customer to MusicCorp. We've reached the end of the process, which involves changing the Status of customer 2346 from PENDING to VERIFIED. As the enrollment is now complete, we also want to remove the matching row from the PendingEnrollments table. With a single database, this is done in the scope of a single ACID database transaction— either both of these state changes occur, or neither occurs.

Tôi muốn nói rõ rằng chúng ta vẫn có thể sử dụng các giao dịch kiểu ACID khi dùng microservice. Một microservice hoàn toàn có thể tự do sử dụng một giao dịch ACID cho các thao tác đối với cơ sở dữ liệu của chính nó, chẳng hạn. Chỉ là phạm vi của các giao dịch này bị thu hẹp lại còn các thay đổi trạng thái xảy ra cục bộ bên trong microservice duy nhất đó. Hãy xem Hình 6-1. Ở đây, chúng ta đang theo dõi quy trình liên quan đến việc đưa một khách hàng mới vào MusicCorp. Chúng ta đã đến cuối quy trình, vốn liên quan đến việc thay đổi Status của khách hàng 2346 từ PENDING sang VERIFIED. Vì việc đăng ký giờ đã hoàn tất, chúng ta cũng muốn xóa dòng tương ứng khỏi bảng PendingEnrollments

. Với một cơ sở dữ liệu duy nhất, việc này được thực hiện trong phạm vi của một giao dịch cơ sở dữ liệu ACID duy nhất—hoặc cả hai thay đổi trạng thái này đều xảy ra, hoặc không cái nào xảy ra.

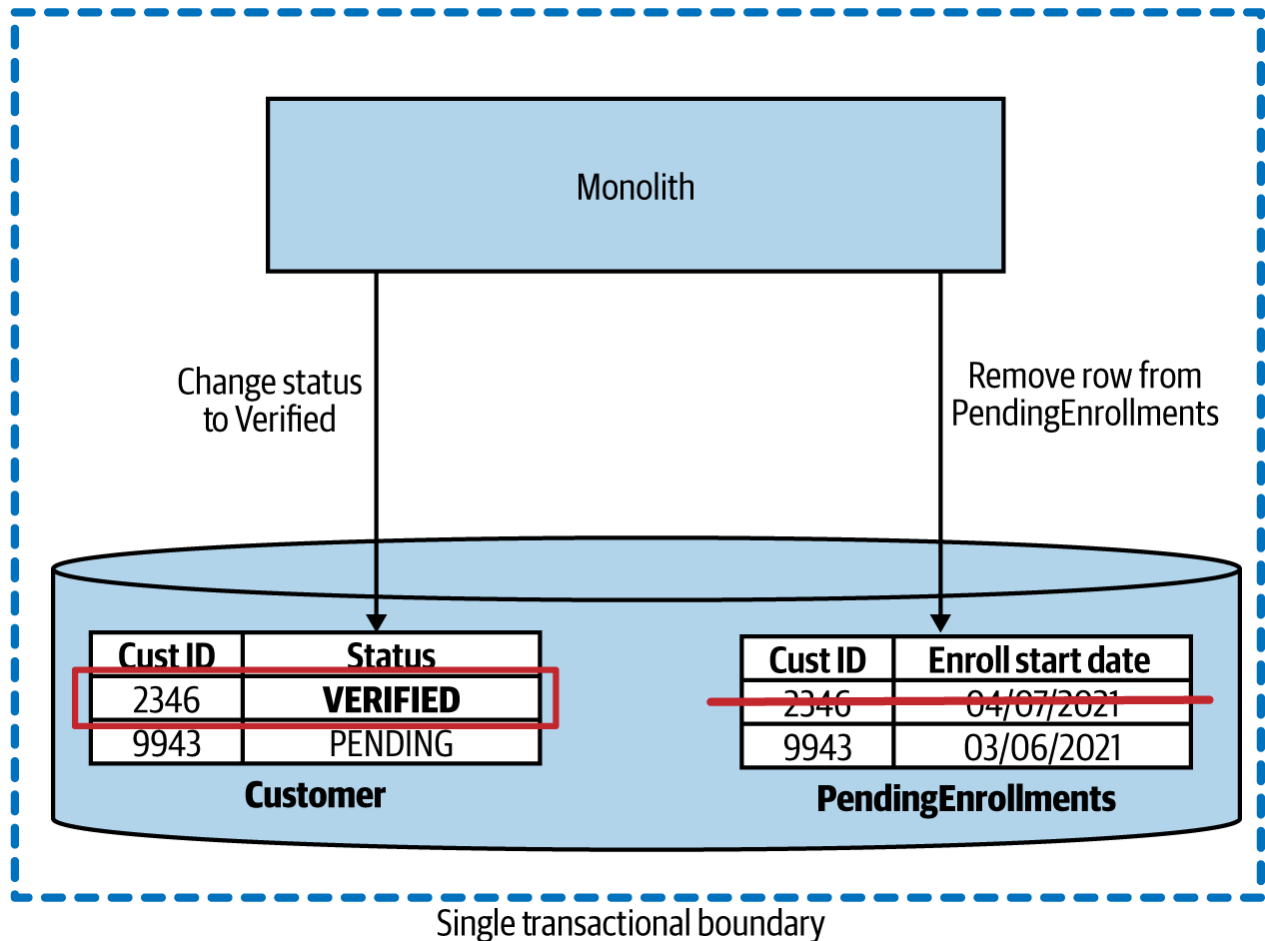


Figure 6-1. Updating two tables in the scope of a single ACID transaction

Hình 6-1. Cập nhật hai bảng trong phạm vi một giao dịch ACID duy nhất

Compare this with Figure 6-2 , where we're making exactly the same change, but each change is made in a different database. This means there are two transactions to consider, each of which could work or fail independently of the other.

Hãy so sánh điều này với Hình 6-2 , nơi chúng ta đang thực hiện chính xác cùng một thay đổi, nhưng mỗi thay đổi được thực hiện ở một cơ sở dữ liệu khác nhau. Điều này có nghĩa là có hai giao dịch cần cân nhắc, mỗi giao dịch có thể thành công hoặc thất bại độc lập với giao dịch kia.

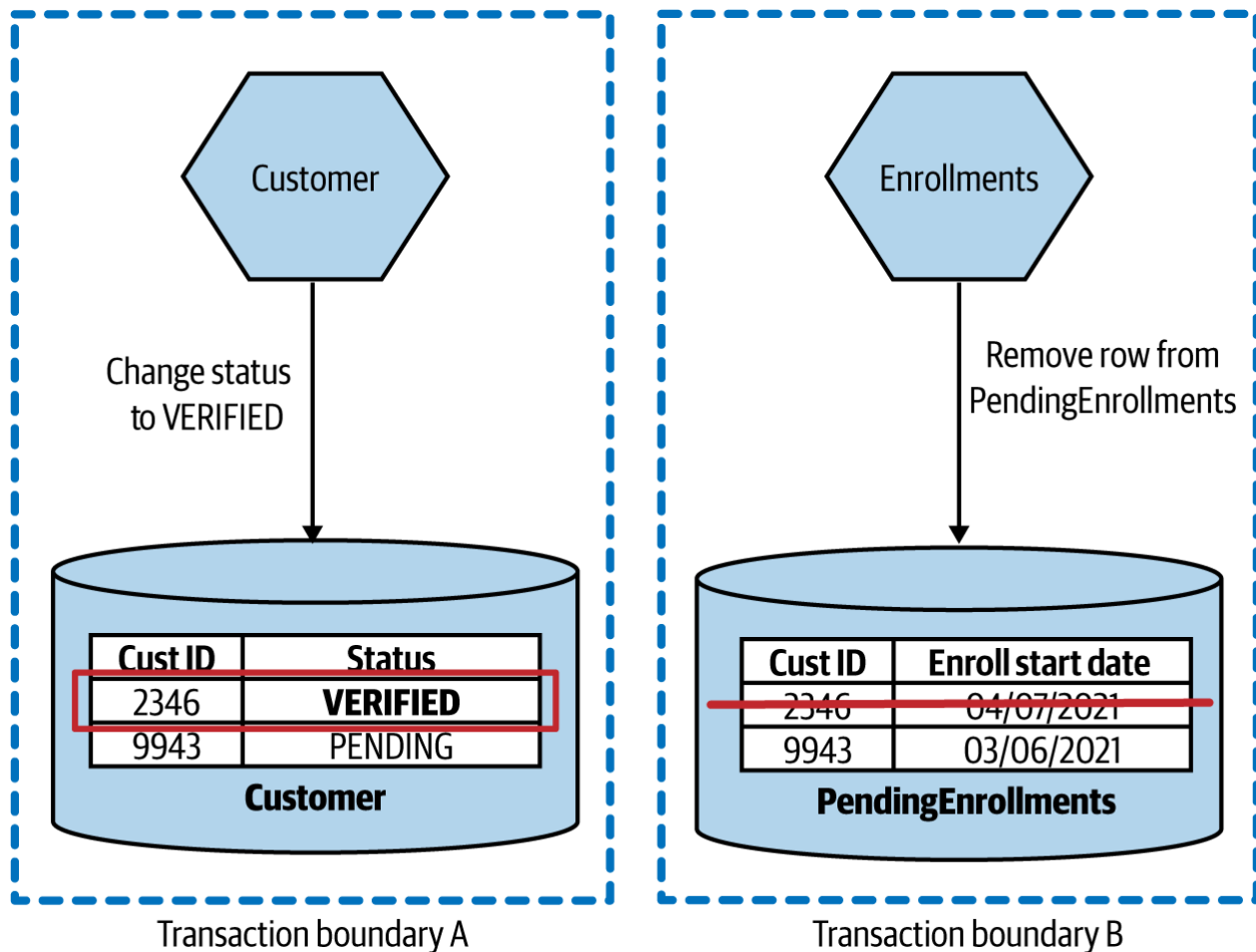


Figure 6-2. Changes made by the Customer and Enrollments microservices are now done in the scope of two different transactions

Hình 6-2. Các thay đổi do microservice Customer và Enrollments thực hiện giờ đây được làm trong phạm vi của hai giao dịch khác nhau

We could decide to sequence these two transactions, of course, removing a row from the PendingEnrollments table only if we could change the row in the Customer table. But we'd still have to reason about what to do if the deletion from the PendingEnrollments table then failed—all logic that we'd need to implement ourselves. Being able to reorder steps to better handle these use cases can be a really useful idea, though (one we'll come back to when we explore sagas). But fundamentally, we have to accept that by decomposing this operation into two separate database transactions, we've lost guaranteed atomicity of the operation as a whole.

Tất nhiên chúng ta có thể quyết định sắp xếp tuần tự hai giao dịch này, chỉ xóa một dòng khỏi bảng PendingEnrollments nếu chúng ta đã có thể thay đổi dòng trong bảng Customer. Nhưng chúng ta vẫn sẽ phải suy luận về việc cần làm gì nếu việc xóa khỏi bảng PendingEnrollments sau đó thất bại—tất cả logic mà chúng ta sẽ phải tự triển khai. Tuy nhiên, khả năng sắp xếp lại thứ tự các bước để xử lý những trường hợp này tốt hơn có thể là một ý tưởng thực sự hữu ích (một ý mà ta sẽ quay lại khi khám phá saga). Nhưng về cơ bản, chúng ta phải chấp nhận rằng bằng cách phân rã thao tác này thành hai giao dịch cơ sở dữ liệu riêng biệt, chúng ta đã mất đi tính nguyên tử được đảm bảo của thao tác như một tổng thể.

This lack of atomicity can start to cause significant problems, especially if we are migrating systems that previously relied on this property. Normally, the first option that people start considering is still using a single transaction, but one that now spans multiple processes—a **distributed transaction**. Unfortunately, as we'll see, distributed transactions may not be the right way forward. Let's look at one of the most common algorithms for implementing distributed transactions, the **two-phase commit**, as a way of exploring the challenges associated with distributed transactions as a whole.

Việc thiếu tính nguyên tử này có thể bắt đầu gây ra những vấn đề đáng kể, đặc biệt nếu chúng ta đang di trú (migrate) các hệ thống trước đây vốn dựa vào thuộc tính này. Thông thường, lựa chọn đầu tiên mà mọi người bắt đầu cân nhắc vẫn là sử dụng một giao dịch duy nhất, nhưng là một giao dịch giờ đây trải rộng trên nhiều tiến trình—một giao dịch phân tán. Tiếc thay, như chúng ta sẽ thấy, giao dịch phân tán có thể không phải là con đường đúng đắn để tiến lên. Hãy xem một trong những thuật toán phổ biến nhất để triển khai giao dịch phân tán, đó là two-phase commit (2PC), như một cách để khám phá những thách thức liên quan đến giao dịch phân tán nói chung.

Distributed Transactions—Two-Phase Commits — Giao dịch phân tán—Two-Phase Commit

The two-phase commit algorithm (sometimes shortened to 2PC) is frequently used in an attempt to give us the ability to make transactional changes in a **distributed system**, where multiple separate processes may need to be updated as part of the overall operation. Distributed transactions, and two-phased commits more specifically, are frequently considered by teams moving to microservice architectures as a way of solving challenges they face. But as we'll see, they might not solve your problems and may bring even more confusion to your system.

Thuật toán two-phase commit (đôi khi được viết tắt là 2PC) thường được sử dụng nhằm cố gắng cho chúng ta khả năng thực hiện các thay đổi mang tính giao dịch trong một hệ thống phân tán, nơi nhiều tiến trình riêng biệt có thể cần được cập nhật như một phần của thao tác tổng thể. Giao dịch phân tán, và two-phase commit nói riêng, thường được các nhóm chuyển sang kiến trúc microservice cân nhắc như một cách giải quyết những thách thức họ gặp phải. Nhưng như chúng ta sẽ thấy, chúng có thể không giải quyết được vấn đề của bạn và có thể mang lại thậm chí nhiều sự rối ren hơn cho hệ thống của bạn.

The 2PC is broken into two phases (hence the name two-phase commit): a voting phase and a commit phase. During the voting phase , a central coordinator contacts all the workers who are going to be part of the transaction and asks for confirmation as to whether or not some state change can be made. In Figure 6-3 , we see two requests: one to change a customer status to VERIFIED , and another to remove a row from our PendingEnrollments table. If all workers agree that the state change they are asked for can take place, the algorithm proceeds to the next phase. If any worker says the change cannot take place, perhaps because the requested state change violates some local condition, the entire operation aborts.

2PC được chia thành hai pha (do đó có tên two-phase commit): một pha bỏ phiếu (voting phase) và một pha commit (commit phase). Trong pha bỏ phiếu , một bộ điều phối trung tâm (central coordinator) liên hệ với tất cả các worker sẽ tham gia vào giao dịch và yêu cầu xác nhận xem một thay đổi trạng thái nào đó có thể được thực hiện hay không. Trong Hình 6-3 , chúng ta thấy hai yêu cầu: một để thay đổi trạng thái khách hàng thành VERIFIED , và một để xóa một dòng khỏi bảng PendingEnrollments của chúng ta. Nếu tất cả các worker đồng ý rằng thay đổi trạng thái mà chúng được yêu cầu có thể diễn ra, thuật toán tiến sang pha tiếp theo. Nếu bất kỳ worker nào nói rằng thay đổi không thể diễn ra, có lẽ vì thay đổi

trạng thái được yêu cầu vi phạm một điều kiện cục bộ nào đó, thì toàn bộ thao tác bị hủy bỏ.

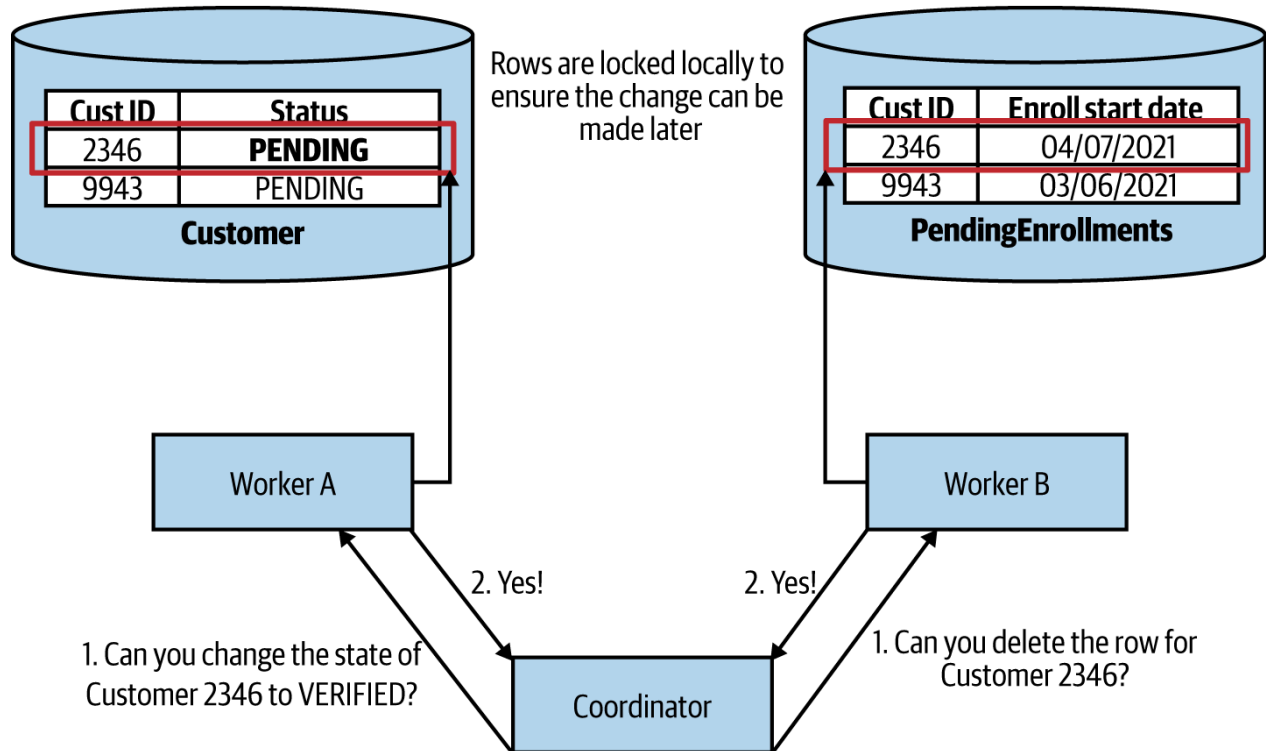


Figure 6-3. In the first phase of a two-phase commit, workers vote to decide if they can carry out some local state change

Hình 6-3. Trong pha đầu tiên của một two-phase commit, các worker bỏ phiếu để quyết định xem chúng có thể thực hiện một thay đổi trạng thái cục bộ nào đó hay không

It's important to highlight that the change does not take effect immediately after a worker indicates that it can make the change. Instead, the worker is guaranteeing that it will be able to make that change at some point in the future. How would the worker make such a guarantee? In Figure 6-3 , for example, Worker A has said it will be able to change the state of the row in the Customer table to update that specific customer's status to VERIFIED . What if a different operation at some later point deletes the row, or makes some other smaller change that nonetheless means that a change to VERIFIED later is invalid? To guarantee that the change to VERIFIED can be made later, Worker A will likely have to lock the record to ensure that other changes cannot take place.

Điều quan trọng cần nhấn mạnh là thay đổi không có hiệu lực ngay lập tức sau khi một worker cho biết nó có thể thực hiện thay đổi. Thay vào đó, worker đang bảo đảm rằng nó sẽ có thể thực hiện thay đổi đó vào một thời điểm nào đó trong tương lai. Worker làm sao có thể đưa ra một bảo đảm như vậy? Trong Hình 6-3 , chẳng hạn, Worker A đã nói nó sẽ có thể thay đổi trạng thái của dòng trong bảng Customer để cập nhật trạng thái của khách hàng cụ thể đó thành VERIFIED . Nếu một thao tác khác vào một thời điểm sau đó xóa dòng đó, hoặc thực hiện một thay đổi nhỏ hơn nào khác mà dù sao cũng khiến cho việc thay đổi thành VERIFIED sau này trở nên không hợp lệ thì sao? Để bảo đảm rằng thay đổi thành VERIFIED có thể được thực hiện sau này, Worker A nhiều khả năng sẽ phải khóa (lock) bản ghi để đảm bảo rằng các thay đổi khác không thể diễn ra.

If any workers didn't vote in favor of the commit, a rollback message needs to be sent to all parties

to ensure that they can clean up locally, which allows the workers to **release** any locks they may be holding. If all workers agreed to make the change, we move to the commit phase, as in Figure 6-4. Here, the changes are actually made, and associated locks are released.

Nếu có bất kỳ worker nào không bỏ phiếu ủng hộ việc commit, một thông điệp rollback cần được gửi đến tất cả các bên để đảm bảo rằng chúng có thể dọn dẹp cục bộ, điều này cho phép các worker giải phóng bất kỳ khóa nào chúng có thể đang giữ. Nếu tất cả các worker đồng ý thực hiện thay đổi, chúng ta chuyển sang pha commit, như trong Hình 6-4. Ở đây, các thay đổi thực sự được thực hiện, và các khóa liên quan được giải phóng.

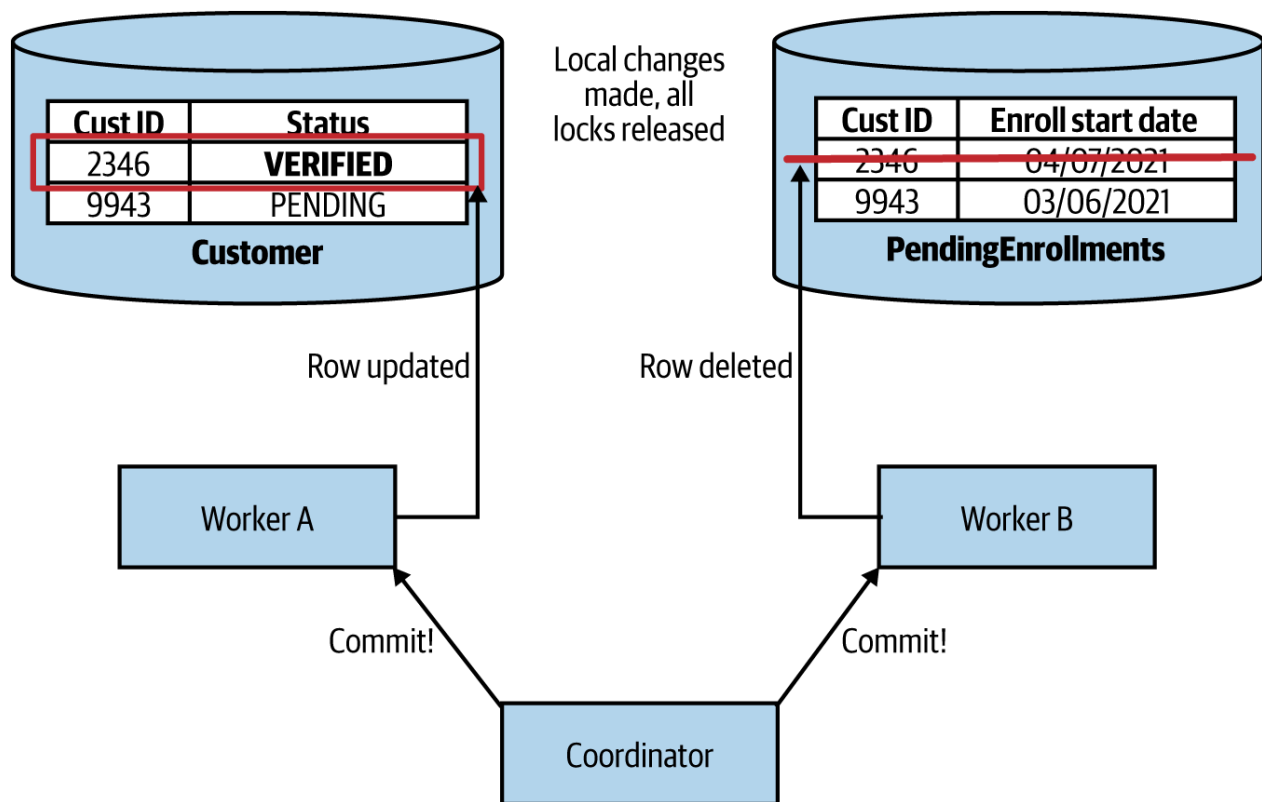


Figure 6-4. In the commit phase of a two-phase commit, changes are actually applied

Hình 6-4. Trong pha commit của một two-phase commit, các thay đổi thực sự được áp dụng

It's important to note that in such a system, we cannot in any way guarantee that these commits will occur at exactly the same time. The Coordinator needs to send the commit request to all participants, and that message could arrive and be processed at different times. This means that it's possible we could see the change made to Worker A but not yet to Worker B, if we could directly observe the state of either worker. The more latency there is between the Coordinator and the participants in the two-phase commit, and the more slowly the workers process the response, the wider this window of inconsistency might be. Coming back to our definition of ACID, isolation ensures that we don't see intermediate states during a transaction. But with this two-phase commit, we've lost that guarantee.

Điều quan trọng cần lưu ý là trong một hệ thống như vậy, chúng ta không thể bằng bất kỳ cách nào bảo đảm rằng các commit này sẽ xảy ra đúng vào cùng một thời điểm. Coordinator cần gửi yêu cầu commit đến tất cả các bên tham gia, và thông điệp đó có thể đến và được xử lý vào những thời điểm khác nhau. Điều này có nghĩa là có khả năng chúng ta có thể thấy thay đổi đã được thực hiện ở Worker A nhưng chưa ở Worker B, nếu chúng ta có thể trực tiếp quan sát trạng thái của một trong hai worker. Độ trễ (latency)

giữa Coordinator và các bên tham gia trong two-phase commit càng lớn, và các worker xử lý phản hồi càng chậm, thì cửa sổ không nhất quán này có thể càng rộng. Quay lại định nghĩa ACID của chúng ta, tính cô lập (isolation) đảm bảo rằng chúng ta không thấy các trạng thái trung gian trong một giao dịch. Nhưng với two-phase commit này, chúng ta đã mất đi sự bảo đảm đó.

When a two-phase commit works, at its heart it is very often just coordinating distributed locks. The workers need to lock local resources to ensure that the commit can take place during the second phase. Managing locks and avoiding deadlocks in a single-process system isn't fun. Now imagine the challenges of coordinating locks among multiple participants. It's not pretty.

Khi một two-phase commit hoạt động, ở cốt lõi của nó, rất thường thì nó chỉ đang điều phối các khóa phân tán (distributed lock). Các worker cần khóa các tài nguyên cục bộ để đảm bảo rằng việc commit có thể diễn ra trong pha thứ hai. Việc quản lý các khóa và tránh deadlock trong một hệ thống một-tiến-trình đã chẳng vui vẻ gì. Giờ hãy tưởng tượng những thách thức của việc điều phối các khóa giữa nhiều bên tham gia. Chẳng đẹp đẽ gì.

There are a host of failure modes associated with two-phase commits that we don't have time to explore. Consider the problem of a worker voting to proceed with the transaction but then not responding when asked to commit. What should we do then? Some of these failure modes can be handled automatically, but some can leave the system in such a state that things need to be fixed manually by an operator.

Có cả một loạt chế độ lỗi (failure mode) liên quan đến two-phase commit mà chúng ta không có thời gian để khám phá. Hãy xem xét vấn đề của một worker bỏ phiếu tiếp tục với giao dịch nhưng sau đó không phản hồi khi được yêu cầu commit. Khi đó chúng ta nên làm gì? Một số trong những chế độ lỗi này có thể được xử lý tự động, nhưng một số có thể để hệ thống ở một trạng thái mà mọi thứ cần được sửa thủ công bởi một người vận hành.

The more participants you have, and the more latency you have in the system, the more issues a two-phase commit will have. 2PC can be a quick way to inject huge amounts of latency into your system, especially if the scope of locking is large, or if the duration of the transaction is large. It's for this reason two-phase commits are typically used only for very short-lived operations. The longer the operation takes, the longer you've got resources locked!

Bạn càng có nhiều bên tham gia, và càng có nhiều độ trễ trong hệ thống, thì một two-phase commit sẽ càng gặp nhiều vấn đề. 2PC có thể là một cách nhanh chóng để bơm một lượng độ trễ khổng lồ vào hệ thống của bạn, đặc biệt nếu phạm vi khóa lớn, hoặc nếu thời lượng của giao dịch lớn. Chính vì lý do này mà two-phase commit thường chỉ được dùng cho các thao tác có vòng đời rất ngắn. Thao tác càng kéo dài, bạn càng giữ tài nguyên bị khóa lâu hơn!

Distributed Transactions—Just Say No — Giao dịch phân tán— Hãy cứ nói không

For all the reasons outlined so far, I strongly suggest you avoid the use of distributed transactions like the two-phase commit to coordinate changes in state across your microservices. So what else can you do?

Vì tất cả các lý do đã trình bày cho đến nay, tôi mạnh mẽ đề nghị bạn tránh việc sử dụng giao dịch phân tán như two-phase commit để điều phối các thay đổi trạng thái trên các microservice của mình. Vậy thì bạn có thể làm gì khác?

Well, the first option could be to just not split the data apart in the first place. If you have pieces of state that you want to manage in a truly atomic and consistent way, and you cannot work out how to sensibly get these characteristics without an ACID-style transaction, then leave that state in a single database, and leave the functionality that manages that state in a single service (or in your **monolith**). If you're in the process of working out where to split your monolith and what decompositions might be easy (or hard), then you could well decide that splitting apart data that is currently managed in a transaction is just too difficult to handle right now. Work on some other area of the system, and come back to this later.

Vâng, lựa chọn đầu tiên có thể là ngay từ đầu đừng chia tách dữ liệu ra. Nếu bạn có những mảnh trạng thái mà bạn muốn quản lý theo cách thực sự nguyên tử và nhất quán, và bạn không thể tìm ra cách hợp lý để đạt được những đặc tính này mà không có một giao dịch kiểu ACID, thì hãy để trạng thái đó trong một cơ sở dữ liệu duy nhất, và để chức năng quản lý trạng thái đó trong một dịch vụ duy nhất (hoặc trong monolith của bạn). Nếu bạn đang trong quá trình tìm ra nơi để chia tách monolith của mình và những cách phân rã nào có thể là dễ (hoặc khó), thì bạn hoàn toàn có thể quyết định rằng việc chia tách dữ liệu hiện đang được quản lý trong một giao dịch là quá khó để xử lý ngay lúc này. Hãy làm việc trên một khu vực nào khác của hệ thống, và quay lại điều này sau.

But what happens if you really do need to break this data apart, but you don't want all the pain of managing distributed transactions? How can you carry out operations in multiple services but avoid locking? What if the operation is going to take minutes, days, or perhaps even months? In cases like this, you may consider an alternative approach: sagas.

Nhưng điều gì xảy ra nếu bạn thực sự cần chia tách dữ liệu này ra, nhưng bạn không muốn tất cả nỗi đau của việc quản lý giao dịch phân tán? Làm sao bạn có thể thực hiện các thao tác trong nhiều dịch vụ nhưng tránh việc khóa? Nếu thao tác sẽ mất hàng phút, hàng ngày, hoặc thậm chí có lẽ hàng tháng thì sao? Trong những trường hợp như vậy, bạn có thể cân nhắc một cách tiếp cận khác: saga.

Database Distributed Transactions — Giao dịch phân tán cơ sở dữ liệu

I am arguing against the general use of distributed transactions to coordinate state change across microservices. In such situations, each microservice is managing its own local durable state (e.g., in its database). Distributed transactional algorithms are being used successfully for some large-scale databases, Google's Spanner being one such system. In this situation, the distributed transaction is being applied transparently from an application's point of view by the underlying database, and the distributed transaction is just being used to coordinate state changes within a single logical database (albeit one that might be distributed across multiple machines, and potentially across multiple data centers).

Tôi đang lập luận chống lại việc sử dụng chung chung giao dịch phân tán để điều phối thay đổi trạng thái trên các microservice. Trong những tình huống như vậy, mỗi microservice đang quản lý trạng thái bền vững cục bộ của chính nó (ví dụ, trong cơ sở dữ liệu của nó). Các thuật toán giao dịch phân tán đang được sử dụng thành công cho một số cơ sở dữ liệu quy mô lớn, Spanner của Google là một hệ thống như vậy. Trong tình huống này, giao dịch phân tán đang được áp dụng một cách trong suốt (transparently) từ góc nhìn của ứng dụng bởi cơ sở dữ liệu nằm dưới, và giao dịch phân tán chỉ đang được dùng để điều phối các thay đổi trạng thái trong phạm vi một cơ sở dữ liệu logic duy nhất (mặc dù là một cơ sở dữ liệu có thể được phân tán trên nhiều máy, và có khả năng trên nhiều trung tâm dữ

liệu).

What Google has managed to achieve with Spanner is impressive, but it's also worth noting that what it had to do to make this work gives you an idea of the challenges involved. Let's just say it involves very expensive data centers and satellite-based atomic clocks (really). For a good overview of how Spanner makes this work, I recommend the presentation “Google Cloud Spanner: Global Consistency at Scale.”³

Những gì Google đã đạt được với Spanner thật ấn tượng, nhưng cũng đáng lưu ý rằng những gì họ phải làm để khiến nó hoạt động cho bạn một ý niệm về những thách thức liên quan. Cứ tạm nói rằng nó liên quan đến những trung tâm dữ liệu rất đắt đỏ và đồng hồ nguyên tử dựa trên vệ tinh (thật đấy). Để có một cái nhìn tổng quan tốt về cách Spanner khiến điều này hoạt động, tôi khuyên xem bài thuyết trình “Google Cloud Spanner: Global Consistency at Scale.”³

Sagas — Saga

Unlike a two-phase commit, a **saga** is by design an algorithm that can coordinate multiple changes in state, but avoids the need for locking resources for long periods of time. A saga does this by modeling the steps involved as discrete activities that can be executed independently. Using sagas comes with the added benefit of forcing us to explicitly model our business processes, which can have significant benefits.

Khác với two-phase commit, một saga theo thiết kế là một thuật toán có thể điều phối nhiều thay đổi trạng thái, nhưng tránh được nhu cầu khóa tài nguyên trong những khoảng thời gian dài. Một saga làm điều này bằng cách mô hình hóa các bước liên quan thành những hoạt động rời rạc có thể được thực thi độc lập. Việc sử dụng saga đi kèm với lợi ích bổ sung là buộc chúng ta phải mô hình hóa một cách tường minh các quy trình nghiệp vụ của mình, điều này có thể mang lại những lợi ích đáng kể.

The core idea, first outlined in “Sagas” by Hector Garcia-Molina and Kenneth Salem,⁴ addresses how best to handle operations known as long lived transactions (LLTs). These transactions might take a long time (minutes, hours, or perhaps even days) and as part of that process require changes to be made to a database.

Ý tưởng cốt lõi, lần đầu được phác thảo trong “Sagas” của Hector Garcia-Molina và Kenneth Salem,⁴ giải quyết cách xử lý tốt nhất các thao tác được gọi là giao dịch tồn tại lâu (long lived transactions, LLT). Những giao dịch này có thể mất một thời gian dài (hàng phút, hàng giờ, hoặc thậm chí có lẽ hàng ngày) và như một phần của quy trình đó đòi hỏi các thay đổi phải được thực hiện đối với một cơ sở dữ liệu.

If you directly mapped an LLT to a normal database transaction, a single database transaction would span the entire life cycle of the LLT. This could result in multiple rows or even full tables being locked for long periods of time while the LLT is taking place, causing significant issues if other processes are trying to read or modify these locked resources.

Nếu bạn ánh xạ trực tiếp một LLT sang một giao dịch cơ sở dữ liệu thông thường, thì một giao dịch cơ sở dữ liệu duy nhất sẽ trải dài toàn bộ vòng đời của LLT. Điều này có thể dẫn đến việc nhiều dòng hoặc thậm chí toàn bộ các bảng bị khóa trong những khoảng thời gian dài trong khi LLT đang diễn ra, gây ra những vấn đề đáng kể nếu các tiến trình khác đang cố đọc hoặc sửa đổi các tài nguyên bị khóa này.

³ Robert Kubis, “Google Cloud Spanner: Global Consistency at Scale,” DevOxx, November 7, 2017, YouTube video, 33:22, <https://oreil.ly/XHvY5> . ⁴ Hector Garcia-Molina and Kenneth Salem, “Sagas,”

ACM Sigmod Record 16, no. 3 (1987): 249–59.

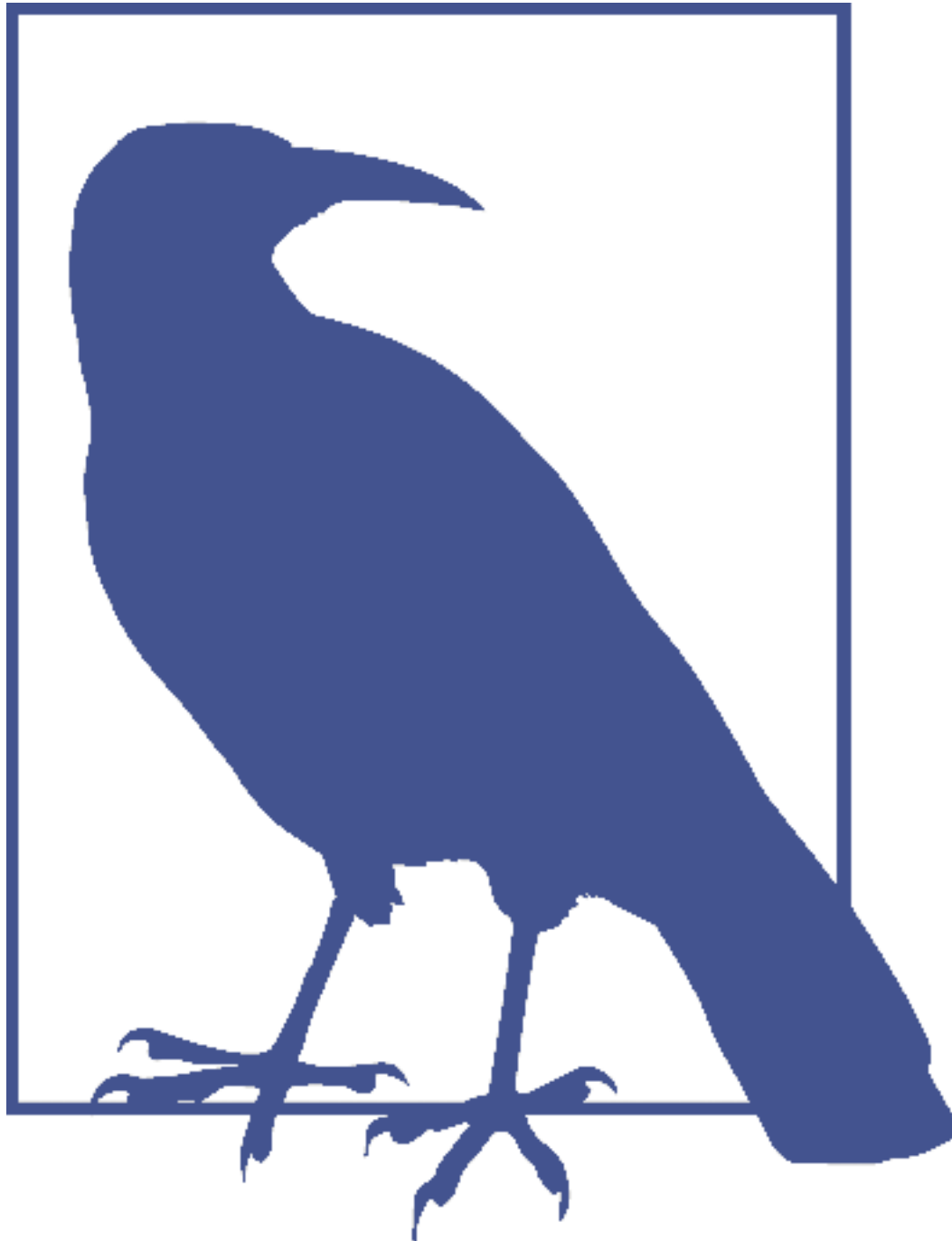
3 Robert Kubis, “Google Cloud Spanner: Global Consistency at Scale,” Devovx, November 7, 2017, YouTube video, 33:22, <https://oreil.ly/XHvY5>. 4 Hector Garcia-Molina and Kenneth Salem, “Sagas,” ACM Sigmod Record 16, no. 3 (1987): 249–59.

Instead, the authors of the paper suggest we should break down these LLTs into a sequence of transactions, each of which can be handled independently. The idea is that the duration of each of these “sub” transactions will be shorter, and will modify only part of the data affected by the entire LLT. As a result, there will be far less contention in the underlying database as the scope and duration of locks is greatly reduced.

Thay vào đó, các tác giả của bài báo gợi ý rằng chúng ta nên chia nhỏ những LLT này thành một chuỗi các giao dịch, mỗi giao dịch có thể được xử lý độc lập. Ý tưởng là thời lượng của mỗi giao dịch “con” này sẽ ngắn hơn, và sẽ chỉ sửa đổi một phần của dữ liệu bị ảnh hưởng bởi toàn bộ LLT. Kết quả là, sẽ có ít tranh chấp (contention) trong cơ sở dữ liệu nằm dưới hơn nhiều vì phạm vi và thời lượng của các khóa được giảm đi đáng kể.

While sagas were originally envisaged as a mechanism to help with LLTs acting against a single database, the model works just as well for coordinating change across multiple services. We can break a single business process into a set of calls that will be made to collaborating services—this is what constitutes a saga.

Mặc dù saga ban đầu được hình dung như một cơ chế để giúp với các LLT tác động lên một cơ sở dữ liệu duy nhất, mô hình này hoạt động tốt không kém để điều phối thay đổi trên nhiều dịch vụ. Chúng ta có thể chia một quy trình nghiệp vụ duy nhất thành một tập hợp các lời gọi sẽ được thực hiện đến các dịch vụ cộng tác—đây chính là điều cấu thành nên một saga.



Before we go any further, you need to understand that a saga does not give us atomicity in ACID terms such as we are used to with a normal database transaction. As we break the LLT into individual transactions, we don't have atomicity at the level of the saga itself. We do have atomicity for each individual transaction inside the overall saga, as each one of them can relate to an ACID transactional change if needed. What a saga gives us is enough information to reason about which state it's in; it's up to us to handle the implications of this.

Trước khi chúng ta đi xa hơn, bạn cần hiểu rằng một saga không cho chúng ta tính nguyên tử theo nghĩa ACID như chúng ta đã quen với một giao dịch cơ sở dữ liệu thông thường. Khi chúng ta chia LLT thành các giao dịch riêng lẻ, chúng ta không có tính nguyên tử ở

cấp độ của bản thân saga. Chúng ta có tính nguyên tử cho mỗi giao dịch riêng lẻ bên trong saga tổng thể, vì mỗi giao dịch trong số đó có thể liên hệ với một thay đổi giao dịch ACID nếu cần. Những gì một saga cho chúng ta là đủ thông tin để suy luận về việc nó đang ở trạng thái nào; còn việc xử lý các hệ quả của điều này là tùy thuộc vào chúng ta.

Let's take a look at a simple order fulfillment flow for MusicCorp, outlined in Figure 6-5 , which we can use to further explore sagas in the context of a microservice **architecture**.

Hãy xem một luồng thực hiện đơn hàng (order fulfillment) đơn giản cho MusicCorp, được phác thảo trong Hình 6-5 , mà chúng ta có thể dùng để khám phá thêm về saga trong bối cảnh của một kiến trúc microservice.

Here, the order fulfillment process is represented as a single saga, with each step in this flow representing an operation that can be carried out by a different service. Within each service, any state change can be handled within a local ACID transaction. For example, when we check and reserve stock using the Warehouse service, internally the Warehouse service might create a row in its local Reservation table recording the reservation; this change would be handled within a normal database transaction.

Ở đây, quy trình thực hiện đơn hàng được biểu diễn dưới dạng một saga duy nhất, với mỗi bước trong luồng này đại diện cho một thao tác có thể được thực hiện bởi một dịch vụ khác nhau. Bên trong mỗi dịch vụ, bất kỳ thay đổi trạng thái nào cũng có thể được xử lý trong một giao dịch ACID cục bộ. Ví dụ, khi chúng ta kiểm tra và giữ chỗ (reserve) hàng tồn kho bằng dịch vụ Warehouse, bên trong dịch vụ Warehouse có thể tạo một dòng trong bảng Reservation cục bộ của nó để ghi lại việc giữ chỗ; thay đổi này sẽ được xử lý trong một giao dịch cơ sở dữ liệu thông thường.

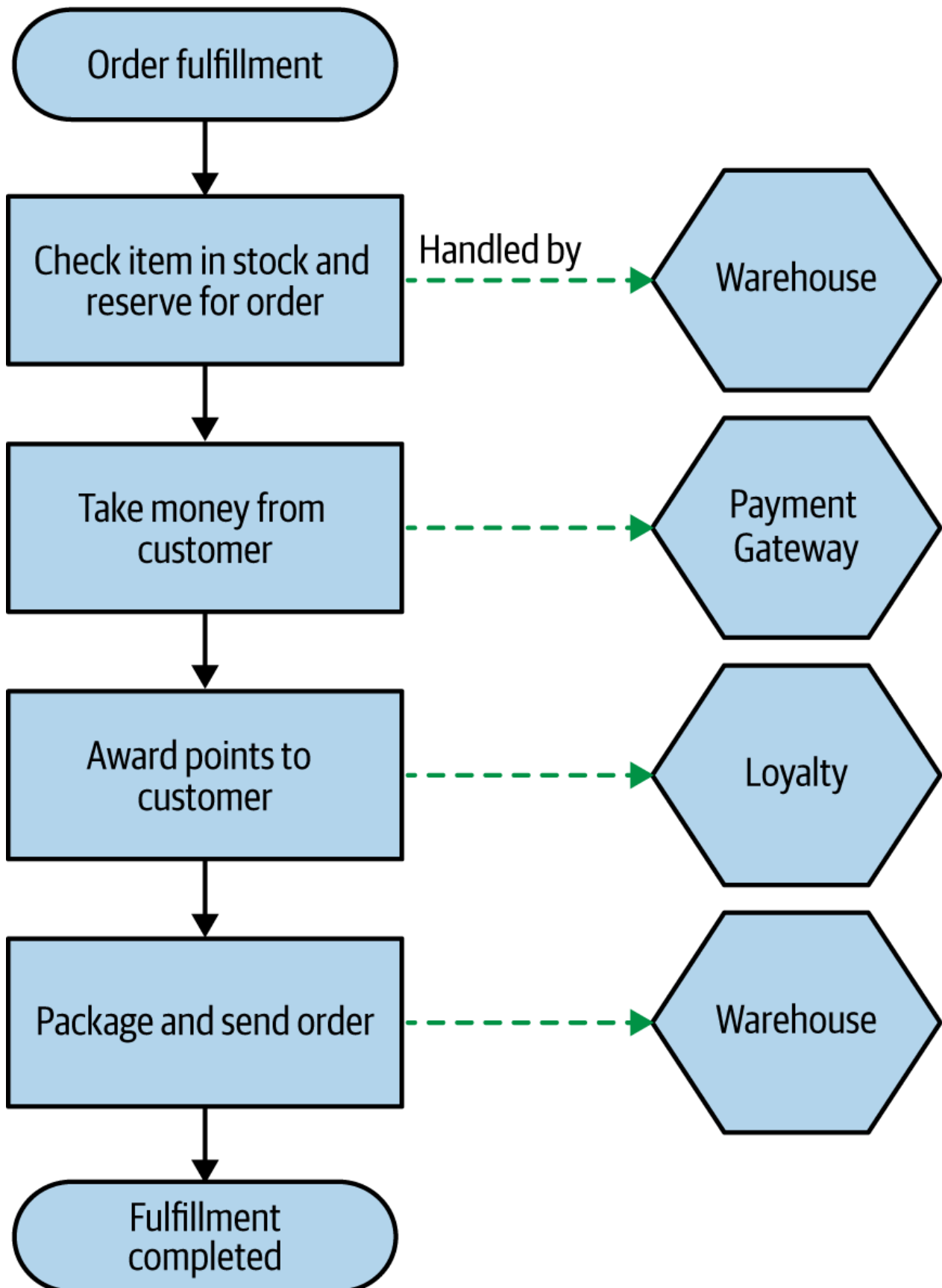


Figure 6-5. An example order fulfillment flow, along with the services responsible for carrying out the operation

Hình 6-5. Một ví dụ về luồng thực hiện đơn hàng, kèm theo các dịch vụ chịu trách nhiệm thực hiện thao tác

Saga Failure Modes — Các chế độ lỗi của Saga

With a saga being broken into individual transactions, we need to consider how to handle failure—or, more specifically, how to recover when a failure happens. The original saga paper describes two types of recovery: backward recovery and forward recovery.

Với việc một saga được chia thành các giao dịch riêng lẻ, chúng ta cần cân nhắc cách xử lý lỗi—hay, cụ thể hơn, cách phục hồi khi có lỗi xảy ra. Bài báo gốc về saga mô tả hai loại phục hồi: phục hồi lùi (backward recovery) và phục hồi tiến (forward recovery).

Backward recovery involves reverting the failure and cleaning up afterwards—a rollback. For this to work, we need to define compensating actions that allow us to undo previously committed transactions. Forward recovery allows us to pick up from the point where the failure occurred and keep processing. For that to work, we need to be able to **retry** transactions, which in turn implies that our system is persisting enough information to allow this retry to take place.

Phục hồi lùi liên quan đến việc đảo ngược lỗi và dọn dẹp sau đó—một rollback. Để điều này hoạt động, chúng ta cần định nghĩa các hành động bù trừ (compensating action) cho phép ta hoàn tác các giao dịch đã commit trước đó. Phục hồi tiến cho phép chúng ta tiếp tục từ thời điểm xảy ra lỗi và tiếp tục xử lý. Để điều đó hoạt động, chúng ta cần có khả năng thử lại các giao dịch, điều này lại ngụ ý rằng hệ thống của chúng ta đang lưu giữ đủ thông tin để cho phép việc thử lại này diễn ra.

Depending on the nature of the business process being modeled, you may expect that any failure mode triggers a backward recovery, a forward recovery, or perhaps a mix of the two.

Tùy thuộc vào bản chất của quy trình nghiệp vụ đang được mô hình hóa, bạn có thể trông đợi rằng bất kỳ chế độ lỗi nào cũng kích hoạt một phục hồi lùi, một phục hồi tiến, hoặc có lẽ là sự pha trộn của cả hai.

It's really important to note that a saga allows us to recover from business failures, not technical failures. For example, if we try and take payment from the customer but the customer has insufficient funds, then this is a business failure that the saga should be expected to handle. On the other hand, if the Payment Gateway times out or throws a 500 Internal Service Error, then this is a technical failure that we need to handle separately. The saga assumes the underlying components are working properly—that the underlying system is reliable, and that we are then coordinating the work of reliable components. We'll explore some of the ways we can make our technical components more reliable in Chapter 12, but for more on this limitation of sagas, I recommend “The Limits of the Saga Pattern” by Uwe Friedrichsen.

Điều thực sự quan trọng cần lưu ý là một saga cho phép chúng ta phục hồi từ các lỗi nghiệp vụ (business failure), chứ không phải lỗi kỹ thuật (technical failure). Ví dụ, nếu chúng ta cố thu tiền từ khách hàng nhưng khách hàng không đủ tiền, thì đây là một lỗi nghiệp vụ mà saga được trông đợi sẽ xử lý. Mặt khác, nếu Payment Gateway hết thời gian chờ (time out) hoặc ném ra một lỗi 500 Internal Service Error, thì đây là một lỗi kỹ thuật mà chúng ta cần xử lý riêng. Saga giả định rằng các thành phần nằm dưới đang hoạt động đúng đắn—rằng hệ thống nằm dưới là đáng tin cậy, và rằng khi đó chúng ta đang điều phối công việc của các thành phần đáng tin cậy. Chúng ta sẽ khám phá một số cách có thể làm cho các thành

phần kỹ thuật của mình đáng tin cậy hơn ở Chương 12 , nhưng để tìm hiểu thêm về giới hạn này của saga, tôi khuyên đọc “The Limits of the Saga Pattern” của Uwe Friedrichsen.

Saga rollbacks

Rollback của Saga

With an ACID transaction, if we hit a problem, we trigger a rollback before a commit occurs. After the rollback, it is like nothing ever happened: the change we were trying to make didn’t take place. With our saga, though, we have multiple transactions involved, and some of those may have already committed before we decide to roll back the entire operation. So how can we roll back transactions after they have already been committed?

Với một giao dịch ACID, nếu chúng ta gặp vấn đề, chúng ta kích hoạt một rollback trước khi một commit xảy ra. Sau rollback, mọi thứ như thể chưa từng xảy ra: thay đổi mà chúng ta đang cố thực hiện đã không diễn ra. Tuy nhiên với saga của chúng ta, chúng ta có nhiều giao dịch liên quan, và một số trong số đó có thể đã commit trước khi chúng ta quyết định rollback toàn bộ thao tác. Vậy làm sao chúng ta có thể rollback các giao dịch sau khi chúng đã được commit?

Let’s come back to our example of processing an order, as outlined in Figure 6-5 . Consider a potential failure mode. We’ve gotten as far as trying to package the item, only to find the item can’t be found in the warehouse, as shown in Figure 6-6 . Our system thinks the item exists, but it’s just not on the shelf!

Hãy quay lại ví dụ xử lý một đơn hàng của chúng ta, như được phác thảo trong Hình 6-5 . Hãy xem xét một chế độ lỗi tiềm tàng. Chúng ta đã đi xa đến mức cố đóng gói món hàng, để rồi phát hiện món hàng không thể tìm thấy trong kho, như thể hiện trong Hình 6-6 . Hệ thống của chúng ta nghĩ rằng món hàng tồn tại, nhưng nó chỉ là không có trên kệ!

Now, let’s assume that we decide we want to just roll back the entire order, rather than giving the customer the option for the item to be placed on back order. The problem is that we’ve already taken payment and awarded loyalty points for the order.

Bây giờ, hãy giả sử rằng chúng ta quyết định muốn chỉ rollback toàn bộ đơn hàng, thay vì cho khách hàng lựa chọn đặt món hàng vào diện chờ hàng về (back order). Vấn đề là chúng ta đã thu tiền và trao điểm thưởng (loyalty point) cho đơn hàng rồi.

If these steps had all been done in a single database transaction, a simple rollback would clean it all up. However, each step in the order fulfillment process was handled by a different service call, each of which operated in a different transactional scope. There is no simple “rollback” for the entire operation.

Nếu tất cả các bước này đã được thực hiện trong một giao dịch cơ sở dữ liệu duy nhất, thì một rollback đơn giản sẽ dọn dẹp tất cả. Tuy nhiên, mỗi bước trong quy trình thực hiện đơn hàng được xử lý bởi một lời gọi dịch vụ khác nhau, mỗi lời gọi hoạt động trong một phạm vi giao dịch khác nhau. Không có một “rollback” đơn giản nào cho toàn bộ thao tác.

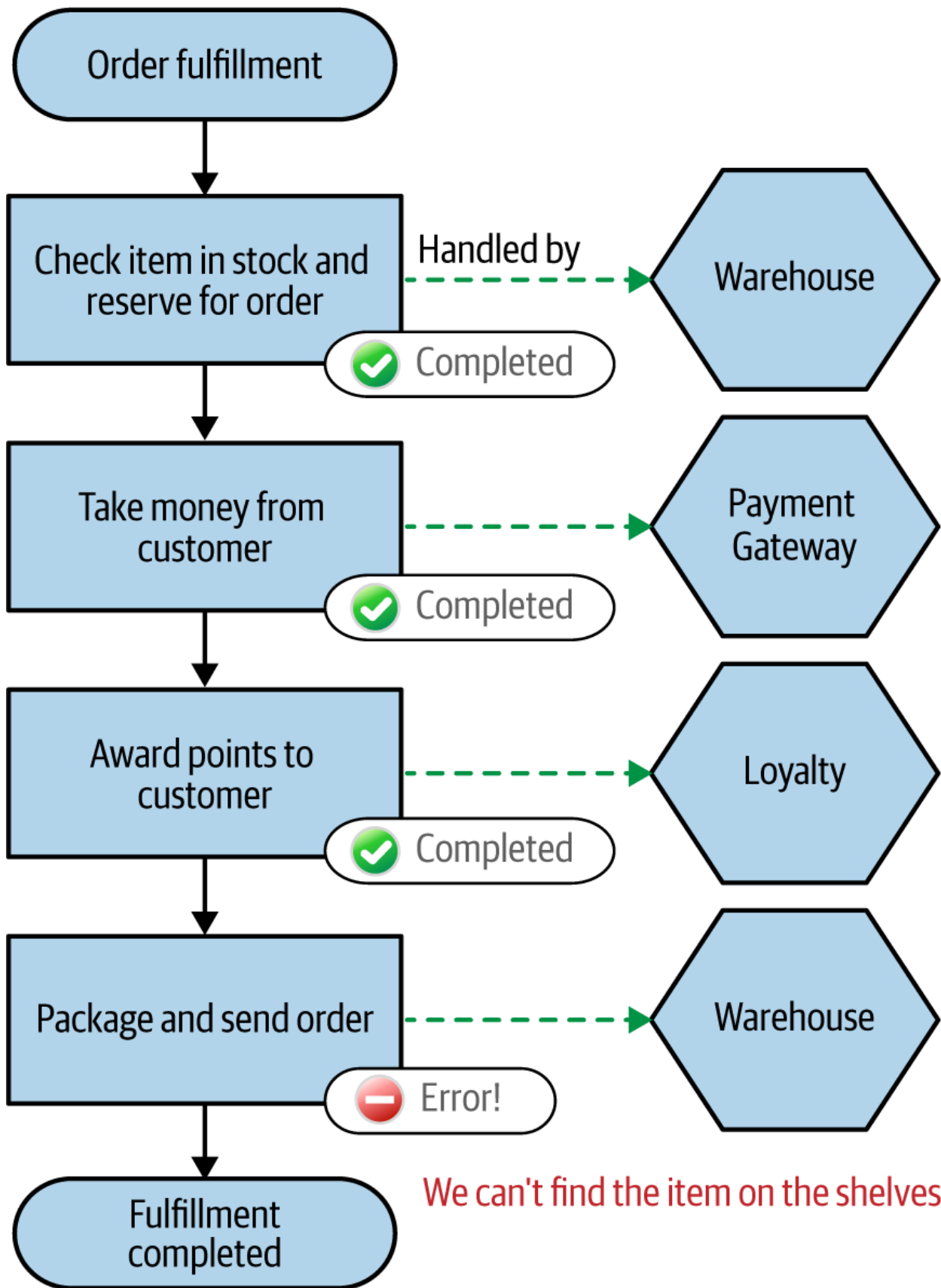


Figure 6-6. We've tried to package our item, but we can't find it in the warehouse

Hình 6-6. Chúng ta đã cố đóng gói món hàng, nhưng không thể tìm thấy nó trong kho

Instead, if you want to implement a rollback, you need to implement a compensating transaction. A compensating transaction is an operation that undoes a previously committed transaction. To roll back our order fulfillment process, we would trigger the compensating transaction for each step in our saga that has already been committed, as shown in Figure 6-7 .

Thay vào đó, nếu bạn muốn triển khai một rollback, bạn cần triển khai một giao dịch bù trừ (compensating transaction). Một giao dịch bù trừ là một thao tác hoàn tác một giao dịch đã commit trước đó. Để rollback quy trình thực hiện đơn hàng của mình, chúng ta sẽ kích hoạt giao dịch bù trừ cho mỗi bước trong saga của ta đã được commit, như thể hiện trong Hình 6-7 .

It's worth calling out the fact that these compensating transactions may not behave exactly as those of a normal database rollback. A database rollback happens before the commit, and after the rollback, it is as though the transaction never happened. In this situation, of course, these transactions did happen. We are creating a new transaction that reverts the changes made by the original transaction, but we can't roll back time and make it as though the original transaction didn't occur.

Đáng nêu ra một sự thật là những giao dịch bù trừ này có thể không hành xử chính xác như những giao dịch của một rollback cơ sở dữ liệu thông thường. Một rollback cơ sở dữ liệu xảy ra trước commit, và sau rollback, mọi thứ như thể giao dịch chưa từng xảy ra. Trong tình huống này, tất nhiên, những giao dịch này đã xảy ra. Chúng ta đang tạo ra một giao dịch mới đảo ngược các thay đổi do giao dịch gốc thực hiện, nhưng chúng ta không thể quay ngược thời gian và khiến mọi thứ như thể giao dịch gốc chưa từng xảy ra.

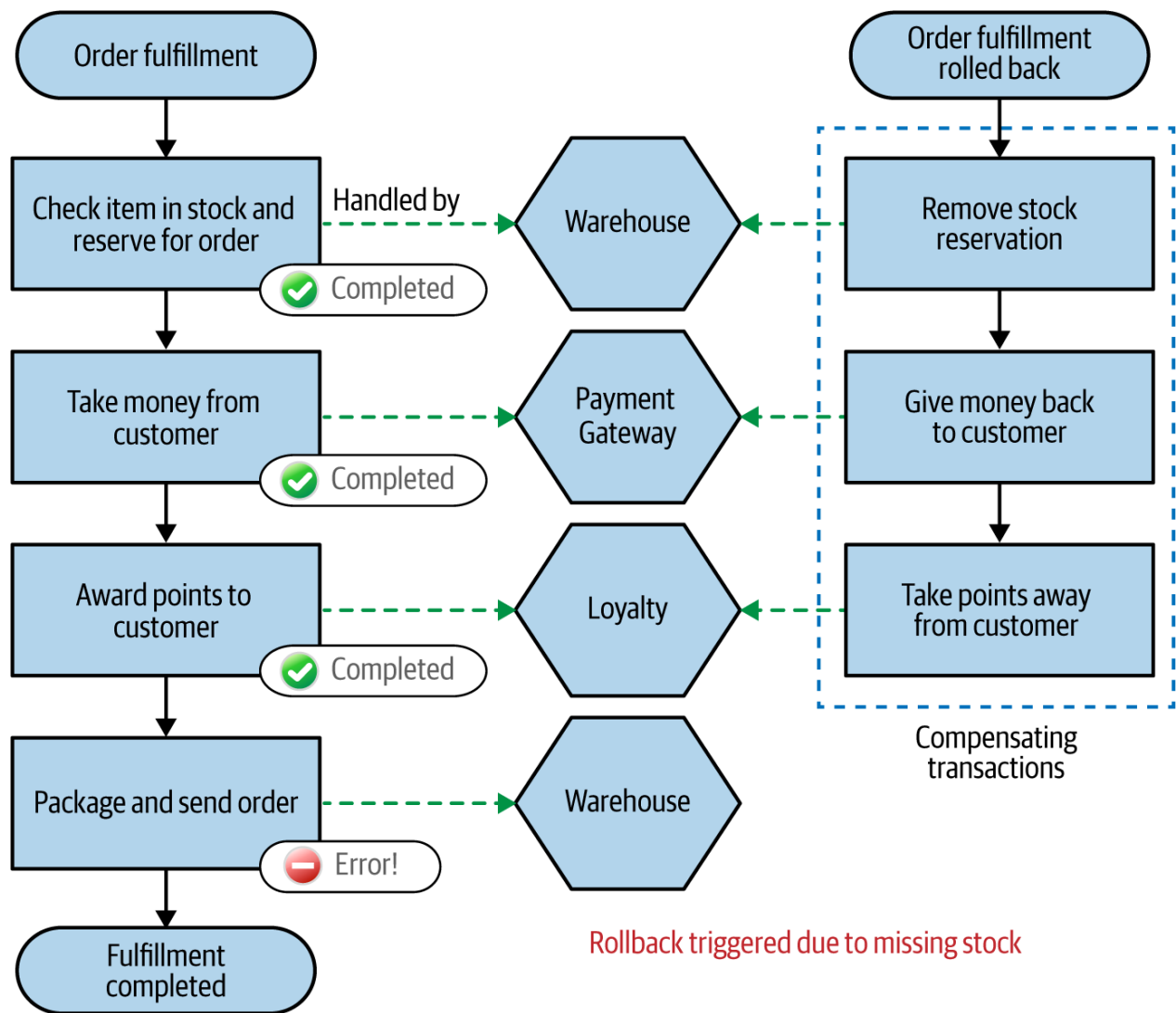


Figure 6-7. Triggering a rollback of the entire saga

Hình 6-7. Kích hoạt một rollback của toàn bộ saga

Because we cannot always cleanly revert a transaction, we say that these compensating transactions are semantic rollbacks. We cannot always clean up everything, but we do enough for the context of our saga. As an example, one of our steps may have involved sending an email to a customer to tell them their order was on the way. If we decide to roll that back, we can't unsend an email! 5 Instead, our compensating transaction could cause a second email to be sent to the customer, informing them that there was a problem with the order and it has canceled.

Vì chúng ta không phải lúc nào cũng có thể đảo ngược một giao dịch một cách sạch sẽ, nên chúng ta nói rằng những giao dịch bù trừ này là rollback ngữ nghĩa (semantic rollback). Chúng ta không phải lúc nào cũng có thể dọn dẹp mọi thứ, nhưng chúng ta làm đủ cho bối cảnh của saga của mình. Ví dụ, một trong các bước của chúng ta có thể đã liên quan đến việc gửi một email cho khách hàng để báo họ biết đơn hàng của họ đang trên đường. Nếu chúng ta quyết định rollback điều đó, chúng ta không thể thu hồi một email đã gửi! 5 Thay vào đó, giao dịch bù trừ của chúng ta có thể làm cho một email thứ hai được gửi đến khách hàng, thông báo cho họ rằng đã có một vấn đề với đơn hàng và nó đã bị hủy.

It is totally appropriate for information related to the rollback to persist in the system. In fact, this may be very important information. You may want to keep a record in the Order service for this aborted order, along with information about what happened, for a whole host of reasons.

Việc thông tin liên quan đến rollback được lưu giữ lại trong hệ thống là hoàn toàn phù hợp. Thực tế, đây có thể là thông tin rất quan trọng. Bạn có thể muốn giữ một bản ghi trong dịch vụ Order cho đơn hàng bị hủy bỏ này, cùng với thông tin về những gì đã xảy ra, vì cả một loạt lý do.

5 We really can't. I've tried!

5 Chúng ta thực sự không thể. Tôi đã thử rồi!

Reordering **workflow** steps to reduce rollbacks

Sắp xếp lại các bước workflow để giảm rollback

In Figure 6-7 , we could have made our likely rollback scenarios somewhat simpler by reordering the steps in our original workflow. A simple change would be to award points only when the order is actually dispatched, as seen in Figure 6-8 .

Trong Hình 6-7 , chúng ta có thể đã làm cho các kịch bản rollback khả dĩ của mình đơn giản hơn phần nào bằng cách sắp xếp lại các bước trong workflow ban đầu. Một thay đổi đơn giản sẽ là chỉ trao điểm khi đơn hàng thực sự được gửi đi (dispatch), như thấy trong Hình 6-8 .

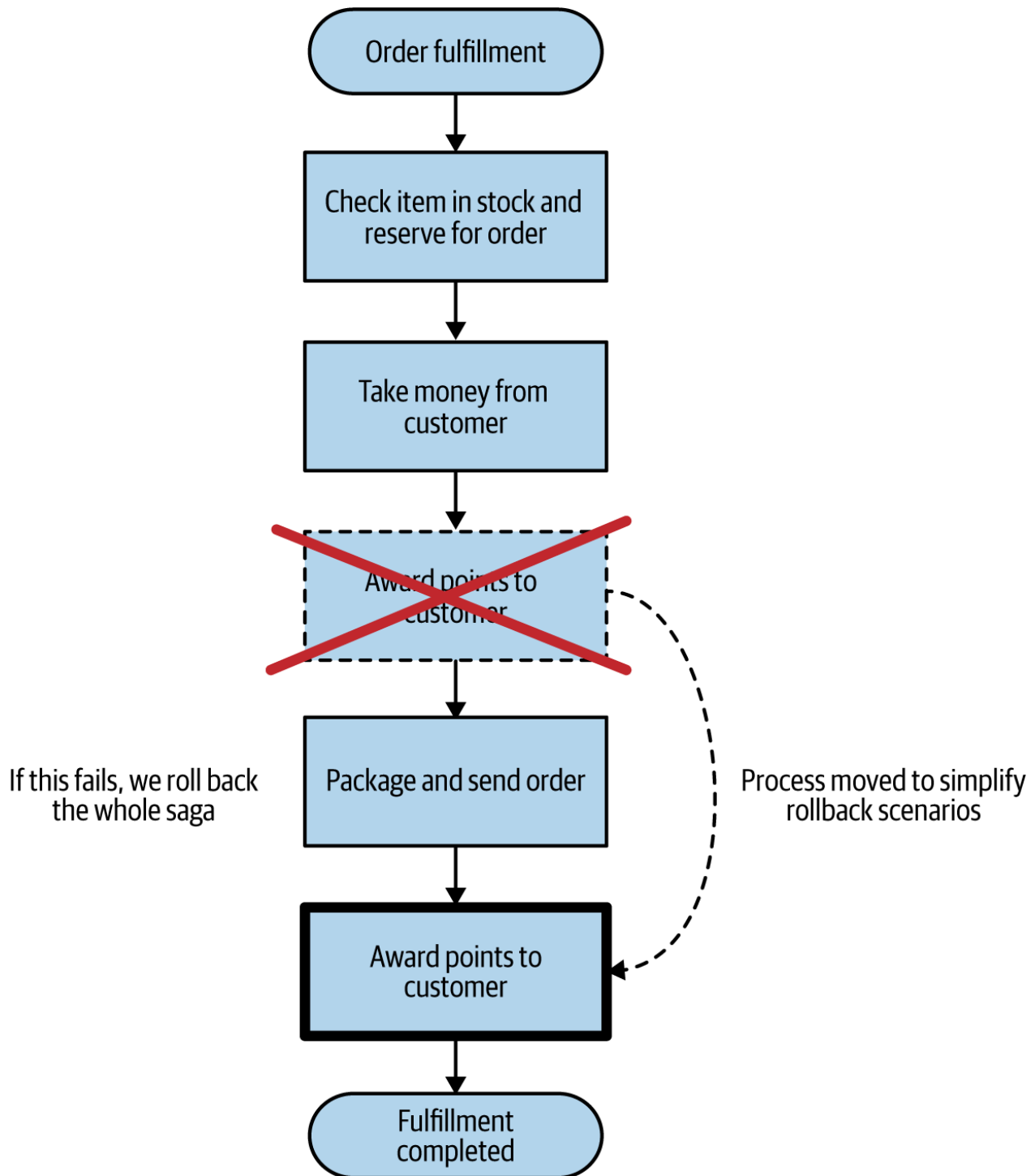


Figure 6-8. Moving steps later in the saga can reduce what has to be rolled back in case of a failure

Hình 6-8. Việc dời các bước về sau trong saga có thể giảm bớt những gì phải được rollback trong trường hợp có lỗi

In this way we would avoid having to worry about that stage being rolled back if we were to have a problem while trying to package and send the order. Sometimes you can simplify your rollback operations just by tweaking how your workflow is carried out. By pulling forward those steps that are most likely to fail and failing the process earlier, you avoid having to trigger later compensating

transactions, as those steps weren't even triggered in the first place.

Theo cách này chúng ta sẽ tránh được việc phải lo lắng về việc giai đoạn đó bị rollback nếu chúng ta gặp vấn đề trong lúc cố đóng gói và gửi đơn hàng. Đôi khi bạn có thể đơn giản hóa các thao tác rollback của mình chỉ bằng cách điều chỉnh cách thức thực hiện workflow. Bằng cách kéo các bước có nhiều khả năng thất bại nhất lên trước và làm cho quy trình thất bại sớm hơn, bạn tránh được việc phải kích hoạt các giao dịch bù trừ sau đó, vì các bước đó thậm chí còn chưa được kích hoạt ngay từ đầu.

These changes, if they can be accommodated, can make your life much easier, avoiding the need to even create compensating transactions for some steps. This can be especially important if implementing a compensating transaction is difficult. You may be able to move a step later in the process to a stage at which it never needs to be rolled back.

Những thay đổi này, nếu chúng có thể được áp dụng, có thể khiến cuộc sống của bạn dễ dàng hơn nhiều, tránh được nhu cầu thậm chí phải tạo các giao dịch bù trừ cho một số bước. Điều này có thể đặc biệt quan trọng nếu việc triển khai một giao dịch bù trừ là khó khăn. Bạn có thể dời một bước về sau trong quy trình đến một giai đoạn mà nó không bao giờ cần phải được rollback.

Mixing fail-backward and fail-forward situations

Pha trộn các tình huống fail-backward và fail-forward

It is totally appropriate to have a mix of failure recovery modes. Some failures may require a rollback (fail backward); others may be fail forward. For the order processing, for example, once we've taken money from the customer, and the item has been packaged, the only step left is to dispatch the package. If for whatever reason we can't dispatch the package (perhaps the delivery firm we use doesn't have space in its vans to take an order today), it seems very odd to roll the whole order back. Instead, we'd probably just retry the dispatch (perhaps queuing it for the following day), and if that fails, we'd require human intervention to resolve the situation.

Việc có một sự pha trộn các chế độ phục hồi lỗi là hoàn toàn phù hợp. Một số lỗi có thể đòi hỏi một rollback (fail backward); những lỗi khác có thể là fail forward. Đối với việc xử lý đơn hàng, chẳng hạn, một khi chúng ta đã thu tiền từ khách hàng, và món hàng đã được đóng gói, thì bước duy nhất còn lại là gửi đi gói hàng. Nếu vì lý do gì đó chúng ta không thể gửi đi gói hàng (có lẽ hãng giao hàng chúng ta dùng không còn chỗ trong xe tải để nhận thêm một đơn hôm nay), thì việc rollback toàn bộ đơn hàng có vẻ rất kỳ cục. Thay vào đó, chúng ta có lẽ sẽ chỉ thử lại việc gửi hàng (có thể xếp hàng đợi cho ngày hôm sau), và nếu việc đó thất bại, chúng ta sẽ cần đến sự can thiệp của con người để giải quyết tình huống.

Implementing Sagas — Triển khai Saga

So far we've looked at the logical model for how sagas work, but we need to go a bit deeper to examine ways of implementing the saga itself. We can look at two styles of saga implementation. Orchestrated sagas more closely follow the original solution space and rely primarily on centralized coordination and tracking. These can be compared to choreographed sagas, which avoid the need for centralized coordination in favor of a more loosely coupled model but can make tracking the progress of a saga more complicated.

Cho đến giờ chúng ta đã xem mô hình logic về cách saga hoạt động, nhưng chúng ta cần đi sâu hơn một chút để xem xét các cách triển khai bản thân saga. Chúng ta có thể xem hai phong cách triển khai saga. Saga điều phối (orchestrated saga) theo sát hơn không gian giải pháp ban đầu và chủ yếu dựa vào sự điều phối và theo dõi tập trung. Chúng có

thể được so sánh với saga vũ đạo (choreographed saga) , vốn tránh nhu cầu điều phối tập trung để chuộg một mô hình ít phụ thuộc (loosely coupled) hơn nhưng có thể khiến việc theo dõi tiến độ của một saga trở nên phức tạp hơn.

Orchestrated sagas

Saga điều phối

Orchestrated sagas use a central coordinator (what we'll call an orchestrator from now on) to define the order of execution and to trigger any required compensating action. You can think of orchestrated sagas as a command-and-control approach: the orchestrator controls what happens and when, and with that comes a good degree of visibility into what is happening with any given saga.

Saga điều phối sử dụng một bộ điều phối trung tâm (từ giờ chúng ta sẽ gọi là orchestrator) để định nghĩa thứ tự thực thi và để kích hoạt bất kỳ hành động bù trừ nào cần thiết. Bạn có thể nghĩ về saga điều phối như một cách tiếp cận chỉ-huy-và-kiểm-soát (command-and-control): orchestrator kiểm soát điều gì xảy ra và khi nào, và đi kèm với điều đó là một mức độ tốt về khả năng thấy được những gì đang xảy ra với một saga bất kỳ.

Taking the order fulfillment process in Figure 6-5 , let's see how this central coordination process would work as a set of collaborating services, as shown in Figure 6-9 .

Lấy quy trình thực hiện đơn hàng trong Hình 6-5 , hãy xem quy trình điều phối tập trung này sẽ hoạt động như thế nào dưới dạng một tập hợp các dịch vụ cộng tác, như thể hiện trong Hình 6-9 .

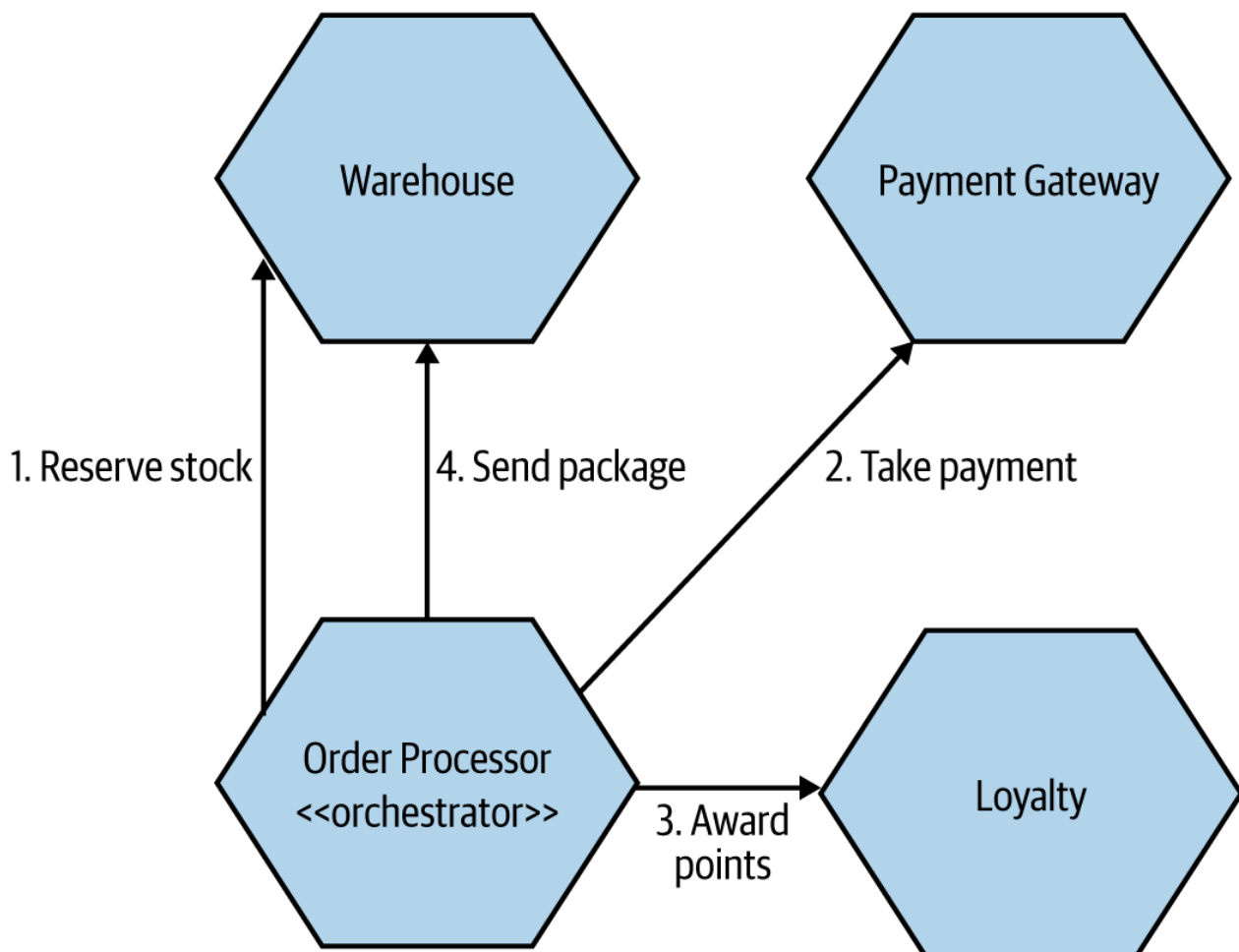


Figure 6-9. An example of how an orchestrated saga may be used to implement our order fulfillment process

Hình 6-9. Một ví dụ về cách một saga điều phối có thể được dùng để triển khai quy trình thực hiện đơn hàng của chúng ta

Here, our central Order Processor , playing the role of the orchestrator, coordinates our fulfillment process. It knows what services are needed to carry out the operation, and it decides when to make calls to those services. If the calls fail, it can decide what to do as a result. In general, orchestrated sagas tend to make heavy use of **request-response** interactions between services: the Order Processor sends a request to services (such as a Payment Gateway) and expects a response to let it know if the request was successful and provide the results of the request.

Ở đây, Order Processor trung tâm của chúng ta, đóng vai trò orchestrator, điều phối quy trình thực hiện đơn hàng của ta. Nó biết những dịch vụ nào cần thiết để thực hiện thao tác, và nó quyết định khi nào thực hiện các lời gọi đến các dịch vụ đó. Nếu các lời gọi thất bại, nó có thể quyết định phải làm gì như là kết quả. Nói chung, các saga điều phối có xu hướng sử dụng nhiều các tương tác request-response giữa các dịch vụ: Order Processor gửi một yêu cầu đến các dịch vụ (chẳng hạn như một Payment Gateway) và trông đợi một phản hồi để cho nó biết liệu yêu cầu có thành công hay không và cung cấp kết quả của yêu cầu.

Having our business process explicitly modeled inside the Order Processor is extremely beneficial. It allows us to look at one place in our system and understand how this process is supposed to work. That can make the onboarding of new people easier and help impart a better understanding of the core parts of the system.

Việc có quy trình nghiệp vụ của chúng ta được mô hình hóa một cách tường minh bên trong Order Processor là cực kỳ có lợi. Nó cho phép chúng ta nhìn vào một nơi trong hệ thống của mình và hiểu được quy trình này được cho là hoạt động ra sao. Điều đó có thể làm cho việc đưa người mới vào (onboarding) dễ dàng hơn và giúp truyền đạt một sự hiểu biết tốt hơn về các phần cốt lõi của hệ thống.

There are a few downsides to consider, though. First, by its nature, this is a somewhat coupled approach. Our Order Processor needs to know about all the associated services, resulting in a higher degree of domain **coupling**. While domain coupling is not inherently bad, we'd still like to keep it to a minimum, if possible. Here, our Order Processor needs to know about and control so many things that this form of coupling is hard to avoid.

Tuy nhiên, có một vài nhược điểm cần cân nhắc. Thứ nhất, về bản chất của nó, đây là một cách tiếp cận có phần phụ thuộc (coupled). Order Processor của chúng ta cần biết về tất cả các dịch vụ liên quan, dẫn đến một mức độ phụ thuộc theo miền (domain coupling) cao hơn. Mặc dù domain coupling vốn không phải là điều xấu, chúng ta vẫn muốn giữ nó ở mức tối thiểu, nếu có thể. Ở đây, Order Processor của chúng ta cần biết về và kiểm soát quá nhiều thứ đến mức hình thức phụ thuộc này khó tránh được.

The other issue, which is more subtle, is that logic that should otherwise be pushed into the services can start to become absorbed in the orchestrator instead. If this begins to happen, you may find that your services become anemic, with little behavior of their own, just taking orders from orchestrators like the Order Processor . It's important you still consider the services that make up these orchestrated flows as entities that have their own local state and behavior. They are in charge of their own local state machines.

Vấn đề còn lại, vốn tinh tế hơn, là logic mà lẽ ra nên được đẩy vào trong các dịch vụ có thể bắt đầu trở nên bị thu hút vào trong orchestrator thay vì vậy. Nếu điều này bắt đầu xảy ra,

bạn có thể thấy rằng các dịch vụ của mình trở nên thiếu máu (anemic), với ít hành vi của riêng chúng, chỉ nhận lệnh từ các orchestrator như Order Processor . Điều quan trọng là bạn vẫn cần xem các dịch vụ cấu thành nên những luồng điều phối này như những thực thể có trạng thái và hành vi cục bộ của riêng chúng. Chúng phụ trách các máy trạng thái (state machine) cục bộ của chính mình.



If logic has a place where it can be centralized, it will become centralized!

Nếu logic có một nơi mà nó có thể được tập trung hóa, nó sẽ bị tập trung hóa!

One way to avoid too much centralization with orchestrated flows is to ensure you have different services playing the role of the orchestrator for different flows. You might have an Order Processor microservice that handles placing an order, a Returns microservice to handle the return and refund process, a Goods Receiving microservice that handles new stock arriving and being put on the shelves, and so on. Something like our Warehouse microservice may be used by all those orchestrators; such a model makes it easier for you to keep functionality in the Warehouse

microservice itself, allowing you to reuse functionality across all those flows.

Một cách để tránh việc tập trung hóa quá mức với các luồng điều phối là đảm bảo bạn có những dịch vụ khác nhau đóng vai trò orchestrator cho những luồng khác nhau. Bạn có thể có một microservice Order Processor xử lý việc đặt một đơn hàng, một microservice Returns để xử lý quy trình trả hàng và hoàn tiền, một microservice Goods Receiving xử lý hàng tồn kho mới đến và được đưa lên kệ, v.v. Một thứ gì đó như microservice Warehouse của chúng ta có thể được sử dụng bởi tất cả các orchestrator đó; một mô hình như vậy giúp bạn dễ dàng hơn trong việc giữ chức năng ở bên trong chính microservice Warehouse, cho phép bạn tái sử dụng chức năng trên tất cả các luồng đó.

BPM Tools — Công cụ BPM

Business process modeling (BPM) tools have been available for many years. By and large, they are designed to allow nondevelopers to define business process flows, often using visual drag-and-drop tools. The idea is that developers would create the building blocks of these processes, and then nondevelopers would wire these building blocks together into the larger process flows. The use of such tools seems to line up really nicely as a way of implementing orchestrated sagas, and indeed, process orchestration is pretty much the main use case for BPM tools (or, in reverse, the use of BPM tools results in you having to adopt orchestration).

Các công cụ mô hình hóa quy trình nghiệp vụ (business process modeling, BPM) đã có sẵn trong nhiều năm. Nhìn chung, chúng được thiết kế để cho phép những người không phải lập trình viên định nghĩa các luồng quy trình nghiệp vụ, thường sử dụng các công cụ kéo-thả trực quan. Ý tưởng là các lập trình viên sẽ tạo ra các khối xây dựng (building block) của những quy trình này, và rồi những người không phải lập trình viên sẽ nối các khối xây dựng này lại với nhau thành các luồng quy trình lớn hơn. Việc sử dụng những công cụ như vậy có vẻ rất phù hợp như một cách triển khai saga điều phối, và quả thực, điều phối quy trình (process orchestration) gần như là ca sử dụng chính cho các công cụ BPM (hoặc, ngược lại, việc sử dụng các công cụ BPM dẫn đến việc bạn phải áp dụng điều phối).

In my experience, I've come to greatly dislike BPM tools. The main reason is that the central conceit—that nondevelopers will define the business process—has in my experience almost never been true. The tooling aimed at nondevelopers ends up getting used by developers, and unfortunately these tools often work in ways that are alien to how developers like to work. They often require the use of GUIs to change the flows, the flows they create may be difficult (or impossible) to version control, the flows themselves may not be designed with **testing** in mind, and more.

Theo kinh nghiệm của tôi, tôi đã đi đến chỗ rất không ưa các công cụ BPM. Lý do chính là cái niềm tin cốt lõi—rằng những người không phải lập trình viên sẽ định nghĩa quy trình nghiệp vụ—theo kinh nghiệm của tôi gần như chưa bao giờ là đúng. Công cụ nhắm đến những người không phải lập trình viên rốt cuộc lại bị các lập trình viên sử dụng, và tiếc thay những công cụ này thường hoạt động theo những cách xa lạ với cách mà lập trình viên thích làm việc. Chúng thường đòi hỏi việc sử dụng GUI để thay đổi các luồng, các luồng mà chúng tạo ra có thể khó (hoặc không thể) đưa vào quản lý phiên bản (version control), bản thân các luồng có thể không được thiết kế với việc kiểm thử trong đầu, và còn nhiều nữa.

If your developers are going to be implementing your business processes, let them use tooling that they know and understand and that is fit for their workflows. In general, this means just letting them use code to implement these things! If you need visibility into how a business process has

been implemented or how it is operating, then it is far easier to project a visual representation of a workflow from code than it is to use a visual representation of your workflow to describe how your code should work.

Nếu các lập trình viên của bạn sẽ triển khai các quy trình nghiệp vụ của bạn, hãy để họ sử dụng công cụ mà họ biết và hiểu và phù hợp với workflow của họ. Nói chung, điều này có nghĩa là chỉ cần để họ dùng mã để triển khai những thứ này! Nếu bạn cần khả năng thấy được một quy trình nghiệp vụ đã được triển khai như thế nào hoặc nó đang vận hành ra sao, thì việc phóng chiếu (project) một biểu diễn trực quan của một workflow từ mã dễ dàng hơn nhiều so với việc sử dụng một biểu diễn trực quan của workflow của bạn để mô tả cách mã của bạn nên hoạt động.

There are efforts to create more developer-friendly BPM tools. Feedback on these tools from developers seems to be mixed, but the tools have worked well for some, and it's good to see people trying to improve on these frameworks. If you feel the need to explore these tools further, do take a look at Camunda and Zeebe , both of which are open source orchestration frameworks targeting microservice developers, and which would be at the top of my list if I decided that a BPM tool was for me.

Có những nỗ lực để tạo ra các công cụ BPM thân thiện với lập trình viên hơn. Phản hồi về những công cụ này từ các lập trình viên có vẻ pha tạp, nhưng các công cụ đã hoạt động tốt với một số người, và thật tốt khi thấy mọi người cố gắng cải thiện những framework này. Nếu bạn cảm thấy cần khám phá những công cụ này sâu hơn, hãy thử xem Camunda và Zeebe , cả hai đều là các framework điều phối mã nguồn mở nhắm đến các lập trình viên microservice, và sẽ đứng đầu danh sách của tôi nếu tôi quyết định rằng một công cụ BPM là dành cho mình.

Choreographed sagas

Saga vũ đạo

A choreographed saga aims to distribute responsibility for the operation of the saga among multiple collaborating services. If orchestration is a command-and-control approach, choreographed sagas represent a trust-but-verify architecture. As we'll see in our example in Figure 6-10 , choreographed sagas will often make heavy use of events for collaboration between services.

Một saga vũ đạo nhắm đến việc phân tán trách nhiệm vận hành saga giữa nhiều dịch vụ cộng tác. Nếu điều phối là một cách tiếp cận chỉ-huy-và-kiểm-soát, thì saga vũ đạo đại diện cho một kiến trúc tin-nhưng-có-kiểm-chứng (trust-but-verify). Như chúng ta sẽ thấy trong ví dụ của mình ở Hình 6-10 , các saga vũ đạo thường sẽ sử dụng nhiều các sự kiện (event) cho việc cộng tác giữa các dịch vụ.

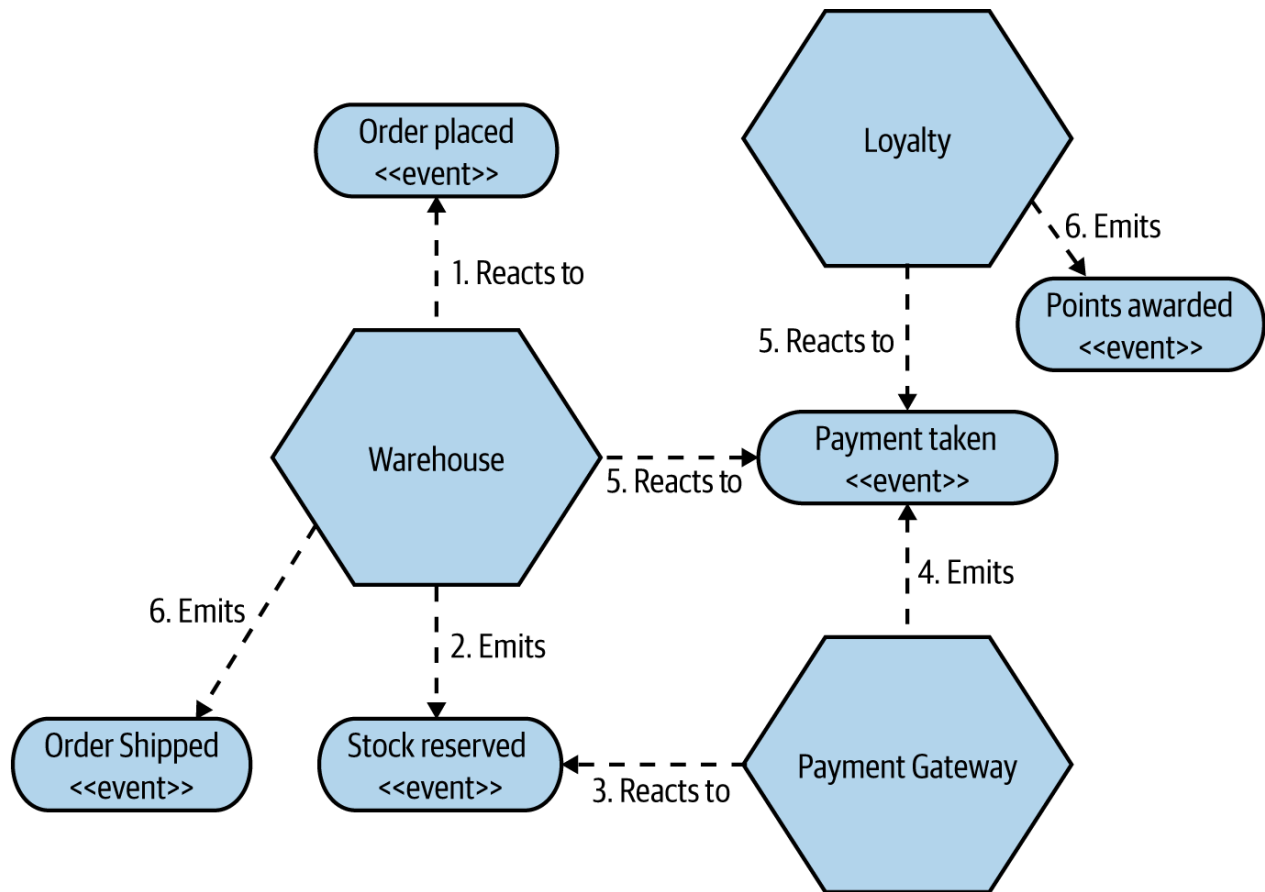


Figure 6-10. An example of a choreographed saga for implementing order fulfillment

Hình 6-10. Một ví dụ về một saga vũ đạo để triển khai việc thực hiện đơn hàng

There's quite a bit going on here, so it's worth exploring in more detail. First, these microservices are reacting to events being received. Conceptually, events are broadcast in the system, and interested parties are able to receive them. Remember, as we discussed in Chapter 4, you don't send events to a microservice; you just fire them out, and the microservices that are interested in these events are able to receive them and act accordingly. In our example, when the Warehouse service receives that first Order Placed event, it knows its job is to reserve the appropriate stock and fire an event once that is done. If the stock couldn't be received, the Warehouse would need to raise an appropriate event (an Insufficient Stock event, perhaps), which might lead to the order being aborted.

Có khá nhiều thứ đang diễn ra ở đây, nên đáng để khám phá chi tiết hơn. Thứ nhất, các microservice này đang phản ứng với các sự kiện đang được nhận. Về mặt khái niệm, các sự kiện được phát quảng bá (broadcast) trong hệ thống, và các bên quan tâm có thể nhận chúng. Hãy nhớ, như chúng ta đã thảo luận ở Chương 4, bạn không gửi các sự kiện đến một microservice; bạn chỉ phát chúng ra, và những microservice nào quan tâm đến những sự kiện này có thể nhận chúng và hành động tương ứng. Trong ví dụ của chúng ta, khi dịch vụ Warehouse nhận được sự kiện Order Placed đầu tiên đó, nó biết công việc của nó là giữ chỗ hàng tồn kho phù hợp và phát ra một sự kiện một khi việc đó hoàn tất. Nếu hàng không thể được nhận, Warehouse sẽ cần phát ra một sự kiện phù hợp (một sự kiện Insufficient Stock, chẳng hạn), điều này có thể dẫn đến việc đơn hàng bị hủy bỏ.

We also see in this example how events can facilitate parallel processing. When the Payment Taken event is fired by the Payment Gateway, it causes reactions in both the Loyalty and Warehouse

microservices. The Warehouse reacts by dispatching the package, while the Loyalty microservice reacts by awarding points.

Chúng ta cũng thấy trong ví dụ này cách các sự kiện có thể tạo điều kiện cho việc xử lý song song. Khi sự kiện Payment Taken được phát ra bởi Payment Gateway, nó gây ra phản ứng ở cả microservice Loyalty và Warehouse. Warehouse phản ứng bằng cách gửi đi gói hàng, trong khi microservice Loyalty phản ứng bằng cách trao điểm.

Typically, you'd use some sort of **message broker** to manage the reliable broadcast and delivery of events. It's possible that multiple microservices may react to the same event, and that is where you would use a topic. Parties interested in a certain type of event would subscribe to a specific topic without having to worry about where these events came from, and the broker ensures the durability of the topic and that the events on it are successfully delivered to subscribers. As an example, we might have a Recommendation service that also listens to Order Placed events and uses that to construct a database of music choices you might like.

Thông thường, bạn sẽ sử dụng một loại message broker nào đó để quản lý việc phát quảng bá và phân phối các sự kiện một cách đáng tin cậy. Có khả năng nhiều microservice có thể phản ứng với cùng một sự kiện, và đó là nơi bạn sẽ sử dụng một topic. Các bên quan tâm đến một loại sự kiện nhất định sẽ đăng ký (subscribe) vào một topic cụ thể mà không phải lo lắng về việc những sự kiện này đến từ đâu, và broker đảm bảo tính bền vững của topic và đảm bảo rằng các sự kiện trên đó được phân phối thành công đến những bên đăng ký. Ví dụ, chúng ta có thể có một dịch vụ Recommendation cũng lắng nghe các sự kiện Order Placed và dùng nó để xây dựng một cơ sở dữ liệu về các lựa chọn âm nhạc mà bạn có thể thích.

In the preceding architecture, no one service knows about any other microservice. They need to know only what to do when a certain event is received—we've drastically reduced the amount of domain coupling. Inherently, this makes for a much less coupled architecture. As the implementation of the process is decomposed and distributed among the three microservices here, we also avoid the concerns about centralization of logic (if you don't have a place where logic can be centralized, then it won't be centralized!).

Trong kiến trúc nói trên, không dịch vụ nào biết về bất kỳ microservice nào khác. Chúng chỉ cần biết phải làm gì khi một sự kiện nhất định được nhận—chúng ta đã giảm đáng kể lượng domain coupling. Về bản chất, điều này tạo nên một kiến trúc ít phụ thuộc hơn nhiều. Khi việc triển khai quy trình được phân rã và phân tán giữa ba microservice ở đây, chúng ta cũng tránh được những lo ngại về việc tập trung hóa logic (nếu bạn không có một nơi mà logic có thể được tập trung hóa, thì nó sẽ không bị tập trung hóa!).

The flip side of this is that it can be harder to work out what is going on. With orchestration, our process was explicitly modeled in our orchestrator. Now, with this architecture as it is presented, how would you build up a mental model of what the process is supposed to be? You'd have to look at the behavior of each service in isolation and reconstitute this picture in your own head—far from straightforward, even with a simple business process like this one.

Mặt trái của điều này là có thể khó hơn để tìm ra điều gì đang diễn ra. Với điều phối, quy trình của chúng ta được mô hình hóa một cách tường minh trong orchestrator của ta. Bây giờ, với kiến trúc như nó được trình bày, bạn sẽ xây dựng một mô hình tư duy về việc quy trình được cho là như thế nào ra sao? Bạn sẽ phải nhìn vào hành vi của mỗi dịch vụ một cách riêng lẻ và tái cấu trúc bức tranh này trong đầu mình—chẳng hề đơn giản, ngay cả với một quy trình nghiệp vụ đơn giản như cái này.

The lack of an explicit representation of our business process is bad enough, but we also lack a way of

knowing what state a saga is in, which can deny us the chance to attach compensating actions when required. We can push some responsibility to the individual services for carrying out compensating actions, but fundamentally we need a way of knowing what state a saga is in for some kinds of recovery. The lack of a central place to interrogate around the status of a saga is a big problem. We get that with orchestration, so how do we solve that here?

Việc thiếu một biểu diễn tường minh của quy trình nghiệp vụ của chúng ta đã đủ tệ rồi, nhưng chúng ta còn thiếu một cách để biết một saga đang ở trạng thái nào, điều này có thể tước đi cơ hội đính kèm các hành động bù trừ khi cần. Chúng ta có thể đẩy một phần trách nhiệm cho các dịch vụ riêng lẻ trong việc thực hiện các hành động bù trừ, nhưng về cơ bản chúng ta cần một cách để biết một saga đang ở trạng thái nào cho một số kiểu phục hồi. Việc thiếu một nơi trung tâm để truy vấn về trạng thái của một saga là một vấn đề lớn. Chúng ta có được điều đó với điều phối, vậy làm sao chúng ta giải quyết điều đó ở đây?

One of the easiest ways of doing this is to project a view regarding the state of a saga by consuming the events being emitted. If we generate a unique ID for the saga, what is known as a correlation ID , we can put it into all of the events that are emitted as part of this saga. When one of our services reacts to an event, the correlation ID is extracted and used for any local logging processes, and it's also passed downstream with any further calls or events that are fired. We could then have a service whose job it is to just vacuum up all these events and present a view of what state each order is in, and perhaps programmatically carry out actions to resolve issues as part of the fulfillment process if the other services couldn't do it themselves. I consider some form of correlation ID essential for choreographed sagas like this, but correlation IDs also have a lot of value more generally, something we explore in more depth in Chapter 10 .

Một trong những cách dễ nhất để làm điều này là phóng chiếu một góc nhìn về trạng thái của một saga bằng cách tiêu thụ (consume) các sự kiện đang được phát ra. Nếu chúng ta sinh ra một ID duy nhất cho saga, cái được gọi là correlation ID , chúng ta có thể đặt nó vào tất cả các sự kiện được phát ra như một phần của saga này. Khi một trong các dịch vụ của chúng ta phản ứng với một sự kiện, correlation ID được trích xuất và dùng cho bất kỳ quy trình ghi log cục bộ nào, và nó cũng được truyền xuống phía dưới (downstream) với bất kỳ lời gọi hoặc sự kiện nào tiếp theo được phát ra. Khi đó chúng ta có thể có một dịch vụ mà công việc của nó chỉ là hút lấy tất cả các sự kiện này và trình bày một góc nhìn về việc mỗi đơn hàng đang ở trạng thái nào, và có lẽ thực hiện bằng chương trình các hành động để giải quyết các vấn đề như một phần của quy trình thực hiện đơn hàng nếu các dịch vụ khác không thể tự làm điều đó. Tôi xem một dạng correlation ID nào đó là thiết yếu cho các saga vũ đạo như thế này, nhưng correlation ID cũng có rất nhiều giá trị một cách tổng quát hơn, một điều mà chúng ta khám phá sâu hơn ở Chương 10 .

Mixing styles

Pha trộn các phong cách

While it may seem that orchestrated and choreographed sagas are diametrically opposing views of how sagas could be implemented, you could easily consider mixing and matching models. You may have some business processes in your system that more naturally fit one model or another. You may also have a single saga that has a mix of styles. In the order fulfillment use case, for example, inside the boundary of the Warehouse service, when managing the packaging and dispatch of an order, we may use an orchestrated flow even if the original request was made as part of a larger choreographed saga. 6

Mặc dù có vẻ như saga điều phối và saga vũ đạo là những góc nhìn đối lập hoàn toàn về cách saga có thể được triển khai, bạn hoàn toàn có thể cân nhắc việc trộn và phối hợp các

mô hình. Bạn có thể có một số quy trình nghiệp vụ trong hệ thống của mình phù hợp một cách tự nhiên hơn với mô hình này hay mô hình khác. Bạn cũng có thể có một saga duy nhất mà có sự pha trộn các phong cách. Trong ca sử dụng thực hiện đơn hàng, chẳng hạn, bên trong ranh giới của dịch vụ Warehouse, khi quản lý việc đóng gói và gửi đi một đơn hàng, chúng ta có thể sử dụng một luồng điều phối ngay cả khi yêu cầu ban đầu được thực hiện như một phần của một saga vũ đạo lớn hơn. 6

If you do decide to mix styles, it's important that you still have a clear way to understand what state a saga is in, and what activities have already happened as part of a saga. Without this, understanding failure modes becomes complex, and recovery from failure is difficult.

Nếu bạn quyết định trộn các phong cách, điều quan trọng là bạn vẫn cần có một cách rõ ràng để hiểu một saga đang ở trạng thái nào, và những hoạt động nào đã xảy ra như một phần của một saga. Không có điều này, việc hiểu các chế độ lỗi trở nên phức tạp, và việc phục hồi từ lỗi trở nên khó khăn.

Tracing Calls — Truy vết các lời gọi

Whether you chose choreography or orchestration, when implementing a business process using multiple microservices, it's common to want to be able to trace all the calls related to the process. This can sometimes be just to help you understand if the business process is working correctly, or it could be to help you diagnose a problem. In Chapter 10 we'll look at concepts such as correlation IDs and **log aggregation** and how they can help in this regard.

Dù bạn chọn vũ đạo hay điều phối, khi triển khai một quy trình nghiệp vụ sử dụng nhiều microservice, người ta thường muốn có khả năng truy vết tất cả các lời gọi liên quan đến quy trình. Điều này đôi khi chỉ để giúp bạn hiểu liệu quy trình nghiệp vụ có đang hoạt động đúng không, hoặc nó có thể là để giúp bạn chẩn đoán một vấn đề. Ở Chương 10 chúng ta sẽ xem xét các khái niệm như correlation ID và gom log (log aggregation) và cách chúng có thể giúp ích về mặt này.

6 It's outside the scope of this book, but Hector Garcia-Molina and Kenneth Salem went on to explore how multiple sagas could be “nested” to implement more complex processes. To read more on this topic, see Hector Garcia-Molina et al., “Modeling Long-Running Activities as Nested Sagas,” *Data Engineering* 14, no. 1 (March 1991: 14–18).

6 Điều này nằm ngoài phạm vi của cuốn sách này, nhưng Hector Garcia-Molina và Kenneth Salem đã tiếp tục khám phá cách nhiều saga có thể được “lồng nhau” (nested) để triển khai các quy trình phức tạp hơn. Để đọc thêm về chủ đề này, xem Hector Garcia-Molina et al., “Modeling Long-Running Activities as Nested Sagas,” *Data Engineering* 14, no. 1 (March 1991: 14–18).

Should I use choreography or orchestration (or a mix)?

Tôi nên dùng vũ đạo hay điều phối (hay pha trộn)?

The implementation of choreographed sagas can bring with it ideas that may be unfamiliar to you and your team. They typically assume heavy use of **event-driven** collaboration, which isn't widely understood. However, in my experience, the extra complexity associated with tracking the progress of a saga is almost always outweighed by the benefits associated with having a more loosely coupled architecture.

Việc triển khai các saga vũ đạo có thể mang theo những ý tưởng có thể xa lạ với bạn và nhóm của bạn. Chúng thường giả định việc sử dụng nhiều sự cộng tác hướng sự kiện

(event-driven), điều vốn chưa được hiểu rộng rãi. Tuy nhiên, theo kinh nghiệm của tôi, sự phức tạp tăng thêm liên quan đến việc theo dõi tiến độ của một saga gần như luôn được bù đắp nhiều hơn bởi những lợi ích liên quan đến việc có một kiến trúc ít phụ thuộc hơn.

Stepping aside from my own personal tastes, though, the general advice I give regarding orchestration versus choreography is that I am very relaxed in the use of orchestrated sagas when one team owns implementation of the entire saga. In such a situation, the more inherently coupled architecture is much easier to manage within the team boundary. If you have multiple teams involved, I greatly prefer the more decomposed choreographed saga, as it is easier to distribute responsibility for implementing the saga to the teams, with the more loosely coupled architecture allowing these teams to work more in isolation.

Tuy nhiên, gác lại sở thích cá nhân của riêng tôi, lời khuyên chung mà tôi đưa ra liên quan đến điều phối so với vũ đạo là tôi rất thoải mái trong việc sử dụng các saga điều phối khi một nhóm sở hữu việc triển khai toàn bộ saga. Trong tình huống như vậy, kiến trúc vốn dĩ phụ thuộc nhiều hơn dễ quản lý hơn nhiều trong phạm vi ranh giới của nhóm. Nếu bạn có nhiều nhóm liên quan, tôi rất thích saga vũ đạo phân rã hơn, vì việc phân tán trách nhiệm triển khai saga cho các nhóm dễ dàng hơn, với kiến trúc ít phụ thuộc hơn cho phép các nhóm này làm việc độc lập hơn.

It's worth noting that as a general rule, you'll be more likely to gravitate toward request-response-based calls with orchestration, whereas choreography tends to make heavier use of events. This isn't a hard rule, just a general observation. My own general leaning toward choreography is likely a function of the fact that I tend to gravitate toward event-driven interaction models—if you are finding the use of event-driven collaboration difficult to get your head around, choreography might not be for you.

Đáng lưu ý rằng theo một quy tắc chung, bạn sẽ có nhiều khả năng nghiêng về các lời gọi dựa trên request-response với điều phối, trong khi vũ đạo có xu hướng sử dụng nhiều sự kiện hơn. Đây không phải là một quy tắc cứng nhắc, chỉ là một quan sát chung. Việc bản thân tôi nói chung nghiêng về vũ đạo nhiều khả năng là một hệ quả của việc tôi có xu hướng nghiêng về các mô hình tương tác hướng sự kiện—nếu bạn đang thấy việc sử dụng sự cộng tác hướng sự kiện khó hình dung, thì vũ đạo có thể không dành cho bạn.

Sagas Versus Distributed Transactions — Saga so với Giao dịch phân tán

As I hope I have broken down by now, distributed transactions come with some significant challenges, and outside of some very specific situations I tend to avoid them. Pat Helland, a pioneer in distributed systems, distills the fundamental challenges that come with implementing distributed transactions for the kinds of applications we build today: 7

Như tôi hy vọng mình đã phân tích đến giờ, giao dịch phân tán đi kèm với một số thách thức đáng kể, và ngoài một số tình huống rất cụ thể tôi có xu hướng tránh chúng. Pat Helland, một người tiên phong trong lĩnh vực hệ thống phân tán, chắt lọc những thách thức nền tảng đi kèm với việc triển khai giao dịch phân tán cho các loại ứng dụng mà chúng ta xây dựng ngày nay: 7

In most distributed transaction systems, the failure of a single node causes transaction commit to stall. This in turn causes the application to get wedged. In such systems, the larger it gets, the more likely the system is going to be down. When flying an airplane that needs all of its engines to work, adding an engine reduces the availability of the airplane.

Trong hầu hết các hệ thống giao dịch phân tán, sự cố của một nút (node) duy nhất khiến cho việc commit giao dịch bị đình trệ. Điều này lại khiến cho ứng dụng bị kẹt cứng. Trong những hệ thống như vậy, nó càng lớn, thì hệ thống càng có nhiều khả năng bị sập. Khi lái một chiếc máy bay cần tất cả động cơ của nó hoạt động, việc thêm một động cơ làm giảm tính sẵn sàng (availability) của máy bay.

7 See Pat Helland, “Life Beyond Distributed Transactions: An Apostate’s Opinion,” *acmqueue* 14, no. 5 (December 12, 2016).

7 Xem Pat Helland, “Life Beyond Distributed Transactions: An Apostate’s Opinion,” *acmqueue* 14, no. 5 (December 12, 2016).

In my experience, explicitly modeling business processes as a saga avoids many of the challenges of distributed transactions, while having the added benefit of making what might otherwise be implicitly modeled processes much more explicit and obvious to your developers. Making the core business processes of your system a first-class concept will have a host of advantages.

Theo kinh nghiệm của tôi, việc mô hình hóa một cách tường minh các quy trình nghiệp vụ thành một saga tránh được nhiều thách thức của giao dịch phân tán, đồng thời có thêm lợi ích là làm cho những quy trình mà lẽ ra có thể được mô hình hóa một cách ngầm định trở nên tường minh và rõ ràng hơn nhiều đối với các lập trình viên của bạn. Việc làm cho các quy trình nghiệp vụ cốt lõi của hệ thống của bạn trở thành một khái niệm hạng nhất (first-class concept) sẽ mang lại một loạt lợi thế.

Summary — Tóm tắt

So as we can see, the path to implementing workflows in our microservice architecture comes down to explicitly modeling the business process we are trying to implement. This brings us back to the idea of modeling aspects of our **business domain** in our microservice architecture—explicitly modeling business processes makes sense if our microservice boundaries are also defined primarily in terms of our business domain.

Vậy như chúng ta có thể thấy, con đường để triển khai các workflow trong kiến trúc microservice của chúng ta rất cuộc quy về việc mô hình hóa một cách tường minh quy trình nghiệp vụ mà chúng ta đang cố triển khai. Điều này đưa chúng ta trở lại với ý tưởng mô hình hóa các khía cạnh của miền nghiệp vụ của chúng ta trong kiến trúc microservice—việc mô hình hóa tường minh các quy trình nghiệp vụ là hợp lý nếu các ranh giới microservice của chúng ta cũng được định nghĩa chủ yếu theo miền nghiệp vụ của ta.

Whether you decide to gravitate more toward orchestration or toward choreography, hopefully you’re much better placed to know what model will fit your problem space better.

Dù bạn quyết định nghiêng nhiều hơn về điều phối hay về vũ đạo, hy vọng bạn đang ở vị thế tốt hơn nhiều để biết mô hình nào sẽ phù hợp hơn với không gian vấn đề của mình.

If you want to explore this space in more detail, although sagas aren’t explicitly covered, Enterprise Integration Patterns by Gregor Hohpe and Bobby Woolf has a number of patterns that can be incredibly useful when implementing different types of workflow. 8 I can also heartily recommend Practical Process Automation by Bernd Ruecker. 9 Bernd’s book focuses much more on the orchestration side of sagas, but it is packed with useful information that makes it a natural following-on point for this topic.

Nếu bạn muốn khám phá không gian này chi tiết hơn, mặc dù saga không được đề cập một cách tường minh, Enterprise Integration Patterns của Gregor Hohpe và Bobby Woolf có một số pattern có thể cực kỳ hữu ích khi triển khai các loại workflow khác nhau. 8 Tôi cũng nhiệt thành khuyên đọc Practical Process Automation của Bernd Ruecker. 9 Cuốn sách của Bernd tập trung nhiều hơn vào khía cạnh điều phối của saga, nhưng nó chứa đầy thông tin hữu ích khiến nó trở thành một điểm tiếp nối tự nhiên cho chủ đề này.

Now we have a sense of how our microservices can communicate and coordinate with each other, but how do we build them in the first place? In our next chapter, we'll look at how to apply source control, **continuous integration**, and **continuous delivery** in the context of a microservice architecture.

Bây giờ chúng ta đã có cảm nhận về cách các microservice của mình có thể giao tiếp và điều phối lẫn nhau, nhưng làm sao chúng ta xây dựng chúng ngay từ đầu? Trong chương tiếp theo, chúng ta sẽ xem cách áp dụng quản lý mã nguồn (source control), tích hợp liên tục (CI), và chuyển giao liên tục (CD) trong bối cảnh của một kiến trúc microservice.

8 Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns (Boston: Addison-Wesley, 2003). 9 Bernd Ruecker, Practical Process Automation (Sebastopol: O'Reilly, 2021).

8 Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns (Boston: Addison-Wesley, 2003). 9 Bernd Ruecker, Practical Process Automation (Sebastopol: O'Reilly, 2021).

CHAPTER 7 Build — Chương 7: Build

We’ve spent a lot of time covering the design aspects of **microservices**, but we need to start getting a bit deeper into how your development process may need to change to accommodate this new style of **architecture**. In the following chapters, we’ll look at how we **deploy** and test our microservices, but before that we need to look at what comes first—what happens when a developer has a change ready to check in?

Chúng ta đã dành khá nhiều thời gian bàn về các khía cạnh thiết kế của microservices, nhưng giờ cần đi sâu hơn một chút vào việc quy trình phát triển của bạn có thể phải thay đổi như thế nào để thích ứng với phong cách kiến trúc (architecture) mới này. Trong các chương tiếp theo, chúng ta sẽ xem xét cách triển khai (deploy) và kiểm thử (testing) microservices, nhưng trước đó cần nhìn vào thứ diễn ra đầu tiên—điều gì xảy ra khi một lập trình viên đã sẵn sàng đẩy (check in) một thay đổi?

We’ll start this exploration by reviewing some foundational concepts—**continuous integration** and **continuous delivery**. They’re important concepts no matter what kind of systems architecture you might be using, but microservices open up a host of unique questions. From there we’ll look at pipelines and at different ways of managing source code for your services.

Chúng ta sẽ bắt đầu hành trình khám phá này bằng việc xem lại vài khái niệm nền tảng—tích hợp liên tục (continuous integration) và chuyển giao liên tục (continuous delivery). Đây là những khái niệm quan trọng bất kể bạn dùng kiểu kiến trúc hệ thống nào, nhưng microservices lại mở ra hàng loạt câu hỏi đặc thù. Từ đó, chúng ta sẽ xem xét pipeline và những cách khác nhau để quản lý mã nguồn cho các service của bạn.

A Brief Introduction to Continuous Integration — Giới thiệu ngắn gọn về Tích hợp liên tục (Continuous Integration)

Continuous integration (CI) has been around for a number of years. However, it’s worth spending a bit of time going over the basics, as there are some different options to consider, especially when we think about the mapping between microservices, builds, and version control repositories.

Tích hợp liên tục (continuous integration - CI) đã xuất hiện được nhiều năm. Tuy nhiên, vẫn đáng để dành chút thời gian điểm lại những điều cơ bản, bởi có một số lựa chọn khác nhau cần cân nhắc, đặc biệt khi ta nghĩ về cách ánh xạ giữa microservices, các bản build, và các kho lưu trữ kiểm soát phiên bản (version control repository).

With CI, the core goal is to keep everyone in sync with each other, which we achieve by frequently making sure that newly checked-in code properly integrates with existing code. To do this, a CI server detects that the code has been committed, checks it out, and carries out some verification such as making sure that the code compiles and that tests pass. As a bare minimum, we expect this integration

to be done on a daily basis, although in practice I've worked in multiple teams in which a developer has in fact integrated their changes multiple times per day.

Với CI, mục tiêu cốt lõi là giữ cho mọi người đồng bộ với nhau, điều mà ta đạt được bằng cách thường xuyên đảm bảo rằng mã vừa được đẩy lên tích hợp đúng đắn với mã hiện có. Để làm điều này, một CI server phát hiện rằng mã đã được commit, lấy mã đó về, và thực hiện một số bước xác minh chẳng hạn như đảm bảo mã biên dịch được và các bài test đều pass. Ở mức tối thiểu nhất, ta kỳ vọng việc tích hợp này được thực hiện hằng ngày, dù trên thực tế tôi đã làm việc trong nhiều nhóm mà ở đó một lập trình viên thực sự tích hợp các thay đổi của họ nhiều lần mỗi ngày.

As part of this process, we often create artifacts that are used for further validation, such as deploying a running **service** to run tests against it (we'll explore **testing** in depth in Chapter 9). Ideally, we want to **build** these artifacts once and once only and use them for all deployments of that version of the code. This is so we can avoid doing the same thing over and over again, and so we can confirm that the artifacts we deploy are the ones we tested. To enable these artifacts to be reused, we place them in a **repository** of some sort, either provided by the CI tool itself or in a separate system.

Là một phần của quá trình này, ta thường tạo ra các artifact dùng cho việc kiểm chứng sâu hơn, chẳng hạn triển khai một service đang chạy để chạy test đối chiếu với nó (chúng ta sẽ khám phá kiểm thử chuyên sâu ở Chương 9). Lý tưởng nhất là ta muốn build các artifact này đúng một lần duy nhất và dùng chúng cho mọi lần triển khai phiên bản mã đó. Như vậy ta tránh làm đi làm lại cùng một việc, và có thể xác nhận rằng các artifact ta triển khai chính là những artifact ta đã test. Để cho phép tái sử dụng các artifact này, ta đặt chúng vào một kho lưu trữ nào đó, hoặc do chính công cụ CI cung cấp hoặc trong một hệ thống riêng.

We'll be looking at the role of artifacts in more depth shortly, and we'll look in depth at testing in Chapter 9.

Chúng ta sẽ xem xét vai trò của các artifact sâu hơn trong chốc lát, và sẽ tìm hiểu kỹ về kiểm thử ở Chương 9.

CI has a number of benefits. We get fast feedback as to the quality of our code, through the use of static analysis and testing. CI also allows us to automate the creation of our binary artifacts. All the code required to build the **artifact** is itself version controlled, so we can re-create the artifact if needed. We can also trace from a deployed artifact back to the code, and, depending on the capabilities of the CI tool itself, we can see what tests were run on the code and artifact too. If embracing **infrastructure as code**, we can also version control all the code needed to configure the infrastructure for our **microservice** alongside the code for the microservice itself, improving transparency around changes and making it even easier to reproduce builds. It's for these reasons that CI has been so successful.

CI mang lại nhiều lợi ích. Ta nhận được phản hồi nhanh về chất lượng mã, thông qua việc dùng phân tích tĩnh (static analysis) và kiểm thử. CI cũng cho phép ta tự động hóa việc tạo ra các artifact nhị phân (binary artifact). Toàn bộ mã cần để build artifact đều được kiểm soát phiên bản, nên ta có thể tái tạo artifact nếu cần. Ta cũng có thể truy vết từ một artifact đã triển khai ngược về mã nguồn, và tùy vào năng lực của chính công cụ CI, ta còn có thể thấy những bài test nào đã chạy trên mã và artifact đó. Nếu áp dụng infrastructure as code, ta cũng có thể kiểm soát phiên bản toàn bộ mã cần để cấu hình hạ tầng cho microservice của mình song song với mã của chính microservice đó, giúp tăng tính minh bạch quanh các thay đổi và càng dễ tái tạo các bản build hơn. Chính vì những lý do này mà CI đã thành công đến vậy.

Are You Really Doing CI? — Bạn có thực sự đang làm CI không?

CI is a key practice that allows us to make changes quickly and easily, and without which the journey into microservices will be painful. I suspect you are probably using a CI tool in your own organization, but that might not be the same thing as actually doing CI. I've seen many people confuse adopting a CI tool with actually embracing CI. A CI tool, used well, will help you do CI—but using a tool like Jenkins, CircleCI, Travis, or one of the many other options out there doesn't guarantee you're actually doing CI right.

CI là một thực hành then chốt cho phép ta thực hiện thay đổi nhanh chóng và dễ dàng, mà thiếu nó thì hành trình đến với microservices sẽ rất gian nan. Tôi ngờ rằng bạn có lẽ đang dùng một công cụ CI trong tổ chức của mình, nhưng điều đó có thể không đồng nghĩa với việc thực sự làm CI. Tôi đã thấy nhiều người nhầm lẫn việc áp dụng một công cụ CI với việc thực sự đón nhận CI. Một công cụ CI, nếu dùng tốt, sẽ giúp bạn làm CI—nhưng việc dùng một công cụ như Jenkins, CircleCI, Travis, hay một trong vô số lựa chọn khác ngoài kia không đảm bảo rằng bạn đang thực sự làm CI đúng cách.

So how do you know if you're actually practicing CI? I really like Jez Humble's three questions he asks people to test if they really understand what CI is about—it might be interesting to ask yourself these same questions:

Vậy làm sao bạn biết mình có đang thực sự thực hành CI hay không? Tôi rất thích ba câu hỏi mà Jez Humble đặt ra cho mọi người để kiểm tra xem họ có thực sự hiểu CI là gì hay không—có thể sẽ thú vị nếu bạn tự hỏi mình chính những câu hỏi này:

Do you check in to mainline once per day? You need to make sure your code integrates. If you don't check your code together with everyone else's changes frequently, you end up making future integration harder. Even if you are using short-lived branches to manage changes, integrate as frequently as you can into a single mainline branch—at least once a day.

Bạn có check in vào mainline ít nhất một lần mỗi ngày không? Bạn cần đảm bảo mã của mình được tích hợp. Nếu bạn không thường xuyên kiểm tra mã của mình cùng với các thay đổi của mọi người khác, bạn sẽ khiến việc tích hợp về sau trở nên khó khăn hơn. Ngay cả khi bạn dùng các nhánh ngắn hạn (short-lived branch) để quản lý thay đổi, hãy tích hợp thường xuyên hết mức có thể vào một nhánh mainline duy nhất—ít nhất một lần mỗi ngày.

Do you have a suite of tests to validate your changes? Without tests, we just know that syntactically our integration has worked, but we don't know if we have broken the behavior of the system. CI without some verification that our code behaves as expected isn't CI.

Bạn có một bộ test để kiểm chứng các thay đổi của mình không? Không có test, ta chỉ biết rằng về mặt cú pháp thì việc tích hợp đã thành công, nhưng ta không biết liệu mình có làm hỏng hành vi của hệ thống hay không. CI mà không có bước kiểm chứng nào cho thấy mã hoạt động như kỳ vọng thì không phải là CI.

When the build is broken, is it the #1 priority of the team to fix it? A passing green build means our changes have safely been integrated. A red build means the last change possibly did not integrate. You need to stop all further check-ins that aren't involved in fixing the builds to get it passing again. If you let more changes pile up, the time it takes to fix the build will increase drastically. I've worked with teams where the build has been broken for days, resulting in substantial efforts to eventually get a passing build.

Khi bản build bị hỏng, việc sửa nó có phải là ưu tiên số 1 của nhóm không? Một bản build xanh (pass) nghĩa là các thay đổi của ta đã được tích hợp an toàn. Một bản build đỏ nghĩa

là thay đổi gần nhất có thể đã không tích hợp được. Bạn cần dừng tất cả các lần check-in tiếp theo không liên quan đến việc sửa bản build để đưa nó trở lại trạng thái pass. Nếu bạn để các thay đổi chồng chất thêm, thời gian cần để sửa bản build sẽ tăng vọt. Tôi đã làm việc với những nhóm mà bản build bị hỏng suốt nhiều ngày, dẫn đến những nỗ lực đáng kể mới có được một bản build pass.

Branching Models — Các mô hình phân nhánh (Branching Models)

Few topics around build and **deployment** seem to cause as much of a controversy as that of using source code branching for feature development. Branching in source code allows for development to be done in isolation without disrupting the work being done by others. On the surface, creating a source code branch for each feature being worked on—otherwise known as feature branching—seems like a useful concept.

Ít chủ đề nào quanh việc build và triển khai lại gây nhiều tranh cãi như việc dùng phân nhánh mã nguồn (source code branching) cho phát triển tính năng. Phân nhánh trong mã nguồn cho phép việc phát triển được thực hiện một cách độc lập mà không làm gián đoạn công việc của người khác. Nhìn bề ngoài, việc tạo một nhánh mã nguồn cho mỗi tính năng đang được làm—còn gọi là feature branching—có vẻ là một khái niệm hữu ích.

The problem is that when you work on a feature branch, you aren't regularly integrating your changes with everyone else. Fundamentally, you are delaying integration. And when you finally decide to integrate your changes with everyone else, you'll have a much more complex merge.

Vấn đề là khi bạn làm việc trên một feature branch, bạn không thường xuyên tích hợp các thay đổi của mình với mọi người khác. Về cơ bản, bạn đang trì hoãn việc tích hợp. Và khi cuối cùng bạn quyết định tích hợp các thay đổi của mình với mọi người khác, bạn sẽ phải đối mặt với một lần merge phức tạp hơn nhiều.

The alternative approach is to have everyone check in to the same “trunk” of source code. To keep changes from impacting other people, techniques like feature flags are used to “hide” incomplete work. This technique of everyone working off the same trunk is called trunk-based development .

Cách tiếp cận thay thế là để mọi người cùng check in vào cùng một “trunk” của mã nguồn. Để các thay đổi không ảnh hưởng đến người khác, người ta dùng các kỹ thuật như feature flag để “ẩn” phần công việc chưa hoàn thiện. Kỹ thuật mà mọi người cùng làm việc trên cùng một trunk này được gọi là trunk-based development.

The discussion around this topic is nuanced, but my own take is that the benefits of frequent integration—and validation of that integration—are significant enough that trunk-based development is my preferred style of development. Moreover, the work to implement feature flags is frequently beneficial in terms of progressive delivery, a concept we'll explore in Chapter 8 .

Cuộc tranh luận quanh chủ đề này khá tinh tế, nhưng quan điểm riêng của tôi là lợi ích của việc tích hợp thường xuyên—và việc kiểm chứng quá trình tích hợp đó—đủ lớn để trunk-based development trở thành phong cách phát triển tôi ưa thích. Hơn nữa, công sức bỏ ra để triển khai feature flag thường mang lại lợi ích về mặt progressive delivery, một khái niệm chúng ta sẽ khám phá ở Chương 8.



Be Careful About Branches

Hãy thận trọng với các nhánh

Integrate early, and integrate often. Avoid the use of long-lived branches for feature development, and consider trunk-based development instead. If you really have to use branches, keep them short!

Tích hợp sớm, và tích hợp thường xuyên. Tránh dùng các nhánh dài hạn (long-lived branch) cho phát triển tính năng, và thay vào đó hãy cân nhắc trunk-based development. Nếu bạn thực sự bắt buộc phải dùng nhánh, hãy giữ chúng thật ngắn!

Quite aside from my own anecdotal experience, there is a growing body of research that shows the efficacy of reducing the number of branches and adopting trunk-based development. The 2016 State of DevOps report by DORA and Puppet 1 carries out rigorous research into the delivery practices of organizations around the world and studies which practices are commonly used by high-performing teams:

Ngoài kinh nghiệm cá nhân mang tính giai thoại của riêng tôi, ngày càng có nhiều nghiên cứu cho thấy hiệu quả của việc giảm số lượng nhánh và áp dụng trunk-based development. Báo cáo State of DevOps năm 2016 của DORA và Puppet 1 thực hiện nghiên cứu nghiêm túc về các thực hành chuyển giao của các tổ chức trên khắp thế giới và khảo sát những thực hành nào thường được các nhóm hiệu suất cao sử dụng:

We found that having branches or forks with very short lifetimes (less than a day) before being merged into trunk, and less than three active branches in total, are important aspects of continuous delivery, and all contribute to higher performance. So does merging code into trunk or master on a daily basis. The State of DevOps report has continued to explore this topic in more depth in subsequent years, and has continued to find evidence for the efficacy of this approach.

Chúng tôi nhận thấy rằng việc có các nhánh hoặc fork với vòng đời rất ngắn (dưới một ngày) trước khi được merge vào trunk, và tổng cộng có ít hơn ba nhánh đang hoạt động, là những khía cạnh quan trọng của continuous delivery, và tất cả đều góp phần tạo nên hiệu suất cao hơn. Việc merge mã vào trunk hoặc master hằng ngày cũng vậy. Báo cáo State of DevOps đã tiếp tục khám phá chủ đề này sâu hơn trong những năm tiếp theo, và liên tục tìm thấy bằng chứng về hiệu quả của cách tiếp cận này.

A branch-heavy approach is still common in open source development, often through adopting the “GitFlow” development model. It’s worth noting that open source development is not the same as normal day-to-day development. Open source development is characterized by a large number of ad hoc contributions from time-poor “untrusted” committers, whose changes require vetting by a smaller number of “trusted” contributors. Typical day-to-day closed source development is normally done by a tight-knit team whose members all have commit rights, even if they decide to adopt some form of code review process. So what might work for open source development may not work for your day job. Even then, the State of DevOps report for 2019, 2 further exploring this topic, found some interesting insights into open source development and the impact of “long lived” branches:

Cách tiếp cận nặng về phân nhánh vẫn còn phổ biến trong phát triển mã nguồn mở, thường thông qua việc áp dụng mô hình phát triển “GitFlow”. Cần lưu ý rằng phát triển mã nguồn mở không giống với việc phát triển hằng ngày thông thường. Phát triển mã nguồn mở được đặc trưng bởi một lượng lớn các đóng góp tự phát từ những người đóng góp (committer) “không đáng tin cậy” eo hẹp về thời gian, mà các thay đổi của họ cần được kiểm duyệt bởi một số ít người đóng góp “đáng tin cậy” hơn. Việc phát triển mã nguồn đóng hằng ngày thông thường thì lại thường được thực hiện bởi một nhóm gắn bó chặt chẽ mà tất cả thành viên đều có quyền commit, ngay cả khi họ quyết định áp dụng một dạng quy trình review mã nào đó. Vậy nên điều gì hiệu quả với phát triển mã nguồn mở có thể không hiệu quả với công việc thường ngày của bạn. Ngay cả như vậy, báo cáo State of DevOps năm 2019, 2 khám phá sâu hơn chủ đề này, đã tìm ra một số phát hiện thú vị về phát triển mã nguồn mở và tác động của các nhánh “dài hạn”:

Our research findings extend to open source development in some areas:

Các phát hiện nghiên cứu của chúng tôi áp dụng được cho phát triển mã nguồn mở ở một số khía cạnh:

- Committing code sooner is better: In open source projects, many have observed that merging patches faster to prevent rebases helps developers move faster.
- Working in small batches is better: Large “patch bombs” are harder and slower to merge into a project than smaller, more readable patchsets since maintainers need more time to review the changes.
 - Commit mã sớm hơn thì tốt hơn: Trong các dự án mã nguồn mở, nhiều người đã quan sát thấy rằng merge các bản vá (patch) nhanh hơn để tránh phải rebase giúp lập trình viên di chuyển nhanh hơn.
 - Làm việc theo lô nhỏ thì tốt hơn: Các “patch bomb” lớn khó và chậm merge vào một dự án hơn so với những bộ vá nhỏ, dễ đọc hơn, vì người bảo trì cần nhiều thời gian hơn để review các thay đổi.

Whether you are working on a closed-source code base or an open source project, short-lived branches; small, readable patches; and automatic testing of changes make everyone more productive.

Dù bạn đang làm việc trên một mã nền nguồn đóng hay một dự án mã nguồn mở, các nhánh ngắn hạn; các bản vá nhỏ, dễ đọc; và việc tự động test các thay đổi đều khiến mọi người làm việc năng suất hơn.

1 Alanna Brown, Nicole Forsgren, Jez Humble, Nigel Kersten, and Gene Kim, 2016 State of DevOps Report , <https://oreil.ly/YqEEh> .

1 Alanna Brown, Nicole Forsgren, Jez Humble, Nigel Kersten, and Gene Kim, 2016 State of DevOps Report , <https://oreil.ly/YqEEh> .

2 Nicole Forsgren, Dustin Smith, Jez Humble, and Jessie Frazelle, Accelerate: State of DevOps 2019, <https://oreil.ly/mfkIJ> .

2 Nicole Forsgren, Dustin Smith, Jez Humble, and Jessie Frazelle, Accelerate: State of DevOps 2019, <https://oreil.ly/mfkIJ> .

Build Pipelines and Continuous Delivery — Build Pipeline và Continuous Delivery

Very early on in doing CI, my then-colleagues at Thoughtworks and I realized the value in sometimes having multiple stages inside a build. Tests are a very common case in which this comes into play. I may have a lot of fast, small-scoped tests, and a small number of slow, large-scoped tests. If we run all the tests together, and if we’re waiting for our large-scoped slow tests to finish, we may not be able to get fast feedback when our fast tests fail. And if the fast tests fail, there probably isn’t much sense in running the slower tests anyway! A solution to this problem is to have different stages in our build, creating what is known as a **build pipeline** . So we can have a dedicated stage for all the fast tests, which we run first, and if they all pass, we then run a separate stage for the slower tests.

Rất sớm trong quá trình làm CI, các đồng nghiệp khi đó của tôi tại Thoughtworks và tôi đã nhận ra giá trị của việc đôi khi có nhiều giai đoạn (stage) bên trong một bản build. Test là một trường hợp rất phổ biến mà điều này phát huy tác dụng. Tôi có thể có rất nhiều test nhanh, phạm vi nhỏ, và một số ít test chậm, phạm vi lớn. Nếu ta chạy tất cả test cùng nhau, và nếu ta đang chờ các test chậm phạm vi lớn hoàn thành, ta có thể không nhận được phản hồi nhanh khi các test nhanh của mình thất bại. Và nếu các test nhanh đã thất

bại, thì có lẽ cũng chẳng có nhiều ý nghĩa khi chạy các test chậm hơn nữa! Một giải pháp cho vấn đề này là có các giai đoạn khác nhau trong bản build, tạo ra cái được gọi là build pipeline. Vậy là ta có thể có một giai đoạn riêng cho tất cả các test nhanh, được chạy trước, và nếu tất cả đều pass, ta sẽ chạy một giai đoạn riêng cho các test chậm hơn.

This build pipeline concept gives us a nice way of tracking the progress of our software as it clears each stage, helping give us insight into the quality of our software. We create a deployable artifact, the thing that will ultimately be deployed into production, and use this artifact throughout the pipeline. In our context, this artifact will relate to a microservice we want to deploy. In Figure 7-1, we see this happening—the same artifact is used in each stage of the pipeline, giving us more and more confidence that the software will work in production.

Khái niệm build pipeline này cho ta một cách hay để theo dõi tiến trình của phần mềm khi nó vượt qua từng giai đoạn, giúp ta hiểu rõ hơn về chất lượng phần mềm của mình. Ta tạo ra một artifact có thể triển khai được, thứ rất cuộc sẽ được triển khai vào môi trường production, và dùng artifact này xuyên suốt pipeline. Trong ngữ cảnh của ta, artifact này sẽ tương ứng với một microservice mà ta muốn triển khai. Trong Hình 7-1, ta thấy điều này diễn ra—cùng một artifact được dùng ở mỗi giai đoạn của pipeline, mang lại cho ta ngày càng nhiều niềm tin rằng phần mềm sẽ hoạt động trong production.

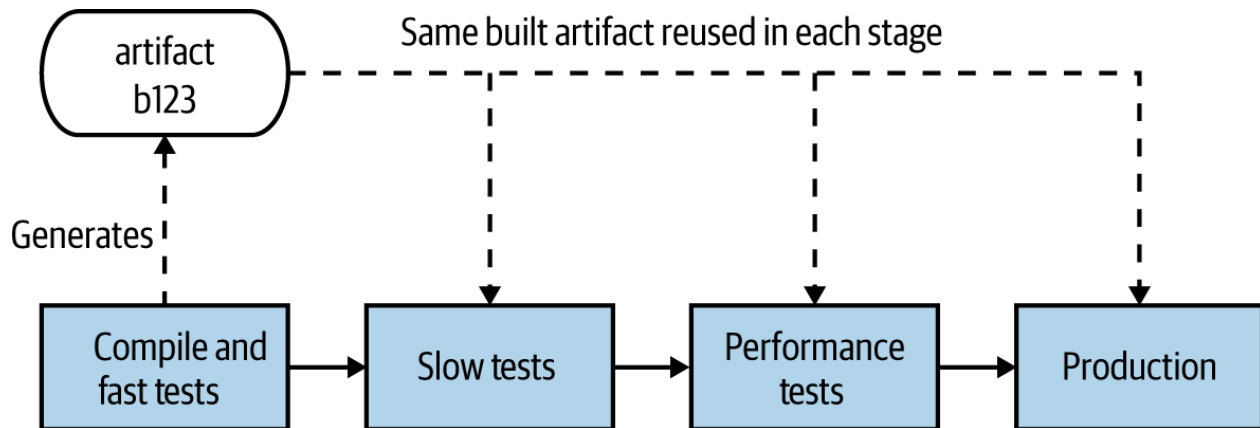


Figure 7-1. A simple **release** process for our Catalog service modeled as a build pipeline

Hình 7-1. Một quy trình phát hành đơn giản cho service Catalog của ta được mô hình hóa thành một build pipeline

Continuous delivery (CD) builds on this concept, and then some. As outlined in Jez Humble and Dave Farley's book of the same name, 3 CD is the approach whereby we get constant feedback on the production readiness of each and every check-in, and furthermore treat each and every check-in as a release candidate.

Chuyển giao liên tục (continuous delivery - CD) xây dựng trên khái niệm này, và còn hơn thế nữa. Như được trình bày trong cuốn sách cùng tên của Jez Humble và Dave Farley, 3 CD là cách tiếp cận trong đó ta nhận được phản hồi liên tục về mức độ sẵn sàng cho production của từng lần check-in, và hơn nữa coi mỗi và mọi lần check-in như một ứng viên phát hành (release candidate).

To fully embrace this concept, we need to model all the processes involved in getting our software from check-in to production, and we need to know where any given version of the software is in terms of being cleared for release. In CD, we do this by

Để đón nhận trọn vẹn khái niệm này, ta cần mô hình hóa tất cả các quy trình liên quan

đến việc đưa phần mềm từ lúc check-in đến production, và ta cần biết một phiên bản phần mềm bất kỳ đang ở đâu trên hành trình được duyệt để phát hành. Trong CD, ta làm điều này bằng cách

3 For more details, see Jez Humble and David Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation* (Upper Saddle River, NJ: Addison-Wesley, 2010).

3 Để biết thêm chi tiết, xem Jez Humble and David Farley, *Continuous Delivery: Reliable Software Releases Through Build, Test, and Deployment Automation* (Upper Saddle River, NJ: Addison-Wesley, 2010).

modeling each and every stage our software has to go through, both manual and automated, an example of which I shared for our Catalog service in Figure 7-1 . Most CI tools nowadays provide some support for defining and visualizing the state of build pipelines like this.

mô hình hóa từng giai đoạn mà phần mềm của ta phải đi qua, cả thủ công lẫn tự động, một ví dụ mà tôi đã chia sẻ cho service Catalog của ta ở Hình 7-1. Hầu hết các công cụ CI ngày nay đều cung cấp một mức hỗ trợ nào đó cho việc định nghĩa và trực quan hóa trạng thái của các build pipeline như thế này.

If the new Catalog service passes whatever checks are carried out at a stage in the pipeline, it can then move on to the next step. If it doesn't pass a stage, our CI tool can let us know which stages the build has passed and can get visibility about what failed. If we need to do something to fix it, we'd make a change and check it in, allowing the new version of our microservice to try and pass all the stages before being available for deployment. In Figure 7-2 , we see an example of this: build-120 failed the fast test stage, build-121 failed at the performance tests, but build-122 made it all the way to production.

Nếu service Catalog mới vượt qua bất kỳ kiểm tra nào được thực hiện ở một giai đoạn trong pipeline, nó có thể chuyển sang bước tiếp theo. Nếu nó không vượt qua một giai đoạn, công cụ CI của ta có thể cho ta biết bản build đã vượt qua những giai đoạn nào và cho ta thấy rõ điều gì đã thất bại. Nếu cần làm gì đó để sửa, ta sẽ thực hiện một thay đổi và check in nó, cho phép phiên bản mới của microservice thử vượt qua tất cả các giai đoạn trước khi sẵn sàng để triển khai. Trong Hình 7-2, ta thấy một ví dụ về điều này: build-120 thất bại ở giai đoạn test nhanh, build-121 thất bại ở các test hiệu năng, nhưng build-122 đã đi suốt đến tận production.

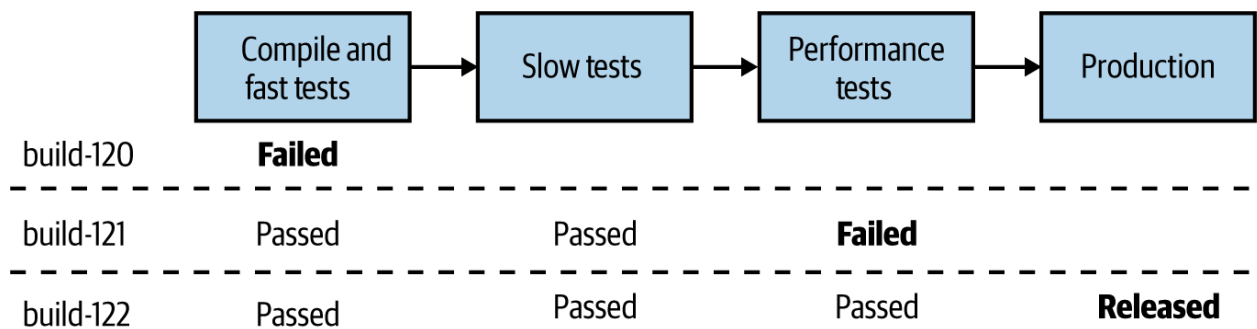


Figure 7-2. Our Catalog microservice can get deployed only if it passes each step in our pipeline

Hình 7-2. Microservice Catalog của ta chỉ được triển khai nếu nó vượt qua từng bước trong pipeline của ta

Continuous Delivery Versus Continuous Deployment — Continuous Delivery so với Continuous Deployment

I have on occasion seen some confusion around the terms continuous delivery and continuous deployment. As we've already discussed, continuous delivery is the concept whereby each check-in is treated as a release candidate, and whereby we can assess the quality of each release candidate to decide if it's ready to be deployed. With continuous deployment on the other hand, all check-ins have to be validated using automated mechanisms (tests for example), and any software that passes these verification checks is deployed automatically, without human intervention. Continuous deployment can therefore be considered an extension of continuous delivery. Without continuous delivery, you can't do continuous deployment. But you can do continuous delivery without doing continuous deployment.

Đôi khi tôi thấy có chút nhầm lẫn quanh các thuật ngữ continuous delivery và continuous deployment. Như ta đã bàn, continuous delivery là khái niệm trong đó mỗi lần check-in được coi như một release candidate, và ta có thể đánh giá chất lượng của từng release candidate để quyết định xem nó đã sẵn sàng để triển khai chưa. Mặt khác, với continuous deployment, tất cả các lần check-in phải được kiểm chứng bằng các cơ chế tự động (ví dụ test), và bất kỳ phần mềm nào vượt qua các bước kiểm tra xác minh này đều được triển khai tự động, không cần sự can thiệp của con người. Do đó, continuous deployment có thể được xem là một phần mở rộng của continuous delivery. Không có continuous delivery, bạn không thể làm continuous deployment. Nhưng bạn có thể làm continuous delivery mà không làm continuous deployment.

Continuous deployment isn't right for everyone—many people want some human interaction to decide whether software should be deployed, something that is totally compatible with continuous delivery. However, adopting continuous delivery does imply continual focus on optimizing your path to production, the increased visibility making it easier to see where optimizations should be made. Often human involvement in the post-check-in process is a bottleneck that needs addressing—see the shift from manual regression testing to automated functional testing, for example. As a result, as you automate more and more of your build, deployment, and release process, you may find yourself getting closer and closer to continuous deployment.

Continuous deployment không phù hợp với tất cả mọi người—nhiều người muốn có một mức tương tác của con người để quyết định xem phần mềm có nên được triển khai hay không, điều này hoàn toàn tương thích với continuous delivery. Tuy nhiên, việc áp dụng continuous delivery quả thực hàm ý sự tập trung liên tục vào việc tối ưu hóa con đường đến production của bạn, sự minh bạch gia tăng giúp dễ thấy hơn những chỗ cần tối ưu. Sự tham gia của con người trong quy trình sau check-in thường là một nút thắt cổ chai cần được giải quyết—ví dụ như sự dịch chuyển từ kiểm thử hồi quy thủ công sang kiểm thử chức năng tự động. Kết quả là, khi bạn tự động hóa ngày càng nhiều quy trình build, triển khai, và phát hành của mình, bạn có thể thấy mình đang tiến ngày càng gần đến continuous deployment.

Tooling — Công cụ

Ideally, you want a tool that embraces continuous delivery as a first-class concept. I have seen many people try to hack and extend CI tools to make them do CD, often resulting in complex systems that are nowhere near as easy to use as tools that build in CD from the beginning. Tools that fully support CD allow you to define and visualize these pipelines, modeling the entire path to production for your

software. As a version of our code moves through the pipeline, if it passes one of these automated verification stages, it moves to the next stage.

Lý tưởng nhất là bạn muốn một công cụ đón nhận continuous delivery như một khái niệm hạng nhất (first-class concept). Tôi đã thấy nhiều người cố gắng “chế cháo” và mở rộng các công cụ CI để bắt chúng làm CD, thường dẫn đến những hệ thống phức tạp khó dùng hơn nhiều so với các công cụ được xây dựng để làm CD ngay từ đầu. Các công cụ hỗ trợ đầy đủ CD cho phép bạn định nghĩa và trực quan hóa các pipeline này, mô hình hóa toàn bộ con đường đến production cho phần mềm của bạn. Khi một phiên bản mã của ta di chuyển qua pipeline, nếu nó vượt qua một trong các giai đoạn kiểm chứng tự động này, nó chuyển sang giai đoạn tiếp theo.

Some stages may be manual. For example, if we have a manual user acceptance testing (UAT) process, I should be able to use a CD tool to model it. I can see the next available build ready to be deployed into our UAT environment and then deploy it, and then if it passes our manual checks, mark that stage as being successful so it can move to the next one. If the subsequent stage is automated, it will then get triggered automatically.

Một số giai đoạn có thể là thủ công. Ví dụ, nếu ta có một quy trình kiểm thử chấp nhận của người dùng (user acceptance testing - UAT) thủ công, tôi sẽ có thể dùng một công cụ CD để mô hình hóa nó. Tôi có thể thấy bản build tiếp theo sẵn sàng để được triển khai vào môi trường UAT của ta và rồi triển khai nó, sau đó nếu nó vượt qua các kiểm tra thủ công của ta, đánh dấu giai đoạn đó là thành công để nó có thể chuyển sang giai đoạn tiếp theo. Nếu giai đoạn kế tiếp là tự động, nó sẽ được kích hoạt tự động.

Trade-Offs and Environments — Đánh đổi và Môi trường

As we move our microservice artifact through this pipeline, our microservice gets deployed into different environments. Different environments serve different purposes, and they may have different characteristics.

Khi ta di chuyển artifact microservice của mình qua pipeline này, microservice của ta được triển khai vào các môi trường khác nhau. Các môi trường khác nhau phục vụ những mục đích khác nhau, và chúng có thể có những đặc tính khác nhau.

Structuring a pipeline, and therefore working out what environments you’ll need, is in and of itself a balancing act. Early on in the pipeline, we’re looking for fast feedback on the production readiness of our software. We want to let developers know as soon as possible if there is a problem—the sooner we get feedback about a problem occurring, the quicker it is to fix it. As our software gets closer to production, we want more certainty that the software will work, and we’ll therefore be deploying into increasingly production-like environments—we can see this trade-off in Figure 7-3 .

Việc cấu trúc một pipeline, và qua đó tìm ra những môi trường bạn sẽ cần, tự bản thân nó đã là một hành động cân bằng. Ở giai đoạn đầu của pipeline, ta đang tìm phản hồi nhanh về mức độ sẵn sàng cho production của phần mềm. Ta muốn cho lập trình viên biết sớm nhất có thể nếu có vấn đề—càng nhận được phản hồi sớm về một vấn đề đang xảy ra, càng sửa được nhanh. Khi phần mềm của ta tiến gần production hơn, ta muốn chắc chắn hơn rằng phần mềm sẽ hoạt động, và do đó ta sẽ triển khai vào các môi trường ngày càng giống production—ta có thể thấy sự đánh đổi này ở Hình 7-3.

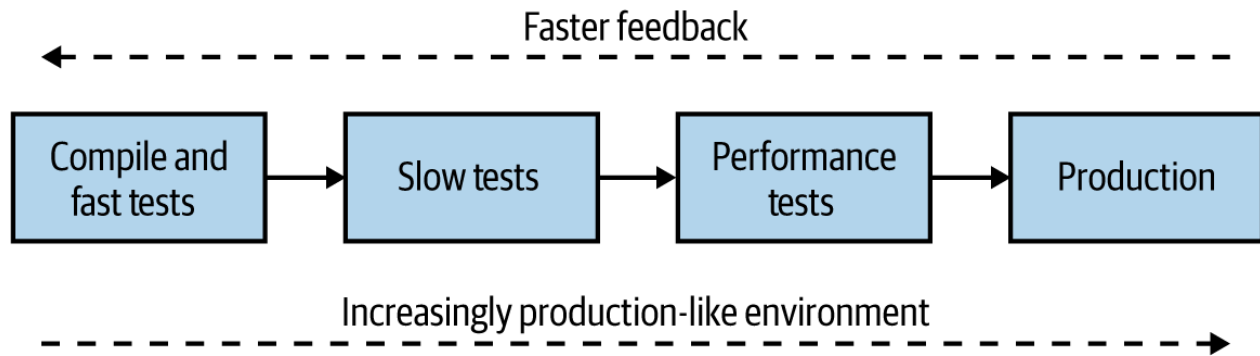


Figure 7-3. Balancing a build pipeline for fast feedback and production-like execution environments. You get the fastest feedback on your development laptop—but that is far from production-like. You could roll out every commit to an environment that is a faithful reproduction of your actual production environment, but that will likely take longer and cost more. So finding the balance is key, and continuing to review the trade-off between fast feedback and the need for production-like environments can be an incredibly important ongoing activity.

Hình 7-3. Cân bằng một build pipeline để có phản hồi nhanh và các môi trường thực thi giống production. Bạn nhận được phản hồi nhanh nhất trên chiếc laptop phát triển của mình—nhưng nó còn xa mới giống production. Bạn có thể tung mỗi commit ra một môi trường tái tạo trung thực môi trường production thực tế của bạn, nhưng điều đó nhiều khả năng sẽ tốn nhiều thời gian và chi phí hơn. Vậy nên tìm ra điểm cân bằng là then chốt, và việc tiếp tục xem xét lại sự đánh đổi giữa phản hồi nhanh và nhu cầu về các môi trường giống production có thể là một hoạt động liên tục cực kỳ quan trọng.

The challenges of creating a production-like environment are also part of why more people are doing forms of testing in production, including techniques such as smoke testing and parallel runs. We'll come back to this topic in Chapter 8 .

Những thách thức của việc tạo ra một môi trường giống production cũng là một phần lý do vì sao ngày càng nhiều người thực hiện các hình thức kiểm thử trong production, bao gồm các kỹ thuật như smoke testing và parallel run. Chúng ta sẽ quay lại chủ đề này ở Chương 8.

Artifact Creation — Tạo Artifact

As we move our microservice into different environments, we actually have to have something to deploy. It turns out there are a number of different options for what type of deployment artifact you can use. In general, which artifact you create will depend greatly on the technology you have chosen to adopt for deployment. We'll be looking at that in depth in the next chapter, but I wanted to give you some very important tips about how artifact creation should fit into your CI/CD build process.

Khi ta di chuyển microservice của mình vào các môi trường khác nhau, ta thực ra phải có một thứ gì đó để triển khai. Hóa ra có một số lựa chọn khác nhau về loại artifact triển khai mà bạn có thể dùng. Nhìn chung, artifact bạn tạo ra sẽ phụ thuộc rất nhiều vào công nghệ mà bạn đã chọn để triển khai. Chúng ta sẽ xem xét điều đó chuyên sâu ở chương tiếp theo, nhưng tôi muốn cho bạn một vài mẹo rất quan trọng về cách việc tạo artifact nên ăn khớp vào quy trình build CI/CD của bạn.

To keep things simple, we'll sidestep exactly what type of artifact we are creating— just consider it a single deployable blob for the moment. Now, there are two important rules we need to consider.

Firstly, as I mentioned earlier, we should build an artifact once and once only. Building the same thing over and over again is a waste of time and bad for the planet, and it can theoretically introduce problems if the build configuration isn't exactly the same each time. On some programming languages, a different build flag can make the software behave quite differently. Secondly, the artifact you verify should be the artifact you deploy! If you build a microservice, test it, say “yes, it's working,” and then build it again for deployment into production, how do you know that the software you validated is the same software you deployed?

Để cho đơn giản, ta sẽ tạm gác lại chính xác loại artifact ta đang tạo—chỉ cần coi nó như một khối blob có thể triển khai được duy nhất vào lúc này. Bây giờ, có hai quy tắc quan trọng ta cần cân nhắc. Thứ nhất, như tôi đã đề cập trước đó, ta nên build một artifact đúng một lần duy nhất. Build đi build lại cùng một thứ là lãng phí thời gian và có hại cho hành tinh này, và về lý thuyết nó có thể gây ra vấn đề nếu cấu hình build không hoàn toàn giống nhau ở mỗi lần. Trên một số ngôn ngữ lập trình, một cờ build (build flag) khác có thể khiến phần mềm hành xử khá khác biệt. Thứ hai, artifact bạn kiểm chứng phải là artifact bạn triển khai! Nếu bạn build một microservice, test nó, nói “vâng, nó hoạt động,” rồi lại build nó lần nữa để triển khai vào production, làm sao bạn biết rằng phần mềm bạn đã kiểm chứng cũng chính là phần mềm bạn đã triển khai?

Taking these two ideas together, we have a pretty simple approach. Build your deployable artifact once and once only, and ideally do it pretty early in the pipeline. I would typically do this after compiling the code (if required) and running my fast tests. Once created, this artifact is stored in an appropriate repository—this could be something like Artifactory or Nexus, or perhaps a **container** registry. Your choice of deployment artifact likely dictates the nature of the artifact store. This same artifact can then be used for all stages in the pipeline that follow, up to and including deployment into production. So coming back to our earlier pipeline, we can see in Figure 7-4 that we create an artifact for our Catalog service during the first stage of the pipeline and then deploy the same build-123 artifact as part of the slow tests, performance tests, and production stages.

Gộp hai ý tưởng này lại, ta có một cách tiếp cận khá đơn giản. Build artifact triển khai của bạn đúng một lần duy nhất, và lý tưởng nhất là làm điều đó khá sớm trong pipeline. Tôi thường làm điều này sau khi biên dịch mã (nếu cần) và chạy các test nhanh của mình. Sau khi được tạo, artifact này được lưu vào một kho lưu trữ phù hợp—đó có thể là thứ gì đó như Artifactory hay Nexus, hoặc có lẽ là một container registry. Lựa chọn artifact triển khai của bạn nhiều khả năng quyết định bản chất của kho lưu trữ artifact. Cùng artifact này sau đó có thể được dùng cho tất cả các giai đoạn tiếp theo trong pipeline, cho đến và bao gồm cả việc triển khai vào production. Vậy quay lại pipeline trước đó của ta, ta có thể thấy ở Hình 7-4 rằng ta tạo một artifact cho service Catalog trong giai đoạn đầu tiên của pipeline rồi triển khai cùng artifact build-123 đó như một phần của các giai đoạn test chậm, test hiệu năng, và production.

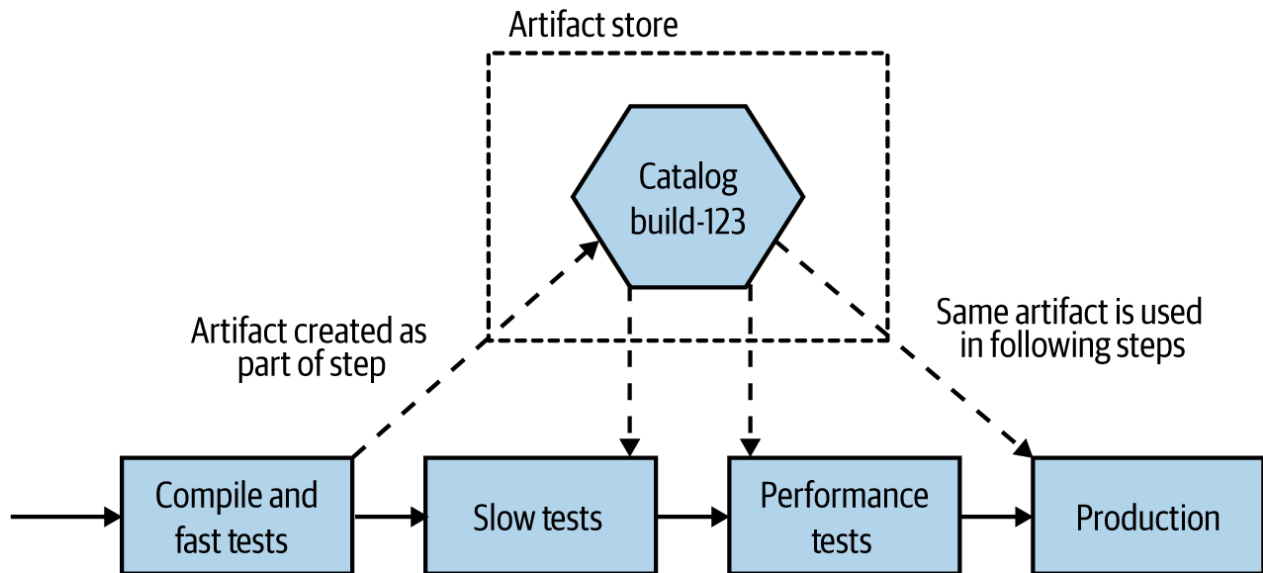


Figure 7-4. The same artifact is deployed into each environment

Hình 7-4. Cùng một artifact được triển khai vào mỗi môi trường

If the same artifact is going to be used across multiple environments, any aspects of configuration that vary from environment to environment need to be kept outside the artifact itself. As a simple example, I might want to configure application logs so that everything at DEBUG level and above is logged when running the Slow Tests stage, giving me more information to diagnose why a test fails. I might decide, though, to change this to INFO to reduce the log volume for the Performance Tests and Production deployment.

Nếu cùng một artifact sẽ được dùng trên nhiều môi trường, thì bất kỳ khía cạnh cấu hình nào thay đổi từ môi trường này sang môi trường khác đều cần được giữ bên ngoài bản thân artifact. Lấy một ví dụ đơn giản, tôi có thể muốn cấu hình log ứng dụng sao cho mọi thứ ở mức DEBUG trở lên được ghi lại khi chạy giai đoạn Slow Tests, cho tôi nhiều thông tin hơn để chẩn đoán vì sao một test thất bại. Tuy vậy, tôi có thể quyết định đổi điều này thành INFO để giảm khối lượng log cho các giai đoạn Performance Tests và triển khai Production.



Artifact Creation Tips

Mẹo tạo Artifact

Build a deployment artifact for your microservice once. Reuse this artifact everywhere you want to deploy that version of your microservice. Keep your deployment artifact environment-agnostic—

store environment-specific configuration elsewhere.

Build một artifact triển khai cho microservice của bạn một lần. Tái sử dụng artifact này ở mọi nơi bạn muốn triển khai phiên bản đó của microservice. Giữ cho artifact triển khai của bạn không phụ thuộc vào môi trường (environment-agnostic)—lưu cấu hình đặc thù theo môi trường ở nơi khác.

Mapping Source Code and Builds to Microservices — Ánh xạ Mã nguồn và các bản Build với Microservices

We’ve already looked at one topic that can excite warring factions—feature branching versus trunk-based development—but it turns out that the controversy isn’t over for this chapter. Another topic that is likely to elicit some pretty diverse opinions is the organization of code for our microservices. I have my own preferences, but before we get to those, let’s explore the main options for how we organize code for our microservices.

Ta đã xem xét một chủ đề có thể làm dấy lên các phe phái xung khắc—feature branching so với trunk-based development—nhưng hóa ra sự tranh cãi của chương này vẫn chưa hết. Một chủ đề khác nhiều khả năng cũng gợi lên những ý kiến khá đa dạng là việc tổ chức mã cho microservices của ta. Tôi có những sở thích riêng, nhưng trước khi đến với chúng, hãy cùng khám phá các lựa chọn chính về cách tổ chức mã cho microservices của ta.

One Giant Repo, One Giant Build — Một Repo khổng lồ, một Build khổng lồ

If we start with the simplest option, we could lump everything in together. We have a single, giant repository storing all our code, and we have a single build, as we see in Figure 7-5 . Any check-in to this source code repository will cause our build to trigger, where we will run all the verification steps associated with all our microservices and produce multiple artifacts, all tied back to the same build.

Nếu bắt đầu với lựa chọn đơn giản nhất, ta có thể gộp tất cả mọi thứ vào với nhau. Ta có một repository khổng lồ duy nhất lưu trữ toàn bộ mã của ta, và ta có một bản build duy nhất, như ta thấy ở Hình 7-5. Bất kỳ lần check-in nào vào kho mã nguồn này đều khiến bản build của ta được kích hoạt, nơi ta sẽ chạy tất cả các bước kiểm chứng liên quan đến tất cả microservices của ta và tạo ra nhiều artifact, tất cả đều gắn ngược về cùng một bản build.

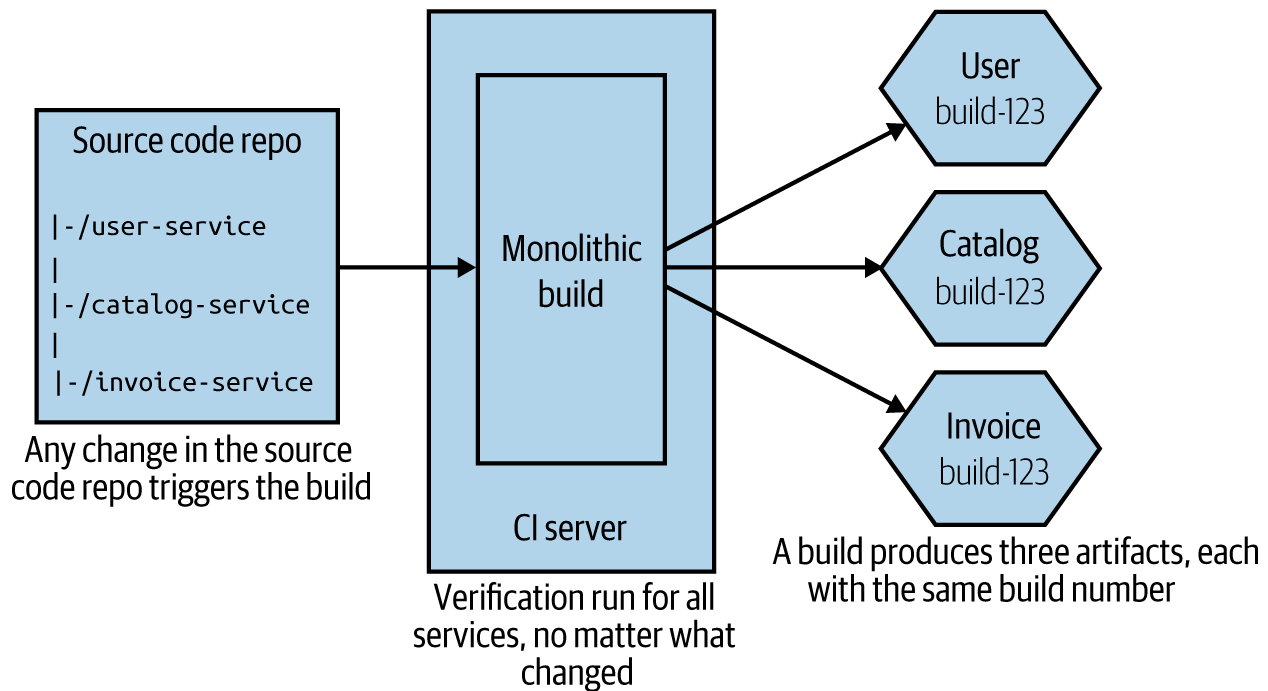


Figure 7-5. Using a single source code repository and CI build for all microservices

Hình 7-5. Dùng một kho mã nguồn duy nhất và bản build CI duy nhất cho tất cả microservices

Compared to other approaches, this seems much simpler on the surface: fewer repositories to be concerned about, and a conceptually simpler build. From a developer point of view, things are pretty straightforward too. I just check code in. If I have to work on multiple services at once, I just have to worry about one commit.

So với các cách tiếp cận khác, nhìn bề ngoài cách này có vẻ đơn giản hơn nhiều: ít repository phải bận tâm hơn, và một bản build đơn giản hơn về mặt khái niệm. Từ góc nhìn của lập trình viên, mọi thứ cũng khá đơn giản. Tôi chỉ việc check in mã. Nếu tôi phải làm việc trên nhiều service cùng lúc, tôi chỉ phải bận tâm về một commit.

This model can work perfectly well if you buy into the idea of lockstep releases, where you don't mind deploying multiple services at once. In general, this is absolutely a pattern to avoid, but very early on in a project, especially if only one team is working on everything, this model might make sense for short periods of time.

Mô hình này có thể hoạt động hoàn hảo nếu bạn chấp nhận ý tưởng phát hành đồng loạt (lockstep release), khi bạn không ngại triển khai nhiều service cùng một lúc. Nhìn chung, đây tuyệt đối là một mẫu cần tránh, nhưng ở giai đoạn rất sớm của một dự án, đặc biệt nếu chỉ có một nhóm làm việc trên tất cả mọi thứ, mô hình này có thể hợp lý trong những khoảng thời gian ngắn.

Now let me explain some of the significant downsides to this approach. If I make a one-line change to a single service—for example, changing behavior in the User service in Figure 7-5—all the other services get verified and built. This could take more time than needed—I'm waiting for things that probably don't need to be tested. This impacts our cycle time, the speed at which we can move a single change from development to live. More troubling, though, is knowing what artifacts should or shouldn't be deployed. Do I now need to deploy all the build services to push my small change into production? It can be hard to tell; trying to guess which services really changed just by reading

the commit messages is difficult. Organizations using this approach often fall back to just deploying everything together, which we really want to avoid.

Giờ hãy để tôi giải thích một số nhược điểm đáng kể của cách tiếp cận này. Nếu tôi thực hiện một thay đổi một dòng cho một service duy nhất—ví dụ, thay đổi hành vi trong service User ở Hình 7-5—tất cả các service khác đều được kiểm chứng và build. Điều này có thể tốn nhiều thời gian hơn cần thiết—tôi đang chờ những thứ có lẽ chẳng cần được test. Điều này ảnh hưởng đến cycle time của ta, tức tốc độ ta có thể đưa một thay đổi đơn lẻ từ phát triển đến chạy thật. Tuy nhiên, điều phiền toái hơn là việc biết artifact nào nên hoặc không nên được triển khai. Giờ tôi có cần triển khai tất cả các service đã build để đẩy thay đổi nhỏ của mình vào production không? Khó mà nói được; cố gắng đoán xem service nào thực sự thay đổi chỉ bằng cách đọc các thông điệp commit là việc khó khăn. Các tổ chức dùng cách tiếp cận này thường lười về việc cứ triển khai tất cả cùng nhau, điều mà ta thực sự muốn tránh.

Furthermore, if my one-line change to the User service breaks the build, no other changes can be made to the other services until that break is fixed. And think about a scenario in which you have multiple teams all sharing this giant build. Who is in charge?

Hơn nữa, nếu thay đổi một dòng của tôi cho service User làm hỏng bản build, thì không thể thực hiện thay đổi nào khác cho các service khác cho đến khi chỗ hỏng đó được sửa. Và hãy nghĩ đến một kịch bản trong đó bạn có nhiều nhóm cùng chia sẻ bản build khổng lồ này. Ai là người chịu trách nhiệm?

Arguably, this approach is a form of **monorepo**. In practice, however, most of the monorepo implementations I've seen map multiple builds to different parts of the repo, something we'll explore in more depth shortly. So you could see this pattern of one repo mapping to a single build as the worst form of monorepo for those wanting to build multiple independently deployable microservices.

Có thể lập luận rằng cách tiếp cận này là một dạng monorepo. Tuy nhiên, trong thực tế, hầu hết các triển khai monorepo mà tôi từng thấy đều ánh xạ nhiều bản build với những phần khác nhau của repo, điều mà ta sẽ khám phá sâu hơn trong chốc lát. Vậy nên bạn có thể coi mẫu một repo ánh xạ với một bản build duy nhất này là dạng monorepo tệ nhất đối với những ai muốn build nhiều microservice có thể triển khai độc lập.

In practice, I almost never see this approach used, except in the earliest stages of projects. To be honest, either of the two following approaches are significantly preferable, so we'll focus on those instead.

Trong thực tế, tôi gần như không bao giờ thấy cách tiếp cận này được dùng, ngoại trừ ở những giai đoạn sớm nhất của các dự án. Thành thật mà nói, bất kỳ cách nào trong hai cách tiếp cận tiếp theo đều ưu việt hơn đáng kể, nên ta sẽ tập trung vào chúng thay vào đó.

Pattern: One Repository per Microservice (aka Multirepo) — Mẫu: Mỗi Microservice một Repository (còn gọi là Multirepo)

With the one repository per microservice pattern (more commonly referred to as the **multirepo** pattern when being compared to the monorepo pattern), the code for each microservice is stored in its own source code repository, as we see in Figure 7-6 . This approach leads to a straightforward mapping between source code changes and CI builds.

Với mẫu mỗi microservice một repository (thường được gọi là mẫu multirepo khi so sánh với mẫu monorepo), mã cho mỗi microservice được lưu trong kho mã nguồn riêng của nó, như ta thấy ở Hình 7-6. Cách tiếp cận này dẫn đến một sự ánh xạ trực tiếp giữa các thay đổi mã nguồn và các bản build CI.

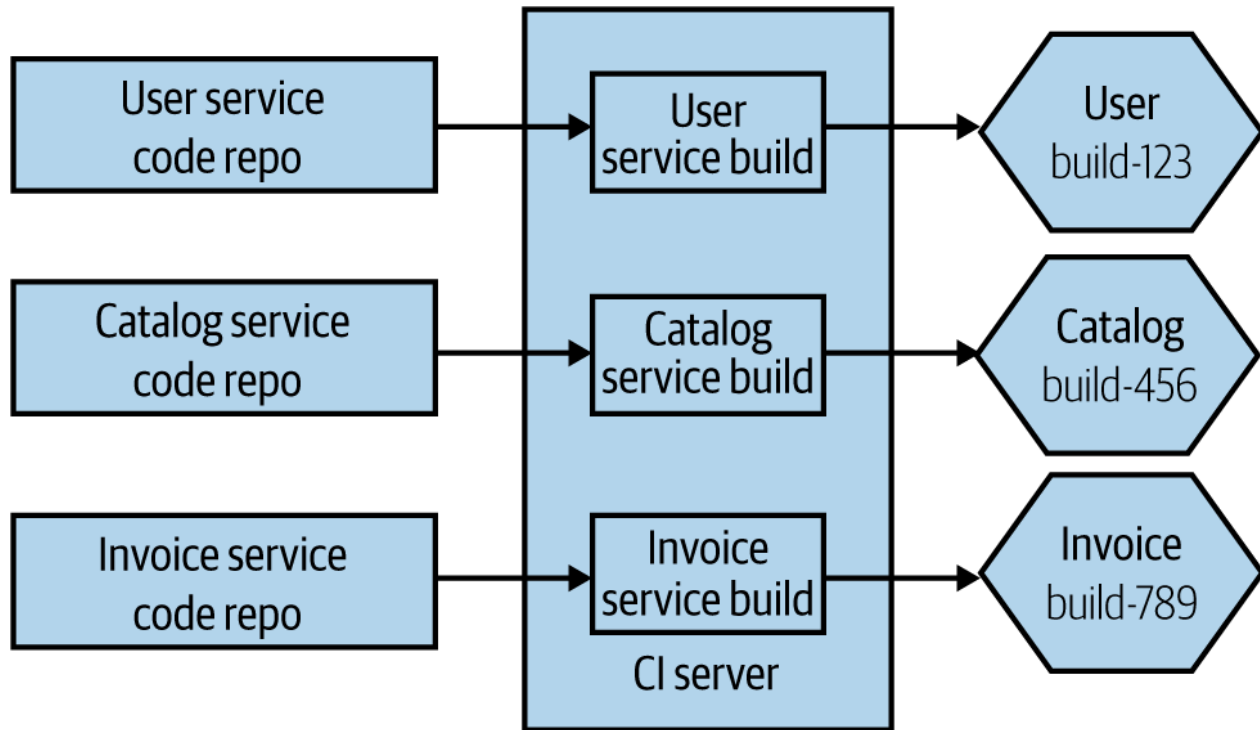


Figure 7-6. The source code for each microservice is stored in a separate source code repository

Hình 7-6. Mã nguồn cho mỗi microservice được lưu trong một kho mã nguồn riêng

Any change to the User source code repository triggers the matching build, and if that passes, I'll have a new version of my User microservice available for deployment. Having a separate repository for each microservice also allows you to change **ownership** on a per-repository basis, something that makes sense if you want to consider a strong ownership model for your microservices (more on that shortly).

Bất kỳ thay đổi nào với kho mã nguồn User đều kích hoạt bản build tương ứng, và nếu nó pass, tôi sẽ có một phiên bản mới của microservice User sẵn sàng để triển khai. Việc có một repository riêng cho mỗi microservice cũng cho phép bạn thay đổi quyền sở hữu trên cơ sở từng repository, điều này hợp lý nếu bạn muốn cân nhắc một mô hình quyền sở hữu mạnh (strong ownership) cho microservices của mình (sẽ nói thêm về điều này sớm thôi).

The straightforward nature of this pattern does create some challenges, however. Specifically, developers may find themselves working with multiple repositories at a time, which is especially painful if they are trying to make changes across multiple repositories at once. Additionally, changes cannot be made in an atomic fashion across separate repositories, at least not with Git.

Tuy nhiên, bản chất trực tiếp của mẫu này quả thực tạo ra một số thách thức. Cụ thể, lập trình viên có thể thấy mình đang làm việc với nhiều repository cùng lúc, điều này đặc biệt khổ sở nếu họ đang cố thực hiện các thay đổi trải khắp nhiều repository cùng một lúc. Thêm nữa, các thay đổi không thể được thực hiện theo kiểu nguyên tử (atomic) trải qua các repository riêng biệt, ít nhất là không với Git.

Reusing code across repositories

Tái sử dụng mã qua các repository

When using this pattern, there is nothing to stop a microservice from depending on other code that is managed in different repositories. A simple mechanism for doing this is to have the code you want to reuse packaged into a library that then becomes an explicit dependency of the downstream microservices. We can see an example of that in Figure 7-7, where the Invoice and Payroll services both make use of the Connection library.

Khi dùng mẫu này, không có gì ngăn cản một microservice phụ thuộc vào mã khác được quản lý trong các repository khác nhau. Một cơ chế đơn giản để làm điều này là đóng gói mã bạn muốn tái sử dụng thành một thư viện (library), thư viện đó rồi trở thành một phụ thuộc tường minh của các microservice phía dưới. Ta có thể thấy một ví dụ về điều đó ở Hình 7-7, nơi cả service Invoice và Payroll đều sử dụng thư viện Connection.

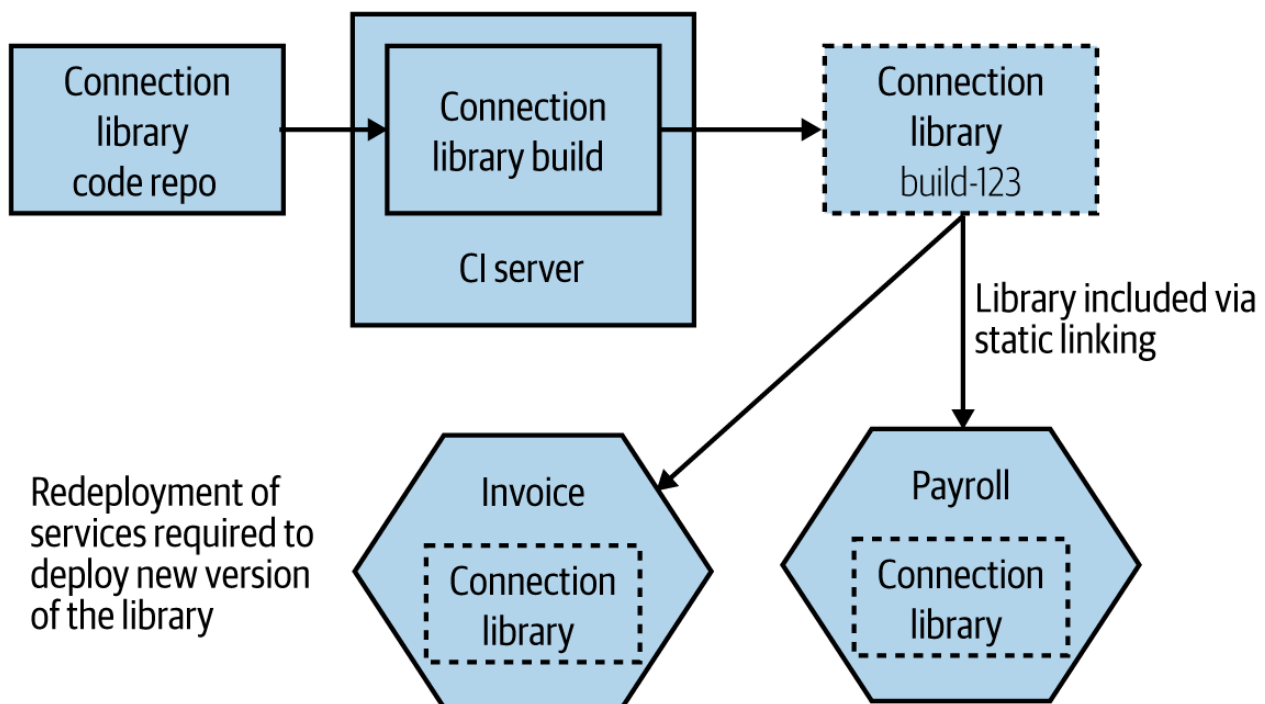


Figure 7-7. Reusing code across different repositories

Hình 7-7. Tái sử dụng mã qua các repository khác nhau

If you wanted to roll out a change to the Connection library, you'd have to make the changes in the matching source code repository and wait for its build to complete, giving you a new versioned artifact. To actually deploy new versions of the Invoice or Payroll services using this new version of the library, you'd need to change the version of the Connection library they use. This might require a manual change (if you are depending on a specific version), or it could be configured to happen dynamically, depending on the nature of the CI tooling you are using. The concepts behind this are outlined in more detail in the book *Continuous Delivery* by Jez Humble and Dave Farley. 4

Nếu bạn muốn tung ra một thay đổi cho thư viện Connection, bạn sẽ phải thực hiện các thay đổi trong kho mã nguồn tương ứng và chờ bản build của nó hoàn tất, mang lại cho bạn một artifact mới được gắn phiên bản. Để thực sự triển khai các phiên bản mới của service Invoice hay Payroll dùng phiên bản mới này của thư viện, bạn sẽ cần thay đổi phiên bản thư viện Connection mà chúng dùng. Điều này có thể đòi hỏi một thay đổi thủ

công (nếu bạn đang phụ thuộc vào một phiên bản cụ thể), hoặc có thể được cấu hình để xảy ra động, tùy vào bản chất của công cụ CI bạn đang dùng. Các khái niệm đằng sau điều này được trình bày chi tiết hơn trong cuốn sách Continuous Delivery của Jez Humble và Dave Farley. 4

The important thing to remember, of course, is that if you want to roll out the new version of the Connection library, then you also need to deploy the newly built Invoice and Payroll services. Remember, all the caveats we explored in “DRY and

Tất nhiên, điều quan trọng cần nhớ là nếu bạn muốn tung ra phiên bản mới của thư viện Connection, thì bạn cũng cần triển khai các service Invoice và Payroll vừa được build lại. Hãy nhớ, tất cả những điều cần lưu ý mà ta đã khám phá trong “DRY and

4 See “Managing Dependency Graphs” in Continuous Delivery , pp. 363–73.

4 Xem “Managing Dependency Graphs” trong Continuous Delivery , trang 363–73.

the Perils of Code Reuse in a Microservice World” on page 154 regarding reuse and microservices still apply—if you choose to reuse code via libraries, then you must be OK with the fact that these changes cannot be rolled out in an atomic fashion, or else we undermine our goal of **independent deployability**. You also have to be aware that it can be more challenging to know if some microservices are using a specific version of a library, which may be problematic if you’re trying to deprecate the use of an old version of the library.

the Perils of Code Reuse in a Microservice World” ở trang 154 liên quan đến việc tái sử dụng và microservices vẫn áp dụng—nếu bạn chọn tái sử dụng mã thông qua các thư viện, thì bạn phải chấp nhận được sự thật rằng những thay đổi này không thể được tung ra theo kiểu nguyên tử, nếu không ta sẽ làm suy yếu mục tiêu khả năng triển khai độc lập của mình. Bạn cũng phải nhận thức được rằng việc biết liệu một số microservice có đang dùng một phiên bản cụ thể của một thư viện hay không có thể khó khăn hơn, điều này có thể gây trở ngại nếu bạn đang cố loại bỏ dần (deprecate) việc dùng một phiên bản cũ của thư viện.

Working across multiple repositories

Làm việc trên nhiều repository

So, aside from reusing code via libraries, how else can we make a change across more than one repository? Let’s look at another example. In Figure 7-8 , I want to change the **API** exposed by the Inventory service, and I also need to update the Shipping service so it can make use of the new change. If the code for both Inventory and Shipping was in the same repository, I could commit the code once. Instead, I’ll have to break the changes into two commits—one for Inventory and another for Shipping .

Vậy, ngoài việc tái sử dụng mã thông qua các thư viện, ta còn có thể thực hiện một thay đổi trải qua nhiều hơn một repository bằng cách nào khác? Hãy xem một ví dụ khác. Trong Hình 7-8, tôi muốn thay đổi API mà service Inventory phơi bày, và tôi cũng cần cập nhật service Shipping để nó có thể tận dụng thay đổi mới. Nếu mã cho cả Inventory và Shipping nằm trong cùng một repository, tôi có thể commit mã một lần. Thay vào đó, tôi sẽ phải chia các thay đổi thành hai commit—một cho Inventory và một cho Shipping.

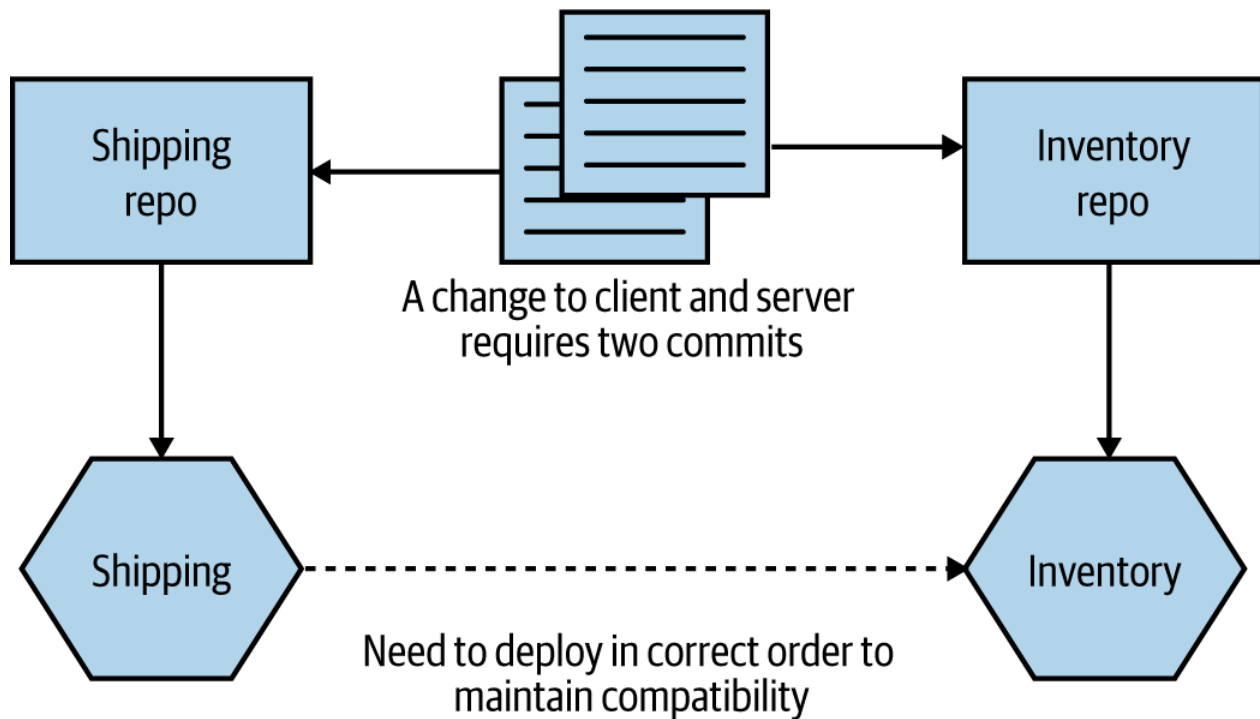


Figure 7-8. Changes across repository boundaries require multiple commits

Hình 7-8. Các thay đổi trải qua ranh giới repository đòi hỏi nhiều commit

Having these changes split could cause problems if one commit fails but the other works—I may need to make two changes to roll back the change, for example, and that could be complicated if other people have checked in in the meantime. The reality is that in this specific situation, I’d likely want to stage the commits somewhat, in any case. I’d want to make sure the commit to change the Inventory service worked before I change any client code in the Shipping service—if the new functionality in the API isn’t present, there is no point having client code that makes use of it.

Việc chia tách các thay đổi này có thể gây ra vấn đề nếu một commit thất bại nhưng commit kia thành công—chẳng hạn tôi có thể cần thực hiện hai thay đổi để rollback thay đổi đó, và điều đó có thể phức tạp nếu trong lúc ấy người khác đã check in. Thực tế là trong tình huống cụ thể này, dù sao tôi cũng sẽ muốn dàn xếp các commit theo từng bước. Tôi sẽ muốn đảm bảo rằng commit để thay đổi service Inventory đã hoạt động trước khi tôi thay đổi bất kỳ mã client nào trong service Shipping—nếu chức năng mới trong API chưa hiện diện, thì chẳng có ý nghĩa gì khi có mã client tận dụng nó.

I’ve spoken to multiple people who find the lack of atomic deployment with this to be a significant problem. I can certainly appreciate the complexity this brings, but I think that in most cases it points to a bigger underlying issue. If you are continually making changes across multiple microservices, then your service boundaries might not be in the right place, and it could imply too much **coupling** between your services. As we’ve already discussed, we’re trying to optimize our architecture, and our microservice boundaries, so that changes are more likely going to apply within a microservice boundary. Cross-cutting changes should be the exception, not the norm.

Tôi đã trao đổi với nhiều người thấy rằng việc thiếu triển khai nguyên tử ở đây là một vấn đề đáng kể. Tôi chắc chắn có thể thấu hiểu sự phức tạp mà điều này mang lại, nhưng tôi nghĩ rằng trong hầu hết các trường hợp, nó chỉ ra một vấn đề tiềm ẩn lớn hơn. Nếu bạn liên tục thực hiện các thay đổi trải khắp nhiều microservice, thì có thể ranh giới

service của bạn không nằm đúng chỗ, và điều đó có thể hàm ý quá nhiều coupling giữa các service của bạn. Như ta đã bàn, ta đang cố tối ưu hóa kiến trúc của mình, và các ranh giới microservice của mình, sao cho các thay đổi nhiều khả năng sẽ áp dụng bên trong một ranh giới microservice. Các thay đổi cắt ngang (cross-cutting) nên là ngoại lệ, chứ không phải chuẩn mực.

In fact, I'd argue that the pain of working across multiple repos can be useful in helping enforce microservice boundaries, as it forces you to think carefully about where these boundaries are, and about the nature of the interactions between them.

Thực ra, tôi cho rằng nỗi đau khi làm việc trải khắp nhiều repo có thể hữu ích trong việc giúp thực thi các ranh giới microservice, bởi nó buộc bạn phải suy nghĩ cẩn thận về vị trí của những ranh giới này, và về bản chất của các tương tác giữa chúng.



If you are constantly making changes across multiple microservices, it's likely that your microservice boundaries are in the wrong place. It may be worth considering merging microservices back together if you spot this happening.

Nếu bạn liên tục thực hiện các thay đổi trải khắp nhiều microservice, thì nhiều khả năng các ranh giới microservice của bạn nằm sai chỗ. Có thể đáng để cân nhắc gộp các

microservice trở lại với nhau nếu bạn phát hiện điều này đang xảy ra.

Then there is the hassle of having to pull from multiple repos and push to multiple repos as part of your normal **workflow**. In my experience, this can be simplified either by using an IDE that supports multiple repositories (this is something that all IDEs I've used over the last five years can handle) or by writing simple wrapper scripts to simplify things when working on the command line.

Rồi còn nỗi phiền toái khi phải pull từ nhiều repo và push tới nhiều repo như một phần của quy trình làm việc thông thường của bạn. Theo kinh nghiệm của tôi, điều này có thể được đơn giản hóa hoặc bằng cách dùng một IDE hỗ trợ nhiều repository (đây là thứ mà tất cả các IDE tôi đã dùng trong năm năm qua đều xử lý được) hoặc bằng cách viết các script bao bọc (wrapper script) đơn giản để đơn giản hóa mọi thứ khi làm việc trên dòng lệnh.

Where to use this pattern

Khi nào dùng mẫu này

Using the one repository per microservice approach works just as well for small teams as it does for large teams, but if you find yourself making lots of changes across microservice boundaries, then it may not be for you, and the monorepo pattern we discuss next may be a better fit—although making lots of changes across service boundaries can be considered a warning sign that something isn't right, as we've discussed previously. It can also make code reuse more complex than using a monorepo approach, as you need to depend on code being packaged into version artifacts.

Dùng cách tiếp cận mỗi microservice một repository hoạt động tốt cho các nhóm nhỏ cũng như cho các nhóm lớn, nhưng nếu bạn thấy mình thực hiện rất nhiều thay đổi trải qua các ranh giới microservice, thì nó có thể không dành cho bạn, và mẫu monorepo mà ta bàn tiếp theo có thể phù hợp hơn—mặc dù việc thực hiện nhiều thay đổi trải qua các ranh giới service có thể được coi là một dấu hiệu cảnh báo rằng có điều gì đó không ổn, như ta đã bàn trước đó. Nó cũng có thể khiến việc tái sử dụng mã phức tạp hơn so với dùng cách tiếp cận monorepo, vì bạn cần phụ thuộc vào việc mã được đóng gói thành các artifact có gắn phiên bản.

Pattern: Monorepo — Mẫu: Monorepo

With a monorepo approach, code for multiple microservices (or other types of projects) is stored in the same source code repository. I have seen situations in which a monorepo is used just by one team to manage source control for all its services, although the concept has been popularized by some very large tech companies where multiple teams and hundreds if not thousands of developers can all work on the same source code repository.

Với cách tiếp cận monorepo, mã cho nhiều microservice (hoặc các loại dự án khác) được lưu trong cùng một kho mã nguồn. Tôi đã thấy những tình huống trong đó một monorepo chỉ được một nhóm duy nhất dùng để quản lý kiểm soát mã nguồn cho tất cả các service của nó, mặc dù khái niệm này đã được phổ biến bởi một số công ty công nghệ rất lớn nơi nhiều nhóm và hàng trăm nếu không nói hàng nghìn lập trình viên đều có thể làm việc trên cùng một kho mã nguồn.

By having all the source code in the same repository, you allow for source code changes to be made across multiple projects in an atomic fashion, and for finer-grained reuse of code from one project to the next. Google is probably the best-known example of a company using a monorepo approach, although it's far from the only one. Although there are some other benefits to this approach, such as

improved visibility of other people’s code, the ability to reuse code easily and to make changes that impact multiple different projects is often cited as the major reason for adopting this pattern.

Bằng cách có tất cả mã nguồn trong cùng một repository, bạn cho phép các thay đổi mã nguồn được thực hiện trải qua nhiều dự án theo kiểu nguyên tử, và cho phép tái sử dụng mã ở mức tinh hơn từ dự án này sang dự án khác. Google có lẽ là ví dụ nổi tiếng nhất về một công ty dùng cách tiếp cận monorepo, mặc dù nó còn xa mới là công ty duy nhất. Mặc dù có một số lợi ích khác của cách tiếp cận này, chẳng hạn cải thiện khả năng nhìn thấy mã của người khác, nhưng khả năng tái sử dụng mã dễ dàng và thực hiện các thay đổi ảnh hưởng đến nhiều dự án khác nhau thường được nêu ra như lý do chính để áp dụng mẫu này.

If we take the example we just discussed, where we want to make a change to the Inventory so that it exposes some new behavior and also update the Shipping service to make use of this new functionality that we’ve exposed, then these changes can be made in a single commit, as we see in Figure 7-9.

Nếu ta lấy ví dụ ta vừa bàn, nơi ta muốn thực hiện một thay đổi cho Inventory để nó phơi bày một số hành vi mới và đồng thời cập nhật service Shipping để tận dụng chức năng mới mà ta đã phơi bày, thì những thay đổi này có thể được thực hiện trong một commit duy nhất, như ta thấy ở Hình 7-9.

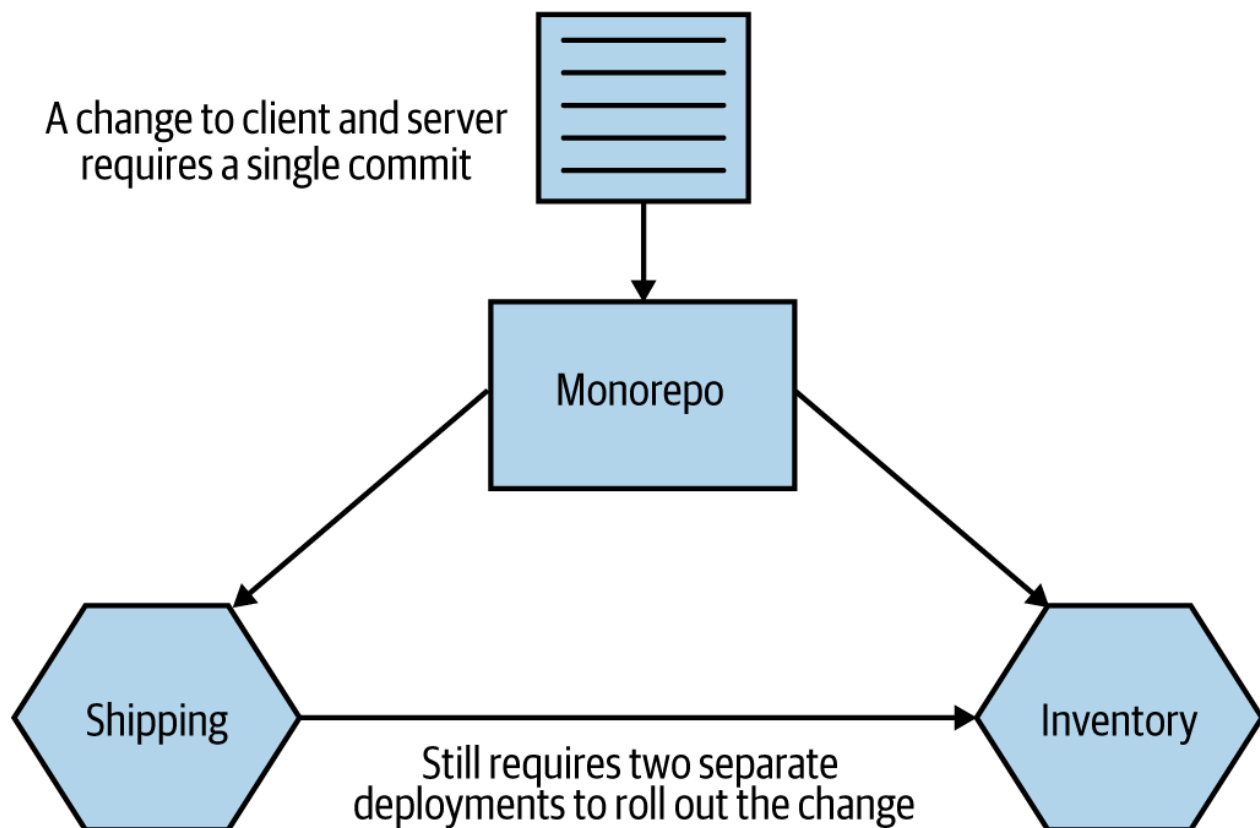


Figure 7-9. Using a single commit to make changes across two microservices using a monorepo

Hình 7-9. Dùng một commit duy nhất để thực hiện các thay đổi trải qua hai microservice bằng cách dùng monorepo

Of course, as with the multirepo pattern discussed previously, we still need to deal with the deployment side of this. We’d likely need to carefully consider the order of deployment if we want

to avoid a lockstep deployment.

Tất nhiên, cũng như với mẫu multirepo đã bàn trước đó, ta vẫn cần xử lý mặt triển khai của việc này. Ta nhiều khả năng sẽ cần cân nhắc cẩn thận thứ tự triển khai nếu ta muốn tránh một lần triển khai đồng loạt (lockstep deployment).



Atomic Commits Versus Atomic Deploy

Commit nguyên tử so với Triển khai nguyên tử

Being able to make an atomic commit across multiple services doesn't give you atomic rollout. If you find yourself wanting to change code across multiple services at once and roll it out into production all at the same time, this violates the core principle of independent deployability. For more on this, see “DRY and the Perils of Code Reuse in a Microservice World” on page 154 .

Việc có thể thực hiện một commit nguyên tử trải qua nhiều service không cho bạn một lần tung ra nguyên tử. Nếu bạn thấy mình muốn thay đổi mã trải qua nhiều service cùng

lúc và tung tất cả ra production cùng một lúc, điều này vi phạm nguyên tắc cốt lõi của khả năng triển khai độc lập. Để biết thêm về điều này, xem “DRY and the Perils of Code Reuse in a Microservice World” ở trang 154.

Mapping to build

Ánh xạ với build

With a single source code repository per microservice, mapping from the source code to a build process is straightforward. Any change in that source code repository can trigger a matching CI build. With a monorepo, it gets a bit more complex.

Với một kho mã nguồn duy nhất cho mỗi microservice, việc ánh xạ từ mã nguồn đến một quy trình build là trực tiếp. Bất kỳ thay đổi nào trong kho mã nguồn đó đều có thể kích hoạt một bản build CI tương ứng. Với một monorepo, nó trở nên phức tạp hơn một chút.

A simple starting point is to map folders inside the monorepo to a build, as shown in Figure 7-10 . A change made to the user-service folder would trigger the User service build, for example. If you checked in code that changed files both in the user-service folder and the catalog-service folder, then both the User build and the Catalog build would get triggered.

Một điểm khởi đầu đơn giản là ánh xạ các thư mục bên trong monorepo với một bản build, như trong Hình 7-10. Một thay đổi được thực hiện với thư mục user-service sẽ kích hoạt bản build của service User, chẳng hạn vậy. Nếu bạn check in mã làm thay đổi các tệp trong cả thư mục user-service lẫn thư mục catalog-service, thì cả bản build User lẫn bản build Catalog đều sẽ được kích hoạt.

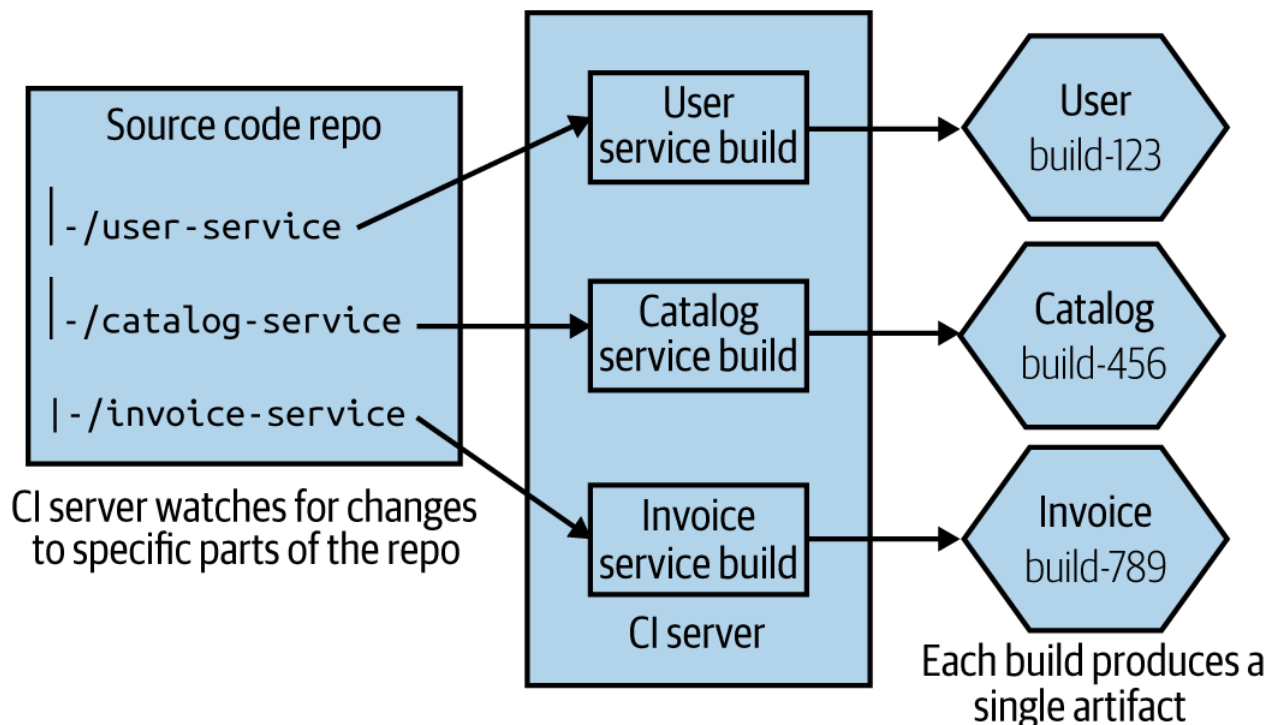


Figure 7-10. A single source repo with subdirectories mapped to independent builds

Hình 7-10. Một kho mã nguồn duy nhất với các thư mục con được ánh xạ tới các bản build độc lập

This gets more complex as you have more involved folder structures. On larger projects you can end

up with multiple different folders wanting to trigger the same build, and with some folders triggering more than one build. At the simple end of the spectrum, you might have a “common” folder used by all microservices, a change to which causes all microservices to be rebuilt. At the more complex end, teams end up needing to adopt more graph-based build tools like the open source Bazel tool to manage these dependencies more effectively (Bazel is an open source version of Google’s own internal build tool). Implementing a new build system can be a significant undertaking, so it is not something to be done lightly—but Google’s own monorepo would be impossible without tools like this.

Điều này trở nên phức tạp hơn khi bạn có các cấu trúc thư mục rắc rối hơn. Trên các dự án lớn hơn, bạn có thể rơi vào tình huống có nhiều thư mục khác nhau muốn kích hoạt cùng một bản build, và có một số thư mục kích hoạt nhiều hơn một bản build. Ở đầu đơn giản của phổ này, bạn có thể có một thư mục “common” được tất cả microservices dùng, mà một thay đổi với nó khiến tất cả microservices được build lại. Ở đầu phức tạp hơn, các nhóm rất cuộc cần phải áp dụng các công cụ build dựa trên đồ thị (graph-based) nhiều hơn như công cụ mã nguồn mở Bazel để quản lý các phụ thuộc này hiệu quả hơn (Bazel là một phiên bản mã nguồn mở của chính công cụ build nội bộ của Google). Việc triển khai một hệ thống build mới có thể là một việc lớn, nên đó không phải là điều nên làm một cách nhẹ dạ—nhưng chính monorepo của Google sẽ không thể tồn tại nếu thiếu các công cụ như thế này.

One of the benefits of a monorepo approach is that we can practice finer-grained reuse across projects. With a multirepo model, if I want to reuse someone else’s code, it will likely have to be packaged as a versioned artifact that I can then include as part of my build (such as a Nuget package, a JAR file, or an NPM). With our unit of reuse being a library, we are potentially pulling in more code than we really want. Theoretically, with a monorepo I could just depend on a single source file from another project—although this of course will cause me to have a more complex build mapping.

Một trong những lợi ích của cách tiếp cận monorepo là ta có thể thực hành tái sử dụng ở mức tinh hơn trải qua các dự án. Với một mô hình multirepo, nếu tôi muốn tái sử dụng mã của người khác, nó nhiều khả năng sẽ phải được đóng gói thành một artifact có gắn phiên bản mà tôi rồi có thể đưa vào như một phần của bản build của mình (chẳng hạn một gói Nuget, một tệp JAR, hay một NPM). Với đơn vị tái sử dụng của ta là một thư viện, ta có khả năng kéo vào nhiều mã hơn ta thực sự muốn. Về lý thuyết, với một monorepo tôi có thể chỉ phụ thuộc vào một tệp nguồn duy nhất từ một dự án khác—mặc dù điều này tất nhiên sẽ khiến tôi có một ánh xạ build phức tạp hơn.

Defining ownership

Định nghĩa quyền sở hữu

With smaller team sizes and small codebase sizes, monorepos can likely work well with the traditional build and source code management tools that you are used to. However, as your monorepo gets bigger, you’ll likely need to start looking at different types of tools. We’ll explore ownership models in more detail in Chapter 15, but in the meantime it’s worth exploring briefly how this plays out when we think about source control.

Với quy mô nhóm nhỏ hơn và quy mô mã nền nhỏ, monorepo nhiều khả năng có thể hoạt động tốt với các công cụ quản lý build và mã nguồn truyền thống mà bạn đã quen. Tuy nhiên, khi monorepo của bạn lớn hơn, bạn nhiều khả năng sẽ cần bắt đầu xem xét các loại công cụ khác. Chúng ta sẽ khám phá các mô hình quyền sở hữu chi tiết hơn ở Chương 15, nhưng trong lúc đó cũng đáng để khám phá ngắn gọn xem điều này diễn ra như thế nào khi ta nghĩ về kiểm soát mã nguồn.

Martin Fowler has previously written about different ownership models, outlining a sliding scale of ownership from strong ownership through weak ownership and on to collective ownership . Since Martin captured those terms, development practices have changed, so it's perhaps worth revisiting and redefining these terms.

Martin Fowler trước đây đã viết về các mô hình quyền sở hữu khác nhau, phác họa một thang đo trượt của quyền sở hữu từ quyền sở hữu mạnh (strong ownership) qua quyền sở hữu yếu (weak ownership) và đến quyền sở hữu tập thể (collective ownership). Kể từ khi Martin nắm bắt những thuật ngữ đó, các thực hành phát triển đã thay đổi, nên có lẽ đáng để xem lại và định nghĩa lại những thuật ngữ này.

With strong ownership, some code is owned by a specific group of people. If someone from outside that group wants to make a change, they have to ask the owners to make the change for them. Weak ownership still has the concept of defined owners, but people outside the ownership group are allowed to make changes, although any of these changes must be reviewed and accepted by someone in the ownership group. This would cover a pull request being sent to the core ownership team for review, before the pull request is merged. With collective ownership, any developer can change any piece of code.

Với strong ownership, một số mã được sở hữu bởi một nhóm người cụ thể. Nếu ai đó bên ngoài nhóm đó muốn thực hiện một thay đổi, họ phải nhờ những người sở hữu thực hiện thay đổi giúp họ. Weak ownership vẫn có khái niệm về những người sở hữu được định nghĩa rõ, nhưng những người bên ngoài nhóm sở hữu được phép thực hiện các thay đổi, mặc dù bất kỳ thay đổi nào trong số này đều phải được ai đó trong nhóm sở hữu review và chấp nhận. Điều này sẽ bao gồm một pull request được gửi đến nhóm sở hữu cốt lõi để review, trước khi pull request được merge. Với collective ownership, bất kỳ lập trình viên nào cũng có thể thay đổi bất kỳ đoạn mã nào.

With a small number of developers (20 or fewer, as a general guide), you can afford to practice collective ownership—where any developer can change any other microservice. As you have more people, though, you're more likely to want to move toward either a strong or weak ownership model to create more defined boundaries of responsibility. This can cause a challenge for teams using monorepos if their source control tool doesn't support finer-grained ownership controls.

Với một số lượng nhỏ lập trình viên (20 người hoặc ít hơn, như một hướng dẫn chung), bạn có thể đủ điều kiện thực hành collective ownership—nơi bất kỳ lập trình viên nào cũng có thể thay đổi bất kỳ microservice nào khác. Tuy nhiên, khi bạn có nhiều người hơn, bạn nhiều khả năng sẽ muốn dịch chuyển về phía một mô hình strong hoặc weak ownership để tạo ra các ranh giới trách nhiệm rõ ràng hơn. Điều này có thể gây ra một thách thức cho các nhóm dùng monorepo nếu công cụ kiểm soát mã nguồn của họ không hỗ trợ các kiểm soát quyền sở hữu ở mức tinh hơn.

Some source code tools allow you to specify ownership of specific directories or even specific filepaths inside a single repository. Google initially implemented this system on top of Perforce for its own monorepo before developing its own source control system, and it's also something that GitHub has supported since 2016. With GitHub, you create a CODEOWNERS file, which lets you map owners to directories or filepaths. You can see some examples in Example 7-1 , drawn from GitHub's own documentation, that show the kinds of flexibility these systems can bring.

Một số công cụ mã nguồn cho phép bạn chỉ định quyền sở hữu của các thư mục cụ thể hoặc thậm chí các đường dẫn tệp cụ thể bên trong một repository duy nhất. Google ban đầu triển khai hệ thống này trên nền Perforce cho monorepo của riêng mình trước khi phát triển hệ thống kiểm soát mã nguồn của riêng họ, và đó cũng là thứ mà GitHub đã hỗ trợ kể từ năm 2016. Với GitHub, bạn tạo một tệp CODEOWNERS, cho phép bạn ánh xạ

những người sở hữu với các thư mục hoặc đường dẫn tệp. Bạn có thể thấy một số ví dụ trong Ví dụ 7-1, được trích từ chính tài liệu của GitHub, cho thấy những kiểu linh hoạt mà các hệ thống này có thể mang lại.

Example 7-1. Examples of how to specify ownership in specific directories in a GitHub CODEOWNERS file

Ví dụ 7-1. Các ví dụ về cách chỉ định quyền sở hữu trong các thư mục cụ thể trong một tệp CODEOWNERS của GitHub

In this example, @doctocat owns any files in the build/logs # directory at the root of the repository and any of its # subdirectories. /build/logs/ @doctocat —

In this example, @octocat owns any file in an apps directory # anywhere in your repository. apps/ @octocat —

In this example, @doctocat owns any file in the /docs # directory in the root of your repository. /docs/ @doctocat — In this example, @doctocat owns any file in the /docs # directory in the root of your repository. /docs/ @doctocat

GitHub's own code ownership concept ensures that code owners for source files are requested for review whenever a pull request is raised for the relevant files. This could be a problem with larger pull requests, as you could end up needing sign-off from multiple reviewers, but there are lots of good reasons to aim for smaller pull requests, in any case.

Khái niệm quyền sở hữu mã của riêng GitHub đảm bảo rằng những người sở hữu mã đối với các tệp nguồn được yêu cầu review mỗi khi một pull request được tạo cho các tệp liên quan. Điều này có thể là một vấn đề với các pull request lớn hơn, vì bạn có thể rất cuộc cần sự phê duyệt từ nhiều người review, nhưng dù sao cũng có rất nhiều lý do chính đáng để hướng tới các pull request nhỏ hơn.

Tooling

Công cụ

Google's own monorepo is massive, and it takes significant amounts of engineering to make it work at scale. Consider things like a graph-based build system that has gone through multiple generations, a distributed object linker to speed up build times, plug-ins for IDEs and text editors that can dynamically keep dependency files in check—it's an enormous amount of work. As Google grew, it increasingly hit limitations on its use of Perforce and ended up having to create its own proprietary source control tool called Piper. When I worked in this part of Google back in 2007–2008, there were over a hundred people maintaining various developer tools, with a significant part of this effort given over to dealing with implications of the monorepo approach. That's something that you can justify if you have tens of thousands of engineers, of course.

Monorepo của riêng Google là đồ sộ, và phải mất một lượng lớn công sức kỹ thuật để khiến

nó hoạt động ở quy mô lớn. Hãy xét những thứ như một hệ thống build dựa trên đồ thị đã trải qua nhiều thế hệ, một bộ liên kết đối tượng phân tán (distributed object linker) để tăng tốc thời gian build, các plug-in cho IDE và trình soạn thảo văn bản có thể tự động giữ cho các tệp phụ thuộc luôn được kiểm soát—đó là một khối lượng công việc khổng lồ. Khi Google lớn lên, nó ngày càng vấp phải những giới hạn về việc dùng Perforce và rốt cuộc đã phải tạo ra công cụ kiểm soát mã nguồn độc quyền của riêng mình tên là Piper. Khi tôi làm việc ở bộ phận này của Google hồi 2007–2008, có hơn một trăm người duy trì các công cụ phát triển khác nhau, với một phần đáng kể của nỗ lực này dành cho việc xử lý các hệ quả của cách tiếp cận monorepo. Đó là điều bạn có thể biện minh được nếu bạn có hàng chục nghìn kỹ sư, tất nhiên.

For a more detailed overview of the rationale behind Google’s use of a monorepo, I recommend “Why Google Stores Billions of Lines of Code in a Single Repository” by Rachel Potvin and Josh Levenberg.⁵ In fact, I’d suggest it is required reading for anyone thinking, “We should use a monorepo, because Google does!” Your organization probably isn’t Google and probably doesn’t have Google-type problems, constraints,

Để có một cái nhìn tổng quan chi tiết hơn về cơ sở lý luận đằng sau việc Google dùng monorepo, tôi khuyên đọc “Why Google Stores Billions of Lines of Code in a Single Repository” của Rachel Potvin và Josh Levenberg.⁵ Thực ra, tôi sẽ đề xuất nó là tài liệu bắt buộc phải đọc cho bất kỳ ai đang nghĩ, “Chúng ta nên dùng monorepo, vì Google làm thế!” Tổ chức của bạn có lẽ không phải là Google và có lẽ không có các vấn đề, ràng buộc,

⁵ Rachel Potvin and Josh Levenberg, “Why Google Stores Billions of Lines of Code in a Single Repository,” Communications of the ACM 59, no. 7 (July 2016): 78–87.

⁵ Rachel Potvin and Josh Levenberg, “Why Google Stores Billions of Lines of Code in a Single Repository,” Communications of the ACM 59, no. 7 (July 2016): 78–87.

or resources. Put another way, whatever monorepo you end up with probably won’t be Google’s.

hay nguồn lực kiểu Google. Nói cách khác, dù bạn rốt cuộc có một monorepo như thế nào đi nữa thì nó có lẽ cũng sẽ không phải là monorepo của Google.

Microsoft experienced similar issues with scale. It adopted Git to help manage the main source code repository for Windows. A full working directory for this codebase is around 270 GB of source files.⁶ Downloading all of that would take an age, and it’s also not necessary—developers will end up working on just one small part of the overall system. So Microsoft had to create a dedicated virtual file system, VFS for Git (previously known as GVFS), that ensures only the source files that a developer needs are actually downloaded.

Microsoft đã trải qua những vấn đề tương tự với quy mô. Họ đã áp dụng Git để giúp quản lý kho mã nguồn chính cho Windows. Một thư mục làm việc đầy đủ cho mã nền này vào khoảng 270 GB tệp nguồn.⁶ Tải toàn bộ chỗ đó về sẽ mất một thời gian dài, và nó cũng không cần thiết—lập trình viên rốt cuộc sẽ chỉ làm việc trên một phần nhỏ của toàn bộ hệ thống. Vậy nên Microsoft đã phải tạo ra một hệ thống tệp ảo (virtual file system) chuyên dụng, VFS for Git (trước đây gọi là GVFS), đảm bảo rằng chỉ những tệp nguồn mà một lập trình viên cần mới thực sự được tải về.

VFS for Git is an impressive achievement, as is Google’s own toolchain, although justifying these kinds of investments in this sort of technology is much easier for companies like this. It’s also worth pointing out that although VFS for Git is open source, I’ve yet to meet a team outside Microsoft using it—and the vast bulk of Google’s own toolchain supporting its monorepo is closed source (Bazel is a notable exception, but it’s unclear to what extent the open source Bazel actually mirrors what is used inside Google itself).

VFS for Git là một thành tựu ấn tượng, cũng như chuỗi công cụ của riêng Google, mặc dù việc biện minh cho những kiểu đầu tư vào loại công nghệ này dễ dàng hơn nhiều đối với các công ty như thế. Cũng đáng để chỉ ra rằng mặc dù VFS for Git là mã nguồn mở, tôi vẫn chưa gặp một nhóm nào ngoài Microsoft dùng nó—và phần lớn áp đảo chuỗi công cụ của riêng Google hỗ trợ monorepo của họ là mã nguồn đóng (Bazel là một ngoại lệ đáng chú ý, nhưng không rõ Bazel mã nguồn mở thực sự phản chiếu đến mức nào những gì được dùng bên trong chính Google).

Markus Oberlehner’s piece “Monorepos in the Wild” introduced me to Lerna, a tool created by the team behind the Babel JavaScript compiler. Lerna is designed to make it easier to produce multiple versioned artifacts from the same source code repository. I can’t speak directly to how effective Lerna is at this task (in addition to a number of other notable deficiencies, I am not an experienced JavaScript developer), but it seems from a surface examination to simplify this approach somewhat.

Bài viết “Monorepos in the Wild” của Markus Oberlehner đã giới thiệu cho tôi về Lerna, một công cụ được tạo bởi nhóm đứng sau trình biên dịch JavaScript Babel. Lerna được thiết kế để giúp việc tạo ra nhiều artifact có gắn phiên bản từ cùng một kho mã nguồn dễ dàng hơn. Tôi không thể trực tiếp nói về việc Lerna hiệu quả đến đâu ở nhiệm vụ này (ngoài một số thiếu sót đáng chú ý khác, tôi không phải là một lập trình viên JavaScript có kinh nghiệm), nhưng nhìn qua thì có vẻ nó phần nào đơn giản hóa cách tiếp cận này.

How “mono” is mono?

“Mono” đến mức nào là mono?

Google doesn’t store all of its code in a monorepo. There are some projects, especially those being developed in the open, that are held elsewhere. Nonetheless, at least based on the previously mentioned ACM article, 95% of Google’s code was stored in the monorepo as of 2016. In other organizations, a monorepo may be scoped to only one system, or to a small number of systems. This means a company could have a small number of monorepos for different parts of the organization.

Google không lưu trữ toàn bộ mã của họ trong một monorepo. Có một số dự án, đặc biệt là những dự án đang được phát triển công khai, được giữ ở nơi khác. Tuy nhiên, ít nhất là dựa trên bài báo ACM đã đề cập trước đó, 95% mã của Google được lưu trong monorepo tính đến năm 2016. Trong các tổ chức khác, một monorepo có thể chỉ giới hạn ở một hệ thống, hoặc ở một số ít hệ thống. Điều này nghĩa là một công ty có thể có một số ít monorepo cho các phần khác nhau của tổ chức.

I’ve also spoken to teams that practice per-team monorepos. While technically speaking this probably doesn’t match up to the original definition of this pattern (which typically talks in terms of multiple teams sharing the same repository), I still think it’s more “monorepo” than anything else. In this situation, each team has its own monorepo that is fully under its control. All microservices owned by that team have their code stored in that team’s monorepo, as shown in Figure 7-11.

Tôi cũng đã trao đổi với các nhóm thực hành monorepo theo từng nhóm. Mặc dù về mặt kỹ thuật mà nói điều này có lẽ không khớp với định nghĩa gốc của mẫu này (vốn thường nói theo nghĩa nhiều nhóm cùng chia sẻ cùng một repository), tôi vẫn nghĩ nó “monorepo” hơn bất cứ thứ gì khác. Trong tình huống này, mỗi nhóm có monorepo riêng của mình hoàn toàn dưới sự kiểm soát của nó. Tất cả microservices được nhóm đó sở hữu đều có mã được lưu trong monorepo của nhóm đó, như trong Hình 7-11.

6 See Git Virtual File System Design History .

6 Xem Git Virtual File System Design History .

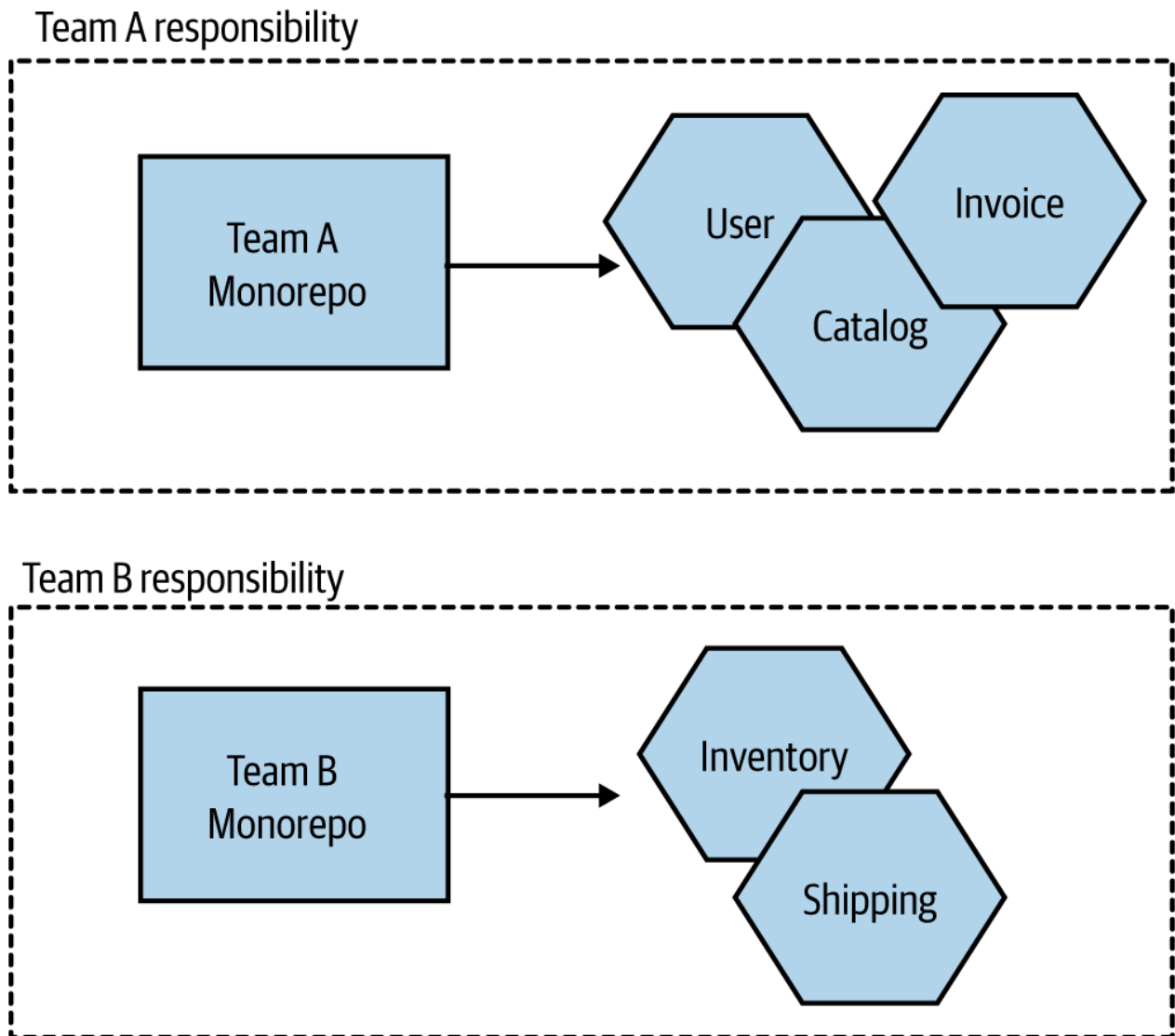


Figure 7-11. A pattern variation in which each team has its own monorepo

Hình 7-11. Một biến thể của mẫu trong đó mỗi nhóm có monorepo riêng của mình

For teams practicing collective ownership, this model has a lot of benefits, arguably providing most of the advantages of a monorepo approach while sidestepping some of the challenges that occur at larger scale. This halfway house can make a lot of sense in terms of working within existing organizational ownership boundaries, and it can somewhat mitigate the concerns about the use of this pattern at larger scale.

Đối với các nhóm thực hành collective ownership, mô hình này có rất nhiều lợi ích, có thể nói là mang lại hầu hết các ưu điểm của cách tiếp cận monorepo trong khi tránh được một số thách thức xảy ra ở quy mô lớn hơn. Giải pháp trung dung này có thể rất hợp lý xét về việc làm việc trong các ranh giới quyền sở hữu hiện có của tổ chức, và nó có thể phần nào giảm bớt những lo ngại về việc dùng mẫu này ở quy mô lớn hơn.

Where to use this pattern

Khi nào dùng mẫu này

Some organizations working at very large scale have found the monorepo approach to work very well for them. We’ve already mentioned Google and Microsoft, and we can add Facebook, Twitter, and Uber to the list. These organizations all have one thing in common—they are big, tech-focused companies that are able to dedicate significant resources to getting the best out of this pattern. Where I see monorepos work well is at the other end of the spectrum, with smaller numbers of developers and teams. With 10 to 20 developers, it is easier to manage ownership boundaries and keep the build process simple with the monorepo approach. Pain points seem to emerge for organizations in the middle—those with the scale to start hitting issues that require new tooling or ways of working, but without the spare bandwidth to invest in these ideas.

Một số tổ chức làm việc ở quy mô rất lớn đã nhận thấy cách tiếp cận monorepo hoạt động rất tốt với họ. Ta đã đề cập đến Google và Microsoft, và ta có thể thêm Facebook, Twitter, và Uber vào danh sách. Các tổ chức này đều có một điểm chung—chúng là những công ty lớn, tập trung vào công nghệ, có khả năng dành những nguồn lực đáng kể để khai thác tốt nhất mẫu này. Nơi tôi thấy monorepo hoạt động tốt là ở đầu kia của phổ, với số lượng lập trình viên và nhóm nhỏ hơn. Với 10 đến 20 lập trình viên, việc quản lý các ranh giới quyền sở hữu và giữ quy trình build đơn giản với cách tiếp cận monorepo dễ dàng hơn. Các điểm đau dường như nổi lên đối với các tổ chức ở khoảng giữa—những tổ chức có quy mô đủ lớn để bắt đầu vấp phải các vấn đề đòi hỏi công cụ hoặc cách làm việc mới, nhưng lại không có dư bằng thông để đầu tư vào những ý tưởng này.

Which Approach Would I Use? — Tôi sẽ dùng cách tiếp cận nào?

In my experience, the main advantages of a monorepo approach—finer-grained reuse and atomic commits—don’t seem to outweigh the challenges that emerge at scale. For smaller teams, either approach is fine, but as you scale, I feel that the one repository per microservice (multirepos) approach is more straightforward. Fundamentally, I’m concerned about the encouragement of cross-service changes, the more confused lines of ownership, and the need for new tooling that monorepos can bring.

Theo kinh nghiệm của tôi, những ưu điểm chính của cách tiếp cận monorepo—tái sử dụng ở mức tinh hơn và các commit nguyên tử—dường như không vượt trội hơn những thách thức nảy sinh ở quy mô lớn. Đối với các nhóm nhỏ hơn, cách nào cũng ổn, nhưng khi bạn mở rộng quy mô, tôi cảm thấy rằng cách tiếp cận mỗi microservice một repository (multirepo) trực tiếp hơn. Về cơ bản, tôi lo ngại về việc khuyến khích các thay đổi xuyên service, các lần ranh quyền sở hữu mơ hồ hơn, và nhu cầu về công cụ mới mà monorepo có thể mang lại.

A problem I’ve seen repeatedly is that organizations that started small, where collective ownership (and therefore monorepos) worked well initially, have struggled to move to different models later on, as the concept of the monorepo is so ingrained. As the delivery organization grows, the pain of the monorepo increases, but so too does the cost of migrating to an alternative approach. This is even more challenging for organizations that grew rapidly, as it’s often only after that rapid growth has occurred that the problems become evident, at which point the cost of migration to a multirepo approach looks too high. This can lead to the sunk cost fallacy: you’ve invested so much in making the monorepo work up to this point—just a bit more investment will make it work as well as it used to, right? Perhaps not—but it’s a brave soul who can recognize that they are throwing good money after bad and make a decision to change course.

Một vấn đề tôi đã thấy lặp đi lặp lại là các tổ chức bắt đầu từ quy mô nhỏ, nơi collective ownership (và do đó monorepo) ban đầu hoạt động tốt, đã chật vật chuyển sang các mô

hình khác về sau, vì khái niệm monorepo đã ăn sâu đến vậy. Khi tổ chức chuyển giao lớn lên, nỗi đau của monorepo tăng lên, nhưng chi phí di chuyển sang một cách tiếp cận thay thế cũng vậy. Điều này còn thách thức hơn đối với các tổ chức tăng trưởng nhanh, vì thường chỉ sau khi sự tăng trưởng nhanh đó đã xảy ra thì các vấn đề mới trở nên rõ ràng, và đến lúc đó chi phí di chuyển sang một cách tiếp cận multirepo trông quá cao. Điều này có thể dẫn đến nguy biện chi phí chìm (sunk cost fallacy): bạn đã đầu tư quá nhiều để khiến monorepo hoạt động đến tận thời điểm này rồi—chỉ cần đầu tư thêm một chút nữa sẽ khiến nó hoạt động tốt như trước đây, đúng không? Có lẽ không—nhưng phải là một tâm hồn dũng cảm mới có thể nhận ra rằng mình đang ném tiền tốt theo tiền xấu và đưa ra quyết định đổi hướng.

The concerns about ownership and monorepos can be alleviated through the use of fine-grained ownership controls, but that tends to require tooling and/or an increased level of diligence. My opinion on this might change as the maturity of tooling around monorepos improves, but despite a lot of work being done in regard to the open source development of graph-based build tools, I'm still seeing very low take-up of these toolchains. So it's multirepos for me.

Những lo ngại về quyền sở hữu và monorepo có thể được xoa dịu thông qua việc dùng các kiểm soát quyền sở hữu ở mức tinh, nhưng điều đó có xu hướng đòi hỏi công cụ và/hoặc một mức độ cần mẫn gia tăng. Ý kiến của tôi về điều này có thể thay đổi khi độ trưởng thành của công cụ quanh monorepo cải thiện, nhưng bất chấp rất nhiều công sức đã được bỏ ra cho việc phát triển mã nguồn mở các công cụ build dựa trên đồ thị, tôi vẫn thấy mức độ tiếp nhận các chuỗi công cụ này rất thấp. Vậy nên với tôi là multirepo.

Summary — Tóm tắt

We've covered some important ideas in this chapter that should stand you in good stead whether or not you end up using microservices. There are many more aspects to explore around these ideas, from continuous delivery to trunk-based development, monorepos to multirepos. I've given you a host of resources and further reading, but it's time for us to move on to a subject that is important to explore in some depth— deployment.

Chúng ta đã đề cập đến một số ý tưởng quan trọng trong chương này, những ý tưởng sẽ giúp ích cho bạn dù bạn có rốt cuộc dùng microservices hay không. Còn nhiều khía cạnh nữa để khám phá quanh những ý tưởng này, từ continuous delivery đến trunk-based development, từ monorepo đến multirepo. Tôi đã cho bạn vô số tài nguyên và tài liệu đọc thêm, nhưng đã đến lúc chúng ta chuyển sang một chủ đề quan trọng cần khám phá khá sâu—triển khai.

CHAPTER 8 Deployment — Chương 8: Triển khai

Deploying a single-process monolithic application is a fairly straightforward process. **Microservices**, with their interdependence and wealth of technology options, are a different kettle of fish altogether. When I wrote the first edition of this book, this chapter already had a lot to say about the huge variety of options available to you. Since then, **Kubernetes** has come to the fore, and **Function as a Service** (FaaS) platforms have given us even more ways to think about how to actually ship our software.

Triển khai (deployment) một ứng dụng monolith đơn tiến trình là một quá trình khá đơn giản. Microservices, với sự phụ thuộc lẫn nhau và vô số lựa chọn công nghệ, lại là một câu chuyện hoàn toàn khác. Khi tôi viết ấn bản đầu tiên của cuốn sách này, chương này đã có rất nhiều điều để nói về vô vàn lựa chọn sẵn có cho bạn. Kể từ đó, Kubernetes đã vươn lên hàng đầu, và các nền tảng Function as a Service (FaaS) đã cho chúng ta thêm nhiều cách nữa để nghĩ về việc thực sự đưa phần mềm của mình ra thị trường.

Although the technology may have changed over the last decade, I think many of the core principles associated with building software haven't changed. In fact, I think it's all the more important that we thoroughly understand these foundational ideas, as they can help us understand how to navigate this chaotic landscape of new technology. With that in mind, this chapter will highlight some core principles related to **deployment** that are important to understand, while also showing how the different tools available to you may help (or hinder) in regard to putting these principles into practice.

Mặc dù công nghệ có thể đã thay đổi trong suốt thập kỷ qua, tôi cho rằng nhiều nguyên tắc cốt lõi gắn với việc xây dựng phần mềm thì không hề thay đổi. Thực tế, tôi nghĩ việc thấu hiểu cận kề những ý tưởng nền tảng này càng trở nên quan trọng hơn, bởi chúng có thể giúp ta định hướng trong bối cảnh hỗn loạn của công nghệ mới. Với suy nghĩ đó, chương này sẽ làm nổi bật một số nguyên tắc cốt lõi liên quan đến triển khai mà bạn cần hiểu, đồng thời cho thấy các công cụ khác nhau hiện có có thể giúp (hoặc cản trở) bạn như thế nào trong việc đưa những nguyên tắc này vào thực tiễn.

To start off with, though, let's peek behind the curtain a bit and look at what happens as we move from a logical view of our systems **architecture** toward a real physical deployment topology.

Tuy vậy, để bắt đầu, hãy vén bức màn lên một chút và xem điều gì xảy ra khi ta chuyển từ góc nhìn logic về kiến trúc hệ thống sang một topology triển khai vật lý thực sự.

From Logical to Physical — Từ Logic đến Vật lý

To this point, when we've discussed microservices, we've spoken about them in a logical sense rather than in a physical sense. We could talk about how our Invoice **microservice** communicates with the

Order microservice, as shown in Figure 8-1 , without actually looking at the physical topology of how these services are deployed. A logical view of an architecture typically abstracts away underlying physical deployment concerns—that notion needs to change for the scope of this chapter.

Cho đến giờ, khi bàn về microservices, chúng ta đã nói về chúng theo nghĩa logic hơn là theo nghĩa vật lý. Ta có thể nói về việc microservice Invoice giao tiếp với microservice Order ra sao, như minh họa trong Hình 8-1, mà không thực sự xem xét topology vật lý về cách các service này được triển khai. Một góc nhìn logic về kiến trúc thường trừu tượng hóa, gạt bỏ những mối bận tâm về triển khai vật lý bên dưới—và quan niệm đó cần thay đổi trong phạm vi chương này.

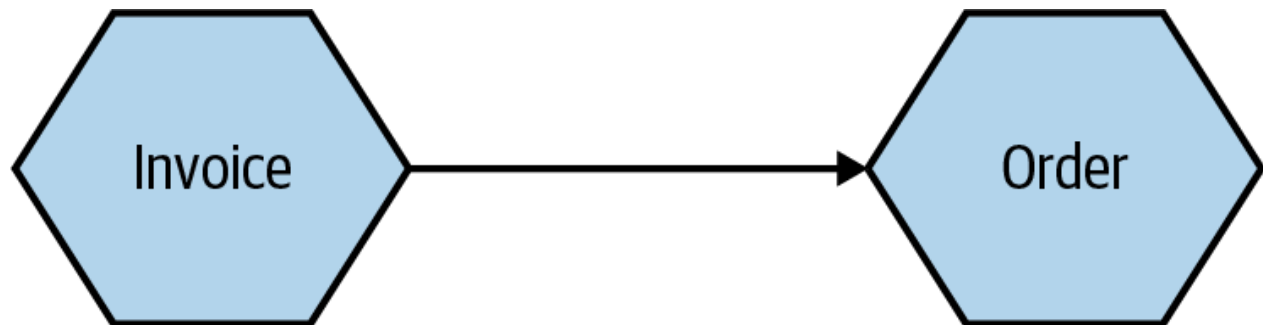


Figure 8-1. A simple, logical view of two microservices

Hình 8-1. Một góc nhìn logic đơn giản về hai microservice

This logical view of our microservices can hide a wealth of complexity when it comes to actually running them on real infrastructure. Let's take a look at what sorts of details might be hidden by a diagram like this.

Góc nhìn logic này về các microservice của ta có thể che giấu vô số sự phức tạp khi nói đến việc thực sự chạy chúng trên hạ tầng thật. Hãy cùng xem những loại chi tiết nào có thể bị ẩn đi bởi một sơ đồ như thế này.

Multiple Instances — Nhiều Instance

When we think about the deployment topology of the two microservices (in Figure 8-2), it's not as simple as one thing talking to another. To start with, it seems quite likely that we'll have more than one instance of each **service**. Having multiple instances of a service allows you to handle more load and can also improve the **robustness** of your system, as you can more easily tolerate the failure of a single instance. So we potentially have one or more instances of Invoice talking to one or more instances of Order . Exactly how the communication between these instances is handled will depend on the nature of the communication mechanism, but if we assume that in this situation we're using some form of HTTP-based **API**, a load balancer would be enough to handle routing of requests to different instances, as we see in Figure 8-2 .

Khi nghĩ về topology triển khai của hai microservice (trong Hình 8-2), mọi chuyện không đơn giản như một thứ nói chuyện với một thứ khác. Để bắt đầu, có vẻ rất khả dĩ là ta sẽ có nhiều hơn một instance của mỗi service. Việc có nhiều instance của một service cho phép bạn xử lý nhiều tải hơn và cũng có thể cải thiện độ vững chắc (robustness) của hệ thống, vì bạn dễ dàng chịu đựng được sự cố của một instance đơn lẻ hơn. Vậy nên ta có khả năng sẽ có một hoặc nhiều instance của Invoice trò chuyện với một hoặc nhiều instance của Order. Việc giao tiếp giữa các instance này được xử lý chính xác ra sao sẽ phụ thuộc vào bản chất của cơ chế giao tiếp, nhưng nếu giả định rằng trong tình huống này ta đang

dùng một dạng API dựa trên HTTP, thì một load balancer là đủ để xử lý việc định tuyến các request tới các instance khác nhau, như ta thấy trong Hình 8-2.

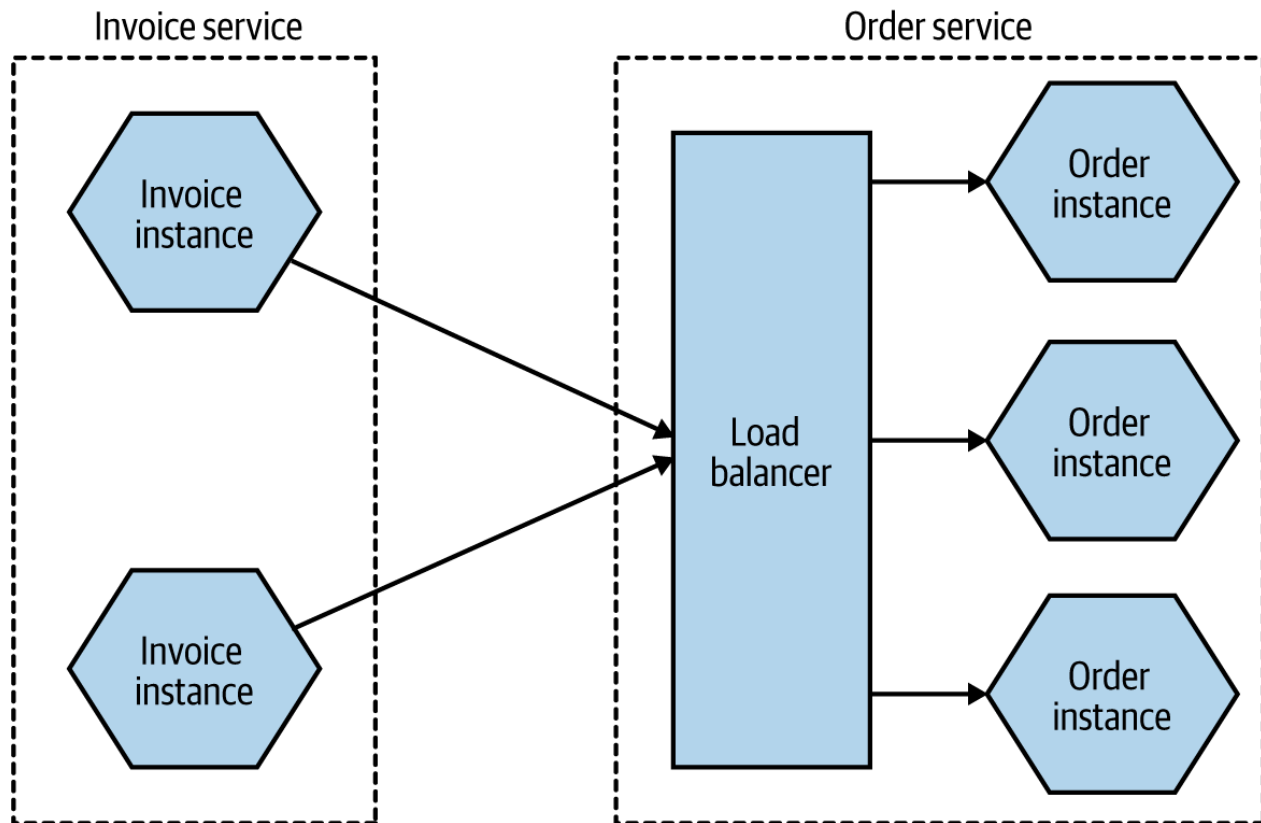


Figure 8-2. Using a load balancer to map requests to specific instances of the Order microservice The number of instances you'll want will depend on the nature of your application— you'll need to assess the required redundancy, expected load levels, and the like to come up with a workable number. You may also need to take into account where these instances will run. If you have multiple instances of a service for robustness reasons, you likely want to make sure that these instances aren't all on the same underlying hardware. Taken further, this might require that you have different instances distributed not only across multiple machines but also across different data centers, to give you protection against a whole data center being made unavailable. This might lead to a deployment topology like the one in Figure 8-3 .

Hình 8-2. Dùng load balancer để ánh xạ request tới các instance cụ thể của microservice Order. Số lượng instance bạn muốn sẽ phụ thuộc vào bản chất của ứng dụng—bạn cần đánh giá mức dự phòng (redundancy) cần thiết, mức tải dự kiến và những thứ tương tự để đưa ra một con số khả thi. Bạn cũng có thể cần tính đến việc các instance này sẽ chạy ở đâu. Nếu bạn có nhiều instance của một service vì lý do robustness, thì có lẽ bạn muốn đảm bảo rằng các instance này không nằm cả trên cùng một phần cứng bên dưới. Đi xa hơn, điều này có thể đòi hỏi bạn phân bố các instance khác nhau không chỉ trên nhiều máy mà còn trên nhiều trung tâm dữ liệu (data center) khác nhau, để bảo vệ bạn trước trường hợp cả một data center bị mất khả dụng. Điều này có thể dẫn đến một topology triển khai như trong Hình 8-3.

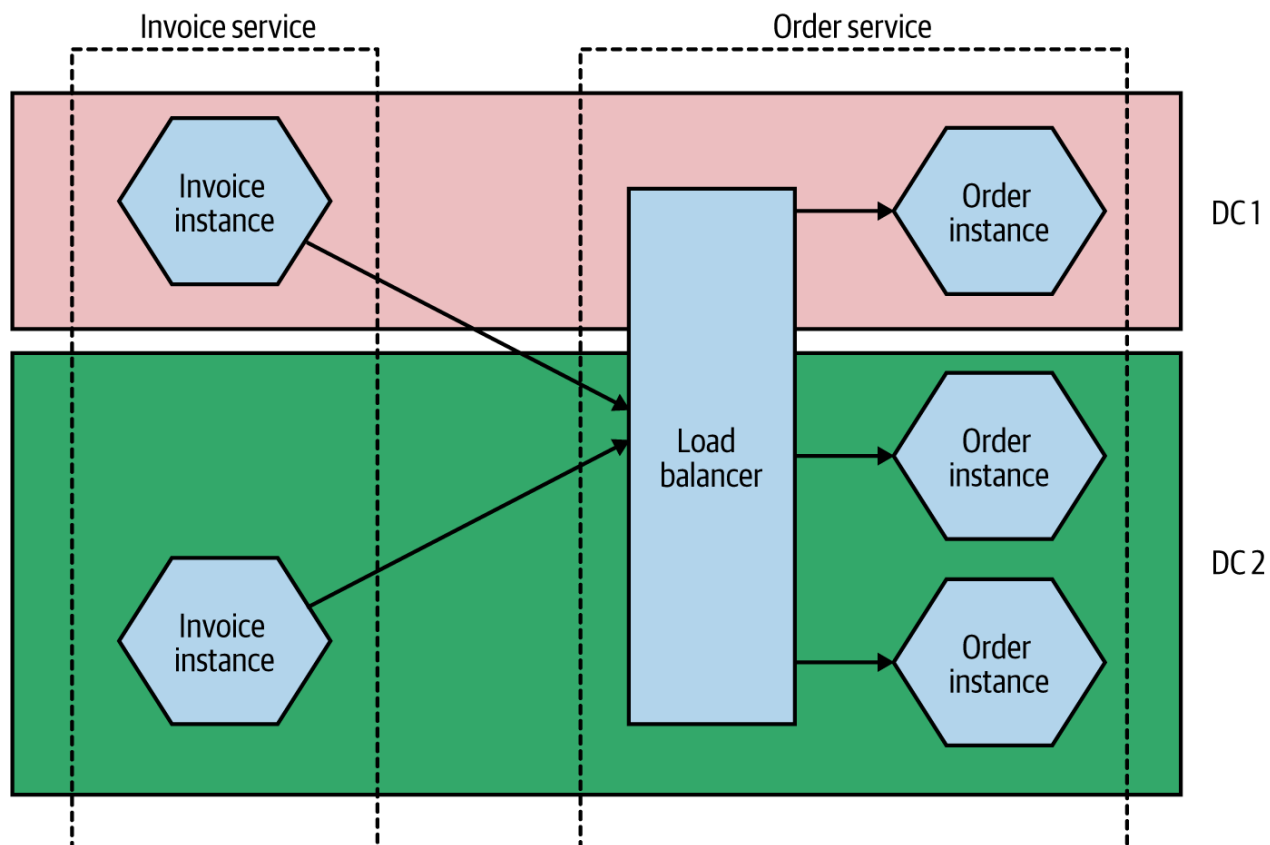


Figure 8-3. Distributing instances across multiple different data centers

Hình 8-3. Phân bố các instance trên nhiều data center khác nhau

This might seem overly cautious—what are the chances of an entire data center being unavailable? Well, I can’t answer that question for every situation, but at least when dealing with the main cloud providers, this is absolutely something you have to take account of. When it comes to something like a managed virtual machine, neither AWS nor Azure nor Google will give you an SLA for a single machine, nor do they give you an SLA for a single availability zone (which is the closest equivalent to a data center for these providers). In practice, this means that any solution you **deploy** should be distributed across multiple availability zones.

Điều này nghe có vẻ quá thận trọng—xác suất cả một data center bị mất khả dụng là bao nhiêu cơ chứ? Vâng, tôi không thể trả lời câu hỏi đó cho mọi tình huống, nhưng ít nhất khi làm việc với các nhà cung cấp đám mây lớn, đây tuyệt đối là điều bạn phải tính đến. Khi nói đến thứ như một máy ảo được quản lý (managed virtual machine), không AWS, Azure hay Google nào cho bạn một SLA cho một máy đơn lẻ, và họ cũng không cho bạn SLA cho một availability zone đơn lẻ (vốn là thứ gần tương đương nhất với một data center đối với các nhà cung cấp này). Trên thực tế, điều này có nghĩa là bất kỳ giải pháp nào bạn triển khai cũng nên được phân bố trên nhiều availability zone.

The Database — Cơ sở Dữ liệu

Taking this further, there is another major component that we’ve ignored up until this point—the **database**. As I’ve already discussed, we want a microservice to hide its internal state management, so any database used by a microservice for managing its state is considered to be hidden inside the

microservice. This leads to the oft-stated mantra of “don’t share databases,” the case for which I hope has already been made sufficiently by now.

Đi xa hơn nữa, có một thành phần lớn khác mà chúng ta đã bỏ qua cho đến giờ—cơ sở dữ liệu (database). Như tôi đã bàn, ta muốn một microservice che giấu việc quản lý trạng thái nội bộ của nó, nên bất kỳ database nào được một microservice dùng để quản lý trạng thái đều được coi là ẩn bên trong microservice đó. Điều này dẫn đến câu thần chú thường được nhắc đến là “đừng dùng chung database”, một luận điểm mà tôi hy vọng đến giờ đã được trình bày đầy đủ.

But how does this work when we consider the fact that I have multiple microservice instances? Should each microservice instance have its own database? In a word, no. In most cases, if I go to any instance of my Order service, I want to be able to get information about the same order. So we need some degree of shared state between different instances of the same logical service. This is shown in Figure 8-4 .

Nhưng điều này hoạt động ra sao khi ta xét đến thực tế là tôi có nhiều instance của microservice? Mỗi instance của microservice có nên có database riêng của nó không? Nói gọn lại là: không. Trong hầu hết trường hợp, nếu tôi đi tới bất kỳ instance nào của service Order, tôi đều muốn lấy được thông tin về cùng một đơn hàng. Vậy nên ta cần một mức độ trạng thái dùng chung nào đó giữa các instance khác nhau của cùng một service logic. Điều này được thể hiện trong Hình 8-4.

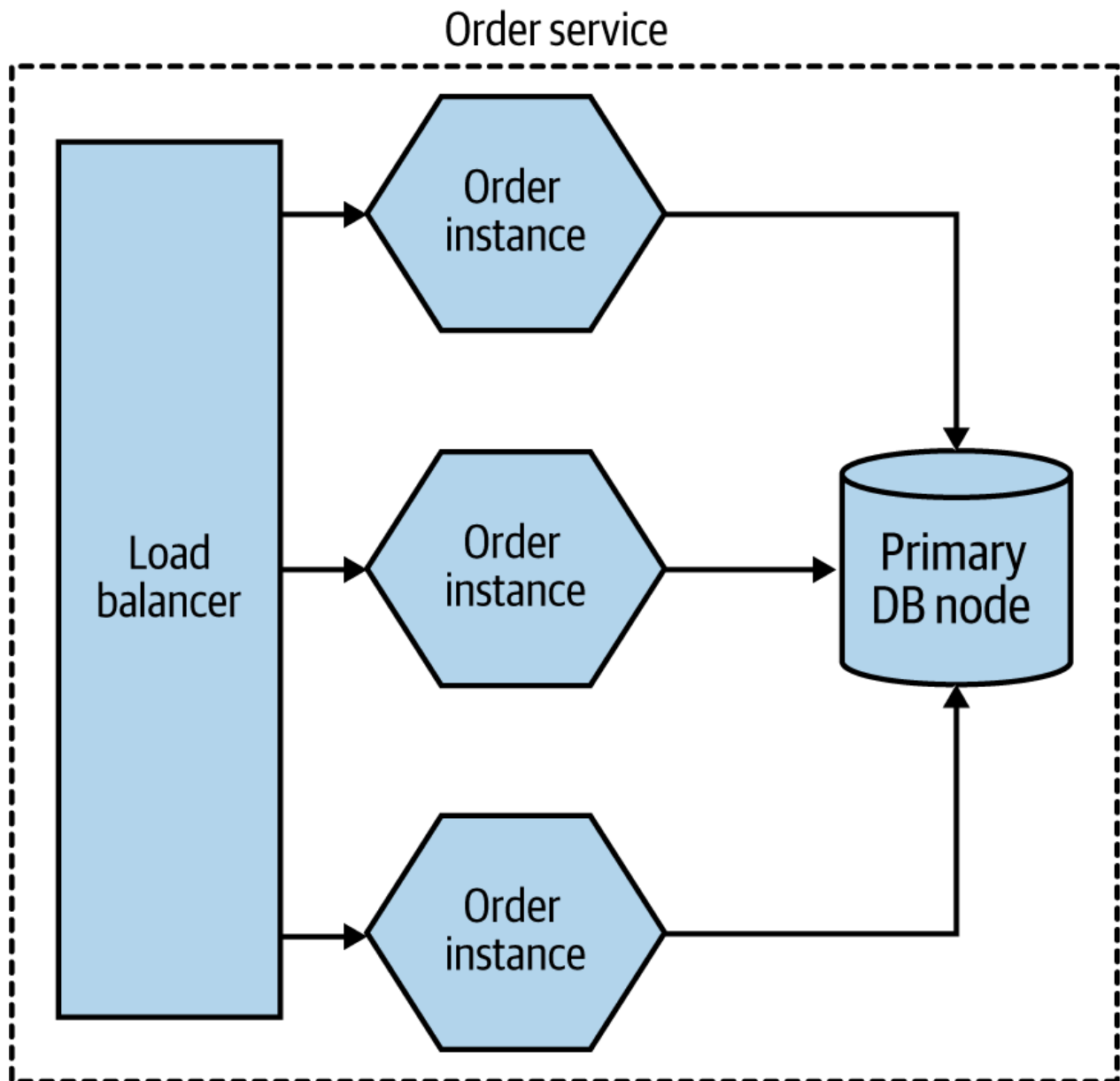


Figure 8-4. Multiple instances of the same microservice can share a database

Hình 8-4. Nhiều instance của cùng một microservice có thể dùng chung một database

But doesn't this violate our "don't share databases" rule? Not really. One of our major concerns is that when sharing a database across multiple different microservices, the logic associated with accessing and manipulating that state is now spread across different microservices. But here the data is being shared by different instances of the same microservice. The logic for accessing and manipulating state is still held within a single logical microservice.

Nhưng điều này chẳng phải vi phạm quy tắc “đừng dùng chung database” của ta sao? Không hẳn. Một trong những mối bận tâm chính của ta là khi dùng chung một database giữa nhiều microservice khác nhau, logic gắn với việc truy cập và thao tác trạng thái đó giờ đây bị trải rộng ra nhiều microservice khác nhau. Nhưng ở đây, dữ liệu đang được dùng chung bởi các instance khác nhau của cùng một microservice. Logic để truy cập và thao tác trạng thái vẫn nằm gọn trong một microservice logic duy nhất.

Database deployment and **scaling**

Triển khai và mở rộng database

As with our microservices, we've so far mostly talked about a database in a logical sense. In Figure 8-3 , we ignored any concerns about the redundancy or scaling needs of the underlying database.

Cũng như với các microservice, đến giờ ta chủ yếu mới nói về database theo nghĩa logic. Trong Hình 8-3, ta đã bỏ qua mọi mối bận tâm về nhu cầu dự phòng hay mở rộng (scaling) của database bên dưới.

Broadly speaking, a physical database deployment might be hosted on multiple machines, for a host of reasons. A common example would be to split load for reads and writes between a primary node and one or more nodes that are designated for read-only purposes (these nodes are typically referred to as read replicas). If we were implementing this idea for our Order service, we might end up with a situation like the one shown in Figure 8-5 .

Nói rộng ra, một triển khai database vật lý có thể được lưu trữ trên nhiều máy, vì vô số lý do. Một ví dụ phổ biến là tách tải đọc và ghi giữa một node chính (primary) và một hoặc nhiều node được chỉ định chỉ để đọc (các node này thường được gọi là read replica). Nếu ta hiện thực ý tưởng này cho service Order, có thể ta sẽ rơi vào tình huống như trong Hình 8-5.

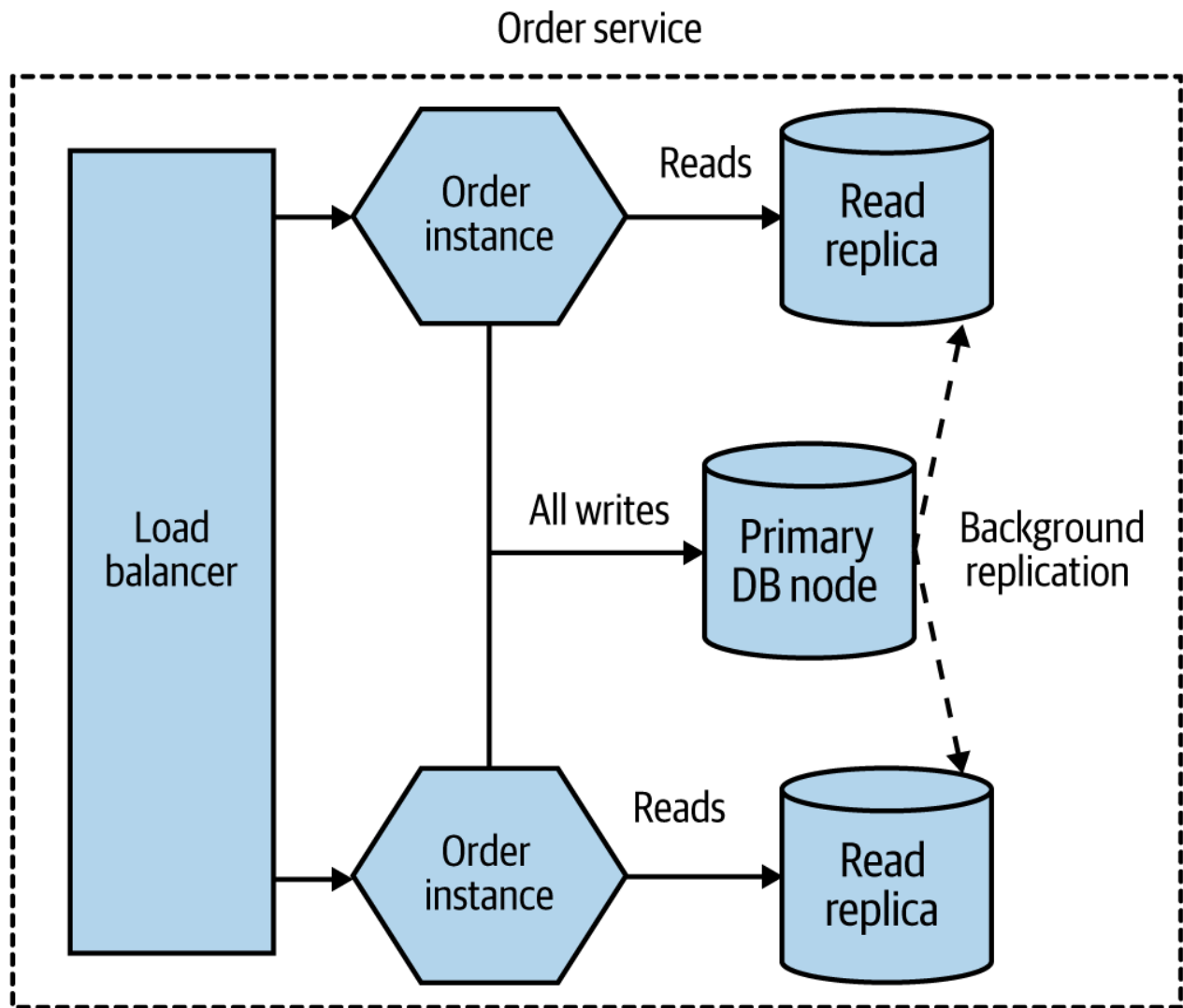


Figure 8-5. Using read replicas to distribute load

Hình 8-5. Dùng read replica để phân tán tải

All read-only traffic goes to one of the read replica nodes, and you can further scale read traffic by adding additional read nodes. Due to the way that relational databases work, it's more difficult to scale writes by adding additional machines (typically sharding models are required, which adds additional complexity), so moving read-only traffic to these read replicas can often free up more capacity on the write node to allow for more scaling.

Toàn bộ lưu lượng chỉ-đọc đi tới một trong các node read replica, và bạn có thể mở rộng lưu lượng đọc thêm nữa bằng cách bổ sung các node đọc. Do cách thức hoạt động của cơ sở dữ liệu quan hệ, việc mở rộng ghi bằng cách bổ sung máy là khó hơn (thường cần đến các mô hình sharding, vốn làm tăng thêm độ phức tạp), nên việc chuyển lưu lượng chỉ-đọc sang các read replica này thường có thể giải phóng thêm năng lực trên node ghi, cho phép mở rộng nhiều hơn.

Added to this complex picture is the fact that the same database infrastructure can support multiple logically isolated databases. So the databases for Invoice and Order might both be served by the same underlying database engine and hardware, as shown in Figure 8-6 . This can have significant

benefits—it allows you to pool hardware to serve multiple microservices, it can reduce licensing costs, and it can also help reduce the work around management of the database itself.

Thêm vào bức tranh phức tạp này là việc cùng một hạ tầng database có thể phục vụ nhiều database tách biệt về mặt logic. Vậy nên database cho Invoice và Order có thể đều được phục vụ bởi cùng một engine database và phần cứng bên dưới, như minh họa trong Hình 8-6. Điều này có thể mang lại lợi ích đáng kể—nó cho phép bạn gộp chung phần cứng để phục vụ nhiều microservice, có thể giảm chi phí bản quyền, và cũng giúp giảm khối lượng công việc quản lý bản thân database.

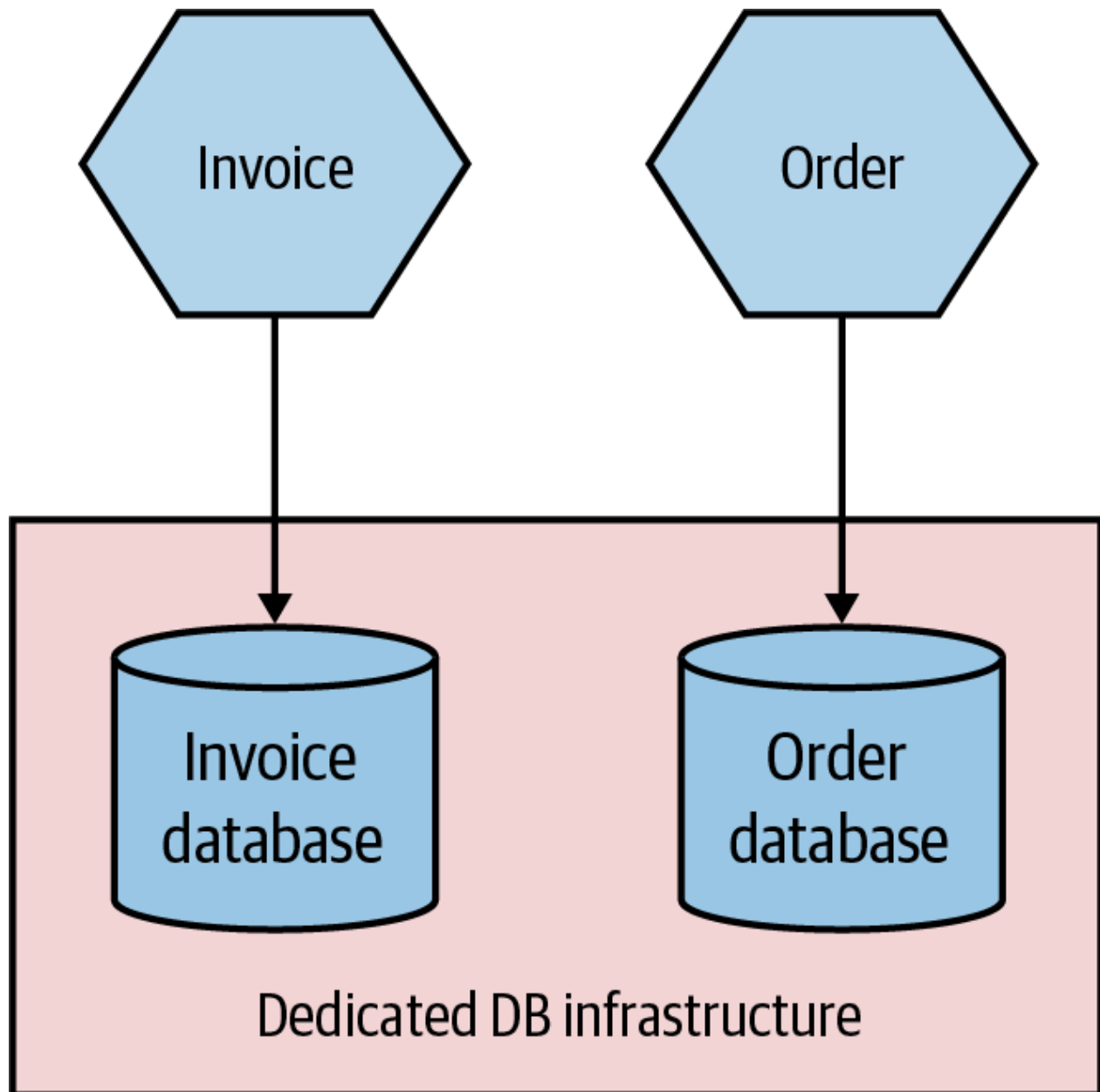


Figure 8-6. The same physical database infrastructure hosting two logically isolated databases

Hình 8-6. Cùng một hạ tầng database vật lý lưu trữ hai database tách biệt về mặt logic

The important thing to realize here is that although these two databases might be run from the same

hardware and database engine, they are still logically isolated databases. They cannot interfere with each other (unless you allow that). The one major thing to consider is that if this **shared database** infrastructure fails, you might impact multiple microservices, which could have catastrophic impact.

Điều quan trọng cần nhận ra ở đây là mặc dù hai database này có thể chạy từ cùng một phần cứng và engine database, chúng vẫn là các database tách biệt về mặt logic. Chúng không thể can thiệp lẫn nhau (trừ khi bạn cho phép điều đó). Điều quan trọng cần nhắc là nếu hạ tầng database dùng chung này gặp sự cố, bạn có thể ảnh hưởng đến nhiều microservice, điều có thể gây tác động thảm khốc.

In my experience, organizations that manage their own infrastructure and run in an “on-prem” fashion tend to be much more likely to have multiple different databases hosted from shared database infrastructure, for the cost reasons I outlined before. Provisioning and managing hardware is painful (and historically at least, databases are less likely to run on virtualized infrastructure), so you want less of that.

Theo kinh nghiệm của tôi, các tổ chức tự quản lý hạ tầng của mình và vận hành theo kiểu “on-prem” có xu hướng dễ có nhiều database khác nhau được lưu trữ từ một hạ tầng database dùng chung hơn, vì những lý do chi phí tôi đã nêu trước đó. Việc cung cấp và quản lý phần cứng là cực nhọc (và ít nhất là về mặt lịch sử, database ít có khả năng chạy trên hạ tầng ảo hóa hơn), nên bạn muốn ít việc đó hơn.

On the other hand, teams that run on public cloud providers are much more likely to provision dedicated database infrastructure on a per-microservice basis, as shown in Figure 8-7 . The costs of provisioning and managing this infrastructure are much lower. AWS’s Relational Database Service (RDS), for example, can automatically handle concerns like backups, upgrades, and multiavailability zone failover, and similar products are available from the other public cloud providers. This makes it much more cost effective to have more isolated infrastructure for your microservice, giving each microservice owner more control rather than having to rely on a shared service.

Mặt khác, các đội vận hành trên nhà cung cấp đám mây công cộng (public cloud) có nhiều khả năng cung cấp hạ tầng database riêng theo từng microservice hơn, như minh họa trong Hình 8-7. Chi phí cung cấp và quản lý hạ tầng này thấp hơn nhiều. Ví dụ, Relational Database Service (RDS) của AWS có thể tự động xử lý các mối bận tâm như sao lưu, nâng cấp và chuyển đổi dự phòng (failover) đa availability zone, và các sản phẩm tương tự cũng có sẵn từ những nhà cung cấp public cloud khác. Điều này làm cho việc có hạ tầng tách biệt hơn cho microservice của bạn trở nên hiệu quả về chi phí hơn nhiều, trao cho mỗi chủ sở hữu microservice nhiều quyền kiểm soát hơn thay vì phải dựa vào một service dùng chung.

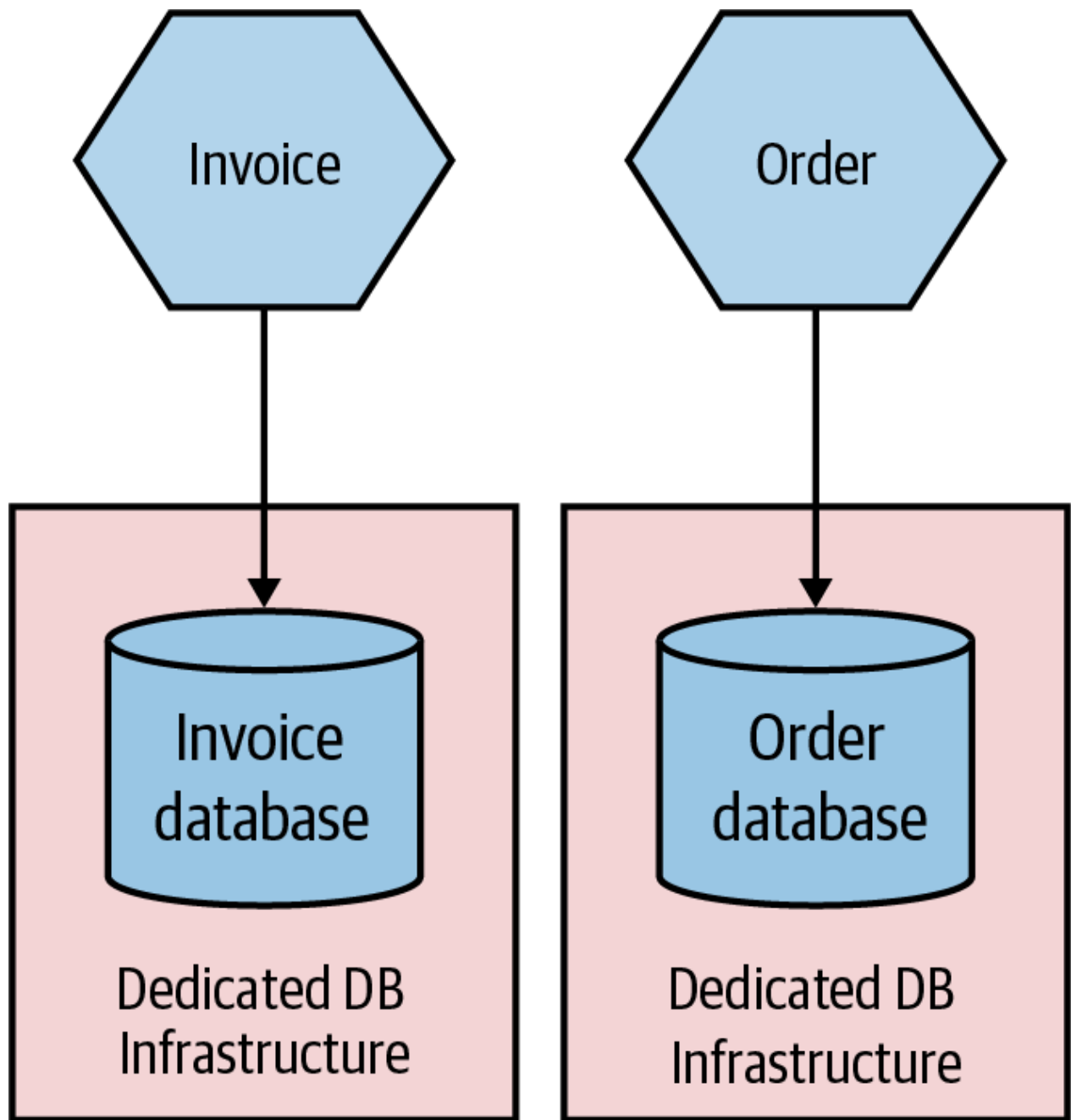


Figure 8-7. Each microservice making use of its own dedicated DB infrastructure

Hình 8-7. Mỗi microservice sử dụng hạ tầng DB riêng của mình

Environments — Môi trường

When you deploy your software, it runs in an environment. Each environment will typically serve different purposes, and the exact number of environments you might have will vary greatly based on how you develop software and how your software is deployed to your end user. Some environments will have production data, while others won't. Some environments may have all services in them; others might have just a small number of services, with any nonpresent services replaced with fake

ones for the purposes of **testing**.

Khi bạn triển khai phần mềm của mình, nó chạy trong một môi trường (environment). Mỗi môi trường thường phục vụ những mục đích khác nhau, và số lượng môi trường chính xác mà bạn có thể có sẽ thay đổi rất nhiều tùy theo cách bạn phát triển phần mềm và cách phần mềm của bạn được triển khai tới người dùng cuối. Một số môi trường sẽ có dữ liệu production, số khác thì không. Một số môi trường có thể có tất cả các service trong đó; số khác có thể chỉ có một số ít service, với mọi service không hiện diện được thay bằng các service giả nhằm mục đích kiểm thử.

Typically, we think of our software as moving through a number of preproduction environments, with each one serving some purpose to allow the software to be developed and its readiness for production to be tested—we explored this in “Trade-Offs and Environments” on page 203 . From a developer laptop to a **continuous integration** server, an integrated test environment, and beyond—the exact nature and number of your environments will depend on a host of factors but will be driven primarily by how you choose to develop software. In Figure 8-8 , we see a pipeline for MusicCorp’s Catalog microservice. The microservice moves through different environments before it finally gets into a production environment, where our users will get to use the new software.

Thông thường, ta hình dung phần mềm của mình di chuyển qua một loạt môi trường tiền-production (preproduction), mỗi môi trường phục vụ một mục đích nào đó để cho phép phần mềm được phát triển và để kiểm thử mức độ sẵn sàng cho production của nó—chúng ta đã khám phá điều này trong “Trade-Offs and Environments” ở trang 203. Từ laptop của lập trình viên đến một máy chủ tích hợp liên tục (continuous integration), một môi trường kiểm thử tích hợp, và xa hơn nữa—bản chất và số lượng môi trường chính xác của bạn sẽ phụ thuộc vào vô số yếu tố nhưng chủ yếu sẽ được dẫn dắt bởi cách bạn chọn để phát triển phần mềm. Trong Hình 8-8, ta thấy một pipeline cho microservice Catalog của MusicCorp. Microservice này di chuyển qua các môi trường khác nhau trước khi cuối cùng vào được môi trường production, nơi người dùng của ta sẽ được dùng phần mềm mới.

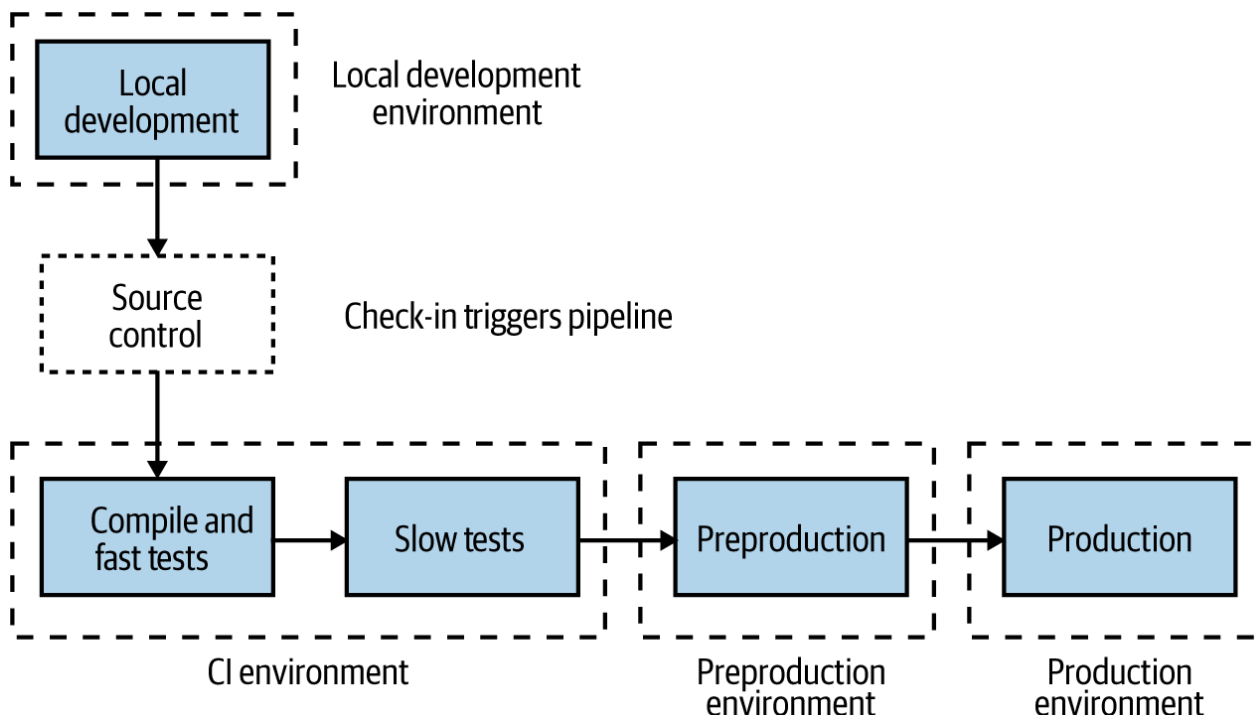


Figure 8-8. Different environments used for different parts of the pipeline

Hình 8-8. Các môi trường khác nhau được dùng cho các phần khác nhau của pipeline

The first environment our microservice runs in is wherever the developer was working on the code prior to check-in—probably their local laptop. After committing the code, the CI process kicks off with the fast tests. Both the fast and slow test stages deploy into our CI environment. If the slow tests pass, the microservice is deployed into the preproduction environment to allow for manual verification (which is entirely optional but still important for many). If this manual verification passes, the microservice is then deployed into production.

Môi trường đầu tiên mà microservice của ta chạy là bất cứ đâu mà lập trình viên làm việc trên mã trước khi check-in—có lẽ là laptop cá nhân của họ. Sau khi commit mã, quá trình CI khởi động với các bài kiểm thử nhanh. Cả giai đoạn kiểm thử nhanh lẫn chậm đều triển khai vào môi trường CI của ta. Nếu các bài kiểm thử chậm vượt qua, microservice được triển khai vào môi trường preproduction để cho phép kiểm chứng thủ công (việc này hoàn toàn tùy chọn nhưng vẫn quan trọng với nhiều người). Nếu việc kiểm chứng thủ công này vượt qua, microservice sau đó được triển khai vào production.

Ideally, each environment in this process would be an exact copy of the production environment. This would give us even more confidence that our software will work when it reaches production. However, in reality, we often can't afford to run multiple copies of our entire production environment due to how expensive this is.

Lý tưởng nhất, mỗi môi trường trong quá trình này sẽ là một bản sao y hệt của môi trường production. Điều này cho ta thêm sự tự tin rằng phần mềm của ta sẽ hoạt động khi đến được production. Tuy nhiên, trên thực tế, ta thường không đủ khả năng chạy nhiều bản sao của toàn bộ môi trường production vì việc đó quá tốn kém.

We also want to tune environments earlier in this process to allow for fast feedback. It's vital that we know as early as possible whether or not our software works so that we can fix things quickly, if needed. The earlier we know about a problem with our software, the faster it is to fix it, and the lower the impact of the break. It's much better to find a problem on our local laptop than pick it up in preproduction testing, but likewise picking up a problem in preproduction testing might be much better for us than picking something up in production (although we will explore some important trade-offs around this in Chapter 9).

Ta cũng muốn tinh chỉnh các môi trường ở giai đoạn sớm hơn trong quá trình này để cho phép phản hồi (feedback) nhanh. Việc biết càng sớm càng tốt liệu phần mềm của ta có hoạt động hay không là tối quan trọng, để ta có thể sửa chữa nhanh nếu cần. Càng biết sớm về một vấn đề với phần mềm của mình, việc sửa nó càng nhanh, và tác động của lỗi càng thấp. Tìm ra một vấn đề ngay trên laptop của ta sẽ tốt hơn nhiều so với việc phát hiện nó trong kiểm thử preproduction, nhưng tương tự, phát hiện một vấn đề trong kiểm thử preproduction có thể tốt cho ta hơn nhiều so với việc phát hiện nó trong production (mặc dù ta sẽ khám phá một số trade-off quan trọng quanh điều này trong Chương 9).

This means that environments closer to the developer will be tuned to provide fast feedback, which may compromise how “production-like” they are. But as environments get closer to production, we will want them to be more and more like the end production environment to ensure that we catch problems.

Điều này có nghĩa là các môi trường gần với lập trình viên hơn sẽ được tinh chỉnh để cung cấp phản hồi nhanh, điều có thể đánh đổi mức độ “giống production” của chúng. Nhưng khi các môi trường tiến gần đến production, ta sẽ muốn chúng ngày càng giống với môi trường production cuối cùng để đảm bảo ta bắt được các vấn đề.

As a simple example of this in action, let's revisit our earlier example of the Catalog service and take a

look at the different environments. In Figure 8-9, the local developer laptop has our service deployed as a single instance running locally. The software is fast to **build** but is deployed as a single instance running on very different hardware from what we expect in production. In the CI environment, we deploy two copies of our service to test against, making sure our load balancing logic is working OK. We deploy both instances to the same machine—this keeps costs down and makes things faster, and it still gives us enough feedback at this stage in the process.

Như một ví dụ đơn giản về điều này trong thực tế, hãy quay lại ví dụ trước đó về service Catalog và xem xét các môi trường khác nhau. Trong Hình 8-9, laptop của lập trình viên cục bộ có service của ta được triển khai dưới dạng một instance đơn lẻ chạy cục bộ. Phần mềm build nhanh nhưng được triển khai dưới dạng một instance đơn lẻ chạy trên phần cứng rất khác so với những gì ta kỳ vọng ở production. Trong môi trường CI, ta triển khai hai bản sao của service để kiểm thử, đảm bảo logic cân bằng tải của ta hoạt động ổn. Ta triển khai cả hai instance lên cùng một máy—điều này giữ chi phí thấp và làm mọi thứ nhanh hơn, mà vẫn cho ta đủ phản hồi ở giai đoạn này trong quá trình.

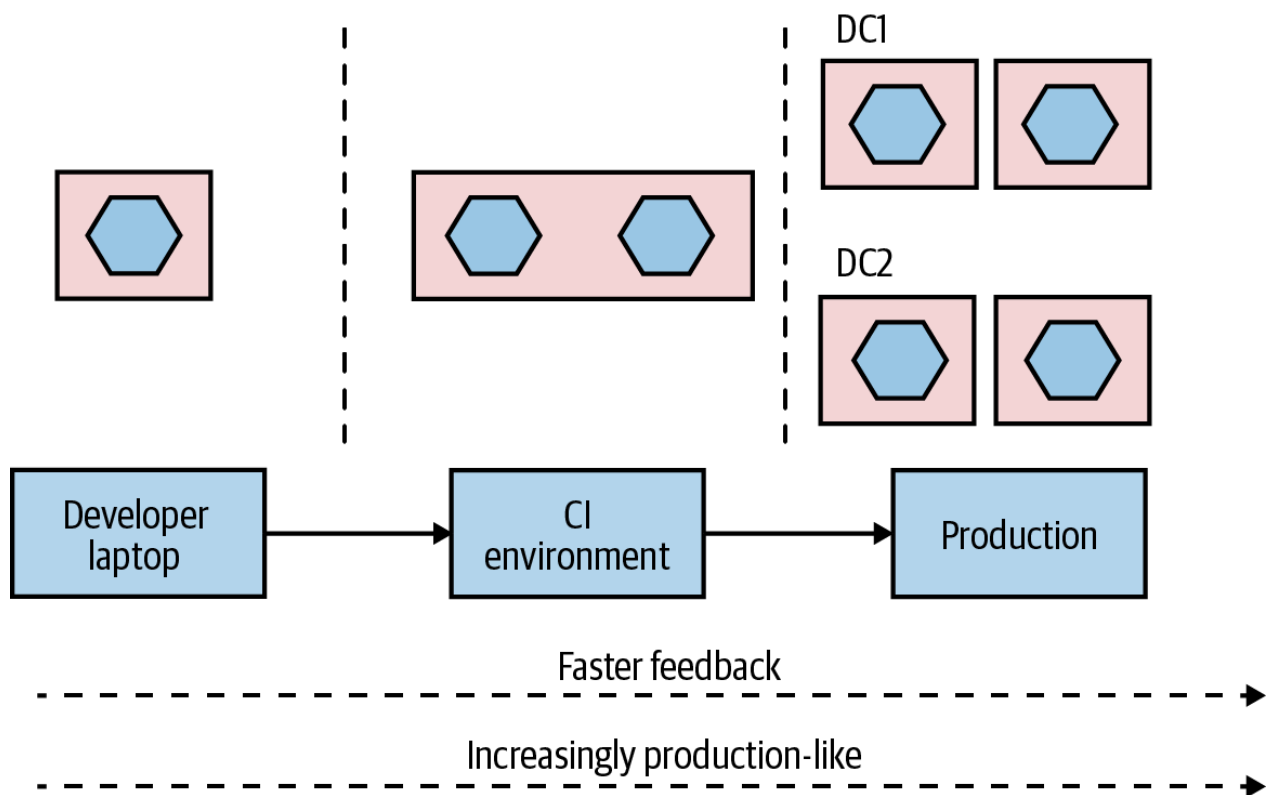


Figure 8-9. A microservice can vary in how it is deployed from one environment to the next

Hình 8-9. Một microservice có thể thay đổi về cách triển khai từ môi trường này sang môi trường khác

Finally, in production, our microservice is deployed as four instances, spread across four machines, which in turn are distributed across two different data centers.

Cuối cùng, trong production, microservice của ta được triển khai dưới dạng bốn instance, trải trên bốn máy, vốn lần lượt được phân bố trên hai data center khác nhau.

This is just an example of how you might use environments; exactly what setup you'll need will vary greatly depending on what you are building and how you deploy it. You might, for example, have multiple production environments if you need to deploy one copy of your software for each customer.

Đây chỉ là một ví dụ về cách bạn có thể dùng các môi trường; chính xác bạn cần thiết lập ra sao sẽ thay đổi rất nhiều tùy theo bạn đang xây dựng cái gì và triển khai nó như thế nào. Chẳng hạn, bạn có thể có nhiều môi trường production nếu bạn cần triển khai một bản sao phần mềm của mình cho mỗi khách hàng.

The key thing, though, is that the exact topology of your microservice will change from environment to environment. You therefore need to find ways to change the number of instances from one environment to another, along with any environment-specific configuration. You also want to build your service instances once and once only, so it follows that any environment-specific information needs to be separate from the deployed service **artifact**.

Tuy nhiên, điều mấu chốt là topology chính xác của microservice của bạn sẽ thay đổi từ môi trường này sang môi trường khác. Do đó bạn cần tìm cách thay đổi số lượng instance từ môi trường này sang môi trường khác, cùng với mọi cấu hình đặc thù theo môi trường. Bạn cũng muốn build các instance service của mình chỉ một lần duy nhất, nên kéo theo là bất kỳ thông tin đặc thù theo môi trường nào cũng cần tách biệt khỏi artifact service đã được triển khai.

How you go about varying the topology of your microservice from one environment to another will depend greatly on the mechanism you use for deployment, and also on how much the topologies vary. If the only thing that changes from one environment to another is the number of microservice instances, this might be as simple as parameterizing this value to allow for different numbers to be passed in as part of the deployment activity.

Cách bạn thay đổi topology của microservice từ môi trường này sang môi trường khác sẽ phụ thuộc rất nhiều vào cơ chế bạn dùng để triển khai, và cũng vào việc các topology khác nhau đến mức nào. Nếu thứ duy nhất thay đổi từ môi trường này sang môi trường khác là số lượng instance của microservice, thì điều này có thể đơn giản như việc tham số hóa giá trị đó để cho phép truyền vào các con số khác nhau như một phần của hoạt động triển khai.

So, to summarize, a single logical microservice can be deployed into multiple environments. From one environment to the next, the number of instances of each microservice can vary based on the requirements of each environment.

Vậy nên, để tóm tắt, một microservice logic duy nhất có thể được triển khai vào nhiều môi trường. Từ môi trường này sang môi trường khác, số lượng instance của mỗi microservice có thể thay đổi dựa trên yêu cầu của từng môi trường.

Principles of Microservice Deployment — Nguyên tắc Triển khai Microservice

With so many options before you for how to deploy your microservices, I think it's important that I establish some core principles in this area. A solid understanding of these principles will stand you in good stead no matter the choices you end up making. We'll look at each principle in detail shortly, but just to get us started, here are the core ideas we'll be covering:

Với rất nhiều lựa chọn trước mắt về cách triển khai microservice, tôi nghĩ việc thiết lập một số nguyên tắc cốt lõi trong lĩnh vực này là quan trọng. Một sự hiểu biết vững chắc về những nguyên tắc này sẽ giúp ích cho bạn dù bạn cuối cùng đưa ra lựa chọn nào. Chúng ta sẽ xem xét từng nguyên tắc một cách chi tiết ngay sau đây, nhưng để bắt đầu, đây là những ý tưởng cốt lõi mà ta sẽ đề cập:

Isolated execution Run microservice instances in an isolated fashion such that they have their own computing resources, and their execution cannot impact other microservice instances running nearby.

Thực thi cô lập (isolated execution) Chạy các instance microservice theo kiểu cô lập sao cho chúng có tài nguyên tính toán riêng, và việc thực thi của chúng không thể tác động đến các instance microservice khác đang chạy gần đó.

Focus on automation As the number of microservices increases, automation becomes increasingly important. Focus on choosing technology that allows for a high degree of automation, and adopt automation as a core part of your culture.

Tập trung vào tự động hóa Khi số lượng microservice tăng lên, tự động hóa ngày càng trở nên quan trọng. Tập trung vào việc chọn công nghệ cho phép mức độ tự động hóa cao, và đưa tự động hóa thành một phần cốt lõi trong văn hóa của bạn.

Infrastructure as code Represent the configuration for your infrastructure to ease automation and promote information sharing. Store this code in source control to allow for environments to be re-created.

Infrastructure as code Biểu diễn cấu hình cho hạ tầng của bạn để giảm bớt việc tự động hóa và thúc đẩy chia sẻ thông tin. Lưu trữ mã này trong hệ thống quản lý mã nguồn để cho phép tái tạo lại các môi trường.

Zero-downtime deployment Take **independent deployability** further and ensure that deploying a new version of a microservice can be done without any downtime to users of your service (be they humans or other microservices).

Triển khai không gián đoạn (zero-downtime deployment) Đẩy khả năng triển khai độc lập đi xa hơn và đảm bảo rằng việc triển khai một phiên bản mới của microservice có thể được thực hiện mà không gây bất kỳ gián đoạn nào cho người dùng service của bạn (dù họ là con người hay các microservice khác).

Desired state management Use a platform that maintains your microservice in a defined state, launching new instances if required in the event of outages or traffic increases.

Quản lý trạng thái mong muốn (desired state management) Dùng một nền tảng duy trì microservice của bạn ở một trạng thái đã định, khởi chạy các instance mới nếu cần trong trường hợp gián đoạn hoặc tăng lưu lượng.

Isolated Execution — Thực thi Cô lập

You may be tempted, especially early on in your microservices journey, to just put all of your microservice instances on a single machine (which could be a single physical machine or a single VM), as shown in Figure 8-10 . Purely from a host management point of view, this model is simpler. In a world in which one team manages the infrastructure and another team manages the software, the infrastructure team's workload is often a function of the number of hosts it has to manage. If more services are packed on to a single host, the host management workload doesn't increase as the number of services increases.

Bạn có thể bị cám dỗ, đặc biệt là vào giai đoạn đầu trong hành trình microservice, đặt tất cả các instance microservice của mình lên một máy đơn lẻ (có thể là một máy vật lý đơn lẻ hoặc một VM đơn lẻ), như minh họa trong Hình 8-10. Thuần túy từ góc độ quản lý máy chủ (host), mô hình này đơn giản hơn. Trong một thế giới mà một đội quản lý hạ tầng còn đội khác quản lý phần mềm, khối lượng công việc của đội hạ tầng thường là hàm của số

lượng host mà họ phải quản lý. Nếu nhồi nhét nhiều service hơn lên một host đơn lẻ, khối lượng công việc quản lý host không tăng theo khi số service tăng.

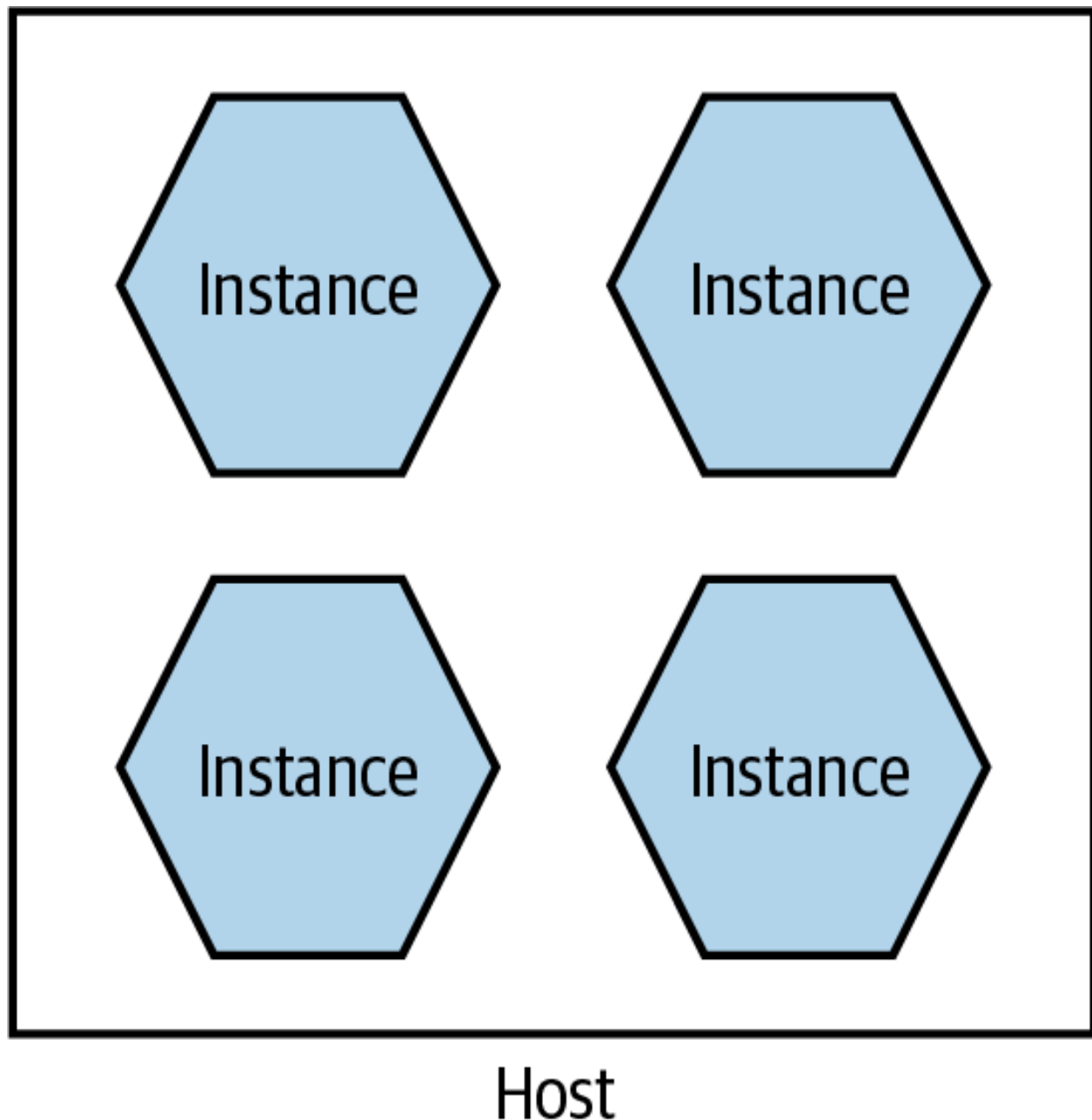


Figure 8-10. Multiple microservices per host

Hình 8-10. Nhiều microservice trên mỗi host

There are some challenges with this model, though. First, it can make **monitoring** more difficult. For example, when tracking CPU, do I need to track the CPU of one service independent of the others? Or do I care about the CPU of the host as a whole? Side effects can also be hard to avoid. If one service is under significant load, it can end up reducing the resources available to other parts of the system. This was an issue that Gilt , an online fashion retailer, encountered. Starting with a Ruby on Rails **monolith**, Gilt decided to move to microservices to make it easier to scale the application and also to

better accomodate a growing number of developers. Initially Gilt coexisted many microservices on a single box, but uneven load on one of the microservices would have an adverse impact on everything else running on that host. This made impact analysis of host failures more complex as well—taking a single host out of commission can have a large ripple effect.

Tuy nhiên, mô hình này có một số thách thức. Thứ nhất, nó có thể làm cho việc giám sát (monitoring) khó khăn hơn. Ví dụ, khi theo dõi CPU, tôi có cần theo dõi CPU của một service độc lập với các service khác không? Hay tôi quan tâm đến CPU của cả host nói chung? Các tác dụng phụ cũng có thể khó tránh. Nếu một service đang chịu tải lớn, nó có thể rất cuộc làm giảm tài nguyên sẵn có cho các phần khác của hệ thống. Đây là một vấn đề mà Gilt, một nhà bán lẻ thời trang trực tuyến, đã gặp phải. Bắt đầu từ một monolith Ruby on Rails, Gilt quyết định chuyển sang microservices để dễ mở rộng ứng dụng hơn và cũng để phù hợp hơn với số lượng lập trình viên ngày càng tăng. Ban đầu Gilt cho nhiều microservice cùng tồn tại trên một máy, nhưng tải không đồng đều trên một trong các microservice sẽ gây tác động bất lợi lên mọi thứ khác đang chạy trên host đó. Điều này cũng làm cho việc phân tích tác động của sự cố host phức tạp hơn—đưa một host đơn lẻ ra khỏi hoạt động có thể tạo ra hiệu ứng lan tỏa lớn.

Deployment of services can be somewhat more complex too, as ensuring one deployment doesn't affect another leads to additional headaches. For example, if each microservice has different (and potentially contradictory) dependencies that need to be installed on the shared host, how can I make that work?

Việc triển khai service cũng có thể phức tạp hơn đôi chút, vì việc đảm bảo một lần triển khai không ảnh hưởng đến lần khác dẫn đến những đau đầu bổ sung. Ví dụ, nếu mỗi microservice có những phụ thuộc khác nhau (và có thể mâu thuẫn nhau) cần được cài đặt trên host dùng chung, làm sao tôi có thể làm cho điều đó hoạt động?

This model can also inhibit the autonomy of teams. If services for different teams are installed on the same host, who gets to configure the host for their services? In all likelihood, this ends up getting handled by a centralized team, meaning it takes more coordination to get services deployed.

Mô hình này cũng có thể kìm hãm tính tự chủ của các đội. Nếu service của các đội khác nhau được cài trên cùng một host, ai sẽ là người cấu hình host cho service của họ? Nhiều khả năng, điều này rất cuộc được xử lý bởi một đội tập trung, nghĩa là cần nhiều sự phối hợp hơn để triển khai được service.

Fundamentally, running lots of microservice instances on the same machine (virtual or physical) ends up drastically undermining one of the key principles of microservices as a whole—independent deployability. It follows, therefore, that we really want to run microservice instances in isolation, as we see in Figure 8-11 .

Về cơ bản, việc chạy nhiều instance microservice trên cùng một máy (ảo hay vật lý) rất cuộc làm suy yếu nghiêm trọng một trong những nguyên tắc then chốt của microservices nói chung—khả năng triển khai độc lập. Do đó, kéo theo là ta thực sự muốn chạy các instance microservice trong sự cô lập, như ta thấy trong Hình 8-11.

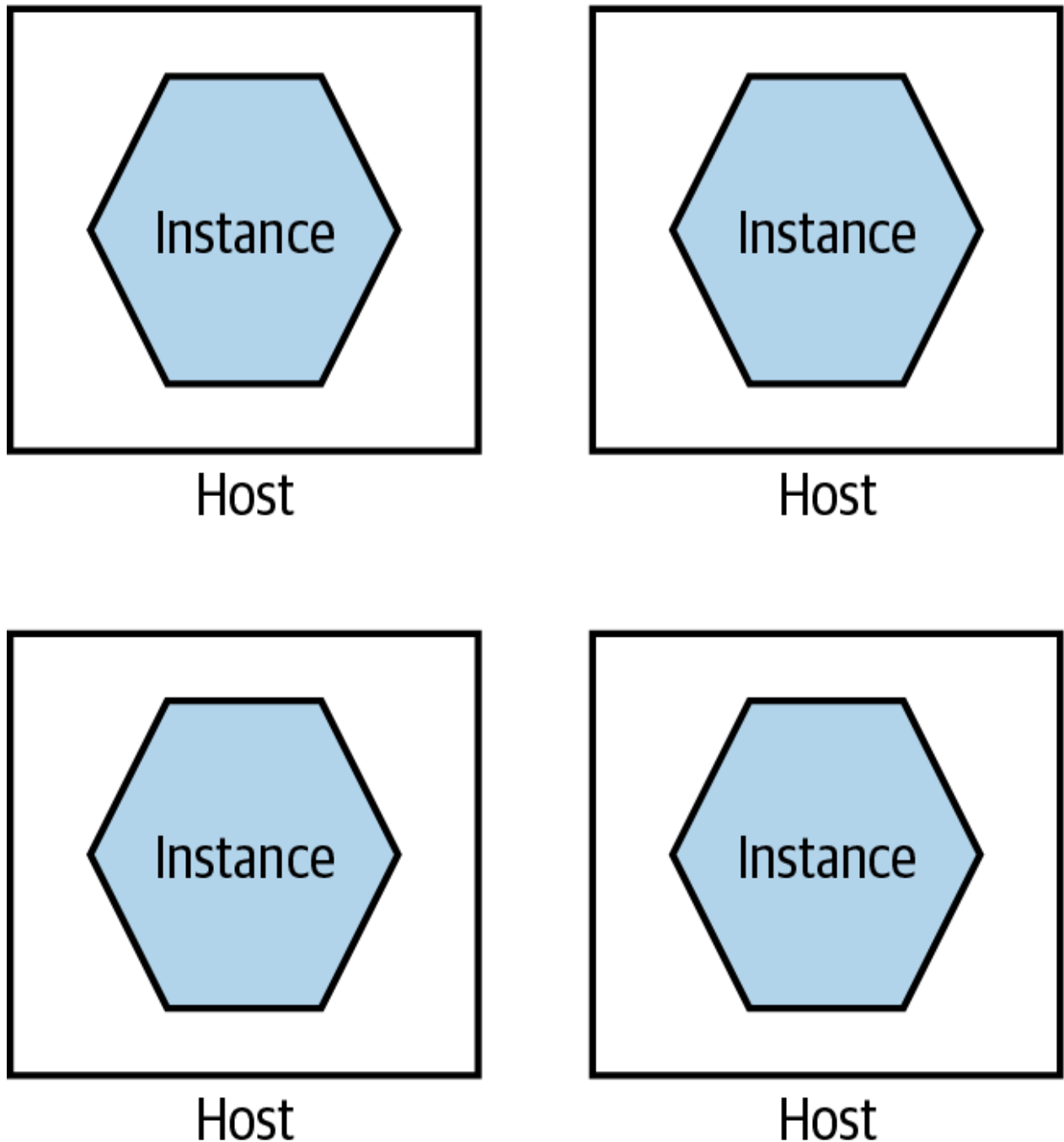


Figure 8-11. A single microservice per host

Hình 8-11. Một microservice duy nhất trên mỗi host

Each microservice instance gets its own isolated execution environment. It can install its own dependencies, and have its own set of ring-fenced resources.

Mỗi instance microservice có được môi trường thực thi cô lập riêng của nó. Nó có thể cài các phụ thuộc riêng và có tập tài nguyên được khoanh vùng riêng (ring-fenced).

As my old colleague Neal Ford puts it, many of our working practices around deployment and host management are an attempt to optimize for scarcity of resources. In the past, if we wanted another

machine to achieve isolation, our only option was to buy or rent another physical machine. This often had a large lead time to it and resulted in a long-term financial commitment. In my experience, it's not uncommon for clients to provision new servers only every two to three years, and trying to get additional machines outside of these timelines is difficult. But on-demand computing platforms have drastically reduced the costs of computing resources, and improvements in virtualization technology mean there is more flexibility, even for in-house hosted infrastructure.

Như đồng nghiệp cũ của tôi Neal Ford diễn đạt, nhiều thực hành làm việc của chúng ta quanh việc triển khai và quản lý host là một nỗ lực tối ưu hóa cho sự khan hiếm tài nguyên. Trong quá khứ, nếu ta muốn thêm một máy để đạt được sự cô lập, lựa chọn duy nhất của ta là mua hoặc thuê thêm một máy vật lý. Việc này thường có thời gian chờ dài và dẫn đến một cam kết tài chính dài hạn. Theo kinh nghiệm của tôi, không hiếm khi khách hàng chỉ cung cấp máy chủ mới mỗi hai đến ba năm một lần, và việc cố lấy thêm máy ngoài những mốc thời gian này là khó. Nhưng các nền tảng tính toán theo yêu cầu (on-demand) đã giảm mạnh chi phí tài nguyên tính toán, và những cải tiến trong công nghệ ảo hóa nghĩa là có nhiều linh hoạt hơn, kể cả cho hạ tầng tự lưu trữ nội bộ.

With containerization joining the mix, we have more options than ever before for provisioning an isolated execution environment. As Figure 8-12 shows, broadly speaking, we go from the extreme of having dedicated physical machines for our services, which gives us the best isolation but probably the highest cost, to containers at the other end, which gives us weaker isolation but tends to be more cost effective and much faster to provision. We'll come back to some of the specifics around technology such as containerization later in this chapter.

Với sự tham gia của container, ta có nhiều lựa chọn hơn bao giờ hết để cung cấp một môi trường thực thi cô lập. Như Hình 8-12 cho thấy, nói rộng ra, ta đi từ thái cực có những máy vật lý chuyên dụng cho service của mình, vốn cho ta sự cô lập tốt nhất nhưng có lẽ chi phí cao nhất, đến container ở đầu kia, vốn cho ta sự cô lập yếu hơn nhưng thường hiệu quả về chi phí hơn và nhanh hơn nhiều để cung cấp. Ta sẽ quay lại một số chi tiết cụ thể quanh công nghệ như container hóa ở phần sau trong chương này.

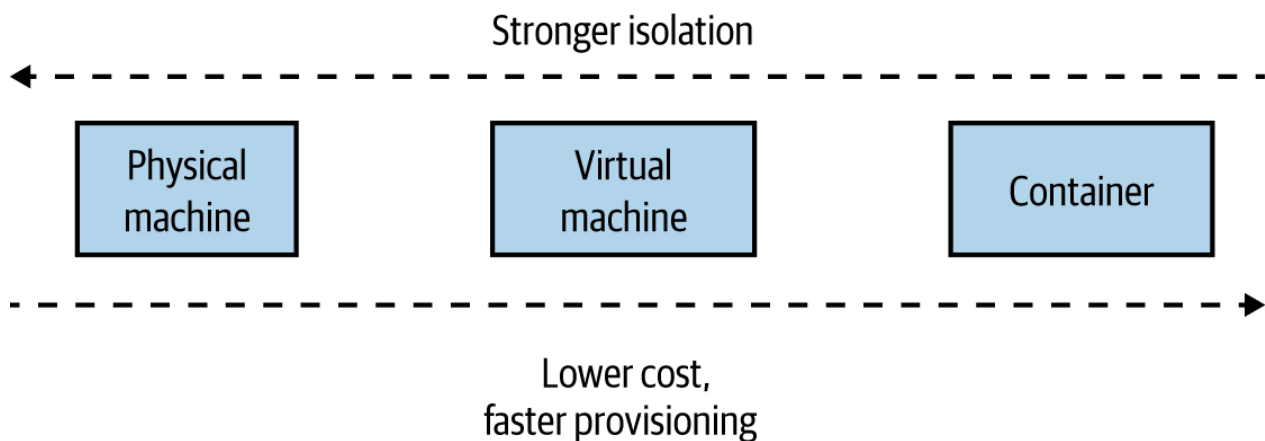


Figure 8-12. Different trade-offs around isolation models

Hình 8-12. Các trade-off khác nhau quanh các mô hình cô lập

If you were deploying your microservices onto more abstracted platforms such as AWS Lambda or Heroku, this isolation would be provided for you. Depending on the nature of the platform itself, you could likely expect your microservice instance to end up running inside a **container** or dedicated VM behind the scenes.

Nếu bạn triển khai microservice của mình lên những nền tảng trừu tượng hơn như AWS

Lambda hay Heroku, sự cô lập này sẽ được cung cấp sẵn cho bạn. Tùy theo bản chất của bản thân nền tảng, nhiều khả năng bạn có thể kỳ vọng instance microservice của mình tốt cuộc chạy bên trong một container hoặc một VM chuyên dụng ở phía hậu trường.

In general, the isolation around containers has improved sufficiently to make them a more natural choice for microservice workloads. The difference in isolation between containers and VMs has reduced to the point that for the vast majority of workloads, containers are “good enough,” which is in large part why they are such a popular choice and why they tend to be my default choice in most situations.

Nhìn chung, sự cô lập quanh container đã cải thiện đủ để biến chúng thành lựa chọn tự nhiên hơn cho các tải công việc microservice. Sự khác biệt về cô lập giữa container và VM đã thu hẹp đến mức mà với đại đa số tải công việc, container là “đủ tốt”, và đó phần lớn là lý do tại sao chúng là một lựa chọn phổ biến đến vậy và tại sao chúng thường là lựa chọn mặc định của tôi trong hầu hết tình huống.

Focus on Automation — Tập trung vào Tự động hóa

As you add more microservices, you’ll have more moving parts to deal with—more processes, more things to configure, more instances to monitor. Moving to microservices pushes a lot of complexity into the operational space, and if you are managing your operational processes in a mostly manual way, this means that more services will require more and more people to do things.

Khi bạn thêm nhiều microservice hơn, bạn sẽ có nhiều bộ phận động hơn để xử lý—nhiều tiến trình hơn, nhiều thứ cần cấu hình hơn, nhiều instance cần giám sát hơn. Việc chuyển sang microservices đẩy rất nhiều sự phức tạp vào không gian vận hành, và nếu bạn quản lý các quy trình vận hành của mình theo cách phần lớn là thủ công, điều này có nghĩa là nhiều service hơn sẽ đòi hỏi ngày càng nhiều người để làm việc.

Instead, you need a relentless focus on automation. Select tooling and technology that allows for things to be done in an automatic fashion, ideally with a view to working with infrastructure as code (which we’ll cover shortly).

Thay vào đó, bạn cần một sự tập trung không ngơi nghỉ vào tự động hóa. Hãy chọn công cụ và công nghệ cho phép làm mọi thứ theo cách tự động, lý tưởng nhất là hướng tới việc làm việc với infrastructure as code (mà ta sẽ đề cập ngay sau đây).

As the number of microservices increases, automation becomes increasingly important. Give serious consideration to technology that allows for a high degree of automation, and adopt automation as a core part of your culture.

Khi số lượng microservice tăng lên, tự động hóa ngày càng trở nên quan trọng. Hãy cân nhắc nghiêm túc công nghệ cho phép mức độ tự động hóa cao, và đưa tự động hóa thành một phần cốt lõi trong văn hóa của bạn.

Automation is also how you can make sure that your developers still remain productive. Giving developers the ability to self-service-provision individual services or groups of services is the key to making their lives easier.

Tự động hóa cũng là cách bạn có thể đảm bảo rằng các lập trình viên của mình vẫn duy trì năng suất. Trao cho lập trình viên khả năng tự phục vụ (self-service) việc cung cấp các service riêng lẻ hay nhóm service là chìa khóa để làm cho cuộc sống của họ dễ dàng hơn.

Picking technology that enables automation starts with the tools used to manage hosts. Can you write a line of code to launch a virtual machine, or shut one down? Can you deploy the software you have

written automatically? Can you deploy database changes without manual intervention? Embracing a culture of automation is key if you want to keep the complexities of microservice architectures in check.

Việc chọn công nghệ cho phép tự động hóa bắt đầu từ các công cụ dùng để quản lý host. Bạn có thể viết một dòng mã để khởi chạy một máy ảo, hay tắt nó đi không? Bạn có thể triển khai phần mềm mình đã viết một cách tự động không? Bạn có thể triển khai các thay đổi database mà không cần can thiệp thủ công không? Đón nhận một văn hóa tự động hóa là then chốt nếu bạn muốn kiểm soát được sự phức tạp của các kiến trúc microservice.

Two case studies on the power of automation

Hai nghiên cứu tình huống về sức mạnh của tự động hóa

It will probably be helpful to give you a couple of concrete examples that explain the power of good automation. The Australian company realestate.com.au (REA) provides real estate listings for retail and commercial customers in Australia and elsewhere in the Asia-Pacific region. Over a number of years, it had been moving its platform toward a distributed microservices design. When it started on this journey, it had to spend a lot of time getting the tooling around the services just right—making it easy for developers to provision machines, deploy their code, and monitor their services. This caused a front-loading of work to get things started.

Có lẽ sẽ hữu ích nếu đưa cho bạn vài ví dụ cụ thể giải thích sức mạnh của tự động hóa tốt. Công ty Úc realestate.com.au (REA) cung cấp các tin đăng bất động sản cho khách hàng bán lẻ và thương mại tại Úc và những nơi khác trong khu vực châu Á - Thái Bình Dương. Qua nhiều năm, công ty đã dịch chuyển nền tảng của mình theo hướng một thiết kế microservices phân tán. Khi bắt đầu hành trình này, họ đã phải dành rất nhiều thời gian để làm cho bộ công cụ quanh các service thật chuẩn xác—giúp lập trình viên dễ dàng cung cấp máy, triển khai mã và giám sát service của họ. Điều này gây ra một khối lượng công việc dồn về phía trước để khởi động mọi thứ.

In the first three months of this exercise, REA was able to move just two new microservices into production, with the development team taking full responsibility for the entire build, deployment, and support of the services. In the next three months, between 10 to 15 services went live in a similar manner. By the end of an 18-month period, REA had more than 70 services in production.

Trong ba tháng đầu của bài tập này, REA chỉ có thể đưa hai microservice mới vào production, với đội phát triển chịu toàn bộ trách nhiệm về việc build, triển khai và hỗ trợ các service. Trong ba tháng tiếp theo, từ 10 đến 15 service được đưa vào hoạt động theo cách tương tự. Đến cuối giai đoạn 18 tháng, REA đã có hơn 70 service trong production.

This sort of pattern is also borne out by the experiences of Gilt, which we mentioned earlier. Again, automation, especially tooling to help developers, drove the explosion in Gilt's use of microservices. A year after starting its migration to microservices, Gilt had around 10 microservices live; by 2012, over 100; and in 2014, over 450 microservices were live—or around three microservices for every developer in Gilt. This sort of ratio of microservices to developers is not uncommon among organizations that are mature in their use of microservices, the Financial Times being a company with a similar ratio.

Kiểu mô thức này cũng được minh chứng qua trải nghiệm của Gilt, công ty ta đã nhắc đến trước đó. Một lần nữa, tự động hóa, đặc biệt là bộ công cụ giúp đỡ lập trình viên, đã thúc đẩy sự bùng nổ trong việc Gilt sử dụng microservices. Một năm sau khi bắt đầu cuộc di trú sang microservices, Gilt có khoảng 10 microservice hoạt động; đến năm 2012, hơn 100; và đến năm 2014, hơn 450 microservice đang hoạt động—tức khoảng ba microservice cho mỗi lập trình viên tại Gilt. Kiểu tỷ lệ microservice trên lập trình viên này không phải là

hiếm trong các tổ chức đã trưởng thành trong việc dùng microservices, Financial Times là một công ty có tỷ lệ tương tự.

Infrastructure as Code (IAC) — Infrastructure as Code (IAC)

Taking the concept of automation further, infrastructure as code (IAC) is a concept whereby your infrastructure is configured by using machine-readable code. You might define your service configuration in a chef or puppet file, or perhaps write some bash scripts to set things up—but whatever tool you end up using, your system can be brought into a known state through the use of source code. Arguably, the concept of IAC could be considered one way to implement automation. I think, though, that it's worth calling it out as its own thing, because it speaks to how automation should be done. Infrastructure as code has brought concepts from software development into the operations space. By defining our infrastructure via code, this configuration can be version controlled, tested, and repeated at will. For more on this topic, I recommend *Infrastructure as Code*, 2nd edition, by Kief Morris. ¹

Đẩy khái niệm tự động hóa đi xa hơn, infrastructure as code (IAC) là một khái niệm theo đó hạ tầng của bạn được cấu hình bằng cách dùng mã mà máy đọc được. Bạn có thể định nghĩa cấu hình service của mình trong một file chef hoặc puppet, hoặc có lẽ viết vài script bash để thiết lập mọi thứ—nhưng dù bạn cuối cùng dùng công cụ nào, hệ thống của bạn có thể được đưa về một trạng thái đã biết thông qua việc sử dụng mã nguồn. Có thể lập luận rằng khái niệm IAC có thể được coi là một cách để hiện thực hóa tự động hóa. Tuy nhiên, tôi nghĩ nó đáng được nêu ra như một thứ riêng, vì nó nói lên cách thức tự động hóa nên được thực hiện. Infrastructure as code đã đưa các khái niệm từ phát triển phần mềm vào không gian vận hành. Bằng cách định nghĩa hạ tầng của ta thông qua mã, cấu hình này có thể được quản lý phiên bản (version control), kiểm thử và lặp lại tùy ý. Để biết thêm về chủ đề này, tôi khuyên đọc *Infrastructure as Code*, ấn bản thứ 2, của Kief Morris.

1

Theoretically, you could use any programming language to apply the ideas of infrastructure as code, but there are specialist tools in this area such as Puppet, Chef, Ansible, and others, all of which took their lead from the earlier CFEngine. These tools are declarative—they allow you to define in textual form what you expect a machine (or other set of resources) to look like, and when these scripts are applied, the infrastructure is brought into that state. More recent tools have gone beyond looking at configuring a machine and moved into looking at how to configure entire sets of cloud resources—Terraform has been very successful in this space, and I'm excited to see the potential of Pulumi, which is aiming to do something similar, albeit by allowing people to use normal programming languages rather than the domain-specific languages that often get used by these tools. AWS CloudFormation and the AWS Cloud Development Kit (CDK) are examples of platform-specific tools, in this case supporting only AWS—although it's worth noting that even if I was working only with AWS, I'd prefer the flexibility of a cross-platform tool like Terraform.

Về lý thuyết, bạn có thể dùng bất kỳ ngôn ngữ lập trình nào để áp dụng các ý tưởng của infrastructure as code, nhưng có những công cụ chuyên biệt trong lĩnh vực này như Puppet, Chef, Ansible và những công cụ khác, tất cả đều lấy cảm hứng từ CFEngine trước đó. Các công cụ này có tính khai báo (declarative)—chúng cho phép bạn định nghĩa dưới dạng văn bản cái mà bạn kỳ vọng một máy (hoặc một tập tài nguyên khác) trông như thế nào, và khi các script này được áp dụng, hạ tầng được đưa về trạng thái đó. Các công cụ gần đây hơn đã vượt khỏi việc nhìn vào cấu hình một máy và chuyển sang nhìn vào cách cấu hình toàn bộ các tập tài nguyên đám mây—Terraform đã rất thành công trong không gian này, và tôi rất hào hứng khi thấy tiềm năng của Pulumi, vốn nhắm tới làm điều gì đó

tương tự, dù bằng cách cho phép mọi người dùng các ngôn ngữ lập trình thông thường thay vì các ngôn ngữ chuyên biệt theo miền (domain-specific language) thường được các công cụ này dùng. AWS CloudFormation và AWS Cloud Development Kit (CDK) là những ví dụ về công cụ đặc thù theo nền tảng, trong trường hợp này chỉ hỗ trợ AWS—mặc dù cũng đáng lưu ý rằng ngay cả khi tôi chỉ làm việc với AWS, tôi vẫn ưa thích sự linh hoạt của một công cụ đa nền tảng như Terraform.

Version controlling your infrastructure code gives you transparency over who has made changes, something that auditors love. It also makes it easier to reproduce an environment at a given point in time. This is something that can be especially useful when trying to track down defects. In one memorable example, one of my clients, as part of a court case, had to re-create an entire running system as of a specific time some years before, down to the patch levels of the operating systems and the contents of message brokers. If the environment configuration had been stored in version control, their job would have been much easier—as it was, they ended up spending over three months painstakingly trying to rebuild a mirror image of an earlier production environment by wading through emails and **release** notes to try and work out what was done by whom. The court case, which had already been going for a long period of time, was still not resolved by the time I ended my work with the client.

Việc quản lý phiên bản mã hạ tầng của bạn cho bạn sự minh bạch về việc ai đã thực hiện thay đổi, điều mà các kiểm toán viên rất thích. Nó cũng giúp tái tạo một môi trường tại một thời điểm nhất định dễ dàng hơn. Đây là điều có thể đặc biệt hữu ích khi cố truy tìm lỗi. Trong một ví dụ đáng nhớ, một trong các khách hàng của tôi, như một phần của một vụ kiện tại tòa, đã phải tái tạo lại toàn bộ một hệ thống đang chạy tại một thời điểm cụ thể vài năm trước, đến tận mức bản vá của hệ điều hành và nội dung của các message broker. Nếu cấu hình môi trường đã được lưu trong version control, công việc của họ sẽ dễ dàng hơn nhiều—nhưng thực tế, họ rất cuộc đã dành hơn ba tháng cặm cụi cố dựng lại một bản sao y của một môi trường production trước đó bằng cách lội qua các email và ghi chú phát hành để cố xác định ai đã làm gì. Vụ kiện, vốn đã kéo dài một thời gian rồi, vẫn chưa được giải quyết vào lúc tôi kết thúc công việc với khách hàng đó.

Zero-Downtime Deployment — Triển khai Không Gián đoạn

As you are probably sick and tired of hearing me say, independent deployability is really important. It is, however, also not an absolute quality. How independent is something exactly? Before this chapter, we'd primarily looked at independent deployability in terms of avoiding implementation **coupling**. Earlier in this chapter, we spoke about the importance of providing a microservice instance with an isolated execution environment, to ensure it has a degree of independence at the physical deployment level. But we can go further.

Như có lẽ bạn đã chán ngấy khi nghe tôi nói, khả năng triển khai độc lập thực sự quan trọng. Tuy nhiên, nó cũng không phải là một phẩm chất tuyệt đối. Một thứ độc lập đến mức nào, chính xác? Trước chương này, ta chủ yếu đã nhìn vào khả năng triển khai độc lập dưới góc độ tránh coupling trong hiện thực. Ở đầu chương này, ta đã nói về tầm quan trọng của việc cung cấp cho một instance microservice một môi trường thực thi cô lập, để đảm bảo nó có một mức độ độc lập ở tầng triển khai vật lý. Nhưng ta có thể đi xa hơn.

1 Kief Morris, *Infrastructure as Code*, 2nd edition (Sebastopol: O'Reilly, 2020).

1 Kief Morris, *Infrastructure as Code*, 2nd edition (Sebastopol: O'Reilly, 2020).

Implementing the ability for zero-downtime deployment can be a huge step up in allowing

microservices to be developed and deployed. Without zero-downtime deployment, I may have to coordinate with upstream consumers when I release software to alert them of a potential outage.

Việc hiện thực hóa khả năng triển khai không gián đoạn có thể là một bước tiến lớn trong việc cho phép microservice được phát triển và triển khai. Không có triển khai không gián đoạn, tôi có thể phải phối hợp với các consumer phía thượng nguồn (upstream) khi phát hành phần mềm để báo cho họ về một sự cố gián đoạn tiềm tàng.

Sarah Wells at the Financial Times cites the ability to implement zero-downtime deployment as being the single biggest benefit in terms of improving the speed of delivery. With the confidence that releases wouldn't interrupt its users, the Financial Times was able to drastically increase the frequency of releases. In addition, a zero-downtime release can be much more easily done during working hours. Quite aside from the fact that doing so improves the quality of life of the people involved with the release (compared to working evenings and weekends), a well-rested team working during the day is less likely to make mistakes and will have support from many of their colleagues when they need to fix issues.

Sarah Wells tại Financial Times nêu rằng khả năng hiện thực hóa triển khai không gián đoạn là lợi ích lớn nhất xét về mặt cải thiện tốc độ chuyển giao. Với sự tự tin rằng các lần phát hành sẽ không làm gián đoạn người dùng, Financial Times đã có thể tăng mạnh tần suất phát hành. Thêm vào đó, một lần phát hành không gián đoạn có thể được thực hiện dễ dàng hơn nhiều trong giờ làm việc. Bỏ qua thực tế rằng làm như vậy cải thiện chất lượng cuộc sống của những người liên quan đến việc phát hành (so với làm việc buổi tối và cuối tuần), một đội được nghỉ ngơi đầy đủ làm việc vào ban ngày ít có khả năng phạm sai lầm hơn và sẽ có sự hỗ trợ từ nhiều đồng nghiệp của họ khi cần sửa lỗi.

The goal here is that upstream consumers shouldn't notice at all when you do a release. Making this possible can depend greatly on the nature of your microservice. If you're already making use of middleware-backed **asynchronous** communication between your microservice and your consumers, this might be trivial to implement—messages sent to you will be delivered when you are back up. If you're making use of **synchronous**-based communication, though, this can be more problematic.

Mục tiêu ở đây là các consumer phía thượng nguồn hoàn toàn không nên nhận ra khi bạn thực hiện một lần phát hành. Việc làm cho điều này khả thi có thể phụ thuộc rất nhiều vào bản chất của microservice của bạn. Nếu bạn đã sử dụng giao tiếp bất đồng bộ có middleware hậu thuẫn giữa microservice và các consumer của mình, điều này có thể rất dễ hiện thực hóa—các message gửi tới bạn sẽ được chuyển giao khi bạn hoạt động trở lại. Tuy nhiên, nếu bạn dùng giao tiếp dựa trên đồng bộ, điều này có thể rắc rối hơn.

Concepts like rolling upgrades can be handy here, and this is one area where the use of a platform like Kubernetes makes your life much easier. With a rolling upgrade, your microservice isn't totally shut down before the new version is deployed, instead instances of your microservice are slowly ramped down as new instances running new versions of your software are ramped up. It's worth noting, though, that if the only thing you are looking for is something to help with zero-downtime deployments, then implementing Kubernetes is likely huge overkill. Something simple like a blue-green deployment mechanism (which we'll explore more in "Separating Deployment from Release" on page 270) can work just as effectively.

Các khái niệm như rolling upgrade có thể hữu ích ở đây, và đây là một lĩnh vực mà việc dùng một nền tảng như Kubernetes làm cho cuộc sống của bạn dễ dàng hơn nhiều. Với một rolling upgrade, microservice của bạn không bị tắt hoàn toàn trước khi phiên bản mới được triển khai, thay vào đó các instance của microservice được giảm dần (ramp down) khi các instance mới chạy phiên bản mới của phần mềm được tăng dần (ramp up). Tuy

vậy, cũng đáng lưu ý rằng nếu thứ duy nhất bạn tìm kiếm là một thứ giúp triển khai không gián đoạn, thì việc hiện thực hóa Kubernetes có lẽ là quá thừa thãi. Một thứ đơn giản như cơ chế triển khai blue-green (mà ta sẽ khám phá thêm trong “Separating Deployment from Release” ở trang 270) có thể hoạt động hiệu quả không kém.

There can be additional challenges in terms of dealing with problems like long-lived connections and the like. It’s certainly true that if you build a microservice with zero-downtime deployment in mind, you’ll likely have a much easier time of it than if you took an existing systems architecture and attempted to retrofit this concept afterwards. Whether or not you are able to implement a zero-downtime deployment for your services initially, if you can get there you’ll certainly appreciate that increased level of independence.

Có thể có những thách thức bổ sung về việc xử lý các vấn đề như kết nối tồn tại lâu dài (long-lived connection) và những thứ tương tự. Chắc chắn là nếu bạn xây dựng một microservice với triển khai không gián đoạn trong đầu ngay từ đầu, bạn có lẽ sẽ thấy mọi việc dễ dàng hơn nhiều so với việc lấy một kiến trúc hệ thống hiện có và cố gắn ghép khái niệm này vào sau. Dù bạn có thể hiện thực hóa triển khai không gián đoạn cho service của mình ngay từ đầu hay không, nếu bạn đạt được điều đó, bạn chắc chắn sẽ trân trọng mức độ độc lập gia tăng ấy.

Desired State Management — Quản lý Trạng thái Mong muốn

Desired state management is the ability to specify the infrastructure requirements you have for your application, and for those requirements to be maintained without manual intervention. If the running system changes in such a way that your desired state is no longer maintained, the underlying platform takes the required steps to bring the system back into desired state.

Quản lý trạng thái mong muốn là khả năng đặc tả các yêu cầu hạ tầng mà bạn có cho ứng dụng của mình, và để những yêu cầu đó được duy trì mà không cần can thiệp thủ công. Nếu hệ thống đang chạy thay đổi theo cách mà trạng thái mong muốn của bạn không còn được duy trì, nền tảng bên dưới sẽ thực hiện các bước cần thiết để đưa hệ thống trở lại trạng thái mong muốn.

As a simple example of how desired state management might work, you could specify the number of instances your microservice requires, perhaps also specifying how much memory and CPU those instances need. Some underlying platform takes this configuration and applies it, bringing the system into the desired state. It’s up to the platform to, among other things, identify which machines have spare resources that can be allocated to run the requested number of instances. As Figure 8-13 shows, if one of those instances dies, the platform recognizes that the current state doesn’t match the desired state and takes appropriate action by launching a replacement instance.

Như một ví dụ đơn giản về cách quản lý trạng thái mong muốn có thể hoạt động, bạn có thể đặc tả số lượng instance mà microservice của mình cần, có lẽ cũng đặc tả các instance đó cần bao nhiêu bộ nhớ và CPU. Một nền tảng bên dưới nào đó nhận cấu hình này và áp dụng nó, đưa hệ thống về trạng thái mong muốn. Việc xác định, trong số những thứ khác, máy nào có tài nguyên dư có thể được phân bổ để chạy số lượng instance được yêu cầu là tùy thuộc vào nền tảng. Như Hình 8-13 cho thấy, nếu một trong những instance đó chết, nền tảng nhận ra rằng trạng thái hiện tại không khớp với trạng thái mong muốn và thực hiện hành động thích hợp bằng cách khởi chạy một instance thay thế.

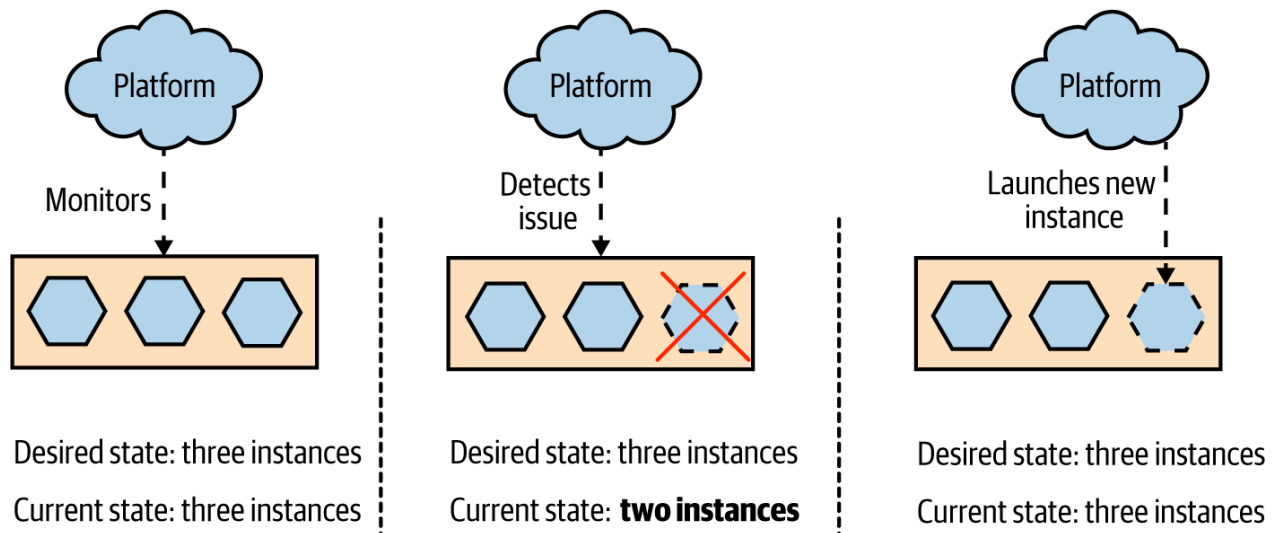


Figure 8-13. A platform providing desired state management, spinning up a new instance when one dies

Hình 8-13. Một nền tảng cung cấp quản lý trạng thái mong muốn, khởi tạo một instance mới khi một instance chết

The beauty of desired state management is that the platform itself manages how the desired state is maintained. It frees development and operations people alike from having to worry about exactly how things are being done—they just have to focus on getting the desired state definition right in the first place. It also means that in the event of a problem occurring, such as an instance dying, the underlying hardware failing, or a data center shutting down, the platform can handle the issue for you without human intervention being required.

Về đẹp của quản lý trạng thái mong muốn là chính nền tảng quản lý cách trạng thái mong muốn được duy trì. Nó giải phóng cả những người phát triển lẫn vận hành khỏi việc phải lo lắng chính xác mọi thứ đang được thực hiện ra sao—they chỉ cần tập trung vào việc đặc tả trạng thái mong muốn cho đúng ngay từ đầu. Nó cũng có nghĩa là trong trường hợp một vấn đề xảy ra, chẳng hạn một instance chết, phần cứng bên dưới hỏng, hay một data center ngừng hoạt động, nền tảng có thể xử lý vấn đề cho bạn mà không cần đến sự can thiệp của con người.

While it's possible to build your own toolchain to apply desired state management, typically you use a platform that already supports it. Kubernetes is one such tool that embraces this idea, and you can also achieve something similar using concepts such as **autoscaling** groups on a public cloud provider like Azure or AWS. Another platform that can provide this capability is Nomad . Unlike Kubernetes, which is focused on deploying and managing container-based workloads, Nomad has a very flexible model around running other sorts of application workloads as well, such as Java applications, VMs, Hadoop jobs, and more. It may be worth a look if you want a platform for managing mixed workloads that still makes use of concepts like desired state management.

Mặc dù có thể xây dựng bộ công cụ riêng của bạn để áp dụng quản lý trạng thái mong muốn, thường thì bạn dùng một nền tảng đã hỗ trợ sẵn điều đó. Kubernetes là một công cụ như vậy, đón nhận ý tưởng này, và bạn cũng có thể đạt được điều gì đó tương tự bằng cách dùng các khái niệm như autoscaling group trên một nhà cung cấp public cloud như Azure hay AWS. Một nền tảng khác có thể cung cấp khả năng này là Nomad. Khác với Kubernetes, vốn tập trung vào triển khai và quản lý các tải công việc dựa trên container, Nomad có một mô hình rất linh hoạt quanh việc chạy các loại tải công việc ứng dụng khác

nữa, chẳng hạn như ứng dụng Java, VM, công việc Hadoop và nhiều hơn nữa. Có thể đáng để xem xét nếu bạn muốn một nền tảng để quản lý các tải công việc hỗn hợp mà vẫn dùng các khái niệm như quản lý trạng thái mong muốn.

These platforms are aware of the underlying availability of resources and are able to match the requests for desired state to the available resources (or else tell you this isn't possible). As an operator, you are distanced from the low-level configuration—you can say something simple like “I want four instances spread across both data centers” and rely on your platform to ensure this is done for you. Different platforms provide different levels of control—you can get much more complex with your desired state definition if you want.

Các nền tảng này nhận biết được mức độ sẵn có của tài nguyên bên dưới và có thể khớp các yêu cầu về trạng thái mong muốn với tài nguyên sẵn có (hoặc nếu không thì báo cho bạn biết điều này là bất khả thi). Là một người vận hành, bạn được tách khỏi cấu hình cấp thấp—bạn có thể nói điều gì đó đơn giản như “Tôi muốn bốn instance trải trên cả hai data center” và dựa vào nền tảng của mình để đảm bảo điều này được thực hiện cho bạn. Các nền tảng khác nhau cung cấp các mức độ kiểm soát khác nhau—bạn có thể làm cho đặc tả trạng thái mong muốn của mình phức tạp hơn nhiều nếu muốn.

The use of desired state management can occasionally cause you problems if you forget you're making use of it. I remember a situation in which I was shutting down a development cluster on AWS before I went home. I was shutting down the managed virtual machine instances (provided by AWS's EC2 product) to save money—they weren't going to be used overnight. However, I found that as soon as I killed one of the instances, another instance popped back up. It took me a while to realize that I had configured an autoscaling group to ensure that there was a minimum number of machines. AWS was seeing an instance die and spinning up a replacement. It took me 15 minutes of playing whack-a-mole like this before I realized what was up. The problem was that we were charged for EC2 on a per-hour basis. Even if an instance ran for only a minute, we got charged for the full hour. So my flailing around at the end of the day ended up being quite costly. In a way, this was a sign of success (at least that's what I told myself)—we'd set up the autoscaling group some time before, and they had just worked to the point that we had forgotten they were there. It was simply a matter of writing a script to disable the autoscaling group as part of the cluster shutdown to fix the problem in the future.

Việc dùng quản lý trạng thái mong muốn đôi khi có thể gây cho bạn rắc rối nếu bạn quên mất rằng mình đang dùng nó. Tôi nhớ một tình huống tôi đang tắt một cluster phát triển trên AWS trước khi về nhà. Tôi đang tắt các instance máy ảo được quản lý (cung cấp bởi sản phẩm EC2 của AWS) để tiết kiệm tiền—chúng sẽ không được dùng qua đêm. Tuy nhiên, tôi phát hiện rằng ngay khi tôi giết một trong các instance, một instance khác lại bật lên. Phải mất một lúc tôi mới nhận ra rằng mình đã cấu hình một autoscaling group để đảm bảo có một số lượng máy tối thiểu. AWS thấy một instance chết và khởi tạo một bản thay thế. Tôi mất 15 phút chơi trò đập-chuột-chũi (whack-a-mole) như thế này trước khi nhận ra chuyện gì đang xảy ra. Vấn đề là chúng tôi bị tính phí EC2 theo giờ. Ngay cả khi một instance chỉ chạy một phút, chúng tôi vẫn bị tính phí trọn giờ. Vậy nên việc tôi vùng vẫy loạn xạ vào cuối ngày rốt cuộc khá tốn kém. Theo một cách nào đó, đây là dấu hiệu của thành công (ít nhất đó là điều tôi tự nhủ với mình)—chúng tôi đã thiết lập autoscaling group từ trước đó, và chúng đã hoạt động tốt đến mức chúng tôi quên rằng chúng tồn tại. Đơn giản chỉ là vấn đề viết một script để vô hiệu hóa autoscaling group như một phần của việc tắt cluster để khắc phục vấn đề này trong tương lai.

Prerequisites

Điều kiện tiên quyết

To take advantage of desired state management, the platform needs some way to automatically launch instances of your microservice. So having a fully automated deployment for microservice instances is a clear prerequisite for desired state management. You may also need to give careful thought to how long it takes your instances to launch. If you are using desired state management to ensure there are enough computing resources to handle user load, then if an instance dies, you'll want a replacement instance as quickly as possible to fill the gap. If provisioning a new instance takes a long time, you may need to have excess capacity in place to handle the load in the event of an instance dying so as to give yourself enough breathing room to bring up a new copy.

Để tận dụng quản lý trạng thái mong muốn, nền tảng cần một cách nào đó để tự động khởi chạy các instance của microservice của bạn. Vậy nên việc có một quy trình triển khai hoàn toàn tự động cho các instance microservice là một điều kiện tiên quyết rõ ràng cho quản lý trạng thái mong muốn. Bạn cũng có thể cần suy nghĩ kỹ về việc các instance của bạn mất bao lâu để khởi chạy. Nếu bạn dùng quản lý trạng thái mong muốn để đảm bảo có đủ tài nguyên tính toán xử lý tải người dùng, thì nếu một instance chết, bạn sẽ muốn một instance thay thế càng nhanh càng tốt để lấp chỗ trống. Nếu việc cung cấp một instance mới mất nhiều thời gian, bạn có thể cần có năng lực dư thừa để xử lý tải trong trường hợp một instance chết, nhằm cho mình đủ khoảng thở để dựng lên một bản sao mới.

Although you could hack together a desired state management solution for yourself, I'm not convinced it's a good use of your time. If you want to embrace this concept, I think you are better off adopting a platform that embraces it as a first-class concept. As this means coming to grips with what might represent a new deployment platform and all the associated ideas and tooling, you might want to delay adopting desired state management until you already have a few microservices up and running. This will allow you to get familiar with the basics of microservices before becoming overloaded with new technology. Platforms like Kubernetes really help when you have lots of things to manage—if you only have a few processes to worry about, you could wait till later on to adopt these tools.

Mặc dù bạn có thể chấp vá một giải pháp quản lý trạng thái mong muốn cho riêng mình, tôi không tin đó là cách dùng thời gian của bạn một cách hợp lý. Nếu bạn muốn đón nhận khái niệm này, tôi nghĩ bạn nên áp dụng một nền tảng đón nhận nó như một khái niệm hạng nhất (first-class concept) thì hơn. Vì điều này nghĩa là phải vật lộn với cái có thể là một nền tảng triển khai mới cùng tất cả các ý tưởng và công cụ liên quan, bạn có thể muốn trì hoãn việc áp dụng quản lý trạng thái mong muốn cho đến khi bạn đã có sẵn vài microservice đang chạy. Điều này sẽ cho phép bạn làm quen với những điều cơ bản của microservices trước khi bị quá tải bởi công nghệ mới. Các nền tảng như Kubernetes thực sự giúp ích khi bạn có rất nhiều thứ để quản lý—nếu bạn chỉ có vài tiến trình cần lo, bạn có thể đợi đến sau này mới áp dụng những công cụ này.

GitOps

GitOps

GitOps, a fairly recent concept pioneered by Weaveworks, brings together the concepts of desired state management and infrastructure as code. GitOps was originally conceived in the context of working with Kubernetes, and this is where the related tooling is focused, although arguably it describes a **workflow** that others have used before.

GitOps, một khái niệm khá gần đây được Weaveworks tiên phong, gắn kết các khái niệm quản lý trạng thái mong muốn và infrastructure as code lại với nhau. GitOps ban đầu được hình thành trong bối cảnh làm việc với Kubernetes, và đây là nơi bộ công cụ liên quan tập trung vào, mặc dù có thể lập luận rằng nó mô tả một workflow mà những người khác đã dùng từ trước.

With GitOps, your desired state for your infrastructure is defined in code and stored in source control. When changes are made to this desired state, some tooling ensures that this updated desired state is applied to the running system. The idea is to give developers a simplified workflow for working with their applications.

Với GitOps, trạng thái mong muốn cho hạ tầng của bạn được định nghĩa trong mã và lưu trữ trong hệ thống quản lý mã nguồn. Khi có thay đổi đối với trạng thái mong muốn này, một bộ công cụ nào đó đảm bảo rằng trạng thái mong muốn đã cập nhật này được áp dụng vào hệ thống đang chạy. Ý tưởng là trao cho lập trình viên một workflow đơn giản hóa để làm việc với ứng dụng của họ.

If you've used infrastructure configuration tools like Chef or Puppet, this model is familiar for managing infrastructure. When using Chef Server or Puppet Master, you had a centralized system capable of pushing out changes dynamically when they were made. The shift with GitOps is that this tooling is making use of capabilities inside Kubernetes to help manage applications rather than just infrastructure.

Nếu bạn đã từng dùng các công cụ cấu hình hạ tầng như Chef hay Puppet, mô hình này quen thuộc cho việc quản lý hạ tầng. Khi dùng Chef Server hoặc Puppet Master, bạn có một hệ thống tập trung có khả năng đẩy các thay đổi ra một cách động khi chúng được thực hiện. Sự dịch chuyển với GitOps là bộ công cụ này tận dụng các khả năng bên trong Kubernetes để giúp quản lý ứng dụng thay vì chỉ hạ tầng.

Tools like Flux are making it much easier to embrace these ideas. It's worth noting, of course, that while tools can make it easier for you to change the way you work, they can't force you into adopting new working approaches. Put differently, just because you have Flux (or another GitOps tool), it doesn't mean you're embracing the ideas of desired state management or infrastructure as code.

Các công cụ như Flux đang làm cho việc đón nhận những ý tưởng này dễ dàng hơn nhiều. Tất nhiên, cũng đáng lưu ý rằng dù công cụ có thể giúp bạn thay đổi cách làm việc dễ dàng hơn, chúng không thể ép buộc bạn áp dụng các cách làm việc mới. Nói cách khác, chỉ vì bạn có Flux (hoặc một công cụ GitOps khác) không có nghĩa là bạn đang đón nhận các ý tưởng quản lý trạng thái mong muốn hay infrastructure as code.

If you're in the world of Kubernetes, adopting a tool like Flux and the workflows it promotes may well speed up the introduction of concepts like desired state management and infrastructure as code. Just make sure you don't lose sight of the goals of the underlying concepts and get blinded by all the new technology in this space!

Nếu bạn ở trong thế giới Kubernetes, việc áp dụng một công cụ như Flux và các workflow nó cổ vũ rất có thể đẩy nhanh việc đưa vào các khái niệm như quản lý trạng thái mong muốn và infrastructure as code. Chỉ cần đảm bảo bạn không đánh mất tầm nhìn về mục tiêu của các khái niệm nền tảng bên dưới và bị lóa mắt bởi tất cả công nghệ mới trong không gian này!

Deployment Options — Các Lựa chọn Triển khai

When it comes to the approaches and tooling we can use for our microservice workloads, we have loads of options. But we should look at these options in terms of the principles I just outlined. We want our microservices to run in an isolated fashion and to ideally be deployed in a way that avoids downtime. We want the tooling we pick to allow us to embrace a culture of automation, define our infrastructure and application configuration in code, and ideally also manage desired state for us.

Khi nói đến các phương pháp tiếp cận và công cụ mà ta có thể dùng cho các tải công việc microservice, ta có vô số lựa chọn. Nhưng ta nên nhìn vào những lựa chọn này dưới góc độ các nguyên tắc mà tôi vừa nêu. Ta muốn các microservice chạy theo kiểu cô lập và lý tưởng nhất là được triển khai theo cách tránh được gián đoạn. Ta muốn công cụ mình chọn cho phép ta đón nhận một văn hóa tự động hóa, định nghĩa cấu hình hạ tầng và ứng dụng của mình bằng mã, và lý tưởng nhất là cũng quản lý trạng thái mong muốn cho ta.

Let's briefly summarize the various deployment options before looking at how well they deliver on these ideas:

Hãy tóm tắt ngắn gọn các lựa chọn triển khai khác nhau trước khi xem chúng đáp ứng những ý tưởng này tốt đến đâu:

Physical machine A microservice instance is deployed directly onto a physical machine, with no virtualization.

Máy vật lý (physical machine) Một instance microservice được triển khai trực tiếp lên một máy vật lý, không có ảo hóa.

Virtual machine A microservice instance is deployed on to a virtual machine.

Máy ảo (virtual machine) Một instance microservice được triển khai lên một máy ảo.

Container A microservice instance runs as a separate container on a virtual or physical machine. That container runtime may be managed by a container orchestration tool like Kubernetes.

Container Một instance microservice chạy như một container riêng trên một máy ảo hoặc vật lý. Container runtime đó có thể được quản lý bởi một công cụ điều phối container (container orchestration) như Kubernetes.

Application container A microservice instance is run inside an application container that manages other application instances, typically on the same runtime.

Application container Một instance microservice được chạy bên trong một application container vốn quản lý các instance ứng dụng khác, thường trên cùng một runtime.

Platform as a Service (PaaS) A more highly abstracted platform is used to deploy microservice instances, often abstracting away all concepts of the underlying servers used to run your microservices. Examples include Heroku, Google App Engine, and AWS Beanstalk.

Platform as a Service (PaaS) Một nền tảng được trừu tượng hóa cao hơn được dùng để triển khai các instance microservice, thường trừu tượng hóa, gạt bỏ mọi khái niệm về các máy chủ bên dưới được dùng để chạy microservice của bạn. Ví dụ bao gồm Heroku, Google App Engine và AWS Beanstalk.

Function as a Service (FaaS) A microservice instance is deployed as one or more functions, which are run and managed by an underlying platform like AWS Lambda or Azure Functions. Arguably, FaaS is a specific type of PaaS, but it deserves exploration in its own right given the recent popularity of the idea and the questions it raises about mapping from a microservice to a deployed artifact.

Function as a Service (FaaS) Một instance microservice được triển khai dưới dạng một hoặc nhiều function, vốn được chạy và quản lý bởi một nền tảng bên dưới như AWS Lambda hay Azure Functions. Có thể lập luận rằng FaaS là một loại PaaS cụ thể, nhưng nó xứng đáng được khám phá theo cách riêng của nó, do sự phổ biến gần đây của ý tưởng và những câu hỏi mà nó đặt ra về việc ánh xạ từ một microservice sang một artifact đã triển khai.

Physical Machines — Máy Vật lý

An increasingly rare option, you may find yourself deploying microservices directly onto physical machines. By “directly,” I mean that there are no layers of virtualization or containerization between you and the underlying hardware. This has become less and less common for a few different reasons. Firstly, deploying directly onto physical hardware can lead to lower utilization across your estate. If I have a single instance of a microservice running on a physical machine and I use only half the CPU, memory, or I/O provided by the hardware, then the remaining resources are wasted. This problem has led to the virtualization of most computing infrastructure, allowing you to coexist multiple virtual machines on the same physical machine. It gives you much higher utilization of your infrastructure, which has some obvious benefits in terms of cost effectiveness.

Một lựa chọn ngày càng hiếm, bạn có thể thấy mình triển khai microservice trực tiếp lên các máy vật lý. Khi nói “trực tiếp”, tôi muốn nói là không có lớp ảo hóa hay container hóa nào giữa bạn và phần cứng bên dưới. Điều này đã ngày càng ít phổ biến vì một vài lý do khác nhau. Thứ nhất, triển khai trực tiếp lên phần cứng vật lý có thể dẫn đến việc sử dụng tài nguyên (utilization) thấp hơn trên toàn bộ hạ tầng của bạn. Nếu tôi có một instance microservice duy nhất chạy trên một máy vật lý và tôi chỉ dùng một nửa CPU, bộ nhớ hay I/O mà phần cứng cung cấp, thì các tài nguyên còn lại bị lãng phí. Vấn đề này đã dẫn đến việc ảo hóa hầu hết hạ tầng tính toán, cho phép bạn cho nhiều máy ảo cùng tồn tại trên cùng một máy vật lý. Nó cho bạn mức sử dụng tài nguyên hạ tầng cao hơn nhiều, điều này có một số lợi ích rõ ràng về mặt hiệu quả chi phí.

If you have direct access to physical hardware without the option for virtualization, the temptation is to then pack multiple microservices on the same machine—of course, this violates the principle we talked about regarding having an isolated execution environment for your services. You could use tools like Puppet or Chef to configure the machine—helping implement infrastructure as code. The problem is that if you are working only at the level of a single physical machine, implementing concepts like desired state management, zero-downtime deployment, and so on requires us to work at a higher level of abstraction, using some sort of management layer on top. These types of systems are more commonly used in conjunction with virtual machines, something we’ll explore further in a moment.

Nếu bạn có quyền truy cập trực tiếp vào phần cứng vật lý mà không có lựa chọn ảo hóa, thì sự cám dỗ là sẽ nhồi nhét nhiều microservice lên cùng một máy—tất nhiên, điều này vi phạm nguyên tắc mà ta đã nói về việc có một môi trường thực thi cô lập cho các service của bạn. Bạn có thể dùng các công cụ như Puppet hay Chef để cấu hình máy—giúp hiện thực hóa infrastructure as code. Vấn đề là nếu bạn chỉ làm việc ở cấp độ một máy vật lý đơn lẻ, việc hiện thực hóa các khái niệm như quản lý trạng thái mong muốn, triển khai không gián đoạn, v.v., đòi hỏi ta làm việc ở một mức trừu tượng cao hơn, dùng một dạng lớp quản lý nào đó ở trên cùng. Những loại hệ thống này phổ biến hơn khi được dùng kết hợp với máy ảo, thứ ta sẽ khám phá thêm trong giây lát.

In general, directly deploying microservices onto physical machines is something I almost never see nowadays, and you’ll likely need to have some very specific requirements (or constraints) in your situation to justify this approach over the increased flexibility that either virtualization or containerization may bring.

Nhìn chung, việc triển khai microservice trực tiếp lên máy vật lý là điều mà ngày nay tôi gần như không bao giờ thấy, và bạn có lẽ cần có một số yêu cầu (hoặc ràng buộc) rất cụ thể trong tình huống của mình để biện minh cho cách tiếp cận này so với sự linh hoạt gia tăng mà ảo hóa hoặc container hóa có thể mang lại.

Virtual Machines — Máy Ảo

Virtualization has transformed data centers, by allowing us to chunk up existing physical machines into smaller, virtual machines. Traditional virtualization like VMware or that used by the main cloud providers, managed virtual machine infrastructure (such as AWS's EC2 service) has yielded huge benefits in increasing the utilization of computing infrastructure, while at the same time reducing the overhead of host management.

Ảo hóa đã biến đổi các trung tâm dữ liệu, bằng cách cho phép ta chia nhỏ các máy vật lý hiện có thành các máy ảo nhỏ hơn. Ảo hóa truyền thống như VMware hay loại được các nhà cung cấp đám mây lớn dùng, hạ tầng máy ảo được quản lý (chẳng hạn như dịch vụ EC2 của AWS) đã mang lại lợi ích to lớn trong việc tăng mức sử dụng hạ tầng tính toán, đồng thời giảm chi phí quản lý host.

Fundamentally, virtualization allows you to split up an underlying machine into multiple smaller “virtual” machines that act just like normal servers to the software running inside the virtual machines. You can assign portions of the underlying CPU, memory, I/O, and storage capability to each virtual machine, which in our context allows you to cram many more isolated execution environments for your microservice instances onto a single physical machine.

Về cơ bản, ảo hóa cho phép bạn chia một máy bên dưới thành nhiều máy “ảo” nhỏ hơn, hành xử y như các máy chủ bình thường đối với phần mềm chạy bên trong các máy ảo. Bạn có thể gán các phần CPU, bộ nhớ, I/O và khả năng lưu trữ bên dưới cho mỗi máy ảo, điều mà trong ngữ cảnh của ta cho phép bạn nhồi nhét nhiều môi trường thực thi cô lập hơn cho các instance microservice của mình lên một máy vật lý đơn lẻ.

Each virtual machine contains a full operating system and set of resources that can be used by the software running inside the VM. This ensures that you have a very good degree of isolation between instances when each instance is deployed onto a separate VM. Each microservice instance can fully configure the operating system in the VM to its own local needs. We still have the issue, though, that if the underlying hardware running these virtual machines fails, we can lose multiple microservice instances. There are ways to help solve that particular problem, including things like desired state management, which we discussed earlier.

Mỗi máy ảo chứa một hệ điều hành đầy đủ và một tập tài nguyên có thể được dùng bởi phần mềm chạy bên trong VM. Điều này đảm bảo bạn có một mức độ cô lập rất tốt giữa các instance khi mỗi instance được triển khai lên một VM riêng. Mỗi instance microservice có thể cấu hình đầy đủ hệ điều hành trong VM theo nhu cầu cục bộ riêng của nó. Tuy nhiên, ta vẫn có vấn đề là nếu phần cứng bên dưới chạy các máy ảo này hỏng, ta có thể mất nhiều instance microservice. Có những cách giúp giải quyết vấn đề cụ thể đó, bao gồm những thứ như quản lý trạng thái mong muốn, mà ta đã bàn trước đó.

Cost of virtualization

Chi phí của ảo hóa

As you pack more and more virtual machines onto the same underlying hardware, you will find that you get diminishing returns in terms of the computing resources available to the VMs themselves. Why is this?

Khi bạn nhồi nhét ngày càng nhiều máy ảo lên cùng một phần cứng bên dưới, bạn sẽ thấy rằng mình nhận được lợi ích giảm dần xét về tài nguyên tính toán sẵn có cho bản thân các VM. Tại sao lại thế?

Think of our physical machine as a sock drawer. If we put lots of wooden dividers into our drawer,

can we store more socks or fewer? The answer is fewer: the dividers themselves take up room too! Our drawer might be easier to deal with and organize, and perhaps we could decide to put T-shirts in one of the spaces now rather than just socks, but more dividers means less overall space.

Hãy nghĩ máy vật lý của ta như một ngăn kéo đựng tất. Nếu ta bỏ nhiều vách ngăn gỗ vào ngăn kéo, ta có thể chứa nhiều tất hơn hay ít hơn? Câu trả lời là ít hơn: chính các vách ngăn cũng chiếm chỗ! Ngăn kéo của ta có thể dễ xử lý và sắp xếp hơn, và có lẽ ta có thể quyết định bỏ áo phông vào một trong các khoảng trống thay vì chỉ tất, nhưng nhiều vách ngăn hơn nghĩa là ít không gian tổng thể hơn.

In the world of virtualization, we have a similar overhead as our sock drawer dividers. To understand where this overhead comes from, let's look at how most virtualization is done. Figure 8-14 shows a comparison of two types of virtualization. On the left, we see the various layers involved in what is called type 2 virtualization, and on the right we see container-based virtualization, which we'll explore more shortly.

Trong thế giới ảo hóa, ta có một chi phí phụ trội (overhead) tương tự như các vách ngăn của ngăn kéo tất. Để hiểu chi phí này đến từ đâu, hãy xem cách hầu hết việc ảo hóa được thực hiện. Hình 8-14 cho thấy sự so sánh hai loại ảo hóa. Bên trái, ta thấy các lớp khác nhau liên quan đến cái được gọi là ảo hóa loại 2 (type 2 virtualization), và bên phải ta thấy ảo hóa dựa trên container (container-based virtualization), thứ ta sẽ khám phá thêm ngay sau đây.

Type 2 virtualization is the sort implemented by AWS, VMware, vSphere, Xen, and KVM. (Type 1 virtualization refers to technology in which the VMs run directly on hardware, not on top of another operating system.) On our physical infrastructure we have a host operating system. On this OS we run something called a hypervisor, which has two key jobs. First, it maps resources like CPU and memory from the virtual host to the physical host. Second, it acts as a control layer, allowing us to manipulate the virtual machines themselves.

Ảo hóa loại 2 là loại được hiện thực bởi AWS, VMware, vSphere, Xen và KVM. (Ảo hóa loại 1 chỉ công nghệ mà trong đó các VM chạy trực tiếp trên phần cứng, không phải trên một hệ điều hành khác.) Trên hạ tầng vật lý của ta có một hệ điều hành chủ (host operating system). Trên OS này ta chạy một thứ gọi là hypervisor, vốn có hai nhiệm vụ then chốt. Thứ nhất, nó ánh xạ các tài nguyên như CPU và bộ nhớ từ host ảo sang host vật lý. Thứ hai, nó hoạt động như một lớp điều khiển, cho phép ta thao tác chính các máy ảo.

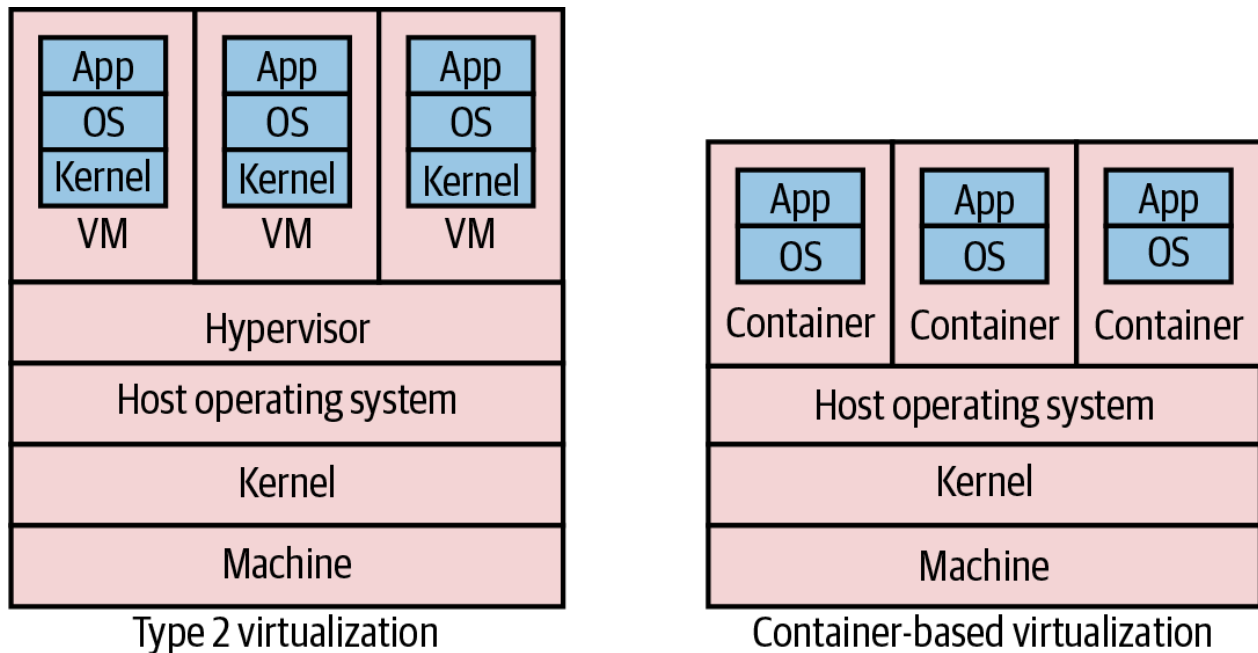


Figure 8-14. Comparison of standard type 2 virtualization and lightweight containers

Hình 8-14. So sánh giữa ảo hóa loại 2 tiêu chuẩn và container nhẹ

Inside the VMs, we get what look like completely different hosts. They can run their own operating systems, with their own kernels. They can be considered almost hermetically sealed machines, kept isolated from the underlying physical host and the other virtual machines by the hypervisor.

Bên trong các VM, ta nhận được những thứ trông như các host hoàn toàn khác biệt. Chúng có thể chạy hệ điều hành riêng, với kernel riêng. Chúng có thể được coi là các máy gần như được niêm phong kín khí, được giữ cô lập khỏi host vật lý bên dưới và các máy ảo khác bởi hypervisor.

The problem with type 2 virtualization is that the hypervisor here needs to set aside resources to do its job. This takes away CPU, I/O, and memory that could be used elsewhere. The more hosts the hypervisor manages, the more resources it needs. At a certain point, this overhead becomes a constraint in slicing up your physical infrastructure any further. In practice, this means that there are often diminishing returns in slicing up a physical box into smaller and smaller parts, as proportionally more and more resources go into the overhead of the hypervisor.

Vấn đề với ảo hóa loại 2 là hypervisor ở đây cần dành ra tài nguyên để làm việc của nó. Điều này lấy đi CPU, I/O và bộ nhớ vốn có thể được dùng ở nơi khác. Hypervisor quản lý càng nhiều host, nó càng cần nhiều tài nguyên. Đến một điểm nhất định, chi phí phụ trội này trở thành một ràng buộc trong việc chia nhỏ hạ tầng vật lý của bạn thêm nữa. Trên thực tế, điều này có nghĩa là thường có lợi ích giảm dần trong việc chia một máy vật lý thành những phần ngày càng nhỏ hơn, vì tỷ lệ tài nguyên ngày càng nhiều đổ vào chi phí phụ trội của hypervisor.

Good for microservices?

Có tốt cho microservices?

Coming back to our principles, virtual machines do very well in terms of isolation, but at a cost. Their ease of automation can vary based on the exact technology being used—managed VMs on Google Cloud, Azure, or AWS, for example, are all easy to automate via well-supported APIs and

an ecosystem of tools that build on these APIs. In addition, these platforms provide concepts like autoscaling groups, helping implement desired state management. Implementing zero-downtime deployment is going to take more work, but if the VM platform you are using gives you a good API, the building blocks are there. The issue is that many people are making use of managed VMs provided by traditional virtualization platforms like the ones provided by VMware, which, while they may theoretically allow for automation, are typically not used in this context. Instead these platforms tend to be under the central control of a dedicated operations team, and the ability to directly automate against them can be restricted as a result.

Quay lại với các nguyên tắc của ta, máy ảo làm rất tốt về mặt cô lập, nhưng có cái giá của nó. Mức độ dễ tự động hóa của chúng có thể thay đổi tùy theo công nghệ chính xác được dùng—chẳng hạn các VM được quản lý trên Google Cloud, Azure hay AWS đều dễ tự động hóa qua các API được hỗ trợ tốt và một hệ sinh thái công cụ xây dựng trên các API này. Thêm vào đó, các nền tảng này cung cấp các khái niệm như autoscaling group, giúp hiện thực hóa quản lý trạng thái mong muốn. Việc hiện thực hóa triển khai không gián đoạn sẽ cần nhiều công sức hơn, nhưng nếu nền tảng VM bạn đang dùng cho bạn một API tốt, thì các viên gạch xây dựng đã có sẵn ở đó. Vấn đề là nhiều người đang dùng các VM được quản lý cung cấp bởi các nền tảng ảo hóa truyền thống như những nền tảng do VMware cung cấp, vốn về lý thuyết có thể cho phép tự động hóa nhưng thường không được dùng theo ngữ cảnh này. Thay vào đó các nền tảng này có xu hướng nằm dưới sự kiểm soát tập trung của một đội vận hành chuyên trách, và khả năng tự động hóa trực tiếp đối với chúng do đó có thể bị hạn chế.

Although containers are proving to be more popular in general for microservice workloads, many organizations have used virtual machines for running large-scale microservice systems, to great effect. Netflix, one of the poster children for microservices, built out much of its microservices on top of AWS's managed virtual machines via EC2. If you need the stricter isolation levels that they can bring, or you don't have the ability to containerize your application, VMs can be a great choice.

Mặc dù container đang chứng tỏ phổ biến hơn nói chung cho các tải công việc microservice, nhiều tổ chức đã dùng máy ảo để chạy các hệ thống microservice quy mô lớn, với hiệu quả tuyệt vời. Netflix, một trong những hình mẫu cho microservices, đã xây dựng phần lớn microservice của mình trên nền các máy ảo được quản lý của AWS qua EC2. Nếu bạn cần các mức cô lập nghiêm ngặt hơn mà chúng có thể mang lại, hoặc bạn không có khả năng container hóa ứng dụng của mình, VM có thể là một lựa chọn tuyệt vời.

Containers — Container

Since the first edition of this book, containers have become a dominant concept in server-side software deployment and for many are the de facto choice for packaging and running microservice architectures. The container concept, popularized by Docker, and allied with a supporting container orchestration platform like Kubernetes, has become many people's go-to choice for running microservice architectures at scale.

Kể từ ấn bản đầu tiên của cuốn sách này, container đã trở thành một khái niệm thống trị trong triển khai phần mềm phía máy chủ và với nhiều người là lựa chọn mặc định (de facto) để đóng gói và chạy các kiến trúc microservice. Khái niệm container, được Docker phổ biến, kết hợp với một nền tảng điều phối container hậu thuẫn như Kubernetes, đã trở thành lựa chọn ưu tiên của nhiều người để chạy các kiến trúc microservice ở quy mô lớn.

Before we get to why this has happened and to the relationship between containers, Kubernetes, and

Docker, we should first explore what a container is exactly, and look specifically at how it differs from virtual machines.

Trước khi đi đến tại sao điều này xảy ra và đến mối quan hệ giữa container, Kubernetes và Docker, đầu tiên ta nên khám phá chính xác một container là gì, và xem cụ thể nó khác máy ảo ra sao.

Isolated, differently

Cô lập, theo một cách khác

Containers first emerged on UNIX-style operating systems and for many years were really only a viable prospect on those operating systems, such as Linux. Although Windows containers are very much a thing, it has been Linux operating systems that containers have had the biggest impact on so far.

Container đầu tiên xuất hiện trên các hệ điều hành kiểu UNIX và trong nhiều năm thực sự chỉ là một triển vọng khả thi trên các hệ điều hành đó, chẳng hạn như Linux. Mặc dù Windows container rất thực sự tồn tại, chính các hệ điều hành Linux mới là nơi container có tác động lớn nhất cho đến nay.

On Linux, processes are run by a given user and have certain capabilities based on how the permissions are set. Processes can spawn other processes. For example, if I launch a process in a terminal, that process is generally considered a child of the terminal process. The Linux kernel's job is maintaining this tree of processes, ensuring that only permitted users can access the processes. Additionally, the Linux kernel is capable of assigning resources to these different processes—this is all part and parcel of building a viable multiuser operating system, where you don't want the activities of one user to kill the **rest** of the system.

Trên Linux, các tiến trình được chạy bởi một người dùng nhất định và có những khả năng nhất định dựa trên cách thiết lập quyền. Các tiến trình có thể sinh ra các tiến trình khác. Ví dụ, nếu tôi khởi chạy một tiến trình trong terminal, tiến trình đó thường được coi là con của tiến trình terminal. Công việc của Linux kernel là duy trì cây tiến trình này, đảm bảo rằng chỉ những người dùng được phép mới có thể truy cập các tiến trình. Thêm vào đó, Linux kernel có khả năng gán tài nguyên cho các tiến trình khác nhau này—đây hoàn toàn là một phần của việc xây dựng một hệ điều hành đa người dùng khả thi, nơi bạn không muốn hoạt động của một người dùng giết chết phần còn lại của hệ thống.

Containers running on the same machine make use of the same underlying kernel (although there are exceptions to this rule that we'll explore shortly). Rather than managing processes directly, you can think of a container as an abstraction over a subtree of the overall system process tree, with the kernel doing all the hard work. These containers can have physical resources allocated to them, something the kernel handles for us. This general approach has been around in many forms, such as Solaris Zones and OpenVZ, but it was with LXC that this idea made its way into the mainstream of Linux operating systems. The concept of Linux containers was further advanced when Docker provided yet a higher level of abstraction over containers, initially using LXC under the hood and then replacing it altogether.

Các container chạy trên cùng một máy dùng chung cùng kernel bên dưới (mặc dù có những ngoại lệ với quy tắc này mà ta sẽ khám phá ngay sau đây). Thay vì quản lý các tiến trình một cách trực tiếp, bạn có thể nghĩ về một container như một sự trừu tượng hóa trên một cây con của cây tiến trình tổng thể của hệ thống, với kernel làm tất cả việc nặng nhọc. Các container này có thể được cấp phát tài nguyên vật lý, một việc mà kernel xử lý cho ta. Cách tiếp cận tổng quát này đã tồn tại dưới nhiều hình thức, chẳng hạn như Solaris Zones và OpenVZ, nhưng chính với LXC mà ý tưởng này đã đi vào dòng chính của các hệ điều hành

Linux. Khái niệm Linux container được phát triển thêm khi Docker cung cấp một mức trừu tượng hóa cao hơn nữa trên container, ban đầu dùng LXC ở phía dưới rồi sau đó thay thế nó hoàn toàn.

If we look at the stack diagram for a host running a container in Figure 8-14, we see a few differences when comparing it with type 2 virtualization. First, we don't need a hypervisor. Second, the container doesn't seem to have a kernel—that's because it makes use of the kernel of the underlying machine. In Figure 8-15 we see this more clearly. A container can run its own operating system, but that operating system makes use of a part of the shared kernel—it's in this kernel that the process tree for each container lives. This means that our host operating system could run Ubuntu, and our containers CentOS, as long as they could both run as part of the same underlying kernel.

Nếu ta nhìn vào sơ đồ ngăn xếp (stack diagram) cho một host chạy container trong Hình 8-14, ta thấy vài khác biệt khi so với ảo hóa loại 2. Thứ nhất, ta không cần một hypervisor. Thứ hai, container có vẻ không có một kernel—đó là vì nó tận dụng kernel của máy bên dưới. Trong Hình 8-15 ta thấy điều này rõ hơn. Một container có thể chạy hệ điều hành riêng của nó, nhưng hệ điều hành đó tận dụng một phần của kernel dùng chung—chính trong kernel này mà cây tiến trình cho mỗi container tồn tại. Điều này nghĩa là hệ điều hành chủ của ta có thể chạy Ubuntu, còn các container của ta chạy CentOS, miễn là cả hai đều có thể chạy như một phần của cùng kernel bên dưới.

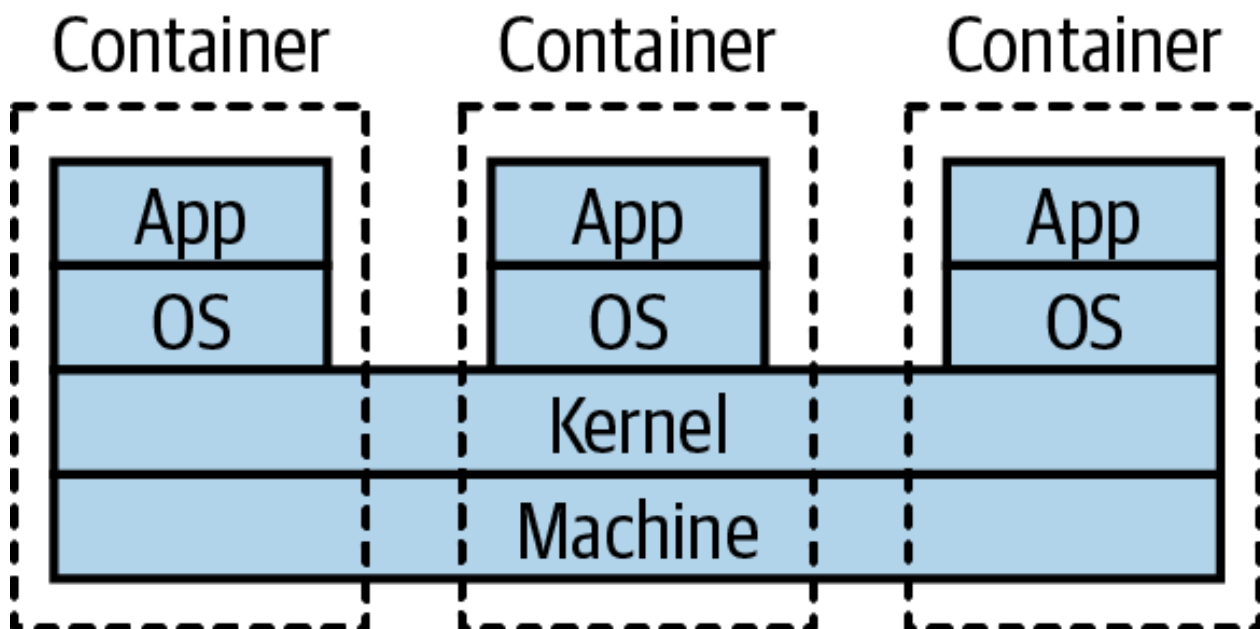


Figure 8-15. Normally, containers on the same machine share the same kernel. With containers, we don't just benefit from the resources saved by not needing a hypervisor; we also gain in terms of feedback. Linux containers are much faster to provision than full-fat virtual machines. It isn't uncommon for a VM to take many minutes to start—but with Linux containers, startup can take just a few seconds. You also have finer-grained control over the containers themselves in terms of assigning resources to them, which makes it much easier to tweak the settings to get the most out of the underlying hardware.

Hình 8-15. Thông thường, các container trên cùng một máy dùng chung cùng kernel. Với container, ta không chỉ hưởng lợi từ tài nguyên tiết kiệm được nhờ không cần hypervisor; ta còn được lợi về mặt phản hồi. Linux container nhanh hơn nhiều để cung cấp so với các máy ảo đầy đủ. Không hiếm khi một VM mất nhiều phút để khởi động—nhưng với Linux

container, việc khởi động có thể chỉ mất vài giây. Bạn cũng có quyền kiểm soát chi tiết hơn đối với chính các container về mặt gán tài nguyên cho chúng, điều này làm cho việc tinh chỉnh các thiết lập để tận dụng tối đa phần cứng bên dưới dễ dàng hơn nhiều.

Due to the more lightweight nature of containers, we can have many more of them running on the same hardware than would be possible with VMs. By deploying one service per container, as in Figure 8-16 , we get a degree of isolation from other containers (although this isn't perfect) and can do so much more cost-effectively than would be possible if we wanted to run each service in its own VM. Coming back to our sock drawer analogy from earlier, with containers the sock drawer dividers are much thinner than they are for VMs, meaning a higher proportion of the sock drawer gets used for socks.

Do bản chất nhẹ hơn của container, ta có thể có nhiều container chạy trên cùng phần cứng hơn so với mức có thể với VM. Bằng cách triển khai một service trên mỗi container, như trong Hình 8-16, ta có được một mức độ cô lập khỏi các container khác (mặc dù điều này không hoàn hảo) và có thể làm vậy hiệu quả về chi phí hơn nhiều so với mức có thể nếu ta muốn chạy mỗi service trong VM riêng của nó. Quay lại với phép loại suy ngăn kéo tất trước đó, với container các vách ngăn của ngăn kéo tất mỏng hơn nhiều so với với VM, nghĩa là một tỷ lệ ngăn kéo tất lớn hơn được dùng để đựng tất.

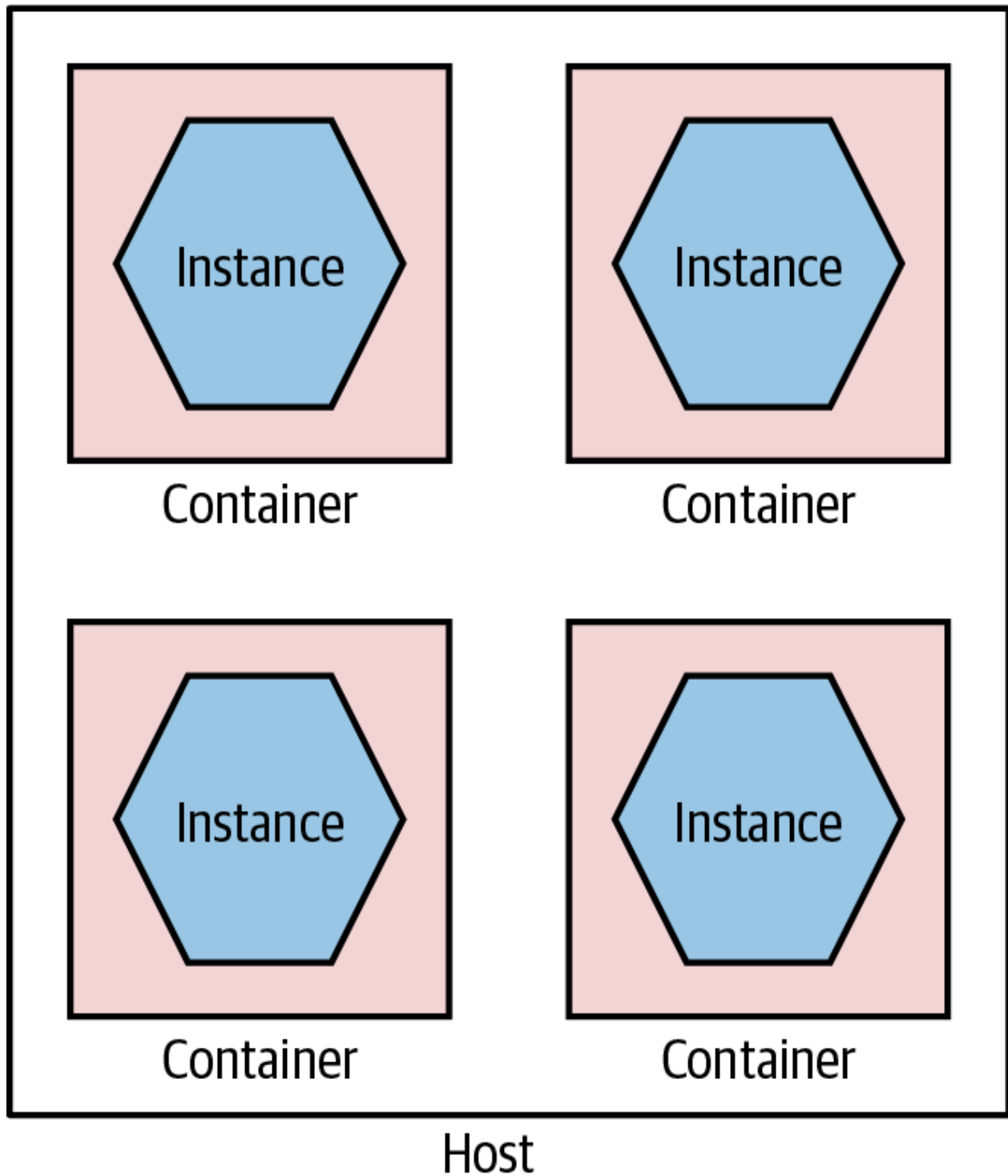


Figure 8-16. Running services in separate containers

Hình 8-16. Chạy các service trong các container riêng biệt

Containers can be used well with full-fat virtualization too; in fact, this is common. I've seen more than one project provision a large AWS EC2 instance and run multiple containers on it to get the best of both worlds: an on-demand ephemeral compute platform in the form of EC2, coupled with highly flexible and fast containers running on top of it.

Container cũng có thể được dùng kết hợp tốt với ảo hóa đầy đủ; thực tế, điều này là phổ biến. Tôi đã thấy hơn một dự án cung cấp một instance AWS EC2 lớn và chạy nhiều container trên đó để có được điều tốt nhất của cả hai thế giới: một nền tảng tính toán phù du (ephemeral) theo yêu cầu dưới dạng EC2, kết hợp với các container linh hoạt và nhanh cao chạy trên đó.

Not perfect

Không hoàn hảo

Linux containers aren't without some problems, however. Imagine I have lots of microservices running in their own containers on a host. How does the outside world see them? You need some way to route the outside world through to the underlying containers, something many of the hypervisors do for you with normal virtualization. With earlier technology like LXC, this was something you had to handle yourself—this is one area where Docker's take on containers has helped hugely.

Tuy nhiên, Linux container không phải không có một số vấn đề. Hãy hình dung tôi có rất nhiều microservice chạy trong các container riêng của chúng trên một host. Thế giới bên ngoài thấy chúng ra sao? Bạn cần một cách nào đó để định tuyến thế giới bên ngoài đến tận các container bên dưới, một thứ mà nhiều hypervisor làm cho bạn với ảo hóa thông thường. Với công nghệ trước đây như LXC, đây là thứ bạn phải tự xử lý—đây là một lĩnh vực mà cách tiếp cận container của Docker đã giúp đỡ rất nhiều.

Another point to bear in mind is that these containers can be considered isolated from a resource point of view—I can allocate ring-fenced sets of CPU, memory, and so on to each container—but this is not necessarily the same degree of isolation as you get from virtual machines, or for that matter by having separate physical machines. Early on, there were a number of documented and known ways in which a process from one container could bust out and interact with other containers or the underlying host.

Một điểm khác cần lưu ý là các container này có thể được coi là cô lập từ góc độ tài nguyên—tôi có thể cấp phát các tập CPU, bộ nhớ, v.v. được khoanh vùng cho mỗi container—nhưng đây không nhất thiết là cùng mức độ cô lập mà bạn nhận được từ máy ảo, hay nói chuyện đó, từ việc có các máy vật lý riêng biệt. Vào thời kỳ đầu, có một số cách đã được ghi nhận và biết đến trong đó một tiến trình từ một container có thể thoát ra và tương tác với các container khác hoặc với host bên dưới.

A huge amount of work has gone into resolving these issues, and the container orchestration systems and underlying container runtimes have done a good job of examining how to better run container workloads so this isolation is improved, but you will need to give due thought to the sorts of workloads you want to run. My own guidance here is that in general you should view containers as a great way of isolating execution of trusted software. If you are running code written by others and are concerned about a malicious party trying to bypass container-level isolation, then you'll want to do some deeper examination yourself regarding the current state of the art for handling such situations—some of which we'll touch on in a moment.

Một khối lượng công việc khổng lồ đã được đổ vào để giải quyết các vấn đề này, và các hệ thống điều phối container cùng các container runtime bên dưới đã làm tốt việc xem xét cách chạy các tải công việc container tốt hơn để sự cô lập này được cải thiện, nhưng bạn sẽ cần suy nghĩ thấu đáo về loại tải công việc bạn muốn chạy. Hướng dẫn của riêng tôi ở đây là nhìn chung bạn nên xem container như một cách tuyệt vời để cô lập việc thực thi phần mềm đáng tin cậy. Nếu bạn đang chạy mã do người khác viết và lo ngại một bên có ác ý cố vượt qua sự cô lập ở cấp container, thì bạn sẽ muốn tự mình kiểm tra sâu hơn về tình trạng tân tiến hiện thời của việc xử lý các tình huống như vậy—một số trong đó ta sẽ

đề cập trong giây lát.

Windows containers

Windows container

Historically, Windows users would look longingly at their Linux-using contemporaries, as containers were something denied to the Windows operating system. Over the last few years, however, this has changed, with containers now being a fully supported concept. The delay was really about the underlying Windows operating system and kernel supporting the same kinds of capabilities as existed in the land of Linux to make containers work. It was with the delivery of Windows Server 2016 that a lot of this changed, and since then Windows containers have continued to evolve.

Về mặt lịch sử, người dùng Windows từng nhìn đầy ao ước về phía những người cùng thời dùng Linux, vì container là thứ bị từ chối với hệ điều hành Windows. Tuy nhiên, trong vài năm qua, điều này đã thay đổi, với container giờ đây là một khái niệm được hỗ trợ đầy đủ. Sự chậm trễ thực sự nằm ở việc hệ điều hành và kernel Windows bên dưới hỗ trợ cùng loại khả năng đã tồn tại trong vùng đất Linux để làm cho container hoạt động. Chính với sự ra đời của Windows Server 2016 mà phần lớn điều này đã thay đổi, và kể từ đó Windows container đã tiếp tục tiến hóa.

One of the initial stumbling blocks in the adoption of Windows containers has been the size of the Windows operating system itself. Remember that you need to run an operating system inside each container, so when downloading a container image, you're also downloading an operating system. Windows, though, is big—so big that it made containers very heavy, not just in terms of the size of the images but also in terms of the resources required to run them.

Một trong những trở ngại ban đầu trong việc áp dụng Windows container là kích thước của chính hệ điều hành Windows. Hãy nhớ rằng bạn cần chạy một hệ điều hành bên trong mỗi container, nên khi tải xuống một container image, bạn cũng đang tải xuống một hệ điều hành. Windows, dù vậy, thì lớn—lớn đến mức nó làm cho container rất nặng nề, không chỉ về kích thước của các image mà còn về tài nguyên cần để chạy chúng.

Microsoft reacted to this by creating a cut-down operating system called Windows Nano Server. The idea is that Nano Server should have a small-footprint OS and be capable of running things like microservice instances. Alongside this, Microsoft also support a larger Windows Server Core, which is there to support running legacy Windows applications as containers. The issue is that these things are still pretty big when compared to their Linux equivalents—early versions of Nano Server would still be well over 1 GB in size, compared to small-footprint Linux operating systems like Alpine that would take up only a few megabytes.

Microsoft đã phản ứng với điều này bằng cách tạo ra một hệ điều hành rút gọn gọi là Windows Nano Server. Ý tưởng là Nano Server nên có một OS dấu chân nhỏ (small-footprint) và có khả năng chạy những thứ như các instance microservice. Song song với đó, Microsoft cũng hỗ trợ một Windows Server Core lớn hơn, vốn dùng để hỗ trợ chạy các ứng dụng Windows kế thừa (legacy) dưới dạng container. Vấn đề là những thứ này vẫn khá lớn so với các bản tương đương Linux của chúng—các phiên bản Nano Server ban đầu vẫn có kích thước hơn 1 GB, so với các hệ điều hành Linux dấu chân nhỏ như Alpine vốn chỉ chiếm vài megabyte.

While Microsoft has continued to try and reduce the size of Nano Server, this size disparity still exists. In practice, though, due to the way that common layers across container images can be cached, this may not be a massive issue.

Mặc dù Microsoft đã tiếp tục cố giảm kích thước của Nano Server, sự chênh lệch kích thước

này vẫn tồn tại. Tuy nhiên, trên thực tế, do cách mà các lớp chung giữa các container image có thể được cache, đây có thể không phải là một vấn đề quá lớn.

Of special interest in the world of Windows containers is the fact that they support different levels of isolation. A standard Windows container uses process isolation, much like its Linux counterparts. With process isolation, each container runs in part of the same underlying kernel, which manages the isolation between the containers. With Windows containers, you also have the option of providing more isolation by running containers inside their own Hyper-V VM. This gives you something closer to the isolation level of full virtualization, but the nice thing is that you can choose between Hyper-V or process isolation when you launch the container—the image doesn’t need to change.

Đặc biệt đáng quan tâm trong thế giới Windows container là việc chúng hỗ trợ các mức cô lập khác nhau. Một Windows container tiêu chuẩn dùng cô lập tiến trình (process isolation), khá giống với các bản tương đương Linux của nó. Với cô lập tiến trình, mỗi container chạy trong một phần của cùng kernel bên dưới, vốn quản lý sự cô lập giữa các container. Với Windows container, bạn cũng có lựa chọn cung cấp thêm cô lập bằng cách chạy các container bên trong VM Hyper-V riêng của chúng. Điều này cho bạn một thứ gần hơn với mức cô lập của ảo hóa đầy đủ, nhưng điều hay là bạn có thể chọn giữa cô lập Hyper-V hay cô lập tiến trình khi bạn khởi chạy container—image không cần thay đổi.

Having flexibility about running images in different types of isolation can have its benefits. In some situations, your threat model may dictate that you want stronger isolation between your running processes than simple process-level isolation. For example, you might be running “untrusted” third-party code alongside your own processes. In such a situation, being able to run those container workloads as Hyper- V containers is very useful. Note, of course, that Hyper-V isolation is likely to have an impact in terms of spin-up time and a runtime cost closer to that of normal virtualization.

Việc có sự linh hoạt trong việc chạy image ở các loại cô lập khác nhau có thể có lợi ích của nó. Trong một số tình huống, mô hình mối đe dọa (threat model) của bạn có thể đòi hỏi rằng bạn muốn sự cô lập mạnh hơn giữa các tiến trình đang chạy so với cô lập ở cấp tiến trình đơn giản. Ví dụ, bạn có thể đang chạy mã bên thứ ba “không đáng tin cậy” cùng với các tiến trình của riêng mình. Trong một tình huống như vậy, khả năng chạy các tải công việc container đó dưới dạng container Hyper-V là rất hữu ích. Tất nhiên cũng lưu ý rằng cô lập Hyper-V nhiều khả năng có tác động về mặt thời gian khởi tạo và một chi phí runtime gần hơn với chi phí của ảo hóa thông thường.

Blurred Lines — Ranh giới Mờ nhạt

There is a growing trend of people looking for solutions that provide the stronger isolation provided by VMs while having the lightweight nature of containers. Examples include Microsoft’s Hyper-V containers, which allow for separate kernels, and Firecracker , which is confusingly called a kernel-based VM. Firecracker has proved popular as an implementation detail of service offerings like AWS Lambda, where there is a need to fully isolate workloads from different customers while still trying to keep spin-up time down and reduce the operational footprint of the workloads.

Có một xu hướng ngày càng tăng về việc mọi người tìm kiếm các giải pháp cung cấp sự cô lập mạnh hơn do VM mang lại trong khi vẫn có bản chất nhẹ của container. Ví dụ bao gồm Hyper-V container của Microsoft, vốn cho phép có các kernel riêng biệt, và Firecracker, thứ một cách khó hiểu được gọi là một VM dựa trên kernel (kernel-based VM). Firecracker đã chứng tỏ phổ biến như một chi tiết hiện thực của các dịch vụ như AWS Lambda, nơi có nhu cầu cô lập hoàn toàn các tải công việc từ các khách hàng khác nhau trong khi vẫn cố giữ thời gian khởi tạo thấp và giảm dấu chân vận hành của các tải công việc.

Docker

Docker

Containers were in limited use before the emergence of Docker pushed the concept mainstream. The Docker toolchain handles much of the work around containers. Docker manages the container provisioning, handles some of the networking problems for you, and even provides its own registry concept that allows you to store Docker applications. Before Docker, we didn't have the concept of an "image" for containers—this aspect, along with a much nicer set of tools for working with containers, helped containers become much easier to use.

Container đã được dùng hạn chế trước khi sự xuất hiện của Docker đẩy khái niệm này vào dòng chính. Bộ công cụ Docker xử lý phần lớn công việc quanh container. Docker quản lý việc cung cấp container, xử lý một số vấn đề mạng cho bạn, và thậm chí cung cấp khái niệm registry riêng cho phép bạn lưu trữ các ứng dụng Docker. Trước Docker, ta không có khái niệm "image" cho container—khía cạnh này, cùng với một bộ công cụ tốt hơn nhiều để làm việc với container, đã giúp container trở nên dễ dùng hơn nhiều.

The Docker image abstraction is a useful one for us, as the details of how our microservice is implemented are hidden. We have the builds for our microservice create a Docker image as a build artifact and store the image in the Docker registry, and away we go. When you launch an instance of a Docker image, you have a generic set of tools for managing that instance, no matter the underlying technology used—microservices written in Go, Python, NodeJS, or whatever can all be treated the same.

Sự trừu tượng hóa Docker image là một thứ hữu ích cho ta, vì các chi tiết về cách microservice của ta được hiện thực bị ẩn đi. Ta cho các bản build cho microservice của mình tạo ra một Docker image như một artifact build và lưu trữ image trong Docker registry, và thế là xong. Khi bạn khởi chạy một instance của một Docker image, bạn có một bộ công cụ chung để quản lý instance đó, bất kể công nghệ bên dưới được dùng—microservice viết bằng Go, Python, NodeJS hay bất cứ gì đều có thể được xử lý như nhau.

Docker can also alleviate some of the downsides of running lots of services locally for dev and test purposes. Previously, I might have used a tool like Vagrant that allows me to host multiple independent VMs on my development machine. This would allow me to have a production-like VM running my service instances locally. This was a pretty heavyweight approach, though, and I'd be limited in how many VMs I could run. With Docker, it's easy just to run Docker directly on my developer machine, probably using Docker Desktop. Now I can just build a Docker image for my microservice instance, or pull down a prebuilt image, and run it locally. These Docker images can (and should) be identical to the container image that I will eventually run in production.

Docker cũng có thể giảm bớt một số nhược điểm của việc chạy nhiều service cục bộ cho mục đích phát triển và kiểm thử. Trước đây, tôi có thể đã dùng một công cụ như Vagrant cho phép tôi lưu trữ nhiều VM độc lập trên máy phát triển của mình. Điều này cho phép tôi có một VM giống production chạy các instance service của tôi cục bộ. Tuy nhiên, đây là một cách tiếp cận khá nặng nề, và tôi sẽ bị hạn chế về số lượng VM tôi có thể chạy. Với Docker, dễ dàng chỉ cần chạy Docker trực tiếp trên máy phát triển của tôi, có lẽ dùng Docker Desktop. Giờ tôi có thể chỉ cần build một Docker image cho instance microservice của mình, hoặc kéo về một image dựng sẵn, và chạy nó cục bộ. Các Docker image này có thể (và nên) giống hệt với container image mà tôi cuối cùng sẽ chạy trong production.

When Docker first emerged, its scope was limited to managing containers on one machine. This was of limited use—what if you wanted to manage containers across multiple machines? This is

something that is essential if you want to maintain system health, if you have a machine die on you, or if you just want to run enough containers to handle the system's load. Docker came out with two totally different products of its own to solve this problem, confusingly called “Docker Swarm” and “Docker Swarm Mode”—who said naming stuff was hard again? Really, though, when it comes to managing lots of containers across many machines, Kubernetes is king here, even if you might use the Docker toolchain for building and managing individual containers.

Khi Docker mới xuất hiện, phạm vi của nó bị giới hạn ở việc quản lý container trên một máy. Điều này hữu ích hạn chế—nếu bạn muốn quản lý container trên nhiều máy thì sao? Đây là điều thiết yếu nếu bạn muốn duy trì sức khỏe hệ thống, nếu bạn có một máy chết, hoặc nếu bạn chỉ muốn chạy đủ container để xử lý tải của hệ thống. Docker đã ra hai sản phẩm hoàn toàn khác nhau của riêng mình để giải quyết vấn đề này, được đặt tên một cách khó hiểu là “Docker Swarm” và “Docker Swarm Mode”—ai bảo việc đặt tên là khó nhỉ? Tuy nhiên, thực sự thì khi nói đến việc quản lý nhiều container trên nhiều máy, Kubernetes là vua ở đây, kể cả khi bạn có thể dùng bộ công cụ Docker để build và quản lý các container riêng lẻ.

Fitness for microservices

Sự phù hợp cho microservices

Containers as a concept work wonderfully well for microservices, and Docker made containers significantly more viable as a concept. We get our isolation but at a manageable cost. We also hide underlying technology, allowing us to mix different tech stacks. When it comes to implementing concepts like desired state management, though, we'll need something like Kubernetes to handle it for us.

Container như một khái niệm hoạt động cực kỳ tốt cho microservices, và Docker đã làm cho container khả thi hơn đáng kể như một khái niệm. Ta có được sự cô lập nhưng với một chi phí có thể quản lý được. Ta cũng ẩn đi công nghệ bên dưới, cho phép ta trộn lẫn các ngăn xếp công nghệ khác nhau. Tuy nhiên, khi nói đến việc hiện thực hóa các khái niệm như quản lý trạng thái mong muốn, ta sẽ cần một thứ như Kubernetes để xử lý nó cho ta.

Kubernetes is important enough to warrant a more detailed discussion, so we'll come back to it later in the chapter. But for now just think of it as a way of managing containers across lots of machines, which is enough for the moment.

Kubernetes đủ quan trọng để đáng có một cuộc thảo luận chi tiết hơn, nên ta sẽ quay lại với nó ở phần sau của chương. Nhưng bây giờ chỉ cần nghĩ về nó như một cách quản lý container trên nhiều máy, thế là đủ cho lúc này.

Application Containers — Application Container

If you're familiar with deploying .NET applications behind IIS or Java applications into something like Weblogic or Tomcat, you will be well acquainted with the model in which multiple distinct services or applications sit inside a single application container, which in turn sits on a single host, as we see in Figure 8-17. The idea is that the application container your services live in gives you benefits in terms of improved manageability, such as clustering support to handle grouping multiple instances together, monitoring tools, and the like.

Nếu bạn quen với việc triển khai các ứng dụng .NET phía sau IIS hoặc các ứng dụng Java vào một thứ như Weblogic hay Tomcat, bạn sẽ rất quen thuộc với mô hình trong đó nhiều service hoặc ứng dụng riêng biệt nằm bên trong một application container duy nhất, vốn lại nằm trên một host duy nhất, như ta thấy trong Hình 8-17. Ý tưởng là application

container nơi các service của bạn sống mang lại lợi ích về mặt khả năng quản lý được cải thiện, chẳng hạn như hỗ trợ clustering để xử lý việc nhóm nhiều instance lại với nhau, các công cụ giám sát và những thứ tương tự.

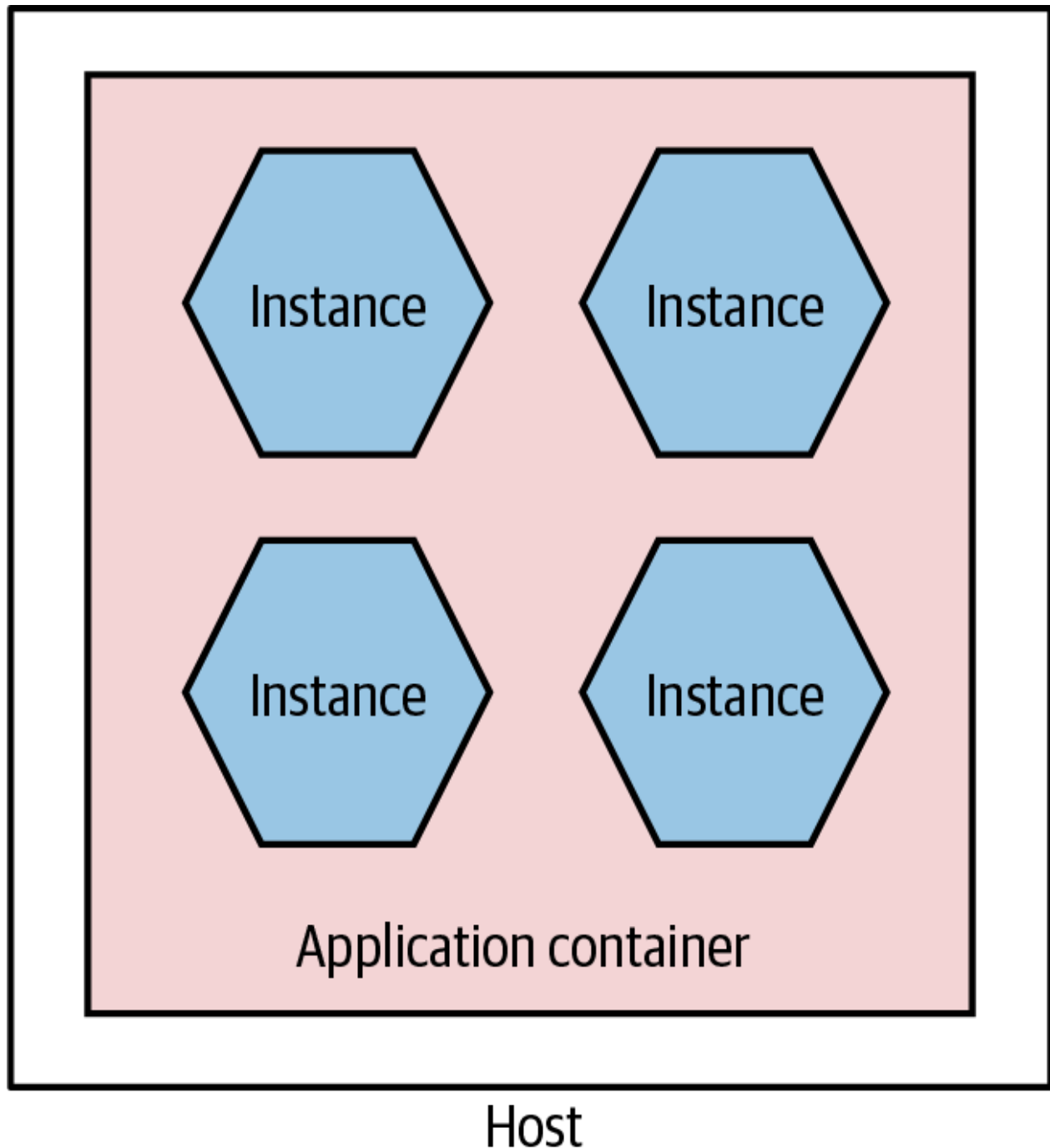


Figure 8-17. Multiple microservices per application container

Hình 8-17. Nhiều microservice trên mỗi application container

This setup can also yield benefits in terms of reducing overhead of language runtimes. Consider running five Java services in a single Java servlet container. I have the overhead of only a single JVM. Compare this with running five independent JVMs on the same host when using containers. That

said, I still feel that these application containers have enough downsides that you should challenge yourself to see whether they are really required.

Thiết lập này cũng có thể mang lại lợi ích về mặt giảm chi phí phụ trội của các runtime ngôn ngữ. Hãy xét việc chạy năm service Java trong một servlet container Java duy nhất. Tôi chỉ có chi phí phụ trội của một JVM duy nhất. So sánh điều này với việc chạy năm JVM độc lập trên cùng một host khi dùng container. Dẫu vậy, tôi vẫn cảm thấy các application container này có đủ nhược điểm để bạn nên thách thức chính mình xem liệu chúng có thực sự cần thiết hay không.

First among the downsides is that they inevitably constrain technology choice. You have to buy into a technology stack. This can limit not only the technology choices for the implementation of the service itself but also the options you have in terms of automation and management of your systems. As we'll discuss shortly, one of the ways we can address the overhead of managing multiple hosts is with automation, and so constraining our options for resolving this may well be doubly damaging.

Đầu tiên trong các nhược điểm là chúng tất yếu ràng buộc lựa chọn công nghệ. Bạn phải cam kết với một ngăn xếp công nghệ. Điều này có thể giới hạn không chỉ các lựa chọn công nghệ cho việc hiện thực bản thân service mà còn các lựa chọn bạn có về mặt tự động hóa và quản lý hệ thống của mình. Như ta sẽ bàn ngay sau đây, một trong những cách ta có thể giải quyết chi phí phụ trội của việc quản lý nhiều host là bằng tự động hóa, nên việc ràng buộc các lựa chọn của ta để giải quyết điều này rất có thể gây tổn hại gấp đôi.

I would also question some of the value of the features provided by these application containers. Many of them tout the ability to manage clusters to support shared in-memory session state, something we absolutely want to avoid in any case due to the challenges this creates when scaling our services. And the monitoring capabilities they provide won't be sufficient when we consider the sorts of joined-up monitoring we want to do in a microservices world, as we'll see in Chapter 10. Many of them also have quite slow spin-up times, impacting developer feedback cycles.

Tôi cũng sẽ đặt câu hỏi về một phần giá trị của các tính năng do các application container này cung cấp. Nhiều trong số chúng khoe khoang khả năng quản lý cluster để hỗ trợ trạng thái phiên (session state) dùng chung trong bộ nhớ, một thứ ta tuyệt đối muốn tránh trong mọi trường hợp do những thách thức mà điều này tạo ra khi mở rộng các service của ta. Và các khả năng giám sát mà chúng cung cấp sẽ không đủ khi ta xét đến kiểu giám sát kết nối liền mạch mà ta muốn làm trong thế giới microservices, như ta sẽ thấy trong Chương 10. Nhiều trong số chúng cũng có thời gian khởi tạo khá chậm, ảnh hưởng đến chu kỳ phản hồi của lập trình viên.

There are other sets of problems too. Attempting to do proper life-cycle management of applications on top of platforms like the JVM can be problematic and more complex than simply restarting a JVM. Analyzing resource use and threads is also much more complex, as you have multiple applications sharing the same process. And remember, even if you do get value from technology-specific containers, they aren't free. Aside from the fact that many of them are commercial and thus have a cost implication, they add a resource overhead in and of themselves.

Còn có những tập vấn đề khác nữa. Việc cố quản lý vòng đời (life-cycle) một cách đúng đắn cho các ứng dụng trên nền các nền tảng như JVM có thể gặp trục trặc và phức tạp hơn việc chỉ đơn giản khởi động lại một JVM. Việc phân tích mức sử dụng tài nguyên và các luồng (thread) cũng phức tạp hơn nhiều, vì bạn có nhiều ứng dụng dùng chung cùng một tiến trình. Và hãy nhớ, kể cả khi bạn nhận được giá trị từ các container đặc thù theo công nghệ, chúng không miễn phí. Bỏ qua thực tế rằng nhiều trong số chúng là thương mại và do đó có hệ quả về chi phí, bản thân chúng cũng thêm vào một chi phí phụ trội về tài nguyên.

Ultimately, this approach is again an attempt to optimize for scarcity of resources that simply may not hold up anymore. Whether or not you decide to have multiple services per host as a deployment model, I would strongly suggest looking at self-contained deployable microservices as artifacts, with each microservice instance running as its own isolated process.

Rốt cuộc, cách tiếp cận này lại một lần nữa là một nỗ lực tối ưu hóa cho sự khan hiếm tài nguyên vốn có thể đơn giản là không còn đứng vững nữa. Dù bạn quyết định có nhiều service trên mỗi host như một mô hình triển khai hay không, tôi sẽ mạnh mẽ đề nghị nhìn vào các microservice triển khai được dạng tự chứa (self-contained) như các artifact, với mỗi instance microservice chạy như tiến trình cô lập riêng của nó.

Fundamentally, the lack of isolation this model provides is one of the main reasons why this model is increasingly rare for people adopting microservice architectures.

Về cơ bản, sự thiếu cô lập mà mô hình này cung cấp là một trong những lý do chính tại sao mô hình này ngày càng hiếm gặp đối với những người áp dụng các kiến trúc microservice.

Platform as a Service (PaaS) — Platform as a Service (PaaS)

When using Platform as a Service (PaaS), you are working at a higher-level abstraction than a single host. Some of these platforms rely on taking a technology-specific artifact, such as a Java WAR file or Ruby gem, and automatically provisioning and running it for you. Some of these platforms will transparently attempt to handle scaling the system up and down for you; others will allow you some control over how many nodes your service might run on, but they handle the rest.

Khi dùng Platform as a Service (PaaS), bạn làm việc ở một mức trừu tượng cao hơn so với một host đơn lẻ. Một số nền tảng này dựa vào việc lấy một artifact đặc thù theo công nghệ, chẳng hạn như một file Java WAR hay một Ruby gem, và tự động cung cấp và chạy nó cho bạn. Một số nền tảng này sẽ một cách trong suốt cố xử lý việc mở rộng hệ thống lên xuống cho bạn; số khác sẽ cho bạn một số quyền kiểm soát về việc service của bạn có thể chạy trên bao nhiêu node, nhưng chúng xử lý phần còn lại.

As was the case when I wrote the first edition, most of the best, most polished PaaS solutions are hosted. Heroku set the benchmark for delivering a developer-friendly **interface** and arguably has remained the gold standard for PaaS, despite limited growth in terms of its featureset over the last few years. Platforms like Heroku don't just run your application instance; they also provide capabilities such as running database instances for you—something that can be very painful to do yourself.

Như trường hợp khi tôi viết ấn bản đầu tiên, hầu hết các giải pháp PaaS tốt nhất, hoàn thiện nhất đều là dạng được lưu trữ (hosted). Heroku đã đặt ra chuẩn mực cho việc cung cấp một giao diện thân thiện với lập trình viên và có thể lập luận rằng vẫn là tiêu chuẩn vàng cho PaaS, dù tăng trưởng hạn chế về mặt bộ tính năng trong vài năm qua. Các nền tảng như Heroku không chỉ chạy instance ứng dụng của bạn; chúng còn cung cấp các khả năng như chạy các instance database cho bạn—một thứ có thể rất cực nhọc nếu tự làm.

When PaaS solutions work well, they work very well indeed. However, when they don't quite work for you, you often don't have much control in terms of getting under the hood to fix things. This is part of the trade-off you make. I would say that in my experience the smarter the PaaS solutions try to be, the more they go wrong. I've used more than one PaaS that attempts to autoscale based on application use, but does it badly. Invariably the heuristics that drive these smarts tend to be tailored for the average application rather than your specific use case. The more nonstandard your application, the more likely it is that it might not play nicely with a PaaS.

Khi các giải pháp PaaS hoạt động tốt, chúng thực sự hoạt động rất tốt. Tuy nhiên, khi chúng không hẳn hoạt động cho bạn, bạn thường không có nhiều quyền kiểm soát về mặt chui xuống dưới nắp ca-pô để sửa chữa mọi thứ. Đây là một phần của sự đánh đổi bạn thực hiện. Tôi sẽ nói rằng theo kinh nghiệm của tôi, các giải pháp PaaS càng cố tỏ ra thông minh, chúng càng đi sai. Tôi đã dùng hơn một PaaS cố autoscale dựa trên mức dùng ứng dụng, nhưng làm việc đó tệ. Hầu như không tránh khỏi, các heuristic dẫn dắt những điều thông minh này có xu hướng được thừa riêng cho ứng dụng trung bình hơn là cho trường hợp dùng cụ thể của bạn. Ứng dụng của bạn càng phi tiêu chuẩn, càng có khả năng nó có thể không hòa hợp với một PaaS.

As the good PaaS solutions handle so much for you, they can be an excellent way of handling the increased overhead we get with having many more moving parts. That said, I'm still not sure that we have all the models right in this space yet, and the limited self-hosted options mean that this approach might not work for you. When I wrote the first edition, I was hopeful that we'd see more growth in this space, but it hasn't happened in the way that I expected. Instead, I think the growth of **serverless** products offered primarily by the public cloud providers has started to fill this need. Rather than offering black-box platforms for hosting an application, they instead provide turnkey managed solutions for things like message brokers, databases, storage, and such that allow us to mix and match the parts we like to build what we need. It is against this backdrop that Function as a Service, a specific type of serverless product, has been getting a lot of traction.

Vì các giải pháp PaaS tốt xử lý rất nhiều thứ cho bạn, chúng có thể là một cách tuyệt vời để xử lý chi phí phụ trội gia tăng mà ta có với việc có nhiều bộ phận động hơn. Dẫu vậy, tôi vẫn không chắc rằng ta đã có tất cả các mô hình đúng trong không gian này. Và các lựa chọn tự lưu trữ (self-hosted) hạn chế nghĩa là cách tiếp cận này có thể không hoạt động cho bạn. Khi tôi viết ấn bản đầu tiên, tôi đã hy vọng rằng ta sẽ thấy nhiều tăng trưởng hơn trong không gian này, nhưng nó đã không xảy ra theo cách tôi kỳ vọng. Thay vào đó, tôi nghĩ sự tăng trưởng của các sản phẩm serverless được cung cấp chủ yếu bởi các nhà cung cấp public cloud đã bắt đầu lấp đầy nhu cầu này. Thay vì cung cấp các nền tảng hộp đen để lưu trữ một ứng dụng, họ cung cấp các giải pháp được quản lý kiểu chìa-khóa-trao-tay (turnkey) cho những thứ như message broker, database, lưu trữ và những thứ tương tự, cho phép ta trộn và khớp các bộ phận ta thích để xây dựng cái ta cần. Chính trong bối cảnh này mà Function as a Service, một loại sản phẩm serverless cụ thể, đã thu hút được rất nhiều sự chú ý.

Assessing the suitability of PaaS offerings for microservices is difficult, as they come in many shapes and sizes. Heroku looks quite different from Netlify, for example, but both could work for you as a deployment platform for your microservices, depending on the nature of your application.

Việc đánh giá sự phù hợp của các sản phẩm PaaS cho microservices là khó, vì chúng có đủ hình dạng và kích cỡ. Chẳng hạn, Heroku trông khá khác Netlify, nhưng cả hai đều có thể hoạt động cho bạn như một nền tảng triển khai cho microservice của bạn, tùy theo bản chất của ứng dụng của bạn.

Function as a Service (FaaS) — Function as a Service (FaaS)

In the last few years, the only technology to get even close to Kubernetes in terms of generating hype (at least in the context of microservices) is serverless. Serverless is actually an umbrella term for a host of different technologies where, from the point of view of the person using them, the underlying computers don't matter. The detail of managing and configuring machines is taken away from you. In the words of Ken Fromm (who as far as I can tell coined the term serverless):

Trong vài năm qua, công nghệ duy nhất tiến gần đến Kubernetes về mặt tạo ra sự cường điệu (hype) (ít nhất là trong ngữ cảnh microservices) là serverless. Serverless thực ra là một thuật ngữ ô che (umbrella term) cho một loạt công nghệ khác nhau mà ở đó, từ góc nhìn của người dùng chúng, các máy tính bên dưới không quan trọng. Chi tiết của việc quản lý và cấu hình máy bị lấy đi khỏi bạn. Theo lời của Ken Fromm (người mà theo những gì tôi biết đã đặt ra thuật ngữ serverless):

The phrase “serverless” doesn’t mean servers are no longer involved. It simply means that developers no longer have to think that much about them. Computing resources get used as services without having to manage around physical capacities or limits. Service providers increasingly take on the responsibility of managing servers, data stores and other infrastructure resources. Developers could set up their own open source solutions, but that means they have to manage the servers and the queues and the loads. —Ken Fromm, “Why the Future of Software and Apps Is Serverless” Function as a Service, or FaaS, has become such a major part of serverless that for many the two terms are interchangeable. This is unfortunate, as it overlooks the importance of other serverless products like databases, queues, storage solutions, and the like. Nonetheless, it speaks to the excitement that FaaS has generated that it’s dominated the discussion.

Cụm từ “serverless” không có nghĩa là các máy chủ không còn dính dáng nữa. Nó chỉ đơn giản nghĩa là lập trình viên không còn phải nghĩ nhiều đến chúng nữa. Tài nguyên tính toán được dùng như các dịch vụ mà không cần quản lý quanh các năng lực hay giới hạn vật lý. Các nhà cung cấp dịch vụ ngày càng đảm nhận trách nhiệm quản lý máy chủ, kho dữ liệu và các tài nguyên hạ tầng khác. Lập trình viên có thể thiết lập các giải pháp mã nguồn mở của riêng họ, nhưng điều đó nghĩa là họ phải quản lý các máy chủ, các hàng đợi và các tải. —Ken Fromm, “Why the Future of Software and Apps Is Serverless” Function as a Service, hay FaaS, đã trở thành một phần lớn của serverless đến mức với nhiều người hai thuật ngữ này có thể thay thế cho nhau. Điều này đáng tiếc, vì nó bỏ qua tầm quan trọng của các sản phẩm serverless khác như database, hàng đợi, các giải pháp lưu trữ và những thứ tương tự. Tuy nhiên, nó nói lên sự phấn khích mà FaaS đã tạo ra đến mức nó đã thống trị cuộc thảo luận.

It was AWS’s Lambda product, launched in 2014, that ignited the excitement around FaaS. At one level, the concept is delightfully simple. You deploy some code (a “function”). That code is dormant, until something happens to trigger that code. You’re in charge of deciding what that trigger might be—it could be a file arriving in a certain location, an item appearing on a message queue, a call coming in via HTTP, or something else.

Chính sản phẩm Lambda của AWS, ra mắt năm 2014, đã thắp lên sự phấn khích quanh FaaS. Ở một mức, khái niệm này đơn giản một cách thú vị. Bạn triển khai một đoạn mã (một “function”). Đoạn mã đó nằm ngủ, cho đến khi điều gì đó xảy ra để kích hoạt (trigger) đoạn mã đó. Bạn là người chịu trách nhiệm quyết định trigger đó có thể là gì—nó có thể là một file đến tại một vị trí nhất định, một mục xuất hiện trên một hàng đợi message, một lời gọi đến qua HTTP, hay điều gì đó khác.

When your function triggers, it runs, and when it finishes, it shuts down. The underlying platform handles spinning these functions up or down on demand and will handle concurrent executions of your functions so that you can have multiple copies running at once where appropriate.

Khi function của bạn được trigger, nó chạy, và khi nó kết thúc, nó tắt đi. Nền tảng bên dưới xử lý việc khởi tạo các function này lên hoặc xuống theo yêu cầu và sẽ xử lý các lần thực thi đồng thời (concurrent) của function của bạn để bạn có thể có nhiều bản sao chạy cùng lúc khi thích hợp.

The benefits here are numerous. Code that isn’t running isn’t costing you money— you pay only

for what you use. This can make FaaS a great option for situations in which you have low or unpredictable load. The underlying platform handles spinning the functions up and down for you, giving you some degree of implicit high availability and robustness without you having to do any work. Fundamentally, the use of a FaaS platform, as with many of the other serverless offerings, allows you to drastically reduce the amount of operational overhead you need to worry about.

Lợi ích ở đây rất nhiều. Mã không chạy thì không tốn tiền của bạn—bạn chỉ trả cho những gì bạn dùng. Điều này có thể làm cho FaaS một lựa chọn tuyệt vời cho các tình huống bạn có tải thấp hoặc không thể đoán trước. Nền tảng bên dưới xử lý việc khởi tạo các function lên xuống cho bạn, cho bạn một mức độ tính sẵn sàng cao (high availability) và độ vững chắc ngầm định mà bạn không phải làm gì. Về cơ bản, việc dùng một nền tảng FaaS, như với nhiều sản phẩm serverless khác, cho phép bạn giảm mạnh lượng chi phí phụ trội vận hành mà bạn cần lo lắng.

Limitations

Hạn chế

Under the hood, all the FaaS implementations I'm aware of make use of some sort of container technology. This is hidden from you—typically you don't have to worry about building a container that will be run, you just provide some packaged form of the code. This means, though, that you lack a degree of control over what exactly can be run; as a result you need the FaaS provider to support your language of choice. Azure Functions have done the best here in terms of the major cloud vendors, supporting a wide variety of different runtimes, whereas Google Cloud's own Cloud Functions offering supports very few languages by comparison (at the time of writing, Google supports only Go, some Node versions, and Python). It's worth noting that AWS does now allow you to define your own custom runtime for your functions, theoretically enabling you to implement support for languages that aren't provided out of the box, although this then becomes another piece of operational overhead you have to maintain.

Ở phía dưới, tất cả các bản hiện thực FaaS mà tôi biết đều tận dụng một dạng công nghệ container nào đó. Điều này được ẩn khỏi bạn—thường bạn không phải lo lắng về việc build một container sẽ được chạy, bạn chỉ cung cấp một dạng đóng gói nào đó của mã. Tuy nhiên, điều này nghĩa là bạn thiếu một mức độ kiểm soát về chính xác cái gì có thể được chạy; kết quả là bạn cần nhà cung cấp FaaS hỗ trợ ngôn ngữ bạn chọn. Azure Functions đã làm tốt nhất ở đây trong số các nhà cung cấp đám mây lớn, hỗ trợ nhiều loại runtime khác nhau, trong khi sản phẩm Cloud Functions của riêng Google Cloud hỗ trợ rất ít ngôn ngữ khi so sánh (vào thời điểm viết, Google chỉ hỗ trợ Go, một số phiên bản Node và Python). Cũng đáng lưu ý rằng AWS giờ cho phép bạn định nghĩa runtime tùy chỉnh của riêng mình cho các function, về lý thuyết cho phép bạn hiện thực việc hỗ trợ các ngôn ngữ không được cung cấp sẵn, mặc dù điều này khi đó trở thành một mảng chi phí phụ trội vận hành khác mà bạn phải duy trì.

This lack of control over the underlying runtime also extends to the lack of control over the resources given to each function invocation. Across Google Cloud, Azure, and AWS, you can only control the memory given to each function. This in turn seems to imply that a certain amount of CPU and I/O is given to your function runtime, but you can't control those aspects directly. This may mean that you end up having to give more memory to a function even if it doesn't need it just to get the CPU you need. Ultimately, if you feel that you need to do a lot of fine tuning around resources available to your functions, then I feel that, at this stage at least, FaaS is probably not a great option for you.

Sự thiếu kiểm soát này đối với runtime bên dưới cũng mở rộng đến sự thiếu kiểm soát đối với tài nguyên được cấp cho mỗi lần gọi function. Trên Google Cloud, Azure và AWS, bạn chỉ có thể kiểm soát bộ nhớ được cấp cho mỗi function. Điều này lần lượt có vẻ ngụ ý

rằng một lượng CPU và I/O nhất định được cấp cho runtime function của bạn, nhưng bạn không thể kiểm soát các khía cạnh đó trực tiếp. Điều này có thể nghĩa là bạn rất cuộc phải cấp nhiều bộ nhớ hơn cho một function dù nó không cần chỉ để có được CPU bạn cần. Rất cuộc, nếu bạn cảm thấy mình cần làm nhiều tinh chỉnh quanh tài nguyên sẵn có cho các function, thì tôi cảm thấy rằng, ít nhất ở giai đoạn này, FaaS có lẽ không phải là một lựa chọn tuyệt vời cho bạn.

Another limitation to be aware of is that function invocations can provide limits in terms of how long they can run for. Google Cloud functions, for example, are currently capped at 9 minutes of execution, while AWS Lambda functions can run for up to 15 minutes. Azure functions can run forever if you want (depending on the type of plan you are on). Personally, I think if you have functions running for long periods of time, this probably points to the sort of problem that functions aren't a good fit for.

Một hạn chế khác cần lưu ý là các lần gọi function có thể đặt ra giới hạn về thời gian chúng có thể chạy. Chẳng hạn, các function Google Cloud hiện bị giới hạn ở 9 phút thực thi, trong khi các function AWS Lambda có thể chạy tới 15 phút. Các function Azure có thể chạy mãi mãi nếu bạn muốn (tùy theo loại gói bạn đang dùng). Cá nhân tôi, tôi nghĩ nếu bạn có các function chạy trong thời gian dài, điều này có lẽ chỉ ra kiểu vấn đề mà function không phù hợp.

Finally, most function invocations are considered to be stateless. Conceptually, this means that a function cannot access state left by a previous function invocation unless that state is stored elsewhere (for example, in a database). This has made it hard to have multiple functions chained together—consider one function orchestrating a series of calls to other downstream functions. A notable exception is Azure Durable Functions, which solves this problem in a really interesting way. Durable Functions supports the ability to suspend the state of a given function and allow it to restart where the invocation left off—this is all handled transparently through the use of reactive extensions. This is a solution that I think is significantly more developer friendly than AWS's own Step Functions, which ties together multiple functions using JSON-based configuration.

Cuối cùng, hầu hết các lần gọi function được coi là không trạng thái (stateless). Về mặt khái niệm, điều này nghĩa là một function không thể truy cập trạng thái để lại bởi một lần gọi function trước trừ khi trạng thái đó được lưu ở nơi khác (ví dụ, trong một database). Điều này đã làm cho việc có nhiều function được xâu chuỗi với nhau trở nên khó khăn—hãy xét một function điều phối một loạt lời gọi tới các function hạ nguồn (downstream) khác. Một ngoại lệ đáng chú ý là Azure Durable Functions, vốn giải quyết vấn đề này theo một cách thực sự thú vị. Durable Functions hỗ trợ khả năng tạm dừng trạng thái của một function nhất định và cho phép nó khởi động lại từ nơi lần gọi đã dừng—tất cả điều này được xử lý một cách trong suốt thông qua việc dùng reactive extension. Đây là một giải pháp mà tôi nghĩ thân thiện với lập trình viên hơn đáng kể so với Step Functions của riêng AWS, vốn buộc nhiều function lại với nhau bằng cấu hình dựa trên JSON.

WebAssembly (Wasm) — WebAssembly (Wasm)

Wasm is an official standard that was originally defined to give developers a way of running sandboxed programs written in a variety of programming languages on client browsers. Defining both a packaging format and a runtime environment, the goal of Wasm is to allow arbitrary code to run in a safe and efficient manner on client devices. This can allow for far more sophisticated client-side applications to be created when using normal web browsers. As a concrete example, eBay used Wasm to deliver barcode scanner software, the core of which was written in C++ and

which was previously available only to native Android or iOS applications, to the web. 2

Wasm là một chuẩn chính thức ban đầu được định nghĩa để cho lập trình viên một cách chạy các chương trình được sandbox viết bằng nhiều ngôn ngữ lập trình khác nhau trên các trình duyệt phía client. Định nghĩa cả một định dạng đóng gói lẫn một môi trường runtime, mục tiêu của Wasm là cho phép mã tùy ý chạy theo cách an toàn và hiệu quả trên các thiết bị client. Điều này có thể cho phép tạo ra các ứng dụng phía client tinh vi hơn nhiều khi dùng các trình duyệt web thông thường. Như một ví dụ cụ thể, eBay đã dùng Wasm để đưa phần mềm quét mã vạch, cốt lõi của nó được viết bằng C++ và trước đó chỉ có sẵn cho các ứng dụng native Android hay iOS, lên web. 2

The WebAssembly System Interface (WASI) was defined as a way to let Wasm move from the browser and work anywhere a compatible WASI implementation can be found. An example of this is the ability to run Wasm on content delivery networks like Fastly or Cloudflare.

The WebAssembly System Interface (WASI) được định nghĩa như một cách để Wasm di chuyển khỏi trình duyệt và hoạt động ở bất cứ đâu có thể tìm thấy một bản hiện thực WASI tương thích. Một ví dụ về điều này là khả năng chạy Wasm trên các mạng phân phối nội dung (content delivery network) như Fastly hay Cloudflare.

2 Senthil Padmanabhan and Pranav Jha, “WebAssembly at eBay: A Real-World Use Case,” eBay, May 22, 2019, <https://oreil.ly/SfvHT>.

2 Senthil Padmanabhan and Pranav Jha, “WebAssembly at eBay: A Real-World Use Case,” eBay, May 22, 2019, <https://oreil.ly/SfvHT>.

Due to its lightweight nature and the strong sandboxing concepts built in to its core specification, Wasm has the potential to challenge the use of containers as the go-to deployment format for server-side applications. In the short term, what is holding it back is likely the server-side platforms available to run Wasm. While you can theoretically run Wasm on Kubernetes, for example, you end up embedding Wasm inside containers, which arguably ends up being somewhat pointless, as you’re running a more lightweight deployment inside a (comparatively) more heavyweight container.

Do bản chất nhẹ và các khái niệm sandbox mạnh được xây dựng sẵn trong đặc tả cốt lõi của nó, Wasm có tiềm năng thách thức việc dùng container như định dạng triển khai ưu tiên cho các ứng dụng phía máy chủ. Trong ngắn hạn, thứ đang kìm hãm nó nhiều khả năng là các nền tảng phía máy chủ sẵn có để chạy Wasm. Chẳng hạn, mặc dù về lý thuyết bạn có thể chạy Wasm trên Kubernetes, bạn rốt cuộc nhúng Wasm bên trong container, điều mà có thể lập luận là rốt cuộc khá vô nghĩa, vì bạn đang chạy một triển khai nhẹ hơn bên trong một container (tương đối) nặng hơn.

A server-side deployment platform with native support for WASI would likely be needed to get the most out of Wasm’s potential. Theoretically at least, a scheduler like Nomad would be better placed to support Wasm, as it supports a pluggable driver model. Time will tell!

Một nền tảng triển khai phía máy chủ với hỗ trợ native cho WASI nhiều khả năng sẽ cần để tận dụng tối đa tiềm năng của Wasm. Ít nhất về mặt lý thuyết, một bộ lập lịch (scheduler) như Nomad sẽ ở vị trí tốt hơn để hỗ trợ Wasm, vì nó hỗ trợ một mô hình driver có thể cắm thêm (pluggable). Thời gian sẽ trả lời!

Challenges

Thách thức

Aside from the limitations we’ve just looked at, there are some other challenges you may experience when using FaaS.

Bên cạnh các hạn chế ta vừa xem, có một số thách thức khác mà bạn có thể gặp khi dùng FaaS.

Firstly, it's important to address a concern that is often raised with FaaS, and that is the notion of spin-up time. Conceptually, functions are not running at all unless they are needed. This means they have to be launched to serve an incoming request. Now, for some runtimes, it takes a long time to spin up a new version of the runtime—often called a “cold start” time. JVM and .NET runtimes suffer a lot from this, so a cold start time for functions using these runtimes can often be significant.

Thứ nhất, quan trọng là phải giải quyết một mối bận tâm thường được nêu ra với FaaS, đó là khái niệm thời gian khởi tạo (spin-up time). Về mặt khái niệm, các function hoàn toàn không chạy trừ khi chúng được cần đến. Điều này nghĩa là chúng phải được khởi chạy để phục vụ một request đến. Giờ đây, với một số runtime, việc khởi tạo một phiên bản mới của runtime mất nhiều thời gian—thường được gọi là thời gian “khởi động lạnh” (cold start). Các runtime JVM và .NET chịu nhiều ảnh hưởng từ điều này, nên thời gian khởi động lạnh cho các function dùng các runtime này thường có thể đáng kể.

In reality, though, these runtimes rarely cold start. On AWS at least, the runtimes are kept “warm,” so that requests that come in are served by already launched and running instances. This happens to such an extent that it can be difficult to gauge the impact of a “cold start” nowadays due to the optimizations being done under the hood by the FaaS providers. Nonetheless, if this is a concern, sticking to languages whose runtimes have fast spin-up times (Go, Python, Node, and Ruby come to mind) can sidestep this issue effectively.

Tuy nhiên, trên thực tế, các runtime này hiếm khi khởi động lạnh. Ít nhất trên AWS, các runtime được giữ “ấm” (warm), để các request đến được phục vụ bởi các instance đã được khởi chạy và đang chạy. Điều này xảy ra đến mức mà ngày nay có thể khó đánh giá tác động của một lần “khởi động lạnh” do các tối ưu hóa đang được các nhà cung cấp FaaS thực hiện ở phía dưới. Tuy nhiên, nếu đây là một mối bận tâm, việc bám vào các ngôn ngữ có runtime với thời gian khởi tạo nhanh (Go, Python, Node và Ruby thoág hiện lên trong đầu) có thể né tránh vấn đề này một cách hiệu quả.

Finally, the dynamic scaling aspect of functions can actually end up being an issue. Functions are launched when triggered. All the platforms I've used have a hard limit on the maximum number of concurrent function invocations, which is something you might have to take careful note of. I've spoken to more than one team that has had the issue of functions scaling up and overwhelming other parts of its infrastructure that didn't have the same scaling properties. Steve Faulkner from Bustle shared one such example, where scaling functions overloaded Bustle's Redis infrastructure, causing production issues. If one part of your system can dynamically scale but the other parts of your system don't, then you might find that this mismatch can cause significant headaches.

Cuối cùng, khía cạnh mở rộng động (dynamic scaling) của các function thực ra rốt cuộc có thể là một vấn đề. Các function được khởi chạy khi được trigger. Tất cả các nền tảng tôi đã dùng đều có một giới hạn cứng về số lượng tối đa các lần gọi function đồng thời, một điều bạn có thể phải lưu ý cẩn thận. Tôi đã nói chuyện với hơn một đội gặp vấn đề function mở rộng lên và áp đảo các phần khác trong hạ tầng của họ vốn không có cùng đặc tính mở rộng. Steve Faulkner từ Bustle đã chia sẻ một ví dụ như vậy, nơi các function mở rộng làm quá tải hạ tầng Redis của Bustle, gây ra các vấn đề trong production. Nếu một phần của hệ thống của bạn có thể mở rộng động nhưng các phần khác của hệ thống thì không, thì bạn có thể thấy rằng sự bất tương xứng này có thể gây ra những đau đầu đáng kể.

Mapping to microservices

Ánh xạ sang microservices

So far in our discussions of the various deployment options, the mapping from a microservice instance to a deployment mechanism has been pretty straightforward. A single microservice instance could be deployed onto a virtual machine, packaged as a single container, or even dropped onto an application container like Tomcat or IIS. With FaaS, things get a bit more confused.

Đến giờ trong các thảo luận của ta về các lựa chọn triển khai khác nhau, việc ánh xạ từ một instance microservice sang một cơ chế triển khai khá đơn giản. Một instance microservice duy nhất có thể được triển khai lên một máy ảo, đóng gói thành một container duy nhất, hay thậm chí thả vào một application container như Tomcat hay IIS. Với FaaS, mọi thứ trở nên hơi rối hơn một chút.

Function per microservice. Now obviously a single microservice instance can be deployed as a single function, as shown in Figure 8-18. This is probably a sensible place to start. This keeps the concept of a microservice instance as a unit of deployment, which is the model we've been exploring the most so far.

Một function cho mỗi microservice. Giờ thì rõ ràng một instance microservice duy nhất có thể được triển khai dưới dạng một function duy nhất, như minh họa trong Hình 8-18. Đây có lẽ là một điểm khởi đầu hợp lý. Điều này giữ khái niệm một instance microservice như một đơn vị triển khai, vốn là mô hình ta đã khám phá nhiều nhất đến giờ.

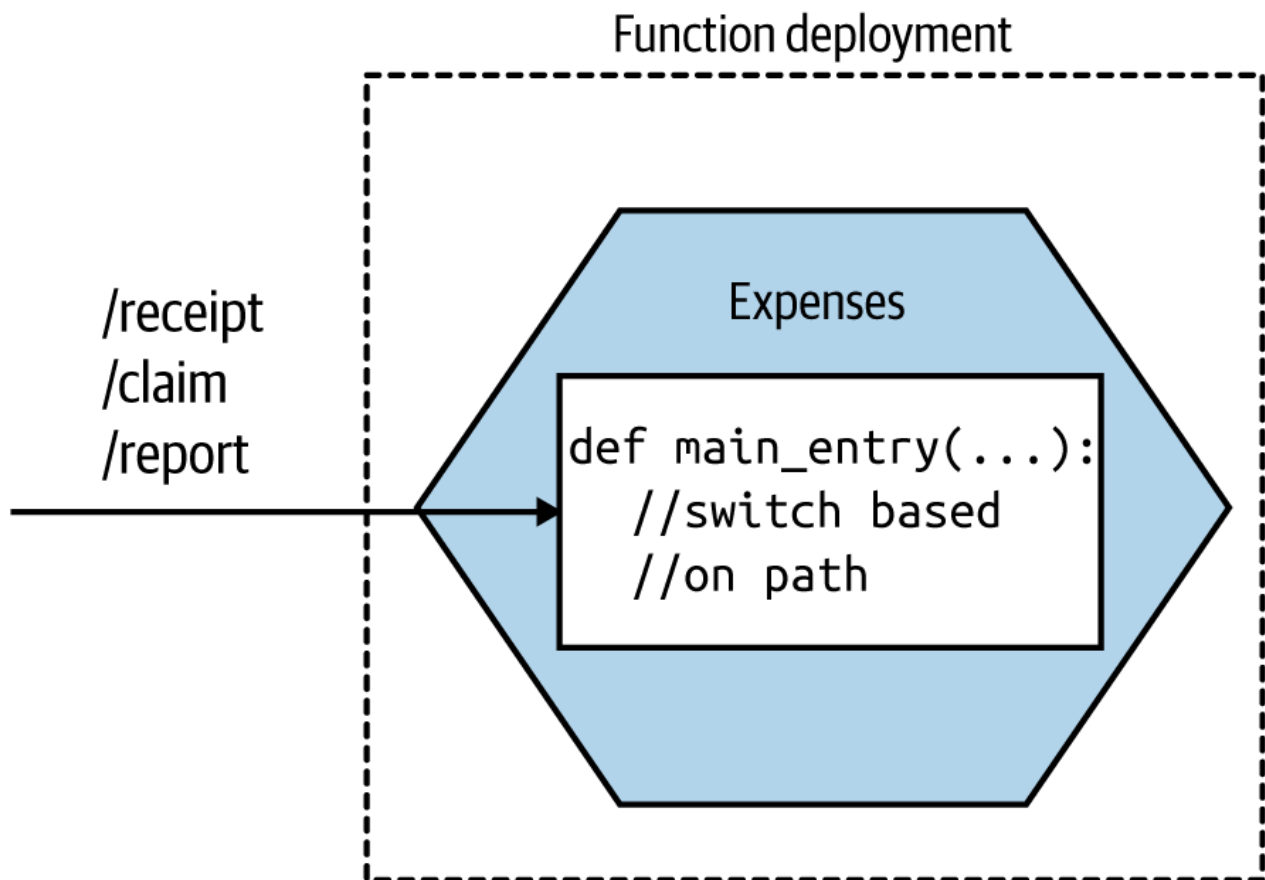


Figure 8-18. Our Expenses service is implemented as a single function

Hình 8-18. Service Expenses của ta được hiện thực dưới dạng một function duy nhất

When invoked, the FaaS platform will trigger a single entry point in your deployed function. This means that if you're going to have a single function deployment for your entire service, you'll need

to have some way of dispatching from that entry point to the different pieces of functionality in your microservice. If you were implementing the Expenses service as a REST-based microservice, you might have various resources exposed, like /receipt , /claim , or /report . With this model, a request for any of these resources would come in through this same entry point, so you'd need to direct the inbound call to the appropriate piece of functionality based on the inbound request path.

Khi được gọi, nền tảng FaaS sẽ trigger một điểm vào (entry point) duy nhất trong function đã triển khai của bạn. Điều này nghĩa là nếu bạn định có một triển khai function duy nhất cho toàn bộ service của mình, bạn sẽ cần một cách nào đó để điều phối từ điểm vào đó tới các mảnh chức năng khác nhau trong microservice của bạn. Nếu bạn hiện thực service Expenses dưới dạng một microservice dựa trên REST, bạn có thể có nhiều resource khác nhau được phơi bày ra, như /receipt, /claim hay /report. Với mô hình này, một request cho bất kỳ resource nào trong số này sẽ đến qua cùng một điểm vào đó, nên bạn sẽ cần định hướng lời gọi đến tới mảnh chức năng thích hợp dựa trên đường dẫn (path) của request đến.

Function per **aggregate**. So how would we break up a microservice instance into smaller functions? If you're making use of **domain-driven design**, you may already have explicitly modeled your aggregates (a collection of objects that are managed as a single entity, typically referring to real-world concepts). If your microservice instance handles multiple aggregates, one model that makes sense to me is to break out a function for each aggregate, as shown in Figure 8-19 . This ensures that all the logic for a single aggregate is self-contained inside the function, making it easier to ensure a consistent implementation of the life-cycle management of the aggregate.

Một function cho mỗi aggregate. Vậy ta sẽ chia một instance microservice thành các function nhỏ hơn ra sao? Nếu bạn dùng domain-driven design, bạn có thể đã mô hình hóa một cách tường minh các aggregate của mình (một tập các đối tượng được quản lý như một thực thể duy nhất, thường ám chỉ các khái niệm thực tế). Nếu instance microservice của bạn xử lý nhiều aggregate, một mô hình hợp lý với tôi là tách ra một function cho mỗi aggregate, như minh họa trong Hình 8-19. Điều này đảm bảo rằng tất cả logic cho một aggregate duy nhất là tự chứa bên trong function, làm cho việc đảm bảo một sự hiện thực nhất quán cho việc quản lý vòng đời của aggregate dễ dàng hơn.

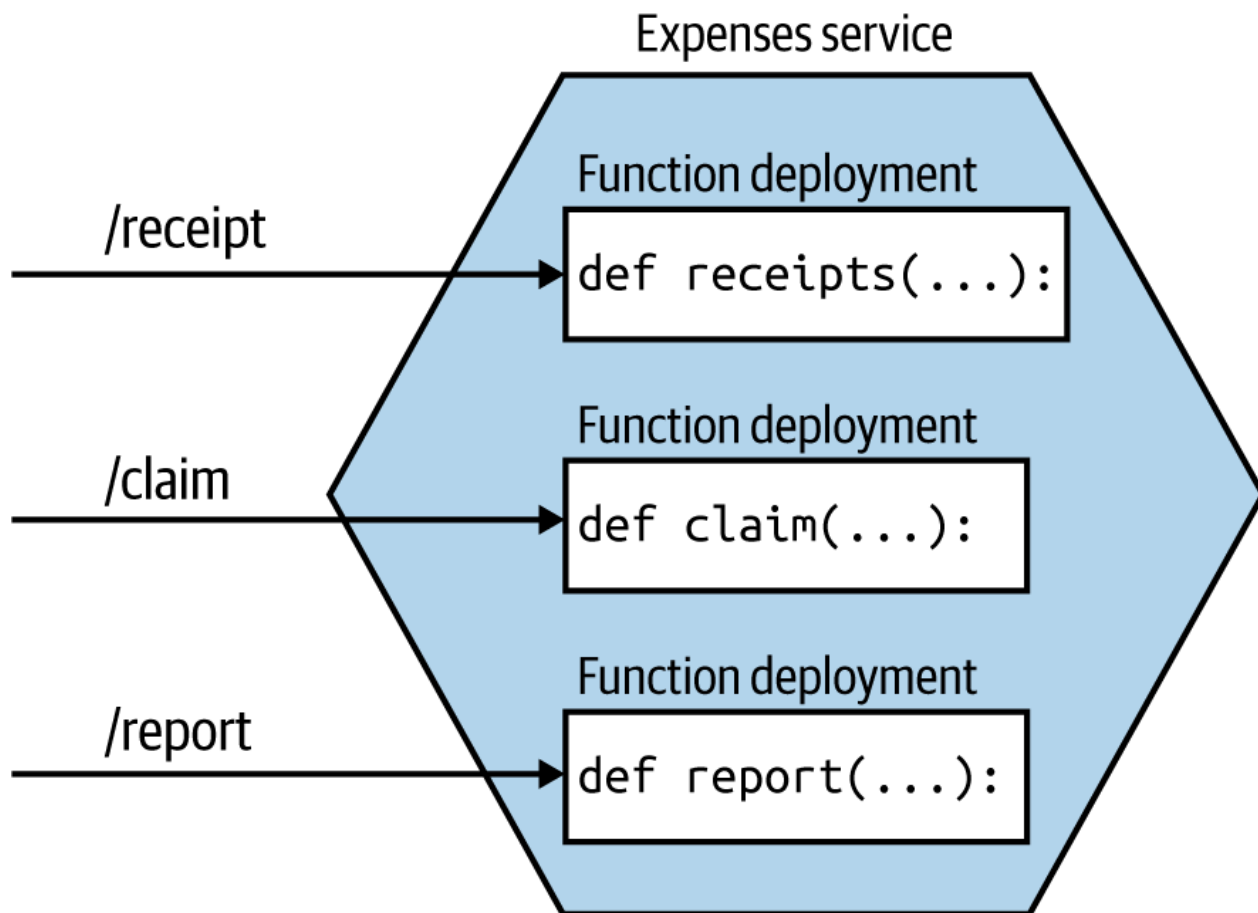


Figure 8-19. An Expenses service being deployed as multiple functions, each one handling a different aggregate

Hình 8-19. Một service Expenses được triển khai dưới dạng nhiều function, mỗi function xử lý một aggregate khác nhau

With this model, our microservice instance no longer maps to a single unit of deployment. Instead, our microservice is now more of a logical concept consisting of multiple different functions that can theoretically be deployed independently of each other.

Với mô hình này, instance microservice của ta không còn ánh xạ sang một đơn vị triển khai duy nhất. Thay vào đó, microservice của ta giờ đây giống một khái niệm logic hơn, gồm nhiều function khác nhau mà về lý thuyết có thể được triển khai độc lập với nhau.

A few caveats here. Firstly, I would strongly urge you to maintain a coarser-grained external interface. To upstream consumers, they are still talking to the Expenses service—they are unaware that requests get mapped to smaller-scoped aggregates. This ensures that should you change your mind and want to recombine things or even restructure the aggregate model, you won't impact upstream consumers.

Vài lưu ý ở đây. Thứ nhất, tôi sẽ mạnh mẽ thúc giục bạn duy trì một interface bên ngoài thô hơn (coarser-grained). Đối với các consumer phía thượng nguồn, họ vẫn đang trò chuyện với service Expenses—they không hề biết rằng các request được ánh xạ sang các aggregate có phạm vi nhỏ hơn. Điều này đảm bảo rằng nếu bạn đổi ý và muốn kết hợp lại mọi thứ hay thậm chí tái cấu trúc mô hình aggregate, bạn sẽ không ảnh hưởng đến các consumer phía thượng nguồn.

The second issue relates to data. Should these aggregates continue to use a shared database? On this issue, I am somewhat relaxed. Assuming that the same team manages all these functions, and that conceptually it remains a single “service,” I’d be OK with them still using the same database, as Figure 8-20 shows.

Vấn đề thứ hai liên quan đến dữ liệu. Liệu các aggregate này có nên tiếp tục dùng một database dùng chung? Về vấn đề này, tôi có phần thoải mái. Giả sử rằng cùng một đội quản lý tất cả các function này, và về mặt khái niệm nó vẫn là một “service” duy nhất, tôi sẽ ổn với việc chúng vẫn dùng cùng một database, như Hình 8-20 cho thấy.

Over time, though, if the needs of each aggregate function diverge, I’d be inclined to look to separate out their data usage, as seen in Figure 8-21, especially if you start to see coupling in the data tier impair your ability to change them easily. At this stage, you could argue that these functions would now be microservices in their own right—although as I’ve just explained, there may be value in still representing them as a single microservice to upstream consumers.

Tuy nhiên, theo thời gian, nếu nhu cầu của mỗi aggregate function phân kỳ, tôi sẽ có khuynh hướng tìm cách tách riêng việc dùng dữ liệu của chúng, như thấy trong Hình 8-21, đặc biệt nếu bạn bắt đầu thấy coupling ở tầng dữ liệu làm suy giảm khả năng thay đổi chúng một cách dễ dàng. Ở giai đoạn này, bạn có thể lập luận rằng các function này giờ đây sẽ tự thân là các microservice—mặc dù như tôi vừa giải thích, có thể có giá trị trong việc vẫn biểu diễn chúng như một microservice duy nhất đối với các consumer phía thượng nguồn.

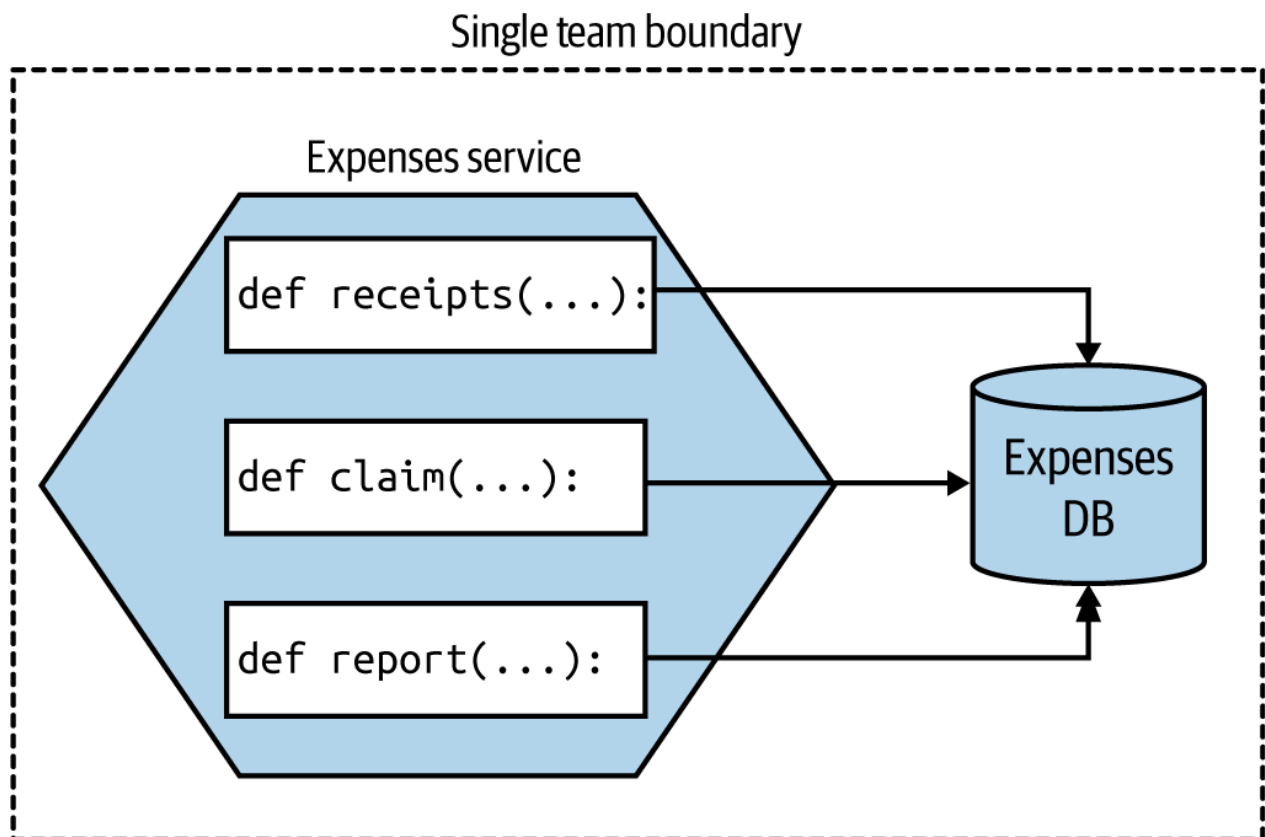


Figure 8-20. Different functions using the same database, as they are all logically part of the same microservice and are managed by the same team

Hình 8-20. Các function khác nhau dùng cùng một database, vì chúng đều logic là một

phần của cùng một microservice và được cùng một đội quản lý

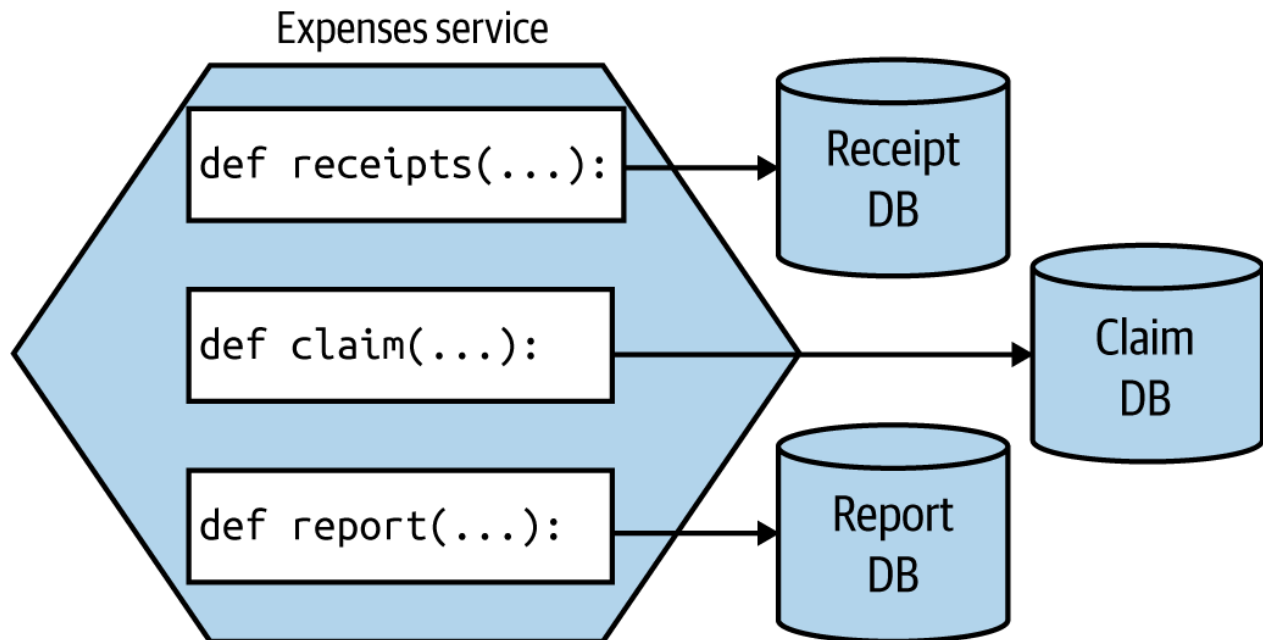


Figure 8-21. Each function using its own database

Hình 8-21. Mỗi function dùng database riêng của nó

This mapping from a single microservice to multiple finer-grained deployable units warps our previous definition of a microservice somewhat. We normally consider a microservice as being an independently deployable unit—now one microservice is made up of multiple different independently deployable units. Conceptually, in this example, the microservice moves toward being more of a logical than a physical concept.

Việc ánh xạ từ một microservice duy nhất sang nhiều đơn vị triển khai được mịn hơn này làm méo mó đôi chút định nghĩa trước đó của ta về một microservice. Ta thường coi một microservice như một đơn vị triển khai được độc lập—giờ đây một microservice được tạo thành từ nhiều đơn vị triển khai được độc lập khác nhau. Về mặt khái niệm, trong ví dụ này, microservice dịch chuyển về phía một khái niệm logic hơn là vật lý.

Get even more fine-grained. If you wanted to go even smaller, there is a temptation to break down your functions per aggregate into smaller pieces. I am much more cautious here. Aside from the explosion of functions this will likely create, it also violates one of the core principles of an aggregate—that we want to treat it as a single unit to ensure we can better manage the integrity of the aggregate itself.

Hãy đi mịn hơn nữa. Nếu bạn muốn đi nhỏ hơn nữa, có một cám dỗ chia các function-cho-mỗi-aggregate của bạn thành các mảnh nhỏ hơn. Tôi thận trọng hơn nhiều ở đây. Bỏ qua sự bùng nổ của các function mà điều này nhiều khả năng tạo ra, nó cũng vi phạm một trong những nguyên tắc cốt lõi của một aggregate—rằng ta muốn xử lý nó như một đơn vị duy nhất để đảm bảo ta có thể quản lý tốt hơn tính toàn vẹn của chính aggregate.

I've previously entertained the idea of making each state transition of an aggregate its own function, but I've backed out of this idea due to the problems associated with inconsistency. When you have different independently deployable things, each managing a different part of an overall state transition, ensuring things are done properly gets really quite difficult. It puts us into the space of sagas, which we discussed in Chapter 6 . When implementing complex business processes, concepts

like sagas are important, and the work is justifiable. I struggle, though, to see the value in adding this complexity at the level of managing a single aggregate that could easily be handled by a single function.

Trước đây tôi đã từng nghĩ tới ý tưởng biến mỗi chuyển dịch trạng thái (state transition) của một aggregate thành một function riêng của nó, nhưng tôi đã rút khỏi ý tưởng này do các vấn đề gắn với sự thiếu nhất quán. Khi bạn có những thứ triển khai được độc lập khác nhau, mỗi thứ quản lý một phần khác nhau của một chuyển dịch trạng thái tổng thể, việc đảm bảo mọi thứ được thực hiện đúng đắn trở nên thực sự khá khó. Nó đặt ta vào không gian của saga, thứ ta đã bàn trong Chương 6. Khi hiện thực các quy trình nghiệp vụ phức tạp, các khái niệm như saga là quan trọng, và công sức là chính đáng. Tuy nhiên, tôi vật lộn để thấy giá trị trong việc thêm sự phức tạp này ở cấp độ quản lý một aggregate duy nhất vốn có thể dễ dàng được xử lý bởi một function duy nhất.

The way forward

Con đường phía trước

I remain convinced that the future for most developers is using a platform that hides much of the underlying detail from them. For many years, Heroku was the closest thing I could point to in terms of something that found the right balance, but now we have FaaS and the wider ecosystem of turnkey serverless offerings that chart a different path.

Tôi vẫn tin chắc rằng tương lai cho hầu hết lập trình viên là dùng một nền tảng ẩn đi phần lớn chi tiết bên dưới khỏi họ. Trong nhiều năm, Heroku là thứ gần nhất tôi có thể chỉ ra về mặt một thứ tìm được sự cân bằng đúng đắn, nhưng giờ đây ta có FaaS và hệ sinh thái rộng hơn của các sản phẩm serverless chìa-khóa-trao-tay vạch ra một con đường khác.

There are still issues to be ironed out with FaaS, but I feel that, while the current crop of offerings still need to change to resolve the issues with them, this is the sort of platform that most developers will end up using. Not all applications will fit neatly into a FaaS ecosystem given the constraints, but for those that do, people are already seeing significant benefits. With more and more work going into Kubernetes-backed FaaS offerings, people who are unable to make direct use of the FaaS solutions provided by the main cloud providers will increasingly be able to take advantage of this new way of working.

Vẫn còn những vấn đề cần được giải quyết với FaaS, nhưng tôi cảm thấy rằng, dù lựa sản phẩm hiện tại vẫn cần thay đổi để giải quyết các vấn đề của chúng, đây là loại nền tảng mà hầu hết lập trình viên rốt cuộc sẽ dùng. Không phải mọi ứng dụng đều sẽ vừa khít vào một hệ sinh thái FaaS do các ràng buộc, nhưng với những ứng dụng phù hợp, người ta đã đang thấy những lợi ích đáng kể. Với ngày càng nhiều công sức đổ vào các sản phẩm FaaS hậu thuẫn bởi Kubernetes, những người không thể trực tiếp dùng các giải pháp FaaS do các nhà cung cấp đám mây lớn cung cấp sẽ ngày càng có thể tận dụng cách làm việc mới này.

So, while FaS may not work for everything, it's certainly something I urge people to explore. And for my clients who are looking at moving to cloud-based Kubernetes solutions, I've been urging many of them to explore FaaS first, as it may give them everything they need while hiding significant complexity and offloading a lot of work.

Vậy nên, dù FaaS có thể không hoạt động cho mọi thứ, nó chắc chắn là một thứ tôi thúc giục mọi người khám phá. Và với các khách hàng của tôi đang xem xét chuyển sang các giải pháp Kubernetes dựa trên đám mây, tôi đã thúc giục nhiều người trong số họ khám phá FaaS trước, vì nó có thể cho họ mọi thứ họ cần trong khi ẩn đi sự phức tạp đáng kể và giảm tải rất nhiều công việc.

I'm seeing more organizations making use of FaaS as part of a wider solution, picking FaaS for specific use cases where it fits well. A good example would be the BBC, which makes use of Lambda functions as part of its core technology stack that provides the BBC News website. The overall system uses a mix of Lambda and EC2 instances—with the EC2 instances often being used in situations in which Lambda function invocations would be too expensive. 3

Tôi đang thấy ngày càng nhiều tổ chức dùng FaaS như một phần của một giải pháp rộng hơn, chọn FaaS cho các trường hợp dùng cụ thể nơi nó phù hợp tốt. Một ví dụ hay sẽ là BBC, vốn dùng các function Lambda như một phần của ngăn xếp công nghệ cốt lõi cung cấp trang web BBC News. Hệ thống tổng thể dùng một sự pha trộn giữa Lambda và các instance EC2—với các instance EC2 thường được dùng trong các tình huống mà các lần gọi function Lambda sẽ quá tốn kém. 3

3 Johnathan Ishmael, “Optimising Serverless for BBC Online,” Technology and Creativity at the BBC (blog), BBC, January 26, 2021, <https://oreil.ly/gkSdp> .

3 Johnathan Ishmael, “Optimising Serverless for BBC Online,” Technology and Creativity at the BBC (blog), BBC, January 26, 2021, <https://oreil.ly/gkSdp> .

Which Deployment Option Is Right for You? — Lựa chọn Triển khai Nào Phù hợp với Bạn?

Yikes. So we have a lot of options, right? And I probably haven't helped too much by going out of my way to share loads of pros and cons for each approach. If you've gotten this far, you might be a bit bewildered about what you should do.

Ồi chà. Vậy là ta có rất nhiều lựa chọn, phải không? Và tôi có lẽ chẳng giúp ích được mấy khi cố hết sức chia sẻ vô số ưu và nhược điểm cho mỗi cách tiếp cận. Nếu bạn đã đọc đến đây, bạn có thể hơi hoang mang về việc mình nên làm gì.



Well, before I go any further, I really hope that it goes without saying that if what you are currently doing works for you, then keep doing it! Don't let fashion dictate your technical decisions.

Vâng, trước khi tôi đi xa hơn, tôi thực sự hy vọng rằng điều này khỏi cần nói, rằng nếu thứ bạn đang làm hiện tại hoạt động tốt cho bạn, thì hãy cứ tiếp tục làm! Đừng để một thời thượng quyết định các lựa chọn kỹ thuật của bạn.

If you think you do need to change how you deploy microservices, then let me try and distill down much of what we've already discussed and come up with some useful guidance.

Nếu bạn nghĩ rằng mình thực sự cần thay đổi cách triển khai microservice, thì hãy để tôi cố chắt lọc phần lớn những gì ta đã bàn và đưa ra một số hướng dẫn hữu ích.

Revisiting our principles of microservice deployment, one of the most important aspects we focused on was that of ensuring isolation of our microservices. But just using that as a guiding principle might guide us toward using dedicated physical machines for each microservice instance! That of course would likely be very expensive, and as we've already discussed, there are some very powerful tools that we wouldn't be able to use if we went down this route.

Xem lại các nguyên tắc triển khai microservice của ta, một trong những khía cạnh quan trọng nhất ta đã tập trung vào là việc đảm bảo sự cô lập của các microservice của ta. Nhưng chỉ dùng điều đó như một nguyên tắc dẫn đường có thể dẫn ta tới việc dùng các máy vật lý chuyên dùng cho mỗi instance microservice! Điều đó tất nhiên rất có thể sẽ rất tốn kém, và như ta đã bàn, có một số công cụ rất mạnh mẽ mà ta sẽ không thể dùng nếu đi theo con đường này.

Trade-offs abound here. Balancing cost against ease of use, isolation, familiarity...it can become overwhelming. So let's review a set of rules I like to call Sam's Really Basic Rules of Thumb for Working Out Where to Deploy Stuff:

Trade-off đây rầy ở đây. Cân bằng chi phí với sự dễ dùng, sự cô lập, sự quen thuộc...có thể trở nên áp đảo. Vậy nên hãy điểm qua một tập quy tắc mà tôi thích gọi là Sam's Really Basic Rules of Thumb for Working Out Where to Deploy Stuff (Các Quy tắc Ngón tay Cái Thực sự Cơ bản của Sam về Việc Xác định Triển khai Mọi thứ ở Đâu):

1. If it ain't broke, don't fix it. 4
2. Give up as much control as you feel happy with, and then give away just a little bit more. If you can offload all your work to a good PaaS like Heroku (or a FaaS platform), then do it and be happy. Do you really need to tinker with every last setting?
3. Containerizing your microservices it is not pain-free, but is a really good compromise around cost of isolation and has some fantastic benefits for local development, while still giving you a degree of control over what happens. Expect Kubernetes in your future.
 1. Nếu nó không hỏng, đừng sửa nó. 4
 2. Từ bỏ càng nhiều quyền kiểm soát mà bạn cảm thấy thoải mái, rồi cho đi thêm một chút nữa. Nếu bạn có thể giảm tải toàn bộ công việc của mình cho một PaaS tốt như Heroku (hoặc một nền tảng FaaS), thì hãy làm thế và vui vẻ. Bạn có thực sự cần mà mò từng thiết lập cuối cùng không?
 3. Container hóa các microservice của bạn không phải không đau đớn, nhưng là một thỏa hiệp thực sự tốt quanh chi phí của sự cô lập và có một số lợi ích tuyệt vời cho việc phát triển cục bộ, trong khi vẫn cho bạn một mức độ kiểm soát về những gì xảy ra. Hãy kỳ vọng Kubernetes trong tương lai của bạn.

Many people are proclaiming "Kubernetes or bust!" which I feel is unhelpful. If you're on the public cloud, and your problem fits FaaS as a deployment model, do that instead and skip Kubernetes. Your developers will likely end up being much

Nhiều người đang tuyên bố "Kubernetes hoặc là chết!" mà tôi cảm thấy là không giúp ích. Nếu bạn ở trên public cloud, và vấn đề của bạn vừa khớp với FaaS như một mô hình triển khai, hãy làm điều đó thay vào đó và bỏ qua Kubernetes. Các lập trình viên của bạn nhiều khả năng rất cuộc sẽ

4 I might not have come up with this rule.

4 Có lẽ tôi không phải người nghĩ ra quy tắc này.

more productive. As we'll discuss more in Chapter 16, don't let the fear of lock-in keep you trapped in a mess of your own making.

có năng suất cao hơn nhiều. Như ta sẽ bàn thêm trong Chương 16, đừng để nỗi sợ bị khóa chặt (lock-in) giữ bạn mắc kẹt trong một mớ hỗn độn do chính mình tạo ra.

Found an awesome PaaS like Heroku or Zeit, and have an application that fits the constraints of the platform? Push all the work to the platform and spend more time working on your product. Both Heroku and Zeit are pretty fantastic platforms with awesome usability from a developer point of view. Don't your developers deserve to be happy after all?

Tìm được một PaaS tuyệt vời như Heroku hay Zeit, và có một ứng dụng vừa khớp với các ràng buộc của nền tảng? Hãy đẩy tất cả công việc cho nền tảng và dành nhiều thời gian hơn để làm việc trên sản phẩm của bạn. Cả Heroku lẫn Zeit đều là những nền tảng khá tuyệt vời với khả năng sử dụng tuyệt hảo từ góc nhìn của lập trình viên. Sau cùng thì các lập trình viên của bạn chẳng đáng được hạnh phúc sao?

For the rest of you, containerization is the way to go, which means we need to talk about Kubernetes.

Còn với phần còn lại các bạn, container hóa là con đường nên đi, điều đó nghĩa là ta cần nói về Kubernetes.

Role for Puppet, Chef, and Other Tools? — Vai trò của Puppet, Chef và Các Công cụ Khác?

This chapter has changed significantly since the first edition. This is due in part to the industry as a whole evolving, but also to new technology that has become increasingly useful. The emergence of new technology has also led to a diminished role for other technology—and so we see tools like Puppet, Chef, Ansible, and Salt playing a much smaller role in deploying microservice architectures than we did back in 2014.

Chương này đã thay đổi đáng kể kể từ ấn bản đầu tiên. Điều này một phần do toàn ngành công nghiệp nói chung tiến hóa, nhưng cũng do công nghệ mới đã ngày càng trở nên hữu ích. Sự xuất hiện của công nghệ mới cũng đã dẫn đến một vai trò giảm sút cho công nghệ khác—và vì vậy ta thấy các công cụ như Puppet, Chef, Ansible và Salt đóng một vai trò nhỏ hơn nhiều trong việc triển khai các kiến trúc microservice so với thời 2014.

The main reason for this is fundamentally the rise of the container. The power of tools like Puppet and Chef is that they give you a way to bring a machine to a desired state, with that desired state defined in some code form. You can define what runtimes you need, where configuration files need to be, etc., in a way that can deterministically be run time and again on the same machine, ensuring it can always be brought to the same state.

Lý do chính cho điều này về cơ bản là sự trỗi dậy của container. Sức mạnh của các công cụ như Puppet và Chef là chúng cho bạn một cách đưa một máy về trạng thái mong muốn, với trạng thái mong muốn đó được định nghĩa ở một dạng mã nào đó. Bạn có thể định nghĩa bạn cần các runtime nào, các file cấu hình cần ở đâu, v.v., theo một cách có thể được chạy một cách tất định lặp đi lặp lại trên cùng một máy, đảm bảo nó luôn có thể được đưa về cùng trạng thái.

The way most people build up a container is by defining a Dockerfile. This allows you to define the same requirements as you would with Puppet or Chef, with some differences. A container is blown away when redeployed, so each container creation is done from scratch (I'm simplifying somewhat here). This means that a lot of the complexity inherent in Puppet and Chef to handle those tools being run over and over on the same machines isn't needed.

Cách hầu hết mọi người dựng lên một container là bằng cách định nghĩa một Dockerfile. Điều này cho phép bạn định nghĩa các yêu cầu giống như bạn làm với Puppet hay Chef, với một số khác biệt. Một container bị thổi bay khi được triển khai lại, nên mỗi lần tạo container được thực hiện từ đầu (tôi đang đơn giản hóa đôi chút ở đây). Điều này nghĩa là rất nhiều sự phức tạp vốn có trong Puppet và Chef để xử lý việc các công cụ này được chạy đi chạy lại trên cùng các máy là không cần thiết.

Puppet, Chef, and similar tools are still incredibly useful, but their role has now been pushed out of the container and further down the stack. People use tools like these for managing legacy applications and infrastructure, or for building up the clusters that container workloads now run on. But developers are even less likely to come into contact with these tools than they were in the past.

Puppet, Chef và các công cụ tương tự vẫn cực kỳ hữu ích, nhưng vai trò của chúng giờ đã bị đẩy ra khỏi container và xuống sâu hơn trong ngăn xếp. Người ta dùng các công cụ như thế này để quản lý các ứng dụng và hạ tầng kế thừa, hoặc để dựng lên các cluster mà các tải công việc container giờ đây chạy trên đó. Nhưng lập trình viên thậm chí còn ít có khả năng tiếp xúc với các công cụ này hơn so với trong quá khứ.

The concept of infrastructure as code is still vitally important. It's just that the type of tools developers are likely to use has changed. For those working with the cloud, for example, things like Terraform can be very useful for provisioning cloud infrastructure. Recently, I've become a big fan of Pulumi, which eschews the use of domain-specific languages (DSLs) in favor of using normal programming languages to help developers manage their cloud infrastructure. I see big things ahead for Pulumi as delivery teams take more and more **ownership** of the operational world, and I suspect that Puppet, Chef, and the like, while they will continue to play a useful role in operations, will likely move further and further away from day-to-day development activities.

Khái niệm infrastructure as code vẫn cực kỳ quan trọng. Chỉ là loại công cụ mà lập trình viên có khả năng dùng đã thay đổi. Chẳng hạn, với những người làm việc với đám mây, những thứ như Terraform có thể rất hữu ích để cung cấp hạ tầng đám mây. Gần đây, tôi đã trở thành một fan lớn của Pulumi, vốn xa lánh việc dùng các ngôn ngữ chuyên biệt theo miền (DSL) để ưu tiên dùng các ngôn ngữ lập trình thông thường nhằm giúp lập trình viên quản lý hạ tầng đám mây của họ. Tôi thấy những điều lớn lao phía trước cho Pulumi khi các đội chuyển giao đảm nhận ngày càng nhiều quyền sở hữu thế giới vận hành, và tôi nghĩ rằng Puppet, Chef và những thứ tương tự, dù chúng sẽ tiếp tục đóng một vai trò hữu ích trong vận hành, nhiều khả năng sẽ dịch chuyển ngày càng xa khỏi các hoạt động phát triển hàng ngày.

Kubernetes and Container Orchestration — Kubernetes và Điều phối Container

As containers started gaining traction, many people started looking at solutions for how to manage containers across multiple machines. Docker had two attempts at this (with Docker Swarm and Docker Swarm Mode, respectively); companies like Rancher and CoreOS came up with their own

takes; and more general purpose platforms like Mesos were used to run containers alongside other sorts of workloads. Ultimately, though, despite a lot of effort on these products, Kubernetes has in the last couple of years come to dominate this space.

Khi container bắt đầu thu hút sự chú ý, nhiều người bắt đầu tìm kiếm các giải pháp về cách quản lý container trên nhiều máy. Docker đã có hai lần thử việc này (lần lượt với Docker Swarm và Docker Swarm Mode); các công ty như Rancher và CoreOS đã đưa ra cách làm riêng của họ; và các nền tảng đa dụng hơn như Mesos được dùng để chạy container song song với các loại tải công việc khác. Tuy nhiên, rốt cuộc, mặc dù có rất nhiều nỗ lực đổ vào các sản phẩm này, Kubernetes trong vài năm gần đây đã thống trị không gian này.

Before we speak to Kubernetes itself, we should discuss why there's a need for a tool like it in the first place.

Trước khi nói về chính Kubernetes, ta nên bàn tại sao lại có nhu cầu cho một công cụ như nó ngay từ đầu.

The Case for Container Orchestration — Lý do cho Điều phối Container

Broadly speaking, Kubernetes can variously be described as a container orchestration platform or, to use a term that has fallen out of favor, a container scheduler. So what are these platforms, and why might we want them?

Nói rộng ra, Kubernetes có thể được mô tả theo nhiều cách như một nền tảng điều phối container (container orchestration) hoặc, dùng một thuật ngữ đã không còn được ưa chuộng, một bộ lập lịch container (container scheduler). Vậy các nền tảng này là gì, và tại sao ta lại muốn chúng?

Containers are created by isolating a set of resources on an underlying machine. Tools like Docker allow us to define what a container should look like and create an instance of that container on a machine. But most solutions require that our software be defined on multiple machines, perhaps to handle sufficient load, or to ensure that the system has redundancy in place to tolerate the failure of a single node. Container orchestration platforms handle how and where container workloads are run. The term “scheduling” starts to make more sense in this context. The operator says, “I want this thing to run,” and the orchestrator works out how to schedule that job— finding available resources, reallocating them if necessary, and handling the details for the operator.

Container được tạo ra bằng cách cô lập một tập tài nguyên trên một máy bên dưới. Các công cụ như Docker cho phép ta định nghĩa một container nên trông như thế nào và tạo một instance của container đó trên một máy. Nhưng hầu hết các giải pháp đòi hỏi rằng phần mềm của ta được định nghĩa trên nhiều máy, có lẽ để xử lý đủ tải, hoặc để đảm bảo hệ thống có dự phòng sẵn sàng nhằm chịu được sự cố của một node đơn lẻ. Các nền tảng điều phối container xử lý cách thức và vị trí các tải công việc container được chạy. Thuật ngữ “lập lịch” (scheduling) bắt đầu có ý nghĩa hơn trong ngữ cảnh này. Người vận hành nói: “Tôi muốn thứ này chạy”, và bộ điều phối tính toán cách lập lịch cho công việc đó— tìm các tài nguyên sẵn có, tái phân bổ chúng nếu cần, và xử lý các chi tiết cho người vận hành.

The various container orchestration platforms also handle desired state management for us, ensuring that the expected state of a set of containers (microservice instances, in our case) is maintained. They also allow us to specify how we want these workloads to be distributed, allowing us to optimize for resource utilization, latency between processes, or robustness reasons.

Các nền tảng điều phối container khác nhau cũng xử lý quản lý trạng thái mong muốn cho ta, đảm bảo rằng trạng thái kỳ vọng của một tập container (các instance microservice, trong trường hợp của ta) được duy trì. Chúng cũng cho phép ta đặc tả ta muốn các tải công việc này được phân bổ ra sao, cho phép ta tối ưu hóa cho việc sử dụng tài nguyên, độ trễ giữa các tiến trình, hoặc vì lý do robustness.

Without such a tool, you'll have to manage the distribution of your containers, something that I can tell you from firsthand experience gets old very fast. Writing scripts to manage launching and networking container instances is not fun.

Không có một công cụ như vậy, bạn sẽ phải quản lý việc phân bổ các container của mình, một việc mà tôi có thể nói với bạn từ kinh nghiệm trực tiếp là rất nhanh chán. Viết các script để quản lý việc khởi chạy và nối mạng các instance container thì không vui chút nào.

Broadly speaking, all of the container orchestration platforms, including Kubernetes, provide these capabilities in some shape or form. If you look at general purpose schedulers like Mesos or Nomad, managed solutions like AWS's ECS, Docker Swarm Mode, and so on, you'll see a similar featureset. But for reasons we'll explore shortly, Kubernetes has won this space. It also has one or two interesting concepts that are worth exploring briefly.

Nói rộng ra, tất cả các nền tảng điều phối container, bao gồm Kubernetes, cung cấp các khả năng này theo hình thức này hay hình thức khác. Nếu bạn nhìn vào các bộ lập lịch đa dụng như Mesos hay Nomad, các giải pháp được quản lý như ECS của AWS, Docker Swarm Mode, v.v., bạn sẽ thấy một bộ tính năng tương tự. Nhưng vì những lý do ta sẽ khám phá ngay sau đây, Kubernetes đã thắng trong không gian này. Nó cũng có một hai khái niệm thú vị đáng để khám phá ngắn gọn.

A Simplified View of Kubernetes Concepts — Một Góc nhìn Đơn giản hóa về Các Khái niệm Kubernetes

There are many other concepts in Kubernetes, so you'll forgive me for not going into all of them (that would definitely justify a book in itself). What I'll try to do here is outline the key ideas you'll need to engage with when you first start working with the tool. Let's look into the concept of a cluster first, as shown in Figure 8-22 .

Có nhiều khái niệm khác trong Kubernetes, nên bạn sẽ thứ lỗi cho tôi vì không đi vào tất cả chúng (điều đó chắc chắn sẽ biện minh cho cả một cuốn sách riêng). Cái tôi sẽ cố làm ở đây là phác họa các ý tưởng then chốt mà bạn cần nắm khi lần đầu bắt đầu làm việc với công cụ này. Hãy đi vào khái niệm một cluster trước, như minh họa trong Hình 8-22.

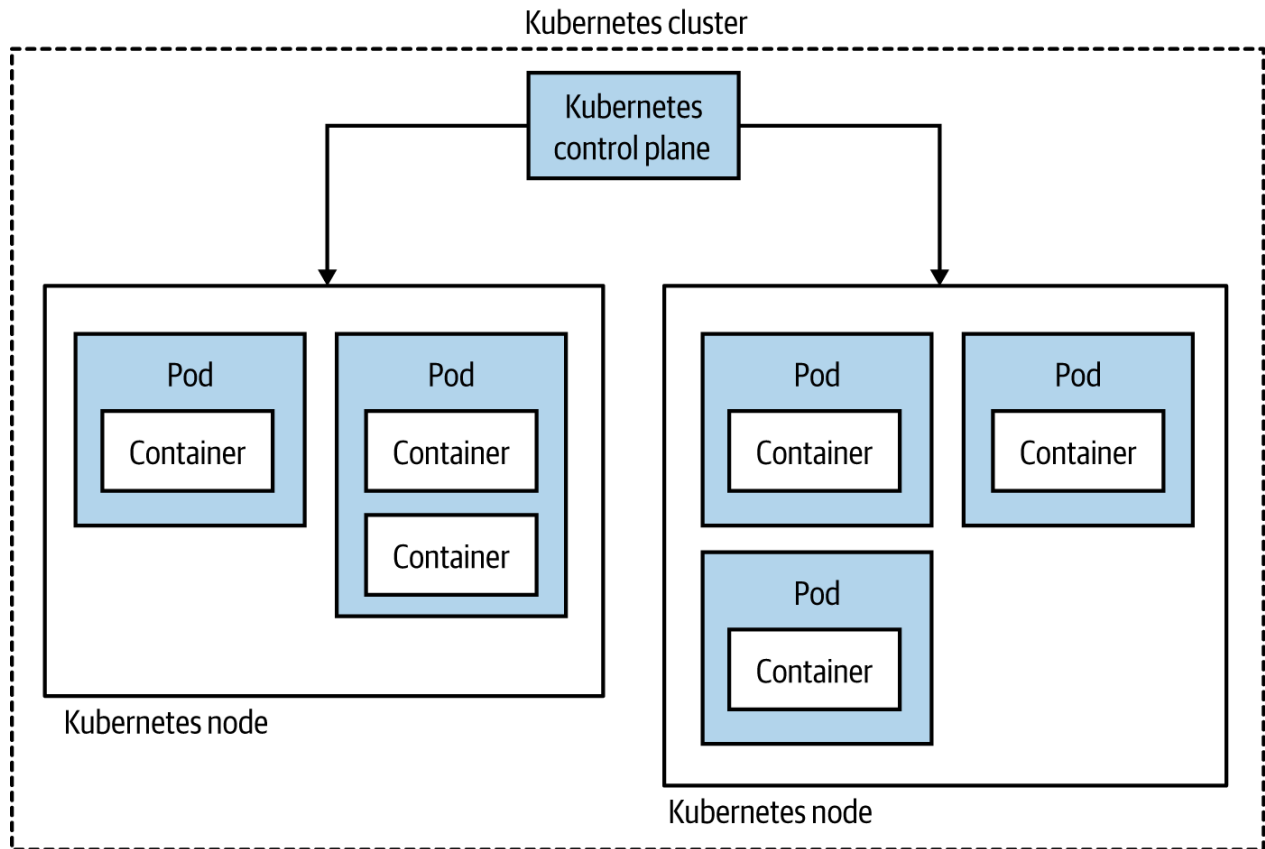


Figure 8-22. A simple overview of Kubernetes topology

Hình 8-22. Một tổng quan đơn giản về topology Kubernetes

Fundamentally, a Kubernetes cluster consists of two things. First, there's a set of machines that the workloads will run on called the nodes. Secondly, there's a set of controlling software that manages these nodes and is referred to as the control plane. These nodes could be running physical machines or virtual machines under the hood. Rather than scheduling a container, Kubernetes instead schedules something it calls a pod. A pod consists of one or more containers that will be deployed together.

Về cơ bản, một Kubernetes cluster gồm hai thứ. Thứ nhất, có một tập máy mà các tải công việc sẽ chạy trên đó gọi là các node. Thứ hai, có một tập phần mềm điều khiển quản lý các node này và được gọi là control plane. Các node này có thể chạy các máy vật lý hoặc máy ảo ở phía dưới. Thay vì lập lịch một container, Kubernetes thay vào đó lập lịch một thứ nó gọi là pod. Một pod gồm một hoặc nhiều container sẽ được triển khai cùng nhau.

Commonly, you'll have only one container in a pod—for example, an instance of your microservice. There are some occasions (rare, in my experience) where having multiple containers deployed together can make sense though. A good example of this is the use of sidecar proxies such as Envoy, often as part of a **service mesh**—a topic we discussed in “Service Meshes and API Gateways” on page 162.

Thông thường, bạn sẽ chỉ có một container trong một pod—ví dụ, một instance của microservice của bạn. Tuy nhiên có một số trường hợp (hiếm, theo kinh nghiệm của tôi) mà việc có nhiều container được triển khai cùng nhau có thể hợp lý. Một ví dụ hay về điều này là việc dùng các sidecar proxy như Envoy, thường như một phần của service mesh—một chủ đề ta đã bàn trong “Service Meshes and API Gateways” ở trang 162.

The next concept that is useful to know about is called a service . In the context of Kubernetes, you can think of a service as a stable routing endpoint—basically, a way to map from the pods you have running to a stable network interface that is available within the cluster. Kubernetes handles routing within the cluster, as we see in Figure 8-23 .

Khái niệm tiếp theo hữu ích cần biết được gọi là một service. Trong ngữ cảnh của Kubernetes, bạn có thể nghĩ về một service như một điểm cuối định tuyến (routing endpoint) ổn định—về cơ bản, một cách để ánh xạ từ các pod bạn đang chạy sang một giao diện mạng ổn định có sẵn bên trong cluster. Kubernetes xử lý việc định tuyến bên trong cluster, như ta thấy trong Hình 8-23.

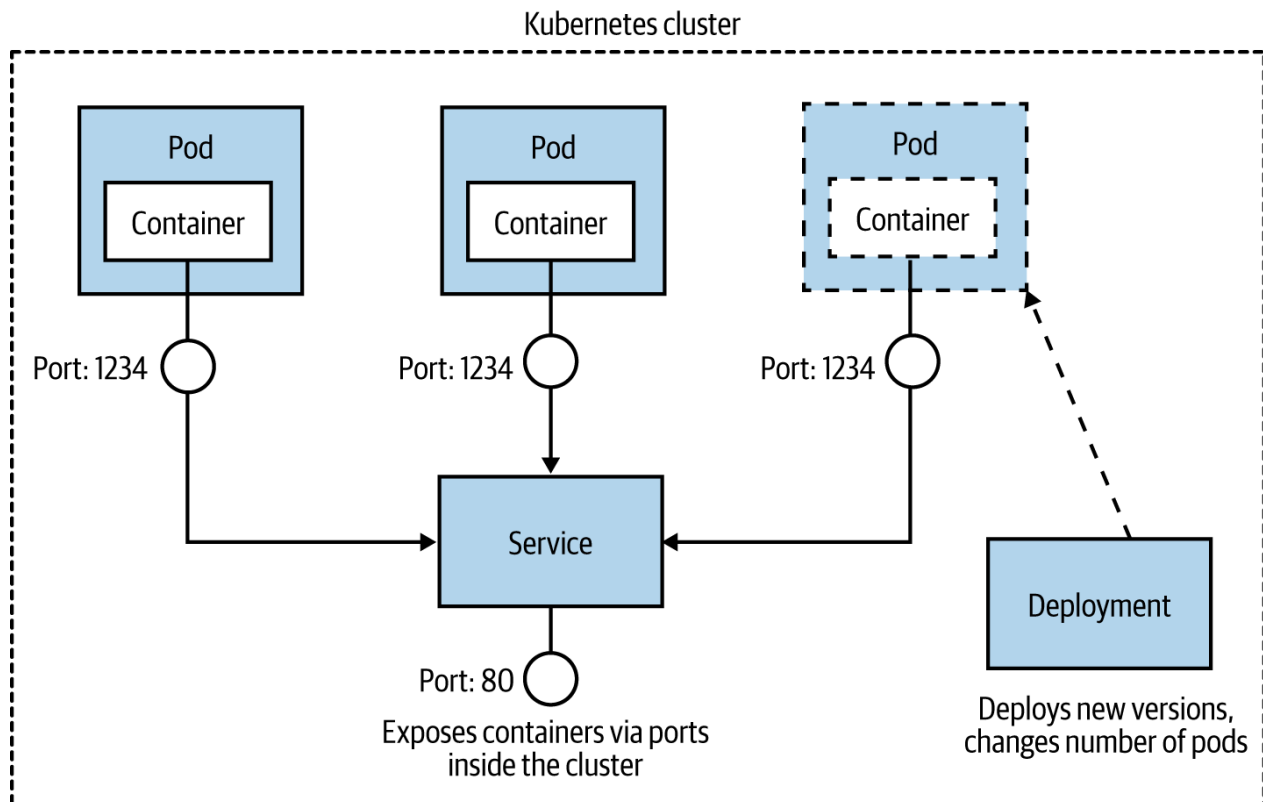


Figure 8-23. How a pod, a service, and a deployment work together

Hình 8-23. Cách một pod, một service và một deployment hoạt động cùng nhau

The idea is that a given pod can be considered ephemeral—it be might shut down for any number of reasons—whereas a service as a whole lives on. The service exists to route calls to and from the pods and can handle pods being shut down or new pods being launched. Purely from a terminology point of view, this can be confusing. We talk more generally about deploying a service, but in Kubernetes you don't deploy a service—you deploy pods that map to a service. It can take a while to get your head around this.

Ý tưởng là một pod nhất định có thể được coi là phù du (ephemeral)—nó có thể bị tắt vì vô số lý do—trong khi một service như một tổng thể vẫn tồn tại. Service tồn tại để định tuyến các lời gọi đến và đi từ các pod và có thể xử lý việc các pod bị tắt hay các pod mới được khởi chạy. Thuận túy từ góc độ thuật ngữ, điều này có thể gây nhầm lẫn. Ta nói chung chung về việc triển khai một service, nhưng trong Kubernetes bạn không triển khai một service—bạn triển khai các pod ánh xạ tới một service. Có thể mất một lúc để thông được điều này.

Next, we have a replica set . With a replica set, you define the desired state of a set of pods. This is where you say, “I want four of these pods,” and Kubernetes handles the rest. In practice, you are no longer expected to work with replica sets directly; instead, they are handled for you via a deployment , the last concept we’ll look at. A deployment is how you apply changes to your pods and replica sets. With a deployment, you can do things like issue rolling upgrades (so you replace pods with a newer version in a gradual fashion to avoid downtime), rollbacks, scaling up the number of nodes, and more.

Tiếp theo, ta có một replica set. Với một replica set, bạn định nghĩa trạng thái mong muốn của một tập pod. Đây là nơi bạn nói: “Tôi muốn bốn pod loại này”, và Kubernetes xử lý phần còn lại. Trên thực tế, bạn không còn được kỳ vọng làm việc trực tiếp với replica set nữa; thay vào đó, chúng được xử lý cho bạn thông qua một deployment, khái niệm cuối cùng ta sẽ xem xét. Một deployment là cách bạn áp dụng các thay đổi cho các pod và replica set của mình. Với một deployment, bạn có thể làm những việc như phát hành các rolling upgrade (để bạn thay thế các pod bằng một phiên bản mới hơn theo kiểu dần dần nhằm tránh gián đoạn), rollback, mở rộng số lượng node và nhiều hơn nữa.

So, to deploy your microservice, you define a pod , which will contain your microservice instance inside it; you define a service , which will let Kubernetes know how your microservice will be accessed; and you apply changes to the running pods using a deployment . It seems easy when I say that, doesn’t it? Let’s just say I’ve left out quite a bit of stuff here for the sake of brevity.

Vậy nên, để triển khai microservice của bạn, bạn định nghĩa một pod, vốn sẽ chứa instance microservice của bạn bên trong nó; bạn định nghĩa một service, vốn sẽ cho Kubernetes biết microservice của bạn sẽ được truy cập ra sao; và bạn áp dụng các thay đổi cho các pod đang chạy bằng một deployment. Nghe có vẻ dễ khi tôi nói thế, phải không? Cứ tạm nói là tôi đã bỏ qua khá nhiều thứ ở đây cho ngắn gọn.

Multitenancy and Federation — Đa người thuê (Multitenancy) và Liên kết (Federation)

From an efficiency point of view, you’d want to pool all the computing resources available to you in a single Kubernetes cluster and have all workloads run there from all across your organization. This would likely give you a higher utilization of the underlying resources, as unused resources could be freely reallocated to whomever needs them. This in turn should reduce costs accordingly.

Từ góc độ hiệu quả, bạn sẽ muốn gộp tất cả tài nguyên tính toán sẵn có của mình vào một Kubernetes cluster duy nhất và để tất cả các tải công việc chạy ở đó từ khắp tổ chức của bạn. Điều này nhiều khả năng cho bạn mức sử dụng cao hơn của các tài nguyên bên dưới, vì các tài nguyên không dùng có thể được tái phân bổ tự do cho bất cứ ai cần chúng. Điều này lần lượt nên giảm chi phí một cách tương ứng.

The challenge is that while Kubernetes is well able to manage different microservices for different purposes, it has limitations regarding how “multitenanted” the platform is. Different departments in your organization might want different degrees of control over various resources. These sorts of controls were not built into Kubernetes, a decision that seems sensible in terms of trying to keep the scope of Kubernetes somewhat limited. To work around this problem, organizations seem to explore a couple of different paths.

Thách thức là dù Kubernetes có khả năng quản lý tốt các microservice khác nhau cho các mục đích khác nhau, nó có những hạn chế về việc nền tảng “đa người thuê” (multitenanted) đến mức nào. Các phòng ban khác nhau trong tổ chức của bạn có thể

muốn các mức độ kiểm soát khác nhau đối với nhiều tài nguyên khác nhau. Những loại kiểm soát này không được xây dựng sẵn trong Kubernetes, một quyết định có vẻ hợp lý về mặt cố giữ phạm vi của Kubernetes ở mức hạn chế phần nào. Để xoay xở quanh vấn đề này, các tổ chức dường như khám phá vài con đường khác nhau.

The first option is to adopt a platform built on top of Kubernetes that provides these capabilities—OpenShift from Red Hat, for example, has a rich set of access controls and other capabilities that are built with larger organizations in mind and can make the concept of multitenancy somewhat easier. Aside from any financial implication of using these sorts of platforms, for them to work you'll sometimes have to work with the abstractions given to you by the vendor you chose—meaning your developers need to know not only how to use Kubernetes but also how to use that specific vendor's platform.

Lựa chọn đầu tiên là áp dụng một nền tảng được xây dựng trên Kubernetes cung cấp các khả năng này—chẳng hạn, OpenShift từ Red Hat có một bộ kiểm soát truy cập phong phú và các khả năng khác được xây dựng với các tổ chức lớn hơn trong đầu và có thể làm cho khái niệm multitenancy dễ dàng hơn phần nào. Bỏ qua bất kỳ hệ quả tài chính nào của việc dùng các loại nền tảng này, để chúng hoạt động đôi khi bạn sẽ phải làm việc với các trừu tượng hóa mà nhà cung cấp bạn chọn đưa cho bạn—nghĩa là lập trình viên của bạn cần biết không chỉ cách dùng Kubernetes mà còn cách dùng nền tảng cụ thể của nhà cung cấp đó.

Another approach is to consider a federated model, outlined in Figure 8-24 . With federation, you have multiple separate clusters, with some layer of software that sits on top allowing you to make changes across all the clusters if needed. In many cases, people would work directly against one cluster, giving them a pretty familiar Kubernetes experience, but in some situations, you may want to distribute an application across multiple clusters, perhaps if those clusters were in different geographies and you wanted your application deployed with some ability to handle the loss of an entire cluster.

Một cách tiếp cận khác là xem xét một mô hình liên kết (federated), được phác thảo trong Hình 8-24. Với liên kết, bạn có nhiều cluster riêng biệt, với một lớp phần mềm nào đó nằm trên cùng cho phép bạn thực hiện các thay đổi trên tất cả các cluster nếu cần. Trong nhiều trường hợp, người ta sẽ làm việc trực tiếp với một cluster, cho họ một trải nghiệm Kubernetes khá quen thuộc, nhưng trong một số tình huống, bạn có thể muốn phân bố một ứng dụng trên nhiều cluster, có lẽ nếu các cluster đó ở các vùng địa lý khác nhau và bạn muốn ứng dụng của mình được triển khai với một khả năng nào đó để xử lý việc mất cả một cluster.

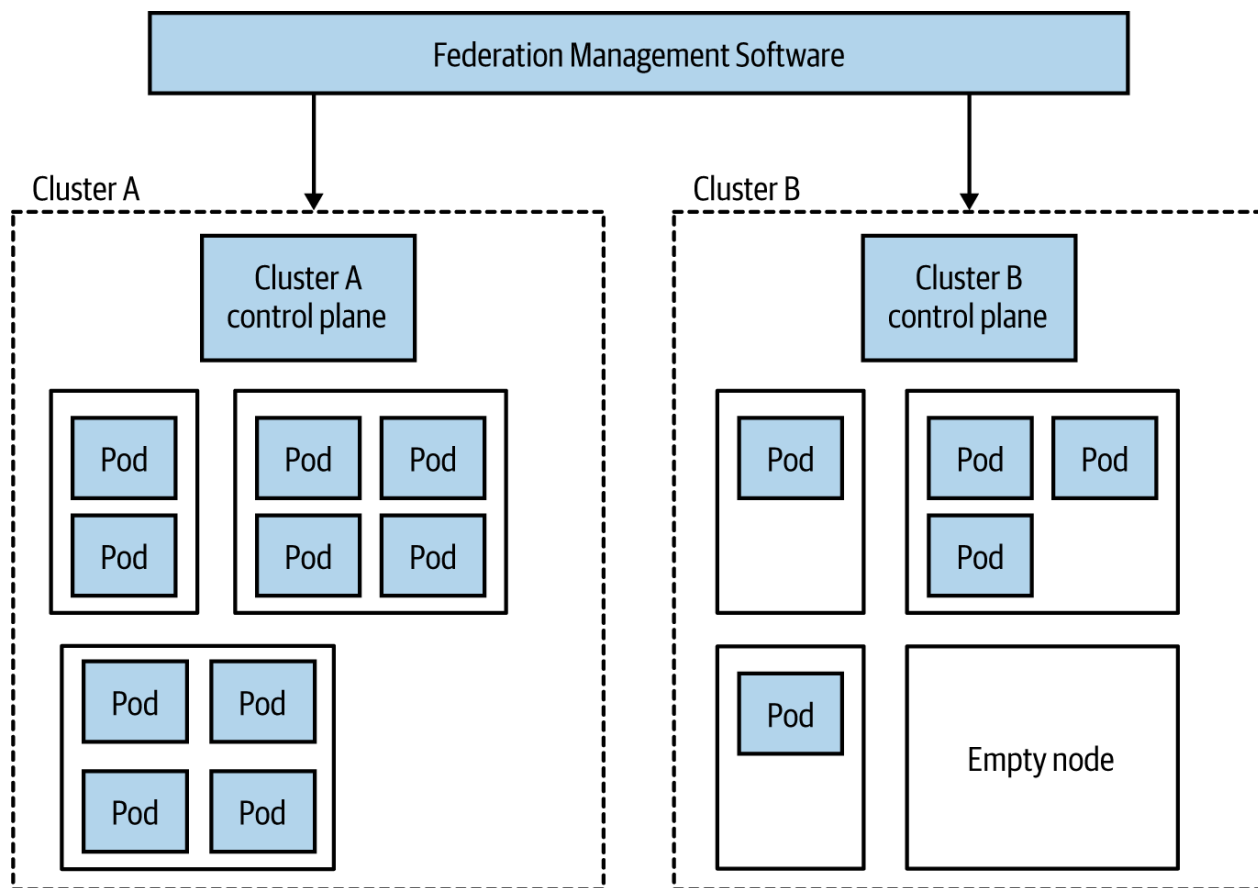


Figure 8-24. An example of federation in Kubernetes

Hình 8-24. Một ví dụ về liên kết trong Kubernetes

The federated nature makes resource pooling more challenging. As we see in Figure 8-24, Cluster A is fully utilized, whereas Cluster B has lots of unused capacity. If we wanted to run more workloads on Cluster A, doing so would be possible only if we could give it more resources, such as moving the empty node on Cluster B over to Cluster A. How easy it will be to move the node from one cluster to the other will depend on the nature of the federation software being used, but I can well imagine this being a nontrivial change. Bear in mind that a single node can be part of either one cluster or another and so cannot run pods for both Cluster A and Cluster B.

Bản chất liên kết làm cho việc gộp tài nguyên thách thức hơn. Như ta thấy trong Hình 8-24, Cluster A được sử dụng đầy đủ, trong khi Cluster B có nhiều năng lực chưa dùng. Nếu ta muốn chạy thêm tải công việc trên Cluster A, làm vậy chỉ khả thi nếu ta có thể cho nó thêm tài nguyên, chẳng hạn như chuyển node trống trên Cluster B sang Cluster A. Việc chuyển node từ cluster này sang cluster khác để đến đâu sẽ phụ thuộc vào bản chất của phần mềm liên kết được dùng, nhưng tôi hoàn toàn có thể hình dung đây là một thay đổi không hề tầm thường. Hãy nhớ rằng một node đơn lẻ có thể là một phần của hoặc cluster này hoặc cluster khác và do đó không thể chạy các pod cho cả Cluster A lẫn Cluster B.

It's worth noting that having multiple clusters can be beneficial when we consider the challenges of upgrading the cluster itself. It may be easier and safer to move a microservice over to a newly upgraded cluster than to upgrade the cluster in-place.

Cũng đáng lưu ý rằng việc có nhiều cluster có thể có lợi khi ta xét đến các thách thức của việc nâng cấp chính cluster đó. Việc chuyển một microservice sang một cluster mới được

nâng cấp có thể dễ hơn và an toàn hơn so với nâng cấp cluster tại chỗ (in-place).

Fundamentally, these are challenges of scale. For some organizations, you'll never have these issues, as you're happy to share a single cluster. For other organizations looking to get efficiencies at larger scale, this is certainly an area that you'll want to explore in more detail. It should be noted that there are a number of different visions for what Kubernetes federation should look like, and a number of different toolchains out there for managing them.

Về cơ bản, đây là những thách thức của quy mô. Với một số tổ chức, bạn sẽ không bao giờ gặp những vấn đề này, vì bạn hài lòng với việc dùng chung một cluster duy nhất. Với các tổ chức khác đang tìm cách đạt hiệu quả ở quy mô lớn hơn, đây chắc chắn là một lĩnh vực bạn sẽ muốn khám phá chi tiết hơn. Cần lưu ý rằng có một số tầm nhìn khác nhau về việc liên kết Kubernetes nên trông như thế nào, và một số bộ công cụ khác nhau ngoài kia để quản lý chúng.

The Background Behind Kubernetes — Bối cảnh Đằng sau Kubernetes

Kubernetes started as an open source project at Google, which drew its inspiration from the earlier container management systems Omega and Borg. Many of the core concepts in Kubernetes are based on concepts around how container workloads are managed inside Google, albeit with a slightly different target in mind. Borg runs systems at a massive global scale, handling tens if not hundreds of thousands of containers across data centers globally. If you want more detail about how the different mindsets behind these three Google platforms compare, albeit from a Google-centric viewpoint, I recommend “Borg, Omega, and Kubernetes” by Brendan Burns et al. as a good overview.

Kubernetes khởi đầu như một dự án mã nguồn mở tại Google, vốn lấy cảm hứng từ các hệ thống quản lý container trước đó là Omega và Borg. Nhiều khái niệm cốt lõi trong Kubernetes dựa trên các khái niệm quanh cách các tải công việc container được quản lý bên trong Google, dù với một mục tiêu hơi khác trong đầu. Borg chạy các hệ thống ở quy mô toàn cầu khổng lồ, xử lý hàng chục nếu không phải hàng trăm ngàn container trên các trung tâm dữ liệu khắp toàn cầu. Nếu bạn muốn thêm chi tiết về cách các tư duy khác nhau đằng sau ba nền tảng này của Google so sánh ra sao, dù từ một góc nhìn lấy Google làm trung tâm, tôi khuyên đọc “Borg, Omega, and Kubernetes” của Brendan Burns và cộng sự như một tổng quan tốt.

While Kubernetes shares some DNA with Borg and Omega, working at massive scale has not been a main driver behind the project. Both Nomad and Mesos (each of which took a cue from Borg) have found a niche in situations in which clusters of thousands of machines are needed, as demonstrated in Apple's use of Mesos for Siri 5 or Roblox's use of Nomad .

Mặc dù Kubernetes chia sẻ một số ADN với Borg và Omega, việc làm việc ở quy mô khổng lồ không phải là động lực chính đằng sau dự án. Cả Nomad lẫn Mesos (mỗi cái lấy gợi ý từ Borg) đã tìm được một thị trường ngách trong các tình huống cần các cluster gồm hàng ngàn máy, như được minh chứng qua việc Apple dùng Mesos cho Siri 5 hay việc Roblox dùng Nomad.

Kubernetes wanted to take ideas from Google but provide a more developer-friendly experience than that delivered by Borg or Omega. It's possible to look at Google's decision to invest a lot of engineering effort in creating an open source tool in a purely altruistic light, and while I'm sure that was the intention of some people, the reality is that this is as much about the risk Google was seeing from competition in the public cloud space, specifically AWS.

Kubernetes muốn lấy các ý tưởng từ Google nhưng cung cấp một trải nghiệm thân thiện với lập trình viên hơn so với cái mà Borg hay Omega mang lại. Có thể nhìn quyết định của Google đầu tư rất nhiều công sức kỹ thuật vào việc tạo ra một công cụ mã nguồn mở dưới ánh sáng thuần túy vị tha, và mặc dù tôi chắc rằng đó là ý định của một số người, thực tế là điều này cũng nhiều phần liên quan đến rủi ro mà Google đang thấy từ sự cạnh tranh trong không gian public cloud, cụ thể là AWS.

In the public cloud market, Google Cloud has gained ground but is still a distant third behind Azure and AWS (who are out in front), and some analysis has it being edged into fourth place by Alibaba Cloud. Despite the improving market share, it is still nowhere near where Google wants it to be.

Trong thị trường public cloud, Google Cloud đã giành được chỗ đứng nhưng vẫn là một hạng ba cách biệt sau Azure và AWS (những kẻ dẫn đầu), và một số phân tích cho rằng nó đang bị Alibaba Cloud đẩy xuống hạng tư. Bất chấp thị phần đang cải thiện, nó vẫn còn xa nơi Google muốn nó ở.

It seems likely that a major concern was that the clear market leader, AWS, could eventually have a near monopoly in the cloud computing space. Moreover, concerns regarding the cost of migration from one provider to another meant that such a position of market dominance would be hard to shift. And then along comes Kubernetes, with its promise of being able to deliver a standard platform for running container workloads that could be run by multiple vendors. The hope was that this would enable migration from one provider to another and avoid an AWS-only future.

Có vẻ khả dĩ rằng một mối bận tâm lớn là kẻ dẫn đầu thị trường rõ ràng, AWS, rốt cuộc có thể có gần như độc quyền trong không gian điện toán đám mây. Hơn nữa, các mối bận tâm về chi phí di trú từ nhà cung cấp này sang nhà cung cấp khác nghĩa là một vị thế thống trị thị trường như vậy sẽ khó lay chuyển. Và rồi Kubernetes xuất hiện, với lời hứa về khả năng cung cấp một nền tảng chuẩn để chạy các tải công việc container vốn có thể được chạy bởi nhiều nhà cung cấp. Hy vọng là điều này sẽ cho phép di trú từ nhà cung cấp này sang nhà cung cấp khác và tránh một tương lai chỉ-có-AWS.

So you can see Kubernetes as a generous contribution from Google to the wider IT industry, or as an attempt by Google to remain relevant in the fast-moving public cloud space. I have no problem seeing both things as equally true.

Vậy nên bạn có thể nhìn Kubernetes như một đóng góp hào phóng từ Google cho ngành CNTT rộng hơn, hoặc như một nỗ lực của Google để duy trì sự liên quan trong không gian public cloud chuyển động nhanh. Tôi không gặp vấn đề gì khi nhìn cả hai điều này đều đúng như nhau.

5 Daniel Bryant, “Apple Rebuilds Siri **Backend** Services Using Apache Mesos,” InfoQ, May 3, 2015, <https://oreil.ly/NLUMX>.

5 Daniel Bryant, “Apple Rebuilds Siri Backend Services Using Apache Mesos,” InfoQ, May 3, 2015, <https://oreil.ly/NLUMX>.

The Cloud Native Computing Federation — Cloud Native Computing Federation

The Cloud Native Computing Foundation (CNCF for short) is an offshoot of the nonprofit Linux Foundation. The CNCF focuses on curating the ecosystem of projects to help promote cloud native development, although in practice this means supporting Kubernetes and projects that work with or build on Kubernetes itself. The projects themselves aren’t created or directly developed by

the CNCF; instead, you can see the CNCF as a place where these projects that might otherwise be developed in isolation can be hosted together in the same place, and where common standards and interoperability can be developed.

Cloud Native Computing Foundation (gọi tắt là CNCF) là một nhánh tách ra của tổ chức phi lợi nhuận Linux Foundation. CNCF tập trung vào việc tuyển chọn hệ sinh thái các dự án nhằm giúp thúc đẩy phát triển cloud native, mặc dù trên thực tế điều này nghĩa là hỗ trợ Kubernetes và các dự án làm việc với hoặc xây dựng trên chính Kubernetes. Bản thân các dự án không được tạo ra hay phát triển trực tiếp bởi CNCF; thay vào đó, bạn có thể nhìn CNCF như một nơi mà các dự án này, vốn nếu không sẽ được phát triển một cách cô lập, có thể được lưu trữ cùng nhau tại cùng một chỗ, và nơi các chuẩn chung cùng khả năng tương tác (interoperability) có thể được phát triển.

In this way, the CNCF reminds me of the role of the Apache Software Foundation— as with the CNCF, a project being part of the Apache Software Foundation normally implies a level of quality and wider community support. All of the projects hosted by the CNCF are open source, although the development of these projects may well be driven by commercial entities.

Theo cách này, CNCF gợi tôi nhớ đến vai trò của Apache Software Foundation—cũng như với CNCF, việc một dự án là một phần của Apache Software Foundation thường ngụ ý một mức độ chất lượng và sự hỗ trợ cộng đồng rộng hơn. Tất cả các dự án được CNCF lưu trữ đều là mã nguồn mở, mặc dù việc phát triển các dự án này rất có thể được dẫn dắt bởi các thực thể thương mại.

Aside from helping guide the development of these associated projects, the CNCF also runs events, provides documentation and training materials, and defines the various certification programs around Kubernetes. The group itself has members from across the industry, and although it can be difficult for smaller groups or independents to play much of a role in the organization itself, the degree of cross-industry support (including many companies who are competitors with each other) is impressive.

Bên cạnh việc giúp dẫn dắt sự phát triển của các dự án liên quan này, CNCF cũng tổ chức các sự kiện, cung cấp tài liệu và tài liệu đào tạo, và định nghĩa các chương trình chứng nhận khác nhau quanh Kubernetes. Bản thân nhóm này có thành viên từ khắp ngành công nghiệp, và mặc dù có thể khó cho các nhóm nhỏ hơn hay các cá nhân độc lập đóng vai trò lớn trong chính tổ chức, mức độ hỗ trợ xuyên ngành (bao gồm nhiều công ty là đối thủ cạnh tranh của nhau) là ấn tượng.

As an outsider, the CNCF seems to have had great success in helping spread the word regarding the usefulness of the projects it curates. It's also acted as a place where the evolution of major projects can be discussed in the open, ensuring a lot of broad input. The CNCF has played a huge part in the success of Kubernetes—it's easy to imagine that without it, we'd still have a fragmented landscape in this area.

Là một người ngoài cuộc, CNCF dường như đã có thành công lớn trong việc giúp lan tỏa thông điệp về tính hữu dụng của các dự án mà nó tuyển chọn. Nó cũng đã hoạt động như một nơi mà sự tiến hóa của các dự án lớn có thể được thảo luận một cách công khai, đảm bảo rất nhiều đầu vào rộng rãi. CNCF đã đóng một vai trò khổng lồ trong thành công của Kubernetes—dễ hình dung rằng nếu không có nó, ta vẫn sẽ có một bối cảnh phân mảnh trong lĩnh vực này.

Platforms and Portability — Nền tảng và Tính Di động

You'll often hear Kubernetes described as a “platform.” It's not really a platform in the sense that a developer would understand the term, though. Out of the box, all it really gives you is the ability to run container workloads. Most folks using Kubernetes end up assembling their own platform by installing supporting software such as service meshes, message brokers, **log aggregation** tools, and more. In larger organizations, this ends up being the responsibility of a platform engineering team, who put this platform together and manage it, and help developers use the platform effectively.

Bạn sẽ thường nghe Kubernetes được mô tả như một “nền tảng” (platform). Tuy nhiên, nó không thực sự là một nền tảng theo nghĩa mà một lập trình viên sẽ hiểu về thuật ngữ đó. Ngay khi mở hộp (out of the box), tất cả những gì nó thực sự cho bạn là khả năng chạy các tải công việc container. Hầu hết những người dùng Kubernetes rất cuộc tự lắp ráp nền tảng riêng của họ bằng cách cài đặt các phần mềm hỗ trợ như service mesh, message broker, các công cụ gom log và nhiều hơn nữa. Trong các tổ chức lớn hơn, điều này rất cuộc trở thành trách nhiệm của một đội kỹ thuật nền tảng (platform engineering team), vốn lắp ráp nền tảng này lại với nhau và quản lý nó, và giúp lập trình viên dùng nền tảng một cách hiệu quả.

This can be both a blessing and a curse. This pick-and-mix approach is made possible due to a fairly compatible ecosystem of tools (thanks in large part to the work by the CNCF). This means you can select your favorite tools for specific tasks if you want. But it can also lead to the tyranny of choice—we can easily become overwhelmed with so many options. Products like Red Hat's OpenShift partly take this choice away from us, as they give us a ready-made platform with some decisions already made for us.

Điều này có thể vừa là một phước lành vừa là một lời nguyền. Cách tiếp cận chọn-và-trộn (pick-and-mix) này khả thi nhờ một hệ sinh thái công cụ khá tương thích (phần lớn nhờ công sức của CNCF). Điều này nghĩa là bạn có thể chọn các công cụ ưa thích của mình cho các tác vụ cụ thể nếu muốn. Nhưng nó cũng có thể dẫn đến sự bạo ngược của lựa chọn—ta có thể dễ dàng bị áp đảo với quá nhiều lựa chọn. Các sản phẩm như OpenShift của Red Hat một phần lấy đi lựa chọn này khỏi ta, vì chúng cho ta một nền tảng làm sẵn với một số quyết định đã được đưa ra sẵn cho ta.

What this means is that although at the base level Kubernetes offers a portable abstraction for container execution, in practice it's not as simple as taking an application that works on one cluster and expecting it will work elsewhere. Your application, your operations and developer workflow, may well rely on your own custom platform. Moving from one Kubernetes cluster to another may well also require that you rebuild that platform on your new destination. I've spoken to many organizations that have adopted Kubernetes primarily because they're worried about being locked in to a single vendor, but these organizations haven't understood this nuance—applications built on Kubernetes are portable across Kubernetes clusters in theory, but not always in practice.

Điều này nghĩa là mặc dù ở mức cơ sở Kubernetes cung cấp một sự trừu tượng hóa di động cho việc thực thi container, trên thực tế nó không đơn giản như lấy một ứng dụng hoạt động trên một cluster và kỳ vọng nó sẽ hoạt động ở nơi khác. Ứng dụng của bạn, các workflow vận hành và phát triển của bạn, rất có thể dựa vào nền tảng tùy chỉnh của riêng bạn. Việc chuyển từ một Kubernetes cluster này sang một cluster khác rất có thể cũng đòi hỏi bạn xây dựng lại nền tảng đó trên đích mới của mình. Tôi đã nói chuyện với nhiều tổ chức đã áp dụng Kubernetes chủ yếu vì họ lo lắng về việc bị khóa chặt vào một nhà cung cấp duy nhất, nhưng các tổ chức này đã không hiểu sắc thái này—các ứng dụng xây dựng trên Kubernetes là di động được giữa các Kubernetes cluster về mặt lý thuyết, nhưng không phải lúc nào cũng trên thực tế.

Helm, Operators, and CRDs, Oh My! — Helm, Operator và CRD, Ôi Trời!

One area of continuing confusion in the space of Kubernetes is how to manage the deployment and life cycle of third-party applications and subsystems. Consider the need to run Kafka on your Kubernetes cluster. You could create your own pod, service, and deployment specifications and run them yourself. But what about managing an upgrade to your Kafka setup? What about other common maintenance tasks you might want to deal with, like upgrading running stateful software?

Một lĩnh vực gây nhầm lẫn liên tục trong không gian Kubernetes là cách quản lý việc triển khai và vòng đời của các ứng dụng và hệ thống con bên thứ ba. Hãy xét nhu cầu chạy Kafka trên Kubernetes cluster của bạn. Bạn có thể tạo các đặc tả pod, service và deployment của riêng mình và tự chạy chúng. Nhưng còn việc quản lý một lần nâng cấp cho thiết lập Kafka của bạn thì sao? Còn các tác vụ bảo trì phổ biến khác mà bạn có thể muốn xử lý, như nâng cấp phần mềm có trạng thái (stateful) đang chạy thì sao?

A number of tools have emerged that aim to give you the ability to manage these types of applications at a more sensible level of abstraction. The idea is that someone creates something akin to a package for Kafka, and you run it on your Kubernetes cluster in a more black-box manner. Two of the best-known solutions in this space are Operator and Helm. Helm bills itself as “the missing package manager” for Kubernetes, and while Operator can manage initial installation, it seems to be focused more on the ongoing management of the application. Confusingly, while you can see Operator and Helm as being alternatives to one another, you can also use both of them together in some situations (Helm for initial install, Operator for life-cycle operations).

Một số công cụ đã xuất hiện nhằm cho bạn khả năng quản lý các loại ứng dụng này ở một mức trừu tượng hợp lý hơn. Ý tưởng là một ai đó tạo ra một thứ na ná một gói (package) cho Kafka, và bạn chạy nó trên Kubernetes cluster của mình theo một cách kiểu hộp-đen hơn. Hai trong số các giải pháp nổi tiếng nhất trong không gian này là Operator và Helm. Helm tự quảng bá mình là “trình quản lý gói còn thiếu” (the missing package manager) cho Kubernetes, và trong khi Operator có thể quản lý việc cài đặt ban đầu, nó dường như tập trung nhiều hơn vào việc quản lý liên tục của ứng dụng. Một cách khó hiểu, dù bạn có thể nhìn Operator và Helm như những lựa chọn thay thế cho nhau, bạn cũng có thể dùng cả hai cùng nhau trong một số tình huống (Helm cho cài đặt ban đầu, Operator cho các thao tác vòng đời).

A more recent evolution in this space is something called custom resource definitions, or CRDs. With CRDs you can extend the core Kubernetes APIs, allowing you to plug in new behavior to your cluster. The nice thing about CRDs is that they integrate fairly seamlessly into the existing command-line interface, access controls, and more—so your custom extension doesn’t feel like an alien addition. They basically allow you to implement your own Kubernetes abstractions. Think of the pod, replica set, service, and deployment abstractions we discussed earlier—with CRDs, you could add your own into the mix.

Một sự tiến hóa gần đây hơn trong không gian này là một thứ gọi là custom resource definition, hay CRD. Với CRD bạn có thể mở rộng các API cốt lõi của Kubernetes, cho phép bạn cắm thêm hành vi mới vào cluster của mình. Điều hay về CRD là chúng tích hợp khá liền mạch vào giao diện dòng lệnh, kiểm soát truy cập và nhiều thứ khác hiện có—nên phần mở rộng tùy chỉnh của bạn không cảm thấy như một bổ sung ngoại lai. Về cơ bản chúng cho phép bạn hiện thực các trừu tượng hóa Kubernetes của riêng bạn. Hãy nghĩ về các trừu tượng hóa pod, replica set, service và deployment mà ta đã bàn trước đó—với CRD, bạn có thể thêm các trừu tượng hóa của riêng mình vào hỗn hợp.

You can use CRDs for everything from managing small bits of configuration to controlling service meshes like Istio or cluster-based software like Kafka. With such a flexible and powerful concept, I find it difficult to understand where CRDs would best be used, and there doesn't seem to be a general consensus out there among the experts I've chatted to either. This whole space still doesn't seem to be settling down as quickly as I'd hoped, and there isn't as much consensus as I'd like—a trend in the Kubernetes ecosystem.

Bạn có thể dùng CRD cho mọi thứ từ việc quản lý các mẫu cấu hình nhỏ đến việc điều khiển các service mesh như Istio hay phần mềm dựa trên cluster như Kafka. Với một khái niệm linh hoạt và mạnh mẽ như vậy, tôi thấy khó hiểu được nơi nào CRD sẽ được dùng tốt nhất, và dường như cũng không có một sự đồng thuận chung ngoài kia giữa các chuyên gia tôi đã trò chuyện. Cả không gian này dường như vẫn chưa lắng xuống nhanh như tôi đã hy vọng, và không có nhiều sự đồng thuận như tôi mong muốn—một xu hướng trong hệ sinh thái Kubernetes.

And Knative — Và Knative

Knative is an open source project that aims to provide FaaS-style workflows to developers, using Kubernetes under the hood. Fundamentally, Kubernetes isn't terribly developer friendly, especially if we compare it to the usability of things like Heroku or similar platforms. The aim with Knative is to bring the developer experience of FaaS to Kubernetes, hiding the complexity of Kubernetes from developers. In turn, this should mean development teams are able to more easily manage the full life cycle of their software.

Knative là một dự án mã nguồn mở nhằm cung cấp các workflow kiểu FaaS cho lập trình viên, dùng Kubernetes ở phía dưới. Về cơ bản, Kubernetes không thân thiện với lập trình viên cho lắm, đặc biệt nếu ta so sánh nó với khả năng sử dụng của những thứ như Heroku hay các nền tảng tương tự. Mục tiêu với Knative là mang trải nghiệm phát triển của FaaS đến Kubernetes, ẩn đi sự phức tạp của Kubernetes khỏi lập trình viên. Đến lượt nó, điều này nên nghĩa là các đội phát triển có thể dễ dàng hơn trong việc quản lý toàn bộ vòng đời của phần mềm của họ.

We've already discussed service meshes, and specifically mentioned Istio, back in Chapter 5 . A service mesh is essential for Knative to run. While Knative theoretically allows you to plug in different service meshes, only Istio is considered stable at this time (with support for other meshes like Ambassador and Gloo still in alpha). In practice, this means that if you want to adopt Knative, you'll also already have to have bought into Istio.

Ta đã bàn về service mesh, và cụ thể đã nhắc đến Istio, từ Chương 5. Một service mesh là thiết yếu để Knative chạy. Mặc dù Knative về lý thuyết cho phép bạn cắm thêm các service mesh khác nhau, chỉ Istio được coi là ổn định vào thời điểm này (với hỗ trợ cho các mesh khác như Ambassador và Gloo vẫn còn ở giai đoạn alpha). Trên thực tế, điều này nghĩa là nếu bạn muốn áp dụng Knative, bạn cũng sẽ phải đã cam kết với Istio rồi.

With both Kubernetes and Istio, projects driven largely by Google, it took a very long time for them to get to a stage where they could be considered stable. Kubernetes still had major shifts after its 1.0 release, and only very recently Istio, which is going to underpin Knative, was completely rearchitected. This track record of delivering stable, production-ready projects makes me think that Knative may well take a lot longer to be ready for use by most of us. While some organizations are using it, and you could probably use it too, experience says it will be only so long before some major shift will take place that will require painful migration. It's partly for this reason that I've suggested that more conservative organizations who are considering an FaaS-like offering for their

Kubernetes cluster look elsewhere—projects like Open- FaaS are already being used in production by organizations all over the world and don't require an underlying service mesh. But if you do jump on the Knative train right now, don't be surprised if you have the odd derailment in your future.

Với cả Kubernetes lẫn Istio, các dự án phần lớn được Google dẫn dắt, đã mất một thời gian rất dài để chúng đạt đến một giai đoạn mà chúng có thể được coi là ổn định. Kubernetes vẫn có những dịch chuyển lớn sau bản phát hành 1.0 của nó, và chỉ rất gần đây Istio, thứ sẽ làm nền cho Knative, đã được kiến trúc lại hoàn toàn. Thành tích về việc chuyển giao các dự án ổn định, sẵn sàng cho production này làm tôi nghĩ rằng Knative rất có thể sẽ mất lâu hơn nhiều để sẵn sàng cho hầu hết chúng ta sử dụng. Mặc dù một số tổ chức đang dùng nó, và bạn có lẽ cũng có thể dùng nó, kinh nghiệm nói rằng chỉ là vấn đề thời gian trước khi một dịch chuyển lớn nào đó diễn ra đòi hỏi một cuộc di trú đau đớn. Một phần vì lý do này mà tôi đã đề nghị rằng các tổ chức bảo thủ hơn đang xem xét một sản phẩm kiểu FaaS cho Kubernetes cluster của họ nên tìm nơi khác—các dự án như OpenFaaS đã đang được dùng trong production bởi các tổ chức khắp thế giới và không đòi hỏi một service mesh bên dưới. Nhưng nếu bạn nhảy lên chuyến tàu Knative ngay bây giờ, đừng ngạc nhiên nếu bạn có một cú trật bánh thi thoảng trong tương lai của mình.

One other note: it's been a shame to see that Google has decided not to make Knative a part of the CNCF—one can only assume this is because Google wants to drive the direction of the tool itself. Kubernetes was a confusing prospect for many when launched, partly because it reflected Google's mindset around how containers should be managed. It benefited hugely from involvement from a broader set of the industry, and it's too bad that at this stage at least, Google has decided it isn't interested in the same broad industry involvement for Knative.

Một lưu ý khác: thật đáng tiếc khi thấy Google đã quyết định không đưa Knative thành một phần của CNCF—người ta chỉ có thể giả định rằng đây là vì Google muốn tự dẫn dắt hướng đi của công cụ. Kubernetes là một triển vọng gây nhầm lẫn cho nhiều người khi ra mắt, một phần vì nó phản ánh tư duy của Google quanh cách container nên được quản lý. Nó đã hưởng lợi rất nhiều từ sự tham gia của một tập rộng hơn của ngành công nghiệp, và thật đáng tiếc rằng ít nhất ở giai đoạn này, Google đã quyết định rằng nó không quan tâm đến cùng sự tham gia rộng rãi của ngành cho Knative.

The Future — Tương lai

Going forward, I see no signs that the rampaging juggernaut of Kubernetes will halt any time soon, and I fully expect to see more organizations implementing their own Kubernetes clusters for private clouds or making use of managed clusters in public cloud settings. However, I think what we're seeing now, with developers having to learn how to use Kubernetes directly, will be a relatively short-lived blip. Kubernetes is great at managing container workloads and providing a platform for other things to be built on. It isn't what could be considered a developer-friendly experience, though. Google itself has shown us that with the push behind Knative, and I think we'll continue to see Kubernetes hidden under higher-level abstraction layers. So in the future, I expect Kubernetes to be everywhere. You just won't know it.

Đi tới phía trước, tôi không thấy dấu hiệu nào cho thấy cỗ máy húc đổ mọi thứ là Kubernetes sẽ dừng lại bất cứ lúc nào sớm, và tôi hoàn toàn kỳ vọng thấy nhiều tổ chức hơn hiện thực các Kubernetes cluster của riêng họ cho các đám mây riêng (private cloud) hoặc dùng các cluster được quản lý trong các thiết lập public cloud. Tuy nhiên, tôi nghĩ điều ta đang thấy bây giờ, với việc lập trình viên phải học cách dùng Kubernetes trực

tiếp, sẽ là một đốm sáng tương đối ngắn ngủi. Kubernetes rất giỏi trong việc quản lý các tải công việc container và cung cấp một nền tảng để các thứ khác được xây dựng trên đó. Tuy nhiên, nó không phải là cái có thể được coi là một trải nghiệm thân thiện với lập trình viên. Bản thân Google đã cho ta thấy điều đó với sự thúc đẩy đằng sau Knative, và tôi nghĩ ta sẽ tiếp tục thấy Kubernetes bị ẩn dưới các lớp trừu tượng hóa cấp cao hơn. Vậy nên trong tương lai, tôi kỳ vọng Kubernetes sẽ ở khắp mọi nơi. Bạn chỉ sẽ không biết điều đó thôi.

This isn't to say that developers can forget they are building a **distributed system**. They'll still need to understand the myriad challenges that this type of architecture brings. It's just that they won't have to worry as much about the detail of how their software is mapped to underlying computing resources.

Điều này không phải để nói rằng lập trình viên có thể quên rằng họ đang xây dựng một hệ thống phân tán. Họ vẫn sẽ cần hiểu vô số thách thức mà loại kiến trúc này mang lại. Chỉ là họ sẽ không phải lo lắng nhiều đến vậy về chi tiết của việc phần mềm của họ được ánh xạ sang các tài nguyên tính toán bên dưới ra sao.

Should You Use It? — Bạn Có Nên Dùng Nó?

So for those of you who aren't already fully paid-up members of the Kubernetes club, should you join? Well, let me share a few guidelines. Firstly, implementing and managing your own Kubernetes cluster is not for the faint of heart—it is a significant undertaking. So much of the quality of experience that your developers will have using your Kubernetes install will depend on the effectiveness of the team running the cluster. For this reason, a number of the larger organizations I've spoken to who have gone down the Kubernetes on-prem path have outsourced this work to specialized companies.

Vậy đối với những bạn chưa phải là thành viên đã đóng phí đầy đủ của câu lạc bộ Kubernetes, bạn có nên gia nhập không? Vâng, để tôi chia sẻ một vài hướng dẫn. Thứ nhất, việc hiện thực và quản lý Kubernetes cluster của riêng bạn không dành cho người yếu tim—đó là một việc đảm đương đáng kể. Phần lớn chất lượng trải nghiệm mà lập trình viên của bạn sẽ có khi dùng bản cài Kubernetes của bạn sẽ phụ thuộc vào hiệu quả của đội vận hành cluster. Vì lý do này, một số trong các tổ chức lớn hơn tôi đã nói chuyện đã đi theo con đường Kubernetes on-prem đã thuê ngoài (outsource) công việc này cho các công ty chuyên biệt.

Even better, use a fully managed cluster. If you can make use of the public cloud, then use fully managed solutions like those provided by Google, Azure, and AWS. What I would say, though, is that if you are able to use the public cloud, then consider whether Kubernetes is actually what you want. If you're after a developer-friendly platform for handling the deployment and life cycle of your microservices, then the FaaS platforms we've already looked at could be a great fit. You could also look at the other PaaS-like offerings, like Azure Web Apps, Google App Engine, or some of the smaller providers like Zeit or Heroku.

Tốt hơn nữa, hãy dùng một cluster được quản lý hoàn toàn (fully managed). Nếu bạn có thể dùng public cloud, thì hãy dùng các giải pháp được quản lý hoàn toàn như những giải pháp do Google, Azure và AWS cung cấp. Tuy nhiên, điều tôi sẽ nói là nếu bạn có khả năng dùng public cloud, thì hãy cân nhắc liệu Kubernetes có thực sự là cái bạn muốn không. Nếu bạn đang tìm một nền tảng thân thiện với lập trình viên để xử lý việc triển khai và vòng đời của các microservice của bạn, thì các nền tảng FaaS ta đã xem có thể là một sự phù hợp tuyệt vời. Bạn cũng có thể xem các sản phẩm kiểu PaaS khác, như Azure Web Apps, Google App Engine, hoặc một số nhà cung cấp nhỏ hơn như Zeit hay Heroku.

Before you decide to start using Kubernetes, get some of your administrators and developers using it. The developers can get started running something lightweight locally, such as minikube or MicroK8s, giving them something pretty close to a full Kubernetes experience, but on their laptops. The people you'll have managing the platform may need a deeper dive. Katacoda has some great online tutorials for coming to grips with the core concepts, and the CNCF helps put out a lot of training materials in this space. Make sure the people who will actually use this stuff get to play with it before you make up your mind.

Trước khi bạn quyết định bắt đầu dùng Kubernetes, hãy cho một số quản trị viên và lập trình viên của bạn dùng thử nó. Các lập trình viên có thể bắt đầu chạy một thứ gì đó nhẹ cục bộ, chẳng hạn như minikube hay MicroK8s, cho họ một thứ khá gần với một trải nghiệm Kubernetes đầy đủ, nhưng trên laptop của họ. Những người bạn sẽ giao quản lý nền tảng có thể cần đào sâu hơn. Katacoda có một số hướng dẫn trực tuyến tuyệt vời để nắm bắt các khái niệm cốt lõi, và CNCF giúp đưa ra rất nhiều tài liệu đào tạo trong không gian này. Hãy đảm bảo rằng những người sẽ thực sự dùng thứ này được nghịch với nó trước khi bạn đưa ra quyết định.

Don't get trapped into thinking that you have to have Kubernetes "because everyone else is doing it." This is just as dangerous a justification for picking Kubernetes as it is for picking microservices. As good as Kubernetes is, it isn't for everyone—carry out your own assessment. But let's be frank—if you've got a handful of developers and only a few microservices, Kubernetes is likely to be huge overkill, even if using a fully managed platform.

Đừng bị mắc bẫy vào việc nghĩ rằng bạn phải có Kubernetes "vì mọi người khác đều đang dùng nó". Đây là một lý do biện minh cho việc chọn Kubernetes cũng nguy hiểm như cho việc chọn microservices. Dù Kubernetes có tốt đến đâu, nó không dành cho tất cả mọi người—hãy tự thực hiện đánh giá của riêng bạn. Nhưng hãy nói thẳng—nếu bạn có một nhóm lập trình viên và chỉ vài microservice, Kubernetes nhiều khả năng sẽ là quá thừa thãi, kể cả khi dùng một nền tảng được quản lý hoàn toàn.

Progressive Delivery — Chuyển giao Tăng tiến (Progressive Delivery)

Over the last decade or so, we've become smarter at deploying software to our users. New techniques have emerged that were driven by a number of different use cases and came from many different parts of the IT industry, but primarily they were all focused around making the act of pushing out new software much less risky. And if releasing software becomes less risky, we can release software more frequently.

Trong khoảng một thập kỷ qua, ta đã trở nên thông minh hơn trong việc triển khai phần mềm tới người dùng của mình. Các kỹ thuật mới đã xuất hiện được dẫn dắt bởi một số trường hợp dùng khác nhau và đến từ nhiều phần khác nhau của ngành CNTT, nhưng chủ yếu tất cả chúng đều tập trung quanh việc làm cho hành động đẩy phần mềm mới ra ngoài ít rủi ro hơn nhiều. Và nếu việc phát hành phần mềm trở nên ít rủi ro hơn, ta có thể phát hành phần mềm thường xuyên hơn.

There are a host of activities we carry out before sending our software live that can help us pick up problems before they impact real users. Preproduction testing is a huge part of this, although, as we'll discuss in Chapter 9, this can only take us so far.

Có vô số hoạt động ta thực hiện trước khi đưa phần mềm của mình lên trực tiếp (live) mà có thể giúp ta phát hiện các vấn đề trước khi chúng tác động đến người dùng thật. Kiểm

thử preproduction là một phần lớn của điều này, mặc dù, như ta sẽ bàn trong Chương 9, điều này chỉ đưa ta đi được tới một mức nhất định.

In their book *Accelerate*, Nicole Forsgren, Jez Humble, and Gene Kim show clear evidence drawn from extensive research that high-performing companies deploy more frequently than their low-performing counterparts and at the same time have much lower change failure rates.

Trong cuốn sách *Accelerate* của họ, Nicole Forsgren, Jez Humble và Gene Kim cho thấy bằng chứng rõ ràng rút ra từ nghiên cứu sâu rộng rằng các công ty hiệu suất cao triển khai thường xuyên hơn các đối tác hiệu suất thấp của họ và đồng thời có tỷ lệ thất bại khi thay đổi (change failure rate) thấp hơn nhiều.

The idea that you “go fast and break stuff” doesn’t really seem to apply when it comes to shipping software—shipping frequently and having lower failure rates goes hand in hand, and organizations that have realized this have changed how they think about releasing software.

Ý tưởng rằng bạn “đi nhanh và đập vỡ mọi thứ” dường như không thực sự áp dụng khi nói đến việc đưa phần mềm ra thị trường—đưa phần mềm ra thường xuyên và có tỷ lệ thất bại thấp hơn đi đôi với nhau, và các tổ chức đã nhận ra điều này đã thay đổi cách họ nghĩ về việc phát hành phần mềm.

These organizations make use of techniques like feature toggles, canary releases, parallel runs, and more, which we’ll detail in this section. This shift in how we think about releasing functionality falls under the banner of what is called progressive delivery. Functionality is released to users in a controlled manner; instead of a big-bang deployment, we can be smart about who sees what functionality—for example, by rolling out a new version of our software to a subset of our users.

Các tổ chức này dùng các kỹ thuật như feature toggle, canary release, parallel run và nhiều hơn nữa, mà ta sẽ trình bày chi tiết trong phần này. Sự dịch chuyển này trong cách ta nghĩ về việc phát hành chức năng nằm dưới ngọn cờ của cái được gọi là chuyển giao tăng tiến (progressive delivery). Chức năng được phát hành tới người dùng theo một cách có kiểm soát; thay vì một lần triển khai kiểu vụ-nổ-lớn (big-bang), ta có thể khôn ngoan về việc ai thấy chức năng nào—ví dụ, bằng cách triển khai một phiên bản mới của phần mềm của ta cho một tập con người dùng của ta.

6 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

6 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

Fundamentally, what all these techniques have at their heart is a simple shift in how we think about shipping software. Namely, that we can separate the concept of deployment from that of release.

Về cơ bản, điều mà tất cả các kỹ thuật này đều có ở cốt lõi là một sự dịch chuyển đơn giản trong cách ta nghĩ về việc đưa phần mềm ra thị trường. Cụ thể, rằng ta có thể tách khái niệm triển khai khỏi khái niệm phát hành.

Separating Deployment from Release — Tách Triển khai khỏi Phát hành

Jez Humble, coauthor of **Continuous Delivery**, makes the case for separating these two ideas, and he makes this a core principle for low-risk software releases :

Jez Humble, đồng tác giả của Continuous Delivery, lập luận cho việc tách hai ý tưởng này, và ông đặt đây là một nguyên tắc cốt lõi cho các lần phát hành phần mềm ít rủi ro:

Deployment is what happens when you install some version of your software into a particular environment (the production environment is often implied). Release is when you make a system or some part of it (for example, a feature) available to users. Jez argues that by separating these two ideas, we can ensure that our software works in its production setting without failures being seen by our users. Blue-green deployment is one of the simplest examples of this concept in action—you have one version of your software live (blue), and then you deploy a new version alongside the old version in production (green). You check to make sure that the new version is working as expected, and if it is, you redirect customers to see the new version of your software. If you find a problem before the switchover, no customer is impacted.

Triển khai là điều xảy ra khi bạn cài đặt một phiên bản nào đó của phần mềm của mình vào một môi trường cụ thể (môi trường production thường được ngầm hiểu). Phát hành là khi bạn làm cho một hệ thống hay một phần nào đó của nó (ví dụ, một tính năng) có sẵn cho người dùng. Jez lập luận rằng bằng cách tách hai ý tưởng này, ta có thể đảm bảo rằng phần mềm của ta hoạt động trong môi trường production của nó mà không có các thất bại bị người dùng của ta nhìn thấy. Triển khai blue-green là một trong những ví dụ đơn giản nhất của khái niệm này trong thực tế—bạn có một phiên bản phần mềm của mình đang live (blue), rồi bạn triển khai một phiên bản mới song song với phiên bản cũ trong production (green). Bạn kiểm tra để đảm bảo rằng phiên bản mới hoạt động như kỳ vọng, và nếu đúng vậy, bạn chuyển hướng khách hàng để thấy phiên bản mới của phần mềm của mình. Nếu bạn tìm thấy một vấn đề trước khi chuyển đổi, không khách hàng nào bị tác động.

While blue-green deployments are among the simplest examples of this principle, there are a number of more sophisticated techniques we can use when we embrace this concept.

Mặc dù triển khai blue-green nằm trong số những ví dụ đơn giản nhất của nguyên tắc này, có một số kỹ thuật tinh vi hơn ta có thể dùng khi ta đón nhận khái niệm này.

On to Progressive Delivery — Tiến tới Chuyển giao Tăng tiến

James Governor, cofounder of developer-focused industry analyst firm RedMonk, first coined the term progressive delivery to cover a number of different techniques being used in this space. He has gone on to describe progressive delivery as “continuous delivery with fine-grained control over the blast radius” —so it’s an extension of continuous delivery but also a technique that gives us the ability to control the potential impact of our newly released software.

James Governor, đồng sáng lập của hãng phân tích ngành tập trung vào lập trình viên RedMonk, lần đầu đặt ra thuật ngữ progressive delivery để bao quát một số kỹ thuật khác nhau đang được dùng trong không gian này. Ông đã tiếp tục mô tả progressive delivery như “continuous delivery với kiểm soát chi tiết đối với bán kính vụ nổ (blast radius)” —nên nó là một phần mở rộng của continuous delivery nhưng cũng là một kỹ thuật cho ta khả năng kiểm soát tác động tiềm tàng của phần mềm mới phát hành của ta.

Picking up this theme, Adam Zimman from LaunchDarkly describes how progressive delivery impacts “the business.” From that point of view, we require a shift in thinking about how new functionality reaches our customers. It’s no longer a single rollout —it can now be a phased activity. Importantly, though, progressive delivery can empower the product owner by, as Adam puts it, “delegating the control of the feature to the owner that is most closely responsible for the outcome.”

For this to work, however, the product owner in question needs to understand the mechanics of the progressive delivery technique being used, implying a somewhat technically savvy product owner, or else support from a suitably savvy set of people.

Tiếp nối chủ đề này, Adam Zimman từ LaunchDarkly mô tả cách progressive delivery tác động đến “hoạt động kinh doanh”. Từ góc nhìn đó, ta đòi hỏi một sự dịch chuyển trong tư duy về cách chức năng mới đến được với khách hàng của ta. Nó không còn là một lần triển khai (rollout) duy nhất—giờ đây nó có thể là một hoạt động theo từng giai đoạn. Tuy nhiên, quan trọng là, progressive delivery có thể trao quyền cho product owner bằng cách, như Adam diễn đạt, “ủy thác quyền kiểm soát tính năng cho người chủ chịu trách nhiệm gần nhất với kết quả”. Tuy nhiên, để điều này hoạt động, product owner nói trên cần hiểu cơ chế của kỹ thuật progressive delivery đang được dùng, ngụ ý một product owner có hiểu biết kỹ thuật phần nào, hoặc có sự hỗ trợ từ một tập người có hiểu biết phù hợp.

We’ve already touched on blue-green deployments as one progressive delivery technique. Let’s briefly take a look at a few more.

Ta đã đề cập đến triển khai blue-green như một kỹ thuật progressive delivery. Hãy điểm qua ngắn gọn một vài kỹ thuật nữa.

Feature Toggles — Feature Toggle

With feature toggles (otherwise known as feature flags), we hide deployed functionality behind a toggle that can be used to switch functionality off or on. This is most commonly used as part of trunk-based development, where functionality that isn’t yet finished can be checked in and deployed but still hidden from end users, but it has lots of applications outside of this. This could be useful to turn on a feature at a specified time, or turn off a feature that is causing problems.

Với feature toggle (còn được gọi là feature flag), ta ẩn chức năng đã triển khai phía sau một toggle có thể được dùng để bật hoặc tắt chức năng. Điều này phổ biến nhất được dùng như một phần của phát triển dựa trên trunk (trunk-based development), nơi chức năng chưa hoàn thành có thể được check in và triển khai nhưng vẫn được ẩn khỏi người dùng cuối, nhưng nó có nhiều ứng dụng bên ngoài điều này. Điều này có thể hữu ích để bật một tính năng vào một thời điểm xác định, hoặc tắt một tính năng đang gây ra vấn đề.

You can also use feature toggles in a more fine-grained manner, perhaps allowing a flag to have a different state based on the nature of the user making a request. So you could for example have a group of customers that see a feature turned on (perhaps a beta test group), whereas most people see the feature as being turned off—this could help you implement a canary rollout, something we discuss next. Fully managed solutions exist for managing feature toggles, including LaunchDarkly and Split. Impressive as these platforms are, I think you can get started with something much simpler—just a configuration file can do for a start, then look at these technologies as you start pushing how you want to use the toggles.

Bạn cũng có thể dùng feature toggle theo cách mịn hơn, có lẽ cho phép một flag có một trạng thái khác dựa trên bản chất của người dùng thực hiện một request. Vậy nên chẳng hạn bạn có thể có một nhóm khách hàng thấy một tính năng được bật (có lẽ một nhóm beta test), trong khi hầu hết mọi người thấy tính năng đó bị tắt—điều này có thể giúp bạn hiện thực một canary rollout, một thứ ta bàn tiếp theo. Các giải pháp được quản lý hoàn toàn tồn tại để quản lý feature toggle, bao gồm LaunchDarkly và Split. Dù các nền tảng này ấn tượng, tôi nghĩ bạn có thể bắt đầu với một thứ đơn giản hơn nhiều—chỉ một file cấu hình là đủ để khởi đầu, rồi xem xét các công nghệ này khi bạn bắt đầu đẩy mạnh cách

bạn muốn dùng các toggle.

For a much deeper dive into the world of feature toggles, I can heartily recommend Pete Hodgson's writeup "Feature Toggles (aka Feature Flags)", which goes into a lot of detail regarding how to implement them and the many different ways they can be used.

Để đào sâu hơn nhiều vào thế giới của feature toggle, tôi hết lòng khuyên đọc bài viết "Feature Toggles (aka Feature Flags)" của Pete Hodgson, vốn đi vào rất nhiều chi tiết về cách hiện thực chúng và nhiều cách khác nhau chúng có thể được dùng.

Canary Release — Canary Release

To err is human, but to really foul things up you need a computer. ⁷ We all make mistakes, and computers can let us make mistakes faster and at larger scale than ever before. Given that mistakes are unavoidable (and trust me, they are), then it makes sense to do things that allow us to limit the impact of these mistakes. Canary releases are one such technique.

Phạm sai lầm là chuyện của con người, nhưng để thực sự làm hỏng hết mọi thứ thì bạn cần một cái máy tính. ⁷ Tất cả chúng ta đều phạm sai lầm, và máy tính có thể cho phép ta phạm sai lầm nhanh hơn và ở quy mô lớn hơn bao giờ hết. Vì sai lầm là không thể tránh khỏi (và tin tôi đi, đúng là vậy), thì việc làm những điều cho phép ta giới hạn tác động của những sai lầm này là hợp lý. Canary release là một kỹ thuật như vậy.

Named for the canaries taken into mines as an early warning system for miners to warn them of the presence of dangerous gases, with a canary rollout the idea is that a limited subset of our customers see new functionality. If there is a problem with the rollout, then only that portion of our customers is impacted. If the feature works for that canary group, then it can be rolled out to more of your customers until everyone sees the new version.

Được đặt tên theo những con chim hoàng yến (canary) được đưa vào hầm mỏ như một hệ thống cảnh báo sớm cho thợ mỏ để cảnh báo họ về sự hiện diện của các khí nguy hiểm, với một canary rollout ý tưởng là một tập con hạn chế các khách hàng của ta thấy chức năng mới. Nếu có một vấn đề với lần triển khai, thì chỉ phần khách hàng đó bị tác động. Nếu tính năng hoạt động cho nhóm canary đó, thì nó có thể được triển khai tới nhiều khách hàng hơn của bạn cho đến khi mọi người thấy phiên bản mới.

For a microservice architecture, a toggle could be configured at an individual microservice level, turning functionality on (or off) for requests to that functionality from the outside world or other microservices. Another technique is to have two different

Đối với một kiến trúc microservice, một toggle có thể được cấu hình ở cấp một microservice riêng lẻ, bật (hoặc tắt) chức năng cho các request tới chức năng đó từ thế giới bên ngoài hay các microservice khác. Một kỹ thuật khác là có hai phiên bản

⁷ This quote is often attributed to biologist Paul Ehrlich, but its actual origins are unclear .

⁷ Câu trích này thường được gán cho nhà sinh học Paul Ehrlich, nhưng nguồn gốc thực sự của nó không rõ ràng.

versions of a microservice running side by side, and use the toggle to route to either the old or the new version. Here, the canary implementation has to be somewhere in the routing/networking path, rather than being in one microservice.

khác nhau của một microservice chạy song song nhau, và dùng toggle để định tuyến tới hoặc phiên bản cũ hoặc phiên bản mới. Ở đây, bản hiện thực canary phải nằm ở đâu đó

trong đường định tuyến/nối mạng, thay vì nằm trong một microservice.

When I first did a canary release we controlled the rollout manually. We could configure the percentage of our traffic seeing the new functionality, and over a period of a week we gradually increased this until everyone saw the new functionality. Over the week, we kept an eye on our error rates, bug reports, and the like. Nowadays, it's more common to see this process handled in an automated fashion. Tools like Spinnaker for example have the ability to automatically ramp up calls based on metrics, such as increasing the percentage of calls to a new microservice version if the error rates are at an acceptable level.

Khi tôi lần đầu thực hiện một canary release, chúng tôi kiểm soát lần triển khai một cách thủ công. Chúng tôi có thể cấu hình tỷ lệ phần trăm lưu lượng của mình thấy chức năng mới, và trong một khoảng thời gian một tuần chúng tôi dần dần tăng con số này cho đến khi mọi người thấy chức năng mới. Trong suốt tuần đó, chúng tôi để mắt đến tỷ lệ lỗi, các báo cáo lỗi và những thứ tương tự. Ngày nay, phổ biến hơn là thấy quá trình này được xử lý theo cách tự động. Chẳng hạn, các công cụ như Spinnaker có khả năng tự động tăng dần các lời gọi dựa trên các metric, chẳng hạn như tăng tỷ lệ phần trăm các lời gọi tới một phiên bản microservice mới nếu tỷ lệ lỗi ở mức chấp nhận được.

Parallel Run — Parallel Run

With a canary release, a request to a piece of functionality will be served by either the old or the new version. This means we can't compare how the two versions of functionality would handle the same request, something that can be important if you want to make sure that the new functionality works in exactly the same way as the old version of functionality.

Với một canary release, một request tới một mảnh chức năng sẽ được phục vụ bởi hoặc phiên bản cũ hoặc phiên bản mới. Điều này nghĩa là ta không thể so sánh cách hai phiên bản của chức năng sẽ xử lý cùng một request ra sao, một điều có thể quan trọng nếu bạn muốn đảm bảo rằng chức năng mới hoạt động theo đúng cùng cách như phiên bản cũ của chức năng.

With a parallel run you do exactly that—you run two different implementations of the same functionality side by side, and send a request to the functionality to both implementations. With a microservice architecture, the most obvious approach might be to dispatch a service call to two different versions of the same service and compare the results. An alternative is to coexist both implementations of the functionality inside the same service, which can often make comparison easier.

Với một parallel run bạn làm đúng điều đó—bạn chạy hai bản hiện thực khác nhau của cùng một chức năng song song nhau, và gửi một request tới chức năng đó cho cả hai bản hiện thực. Với một kiến trúc microservice, cách tiếp cận hiển nhiên nhất có thể là điều phối một lời gọi service tới hai phiên bản khác nhau của cùng một service và so sánh kết quả. Một lựa chọn thay thế là cho cả hai bản hiện thực của chức năng cùng tồn tại bên trong cùng một service, điều thường có thể làm cho việc so sánh dễ dàng hơn.

When executing both implementations, it's important to realize that you likely only want the results of one of the invocations. One implementation is considered the source of truth—this is the implementation you currently trust, and is typically the existing implementation. Depending on the nature of the functionality you are comparing with a parallel run, you might have to give this nuance careful thought—you wouldn't want to send two identical order updates to a customer, or pay an invoice twice for example!

Khi thực thi cả hai bản hiện thực, quan trọng là phải nhận ra rằng bạn nhiều khả năng chỉ muốn kết quả của một trong các lần gọi. Một bản hiện thực được coi là nguồn chân lý (source of truth)—đây là bản hiện thực bạn hiện tin tưởng, và thường là bản hiện thực hiện có. Tùy theo bản chất của chức năng bạn đang so sánh với một parallel run, bạn có thể phải suy nghĩ kỹ về sắc thái này—bạn sẽ không muốn gửi hai bản cập nhật đơn hàng giống hệt nhau cho một khách hàng, hay trả một hóa đơn hai lần chẳng hạn!

I explore the parallel run pattern in a lot more detail in Chapter 3 of my book *Monolith to Microservices*.⁸ There I explore its use in helping migrate functionality from a monolithic system to a microservice architecture, where we want to ensure that our new microservice behaves in the same way as the equivalent monolith functionality. In another context, GitHub makes use of this pattern when reworking core parts of its codebase, and have released an open source tool *Scientist* to help with this process.

Tôi khám phá pattern parallel run chi tiết hơn nhiều trong Chương 3 của cuốn sách *Monolith to Microservices* của tôi.⁸ Ở đó tôi khám phá việc dùng nó để giúp di trú chức năng từ một hệ thống monolith sang một kiến trúc microservice, nơi ta muốn đảm bảo rằng microservice mới của ta hành xử theo cùng cách như chức năng monolith tương đương. Trong một ngữ cảnh khác, GitHub dùng pattern này khi tái cấu trúc các phần cốt lõi của codebase của họ, và đã phát hành một công cụ mã nguồn mở *Scientist* để giúp với quá trình này.

⁸ Sam Newman, *Monolith to Microservices* (Sebastopol: O'Reilly, 2019).

⁸ Sam Newman, *Monolith to Microservices* (Sebastopol: O'Reilly, 2019).

Here, the parallel run is done within a single process, with *Scientist* helping to compare the invocations.

Ở đây, parallel run được thực hiện bên trong một tiến trình duy nhất, với *Scientist* giúp so sánh các lần gọi.



With blue-green deployment, feature toggles, canary releases, and parallel runs we've just scratched the surface of the field of progressive delivery. These ideas can work well together (we've already touched on how you could use feature toggles to implement a canary rollout for example), but you probably want to ease yourself in. To start off, just remember to separate the two concepts of deployment and release. Next, start looking for ways to help you deploy your software more

frequently, but in a safe manner. Work with your product owner or other business stakeholders to understand how some of these techniques can help you go faster, but also help reduce failures too.

Với triển khai blue-green, feature toggle, canary release và parallel run ta mới chỉ chạm bề mặt của lĩnh vực progressive delivery. Những ý tưởng này có thể hoạt động tốt cùng nhau (ta đã đề cập đến cách bạn có thể dùng feature toggle để hiện thực một canary rollout chẳng hạn), nhưng bạn có lẽ muốn từ từ làm quen. Để bắt đầu, chỉ cần nhớ tách hai khái niệm triển khai và phát hành. Tiếp theo, bắt đầu tìm cách giúp bạn triển khai phần mềm của mình thường xuyên hơn, nhưng theo một cách an toàn. Hãy làm việc với product owner của bạn hoặc các bên liên quan kinh doanh khác để hiểu cách một số trong các kỹ thuật này có thể giúp bạn đi nhanh hơn, nhưng cũng giúp giảm các thất bại nữa.

Summary — Tóm tắt

OK, so we covered a lot of ground here. Let's briefly recap before we move on. Firstly, let's remind ourselves of the principles for deployment that I outlined earlier:

OK, vậy là ta đã đi qua rất nhiều thứ ở đây. Hãy điểm lại ngắn gọn trước khi ta đi tiếp. Thứ nhất, hãy tự nhắc mình về các nguyên tắc triển khai mà tôi đã nêu trước đó:

Isolated execution Run microservice instances in an isolated fashion where they have their own computing resources, and their execution cannot impact other microservice instances running nearby.

Thực thi cô lập Chạy các instance microservice theo kiểu cô lập nơi chúng có tài nguyên tính toán riêng, và việc thực thi của chúng không thể tác động đến các instance microservice khác đang chạy gần đó.

Focus on automation Choose technology that allows for a high degree of automation, and adopt automation as a core part of your culture.

Tập trung vào tự động hóa Chọn công nghệ cho phép mức độ tự động hóa cao, và đưa tự động hóa thành một phần cốt lõi trong văn hóa của bạn.

Infrastructure as code Represent the configuration for your infrastructure to ease automation and promote information sharing. Store this code in source control to allow for environments to be re-created.

Infrastructure as code Biểu diễn cấu hình cho hạ tầng của bạn để giảm bớt việc tự động hóa và thúc đẩy chia sẻ thông tin. Lưu trữ mã này trong hệ thống quản lý mã nguồn để cho phép tái tạo lại các môi trường.

Aim for zero-downtime deployment Take independent deployability further, and ensure that deploying a new version of a microservice can be done without any downtime to users of your service (be it humans or other microservices).

Hướng tới triển khai không gián đoạn Đẩy khả năng triển khai độc lập đi xa hơn, và đảm bảo rằng việc triển khai một phiên bản mới của microservice có thể được thực hiện mà không gây bất kỳ gián đoạn nào cho người dùng service của bạn (dù là con người hay các microservice khác).

Desired state management Use a platform that maintains your microservice in a defined state, launching new instances if required in the event of outage or traffic increases.

Quản lý trạng thái mong muốn Dùng một nền tảng duy trì microservice của bạn ở một trạng thái đã định, khởi chạy các instance mới nếu cần trong trường hợp gián đoạn hoặc tăng lưu lượng.

In addition, I shared my own guidelines for selecting the right deployment platform:

Thêm vào đó, tôi đã chia sẻ các hướng dẫn của riêng mình để chọn nền tảng triển khai đúng đắn:

CHAPTER 9 Testing — Chương 9:

Kiểm thử

The world of automated **testing** has advanced significantly since I first started writing code, and every month there seems to be some new tool or technique to make it even better. But challenges remain regarding how to effectively and efficiently test our code's functionality when it spans a **distributed system**. This chapter breaks down the problems associated with testing finer-grained systems and presents some solutions to help you make sure you can **release** your new functionality with confidence.

Thế giới của kiểm thử tự động (automated testing) đã tiến bộ đáng kể kể từ khi tôi bắt đầu viết mã, và dường như mỗi tháng lại có một công cụ hay kỹ thuật mới nào đó giúp nó tốt hơn nữa. Nhưng vẫn còn những thách thức về việc làm sao kiểm thử chức năng của mã một cách hiệu quả và tiết kiệm khi nó trải rộng trên một hệ thống phân tán. Chương này phân tích những vấn đề liên quan đến việc kiểm thử các hệ thống được chia nhỏ hơn, và trình bày một số giải pháp giúp bạn chắc chắn rằng mình có thể phát hành chức năng mới một cách tự tin.

Testing covers a lot of ground. Even when we are talking only about automated tests, there are a large number to consider. With **microservices**, we have added another level of complexity. Understanding what different types of tests we can run is important to help us balance the sometimes-opposing forces of getting our software into production as quickly as possible versus making sure our software is of sufficient quality. Given the scope of testing as a whole, I am not going to attempt a broad exploration of the topic. Instead, this chapter is primarily focused on looking at how testing of a **microservice architecture** is different when compared to less-distributed systems such as single-process monolithic applications.

Kiểm thử (testing) bao trùm một phạm vi rất rộng. Ngay cả khi chỉ nói về các bài kiểm thử tự động, vẫn có vô số loại cần cân nhắc. Với microservices, chúng ta lại thêm vào một tầng phức tạp nữa. Việc hiểu được những loại kiểm thử khác nhau mà ta có thể chạy là rất quan trọng, giúp ta cân bằng giữa hai lực đôi khi đối nghịch nhau: đưa phần mềm vào môi trường production càng nhanh càng tốt, và đảm bảo phần mềm của ta đạt chất lượng đủ tốt. Xét đến quy mô của kiểm thử nói chung, tôi sẽ không cố gắng khảo sát rộng toàn bộ chủ đề này. Thay vào đó, chương này chủ yếu tập trung xem xét việc kiểm thử một kiến trúc microservice khác biệt thế nào so với những hệ thống ít phân tán hơn, chẳng hạn các ứng dụng monolith đơn tiến trình.

Where testing is done has also shifted since the first edition of this book. Previously, testing was predominantly carried out before the software got to production. Increasingly, though, we look at testing our applications once they arrive in production— further blurring the lines between development and production-related activities; this is something we'll explore in this chapter before

exploring testing in production more fully in Chapter 10 .

Nơi diễn ra việc kiểm thử cũng đã thay đổi kể từ lần xuất bản đầu tiên của cuốn sách này. Trước đây, kiểm thử chủ yếu được thực hiện trước khi phần mềm lên production. Tuy nhiên, ngày càng nhiều, chúng ta hướng đến việc kiểm thử ứng dụng ngay khi nó đã ở trên production—càng làm mờ ranh giới giữa các hoạt động phát triển và các hoạt động liên quan đến production; đây là điều chúng ta sẽ khám phá trong chương này trước khi tìm hiểu kỹ hơn về kiểm thử trên production trong Chương 10.

Types of Tests — Các loại kiểm thử

Like many consultants, I'm guilty of occasionally using quadrants as a way of categorizing the world, and I was starting to worry this book wouldn't have one. Luckily, Brian Marick came up with a fantastic categorization system for tests that fits right in. Figure 9-1 shows a variation of Marick's quadrant from Lisa Crispin and Janet Gregory's book *Agile Testing 1* that helps categorize the different types of tests.

Giống như nhiều tư vấn viên khác, đôi khi tôi cũng mắc tật dùng các sơ đồ góc phần tư (quadrant) như một cách phân loại thế giới, và tôi đã bắt đầu lo rằng cuốn sách này sẽ không có cái nào. May thay, Brian Marick đã đưa ra một hệ thống phân loại kiểm thử tuyệt vời rất phù hợp ở đây. Hình 9-1 cho thấy một biến thể của sơ đồ góc phần tư của Marick, trích từ cuốn *Agile Testing* của Lisa Crispin và Janet Gregory,¹ giúp phân loại các loại kiểm thử khác nhau.

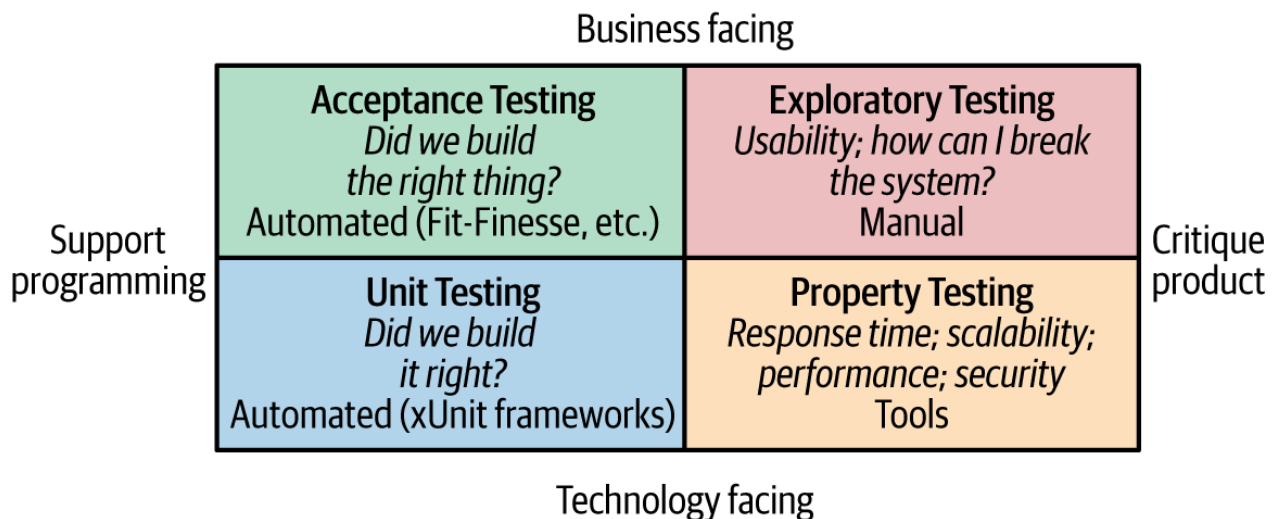


Figure 9-1. Brian Marick's testing quadrant. Lisa Crispin and Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*, © 2009

Hình 9-1. Sơ đồ góc phần tư về kiểm thử của Brian Marick. Lisa Crispin và Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams*, © 2009

At the bottom of the quadrant, we have tests that are technology facing—that is, tests that aid the developers in creating the system in the first place. Property tests such as performance tests and small-scoped unit tests fall into this category; all are typically automated. The top half of the quadrant includes those tests that help the nontechnical stakeholders understand how your system works, which we call business-facing tests. These could be large-scoped, end-to-end tests, as shown in the Acceptance Testing quadrant at top left; or manual testing (as typified by user testing done against a

UAT system), as shown in the Exploratory Testing quadrant.

Ở nửa dưới của sơ đồ, ta có những bài kiểm thử hướng về công nghệ (technology facing)—tức là những bài kiểm thử hỗ trợ lập trình viên trong việc tạo ra hệ thống ngay từ đầu. Các bài kiểm thử thuộc tính (property tests) như kiểm thử hiệu năng (performance tests) và unit test có phạm vi nhỏ đều thuộc nhóm này; tất cả đều thường được tự động hóa. Nửa trên của sơ đồ gồm những bài kiểm thử giúp các bên liên quan phi kỹ thuật hiểu cách hệ thống của bạn vận hành, mà ta gọi là kiểm thử hướng về nghiệp vụ (business-facing tests). Đó có thể là các bài kiểm thử end-to-end phạm vi rộng, như trong góc phần tư Acceptance Testing ở phía trên bên trái; hoặc kiểm thử thủ công (như kiểu kiểm thử người dùng thực hiện trên một hệ thống UAT), như trong góc phần tư Exploratory Testing.

At this point it's worth calling out the fact that the vast majority of these tests focus on preproduction validation . Specifically, we are using these tests to ensure that the software is of sufficient quality before it is deployed into a production environment. Typically, these tests passing (or failing) would be a gating condition for deciding whether the software should be deployed.

Đến đây, cũng đáng nhấn mạnh rằng tuyệt đại đa số các bài kiểm thử này tập trung vào việc kiểm chứng tiền-production (preproduction validation). Cụ thể, ta dùng những bài kiểm thử này để đảm bảo phần mềm đạt chất lượng đủ tốt trước khi nó được triển khai vào môi trường production. Thông thường, việc các bài kiểm thử này pass (hay fail) sẽ là điều kiện cổng để quyết định liệu phần mềm có nên được triển khai hay không.

Increasingly, we are seeing the value of testing our software once we actually get into a production environment. We'll talk more about the balance between these two ideas later in the chapter, but for now it's worth highlighting a limitation of Marick's quadrant in this regard.

Ngày càng nhiều, chúng ta nhận ra giá trị của việc kiểm thử phần mềm ngay khi nó thực sự nằm trong môi trường production. Chúng ta sẽ bàn nhiều hơn về sự cân bằng giữa hai ý tưởng này về sau trong chương, nhưng hiện giờ cũng đáng nêu bật một hạn chế của sơ đồ Marick ở khía cạnh này.

1 Lisa Crispin and Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams* (Upper Saddle River, NJ: Addison-Wesley, 2008).

1 Lisa Crispin và Janet Gregory, *Agile Testing: A Practical Guide for Testers and Agile Teams* (Upper Saddle River, NJ: Addison-Wesley, 2008).

Each type of test shown in this quadrant has a place. Exactly how much of each test you want to do will depend on the nature of your system, but the key point to understand is that you have multiple choices in terms of how to test your system. The trend recently has been away from any large-scale manual testing in favor of automating as much repetitive testing as possible, and I certainly agree with this approach. If you currently carry out large amounts of manual testing, I would suggest you address that before proceeding too far down the path of microservices, as you won't get many of their benefits if you are unable to validate your software quickly and efficiently.

Mỗi loại kiểm thử thể hiện trong sơ đồ này đều có chỗ đứng riêng. Bạn muốn làm bao nhiêu mỗi loại sẽ tùy thuộc vào bản chất hệ thống của bạn, nhưng điểm mấu chốt cần hiểu là bạn có nhiều lựa chọn về cách kiểm thử hệ thống của mình. Xu hướng gần đây là rời xa mọi hình thức kiểm thử thủ công quy mô lớn, chuyển sang tự động hóa càng nhiều phần kiểm thử lặp đi lặp lại càng tốt, và tôi chắc chắn đồng tình với hướng tiếp cận này. Nếu hiện tại bạn đang thực hiện một lượng lớn kiểm thử thủ công, tôi khuyên bạn nên giải quyết vấn đề đó trước khi đi quá xa trên con đường microservices, vì bạn sẽ không gặt hái được nhiều lợi ích của chúng nếu không thể kiểm chứng phần mềm của mình một cách nhanh chóng và hiệu quả.

Manual Exploratory Testing — Kiểm thử khám phá thủ công (Manual Exploratory Testing)

In general, the shift from a monolithic architecture to a microservice architecture will have minimal impact on exploratory testing, aside from any wider organizational shift that might occur. As we'll explore in “Toward Stream-Aligned Teams” on page 459, we'd expect the **user interface** for the application to also be broken down along team lines. In such a context, the **ownership** of manual testing may well change.

Nhìn chung, việc chuyển từ kiến trúc monolith sang kiến trúc microservice sẽ có tác động tối thiểu đến kiểm thử khám phá (exploratory testing), ngoài bất kỳ sự dịch chuyển rộng hơn nào về mặt tổ chức có thể xảy ra. Như chúng ta sẽ khám phá trong mục “Hướng tới các Stream-Aligned Team” ở trang 459, ta cũng kỳ vọng rằng giao diện người dùng (UI) của ứng dụng sẽ được chia nhỏ theo ranh giới các nhóm. Trong bối cảnh như vậy, quyền sở hữu việc kiểm thử thủ công rất có thể sẽ thay đổi.

The ability to automate some tasks, such as verifying how something looks, was previously limited exclusively to manual exploratory testing. The maturing of tooling that enables visual assertions has allowed us to start automating tasks that were previously done manually. You shouldn't see this as a reason to have no manual testing, though; you should instead see it as a chance to free up testers' time to focus on less repetitive, exploratory testing.

Khả năng tự động hóa một số tác vụ, chẳng hạn kiểm chứng xem một thứ gì đó trông ra sao, trước đây chỉ giới hạn ở kiểm thử khám phá thủ công. Sự trưởng thành của các công cụ cho phép kiểm chứng trực quan (visual assertions) đã giúp ta bắt đầu tự động hóa những tác vụ trước kia phải làm thủ công. Tuy nhiên, bạn không nên xem đây là lý do để loại bỏ hoàn toàn kiểm thử thủ công; thay vào đó, hãy xem nó như một cơ hội giải phóng thời gian của những người kiểm thử để họ tập trung vào kiểm thử khám phá, ít lặp lại hơn.

When done well, manual exploratory testing is to a great extent about discovery. Setting aside time to explore the application as an end user can unearth issues that otherwise wouldn't be evident. Manual testing can also be vital in situations in which an automated test is infeasible to implement, perhaps because of the cost of writing the test. Automation is about removing repetitive tasks to free up humans to do the more creative, ad hoc activities. So think of automation as a way of freeing up our brainpower for the things we do best.

Khi được thực hiện tốt, kiểm thử khám phá thủ công phần lớn là về sự khám phá. Dành thời gian khám phá ứng dụng dưới góc nhìn của người dùng cuối có thể phát hiện ra những vấn đề mà nếu không thì sẽ không lộ ra. Kiểm thử thủ công cũng có thể trở nên thiết yếu trong những tình huống mà một bài kiểm thử tự động là bất khả thi để triển khai, có lẽ vì chi phí viết bài kiểm thử đó. Tự động hóa là về việc loại bỏ các tác vụ lặp đi lặp lại để giải phóng con người làm những hoạt động sáng tạo, tùy biến hơn. Vì vậy hãy nghĩ về tự động hóa như một cách giải phóng trí lực của chúng ta cho những việc ta làm tốt nhất.

For the purposes of this chapter, we mostly ignore manual exploratory testing. This isn't to say that this type of testing isn't important, but just that the scope of this chapter is to focus primarily on how testing microservices differs from testing more typical monolithic applications. But when it comes to automated tests, how many of each test do we want? Another model will help us answer this question and understand what the different trade-offs might be.

Trong phạm vi chương này, chúng ta sẽ chủ yếu bỏ qua kiểm thử khám phá thủ công. Điều này không có nghĩa là loại kiểm thử này không quan trọng, mà chỉ vì phạm vi chương này tập trung chủ yếu vào việc kiểm thử microservices khác biệt thế nào so với kiểm thử các

ứng dụng monolith điển hình hơn. Nhưng khi nói đến các bài kiểm thử tự động, ta muốn có bao nhiêu mỗi loại? Một mô hình khác sẽ giúp ta trả lời câu hỏi này và hiểu được các đánh đổi khác nhau có thể là gì.

Test Scope — Phạm vi kiểm thử (Test Scope)

In his book *Succeeding with Agile*,² Mike Cohn outlines a model called the test pyramid to help explain what types of automated tests are needed. The pyramid helps us think about not only the scope of the tests, but also the proportions of the different types of tests we should aim for. Cohn's original model split automated tests into unit tests, **service** tests, and UI tests, as illustrated in Figure 9-2.

Trong cuốn sách *Succeeding with Agile*,² Mike Cohn phác họa một mô hình gọi là kim tự tháp kiểm thử (test pyramid) để giúp giải thích những loại kiểm thử tự động nào là cần thiết. Kim tự tháp giúp ta suy nghĩ không chỉ về phạm vi của các bài kiểm thử, mà còn về tỷ lệ giữa các loại kiểm thử khác nhau mà ta nên hướng tới. Mô hình gốc của Cohn chia kiểm thử tự động thành unit test, service test, và UI test, như minh họa trong Hình 9-2.

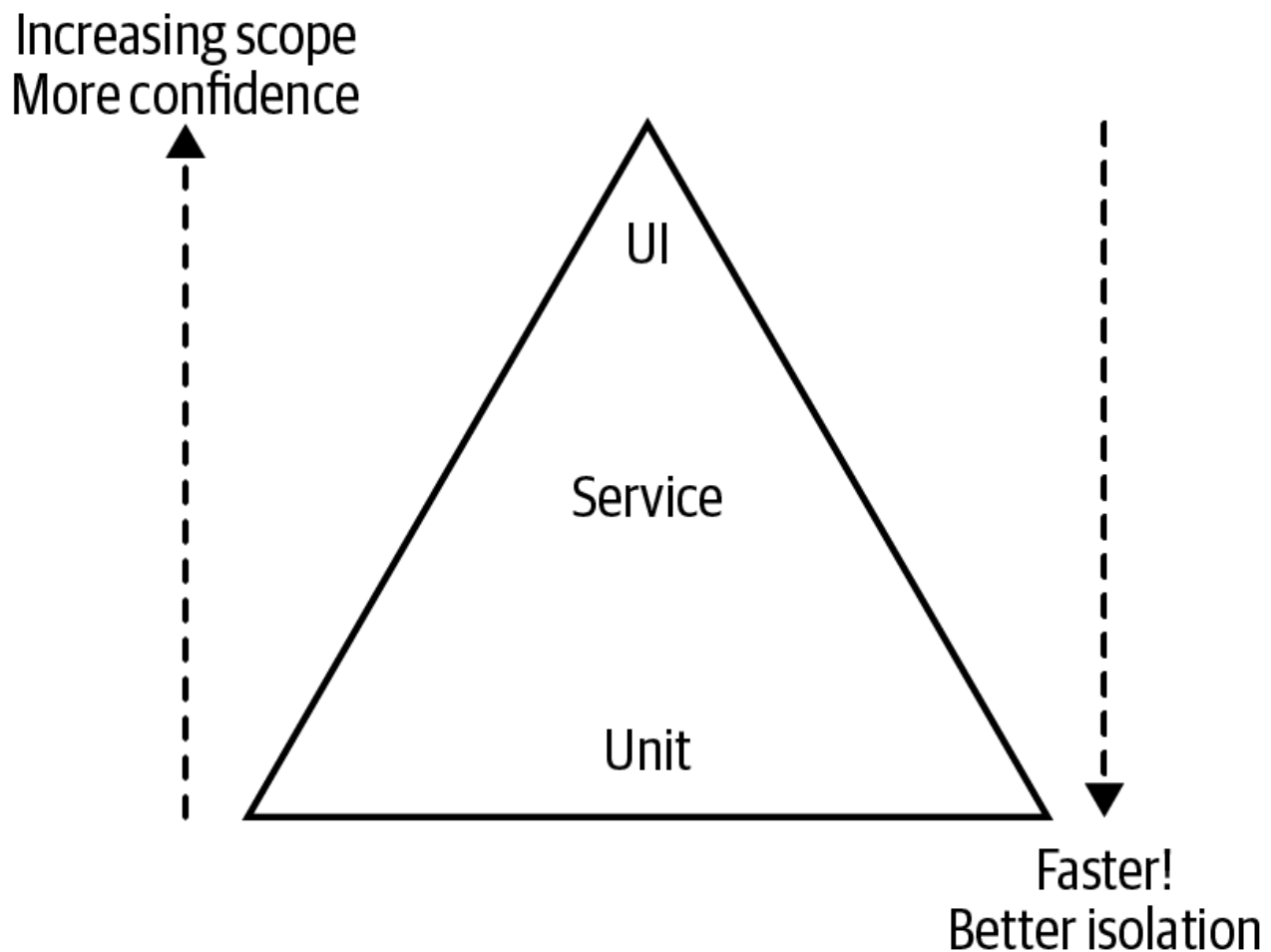


Figure 9-2. Mike Cohn's test pyramid. Mike Cohn, *Succeeding with Agile: Software Development Using Scrum*, 1st ed., © 2010

Hình 9-2. Kim tự tháp kiểm thử của Mike Cohn. Mike Cohn, *Succeeding with Agile: Software*

The key thing to take away when reading the pyramid is that as we go up the pyramid, the test scope increases, as does our confidence that the functionality being tested works. On the other hand, the feedback cycle time increases as the tests take longer to run, and when a test fails, it can be harder to determine which functionality has broken. As we go down the pyramid, in general the tests become much faster, so we get much faster feedback cycles. We find broken functionality faster, our **continuous integration** builds are faster, and we are less likely to move on to a new task before finding out we have broken something. When those smaller-scoped tests fail, we also tend to know what broke, often down to the exact line of code—each test is better isolated, making it easier for us to understand and fix breakages. On the flip side, we don't get a lot of confidence that our system as a whole works if we've tested only one line of code!

Điều mấu chốt cần rút ra khi đọc kim tự tháp là khi ta đi lên trên, phạm vi kiểm thử tăng lên, và cùng với đó là mức độ tin tưởng của ta rằng chức năng đang được kiểm thử thực sự hoạt động. Mặt khác, thời gian của chu kỳ phản hồi (feedback cycle) tăng lên vì các bài kiểm thử mất nhiều thời gian chạy hơn, và khi một bài kiểm thử fail, sẽ khó xác định hơn xem chức năng nào đã hỏng. Khi ta đi xuống dưới kim tự tháp, nhìn chung các bài kiểm thử trở nên nhanh hơn nhiều, nên ta nhận được phản hồi nhanh hơn rất nhiều. Ta phát hiện chức năng bị hỏng nhanh hơn, các bản build tích hợp liên tục (CI) của ta chạy nhanh hơn, và ta ít có khả năng chuyển sang một tác vụ mới trước khi phát hiện ra mình đã làm hỏng thứ gì đó. Khi những bài kiểm thử phạm vi nhỏ này fail, ta cũng thường biết được cái gì đã hỏng, thường là tới tận dòng mã chính xác—mỗi bài kiểm thử được cô lập tốt hơn, giúp ta dễ hiểu và sửa lỗi hơn. Ngược lại, ta sẽ không có nhiều niềm tin rằng toàn bộ hệ thống của mình hoạt động nếu ta chỉ kiểm thử một dòng mã!

The problem with this model is that all these terms mean different things to different people. Service is especially overloaded, and there are many definitions of a **unit test** out there. Is a test a unit test if I test only one line of code? I'd say it is. Is it still a unit test if I test multiple functions or classes? I'd say no, but many would disagree! I tend

Vấn đề với mô hình này là tất cả những thuật ngữ đó mang ý nghĩa khác nhau với những người khác nhau. “Service” đặc biệt bị dùng quá tải, và ngoài kia có rất nhiều định nghĩa về unit test. Một bài kiểm thử có phải là unit test không nếu tôi chỉ kiểm thử một dòng mã? Tôi cho là có. Nó còn là unit test không nếu tôi kiểm thử nhiều hàm hay nhiều lớp? Tôi cho là không, nhưng nhiều người sẽ không đồng ý! Tôi thường có xu hướng

2 Mike Cohn, *Succeeding with Agile* (Upper Saddle River, NJ: Addison-Wesley, 2009).

2 Mike Cohn, *Succeeding with Agile* (Upper Saddle River, NJ: Addison-Wesley, 2009).

to stick with the unit and service names despite their ambiguity, but much prefer calling UI tests end-to-end tests, which I'll do from this point forward.

giữ lại các tên gọi unit và service bất chấp sự mơ hồ của chúng, nhưng tôi thích gọi UI test là end-to-end test hơn nhiều, và đó là cách tôi sẽ dùng từ điểm này trở đi.

Virtually every team I've worked on has used different names for tests than the ones that Cohn uses in the pyramid. Whatever you call them, the key takeaway is that you will want functional automated tests of different scope for different purposes.

Hầu như mọi nhóm tôi từng làm việc cùng đều dùng những tên gọi cho các bài kiểm thử khác với những tên mà Cohn dùng trong kim tự tháp. Dù bạn gọi chúng là gì, điều mấu chốt cần ghi nhớ là bạn sẽ muốn có các bài kiểm thử chức năng tự động với phạm vi khác nhau cho những mục đích khác nhau.

Given the confusion, it's worth looking at what these different layers mean.

Trước sự rối rắm đó, cũng đáng xem xét những tầng khác nhau này có nghĩa là gì.

Let's look at a worked example. In Figure 9-3, we have our helpdesk application and our main website, both of which are interacting with our Customer microservice to retrieve, review, and edit customer details. Our Customer microservice is in turn talking to our Loyalty microservice, where our customers accrue points by buying Justin Bieber CDs. Probably. This is obviously a sliver of our overall MusicCorp system, but it is a good enough slice for us to dive into a few different scenarios we may want to test.

Hãy xem một ví dụ cụ thể. Trong Hình 9-3, ta có ứng dụng helpdesk và website chính, cả hai đều tương tác với microservice Customer của ta để truy xuất, xem lại, và chỉnh sửa thông tin khách hàng. Microservice Customer của ta lần lượt nói chuyện với microservice Loyalty, nơi khách hàng của ta tích điểm bằng cách mua đĩa CD của Justin Bieber. Có lẽ vậy. Đây hiển nhiên chỉ là một mẫu nhỏ trong toàn bộ hệ thống MusicCorp của ta, nhưng nó là một lát cắt đủ tốt để ta đi sâu vào một vài kịch bản khác nhau mà ta có thể muốn kiểm thử.

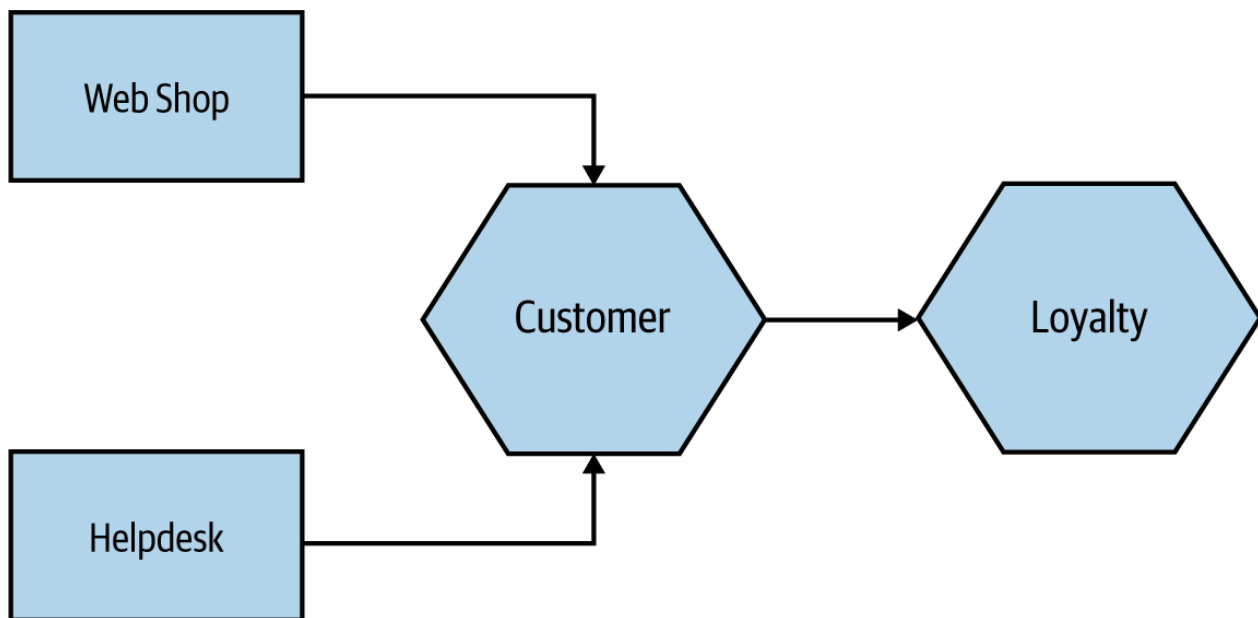


Figure 9-3. Part of our music shop under test

Hình 9-3. Một phần cửa hàng nhạc của ta đang được kiểm thử

Unit Tests — Unit Test

Unit tests typically test a single function or method call. The tests generated as a side effect of test-driven design (TDD) will fall into this category, as do the sorts of tests generated by techniques such as property-based testing. We're not launching microservices here and are limiting the use of external files or network connections. In general, you want a large number of these sorts of tests. Done right, they are very, very fast, and on modern hardware you could expect to run many thousands of these in less than a minute. I know many people who have these tests running automatically when files are changed locally—especially with interpreted languages, this can give very fast feedback cycles.

Unit test thường kiểm thử một hàm hoặc một lời gọi phương thức đơn lẻ. Các bài kiểm

Unit tests help us developers and thus would be technology facing, not business facing, in Marick's terminology. They are also where we hope to catch most of our bugs.

So in our example, when we think about the Customer microservice, unit tests would cover small parts of the code in isolation, as shown in Figure 9-4 .

The diagram shows a central hexagon labeled "Customer". Inside this hexagon, there are three smaller, dashed-line hexagons arranged horizontally. Arrows point from the "Web Shop" and "Helpdesk" rectangles to the "Customer" hexagon. An arrow points from the "Customer" hexagon to a "Loyalty" hexagon. Dashed arrows point from the text "Individual unit test scope" to each of the three smaller hexagons inside the "Customer" hexagon.

Hình 9-4. Phạm vi của unit test trên hệ thống ví dụ của ta

Mục tiêu chính của những bài kiểm thử này là cho ta phản hồi rất nhanh về việc chức năng của ta có tốt hay không. Unit test cũng quan trọng trong việc hỗ trợ tái cấu trúc (refactoring) mã, cho phép ta tái cơ cấu mã trong khi vẫn yên tâm rằng các bài kiểm thử phạm vi nhỏ của ta sẽ bắt được nếu ta mắc sai lầm.

Service Tests — Service Test

Service tests are designed to bypass the user **interface** and test our microservices directly. In a monolithic application, we might just be testing a collection of classes that provide a service to the UI. For a system comprising a number of microservices, a **service test** would test an individual microservice's capabilities.

Service test được thiết kế để bỏ qua giao diện người dùng và kiểm thử trực tiếp các microservice của ta. Trong một ứng dụng monolith, ta có thể chỉ đang kiểm thử một tập hợp các lớp cung cấp một dịch vụ cho UI. Đối với một hệ thống gồm nhiều microservice, một service test sẽ kiểm thử năng lực của một microservice riêng lẻ.

By running tests against a single microservice in this way, we get increased confidence that the service will behave as we expect, but we still keep the scope of the test somewhat isolated. The cause of the test failure should be limited to just the microservice under test. To achieve this isolation, we need to stub out all external collaborators so only the microservice itself is in scope, as Figure 9-5 shows.

Bằng cách chạy kiểm thử trên một microservice đơn lẻ theo cách này, ta có thêm niềm tin rằng dịch vụ sẽ hành xử như ta mong đợi, nhưng ta vẫn giữ cho phạm vi của bài kiểm thử được cô lập phần nào. Nguyên nhân khiến bài kiểm thử fail nên được giới hạn chỉ ở microservice đang được kiểm thử. Để đạt được sự cô lập này, ta cần thay thế (stub out) toàn bộ các cộng tác viên bên ngoài để chỉ riêng microservice đó nằm trong phạm vi, như Hình 9-5 cho thấy.

Some of these tests could be as fast as our small-scoped unit tests, but if you decide to test against a real **database** or to go over networks to stubbed downstream collaborators, test times can increase. These tests also cover more scope than a simple unit test, so when they fail, it can be harder to detect what is broken than with a unit test. However, they have considerably fewer moving parts and are therefore less brittle than larger-scoped tests.

Một số bài kiểm thử này có thể nhanh ngang với các unit test phạm vi nhỏ của ta, nhưng nếu bạn quyết định kiểm thử trên một cơ sở dữ liệu thật hoặc đi qua mạng đến các cộng tác viên hạ nguồn đã được stub, thời gian kiểm thử có thể tăng lên. Những bài kiểm thử này cũng bao phủ phạm vi rộng hơn một unit test đơn giản, nên khi chúng fail, có thể khó phát hiện cái gì đã hỏng hơn so với unit test. Tuy nhiên, chúng có ít bộ phận chuyển động hơn đáng kể và do đó ít giòn (brittle) hơn các bài kiểm thử phạm vi rộng.

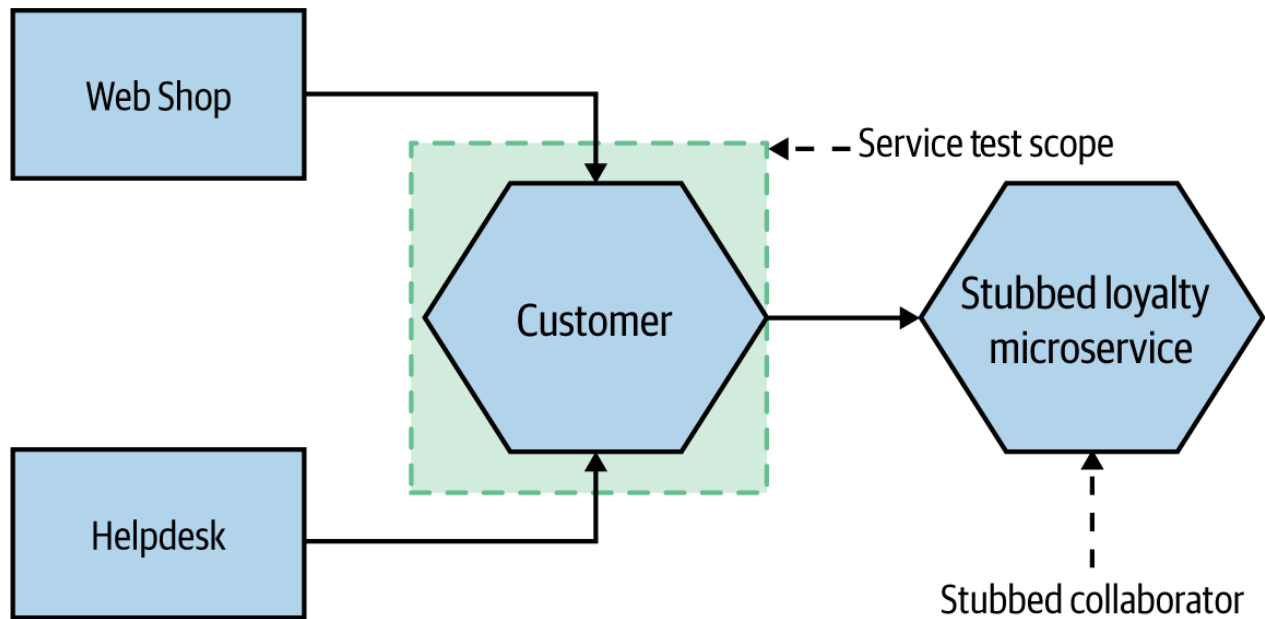


Figure 9-5. Scope of service tests on our example system

Hình 9-5. Phạm vi của service test trên hệ thống ví dụ của ta

End-to-End Tests — End-to-End Test

End-to-end tests are tests run against your entire system. Often they will be driving a GUI through a browser, but they could easily be mimicking other sorts of user interaction, such as uploading a file.

End-to-end test là những bài kiểm thử chạy trên toàn bộ hệ thống của bạn. Thường thì chúng sẽ điều khiển một GUI thông qua trình duyệt, nhưng chúng cũng có thể dễ dàng mô phỏng các loại tương tác người dùng khác, chẳng hạn như tải lên một tệp.

These tests cover a lot of production code, as we see in Figure 9-6 . Thus when they pass, you feel good: you have a high degree of confidence that the code being tested will work in production. But this increased scope comes with downsides, and as we'll see shortly, end-to-end tests can be very tricky to do well in a microservices context.

Những bài kiểm thử này bao phủ rất nhiều mã production, như ta thấy trong Hình 9-6. Do đó khi chúng pass, bạn cảm thấy yên tâm: bạn có mức độ tin tưởng cao rằng mã đang được kiểm thử sẽ hoạt động trên production. Nhưng phạm vi tăng lên này đi kèm với những mặt trái, và như ta sẽ thấy ngay sau đây, end-to-end test có thể rất khó làm cho tốt trong bối cảnh microservices.

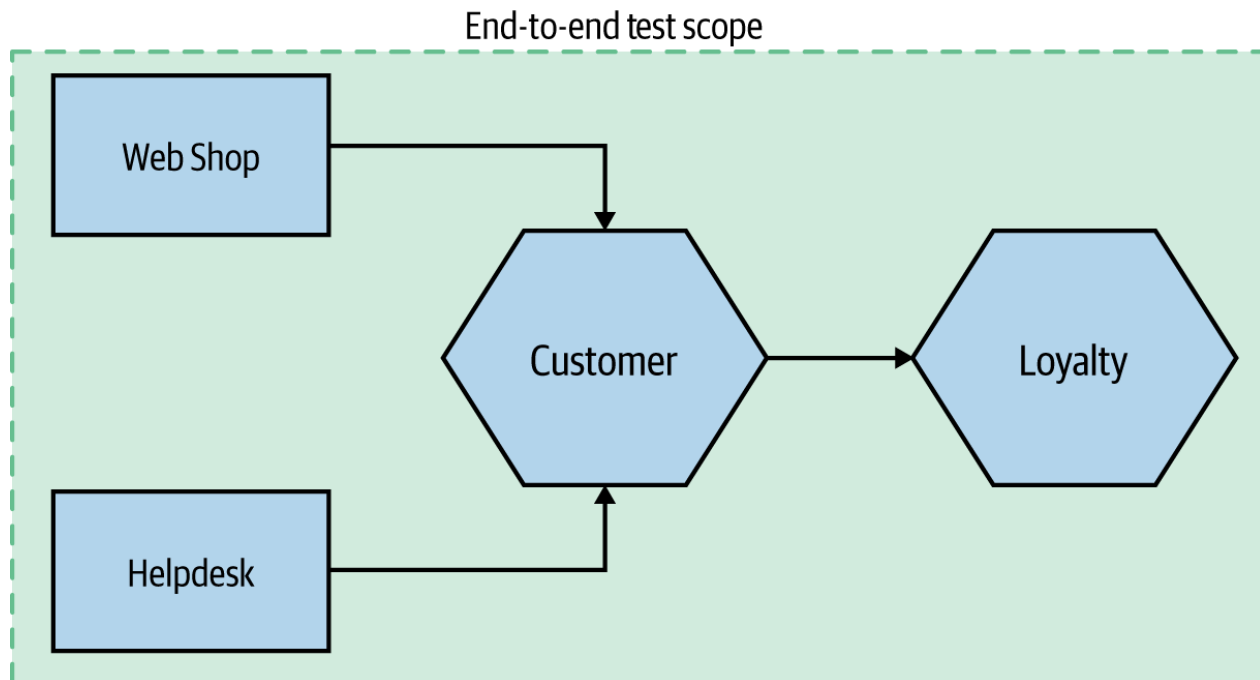


Figure 9-6. Scope of end-to-end tests on our example system

Hình 9-6. Phạm vi của end-to-end test trên hệ thống ví dụ của ta

What About Integration Tests? — Còn kiểm thử tích hợp (Integration Tests) thì sao?

You may notice that I haven’t explicitly described integration tests. This is on purpose. I’ve found that the term is often used by different people to describe different types of tests. For some, an integration test might only look at the interactions between two services, or perhaps the binding between code and a database. For others, integration tests end up being the same as full end-to-end tests. I’ve tried to use more explicit terms in this chapter; I hope it will be easy for you to match whatever you refer to as an “integration test” to the terms I use here.

Bạn có thể để ý rằng tôi đã không mô tả tường minh kiểm thử tích hợp (integration tests). Điều này là có chủ đích. Tôi nhận thấy thuật ngữ này thường được những người khác nhau dùng để mô tả những loại kiểm thử khác nhau. Với một số người, một integration test có thể chỉ xem xét tương tác giữa hai dịch vụ, hoặc có lẽ sự liên kết giữa mã và một cơ sở dữ liệu. Với những người khác, integration test rốt cuộc lại giống hệt end-to-end test đầy đủ. Tôi đã cố dùng những thuật ngữ tường minh hơn trong chương này; tôi hy vọng bạn sẽ dễ dàng đối chiếu bất cứ thứ gì bạn gọi là “integration test” với những thuật ngữ tôi dùng ở đây.

Trade-Offs — Đánh đổi (Trade-Offs)

What we are striving for with the different types of tests that the pyramid covers is a sensible balance. We want fast feedback, and we want confidence that our system works.

Điều ta hướng tới với các loại kiểm thử khác nhau mà kim tự tháp bao quát là một sự cân

bằng hợp lý. Ta muốn phản hồi nhanh, và ta muốn có niềm tin rằng hệ thống của mình hoạt động.

Unit tests are small in scope, so when they fail we can find the problem quickly. They are also quick to write and really quick to run. As our tests get larger in scope, we get more confidence in our system, but our feedback starts to suffer as the tests take longer to run. They are also more costly to write and maintain,

Unit test có phạm vi nhỏ, nên khi chúng fail ta có thể tìm ra vấn đề nhanh chóng. Chúng cũng nhanh để viết và thực sự nhanh để chạy. Khi các bài kiểm thử của ta lớn dần về phạm vi, ta có thêm niềm tin vào hệ thống, nhưng phản hồi của ta bắt đầu bị ảnh hưởng vì các bài kiểm thử mất nhiều thời gian chạy hơn. Chúng cũng tốn kém hơn để viết và bảo trì.

You'll frequently be balancing how many of each type of test you need to find that sweet spot. Finding that your test suite is taking too long to run? When broader-scoped tests like our service or end-to-end tests fail, write a smaller-scoped unit test to pick up that breakage sooner. Look to replace some larger-scoped (and slower) tests with faster, smaller-scoped unit tests. On the other hand, when a bug slips through to production, it may be a sign that you're missing a test.

Bạn sẽ thường xuyên phải cân đối xem mình cần bao nhiêu mỗi loại kiểm thử để tìm ra điểm ngọt ngào đó. Phát hiện ra bộ kiểm thử của bạn mất quá nhiều thời gian để chạy? Khi những bài kiểm thử phạm vi rộng hơn như service test hay end-to-end test fail, hãy viết một unit test phạm vi nhỏ hơn để bắt sự cố đó sớm hơn. Hãy tìm cách thay thế một số bài kiểm thử phạm vi rộng hơn (và chậm hơn) bằng những unit test nhanh hơn, phạm vi nhỏ hơn. Mặt khác, khi một lỗi lọt qua đến production, đó có thể là dấu hiệu bạn đang thiếu một bài kiểm thử.

So if these tests all have trade-offs, how many of each test type do you want? A good rule of thumb is that you probably want an order of magnitude more tests as you descend the pyramid. It's important to know that you do have different types of automated tests, and it's important to understand if your current balance gives you a problem!

Vậy nếu tất cả những bài kiểm thử này đều có đánh đổi, bạn muốn bao nhiêu mỗi loại? Một quy tắc chung tốt là bạn có lẽ muốn nhiều hơn một bậc độ lớn (order of magnitude) khi bạn đi xuống kim tự tháp. Điều quan trọng là biết rằng bạn có nhiều loại kiểm thử tự động khác nhau, và điều quan trọng là hiểu liệu sự cân bằng hiện tại của bạn có đang gây ra vấn đề hay không!

I worked on one monolithic system, for example, where we had 4,000 unit tests, 1,000 service tests, and 60 end-to-end tests. We decided that from a feedback point of view, we had way too many service and end-to-end tests (the latter of which were the worst offenders in impacting feedback loops), so we worked hard to replace the test coverage with smaller-scoped tests.

Chẳng hạn, tôi từng làm việc trên một hệ thống monolith mà ở đó chúng tôi có 4.000 unit test, 1.000 service test, và 60 end-to-end test. Chúng tôi quyết định rằng xét từ góc độ phản hồi, chúng tôi có quá nhiều service test và end-to-end test (loại sau là thủ phạm tồi tệ nhất trong việc ảnh hưởng đến các vòng phản hồi), nên chúng tôi đã nỗ lực thay thế phần độ phủ kiểm thử bằng những bài kiểm thử phạm vi nhỏ hơn.

A common antipattern is what is often referred to as a test snow cone , or inverted pyramid. Here, there are little to no small-scoped tests, with all the coverage in large-scoped tests. These projects often have glacially slow test runs and very long feedback cycles. If these tests are run as part of continuous integration, you won't get many builds, and the nature of the **build** times means that the build can stay broken for a long period when something does break.

Một anti-pattern phổ biến là cái thường được gọi là kem ốc quế kiểm thử (test snow cone), hay kim tự tháp ngược (inverted pyramid). Ở đây, có rất ít hoặc không có bài kiểm thử phạm vi nhỏ, với toàn bộ độ phủ nằm ở các bài kiểm thử phạm vi rộng. Những dự án này thường có thời gian chạy kiểm thử chậm như rùa bò và các chu kỳ phản hồi rất dài. Nếu những bài kiểm thử này được chạy như một phần của tích hợp liên tục, bạn sẽ không có nhiều bản build, và do tính chất của thời gian build, bản build có thể giữ trạng thái hỏng trong một khoảng thời gian dài khi có thứ gì đó thực sự hỏng.

Implementing Service Tests — Triển khai Service Test

Implementing unit tests is a fairly simple affair in the grand scheme of things, and there is plenty of documentation out there explaining how to write them. The service and end-to-end tests are the ones that are more interesting, especially in the context of microservices, so that's what we'll focus on next.

Triển khai unit test là một việc khá đơn giản trong bức tranh tổng thể, và ngoài kia có vô số tài liệu giải thích cách viết chúng. Service test và end-to-end test mới là những thứ thú vị hơn, đặc biệt trong bối cảnh microservices, nên đó là thứ ta sẽ tập trung tiếp theo.

Our service tests want to test a slice of functionality across the whole microservice, and only that microservice. So if we wanted to write a service test for the Customer microservice from Figure 9-3, we would **deploy** an instance of the Customer microservice—and as discussed earlier, we would want to stub out the Loyalty microservice to better ensure that a test breakage can be mapped to an issue with the Customer microservice itself.

Service test của ta muốn kiểm thử một lát cắt chức năng xuyên suốt toàn bộ một microservice, và chỉ riêng microservice đó. Vậy nếu ta muốn viết một service test cho microservice Customer từ Hình 9-3, ta sẽ triển khai một instance của microservice Customer—và như đã bàn ở trên, ta sẽ muốn stub out microservice Loyalty để đảm bảo tốt hơn rằng một sự cố kiểm thử có thể được ánh xạ tới một vấn đề với chính microservice Customer.

As we explored in Chapter 7, once we check in our software, one of the first things our automated build will do is create a binary **artifact** for our microservice—for example, creating a **container** image for that version of the software. So deploying that is pretty straightforward. But how do we handle faking the downstream collaborators?

Như ta đã khám phá trong Chương 7, một khi ta check-in phần mềm của mình, một trong những việc đầu tiên mà bản build tự động của ta sẽ làm là tạo ra một artifact nhị phân cho microservice—ví dụ, tạo một container image cho phiên bản đó của phần mềm. Vậy nên việc triển khai cái đó khá thẳng thắn. Nhưng làm sao ta xử lý việc giả lập các cộng tác viên hạ nguồn?

Our service test suite needs to stub out downstream collaborators and configure the microservice under test to connect to the stub services. We then need to configure the stubs to send back responses to mimic the real-world microservices.

Bộ service test của ta cần stub out các cộng tác viên hạ nguồn và cấu hình microservice đang được kiểm thử để kết nối tới các dịch vụ stub. Sau đó ta cần cấu hình các stub để gửi lại những phản hồi nhằm mô phỏng các microservice thực tế.

Mocking or Stubbing — Mocking hay Stubbing

When I talk about stubbing downstream collaborators, I mean that we create a stub microservice that responds with canned responses to known requests from the microservice under test. For example, I might tell my stubbed Loyalty microservice that when asked for the balance of customer 123, it should return 15,000. The test doesn't care if the stub is called 0, 1, or 100 times. A variation on this is to use a mock instead of a stub.

Khi tôi nói về việc stubbing các cộng tác viên hạ nguồn, ý tôi là ta tạo một microservice stub trả lời bằng những phản hồi đóng hộp (canned responses) cho các yêu cầu đã biết từ microservice đang được kiểm thử. Ví dụ, tôi có thể bảo microservice Loyalty đã được stub của mình rằng khi được hỏi về số dư của khách hàng 123, nó nên trả về 15.000. Bài kiểm thử không quan tâm stub được gọi 0, 1, hay 100 lần. Một biến thể của cách này là dùng một mock thay cho một stub.

When using a mock, I actually go further and make sure the call was made. If the expected call is not made, the test fails. Implementing this approach requires more smarts in the fake collaborators that we create, and if overused it can cause tests to become brittle. As noted, however, a stub doesn't care if it is called 0, 1, or many times.

Khi dùng mock, tôi thực ra đi xa hơn và đảm bảo rằng lời gọi đã được thực hiện. Nếu lời gọi mong đợi không được thực hiện, bài kiểm thử fail. Triển khai cách tiếp cận này đòi hỏi nhiều “thông minh” hơn trong các cộng tác viên giả mà ta tạo ra, và nếu lạm dụng nó có thể khiến các bài kiểm thử trở nên giòn. Tuy nhiên, như đã nói, một stub không quan tâm nó được gọi 0, 1, hay nhiều lần.

Sometimes, though, mocks can be very useful to ensure that the expected side effects happen. For example, I might want to check that when I create a customer, a new points balance is set up for that customer. The balance between stubbing and mocking calls is a delicate one and is just as fraught in service tests as in unit tests. In general, though, I use stubs far more than mocks for service tests. For a more in-depth discussion of this trade-off, take a look at *Growing Object-Oriented Software, Guided by Tests* by Steve Freeman and Nat Pryce.³

Dù vậy, đôi khi mock có thể rất hữu ích để đảm bảo rằng các tác dụng phụ (side effects) mong đợi xảy ra. Ví dụ, tôi có thể muốn kiểm tra rằng khi tôi tạo một khách hàng, một số dư điểm mới được thiết lập cho khách hàng đó. Sự cân bằng giữa stubbing và mocking các lời gọi là một việc tinh tế và cũng đầy gai góc trong service test hệt như trong unit test. Tuy nhiên, nhìn chung tôi dùng stub nhiều hơn mock cho service test. Để có một cuộc thảo luận sâu hơn về đánh đổi này, hãy xem cuốn *Growing Object-Oriented Software, Guided by Tests* của Steve Freeman và Nat Pryce.³

In general, I rarely use mocks for this sort of testing. But having a tool that can implement both mocks and stubs is useful.

Nhìn chung, tôi hiếm khi dùng mock cho loại kiểm thử này. Nhưng việc có một công cụ có thể triển khai cả mock lẫn stub là hữu ích.

While I feel that stubs and mocks are actually fairly well differentiated, I know the distinction can be confusing to some, especially when some people throw in other terms like fakes, spies, and dummies. Gerard Meszaros calls all of these things, including stubs and mocks, “Test Doubles”.

Mặc dù tôi cảm thấy stub và mock thực sự được phân biệt khá rõ ràng, tôi biết sự phân biệt này có thể gây bối rối cho một số người, đặc biệt khi một số người ném thêm vào những thuật ngữ khác như fakes, spies, và dummies. Gerard Meszaros gọi tất cả những thứ này, bao gồm cả stub và mock, là “Test Doubles”.

A Smarter Stub Service — Một dịch vụ Stub thông minh hơn

Normally, for stub services, I've rolled them myself. I've used everything from the Apache web server or nginx to embedded Jetty containers or even command-line-launched Python web servers to launch stub servers for such test cases. I've probably reproduced the same work time and time again in creating these stubs. An old Thoughtworks colleague of mine, Brandon Byars, has potentially saved many of us a chunk of work with his stub/mock server called mountebank.

Thông thường, đối với các dịch vụ stub, tôi tự viết lấy. Tôi đã dùng đủ thứ từ máy chủ web Apache hay nginx đến các container Jetty nhúng, hay thậm chí các máy chủ web Python khởi chạy từ dòng lệnh để dựng các máy chủ stub cho những tình huống kiểm thử như vậy. Tôi có lẽ đã lặp lại cùng một công việc hết lần này đến lần khác trong việc tạo ra những stub này. Một đồng nghiệp cũ của tôi ở Thoughtworks, Brandon Byars, có lẽ đã giúp nhiều người trong chúng ta tiết kiệm được một mớ công việc với máy chủ stub/mock của anh ấy mang tên mountebank.

You can think of mountebank as a small software appliance that is programmable via HTTP. The fact that it happens to be written in NodeJS is completely opaque to any calling service. When mountebank launches, you send it commands telling it to create one or more “imposters,” which will respond on a given port with a specific protocol (currently TCP, HTTP, HTTPS, and SMTP are supported), and what responses these imposters should send when requests are sent. It also supports setting expectations if you want to use it as a mock. Because a single mountebank instance can support the creation of multiple imposters, you can use it to stub out multiple downstream microservices.

Bạn có thể nghĩ về mountebank như một thiết bị phần mềm nhỏ có thể lập trình được qua HTTP. Việc nó tình cờ được viết bằng NodeJS hoàn toàn trong suốt (opaque) đối với bất kỳ dịch vụ gọi nào. Khi mountebank khởi chạy, bạn gửi cho nó các lệnh bảo nó tạo một hoặc nhiều “imposters” (kẻ mạo danh), những thứ này sẽ trả lời trên một cổng nhất định với một giao thức cụ thể (hiện hỗ trợ TCP, HTTP, HTTPS, và SMTP), và những phản hồi nào mà các imposter này nên gửi khi nhận được yêu cầu. Nó cũng hỗ trợ thiết lập các kỳ vọng (expectations) nếu bạn muốn dùng nó như một mock. Vì một instance mountebank đơn lẻ có thể hỗ trợ việc tạo nhiều imposter, bạn có thể dùng nó để stub out nhiều microservice hạ nguồn.

mountebank does have uses outside of automated functional testing. Capital One made use of mountebank to replace existing mocking infrastructure for their large-scale performance tests, for example. 4

mountebank cũng có những công dụng ngoài kiểm thử chức năng tự động. Chẳng hạn, Capital One đã dùng mountebank để thay thế hạ tầng mocking hiện có cho các bài kiểm thử hiệu năng quy mô lớn của họ.⁴

One limitation of mountebank is that it doesn't support stubbing for messaging protocols—for example, if you wanted to make sure that an event was properly sent (and

Một hạn chế của mountebank là nó không hỗ trợ stubbing cho các giao thức nhắn tin (messaging protocols)—ví dụ, nếu bạn muốn chắc chắn rằng một sự kiện được gửi đúng cách (và

3 Steve Freeman and Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (Upper Saddle River, NJ: Addison-Wesley, 2009).

3 Steve Freeman và Nat Pryce, *Growing Object-Oriented Software, Guided by Tests* (Upper Saddle River, NJ: Addison-Wesley, 2009).

4 Jason D. Valentino, “Moving One of Capital One’s Largest Customer-Facing Apps to AWS,” Capital One Tech, May 24, 2017, <https://oreil.ly/5UM5W> .

4 Jason D. Valentino, “Moving One of Capital One’s Largest Customer-Facing Apps to AWS,” Capital One Tech, May 24, 2017, <https://oreil.ly/5UM5W> .

maybe received) via a broker, you’ll have to look elsewhere for a solution. This is one area where Pact might be able to help—that’s something we’ll look at more shortly.

có thể được nhận) qua một broker, bạn sẽ phải tìm giải pháp ở nơi khác. Đây là một lĩnh vực mà Pact có thể giúp được—đó là thứ ta sẽ xem xét kỹ hơn ngay sau đây.

So if we want to run our service tests for just our Customer microservice, we can launch the Customer microservice and a mountebank instance that acts as our Loyalty microservice on the same machine. And if those tests pass, I can deploy the Customer service straightaway! Or can I? What about the services that call the Customer microservice—the helpdesk and the web shop? Do we know if we have made a change that may break them? Of course, we have forgotten the important tests at the top of the pyramid: the end-to-end tests.

Vậy nếu ta muốn chạy service test chỉ cho riêng microservice Customer, ta có thể khởi chạy microservice Customer và một instance mountebank đóng vai microservice Loyalty trên cùng một máy. Và nếu những bài kiểm thử đó pass, tôi có thể triển khai dịch vụ Customer ngay lập tức! Hay là chưa thể? Còn các dịch vụ gọi đến microservice Customer thì sao—helpdesk và cửa hàng web? Ta có biết liệu mình đã tạo ra một thay đổi có thể làm hỏng chúng hay không? Tất nhiên, ta đã quên mất những bài kiểm thử quan trọng ở đỉnh kim tự tháp: các end-to-end test.

Implementing (Those Tricky) End-to-End Tests — Triển khai (Những) End-to-End Test (gai góc đó)

In a microservice system, the capabilities we expose via our user interfaces are delivered by a number of microservices. The point of the end-to-end tests as outlined in Mike Cohn’s pyramid is to drive functionality through these user interfaces against everything underneath to give us some feedback on the quality of the system as a whole.

Trong một hệ thống microservice, các năng lực mà ta phơi bày qua giao diện người dùng được cung cấp bởi nhiều microservice. Mục đích của end-to-end test như được phác họa trong kim tự tháp của Mike Cohn là điều khiển chức năng thông qua các giao diện người dùng này, dựa trên mọi thứ bên dưới, để cho ta một số phản hồi về chất lượng của toàn hệ thống nói chung.

So, to implement an **end-to-end test**, we need to deploy multiple microservices together, and then run a test against all of them. Obviously, this test has a much larger scope, resulting in more confidence that our system works! On the other hand, these tests are liable to be slower and make it harder to diagnose failure. Let’s dig into them a bit more using our previous example to see how these tests can fit in.

Vậy nên, để triển khai một end-to-end test, ta cần triển khai nhiều microservice cùng nhau, rồi chạy một bài kiểm thử trên tất cả chúng. Hiển nhiên, bài kiểm thử này có phạm vi lớn hơn nhiều, dẫn đến nhiều niềm tin hơn rằng hệ thống của ta hoạt động! Mặt khác, những bài kiểm thử này dễ chậm hơn và làm cho việc chẩn đoán sự cố khó hơn. Hãy đào sâu hơn một chút bằng cách dùng ví dụ trước đó để xem những bài kiểm thử này có thể phù hợp ở đâu.

Imagine we want to push out a new version of the Customer microservice. We want to deploy our changes into production as soon as possible but are concerned that we may have introduced a change that could break either the helpdesk or the web shop. No problem—let’s deploy all our services together and run some tests against the helpdesk and web shop to see if we’ve introduced a bug. Now, a naive approach would be to just add these tests onto the end of our customer service pipeline, as in Figure 9-7 .

Hãy hình dung ta muốn đẩy ra một phiên bản mới của microservice Customer. Ta muốn triển khai các thay đổi của mình vào production càng sớm càng tốt nhưng lo ngại rằng mình có thể đã đưa vào một thay đổi có thể làm hỏng helpdesk hoặc cửa hàng web. Không vấn đề gì—hãy triển khai tất cả các dịch vụ của ta cùng nhau và chạy một số bài kiểm thử trên helpdesk và cửa hàng web để xem ta có đưa vào lỗi nào không. Giờ thì, một cách tiếp cận ngây thơ sẽ là chỉ thêm những bài kiểm thử này vào cuối pipeline của dịch vụ Customer, như trong Hình 9-7.

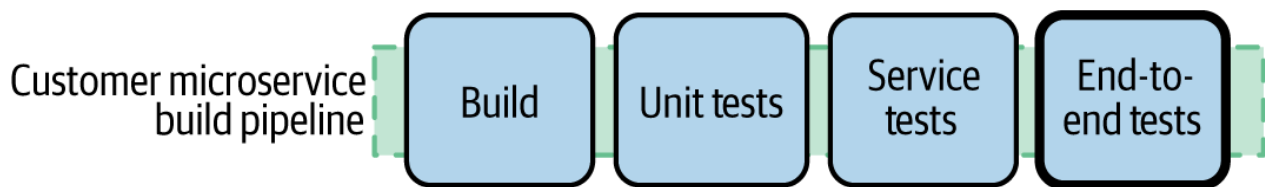


Figure 9-7. Adding our end-to-end tests stage: the right approach?

Hình 9-7. Thêm giai đoạn end-to-end test của ta: cách tiếp cận đúng?

So far, so good. But the first question we have to ask ourselves is, which version of the other microservices should we use? Should we run our tests against the versions of helpdesk and web shop that are in production? It’s a sensible assumption—but what if a new version of either the helpdesk or the web shop is queued up to go live? What should we do then?

Đến đây, mọi thứ vẫn ổn. Nhưng câu hỏi đầu tiên ta phải tự hỏi là, ta nên dùng phiên bản nào của các microservice khác? Ta có nên chạy kiểm thử trên các phiên bản helpdesk và cửa hàng web đang ở trên production không? Đó là một giả định hợp lý—nhưng nếu một phiên bản mới của helpdesk hay cửa hàng web đang xếp hàng chờ lên production thì sao? Khi đó ta nên làm gì?

Here’s another problem: if we have a set of Customer end-to-end tests that deploy lots of microservices and run tests against them, what about the end-to-end tests that the other microservices run? If they are testing the same thing, we may find ourselves covering lots of the same ground and may duplicate much of the effort to deploy all those microservices in the first place.

Đây là một vấn đề khác: nếu ta có một bộ end-to-end test cho Customer mà chúng triển khai rất nhiều microservice và chạy kiểm thử trên chúng, thì còn các end-to-end test mà những microservice khác chạy thì sao? Nếu chúng đang kiểm thử cùng một thứ, ta có thể thấy mình đang bao phủ rất nhiều phần giống nhau và có thể lặp lại nhiều công sức để triển khai tất cả những microservice đó ngay từ đầu.

We can deal with both of these problems elegantly by having multiple pipelines “fan-in” to a single end-to-end test stage. Here, when one of a number of different builds is triggered, it can result in shared build steps being triggered. For example, in Figure 9-8 , a successful build for any of the four microservices would end up triggering the shared end-to-end tests stage. Some CI tools with better **build pipeline** support will enable fan-in models like this out of the box.

Ta có thể xử lý cả hai vấn đề này một cách thanh lịch bằng cách cho nhiều pipeline “gộp

vào” (fan-in) một giai đoạn end-to-end test duy nhất. Ở đây, khi một trong số nhiều bản build khác nhau được kích hoạt, nó có thể dẫn đến việc các bước build dùng chung được kích hoạt. Ví dụ, trong Hình 9-8, một bản build thành công cho bất kỳ microservice nào trong bốn microservice tốt cuộc sẽ kích hoạt giai đoạn end-to-end test dùng chung. Một số công cụ CI có hỗ trợ build pipeline tốt hơn sẽ cho phép các mô hình fan-in như thế này ngay từ đầu mà không cần cấu hình thêm.

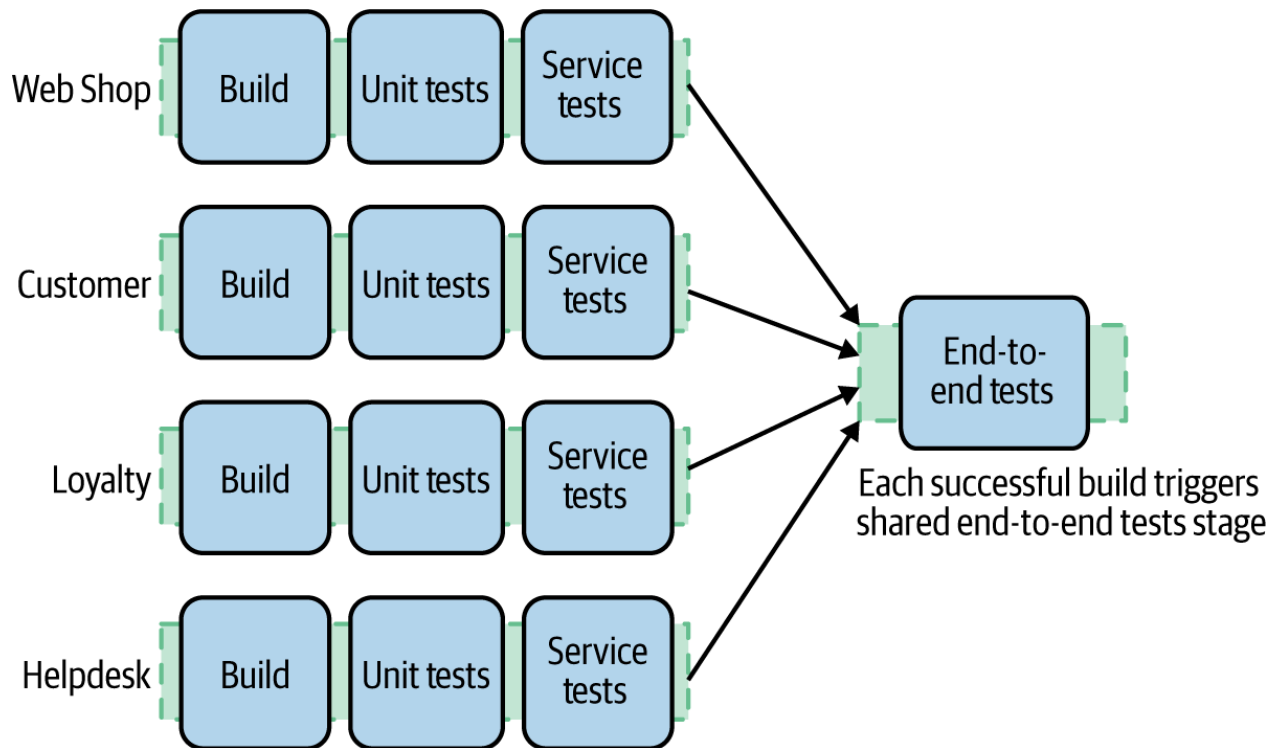


Figure 9-8. A standard way to handle end-to-end tests across services

Hình 9-8. Một cách chuẩn để xử lý end-to-end test xuyên suốt các dịch vụ

So any time one of our services changes, we run the tests local to that service. If those tests pass, we trigger our integration tests. Great, eh? Well, there are unfortunately many disadvantages to end-to-end testing.

Vậy nên bất cứ khi nào một trong các dịch vụ của ta thay đổi, ta chạy các bài kiểm thử cục bộ của dịch vụ đó. Nếu những bài kiểm thử đó pass, ta kích hoạt các bài kiểm thử tích hợp. Tuyệt nhỉ? Ừ thì, thật không may lại có rất nhiều bất lợi đối với kiểm thử end-to-end.

Flaky and Brittle Tests — Kiểm thử chập chờn và giòn (Flaky and Brittle Tests)

As test scope increases, so too does the number of moving parts. These moving parts can introduce test failures that do not show that the functionality under test is broken but indicate that some other problem has occurred. As an example, if we have a test to verify that we can place an order for a single CD and we are running that test against four or five microservices, if any of them is down, we could get a failure that has nothing to do with the nature of the test itself. Likewise, a temporary network glitch could cause a test to fail without saying anything about the functionality under test.

Khi phạm vi kiểm thử tăng lên, số lượng các bộ phận chuyển động cũng tăng theo. Những bộ phận chuyển động này có thể gây ra các sự cố kiểm thử không cho thấy rằng chức năng đang được kiểm thử bị hỏng mà chỉ báo hiệu rằng một vấn đề nào khác đã xảy ra. Ví dụ, nếu ta có một bài kiểm thử để kiểm chứng rằng ta có thể đặt mua một đĩa CD và ta đang chạy bài kiểm thử đó trên bốn hay năm microservice, nếu bất kỳ cái nào trong số chúng bị down, ta có thể nhận một kết quả fail chẳng liên quan gì đến bản chất của bài kiểm thử. Tương tự, một trục trặc mạng tạm thời có thể khiến một bài kiểm thử fail mà không nói lên điều gì về chức năng đang được kiểm thử.

The more moving parts there are, the more brittle our tests may be and the less deterministic they are. If you have tests that sometimes fail, but everyone just reruns them because they may pass again later, then you have flaky tests. And tests covering lots of different processes are not the only culprit. Tests that cover functionality being exercised on multiple threads (and across multiple processes) are also often problematic; a failure could mean a race condition or a **time-out**, or that the functionality is actually broken. Flaky tests are the enemy. When they fail, they don't tell us much. We rerun our CI builds in the hope that they will pass again later, only to see check-ins pile up, and suddenly we find ourselves with a load of broken functionality.

Càng có nhiều bộ phận chuyển động, các bài kiểm thử của ta càng giòn và càng kém ổn định. Nếu bạn có những bài kiểm thử đôi khi fail, nhưng mọi người cứ chạy lại chúng vì chúng có thể sẽ pass lại sau đó, thì bạn đang có những bài kiểm thử chập chờn (flaky tests). Và những bài kiểm thử bao phủ nhiều tiến trình khác nhau không phải là thủ phạm duy nhất. Những bài kiểm thử bao phủ chức năng được thực thi trên nhiều luồng (và xuyên nhiều tiến trình) cũng thường có vấn đề; một sự cố có thể nghĩa là một điều kiện tranh chấp (race condition) hoặc một time-out, hoặc rằng chức năng thực sự bị hỏng. Kiểm thử chập chờn là kẻ thù. Khi chúng fail, chúng không cho ta biết nhiều. Ta chạy lại các bản build CI với hy vọng chúng sẽ pass lại sau đó, để rồi thấy các lần check-in chồng chất lên nhau, và đột nhiên ta thấy mình với một đồng chức năng bị hỏng.

When we detect flaky tests, it is essential that we do our best to remove them. Otherwise, we start to lose faith in a test suite that “always fails like that.” A test suite with flaky tests can become a victim of what Diane Vaughan calls the normalization of deviance—the idea that over time we can become so accustomed to things being wrong that we start to accept them as being normal and not a problem.⁵ This very human tendency means we need to find and eliminate these flaky tests as soon as we can before we start to assume that failing tests are OK.

Khi ta phát hiện ra kiểm thử chập chờn, điều thiết yếu là ta phải cố gắng hết sức để loại bỏ chúng. Nếu không, ta bắt đầu mất niềm tin vào một bộ kiểm thử “lúc nào cũng fail kiểu đó”. Một bộ kiểm thử với những bài kiểm thử chập chờn có thể trở thành nạn nhân của thứ mà Diane Vaughan gọi là sự bình thường hóa của lệch lạc (normalization of deviance)—ý tưởng rằng theo thời gian ta có thể trở nên quen với việc mọi thứ sai đến mức ta bắt đầu chấp nhận chúng là bình thường và không phải vấn đề.⁵ Khuynh hướng rất con người này nghĩa là ta cần tìm và loại bỏ những bài kiểm thử chập chờn này càng sớm càng tốt trước khi ta bắt đầu mặc định rằng các bài kiểm thử fail là chuyện chấp nhận được.

In “Eradicating Non-Determinism in Tests,”⁶ Martin Fowler advocates the approach that if you have flaky tests, you should track them down and—if you can't immediately fix them—remove them from the suite so you can treat them. See if you can rewrite them to avoid testing code running on multiple threads. See if you can make the underlying environment more stable. Better yet, see if you can replace the flaky test with a smaller-scoped test that is less likely to exhibit problems. In some cases, changing the software under test to make it easier to test can also be the right way forward.

Trong bài “Eradicating Non-Determinism in Tests,”⁶ Martin Fowler ủng hộ cách tiếp cận

rằng nếu bạn có những bài kiểm thử chậm chạp, bạn nên truy lùng chúng và—nếu bạn không thể sửa chúng ngay—loại chúng khỏi bộ kiểm thử để có thể xử lý riêng. Hãy xem liệu bạn có thể viết lại chúng để tránh kiểm thử mã chạy trên nhiều luồng không. Hãy xem liệu bạn có thể làm cho môi trường nền tảng ổn định hơn không. Tốt hơn nữa, hãy xem liệu bạn có thể thay thế bài kiểm thử chậm chạp bằng một bài kiểm thử phạm vi nhỏ hơn, ít có khả năng bộc lộ vấn đề hơn không. Trong một số trường hợp, thay đổi chính phần mềm đang được kiểm thử để nó dễ kiểm thử hơn cũng có thể là hướng đi đúng đắn.

Who Writes These End-to-End Tests? — Ai viết những End-to-End Test này?

With the tests that run as part of the pipeline for a specific microservice, the sensible starting point is for the team that owns that service to write those tests (we'll talk more about service ownership in Chapter 15). But if we consider that we might have multiple teams involved, and the end-to-end tests step is now effectively shared among the teams, who writes and looks after these tests?

Với các bài kiểm thử chạy như một phần của pipeline cho một microservice cụ thể, điểm khởi đầu hợp lý là để nhóm sở hữu dịch vụ đó viết những bài kiểm thử ấy (ta sẽ bàn nhiều hơn về quyền sở hữu dịch vụ trong Chương 15). Nhưng nếu ta xét đến việc có thể có nhiều nhóm tham gia, và bước end-to-end test giờ thực chất được chia sẻ giữa các nhóm, thì ai viết và trông coi những bài kiểm thử này?

I have seen a number of problems caused here. These tests become a free-for-all, with all teams granted access to add tests without any understanding of the health of the whole suite. This can often result in an explosion of test cases, sometimes

Tôi đã chứng kiến một số vấn đề nảy sinh ở đây. Những bài kiểm thử này trở thành một cuộc hỗn chiến tự do, với mọi nhóm được cấp quyền thêm kiểm thử mà không hề hiểu gì về tình trạng sức khỏe của toàn bộ bộ kiểm thử. Điều này thường có thể dẫn đến sự bùng nổ các tình huống kiểm thử, đôi khi

5 Diane Vaughan, *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA* (Chicago: University of Chicago Press, 1996).

5 Diane Vaughan, *The Challenger Launch Decision: Risky Technology, Culture, and Deviance at NASA* (Chicago: University of Chicago Press, 1996).

6 Martin Fowler, “Eradicating Non-Determinism in Tests,” [martinfowler.com](https://martinfowler.com/7Ve7e), April 14, 2011, <https://oreil.ly/7Ve7e> .

6 Martin Fowler, “Eradicating Non-Determinism in Tests,” [martinfowler.com](https://martinfowler.com/7Ve7e), April 14, 2011, <https://oreil.ly/7Ve7e> .

culminating in the test snow cone we talked about earlier. I have also seen situations in which, because there was no real obvious ownership of these tests, their results get ignored. When they break, everyone assumes it is someone else's problem, so they don't care whether the tests are passing.

lên đến đỉnh điểm là cái kem ốc quế kiểm thử mà ta đã nói trước đó. Tôi cũng đã chứng kiến những tình huống mà, vì không có quyền sở hữu thực sự rõ ràng đối với những bài kiểm thử này, kết quả của chúng bị phớt lờ. Khi chúng hỏng, mọi người đều cho rằng đó là vấn đề của người khác, nên họ chẳng quan tâm liệu các bài kiểm thử có đang pass hay không.

One solution I've seen here is to designate certain end-to-end tests as being the responsibility of a given team, even though they might cut across microservices being worked on by multiple different teams. I first learned of this approach from Emily Bache.⁷ The idea is that even though we make use of a “fan in” stage in our pipeline, they would then split the end-to-end test suite into groups of functionality that were owned by different teams, as we'll see in Figure 9-9.

Một giải pháp tôi từng thấy ở đây là chỉ định một số end-to-end test cụ thể thuộc trách nhiệm của một nhóm nhất định, mặc dù chúng có thể cắt ngang các microservice đang được nhiều nhóm khác nhau làm việc. Tôi lần đầu biết đến cách tiếp cận này từ Emily Bache.⁷ Ý tưởng là dù ta vẫn dùng một giai đoạn “fan-in” trong pipeline của mình, họ sau đó sẽ chia bộ end-to-end test thành các nhóm chức năng được sở hữu bởi các nhóm khác nhau, như ta sẽ thấy trong Hình 9-9.

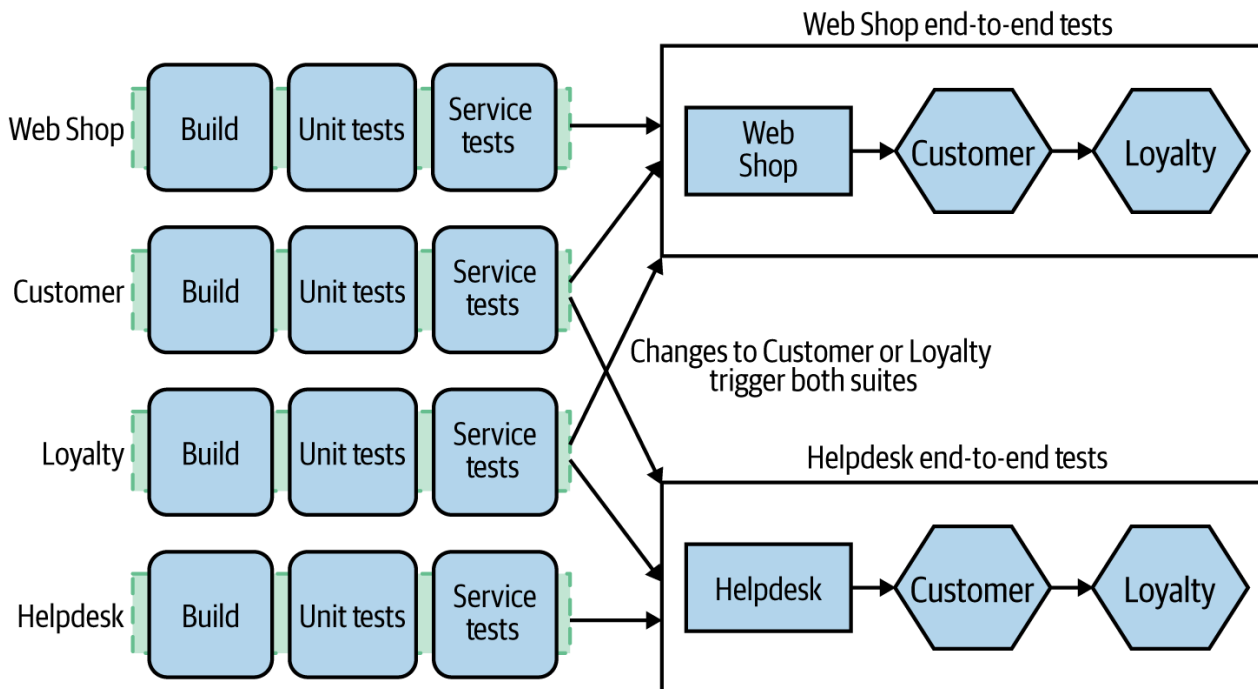


Figure 9-9. A standard way to handle end-to-end tests across services

Hình 9-9. Một cách chuẩn để xử lý end-to-end test xuyên suốt các dịch vụ

In this particular example, a change to the Web Shop that passes the service test stage would trigger the associated end-to-end tests, with the suite being owned by the same team that owns the Web Shop. Likewise, any changes to the Helpdesk only will trigger the associated end-to-end tests. But changes to either Customer or Loyalty trigger both sets of tests. This can lead us to a situation in which a change made to the Loyalty microservice could break both sets of end-to-end tests, potentially requiring the teams who own these two test suites to chase the owner of the Loyalty microservice for a fix. Although this model helped in Emily's case, as we can see, it still has its

Trong ví dụ cụ thể này, một thay đổi đối với Web Shop mà vượt qua giai đoạn service test sẽ kích hoạt các end-to-end test liên quan, với bộ kiểm thử được sở hữu bởi chính nhóm sở hữu Web Shop. Tương tự, bất kỳ thay đổi nào chỉ đối với Helpdesk sẽ kích hoạt các end-to-end test liên quan. Nhưng các thay đổi đối với Customer hoặc Loyalty đều kích hoạt cả hai bộ kiểm thử. Điều này có thể đưa ta tới một tình huống mà một thay đổi thực hiện trên microservice Loyalty có thể làm hỏng cả hai bộ end-to-end test, có khả năng buộc các nhóm sở hữu hai bộ kiểm thử này phải đi đòi chủ sở hữu microservice Loyalty sửa lỗi. Mặc

dù mô hình này có ích trong trường hợp của Emily, như ta thấy, nó vẫn có những

7 Emily Bache, “End-to-End Automated Testing in a Microservices Architecture—Emily Bache,” NDC Conferences, July 5, 2017, YouTube video, 56:48, NDC Oslo 2017, <https://oreil.ly/QX3EK>.

7 Emily Bache, “End-to-End Automated Testing in a Microservices Architecture—Emily Bache,” NDC Conferences, July 5, 2017, YouTube video, 56:48, NDC Oslo 2017, <https://oreil.ly/QX3EK>.

challenges. Fundamentally, it's problematic to have a team own responsibility for tests where people from a different team can cause these tests to break.

thách thức của nó. Về cơ bản, sẽ có vấn đề khi để một nhóm gánh trách nhiệm cho những bài kiểm thử mà người từ một nhóm khác có thể làm hỏng.

Sometimes organizations react by having a dedicated team write these tests. This can be disastrous. The team developing the software becomes increasingly distant from the tests for its code. Cycle times increase, as service owners end up waiting for the test team to write end-to-end tests for the functionality it just wrote. Because another team writes these tests, the team that wrote the service is less involved with, and therefore less likely to know, how to run and fix these tests. Although it is unfortunately still a common organizational pattern, I see significant harm done whenever a team is distanced from writing tests for the code it wrote in the first place.

Đôi khi các tổ chức phản ứng bằng cách lập một nhóm chuyên trách viết những bài kiểm thử này. Điều này có thể là thảm họa. Nhóm phát triển phần mềm trở nên ngày càng xa cách với các bài kiểm thử cho mã của chính họ. Thời gian chu kỳ (cycle time) tăng lên, khi các chủ sở hữu dịch vụ rốt cuộc phải chờ nhóm kiểm thử viết end-to-end test cho chức năng mà họ vừa mới viết. Vì một nhóm khác viết những bài kiểm thử này, nhóm viết ra dịch vụ ít tham gia hơn, và do đó ít có khả năng biết, cách chạy và sửa những bài kiểm thử này. Mặc dù đây vẫn còn là một mẫu tổ chức phổ biến đáng tiếc, tôi thấy có sự tổn hại đáng kể xảy ra bất cứ khi nào một nhóm bị tách rời khỏi việc viết kiểm thử cho mã mà chính họ đã viết ngay từ đầu.

Getting this aspect right is really hard. We don't want to duplicate effort, nor do we want to completely centralize this to the extent that the teams building services are too far removed from things. If you can find a clean way to assign end-to-end tests to a specific team, then do it. If not, and if you can't find a way to remove end-to-end tests and replace them with something else, then you'll likely need to treat the end-to-end test suite as a shared codebase, but with joint ownership. Teams are free to check in to this suite, but ownership of the health of the suite has to be shared between the teams developing the services themselves. If you want to make extensive use of end-to-end tests with multiple teams, I think this approach is essential, and yet I have seen it done only rarely, and never without issues. Ultimately, I am convinced that at a certain level of organizational scale, you need to move away from cross-team end-to-end tests for this reason.

Làm cho khía cạnh này đúng đắn thực sự khó. Ta không muốn lặp lại công sức, cũng không muốn tập trung hóa hoàn toàn tới mức các nhóm xây dựng dịch vụ bị tách quá xa khỏi mọi thứ. Nếu bạn có thể tìm ra một cách rõ ràng để giao end-to-end test cho một nhóm cụ thể, thì hãy làm vậy. Nếu không, và nếu bạn không thể tìm ra cách loại bỏ end-to-end test và thay thế chúng bằng thứ khác, thì bạn có lẽ sẽ cần coi bộ end-to-end test như một codebase dùng chung, nhưng với quyền sở hữu chung (joint ownership). Các nhóm được tự do check-in vào bộ kiểm thử này, nhưng quyền sở hữu đối với sức khỏe của bộ kiểm thử phải được chia sẻ giữa các nhóm tự mình phát triển các dịch vụ. Nếu bạn muốn sử dụng end-to-end test một cách rộng rãi với nhiều nhóm, tôi cho rằng cách tiếp cận này là thiết yếu, vậy mà tôi chỉ hiếm khi thấy nó được thực hiện, và chưa bao giờ thấy nó được

làm mà không gặp vấn đề. Rốt cuộc, tôi tin chắc rằng ở một mức độ quy mô tổ chức nhất định, bạn cần rời bỏ end-to-end test liên-nhóm vì lý do này.

How Long Should End-to-End Tests Run? — End-to-End Test nên chạy trong bao lâu?

These end-to-end tests can take a while. I have seen them take up to a day to run, if not longer, and on one project I worked on, a full regression suite took six weeks! I rarely see teams actually curate their end-to-end test suites to reduce overlap in test coverage, or spend enough time in making them fast.

Những end-to-end test này có thể mất một lúc. Tôi đã thấy chúng mất tới cả ngày để chạy, nếu không nói là lâu hơn, và trên một dự án tôi từng làm, một bộ kiểm thử hồi quy đầy đủ mất sáu tuần! Tôi hiếm khi thấy các nhóm thực sự chăm chút bộ end-to-end test của họ để giảm sự chồng lấn về độ phủ kiểm thử, hay dành đủ thời gian để làm chúng nhanh hơn.

This slowness, combined with the fact that these tests can often be flaky, can be a major problem. A test suite that takes all day and often has breakages that have nothing to do with broken functionality is a disaster. Even if your functionality is broken, it could take you many hours to find out—at which point you likely would have already moved on to other activities, and the context switch in shifting your brain back to fix the issue would be painful.

Sự chậm chạp này, kết hợp với việc những bài kiểm thử này thường có thể chập chờn, có thể là một vấn đề lớn. Một bộ kiểm thử mất cả ngày và thường gặp sự cố chẳng liên quan gì đến chức năng bị hỏng là một thảm họa. Ngay cả khi chức năng của bạn bị hỏng, bạn có thể mất nhiều giờ để phát hiện ra—đến lúc đó bạn có lẽ đã chuyển sang các hoạt động khác, và việc chuyển ngữ cảnh để đưa não bộ quay lại sửa vấn đề sẽ rất khổ sở.

We can ameliorate some of this by running tests in parallel—for example, making use of tools like Selenium Grid. However, this approach is not a substitute for actually understanding what needs to be tested and actively removing tests that are no longer needed.

Ta có thể giảm bớt phần nào điều này bằng cách chạy kiểm thử song song—ví dụ, tận dụng các công cụ như Selenium Grid. Tuy nhiên, cách tiếp cận này không thể thay thế cho việc thực sự hiểu cái gì cần được kiểm thử và chủ động loại bỏ những bài kiểm thử không còn cần thiết.

Removing tests is sometimes a fraught exercise, and I suspect those who attempt it have much in common with people who want to remove certain airport **security** measures. No matter how ineffective the security measures might be, any conversation about removing them is often countered with knee-jerk reactions about not caring about people's safety or wanting terrorists to win. It is hard to have a balanced conversation about the value something adds versus the burden it entails. It can also be a difficult risk/reward trade-off. Do you get thanked if you remove a test? Maybe. But you'll certainly get blamed if a test you removed lets a bug through. When it comes to the larger-scoped test suites, however, this is exactly what we need to be able to do. If the same feature is covered in 20 different tests, perhaps we can get rid of half of them, as those 20 tests take 10 minutes to run! What this requires is a better understanding of risk, which is something humans are famously bad at. As a result, this intelligent curation and management of larger-scoped, high-burden tests happens incredibly infrequently. Wishing people did this more isn't the same thing as making it happen.

Loại bỏ kiểm thử đôi khi là một việc đầy gai góc, và tôi ngờ rằng những ai cố làm điều đó có nhiều điểm chung với những người muốn dỡ bỏ một số biện pháp an ninh sân bay. Cho

dù các biện pháp an ninh có thể kém hiệu quả đến đâu, mọi cuộc trò chuyện về việc dỡ bỏ chúng thường vấp phải những phản ứng bộc phát về việc không quan tâm đến sự an toàn của con người hay muốn bọn khủng bố thắng. Thật khó để có một cuộc trò chuyện cân bằng về giá trị mà một thứ gì đó mang lại so với gánh nặng mà nó kéo theo. Nó cũng có thể là một đánh đổi rủi ro/lợi ích khó khăn. Bạn có được cảm ơn nếu loại bỏ một bài kiểm thử không? Có thể. Nhưng bạn chắc chắn sẽ bị đổ lỗi nếu một bài kiểm thử mà bạn loại bỏ để lọt một lỗi. Tuy nhiên, khi nói đến các bộ kiểm thử phạm vi rộng hơn, đây chính xác là điều ta cần phải làm được. Nếu cùng một tính năng được bao phủ trong 20 bài kiểm thử khác nhau, có lẽ ta có thể bỏ đi một nửa, vì 20 bài kiểm thử đó mất 10 phút để chạy! Điều này đòi hỏi sự hiểu biết tốt hơn về rủi ro, vốn là thứ mà con người nổi tiếng là kém. Kết quả là, sự chậm chạp và quản lý thông minh đối với các bài kiểm thử phạm vi rộng, gánh nặng cao này xảy ra cực kỳ hiếm hoi. Ước gì người ta làm điều này nhiều hơn không giống với việc khiến nó thực sự xảy ra.

The Great Pile-Up — Sự dồn ứ lớn (The Great Pile-Up)

The long feedback cycles associated with end-to-end tests aren't just a problem when it comes to developer productivity. With a long test suite, any breaks take a while to fix, which reduces the amount of time that the end-to-end tests can be expected to be passing. If we deploy only software that has passed through all our tests successfully (which we should!), it means fewer of our services get through to the point of being deployable into production.

Các chu kỳ phản hồi dài gắn với end-to-end test không chỉ là vấn đề khi nói đến năng suất của lập trình viên. Với một bộ kiểm thử dài, bất kỳ sự cố nào cũng mất một lúc để sửa, điều này làm giảm khoảng thời gian mà các end-to-end test có thể được kỳ vọng là đang pass. Nếu ta chỉ triển khai phần mềm đã vượt qua thành công tất cả các bài kiểm thử của mình (mà ta nên làm vậy!), điều đó nghĩa là ít dịch vụ hơn của ta đạt tới điểm có thể triển khai được vào production.

This can lead to a pile-up. While a broken integration test stage is being fixed, more changes from upstream teams can pile in. Aside from the fact that this can make fixing the build harder, it means the scope of changes to be deployed increases. The ideal way to handle this is to not let people check in if the end-to-end tests are failing, but given a long test suite time, that is often impractical. Try saying, “You 30 developers: no check-ins until we fix this seven-hour-long build!” Allowing check-ins on a broken end-to-end test suite, though, is really fixing the wrong problem. If you allow check-ins on a broken build, the build may stay broken for longer, undermining its effectiveness as a way to give you fast feedback about the quality of the code. The right answer is to make the test suite faster.

Điều này có thể dẫn đến sự dồn ứ. Trong khi một giai đoạn kiểm thử tích hợp bị hỏng đang được sửa, nhiều thay đổi hơn từ các nhóm thượng nguồn có thể dồn vào. Ngoài việc điều này có thể làm cho việc sửa bản build khó hơn, nó còn nghĩa là phạm vi các thay đổi cần triển khai tăng lên. Cách lý tưởng để xử lý điều này là không cho phép người ta check-in nếu các end-to-end test đang fail, nhưng với một bộ kiểm thử mất nhiều thời gian, điều đó thường là bất khả thi. Thử mà nói: “Này 30 lập trình viên: không ai được check-in cho đến khi chúng ta sửa xong bản build dài bảy tiếng này!” Tuy nhiên, cho phép check-in trên một bộ end-to-end test đang hỏng thực ra là đang sửa nhầm vấn đề. Nếu bạn cho phép check-in trên một bản build hỏng, bản build có thể giữ trạng thái hỏng lâu hơn, làm xói mòn tính hiệu quả của nó như một cách cho bạn phản hồi nhanh về chất lượng của mã. Câu trả lời đúng là làm cho bộ kiểm thử nhanh hơn.

The larger the scope of a **deployment** and the higher the risk of a release, the more likely we are

to break something. So we want to make sure we can release small, well-tested changes frequently. When end-to-end tests slow down our ability to release small changes, they can end up doing more harm than good.

Phạm vi của một lần triển khai càng lớn và rủi ro của một lần phát hành càng cao, ta càng có khả năng làm hỏng thứ gì đó. Vậy nên ta muốn đảm bảo rằng mình có thể phát hành các thay đổi nhỏ, được kiểm thử kỹ một cách thường xuyên. Khi end-to-end test làm chậm khả năng phát hành các thay đổi nhỏ của ta, chúng có thể rất cuộc gây hại nhiều hơn lợi.

The Metaversion — Siêu phiên bản (The Metaversion)

With the end-to-end tests step, it is easy to start thinking, I know all these services at these versions work together, so why not deploy them all together? This very quickly becomes a conversation along the lines of, So why not use a version number for the whole system? To quote Brandon Byars, “Now you have 2.1.0 problems.”

Với bước end-to-end test, ta dễ bắt đầu nghĩ rằng, mình biết tất cả những dịch vụ này ở những phiên bản này hoạt động cùng nhau, vậy sao không triển khai chúng tất cả cùng nhau? Điều này rất nhanh chóng trở thành một cuộc trò chuyện kiểu, Vậy sao không dùng một số phiên bản cho toàn hệ thống? Trích lời Brandon Byars, “Giờ thì bạn có 2.1.0 vấn đề.”

By versioning together changes made to multiple services, we effectively embrace the idea that changing and deploying multiple services at once is acceptable. It becomes the norm; it becomes OK. In doing so, we cede one of the main advantages of a microservice architecture: the ability to deploy one service by itself, independently of other services.

Bằng cách gán phiên bản chung cho các thay đổi thực hiện trên nhiều dịch vụ, ta thực chất chấp nhận ý tưởng rằng việc thay đổi và triển khai nhiều dịch vụ cùng một lúc là chuyện chấp nhận được. Nó trở thành thông lệ; nó trở nên bình thường. Khi làm vậy, ta nhượng bộ một trong những lợi thế chính của kiến trúc microservice: khả năng triển khai một dịch vụ tự thân nó, độc lập với các dịch vụ khác.

All too often, the approach of accepting multiple services being deployed together drifts into a situation in which services become coupled. Before long, nicely separate services become increasingly tangled with others, and you never notice, because you never try to deploy them by themselves. You end up with a tangled mess where you have to orchestrate the deployment of multiple services at once, and as we discussed previously, this sort of **coupling** can leave us in a worse place than we would be with a single monolithic application.

Quá thường xuyên, cách tiếp cận chấp nhận nhiều dịch vụ được triển khai cùng nhau trôi dạt vào một tình huống mà các dịch vụ trở nên dính chặt (coupled) với nhau. Chẳng mấy chốc, những dịch vụ vốn được tách bạch đẹp đẽ trở nên ngày càng rối rắm với những dịch vụ khác, và bạn không bao giờ nhận ra, vì bạn không bao giờ thử triển khai chúng tự thân. Bạn rất cuộc có một mớ bòng bong nơi bạn phải điều phối việc triển khai nhiều dịch vụ cùng một lúc, và như ta đã bàn trước đó, kiểu coupling này có thể đẩy ta vào một chỗ tồi tệ hơn cả khi ta dùng một ứng dụng monolith đơn lẻ.

This is bad.

Điều này thật tệ.

Lack of Independent Testability — Thiếu khả năng kiểm thử độc lập (Lack of Independent Testability)

We've come back frequently to the topic of **independent deployability** being an important property to facilitate teams working in a more autonomous way, allowing for software to be shipped more effectively. If your teams work independently, it follows that they should be able to test independently. As we've seen, end-to-end tests can reduce the autonomy of teams and can force increased levels of coordination, with the associated challenges that can bring.

Ta đã thường xuyên quay lại chủ đề khả năng triển khai độc lập như một thuộc tính quan trọng giúp các nhóm làm việc theo cách tự chủ hơn, cho phép phần mềm được giao một cách hiệu quả hơn. Nếu các nhóm của bạn làm việc độc lập, thì hệ quả là họ nên có khả năng kiểm thử độc lập. Như ta đã thấy, end-to-end test có thể làm giảm tính tự chủ của các nhóm và có thể ép buộc mức độ phối hợp tăng lên, với những thách thức đi kèm mà điều đó có thể mang lại.

The drive toward independent testability extends to our use of the infrastructure associated with testing. Often, I see people having to make use of shared testing environments in which tests from multiple teams are executed. Such an environment is often highly constrained, and any issue can cause significant problems. Ideally, if you want your teams to be able to develop and test in an independent fashion, they should have their own test environments too.

Động lực hướng tới khả năng kiểm thử độc lập mở rộng đến cả việc ta sử dụng hạ tầng gắn với kiểm thử. Tôi thường thấy người ta phải dùng những môi trường kiểm thử dùng chung, nơi kiểm thử từ nhiều nhóm được thực thi. Một môi trường như vậy thường bị ràng buộc cao, và bất kỳ vấn đề nào cũng có thể gây ra những trục trặc đáng kể. Lý tưởng nhất, nếu bạn muốn các nhóm của mình có thể phát triển và kiểm thử một cách độc lập, họ cũng nên có môi trường kiểm thử riêng của mình.

The research summarized in Accelerate found that high-performing teams were more likely to “do most of their testing on demand, without requiring an integrated test environment.” 8

Nghiên cứu được tóm lược trong cuốn Accelerate phát hiện rằng các nhóm hiệu suất cao có nhiều khả năng “thực hiện hầu hết kiểm thử của họ theo yêu cầu (on demand), mà không cần một môi trường kiểm thử tích hợp.”⁸

Should You Avoid End-to-End Tests? — Bạn có nên tránh End-to-End Test không?

Despite the disadvantages just outlined, for many users end-to-end tests can still be manageable with a small number of microservices, and in these situations they still make a lot of sense. But what happens with 3, 4, 10, or 20 services? Very quickly these test suites become hugely bloated, and in the worst case they can result in a Cartesian-like explosion in the scenarios under test.

Bất chấp những bất lợi vừa nêu, đối với nhiều người dùng, end-to-end test vẫn có thể quản lý được với một số lượng nhỏ microservice, và trong những tình huống đó chúng vẫn rất hợp lý. Nhưng điều gì xảy ra với 3, 4, 10, hay 20 dịch vụ? Rất nhanh chóng những bộ kiểm thử này trở nên phình to khủng khiếp, và trong trường hợp tệ nhất chúng có thể dẫn đến một sự bùng nổ kiểu tích Descartes (Cartesian-like explosion) trong các kịch bản đang được kiểm thử.

In fact, even with a small number of microservices, these tests become difficult when you have multiple teams sharing end-to-end tests. With a shared end-to-end test suite, you undermine your goal of independent deployability. Your ability as a team to deploy a microservice now requires that a test suite shared by multiple teams passes.

Thực ra, ngay cả với một số lượng nhỏ microservice, những bài kiểm thử này trở nên khó khăn khi bạn có nhiều nhóm chia sẻ end-to-end test. Với một bộ end-to-end test dùng chung, bạn làm xói mòn mục tiêu khả năng triển khai độc lập của mình. Khả năng triển khai một microservice của bạn với tư cách một nhóm giờ đây đòi hỏi một bộ kiểm thử được chia sẻ bởi nhiều nhóm phải pass.

What is one of the key problems we are trying to address when we use the end-to-end tests outlined previously? We are trying to ensure that when we deploy a new service to production, our changes won't break consumers. Now, as we covered in detail in “Structural Versus Semantic Contract Breakages” on page 142, having explicit schemas for our microservice interfaces can help us catch structural breakages, and that can definitely reduce the need for more complex end-to-end tests.

Một trong những vấn đề mấu chốt mà ta đang cố giải quyết khi dùng end-to-end test như đã phác họa trước đó là gì? Ta đang cố đảm bảo rằng khi ta triển khai một dịch vụ mới lên production, các thay đổi của ta sẽ không làm hỏng những bên tiêu thụ (consumers). Giờ thì, như ta đã trình bày chi tiết trong mục “Phá vỡ hợp đồng về Cấu trúc so với về Ngữ nghĩa” ở trang 142, việc có các schema tường minh cho các interface microservice của ta có thể giúp ta bắt được những vi phạm về cấu trúc, và điều đó chắc chắn có thể giảm nhu cầu về các end-to-end test phức tạp hơn.

However, schemas can't pick up semantic breakages, namely changes in behavior that cause breakages due to backward incompatibility. End-to-end tests absolutely can help catch these semantic breakages, but they do so at a great cost. Ideally, we'd want to have some type of test that can pick up semantic breaking changes and run in a reduced scope, improving test isolation (and therefore speed of feedback). This is where contract tests and consumer-driven contracts come in.

Tuy nhiên, schema không thể bắt được những vi phạm về ngữ nghĩa, tức là những thay đổi trong hành vi gây ra sự cố do không tương thích ngược (backward incompatibility). End-to-end test hoàn toàn có thể giúp bắt được những vi phạm ngữ nghĩa này, nhưng chúng làm vậy với một cái giá rất lớn. Lý tưởng nhất, ta sẽ muốn có một loại kiểm thử nào đó có thể bắt được các thay đổi phá vỡ về ngữ nghĩa và chạy trong một phạm vi thu hẹp, cải thiện sự cô lập của kiểm thử (và do đó tốc độ phản hồi). Đây là lúc contract test và consumer-driven contracts xuất hiện.

Contract Tests and Consumer-Driven Contracts (CDCs) — Contract Test và Consumer-Driven Contracts (CDCs)

With contract tests, a team whose microservice consumes an external service writes tests that describe how it expects an external service will behave. This is less about testing your own microservice and more about specifying how you expect an external service to behave. One of the main reasons that these contract tests can be useful is that they can be run against any stubs or mocks you are using that represent external

Với contract test, một nhóm có microservice tiêu thụ một dịch vụ bên ngoài sẽ viết các bài kiểm thử mô tả cách họ kỳ vọng một dịch vụ bên ngoài sẽ hành xử. Điều này ít liên quan đến việc kiểm thử chính microservice của bạn mà thiên về việc đặc tả cách bạn kỳ vọng một dịch vụ bên ngoài sẽ hành xử. Một trong những lý do chính khiến những contract test

này có thể hữu ích là chúng có thể được chạy trên bất kỳ stub hay mock nào bạn đang dùng để đại diện cho các dịch vụ

8 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and **Scaling** High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

8 Nicole Forsgren, Jez Humble, và Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

services—your contract tests should pass when you run your own stubs, just as they should the real external service.

bên ngoài—các contract test của bạn nên pass khi bạn chạy stub của chính mình, cũng như chúng nên pass với dịch vụ bên ngoài thật.

Contract tests become very useful when used as part of consumer-driven contracts (CDCs). The contract tests are in effect an explicit, programmatic representation of how the consumer (upstream) microservice expects the producer (downstream) microservice to behave. With CDCs, the consumer team ensures that these contract tests are shared with the producer team to allow the producer team to ensure that its microservice meets these expectations. Typically, this is done by having the downstream producer team run the consumer contracts for each consuming microservice as part of its test suite that would be run on every build. Very importantly from a test feedback point of view, these tests need to be run only against a single producer in isolation, so they can be faster and more reliable than the end-to-end tests they might replace.

Contract test trở nên rất hữu ích khi được dùng như một phần của consumer-driven contracts (CDCs). Các contract test thực chất là một biểu diễn tường minh, có tính lập trình về cách mà microservice tiêu thụ (thượng nguồn) kỳ vọng microservice sản xuất (hạ nguồn) sẽ hành xử. Với CDCs, nhóm tiêu thụ đảm bảo rằng những contract test này được chia sẻ với nhóm sản xuất để cho phép nhóm sản xuất đảm bảo rằng microservice của họ đáp ứng những kỳ vọng này. Thông thường, điều này được thực hiện bằng cách để nhóm sản xuất hạ nguồn chạy các consumer contract cho mỗi microservice tiêu thụ như một phần của bộ kiểm thử của họ, vốn sẽ được chạy trên mỗi bản build. Rất quan trọng từ góc độ phản hồi kiểm thử, những bài kiểm thử này chỉ cần được chạy trên một nhà sản xuất duy nhất trong sự cô lập, nên chúng có thể nhanh hơn và đáng tin cậy hơn so với các end-to-end test mà chúng có thể thay thế.

As an example, let's revisit our scenario from earlier. The Customer microservice has two separate consumers: the helpdesk and the web shop. Both of these consuming applications have expectations for how the Customer microservice will behave. In this example, you create a set of tests for each consumer: one representing the helpdesk's expectations of the Customer microservice, and another set representing the expectations that the web shop has.

Lấy một ví dụ, hãy quay lại kịch bản từ trước đó của ta. Microservice Customer có hai bên tiêu thụ riêng biệt: helpdesk và cửa hàng web. Cả hai ứng dụng tiêu thụ này đều có những kỳ vọng về cách microservice Customer sẽ hành xử. Trong ví dụ này, bạn tạo một bộ kiểm thử cho mỗi bên tiêu thụ: một bộ đại diện cho kỳ vọng của helpdesk đối với microservice Customer, và một bộ khác đại diện cho những kỳ vọng mà cửa hàng web có.

Because these CDCs are expectations for how the Customer microservice should behave, we need only to run the Customer microservice itself, meaning we have the same effective test scope as our service tests. They'd have similar performance characteristics and would require us to run only the Customer microservice itself, with any external dependencies stubbed out.

Vì những CDC này là kỳ vọng về cách microservice Customer nên hành xử, ta chỉ cần chạy

chính microservice Customer, nghĩa là ta có cùng phạm vi kiểm thử hiệu dụng như các service test của mình. Chúng sẽ có những đặc tính hiệu năng tương tự và sẽ chỉ đòi hỏi ta chạy chính microservice Customer, với mọi phụ thuộc bên ngoài được stub out.

A good practice here is to have someone from the producer and consumer teams collaborate on creating the tests, so perhaps people from the web shop and helpdesk teams pair with people from the customer service team. Arguably, consumer-driven contracts are as much about fostering clear lines of communication and collaboration, where needed, between microservices and the teams that consume them. It could be argued, in fact, that implementing CDCs is just making more explicit the communication between the teams that must already exist. In cross-team collaboration, CDCs are an explicit reminder of Conway's law.

Một thực hành tốt ở đây là để một người từ nhóm sản xuất và một người từ nhóm tiêu thụ cùng cộng tác tạo ra các bài kiểm thử, nên có lẽ những người từ nhóm cửa hàng web và helpdesk sẽ kết đôi (pair) với những người từ nhóm dịch vụ Customer. Có thể lập luận rằng consumer-driven contracts cũng nhiều phần là về việc nuôi dưỡng những đường liên lạc và cộng tác rõ ràng, ở những nơi cần thiết, giữa các microservice và các nhóm tiêu thụ chúng. Thực ra, có thể lập luận rằng việc triển khai CDCs chỉ là làm cho sự liên lạc giữa các nhóm vốn đã phải tồn tại trở nên tường minh hơn. Trong cộng tác liên-nhóm, CDCs là một lời nhắc tường minh về Định luật Conway.

CDCs sit at the same level in the test pyramid as service tests, albeit with a very different focus, as shown in Figure 9-10 . These tests are focused on how a consumer will use the service, and the trigger if they break is very different when compared with service tests. If one of these CDCs breaks during a build of the Customer service, it becomes obvious which consumer would be impacted. At this point, you can either fix the problem or else start the discussion about introducing a breaking change in the manner we discussed in “Handling Change Between Microservices” on page 144 .

CDCs nằm ở cùng tầng trong kim tự tháp kiểm thử với service test, dù với một trọng tâm rất khác, như Hình 9-10 cho thấy. Những bài kiểm thử này tập trung vào cách một bên tiêu thụ sẽ sử dụng dịch vụ, và yếu tố kích hoạt khi chúng hỏng rất khác so với service test. Nếu một trong những CDC này hỏng trong một bản build của dịch vụ Customer, sẽ rõ ràng bên tiêu thụ nào sẽ bị ảnh hưởng. Tại điểm này, bạn có thể hoặc sửa vấn đề, hoặc bắt đầu cuộc thảo luận về việc đưa ra một thay đổi phá vỡ theo cách ta đã bàn trong mục “Xử lý Thay đổi Giữa các Microservice” ở trang 144.

So with CDCs, we can identify a breaking change prior to our software going into production without having to use a potentially expensive end-to-end test.

Vậy với CDCs, ta có thể nhận diện một thay đổi phá vỡ trước khi phần mềm của ta lên production mà không cần dùng đến một end-to-end test có khả năng tốn kém.

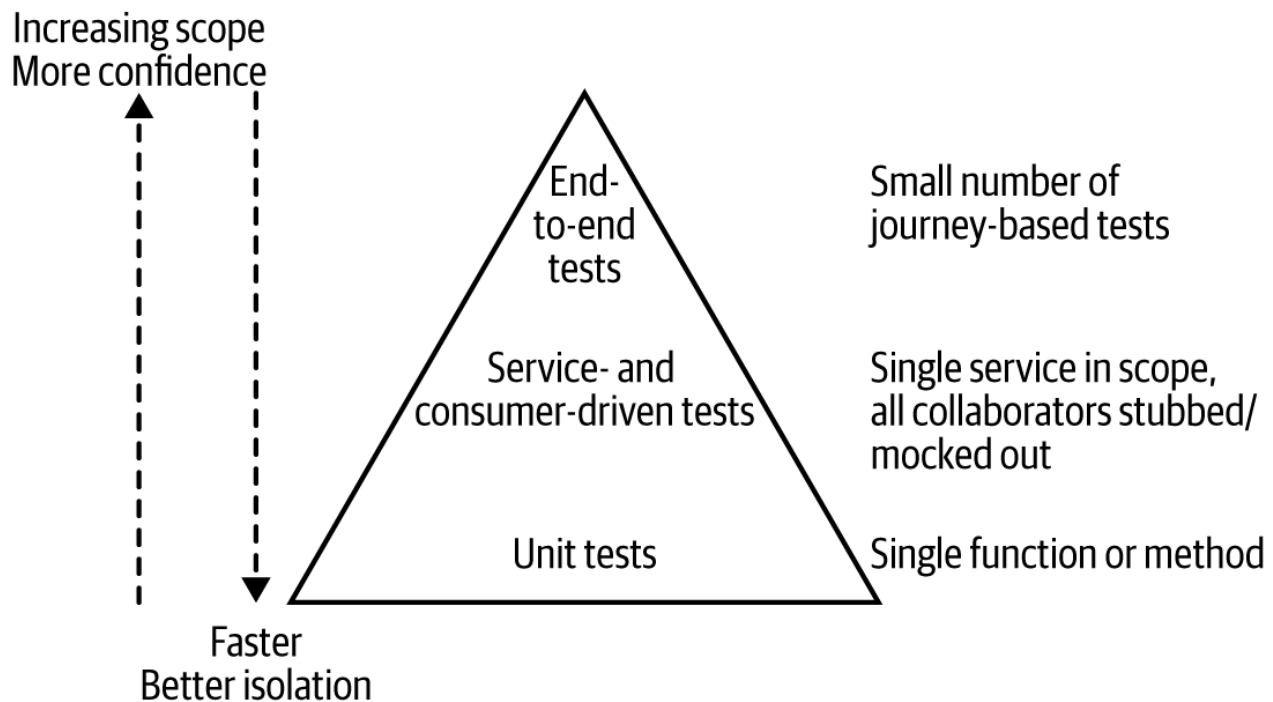


Figure 9-10. Integrating consumer-driven tests into the test pyramid

Hình 9-10. Tích hợp consumer-driven test vào kim tự tháp kiểm thử

Pact

Pact

Pact is a consumer-driven testing tool that was originally developed in-house at realestate.com.au but is now open source. Originally just for Ruby and focused only on HTTP protocols, Pact now supports multiple languages and platforms, such as the JVM, JavaScript, Python, and .NET, and can also be used with messaging interactions.

Pact là một công cụ kiểm thử hướng-người-tiêu-thụ (consumer-driven) ban đầu được phát triển nội bộ tại realestate.com.au nhưng giờ đã là mã nguồn mở. Ban đầu chỉ dành cho Ruby và chỉ tập trung vào các giao thức HTTP, Pact giờ hỗ trợ nhiều ngôn ngữ và nền tảng, chẳng hạn JVM, JavaScript, Python, và .NET, và cũng có thể được dùng với các tương tác nhắn tin.

With Pact, you start by defining the expectations of the producer using a DSL in one of the supported languages. Then you launch a local Pact server and run this expectation against it to create the Pact specification file. The Pact file is just a formal JSON specification; you could obviously handcode this, but using the language-specific SDK is much easier.

Với Pact, bạn bắt đầu bằng cách định nghĩa các kỳ vọng đối với nhà sản xuất bằng một DSL trong một trong các ngôn ngữ được hỗ trợ. Sau đó bạn khởi chạy một máy chủ Pact cục bộ và chạy kỳ vọng này trên nó để tạo ra tệp đặc tả Pact. Tệp Pact chỉ là một đặc tả JSON chính quy; bạn hiển nhiên có thể tự viết tay nó, nhưng dùng SDK riêng cho từng ngôn ngữ thì dễ hơn nhiều.

A really nice property of this model is that the locally running mock server used to generate the Pact file also works as a local stub for downstream microservices. By defining your expectations locally, you are defining how this local stub service should respond. This can replace the need for tools like

mountebank (or your own hand-rolled stubbing or mocking solutions).

Một thuộc tính thực sự hay ho của mô hình này là máy chủ mock chạy cục bộ được dùng để tạo tệp Pact cũng hoạt động như một stub cục bộ cho các microservice hạ nguồn. Bằng cách định nghĩa các kỳ vọng của bạn một cách cục bộ, bạn đang định nghĩa dịch vụ stub cục bộ này nên trả lời ra sao. Điều này có thể thay thế nhu cầu về các công cụ như mountebank (hay các giải pháp stubbing hoặc mocking tự viết tay của riêng bạn).

On the producer side, you then verify that this consumer specification is met by using the JSON Pact specification to drive calls against your microservice and verify responses. For this to work, the producer needs access to the Pact file. As we discussed earlier in “Mapping Source Code and Builds to Microservices” on page 205, we expect the consumer and producer to be in different builds. This means that we need some way for this JSON file, which will be generated to the consumer build, to be made available by the producer.

Về phía nhà sản xuất, sau đó bạn kiểm chứng rằng đặc tả của bên tiêu thụ này được đáp ứng bằng cách dùng đặc tả Pact JSON để điều khiển các lời gọi đến microservice của bạn và kiểm chứng các phản hồi. Để điều này hoạt động, nhà sản xuất cần truy cập được tệp Pact. Như ta đã bàn trước đó trong mục “Ánh xạ Mã nguồn và Bản build tới các Microservice” ở trang 205, ta kỳ vọng bên tiêu thụ và nhà sản xuất nằm trong các bản build khác nhau. Điều này nghĩa là ta cần một cách nào đó để tệp JSON này, vốn sẽ được tạo ra trong bản build của bên tiêu thụ, được nhà sản xuất truy cập được.

You could store the Pact file in your CI/CD tool’s artifact **repository**, or else you could use the Pact Broker, which allows you to store multiple versions of your Pact specifications. This could let you run your **consumer-driven contract** tests against multiple different versions of the consumers, if you wanted to test against, say, the version of the consumer in production and the version of the consumer that was most recently built.

Bạn có thể lưu tệp Pact trong artifact repository của công cụ CI/CD của bạn, hoặc bạn có thể dùng Pact Broker, thứ cho phép bạn lưu nhiều phiên bản của các đặc tả Pact. Điều này có thể cho phép bạn chạy các consumer-driven contract test trên nhiều phiên bản khác nhau của các bên tiêu thụ, nếu bạn muốn kiểm thử trên, ví dụ, phiên bản của bên tiêu thụ đang ở trên production và phiên bản của bên tiêu thụ được build gần đây nhất.

The Pact Broker actually has a host of useful capabilities. Aside from acting as a place where contracts can be stored, you can also find out when those contracts were validated. Also, because the Pact Broker knows about the relationship between the consumer and producer, it’s able to show you which microservices depend on which other microservices.

Pact Broker thực ra có một loạt năng lực hữu ích. Ngoài việc đóng vai một nơi để lưu các hợp đồng, bạn còn có thể tìm ra khi nào những hợp đồng đó được kiểm chứng. Ngoài ra, vì Pact Broker biết về mối quan hệ giữa bên tiêu thụ và nhà sản xuất, nó có thể cho bạn thấy microservice nào phụ thuộc vào microservice nào khác.

Other options

Các lựa chọn khác

Pact isn’t the only option for tooling around consumer-driven contracts. Spring Cloud Contract is one such example. However, it’s worth noting that—unlike Pact, which was designed from the beginning to support different technology stacks— Spring Cloud Contract is really only useful in a pure JVM ecosystem.

Pact không phải lựa chọn duy nhất cho công cụ xoay quanh consumer-driven contracts. Spring Cloud Contract là một ví dụ như vậy. Tuy nhiên, đáng lưu ý rằng—không như Pact,

vốn được thiết kế ngay từ đầu để hỗ trợ các ngăn xếp công nghệ khác nhau—Spring Cloud Contract thực sự chỉ hữu dụng trong một hệ sinh thái thuần JVM.

It's about conversations

Đây là chuyện về các cuộc trò chuyện

In Agile, stories are often referred to as a placeholder for a conversation. CDCs are just like that. They become the codification of a set of discussions about what a service **API** should look like, and when they break, they become a trigger point to have conversations about how that API should evolve.

Trong Agile, các story thường được nhắc đến như một chỗ giữ chỗ (placeholder) cho một cuộc trò chuyện. CDCs cũng giống hệt như vậy. Chúng trở thành sự mã hóa của một loạt thảo luận về việc một API của dịch vụ nên trông như thế nào, và khi chúng hỏng, chúng trở thành một điểm kích hoạt để có những cuộc trò chuyện về việc API đó nên tiến hóa ra sao.

It is important to understand that CDCs require good communication and trust between the consumer and producing service. If both parties are in the same team (or are the same person!), then this shouldn't be hard. However, if you are consuming a service provided with a third party, you may not have the frequency of communication, or trust, to make CDCs work. In these situations, you may have to make do with limited larger-scoped integration tests just around the untrusted component. Alternatively, if you are creating an API for thousands of potential consumers, such as with a publicly available web service API, you may have to play the role of the consumer yourself (or perhaps work with a subset of your consumers) in defining these tests. Breaking huge numbers of external consumers is a pretty bad idea, so if anything the importance of CDCs is increased!

Điều quan trọng là phải hiểu rằng CDCs đòi hỏi sự liên lạc tốt và sự tin tưởng giữa dịch vụ tiêu thụ và dịch vụ sản xuất. Nếu cả hai bên đều ở trong cùng một nhóm (hoặc là cùng một người!), thì điều này không khó. Tuy nhiên, nếu bạn đang tiêu thụ một dịch vụ do một bên thứ ba cung cấp, bạn có thể không có tần suất liên lạc, hay sự tin tưởng, để khiến CDCs hoạt động. Trong những tình huống này, bạn có thể phải tạm hài lòng với các integration test phạm vi rộng hơn hạn chế, chỉ xoay quanh thành phần không đáng tin cậy. Hoặc, nếu bạn đang tạo một API cho hàng nghìn bên tiêu thụ tiềm năng, chẳng hạn với một API dịch vụ web công khai, bạn có thể phải tự mình đóng vai bên tiêu thụ (hoặc có lẽ làm việc với một tập con các bên tiêu thụ của bạn) trong việc định nghĩa những bài kiểm thử này. Phá vỡ một số lượng khổng lồ các bên tiêu thụ bên ngoài là một ý tưởng khá tệ, nên nếu có gì thì tầm quan trọng của CDCs lại càng tăng lên!

The Final Word — Lời cuối (The Final Word)

As outlined in detail earlier in the chapter, end-to-end tests have a large number of disadvantages that grow significantly as you add more moving parts under test. From speaking to people who have been implementing microservices at scale for a while now, I have learned that most of them over time remove the need entirely for end-to-end tests in favor of other mechanisms to validate the quality of their software—for example, the use of explicit schemas and CDCs, in-production testing, or perhaps some of the progressive delivery techniques we've discussed, such as canary releases.

Như đã phác họa chi tiết trước đó trong chương, end-to-end test có một số lượng lớn các bất lợi tăng lên đáng kể khi bạn thêm nhiều bộ phận chuyển động hơn vào diện được kiểm thử. Qua trò chuyện với những người đã triển khai microservices ở quy mô lớn được một thời gian, tôi học được rằng hầu hết họ theo thời gian đều loại bỏ hoàn toàn nhu cầu về end-to-end test để chuyển sang các cơ chế khác nhằm kiểm chứng chất lượng phần mềm

của họ—ví dụ, việc sử dụng các schema tường minh và CDCs, kiểm thử trên production, hoặc có lẽ một số kỹ thuật giao hàng tiệm tiến (progressive delivery) mà ta đã bàn, chẳng hạn các bản phát hành canary.

You can view the running of end-to-end tests prior to production deployment as training wheels. While you are learning how CDCs work and improving your production **monitoring** and deployment techniques, these end-to-end tests may form a useful safety net, where you are trading off cycle time for decreased risk. But as you improve those other areas, and as the relative cost of creating the end-to-end tests increases, you can start to reduce your reliance on end-to-end tests to the point that they are no longer needed. Ditching end-to-end tests without fully understanding what you’ve lost is probably a bad idea.

Bạn có thể xem việc chạy end-to-end test trước khi triển khai lên production như một bộ bánh phụ tập đi (training wheels). Trong khi bạn đang học cách CDCs hoạt động và cải thiện các kỹ thuật giám sát và triển khai production của mình, những end-to-end test này có thể tạo thành một tấm lưới an toàn hữu ích, nơi bạn đánh đổi thời gian chu kỳ để giảm rủi ro. Nhưng khi bạn cải thiện những lĩnh vực khác đó, và khi chi phí tương đối của việc tạo ra end-to-end test tăng lên, bạn có thể bắt đầu giảm sự phụ thuộc của mình vào end-to-end test đến mức chúng không còn cần thiết nữa. Vứt bỏ end-to-end test mà không hiểu rõ hoàn toàn mình đã mất gì thì có lẽ là một ý tưởng tồi.

Obviously you’ll have a better understanding of your own organization’s risk profile than I, but I would challenge you to think long and hard about how much end-to-end testing you really need to do.

Hiển nhiên bạn sẽ hiểu rõ hồ sơ rủi ro của tổ chức của chính mình hơn tôi, nhưng tôi muốn thách thức bạn suy nghĩ thật lâu và thật kỹ về việc bạn thực sự cần làm bao nhiêu kiểm thử end-to-end.

Developer Experience — Trải nghiệm của lập trình viên (Developer Experience)

One of the significant challenges that can emerge as developers find the need to work on more and more microservices is that the developer experience can start to suffer, for the simple reason that they are trying to run more and more microservices locally. This often arises in situations in which a developer needs to run a large-scoped test that bridges multiple nonstubbed microservices.

Một trong những thách thức đáng kể có thể nảy sinh khi các lập trình viên thấy cần phải làm việc trên ngày càng nhiều microservice là trải nghiệm của lập trình viên có thể bắt đầu sa sút, vì một lý do đơn giản là họ đang cố chạy ngày càng nhiều microservice một cách cục bộ. Điều này thường nảy sinh trong những tình huống mà một lập trình viên cần chạy một bài kiểm thử phạm vi rộng bắc cầu qua nhiều microservice không được stub.

How quickly this becomes an issue will depend on a number of factors. How many microservices a developer needs to run locally, which technology stack those microservices are written in, and the power of the local machine can all play a part. Some technology stacks are more resource intensive in terms of their initial footprint—JVM-based microservices come to mind. On the other hand, some tech stacks can result in microservices with a faster and more lightweight resource footprint, perhaps allowing you to run many more microservices locally.

Việc này trở thành vấn đề nhanh đến mức nào sẽ phụ thuộc vào một số yếu tố. Lập trình viên cần chạy bao nhiêu microservice cục bộ, những microservice đó được viết bằng ngôn ngữ công nghệ nào, và sức mạnh của máy cục bộ đều có thể góp phần. Một số ngôn ngữ

công nghệ tiêu tốn tài nguyên hơn xét về dấu chân (footprint) ban đầu của chúng—các microservice dựa trên JVM là một ví dụ. Mặt khác, một số ngôn ngữ có thể cho ra các microservice với dấu chân tài nguyên nhanh hơn và nhẹ hơn, có lẽ cho phép bạn chạy nhiều microservice hơn một cách cục bộ.

One approach to dealing with this challenge is to instead have developers do their development and test work in a cloud environment. The idea is that you can have many more resources available to you to run the microservices you need. Aside from the fact that this model requires that you always have connectivity to your cloud resources, the other main issue is that your feedback cycles can suffer. If you need to make a code change locally and upload the new version of this code (or a locally built artifact) to the cloud, it can add significant delay to your development and test cycles, especially if you are operating in a part of the world with more constrained internet connectivity.

Một cách tiếp cận để giải quyết thách thức này là thay vào đó cho các lập trình viên làm công việc phát triển và kiểm thử của họ trong một môi trường đám mây. Ý tưởng là bạn có thể có nhiều tài nguyên hơn sẵn dùng để chạy các microservice mà bạn cần. Ngoài việc mô hình này đòi hỏi bạn luôn phải có kết nối tới các tài nguyên đám mây của mình, vấn đề chính khác là các chu kỳ phản hồi của bạn có thể bị ảnh hưởng. Nếu bạn cần thực hiện một thay đổi mã cục bộ và tải lên phiên bản mới của mã này (hoặc một artifact build cục bộ) lên đám mây, nó có thể thêm độ trễ đáng kể vào các chu kỳ phát triển và kiểm thử của bạn, đặc biệt nếu bạn đang hoạt động ở một vùng trên thế giới với kết nối internet bị hạn chế hơn.

Full in-the-cloud development is one possibility for addressing the issue of feedback cycles; cloud-based IDEs like Cloud9, now owned by AWS, have shown that this is possible. However, while something like this might be the future for development, it's certainly not the present for the vast majority of us.

Phát triển hoàn toàn trên đám mây là một khả năng để giải quyết vấn đề chu kỳ phản hồi; các IDE dựa trên đám mây như Cloud9, giờ thuộc sở hữu của AWS, đã cho thấy điều này là khả thi. Tuy nhiên, mặc dù một thứ như thế này có thể là tương lai của việc phát triển, nó chắc chắn không phải là hiện tại đối với tuyệt đại đa số chúng ta.

Fundamentally, I do think that the use of cloud environments to allow a developer to run more microservices for their development and test cycles is missing the point, resulting in more complexity than is needed, in addition to higher costs. Ideally, you want to aim for a developer needing to run only the microservices they actually work on. If a developer is part of a team that owns five microservices, then that developer needs to be able to run those microservices as effectively as possible, and for fast feedback my preference would always be for them to run locally.

Về cơ bản, tôi quả thực cho rằng việc sử dụng các môi trường đám mây để cho phép một lập trình viên chạy nhiều microservice hơn cho các chu kỳ phát triển và kiểm thử của họ là đang lạc đề, dẫn đến nhiều phức tạp hơn mức cần thiết, bên cạnh chi phí cao hơn. Lý tưởng nhất, bạn muốn nhắm tới việc một lập trình viên chỉ cần chạy những microservice mà họ thực sự làm việc trên đó. Nếu một lập trình viên là một phần của nhóm sở hữu năm microservice, thì lập trình viên đó cần có khả năng chạy những microservice đó một cách hiệu quả nhất có thể, và để có phản hồi nhanh, sở thích của tôi sẽ luôn là họ chạy chúng cục bộ.

But what if the five microservices your team owns want to call out to other systems and microservices that are owned by other teams? Without them, the local development and test environment won't function, will it? Here again, stubbing comes to the rescue. I should be able to stand up local stubs that mimic the microservices that are out of scope for my team. The only real microservices you should be running locally are the ones you are working on. If you are working in an organization

where you are expected to work on hundreds of different microservices, well then, you’ve got much bigger problems to deal with—this is a topic we’ll explore in more depth in “Strong Versus Collective Ownership” on page 499 .

Nhưng nếu năm microservice mà nhóm của bạn sở hữu lại muốn gọi ra các hệ thống và microservice khác do các nhóm khác sở hữu thì sao? Không có chúng, môi trường phát triển và kiểm thử cục bộ sẽ không hoạt động, đúng không? Ở đây một lần nữa, stubbing đến giải cứu. Tôi nên có thể dựng lên các stub cục bộ mô phỏng những microservice nằm ngoài phạm vi của nhóm tôi. Những microservice thật duy nhất mà bạn nên chạy cục bộ là những cái bạn đang làm việc trên đó. Nếu bạn đang làm việc trong một tổ chức nơi bạn được kỳ vọng làm việc trên hàng trăm microservice khác nhau, thì bạn đang có những vấn đề lớn hơn nhiều để xử lý—đây là một chủ đề ta sẽ khám phá sâu hơn trong mục “Sở hữu Mạnh so với Sở hữu Tập thể” ở trang 499.

From Preproduction to In-Production Testing — Từ kiểm thử tiền-production đến kiểm thử trên production

Historically, most of the focus of testing has been around testing our systems before we get to production. With our tests, we are defining a series of models with which we hope to prove whether our system works and behaves as we would like, both functionally and nonfunctionally. But if our models are not perfect, we will encounter problems when our systems are used in anger. Bugs slip into production, new failure modes are discovered, and our users use the system in ways we could never expect.

Về mặt lịch sử, hầu hết trọng tâm của kiểm thử đã xoay quanh việc kiểm thử các hệ thống của ta trước khi ta lên production. Với các bài kiểm thử của mình, ta đang định nghĩa một loạt các mô hình mà qua đó ta hy vọng chứng minh được liệu hệ thống của ta có hoạt động và hành xử như ta mong muốn hay không, cả về chức năng lẫn phi chức năng. Nhưng nếu các mô hình của ta không hoàn hảo, ta sẽ gặp vấn đề khi hệ thống của ta được dùng trong thực chiến. Lỗi lọt vào production, các chế độ thất bại (failure modes) mới được phát hiện, và người dùng của ta dùng hệ thống theo những cách ta không bao giờ có thể ngờ tới.

One reaction to this is often to define more and more tests, and refine our models, to catch more issues early and reduce the number of problems we encounter with our running production system. However, at a certain point we have to accept that we hit diminishing returns with this approach. With testing prior to deployment, we cannot reduce the chance of failure to zero.

Một phản ứng đối với điều này thường là định nghĩa ngày càng nhiều bài kiểm thử, và tinh chỉnh các mô hình của ta, để bắt được nhiều vấn đề sớm hơn và giảm số lượng vấn đề ta gặp phải với hệ thống production đang chạy. Tuy nhiên, đến một điểm nào đó ta phải chấp nhận rằng ta chạm tới lợi ích cận biên giảm dần với cách tiếp cận này. Với kiểm thử trước khi triển khai, ta không thể giảm khả năng thất bại xuống mức không.

The complexity of a distributed system is such that it can be infeasible to catch all potential problems that might occur before we hit production itself.

Sự phức tạp của một hệ thống phân tán đến mức việc bắt được tất cả các vấn đề tiềm tàng có thể xảy ra trước khi ta tới được chính production có thể là bất khả thi.

Generically speaking, the purpose of a test is to give us feedback as to whether or not our software is of sufficient quality. Ideally, we want that feedback as soon as possible, and we’d like to be able to spot if there is a problem with our software before an end user experiences that issue. This is why a lot of testing is done before we release our software.

Nói một cách tổng quát, mục đích của một bài kiểm thử là cho ta phản hồi về việc liệu phần mềm của ta có đạt chất lượng đủ tốt hay không. Lý tưởng nhất, ta muốn phản hồi đó càng sớm càng tốt, và ta muốn có khả năng phát hiện nếu có vấn đề với phần mềm của mình trước khi một người dùng cuối trải nghiệm vấn đề đó. Đây là lý do tại sao rất nhiều kiểm thử được thực hiện trước khi ta phát hành phần mềm của mình.

Limiting ourselves to testing only in a preproduction environment, though, is hamstringing us. We are reducing the places in which we can pick up issues, and we're also eliminating the possibility of testing the quality of our software in the most important location—where it is going to be used.

Tuy nhiên, tự giới hạn mình ở việc chỉ kiểm thử trong một môi trường tiền-production là đang trói tay ta. Ta đang thu hẹp những nơi mà ta có thể phát hiện vấn đề, và ta cũng đang loại bỏ khả năng kiểm thử chất lượng phần mềm của mình ở vị trí quan trọng nhất—nơi nó sẽ được sử dụng.

We can and should also look to apply testing in a production environment. This can be done in a safe manner, can provide higher-quality feedback than preproduction testing, and, as we see, is likely something you're already doing, whether you realize it or not.

Ta có thể và cũng nên tìm cách áp dụng kiểm thử trong một môi trường production. Điều này có thể được thực hiện một cách an toàn, có thể cung cấp phản hồi chất lượng cao hơn so với kiểm thử tiền-production, và, như ta sẽ thấy, có lẽ là thứ bạn đã đang làm rồi, dù bạn có nhận ra hay không.

Types of In-Production Testing — Các loại kiểm thử trên production

There is a long list of different tests that we can carry out in production, ranging from the simple to the complex. To start off with, let's think about something as simple as a ping check to make sure that a microservice is live. Simply checking to see if a microservice instance is running is a type of test—we just don't see it as that, as it is an activity that is typically handled by “operations” people. But fundamentally, something as simple as determining whether or not a microservice is up can be viewed as a test—one that we run frequently on our software.

Có một danh sách dài các bài kiểm thử khác nhau mà ta có thể thực hiện trên production, từ đơn giản đến phức tạp. Để bắt đầu, hãy nghĩ về một thứ đơn giản như một bài kiểm tra ping để chắc chắn rằng một microservice đang sống. Việc đơn giản kiểm tra xem một instance microservice có đang chạy hay không là một loại kiểm thử—chỉ là ta không xem nó như vậy, vì nó là một hoạt động thường được những người “vận hành” (operations) xử lý. Nhưng về cơ bản, một thứ đơn giản như việc xác định xem một microservice có đang hoạt động hay không có thể được xem là một bài kiểm thử—một bài ta chạy thường xuyên trên phần mềm của mình.

Smoke tests are another example of in-production testing. Typically done as part of the deployment activities, a smoke test ensures that the deployed software is operating correctly. These smoke tests will normally be done on the real, running software before it is released to the users (more on that shortly).

Smoke test là một ví dụ khác về kiểm thử trên production. Thường được thực hiện như một phần của các hoạt động triển khai, một smoke test đảm bảo rằng phần mềm đã triển khai đang vận hành đúng. Những smoke test này thường sẽ được thực hiện trên phần mềm thật, đang chạy trước khi nó được phát hành tới người dùng (sẽ nói thêm về điều này ngay sau đây).

Canary releases, which we covered in Chapter 8, are also a mechanism that is arguably about testing. We release a new version of our software to a small portion of users to “test” that it works correctly. If it does, we can roll out the software to a larger part of our user base, perhaps in a fully automated way.

Các bản phát hành canary, mà ta đã trình bày trong Chương 8, cũng là một cơ chế mà có thể lập luận là về kiểm thử. Ta phát hành một phiên bản mới của phần mềm tới một phần nhỏ người dùng để “kiểm thử” rằng nó hoạt động đúng. Nếu đúng, ta có thể triển khai phần mềm ra một phần lớn hơn của tệp người dùng, có lẽ theo một cách hoàn toàn tự động.

Another example of in-production testing is injecting fake user behavior into the system to make sure it works as expected—for example, placing an order for a fake customer, or registering a new (fake) user in the real production system. This type of test can sometimes get pushback, because people are worried about the impact it might have on the production system. So if you create tests like this, be sure to make them safe.

Một ví dụ khác về kiểm thử trên production là tiêm hành vi người dùng giả vào hệ thống để chắc chắn rằng nó hoạt động như mong đợi—ví dụ, đặt một đơn hàng cho một khách hàng giả, hoặc đăng ký một người dùng (giả) mới trong hệ thống production thật. Loại kiểm thử này đôi khi có thể vấp phải sự phản đối, vì người ta lo lắng về tác động mà nó có thể gây ra cho hệ thống production. Vậy nên nếu bạn tạo những bài kiểm thử như thế này, hãy chắc chắn làm cho chúng an toàn.

Making Testing in Production Safe — Làm cho kiểm thử trên production trở nên an toàn

If you decide to do testing in production (and you should!), it’s important that the tests not cause production issues, either by introducing system instability or by tainting production data. Something as simple as pinging a microservice instance to make sure it is live is likely to be a safe operation—if this causes system instability, you likely have pretty serious issues that need addressing, unless you’ve accidentally made your health check system an internal denial of service attack.

Nếu bạn quyết định thực hiện kiểm thử trên production (và bạn nên làm!), điều quan trọng là các bài kiểm thử không gây ra các vấn đề production, dù bằng cách đưa vào sự bất ổn định cho hệ thống hay bằng cách làm bẩn (tainting) dữ liệu production. Một thứ đơn giản như ping một instance microservice để chắc chắn nó đang sống thì có khả năng là một thao tác an toàn—nếu điều này gây ra sự bất ổn định cho hệ thống, bạn có khả năng đang gặp những vấn đề khá nghiêm trọng cần được giải quyết, trừ phi bạn vô tình biến hệ thống health check của mình thành một cuộc tấn công từ chối dịch vụ nội bộ.

Smoke tests are typically safe, as the operations they carry out are often done on software before it is released. As we explored in “Separating Deployment from Release” on page 270, separating the concept of deployment from release can be incredibly useful. When it comes to in-production testing, tests carried out on software that is deployed into production, before it is released, should be safe.

Smoke test thường an toàn, vì các thao tác chúng thực hiện thường được làm trên phần mềm trước khi nó được phát hành. Như ta đã khám phá trong mục “Tách Triển khai khỏi Phát hành” ở trang 270, việc tách khái niệm triển khai khỏi phát hành có thể cực kỳ hữu ích. Khi nói đến kiểm thử trên production, các bài kiểm thử được thực hiện trên phần mềm đã được triển khai vào production, trước khi nó được phát hành, nên là an toàn.

People tend to get most concerned about the safety of things like injecting fake user behavior into the system. We don't really want the order to be shipped, or the payment made. This is something that needs due care and attention, and despite the challenges, this type of testing can be hugely beneficial. We'll return to this in “**Semantic Monitoring**” on page 333 .

Người ta có xu hướng lo lắng nhất về sự an toàn của những thứ như tiêm hành vi người dùng giả vào hệ thống. Ta không thực sự muốn đơn hàng được giao đi, hay khoản thanh toán được thực hiện. Đây là thứ cần sự cẩn trọng và chú ý đúng mức, và bất chấp những thách thức, loại kiểm thử này có thể vô cùng có lợi. Ta sẽ quay lại điều này trong mục “Giám sát Ngữ nghĩa (Semantic Monitoring)” ở trang 333.

Mean Time to Repair over Mean Time Between Failures? — Thời gian trung bình để sửa chữa hơn là Thời gian trung bình giữa các lần thất bại?

So by looking at techniques like blue-green deployment or canary releasing, we find a way to test closer to (or even in) production, and we also build tools to help us manage a failure if it occurs. Using these approaches is a tacit acknowledgment that we cannot spot and catch all problems before we actually release our software.

Vậy nên bằng cách xem xét các kỹ thuật như triển khai blue-green hay phát hành canary, ta tìm ra một cách để kiểm thử gần hơn (hoặc thậm chí ngay trong) production, và ta cũng xây dựng các công cụ để giúp ta xử lý một sự cố nếu nó xảy ra. Sử dụng những cách tiếp cận này là một sự thừa nhận ngầm rằng ta không thể phát hiện và bắt được tất cả các vấn đề trước khi ta thực sự phát hành phần mềm của mình.

Sometimes expending the same effort on getting better at fixing problems when they occur can be significantly more beneficial than adding more automated functional tests. In the web operations world, this is often referred to as the trade-off between optimizing for mean time between failures (MTBF) and optimizing for mean time to repair (MTTR) .

Đôi khi bỏ ra cùng một công sức vào việc giỏi hơn trong sửa chữa các vấn đề khi chúng xảy ra có thể có lợi hơn đáng kể so với việc thêm nhiều bài kiểm thử chức năng tự động. Trong thế giới vận hành web, điều này thường được nhắc đến như sự đánh đổi giữa việc tối ưu cho thời gian trung bình giữa các lần thất bại (mean time between failures, MTBF) và việc tối ưu cho thời gian trung bình để sửa chữa (mean time to repair, MTTR).

Techniques to reduce the time to recovery can be as simple as very fast rollbacks coupled with good monitoring (which we'll discuss in Chapter 10). If we can spot a problem in production early and roll back early, we reduce the impact on our customers.

Các kỹ thuật để giảm thời gian phục hồi có thể đơn giản như các lần rollback rất nhanh kết hợp với giám sát tốt (mà ta sẽ bàn trong Chương 10). Nếu ta có thể phát hiện một vấn đề trên production sớm và rollback sớm, ta giảm tác động lên khách hàng của mình.

For different organizations, the trade-off between MTBF and MTTR will vary, and much of this lies with understanding the true impact of failure in a production environment. However, most organizations that I see spending time creating functional test suites often expend little to no effort on better monitoring or recovering from failure. So while they may reduce the number of defects that occur in the first place, they can't eliminate all of them, and are unprepared for dealing with them if they pop up in production.

Đối với các tổ chức khác nhau, sự đánh đổi giữa MTBF và MTTR sẽ khác nhau, và phần lớn điều này nằm ở việc hiểu được tác động thực sự của một sự cố trong một môi trường production. Tuy nhiên, hầu hết các tổ chức mà tôi thấy dành thời gian tạo các bộ kiểm thử chức năng lại thường bỏ ra ít đến không có công sức nào vào việc giám sát hay phục hồi khỏi sự cố tốt hơn. Vậy nên dù họ có thể giảm số lượng khiếm khuyết xảy ra ngay từ đầu, họ không thể loại bỏ hết tất cả chúng, và không hề chuẩn bị để xử lý chúng nếu chúng bật ra trên production.

Trade-offs other than those between MTBF and MTTR exist. For example, whether you are trying to work out if anyone will actually use your software, it may make much more sense to get something out now, to prove the idea or the business model before building robust software. In an environment where this is the case, testing may be overkill, as the impact of not knowing if your idea works is much higher than having a defect in production. In these situations, it can be quite sensible to avoid testing prior to production altogether.

Có những đánh đổi khác ngoài những đánh đổi giữa MTBF và MTTR. Ví dụ, nếu bạn đang cố tìm hiểu xem liệu có ai sẽ thực sự dùng phần mềm của bạn hay không, có thể hợp lý hơn nhiều khi đưa một thứ gì đó ra ngay bây giờ, để chứng minh ý tưởng hay mô hình kinh doanh trước khi xây dựng phần mềm vững chắc. Trong một môi trường mà điều này đúng, kiểm thử có thể là thừa thãi, vì tác động của việc không biết liệu ý tưởng của bạn có hiệu quả hay không cao hơn nhiều so với việc có một khiếm khuyết trên production. Trong những tình huống này, việc tránh kiểm thử trước khi lên production hoàn toàn có thể khá hợp lý.

Cross-Functional Testing — Kiểm thử liên-chức năng (Cross-Functional Testing)

The bulk of this chapter has been focused on testing specific pieces of functionality, and how this differs when you are testing a microservice-based system. However, there is another category of testing that is important to discuss. Nonfunctional requirements is an umbrella term used to describe those characteristics your system exhibits that cannot simply be implemented like a normal feature. They include aspects like the acceptable latency of a web page, the number of users a system should support, how accessible your user interface should be to people with disabilities, or how secure your customer data should be.

Phần lớn chương này đã tập trung vào việc kiểm thử các phần chức năng cụ thể, và điều này khác biệt thế nào khi bạn đang kiểm thử một hệ thống dựa trên microservice. Tuy nhiên, có một thể loại kiểm thử khác đáng để bàn. Yêu cầu phi chức năng (nonfunctional requirements) là một thuật ngữ bao trùm được dùng để mô tả những đặc tính mà hệ thống của bạn bộc lộ mà không thể đơn giản được triển khai như một tính năng thông thường. Chúng bao gồm các khía cạnh như độ trễ chấp nhận được của một trang web, số lượng người dùng mà một hệ thống nên hỗ trợ, giao diện người dùng của bạn nên dễ tiếp cận thế nào với người khuyết tật, hay dữ liệu khách hàng của bạn nên được bảo mật ra sao.

The term nonfunctional never sat well with me. Some of the things that get covered by this term seem very functional in nature! A previous colleague of mine, Sarah Taraporewalla, coined the phrase cross-functional requirements (CFR) instead, which I greatly prefer. It speaks more to the fact that these system behaviors really only emerge as the result of lots of cross-cutting work.

Thuật ngữ “phi chức năng” chưa bao giờ làm tôi thấy thoải mái. Một số thứ được bao trùm bởi thuật ngữ này trông rất mang tính chức năng về bản chất! Một đồng nghiệp trước đây

của tôi, Sarah Taraporewalla, đã đặt ra cụm từ yêu cầu liên-chức năng (cross-functional requirements, CFR) thay thế, và tôi thích nó hơn nhiều. Nó nói lên rõ hơn thực tế rằng những hành vi hệ thống này thực sự chỉ nổi lên như kết quả của rất nhiều công việc cắt ngang (cross-cutting).

Many if not most CFRs can really only be met in production. That said, we can define test strategies to help us see if we are at least moving toward meeting these goals. These sorts of tests fall into the Property Testing quadrant. A great example of this type of test is the performance test, which we'll discuss in more depth shortly.

Nhiều, nếu không nói là hầu hết, các CFR thực sự chỉ có thể được đáp ứng trên production. Dù vậy, ta có thể định nghĩa các chiến lược kiểm thử để giúp ta thấy liệu ta có ít nhất đang tiến tới việc đáp ứng những mục tiêu này hay không. Những loại kiểm thử này rơi vào góc phần tư Property Testing. Một ví dụ tuyệt vời về loại kiểm thử này là kiểm thử hiệu năng, mà ta sẽ bàn sâu hơn ngay sau đây.

You may want to track some CFRs at an individual microservice level. For example, you may decide that the durability of service you require from your payment service is significantly higher, but you are happy with more downtime for your music recommendation service, knowing that your core business can survive if you are unable to recommend artists similar to Metallica for 10 minutes or so. These trade-offs will end up having a large impact on how you design and evolve your system, and once again the fine-grained nature of a microservice-based system gives you many more chances to make these trade-offs. When looking at the CFRs a given microservice or team might have to take responsibility for, it's common for them to surface as part of a team's service-level objectives (SLOs), a topic we explore further in "Are We Doing OK?" on page 327 .

Bạn có thể muốn theo dõi một số CFR ở cấp độ từng microservice riêng lẻ. Ví dụ, bạn có thể quyết định rằng độ bền (durability) của dịch vụ mà bạn yêu cầu từ dịch vụ thanh toán của mình là cao hơn đáng kể, nhưng bạn lại hài lòng với nhiều downtime hơn cho dịch vụ gợi ý nhạc của mình, biết rằng nghiệp vụ cốt lõi của bạn vẫn có thể tồn tại nếu bạn không thể gợi ý các nghệ sĩ tương tự như Metallica trong khoảng 10 phút. Những đánh đổi này rất cuộc sẽ có tác động lớn đến cách bạn thiết kế và tiến hóa hệ thống của mình, và một lần nữa bản chất chia nhỏ của một hệ thống dựa trên microservice cho bạn nhiều cơ hội hơn để thực hiện những đánh đổi này. Khi xem xét các CFR mà một microservice hay một nhóm nhất định có thể phải gánh trách nhiệm, thường thấy chúng nổi lên như một phần của các mục tiêu mức dịch vụ (service-level objectives, SLOs) của nhóm, một chủ đề ta khám phá thêm trong mục "Liệu Chúng Ta Có Đang Ổn Không?" ở trang 327.

Tests around CFRs should follow the pyramid too. Some tests will have to be end-to-end, like load tests, but others won't. For example, once you've found a performance bottleneck in an end-to-end load test, write a smaller-scoped test to help you catch the problem in the future. Other CFRs fit faster tests quite easily. I remember working on a project for which we had insisted on ensuring that our HTML markup was using proper accessibility features to help people with disabilities use our website. Checking the generated markup to make sure that the appropriate controls were there could be done very quickly without the need for any networking round trips.

Các bài kiểm thử xoay quanh CFR cũng nên tuân theo kim tự tháp. Một số kiểm thử sẽ phải là end-to-end, như kiểm thử tải (load tests), nhưng những cái khác thì không. Ví dụ, một khi bạn đã tìm ra một điểm nghẽn hiệu năng trong một bài kiểm thử tải end-to-end, hãy viết một bài kiểm thử phạm vi nhỏ hơn để giúp bạn bắt được vấn đề trong tương lai. Các CFR khác khớp với những bài kiểm thử nhanh hơn khá dễ dàng. Tôi nhớ từng làm việc trên một dự án mà ở đó chúng tôi đã khẳng định đảm bảo rằng phần đánh dấu HTML (HTML markup) của mình đang dùng các tính năng trợ năng (accessibility) đúng cách để

giúp người khuyết tật sử dụng website của chúng tôi. Việc kiểm tra phần markup được tạo ra để chắc chắn rằng các điều khiển phù hợp có ở đó có thể được làm rất nhanh mà không cần bất kỳ vòng đi-về qua mạng nào.

All too often, considerations about CFRs come far too late. I strongly suggest looking at your CFRs as early as possible and reviewing them regularly.

Quá thường xuyên, những cân nhắc về CFR đến quá muộn. Tôi thực sự khuyên bạn xem xét các CFR của mình càng sớm càng tốt và rà soát chúng đều đặn.

Performance Tests — Kiểm thử hiệu năng (Performance Tests)

Performance tests are worth calling out explicitly as a way of ensuring that some of our cross-functional requirements can be met. When decomposing systems into smaller microservices, we increase the number of calls that will be made across network boundaries. Where previously an operation might have involved one database call, it may now involve three or four calls across network boundaries to other services, with a matching number of database calls. All of this can decrease the speed at which our systems operate. Tracking down sources of latency is especially important. When you have a call chain of multiple **synchronous** calls, if any part of the chain starts acting slowly, everything is affected, potentially leading to a significant impact. This makes having some way to performance test your applications even more important than it might be with a more monolithic system. Often the reason this sort of testing gets delayed is because initially there isn't enough of the system there to test. I understand this problem, but all too often it leads to kicking the can down the road, with performance testing often only being done just before going live for the first time, if at all! Don't fall into this trap.

Kiểm thử hiệu năng đáng được nêu rõ ra như một cách đảm bảo rằng một số yêu cầu liên-chức năng của ta có thể được đáp ứng. Khi phân rã các hệ thống thành những microservice nhỏ hơn, ta làm tăng số lượng lời gọi sẽ được thực hiện xuyên các ranh giới mạng. Nơi mà trước đây một thao tác có thể chỉ liên quan đến một lời gọi cơ sở dữ liệu, giờ đây nó có thể liên quan đến ba hay bốn lời gọi xuyên các ranh giới mạng đến các dịch vụ khác, với một số lượng lời gọi cơ sở dữ liệu tương ứng. Tất cả những điều này có thể làm giảm tốc độ mà hệ thống của ta vận hành. Việc truy lùng các nguồn gây độ trễ là đặc biệt quan trọng. Khi bạn có một chuỗi lời gọi gồm nhiều lời gọi đồng bộ, nếu bất kỳ phần nào của chuỗi bắt đầu hoạt động chậm, mọi thứ đều bị ảnh hưởng, có khả năng dẫn đến một tác động đáng kể. Điều này khiến việc có cách nào đó để kiểm thử hiệu năng các ứng dụng của bạn còn quan trọng hơn cả với một hệ thống mang tính monolith hơn. Thường thì lý do loại kiểm thử này bị trì hoãn là vì ban đầu chưa có đủ phần hệ thống ở đó để kiểm thử. Tôi hiểu vấn đề này, nhưng quá thường xuyên nó dẫn đến việc đá quả lon xuống đường (kicking the can down the road), với kiểm thử hiệu năng thường chỉ được làm ngay trước khi lên production lần đầu tiên, nếu có làm! Đừng rơi vào cái bẫy này.

As with functional tests, you may want a mix. You may decide that you want performance tests that isolate individual services, but start with tests that check core journeys in your system. You may be able to take end-to-end journey tests and simply run these at volume.

Cũng như với kiểm thử chức năng, bạn có thể muốn một sự pha trộn. Bạn có thể quyết định rằng mình muốn các kiểm thử hiệu năng cô lập từng dịch vụ riêng lẻ, nhưng hãy bắt đầu với các kiểm thử kiểm tra những hành trình cốt lõi trong hệ thống của bạn. Bạn có thể lấy các bài kiểm thử hành trình end-to-end và đơn giản chạy chúng ở mức tải cao.

To generate worthwhile results, you'll often need to run given scenarios with gradually increasing

numbers of simulated customers. This allows you to see how latency of calls varies with increasing load. This means that performance tests can take a while to run. In addition, you'll want the system to match production as closely as possible, to ensure that the results you see will be indicative of the performance you can expect on the production systems. This can mean that you'll need to acquire a more production-like volume of data and may need more machines to match the infrastructure—tasks that can be challenging. Even if you struggle to make the performance environment truly production-like, the tests may still have value in tracking down bottlenecks. Just be aware that you may get false negatives—or, even worse, false positives.

Để tạo ra những kết quả đáng giá, bạn sẽ thường cần chạy các kịch bản cho trước với số lượng khách hàng giả lập tăng dần. Điều này cho phép bạn thấy độ trễ của các lời gọi biến đổi ra sao khi tải tăng. Điều này nghĩa là các kiểm thử hiệu năng có thể mất một lúc để chạy. Ngoài ra, bạn sẽ muốn hệ thống khớp với production càng sát càng tốt, để đảm bảo rằng những kết quả bạn thấy sẽ phản ánh đúng hiệu năng mà bạn có thể kỳ vọng trên các hệ thống production. Điều này có thể nghĩa là bạn sẽ cần thu được một khối lượng dữ liệu giống production hơn và có thể cần nhiều máy hơn để khớp với hạ tầng—những tác vụ có thể đầy thử thách. Ngay cả khi bạn vật lộn để làm cho môi trường hiệu năng thực sự giống production, các bài kiểm thử vẫn có thể có giá trị trong việc truy lùng các điểm nghẽn. Chỉ cần lưu ý rằng bạn có thể nhận được kết quả âm tính giả (false negatives)—hoặc, thậm chí tệ hơn, dương tính giả (false positives).

Due to the time it takes to run performance tests, it isn't always feasible to run them on every check-in. It is a common practice to run a subset every day, and a larger set every week. Whatever approach you pick, make sure you run tests as regularly as you can. The longer you go without running performance tests, the harder it can be to track down the culprit. Performance problems are especially difficult to resolve, so if you can reduce the number of commits you need to look at to see a newly introduced problem, your life will be much easier.

Do thời gian cần để chạy các kiểm thử hiệu năng, không phải lúc nào cũng khả thi để chạy chúng trên mỗi lần check-in. Một thực hành phổ biến là chạy một tập con mỗi ngày, và một tập lớn hơn mỗi tuần. Dù bạn chọn cách tiếp cận nào, hãy chắc chắn rằng bạn chạy kiểm thử đều đặn nhất có thể. Càng đi lâu mà không chạy kiểm thử hiệu năng, càng khó truy lùng thủ phạm. Các vấn đề hiệu năng đặc biệt khó giải quyết, nên nếu bạn có thể giảm số lần commit mà bạn cần xem xét để thấy một vấn đề mới được đưa vào, cuộc sống của bạn sẽ dễ dàng hơn nhiều.

And make sure you also look at the results! I've been very surprised by the number of teams I have encountered who have put in a lot of work implementing tests and running them but never actually check the numbers. Often this is because people don't know what a "good" result looks like. You really need to have targets. When you provide a microservice to be used as part of a wider architecture, it's common to have specific expectations that you commit to deliver—the SLOs I mentioned earlier. If as part of this you commit to deliver a certain level of performance, then it makes sense for any automated test to give you feedback as to whether or not you're likely to meet (and hopefully exceed) that target.

Và hãy chắc chắn rằng bạn cũng xem các kết quả! Tôi đã rất ngạc nhiên bởi số lượng nhóm tôi từng gặp đã bỏ ra rất nhiều công sức triển khai các bài kiểm thử và chạy chúng nhưng không bao giờ thực sự kiểm tra các con số. Thường thì điều này là vì người ta không biết một kết quả "tốt" trông như thế nào. Bạn thực sự cần có các mục tiêu. Khi bạn cung cấp một microservice để được dùng như một phần của một kiến trúc rộng lớn hơn, thường có những kỳ vọng cụ thể mà bạn cam kết sẽ thực hiện—các SLO mà tôi đã nhắc đến trước đó. Nếu như một phần của điều này bạn cam kết thực hiện một mức hiệu năng nhất định, thì việc bất kỳ bài kiểm thử tự động nào cho bạn phản hồi về việc liệu bạn có khả năng đạt

(và hy vọng vượt) mục tiêu đó hay không là hợp lý.

In lieu of specific performance targets, automated performance tests can still be very useful in helping you see how the performance of your microservice varies as you make changes. It can be a safety net to catch you if you make a change that causes a drastic degradation in performance. So an alternative to a specific target might be to fail the test if the delta in performance from one build to the next varies too much.

Trong trường hợp thiếu các mục tiêu hiệu năng cụ thể, các kiểm thử hiệu năng tự động vẫn có thể rất hữu ích trong việc giúp bạn thấy hiệu năng của microservice của mình biến đổi ra sao khi bạn thực hiện các thay đổi. Nó có thể là một tấm lưới an toàn để bắt được bạn nếu bạn thực hiện một thay đổi gây ra một sự suy giảm hiệu năng nghiêm trọng. Vậy nên một lựa chọn thay thế cho một mục tiêu cụ thể có thể là làm fail bài kiểm thử nếu độ chênh (delta) về hiệu năng từ bản build này sang bản build kế tiếp biến đổi quá nhiều.

Performance testing needs to be done in concert with understanding the real system performance (which we'll discuss more in Chapter 10), and ideally you would use the same tools in your performance test environment for visualizing system behavior as those you use in production. This approach can make it much easier to compare like with like.

Kiểm thử hiệu năng cần được làm song hành với việc hiểu hiệu năng hệ thống thực (mà ta sẽ bàn nhiều hơn trong Chương 10), và lý tưởng nhất bạn sẽ dùng cùng những công cụ trong môi trường kiểm thử hiệu năng của mình để trực quan hóa hành vi hệ thống như những công cụ bạn dùng trên production. Cách tiếp cận này có thể làm cho việc so sánh tương đồng với tương đồng dễ hơn nhiều.

Robustness Tests — Kiểm thử độ vững chắc (Robustness Tests)

A microservice architecture is often only as reliable as its weakest link, and as a result it's common for our microservices to build in mechanisms to allow them to improve their **robustness** in order to improve system reliability. We'll explore this topic more in “Stability Patterns” on page 395 , but examples include running multiple instances of a microservice behind a load balancer to tolerate the failure of an instance, or using circuit breakers to programmatically handle situations in which downstream microservices cannot be contacted.

Một kiến trúc microservice thường chỉ đáng tin cậy như mắt xích yếu nhất của nó, và kết quả là việc các microservice của ta xây dựng sẵn các cơ chế cho phép chúng cải thiện độ vững chắc (robustness) nhằm cải thiện độ tin cậy của hệ thống là chuyện thường thấy. Ta sẽ khám phá chủ đề này nhiều hơn trong mục “Các Mẫu về Ổn định (Stability Patterns)” ở trang 395, nhưng các ví dụ bao gồm việc chạy nhiều instance của một microservice phía sau một bộ cân bằng tải để chịu được sự cố của một instance, hoặc dùng circuit breaker để xử lý theo cách lập trình những tình huống mà các microservice hạ nguồn không thể liên lạc được.

In such situations it can be useful to have tests that allow you to re-create certain failures to ensure that your microservice keeps operating as a whole. By their nature these tests can be a bit more tricky to implement. For example, you might need to create an artificial network time-out between a microservice under test and an external stub. That said, they can be worthwhile, especially if you are creating shared functionality that will be used across multiple microservices—for example, using a default **service mesh** implementation to handle circuit breaking.

Trong những tình huống như vậy, có thể hữu ích khi có các bài kiểm thử cho phép bạn tái tạo một số sự cố nhất định để đảm bảo rằng microservice của bạn tiếp tục vận hành như

một tổng thể. Theo bản chất của chúng, những bài kiểm thử này có thể hơi khó triển khai hơn một chút. Ví dụ, bạn có thể cần tạo một time-out mạng nhân tạo giữa một microservice đang được kiểm thử và một stub bên ngoài. Dù vậy, chúng có thể đáng giá, đặc biệt nếu bạn đang tạo chức năng dùng chung sẽ được dùng xuyên nhiều microservice—ví dụ, dùng một triển khai service mesh mặc định để xử lý việc ngắt mạch (circuit breaking).

Summary — Tóm tắt

Bringing this all together, what I have outlined in this chapter is a holistic approach to testing that hopefully gives you some general guidance on how to proceed when testing your own systems. To reiterate the basics:

Gói tất cả những điều này lại với nhau, thứ tôi đã phác họa trong chương này là một cách tiếp cận toàn diện đối với kiểm thử mà hy vọng sẽ cho bạn một số hướng dẫn chung về cách tiến hành khi kiểm thử các hệ thống của chính bạn. Để nhắc lại những điều cơ bản:

- Optimize for fast feedback, and separate types of tests accordingly.
- Avoid the need for end-to-end tests that span more than one team—consider using consumer-driven contracts instead.
- Use consumer-driven contracts to provide focus points for conversations between teams.
- Try to understand the trade-off between putting more effort into testing and detecting issues faster in production (optimizing for MTBF versus optimizing for MTTR).
- Give testing in production a try!
 - Tối ưu cho phản hồi nhanh, và tách các loại kiểm thử cho phù hợp.
 - Tránh nhu cầu về các end-to-end test trải dài qua nhiều hơn một nhóm—cân nhắc dùng consumer-driven contracts thay thế.
 - Dùng consumer-driven contracts để cung cấp các điểm tập trung cho những cuộc trò chuyện giữa các nhóm.
 - Cố gắng hiểu sự đánh đổi giữa việc bỏ thêm công sức vào kiểm thử và việc phát hiện các vấn đề nhanh hơn trên production (tối ưu cho MTBF so với tối ưu cho MTTR).
 - Hãy thử kiểm thử trên production một lần xem sao!

If you are interested in reading more about testing, I recommend *Agile Testing* by Lisa Crispin and Janet Gregory (Addison-Wesley), which among other things covers the use of the testing quadrant in more detail. For a deeper dive into the test pyramid, along with some code samples and more tool references, I also recommend “The Practical Test Pyramid” by Ham Vocke.⁹

Nếu bạn quan tâm đến việc đọc thêm về kiểm thử, tôi giới thiệu cuốn *Agile Testing* của Lisa Crispin và Janet Gregory (Addison-Wesley), trong đó, cùng với những thứ khác, có bàn về việc sử dụng sơ đồ góc phần tư kiểm thử chi tiết hơn. Để đào sâu hơn vào kim tự tháp kiểm thử, kèm theo một số mẫu mã và nhiều tham chiếu công cụ hơn, tôi cũng giới thiệu bài “The Practical Test Pyramid” của Ham Vocke.⁹

This chapter focused mostly on making sure our code works before it hits production, but we also started to look at testing our application once it reaches production. This is something we need to explore in a lot more detail. It turns out that microservices cause a multitude of challenges for understanding how our software is behaving in production—a topic we’ll cover in more depth next.

Chương này chủ yếu tập trung vào việc đảm bảo mã của ta hoạt động trước khi nó lên production, nhưng ta cũng đã bắt đầu xem xét việc kiểm thử ứng dụng của mình một khi nó tới production. Đây là thứ ta cần khám phá chi tiết hơn nhiều. Hóa ra microservices gây

ra vô số thách thức cho việc hiểu phần mềm của ta đang hành xử ra sao trên production—
một chủ đề ta sẽ bàn sâu hơn ngay sau đây.

9 Ham Vocke, “The Practical Test Pyramid,” martinfowler.com, February 26, 2018, <https://oreil.ly/J7lc6>

.

9 Ham Vocke, “The Practical Test Pyramid,” martinfowler.com, February 26, 2018, <https://oreil.ly/J7lc6>.

CHAPTER 10 From Monitoring to Observability — Chương 10: Từ Giám sát đến Khả năng quan sát (From Monitoring to Observability)

As I've shown so far, I hope, breaking our system up into smaller, fine-grained **microservices** results in multiple benefits. It also, as we've also covered in some depth, adds significant sources of new complexity. In no situation is this increased complexity more evident than when it comes to understanding the behavior of our systems in a production environment. Very early on, you'll find that the tools and techniques that worked well for relatively simpler single-process monolithic applications don't work as well for your **microservice architecture**.

Như tôi đã trình bày từ đầu đến giờ, tôi hy vọng bạn đã thấy rằng việc chia hệ thống của chúng ta thành các microservice nhỏ hơn, mịn hơn (fine-grained) mang lại nhiều lợi ích. Nhưng nó cũng, như chúng ta đã bàn khá sâu, bổ sung thêm những nguồn phức tạp mới đáng kể. Sự gia tăng phức tạp này không ở đâu rõ rệt hơn so với khi ta cần hiểu hành vi của hệ thống trong môi trường production. Rất sớm thôi, bạn sẽ nhận ra rằng các công cụ và kỹ thuật từng hoạt động tốt cho các ứng dụng monolith đơn tiến trình tương đối đơn giản lại không còn hiệu quả với kiến trúc microservice của bạn.

In this chapter, we'll look at the challenges associated with **monitoring** our microservice architecture, and I'll show that although new tools can help, fundamentally you may need to shift your entire mindset when it comes to working out what the hell is going on in production. We'll also talk about the increased focus on the concept of **observability**—understanding how to make it possible to ask questions of our system so we can work out what is going wrong.

Trong chương này, chúng ta sẽ xem xét những thách thức gắn với việc giám sát (monitoring) kiến trúc microservice, và tôi sẽ chỉ ra rằng mặc dù các công cụ mới có thể giúp ích, về cơ bản bạn có thể cần thay đổi toàn bộ tư duy của mình khi cố tìm hiểu rốt cuộc chuyện quái gì đang diễn ra trong production. Chúng ta cũng sẽ nói về sự chú trọng ngày càng tăng vào khái niệm khả năng quan sát (observability)—tức là hiểu cách làm cho việc đặt câu hỏi về hệ thống trở nên khả thi để ta tìm ra điều gì đang sai.

Production Pain

Nỗi đau Production



You won't truly appreciate the potential pain, suffering, and anguish caused by a microservice architecture until you have it running in production and serving real traffic.

Bạn sẽ không thực sự thấm thía những nỗi đau, sự khổ sở và quần quai tiềm tàng mà một kiến trúc microservice gây ra cho đến khi bạn chạy nó trong production và phục vụ lưu lượng thật.

Disruption, Panic, and Confusion — Gián đoạn, Hoảng loạn và Bối rối

Picture the scene: it's a quiet Friday afternoon, and the team is looking forward to slipping out early to the pub as a way to start a weekend away from work. Then suddenly the emails arrive. The website is misbehaving! Twitter is ablaze with your company's failings, your boss is chewing your ear off, and the prospects of a quiet weekend vanish.

Hãy hình dung cảnh này: một chiều thứ Sáu yên ả, cả đội đang mong được tan sở sớm để ra quán bia khởi đầu cho một kỳ cuối tuần xa rời công việc. Rồi đột nhiên email ập đến. Website đang trực trực! Twitter rục rủa với những lời chê bai về thất bại của công ty bạn, sếp thì la rầy bên tai, và viễn cảnh một cuối tuần yên bình tan biến.

Few things sum up the problem up as well as the following tweet:

Hiếm có gì tóm gọn vấn đề hay như dòng tweet sau:

We replaced our **monolith** with micro services so that every outage could be more like a murder mystery. 1 Tracking down what's gone wrong, and what caused it, is our first port of call. But this gets difficult if we have a long list of suspects.

Chúng tôi thay thế monolith bằng microservice để mỗi lần sập hệ thống đều giống một vụ án mạng ly kỳ hơn. 1 Truy tìm xem cái gì đã hỏng, và điều gì gây ra nó, là việc đầu tiên chúng ta phải làm. Nhưng điều này trở nên khó khăn nếu ta có một danh sách dài các nghi phạm.

In the world of the single-process monolithic application, we at least have a very obvious place to start our investigations. Website slow? It's the monolith. Website giving odd errors? It's the monolith. CPU at 100%? Monolith. A burning smell? Well, you get the idea. 2 Having a single point of failure makes failure investigation somewhat simpler!

Trong thế giới của ứng dụng monolith đơn tiến trình, ít nhất chúng ta có một điểm xuất phát rất rõ ràng để bắt đầu điều tra. Website chậm? Là do monolith. Website báo lỗi kỳ lạ? Là do monolith. CPU lên 100%? Monolith. Có mùi khét? À, bạn hiểu ý tôi rồi đấy. 2 Có một điểm hỏng duy nhất khiến việc điều tra sự cố đơn giản hơn phần nào!

Now let's think about our own microservice-based system. The capabilities we offer our users are served from multiple microservices, some of which communicate with yet more microservices to accomplish their tasks. There are lots of advantages to such an approach (which is good, as otherwise this book would be a waste of time), but in the world of monitoring, we have a more complex problem on our hands.

Giờ hãy nghĩ về hệ thống dựa trên microservice của chính chúng ta. Các năng lực mà chúng ta cung cấp cho người dùng được phục vụ bởi nhiều microservice, một số trong đó lại giao tiếp với nhiều microservice khác nữa để hoàn thành nhiệm vụ. Cách tiếp cận như vậy có rất nhiều ưu điểm (may thay, nếu không thì cuốn sách này đã là một sự phí phạm thời gian), nhưng trong thế giới của monitoring, chúng ta lại có một bài toán phức tạp hơn trong tay.

We now have multiple servers to monitor, multiple log files to sift through, and multiple places where network latency could cause problems. Our surface area of failure has increased, and so have the things that need to be investigated. So how do we approach this? We need to make sense of what otherwise might be a chaotic, tangled mess—the last thing any of us wants to deal with on a Friday afternoon (or at any time, come to that!).

Giờ đây chúng ta có nhiều máy chủ phải giám sát, nhiều tập tin log phải lần giở, và nhiều nơi mà độ trễ mạng (network latency) có thể gây ra vấn đề. Bề mặt sự cố của chúng ta đã tăng lên, và những thứ cần điều tra cũng vậy. Vậy ta tiếp cận điều này thế nào? Chúng ta cần giải mã được thứ mà nếu không sẽ là một mớ hỗn độn, rối ren—điều cuối cùng mà bất kỳ ai trong chúng ta muốn phải xử lý vào một chiều thứ Sáu (hay vào bất kỳ lúc nào, nói cho cùng!).

Firstly, we need to monitor the small things and provide aggregation to allow us to see the bigger picture. Then, we need to make sure we have tools available to us to slice and dice this data as part

of our investigation. Finally, we need to get smarter about how we think about system health, by embracing concepts like **testing** in production. We'll discuss each of these necessities in this chapter. Let's get started.

Trước tiên, chúng ta cần giám sát những thứ nhỏ và cung cấp khả năng tổng hợp (aggregation) để có thể thấy bức tranh lớn hơn. Sau đó, chúng ta cần đảm bảo có sẵn các công cụ để mổ xẻ, phân tích dữ liệu này như một phần của cuộc điều tra. Cuối cùng, chúng ta cần thông minh hơn trong cách suy nghĩ về sức khỏe hệ thống, bằng cách đón nhận các khái niệm như kiểm thử trong production (testing in production). Chúng ta sẽ bàn về từng nhu cầu này trong chương. Bắt đầu nào.

Single Microservice, Single Server — Một Microservice, Một Máy chủ

Figure 10-1 presents a very simple setup: one host that is running one instance of a microservice. Now we need to monitor it in order to know when something goes wrong, so we can fix it. So what should we look for?

Hình 10-1 trình bày một thiết lập rất đơn giản: một host chạy một instance của một microservice. Giờ chúng ta cần giám sát nó để biết khi nào có gì đó trục trặc, để còn sửa chữa. Vậy ta nên tìm gì?

1 Honestly Black Lives Matter (@honest_update), October 7, 2015, 7:10 p.m., <https://oreil.ly/Z28BA> .

1 Honestly Black Lives Matter (@honest_update), 7 tháng 10 năm 2015, 7:10 chiều, <https://oreil.ly/Z28BA> .

2 Fire as a cause of system outage isn't a totally far-fetched idea. I once helped in the aftermath of a production outage caused by a storage area network (SAN) catching fire. That it took several days for us to be told a fire had occurred is a story for another day.

2 Hỏa hoạn như một nguyên nhân gây sập hệ thống không phải là ý tưởng quá xa vời. Tôi từng giúp xử lý hậu quả của một sự cố production do một mạng lưu trữ (storage area network—SAN) bốc cháy. Việc phải mất vài ngày chúng tôi mới được báo rằng đã có hỏa hoạn là một câu chuyện để kể vào dịp khác.

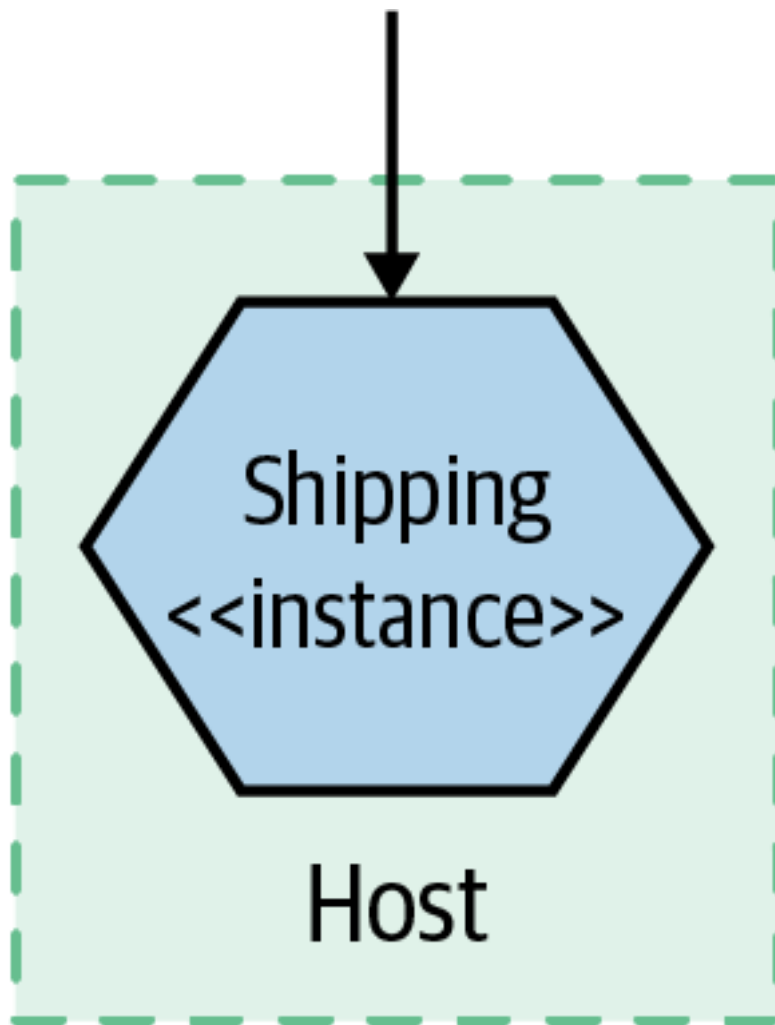


Figure 10-1. A single microservice instance on a single host

Hình 10-1. Một instance microservice duy nhất trên một host duy nhất

First, we'll want to get information from the host itself. CPU, memory—all of these things can be useful. Next, we'll want to have access to the logs from the microservice instance itself. If a user reports an error, we should be able to see the error in these logs, hopefully giving us a way to work out what went wrong. At this point, with our single host, we can probably get by with just logging locally to the host and using command-line tools to look at the log.

Trước tiên, chúng ta sẽ muốn lấy thông tin từ chính host. CPU, bộ nhớ—tất cả những thứ này đều có thể hữu ích. Tiếp theo, chúng ta sẽ muốn truy cập được log từ chính instance microservice. Nếu một người dùng báo lỗi, chúng ta phải thấy được lỗi đó trong các log này, hy vọng nó cho ta cách lần ra điều gì đã sai. Tại thời điểm này, với một host duy nhất, chúng ta có thể tạm ổn nếu chỉ ghi log cục bộ trên host và dùng các công cụ dòng lệnh để xem log.

Finally, we might want to monitor the application itself—observing it from the outside. At a bare minimum, monitoring the response time of the microservice is a good idea. If you have a web server fronting your microservice instance, you could perhaps just look at the logs of the web server. Or perhaps you can get a bit more advanced, using something like a health check endpoint to see if the microservice is up and “healthy” (we'll explore what this means later).

Cuối cùng, chúng ta có thể muốn giám sát chính ứng dụng—quan sát nó từ bên ngoài. Ở mức tối thiểu, giám sát thời gian phản hồi (response time) của microservice là một ý hay. Nếu bạn có một web server đứng trước instance microservice của mình, có lẽ bạn chỉ cần xem log của web server đó. Hoặc bạn có thể nâng cao hơn một chút, dùng thứ gì đó như một health check endpoint để xem microservice có đang chạy và “khỏe mạnh” hay không (chúng ta sẽ khám phá điều này nghĩa là gì ở phần sau).

Time passes, loads increase, and we find ourselves needing to scale...

Thời gian trôi qua, tải tăng lên, và chúng ta nhận ra mình cần mở rộng (scaling)...

Single Microservice, Multiple Servers — Một Microservice, Nhiều Máy chủ

Now we have multiple copies of the **service** running on separate hosts, as shown in Figure 10-2, with requests to the different instances distributed via a load balancer. Things start to get a bit trickier now. We still want to monitor all the same things as before, but we need to do so in such a way that we can isolate the problem. When the CPU is high, is it a problem we are seeing on all hosts, which would point to an issue with the service itself? Or is it isolated to a single host, implying that the host itself has the problem—perhaps a rogue OS process?

Giờ chúng ta có nhiều bản sao của dịch vụ chạy trên các host riêng biệt, như minh họa trong Hình 10-2, với các yêu cầu tới các instance khác nhau được phân phối qua một load balancer. Mọi thứ bắt đầu trở nên hơi rắc rối. Chúng ta vẫn muốn giám sát tất cả những thứ như trước, nhưng phải làm theo cách cho phép cô lập được vấn đề. Khi CPU cao, đó là vấn đề ta thấy trên tất cả các host—điều này chỉ ra trực tiếp với chính dịch vụ? Hay nó chỉ giới hạn ở một host duy nhất, ngụ ý rằng bản thân host đó có vấn đề—có lẽ một tiến trình OS chạy loạn?

At this point, we still want to track the host-level metrics, and perhaps maybe even alert on them when they pass some sort of threshold. But now we want to see what they are across all hosts, as well as individual hosts. In other words, we want to **aggregate** them up and still be able to drill down. So we need something that can collect all these metrics from across the hosts and allow us to slice and dice them.

Tại thời điểm này, chúng ta vẫn muốn theo dõi các metric ở cấp host, và có lẽ thậm chí cảnh báo (alert) về chúng khi chúng vượt một ngưỡng nào đó. Nhưng giờ chúng ta muốn thấy chúng trên tất cả các host, cũng như trên từng host riêng lẻ. Nói cách khác, chúng ta muốn tổng hợp chúng lại mà vẫn có thể khoan sâu xuống (drill down). Vậy nên chúng ta cần thứ gì đó có thể thu thập tất cả các metric này từ khắp các host và cho phép ta mổ xẻ, phân tích chúng.

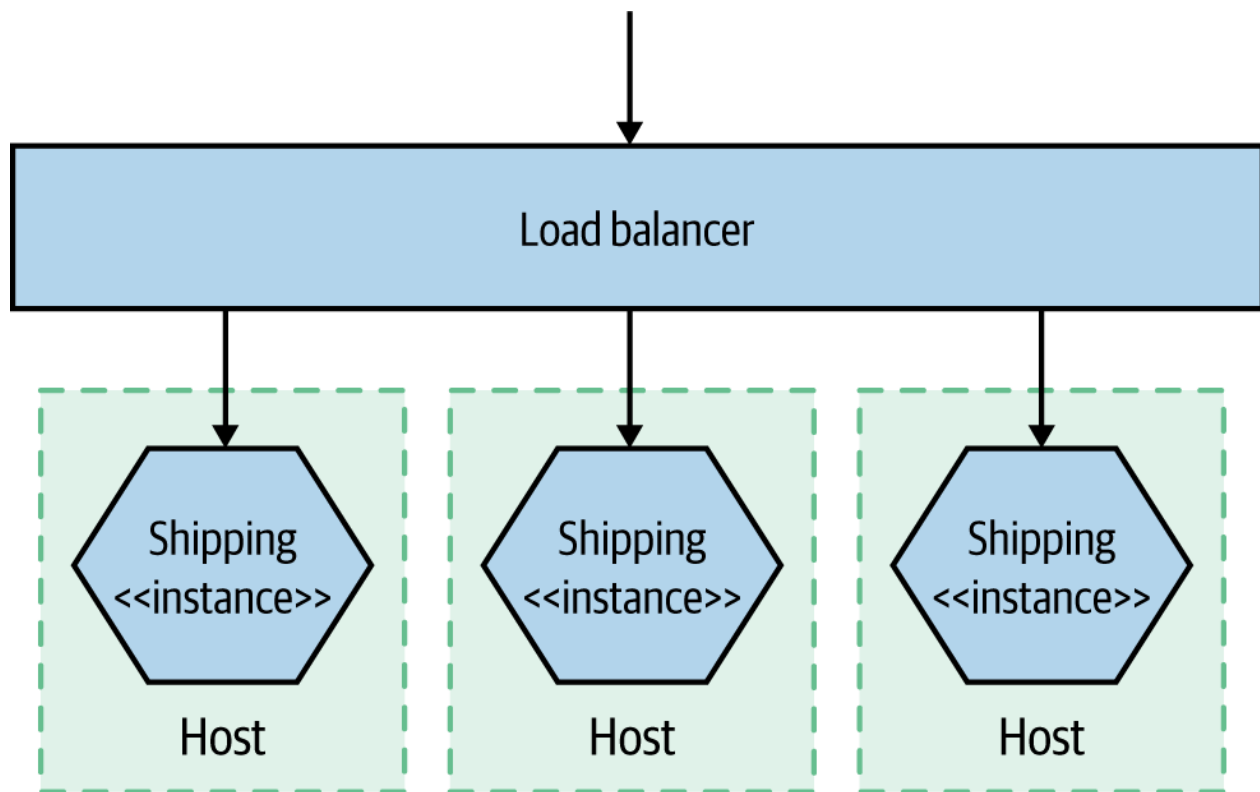


Figure 10-2. A single service distributed across multiple hosts

Hình 10-2. Một dịch vụ duy nhất được phân tán trên nhiều host

Then we have our logs. With our service running on more than one server, we'll probably get tired of logging into each box to look at it. With just a few hosts, though, we can use tools like SSH multiplexers, which allow us to run the same commands on multiple hosts. With the help of a big monitor, and running `grep "Error"` on our microservice log, we can find our culprit. I mean, this isn't great, but it can be good enough for a while. It will get old pretty fast though.

Tiếp đến là log của chúng ta. Với dịch vụ chạy trên nhiều hơn một máy chủ, có lẽ chúng ta sẽ phát mệt khi phải đăng nhập vào từng máy để xem log. Tuy nhiên, với chỉ vài host, chúng ta có thể dùng các công cụ như SSH multiplexer, cho phép chạy cùng một lệnh trên nhiều host. Nhờ một màn hình lớn, và chạy `grep "Error"` trên log microservice, chúng ta có thể tìm ra thủ phạm. Ý tôi là, cách này không tuyệt vời gì, nhưng nó có thể tạm đủ dùng một thời gian. Dù vậy nó sẽ sớm trở nên lỗi thời.

For tasks like response time tracking, we can capture response times at the load balancer for downstream calls to microservices. However, we also have to consider what happens if the load balancer is turning out to be the bottleneck in our system—capturing response times both at the load balancer and at the microservices themselves could be needed. At this point, we probably also care a lot more about what a healthy service looks like, as we'll configure our load balancer to remove unhealthy nodes from our application. Hopefully by the time we get here, we have at least some idea of what healthy looks like.

Với những việc như theo dõi thời gian phản hồi, chúng ta có thể bắt thời gian phản hồi tại load balancer cho các lời gọi downstream tới microservice. Tuy nhiên, chúng ta cũng phải cân nhắc điều gì xảy ra nếu chính load balancer hóa ra là nút thắt cổ chai trong hệ thống—có thể cần bắt thời gian phản hồi cả tại load balancer lẫn tại chính các microservice. Tại

thời điểm này, có lẽ chúng ta cũng quan tâm nhiều hơn nhiều đến việc một dịch vụ khỏe mạnh trông như thế nào, vì chúng ta sẽ cấu hình load balancer để loại bỏ các node không khỏe khỏi ứng dụng. Hy vọng đến lúc này, chúng ta ít nhất đã có chút ý niệm về thế nào là khỏe mạnh.

Multiple Services, Multiple Servers — Nhiều Dịch vụ, Nhiều Máy chủ

In Figure 10-3 , things get much more interesting. Multiple services are collaborating to provide capabilities to our users, and those services are running on multiple hosts, be they physical or virtual. How do you find the error you're looking for in thousands of lines of logs on multiple hosts? How do you determine if a server is misbehaving, or if it is a systemic issue? And how do you track back an error found deep down in a call chain between multiple hosts and work out what caused it?

Trong Hình 10-3 , mọi thứ trở nên thú vị hơn nhiều. Nhiều dịch vụ cộng tác để cung cấp năng lực cho người dùng của chúng ta, và những dịch vụ đó chạy trên nhiều host, dù là vật lý hay ảo. Làm sao bạn tìm được lỗi mình đang truy trong hàng nghìn dòng log trên nhiều host? Làm sao bạn xác định một máy chủ đang trục trặc, hay đó là vấn đề mang tính hệ thống? Và làm sao bạn lần ngược lại một lỗi nằm sâu trong một chuỗi lời gọi giữa nhiều host và tìm ra điều gì đã gây ra nó?

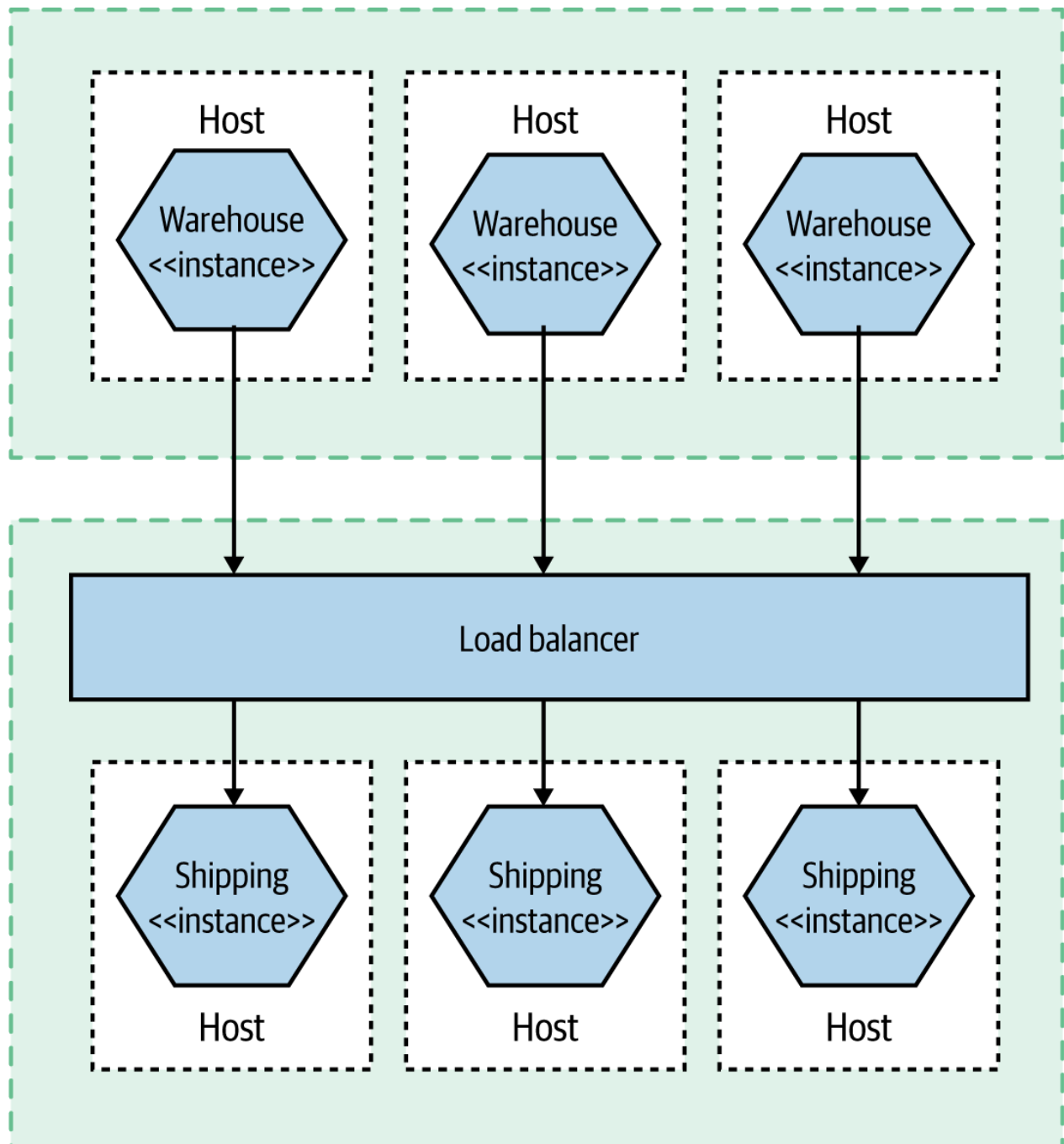


Figure 10-3. Multiple collaborating services distributed across multiple hosts

Hình 10-3. Nhiều dịch vụ cộng tác được phân tán trên nhiều host

Aggregation of information—metrics and logs—play a vital part in making this happen. But this is not the only thing we need to consider. We need to work out how to sift this huge influx of data and try to make sense of it all. Above all, this is largely about a mindset shift, from a fairly static landscape of monitoring to the more active world of observability and testing in production.

Việc tổng hợp thông tin—metric và log—đóng vai trò sống còn trong việc làm cho điều này khả thi. Nhưng đây không phải là thứ duy nhất ta cần cân nhắc. Chúng ta cần tìm ra cách sàng lọc lượng dữ liệu khổng lồ ủa vào này và cố gắng giải mã tất cả. Trên hết, đây phần

lớn là một sự chuyển dịch tư duy, từ một khung cảnh giám sát khá tĩnh sang thế giới chủ động hơn của khả năng quan sát và kiểm thử trong production.

Observability Versus Monitoring — Khả năng quan sát so với Giám sát (Observability Versus Monitoring)

We're going to start delving into how we begin solving some of the problems we've just outlined, but before we do, I think it's important that we explore a term that has gained a lot of popularity since I wrote the first edition of this book— observability .

Chúng ta sắp đào sâu vào cách bắt đầu giải quyết một số vấn đề vừa nêu, nhưng trước khi làm vậy, tôi nghĩ điều quan trọng là khám phá một thuật ngữ đã trở nên rất phổ biến kể từ khi tôi viết phiên bản đầu tiên của cuốn sách này—observability .

As is often the case, the concept of observability has been around for decades but has only recently come to software development. The observability of a system is the extent to which you can understand the internal state of the system from external outputs. It typically requires a more holistic understanding of your software—seeing it more as a system rather than a set of disparate entities.

Như thường thấy, khái niệm observability đã tồn tại hàng thập kỷ nhưng chỉ gần đây mới đến với phát triển phần mềm. Khả năng quan sát của một hệ thống là mức độ bạn có thể hiểu trạng thái nội tại của hệ thống từ các đầu ra bên ngoài (external outputs). Nó thường đòi hỏi một sự hiểu biết toàn diện hơn về phần mềm của bạn—nhìn nhận nó như một hệ thống hơn là một tập hợp các thực thể rời rạc.

In practice, the more observable a system is, the easier it will be for us to understand what the problem is when something goes wrong. Our understanding of the external outputs helps us track down the underlying issue more quickly. The challenge is that often we will need to create these external outputs, and use different types of tools to understand the outputs.

Trong thực tế, một hệ thống càng có khả năng quan sát cao thì càng dễ để chúng ta hiểu vấn đề là gì khi có sự cố. Sự hiểu biết của chúng ta về các đầu ra bên ngoài giúp ta truy ra vấn đề cốt lõi nhanh hơn. Thách thức là thường chúng ta sẽ cần tạo ra các đầu ra bên ngoài này, và dùng các loại công cụ khác nhau để hiểu chúng.

Monitoring, on the other hand, is something we do. We monitor the system. We look at it. Things will start to go wrong if you focus on just the monitoring—the activity— without thinking about what you expect that activity to achieve.

Giám sát, mặt khác, là một việc chúng ta làm. Chúng ta giám sát hệ thống. Chúng ta nhìn vào nó. Mọi thứ sẽ bắt đầu trục trặc nếu bạn chỉ chăm chăm vào việc giám sát—tức bản thân hoạt động đó—mà không nghĩ về việc bạn kỳ vọng hoạt động ấy đạt được điều gì.

More traditional approaches to monitoring would have you think in advance about what could go wrong and define **alerting** mechanisms to tell you when these things happened. But as the system becomes increasingly distributed, you'll encounter issues that would never have occurred to you. With a highly observable system, you'll have a collection of external outputs that you can interrogate in different ways—the concrete outcome of having an observable system is that you can ask questions of your production system that you never would have thought to ask before.

Các cách tiếp cận truyền thống hơn đối với giám sát sẽ buộc bạn nghĩ trước về những gì có thể sai và định nghĩa các cơ chế cảnh báo để báo cho bạn biết khi những điều đó xảy ra. Nhưng khi hệ thống ngày càng phân tán, bạn sẽ gặp những vấn đề mà bạn không bao giờ

nghĩ tới. Với một hệ thống có khả năng quan sát cao, bạn sẽ có một bộ sưu tập các đầu ra bên ngoài mà bạn có thể truy vấn theo nhiều cách khác nhau—kết quả cụ thể của việc có một hệ thống quan sát được là bạn có thể đặt cho hệ thống production của mình những câu hỏi mà trước đây bạn chưa từng nghĩ đến.

We can therefore see monitoring as an activity—something we do—with observability being a property of the system.

Do đó chúng ta có thể xem giám sát như một hoạt động—thứ chúng ta làm—còn khả năng quan sát là một thuộc tính của hệ thống.

The Pillars of Observability? Not So Fast — Các Trụ cột của Khả năng quan sát? Khoan đã

Some people have attempted to distill down the idea of observability to a few core concepts. Some have focused on “the three pillars” of observability in the form of metrics, logging, and **distributed tracing**. New Relic even coined the term MELT (metrics, event, logs, and traces), which hasn’t really caught on, but at least New Relic is trying. Although this simple model initially appealed to me greatly (and I’m a sucker for an acronym!), over time I’ve really moved away from this thinking as being overly reductive but also potentially missing the point.

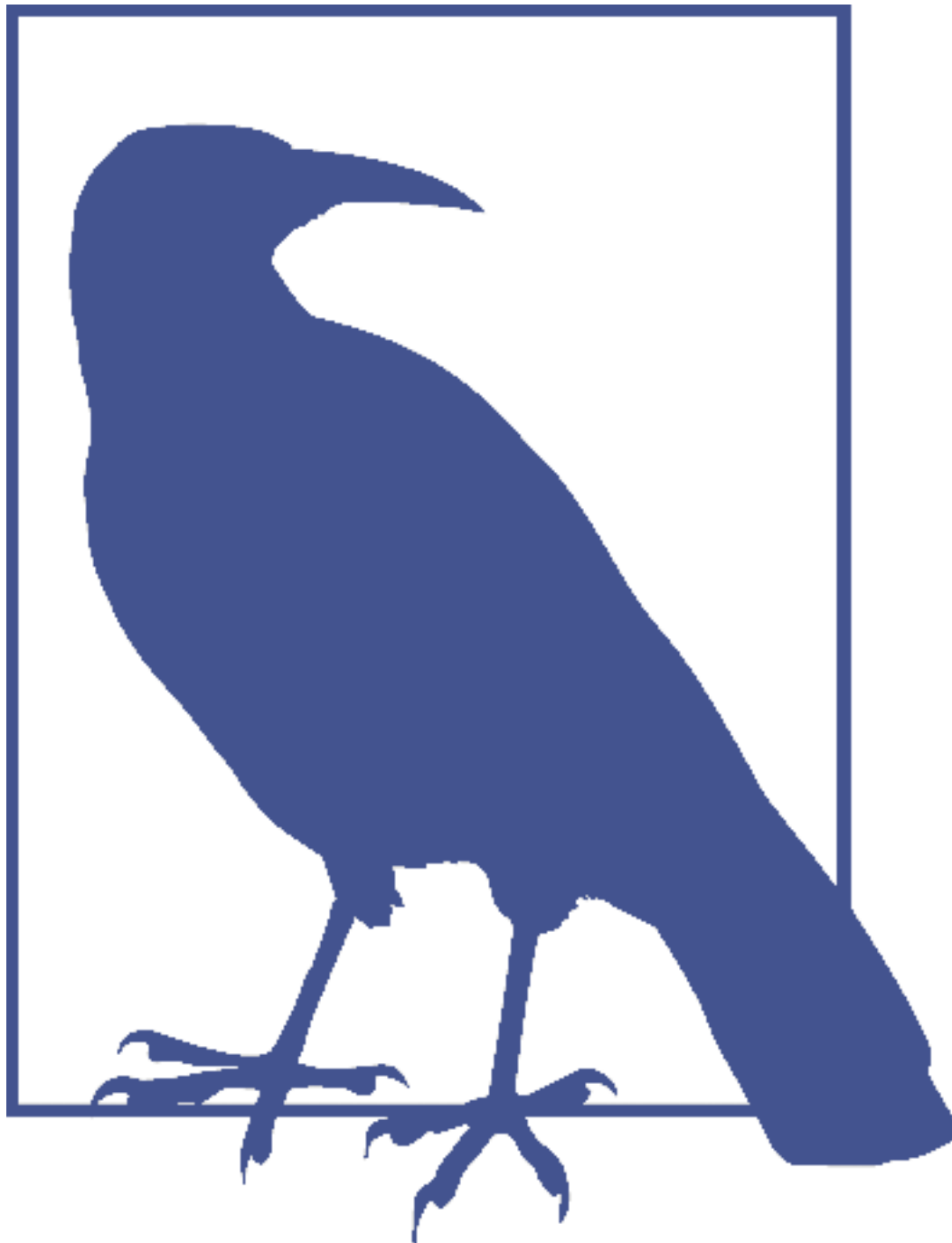
Một số người đã cố gắng chắt lọc ý tưởng về observability xuống còn vài khái niệm cốt lõi. Một số tập trung vào “ba trụ cột” của observability dưới dạng metric, ghi log (logging), và distributed tracing. New Relic thậm chí đặt ra thuật ngữ MELT (metrics, event, logs, and traces), thứ chưa thực sự phổ biến, nhưng ít nhất New Relic cũng đang cố. Mặc dù mô hình đơn giản này ban đầu hấp dẫn tôi vô cùng (và tôi thì cực mê các từ viết tắt!), theo thời gian tôi đã thực sự rời xa lối tư duy này vì thấy nó vừa quá rút gọn vừa có khả năng bỏ lỡ điểm cốt yếu.

Firstly, reducing a property of a system to implementation details in this way seems backward to me. Observability is a property, and there are many ways I might be able to achieve that property. Focusing too much on specific implementation details runs the risk of focusing on activity versus outcome. It’s analogous to the current world of IT, where hundreds if not thousands of organizations have fallen in love with building microservice-based systems without really understanding what they are trying to achieve!

Trước hết, việc rút gọn một thuộc tính của hệ thống xuống thành các chi tiết hiện thực theo kiểu này có vẻ ngược đời đối với tôi. Observability là một thuộc tính, và có nhiều cách tôi có thể đạt được thuộc tính đó. Tập trung quá nhiều vào các chi tiết hiện thực cụ thể có nguy cơ chú trọng hoạt động thay vì kết quả. Nó tương tự như thế giới CNTT hiện nay, nơi hàng trăm nếu không phải hàng nghìn tổ chức đã đắm say việc xây dựng các hệ thống dựa trên microservice mà không thực sự hiểu họ đang cố đạt được điều gì!

Secondly, are there always concrete lines between these concepts? I’d argue that many of them overlap. I can put metrics in a logfile if I want. Likewise, I can construct a distributed trace from a series of log lines, something that is commonly done.

Thứ hai, liệu có luôn có những lần ranh rõ ràng giữa các khái niệm này không? Tôi cho rằng nhiều cái trong số chúng chồng lấn lên nhau. Tôi có thể đặt metric vào một tập tin log nếu muốn. Tương tự, tôi có thể dựng một distributed trace từ một loạt dòng log, điều thường được làm.



Observability is the extent to which you can understand what the system is doing based on external inputs. Logs, events, and metrics might help you make things observable, but be sure to focus on making the system understandable rather than throwing in lots of tools.

Khả năng quan sát là mức độ bạn có thể hiểu hệ thống đang làm gì dựa trên các đầu vào bên ngoài. Log, event và metric có thể giúp bạn làm cho mọi thứ quan sát được, nhưng hãy chắc rằng bạn tập trung vào việc làm cho hệ thống dễ hiểu thay vì ném vào thật nhiều công cụ.

Cynically, I could suggest that the pushing of this simplistic narrative is a way to sell you tools. You need one tool for metrics, a different tool for logs, and yet another tool for traces! And you need to

send all that information differently! It's much easier to sell features by way of a box-ticking exercise when trying to market a product, rather than talking about outcomes. As I said, I could suggest this with a cynical view, but it's 2021, and I'm trying to be a bit more positive in my thinking. 3

Một cách hoài nghi, tôi có thể gợi ý rằng việc đẩy mạnh câu chuyện đơn giản hóa này là một cách để bán công cụ cho bạn. Bạn cần một công cụ cho metric, một công cụ khác cho log, và lại thêm một công cụ nữa cho trace! Và bạn cần gửi tất cả thông tin đó theo những cách khác nhau! Bán tính năng theo kiểu một bài tập đánh dấu vào ô danh sách thì dễ hơn nhiều khi tiếp thị một sản phẩm, thay vì nói về kết quả. Như tôi đã nói, tôi có thể gợi ý điều này với một góc nhìn hoài nghi, nhưng giờ là năm 2021, và tôi đang cố tích cực hơn một chút trong suy nghĩ của mình. 3

It's arguable all these three (or four!) concepts are really just specific examples of a more generic concept. Fundamentally, we can see any piece of information we can get from our system—any of these external outputs—generically as an event. A given event might contain a small or large amount of information. It could contain a CPU rate, information about a failed payment, the fact that a customer has logged in, or any number of things. We can project from this event stream a trace (assuming we can correlate these events), a searchable index, or an aggregation of numbers. Although at present we chose to collect this information in different ways, using different tools and different protocols, our current toolchains shouldn't limit our thinking in terms of how best to get the information we need.

Có thể lập luận rằng cả ba (hay bốn!) khái niệm này thực ra chỉ là các ví dụ cụ thể của một khái niệm tổng quát hơn. Về cơ bản, chúng ta có thể xem bất kỳ mẫu thông tin nào lấy được từ hệ thống—bất kỳ đâu ra bên ngoài nào trong số này—một cách tổng quát như một event. Một event nhất định có thể chứa một lượng thông tin nhỏ hay lớn. Nó có thể chứa một tỉ lệ CPU, thông tin về một khoản thanh toán thất bại, sự kiện một khách hàng đã đăng nhập, hoặc vô số thứ khác. Từ luồng event này, chúng ta có thể chiếu ra một trace (giả định ta có thể tương quan các event này), một chỉ mục tìm kiếm được, hoặc một phép tổng hợp các con số. Mặc dù hiện tại chúng ta chọn thu thập thông tin này theo những cách khác nhau, dùng các công cụ khác nhau và giao thức khác nhau, các chuỗi công cụ hiện tại không nên giới hạn tư duy của ta về cách tốt nhất để lấy được thông tin mình cần.

When it comes to making your system observable, think about the outputs you need from your system in terms of events that you can collect and interrogate. You might need to use different tooling to expose different types of events right now, but that may not be the case in the future.

Khi nói đến việc làm cho hệ thống của bạn quan sát được, hãy nghĩ về các đầu ra bạn cần từ hệ thống dưới dạng các event mà bạn có thể thu thập và truy vấn. Bạn có thể cần dùng các công cụ khác nhau để phơi bày các loại event khác nhau ngay lúc này, nhưng điều đó có thể sẽ không còn đúng trong tương lai.

Building Blocks for Observability — Các Khối xây dựng cho Khả năng quan sát

So what do we need? We need to know that the users of our software are happy. If there is a problem, we want to know about it—ideally before our users find an issue themselves. When a problem does occur, we need to work out what we can do to get the system up and running again, and once the dust has settled we want to have enough information in hand to work out what the hell went wrong and what we can do to avoid the problem happening again.

Vậy chúng ta cần gì? Chúng ta cần biết rằng người dùng phần mềm của chúng ta đang hài

lòng. Nếu có vấn đề, chúng ta muốn biết về nó—lý tưởng nhất là trước khi người dùng tự phát hiện ra trục trặc. Khi một vấn đề thực sự xảy ra, chúng ta cần tìm ra mình có thể làm gì để đưa hệ thống chạy lại, và một khi bụi đã lắng, chúng ta muốn có đủ thông tin trong tay để tìm ra rốt cuộc chuyện quái gì đã sai và chúng ta có thể làm gì để tránh vấn đề tái diễn.

In the **rest** of this chapter we will be looking at how to make this all happen. We will be covering a number of building blocks that can help improve the observability of your **system architecture**:

Trong phần còn lại của chương này, chúng ta sẽ xem xét cách làm cho tất cả điều này xảy ra. Chúng ta sẽ bàn về một số khối xây dựng có thể giúp cải thiện khả năng quan sát của kiến trúc hệ thống của bạn:

3 I didn't say I was succeeding.

3 Tôi không nói là mình đang thành công.

Log aggregation Collecting information across multiple microservices, a vital building block of any monitoring or observability solution

Gom log (Log aggregation) Thu thập thông tin trên nhiều microservice, một khối xây dựng sống còn của bất kỳ giải pháp giám sát hay khả năng quan sát nào

Metrics aggregation Capturing raw numbers from our microservices and infrastructure to help detect problems, drive capacity planning, and perhaps even scale our applications

Tổng hợp metric (Metrics aggregation) Bắt các con số thô từ các microservice và hạ tầng của chúng ta để giúp phát hiện vấn đề, định hướng việc lập kế hoạch dung lượng (capacity planning), và thậm chí có thể mở rộng ứng dụng của chúng ta

Distributed tracing Tracking a flow of calls across multiple microservice boundaries to work out what went wrong and derive accurate latency information

Distributed tracing Theo dõi một luồng lời gọi qua nhiều ranh giới microservice để tìm ra điều gì đã sai và rút ra thông tin độ trễ chính xác

Are you doing OK? Looking at error budgets, SLAs, SLOs, and so on to see how they can be used as part of making sure our microservice is meeting the needs of its consumers

Bạn vẫn ổn chứ? Xem xét các ngân sách lỗi (error budget), SLA, SLO, v.v. để thấy cách chúng có thể được dùng như một phần của việc đảm bảo microservice của chúng ta đáp ứng nhu cầu của những bên tiêu thụ nó

Alerting What should you alert on? What does a good alert look like?

Cảnh báo (Alerting) Bạn nên cảnh báo về điều gì? Một cảnh báo tốt trông như thế nào?

Semantic monitoring Thinking differently about the health of our systems, and about what should wake us up at 3 a.m.

Semantic monitoring Suy nghĩ khác đi về sức khỏe của hệ thống, và về điều gì nên đánh thức chúng ta dậy lúc 3 giờ sáng

Testing in production A summary of various testing in production techniques

Kiểm thử trong production (Testing in production) Tóm lược các kỹ thuật kiểm thử trong production khác nhau

Let's start with perhaps the simplest thing to get up and running, but something that will pay for itself many times over: log aggregation.

Hãy bắt đầu với điều có lẽ đơn giản nhất để triển khai, nhưng là thứ sẽ hoàn vốn cho chính nó nhiều lần: gom log.

Log Aggregation — Gom log (Log Aggregation)

With many servers and microservice instances in even a modest microservice architecture, logging into machines or SSH-multiplexing to retrieve logs doesn't really cut it. Instead, we're looking to use specialized subsystems to grab our logs and make them available centrally.

Với nhiều máy chủ và nhiều instance microservice ngay cả trong một kiến trúc microservice khiêm tốn, việc đăng nhập vào các máy hay dùng SSH-multiplexing để lấy log thực sự không kham nổi. Thay vào đó, chúng ta hướng đến việc dùng các hệ thống con chuyên dụng để gom log và làm cho chúng có sẵn ở một nơi tập trung.

Logs will very quickly become one of the most vital mechanisms to help you understand what is happening in your production system. With simpler **deployment** architectures, our logfiles, what we put into them, and how we treat them are often an afterthought. With an increasingly **distributed system**, they will become a vital tool, not only helping you diagnose what went wrong when you find out you have a problem but also telling you that there was an issue that needed your attention in the first place.

Log sẽ rất nhanh chóng trở thành một trong những cơ chế sống còn nhất giúp bạn hiểu điều gì đang diễn ra trong hệ thống production của mình. Với các kiến trúc triển khai đơn giản hơn, các tập tin log, những gì ta đặt vào đó, và cách ta đối xử với chúng thường chỉ là một suy nghĩ phụ. Với một hệ thống ngày càng phân tán, chúng sẽ trở thành một công cụ sống còn, không chỉ giúp bạn chẩn đoán điều gì đã sai khi bạn phát hiện mình có vấn đề mà còn báo cho bạn biết rằng đã có một sự cố cần bạn chú ý ngay từ đầu.

As we'll discuss shortly, there are a variety of tools in this space, but they all largely operate in the same way, as outlined in Figure 10-4 . Processes (like our microservice instances) log to their local filesystem. A local daemon process periodically collects and forwards this log to some sort of store that can be queried by operators. One of the nice aspects of these systems is that your microservice architecture can be largely unaware of them. You don't need to change your code to use some sort of special **API**; you just log to a local filesystem. You do need to understand the failure modes around this log shipping process, though, especially if you want to understand the situations logs can get lost in.

Như chúng ta sẽ bàn ngay sau đây, có nhiều loại công cụ trong không gian này, nhưng chúng đều hoạt động theo cùng một cách về cơ bản, như được phác họa trong Hình 10-4 . Các tiến trình (như các instance microservice của chúng ta) ghi log vào hệ thống tập tin cục bộ. Một tiến trình daemon cục bộ định kỳ thu thập và chuyển tiếp log này đến một kho lưu trữ nào đó có thể được truy vấn bởi các operator. Một trong những khía cạnh hay của các hệ thống này là kiến trúc microservice của bạn phần lớn có thể không cần biết đến chúng. Bạn không cần thay đổi mã của mình để dùng một loại API đặc biệt nào đó; bạn chỉ cần ghi log vào hệ thống tập tin cục bộ. Tuy vậy, bạn cần hiểu các chế độ hỏng (failure modes) quanh quá trình chuyển tiếp log này, đặc biệt nếu bạn muốn hiểu những tình huống mà log có thể bị mất.

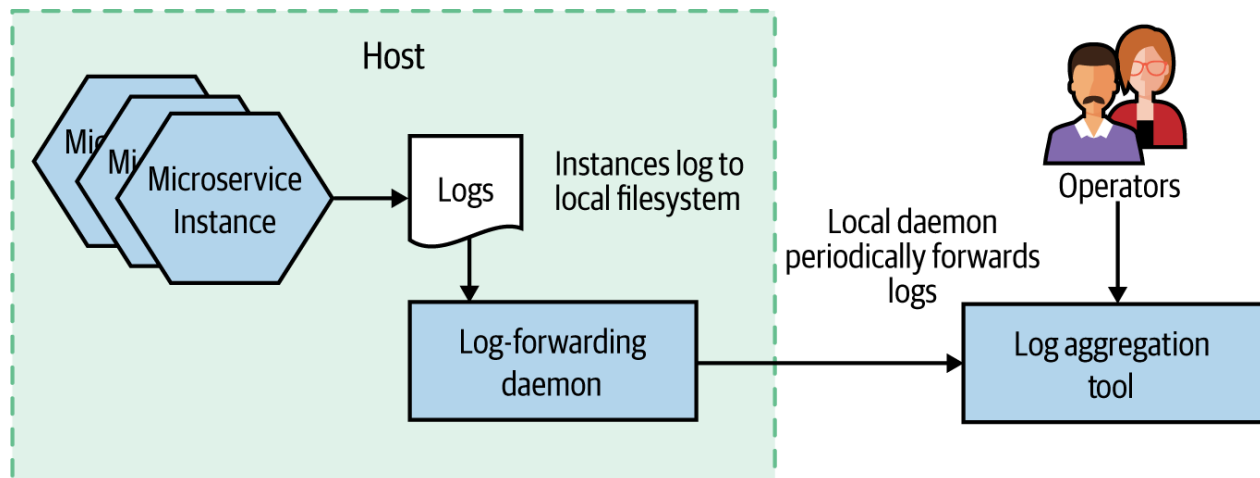


Figure 10-4. An overview of how logs are collected as part of log aggregation

Hình 10-4. Tổng quan về cách log được thu thập như một phần của gom log

Now, I hope you’ve perceived that I try to avoid being dogmatic about things. Rather than just saying you must do X or Y, I’ve tried to give you context and guidance and explain the nuances of certain decisions—that is, I’ve tried to give you the tools to make the right choice for your context. But on the subject of log aggregation, I’ll come as close I get to giving one-size-fits-all advice: you should view implementing a log aggregation tool as a prerequisite for implementing a microservice architecture.

Bây giờ, tôi hy vọng bạn đã nhận ra rằng tôi cố tránh giáo điều về mọi thứ. Thay vì chỉ nói bạn phải làm X hay Y, tôi đã cố cung cấp cho bạn bối cảnh và hướng dẫn cùng giải thích các sắc thái của một số quyết định—tức là, tôi đã cố trao cho bạn các công cụ để đưa ra lựa chọn đúng cho bối cảnh của mình. Nhưng về chủ đề gom log, tôi sẽ tiến gần nhất có thể đến việc đưa ra lời khuyên kiểu “một cỡ vừa cho tất cả”: bạn nên xem việc triển khai một công cụ gom log là điều kiện tiên quyết để triển khai một kiến trúc microservice.

My reasons for this point of view are twofold. Firstly, log aggregation is incredibly useful. For those of you who treat your logfiles as a dumping ground for misinformation, this will come as a surprise. But trust me—when done right, log aggregation can be incredibly valuable, especially when used with another concept we’ll cover shortly, correlation IDs.

Lý do của tôi cho quan điểm này gồm hai phần. Thứ nhất, gom log cực kỳ hữu ích. Với những bạn coi tập tin log như một bãi rác chứa thông tin sai lệch, điều này sẽ là một bất ngờ. Nhưng tin tôi đi—khi làm đúng cách, gom log có thể cực kỳ giá trị, đặc biệt khi dùng cùng với một khái niệm khác mà chúng ta sẽ bàn ngay sau đây, các correlation ID.

Secondly, implementing a log aggregation, when compared to the other sources of pain and suffering that a microservice architecture can bring, isn’t that difficult. If your organization is unable to successfully implement a simple log aggregation solution, it will likely find the other aspects of a microservice architecture too much to handle. So consider using implementation of such a solution as a way of testing your organization’s readiness for the rest of the horror that will follow.

Thứ hai, triển khai gom log, so với những nguồn đau khổ và thống khổ khác mà một kiến trúc microservice có thể mang lại, thì không khó đến vậy. Nếu tổ chức của bạn không thể triển khai thành công một giải pháp gom log đơn giản, thì nhiều khả năng nó sẽ thấy các khía cạnh khác của kiến trúc microservice là quá sức để xử lý. Vậy nên hãy cân nhắc dùng việc triển khai một giải pháp như vậy như một cách kiểm tra mức độ sẵn sàng của tổ chức

bạn cho phần còn lại của những nỗi kinh hoàng sắp tới.



Before Anything Else

Trước Hết Mọi Điều Khác

Before you do anything else to **build** out your microservice architecture, get a log aggregation tool

up and running. Consider it a prerequisite for building a microservice architecture. You'll thank me later.

Trước khi bạn làm bất cứ điều gì khác để xây dựng kiến trúc microservice của mình, hãy đưa một công cụ gom log vào vận hành. Hãy xem nó như một điều kiện tiên quyết để xây dựng một kiến trúc microservice. Sau này bạn sẽ cảm ơn tôi.

Now, it's also true to say that log aggregation has its limitations, and over time you may well want to look at more sophisticated tooling to augment or even replace some of what log aggregation provides. All that being said, it's still an excellent place to start.

Bây giờ, cũng đúng khi nói rằng gom log có những hạn chế của nó, và theo thời gian bạn rất có thể sẽ muốn xem xét các công cụ tinh vi hơn để bổ trợ hoặc thậm chí thay thế một phần những gì gom log mang lại. Dù vậy, nó vẫn là một điểm khởi đầu xuất sắc.

Common format

Định dạng chung

If you are going to aggregate your logs, you'll want to be able to run queries across them to extract useful information. For this to work, it's important that you pick a sensible standard log format—otherwise your queries will end up being difficult or perhaps impossible to write. You want the date, time, microservice name, log level, and so on in consistent places in each log.

Nếu bạn định tổng hợp log của mình, bạn sẽ muốn có thể chạy các truy vấn trên chúng để trích xuất thông tin hữu ích. Để điều này hoạt động, điều quan trọng là bạn chọn một định dạng log chuẩn hợp lý—nếu không các truy vấn của bạn rất cuộc sẽ khó viết hoặc thậm chí không thể viết. Bạn muốn ngày, giờ, tên microservice, mức log (log level), v.v. nằm ở những vị trí nhất quán trong mỗi log.

Some log forwarding agents give you the ability to reformat logs prior to forwarding them to your central log store. Personally, I'd avoid this wherever possible. The issue is that reformatting logs can be computationally intensive, to the point where I've seen real production issues caused by CPU being tied up carrying out this task. It's much better to change the logs as they are written by your microservice itself. I'd keep the use of log forwarding agents to do log reformatting to places where I am unable to change the source log format—for example, legacy or third-party software.

Một số tác nhân chuyển tiếp log (log forwarding agent) cho bạn khả năng định dạng lại log trước khi chuyển tiếp chúng đến kho log trung tâm. Cá nhân tôi, tôi sẽ tránh điều này bất cứ khi nào có thể. Vấn đề là việc định dạng lại log có thể tốn nhiều tính toán, đến mức tôi từng thấy những sự cố production thực sự gây ra bởi CPU bị chiếm dụng để thực hiện tác vụ này. Tốt hơn nhiều là thay đổi log ngay khi chúng được ghi bởi chính microservice của bạn. Tôi sẽ giới hạn việc dùng tác nhân chuyển tiếp log để định dạng lại log ở những chỗ mà tôi không thể thay đổi định dạng log gốc—ví dụ, phần mềm cũ (legacy) hay phần mềm của bên thứ ba.

I would have thought that in the time since I wrote the first edition of this book a common industry standard for logging would have gained traction, but that doesn't seem to have happened. Lots of variations seem to exist; they normally involve taking the standard access log format supported by web servers like Apache and nginx and expanding on it by adding more columns of data. The key thing is that within your own microservice architecture you pick a format that you standardize on internally.

Tôi đã nghĩ rằng trong khoảng thời gian kể từ khi tôi viết phiên bản đầu của cuốn sách này, một chuẩn ngành chung cho việc ghi log sẽ được áp dụng rộng rãi, nhưng điều đó dường như đã không xảy ra. Có vẻ tồn tại rất nhiều biến thể; chúng thường liên quan đến

việc lấy định dạng access log chuẩn được hỗ trợ bởi các web server như Apache và nginx rồi mở rộng nó bằng cách thêm các cột dữ liệu. Điều quan trọng là trong chính kiến trúc microservice của bạn, bạn chọn một định dạng để chuẩn hóa nội bộ.

If you are using a fairly simple log format, you'll just be emitting simple lines of text that have specific pieces of information at specific places in the log line. In Example 10-1, we see an example format.

Nếu bạn dùng một định dạng log khá đơn giản, bạn sẽ chỉ phát ra các dòng văn bản đơn giản với những mẫu thông tin cụ thể ở những vị trí cụ thể trong dòng log. Trong Ví dụ 10-1, chúng ta thấy một định dạng mẫu.

Example 10-1. Some example logs

Ví dụ 10-1. Một số log mẫu

15-02-2020 16:00:58 Order INFO [abc-123] Customer 2112 has placed order 988827 15-02-2020 16:01:01 Payment INFO [abc-123] Payment \$20.99 for 988827 by cust 2112 The log aggregation tool will need to know how to parse this string to extract the information we might want to query on—the timestamp, microservice name, or log level, for example. In this example, that is workable, as these pieces of data occur in static places across our logs—the date is the first column, the time is the second column, and so on. This is more problematic, though, if we want to find log lines related to a given customer—the customer ID is in both log lines but is shown in different locations. This is where we might start thinking about writing out more structured log lines, perhaps using a JSON format, so we can find information like a customer or order ID in a consistent location. Again, the log aggregation tool will need to be configured to parse and extract the required information from the log. The other thing to note is that if you log JSON, that might make it more difficult to be directly read by a human without additional tooling to parse out the required values—simply reading the log in a plain text viewer may not be terribly useful.

15-02-2020 16:00:58 Order INFO [abc-123] Customer 2112 has placed order 988827 15-02-2020 16:01:01 Payment INFO [abc-123] Payment \$20.99 for 988827 by cust 2112 Công cụ gom log sẽ cần biết cách phân tích chuỗi này để trích xuất thông tin mà ta có thể muốn truy vấn—ví dụ timestamp, tên microservice, hay mức log. Trong ví dụ này, điều đó khả thi, vì các mẫu dữ liệu này xuất hiện ở những vị trí cố định xuyên suốt log—ngày là cột đầu tiên, giờ là cột thứ hai, v.v. Tuy nhiên, điều này khó hơn nếu ta muốn tìm các dòng log liên quan đến một khách hàng nhất định—ID khách hàng có trong cả hai dòng log nhưng hiển thị ở các vị trí khác nhau. Đây là lúc ta có thể bắt đầu nghĩ đến việc ghi ra các dòng log có cấu trúc hơn, có lẽ dùng định dạng JSON, để ta có thể tìm thông tin như ID khách hàng hay ID đơn hàng ở một vị trí nhất quán. Một lần nữa, công cụ gom log sẽ cần được cấu hình để phân tích và trích xuất thông tin cần thiết từ log. Điều khác cần lưu ý là nếu bạn ghi log JSON, điều đó có thể khiến nó khó được con người đọc trực tiếp hơn nếu không có công cụ bổ sung để phân tích ra các giá trị cần thiết—chỉ đơn giản đọc log trong một trình xem văn bản thuần có thể không hữu ích lắm.

Correlating log lines

Tương quan các dòng log

With a large number of services interacting to provide any given end-user capability, a single initiating call can end up generating multiple downstream service calls. For example, let's consider an example in MusicCorp, as shown in Figure 10-5. We are signing up a customer to our new streaming service. The customer selects their chosen streaming package and clicks submit. Behind the scenes, when the button is clicked in the UI, it hits the Gateway that sits on the perimeter of our system. This in turn passes the call on to the Streaming microservice. This microservice communicates with Payment to take the first payment, uses the Customer microservice to update

the fact that this customer now has streaming enabled, and sends an email to the customer using our Email microservice confirming they are now subscribed.

Với một số lượng lớn dịch vụ tương tác để cung cấp bất kỳ năng lực nào cho người dùng cuối, một lời gọi khởi tạo duy nhất có thể rất cuộc sinh ra nhiều lời gọi dịch vụ downstream. Ví dụ, hãy xét một tình huống ở MusicCorp, như minh họa trong Hình 10-5 . Chúng ta đang đăng ký một khách hàng vào dịch vụ streaming mới của mình. Khách hàng chọn gói streaming họ muốn và nhấn submit. Đằng sau hậu trường, khi nút được nhấn trong UI, nó chạm tới Gateway nằm ở vành đai của hệ thống. Cái này lần lượt chuyển lời gọi tới microservice Streaming . Microservice này giao tiếp với Payment để thu khoản thanh toán đầu tiên, dùng microservice Customer để cập nhật sự kiện khách hàng này giờ đã bật streaming, và gửi một email cho khách hàng bằng microservice Email của chúng ta để xác nhận họ đã đăng ký.

What happens if the call to the Payment microservice ends up generating an odd error? We'll talk at length about handling failures in Chapter 12 , but consider the difficulty of diagnosing what happened.

Điều gì xảy ra nếu lời gọi tới microservice Payment rất cuộc sinh ra một lỗi kỳ lạ? Chúng ta sẽ bàn dài về xử lý sự cố trong Chương 12 , nhưng hãy xét đến cái khó của việc chẩn đoán chuyện gì đã xảy ra.

The problem is that the only microservice registering an error is our Payment microservice. If we are lucky, we can work out what request caused the problem, and we may even be able to look at the parameters of the call. But we can't see this error in the wider context in which it occurs. In this particular example, even if we assume that each interaction generates only a single log line, we'd have five log lines with information about this flow of calls. Being able to see these log lines grouped together can be incredibly useful.

Vấn đề là microservice duy nhất ghi nhận lỗi là microservice Payment của chúng ta. Nếu may mắn, chúng ta có thể tìm ra yêu cầu nào gây ra vấn đề, và thậm chí có thể xem các tham số của lời gọi. Nhưng chúng ta không thể thấy lỗi này trong bối cảnh rộng hơn nơi nó diễn ra. Trong ví dụ cụ thể này, ngay cả khi ta giả định mỗi tương tác chỉ sinh ra một dòng log duy nhất, chúng ta sẽ có năm dòng log chứa thông tin về luồng lời gọi này. Có khả năng thấy các dòng log này được nhóm lại với nhau có thể cực kỳ hữu ích.

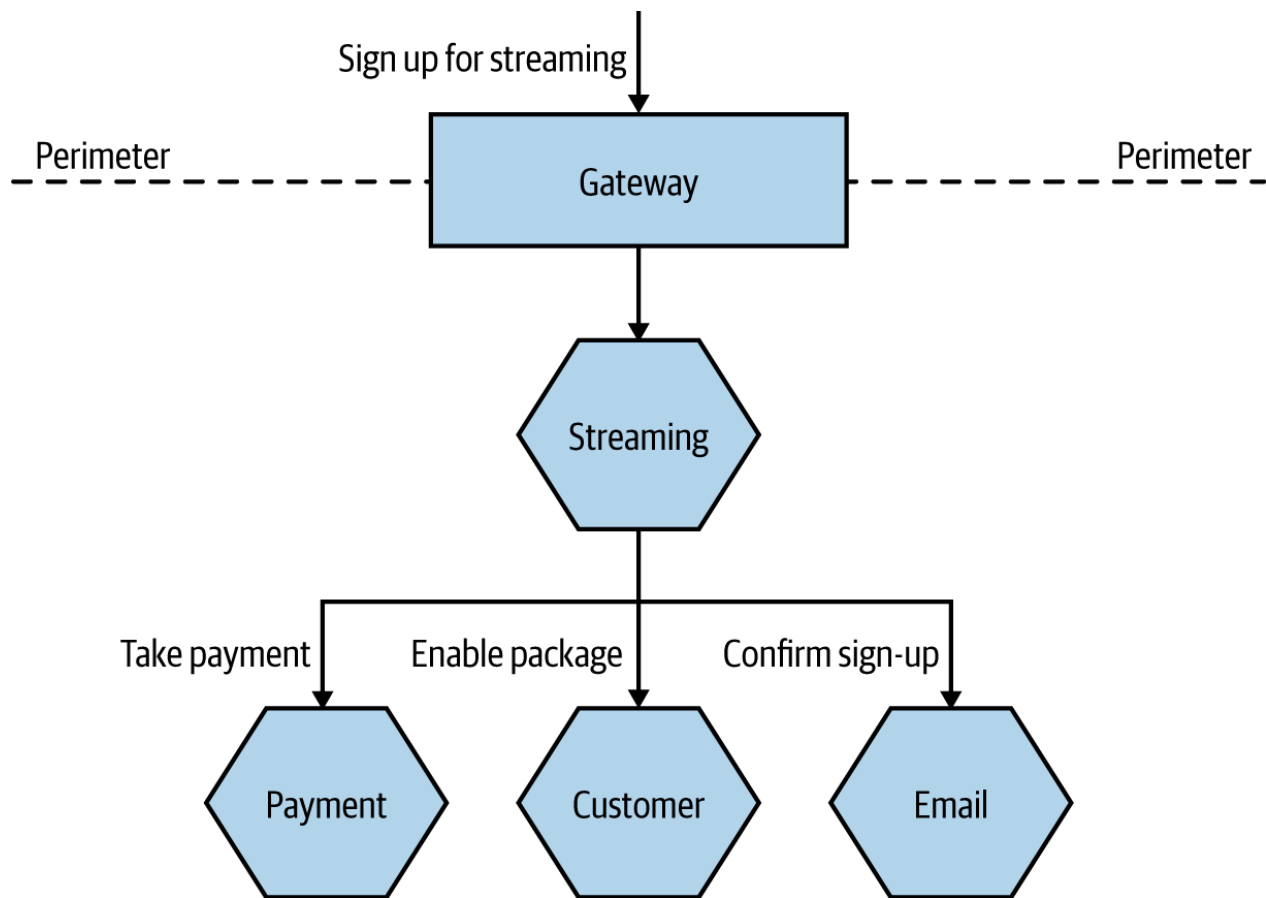


Figure 10-5. A series of calls across multiple microservices related to registering a customer

Hình 10-5. Một loạt lời gọi qua nhiều microservice liên quan đến việc đăng ký một khách hàng

One approach that can be useful here is to use correlation IDs—something we first mentioned in Chapter 6 when discussing sagas. When the first call is made, you generate a unique ID that will be used to correlate all subsequent calls related to the request. In Figure 10-6, we generate this ID in the Gateway, and it is then passed along as a parameter to all subsequent calls.

Một cách tiếp cận có thể hữu ích ở đây là dùng các correlation ID—thứ chúng ta đã đề cập lần đầu trong Chương 6 khi bàn về saga. Khi lời gọi đầu tiên được thực hiện, bạn sinh ra một ID duy nhất sẽ được dùng để tương quan tất cả các lời gọi tiếp theo liên quan đến yêu cầu. Trong Hình 10-6, chúng ta sinh ID này trong Gateway, và nó sau đó được truyền đi như một tham số tới tất cả các lời gọi tiếp theo.

Logging of any activity by a microservice caused by this incoming call will be recorded along with the same correlation ID, which we place in a consistent location in each log line, as shown in Example 10-2. This makes it easy to extract all logs associated with a given correlation ID at a later date.

Việc ghi log bất kỳ hoạt động nào của một microservice gây ra bởi lời gọi đến này sẽ được ghi lại kèm theo cùng một correlation ID, mà chúng ta đặt ở một vị trí nhất quán trong mỗi dòng log, như minh họa trong Ví dụ 10-2. Điều này giúp dễ dàng trích xuất tất cả log gắn với một correlation ID nhất định vào một thời điểm sau này.

Example 10-2. Using a correlation ID in a fixed location in a log line

Ví dụ 10-2. Dùng một correlation ID ở vị trí cố định trong dòng log

15-02-2020 16:01:01 Gateway INFO [abc-123] Signup for streaming 15-02-2020 16:01:02 Streaming INFO [abc-123] Cust 773 signs up ... 15-02-2020 16:01:03 Customer INFO [abc-123] Streaming package added ... 15-02-2020 16:01:03 Email INFO [abc-123] Send streaming welcome ... 15-02-2020 16:01:03 Payment ERROR [abc-123] ValidatePayment ...

15-02-2020 16:01:01 Gateway INFO [abc-123] Signup for streaming 15-02-2020 16:01:02 Streaming INFO [abc-123] Cust 773 signs up ... 15-02-2020 16:01:03 Customer INFO [abc-123] Streaming package added ... 15-02-2020 16:01:03 Email INFO [abc-123] Send streaming welcome ... 15-02-2020 16:01:03 Payment ERROR [abc-123] ValidatePayment ...

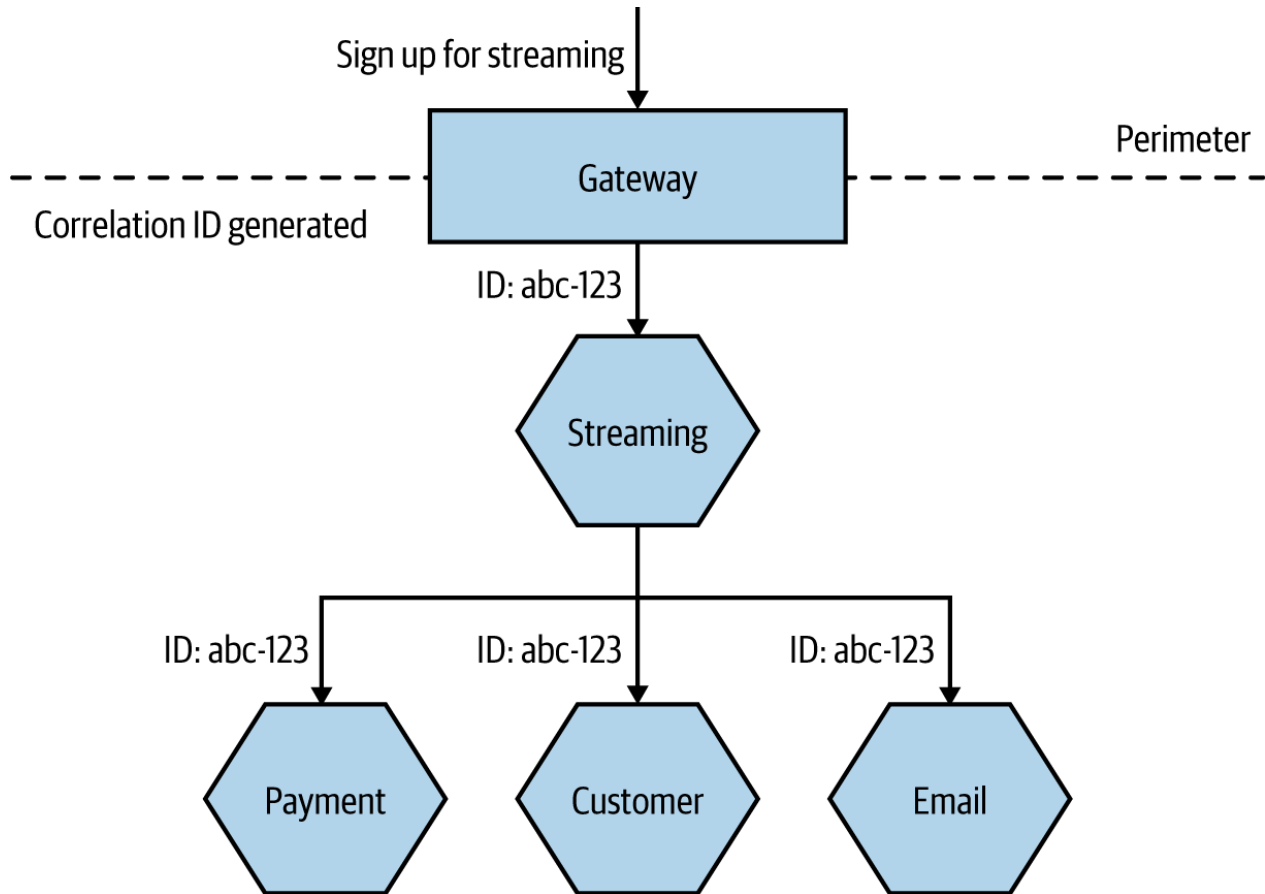


Figure 10-6. Generating a correlation ID for a set of calls

Hình 10-6. Sinh một correlation ID cho một tập lời gọi

You will, of course, need to ensure that each service knows to pass on the correlation ID. This is where you need to standardize and be stronger in enforcing this across your system. But once you have done this, you can actually create tooling to track all sorts of interactions. Such tooling can be useful in tracking down event storms or odd corner cases, or even in identifying especially costly transactions, as you can picture the whole cascade of calls.

Bạn sẽ, dĩ nhiên, cần đảm bảo rằng mỗi dịch vụ biết phải truyền tiếp correlation ID. Đây là chỗ bạn cần chuẩn hóa và cứng rắn hơn trong việc thực thi điều này xuyên suốt hệ thống. Nhưng một khi bạn đã làm được điều này, bạn thực sự có thể tạo ra công cụ để theo dõi đủ loại tương tác. Công cụ như vậy có thể hữu ích trong việc truy tìm các cơn bão event hay các tình huống biên kỳ lạ, hoặc thậm chí trong việc xác định các giao dịch đặc biệt tốn

kém, vì bạn có thể hình dung toàn bộ thác lời gọi.

Correlation IDs in logs are the sorts of things that don't seem that useful initially, but trust me—over time, they can be incredibly useful. Unfortunately, they can be a pain to retrofit into a system. It's for this reason I strongly suggest that you implement correlation IDs in logging as early as possible. Logs can of course only take you so far in this regard—some sorts of problems are better solved by distributed tracing tools, which we'll explore shortly. But a simple correlation ID in a logfile can be incredibly useful initially, meaning you can defer the use of a dedicated tracing tool until your system is complex enough to warrant it.

Các correlation ID trong log là loại thứ thoát đầu trông không hữu ích lắm, nhưng tin tôi đi—theo thời gian, chúng có thể cực kỳ hữu ích. Tiếc thay, chúng có thể là một nỗi đau khi cài ghép vào một hệ thống đã có sẵn. Chính vì lý do này, tôi mạnh mẽ khuyên bạn nên triển khai correlation ID trong việc ghi log càng sớm càng tốt. Dĩ nhiên log chỉ đưa bạn đi xa được đến một mức trong chuyện này—một số loại vấn đề được giải quyết tốt hơn bằng các công cụ distributed tracing, mà chúng ta sẽ khám phá ngay sau đây. Nhưng một correlation ID đơn giản trong tập tin log có thể cực kỳ hữu ích ban đầu, nghĩa là bạn có thể trì hoãn việc dùng một công cụ tracing chuyên dụng cho đến khi hệ thống của bạn đủ phức tạp để đáng phải dùng.



Once you have log aggregation, get correlation IDs in as soon as possible. Easy to do at the start and hard to retrofit later, they will drastically improve the value of your logs.

Một khi bạn đã có gom log, hãy đưa correlation ID vào sớm nhất có thể. Dễ làm lúc đầu và khó cài ghép về sau, chúng sẽ cải thiện đáng kể giá trị của log của bạn.

Timing

Định thời

When looking at a list of log lines, we can be fooled into thinking we are seeing an accurate chronology that will help us understand what happened and in what order. After all, each line of our logs includes a date and time—so why can't we use this as a way to determine the order in which things occurred? In the sequence of calls in Example 10-2, we see a log line from Gateway, followed by logs from Streaming, Customer, Email, and then the Payment microservice. We might conclude that this is the order in which calls actually happened. Unfortunately, we can't always count on that to be true.

Khi nhìn vào một danh sách các dòng log, chúng ta có thể bị đánh lừa rằng mình đang thấy một dòng thời gian chính xác giúp ta hiểu điều gì đã xảy ra và theo thứ tự nào. Suy cho cùng, mỗi dòng log của chúng ta bao gồm một ngày và giờ—vậy tại sao ta không thể dùng điều này như một cách để xác định thứ tự các sự việc diễn ra? Trong chuỗi lời gọi ở Ví dụ 10-2, chúng ta thấy một dòng log từ Gateway, tiếp theo là log từ Streaming, Customer, Email, rồi đến microservice Payment. Chúng ta có thể kết luận rằng đây là thứ tự các lời gọi thực sự diễn ra. Tiếc thay, chúng ta không phải lúc nào cũng có thể trông cậy vào điều đó là đúng.

The log lines are generated on the machines where these microservice instances are running. After being written locally, at some point these logs are forwarded. This means that the date stamps in the log lines are generated on the machines where the microservices are running. Unfortunately, we cannot guarantee that the clocks on these different machines are in sync. This means that the clock on the machine where the Email microservice is running might be a few seconds ahead of the clock on the machine where Payment is running—this could result in it looking like something happened in the Email microservice before it happened in the Payment microservice, but that could just be because of this clock skew.

Các dòng log được sinh ra trên các máy nơi các instance microservice này đang chạy. Sau khi được ghi cục bộ, tại một thời điểm nào đó các log này được chuyển tiếp. Điều này nghĩa là các dấu thời gian trong các dòng log được sinh ra trên các máy nơi các microservice đang chạy. Tiếc thay, chúng ta không thể đảm bảo rằng đồng hồ trên các máy khác nhau này được đồng bộ. Điều này nghĩa là đồng hồ trên máy nơi microservice Email đang chạy có thể nhanh hơn vài giây so với đồng hồ trên máy nơi Payment đang chạy—điều này có thể dẫn đến việc trông như có gì đó xảy ra trong microservice Email trước khi nó xảy ra trong microservice Payment, nhưng đó có thể chỉ là do sự lệch đồng hồ (clock skew) này.

The problem of clock skew causes all sorts of issues in distributed systems. Protocols do exist to try and reduce clock skew in systems—the Network Time Protocol (NTP) being the most widely used example. NTP, however, isn't guaranteed to work, and even when it does work, all it can do is reduce skew, not eliminate it. If you have a sequence of calls that happen quite close together, you may find that even a second of skew across machines is enough for your understanding of the sequence of calls to be totally changed.

Vấn đề lệch đồng hồ gây ra đủ loại rắc rối trong các hệ thống phân tán. Các giao thức có tồn tại để cố giảm lệch đồng hồ trong hệ thống—Network Time Protocol (NTP) là ví dụ được dùng rộng rãi nhất. Tuy nhiên, NTP không được đảm bảo hoạt động, và ngay cả khi nó hoạt động, tất cả những gì nó làm được là giảm độ lệch, chứ không loại bỏ. Nếu bạn có một chuỗi lời gọi diễn ra khá sát nhau, bạn có thể thấy rằng dù chỉ một giây lệch giữa các máy cũng đủ để hiểu biết của bạn về thứ tự các lời gọi bị thay đổi hoàn toàn.

Fundamentally, this means we have two limitations when it comes to time in logs. We can't get fully

accurate timing information for the overall flow of calls, nor can we understand causality.

Về cơ bản, điều này nghĩa là chúng ta có hai giới hạn khi nói đến thời gian trong log. Chúng ta không thể có thông tin định thời hoàn toàn chính xác cho toàn bộ luồng lời gọi, cũng không thể hiểu được tính nhân quả (causality).

In terms of helping to solve this problem so we can understand the true order of things, Leslie Lamport ⁴ has proposed a logical clock system, where a counter is used to keep track of the order of calls. You could implement a similar scheme if you wanted to, and a number of variations to this scheme exist. Personally, though, if I wanted more accurate information regarding the order of calls, and I also wanted more accurate timing, I'd be more inclined to make use of a distributed tracing tool, which will address both issues for me. We'll look at distributed tracing in more depth later in this chapter.

Để giúp giải quyết vấn đề này nhằm hiểu được thứ tự thực sự của các sự việc, Leslie Lamport ⁴ đã đề xuất một hệ thống đồng hồ logic, trong đó một bộ đếm được dùng để theo dõi thứ tự các lời gọi. Bạn có thể hiện thực một sơ đồ tương tự nếu muốn, và tồn tại một số biến thể của sơ đồ này. Tuy nhiên, cá nhân tôi, nếu muốn thông tin chính xác hơn về thứ tự các lời gọi, và đồng thời muốn định thời chính xác hơn, tôi sẽ nghiêng về việc tận dụng một công cụ distributed tracing, thứ sẽ giải quyết cả hai vấn đề cho tôi. Chúng ta sẽ xem xét distributed tracing sâu hơn ở phần sau của chương này.

⁴ Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM 21, no. 7 (July 1978): 558–65, <https://oreil.ly/qzYmh>.

⁴ Leslie Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” Communications of the ACM 21, no. 7 (July 1978): 558–65, <https://oreil.ly/qzYmh>.

Implementations

Các hiện thực

Few spaces in our industry are as contested as that of log aggregation, and a large variety of solutions exist in this space.

Ít có không gian nào trong ngành của chúng ta lại cạnh tranh gay gắt như không gian gom log, và tồn tại vô số giải pháp trong lĩnh vực này.

A popular open source toolchain for log aggregation has been to make use of a log forwarding agent like Fluentd to send logs to Elasticsearch, using Kibana as a way to slice and dice the resulting stream of logs. The biggest challenge with this stack tends to be the overhead of managing Elasticsearch itself, but this may be less of an issue if you need to run Elasticsearch for other purposes, or if you make use of a managed provider. I will sound two additional notes of caution about the use of this toolchain. Firstly, a lot of effort has gone into marketing Elasticsearch as a **database**. Personally, this has always sat uneasily with me. Taking something that was always billed as a search index and rebranding it as a database can be highly problematic. We implicitly make assumptions about how databases act and behave, and we treat them accordingly, regarding them as a source of truth for vital data. But by design, a search index isn't the source of truth; it's a projection of the source of truth. Elasticsearch has suffered from problems in the past that give me pause for thought. ⁵ While I'm sure many of these issues have been resolved, my own reading of these problems has made me cautious regarding the use of Elasticsearch in certain situations, and certainly when considering it as a database. Having a search index that might occasionally lose data isn't an issue if you can already re-index. But treating it like a database is another thing entirely. If I were using this stack and couldn't afford to lose log information, I would want to ensure that I can re-index the original logs if anything went wrong.

Một chuỗi công cụ mã nguồn mở phổ biến cho gom log là dùng một tác nhân chuyển tiếp

log như Fluentd để gửi log tới Elasticsearch, dùng Kibana như một cách để mở xẻ, phân tích luồng log thu được. Thách thức lớn nhất với stack này có xu hướng là gánh nặng quản lý chính Elasticsearch, nhưng điều này có thể ít là vấn đề hơn nếu bạn cần chạy Elasticsearch cho các mục đích khác, hoặc nếu bạn dùng một nhà cung cấp được quản lý (managed provider). Tôi sẽ gióng thêm hai lời cảnh báo về việc dùng chuỗi công cụ này. Thứ nhất, rất nhiều nỗ lực đã được đổ vào việc tiếp thị Elasticsearch như một cơ sở dữ liệu. Cá nhân tôi luôn thấy điều này không thoải mái. Lấy một thứ luôn được giới thiệu là một chỉ mục tìm kiếm (search index) rồi gắn lại nhãn là một cơ sở dữ liệu có thể rất có vấn đề. Chúng ta ngầm đưa ra các giả định về cách cơ sở dữ liệu hành xử và ứng xử, và đối xử với chúng tương ứng, coi chúng như một nguồn sự thật (source of truth) cho dữ liệu sống còn. Nhưng về thiết kế, một chỉ mục tìm kiếm không phải là nguồn sự thật; nó là một phép chiếu của nguồn sự thật. Elasticsearch đã gặp những vấn đề trong quá khứ khiến tôi phải dè chừng. 5 Dù tôi chắc rằng nhiều vấn đề trong số này đã được giải quyết, việc tự đọc về những vấn đề này khiến tôi thận trọng về việc dùng Elasticsearch trong một số tình huống, và chắc chắn là khi cân nhắc nó như một cơ sở dữ liệu. Có một chỉ mục tìm kiếm thì thoả có thể mất dữ liệu không phải là vấn đề nếu bạn đã có thể lập chỉ mục lại (re-index). Nhưng đối xử với nó như một cơ sở dữ liệu lại là chuyện hoàn toàn khác. Nếu tôi dùng stack này và không thể chấp nhận mất thông tin log, tôi sẽ muốn đảm bảo rằng mình có thể lập chỉ mục lại các log gốc nếu có gì đó trục trặc.

The second set of concerns is less about the technical aspects of Elasticsearch and Kibana and more about the behaviors of Elastic, the company behind these projects. Recently, Elastic made a decision to change the license for the source code for both the core Elasticsearch database and Kibana from a widely used and accepted open source license (Apache 2.0) to a non-open source Server Side Public License (SSPL). 6 The driver for this change in license seems to have been the fact that Elastic is frustrated that organizations like AWS have made successful commercial offerings building on this technology that have undercut Elastic's own commercial offerings. Aside from the concerns that the SSPL can be “viral” in nature (in a similar way to the GNU General Public License), this decision has left many incensed. Well over a thousand people contributed code to Elasticsearch under the expectation that they were donating to an open source product. There is the added irony that Elasticsearch itself, and therefore much of the Elastic company as a whole, was built on technology from the

Tập lo ngại thứ hai ít liên quan đến các khía cạnh kỹ thuật của Elasticsearch và Kibana mà liên quan nhiều hơn đến hành xử của Elastic, công ty đứng sau các dự án này. Gần đây, Elastic đã quyết định thay đổi giấy phép cho mã nguồn của cả cơ sở dữ liệu lõi Elasticsearch lẫn Kibana từ một giấy phép mã nguồn mở được dùng và chấp nhận rộng rãi (Apache 2.0) sang một giấy phép không phải mã nguồn mở là Server Side Public License (SSPL). 6 Động lực cho sự thay đổi giấy phép này dường như là việc Elastic thất vọng vì các tổ chức như AWS đã xây dựng các sản phẩm thương mại thành công dựa trên công nghệ này, vốn cắt giảm giá so với các sản phẩm thương mại của chính Elastic. Bên cạnh những lo ngại rằng SSPL có thể mang tính “lây lan” (theo cách tương tự GNU General Public License), quyết định này đã khiến nhiều người phẫn nộ. Hơn một nghìn người đã đóng góp mã cho Elasticsearch với kỳ vọng rằng họ đang đóng góp cho một sản phẩm mã nguồn mở. Còn có sự trớ trêu thêm vào là chính Elasticsearch, và do đó phần lớn công ty Elastic nói chung, được xây dựng trên công nghệ từ dự án mã nguồn mở

5 See the analysis Kyle Kingsbury did on Elasticsearch 1.1.0 in “Jepsen: Elasticsearch,” <https://oreil.ly/uO9wU> and on Elasticsearch 1.5.0 in “Jepsen: Elasticsearch 1.5.0,” <https://oreil.ly/8fBct>.

5 Xem phân tích mà Kyle Kingsbury thực hiện trên Elasticsearch 1.1.0 trong “Jepsen: Elasticsearch,” <https://oreil.ly/uO9wU> và trên Elasticsearch 1.5.0 trong “Jepsen: Elasticsearch

1.5.0,” <https://oreil.ly/8fBCt> .

6 Renato Losio, “Elastic Changes Licences for Elasticsearch and Kibana: AWS Forks Both,” InfoQ, January 25, 2021, <https://oreil.ly/VdWzD> .

6 Renato Losio, “Elastic Changes Licences for Elasticsearch and Kibana: AWS Forks Both,” InfoQ, January 25, 2021, <https://oreil.ly/VdWzD> .

Lucene open source project. At the time of writing, AWS had predictably committed to creating and maintaining an open source fork of both Elasticsearch and Kibana under the previously used open source Apache 2.0 license.

Lucene. Tại thời điểm viết, AWS đã, đúng như dự đoán, cam kết tạo ra và duy trì một bản fork mã nguồn mở của cả Elasticsearch lẫn Kibana dưới giấy phép mã nguồn mở Apache 2.0 từng được dùng trước đó.

In many ways, Kibana was a laudable attempt to create an open source alternative to expensive commercial options like Splunk. As good as Splunk seems to be, every Splunk customer I’ve spoken to has also told me that it can be eye-wateringly expensive, in terms of both the licensing fees and the hardware costs. Many of these customers do see its value, though. That said, there are a wealth of commercial options out there. I’m personally a big fan of Humio , many people like to use Datadog for log aggregation, and you have basic but workable out-of-the-box solutions for log aggregation with some public cloud providers, such as CloudWatch for AWS or Application Insights for Azure.

Theo nhiều cách, Kibana là một nỗ lực đáng khen nhằm tạo ra một giải pháp thay thế mã nguồn mở cho các lựa chọn thương mại đắt đỏ như Splunk. Splunk dù có vẻ tốt thế nào đi nữa, mọi khách hàng Splunk tôi từng nói chuyện cũng đều bảo tôi rằng nó có thể đắt đến chảy nước mắt, cả về phí giấy phép lẫn chi phí phần cứng. Tuy nhiên, nhiều khách hàng trong số này vẫn thấy giá trị của nó. Dù vậy, ngoài kia có vô số lựa chọn thương mại. Cá nhân tôi rất hâm mộ Humio , nhiều người thích dùng Datadog cho gom log, và bạn có các giải pháp gom log cơ bản nhưng dùng được ngay (out-of-the-box) với một số nhà cung cấp public cloud, chẳng hạn CloudWatch cho AWS hay Application Insights cho Azure.

The reality is that you have a huge array of options in this space, from open source to commercial, and from self-hosted to fully hosted. If you want to build a microservice architecture, this isn’t one of the things you should find difficult to get sorted out.

Thực tế là bạn có một mảng lựa chọn khổng lồ trong lĩnh vực này, từ mã nguồn mở đến thương mại, từ tự host đến được host hoàn toàn. Nếu bạn muốn xây dựng một kiến trúc microservice, đây không phải là một trong những thứ bạn thấy khó để giải quyết ổn thỏa.

Shortcomings

Những hạn chế

Logs are a fantastic and easy way to rapidly get information out of your running systems. I remain convinced that for an early-stage microservice architecture, there are few places that will return investment more than logs when it comes to improving visibility of your application in production. They’ll become the lifeblood for information gathering and diagnosis. That said, you do need to be aware of some potentially significant challenges with logs.

Log là một cách tuyệt vời và dễ dàng để nhanh chóng lấy thông tin ra khỏi các hệ thống đang chạy của bạn. Tôi vẫn tin chắc rằng đối với một kiến trúc microservice ở giai đoạn đầu, ít có chỗ nào hoàn vốn đầu tư tốt hơn log khi nói đến việc cải thiện khả năng nhìn thấy ứng dụng của bạn trong production. Chúng sẽ trở thành dòng máu sống cho việc thu thập thông tin và chẩn đoán. Dù vậy, bạn cần ý thức về một số thách thức có thể đáng kể với log.

Firstly, as we've already touched on, due to clock skew they can't always be relied on to help you understand the order in which calls occurred. This clock skew between machines also means the accurate timing of a sequence of calls will be problematic, potentially limiting the logs' usefulness in tracking down latency bottlenecks.

Thứ nhất, như chúng ta đã chạm tới, do lệch đồng hồ, không phải lúc nào cũng có thể trông cậy vào chúng để hiểu thứ tự các lời gọi diễn ra. Sự lệch đồng hồ giữa các máy này cũng nghĩa là việc định thời chính xác cho một chuỗi lời gọi sẽ có vấn đề, có khả năng giới hạn tính hữu ích của log trong việc truy tìm các nút thắt cổ chai về độ trễ.

The main issue with logs, though, is that as you have more microservices and more calls, you end up generating a LOT of data. Loads. Huge quantities. This can result in higher costs in terms of requiring more hardware, and it can also increase the fee you pay to your service provider (some providers charge by use). And depending on how your log aggregation toolchain is built, this can also result in **scaling** challenges. Some log aggregation solutions try and create an index when receiving log data to make queries faster. The problem is that maintaining an index is computationally expensive—and the more logs you receive, and the larger the index grows, the more problematic this can become. This results in the need to be more tailored in what you log to reduce this issue, which in turn can generate more work and which runs the risk that you might put off logging information that would otherwise be valuable. I spoke to a team that managed the Elasticsearch cluster for SaaS-based developer tooling; it found that the largest Elasticsearch cluster it could happily run would only be able to handle six weeks of logging for one of its products, causing the team to constantly have to shift data around to keep things manageable. Part of the reason I like Humio is that this was something its developers built for—rather than maintaining an index, they focus on efficient and scalable ingestion of data with some smart solutions to try and keep query times down.

Tuy nhiên, vấn đề chính với log là khi bạn có nhiều microservice hơn và nhiều lời gọi hơn, bạn rốt cuộc sinh ra MỘT LƯỢNG dữ liệu khổng lồ. Chất đông. Số lượng cực lớn. Điều này có thể dẫn đến chi phí cao hơn về việc cần thêm phần cứng, và nó cũng có thể làm tăng khoản phí bạn trả cho nhà cung cấp dịch vụ (một số nhà cung cấp tính phí theo mức dùng). Và tùy theo cách chuỗi công cụ gom log của bạn được xây dựng, điều này cũng có thể dẫn đến các thách thức về mở rộng. Một số giải pháp gom log cố tạo một chỉ mục khi nhận dữ liệu log để truy vấn nhanh hơn. Vấn đề là việc duy trì một chỉ mục tốn nhiều tính toán—và càng nhận nhiều log, chỉ mục càng lớn, điều này càng trở nên có vấn đề. Điều này dẫn đến nhu cầu phải chọn lọc hơn về những gì bạn ghi log để giảm vấn đề này, mà đến lượt nó có thể sinh thêm công việc và kéo theo nguy cơ bạn có thể bỏ qua việc ghi những thông tin đáng lẽ có giá trị. Tôi từng nói chuyện với một đội quản lý cụm Elasticsearch cho công cụ phát triển dạng SaaS; họ thấy rằng cụm Elasticsearch lớn nhất họ có thể vận hành thoải mái chỉ kham nổi sáu tuần log cho một trong các sản phẩm của họ, khiến đội phải liên tục di chuyển dữ liệu để giữ mọi thứ trong tầm kiểm soát. Một phần lý do tôi thích Humio là vì đây là điều mà các nhà phát triển của nó đã xây dựng sẵn—thay vì duy trì một chỉ mục, họ tập trung vào việc thu nạp dữ liệu hiệu quả và mở rộng được cùng một số giải pháp thông minh để cố giữ thời gian truy vấn ở mức thấp.

Even if you do have a solution that can store the volume of logs you want, these logs can end up containing a lot of valuable and sensitive information. This means that you might have to limit access to the logs (which could further complicate your efforts to have collective **ownership** of your microservices in production), and the logs could be a target for malicious parties. You might therefore need to consider not logging certain types of information (as we'll touch on in “Be frugal” on page 373, if you don't store data, it can't be stolen) to reduce the impact of access by unauthorized parties.

Ngay cả khi bạn có một giải pháp có thể lưu trữ khối lượng log bạn muốn, các log này rốt cuộc có thể chứa rất nhiều thông tin giá trị và nhạy cảm. Điều này nghĩa là bạn có thể

phải giới hạn quyền truy cập vào log (điều này có thể làm phức tạp thêm nỗ lực sở hữu chung các microservice trong production của bạn), và log có thể trở thành mục tiêu cho các bên độc hại. Do đó bạn có thể cần cân nhắc không ghi log một số loại thông tin nhất định (như chúng ta sẽ chạm tới trong “Hãy tần tiện” ở trang 373, nếu bạn không lưu dữ liệu, nó không thể bị đánh cắp) để giảm tác động của việc bị các bên không được phép truy cập.

Metrics Aggregation — Tổng hợp Metric (Metrics Aggregation)

As with the challenge of looking at logs for different hosts, we need to look at better ways to gather and view data about our systems. It can be hard to know what “good” looks like when we’re looking at metrics for a more complex system. Our website is seeing nearly 50 4XX HTTP error codes per second. Is that bad? The CPU load on the catalog service has increased by 20% since lunch; has something gone wrong? The secret to knowing when to panic and when to relax is to gather metrics about how your system behaves over a long-enough period of time that clear patterns emerge.

Cũng như thách thức của việc xem log cho các host khác nhau, chúng ta cần tìm những cách tốt hơn để thu thập và xem dữ liệu về hệ thống của mình. Có thể khó biết “tốt” trông như thế nào khi chúng ta nhìn vào metric cho một hệ thống phức tạp hơn. Website của chúng ta đang thấy gần 50 mã lỗi HTTP 4XX mỗi giây. Như vậy là tệ ư? Tải CPU trên dịch vụ catalog đã tăng 20% kể từ giờ ăn trưa; có gì đó đã sai chăng? Bí quyết để biết khi nào nên hoảng và khi nào nên thư giãn là thu thập metric về cách hệ thống của bạn hành xử trong một khoảng thời gian đủ dài để các mẫu hình (pattern) rõ ràng hiện ra.

In a more complex environment, we’ll be provisioning new instances of our microservices pretty frequently, so we want the system we pick to make it very easy to collect metrics from new hosts. We’ll want to be able to look at a **metric** aggregated for the whole system—for example, the average CPU load—but we’ll also want to aggregate that metric for all the instances of a given service, or even for a single instance of that service. That means we’ll need to be able to associate metadata with the metric to allow us to infer this structure.

Trong một môi trường phức tạp hơn, chúng ta sẽ cấp phát các instance mới của microservice khá thường xuyên, nên chúng ta muốn hệ thống mình chọn khiến việc thu thập metric từ các host mới rất dễ dàng. Chúng ta sẽ muốn có thể xem một metric được tổng hợp cho toàn hệ thống—ví dụ, tải CPU trung bình—nhưng chúng ta cũng sẽ muốn tổng hợp metric đó cho tất cả các instance của một dịch vụ nhất định, hoặc thậm chí cho một instance đơn lẻ của dịch vụ đó. Điều đó nghĩa là chúng ta sẽ cần có thể gắn metadata với metric để cho phép suy ra cấu trúc này.

Another key benefit of understanding your trends is when it comes to capacity planning. Are we reaching our limit? How long until we need more hosts? In the past when we bought physical hosts, this was often an annual job. In the new age of on-demand computing provided by infrastructure as a service (IaaS) vendors, we can scale up or down in minutes, if not seconds. This means that if we understand our usage patterns, we can make sure we have just enough infrastructure to serve our needs. The smarter we are in tracking our trends and knowing what to do with them, the more cost effective and responsive our systems can be.

Một lợi ích then chốt khác của việc hiểu các xu hướng của bạn là khi nói đến việc lập kế hoạch dung lượng (capacity planning). Chúng ta đã chạm tới giới hạn chưa? Còn bao lâu nữa thì cần thêm host? Trước đây khi chúng ta mua các host vật lý, đây thường là một công việc hằng năm. Trong thời đại mới của điện toán theo nhu cầu (on-demand) được cung cấp bởi các nhà cung cấp hạ tầng như một dịch vụ (IaaS), chúng ta có thể mở rộng

lên hoặc xuống trong vài phút, nếu không phải vài giây. Điều này nghĩa là nếu chúng ta hiểu các mẫu sử dụng của mình, chúng ta có thể đảm bảo có vừa đủ hạ tầng để phục vụ nhu cầu. Càng thông minh trong việc theo dõi xu hướng và biết phải làm gì với chúng, hệ thống của chúng ta càng tiết kiệm chi phí và phản ứng nhanh hơn.

Due to the nature of this sort of data, we may want to store and report these metrics at different resolutions. For example, I might want a CPU sample for my servers at the resolution of one sample every 10 seconds for the last 30 minutes, in order to better react to a situation that is currently unfolding. On the other hand, the CPU samples from my servers from last month are likely needed only for general trend analysis, so I might be happy with calculating an average CPU sample on a per-hour basis. This is often done on standard metrics platforms to help keep query times down and also reduce data storage. For something as simple as a CPU rate this might be fine, but the process of aggregating old data does cause us to lose information. The issue with the need for this data to be aggregated is that you often have to decide what to aggregate beforehand—you have to guess in advance as to what information it's OK to lose.

Do bản chất của loại dữ liệu này, chúng ta có thể muốn lưu trữ và báo cáo các metric này ở các độ phân giải (resolution) khác nhau. Ví dụ, tôi có thể muốn một mẫu CPU cho các máy chủ của mình ở độ phân giải một mẫu mỗi 10 giây cho 30 phút vừa qua, để phản ứng tốt hơn với một tình huống đang diễn ra. Mặt khác, các mẫu CPU từ các máy chủ của tôi từ tháng trước có lẽ chỉ cần cho việc phân tích xu hướng chung, nên tôi có thể hài lòng với việc tính một mẫu CPU trung bình trên cơ sở mỗi giờ. Điều này thường được làm trên các nền tảng metric chuẩn để giúp giữ thời gian truy vấn ở mức thấp và cũng giảm dung lượng lưu trữ dữ liệu. Với thứ đơn giản như một tỉ lệ CPU thì điều này có thể ổn, nhưng quá trình tổng hợp dữ liệu cũ khiến chúng ta mất thông tin. Vấn đề với nhu cầu phải tổng hợp dữ liệu này là bạn thường phải quyết định tổng hợp cái gì từ trước—bạn phải đoán trước thông tin nào thì có thể chấp nhận mất.

Standard metrics tools can be absolutely fine for understanding trends or simple failure modes. They can be vital, in fact. But they often don't help us make our systems more observable, as they restrict the sorts of questions we want to ask. Things start getting interesting when we go from simple pieces of information like response time, CPU, or disk space use to thinking more broadly about the types of information we want to capture.

Các công cụ metric chuẩn có thể hoàn toàn ổn để hiểu các xu hướng hay các chế độ hỏng đơn giản. Thực ra chúng có thể là sống còn. Nhưng chúng thường không giúp ta làm cho hệ thống quan sát được hơn, vì chúng giới hạn các loại câu hỏi ta muốn đặt. Mọi thứ bắt đầu trở nên thú vị khi ta đi từ các mẫu thông tin đơn giản như thời gian phản hồi, CPU, hay mức dùng dung lượng đĩa sang suy nghĩ rộng hơn về các loại thông tin ta muốn bắt.

Low versus high cardinality

Cardinality thấp so với cao

Many tools, especially the more recent ones, have been built to accommodate the storage and retrieval of high-cardinality data. There are a number of ways to describe cardinality, but you can think of it as the number of fields that can be easily queried in a given data point. The more potential fields we might want to query of our data, the higher the cardinality we need to support. Fundamentally, this gets more problematic with time-series databases for reasons that I won't expand on here but which are related to the way many of these systems are built.

Nhiều công cụ, đặc biệt là các công cụ gần đây hơn, được xây dựng để hỗ trợ việc lưu trữ và truy xuất dữ liệu cardinality cao. Có một số cách để mô tả cardinality, nhưng bạn có thể coi nó là số lượng trường có thể được truy vấn dễ dàng trong một điểm dữ liệu nhất

định. Càng có nhiều trường tiềm năng mà ta có thể muốn truy vấn từ dữ liệu, cardinality mà ta cần hỗ trợ càng cao. Về cơ bản, điều này càng trở nên có vấn đề với các cơ sở dữ liệu chuỗi thời gian (time-series database) vì những lý do mà tôi sẽ không khai triển ở đây nhưng liên quan đến cách nhiều hệ thống trong số này được xây dựng.

For example, I might want to capture and query on the name of the microservice, customer ID, request ID, build number of software, and product ID, all over time. Then I decide to capture information about the machine at that stage—the OS, system architecture, cloud provider, and so on. I could need to capture all of that information for each data point I collect. As I increase the number of things I might want to query on, the cardinality increases, and the more problems systems will have that aren't built with this use case in mind. As Charity Majors, 7 founder of Honeycomb, explains:

Ví dụ, tôi có thể muốn bắt và truy vấn trên tên microservice, ID khách hàng, ID yêu cầu, số build của phần mềm, và ID sản phẩm, tất cả theo thời gian. Rồi tôi quyết định bắt thông tin về cỗ máy ở giai đoạn đó—OS, kiến trúc hệ thống, nhà cung cấp cloud, v.v. Tôi có thể cần bắt tất cả thông tin đó cho mỗi điểm dữ liệu tôi thu thập. Khi tôi tăng số lượng thứ tôi có thể muốn truy vấn, cardinality tăng lên, và các hệ thống không được xây dựng với trường hợp dùng này trong đầu sẽ càng gặp nhiều vấn đề. Như Charity Majors, 7 nhà sáng lập Honeycomb, giải thích:

7 Charity Majors, “Metrics: Not the Observability Droids You’re Looking For,” Honeycomb (blog), October 24, 2017, <https://oreil.ly/TEETp> .

7 Charity Majors, “Metrics: Not the Observability Droids You’re Looking For,” Honeycomb (blog), October 24, 2017, <https://oreil.ly/TEETp> .

It boils down, essentially, to the metric. The metric is a dot of data, a single number with a name and some identifying tags. All of the context you can get has to be stuffed into those tags. But the write explosion of writing all those tags is expensive because of how metrics are stored on disk. Storing a metric is dirt cheap, but storing a tag is expensive; and storing lots of tags per metric will bring your storage engine to a halt fast. Practically speaking, systems built with low cardinality in mind will struggle greatly if you try to put higher-cardinality data into them. Systems like Prometheus, for example, were built to store fairly simple pieces of information, such as the CPU rate for a given machine. In many ways, we can see Prometheus and similar tools as being a great implementation of traditional metrics storage and querying. But the lack of ability to support higher-cardinality data can be a limiting factor. The Prometheus devs are quite open about this limitation :

Về cơ bản, mọi thứ quy về metric. Metric là một chấm dữ liệu, một con số duy nhất với một cái tên và vài thẻ (tag) định danh. Tất cả bối cảnh bạn có thể lấy được phải được nhồi vào các thẻ đó. Nhưng sự bùng nổ ghi (write explosion) khi ghi tất cả các thẻ đó lại đắt đỏ vì cách metric được lưu trên đĩa. Lưu một metric thì rẻ như bèo, nhưng lưu một thẻ thì đắt; và lưu nhiều thẻ cho mỗi metric sẽ làm bộ máy lưu trữ của bạn khựng lại nhanh chóng. Nói một cách thực tế, các hệ thống được xây dựng với cardinality thấp trong đầu sẽ chật vật rất nhiều nếu bạn cố nhét dữ liệu cardinality cao hơn vào chúng. Các hệ thống như Prometheus, chẳng hạn, được xây dựng để lưu trữ các mẫu thông tin khá đơn giản, như tỉ lệ CPU cho một cỗ máy nhất định. Theo nhiều cách, chúng ta có thể xem Prometheus và các công cụ tương tự là một hiện thực tuyệt vời của việc lưu trữ và truy vấn metric truyền thống. Nhưng việc thiếu khả năng hỗ trợ dữ liệu cardinality cao hơn có thể là một yếu tố giới hạn. Các nhà phát triển Prometheus khá thẳng thắn về giới hạn này :

Remember that every unique combination of key-value label pairs represents a new time series, which can dramatically increase the amount of data stored. Do not use labels to store dimensions with high cardinality (many different label values), such as user IDs, email addresses, or other

unbounded sets of values. Systems that are capable of handling high cardinality are more able to allow you to ask a host of different questions of your systems—often questions you didn’t know you needed to ask beforehand. Now, this might be a hard concept to grasp, especially if you’ve been managing your single-process monolithic system quite happily with more “conventional” tools. Even those people with larger systems have made do with lower-cardinality systems, often because they had no choice. But as your system increases in complexity, you’ll need to improve the quality of the outputs your system provides to allow you to improve its observability. This means collecting more information and having tooling that allows you to slice and dice this data.

Hãy nhớ rằng mỗi tổ hợp duy nhất của các cặp nhãn key-value đại diện cho một chuỗi thời gian mới, điều này có thể làm tăng đáng kể lượng dữ liệu được lưu trữ. Đừng dùng nhãn để lưu các chiều có cardinality cao (nhiều giá trị nhãn khác nhau), chẳng hạn ID người dùng, địa chỉ email, hay các tập giá trị không bị giới hạn khác. Các hệ thống có khả năng xử lý cardinality cao thì có khả năng hơn trong việc cho phép bạn đặt cho hệ thống của mình một loạt câu hỏi khác nhau—thường là những câu hỏi mà bạn không biết trước rằng mình cần đặt. Bây giờ, đây có thể là một khái niệm khó nắm bắt, đặc biệt nếu bạn đã quản lý hệ thống monolith đơn tiến trình của mình khá thoải mái với các công cụ “thông thường” hơn. Ngay cả những người có hệ thống lớn hơn cũng đã xoay sở với các hệ thống cardinality thấp hơn, thường là vì họ không có lựa chọn. Nhưng khi hệ thống của bạn tăng độ phức tạp, bạn sẽ cần cải thiện chất lượng các đầu ra mà hệ thống cung cấp để cho phép bạn cải thiện khả năng quan sát của nó. Điều này nghĩa là thu thập nhiều thông tin hơn và có công cụ cho phép bạn mổ xẻ, phân tích dữ liệu này.

Implementations

Các hiện thực

Since the first edition of this book, Prometheus has become a popular open source tool for the use of gathering and aggregating metrics, and in cases in which I might have previously recommended the use of Graphite (which got my recommendation in the first edition), Prometheus can be a sensible replacement. The commercial space has also greatly expanded in this area, with new vendors and old vendors alike building or retooling existing solutions to target microservice users.

Kể từ phiên bản đầu của cuốn sách này, Prometheus đã trở thành một công cụ mã nguồn mở phổ biến cho việc thu thập và tổng hợp metric, và trong những trường hợp mà trước đây tôi có thể đã khuyên dùng Graphite (thứ tôi đã khuyên dùng trong phiên bản đầu), Prometheus có thể là một sự thay thế hợp lý. Không gian thương mại cũng đã mở rộng rất nhiều trong lĩnh vực này, với cả nhà cung cấp mới lẫn cũ xây dựng hoặc tái trang bị các giải pháp hiện có để nhắm tới người dùng microservice.

Bear in mind, though, my concerns regarding low- versus high-cardinality data. Systems built to handle low-cardinality data will be very difficult to retrofit to support high-cardinality storage and processing. If you are looking for systems that are able to store and manage high cardinality data, allowing for much more sophisticated observation (and questioning) of your system’s behavior, I’d strongly suggest looking at either Honeycomb or Lightstep . Although these tools are often seen as solutions to distributed tracing (which we’ll explore more later), they are highly capable at storing, filtering, and querying high-cardinality data.

Tuy nhiên, hãy ghi nhớ những lo ngại của tôi về dữ liệu cardinality thấp so với cao. Các hệ thống được xây dựng để xử lý dữ liệu cardinality thấp sẽ rất khó cài ghép để hỗ trợ lưu trữ và xử lý cardinality cao. Nếu bạn đang tìm các hệ thống có khả năng lưu trữ và quản lý dữ liệu cardinality cao, cho phép quan sát (và đặt câu hỏi) về hành vi hệ thống của bạn tinh vi hơn nhiều, tôi mạnh mẽ khuyên bạn xem xét hoặc Honeycomb hoặc Lightstep . Mặc dù các công cụ này thường được coi là giải pháp cho distributed tracing (mà chúng ta sẽ

khám phá nhiều hơn sau), chúng có năng lực cao trong việc lưu trữ, lọc và truy vấn dữ liệu cardinality cao.

Monitoring and Observability Systems Are Production Systems — Hệ thống Giám sát và Khả năng quan sát là các Hệ thống Production

With a growing set of tools to help us manage our microservice architecture, we must remember that these tools are themselves production systems. Log aggregation platforms, distributed tracing tools, alerting systems—they all are mission-critical applications that are just as vital as our own software, if not more so. The same degree of diligence needs to be applied in terms of maintaining our production monitoring tools as we apply to the software that we write and maintain.

Với một tập công cụ ngày càng tăng để giúp chúng ta quản lý kiến trúc microservice, chúng ta phải nhớ rằng bản thân các công cụ này cũng là hệ thống production. Các nền tảng gom log, công cụ distributed tracing, hệ thống cảnh báo—tất cả đều là các ứng dụng trọng yếu (mission-critical) quan trọng không kém phần mềm của chính chúng ta, nếu không nói là hơn. Cần áp dụng cùng mức độ cẩn trọng trong việc duy trì các công cụ giám sát production của chúng ta như mức ta áp dụng cho phần mềm mà chúng ta viết và duy trì.

We should also recognize that these tools can become potential vectors of attack from outside parties. At the time of writing, the US government and other organizations around the world are dealing with the discovery of a breach in network management software from SolarWinds. Although the exact nature of the breach is still being explored, it is believed that this is what's known as a supply chain attack. Once installed on customer sites (and SolarWinds is used by 425 of the US Fortune 500 companies), this software allowed malicious parties to gain external access to customers' networks, including the US Treasury's network.

Chúng ta cũng nên nhận ra rằng các công cụ này có thể trở thành các vector tấn công tiềm năng từ các bên bên ngoài. Tại thời điểm viết, chính phủ Mỹ và các tổ chức khác trên khắp thế giới đang đối phó với việc phát hiện một vụ xâm phạm trong phần mềm quản lý mạng từ SolarWinds. Mặc dù bản chất chính xác của vụ xâm phạm vẫn đang được khám phá, người ta tin rằng đây là cái được gọi là tấn công chuỗi cung ứng (supply chain attack). Một khi được cài đặt tại các site khách hàng (và SolarWinds được dùng bởi 425 trong số 500 công ty thuộc Fortune 500 của Mỹ), phần mềm này cho phép các bên độc hại có được quyền truy cập từ bên ngoài vào mạng của khách hàng, bao gồm cả mạng của Bộ Tài chính Mỹ.

Distributed Tracing — Distributed Tracing

So far, I've primarily been talking about collecting information in isolation. Yes, we're aggregating that information, but understanding the wider context in which this information has been captured can be key. Fundamentally, a microservice architecture is a set of processes that work together to carry out some kind of task—we explored the various different ways we can coordinate these activities in Chapter 6. Thus it makes sense, when wanting to understand how our system is actually behaving in a production environment, that we are able to see the relationships between our microservices. This can help us better understand how our system is behaving, assess the impact of an issue, or get better at figuring out what exactly is not working as we'd expect.

Cho đến giờ, tôi chủ yếu nói về việc thu thập thông tin một cách cô lập. Đúng, chúng ta đang tổng hợp thông tin đó, nhưng việc hiểu bối cảnh rộng hơn nơi thông tin này được bắt có thể là then chốt. Về cơ bản, một kiến trúc microservice là một tập các tiến trình cùng làm việc để thực hiện một loại nhiệm vụ nào đó—chúng ta đã khám phá các cách khác nhau mà ta có thể điều phối các hoạt động này trong Chương 6 . Vì vậy việc chúng ta có thể thấy các mối quan hệ giữa các microservice là hợp lý, khi muốn hiểu hệ thống của mình thực sự hành xử thế nào trong môi trường production. Điều này có thể giúp ta hiểu rõ hơn cách hệ thống đang hành xử, đánh giá tác động của một vấn đề, hoặc giải hơn trong việc tìm ra chính xác điều gì không hoạt động như ta kỳ vọng.

As our systems become more complex, it therefore becomes important to have a way of seeing these traces through our systems. We need to be able to pull this disparate data to give us a joined-up view of a set of correlated calls. As we've already seen, doing something simple like putting correlation IDs into our logfiles is a good start, but it's a fairly unsophisticated solution, especially as we'll end up having to create our own custom tooling to help visualize, slice, and dice this data. This is where distributed tracing comes in.

Khi hệ thống của chúng ta trở nên phức tạp hơn, do đó việc có một cách để thấy các trace này xuyên qua hệ thống trở nên quan trọng. Chúng ta cần có thể kéo dữ liệu rời rạc này lại để cho ta một cái nhìn ghép nối của một tập các lời gọi tương quan. Như chúng ta đã thấy, làm điều gì đó đơn giản như đặt correlation ID vào tập tin log là một khởi đầu tốt, nhưng đó là một giải pháp khá thô sơ, đặc biệt vì rốt cuộc chúng ta sẽ phải tự tạo công cụ tùy biến riêng để giúp trực quan hóa và mổ xẻ, phân tích dữ liệu này. Đây là lúc distributed tracing xuất hiện.

How it works

Cách nó hoạt động

Although the exact implementations vary, broadly speaking, distributed tracing tools all work in a similar way. Local activity within a thread is captured in a span . These individual spans are correlated using some unique identifier. The spans are then sent to a central collector, which is able to construct these related spans into a single trace . In Figure 10-7 , we see a picture from Honeycomb that shows a trace across a microservice architecture.

Mặc dù các hiện thực cụ thể khác nhau, nói rộng ra, các công cụ distributed tracing đều hoạt động theo một cách tương tự. Hoạt động cục bộ trong một thread được bắt trong một span . Các span riêng lẻ này được tương quan bằng một định danh duy nhất nào đó. Các span sau đó được gửi tới một bộ thu thập (collector) trung tâm, thứ có thể dựng các span liên quan này thành một trace duy nhất. Trong Hình 10-7 , chúng ta thấy một hình từ Honeycomb cho thấy một trace xuyên qua một kiến trúc microservice.

Query at 5/31 3:35PM > Trace e6ee35b206e1c9e5

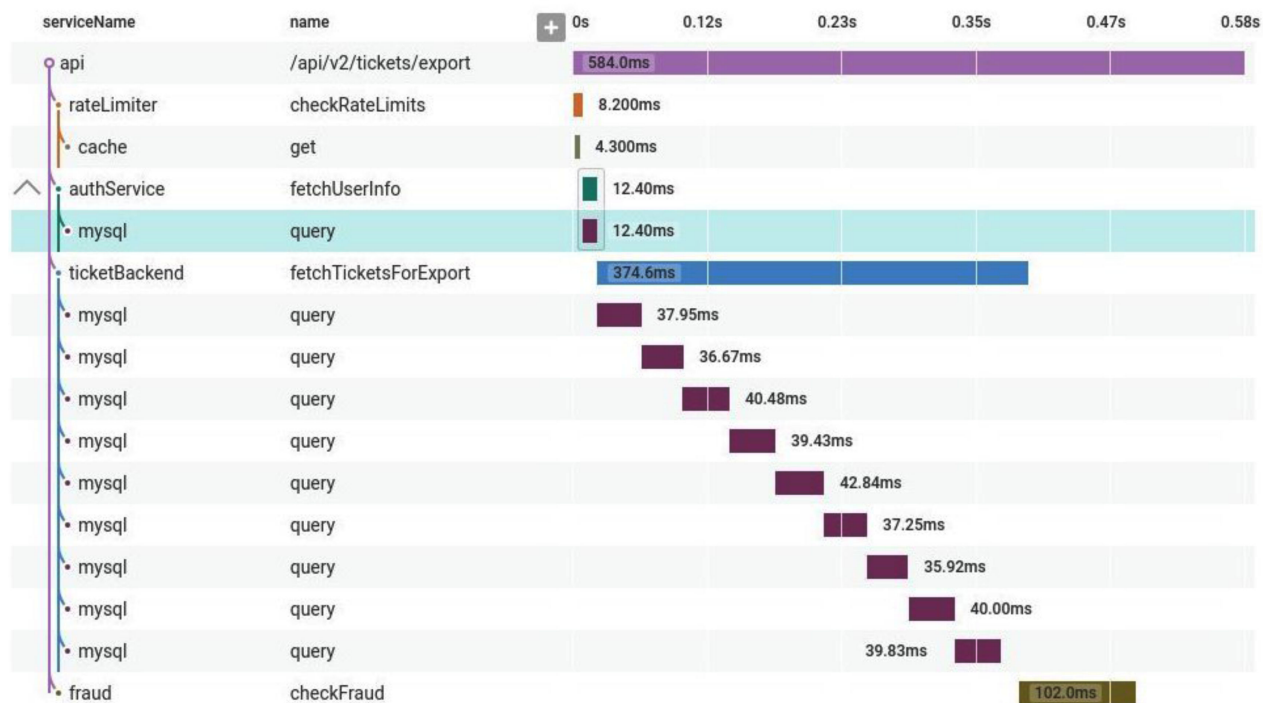


Figure 10-7. A distributed trace shown in Honeycomb, allowing you to identify where time is being spent for operations that can span multiple microservices

Hình 10-7. Một distributed trace được hiển thị trong Honeycomb, cho phép bạn xác định nơi thời gian đang bị tiêu tốn cho các thao tác có thể trải dài qua nhiều microservice

These spans allow you to collect a host of information. Exactly what data you collect will depend on the protocol you are using, but in the case of the OpenTracing API, each span contains a start and end time, a set of logs associated with the span, and an arbitrary set of key-value pairs to help with later querying (these could be used to send things like a customer ID, an order ID, a hostname, a build number, and the like).

Các span này cho phép bạn thu thập một loạt thông tin. Chính xác bạn thu thập dữ liệu gì sẽ phụ thuộc vào giao thức bạn dùng, nhưng trong trường hợp của OpenTracing API, mỗi span chứa một thời điểm bắt đầu và kết thúc, một tập log gắn với span, và một tập tùy ý các cặp key-value để giúp truy vấn về sau (những cái này có thể được dùng để gửi những thứ như ID khách hàng, ID đơn hàng, hostname, số build, và những thứ tương tự).

Collecting enough information to allow us to trace calls in our system can have a direct impact on the system itself. This results in the need for some form of sampling, in which some information is explicitly excluded from our trace collection to ensure the system can still operate. The challenge is ensuring that the right information is dropped, and that we still collect sufficient samples to allow us to correctly extrapolate observations.

Việc thu thập đủ thông tin để cho phép chúng ta lần theo các lời gọi trong hệ thống có thể có tác động trực tiếp lên chính hệ thống. Điều này dẫn đến nhu cầu về một dạng lấy mẫu (sampling) nào đó, trong đó một số thông tin được loại trừ một cách tường minh khỏi việc thu thập trace để đảm bảo hệ thống vẫn có thể vận hành. Thách thức là đảm bảo rằng đúng thông tin bị bỏ đi, và rằng chúng ta vẫn thu thập đủ mẫu để cho phép ngoại suy các quan sát một cách chính xác.

Sampling strategies can be very basic. Google's Dapper system, which inspired many of the distributed tracing tools that came afterwards, performed a highly aggressive random sampling. A certain percentage of calls were sampled, and that was it. Jaeger, for example, will capture only 1 in 1,000 calls in its default setting. The idea here is to capture enough information to understand what our system is doing but not capture so much information that the system itself cannot cope. Tools like Honeycomb and Lightstep can provide more nuanced, dynamic sampling than this simple random-based sampling. An example of dynamic sampling could be where you want more samples for certain types of events—for example, you might want to sample anything that generates an error but would be happy sampling only 1 in 100 successful operations if they are all pretty similar.

Các chiến lược lấy mẫu có thể rất cơ bản. Hệ thống Dapper của Google, thứ đã truyền cảm hứng cho nhiều công cụ distributed tracing ra đời sau đó, thực hiện lấy mẫu ngẫu nhiên cực kỳ quyết liệt. Một tỉ lệ phần trăm nhất định các lời gọi được lấy mẫu, và thế là xong. Jaeger, chẳng hạn, sẽ chỉ bắt 1 trong 1.000 lời gọi ở thiết lập mặc định. Ý tưởng ở đây là bắt đủ thông tin để hiểu hệ thống của ta đang làm gì nhưng không bắt nhiều thông tin đến mức bản thân hệ thống không kham nổi. Các công cụ như Honeycomb và Lightstep có thể cung cấp việc lấy mẫu động (dynamic sampling) tinh tế hơn so với việc lấy mẫu ngẫu nhiên đơn giản này. Một ví dụ về lấy mẫu động có thể là khi bạn muốn nhiều mẫu hơn cho một số loại event nhất định—ví dụ, bạn có thể muốn lấy mẫu bất cứ thứ gì sinh ra lỗi nhưng sẽ hài lòng với việc chỉ lấy mẫu 1 trong 100 thao tác thành công nếu chúng đều khá giống nhau.

Implementing distributed tracing

Triển khai distributed tracing

Getting distributed tracing up and running for your system requires a few things. Firstly, you need to capture span information inside your microservices. If you are using a standard API like OpenTracing or the newer OpenTelemetry API, you might find that some of the third-party libraries and frameworks will come with support for these APIs built in and will already send useful information (for example, automatically capturing information about HTTP calls). But even if they do, chances are you'll still want to instrument your own code, providing useful information about what your microservice is doing at any given point of time.

Để đưa distributed tracing vào vận hành cho hệ thống của bạn cần một vài thứ. Thứ nhất, bạn cần bắt thông tin span bên trong các microservice của mình. Nếu bạn dùng một API chuẩn như OpenTracing hay OpenTelemetry API mới hơn, bạn có thể thấy rằng một số thư viện và framework của bên thứ ba sẽ đi kèm hỗ trợ tích hợp sẵn cho các API này và sẽ đã gửi thông tin hữu ích (ví dụ, tự động bắt thông tin về các lời gọi HTTP). Nhưng ngay cả khi chúng làm vậy, nhiều khả năng bạn vẫn sẽ muốn trang bị (instrument) mã của chính mình, cung cấp thông tin hữu ích về việc microservice của bạn đang làm gì tại bất kỳ thời điểm nào.

Next, you'll need some way to send this span information to your collector—it may be that you send this data directly from your microservice instance to the central collector, but it's much more common to use a local forwarding agent. So as with log aggregation, you run an agent locally to your microservice instance, which will periodically send the span information to a central collector. The use of a local agent typically allows for some more advanced capabilities, such as changing sampling or adding additional tags, and can also more effectively buffer information being sent.

Tiếp theo, bạn sẽ cần một cách nào đó để gửi thông tin span này tới collector của bạn—có thể bạn gửi dữ liệu này trực tiếp từ instance microservice tới collector trung tâm, nhưng phổ biến hơn nhiều là dùng một tác nhân chuyển tiếp cục bộ. Vậy nên cũng như với gom log, bạn chạy một tác nhân cục bộ với instance microservice của mình, thứ sẽ định kỳ gửi

thông tin span tới một collector trung tâm. Việc dùng một tác nhân cục bộ thường cho phép một số năng lực nâng cao hơn, chẳng hạn thay đổi việc lấy mẫu hay thêm các thẻ bổ sung, và cũng có thể đệm (buffer) thông tin được gửi đi hiệu quả hơn.

Lastly, of course, you need a collector able to receive this information and make sense of it all.

Cuối cùng, dĩ nhiên, bạn cần một collector có khả năng nhận thông tin này và giải mã tất cả.

In the open source space, Jaeger has emerged as a popular choice for distributed tracing. For commercial tooling, I'd start by looking at the already mentioned Lightstep and Honeycomb. I'd urge you, though, to pick something that is committed to supporting the OpenTelemetry API. OpenTelemetry is an open API specification that makes it much easier to have code such as database drivers or web frameworks come out of the box with support for tracing, and it can also give you easier portability across different vendors on the collection side. Based on work done by the earlier OpenTracing and OpenConsensus APIs, this API now has broad industry support.

Trong không gian mã nguồn mở, Jaeger đã nổi lên như một lựa chọn phổ biến cho distributed tracing. Về công cụ thương mại, tôi sẽ bắt đầu bằng việc xem xét Lightstep và Honeycomb đã đề cập. Tuy nhiên, tôi thúc giục bạn chọn thứ gì đó cam kết hỗ trợ OpenTelemetry API. OpenTelemetry là một đặc tả API mở giúp dễ hơn nhiều để có mã như driver cơ sở dữ liệu hay web framework đi kèm hỗ trợ tracing ngay từ đầu, và nó cũng có thể cho bạn khả năng dễ chuyển đổi (portability) hơn giữa các nhà cung cấp khác nhau ở phía thu thập. Dựa trên công việc được thực hiện bởi các API OpenTracing và OpenConsensus trước đó, API này nay đã có sự ủng hộ rộng rãi của ngành.

Are We Doing OK? — Chúng ta Đang Ổn chứ? (Are We Doing OK?)

We've talked a lot about the things you could be doing as the operator of a system—the mindset you need, the information you might need to gather. But how do you know if you're doing too much—or not enough? How do you know if you are doing a good enough job, or that your system is working well enough?

Chúng ta đã nói rất nhiều về những điều bạn có thể làm với tư cách người vận hành một hệ thống—tư duy bạn cần, thông tin bạn có thể cần thu thập. Nhưng làm sao bạn biết liệu mình đang làm quá nhiều—hay chưa đủ? Làm sao bạn biết liệu mình đang làm đủ tốt, hay rằng hệ thống của bạn đang hoạt động đủ tốt?

Binary concepts of a system being “up” or “down” start to have less and less meaning as the system becomes more complex. With a single-process monolithic system, it's easier to see system health as a black-and-white quality. But what about a distributed system? If one instance of a microservice is unreachable, is that a problem? Is a microservice “healthy” if it is reachable? What about a situation in which our Returns microservice is available, but half the functionality it provides requires the use of the downstream Inventory microservice, which is currently experiencing issues? Does that mean we consider the Returns microservice healthy or not healthy?

Các khái niệm nhị phân về việc một hệ thống “lên” (up) hay “xuống” (down) bắt đầu ngày càng ít ý nghĩa khi hệ thống trở nên phức tạp hơn. Với một hệ thống monolith đơn tiến trình, dễ hơn để thấy sức khỏe hệ thống như một thuộc tính trắng-đen. Nhưng còn một hệ thống phân tán thì sao? Nếu một instance của một microservice không thể tiếp cận được, đó có phải là vấn đề không? Một microservice có “khỏe mạnh” không nếu nó tiếp cận được? Còn tình huống mà microservice Returns của chúng ta khả dụng, nhưng một nửa chức năng nó cung cấp yêu cầu dùng microservice Inventory downstream, thứ hiện

đang gặp trục trặc thì sao? Điều đó nghĩa là chúng ta coi microservice Returns là khỏe mạnh hay không khỏe mạnh?

As things become more complex, it becomes increasingly important to take a step back and think about things from a different vantage point. Think of a beehive. You could look at an individual bee and determine that it's not happy. Perhaps it's lost one of its wings and thus can no longer fly. This is certainly a problem for that individual bee, but can you extend from that any observation about the health of the hive itself? No—you'd need to look at the health of the hive in a more holistic fashion. One bee being sick doesn't mean the whole hive is sick.

Khi mọi thứ trở nên phức tạp hơn, ngày càng quan trọng để lùi lại một bước và suy nghĩ về mọi thứ từ một góc nhìn khác. Hãy nghĩ về một tổ ong. Bạn có thể nhìn vào một con ong riêng lẻ và xác định rằng nó không vui. Có lẽ nó đã mất một cánh và do đó không thể bay nữa. Điều này chắc chắn là một vấn đề với con ong riêng lẻ đó, nhưng từ đó bạn có thể rút ra bất kỳ quan sát nào về sức khỏe của chính tổ ong không? Không—bạn sẽ cần nhìn vào sức khỏe của tổ ong theo một cách toàn diện hơn. Một con ong bị ốm không có nghĩa là cả tổ bị ốm.

We can try to work out if a service is healthy by deciding, for example, what a good CPU level is, or what makes for an acceptable response time. If our monitoring system detects that the actual values fall outside this safe level, we can trigger an alert. However, in many ways, these values are one step removed from what we actually want to track—namely, is the system working? The more complex the interactions between the services, the further removed we are from actually answering that question by just looking at one metric in isolation.

Chúng ta có thể cố tìm ra một dịch vụ có khỏe mạnh không bằng cách quyết định, chẳng hạn, mức CPU tốt là bao nhiêu, hay thế nào là một thời gian phản hồi chấp nhận được. Nếu hệ thống giám sát của chúng ta phát hiện rằng các giá trị thực tế nằm ngoài mức an toàn này, chúng ta có thể kích hoạt một cảnh báo. Tuy nhiên, theo nhiều cách, các giá trị này cách một bước so với cái chúng ta thực sự muốn theo dõi—cụ thể là, hệ thống có đang hoạt động không? Tương tác giữa các dịch vụ càng phức tạp, chúng ta càng cách xa việc thực sự trả lời câu hỏi đó bằng cách chỉ nhìn vào một metric một cách cô lập.

So we can gather a lot of information, but by itself that doesn't help us answer the question of whether the system is working properly. For that, we need to start thinking a bit more in terms of defining what acceptable behavior looks like. Much work has been done in the space of site reliability engineering (SRE), the focus of which is how we can ensure that our systems can be reliable while still allowing for change. From this space, we have a few useful concepts to explore.

Vậy chúng ta có thể thu thập rất nhiều thông tin, nhưng tự bản thân nó điều đó không giúp ta trả lời câu hỏi liệu hệ thống có đang hoạt động đúng không. Để làm được điều đó, chúng ta cần bắt đầu suy nghĩ nhiều hơn theo hướng định nghĩa thế nào là hành vi chấp nhận được. Rất nhiều công việc đã được làm trong không gian kỹ thuật độ tin cậy của site (site reliability engineering—SRE), trọng tâm của nó là cách chúng ta có thể đảm bảo hệ thống của mình có thể tin cậy được trong khi vẫn cho phép thay đổi. Từ không gian này, chúng ta có một vài khái niệm hữu ích để khám phá.

Strap in—we're about to enter acronym city.

Thắt dây an toàn vào—chúng ta sắp bước vào thành phố của các từ viết tắt.

Service-level agreement

Thỏa thuận mức dịch vụ (Service-level agreement)

A service-level agreement (SLA) is an agreement reached between the people building the system and the people using the system. It describes not only what the users can expect but also what happens if the system doesn't reach this level of acceptable behavior. SLAs tend to be very much at the “bare minimum” level of things, often to the point that if the system only just achieved its targets, the end user would still be unhappy. As an example, AWS has an SLA for its compute service. It makes it clear there is no effective guarantee for uptime for a single EC2 instance (a managed virtual machine 8). AWS states that it makes its best efforts to ensure 90% uptime for a given instance, but if that isn't achieved, then it just doesn't charge you for the given hour in which the instance was unavailable. Now, if your EC2 instances were constantly failing to reach 90% availability in a given hour, causing significant system instability, you might not be getting charged, but you also wouldn't be very happy. In my experience, AWS in practice achieves far in excess of what the SLA outlines, as is often the case with SLAs.

Một thỏa thuận mức dịch vụ (service-level agreement—SLA) là một thỏa thuận đạt được giữa những người xây dựng hệ thống và những người dùng hệ thống. Nó mô tả không chỉ những gì người dùng có thể kỳ vọng mà còn điều gì xảy ra nếu hệ thống không đạt mức hành vi chấp nhận được này. SLA có xu hướng nằm ở mức “tối thiểu trần trụi” của mọi thứ, thường đến mức nếu hệ thống chỉ vừa khít đạt mục tiêu của nó, người dùng cuối vẫn sẽ không hài lòng. Ví dụ, AWS có một SLA cho dịch vụ điện toán của họ. Nó nói rõ rằng không có bảo đảm hiệu lực nào về thời gian hoạt động (uptime) cho một instance EC2 đơn lẻ (một máy ảo được quản lý 8). AWS tuyên bố rằng họ nỗ lực hết sức để đảm bảo 90% uptime cho một instance nhất định, nhưng nếu điều đó không đạt được, thì họ chỉ đơn giản không tính phí bạn cho giờ cụ thể mà instance không khả dụng. Bây giờ, nếu các instance EC2 của bạn liên tục không đạt 90% khả dụng trong một giờ nhất định, gây bất ổn hệ thống đáng kể, bạn có thể không bị tính phí, nhưng bạn cũng sẽ không vui chút nào. Theo kinh nghiệm của tôi, trên thực tế AWS đạt được vượt xa những gì SLA phác ra, như thường thấy với các SLA.

Service-level objectives

Mục tiêu mức dịch vụ (Service-level objectives)

Mapping an SLA down to a team is problematic, especially if the SLA is somewhat broad and cross-cutting. At a team level, we instead talk about service-level objectives (SLOs). SLOs define what the team signs up to provide. Achieving the SLOs of every team across the organization will satisfy (and likely greatly exceed) the requirements of the organization's SLAs. Example SLOs could include things like expected uptime or acceptable response times for a given operation.

Ánh xạ một SLA xuống một đội là điều có vấn đề, đặc biệt nếu SLA hơi rộng và mang tính xuyên suốt. Ở cấp đội, thay vào đó chúng ta nói về các mục tiêu mức dịch vụ (service-level objectives—SLO). SLO định nghĩa những gì đội cam kết cung cấp. Việc đạt được các SLO của mọi đội trong toàn tổ chức sẽ thỏa mãn (và rất có thể vượt xa) các yêu cầu của các SLA của tổ chức. Các SLO ví dụ có thể bao gồm những thứ như uptime kỳ vọng hay thời gian phản hồi chấp nhận được cho một thao tác nhất định.

It is too simplistic to think of SLOs as being what a team needs to do for the organization to achieve its SLAs. Yes, if the entire organization achieves all of its SLOs, we would assume that all the SLAs have also been achieved, but SLOs can speak to other goals not outlined in the SLA—or they might be aspirational, they could be inward facing (trying to carry out some internal change). SLOs can often reflect something that the team itself wants to achieve that may have no relation to an SLA.

Thật quá đơn giản khi nghĩ về SLO như là những gì một đội cần làm để tổ chức đạt được các SLA của nó. Đúng, nếu toàn bộ tổ chức đạt được tất cả các SLO của nó, chúng ta sẽ giả định rằng tất cả các SLA cũng đã được đạt, nhưng các SLO có thể hướng tới các mục tiêu

khác không được phác trong SLA—hoặc chúng có thể mang tính khát vọng, chúng có thể hướng nội (cố thực hiện một thay đổi nội bộ nào đó). SLO thường có thể phản ánh điều gì đó mà bản thân đội muốn đạt được mà có thể không liên quan gì đến một SLA.

Service-level indicators

Chỉ báo mức dịch vụ (Service-level indicators)

To determine if we are meeting our SLOs, we need to gather real data. This is what our service-level indicators (SLI) are. An SLI is a measure of something our software does. It could, for example, be a response time from a process, a customer being registered, an error being raised to the customer, or an order being placed. We need to collect and surface these SLIs to make sure we are meeting our SLOs.

Để xác định liệu chúng ta có đang đạt các SLO của mình không, chúng ta cần thu thập dữ liệu thật. Đây là cái mà các chỉ báo mức dịch vụ (service-level indicators—SLI) của chúng ta là. Một SLI là một phép đo về một điều gì đó phần mềm của chúng ta làm. Nó có thể, chẳng hạn, là thời gian phản hồi từ một tiến trình, một khách hàng được đăng ký, một lỗi được nêu ra cho khách hàng, hay một đơn hàng được đặt. Chúng ta cần thu thập và phơi bày các SLI này để đảm bảo mình đang đạt các SLO.

Error budgets

Ngân sách lỗi (Error budgets)

When we try new things, we inject more potential instability into our systems. As such, a desire to maintain (or improve) system stability could result in change being discouraged. Error budgets are an attempt to avoid this problem by being clear about how much error is acceptable in a system.

Khi chúng ta thử những điều mới, chúng ta tiêm thêm bất ổn tiềm năng vào hệ thống của mình. Như vậy, một mong muốn duy trì (hay cải thiện) sự ổn định của hệ thống có thể dẫn đến việc nản lòng với thay đổi. Ngân sách lỗi là một nỗ lực nhằm tránh vấn đề này bằng cách làm rõ về việc bao nhiêu lỗi là chấp nhận được trong một hệ thống.

8 Mostly. AWS does now provide bare metal instances, which messes with my brain a bit.

8 Phần lớn là vậy. AWS giờ đây cũng cung cấp các instance bare metal, điều này làm não tôi hơi rối một chút.

If you have already decided on an SLO, then working out your error budget should be pretty clear. For example, you might say that your microservice needs to be available 99.9% of the time per quarter, on a 24/7 basis. That means you are actually allowed to be down for 2 hours and 11 minutes per quarter. In terms of that SLO, this is your error budget.

Nếu bạn đã quyết định một SLO, thì việc tính ra ngân sách lỗi của bạn sẽ khá rõ ràng. Ví dụ, bạn có thể nói rằng microservice của bạn cần khả dụng 99,9% thời gian mỗi quý, trên cơ sở 24/7. Điều đó nghĩa là bạn thực sự được phép sập trong 2 giờ 11 phút mỗi quý. Xét theo SLO đó, đây là ngân sách lỗi của bạn.

Error budgets help give you a clear understanding of how well you are achieving (or not) an SLO, allowing you to make better decisions about what risks to take. If you are well under your error budget for the quarter, perhaps you are OK with rolling out that microservice written in a new programming language. If you've already exceeded your error budget, maybe you put off that rollout, and instead focus more of the team's time on improving the reliability of your system.

Ngân sách lỗi giúp cho bạn một sự hiểu biết rõ ràng về việc bạn đang đạt (hay không) một SLO tốt thế nào, cho phép bạn ra quyết định tốt hơn về việc chấp nhận rủi ro nào. Nếu bạn đang ở dưới ngân sách lỗi của mình khá nhiều trong quý, có lẽ bạn ổn với việc tung ra

microservice được viết bằng một ngôn ngữ lập trình mới. Nếu bạn đã vượt ngân sách lỗi rồi, có lẽ bạn hoãn việc tung ra đó, và thay vào đó tập trung nhiều thời gian của đội hơn vào việc cải thiện độ tin cậy của hệ thống.

Error budgets are as much about giving teams breathing room to try new things as anything.

Ngân sách lỗi cũng chủ yếu là về việc cho các đội khoảng thở để thử những điều mới như bất cứ điều gì khác.

Alerting — Cảnh báo (Alerting)

On occasion (hopefully rarely, but likely more than we'd like), something will happen in our systems that will require a human operator to be notified to take action. A microservice may have unexpectedly become unavailable, we might be seeing a larger number of errors than we'd expect, or perhaps the entire system has become unavailable for our users. In these situations, we need people to be made aware of what's happening so they can try and fix things.

Đôi khi (hy vọng là hiếm khi, nhưng có lẽ thường hơn ta muốn), điều gì đó sẽ xảy ra trong hệ thống của chúng ta đòi hỏi một operator con người phải được thông báo để hành động. Một microservice có thể bất ngờ trở nên không khả dụng, chúng ta có thể đang thấy một số lượng lỗi lớn hơn ta kỳ vọng, hoặc có lẽ toàn bộ hệ thống đã trở nên không khả dụng cho người dùng của chúng ta. Trong những tình huống này, chúng ta cần người được làm cho biết về việc đang xảy ra để họ có thể cố sửa chữa.

The problem is that with a microservice architecture, given the larger number of calls, the larger number of processes, and the more complex underlying infrastructure, there will often be things going wrong. The challenge in a microservice environment is working out exactly what types of problems should cause a human to be informed and exactly how they should be told.

Vấn đề là với một kiến trúc microservice, do số lượng lời gọi lớn hơn, số lượng tiến trình lớn hơn, và hạ tầng nền tảng phức tạp hơn, sẽ thường có những thứ trục trặc. Thách thức trong một môi trường microservice là tìm ra chính xác những loại vấn đề nào nên khiến một con người được thông báo và chính xác họ nên được báo như thế nào.

Some problems are worse than others

Một số vấn đề tệ hơn vấn đề khác

When something goes wrong, we want to know about it. Or do we? Are all problems the same? As the sources of problems increase, it can become more important to be able to prioritize these issues to decide if, and how, a human operator should be involved. Often the biggest question I find myself asking when it comes to alerting is, “Should this problem cause someone to be woken up at 3 a.m.?”

Khi điều gì đó trục trặc, chúng ta muốn biết về nó. Hay không nhỉ? Phải chăng mọi vấn đề đều như nhau? Khi các nguồn vấn đề tăng lên, có thể trở nên quan trọng hơn để có khả năng ưu tiên các vấn đề này nhằm quyết định liệu, và làm sao, một operator con người nên được liên đới. Thường câu hỏi lớn nhất tôi thấy mình tự hỏi khi nói đến cảnh báo là, “Vấn đề này có nên khiến ai đó bị đánh thức lúc 3 giờ sáng không?”

I saw one example of this thinking while spending some time on the Google campus many years ago. In the reception area of one of the buildings in Mountain View was an old rack of machines, there as a sort of exhibit. I noticed a couple of things. First, these servers weren't in server enclosures; they were just bare motherboards slotted into the rack. The main thing I noticed, though, was that the hard drives were attached by Velcro. I asked one of the Googlers why that was. “Oh,” he said, “the

hard drives fail so much we don't want them screwed in. We just rip them out, throw them in the bin, and velcro in a new one."

Tôi đã thấy một ví dụ về lối tư duy này khi dành thời gian ở khuôn viên Google nhiều năm trước. Ở khu lễ tân của một trong những tòa nhà tại Mountain View có một giá đỡ máy cũ, đặt ở đó như một dạng vật trưng bày. Tôi để ý vài điều. Thứ nhất, các máy chủ này không nằm trong khung kín máy chủ; chúng chỉ là các bo mạch chủ trần được cắm vào giá. Tuy nhiên, điều chính tôi để ý là các ổ cứng được gắn bằng dây dán Velcro. Tôi hỏi một nhân viên Google tại sao lại thế. "Ồ," anh ta nói, "ổ cứng hỏng nhiều đến mức chúng tôi không muốn vặn vít cố định chúng. Chúng tôi chỉ giật chúng ra, ném vào thùng rác, và dán bằng Velcro một cái mới vào."

The systems Google built assumed that hard drives would fail. It optimized the design of these servers to ensure that hard drive replacement was as easy as possible. As the system had been built to tolerate the failure of a hard drive, while it was important that the drive was replaced eventually, chances are a single hard drive failure wasn't going to cause any significant issues that would be visible to the user. With thousands of servers in a Google data center, it would be someone's daily task to just walk along a row of racks and replace hard drives as they went. Sure, the drive failure was an issue, but one that could be dealt with in a routine fashion. A hard drive failure was considered routine—not worth calling someone about outside of regular work hours, but perhaps just something they needed to be told about during their normal working day.

Các hệ thống Google xây dựng đều giả định rằng ổ cứng sẽ hỏng. Nó tối ưu hóa thiết kế của các máy chủ này để đảm bảo việc thay ổ cứng dễ nhất có thể. Vì hệ thống đã được xây dựng để chịu đựng sự hỏng của một ổ cứng, dù việc ổ đĩa cuối cùng được thay là quan trọng, nhiều khả năng việc hỏng một ổ cứng đơn lẻ sẽ không gây ra bất kỳ vấn đề đáng kể nào mà người dùng có thể thấy. Với hàng nghìn máy chủ trong một trung tâm dữ liệu của Google, sẽ là nhiệm vụ hằng ngày của ai đó chỉ việc đi dọc theo một hàng giá đỡ và thay ổ cứng khi họ đi qua. Chắc chắn, việc ổ đĩa hỏng là một vấn đề, nhưng là một vấn đề có thể được xử lý theo cách thường nhật. Việc ổ cứng hỏng được coi là thường nhật—không đáng để gọi điện cho ai đó ngoài giờ làm việc thông thường, mà có lẽ chỉ là điều họ cần được báo trong ngày làm việc bình thường.

As your sources of potential issues increase, you'll need to get better at prioritizing what things cause what types of alerts. Otherwise, you may well find yourself struggling to separate the trivial from the urgent.

Khi các nguồn vấn đề tiềm năng của bạn tăng lên, bạn sẽ cần giỏi hơn trong việc ưu tiên những thứ gây ra những loại cảnh báo nào. Nếu không, bạn rất có thể thấy mình chật vật tách cái tầm thường khỏi cái khẩn cấp.

Alert fatigue

Mệt mỏi vì cảnh báo (Alert fatigue)

Often, too many alerts can cause significant issues. In 1979, there was a partial meltdown of a reactor at the Three Mile Island nuclear power station in the US. The investigation into the incident highlighted the fact that the operators of the facility were so overwhelmed by the alerts they were seeing that it was impossible to determine what action needed to be taken. There was an alert indicating the fundamental problem that needed to be addressed, but this wasn't something that was obvious to the operators, as so many other alerts were going off at the same time. During the public hearing into the incident, one of the operators, Craig Faust, recalled that "I would have liked to have thrown away the alarm panel. It wasn't giving us any useful information." The report into the incident concluded that the control room "was greatly inadequate for managing an accident." 9

Thường thì quá nhiều cảnh báo có thể gây ra những vấn đề đáng kể. Năm 1979, đã có một vụ tan chảy một phần lò phản ứng tại nhà máy điện hạt nhân Three Mile Island ở Mỹ. Cuộc điều tra về sự cố đã làm nổi bật sự thật rằng những người vận hành cơ sở bị choáng ngợp bởi các cảnh báo họ thấy đến mức không thể xác định cần thực hiện hành động nào. Có một cảnh báo chỉ ra vấn đề cốt lõi cần được xử lý, nhưng đây không phải là điều hiển nhiên với các operator, vì có quá nhiều cảnh báo khác đang réo cùng lúc. Trong phiên điều trần công khai về sự cố, một trong các operator, Craig Faust, nhớ lại rằng “Tôi đã muốn vứt cái bảng báo động đi. Nó chẳng cho chúng tôi bất kỳ thông tin hữu ích nào.” Báo cáo về sự cố kết luận rằng phòng điều khiển “hoàn toàn không đủ năng lực để quản lý một tai nạn.” 9

More recently, we have seen this issue of too many alerts being raised in the context of a series of incidents with the 737 Max aircraft that included two separate air crashes that killed 346 people in total. The initial report 10 into the issues by the US National Transportation Safety Board (NTSB) drew attention to the confusing alerts

Gần đây hơn, chúng ta đã thấy vấn đề quá nhiều cảnh báo được nêu ra này trong bối cảnh một loạt sự cố với máy bay 737 Max bao gồm hai vụ rơi máy bay riêng biệt giết chết tổng cộng 346 người. Báo cáo ban đầu 10 về các vấn đề này của Ủy ban An toàn Giao thông Quốc gia Mỹ (NTSB) đã thu hút sự chú ý đến các cảnh báo gây bối rối

9 United States President’s Commission on the Accident at Three Mile Island, The Need for Change, the Legacy of TMI: Report of the President’s Commission on the Accident at Three Mile Island (Washington, DC: The Commission, 1979).

9 United States President’s Commission on the Accident at Three Mile Island, The Need for Change, the Legacy of TMI: Report of the President’s Commission on the Accident at Three Mile Island (Washington, DC: The Commission, 1979).

10 National Transportation Safety Board, Safety Recommendation Report: Assumptions Used in the Safety Assessment Process and the Effects of Multiple Alerts and Indications on Pilot Performance (Washington, DC: NTSB, 2019).

10 National Transportation Safety Board, Safety Recommendation Report: Assumptions Used in the Safety Assessment Process and the Effects of Multiple Alerts and Indications on Pilot Performance (Washington, DC: NTSB, 2019).

that were triggered in real-world conditions and were considered to be contributing factors in the crashes. From the report:

được kích hoạt trong điều kiện thực tế và được coi là các yếu tố góp phần vào các vụ rơi.
Từ báo cáo:

Human factors research has identified that, for non-normal conditions, such as those involving a system failure with multiple alerts, where there may be multiple flight crew actions required, providing pilots with understanding as to which actions must take priority is a critical need. This is particularly true in the case of functions implemented across multiple airplane systems because a failure in one system within highly integrated system architectures can present multiple alerts and indications to the flight crew as each interfacing system registers the failure...Thus, it is important that system interactions and the flight deck **interface** be designed to help direct pilots to the highest priority action(s). So here we’re talking about operating a nuclear reactor and flying an aircraft. I suspect that many of you right now are wondering what this has to do with the system you are building. Now it’s possible (if not likely) that you aren’t building safety-critical systems like this, but there is a lot we can learn from these examples. Both involve highly complex, interrelated systems in which a problem in one area can cause a problem in another. And when we generate too many

alerts, or we don't give operators the ability to prioritize what alerts should be focused on, disaster can follow. Overwhelming an operator with alerts can cause real issues. From the report again:

Nghiên cứu về các yếu tố con người đã xác định rằng, với các điều kiện bất thường, chẳng hạn những điều kiện liên quan đến một sự cố hệ thống với nhiều cảnh báo, nơi có thể cần nhiều hành động của tổ bay, việc cung cấp cho phi công sự hiểu biết về việc hành động nào phải được ưu tiên là một nhu cầu thiết yếu. Điều này đặc biệt đúng trong trường hợp các chức năng được hiện thực qua nhiều hệ thống máy bay, vì một sự cố trong một hệ thống bên trong các kiến trúc hệ thống tích hợp cao có thể tạo ra nhiều cảnh báo và chỉ báo cho tổ bay khi mỗi hệ thống giao tiếp ghi nhận sự cố... Vì vậy, điều quan trọng là các tương tác hệ thống và giao diện buồng lái được thiết kế để giúp hướng phi công tới (các) hành động ưu tiên cao nhất. Vậy ở đây chúng ta đang nói về việc vận hành một lò phản ứng hạt nhân và lái một chiếc máy bay. Tôi ngờ rằng nhiều người trong các bạn lúc này đang tự hỏi điều này thì liên quan gì đến hệ thống mà bạn đang xây dựng. Bây giờ, có khả năng (nếu không nói là rất có khả năng) bạn không xây dựng các hệ thống an toàn-trọng yếu (safety-critical) như thế này, nhưng có rất nhiều điều chúng ta có thể học từ những ví dụ này. Cả hai đều liên quan đến các hệ thống phức tạp cao, liên kết qua lại, trong đó một vấn đề ở một khu vực có thể gây ra một vấn đề ở khu vực khác. Và khi chúng ta sinh ra quá nhiều cảnh báo, hoặc chúng ta không cho các operator khả năng ưu tiên những cảnh báo nào nên tập trung vào, thậm họa có thể nổi tiếp. Làm operator quá tải với các cảnh báo có thể gây ra những vấn đề thực sự. Lại từ báo cáo:

In addition, research into pilot responses to multiple/simultaneous anomalous situations, along with data from accidents, indicates that multiple competing alerts may exceed available mental resources and narrow attentional focus leading to delayed or inadequately prioritized responses. So think twice about simply throwing up more alerts to an operator—you might not get what you want.

Ngoài ra, nghiên cứu về phản ứng của phi công trước nhiều tình huống bất thường đồng thời/cùng lúc, cùng với dữ liệu từ các tai nạn, chỉ ra rằng nhiều cảnh báo cạnh tranh nhau có thể vượt quá nguồn lực tinh thần sẵn có và thu hẹp tiêu điểm chú ý dẫn đến phản ứng bị trì hoãn hoặc được ưu tiên không thỏa đáng. Vậy hãy nghĩ kỹ trước khi chỉ đơn giản ném thêm cảnh báo cho một operator—bạn có thể không nhận được điều mình muốn.

Alarm Versus Alert — Báo động so với Cảnh báo (Alarm Versus Alert)

When looking more broadly into the topic of alerting, I found a lot of incredibly useful research and practice from a number of contexts, many of which were not specifically talking about alerting in IT systems. The term alarm is commonly encountered when looking into this topic in engineering and beyond, whereas we tend to use the term alert more commonly in IT. I spoke to a few people who see a distinction between these two terms, but oddly enough, what distinctions people drew between these two terms didn't seem consistent. Based on the fact that most people seem to see the terms alert and alarm as practically the same, and on the fact that where a distinction was drawn between the two it wasn't consistent, I have decided to standardize on the term alert for this book.

Khi tìm hiểu rộng hơn về chủ đề cảnh báo, tôi đã tìm thấy rất nhiều nghiên cứu và thực hành cực kỳ hữu ích từ một số bối cảnh, nhiều trong số đó không nói cụ thể về cảnh báo trong các hệ thống CNTT. Thuật ngữ báo động (alarm) thường gặp khi tìm hiểu chủ đề này trong kỹ thuật và xa hơn, trong khi chúng ta có xu hướng dùng thuật ngữ cảnh báo (alert) phổ biến hơn trong CNTT. Tôi đã nói chuyện với một vài người thấy có sự phân biệt giữa hai thuật ngữ này, nhưng kỳ lạ thay, những phân biệt mà người ta vạch ra giữa hai thuật

ngữ này dường như không nhất quán. Dựa trên việc hầu hết mọi người dường như xem các thuật ngữ alert và alarm thực tế là như nhau, và dựa trên việc nơi nào có vạch ra sự phân biệt giữa hai cái thì nó cũng không nhất quán, tôi đã quyết định chuẩn hóa dùng thuật ngữ alert cho cuốn sách này.

Toward better alerting

Hướng tới cảnh báo tốt hơn

So we want to avoid having too many alerts, as well as having alerts that aren't useful. What guidelines can we look at to help us create better alerts?

Vậy chúng ta muốn tránh có quá nhiều cảnh báo, cũng như tránh có những cảnh báo không hữu ích. Chúng ta có thể xem những hướng dẫn nào để giúp tạo ra các cảnh báo tốt hơn?

Steven Shorrock expands on this subject in his article “Alarm Design: From Nuclear Power to WebOps,” 11 which is a great read and a good jumping-off point for more reading in this area. From the article:

Steven Shorrock khai triển chủ đề này trong bài viết “Alarm Design: From Nuclear Power to WebOps” của ông, 11 một bài đọc tuyệt vời và một điểm khởi đầu tốt để đọc thêm trong lĩnh vực này. Từ bài viết:

The purpose of [alerts] is to direct the user's attention towards significant aspects of the operation or equipment that require timely attention. Drawing on work from outside software development, we have a useful set of rules from, of all places, the Engineering Equipment and Materials Users Association (EEMUA), which has come up with as good a description of what makes for a good alert as I've seen:

Mục đích của [các cảnh báo] là hướng sự chú ý của người dùng tới các khía cạnh quan trọng của hoạt động hay thiết bị đòi hỏi sự chú ý kịp thời. Rút ra từ công việc bên ngoài phát triển phần mềm, chúng ta có một tập quy tắc hữu ích từ, ai mà ngờ, Hiệp hội Người dùng Thiết bị và Vật liệu Kỹ thuật (Engineering Equipment and Materials Users Association—EEMUA), nơi đã đưa ra một mô tả về thế nào là một cảnh báo tốt hay nhất mà tôi từng thấy:

Relevant Make sure the alert is of value.

Liên quan (Relevant) Đảm bảo cảnh báo có giá trị.

Unique Ensure that the alert isn't duplicating another.

Duy nhất (Unique) Đảm bảo cảnh báo không trùng lặp với một cảnh báo khác.

Timely We need to get the alert quickly enough to make use of it.

Kịp thời (Timely) Chúng ta cần nhận cảnh báo đủ nhanh để tận dụng nó.

Prioritized Give the operator enough information to decide in what order the alerts should be dealt with.

Được ưu tiên (Prioritized) Cho operator đủ thông tin để quyết định thứ tự xử lý các cảnh báo.

Understandable The information in the alert needs to be clear and readable.

Dễ hiểu (Understandable) Thông tin trong cảnh báo cần rõ ràng và dễ đọc.

Diagnostic It needs to be clear what is wrong.

Chẩn đoán (Diagnostic) Cần rõ điều gì đang sai.

Advisory Help the operator understand what actions need to be taken.

Tư vấn (Advisory) Giúp operator hiểu cần thực hiện những hành động gì.

Focusing Draw attention to the most important issues.

Tập trung (Focusing) Hướng sự chú ý tới những vấn đề quan trọng nhất.

Looking back over my career to times when I've worked in production support, it's depressing to think how rarely the alerts I've had to deal with follow any of these rules.

Nhìn lại sự nghiệp của mình tới những lúc tôi làm hỗ trợ production, thật buồn khi nghĩ về việc các cảnh báo tôi phải xử lý hiếm khi tuân theo bất kỳ quy tắc nào trong số này.

11 Steven Shorrock, "Alarm Design: From Nuclear Power to WebOps," Humanistic Systems (blog), October 16, 2015, <https://oreil.ly/RCHDL>.

11 Steven Shorrock, "Alarm Design: From Nuclear Power to WebOps," Humanistic Systems (blog), October 16, 2015, <https://oreil.ly/RCHDL>.

All too often, unfortunately, the people providing information to our alerting systems and the people actually on the receiving ends of our alerts are different people. From Shorrock again:

Quá thường xuyên, tiếc thay, những người cung cấp thông tin cho hệ thống cảnh báo của chúng ta và những người thực sự ở đầu nhận cảnh báo lại là những người khác nhau. Lại từ Shorrock:

Understanding the nature of alarm handling, and the associated design issues, can help you—the expert in your work—to be a more informed user, helping to bring about the best alarm systems to support your work. One technique that can help reduce the number of alerts vying for our attention involves shifting how we think about which issues require us to bring them to operators' attention in the first place. Let's explore this topic next.

Hiểu được bản chất của việc xử lý báo động, và các vấn đề thiết kế liên quan, có thể giúp bạn—chuyên gia trong công việc của mình—trở thành một người dùng am hiểu hơn, giúp tạo ra các hệ thống báo động tốt nhất để hỗ trợ công việc của bạn. Một kỹ thuật có thể giúp giảm số lượng cảnh báo tranh giành sự chú ý của chúng ta liên quan đến việc thay đổi cách chúng ta nghĩ về việc vấn đề nào cần được đưa ra cho sự chú ý của các operator ngay từ đầu. Hãy khám phá chủ đề này tiếp theo.

Semantic Monitoring — Semantic Monitoring

With semantic monitoring, we are defining a model for what the acceptable semantics of our system are. What are the properties the system has to have for us to think it is operating within acceptable means? To a great extent, semantic monitoring requires a shift in our behavior. Rather than looking for the presence of errors, we instead need to be constantly asking one question: is the system behaving the way that we expect? If it is behaving correctly, then this better helps us understand how to prioritize dealing with the errors we are seeing.

Với semantic monitoring, chúng ta định nghĩa một mô hình về thế nào là ngữ nghĩa (semantics) chấp nhận được của hệ thống. Hệ thống phải có những thuộc tính nào để chúng ta nghĩ nó đang vận hành trong các phương tiện chấp nhận được? Ở mức độ lớn, semantic monitoring đòi hỏi một sự thay đổi trong hành vi của chúng ta. Thay vì tìm kiếm sự hiện diện của các lỗi, thay vào đó chúng ta cần liên tục đặt một câu hỏi: hệ thống

có đang hành xử theo cách mà chúng ta kỳ vọng không? Nếu nó đang hành xử đúng, thì điều này giúp chúng ta hiểu rõ hơn cách ưu tiên xử lý các lỗi mà ta đang thấy.

The next thing to work out is how do you define a model for a correctly behaving system. You can get super formal with this approach (quite literally, with some organizations making use of formal methods for this), but making some simple value statements can get you a long way. For example, in the case of MusicCorp, what has to be true for us to be satisfied that the system is working correctly? Well, perhaps we say that:

Điều tiếp theo cần tìm ra là làm sao bạn định nghĩa một mô hình cho một hệ thống hành xử đúng. Bạn có thể cực kỳ hình thức với cách tiếp cận này (theo nghĩa đen, với một số tổ chức dùng các phương pháp hình thức—formal methods—cho việc này), nhưng đưa ra một số phát biểu giá trị đơn giản có thể đưa bạn đi xa. Ví dụ, trong trường hợp của MusicCorp, điều gì phải đúng để chúng ta hài lòng rằng hệ thống đang hoạt động đúng? Vâng, có lẽ chúng ta nói rằng:

- New customers can register to join.
- We are selling at least \$20,000 worth of products per hour during our peak time.
- We are shipping orders out at a rate that is normal.
 - Khách hàng mới có thể đăng ký tham gia.
 - Chúng ta đang bán được ít nhất 20.000 đô-la giá trị sản phẩm mỗi giờ trong thời gian cao điểm.
 - Chúng ta đang giao các đơn hàng ra với tốc độ bình thường.

If these three statements can prove to be correct, then broadly speaking we feel that the system is operating well enough. Coming back to our earlier discussions of SLAs and SLOs, our model for semantic correctness would be expected to greatly exceed our obligations in an SLA, and we would expect to have concrete SLOs that allow us to track against this model. Put another way, making these statements about how we expect our software to be acting will go a long way toward our identifying SLOs.

Nếu ba phát biểu này có thể chứng minh là đúng, thì nói rộng ra chúng ta cảm thấy hệ thống đang vận hành đủ tốt. Quay lại các thảo luận trước đó của chúng ta về SLA và SLO, mô hình về tính đúng đắn ngữ nghĩa của chúng ta sẽ được kỳ vọng vượt xa các nghĩa vụ trong một SLA, và chúng ta sẽ kỳ vọng có các SLO cụ thể cho phép theo dõi đối chiếu với mô hình này. Nói cách khác, đưa ra những phát biểu này về cách chúng ta kỳ vọng phần mềm của mình hành động sẽ đi một chặng đường dài hướng tới việc xác định các SLO của chúng ta.

One of the biggest challenges is in getting agreement as to what this model is. As you can see, we aren't talking about low level things like "disk usage shouldn't exceed 95%"; we're making higher-level statements about our system. As the operator of the system, or the person who wrote and tested the microservice, you may not be in a position to decide what these value statements should be. In a product-driven delivery organization, this is where the product owner should come in—but it might be your job as the operator to make sure the discussion with the product owner actually happens.

Một trong những thách thức lớn nhất là đạt được sự đồng thuận về mô hình này là gì. Như bạn có thể thấy, chúng ta không nói về những thứ cấp thấp như "mức dùng đĩa không nên vượt 95%"; chúng ta đang đưa ra những phát biểu cấp cao hơn về hệ thống của mình. Với tư cách operator của hệ thống, hoặc người đã viết và kiểm thử microservice, bạn có thể không ở vị thế để quyết định những phát biểu giá trị này nên là gì. Trong một tổ chức chuyển giao hướng sản phẩm (product-driven), đây là lúc product owner nên vào cuộc—nhưng có thể đó là việc của bạn với tư cách operator để đảm bảo cuộc thảo luận với product owner

thực sự diễn ra.

Once you’ve decided on what your model is, then it comes down to working out if the current system behavior meets this model. Broadly speaking, we have two key ways to do this—real user monitoring and synthetic transactions. We’ll look at synthetic transactions in a moment, as they fall under the umbrella of testing in production, but let’s look at real user monitoring first.

Một khi bạn đã quyết định mô hình của mình là gì, thì việc còn lại là tìm ra liệu hành vi hệ thống hiện tại có đáp ứng mô hình này không. Nói rộng ra, chúng ta có hai cách chính để làm điều này—giám sát người dùng thật (real user monitoring) và các giao dịch tổng hợp (synthetic transactions). Chúng ta sẽ xem xét các giao dịch tổng hợp trong chốc lát, vì chúng nằm dưới cái ô của kiểm thử trong production, nhưng hãy xem giám sát người dùng thật trước.

Real user monitoring

Giám sát người dùng thật (Real user monitoring)

With real user monitoring, we look at what is actually happening in our production system and compare that with our semantic model. In MusicCorp, we’d look at how many customers signed up, how many orders we sent, and so on

Với giám sát người dùng thật, chúng ta nhìn vào những gì thực sự đang diễn ra trong hệ thống production của mình và so sánh điều đó với mô hình ngữ nghĩa của chúng ta. Trong MusicCorp, chúng ta sẽ nhìn vào bao nhiêu khách hàng đã đăng ký, bao nhiêu đơn hàng chúng ta đã gửi, v.v.

The challenge with real user monitoring is that often the information we need isn’t available to us in a timely manner. Consider the expectation that MusicCorp should be selling at least \$20,000 in products per hour. If this information is locked away in a database somewhere, we may not be able to collect this information and act on it accordingly. This is why you may need to get better at exposing access to information that you would previously consider to be “business” metrics to your production tooling. If you can emit a CPU rate to your metrics store, and this metric store could be used to alert on this condition, then why can’t you also record a sale and a dollar value into this same store?

Thách thức với giám sát người dùng thật là thường thông tin chúng ta cần không có sẵn cho ta một cách kịp thời. Hãy xét kỳ vọng rằng MusicCorp nên bán được ít nhất 20.000 đô-la sản phẩm mỗi giờ. Nếu thông tin này bị khóa đâu đó trong một cơ sở dữ liệu, chúng ta có thể không thu thập được thông tin này và hành động theo nó tương ứng. Đây là lý do bạn có thể cần giỏi hơn trong việc phơi bày quyền truy cập tới thông tin mà trước đây bạn coi là các metric “nghiệp vụ” cho công cụ production của mình. Nếu bạn có thể phát ra một tỉ lệ CPU tới kho metric của mình, và kho metric này có thể được dùng để cảnh báo về điều kiện này, thì tại sao bạn không thể cũng ghi một lượt bán hàng và một giá trị đô-la vào cùng kho này?

One of the main drawbacks of real user monitoring is that it is fundamentally noisy. You are getting a lot of information—sifting through it to find out if there is a problem can be difficult. It’s also worth realizing that real user monitoring tells you what already happened, and as a result you may not catch an issue til after it occurred. If a customer failed to get registered, that is an unhappy customer. With synthetic transactions, another form of testing in production that we’ll look at in a moment, we have the opportunity to not just cut down on the noise but also catch issues before our users are aware of them.

Một trong những nhược điểm chính của giám sát người dùng thật là nó về cơ bản nhiều (noisy). Bạn đang nhận được rất nhiều thông tin—sàng lọc qua nó để tìm ra liệu có vấn đề

không có thể khó. Cũng đáng nhận ra rằng giám sát người dùng thật cho bạn biết những gì đã xảy ra, và kết quả là bạn có thể không bắt được một vấn đề cho đến sau khi nó đã xảy ra. Nếu một khách hàng không đăng ký được, đó là một khách hàng không vui. Với các giao dịch tổng hợp, một dạng kiểm thử trong production khác mà chúng ta sẽ xem trong chốc lát, chúng ta có cơ hội không chỉ cắt giảm nhiều mà còn bắt các vấn đề trước khi người dùng của chúng ta nhận ra chúng.

Testing in Production — Kiểm thử trong Production (Testing in Production)

Not testing in prod is like not practicing with the full orchestra because your solo sounded fine at home. 12 —Charity Majors

Không kiểm thử trong prod thì giống như không tập luyện với cả dàn nhạc giao hưởng chỉ vì màn độc tấu của bạn nghe ổn ở nhà. 12 —Charity Majors

As we’ve covered multiple times throughout the book, from our discussion of concepts like canary deployments in “Canary **Release**” on page 271 to our look at the balancing act with regard to pre- and post-production testing, carrying out some form of testing in production can be an incredibly useful—and safe—activity. We’ve looked at a number of different types of in-production testing in this book, and there are more forms besides, so I felt that it would be useful to summarize some of the different types of testing in production we’ve already looked at and also share some other examples of testing in production that are commonly used. It surprises me how many people who are scared by the concept of testing in production are already doing it without really realizing they are.

Như chúng ta đã đề cập nhiều lần xuyên suốt cuốn sách, từ thảo luận của chúng ta về các khái niệm như canary deployment trong “Canary Release” ở trang 271 đến việc xem xét sự cân bằng đối với kiểm thử tiền- và hậu-production, việc thực hiện một dạng kiểm thử trong production nào đó có thể là một hoạt động cực kỳ hữu ích—và an toàn. Chúng ta đã xem xét một số loại kiểm thử trong production khác nhau trong cuốn sách này, và còn có nhiều dạng nữa, nên tôi cảm thấy sẽ hữu ích nếu tóm lược một số loại kiểm thử trong production khác nhau mà chúng ta đã xem và cũng chia sẻ một số ví dụ khác về kiểm thử trong production thường được dùng. Tôi ngạc nhiên trước việc bao nhiêu người sợ khái niệm kiểm thử trong production lại đã đang làm nó mà không thực sự nhận ra.

All forms of testing in production are arguably a form of “monitoring” activity. We are carrying out these forms of testing in production to ensure that our production system is running as we expect, and many forms of testing in production can be incredibly effective in picking up problems before our users even notice.

Mọi dạng kiểm thử trong production có thể lập luận là một dạng hoạt động “giám sát”. Chúng ta thực hiện các dạng kiểm thử trong production này để đảm bảo hệ thống production của chúng ta đang chạy như ta kỳ vọng, và nhiều dạng kiểm thử trong production có thể cực kỳ hiệu quả trong việc bắt các vấn đề trước khi người dùng của chúng ta thậm chí nhận ra.

Synthetic transactions

Giao dịch tổng hợp (Synthetic transactions)

With synthetic transactions, we inject fake user behavior into our production system. This fake user behavior has known inputs and expected outputs. For MusicCorp, for example, we could artificially

create a new customer and then check that the customer was successfully created. These transactions would be fired on a regular basis, giving us the chance to pick up problems as quickly as possible.

Với các giao dịch tổng hợp, chúng ta tiêm hành vi người dùng giả vào hệ thống production của mình. Hành vi người dùng giả này có các đầu vào đã biết và các đầu ra kỳ vọng. Với MusicCorp, chẳng hạn, chúng ta có thể tạo một cách nhân tạo một khách hàng mới rồi kiểm tra rằng khách hàng đó đã được tạo thành công. Các giao dịch này sẽ được kích hoạt một cách định kỳ, cho chúng ta cơ hội bắt các vấn đề nhanh nhất có thể.

I first did this back in 2005. I was part of a small Thoughtworks team that was building a system for an investment bank. Throughout the trading day, lots of events came in that represented changes in the market. Our job was to react to these changes and look at the impact on the bank's portfolio. We were working under some fairly tight deadlines, a sour goal was to complete all our calculations less than 10 seconds after the event arrived. The system itself consisted of around five discrete services, at least one of which was running on a computing grid that, among other things, was scavenging unused CPU cycles on around 250 desktop hosts in the bank's disaster recovery center.

Tôi lần đầu làm điều này hồi năm 2005. Tôi là một phần của một đội nhỏ ở Thoughtworks đang xây dựng một hệ thống cho một ngân hàng đầu tư. Trong suốt ngày giao dịch, rất nhiều event ủa vào đại diện cho các thay đổi trên thị trường. Việc của chúng tôi là phản ứng với những thay đổi này và xem xét tác động lên danh mục đầu tư của ngân hàng. Chúng tôi làm việc dưới một số hạn chót khá gắt gao, một mục tiêu là hoàn thành tất cả tính toán của mình trong vòng dưới 10 giây sau khi event đến. Bản thân hệ thống gồm khoảng năm dịch vụ rời rạc, ít nhất một trong số đó chạy trên một lưới tính toán (computing grid) mà, trong số những thứ khác, đang tận dụng các chu kỳ CPU không dùng đến trên khoảng 250 host máy bàn ở trung tâm phục hồi sau thảm họa của ngân hàng.

The number of moving parts in the system meant a lot of noise was being generated from many of the lower-level metrics we were gathering. We also didn't have the

Số lượng các bộ phận chuyển động trong hệ thống có nghĩa là rất nhiều nhiễu được sinh ra từ nhiều metric cấp thấp hơn mà chúng tôi đang thu thập. Chúng tôi cũng không có

12 Charity Majors (@mipsytipsy), Twitter, July 7, 2019, 9:48 a.m., <https://oreil.ly/4VUAX>.

12 Charity Majors (@mipsytipsy), Twitter, July 7, 2019, 9:48 a.m., <https://oreil.ly/4VUAX>.

benefit of scaling gradually or having the system run for a few months to understand what “good” looked like in terms of low-level metrics like CPU rate or response time. Our approach was to generate fake events to price part of the portfolio that was not booked into the downstream systems. Every minute or so, we used a tool called Nagios to run a command-line job that inserted a fake event into one of our queues. Our system picked it up and ran all the various calculations just like any other job, except the results appeared in the “junk” book, which was used only for testing. If a repricing wasn't seen within a given time, Nagios reported this as an issue.

lợi thế của việc mở rộng dần dần hay có hệ thống chạy vài tháng để hiểu thế nào là “tốt” về các metric cấp thấp như tỉ lệ CPU hay thời gian phản hồi. Cách tiếp cận của chúng tôi là sinh các event giả để định giá một phần danh mục đầu tư chưa được ghi vào các hệ thống downstream. Cứ khoảng mỗi phút, chúng tôi dùng một công cụ gọi là Nagios để chạy một tác vụ dòng lệnh chèn một event giả vào một trong các hàng đợi của chúng tôi. Hệ thống của chúng tôi nhặt nó lên và chạy tất cả các tính toán khác nhau giống như bất kỳ tác vụ nào khác, ngoại trừ kết quả xuất hiện trong sổ “rác” (junk book), thứ chỉ được dùng để kiểm thử. Nếu một lượt định giá lại không được thấy trong một khoảng thời gian nhất định, Nagios báo cáo điều này như một vấn đề.

In practice, I've found the use of synthetic transactions to perform semantic monitoring like this to be a far better indicator of issues in systems than alerting on the lower-level metrics. They don't replace the need for the lower-level detail, though—we'll still want that information when we need to find out why a synthetic **transaction** failed.

Trong thực tế, tôi đã thấy việc dùng các giao dịch tổng hợp để thực hiện semantic monitoring kiểu này là một chỉ báo về các vấn đề trong hệ thống tốt hơn nhiều so với cảnh báo trên các metric cấp thấp. Tuy nhiên, chúng không thay thế nhu cầu về chi tiết cấp thấp—chúng ta vẫn sẽ muốn thông tin đó khi cần tìm ra tại sao một giao dịch tổng hợp lại thất bại.

Implementing synthetic transactions. In the past, implementing synthetic transactions was a fairly daunting task. But the world has moved on, and the means to implement them is at our fingertips! You are running tests for your systems, right? If not, go read Chapter 9 and come back. All done? Good!

Triển khai các giao dịch tổng hợp. Trước đây, triển khai các giao dịch tổng hợp là một tác vụ khá đáng sợ. Nhưng thế giới đã tiến lên, và phương tiện để triển khai chúng nay nằm trong tầm tay! Bạn đang chạy các bài kiểm thử cho hệ thống của mình, phải không? Nếu không, hãy đi đọc Chương 9 rồi quay lại. Xong cả chưa? Tốt!

If we look at the tests we have that test a given service end to end, or even our whole system end to end, we have much of what we need to implement semantic monitoring. Our system already exposes the hooks needed to launch the test and check the result. So why not just run a subset of these tests, on an ongoing basis, as a way of monitoring our system?

Nếu chúng ta nhìn vào các bài kiểm thử mà chúng ta có để kiểm thử một dịch vụ nhất định từ đầu đến cuối (end to end), hay thậm chí toàn bộ hệ thống của chúng ta từ đầu đến cuối, chúng ta đã có phần lớn những gì cần để triển khai semantic monitoring. Hệ thống của chúng ta đã phơi bày các móc nối (hook) cần thiết để khởi chạy bài kiểm thử và kiểm tra kết quả. Vậy tại sao không chỉ chạy một tập con của các bài kiểm thử này, trên cơ sở liên tục, như một cách giám sát hệ thống của chúng ta?

There are some things we need to do, of course. First, we need to be careful about the data requirements of our tests. We may need to find a way for our tests to adapt to different live data if it changes over time, or else set a different source of data. For example, we could have a set of fake users we use in production with a known set of data.

Có một số điều chúng ta cần làm, dĩ nhiên. Thứ nhất, chúng ta cần cẩn thận về các yêu cầu dữ liệu của các bài kiểm thử của mình. Chúng ta có thể cần tìm cách để các bài kiểm thử của mình thích nghi với dữ liệu trực tiếp (live data) khác nhau nếu nó thay đổi theo thời gian, hoặc nếu không thì đặt một nguồn dữ liệu khác. Ví dụ, chúng ta có thể có một tập người dùng giả mà chúng ta dùng trong production với một tập dữ liệu đã biết.

Likewise, we have to make sure we don't accidentally trigger unforeseen side effects. A friend told me a story about an ecommerce company that accidentally ran its tests against its production ordering systems. It didn't realize its mistake until a large number of washing machines arrived at the head office.

Tương tự, chúng ta phải đảm bảo mình không vô tình kích hoạt các tác dụng phụ không lường trước. Một người bạn kể tôi nghe một câu chuyện về một công ty thương mại điện tử vô tình chạy các bài kiểm thử của họ trên hệ thống đặt hàng production của họ. Họ không nhận ra sai lầm của mình cho đến khi một số lượng lớn máy giặt đến tại trụ sở chính.

A/B testing

Kiểm thử A/B (A/B testing)

With an A/B test, you **deploy** two different versions of the same functionality, with users seeing either the “A” or the “B” functionality. You are then able to see which version of the functionality performs best. This is commonly used when trying to decide between two different approaches for how something should be done—for example, you might try two different customer registration forms to see which one is more effective in driving sign-ups.

Với một bài kiểm thử A/B, bạn triển khai hai phiên bản khác nhau của cùng một chức năng, với người dùng thấy hoặc chức năng “A” hoặc “B”. Bạn sau đó có thể thấy phiên bản nào của chức năng hoạt động tốt nhất. Điều này thường được dùng khi cố quyết định giữa hai cách tiếp cận khác nhau về việc một điều gì đó nên được làm thế nào—ví dụ, bạn có thể thử hai biểu mẫu đăng ký khách hàng khác nhau để xem cái nào hiệu quả hơn trong việc thúc đẩy đăng ký.

Canary release

Phát hành canary (Canary release)

A small portion of your user base gets to see the new release of functionality. If this new functionality works well, you can increase the portion of your user base that sees the new functionality to the point that the new version of the functionality is now used by all users. On the other hand, if the new functionality doesn’t work as intended, you’ve impacted only a small portion of your user base and can either revert the change or try to fix whatever problem you have identified.

Một phần nhỏ trong cơ sở người dùng của bạn được thấy bản phát hành chức năng mới. Nếu chức năng mới này hoạt động tốt, bạn có thể tăng phần cơ sở người dùng được thấy chức năng mới đến mức phiên bản mới của chức năng giờ được dùng bởi tất cả người dùng. Mặt khác, nếu chức năng mới không hoạt động như dự định, bạn chỉ tác động đến một phần nhỏ cơ sở người dùng và có thể hoặc hoàn nguyên thay đổi hoặc cố sửa bất kỳ vấn đề nào bạn đã xác định.

Parallel run

Chạy song song (Parallel run)

With a parallel run, you execute two different equivalent implementations of the same functionality side by side. Any user request is routed to both versions, and their results can be compared. So instead of directing a user to either the old or the new version, as in a canary release we execute both versions—but the user sees only one. This allows for a full comparison between the two different versions, which is incredibly useful when we want a better understanding of aspects like the load characteristics of a new implementation of some key functionality.

Với một lần chạy song song, bạn thực thi hai hiện thực tương đương khác nhau của cùng một chức năng cạnh nhau. Bất kỳ yêu cầu người dùng nào cũng được định tuyến tới cả hai phiên bản, và kết quả của chúng có thể được so sánh. Vậy nên thay vì hướng một người dùng tới hoặc phiên bản cũ hoặc mới, như trong một phát hành canary, chúng ta thực thi cả hai phiên bản—nhưng người dùng chỉ thấy một. Điều này cho phép một sự so sánh đầy đủ giữa hai phiên bản khác nhau, điều cực kỳ hữu ích khi chúng ta muốn hiểu rõ hơn các khía cạnh như đặc tính tải của một hiện thực mới của một chức năng then chốt nào đó.

Smoke tests

Smoke test

Used after the software is deployed into production but before it is released, smoke tests are run against the software to make sure it is working appropriately. These tests are normally fully

automated and can range from very simple activities like making sure a given microservice is up and running to actually executing full-blown synthetic transactions.

Được dùng sau khi phần mềm được triển khai vào production nhưng trước khi nó được phát hành, smoke test được chạy đối với phần mềm để đảm bảo nó đang hoạt động phù hợp. Các bài kiểm thử này thường hoàn toàn tự động và có thể từ các hoạt động rất đơn giản như đảm bảo một microservice nhất định đang chạy đến việc thực sự thực thi các giao dịch tổng hợp đầy đủ.

Synthetic transactions

Giao dịch tổng hợp (Synthetic transactions)

A full-blown, fake user interaction is injected into the system. It's often very close to the sort of **end-to-end test** you might write.

Một tương tác người dùng giả, đầy đủ, được tiêm vào hệ thống. Nó thường rất gần với loại bài kiểm thử end-to-end mà bạn có thể viết.

Chaos engineering

Chaos engineering

A topic we'll discuss more in Chapter 12, chaos engineering can involve injection of faults into a production system to ensure that it is able to handle these expected issues. The best-known example of this technique is probably Netflix's Chaos Monkey, which is able to turn off virtual machines in production, with the expectation that the system is robust enough that these shutdowns don't interrupt the end-user functionality.

Một chủ đề chúng ta sẽ bàn thêm trong Chương 12, chaos engineering có thể liên quan đến việc tiêm các lỗi vào một hệ thống production để đảm bảo rằng nó có thể xử lý được các vấn đề kỳ vọng này. Ví dụ nổi tiếng nhất của kỹ thuật này có lẽ là Chaos Monkey của Netflix, thứ có thể tắt các máy ảo trong production, với kỳ vọng rằng hệ thống đủ vững chắc để các lần tắt máy này không làm gián đoạn chức năng cho người dùng cuối.

Standardization — Chuẩn hóa (Standardization)

As we've covered previously, one of the ongoing balancing acts you'll need to pull off is where to allow for decisions to be made narrowly for a single microservice versus where you need to standardize across your system. In my opinion, monitoring and observability is one area in which standardization can be incredibly important. With microservices collaborating in lots of different ways to provide capabilities to users using multiple interfaces, you need to view the system in a holistic way.

Như chúng ta đã đề cập trước đó, một trong những hành động cân bằng liên tục mà bạn sẽ cần thực hiện là chỗ nào nên cho phép các quyết định được đưa ra hẹp cho một microservice đơn lẻ so với chỗ nào bạn cần chuẩn hóa xuyên suốt hệ thống của mình. Theo ý kiến của tôi, giám sát và khả năng quan sát là một lĩnh vực mà việc chuẩn hóa có thể cực kỳ quan trọng. Với các microservice cộng tác theo nhiều cách khác nhau để cung cấp năng lực cho người dùng sử dụng nhiều giao diện, bạn cần nhìn hệ thống một cách toàn diện.

You should try to write your logs out in a standard format. You definitely want to have all your metrics in one place, and you may want to have a list of standard names for your metrics too; it would be

very annoying for one service to have a metric called `ResponseTime` , and another to have one called `RspTimeSecs` , when they mean the same thing.

Bạn nên cố ghi log của mình ra theo một định dạng chuẩn. Bạn chắc chắn muốn có tất cả các metric của mình ở một nơi, và bạn có thể muốn có một danh sách tên chuẩn cho các metric của mình nữa; sẽ rất khó chịu nếu một dịch vụ có một metric gọi là `ResponseTime` , và một dịch vụ khác có một cái gọi là `RspTimeSecs` , khi chúng có cùng ý nghĩa.

As always with standardization, tools can help. As I've said before, the key is to make it easy to do the right thing—so having a platform with many of the basic building blocks in place, like log aggregation, makes a lot of sense. Increasingly, much of this falls to the **platform team**, the role of which we explore more fully in Chapter 15 .

Như luôn luôn với chuẩn hóa, công cụ có thể giúp ích. Như tôi đã nói trước đó, điều then chốt là làm cho việc làm đúng trở nên dễ dàng—vậy nên có một nền tảng với nhiều khối xây dựng cơ bản sẵn có, như gom log, là rất hợp lý. Ngày càng nhiều, phần lớn việc này rơi vào platform team, vai trò của nó chúng ta khám phá đầy đủ hơn trong Chương 15 .

Selecting Tools — Lựa chọn Công cụ (Selecting Tools)

As we've already covered, there are potentially a host of different tools you may need to bring to bear to help improve the observability of your system. But as I've already touched on, this is a fast-emerging space, and it's highly likely that the tooling we'll be using in the future will look very different from what we have now. With platforms like Honeycomb and Lightstep leading the way in terms of what observability tooling for microservices looks like, and with the rest of the market to an extent playing catch-up, I fully expect this space to see a lot of churn in the future.

Như chúng ta đã đề cập, có khả năng có cả một loạt công cụ khác nhau mà bạn có thể cần huy động để giúp cải thiện khả năng quan sát của hệ thống. Nhưng như tôi đã chạm tới, đây là một không gian đang nổi lên nhanh chóng, và rất có khả năng các công cụ mà chúng ta sẽ dùng trong tương lai sẽ trông rất khác so với những gì chúng ta có hiện nay. Với các nền tảng như Honeycomb và Lightstep dẫn đầu về việc công cụ khả năng quan sát cho microservice trông như thế nào, và với phần còn lại của thị trường ở một mức nào đó đang đuổi theo, tôi hoàn toàn kỳ vọng không gian này sẽ thấy nhiều xáo trộn trong tương lai.

So it's quite possible you'll need different tools from what you have right now if you're only just embracing microservices, and it's possible you'll want different tooling in the future too, as the solutions in this space continue to improve. With that in mind, I want to share a few thoughts about criteria that I think are important for any tooling in this area to have.

Vậy nên hoàn toàn có khả năng bạn sẽ cần các công cụ khác với những gì bạn có hiện nay nếu bạn chỉ vừa mới đón nhận microservice, và có khả năng bạn sẽ muốn công cụ khác trong tương lai nữa, khi các giải pháp trong không gian này tiếp tục cải thiện. Với điều đó trong đầu, tôi muốn chia sẻ một vài suy nghĩ về các tiêu chí mà tôi nghĩ là quan trọng cho bất kỳ công cụ nào trong lĩnh vực này cần có.

Democratic — Mang tính Dân chủ (Democratic)

If you have tools that are so hard to work with only experienced operators can make use of them, then you limit the number of people who can participate in production activities. Likewise, if you

pick tools that are so expensive as to prohibit their use in any situation other than critical production environments, then developers will not have exposure to these tools until it's too late.

Nếu bạn có các công cụ khó dùng đến mức chỉ các operator giàu kinh nghiệm mới có thể tận dụng chúng, thì bạn giới hạn số lượng người có thể tham gia vào các hoạt động production. Tương tự, nếu bạn chọn các công cụ đắt đến mức cấm dùng chúng trong bất kỳ tình huống nào ngoài các môi trường production trọng yếu, thì các nhà phát triển sẽ không được tiếp xúc với các công cụ này cho đến khi quá muộn.

Pick tools that consider the needs of all the people whom you will want using them. If you really want to move to a model of more collective ownership of your software, then the software needs to be usable by everyone in the team. Making sure that whatever tooling you pick will also be used in development and test environments will go a long way toward making this goal a reality.

Hãy chọn các công cụ xét đến nhu cầu của tất cả những người mà bạn sẽ muốn dùng chúng. Nếu bạn thực sự muốn chuyển sang một mô hình sở hữu chung phần mềm của mình hơn, thì phần mềm cần dùng được bởi mọi người trong đội. Đảm bảo rằng bất kỳ công cụ nào bạn chọn cũng sẽ được dùng trong các môi trường phát triển và kiểm thử sẽ đi một chặng đường dài hướng tới việc biến mục tiêu này thành hiện thực.

Easy to Integrate — Dễ Tích hợp (Easy to Integrate)

Getting the right information out of your application architecture and the systems you run on is vital, and as we've already covered, you may need to be extracting more information than before, and in different formats. Making this process as easy as possible is crucial. Initiatives such as OpenTracing have helped in terms of providing standard APIs that client libraries and platforms can support, making integration and portability across toolchains easier. Of special interest, as I've discussed, is the new OpenTelemetry initiative, which is being driven by a large number of parties.

Lấy đúng thông tin ra khỏi kiến trúc ứng dụng của bạn và các hệ thống bạn chạy là sống còn, và như chúng ta đã đề cập, bạn có thể cần trích xuất nhiều thông tin hơn trước, và ở các định dạng khác nhau. Làm cho quá trình này dễ nhất có thể là then chốt. Các sáng kiến như OpenTracing đã giúp ích về việc cung cấp các API chuẩn mà các thư viện client và nền tảng có thể hỗ trợ, làm cho việc tích hợp và chuyển đổi (portability) xuyên các chuỗi công cụ dễ hơn. Đặc biệt đáng quan tâm, như tôi đã bàn, là sáng kiến OpenTelemetry mới, thứ đang được thúc đẩy bởi một số lượng lớn các bên.

Picking tools that support these open standards will ease integration efforts and also perhaps help make it easier to change vendors later on.

Chọn các công cụ hỗ trợ các tiêu chuẩn mở này sẽ giảm bớt nỗ lực tích hợp và cũng có lẽ giúp dễ hơn để đổi nhà cung cấp về sau.

Provide Context — Cung cấp Bối cảnh (Provide Context)

When looking at a piece of information, I need the tool to provide me with as much context as possible to help me understand what needs to happen next. I really like the following categorization system for different types of context that I found via a Lightstep blog post: 13

Khi nhìn vào một mẫu thông tin, tôi cần công cụ cung cấp cho tôi càng nhiều bối cảnh càng tốt để giúp tôi hiểu điều gì cần xảy ra tiếp theo. Tôi thực sự thích hệ thống phân loại sau đây cho các loại bối cảnh khác nhau mà tôi tìm thấy qua một bài blog của Lightstep: 13

Temporal context How does this look compared to a minute, hour, day, or month ago?

Bối cảnh thời gian (Temporal context) Cái này trông thế nào so với một phút, một giờ, một ngày, hay một tháng trước?

Relative context How has this changed in relation to other things in the system?

Bối cảnh tương đối (Relative context) Cái này đã thay đổi thế nào so với các thứ khác trong hệ thống?

Relational context Is something depending on this? Is this depending on something else?

Bối cảnh quan hệ (Relational context) Có thứ gì đó đang phụ thuộc vào cái này không? Cái này có phụ thuộc vào thứ gì khác không?

Proportional context How bad is this? Is it large or small scoped? Who is impacted?

Bối cảnh tỉ lệ (Proportional context) Cái này tệ đến mức nào? Phạm vi nó lớn hay nhỏ? Ai bị tác động?

Real-Time — Thời gian thực (Real-Time)

You can't wait ages for this information. You need it now. Your definition of "now" can of course vary somewhat, but in the context of your systems, you need information quickly enough that you have a chance of spotting a problem before a user does, or at the very least of having the information at hand when someone does complain. In practice, we're talking about seconds, not minutes or hours.

Bạn không thể chờ đợi cả thế kỷ cho thông tin này. Bạn cần nó ngay bây giờ. Định nghĩa của bạn về "ngay bây giờ" dĩ nhiên có thể khác nhau phần nào, nhưng trong bối cảnh hệ thống của bạn, bạn cần thông tin đủ nhanh để bạn có cơ hội phát hiện một vấn đề trước khi người dùng làm vậy, hoặc ít nhất là có thông tin trong tay khi ai đó thực sự than phiền. Trong thực tế, chúng ta đang nói về vài giây, chứ không phải vài phút hay vài giờ.

13 "Observability: A Complete Overview for 2021," Lightstep, accessed June 16, 2021, <https://oreil.ly/a1ERu>

13 "Observability: A Complete Overview for 2021," Lightstep, accessed June 16, 2021, <https://oreil.ly/a1ERu>

Suitable for Your Scale — Phù hợp với Quy mô của Bạn (Suitable for Your Scale)

Much of the work in the space of distributed systems observability has been inspired by work done in large-scale distributed systems. This unfortunately can lead to us trying to re-create solutions for systems of much larger scale than our own without understanding the trade-offs.

Phần lớn công việc trong không gian khả năng quan sát các hệ thống phân tán đã được truyền cảm hứng bởi công việc làm trong các hệ thống phân tán quy mô lớn. Điều này tiếc thay có thể dẫn đến việc chúng ta cố tái tạo các giải pháp cho các hệ thống có quy mô lớn hơn nhiều so với của chính mình mà không hiểu các đánh đổi (trade-off).

Systems with massive scale will often have to make specific trade-offs to reduce the functionality of their systems in order to handle the scale they operate at. Dapper, for example, had to make use of

a highly aggressive random sampling of data (effectively “dropping” a lot of information) to be able to cope with Google scale. As Ben Sigelman, founder of LightStep and creator of Dapper, puts it: 14

Các hệ thống với quy mô khổng lồ thường sẽ phải thực hiện các đánh đổi cụ thể để giảm chức năng của hệ thống nhằm xử lý được quy mô mà chúng vận hành. Dapper, chẳng hạn, đã phải dùng đến việc lấy mẫu ngẫu nhiên cực kỳ quyết liệt (thực chất “bỏ rơi” rất nhiều thông tin) để có thể kham nổi quy mô Google. Như Ben Sigelman, nhà sáng lập LightStep và người tạo ra Dapper, diễn đạt: 14

Google’s microservices generate about 5 billion RPCs per second; building observability tools that scale to 5B RPCs/sec therefore boils down to building observability tools that are profoundly feature poor. If your organization is doing more like 5 million RPCs/sec, that’s still quite impressive, but you should almost certainly not use what Google uses: at 1/1000th the scale, you can afford much more powerful features. You also ideally want a tool that can scale as you scale. Again, cost effectiveness can come into play here. Even if your tool of choice can technically scale to support the expected growth of your system, can you afford to keep paying for it?

Các microservice của Google sinh ra khoảng 5 tỉ RPC mỗi giây; do đó xây dựng các công cụ khả năng quan sát mở rộng được tới 5 tỉ RPC/giây rất cuộc quy về xây dựng các công cụ khả năng quan sát nghèo nàn về tính năng sâu sắc. Nếu tổ chức của bạn đang làm kiểu như 5 triệu RPC/giây, thì điều đó vẫn khá ấn tượng, nhưng bạn gần như chắc chắn không nên dùng cái Google dùng: ở quy mô bằng 1/1000, bạn có thể đủ khả năng cho các tính năng mạnh hơn nhiều. Bạn cũng lý tưởng nhất là muốn một công cụ có thể mở rộng khi bạn mở rộng. Một lần nữa, hiệu quả chi phí có thể vào cuộc ở đây. Ngay cả khi công cụ bạn chọn về mặt kỹ thuật có thể mở rộng để hỗ trợ sự tăng trưởng kỳ vọng của hệ thống bạn, liệu bạn có đủ khả năng tiếp tục trả tiền cho nó không?

The Expert in the Machine — Chuyên gia trong Cỗ máy (The Expert in the Machine)

I’ve talked a lot about tools in this chapter, perhaps more than in any other chapter in the book. This is partly due to the fundamental shift from viewing the world purely in terms of monitoring to instead thinking about how to make our systems more observable; this change in behavior requires tooling to help support it. It would be a mistake, however, to see this shift as being purely about new tools, as I hope I’ve already explained. Nonetheless, with a large number of different vendors vying for our attention, we also have to be cautious.

Tôi đã nói rất nhiều về các công cụ trong chương này, có lẽ nhiều hơn bất kỳ chương nào khác trong cuốn sách. Điều này một phần là do sự chuyển dịch cơ bản từ việc nhìn thế giới thuần túy theo nghĩa giám sát sang thay vào đó suy nghĩ về cách làm cho hệ thống của chúng ta quan sát được hơn; sự thay đổi hành vi này đòi hỏi công cụ để giúp hỗ trợ nó. Tuy nhiên, sẽ là một sai lầm khi xem sự chuyển dịch này thuần túy là về các công cụ mới, như tôi hy vọng mình đã giải thích. Tuy vậy, với một số lượng lớn các nhà cung cấp khác nhau tranh giành sự chú ý của chúng ta, chúng ta cũng phải thận trọng.

For over a decade now, I have seen multiple vendors claim some degree of smarts in terms of how their system will be the one that will magically detect problems and tell us exactly what we need to do to fix things. This seems to come in waves, but with the recent buzz around machine learning (ML) and artificial intelligence (AI) only increasing, I’m seeing more claims around automated anomaly detection being made. I am dubious as to how effective this can be in a fully automated fashion, and even then it would be problematic to assume that all the expertise you need could be automated

away.

Trong hơn một thập kỷ nay, tôi đã thấy nhiều nhà cung cấp tuyên bố một mức độ thông minh nào đó về việc hệ thống của họ sẽ là cái phát hiện vấn đề một cách thần kỳ và nói cho chúng ta biết chính xác chúng ta cần làm gì để sửa chữa mọi thứ. Điều này dường như đến theo từng đợt, nhưng với sự xôn xao gần đây quanh học máy (ML) và trí tuệ nhân tạo (AI) chỉ ngày càng tăng, tôi đang thấy nhiều tuyên bố hơn quanh việc phát hiện bất thường tự động (automated anomaly detection) được đưa ra. Tôi hoài nghi về việc điều này có thể hiệu quả thế nào theo kiểu hoàn toàn tự động, và ngay cả khi đó thì cũng sẽ là vấn đề khi giả định rằng tất cả chuyên môn bạn cần có thể được tự động hóa đi.

14 Ben Sigelman, “Three Pillars with Zero Answers—Towards a New Scorecard for Observability,” Lightstep (blog post), December 5, 2018, <https://oreil.ly/R3LwC> .

14 Ben Sigelman, “Three Pillars with Zero Answers—Towards a New Scorecard for Observability,” Lightstep (blog post), December 5, 2018, <https://oreil.ly/R3LwC> .

Much of the drive around AI has always been to try and codify expert knowledge into an automated system. The idea that we can automate away expertise might be attractive to some, but it’s also a potentially dangerous idea in this space, at least with our current understanding. Why automate away expertise? So then you don’t have to invest in having expert operators run your system. I’m not trying to make some point here about the upheaval of labor caused by technological advancement—more that people are selling this idea now, and companies are buying this idea now, in the hope that these are fully solvable (and automatable) problems. The reality is that right now, they aren’t.

Phần lớn động lực quanh AI luôn là cố mã hóa kiến thức chuyên gia vào một hệ thống tự động. Ý tưởng rằng chúng ta có thể tự động hóa đi chuyên môn có thể hấp dẫn với một số người, nhưng nó cũng là một ý tưởng có khả năng nguy hiểm trong không gian này, ít nhất với sự hiểu biết hiện tại của chúng ta. Tại sao lại tự động hóa đi chuyên môn? Để rồi bạn không phải đầu tư vào việc có các operator chuyên gia vận hành hệ thống của mình. Tôi không cố đưa ra một luận điểm nào đó ở đây về sự xáo trộn lao động do tiến bộ công nghệ gây ra—mà hơn thế là việc người ta đang bán ý tưởng này lúc này, và các công ty đang mua ý tưởng này lúc này, với hy vọng rằng đây là những vấn đề hoàn toàn giải được (và tự động hóa được). Thực tế là ngay bây giờ, chúng không phải vậy.

I worked recently with a data science–focused startup in Europe. The startup was working with a company that provided bed-monitoring hardware, which could gather various pieces of data about a patient. The data scientists were able to help see patterns in the data, showing odd clusters of patients that could be determined by correlating various facets of the data. The data scientists could say “these patients seem related” but had no awareness as to what the meaning of that relationship was. It took a clinician to explain that some of these clusters referred to patients who were, in general, more sick than others. It required expertise to identify the cluster, and a different expertise to understand what this cluster meant and put that knowledge into action. Coming back to our tooling for monitoring and observability tooling, I could see such a tool alerting someone to the fact that “something looks odd,” but knowing what to do with that information still requires a degree of expertise.

Gần đây tôi làm việc với một startup tập trung vào khoa học dữ liệu ở châu Âu. Startup này đang làm việc với một công ty cung cấp phần cứng giám sát giường bệnh, thứ có thể thu thập nhiều mẫu dữ liệu khác nhau về một bệnh nhân. Các nhà khoa học dữ liệu có thể giúp thấy các mẫu hình trong dữ liệu, cho thấy các cụm bệnh nhân kỳ lạ có thể được xác định bằng cách tương quan các khía cạnh khác nhau của dữ liệu. Các nhà khoa học dữ liệu có thể nói “những bệnh nhân này dường như liên quan” nhưng không có nhận thức nào về ý nghĩa của mối quan hệ đó là gì. Phải cần một bác sĩ lâm sàng giải thích rằng một số cụm này quy về các bệnh nhân, nói chung, ốm nặng hơn những người khác. Cần chuyên

môn để xác định cụm, và một chuyên môn khác để hiểu cụm này nghĩa là gì và đưa kiến thức đó vào hành động. Quay lại với công cụ của chúng ta cho giám sát và khả năng quan sát, tôi có thể thấy một công cụ như vậy cảnh báo cho ai đó về sự thật rằng “có gì đó trông kỳ lạ,” nhưng biết phải làm gì với thông tin đó vẫn đòi hỏi một mức độ chuyên môn.

While I’m sure that features such as “automated anomaly detection” may well continue to improve, we have to recognize that right now, the expert in the system is, and will remain for some time, a human. We can create tools that can better inform the operator of what needs to be done, and we can provide automation to help the operator carry out their decisions in a more effective manner. But the fundamentally varied and complicated environment of a distributed system means that we need skilled and supported human operators. We want our experts to use their expertise in asking the right questions and making the best decisions they can. We shouldn’t be asking them to use their expertise to deal with the shortcomings of poor tools. Nor should we succumb to the convenient notion that some fancy new tool will solve all our problems.

Trong khi tôi chắc rằng các tính năng như “phát hiện bất thường tự động” rất có thể sẽ tiếp tục cải thiện, chúng ta phải nhận ra rằng ngay bây giờ, chuyên gia trong hệ thống là, và sẽ vẫn là trong một thời gian nữa, một con người. Chúng ta có thể tạo ra các công cụ có thể thông báo tốt hơn cho operator về việc cần làm gì, và chúng ta có thể cung cấp tự động hóa để giúp operator thực hiện các quyết định của họ một cách hiệu quả hơn. Nhưng môi trường vốn đa dạng và phức tạp một cách cơ bản của một hệ thống phân tán nghĩa là chúng ta cần các operator con người có kỹ năng và được hỗ trợ. Chúng ta muốn các chuyên gia của mình dùng chuyên môn của họ vào việc đặt đúng câu hỏi và đưa ra những quyết định tốt nhất có thể. Chúng ta không nên yêu cầu họ dùng chuyên môn của mình để đối phó với các thiếu sót của những công cụ tồi. Chúng ta cũng không nên khuất phục trước cái ý niệm tiện lợi rằng một công cụ mới hào nhoáng nào đó sẽ giải quyết tất cả các vấn đề của chúng ta.

Getting Started — Bắt đầu (Getting Started)

As I’ve outlined, there is a lot to think about here. But I want to provide a basic starting point for a simple microservice architecture in terms of what and how you should capture things.

Như tôi đã phác ra, có rất nhiều điều để nghĩ ở đây. Nhưng tôi muốn cung cấp một điểm khởi đầu cơ bản cho một kiến trúc microservice đơn giản về việc bạn nên bắt và bắt những thứ gì như thế nào.

To start off with, you want to be able to capture basic information about the hosts your microservices are running on—CPU rate, I/O, and so on—and make sure you can match a microservice instance back to the host it is running on. For each microservice instance, you want to capture response times for its service interfaces and record all downstream calls in logs. Get correlation IDs into your logs from the start. Log other major steps in your business processes. This will require that you have at least a basic metric and log aggregation toolchain in place.

Để bắt đầu, bạn muốn có thể bắt thông tin cơ bản về các host mà các microservice của bạn đang chạy trên đó—tỉ lệ CPU, I/O, v.v.—và đảm bảo bạn có thể khớp một instance microservice trở lại với host mà nó đang chạy trên đó. Với mỗi instance microservice, bạn muốn bắt thời gian phản hồi cho các giao diện dịch vụ của nó và ghi lại tất cả các lời gọi downstream trong log. Đưa correlation ID vào log của bạn ngay từ đầu. Ghi log các bước chính khác trong các quy trình nghiệp vụ của bạn. Điều này sẽ đòi hỏi rằng bạn có ít nhất một chuỗi công cụ tổng hợp metric và gom log cơ bản tại chỗ.

I'd hesitate to say that you need to start with a dedicated distributed tracing tool. If you have to run and host the tool yourself, this can add significant complexity. On the other hand, if you can make use of a fully managed service offering easily, instrumenting your microservices from the start can make a lot of sense.

Tôi sẽ ngần ngại nói rằng bạn cần bắt đầu với một công cụ distributed tracing chuyên dụng. Nếu bạn phải tự chạy và host công cụ, điều này có thể thêm độ phức tạp đáng kể. Mặt khác, nếu bạn có thể tận dụng dễ dàng một dịch vụ được quản lý hoàn toàn, việc trang bị (instrument) các microservice của bạn ngay từ đầu có thể rất hợp lý.

For key operations, strongly consider creating synthetic transactions as a way of better understanding if the vital aspects of your system are working properly. Build your system with this capability in mind.

Với các thao tác then chốt, hãy mạnh mẽ cân nhắc tạo các giao dịch tổng hợp như một cách hiểu rõ hơn liệu các khía cạnh sống còn của hệ thống bạn có đang hoạt động đúng không. Hãy xây dựng hệ thống của bạn với năng lực này trong đầu.

All of this is just the basic information gathering. More importantly, you need to make sure you can sift this information to ask questions of the running system. Are you able to say with confidence that the system is properly working for your users? Over time, you'll need to collect more information and improve your tooling (and the way you use it) to better improve the observability of your platform.

Tất cả những điều này chỉ là việc thu thập thông tin cơ bản. Quan trọng hơn, bạn cần đảm bảo mình có thể sàng lọc thông tin này để đặt câu hỏi về hệ thống đang chạy. Bạn có thể nói với sự tự tin rằng hệ thống đang hoạt động đúng cho người dùng của bạn không? Theo thời gian, bạn sẽ cần thu thập nhiều thông tin hơn và cải thiện công cụ của mình (và cách bạn dùng nó) để cải thiện tốt hơn khả năng quan sát của nền tảng của bạn.

Summary — Tóm tắt (Summary)

Distributed systems can be complicated to understand, and the more distributed they are, the more difficult the task of production troubleshooting becomes. When the pressure is on, alerts are blazing, and customers are screaming, it's important you have the right information available to you to work out what the hell is going on and what you need to do to fix it.

Các hệ thống phân tán có thể phức tạp để hiểu, và chúng càng phân tán, nhiệm vụ xử lý sự cố production càng trở nên khó khăn. Khi áp lực đè nặng, các cảnh báo đang réo inh ỏi, và khách hàng đang gào thét, điều quan trọng là bạn có thông tin đúng sẵn có để tìm ra root cuộc chuyện quái gì đang diễn ra và bạn cần làm gì để sửa nó.

As your microservice architecture becomes more complex, it becomes less easy to know in advance what problems may happen. Instead, you will frequently be surprised by the types of problems you will encounter. It thus becomes essential to shift your thinking away from the mostly (passive) activity of monitoring and toward actively making your system observable. This involves not just potentially changing your toolset but also shifting away from static dashboards to more dynamic slicing and dicing activities.

Khi kiến trúc microservice của bạn trở nên phức tạp hơn, càng khó để biết trước những vấn đề nào có thể xảy ra. Thay vào đó, bạn sẽ thường xuyên bị bất ngờ bởi các loại vấn đề bạn sẽ gặp. Do đó việc chuyển dịch tư duy của bạn ra khỏi hoạt động chủ yếu (thụ động) là giám sát và hướng tới việc chủ động làm cho hệ thống của bạn quan sát được trở nên thiết yếu. Điều này liên quan không chỉ đến việc có khả năng thay đổi bộ công cụ của bạn mà

còn việc chuyển dịch khỏi các dashboard tĩnh sang các hoạt động mổ xẻ, phân tích động hơn.

With a simple system, the basics will get you a long way. Get log aggregation in from the start, and get correlation IDs in your log lines too. Distributed tracing can follow later, but be on the lookout for when the time comes to put that into place.

Với một hệ thống đơn giản, những điều cơ bản sẽ đưa bạn đi xa. Đưa gom log vào ngay từ đầu, và đưa correlation ID vào các dòng log của bạn nữa. Distributed tracing có thể theo sau về sau, nhưng hãy để mắt đến khi nào đến lúc đưa nó vào.

Shift your understanding of system or microservice health away from a binary state of “happy” or “sad”; realize instead that the truth is always more nuanced than that. Switch from having every little problem generating an alert to thinking more holistically about what is acceptable. Strongly consider embracing SLOs and alerting based on these precepts to reduce alert fatigue and properly focus attention.

Hãy chuyển dịch sự hiểu biết của bạn về sức khỏe của hệ thống hay microservice ra khỏi một trạng thái nhị phân “vui” hay “buồn”; thay vào đó hãy nhận ra rằng sự thật luôn nhiều sắc thái hơn thế. Hãy chuyển từ việc có mọi vấn đề nhỏ nhặt sinh ra một cảnh báo sang suy nghĩ toàn diện hơn về việc thế nào là chấp nhận được. Hãy mạnh mẽ cân nhắc đón nhận các SLO và cảnh báo dựa trên các nguyên tắc này để giảm mệt mỏi vì cảnh báo và tập trung sự chú ý đúng cách.

CHAPTER 11 Security — Chương 11:

Bảo mật

I want to preface this chapter by saying that I do not consider myself to be an expert in the space of application **security**. I aim simply to be a conscious incompetent—in other words, I want to understand what I don’t know and be aware of my limits. Even as I learn more about this space, I learn that there is still more to know. This isn’t to say that educating yourself about topics like this is pointless—I feel that everything I’ve learned about this space over the last ten years has made me a more effective developer and architect.

Tôi muốn mở đầu chương này bằng việc nói rằng tôi không tự coi mình là một chuyên gia trong lĩnh vực bảo mật ứng dụng. Tôi chỉ đơn giản muốn trở thành một người “thiếu năng lực nhưng có ý thức” — nói cách khác, tôi muốn hiểu được những gì mình chưa biết và nhận thức được giới hạn của bản thân. Ngay cả khi học thêm về lĩnh vực này, tôi vẫn nhận ra rằng còn rất nhiều điều cần phải biết. Điều này không có nghĩa là việc tự trang bị kiến thức về những chủ đề như vậy là vô ích — tôi cảm thấy rằng mọi thứ tôi học được về lĩnh vực này trong mười năm qua đã giúp tôi trở thành một lập trình viên và kiến trúc sư hiệu quả hơn.

In this chapter, I highlight those aspects of security that I consider worthwhile for a general developer, architect, or operations person working on a **microservice architecture** to understand. There is still a need to have the support of experts in the space of application security—but even if you have access to such people, it’s still important for you to have some grounding in these topics. In the same way that developers have learned more about **testing** or data management, topics previously restricted to specialists, having a general awareness of security topics can be vital in terms of building security into our software from the beginning.

Trong chương này, tôi sẽ làm nổi bật những khía cạnh của bảo mật (security) mà tôi cho là đáng để một lập trình viên, kiến trúc sư hay người vận hành nói chung làm việc trên kiến trúc microservice nên hiểu. Vẫn cần có sự hỗ trợ của các chuyên gia trong lĩnh vực bảo mật ứng dụng — nhưng ngay cả khi bạn có quyền tiếp cận những người như vậy, việc bạn có một nền tảng nhất định về các chủ đề này vẫn rất quan trọng. Cũng giống như cách các lập trình viên đã học thêm về kiểm thử (testing) hay quản lý dữ liệu, những chủ đề trước đây vốn chỉ dành riêng cho các chuyên gia, việc có nhận thức chung về các chủ đề bảo mật có thể là yếu tố sống còn để đưa tính bảo mật vào phần mềm của chúng ta ngay từ đầu.

When comparing **microservices** to less distributed architectures, it turns out we are presented with an interesting dichotomy. On the one hand, we now have more data flowing over networks that previously would stay on a single machine, and we have more complex infrastructure running our architecture—our surface area of attack is much greater. On the other hand, microservices give

us more opportunity to defend in depth and limit the scope of access, potentially increasing the projection of our system while also reducing the impact of an attack should it occur. This apparent paradox—that microservices can make our systems both less secure and more secure—is really just a delicate balancing act. My hope is that by the end of this chapter you’ll end up on the right side of this equation.

Khi so sánh microservice với các kiến trúc ít phân tán hơn, hóa ra chúng ta lại đối mặt với một nghịch lý thú vị. Một mặt, giờ đây chúng ta có nhiều dữ liệu hơn truyền qua mạng, trong khi trước kia chúng vẫn nằm trên một máy duy nhất, và chúng ta có hạ tầng phức tạp hơn để chạy kiến trúc của mình — diện tích bề mặt tấn công của chúng ta lớn hơn nhiều. Mặt khác, microservice cho chúng ta nhiều cơ hội hơn để phòng thủ nhiều lớp (defense in depth) và giới hạn phạm vi truy cập, qua đó vừa có thể tăng khả năng bảo vệ hệ thống vừa giảm tác động của một cuộc tấn công nếu nó xảy ra. Nghịch lý hiển nhiên này — rằng microservice có thể khiến hệ thống của chúng ta vừa kém an toàn hơn vừa an toàn hơn — thực ra chỉ là một sự cân bằng tinh tế. Hy vọng của tôi là đến cuối chương này, bạn sẽ kết thúc ở phía đúng của phương trình này.

To help you find the right balance when it comes to security of your microservice architecture, we’ll cover the following topics:

Để giúp bạn tìm ra sự cân bằng phù hợp khi nói đến bảo mật trong kiến trúc microservice của mình, chúng ta sẽ đề cập đến các chủ đề sau:

Core principles Fundamental concepts that are useful to embrace when looking to **build** more secure software

Nguyên tắc cốt lõi Các khái niệm nền tảng hữu ích nên đón nhận khi muốn xây dựng phần mềm an toàn hơn

The five functions of cybersecurity Identify, protect, detect, respond, and recover—an overview of the five key function areas for application security

Năm chức năng của an ninh mạng Nhận diện, bảo vệ, phát hiện, ứng phó và phục hồi — tổng quan về năm vùng chức năng then chốt của bảo mật ứng dụng

Foundations of application security Some specific fundamental concepts of application security and how they apply to microservices, including credentials and secrets, patching, backups, and rebuild

Nền tảng của bảo mật ứng dụng Một số khái niệm nền tảng cụ thể của bảo mật ứng dụng và cách chúng áp dụng cho microservice, bao gồm credential và secret, vá lỗi (patching), sao lưu (backup), và dựng lại (rebuild)

Implicit trust versus **zero trust** Different approaches for trust in our microservice environment and how this impacts security-related activities

Tin cậy ngầm định và zero trust Các cách tiếp cận khác nhau về sự tin cậy trong môi trường microservice của chúng ta và việc điều này tác động ra sao đến các hoạt động liên quan đến bảo mật

Securing data How we protect data as it travels over networks and as it rests on disk

Bảo vệ dữ liệu Cách chúng ta bảo vệ dữ liệu khi nó di chuyển qua mạng và khi nó nằm yên trên đĩa

Authentication and **authorization** How single sign-on (SSO) works in a microservice architecture, centralized versus decentralized authorization models, and the role of JWT tokens as part of this

Xác thực và phân quyền Cách single sign-on (SSO) hoạt động trong kiến trúc microservice, các mô hình phân quyền tập trung và phi tập trung, cùng vai trò của JWT token trong việc này

Core Principles — Các nguyên tắc cốt lõi

Often when the topic of microservice security comes up, people want to start talking about reasonably sophisticated technological issues like the use of JWT tokens or the need for mutual TLS (topics we will explore later in this chapter). However, the problem with security is that you're only as secure as your least secure aspect. To use an analogy, if you're looking to secure your home, it would be a mistake to focus all your efforts on having a front door that is pick resistant, with lights and cameras to deter malicious parties, if you leave your back door open.

Thường thì khi chủ đề bảo mật microservice được nhắc tới, mọi người muốn bắt đầu bàn về những vấn đề công nghệ khá phức tạp như việc sử dụng JWT token hay nhu cầu dùng mutual TLS (những chủ đề chúng ta sẽ khám phá sau trong chương này). Tuy nhiên, vấn đề của bảo mật là bạn chỉ an toàn ngang với khía cạnh kém an toàn nhất của mình. Lấy một phép ẩn dụ, nếu bạn muốn bảo vệ ngôi nhà của mình, sẽ là một sai lầm nếu dồn toàn bộ công sức vào việc có một cửa trước chống cạy phá, với đèn và camera để răn đe những kẻ có ý đồ xấu, mà lại để cửa sau mở toang.

Thus, there are some fundamental aspects of application security that we need to look at, however briefly, to highlight the plethora of issues you need to be aware of. We will look at how these core issues are made more (or less) complex in the context of microservices, but they should also be generally applicable to software development as a whole. For those of you who want to jump ahead to all that “good stuff,” please just make sure you aren't focusing too much on securing your front door while leaving the back door open.

Vì vậy, có một số khía cạnh nền tảng của bảo mật ứng dụng mà chúng ta cần xem xét, dù chỉ là sơ qua, để làm nổi bật vô số vấn đề mà bạn cần lưu ý. Chúng ta sẽ xem xét cách những vấn đề cốt lõi này trở nên phức tạp hơn (hoặc bớt phức tạp hơn) trong bối cảnh microservice, nhưng chúng cũng nên áp dụng được một cách tổng quát cho phát triển phần mềm nói chung. Đối với những bạn muốn nhảy ngay đến tất cả những thứ “hay ho” đó, hãy chỉ cần chắc chắn rằng bạn không quá tập trung vào việc bảo vệ cửa trước trong khi để cửa sau mở.

Principle of Least Privilege — Nguyên tắc Đặc quyền Tối thiểu

When granting application access to individuals, external or internal systems, or even our own microservices, we should pay careful attention to what access we grant. The principle of least privilege describes the idea that when granting access we should grant the minimum access a party needs to carry out the required functionality, and only for the time period they need it. The main benefit of this is to ensure that if credentials are compromised by an attacker, those credentials will give the malicious party as limited access as possible.

Khi cấp quyền truy cập ứng dụng cho các cá nhân, các hệ thống bên ngoài hoặc nội bộ, hay thậm chí cho chính các microservice của mình, chúng ta nên hết sức chú ý đến quyền truy cập mà mình cấp. Nguyên tắc đặc quyền tối thiểu (principle of least privilege) mô tả ý tưởng rằng khi cấp quyền truy cập, chúng ta chỉ nên cấp mức truy cập tối thiểu mà một bên cần để thực hiện chức năng được yêu cầu, và chỉ trong khoảng thời gian họ cần. Lợi

ích chính của điều này là đảm bảo rằng nếu credential bị kẻ tấn công xâm phạm, những credential đó sẽ trao cho kẻ xấu mức truy cập hạn chế nhất có thể.

If a microservice only has read-only access to a **database**, then an attacker gaining access to those database credentials will only gain read-only access, and only to that database. If the **credential** to the database expires before it is compromised, then the credential becomes useless. This concept can be extended to limit which microservices can be communicated with by specific parties.

Nếu một microservice chỉ có quyền truy cập chỉ-đọc vào một cơ sở dữ liệu, thì kẻ tấn công lấy được quyền truy cập vào credential của cơ sở dữ liệu đó cũng chỉ có được quyền chỉ-đọc, và chỉ với cơ sở dữ liệu đó. Nếu credential đối với cơ sở dữ liệu hết hạn trước khi nó bị xâm phạm, thì credential đó trở nên vô dụng. Khái niệm này có thể được mở rộng để giới hạn những microservice nào có thể được giao tiếp bởi các bên cụ thể.

As we'll see later in this chapter, the principle of least privilege can extend to ensuring that access controls are only granted for limited timeframes as well, further limiting any bad outcomes that can occur in the event of a compromise.

Như chúng ta sẽ thấy sau trong chương này, nguyên tắc đặc quyền tối thiểu còn có thể mở rộng đến việc đảm bảo rằng các quyền kiểm soát truy cập chỉ được cấp trong những khung thời gian giới hạn, qua đó càng hạn chế hơn những hậu quả xấu có thể xảy ra trong trường hợp bị xâm phạm.

Defense in Depth — Phòng thủ nhiều lớp

The UK, where I live, is full of castles. These castles are a partial reminder of our country's history (not least a reminder of a time before the United Kingdom was, well, united to some degree). They remind us of a time when people felt the need to defend their property from foes. Sometimes the perceived foes were different—many of the castles near where I live in Kent were designed to defend against coastal invasions from France. 1 Whatever the reason, castles can be a great example of the principle of **defense in depth**.

Vương quốc Anh, nơi tôi sống, đầy rẫy lâu đài. Những lâu đài này phần nào là lời nhắc nhở về lịch sử của đất nước chúng tôi (chỉ ít là lời nhắc về một thời trước khi Vương quốc Anh, à thì, được hợp nhất ở một mức độ nào đó). Chúng nhắc chúng ta về một thời kỳ khi con người cảm thấy cần phải bảo vệ tài sản của mình khỏi kẻ thù. Đôi khi những kẻ thù được nhận thức là khác nhau — nhiều lâu đài gần nơi tôi sống ở Kent được thiết kế để phòng thủ trước các cuộc xâm lược từ biển của nước Pháp. 1 Dù lý do là gì, lâu đài có thể là một ví dụ tuyệt vời cho nguyên tắc phòng thủ nhiều lớp.

Having only one protection mechanism is a problem if an attacker finds a way to breach that defense, or if the protection mechanism defends against only certain types of attackers. Think of a coastal defense fort whose only wall faces the sea, leaving it totally defenseless against an attack by land. If you look at Dover Castle, which is very close to where I live, there are multiple protections that are easy to see. Firstly, it is on a large hill, making an approach to the castle by land difficult. It has not one wall but two—a breach in the first wall still requires that an attacker deals with the second one. And once you're through the final wall, you have a large and imposing keep (tower) to deal with.

Chỉ có một cơ chế bảo vệ duy nhất là một vấn đề nếu kẻ tấn công tìm ra cách phá vỡ phòng tuyến đó, hoặc nếu cơ chế bảo vệ chỉ chống được một số loại kẻ tấn công nhất định. Hãy nghĩ đến một pháo đài phòng thủ ven biển mà bức tường duy nhất của nó hướng ra biển, khiến nó hoàn toàn không có khả năng phòng thủ trước một cuộc tấn công từ đất liền. Nếu bạn nhìn vào Lâu đài Dover, vốn rất gần nơi tôi sống, có nhiều lớp bảo vệ dễ thấy. Thứ

nhất, nó nằm trên một ngọn đồi lớn, khiến việc tiếp cận lâu đài bằng đường bộ trở nên khó khăn. Nó không chỉ có một bức tường mà có hai — một lỗ thủng ở bức tường thứ nhất vẫn đòi hỏi kẻ tấn công phải đối phó với bức tường thứ hai. Và một khi bạn đã vượt qua bức tường cuối cùng, bạn còn phải đối mặt với một tháp chính (keep) lớn và uy nghi.

The same principle must apply when we build protections into our application security. Having multiple protections in place to defend against attackers is vital. With

Nguyên tắc tương tự phải được áp dụng khi chúng ta xây dựng các lớp bảo vệ vào bảo mật ứng dụng của mình. Việc có nhiều lớp bảo vệ tại chỗ để chống lại những kẻ tấn công là điều sống còn. Với

1 Please, let's not make everything about Brexit.

1 Làm ơn, đừng biến mọi thứ thành chuyện Brexit.

microservice architectures, we have many more places where we are able to protect our systems. By breaking our functionality apart into different microservices and limiting the scope of what those microservices can do, we're already applying defense in depth. We can also run microservices on different network segments, apply network-based protections at more places, and even make use of a mix of technology for building and running these microservices such that a single zero-day exploit may not impact everything we have.

kiến trúc microservice, chúng ta có nhiều nơi hơn để có thể bảo vệ hệ thống của mình. Bằng cách chia nhỏ chức năng thành các microservice khác nhau và giới hạn phạm vi những gì các microservice đó có thể làm, chúng ta đã đang áp dụng phòng thủ nhiều lớp. Chúng ta cũng có thể chạy các microservice trên các phân đoạn mạng khác nhau, áp dụng các lớp bảo vệ dựa trên mạng tại nhiều nơi hơn, và thậm chí dùng một hỗn hợp công nghệ để xây dựng và chạy các microservice này sao cho một lỗ hổng zero-day đơn lẻ có thể không tác động đến tất cả những gì chúng ta có.

Microservices provide more ability to defend in depth than equivalent single-process monolithic applications, and as a result, they can help organizations build more secure systems.

Microservice mang lại nhiều khả năng phòng thủ nhiều lớp hơn so với các ứng dụng monolith đơn-tiến-trình tương đương, và nhờ đó, chúng có thể giúp các tổ chức xây dựng những hệ thống an toàn hơn.

Types of Security Controls — Các loại kiểm soát bảo mật

When considering the security controls we might put in place to secure our system, we can categorize them as: 2

Khi xem xét các kiểm soát bảo mật (security control) mà chúng ta có thể đặt ra để bảo vệ hệ thống, chúng ta có thể phân loại chúng thành: 2

Preventative Stopping an attack from happening. This includes storing secrets securely, encrypting data at **rest** and in transit, and implementing proper authentication and authorization mechanisms.

Phòng ngừa Ngăn một cuộc tấn công xảy ra. Điều này bao gồm việc lưu trữ secret một cách an toàn, mã hóa dữ liệu khi nghỉ và khi truyền, cùng việc triển khai các cơ chế xác thực (authentication) và phân quyền (authorization) đúng đắn.

Detective **Alerting** you to the fact that an attack is happening/has happened. Application firewalls and intrusion detection services are good examples.

Phát hiện Cảnh báo cho bạn về việc một cuộc tấn công đang xảy ra/đã xảy ra. Tường lửa ứng dụng và các dịch vụ phát hiện xâm nhập là những ví dụ điển hình.

Responsive Helping you respond during/after an attack. Having an automated mechanism to rebuild your system, working backups to recover data, and a proper comms plan in place in the wake of an incident can be vital.

Ứng phó Giúp bạn ứng phó trong/sau một cuộc tấn công. Có một cơ chế tự động để dựng lại hệ thống, các bản sao lưu hoạt động được để khôi phục dữ liệu, và một kế hoạch truyền thông phù hợp đặt sẵn cho thời điểm sau sự cố có thể là điều sống còn.

A combination of all three will be required to properly secure a system, and you may have multiples of each type. Coming back to our castle example, we might have multiple walls, which represent multiple preventative controls. We could have watchtowers in place and a beacon system so we can see if an attack is happening. Finally, we might have some carpenters and stonemasons on standby in case we need to strengthen the doors or walls in the wake of an attack. Obviously, you are unlikely to be building castles for a living, so we'll look at examples of these controls for a microservice architecture later in this chapter.

Một sự kết hợp của cả ba loại sẽ là cần thiết để bảo vệ hệ thống một cách đúng đắn, và bạn có thể có nhiều cái thuộc mỗi loại. Quay lại ví dụ lâu đài của chúng ta, chúng ta có thể có nhiều bức tường, đại diện cho nhiều kiểm soát phòng ngừa. Chúng ta có thể có các tháp canh và một hệ thống đèn hiệu để có thể thấy nếu một cuộc tấn công đang diễn ra. Cuối cùng, chúng ta có thể có một số thợ mộc và thợ nề túc trực phòng khi cần gia cố cửa hoặc tường sau một cuộc tấn công. Hiển nhiên, bạn khó có khả năng đang kiếm sống bằng nghề xây lâu đài, nên chúng ta sẽ xem xét các ví dụ về những kiểm soát này cho một kiến trúc microservice sau trong chương này.

2 Try as I might, I can't find the original source for this categorization scheme.

2 Dù đã cố gắng hết sức, tôi vẫn không thể tìm ra nguồn gốc ban đầu của sơ đồ phân loại này.

Automation — Tự động hóa

A recurring theme of this book is automation. As we have so many more moving parts with microservice architectures, automation becomes key to helping us manage the growing complexity of our system. At the same time, we have a drive toward increasing the speed of delivery, and automation is essential here. Computers are way better at doing the same thing over and over again than humans are—they do it faster and more efficiently than we can (and with less variability as well). They can also reduce human error and make it easier to implement the principle of least privilege—we can assign specific privileges to specific scripts, for example.

Một chủ đề lặp đi lặp lại của cuốn sách này là tự động hóa. Vì chúng ta có nhiều thành phần chuyển động hơn rất nhiều với kiến trúc microservice, tự động hóa trở thành chìa khóa giúp chúng ta quản lý độ phức tạp ngày càng tăng của hệ thống. Đồng thời, chúng ta có động lực hướng tới việc tăng tốc độ chuyển giao, và tự động hóa là thiết yếu ở đây. Máy tính làm cùng một việc lặp đi lặp lại tốt hơn nhiều so với con người — chúng làm nhanh hơn và hiệu quả hơn chúng ta (và cũng với ít sự biến thiên hơn). Chúng cũng có thể giảm lỗi do con người và giúp dễ dàng triển khai nguyên tắc đặc quyền tối thiểu hơn — chẳng hạn, chúng ta có thể gán những đặc quyền cụ thể cho những script cụ thể.

As we'll see throughout this chapter, automation can help us recover in the wake of an incident. We can use it to revoke and rotate security keys and also make use of tooling to help detect potential

security issues more easily. As with other aspects of microservice architecture, embracing a culture of automation will help you immensely when it comes to security.

Như chúng ta sẽ thấy xuyên suốt chương này, tự động hóa có thể giúp chúng ta phục hồi sau một sự cố. Chúng ta có thể dùng nó để thu hồi và xoay vòng các khóa bảo mật, đồng thời tận dụng công cụ để giúp phát hiện các vấn đề bảo mật tiềm ẩn dễ dàng hơn. Cũng như các khía cạnh khác của kiến trúc microservice, đón nhận một văn hóa tự động hóa sẽ giúp ích cho bạn rất nhiều khi nói đến bảo mật.

Build Security into the Delivery Process — Đưa bảo mật vào quy trình chuyển giao

Like so many other aspects of software delivery, security is all too often considered an afterthought. Historically at least, addressing security aspects of a system is something that is done after the code has been written, potentially leading to significant reworking later on. Security has often been looked at as something of a blocker to getting software out the door.

Cũng như rất nhiều khía cạnh khác của việc chuyển giao phần mềm, bảo mật quá thường xuyên bị coi là chuyện tính sau. Chỉ ít là trong quá khứ, việc xử lý các khía cạnh bảo mật của một hệ thống là điều được thực hiện sau khi mã đã được viết xong, có khả năng dẫn đến việc phải làm lại đáng kể về sau. Bảo mật thường bị nhìn nhận như một thứ gì đó cản trở việc đưa phần mềm ra cửa.

Over the last 20 years, we’ve seen similar issues with testing, usability, and operations. These aspects of software delivery were frequently delivered in siloed fashion, often after the bulk of the code was finished. An old colleague of mine, Jonny Schneider, once likened the approach toward usability of software in terms of a “Do you want fries with that?” mentality. In other words, usability is an afterthought—something you sprinkle on top of the “main meal.” 3

Trong 20 năm qua, chúng ta đã thấy những vấn đề tương tự với kiểm thử, tính khả dụng (usability), và vận hành. Những khía cạnh này của việc chuyển giao phần mềm thường được thực hiện theo kiểu tách biệt (siloed), thường là sau khi phần lớn mã đã hoàn thành. Một đồng nghiệp cũ của tôi, Jonny Schneider, từng ví cách tiếp cận đối với tính khả dụng của phần mềm theo kiểu tư duy “Bạn có muốn gọi thêm khoai tây chiên không?”. Nói cách khác, tính khả dụng là một thứ tính sau — một thứ bạn rắc lên trên “món chính”. 3

The reality, of course, is that software that isn’t usable, isn’t secure, can’t operate properly in production, and is riddled with bugs isn’t in any way shape or form a “main meal”—it’s a flawed offering at best. We’ve gotten better at pulling testing forward into the main delivery process, as we’ve done with operational aspects (DevOps, anyone?) and usability—security should be no different. We need to ensure that developers have a more general awareness of security-related concerns, that specialists find a way to embed themselves into delivery teams when required, and that tooling improves to allow us to build security-related thinking into our software.

Tất nhiên, thực tế là phần mềm không khả dụng, không an toàn, không thể vận hành đúng cách trong môi trường production, và đầy rẫy lỗi thì không hề là một “món chính” theo bất kỳ hình thức nào — tốt nhất thì nó cũng chỉ là một sản phẩm khiếm khuyết. Chúng ta đã làm tốt hơn trong việc kéo kiểm thử vào sớm trong quy trình chuyển giao chính, như chúng ta đã làm với các khía cạnh vận hành (DevOps, có ai không nhỉ?) và tính khả dụng — bảo mật cũng không nên khác. Chúng ta cần đảm bảo rằng các lập trình viên có nhận thức chung hơn về những mối quan tâm liên quan đến bảo mật, rằng các chuyên gia tìm

được cách nhúng mình vào các nhóm chuyển giao khi cần, và rằng công cụ được cải thiện để cho phép chúng ta đưa tư duy liên quan đến bảo mật vào phần mềm của mình.

3 I recommend Jonny’s Understanding Design Thinking, Lean, and Agile (O’Reilly) for more of his insights.

3 Tôi giới thiệu cuốn Understanding Design Thinking, Lean, and Agile (O’Reilly) của Jonny để có thêm nhiều hiểu biết của anh ấy.

This can create challenges in organizations adopting stream-aligned teams with increased degrees of autonomy around the **ownership** of their microservices. What is the role of security experts? In “Enabling Teams” on page 503, we’ll look at how specialists like security experts can work to support stream-aligned teams and help microservice owners build more security thinking into their software, but also to make sure that you have the right deep expertise on hand when you need it.

Điều này có thể tạo ra thách thức ở các tổ chức áp dụng stream-aligned team với mức độ tự chủ ngày càng cao quanh quyền sở hữu các microservice của họ. Vai trò của các chuyên gia bảo mật là gì? Trong “Enabling Teams” ở trang 503, chúng ta sẽ xem xét cách các chuyên gia như chuyên gia bảo mật có thể làm việc để hỗ trợ các stream-aligned team và giúp những người sở hữu microservice đưa thêm tư duy bảo mật vào phần mềm của họ, đồng thời cũng đảm bảo rằng bạn có sẵn chuyên môn sâu phù hợp khi cần.

There are automated tools that can probe our systems for vulnerabilities, such as by looking for cross-site scripting attacks. The Zed Attack Proxy (aka ZAP) is a good example. Informed by the work of OWASP, ZAP attempts to re-create malicious attacks on your website. Other tools exist that use static analysis to look for common coding mistakes that can open up security holes, such as Brakeman for Ruby; there are also tools like Snyk, which among other things can pick up dependencies on third-party libraries that have known vulnerabilities. Where these tools can be easily integrated into normal CI builds, integrating them into your standard check-ins is a great place to start. Of course, it’s worth noting that many of these types of tools can address only local issues—for example, a vulnerability in a specific piece of code. They don’t replace the need to understand the security of your system at a wider, systemic level.

Có những công cụ tự động có thể dò tìm lỗ hổng trong hệ thống của chúng ta, chẳng hạn bằng cách tìm kiếm các cuộc tấn công cross-site scripting. Zed Attack Proxy (còn gọi là ZAP) là một ví dụ điển hình. Được dẫn dắt bởi công trình của OWASP, ZAP cố gắng tái tạo các cuộc tấn công độc hại lên website của bạn. Có những công cụ khác sử dụng phân tích tĩnh để tìm kiếm các lỗi lập trình phổ biến có thể mở ra lỗ hổng bảo mật, chẳng hạn Brakeman cho Ruby; cũng có những công cụ như Snyk, mà ngoài những thứ khác, có thể phát hiện các phụ thuộc vào thư viện của bên thứ ba có lỗ hổng đã biết. Ở những nơi mà các công cụ này có thể dễ dàng tích hợp vào các bản build CI thông thường, việc tích hợp chúng vào các lần check-in tiêu chuẩn của bạn là một điểm khởi đầu tuyệt vời. Tất nhiên, cũng đáng lưu ý rằng nhiều loại công cụ như thế này chỉ có thể xử lý các vấn đề cục bộ — ví dụ, một lỗ hổng trong một đoạn mã cụ thể. Chúng không thay thế được nhu cầu hiểu về bảo mật của hệ thống ở cấp độ rộng hơn, mang tính hệ thống.

The Five Functions of Cybersecurity — Năm chức năng của an ninh mạng

With those core principles at the back of our minds, let’s now consider the broad-ranging security-related activities that we need to perform. We’ll then go on to understand how these activities vary in the context of a microservice architecture. The model I prefer for describing the universe of

application security comes from the US National Institute of Standards and Technology (NIST), which outlines a useful five-part model for the various activities involved in cybersecurity:

Với những nguyên tắc cốt lõi đó nằm trong tâm trí, giờ hãy cùng xem xét các hoạt động liên quan đến bảo mật trên phạm vi rộng mà chúng ta cần thực hiện. Sau đó chúng ta sẽ tiếp tục hiểu xem các hoạt động này thay đổi như thế nào trong bối cảnh kiến trúc microservice. Mô hình mà tôi ưa thích để mô tả toàn bộ vũ trụ của bảo mật ứng dụng đến từ Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST), nơi phác thảo một mô hình hữu ích gồm năm phần cho các hoạt động khác nhau liên quan đến an ninh mạng:

- Identify who your potential attackers are, what targets they are trying to acquire, and where you are most vulnerable.
- Protect your key assets from would-be hackers.
- Detect if an attack has happened, despite your best efforts.
- Respond when you've found out something bad has occurred.
- Recover in the wake of an incident.

- Nhận diện (Identify) xem ai là những kẻ tấn công tiềm năng của bạn, họ đang cố nhắm tới những mục tiêu nào, và bạn dễ bị tổn thương nhất ở đâu.
- Bảo vệ (Protect) các tài sản then chốt của bạn khỏi những kẻ muốn xâm nhập.
- Phát hiện (Detect) xem một cuộc tấn công đã xảy ra hay chưa, bất chấp những nỗ lực tốt nhất của bạn.
- Ứng phó (Respond) khi bạn đã phát hiện ra điều gì đó tồi tệ đã xảy ra.
- Phục hồi (Recover) sau một sự cố.

I find this model to be especially useful due to its holistic nature. It's very easy to put all your effort into protecting your application without first considering what threats you might actually face, let alone working out what you might do if a clever attacker does slip past your defenses.

Tôi thấy mô hình này đặc biệt hữu ích nhờ bản chất toàn diện của nó. Rất dễ để dồn toàn bộ công sức vào việc bảo vệ ứng dụng của bạn mà chưa cân nhắc trước xem bạn thực sự có thể đối mặt với những mối đe dọa nào, chứ chưa nói đến việc tính ra bạn có thể làm gì nếu một kẻ tấn công thông minh quả thực lọt qua các phòng tuyến của bạn.

Let's explore each of these functions in a bit more depth and look at how a microservice architecture might change how you approach these ideas in comparison to a more traditional monolithic architecture.

Hãy cùng khám phá từng chức năng này sâu hơn một chút và xem một kiến trúc microservice có thể thay đổi cách bạn tiếp cận những ý tưởng này như thế nào so với một kiến trúc monolith truyền thống hơn.

Identify — Nhận diện

Before we can work out what we should protect, we need to work out who might be after our stuff and what exactly they might be looking for. It's often hard to put ourselves into the mindset of an attacker, but that's exactly what we need to do to ensure that we focus our efforts in the right place. Threat modeling is the first thing you should look at when addressing this aspect of application security.

Trước khi có thể tính ra mình nên bảo vệ cái gì, chúng ta cần tính ra ai có thể đang nhắm đến tài sản của mình và chính xác thì họ có thể đang tìm kiếm điều gì. Thường thì rất khó để đặt mình vào tâm trí của một kẻ tấn công, nhưng đó chính xác là điều chúng ta cần làm để đảm bảo rằng mình tập trung công sức đúng chỗ. Mô hình hóa mối đe dọa (threat modeling) là thứ đầu tiên bạn nên xem xét khi giải quyết khía cạnh này của bảo mật ứng dụng.

As human beings, we are quite bad at understanding risk. We often fixate on the wrong things while ignoring the bigger problems that can be just out of sight. This of course extends into the field of

security. Our understanding about what security risks we might be exposed to is often largely colored by our limited view of the system, our skills, and our experiences.

Là con người, chúng ta khá tệ trong việc hiểu rủi ro. Chúng ta thường chăm chăm vào những thứ sai trong khi bỏ qua những vấn đề lớn hơn có thể vừa nằm ngoài tầm mắt. Điều này tất nhiên cũng mở rộng vào lĩnh vực bảo mật. Sự hiểu biết của chúng ta về những rủi ro bảo mật mà mình có thể phơi nhiễm thường bị tô màu phần lớn bởi cái nhìn hạn chế của chúng ta về hệ thống, bởi kỹ năng và kinh nghiệm của mình.

When I chat with developers about security risks in the context of microservice architecture, they immediately start talking about JWTs and mutual TLS. They reach for technical solutions to technical problems they have some visibility of. I don't mean to point the finger just at developers—all of us have a limited view of the world. Coming back to the analogy we used earlier, this is how we can end up with an incredibly secure front door and a wide-open back door.

Khi tôi trò chuyện với các lập trình viên về rủi ro bảo mật trong bối cảnh kiến trúc microservice, họ ngay lập tức bắt đầu nói về JWT và mutual TLS. Họ với tới các giải pháp kỹ thuật cho những vấn đề kỹ thuật mà họ có một chút khả năng nhìn thấy. Tôi không có ý chỉ trỏ ngón tay vào riêng các lập trình viên — tất cả chúng ta đều có một cái nhìn hạn chế về thế giới. Quay lại với phép ẩn dụ chúng ta đã dùng trước đó, đây là cách mà chúng ta có thể kết thúc với một cửa trước cực kỳ an toàn và một cửa sau mở toang.

At one company I worked at, there had been a number of discussions around the need to install closed-circuit TV (CCTV) cameras in the reception areas of the company's offices all over the world. This was due to an incident in which an unauthorized person had gained access to the front office area and then the corporate network. The belief was that a CCTV camera system would not only deter others from trying the same thing again but also help identify the individuals involved after the fact.

Tại một công ty mà tôi từng làm việc, đã có một số cuộc thảo luận quanh nhu cầu lắp đặt camera giám sát mạch kín (CCTV) ở các khu vực lễ tân của các văn phòng công ty trên khắp thế giới. Điều này là do một sự cố trong đó một người không được phép đã giành được quyền vào khu vực văn phòng phía trước rồi sau đó là mạng nội bộ của công ty. Niềm tin là một hệ thống camera CCTV không chỉ răn đe những người khác khỏi thử lại điều tương tự mà còn giúp nhận diện những cá nhân liên quan sau khi sự việc đã xảy ra.

The specter of corporate surveillance unleashed a wave of anguish in the company regarding “big brother”-type concerns. Depending on which side of the argument you were on, this was either about spying on employees (if you were “pro” cameras) or about being happy for intruders to gain access to the buildings (if you were “anti” cameras). Putting aside the problematic nature of such a polarized discussion, 4 one employee spoke up in a rather sheepish manner to suggest that perhaps the discussion was a bit misguided as we were missing some bigger issues—namely the fact that people seemed unconcerned that the front door of one of the main offices had a faulty lock, and that for years people would arrive in the morning to find the door unlocked.

Bóng ma của việc giám sát doanh nghiệp đã làm dấy lên một làn sóng day dứt trong công ty quanh những mối lo kiểu “ông anh cả” (big brother). Tùy bạn đứng về phía nào của cuộc tranh luận, đây hoặc là chuyện do thám nhân viên (nếu bạn “ủng hộ” camera) hoặc là chuyện sẵn lòng để kẻ xâm nhập giành quyền vào các tòa nhà (nếu bạn “phản đối” camera). Gác lại bản chất rắc rối của một cuộc thảo luận phân cực như vậy, 4 một nhân viên đã lên tiếng theo một cách khá rụt rè để gợi ý rằng có lẽ cuộc thảo luận có chút lệch hướng vì chúng ta đang bỏ sót một số vấn đề lớn hơn — cụ thể là việc mọi người có vẻ không bận tâm rằng cửa trước của một trong những văn phòng chính có một ổ khóa bị lỗi, và rằng suốt nhiều năm liền người ta đến vào buổi sáng để thấy cửa không khóa.

4 It was more of a passive-aggressive argument, often without the “passive” bit.

4 Đó là một cuộc tranh luận thiên về thụ động-gây hấn (passive-aggressive) hơn, thường là không có phần “thụ động”.

This extreme (but true) story is a great example of a common problem we face when trying to secure our systems. Without having time to take in all the factors and understand just where your biggest risks are, you may well end up missing the places where your time is better spent. The goal of threat modeling is about helping you understand what an attacker might want from your system. What are they after? Will different types of malicious actors want to gain access to different assets? Threat modeling, when done right, is largely about putting yourself in the mind of the attacker, thinking from the outside in. This outsider view is important, and it's one reason why having an external party help drive a threat modeling exercise can be very useful.

Câu chuyện cực đoan (nhưng có thật) này là một ví dụ tuyệt vời về một vấn đề phổ biến mà chúng ta đối mặt khi cố gắng bảo vệ hệ thống của mình. Nếu không có thời gian để tiếp nhận tất cả các yếu tố và hiểu chính xác đâu là những rủi ro lớn nhất của bạn, rất có thể bạn sẽ bỏ lỡ những nơi mà thời gian của bạn được sử dụng tốt hơn. Mục tiêu của mô hình hóa mối đe dọa là giúp bạn hiểu kẻ tấn công có thể muốn gì từ hệ thống của bạn. Họ đang truy lùng điều gì? Liệu các loại tác nhân độc hại khác nhau có muốn giành quyền truy cập vào các tài sản khác nhau không? Mô hình hóa mối đe dọa, khi được làm đúng, phần lớn là việc đặt mình vào tâm trí của kẻ tấn công, suy nghĩ từ ngoài vào trong. Cái nhìn từ bên ngoài này rất quan trọng, và đó là một lý do vì sao việc có một bên ngoài giúp dẫn dắt một buổi mô hình hóa mối đe dọa có thể rất hữu ích.

The core idea of threat modeling doesn't change much when we look at microservice architectures, aside from the fact that any architecture being analyzed may now be more complex. What shifts is how we take the outcome of a threat model and put it into action. One of the outputs of a threat modeling exercise would be a list of recommendations for what security controls need to be put in place—these controls can include things like a change in process, a shift in technology, or perhaps a modification in **system architecture**. Some of these changes may be cross-cutting and could impact multiple teams and their associated microservices. Others might result in more targeted work. Fundamentally, though, when undertaking threat modeling, you do need to look holistically—focusing this analysis on too small a subset of your system, such as one or two microservices, may result in a false sense of security. You may end up focusing your time on building a fantastically secure front door, only to have left the window open.

Ý tưởng cốt lõi của mô hình hóa mối đe dọa không thay đổi nhiều khi chúng ta xét đến kiến trúc microservice, ngoại trừ việc bất kỳ kiến trúc nào đang được phân tích giờ đây có thể phức tạp hơn. Điều thay đổi là cách chúng ta lấy kết quả của một mô hình mối đe dọa và đưa nó vào hành động. Một trong những đầu ra của một buổi mô hình hóa mối đe dọa sẽ là một danh sách các khuyến nghị về những kiểm soát bảo mật nào cần được đặt ra — những kiểm soát này có thể bao gồm những thứ như một thay đổi trong quy trình, một sự chuyển dịch về công nghệ, hoặc có lẽ một sự thay đổi trong kiến trúc hệ thống. Một số thay đổi này có thể mang tính xuyên suốt và có thể tác động đến nhiều nhóm cùng các microservice liên quan của họ. Số khác có thể dẫn đến công việc nhắm mục tiêu hơn. Tuy nhiên, về cơ bản, khi tiến hành mô hình hóa mối đe dọa, bạn quả thực cần nhìn một cách toàn diện — việc tập trung phân tích này vào một tập con quá nhỏ của hệ thống, chẳng hạn một hai microservice, có thể dẫn đến một cảm giác an toàn giả tạo. Bạn có thể rất cuộc dành thời gian xây một cửa trước an toàn tuyệt vời, để rồi lại để cửa sổ mở.

For a deeper dive into this topic, I can recommend Threat Modeling: Designing for Security 5 by Adam Shostack.

Để tìm hiểu sâu hơn về chủ đề này, tôi có thể giới thiệu cuốn *Threat Modeling: Designing for Security* 5 của Adam Shostack.

Protect — Bảo vệ

Once we've identified our most valuable—and most vulnerable—assets, we need to ensure that they are properly protected. As I've noted, microservice architectures arguably give us a much wider surface area of attack, and we therefore have more things that may need to be protected, but they also give us more options to defend in depth. We'll spend most of this chapter focusing on various aspects of protection, primarily because this is the area where microservice architectures create the most challenges.

Một khi đã nhận diện được các tài sản giá trị nhất — và dễ bị tổn thương nhất — của mình, chúng ta cần đảm bảo rằng chúng được bảo vệ đúng đắn. Như tôi đã lưu ý, kiến trúc microservice có thể nói là cho chúng ta một diện tích bề mặt tấn công rộng hơn nhiều, và do đó chúng ta có nhiều thứ hơn có thể cần được bảo vệ, nhưng chúng cũng cho chúng ta nhiều lựa chọn hơn để phòng thủ nhiều lớp. Chúng ta sẽ dành phần lớn chương này tập trung vào nhiều khía cạnh khác nhau của việc bảo vệ, chủ yếu vì đây là lĩnh vực mà kiến trúc microservice tạo ra nhiều thách thức nhất.

5 Adam Shostack, *Threat Modeling: Designing for Security* (Indianapolis: Wiley, 2014).

5 Adam Shostack, *Threat Modeling: Designing for Security* (Indianapolis: Wiley, 2014).

Detect — Phát hiện

With a microservice architecture, detecting an incident can be more complex. We have more networks to monitor and more machines to keep an eye on. The sources of information are greatly increased, which can make detecting problems all the more difficult. Many of the techniques we explored in Chapter 10, such as **log aggregation**, can help us gather information that will help us detect that something bad might be happening. In addition to those, there are special tools like intrusion detection systems that you might run to spot bad behavior. The software to deal with the increasing complexity of our systems is improving, especially in the space of **container** workloads with tools like Aqua.

Với một kiến trúc microservice, việc phát hiện một sự cố có thể phức tạp hơn. Chúng ta có nhiều mạng hơn để giám sát và nhiều máy hơn để để mắt tới. Các nguồn thông tin tăng lên rất nhiều, điều này có thể khiến việc phát hiện vấn đề càng khó khăn hơn. Nhiều kỹ thuật chúng ta đã khám phá ở Chương 10, chẳng hạn gom log (log aggregation), có thể giúp chúng ta thu thập thông tin sẽ giúp phát hiện rằng điều gì đó tồi tệ có thể đang xảy ra. Ngoài ra, còn có những công cụ đặc biệt như các hệ thống phát hiện xâm nhập mà bạn có thể chạy để phát hiện hành vi xấu. Phần mềm để xử lý độ phức tạp ngày càng tăng của hệ thống đang được cải thiện, đặc biệt trong lĩnh vực workload container với các công cụ như Aqua.

Respond — Ứng phó

If the worst has happened and you've found out about it, what should you do? Working out an effective incident response approach is vital to limiting the damage caused by a breach. This normally starts with understanding the scope of the breach and what data has been exposed. If the data

exposed includes personally identifiable information (PII), then you need to follow both security and privacy incident response and notification processes. This may well mean that you have to talk to different parts of your organization, and in some situations you may be legally obliged to inform an appointed data protection officer when certain types of breach occur.

Nếu điều tồi tệ nhất đã xảy ra và bạn đã phát hiện ra nó, bạn nên làm gì? Việc tính ra một cách tiếp cận ứng phó sự cố hiệu quả là điều sống còn để hạn chế thiệt hại do một vụ xâm phạm gây ra. Điều này thường bắt đầu bằng việc hiểu phạm vi của vụ xâm phạm và dữ liệu nào đã bị phơi lộ. Nếu dữ liệu bị phơi lộ bao gồm thông tin định danh cá nhân (PII), thì bạn cần tuân theo cả các quy trình ứng phó và thông báo sự cố về bảo mật lẫn về quyền riêng tư. Điều này hoàn toàn có thể có nghĩa là bạn phải trao đổi với các bộ phận khác nhau trong tổ chức của mình, và trong một số tình huống bạn có thể bị ràng buộc pháp lý phải thông báo cho một cán bộ bảo vệ dữ liệu được chỉ định khi xảy ra một số loại xâm phạm nhất định.

Many an organization has compounded the impact of a breach through its mishandling of the aftermath, often leading to increased financial penalties quite aside from the damage caused to its brand and its relationship with customers. It's important, therefore, to understand not just what you have to do because of legal or compliance reasons but also what you should do in terms of looking after the users of your software. The GDPR, for example, requires that personal data breaches are reported to the relevant authorities within 72 hours—a timeline that doesn't seem overly onerous. That doesn't mean you couldn't strive to let people know earlier if a breach of their data has occurred.

Nhiều tổ chức đã làm trầm trọng thêm tác động của một vụ xâm phạm thông qua việc xử lý sai hậu quả của nó, thường dẫn đến các khoản phạt tài chính tăng lên hoàn toàn tách biệt với thiệt hại gây ra cho thương hiệu và quan hệ của họ với khách hàng. Do đó, điều quan trọng là phải hiểu không chỉ những gì bạn phải làm vì lý do pháp lý hay tuân thủ, mà còn những gì bạn nên làm xét về mặt chăm sóc người dùng phần mềm của mình. Ví dụ, GDPR yêu cầu các vụ vi phạm dữ liệu cá nhân phải được báo cáo cho các cơ quan liên quan trong vòng 72 giờ — một mốc thời gian dường như không quá nặng nề. Điều đó không có nghĩa là bạn không thể nỗ lực để cho mọi người biết sớm hơn nếu một vụ xâm phạm dữ liệu của họ đã xảy ra.

Aside from the external communication aspects of response, how you handle things internally is also critical. Organizations that have a culture of blame and fear are likely going to fare badly in the wake of a major incident. Lessons won't be learned, and contributing factors won't come to light. On the other hand, an organization that focuses on openness and safety will be best placed to learn the lessons that ensure that similar incidents are less likely to happen. We'll come back to this in “Blame” on page 415 .

Bên cạnh các khía cạnh truyền thông ra bên ngoài của việc ứng phó, cách bạn xử lý mọi việc trong nội bộ cũng then chốt. Các tổ chức có văn hóa đổ lỗi và sợ hãi nhiều khả năng sẽ xoay sở rất tệ sau một sự cố lớn. Các bài học sẽ không được rút ra, và các yếu tố góp phần sẽ không lộ diện. Mặt khác, một tổ chức tập trung vào sự cởi mở và an toàn sẽ ở vị thế tốt nhất để rút ra những bài học đảm bảo rằng các sự cố tương tự ít có khả năng xảy ra hơn. Chúng ta sẽ quay lại điều này trong “Blame” ở trang 415.

Recover — Phục hồi

Recovery refers to our ability to get the system up and running again in the wake of an attack, and also our ability to implement what we have learned to ensure problems are less likely to happen again.

With a microservice architecture, we have many more moving parts, which can make recovery more complex if an issue has a broad impact. So later in this chapter we will look at how simple things like automation and backups can help you rebuild a microservice system on demand and get your system back up and running as quickly as possible.

Phục hồi nói đến khả năng của chúng ta đưa hệ thống hoạt động trở lại sau một cuộc tấn công, và cũng là khả năng triển khai những gì chúng ta đã học được để đảm bảo các vấn đề ít có khả năng xảy ra lại hơn. Với một kiến trúc microservice, chúng ta có nhiều thành phần chuyển động hơn rất nhiều, điều này có thể khiến việc phục hồi phức tạp hơn nếu một vấn đề có tác động rộng. Vì vậy sau trong chương này chúng ta sẽ xem xét cách những thứ đơn giản như tự động hóa và sao lưu có thể giúp bạn dựng lại một hệ thống microservice theo yêu cầu và đưa hệ thống của bạn hoạt động trở lại nhanh nhất có thể.

Foundations of Application Security — Nền tảng của bảo mật ứng dụng

OK, now that we have some core principles in place and some sense of the wide world that security activities can cover, let's look at a few foundational security topics in the context of a microservice architecture if you want to build a more secure system—credentials, patching, backups, and rebuild.

OK, giờ khi chúng ta đã có một số nguyên tắc cốt lõi tại chỗ và một cảm nhận nào đó về thế giới rộng lớn mà các hoạt động bảo mật có thể bao trùm, hãy cùng xem xét một vài chủ đề bảo mật nền tảng trong bối cảnh kiến trúc microservice nếu bạn muốn xây dựng một hệ thống an toàn hơn — credential, vá lỗi, sao lưu, và dựng lại.

Credentials — Credential

Broadly speaking, credentials give a person (or computer) access to some form of restricted resource. This could be a database, a computer, a user account, or something else. With a microservice architecture, in terms of how it compares to an equivalent monolithic architecture, we likely have the same number of humans involved, but we have lots more credentials in the mix representing different microservices, (virtual) machines, databases, and the like. This can lead to a degree of confusion around how to restrict (or not restrict) access, and in many cases it can lead to a “lazy” approach in which a small number of credentials with broad privileges are used in an attempt to simplify things. This can in turn lead to more problems if credentials are compromised.

Nói rộng ra, credential trao cho một người (hoặc máy tính) quyền truy cập vào một dạng tài nguyên bị hạn chế nào đó. Đây có thể là một cơ sở dữ liệu, một máy tính, một tài khoản người dùng, hay một thứ gì khác. Với một kiến trúc microservice, xét về việc nó so sánh thế nào với một kiến trúc monolith tương đương, chúng ta nhiều khả năng có cùng số lượng con người tham gia, nhưng chúng ta có nhiều credential hơn trong cuộc chơi, đại diện cho các microservice, máy (ảo), cơ sở dữ liệu, và những thứ tương tự khác nhau. Điều này có thể dẫn đến một mức độ bối rối nào đó quanh việc làm sao để hạn chế (hay không hạn chế) quyền truy cập, và trong nhiều trường hợp nó có thể dẫn đến một cách tiếp cận “lười biếng” trong đó một số lượng nhỏ credential với đặc quyền rộng được dùng nhằm cố đơn giản hóa mọi thứ. Điều này đến lượt nó có thể dẫn đến nhiều vấn đề hơn nếu credential bị xâm phạm.

We can break the topic of credentials down into two key areas. Firstly, we have the credentials of the users (and operators) of our system. These are often the weakest point of our system and are

commonly used as an attack vector by malicious parties, as we'll see in a moment. Secondly, we can consider secrets—pieces of information that are critical to running our microservices. Across both sets of credentials, we have to consider the issues of rotation, revocation, and limiting scope.

Chúng ta có thể chia chủ đề credential thành hai lĩnh vực chính. Thứ nhất, chúng ta có credential của những người dùng (và người vận hành) của hệ thống. Đây thường là điểm yếu nhất của hệ thống và thường được dùng làm một hướng tấn công bởi các bên độc hại, như chúng ta sẽ thấy trong giây lát. Thứ hai, chúng ta có thể xét đến secret — những mẫu thông tin trọng yếu đối với việc chạy các microservice của chúng ta. Xét trên cả hai tập credential, chúng ta phải xét đến các vấn đề xoay vòng, thu hồi, và giới hạn phạm vi.

User credentials

Credential người dùng

User credentials, such as email and password combinations, remain essential to how many of us work with our software, but they also are a potential weak spot when it comes to our systems being accessed by malicious parties. Verizon's 2020 Data Breach Investigations Report found that some form of credential theft was used in 80% of cases caused by hacking. This includes situations in which credentials were stolen through mechanisms like phishing attacks or where passwords were brute-forced.

Credential người dùng, chẳng hạn các cặp email và mật khẩu, vẫn còn thiết yếu đối với cách nhiều người trong chúng ta làm việc với phần mềm của mình, nhưng chúng cũng là một điểm yếu tiềm tàng khi nói đến việc hệ thống của chúng ta bị các bên độc hại truy cập. Báo cáo Điều tra Vi phạm Dữ liệu 2020 của Verizon phát hiện rằng một dạng đánh cắp credential nào đó đã được dùng trong 80% các vụ do hack gây ra. Điều này bao gồm các tình huống trong đó credential bị đánh cắp qua các cơ chế như tấn công lừa đảo (phishing) hoặc nơi mật khẩu bị brute-force.

There is some excellent advice out there about how to properly handle things like passwords—advice that despite being simple and clear to follow is still not being adopted widely enough. Troy Hunt has an excellent overview of the latest advice from both NIST and the UK's National Cyber Security Centre. ⁶ This advice includes recommendations to use password managers and long passwords, to avoid the use of complex password rules, and—somewhat surprisingly—to avoid mandated regular password changes. Troy's full post is worth reading in detail.

Có một số lời khuyên tuyệt vời ngoài kia về cách xử lý đúng đắn những thứ như mật khẩu — lời khuyên mà dù đơn giản và rõ ràng để làm theo nhưng vẫn chưa được áp dụng đủ rộng rãi. Troy Hunt có một bản tổng quan tuyệt vời về những lời khuyên mới nhất từ cả NIST lẫn Trung tâm An ninh Mạng Quốc gia của Vương quốc Anh. ⁶ Lời khuyên này bao gồm các khuyến nghị dùng trình quản lý mật khẩu và mật khẩu dài, tránh dùng các quy tắc mật khẩu phức tạp, và — khá bất ngờ — tránh bắt buộc đổi mật khẩu định kỳ. Toàn bộ bài viết của Troy đáng đọc kỹ.

In the current era of **API**-driven systems, our credentials also extend to managing things like API keys for third-party systems, such as accounts for your public cloud provider. If a malicious party gains access to your root AWS account, for example, they could decide to destroy everything running in that account. In one extreme example, such an attack resulted in a company called Code Spaces ⁷ going out of business—all of their resources were running in a single account, backups and all. The irony of Code Spaces offering “Rock Solid, Secure and Affordable Svn Hosting, Git Hosting and Project Management” is not lost on me.

Trong kỷ nguyên hiện tại của các hệ thống dựa trên API, credential của chúng ta cũng mở rộng đến việc quản lý những thứ như API key cho các hệ thống của bên thứ ba, chẳng hạn

các tài khoản cho nhà cung cấp đám mây công cộng của bạn. Ví dụ, nếu một bên độc hại giành được quyền truy cập vào tài khoản AWS gốc của bạn, họ có thể quyết định hủy diệt mọi thứ đang chạy trong tài khoản đó. Trong một ví dụ cực đoan, một cuộc tấn công như vậy đã khiến một công ty tên là Code Spaces 7 phải ngừng kinh doanh — tất cả tài nguyên của họ đều đang chạy trong một tài khoản duy nhất, cả bản sao lưu và mọi thứ. Sự trở trêu của việc Code Spaces chào mời “Dịch vụ lưu trữ Svn, lưu trữ Git và quản lý dự án Vững như bàn thạch, An toàn và Phải chăng” không thoát khỏi mắt tôi.

Even if someone gets hold of your API keys for your cloud provider and doesn't decide to destroy everything you've built, they might decide to spin up some expensive virtual machines to run some bitcoin mining in the hope that you won't notice. This happened to one of my clients, who discovered that someone had spent over \$10K doing just this before the account was shut down. It turns out that attackers know how to automate too—there are bots out there that will just scan for credentials and try to use them to launch machines for cryptocurrency mining.

Ngay cả khi ai đó nắm được API key cho nhà cung cấp đám mây của bạn và không quyết định hủy diệt mọi thứ bạn đã xây dựng, họ có thể quyết định khởi tạo một số máy ảo đắt tiền để chạy đào bitcoin với hy vọng bạn sẽ không để ý. Điều này đã xảy ra với một khách hàng của tôi, người phát hiện rằng ai đó đã tiêu hơn 10 nghìn đô la làm đúng việc này trước khi tài khoản bị đóng. Hóa ra những kẻ tấn công cũng biết tự động hóa — có những bot ngoài kia sẽ quét tìm credential và thử dùng chúng để khởi chạy máy để đào tiền mã hóa.

6 Troy Hunt, “Passwords Evolved: Authentication Guidance for the Modern Era,” July 26, 2017, <https://oreil.ly/T7PYM>.

6 Troy Hunt, “Passwords Evolved: Authentication Guidance for the Modern Era,” July 26, 2017, <https://oreil.ly/T7PYM>.

7 Neil McAllister, “Code Spaces Goes Titsup FOREVER After Attacker NUKES Its Amazon-Hosted Data,” The Register, June 18, 2014, <https://oreil.ly/mw7PC>.

7 Neil McAllister, “Code Spaces Goes Titsup FOREVER After Attacker NUKES Its Amazon-Hosted Data,” The Register, June 18, 2014, <https://oreil.ly/mw7PC>.

Secrets

Secret

Broadly speaking, secrets are critical pieces of information that a microservice needs to operate and that are also sensitive enough that they require protecting from malicious parties. Examples of secrets that a microservice might need include:

Nói rộng ra, secret là những mẫu thông tin trọng yếu mà một microservice cần để vận hành và cũng đủ nhạy cảm đến mức chúng đòi hỏi phải được bảo vệ khỏi các bên độc hại. Các ví dụ về secret mà một microservice có thể cần bao gồm:

- Certificates for TLS
- SSH keys
- Public/private API keypairs
- Credentials for accessing databases
 - Chứng chỉ (certificate) cho TLS
 - Khóa SSH
 - Cặp khóa API công khai/riêng tư
 - Credential để truy cập cơ sở dữ liệu

If we consider the life cycle of a secret, we can start to tease apart the various aspects of secrets management that might require different security needs:

Nếu chúng ta xét đến vòng đời của một secret, chúng ta có thể bắt đầu bóc tách các khía cạnh khác nhau của việc quản lý secret vốn có thể đòi hỏi những nhu cầu bảo mật khác nhau:

Creation How do we create the secret in the first place?

Tạo Trước hết, chúng ta tạo ra secret như thế nào?

Distribution Once the secret is created, how do we make sure it gets to the right place (and only the right place)?

Phân phối Một khi secret được tạo, làm sao chúng ta đảm bảo nó đến đúng nơi (và chỉ đúng nơi)?

Storage Is the secret stored in a way that ensures only authorized parties can access it?

Lưu trữ Secret có được lưu trữ theo cách đảm bảo chỉ các bên được ủy quyền mới có thể truy cập không?

Monitoring Do we know how this secret is being used?

Giám sát Chúng ta có biết secret này đang được dùng như thế nào không?

Rotation Are we able to change the secret without causing problems?

Xoay vòng Chúng ta có thể thay đổi secret mà không gây ra vấn đề không?

If we have a number of microservices, each of which might want different sets of secrets, we're going to need to use tooling to help manage all of this.

Nếu chúng ta có một số microservice, mỗi microservice có thể muốn các tập secret khác nhau, chúng ta sẽ cần dùng công cụ để giúp quản lý tất cả những thứ này.

Kubernetes provides a built-in secrets solution. It is somewhat limited in terms of functionality but does come as part of a basic Kubernetes install, so it could be good enough for many use cases. 8

Kubernetes cung cấp một giải pháp secret tích hợp sẵn. Nó hơi hạn chế về mặt chức năng nhưng đi kèm như một phần của một bản cài Kubernetes cơ bản, nên nó có thể đủ tốt cho nhiều trường hợp sử dụng. 8

If you were looking for a more sophisticated tool in this space, Hashicorp's Vault is worth a look. An open source tool with commercial options available, it's a veritable Swiss Army knife of secrets management, handling everything from the basic aspects of distributing secrets to generating time-limited credentials for databases and cloud

Nếu bạn đang tìm một công cụ tinh vi hơn trong lĩnh vực này, Vault của Hashicorp đáng để xem. Là một công cụ mã nguồn mở với các tùy chọn thương mại có sẵn, nó đúng là một con dao Thụy Sĩ của việc quản lý secret, xử lý mọi thứ từ các khía cạnh cơ bản của việc phân phối secret đến việc sinh ra credential có giới hạn thời gian cho cơ sở dữ liệu và

8 Some people are concerned about the fact that secrets are stored in plain text. Whether or not this is an issue for you depends largely on your threat model. For the secrets to be read, an attacker would have to have direct access to the core systems running your cluster, at which point it's arguable that your cluster is already hopelessly compromised.

8 Một số người lo ngại về việc các secret được lưu ở dạng văn bản thuần (plain text). Điều này có phải là vấn đề với bạn hay không phụ thuộc phần lớn vào mô hình mối đe dọa của

bạn. Để đọc được các secret, kẻ tấn công sẽ phải có quyền truy cập trực tiếp vào các hệ thống lõi đang chạy cụm của bạn, và tới thời điểm đó thì có thể lập luận rằng cụm của bạn vốn đã bị xâm phạm vô vọng rồi.

platforms. Vault has the added benefit that the supporting consul-template tool is able to dynamically update secrets in a normal configuration file. This means parts of your system that want to read secrets from a local filesystem don't need to change to support the secrets management tool. When a secret is changed in Vault, consul-template can update this entry in the configuration file, allowing your microservices to dynamically change the secrets they are using. This is fantastic for managing credentials at scale.

các nền tảng đám mây. Vault có thêm lợi ích là công cụ hỗ trợ consul-template có khả năng cập nhật secret một cách động trong một tệp cấu hình thông thường. Điều này có nghĩa là các phần của hệ thống của bạn muốn đọc secret từ một hệ thống tệp cục bộ không cần phải thay đổi để hỗ trợ công cụ quản lý secret. Khi một secret được thay đổi trong Vault, consul-template có thể cập nhật mục này trong tệp cấu hình, cho phép các microservice của bạn thay đổi động các secret mà chúng đang dùng. Điều này thật tuyệt vời để quản lý credential ở quy mô lớn.

Some public cloud providers also offer solutions in this space; for example, AWS Secrets Manager or Azure's Key Vault come to mind. Some people dislike the idea of storing critical secret information in a public cloud **service** like this, however. Again, this comes down to your threat model. If it is a serious concern, there is nothing to stop you from running Vault on your public cloud provider of choice and handling that system yourself. Even if data at rest is stored on the cloud provider, with the appropriate storage **backend** you can ensure that the data is encrypted in such a way that even if an outside party got hold of the data they couldn't do anything with it.

Một số nhà cung cấp đám mây công cộng cũng cung cấp các giải pháp trong lĩnh vực này; ví dụ, AWS Secrets Manager hay Key Vault của Azure xuất hiện trong tâm trí. Tuy nhiên, một số người không thích ý tưởng lưu trữ thông tin secret trọng yếu trong một dịch vụ đám mây công cộng như thế này. Một lần nữa, điều này lại quy về mô hình mối đe dọa của bạn. Nếu đó là một mối lo nghiêm trọng, không có gì ngăn bạn chạy Vault trên nhà cung cấp đám mây công cộng bạn chọn và tự xử lý hệ thống đó. Ngay cả khi dữ liệu khi nghỉ được lưu trên nhà cung cấp đám mây, với backend lưu trữ phù hợp bạn có thể đảm bảo rằng dữ liệu được mã hóa theo cách mà ngay cả khi một bên bên ngoài nắm được dữ liệu họ cũng không thể làm gì với nó.

Rotation

Xoay vòng

Ideally, we want to rotate credentials frequently to limit the damage someone can do if they gain access to the credentials. If a malicious party gains access to your AWS API public/private keypair but that credential is changed once a week, they have only one week in which to make use of the credentials. They can still do a lot of damage in a week of course, but you get the idea. Some types of attackers like to gain access to systems and then stay undetected, allowing them to collect more valuable data over time and find ways into other parts of your system. If they used stolen credentials to gain access, you may be able to stop them in their tracks if the credentials they use expire before they can make much use of them.

Lý tưởng nhất, chúng ta muốn xoay vòng credential thường xuyên để hạn chế thiệt hại mà ai đó có thể gây ra nếu họ giành được quyền truy cập vào credential. Nếu một bên độc hại giành được quyền truy cập vào cặp khóa công khai/riêng tư API AWS của bạn nhưng credential đó được thay đổi mỗi tuần một lần, họ chỉ có một tuần để tận dụng credential.

Tất nhiên họ vẫn có thể gây ra rất nhiều thiệt hại trong một tuần, nhưng bạn hiểu ý rồi đấy. Một số loại kẻ tấn công thích giành quyền truy cập vào các hệ thống rồi sau đó nằm vùng không bị phát hiện, cho phép họ thu thập nhiều dữ liệu giá trị hơn theo thời gian và tìm đường vào các phần khác của hệ thống của bạn. Nếu họ dùng credential bị đánh cắp để giành quyền truy cập, bạn có thể chặn họ ngay từ đầu nếu các credential họ dùng hết hạn trước khi họ có thể tận dụng được nhiều.

A great example of rotation for operator credentials would be the generation of time-limited API keys for using AWS. Many organizations now generate API keys on the fly for their staff, with the public and private keypair being valid only for a short period of time—typically less than an hour. This allows you to generate the API keys you need to carry out whatever operation is needed, safe in the knowledge that even if a malicious party subsequently gains access to these keys, they will be unable to make use of them. Even if you did accidentally check that keypair into public GitHub, it would be no use to anyone once it's expired.

Một ví dụ tuyệt vời về việc xoay vòng cho credential người vận hành sẽ là việc sinh ra các API key có giới hạn thời gian để dùng AWS. Nhiều tổ chức giờ đây sinh API key ngay tức thì cho nhân viên của họ, với cặp khóa công khai và riêng tư chỉ hợp lệ trong một khoảng thời gian ngắn — thường ít hơn một giờ. Điều này cho phép bạn sinh các API key bạn cần để thực hiện bất kỳ thao tác nào cần thiết, yên tâm rằng ngay cả khi một bên độc hại sau đó giành được quyền truy cập vào những khóa này, họ cũng sẽ không thể tận dụng được chúng. Ngay cả khi bạn vô tình check cặp khóa đó vào GitHub công khai, nó cũng sẽ vô dụng với bất kỳ ai một khi đã hết hạn.

The use of time-limited credentials can be useful for systems too. Hashicorp's Vault can generate time-limited credentials for databases. Rather than your microservice instance reading database connection details from a configuration store or a text file, they can instead be generated on the fly for a specific instance of your microservice.

Việc sử dụng credential có giới hạn thời gian cũng có thể hữu ích cho các hệ thống. Vault của Hashicorp có thể sinh credential có giới hạn thời gian cho cơ sở dữ liệu. Thay vì để instance microservice của bạn đọc chi tiết kết nối cơ sở dữ liệu từ một kho cấu hình hay một tệp văn bản, chúng có thể thay vào đó được sinh ra ngay tức thì cho một instance cụ thể của microservice của bạn.

Moving to a process of frequent rotation of credentials like keys can be painful. I've spoken to companies that have experienced incidents as a result of key rotation, where systems stop working when keys are changed. This is often due to the fact that it can be unclear as to what is using a particular credential. If the scope of the credential is limited, the potential impact of rotation is significantly reduced. But if the credential has broad use, working out the impact of a change can be difficult. This isn't to put you off rotation but just to make you aware of the potential risks, and I remain convinced it is the right thing to do. The most sensible way forward would likely be to adopt tooling to help automate this process while also limiting the scope of each set of credentials at the same time.

Việc chuyển sang một quy trình xoay vòng thường xuyên các credential như khóa có thể là một nỗi đau. Tôi đã trò chuyện với các công ty từng gặp sự cố do hệ quả của việc xoay vòng khóa, nơi các hệ thống ngừng hoạt động khi khóa được thay đổi. Điều này thường là do thực tế rằng có thể không rõ thứ gì đang dùng một credential cụ thể. Nếu phạm vi của credential bị giới hạn, tác động tiềm tàng của việc xoay vòng giảm đi đáng kể. Nhưng nếu credential có phạm vi sử dụng rộng, việc tính ra tác động của một thay đổi có thể khó khăn. Điều này không nhằm làm bạn nản lòng với việc xoay vòng mà chỉ để khiến bạn nhận thức được những rủi ro tiềm tàng, và tôi vẫn tin chắc rằng đó là điều đúng đắn nên

làm. Cách tiến hành hợp lý nhất có lẽ là đón nhận công cụ để giúp tự động hóa quy trình này đồng thời cũng giới hạn phạm vi của mỗi tập credential cùng lúc.

Revocation

Thu hồi

Having a policy in place to ensure that key credentials are rotated on a regular basis can be a sensible way to limit the impact of credential leakage, but what happens if you know that a given credential has fallen into the wrong hands? Do you have to wait until a scheduled rotation kicks in for that credential to no longer be valid? That may not be practical—or sensible. Instead, you'd ideally like to be able to automatically revoke and potentially regenerate credentials when something like this happens.

Việc có một chính sách tại chỗ để đảm bảo rằng các credential then chốt được xoay vòng đều đặn có thể là một cách hợp lý để hạn chế tác động của việc rò rỉ credential, nhưng điều gì xảy ra nếu bạn biết rằng một credential cho trước đã rơi vào tay kẻ xấu? Bạn có phải chờ đến khi một lượt xoay vòng theo lịch khởi động để credential đó không còn hợp lệ không? Điều đó có thể không thực tế — hoặc không hợp lý. Thay vào đó, lý tưởng nhất là bạn muốn có khả năng tự động thu hồi và có khả năng sinh lại credential khi điều gì đó như thế này xảy ra.

The use of tools that allow for centralized secrets management can help here, but this may require that your microservices are able to reread newly generated values. If your microservice is directly reading secrets from something like the Kubernetes secrets store or Vault, it can be notified when these values have changed, allowing your microservice to make use of the changed values. Alternatively, if your microservice reads these secrets only on startup, then you may need to do a rolling restart of your system to reload these credentials. If you are regularly rotating credentials, though, chances are you've already had to solve the problem of your microservices being able to reread this information. If you're comfortable with regular rotation of credentials, chances are you're already set up to handle emergency revocation as well.

Việc sử dụng các công cụ cho phép quản lý secret tập trung có thể giúp ích ở đây, nhưng điều này có thể đòi hỏi rằng các microservice của bạn có khả năng đọc lại các giá trị mới sinh ra. Nếu microservice của bạn đang trực tiếp đọc secret từ một thứ như kho secret của Kubernetes hay Vault, nó có thể được thông báo khi các giá trị này đã thay đổi, cho phép microservice của bạn tận dụng các giá trị đã thay đổi. Ngược lại, nếu microservice của bạn chỉ đọc các secret này khi khởi động, thì bạn có thể cần thực hiện một lượt khởi động lại cuốn chiếu (rolling restart) hệ thống của mình để nạp lại các credential này. Tuy nhiên, nếu bạn đang xoay vòng credential đều đặn, nhiều khả năng bạn đã phải giải quyết bài toán microservice của mình có khả năng đọc lại thông tin này rồi. Nếu bạn thoải mái với việc xoay vòng credential đều đặn, nhiều khả năng bạn cũng đã được thiết lập sẵn để xử lý việc thu hồi khẩn cấp.

Scanning for Keys — Quét tìm Khóa

Accidentally checking in private keys to source code repositories is a common way for credentials to be leaked to unauthorized parties—it happens a surprising amount. GitHub automatically scans repositories for some types of secrets, but you can also run your own scanning too. It would be great if you could pick up secrets before you check in, and git-secrets lets you do just that. It can scan existing commits for potential secrets, but by setting it up as a commit hook, it can stop commits even being made. There is also the similar gitleaks, which, in addition to supporting pre-commit

hooks and general scanning of commits, has a few features that make it potentially more useful as a general tool for scanning local files.

Vô tình check khóa riêng tư vào kho mã nguồn là một cách phổ biến để credential bị rò rỉ tới các bên không được phép — nó xảy ra nhiều một cách đáng ngạc nhiên. GitHub tự động quét các kho tìm một số loại secret, nhưng bạn cũng có thể tự chạy quét của riêng mình. Sẽ thật tuyệt nếu bạn có thể phát hiện secret trước khi check in, và git-secrets cho phép bạn làm đúng điều đó. Nó có thể quét các commit hiện có để tìm secret tiềm tàng, nhưng bằng cách thiết lập nó như một commit hook, nó có thể chặn ngay cả việc commit được thực hiện. Cũng có công cụ tương tự gitleaks, mà ngoài việc hỗ trợ pre-commit hook và quét chung các commit, có một vài tính năng khiến nó có khả năng hữu ích hơn như một công cụ chung để quét các tệp cục bộ.

Limiting scope

Giới hạn phạm vi

Limiting the scope of credentials is core to the idea of embracing the principle of least privilege. This can apply across all forms of credentials, but limiting the scope of what a given set of credentials gives you access to can be incredibly useful. For example, in Figure 11-1, each instance of the Inventory microservice is given the same username and password for the supporting database. We also provide read-only access to the supporting Debezium process that will be used to read data and send it out via Kafka as part of an existing ETL process. If the username and password for the microservices are compromised, an external party could theoretically gain read and write access to the database. If they gained access to the Debezium credentials, though, they'd only have read-only access.

Giới hạn phạm vi của credential là cốt lõi của ý tưởng đón nhận nguyên tắc đặc quyền tối thiểu. Điều này có thể áp dụng xuyên suốt mọi dạng credential, nhưng việc giới hạn phạm vi những gì một tập credential cho trước trao cho bạn quyền truy cập có thể cực kỳ hữu ích. Ví dụ, trong Hình 11-1, mỗi instance của microservice Inventory được trao cùng một username và password cho cơ sở dữ liệu hỗ trợ. Chúng ta cũng cung cấp quyền truy cập chỉ-đọc cho tiến trình Debezium hỗ trợ vốn sẽ được dùng để đọc dữ liệu và gửi nó ra qua Kafka như một phần của một quy trình ETL hiện có. Nếu username và password cho các microservice bị xâm phạm, một bên bên ngoài về mặt lý thuyết có thể giành được quyền truy cập đọc và ghi vào cơ sở dữ liệu. Tuy nhiên, nếu họ giành được quyền truy cập vào credential của Debezium, họ chỉ có quyền truy cập chỉ-đọc.

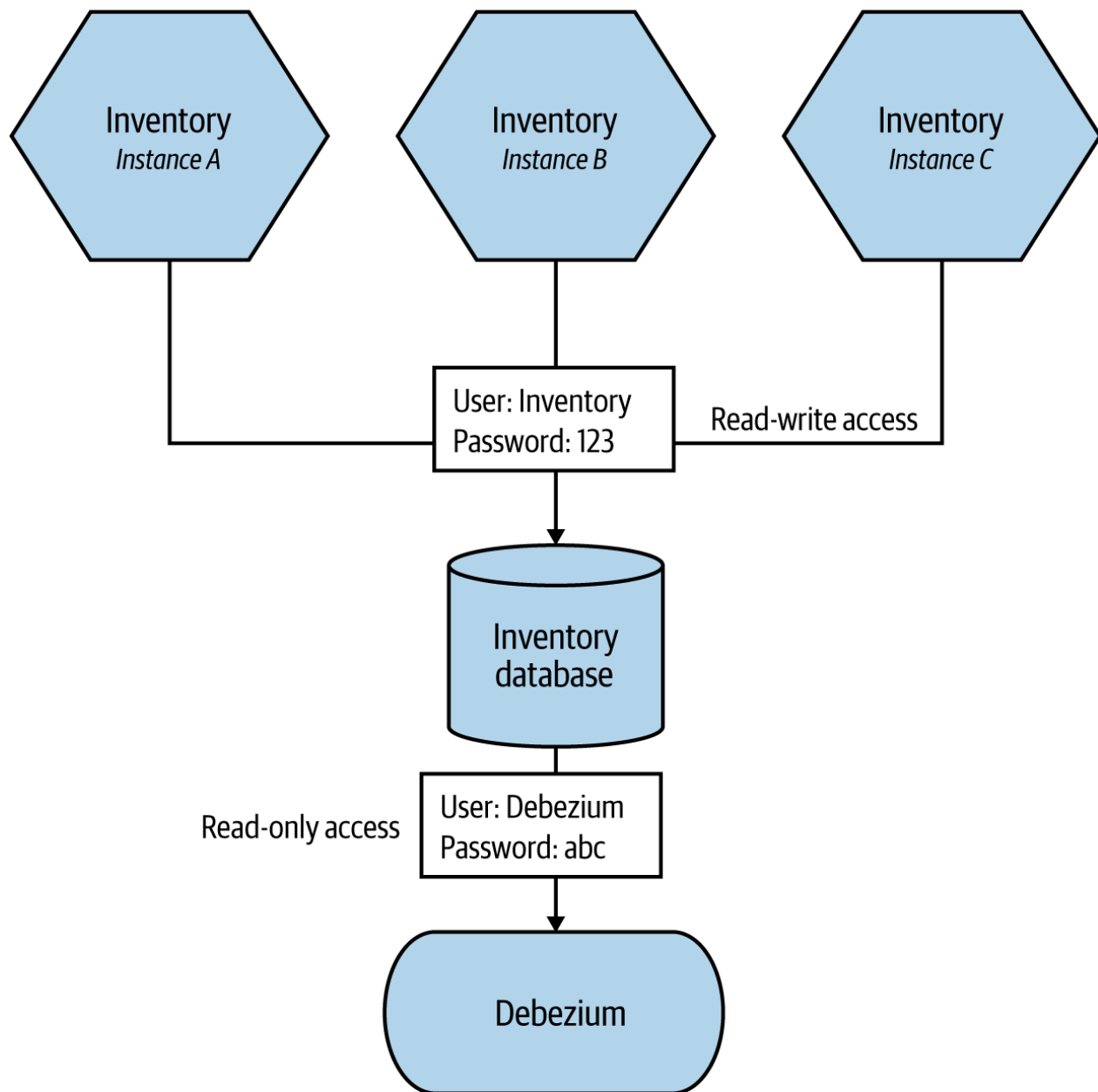


Figure 11-1. Limiting scope of credentials to limit the impact of misuse

Hình 11-1. Giới hạn phạm vi của credential để hạn chế tác động của việc lạm dụng

Limiting scope can apply in terms of both what the set of credentials can access and also who has access to that set of credentials. In Figure 11-2 , we've changed things such that each instance of Inventory gets a different set of credentials. This means that we could rotate each credential independently, or just revoke the credential for one of the instances if that is what becomes compromised. Moreover, with more specific credentials it can be easier to find out from where and how the credential was obtained. There are obviously other benefits that come from having a uniquely identifiable username for a microservice instance here; it might be easier to track down which instance caused an expensive query, for example.

Giới hạn phạm vi có thể áp dụng xét về cả những gì tập credential có thể truy cập lẫn ai có quyền truy cập vào tập credential đó. Trong Hình 11-2, chúng ta đã thay đổi mọi thứ sao

cho mỗi instance của Inventory nhận một tập credential khác nhau. Điều này có nghĩa là chúng ta có thể xoay vòng từng credential một cách độc lập, hoặc chỉ thu hồi credential cho một trong các instance nếu đó là cái bị xâm phạm. Hơn nữa, với những credential cụ thể hơn, có thể dễ dàng tìm ra credential đã bị lấy từ đâu và như thế nào. Hiển nhiên có những lợi ích khác đến từ việc có một username định danh duy nhất cho một instance microservice ở đây; chẳng hạn, có thể dễ dàng hơn để truy ra instance nào đã gây ra một truy vấn tốn kém.

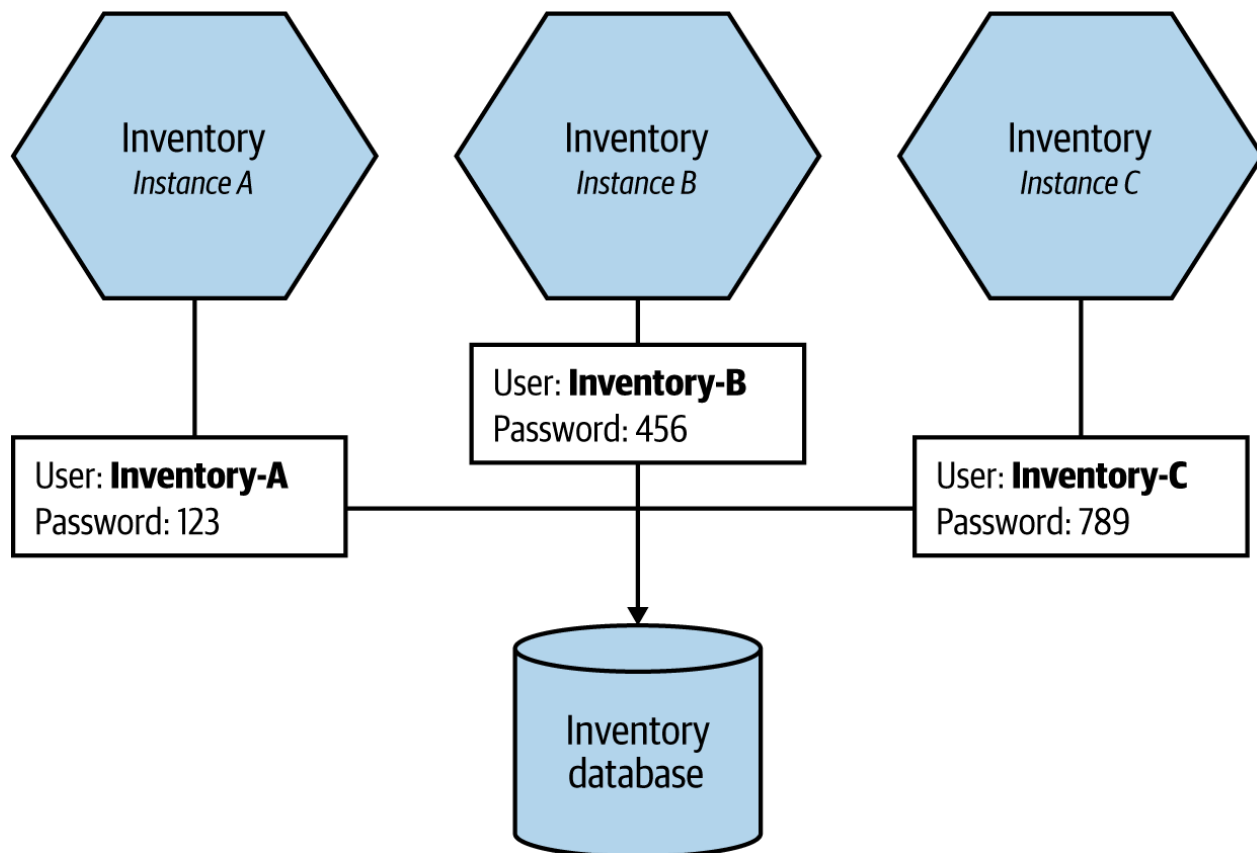


Figure 11-2. Each instance of Inventory has its own access credentials for the database, further limiting access

Hình 11-2. Mỗi instance của Inventory có credential truy cập riêng cho cơ sở dữ liệu, càng giới hạn quyền truy cập hơn

As we’ve already covered, managing fine-grained credentials at scale can be complex, and if you did want to adopt an approach like this, some form of automation would be essential—secret stores like Vault come to mind as perfect ways to implement schemes like this.

Như chúng ta đã đề cập, việc quản lý các credential chi tiết ở quy mô lớn có thể phức tạp, và nếu bạn quả thực muốn đón nhận một cách tiếp cận như thế này, một dạng tự động hóa nào đó sẽ là thiết yếu — các kho secret như Vault xuất hiện trong tâm trí như những cách hoàn hảo để triển khai các sơ đồ kiểu này.

Patching — Vá lỗi

The 2017 Equifax data breach is a great example of the importance of patching. A known vulnerability in Apache Struts was used to gain unauthorized access to data held by Equifax. Because Equifax is a credit bureau, this information was especially sensitive. In the end it was found that the data of more than 160 million people was compromised in the breach. Equifax ended up having to pay a \$700 million settlement.

Vụ vi phạm dữ liệu Equifax năm 2017 là một ví dụ tuyệt vời về tầm quan trọng của việc vá lỗi. Một lỗ hổng đã biết trong Apache Struts đã được dùng để giành quyền truy cập trái phép vào dữ liệu do Equifax nắm giữ. Vì Equifax là một cơ quan tín dụng, thông tin này đặc biệt nhạy cảm. Rất cuộc người ta phát hiện rằng dữ liệu của hơn 160 triệu người đã bị xâm phạm trong vụ vi phạm này. Equifax rất cuộc đã phải trả một khoản dân xếp 700 triệu đô la.

Months before the breach, the vulnerability in Apache Struts had been identified, and a new **release** had been made by the maintainers fixing the issue. Unfortunately, Equifax hadn't updated to the new version of the software, despite it being available for months before the attack. Had Equifax updated this software in a timely manner, it seems probable that the attack wouldn't have been possible.

Nhiều tháng trước vụ vi phạm, lỗ hổng trong Apache Struts đã được nhận diện, và một bản phát hành mới đã được những người bảo trì tạo ra để sửa vấn đề. Đáng tiếc, Equifax đã không cập nhật lên phiên bản mới của phần mềm, mặc dù nó đã có sẵn nhiều tháng trước cuộc tấn công. Giả như Equifax cập nhật phần mềm này một cách kịp thời, có vẻ nhiều khả năng cuộc tấn công đã không thể xảy ra.

The issue of keeping on top of patching is becoming more complex as we **deploy** increasingly complex systems. We need to become more sophisticated in how we handle this fairly basic concept.

Vấn đề bám sát việc vá lỗi đang trở nên phức tạp hơn khi chúng ta triển khai các hệ thống ngày càng phức tạp. Chúng ta cần trở nên tinh vi hơn trong cách xử lý khái niệm khá cơ bản này.

Figure 11-3 shows an example of the layers of infrastructure and software that exist underneath a typical Kubernetes cluster. If you run all of that infrastructure yourself, you're in charge of managing and patching all those layers. How confident are you that you are up to date with your patching? Obviously, if you can offload some of this work to a public cloud provider, you can offload part of this burden too.

Hình 11-3 cho thấy một ví dụ về các lớp hạ tầng và phần mềm tồn tại bên dưới một cụm Kubernetes điển hình. Nếu bạn tự chạy toàn bộ hạ tầng đó, bạn chịu trách nhiệm quản lý và vá lỗi tất cả các lớp đó. Bạn tự tin đến mức nào rằng bạn đã cập nhật việc vá lỗi của mình? Hiển nhiên, nếu bạn có thể chuyển một phần công việc này sang một nhà cung cấp đám mây công cộng, bạn cũng có thể chuyển bớt một phần gánh nặng này.

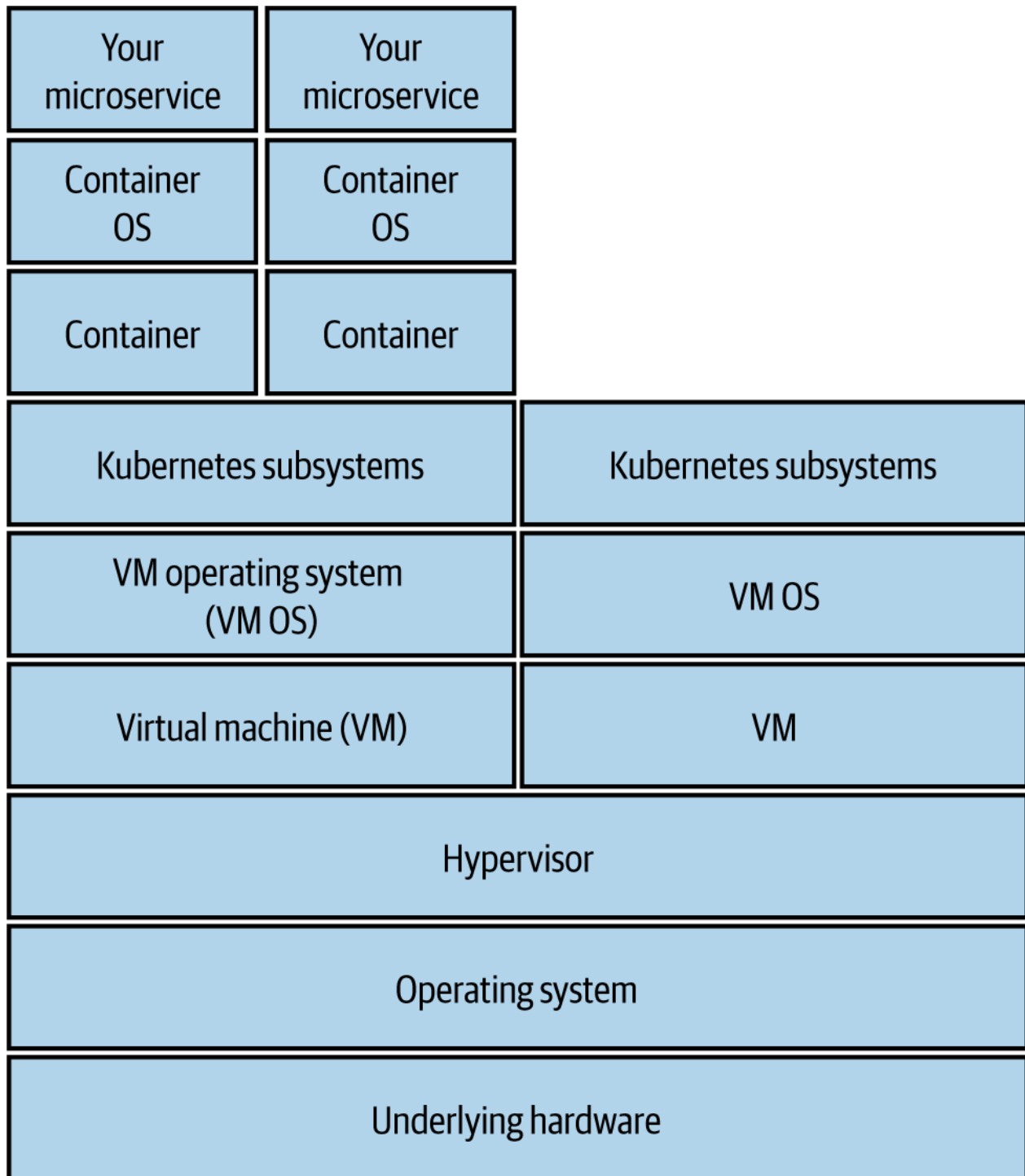


Figure 11-3. The different layers in a modern infrastructure that all require maintenance and patching. If you were to use a managed Kubernetes cluster on one of the main public cloud vendors, for example, you'd drastically reduce your scope of ownership, as we see in Figure 11-4.

Hình 11-3. Các lớp khác nhau trong một hạ tầng hiện đại đều cần được bảo trì và vá lỗi. Nếu bạn dùng một cụm Kubernetes được quản lý trên một trong những nhà cung cấp đám mây công cộng lớn, chẳng hạn, bạn sẽ giảm đáng kể phạm vi sở hữu của mình, như chúng ta thấy trong Hình 11-4.

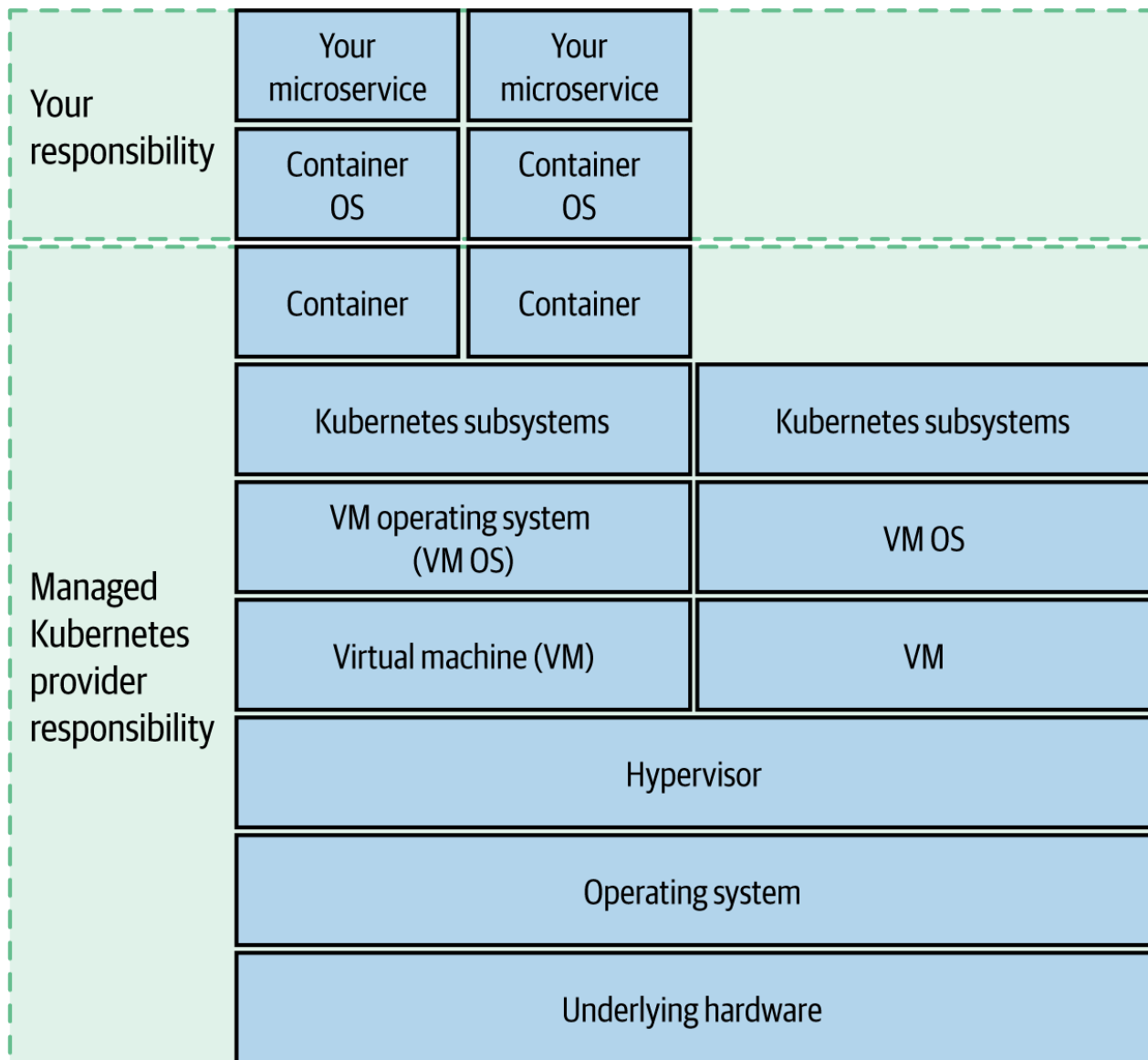


Figure 11-4. Offloading responsibility for some layers of this stack can reduce complexity

Hình 11-4. Việc chuyển trách nhiệm cho một số lớp của ngăn xếp này có thể giảm độ phức tạp

Containers throw us an interesting curveball here. We treat a given container instance as immutable. But a container contains not just our software but also an operating system. And do you know where that container has come from? Containers are based on an image, which in turn can extend other images—are you sure the base images you are using don't already have backdoors in them? If you haven't changed a container instance in six months, that's six months worth of operating system patches that haven't been applied. Keeping on top of this is problematic, which is why companies like Aqua provide tooling to help you analyze your running production containers so you can understand what issues need to be addressed.

Container ném cho chúng ta một quả bóng xoáy thú vị ở đây. Chúng ta coi một instance container cho trước là bất biến (immutable). Nhưng một container không chỉ chứa phần mềm của chúng ta mà còn chứa một hệ điều hành. Và bạn có biết container đó đến từ đâu

không? Container dựa trên một image, mà image này đến lượt nó lại có thể mở rộng các image khác — bạn có chắc các base image bạn đang dùng không sẵn có backdoor trong đó không? Nếu bạn không thay đổi một instance container trong sáu tháng, đó là sáu tháng bản vá hệ điều hành chưa được áp dụng. Việc bám sát điều này là một vấn đề, đó là lý do vì sao các công ty như Aqua cung cấp công cụ để giúp bạn phân tích các container production đang chạy của mình để bạn có thể hiểu những vấn đề nào cần được giải quyết.

At the very top of this set of layers, of course, is our application code. Is that up to date? It's not just the code we write; what about the third-party code we use? A bug in a third-party library can leave our application vulnerable to attack. In the case of the Equifax breach, the unpatched vulnerability was actually in Struts—a Java web framework.

Ở trên cùng của tập các lớp này, tất nhiên, là mã ứng dụng của chúng ta. Cái đó có được cập nhật không? Không chỉ là mã chúng ta viết; còn mã của bên thứ ba mà chúng ta dùng thì sao? Một lỗi trong một thư viện của bên thứ ba có thể khiến ứng dụng của chúng ta dễ bị tấn công. Trong trường hợp vụ vi phạm Equifax, lỗ hổng chưa được vá thực ra nằm trong Struts — một web framework của Java.

At scale, working out which microservices are linking to libraries with known vulnerabilities can be incredibly difficult. This is an area in which I strongly recommend the use of tools like Snyk or GitHub code scanning, which is able to automatically scan your third-party dependencies and alert you if you are linking to libraries with known vulnerabilities. If it finds any, it can send you a pull request to help update to the latest patched versions. You can even build this into your CI process and have a microservice build fail if it links to libraries with issues.

Ở quy mô lớn, việc tính ra microservice nào đang liên kết tới các thư viện có lỗ hổng đã biết có thể cực kỳ khó khăn. Đây là một lĩnh vực mà tôi mạnh mẽ khuyến nghị dùng các công cụ như Snyk hay GitHub code scanning, vốn có khả năng tự động quét các phụ thuộc của bên thứ ba của bạn và cảnh báo bạn nếu bạn đang liên kết tới các thư viện có lỗ hổng đã biết. Nếu nó tìm thấy bất kỳ cái nào, nó có thể gửi cho bạn một pull request để giúp cập nhật lên các phiên bản đã vá mới nhất. Bạn thậm chí có thể đưa điều này vào quy trình CI của mình và làm cho một bản build microservice thất bại nếu nó liên kết tới các thư viện có vấn đề.

Backups — Sao lưu

So I sometimes think that taking backups is like flossing, in that lots more people say they do it than actually do. I don't feel the need to restate the argument for backups too much here, other than to say: you should take backups, because data is valuable, and you don't want to lose it.

Tôi đôi khi nghĩ rằng việc sao lưu cũng giống như việc dùng chỉ nha khoa, ở chỗ có nhiều người nói rằng họ làm hơn so với số người thực sự làm. Tôi không cảm thấy cần phải nhắc lại quá nhiều lập luận cho việc sao lưu ở đây, ngoài việc nói rằng: bạn nên sao lưu, vì dữ liệu là giá trị, và bạn không muốn mất nó.

Data is more valuable than ever, and yet I sometimes wonder if improvements in technology have caused us to deprioritize backups. Disks are more reliable than they used to be. Databases are more likely to have built-in replication to avoid data loss. With such systems, we may convince ourselves that we don't need backups. But what if a catastrophic error occurs and your entire Cassandra cluster is wiped out? Or what if a coding bug means your application actually deletes valuable data? Backups are as important as ever. So please, back up your critical data.

Dữ liệu giá trị hơn bao giờ hết, ấy thế mà tôi đôi khi tự hỏi liệu những cải tiến về công nghệ

có khiến chúng ta hạ thấp ưu tiên cho việc sao lưu không. Đã đáng tin cậy hơn trước kia. Cơ sở dữ liệu nhiều khả năng có sao chép (replication) tích hợp sẵn để tránh mất dữ liệu. Với những hệ thống như vậy, chúng ta có thể tự thuyết phục mình rằng mình không cần sao lưu. Nhưng nếu một lỗi thảm khốc xảy ra và toàn bộ cụm Cassandra của bạn bị xóa sạch thì sao? Hoặc nếu một lỗi lập trình khiến ứng dụng của bạn thực sự xóa dữ liệu giá trị thì sao? Sao lưu quan trọng như mọi khi. Vậy nên làm ơn, hãy sao lưu dữ liệu trọng yếu của bạn.

With the **deployment** of our microservices being automated, we don't need to take full machine backups, as we can rebuild our infrastructure from source code. So we're not trying to copy the state of entire machines; we instead target our backups to the state that is most valuable. This means that our focus for backups is limited to things like the data in our databases, or perhaps our application logs. With the right filesystem technology, it's possible to take near-instantaneous block-level clones of a database's data without noticeably interrupting service.

Với việc triển khai các microservice của chúng ta được tự động hóa, chúng ta không cần phải sao lưu toàn bộ máy, vì chúng ta có thể dựng lại hạ tầng của mình từ mã nguồn. Vậy nên chúng ta không cố sao chép trạng thái của toàn bộ máy; thay vào đó chúng ta nhắm các bản sao lưu của mình vào trạng thái giá trị nhất. Điều này có nghĩa là trọng tâm của chúng ta cho việc sao lưu được giới hạn ở những thứ như dữ liệu trong cơ sở dữ liệu, hoặc có lẽ các log ứng dụng của chúng ta. Với công nghệ hệ thống tệp phù hợp, có thể tạo các bản sao chép (clone) cấp khối gần như tức thời của dữ liệu của một cơ sở dữ liệu mà không làm gián đoạn dịch vụ một cách đáng kể.



Avoid the Schrödinger Backup

Tránh bản sao lưu Schrödinger

When creating backups, you want to avoid what I call the Schrödinger backup.⁹ This is a backup that may or may not actually be a backup. Until you actually try and restore it, you really don't know

if it's actually a backup use, 10 or if it's just a bunch of 1s and 0s written to disk. The best way to avoid this problem is to ensure that the backup is real by actually restoring it. Find ways to build regular restoration of backups into your software development process—for example, by using production backups to build your performance test data.

Khi tạo bản sao lưu, bạn muốn tránh cái mà tôi gọi là bản sao lưu Schrödinger. 9 Đây là một bản sao lưu mà có thể thực sự là một bản sao lưu hoặc cũng có thể không. Cho đến khi bạn thực sự thử và khôi phục nó, bạn thực sự không biết liệu nó có thực sự là một bản sao lưu dùng được không, 10 hay nó chỉ là một mớ số 1 và số 0 được ghi lên đĩa. Cách tốt nhất để tránh vấn đề này là đảm bảo rằng bản sao lưu là thật bằng cách thực sự khôi phục nó. Tìm cách để đưa việc khôi phục bản sao lưu đều đặn vào quy trình phát triển phần mềm của bạn—chẳng hạn, bằng cách dùng các bản sao lưu production để xây dựng dữ liệu kiểm thử hiệu năng của bạn.

The “old” guidance about backups is that they should be kept off-site, the idea being that an incident in your offices or data center wouldn't affect your backups if they were elsewhere. What does “off-site” mean, though, if your application is deployed in a public cloud? What matters is that your backups are stored in a way that they are as isolated as possible from your core systems, so that a compromise in the core system won't also put your backups at risk. Code Spaces, which we mentioned earlier, had backups—but they were stored on AWS in the same account that was compromised. If your application runs on AWS, you could still store your backups there too, but you should do so on a separate account on separate cloud resources—you might even want to consider putting them into a different cloud region to mitigate the risk of a region-wide issue, or you could even store them with another provider.

Hướng dẫn “cũ” về sao lưu là chúng nên được giữ ở nơi khác (off-site), với ý tưởng là một sự cố ở văn phòng hay trung tâm dữ liệu của bạn sẽ không ảnh hưởng đến các bản sao lưu nếu chúng ở chỗ khác. Tuy nhiên, “nơi khác” nghĩa là gì nếu ứng dụng của bạn được triển khai trên một đám mây công cộng? Điều quan trọng là các bản sao lưu của bạn được lưu trữ theo cách chúng được cô lập nhất có thể khỏi các hệ thống lõi của bạn, để một sự xâm phạm trong hệ thống lõi cũng sẽ không đặt các bản sao lưu của bạn vào rủi ro. Code Spaces, mà chúng ta đã nhắc đến trước đó, có các bản sao lưu—nhưng chúng được lưu trên AWS trong cùng một tài khoản đã bị xâm phạm. Nếu ứng dụng của bạn chạy trên AWS, bạn vẫn có thể lưu trữ các bản sao lưu của mình ở đó, nhưng bạn nên làm vậy trên một tài khoản riêng trên các tài nguyên đám mây riêng—bạn thậm chí có thể muốn cân nhắc đặt chúng vào một vùng (region) đám mây khác để giảm thiểu rủi ro của một vấn đề trên toàn vùng, hoặc bạn thậm chí có thể lưu trữ chúng với một nhà cung cấp khác.

So make sure you back up critical data, keep those backups in a system separate to your main production environment, and make sure the backups actually work by regularly restoring them.

Vậy nên hãy chắc chắn bản sao lưu dữ liệu trọng yếu, giữ các bản sao lưu đó trong một hệ thống tách biệt với môi trường production chính của bạn, và chắc chắn các bản sao lưu thực sự hoạt động bằng cách khôi phục chúng đều đặn.

Rebuild — Dựng lại

We can do our best to ensure that a malicious party doesn't gain access to our systems, but what happens if they do? Well, often the most important thing you can do in the initial aftermath is get the system up and running again, but in such a way that you have removed access from the unauthorized party. This isn't always straightforward, though. I recall one of our machines being hacked many years ago by a rootkit. A rootkit is a bundle of software that is designed to hide the

activities of an unauthorized party, and it's a technique commonly used by attackers who want to remain undetected, allowing them time to explore the system. In our case, we found that the rootkit had altered core system commands like `ls` (listing files) or `ps` (to show

Chúng ta có thể làm hết sức để đảm bảo rằng một bên độc hại không giành được quyền truy cập vào hệ thống của mình, nhưng điều gì xảy ra nếu họ làm được? À, thường thì điều quan trọng nhất bạn có thể làm trong giai đoạn ngay sau đó là đưa hệ thống hoạt động trở lại, nhưng theo cách mà bạn đã loại bỏ quyền truy cập của bên không được phép. Tuy nhiên, điều này không phải lúc nào cũng đơn giản. Tôi nhớ một trong các máy của chúng tôi bị hack nhiều năm trước bởi một rootkit. Một rootkit là một gói phần mềm được thiết kế để ẩn giấu hoạt động của một bên không được phép, và đó là một kỹ thuật thường được dùng bởi những kẻ tấn công muốn nằm vùng không bị phát hiện, cho phép họ thời gian để khám phá hệ thống. Trong trường hợp của chúng tôi, chúng tôi phát hiện rằng rootkit đã sửa đổi các lệnh hệ thống lõi như `ls` (liệt kê tệp) hay `ps` (để hiển thị

9 I definitely came up with this term, but I also think it highly probable that I'm not the only one.

9 Tôi chắc chắn đã nghĩ ra thuật ngữ này, nhưng tôi cũng nghĩ rất có khả năng là tôi không phải người duy nhất.

10 Just as Niels Bohr argued that Schrödinger's cat was both alive and dead until you actually opened the box to check.

10 Cũng như Niels Bohr lập luận rằng con mèo của Schrödinger vừa sống vừa chết cho đến khi bạn thực sự mở hộp ra để kiểm tra.

process listings) to hide traces of the external attacker. We spotted this only when we were able to check the hashes of the programs running on the machine against the official packages. In the end, we basically had to reinstall the entire server from scratch.

danh sách tiến trình) để che giấu dấu vết của kẻ tấn công bên ngoài. Chúng tôi chỉ phát hiện điều này khi có thể kiểm tra các giá trị hash của các chương trình đang chạy trên máy so với các gói chính thức. Cuối cùng, về cơ bản chúng tôi đã phải cài lại toàn bộ máy chủ từ đầu.

The ability to simply wipe a server from existence and totally rebuild can be incredibly effective not just in the wake of a known attack but also in terms of reducing the impact of persistent attackers. You might not be aware of the presence of a malicious party on your system, but if you are routinely rebuilding your servers and rotating credentials, you may be drastically limiting the impact of what they can do without your realizing it.

Khả năng đơn giản là xóa sổ một máy chủ khỏi sự tồn tại và dựng lại hoàn toàn có thể cực kỳ hiệu quả không chỉ sau một cuộc tấn công đã biết mà còn xét về việc giảm tác động của những kẻ tấn công nằm vùng dai dẳng. Bạn có thể không nhận thức được sự hiện diện của một bên độc hại trên hệ thống của mình, nhưng nếu bạn thường xuyên dựng lại máy chủ và xoay vòng credential, bạn có thể đang hạn chế một cách đáng kể tác động của những gì họ có thể làm mà bạn không nhận ra.

Your ability to rebuild a given microservice, or even an entire system, comes down to the quality of your automation and backups. If you can deploy and configure each microservice from scratch based on information stored in source control, you're off to a good start. Of course, you need to combine that with a rock-solid backup restoration process for the data. As with backups, the best way to make sure automated deployment and configuration of your microservices works is to do it a lot—and the easiest way to achieve that is just to use the same process for rebuilding a microservice as you do for every deployment. This is of course how most container-based deployment processes work. You

deploy a new set of containers running the new version of your microservice and shut down the old set. Making this normal operating procedure makes a rebuild almost a nonevent.

Khả năng dựng lại một microservice cho trước, hay thậm chí toàn bộ một hệ thống, quy về chất lượng của tự động hóa và sao lưu của bạn. Nếu bạn có thể triển khai và cấu hình từng microservice từ đầu dựa trên thông tin được lưu trong quản lý mã nguồn, bạn đã có một khởi đầu tốt. Tất nhiên, bạn cần kết hợp điều đó với một quy trình khôi phục bản sao lưu vững như bàn thạch cho dữ liệu. Cũng như với sao lưu, cách tốt nhất để chắc chắn rằng việc triển khai và cấu hình tự động các microservice của bạn hoạt động là làm nó nhiều lần — và cách dễ nhất để đạt được điều đó chỉ là dùng cùng một quy trình để dựng lại một microservice như quy trình bạn dùng cho mọi lần triển khai. Tất nhiên đây là cách hầu hết các quy trình triển khai dựa trên container hoạt động. Bạn triển khai một tập container mới chạy phiên bản mới của microservice và tắt tập cũ. Biến điều này thành quy trình vận hành bình thường khiến việc dựng lại gần như không còn là một sự kiện gì cả.

There is one caveat here, especially if you are deploying on a container platform like Kubernetes. You might be frequently blowing away and redeploying container instances, but what about the underlying container platform itself? Do you have the ability to rebuild that from scratch? If you're using a fully managed Kubernetes provider, spinning up a new cluster might not be too difficult, but if you've installed and manage the cluster yourself, then this may be a nontrivial amount of work.

Có một lưu ý ở đây, đặc biệt nếu bạn đang triển khai trên một nền tảng container như Kubernetes. Bạn có thể đang thường xuyên thổi bay và triển khai lại các instance container, nhưng còn nền tảng container nền tảng bên dưới thì sao? Bạn có khả năng dựng lại nó từ đầu không? Nếu bạn đang dùng một nhà cung cấp Kubernetes được quản lý hoàn toàn, việc khởi tạo một cụm mới có thể không quá khó, nhưng nếu bạn đã tự cài và tự quản lý cụm, thì đây có thể là một lượng công việc không hề tầm thường.



Being able to rebuild your microservice and recover its data in an automated fashion helps you recover in the wake of an attack and also has the advantage of making your deployments easier across the board, having positive benefits for development, test, and production operations activities.

Có khả năng dựng lại microservice của bạn và khôi phục dữ liệu của nó một cách tự động giúp bạn phục hồi sau một cuộc tấn công và cũng có lợi thế là khiến các lần triển khai của

bạn dễ dàng hơn trên mọi mặt, mang lại lợi ích tích cực cho các hoạt động vận hành phát triển, kiểm thử, và production.

Implicit Trust Versus Zero Trust — Tin cậy ngầm định và zero trust

Our microservice architecture consists of lots of communication between things. Human users interact with our system via user interfaces. These user interfaces in turn make calls to microservices, and microservices end up calling yet more microservices. When it comes to application security, we need to consider the issue of trust among all those points of contact. How do we establish an acceptable level of trust?

Kiến trúc microservice của chúng ta bao gồm rất nhiều sự giao tiếp giữa các thứ. Người dùng là con người tương tác với hệ thống của chúng ta qua các giao diện người dùng. Các giao diện người dùng này đến lượt chúng thực hiện các lời gọi tới microservice, và microservice rồi cuộc lại gọi thêm nhiều microservice nữa. Khi nói đến bảo mật ứng dụng, chúng ta cần xét đến vấn đề tin cậy giữa tất cả các điểm tiếp xúc đó. Làm sao chúng ta thiết lập một mức độ tin cậy chấp nhận được?

We'll explore this topic shortly in terms of authentication and authorization of both humans and microservices, but before that we should consider some fundamental models around trust.

Chúng ta sẽ khám phá chủ đề này trong giây lát xét về mặt xác thực và phân quyền của cả con người lẫn microservice, nhưng trước đó chúng ta nên xét đến một số mô hình nền tảng quanh sự tin cậy.

Do we trust everything running in our network? Or do we view everything with suspicion? Here, we can consider two mindsets—implicit trust and zero trust.

Chúng ta có tin cậy mọi thứ đang chạy trong mạng của mình không? Hay chúng ta nhìn mọi thứ với sự nghi ngờ? Ở đây, chúng ta có thể xét đến hai tư duy — tin cậy ngầm định (implicit trust) và zero trust.

Implicit Trust — Tin cậy ngầm định

Our first option could be to just assume that any calls to a service made from inside our perimeter are implicitly trusted.

Lựa chọn đầu tiên của chúng ta có thể là chỉ giả định rằng bất kỳ lời gọi nào tới một dịch vụ được thực hiện từ bên trong vành đai (perimeter) của chúng ta đều được tin cậy ngầm định.

Depending on the sensitivity of the data, this might be fine. Some organizations attempt to ensure security at the perimeter of their networks, and they therefore assume they don't need to do anything else when two services are talking together. However, should an attacker penetrate your network, all hell could break loose. If the attacker decides to intercept and read the data being sent, change the data without you knowing, or even in some circumstances pretend to be the thing you are talking to, you may not know much about it.

Tùy vào độ nhạy cảm của dữ liệu, điều này có thể ổn. Một số tổ chức cố gắng đảm bảo bảo mật ở vành đai mạng của họ, và do đó họ giả định rằng họ không cần làm gì khác khi hai dịch vụ trò chuyện với nhau. Tuy nhiên, nếu một kẻ tấn công thâm nhập vào mạng của

bạn, mọi sự hỗn loạn có thể bùng phát. Nếu kẻ tấn công quyết định chặn và đọc dữ liệu đang được gửi, thay đổi dữ liệu mà bạn không hề biết, hoặc thậm chí trong một số trường hợp giả vờ là thứ mà bạn đang trò chuyện cùng, bạn có thể chẳng biết được nhiều về điều đó.

This is by far the most common form of inside-perimeter trust I see in organizations. I'm not saying that is a good thing! For most of the organizations I see using this model, I worry that the implicit trust model is not a conscious decision; rather, people are unaware of the risks in the first place.

Đây là dạng tin cậy bên-trong-vành-đai phổ biến nhất mà tôi thấy ở các tổ chức, vượt xa các dạng khác. Tôi không nói rằng đó là một điều tốt! Đối với hầu hết các tổ chức mà tôi thấy dùng mô hình này, tôi lo rằng mô hình tin cậy ngầm định không phải là một quyết định có ý thức; mà đúng hơn, mọi người không nhận thức được các rủi ro ngay từ đầu.

Zero Trust — Zero trust

“Jill, we’ve traced the call—it’s coming from inside the house!” — When a Stranger Calls

“Jill này, chúng tôi đã truy ra cuộc gọi — nó đến từ bên trong ngôi nhà!” — When a Stranger Calls

When operating in a zero-trust environment, you have to assume that you are operating in an environment that has already been compromised—the computers you are talking to could have been compromised, the inbound connections could be from hostile parties, the data you are writing could be read by bad people. Paranoid? Yes! Welcome to zero trust.

Khi vận hành trong một môi trường zero-trust, bạn phải giả định rằng bạn đang vận hành trong một môi trường đã bị xâm phạm — các máy tính bạn đang trò chuyện cùng có thể đã bị xâm phạm, các kết nối đi vào có thể đến từ các bên thù địch, dữ liệu bạn đang ghi có thể bị những kẻ xấu đọc. Hoang tưởng ư? Đúng vậy! Chào mừng đến với zero trust.

Zero trust, fundamentally, is a mindset. It’s not something you can magically implement using a product or tool; it’s an idea and that idea is that if you operate under the assumption that you are operating in a hostile environment in which bad actors could already be present, then you have to carry out precautions to make sure that you can still operate safely. In effect, the concept of a “perimeter” is meaningless with zero trust (for this reason, zero trust is often also known as “perimeterless computing”).

Zero trust, về cơ bản, là một tư duy. Nó không phải là thứ bạn có thể triển khai một cách thần kỳ bằng một sản phẩm hay công cụ; nó là một ý tưởng và ý tưởng đó là nếu bạn vận hành dưới giả định rằng bạn đang vận hành trong một môi trường thù địch nơi những tác nhân xấu có thể đã hiện diện, thì bạn phải thực hiện các biện pháp phòng ngừa để chắc chắn rằng bạn vẫn có thể vận hành an toàn. Trên thực tế, khái niệm về một “vành đai” trở nên vô nghĩa với zero trust (vì lý do này, zero trust thường còn được gọi là “điện toán không vành đai” — perimeterless computing).

Since you are assuming your system has been compromised, all inbound calls from other microservices have to be properly assessed. Is this really a client I should trust?

Vì bạn đang giả định hệ thống của mình đã bị xâm phạm, tất cả các lời gọi đi vào từ các microservice khác đều phải được đánh giá đúng đắn. Đây có thực sự là một client mà tôi nên tin cậy không?

Likewise, all data should be stored safely and all encryption keys held securely, and as we have to assume someone is listening, all sensitive data in transit within our system needs to be encrypted.

Tương tự, tất cả dữ liệu nên được lưu trữ an toàn và tất cả khóa mã hóa được giữ một cách bảo mật, và vì chúng ta phải giả định rằng có ai đó đang nghe lén, tất cả dữ liệu nhạy cảm khi truyền trong hệ thống của chúng ta đều cần được mã hóa.

Interestingly, if you have properly implemented a zero-trust mindset, you can start to do things that seem pretty odd:

Điều thú vị là, nếu bạn đã triển khai đúng đắn một tư duy zero-trust, bạn có thể bắt đầu làm những việc có vẻ khá kỳ lạ:

[With zero trust] you can actually make certain counter-intuitive access decisions and for example allow connections to internal services from the internet because you treat your “internal” network as equally trustworthy as the internet (i.e., not at all). —Jan Schaumann 11

[Với zero trust] bạn thực sự có thể đưa ra một số quyết định truy cập phản trực giác và chẳng hạn cho phép các kết nối tới các dịch vụ nội bộ từ internet vì bạn coi mạng “nội bộ” của mình đáng tin cậy ngang với internet (tức là, không đáng tin chút nào). —Jan Schaumann 11

Jan’s argument here is that if you assume that nothing inside your network is to be trusted, and that trust has to be reestablished, you can be much more flexible about the environment in which your microservice lives—you aren’t expecting the wider environment to be secure. But remember, zero trust isn’t something you turn on with a switch. It’s an underlying principle for how you decide to do things. It has to drive your decision making about how you build and evolve your system—it’s going to be something that you have to constantly invest in to get the rewards.

Lập luận của Jan ở đây là nếu bạn giả định rằng không có gì bên trong mạng của bạn đáng được tin cậy, và rằng sự tin cậy phải được tái thiết lập, thì bạn có thể linh hoạt hơn nhiều về môi trường mà microservice của bạn sống trong đó — bạn không trông đợi môi trường rộng hơn là an toàn. Nhưng hãy nhớ, zero trust không phải là thứ bạn bật lên bằng một công tắc. Nó là một nguyên tắc nền tảng cho cách bạn quyết định làm mọi việc. Nó phải dẫn dắt việc ra quyết định của bạn về cách bạn xây dựng và tiến hóa hệ thống của mình — nó sẽ là thứ bạn phải liên tục đầu tư vào để gặt hái phần thưởng.

It’s a Spectrum — Đó là một dải quang phổ

I don’t mean to imply that you have a stark choice between implicit and zero trust. The extent to which you trust (or don’t) other parties in your system could change based on the sensitivity of the information being accessed. You might decide, for example, to adopt a concept of zero trust for any microservices handling PII but be more relaxed in other areas. Again, the cost of any security implementation should be justified (and driven) by your threat model. Let your understanding of your threats and their associated impact drive your decision making around whether or not zero trust is worth it to you.

Tôi không có ý ám chỉ rằng bạn có một lựa chọn cứng nhắc giữa tin cậy ngầm định và zero trust. Mức độ bạn tin cậy (hay không) các bên khác trong hệ thống của mình có thể thay đổi dựa trên độ nhạy cảm của thông tin đang được truy cập. Chẳng hạn, bạn có thể quyết định đón nhận khái niệm zero trust cho bất kỳ microservice nào xử lý PII nhưng thoải mái hơn ở các lĩnh vực khác. Một lần nữa, chi phí của bất kỳ việc triển khai bảo mật nào cũng nên được biện minh (và dẫn dắt) bởi mô hình mối đe dọa của bạn. Hãy để sự hiểu biết của bạn về các mối đe dọa và tác động liên quan của chúng dẫn dắt việc ra quyết định của bạn quanh việc liệu zero trust có đáng giá với bạn hay không.

As an example, let's look at MedicalCo, a company I worked with that managed sensitive healthcare data pertaining to individuals. All information it held was classified based on a fairly sensible and straightforward approach:

Lấy một ví dụ, hãy nhìn vào MedicalCo, một công ty mà tôi từng làm việc cùng, vốn quản lý dữ liệu y tế nhạy cảm liên quan đến các cá nhân. Tất cả thông tin mà nó nắm giữ được phân loại dựa trên một cách tiếp cận khá hợp lý và đơn giản:

Public Data that could be freely shared with any external party. This information is effectively in the public domain.

Công khai Dữ liệu có thể được chia sẻ tự do với bất kỳ bên bên ngoài nào. Thông tin này về cơ bản nằm trong phạm vi công cộng.

Private Information that should be available only to logged-in users. Access to this information could be further limited due to authorization restrictions. This might include things such as which insurance plan a customer was on.

Riêng tư Thông tin chỉ nên có sẵn cho những người dùng đã đăng nhập. Quyền truy cập vào thông tin này có thể bị giới hạn thêm do các hạn chế phân quyền. Điều này có thể bao gồm những thứ như gói bảo hiểm mà một khách hàng đang dùng.

11 Jan Schaumann (@jschauma), Twitter, November 5, 2020, 4:22 p.m., <https://oreil.ly/QaCm2> .

11 Jan Schaumann (@jschauma), Twitter, November 5, 2020, 4:22 p.m., <https://oreil.ly/QaCm2>

Secret Incredibly sensitive information about individuals that could be accessed by people other than the individual in question only in extremely specific situations. This includes information about an individual's health data.

Bí mật Thông tin cực kỳ nhạy cảm về các cá nhân mà chỉ có thể được truy cập bởi những người khác ngoài chính cá nhân đó trong những tình huống cực kỳ cụ thể. Điều này bao gồm thông tin về dữ liệu sức khỏe của một cá nhân.

Microservices were then categorized based on the most sensitive data they used and had to run in a matching environment (zone) with matching controls, as we see in Figure 11-5 . A microservice would have to run in the zone matching the most sensitive data it made use of. For example, a microservice running in the public zone could use only public data. On the other hand, a microservice that used public and private data had to run in the private zone, and a microservice that accessed secret information always had to run in the secret zone.

Các microservice sau đó được phân loại dựa trên dữ liệu nhạy cảm nhất mà chúng dùng và phải chạy trong một môi trường (vùng — zone) tương ứng với các kiểm soát tương ứng, như chúng ta thấy trong Hình 11-5. Một microservice sẽ phải chạy trong vùng tương ứng với dữ liệu nhạy cảm nhất mà nó dùng. Ví dụ, một microservice chạy trong vùng công khai chỉ có thể dùng dữ liệu công khai. Mặt khác, một microservice dùng dữ liệu công khai và riêng tư phải chạy trong vùng riêng tư, và một microservice truy cập thông tin bí mật luôn phải chạy trong vùng bí mật.

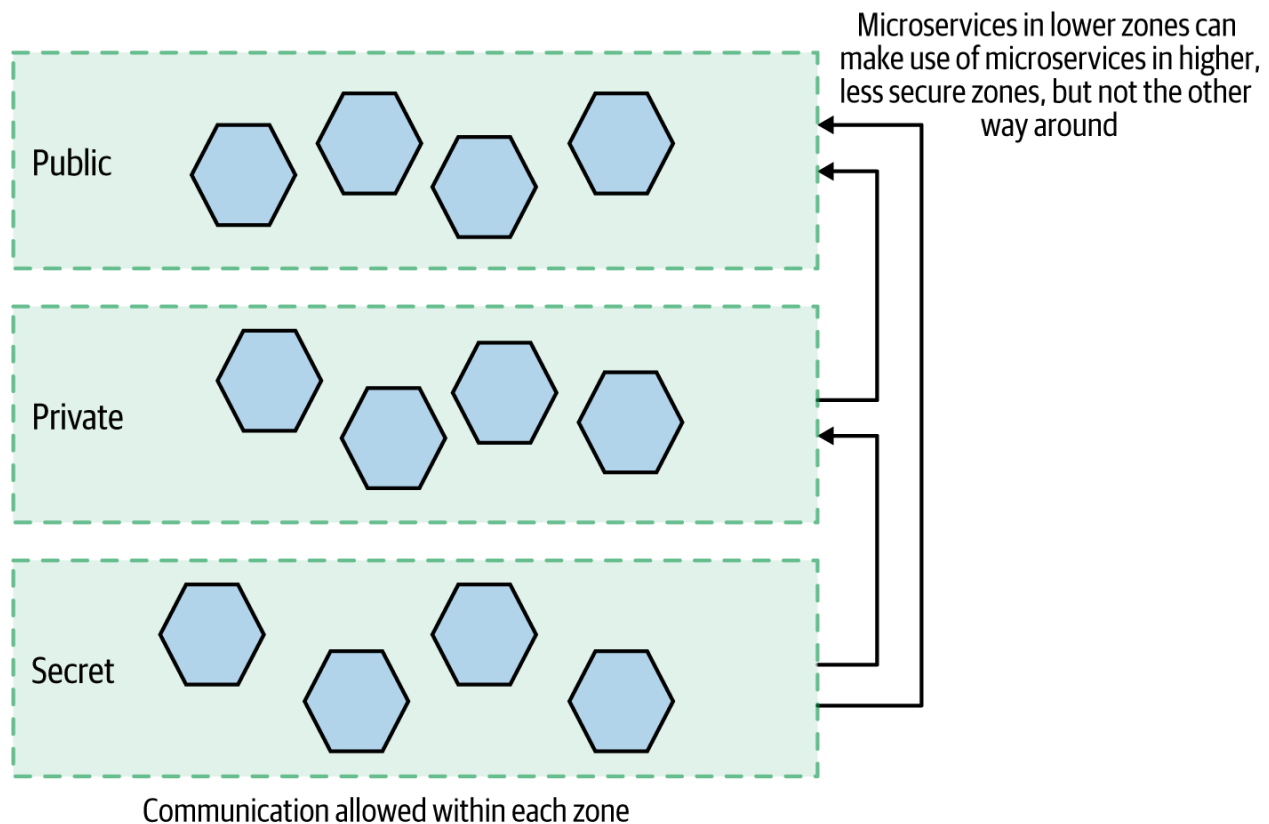


Figure 11-5. Deploying microservices into different zones based on the sensitivity of the data they handle

Hình 11-5. Triển khai microservice vào các vùng khác nhau dựa trên độ nhạy cảm của dữ liệu chúng xử lý

Microservices within each zone could communicate with each other but were unable to directly reach across to access data or functionality in the lower, more secure zones. Microservices in the more secure zones could reach up to access functionality running in the less secure zones, though.

Các microservice trong mỗi vùng có thể giao tiếp với nhau nhưng không thể trực tiếp với tới để truy cập dữ liệu hay chức năng ở các vùng thấp hơn, bảo mật hơn. Tuy nhiên, các microservice ở các vùng bảo mật hơn có thể với lên để truy cập chức năng đang chạy ở các vùng kém bảo mật hơn.

Here, MedicalCo has given itself the flexibility to vary its approach in each zone. The less secure public zone can operate in something closer to an implicit trust environment, whereas the secret zone assumes zero trust. Arguably, if MedicalCo were to adopt a zero-trust approach across its entire system, having microservices deployed into separate zones wouldn't be required, as all inter-microservice calls would require additional authentication and authorization. That said, thinking of defense in depth once again, I can't help thinking I'd still consider this zoned approach given the sensitivity of the data!

Ở đây, MedicalCo đã tự trao cho mình sự linh hoạt để biến đổi cách tiếp cận của mình ở mỗi vùng. Vùng công khai kém bảo mật hơn có thể vận hành trong thứ gì đó gần với một môi trường tin cậy ngầm định, trong khi vùng bí mật giả định zero trust. Có thể lập luận rằng, nếu MedicalCo đón nhận một cách tiếp cận zero-trust trên toàn bộ hệ thống của mình, thì việc có các microservice được triển khai vào các vùng riêng biệt sẽ không cần thiết, vì tất

cả các lời gọi giữa các microservice đều sẽ đòi hỏi xác thực và phân quyền bổ sung. Tuy nhiên, nghĩ về phòng thủ nhiều lớp một lần nữa, tôi không thể không nghĩ rằng tôi vẫn sẽ cân nhắc cách tiếp cận phân vùng này xét đến độ nhạy cảm của dữ liệu!

Securing Data — Bảo vệ dữ liệu

As we break our monolithic software apart into microservices, our data moves around our systems more than before. It doesn't just flow over networks; it also sits on disk. Having more valuable data spread around more places can be a nightmare when it comes to securing our application, if we aren't careful. Let's look in more detail at how we can protect our data as it moves over networks, and as it sits at rest.

Khi chúng ta tách phần mềm monolith của mình ra thành các microservice, dữ liệu của chúng ta di chuyển quanh các hệ thống nhiều hơn trước. Nó không chỉ chảy qua mạng; nó còn nằm trên đĩa. Việc có nhiều dữ liệu giá trị hơn rải rác ở nhiều nơi hơn có thể là một cơn ác mộng khi nói đến việc bảo vệ ứng dụng của chúng ta, nếu chúng ta không cẩn thận. Hãy cùng xem chi tiết hơn cách chúng ta có thể bảo vệ dữ liệu của mình khi nó di chuyển qua mạng, và khi nó nằm yên (at rest).

Data in Transit — Dữ liệu khi truyền

The nature of the protections you have will depend largely on the nature of the communication protocols you have picked. If you are using HTTP, for example, it would be natural to look at using HTTP with Transport Layer Security (TLS), a topic we'll expand on more in the next section; but if you're using alternative protocols such as communication via a **message broker**, you may have to look at that particular technology's support for protecting data in transit. Rather than look at the details of a large range of technology in this space, I think instead it is important to consider more generically the four main areas of interest when it comes to securing data in transit, and to look at how these concerns could be addressed with HTTP as an example. Hopefully, it shouldn't be too difficult for you to map these ideas to whatever communication protocol you are picking.

Bản chất của các lớp bảo vệ bạn có sẽ phụ thuộc phần lớn vào bản chất của các giao thức giao tiếp mà bạn đã chọn. Ví dụ, nếu bạn đang dùng HTTP, sẽ là tự nhiên khi xem xét dùng HTTP với Transport Layer Security (TLS), một chủ đề chúng ta sẽ mở rộng thêm trong phần tiếp theo; nhưng nếu bạn đang dùng các giao thức thay thế chẳng hạn giao tiếp qua một message broker, bạn có thể phải xem xét sự hỗ trợ của công nghệ cụ thể đó cho việc bảo vệ dữ liệu khi truyền. Thay vì xem xét chi tiết của một loạt lớn công nghệ trong lĩnh vực này, tôi nghĩ thay vào đó việc xét một cách tổng quát hơn bốn lĩnh vực quan tâm chính khi nói đến việc bảo vệ dữ liệu khi truyền, và xem những mối quan tâm này có thể được giải quyết như thế nào với HTTP làm ví dụ, là điều quan trọng. Hy vọng rằng sẽ không quá khó để bạn ánh xạ những ý tưởng này sang bất kỳ giao thức giao tiếp nào bạn chọn.

In Figure 11-6 we can see the four key concerns of data in transit.

Trong Hình 11-6 chúng ta có thể thấy bốn mối quan tâm then chốt của dữ liệu khi truyền.

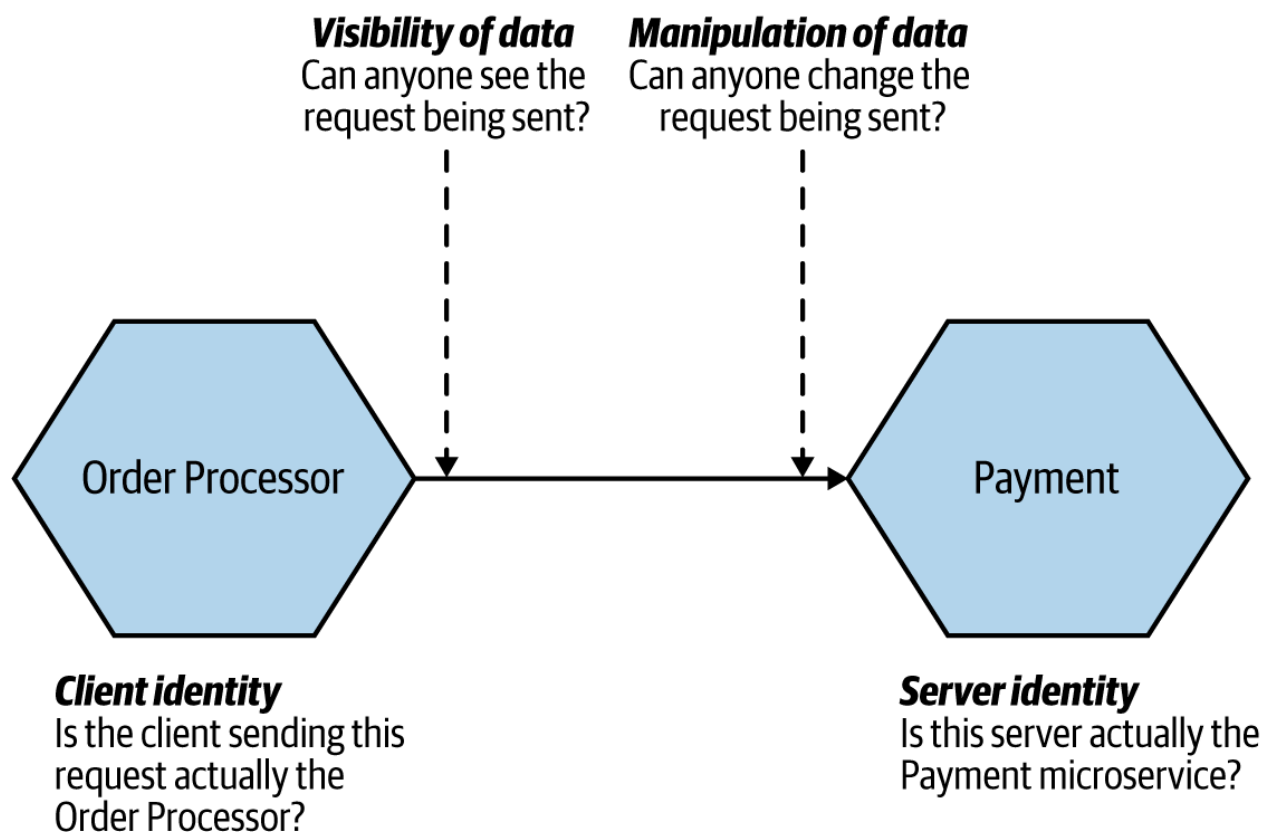


Figure 11-6. The four main concerns when it comes to data in transit

Hình 11-6. Bốn mối quan tâm chính khi nói đến dữ liệu khi truyền

Let's look at each concern in a bit more detail.

Hãy cùng xem từng mối quan tâm chi tiết hơn một chút.

Server identity

Danh tính máy chủ

One of the simplest things to check is that the server you are talking to is exactly who it claims to be. This is important because a malicious party could theoretically impersonate an endpoint—and vacuum up any useful data that you send it. Validating server identity has long been a concern on the public internet, leading to the push for more widespread use of HTTPS—and to an extent, we are able to benefit from the work done to help secure the public internet when it comes to managing internal HTTP endpoints.

Một trong những điều đơn giản nhất để kiểm tra là máy chủ bạn đang trò chuyện cùng đúng là kẻ mà nó tự nhận. Điều này quan trọng vì một bên độc hại về mặt lý thuyết có thể mạo danh một endpoint — và hút sạch mọi dữ liệu hữu ích mà bạn gửi cho nó. Việc xác thực danh tính máy chủ từ lâu đã là một mối quan tâm trên internet công cộng, dẫn đến sự thúc đẩy sử dụng HTTPS rộng rãi hơn — và ở một mức độ nào đó, chúng ta có thể hưởng lợi từ công sức đã bỏ ra để giúp bảo vệ internet công cộng khi nói đến việc quản lý các endpoint HTTP nội bộ.

When people talk about “HTTPS,” they are typically referring to using HTTP with TLS. ¹² With most communication over the public internet, due to the various potential attack vectors out there (unsecured WiFi, **DNS** poisoning, and the like), it is vital to ensure that when we go to a website

it really is the website it claims to be. With HTTPS, our browser can look at the certificate for that website and make sure it is valid. It is a highly sensible safety mechanism—“HTTPS Everywhere” has become a rallying cry for the public internet, and with good reason.

Khi người ta nói về “HTTPS”, họ thường nói về việc dùng HTTP với TLS. 12 Với hầu hết giao tiếp qua internet công cộng, do nhiều hướng tấn công tiềm tàng ngoài kia (WiFi không bảo mật, đầu độc DNS, và những thứ tương tự), việc đảm bảo rằng khi chúng ta vào một website thì nó thực sự là website mà nó tự nhận là điều sống còn. Với HTTPS, trình duyệt của chúng ta có thể nhìn vào chứng chỉ của website đó và chắc chắn rằng nó hợp lệ. Đó là một cơ chế an toàn cực kỳ hợp lý — “HTTPS Everywhere” đã trở thành một khẩu hiệu tập hợp cho internet công cộng, và vì lý do chính đáng.

12 The “S” in “HTTPS” used to relate to the older Secure Socket Layer (SSL), which has been replaced with TLS for a number of reasons. Confusingly, the term SSL still lingers even when TLS is actually being used. The OpenSSL library, for example, is actually widely used to implement TLS, and when you are issued an SSL certificate, it will actually be for TLS. We don’t make things easy for ourselves, do we?

12 Chữ “S” trong “HTTPS” trước kia liên quan đến Secure Socket Layer (SSL) cũ hơn, vốn đã được thay thế bằng TLS vì một số lý do. Một cách khó hiểu, thuật ngữ SSL vẫn còn đây ngay cả khi TLS mới thực sự được dùng. Chẳng hạn, thư viện OpenSSL thực ra được dùng rộng rãi để triển khai TLS, và khi bạn được cấp một chứng chỉ SSL, nó thực ra sẽ là cho TLS. Chúng ta đâu có làm mọi thứ dễ dàng cho chính mình, đúng không?

It’s worth noting that some communication protocols that use HTTP under the hood can take advantage of HTTPS—so we can easily run SOAP or gRPC over HTTPS without issue. HTTPS also gives us additional protections over and above simply confirming that we are talking to whom we expect. We’ll come to that shortly.

Đáng lưu ý rằng một số giao thức giao tiếp dùng HTTP bên dưới có thể tận dụng HTTPS — vậy nên chúng ta có thể dễ dàng chạy SOAP hay gRPC trên HTTPS mà không gặp vấn đề gì. HTTPS cũng cho chúng ta các lớp bảo vệ bổ sung vượt trên việc chỉ xác nhận rằng chúng ta đang trò chuyện với người mà chúng ta mong đợi. Chúng ta sẽ đến với điều đó trong giây lát.

Client identity

Danh tính client

When we refer to client identity in this context, we are referring to the microservice making a call—so we are trying to confirm and authenticate the identity of the upstream microservice. We will be looking at how we authenticate humans (users!) a bit later on.

Khi chúng ta nói đến danh tính client trong bối cảnh này, chúng ta đang nói đến microservice thực hiện một lời gọi — vậy nên chúng ta đang cố xác nhận và xác thực danh tính của microservice phía thượng nguồn (upstream). Chúng ta sẽ xem xét cách chúng ta xác thực con người (người dùng!) một lát sau.

We can verify the identity of a client in a number of ways. We could ask that the client send us some information in the request telling us who they are. An example might be to use some sort of shared secret or a client-side certificate to sign the request. When the server has to verify the client identity, we want this to be as efficient as possible—I’ve seen some solutions (including those pushed by **API gateway** vendors) that involve the server having to make calls to central services to check client identity, which is pretty nuts when you consider the latency implications.

Chúng ta có thể xác minh danh tính của một client theo một số cách. Chúng ta có thể yêu

cầu client gửi cho chúng ta một số thông tin trong yêu cầu cho biết họ là ai. Một ví dụ có thể là dùng một dạng secret chia sẻ nào đó hoặc một chứng chỉ phía client để ký yêu cầu. Khi máy chủ phải xác minh danh tính client, chúng ta muốn điều này hiệu quả nhất có thể — tôi đã thấy một số giải pháp (bao gồm những giải pháp được các nhà cung cấp API gateway thúc đẩy) đòi hỏi máy chủ phải thực hiện các lời gọi tới các dịch vụ trung tâm để kiểm tra danh tính client, điều khá phiền rồ khi bạn xét đến các hàm ý về độ trễ.

I struggle to think of a situation in which I would verify client identity without also verifying server identity—to verify both, you would typically end up implementing some form of mutual authentication . With mutual authentication, both parties authenticate each other. So in Figure 11-6 , the Order Processor authenticates the Payment microservice, and the Payment microservice authenticates the Order Processor .

Tôi khó nghĩ ra một tình huống mà tôi sẽ xác minh danh tính client mà không cũng xác minh danh tính máy chủ — để xác minh cả hai, bạn thường rất cuộc sẽ triển khai một dạng xác thực lẫn nhau (mutual authentication) nào đó. Với xác thực lẫn nhau, cả hai bên đều xác thực lẫn nhau. Vậy nên trong Hình 11-6, Order Processor xác thực microservice Payment, và microservice Payment xác thực Order Processor.

We can do this through the use of mutual TLS , in which case both the client and server make use of certificates. On the public internet, verifying the identity of a client device is typically less important than verifying the identity of the human using that device. As such, mutual TLS is rarely used. In our microservice architecture, though, especially where we might be operating in a zero-trust environment, this is much more common.

Chúng ta có thể làm điều này thông qua việc dùng mutual TLS, trong trường hợp đó cả client và máy chủ đều dùng chứng chỉ. Trên internet công cộng, việc xác minh danh tính của một thiết bị client thường ít quan trọng hơn việc xác minh danh tính của con người đang dùng thiết bị đó. Do đó, mutual TLS hiếm khi được dùng. Tuy nhiên, trong kiến trúc microservice của chúng ta, đặc biệt là nơi chúng ta có thể đang vận hành trong một môi trường zero-trust, điều này phổ biến hơn nhiều.

The challenge with implementing schemes like mutual TLS has historically been tooling. Nowadays this is less of an issue. Tools like Vault can make distributing certificates much easier, and wanting to simplify the use of mutual TLS is one of the main reasons for people to implement service meshes, which we explored in “Service Meshes and API Gateways” on page 162 .

Thách thức với việc triển khai các sơ đồ như mutual TLS trong lịch sử vốn là công cụ. Ngày nay điều này ít là vấn đề hơn. Các công cụ như Vault có thể khiến việc phân phối chứng chỉ dễ dàng hơn nhiều, và mong muốn đơn giản hóa việc dùng mutual TLS là một trong những lý do chính để mọi người triển khai service mesh, mà chúng ta đã khám phá trong “Service Meshes and API Gateways” ở trang 162.

Visibility of data

Khả năng nhìn thấy dữ liệu

When we send data from one microservice to another, can someone view the data? For some information, such as the price of Peter Andre albums, we might not care much since the data is already in the public domain. On the other hand, some data might include PII, which we need to make sure is protected.

Khi chúng ta gửi dữ liệu từ một microservice tới một microservice khác, liệu ai đó có thể xem được dữ liệu không? Đối với một số thông tin, chẳng hạn giá album của Peter Andre, chúng ta có thể không quá bận tâm vì dữ liệu đã nằm trong phạm vi công cộng. Mặt khác,

một số dữ liệu có thể bao gồm PII, mà chúng ta cần đảm bảo được bảo vệ.

When you use either plain old HTTPS or mutual TLS, data won't be visible to intermediate parties—this is because TLS encrypts the data being sent. This can be problematic if you explicitly want the data sent in the open—for example, reverse proxies like Squid or Varnish are able to **cache** HTTP responses, but that isn't possible with HTTPS.

Khi bạn dùng hoặc HTTPS cũ kỹ thông thường hoặc mutual TLS, dữ liệu sẽ không nhìn thấy được đối với các bên trung gian — điều này là vì TLS mã hóa dữ liệu đang được gửi. Điều này có thể gây vấn đề nếu bạn cố ý muốn dữ liệu được gửi công khai — ví dụ, các reverse proxy như Squid hay Varnish có khả năng cache các phản hồi HTTP, nhưng điều đó không thể với HTTPS.

Manipulation of data

Việc thao túng dữ liệu

We could imagine a number of situations in which manipulating data being sent could be bad—changing the amount of money being sent, for example. So in Figure 11-6, we need to make sure the potential attacker is unable to change the request being sent to Payment from the Order Processor.

Chúng ta có thể hình dung một số tình huống trong đó việc thao túng dữ liệu đang được gửi có thể tồi tệ — thay đổi số tiền đang được gửi, chẳng hạn. Vậy nên trong Hình 11-6, chúng ta cần đảm bảo rằng kẻ tấn công tiềm tàng không thể thay đổi yêu cầu đang được gửi tới Payment từ Order Processor.

Typically, the types of protections that make data invisible will also ensure that the data can't be manipulated (HTTPS does that, for instance). However, we could decide to send data in the open but still want to ensure it cannot be manipulated. For HTTP, one such approach is to use a hash-based message authentication code (HMAC) to sign the data being sent. With HMAC, a hash is generated and sent with the data, and the receiver can check the hash against the data to confirm that the data hasn't been changed.

Thông thường, các loại bảo vệ khiến dữ liệu vô hình cũng sẽ đảm bảo rằng dữ liệu không thể bị thao túng (HTTPS làm điều đó, chẳng hạn). Tuy nhiên, chúng ta có thể quyết định gửi dữ liệu công khai nhưng vẫn muốn đảm bảo nó không thể bị thao túng. Đối với HTTP, một cách tiếp cận như vậy là dùng một mã xác thực thông điệp dựa trên hash (HMAC) để ký dữ liệu đang được gửi. Với HMAC, một giá trị hash được sinh ra và gửi cùng dữ liệu, và bên nhận có thể kiểm tra hash so với dữ liệu để xác nhận rằng dữ liệu chưa bị thay đổi.

Data at Rest — Dữ liệu khi nghỉ

Data lying about is a liability, especially if it is sensitive. Hopefully we've done everything we can to ensure attackers cannot breach our network, and also that they cannot breach our applications or operating systems to get access to the underlying data. However, we need to be prepared in case they do—defense in depth is key.

Dữ liệu nằm vương vãi là một mối nguy, đặc biệt nếu nó nhạy cảm. Hy vọng chúng ta đã làm mọi thứ có thể để đảm bảo kẻ tấn công không thể xâm phạm mạng của chúng ta, và cũng đảm bảo rằng chúng không thể xâm phạm ứng dụng hay hệ điều hành của chúng ta để giành quyền truy cập vào dữ liệu bên dưới. Tuy nhiên, chúng ta cần chuẩn bị phòng khi chúng làm được — phòng thủ nhiều lớp là chìa khóa.

Many of the high-profile security breaches we hear of involve data at rest being acquired by an attacker, and that data being readable by the attacker. This happens either because the data was

stored in an unencrypted form or because the mechanism used to protect the data had a fundamental flaw.

Nhiều vụ vi phạm bảo mật nổi cộm mà chúng ta nghe đến liên quan đến việc dữ liệu khi nghỉ bị một kẻ tấn công thâm tóm, và dữ liệu đó đọc được bởi kẻ tấn công. Điều này xảy ra hoặc vì dữ liệu được lưu ở dạng chưa mã hóa hoặc vì cơ chế dùng để bảo vệ dữ liệu có một khiếm khuyết cơ bản.

The mechanisms by which data at rest can be protected are many and varied, but there are some general things to bear in mind.

Các cơ chế mà nhờ đó dữ liệu khi nghỉ có thể được bảo vệ thì nhiều và đa dạng, nhưng có một số điều chung cần ghi nhớ.

Go with the well known

Hãy chọn những thứ đã được biết đến rộng rãi

In some cases, you can offload the job of encrypting data to existing software—for example, by making use of your database's built-in support for encryption. However, if you find a need to encrypt and decrypt data in your own system, make sure you're going with well-known and tested implementations. The easiest way you can mess up data encryption is to try to implement your own encryption algorithms, or even try to implement someone else's. Whatever programming language you use, you'll have access to reviewed, regularly patched implementations of well-regarded encryption algorithms. Use those! And subscribe to the mailing lists/advisory lists for the technology you choose to make sure you are aware of vulnerabilities as they are found, so you can keep them patched and up to date.

Trong một số trường hợp, bạn có thể chuyển công việc mã hóa dữ liệu sang phần mềm hiện có — ví dụ, bằng cách tận dụng sự hỗ trợ tích hợp sẵn cho mã hóa của cơ sở dữ liệu của bạn. Tuy nhiên, nếu bạn thấy cần mã hóa và giải mã dữ liệu trong hệ thống của riêng mình, hãy chắc chắn rằng bạn chọn các bản triển khai đã được biết đến rộng rãi và đã được kiểm nghiệm. Cách dễ nhất để bạn làm hỏng việc mã hóa dữ liệu là cố tự triển khai các thuật toán mã hóa của riêng mình, hoặc thậm chí cố triển khai của người khác. Dù bạn dùng ngôn ngữ lập trình nào, bạn cũng sẽ có quyền truy cập vào các bản triển khai đã được rà soát, được vá lỗi đều đặn của các thuật toán mã hóa được đánh giá cao. Hãy dùng những cái đó! Và hãy đăng ký các danh sách thư/danh sách cảnh báo cho công nghệ bạn chọn để chắc chắn rằng bạn nhận thức được các lỗ hổng ngay khi chúng được phát hiện, để bạn có thể giữ chúng được vá và cập nhật.

For securing passwords, you should absolutely be using a technique called salted password hashing . This ensures that passwords are never held in plain text, and that even if an attacker brute-forces one hashed password they cannot then automatically read other passwords. 13

Để bảo vệ mật khẩu, bạn tuyệt đối nên dùng một kỹ thuật gọi là băm mật khẩu có thêm muối (salted password hashing). Điều này đảm bảo rằng mật khẩu không bao giờ được giữ ở dạng văn bản thuần, và rằng ngay cả khi một kẻ tấn công brute-force được một mật khẩu đã băm thì họ cũng không thể tự động đọc các mật khẩu khác. 13

Badly implemented encryption could be worse than having none, as the false sense of security (pardon the pun) can lead you to take your eye off the ball.

Mã hóa được triển khai tồi có thể còn tệ hơn là không có gì, vì cảm giác an toàn giả tạo (xin lỗi vì cách chơi chữ) có thể khiến bạn lơ là việc theo dõi.

Pick your targets

Hãy chọn mục tiêu của bạn

Assuming everything should be encrypted can simplify things somewhat. There is no guesswork about what should or should not be protected. However, you'll still need to think about what data can be put into logfiles to help with problem identification, and the computational overhead of encrypting everything can become pretty onerous and require more powerful hardware as a result. This is even more challenging when you're applying database migrations as part of refactoring schemas. Depending on the changes that are being made, the data may need to be decrypted, migrated, and reencrypted.

Giả định rằng mọi thứ nên được mã hóa có thể đơn giản hóa mọi việc phần nào. Không có sự phỏng đoán nào về việc cái gì nên hay không nên được bảo vệ. Tuy nhiên, bạn vẫn sẽ cần nghĩ về dữ liệu nào có thể được đưa vào tệp log để giúp nhận diện vấn đề, và chi phí tính toán của việc mã hóa mọi thứ có thể trở nên khá nặng nề và do đó đòi hỏi phần cứng mạnh hơn. Điều này càng thách thức hơn khi bạn đang áp dụng các migration cơ sở dữ liệu như một phần của việc tái cấu trúc schema. Tùy vào các thay đổi đang được thực hiện, dữ liệu có thể cần được giải mã, migrate, và mã hóa lại.

By subdividing your system into more fine-grained services, you might identify an entire data store that can be encrypted wholesale, but that is unlikely. Limiting this encryption to a known set of tables is a sensible approach.

Bằng cách chia nhỏ hệ thống của bạn thành các dịch vụ chi tiết hơn, bạn có thể nhận diện được một kho dữ liệu hoàn chỉnh có thể được mã hóa toàn bộ, nhưng điều đó không có khả năng cao. Việc giới hạn mã hóa này vào một tập bảng đã biết là một cách tiếp cận hợp lý.

Be frugal

Hãy tằn tiện

As disk space becomes cheaper and the capabilities of databases improve, the ease with which bulk amounts of information can be captured and stored is improving rapidly. This data is valuable—not only to businesses themselves, which increasingly see data as a valuable asset, but equally to the users who value their own privacy. The data that pertains to an individual or that could be used to derive information about an individual must be the data that we are most careful with.

Khi không gian đĩa trở nên rẻ hơn và các năng lực của cơ sở dữ liệu được cải thiện, sự dễ dàng mà nhờ đó lượng lớn thông tin có thể được thu thập và lưu trữ đang được cải thiện nhanh chóng. Dữ liệu này là giá trị — không chỉ với chính các doanh nghiệp, vốn ngày càng coi dữ liệu là một tài sản giá trị, mà cũng giá trị tương đương với những người dùng coi trọng quyền riêng tư của chính họ. Dữ liệu liên quan đến một cá nhân hoặc có thể được dùng để suy ra thông tin về một cá nhân phải là dữ liệu mà chúng ta cẩn thận nhất.

However, what if we made our lives a bit easier? Why not scrub as much information as possible that can be personally identifiable, and do it as soon as possible? When logging a request from a user, do we need to store the entire IP address forever, or could we replace the last few digits with x? Do we need to store someone's name, age, gender, and date of birth in order to provide them with product offers, or is their age range and postal code enough information?

Tuy nhiên, sẽ thế nào nếu chúng ta khiến cuộc sống của mình dễ dàng hơn một chút? Tại sao không xóa sạch càng nhiều thông tin có thể định danh cá nhân càng tốt, và làm điều đó càng sớm càng tốt? Khi ghi log một yêu cầu từ một người dùng, chúng ta có cần lưu trữ toàn bộ địa chỉ IP mãi mãi không, hay chúng ta có thể thay vài chữ số cuối bằng x? Chúng ta có cần lưu trữ tên, tuổi, giới tính, và ngày sinh của ai đó để cung cấp cho họ các ưu đãi

sản phẩm không, hay khoảng tuổi và mã bưu chính của họ là đủ thông tin?

13 We don't encrypt passwords at rest, as encryption means that anyone with the right key can read the password back.

13 Chúng ta không mã hóa mật khẩu khi nghỉ, vì mã hóa có nghĩa là bất kỳ ai có đúng khóa đều có thể đọc ngược lại được mật khẩu.

The advantages to being frugal with data collection are manifold. First, if you don't store it, no one can steal it. Second, if you don't store it, no one (e.g., a governmental agency) can ask for it either!

Các lợi thế của việc tằn tiện với việc thu thập dữ liệu thì rất nhiều. Thứ nhất, nếu bạn không lưu trữ nó, không ai có thể đánh cắp nó. Thứ hai, nếu bạn không lưu trữ nó, không ai (ví dụ, một cơ quan chính phủ) có thể yêu cầu nó.

The German phrase *Datensparsamkeit* represents this concept. Originating from German privacy legislation, it encapsulates the concept of storing only as much information as is absolutely required to fulfill business operations or satisfy local laws.

Cụm từ tiếng Đức *Datensparsamkeit* đại diện cho khái niệm này. Bắt nguồn từ luật về quyền riêng tư của Đức, nó gói gọn khái niệm chỉ lưu trữ đúng lượng thông tin tuyệt đối cần thiết để thực hiện các hoạt động kinh doanh hoặc thỏa mãn các luật địa phương.

This is obviously in direct tension with the move toward storing more and more information, but realizing that this tension even exists is a good start!

Điều này hiển nhiên mâu thuẫn trực tiếp với xu hướng lưu trữ ngày càng nhiều thông tin, nhưng nhận ra rằng sự mâu thuẫn này thậm chí tồn tại đã là một khởi đầu tốt!

It's all about the keys

Tất cả là về các khóa

Most forms of encryption involves making use of some key in conjunction with a suitable algorithm to create encrypted data. To decrypt the data so it can be read, authorized parties will need access to a key—either the same key or a different key (in the case of public-key encryption). So where are your keys stored? Now, if I am encrypting my data because I am worried about someone stealing my whole database, and I store the key I use in the same database, then I haven't really achieved much! Therefore, we need to store the keys somewhere else. But where?

Hầu hết các dạng mã hóa đều liên quan đến việc tận dụng một khóa nào đó kết hợp với một thuật toán phù hợp để tạo ra dữ liệu đã mã hóa. Để giải mã dữ liệu sao cho nó có thể đọc được, các bên được ủy quyền sẽ cần quyền truy cập vào một khóa — hoặc cùng một khóa hoặc một khóa khác (trong trường hợp mã hóa khóa công khai). Vậy các khóa của bạn được lưu ở đâu? Giờ thì, nếu tôi đang mã hóa dữ liệu của mình vì tôi lo lắng về việc ai đó đánh cắp toàn bộ cơ sở dữ liệu của tôi, và tôi lưu khóa mà tôi dùng trong cùng cơ sở dữ liệu đó, thì tôi chẳng đạt được mấy! Do đó, chúng ta cần lưu các khóa ở nơi khác. Nhưng ở đâu?

One solution is to use a separate security appliance to encrypt and decrypt data. Another is to use a separate key vault that your service can access when it needs a key. The life-cycle management of the keys (and access to change them) can be a vital operation, and these systems can handle this for you. This is where HashiCorp's Vault can also come in very handy.

Một giải pháp là dùng một thiết bị bảo mật riêng để mã hóa và giải mã dữ liệu. Một giải pháp khác là dùng một kho khóa (key vault) riêng mà dịch vụ của bạn có thể truy cập khi nó cần một khóa. Việc quản lý vòng đời của các khóa (và quyền truy cập để thay đổi

chúng) có thể là một hoạt động sống còn, và những hệ thống này có thể xử lý việc này cho bạn. Đây là nơi Vault của HashiCorp cũng có thể trở nên rất hữu ích.

Some databases even include built-in support for encryption, such as SQL Server's Transparent Data Encryption, which aims to handle this in a transparent fashion. Even if your database of choice does include such support, research how the keys are handled and understand if the threat you are protecting against is actually being mitigated.

Một số cơ sở dữ liệu thậm chí bao gồm hỗ trợ tích hợp sẵn cho mã hóa, chẳng hạn Transparent Data Encryption của SQL Server, vốn nhắm tới việc xử lý điều này một cách trong suốt (transparent). Ngay cả khi cơ sở dữ liệu bạn chọn quả thực bao gồm hỗ trợ như vậy, hãy nghiên cứu cách các khóa được xử lý và hiểu liệu mối đe dọa mà bạn đang phòng vệ có thực sự đang được giảm thiểu hay không.

Again, this stuff is complex. Avoid implementing your own encryption, and do some good research!

Một lần nữa, thứ này phức tạp. Hãy tránh tự triển khai mã hóa của riêng bạn, và hãy nghiên cứu kỹ lưỡng!



Encrypt data when you first see it. Only decrypt on demand, and ensure that data is never stored anywhere.

Hãy mã hóa dữ liệu ngay khi bạn nhìn thấy nó lần đầu. Chỉ giải mã theo yêu cầu, và đảm bảo rằng dữ liệu không bao giờ được lưu trữ ở bất kỳ đâu.

Encrypt backups

Mã hóa các bản sao lưu

Backups are good. We want to back up our important data. And it may seem like an obvious point, but if the data is sensitive enough that we want it to be encrypted in our running production system, then we will probably also want to make sure that any backups of the same data are also encrypted!

Sao lưu là tốt. Chúng ta muốn sao lưu dữ liệu quan trọng của mình. Và có vẻ đây là một điểm hiển nhiên, nhưng nếu dữ liệu đủ nhạy cảm đến mức chúng ta muốn nó được mã hóa trong hệ thống production đang chạy của mình, thì chúng ta có lẽ cũng sẽ muốn đảm bảo rằng bất kỳ bản sao lưu nào của cùng dữ liệu đó cũng được mã hóa!

Authentication and Authorization — Xác thực và phân quyền

Authentication and authorization are core concepts when it comes to people and things that interact with our system. In the context of security, authentication is the process by which we confirm that a party is who they say they are. We typically authenticate a human user by having them type in their username and password. We assume that only the actual user has access to this information, and therefore the person entering this information must be them. Other, more complex systems exist as well, of course. Our phones now let us use our fingerprint or face to confirm that we are who we say we are. Generally, when we're talking abstractly about who or what is being authenticated, we refer to that party as the principal.

Xác thực và phân quyền là những khái niệm cốt lõi khi nói đến những người và những thứ tương tác với hệ thống của chúng ta. Trong bối cảnh bảo mật, xác thực là quá trình mà nhờ đó chúng ta xác nhận rằng một bên đúng là kẻ mà họ tự nhận. Chúng ta thường xác thực một người dùng là con người bằng cách yêu cầu họ gõ vào username và password của mình. Chúng ta giả định rằng chỉ người dùng thực sự mới có quyền truy cập vào thông tin này, và do đó người nhập thông tin này hẳn phải là họ. Tất nhiên, cũng tồn tại những hệ thống khác phức tạp hơn. Điện thoại của chúng ta giờ đây cho phép chúng ta dùng dấu vân tay hay khuôn mặt để xác nhận rằng chúng ta đúng là kẻ mà chúng ta tự nhận. Nói chung, khi chúng ta nói một cách trừu tượng về ai hoặc cái gì đang được xác thực, chúng ta gọi bên đó là principal (chủ thể).

Authorization is the mechanism by which we map from a principal to the action we are allowing them to do. Often, when a principal is authenticated, we will be given information about them that will help us decide what we should let them do. We might, for example, be told what department or office they work in—a piece of information that our system can use to decide what the principal can and cannot do.

Phân quyền là cơ chế mà nhờ đó chúng ta ánh xạ từ một principal sang hành động mà chúng ta cho phép họ làm. Thường thì, khi một principal được xác thực, chúng ta sẽ được cung cấp thông tin về họ vốn sẽ giúp chúng ta quyết định nên cho phép họ làm gì. Chẳng hạn, chúng ta có thể được cho biết họ làm việc ở phòng ban hay văn phòng nào — một mẫu thông tin mà hệ thống của chúng ta có thể dùng để quyết định principal có thể và không thể làm gì.

Ease of use is important—we want to make it easy for our users to access our system. We don't want everyone to have to log in separately to access different microservices, using a different username and password for each one. So we also need to look at how we can implement single sign-on (SSO) in a microservices environment.

Sự dễ sử dụng là quan trọng — chúng ta muốn khiến người dùng của mình dễ dàng truy cập hệ thống. Chúng ta không muốn mọi người phải đăng nhập riêng để truy cập các microservice khác nhau, dùng một username và password khác nhau cho mỗi cái. Vậy nên chúng ta cũng cần xem xét cách chúng ta có thể triển khai single sign-on (SSO) trong một môi trường microservice.

Service-to-Service Authentication — Xác thực giữa dịch vụ với dịch vụ

Earlier we discussed mutual TLS, which, aside from protecting data in transit, also allows us to implement a form of authentication. When a client talks to a server using mutual TLS, the server is able to authenticate the client, and the client is able to authenticate the server—this is a form of service-to-service authentication. Other authentication schemes can be used besides mutual TLS. A common example is the use of API keys, where the client needs to use the key to hash a request in such a way that the server is able to verify that the client used a valid key.

Trước đó chúng ta đã bàn về mutual TLS, vốn, ngoài việc bảo vệ dữ liệu khi truyền, còn cho phép chúng ta triển khai một dạng xác thực. Khi một client trò chuyện với một máy chủ dùng mutual TLS, máy chủ có khả năng xác thực client, và client có khả năng xác thực máy chủ — đây là một dạng xác thực giữa dịch vụ với dịch vụ (service-to-service authentication). Có thể dùng các sơ đồ xác thực khác ngoài mutual TLS. Một ví dụ phổ biến là việc dùng API key, nơi client cần dùng khóa để băm một yêu cầu theo cách mà máy chủ có khả năng xác minh rằng client đã dùng một khóa hợp lệ.

Human Authentication — Xác thực con người

We're accustomed to humans authenticating themselves with the familiar username and password combination. Increasingly, though, this is being used as part of a multifactor authentication approach, where a user may need more than one piece of knowledge (a factor) to authenticate themselves. Most commonly, this takes the form of multifactor authentication (MFA), 14 where more than one factor is needed. MFA would most commonly involve the use of a normal username and password combo, in addition to providing at least one additional factor.

Chúng ta đã quen với việc con người tự xác thực bằng tổ hợp username và password quen thuộc. Tuy nhiên, ngày càng nhiều, điều này đang được dùng như một phần của một cách tiếp cận xác thực đa yếu tố, nơi một người dùng có thể cần nhiều hơn một mẫu kiến thức (một yếu tố — factor) để tự xác thực. Phổ biến nhất, điều này có hình thức xác thực đa yếu tố (MFA), 14 nơi cần nhiều hơn một yếu tố. MFA phổ biến nhất sẽ liên quan đến việc dùng một tổ hợp username và password thông thường, cùng với việc cung cấp ít nhất một yếu tố bổ sung.

The different types of authentication factors have grown in recent years—from codes sent over SMS and magic links sent via email to dedicated mobile apps like Authy and USB and NFC hardware devices like the YubiKey . Biometric factors are more commonly used now as well, as users have more access to hardware that supports things like fingerprint or face recognition. While MFA has shown itself to be much more secure as a general approach, and many public services support it, it hasn't caught on as a mass market authentication scheme, although I do expect that to change. For managing authentication of key services that are vital to the running of your software or allow access to especially sensitive information (e.g., source code access), I'd consider the use of MFA to be a must.

Các loại yếu tố xác thực khác nhau đã tăng lên trong những năm gần đây — từ mã được gửi qua SMS và đường link kỳ diệu (magic link) được gửi qua email đến các ứng dụng di động chuyên dụng như Authy cùng các thiết bị phần cứng USB và NFC như YubiKey. Các yếu tố sinh trắc học cũng được dùng phổ biến hơn bây giờ, khi người dùng có nhiều quyền truy cập hơn vào phần cứng hỗ trợ những thứ như nhận dạng vân tay hay khuôn mặt. Trong khi MFA đã chứng tỏ là an toàn hơn nhiều như một cách tiếp cận chung, và nhiều dịch vụ công cộng hỗ trợ nó, nó vẫn chưa trở nên phổ biến như một sơ đồ xác thực dành cho thị trường đại chúng, mặc dù tôi quả thực kỳ vọng điều đó sẽ thay đổi. Để quản lý việc xác thực các dịch vụ then chốt vốn sống còn đối với việc chạy phần mềm của bạn hoặc cho phép truy cập vào thông tin đặc biệt nhạy cảm (ví dụ, truy cập mã nguồn), tôi sẽ coi việc dùng MFA là điều bắt buộc.

Common Single Sign-On Implementations — Các cách triển khai Single Sign-On phổ biến

A common approach to authentication is to use some sort of single sign-on (SSO) solution to ensure that a user has only to authenticate themselves only once per session, even if during that session they may end up interacting with multiple downstream services or applications. For example, when you log in with your Google account, you are logged in on Google Calendar, Gmail, and Google Docs, even though these are separate systems.

Một cách tiếp cận phổ biến cho xác thực là dùng một dạng giải pháp single sign-on (SSO) nào đó để đảm bảo rằng một người dùng chỉ phải tự xác thực một lần mỗi phiên, ngay cả khi trong phiên đó họ có thể rất cuộc tương tác với nhiều dịch vụ hay ứng dụng phía hạ nguồn. Ví dụ, khi bạn đăng nhập bằng tài khoản Google của mình, bạn được đăng nhập trên Google Calendar, Gmail, và Google Docs, mặc dù đây là các hệ thống riêng biệt.

When a principal tries to access a resource (like a web-based **interface**), they are directed to authenticate with an identity provider . The identity provider may ask them to provide a username and password or might require the use of something more advanced like MFA. Once the identity provider is satisfied that the principal has been authenticated, it gives information to the service provider , allowing it to decide whether to grant them access to the resource.

Khi một principal cố truy cập một tài nguyên (như một giao diện trên nền web), họ được hướng đến để xác thực với một nhà cung cấp danh tính (identity provider). Nhà cung cấp danh tính có thể yêu cầu họ cung cấp một username và password hoặc có thể đòi hỏi việc dùng một thứ gì đó tiên tiến hơn như MFA. Một khi nhà cung cấp danh tính hài lòng rằng principal đã được xác thực, nó cung cấp thông tin cho nhà cung cấp dịch vụ (service provider), cho phép nó quyết định liệu có cấp cho họ quyền truy cập vào tài nguyên hay không.

14 Previously we would talk about two-factor authentication (2FA). MFA is the same concept but introduces the idea that we often now allow our users to provide an additional factor from a variety of devices, such as a secure token, a mobile authentication app, or perhaps biometrics. You can consider 2FA to be a subset of MFA.

14 Trước đây chúng ta sẽ nói về xác thực hai yếu tố (2FA). MFA là cùng một khái niệm nhưng giới thiệu ý tưởng rằng giờ đây chúng ta thường cho phép người dùng cung cấp một yếu tố bổ sung từ nhiều loại thiết bị, chẳng hạn một token bảo mật, một ứng dụng xác thực di động, hoặc có lẽ sinh trắc học. Bạn có thể coi 2FA là một tập con của MFA.

This identity provider could be an externally hosted system or something inside your own

organization. Google, for example, provides an OpenID Connect identity provider. For enterprises, though, it is common to have your own identity provider, which may be linked to your company's directory service. A directory service could be something like the Lightweight Directory Access Protocol (LDAP) or Active Directory. These systems allow you to store information about principals, such as what roles they play in the organization. Often the directory service and the identity provider are one and the same, while at other times they are separate but linked. Okta, for example, is a hosted SAML identity provider that handles tasks like two-factor authentication but can link to your company's directory services as the source of truth.

Nhà cung cấp danh tính này có thể là một hệ thống được host bên ngoài hoặc một thứ gì đó bên trong tổ chức của riêng bạn. Google, chẳng hạn, cung cấp một nhà cung cấp danh tính OpenID Connect. Tuy nhiên, đối với các doanh nghiệp, việc có nhà cung cấp danh tính của riêng mình là phổ biến, vốn có thể được liên kết với dịch vụ thư mục (directory service) của công ty bạn. Một dịch vụ thư mục có thể là một thứ gì đó như Lightweight Directory Access Protocol (LDAP) hay Active Directory. Những hệ thống này cho phép bạn lưu trữ thông tin về các principal, chẳng hạn họ đóng những vai trò nào trong tổ chức. Thường thì dịch vụ thư mục và nhà cung cấp danh tính là một và giống nhau, trong khi những lúc khác chúng tách biệt nhưng được liên kết. Okta, chẳng hạn, là một nhà cung cấp danh tính SAML được host vốn xử lý các tác vụ như xác thực hai yếu tố nhưng có thể liên kết với các dịch vụ thư mục của công ty bạn làm nguồn sự thật.

So the identity provider gives the system information about who the principal is, but the system decides what that principal is allowed to do.

Vậy nên nhà cung cấp danh tính cung cấp cho hệ thống thông tin về principal là ai, nhưng hệ thống quyết định principal đó được phép làm gì.

SAML is a SOAP-based standard and is known for being fairly complex to work with despite the libraries and tooling available to support it, and since the first edition of this book it has rapidly fallen out of favor. 15 OpenID Connect is a standard that has emerged as a specific implementation of OAuth 2.0, based on the way Google and others handle SSO. It uses simpler REST calls, and due in part to its relative simplicity and widespread support, it is the dominant mechanism for end-user SSO, and has gained significant inroads into enterprises.

SAML là một chuẩn dựa trên SOAP và được biết đến là khá phức tạp để làm việc cùng bất chấp các thư viện và công cụ có sẵn để hỗ trợ nó, và kể từ phiên bản đầu của cuốn sách này nó đã nhanh chóng mất đi sự ưa chuộng. 15 OpenID Connect là một chuẩn đã nổi lên như một bản triển khai cụ thể của OAuth 2.0, dựa trên cách Google và những bên khác xử lý SSO. Nó dùng các lời gọi REST đơn giản hơn, và một phần nhờ sự đơn giản tương đối và sự hỗ trợ rộng rãi của nó, nó là cơ chế thống trị cho SSO của người dùng cuối, và đã thâm nhập đáng kể vào các doanh nghiệp.

Single Sign-On Gateway — Cổng Single Sign-On

We could decide to handle the redirection to, and handshaking with, the identity provider within each microservice, so that any unauthenticated request from an outside party is properly dealt with. Obviously, this could mean a lot of duplicated functionality across our microservices. A shared library could help, but we'd have to be careful to avoid the **coupling** that can come from shared code (see “DRY and the Perils of Code Reuse in a Microservice World” on page 154 for more). A shared library also wouldn't help if we had microservices written in different technology stacks.

Chúng ta có thể quyết định xử lý việc chuyển hướng tới, và bắt tay với, nhà cung cấp danh

tính bên trong mỗi microservice, sao cho bất kỳ yêu cầu chưa được xác thực nào từ một bên bên ngoài đều được xử lý đúng đắn. Hiển nhiên, điều này có thể có nghĩa là rất nhiều chức năng trùng lặp xuyên suốt các microservice của chúng ta. Một thư viện chia sẻ có thể giúp ích, nhưng chúng ta sẽ phải cẩn thận để tránh sự coupling vốn có thể đến từ mã chia sẻ (xem “DRY and the Perils of Code Reuse in a Microservice World” ở trang 154 để biết thêm). Một thư viện chia sẻ cũng sẽ không giúp ích nếu chúng ta có các microservice được viết bằng các ngôn ngữ công nghệ khác nhau.

Rather than having each service manage handshaking with our identity provider, a more common approach is to use a gateway to act as a proxy, sitting between your services and the outside world (as shown in Figure 11-7). The idea is that we can centralize the behavior for redirecting the user and perform the handshake in only one place.

Thay vì để mỗi dịch vụ quản lý việc bắt tay với nhà cung cấp danh tính của chúng ta, một cách tiếp cận phổ biến hơn là dùng một cổng (gateway) để hành xử như một proxy, nằm giữa các dịch vụ của bạn và thế giới bên ngoài (như được thể hiện trong Hình 11-7). Ý tưởng là chúng ta có thể tập trung hóa hành vi chuyển hướng người dùng và thực hiện việc bắt tay ở chỉ một nơi.

15 I can't take credit for that!

15 Tôi không thể nhận công cho điều đó!

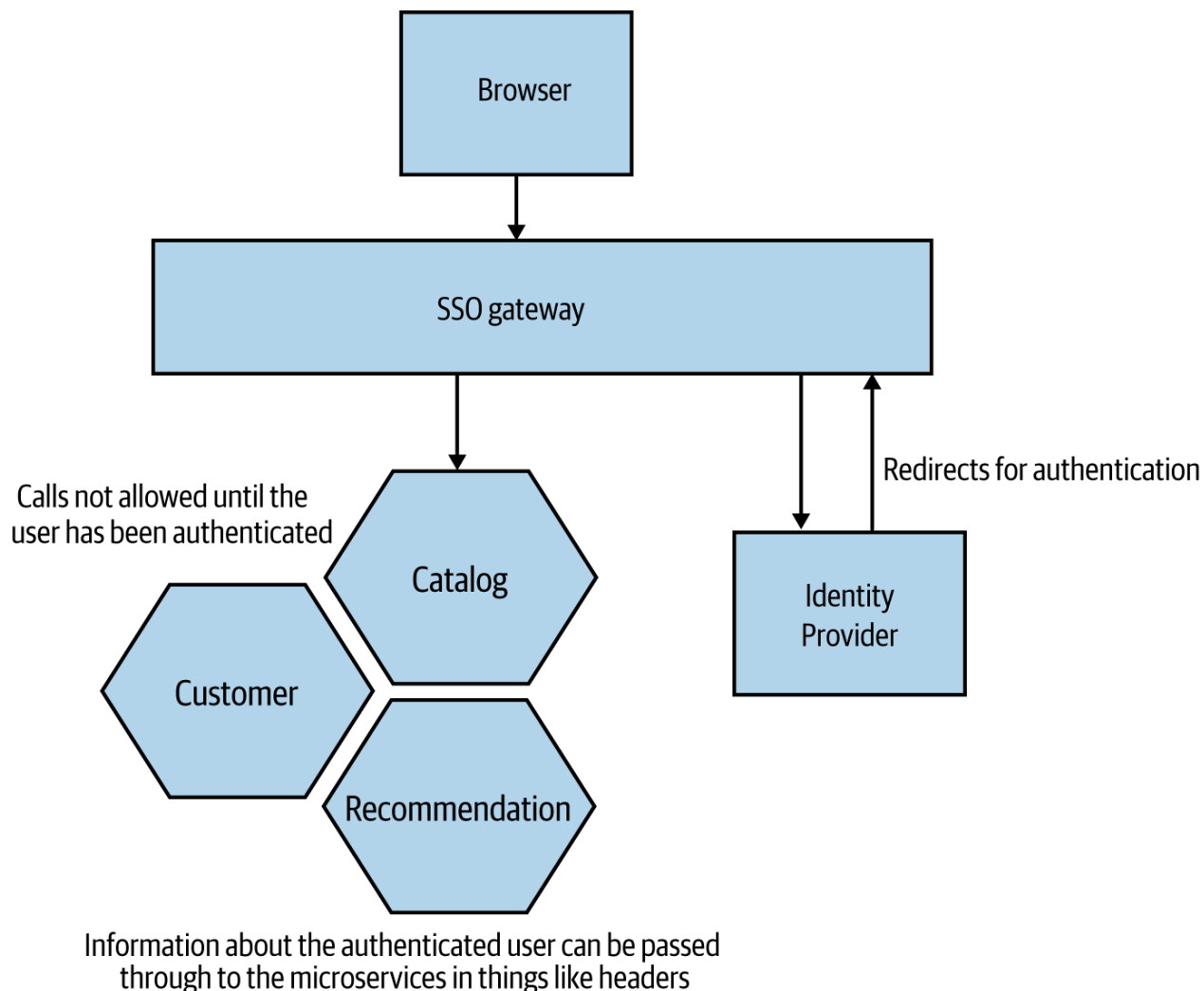


Figure 11-7. Using a gateway to handle SSO

Hình 11-7. Dùng một cổng để xử lý SSO

However, we still need to solve the problem of how the downstream service receives information about principals, such as their username or what roles they play. If you're using HTTP, you could configure your gateway to populate headers with this information. Shibboleth is one tool that can do this for you, and I've seen it used with the Apache web server to handle integration with SAML-based identity providers, to great effect. An alternative, which we'll look at in more detail shortly, is to create a **JSON Web Token (JWT)** containing all the information about the principal; this has a number of benefits, including being something we can more easily pass from microservice to microservice.

Tuy nhiên, chúng ta vẫn cần giải quyết bài toán làm sao dịch vụ hạ nguồn nhận được thông tin về các principal, chẳng hạn username của họ hay các vai trò họ đóng. Nếu bạn đang dùng HTTP, bạn có thể cấu hình cổng của mình để điền thông tin này vào các header. Shibboleth là một công cụ có thể làm điều này cho bạn, và tôi đã thấy nó được dùng với máy chủ web Apache để xử lý việc tích hợp với các nhà cung cấp danh tính dựa trên SAML, với hiệu quả tuyệt vời. Một giải pháp thay thế, mà chúng ta sẽ xem xét chi tiết hơn trong giây lát, là tạo một JSON Web Token (JWT) chứa tất cả thông tin về principal; điều này có một số lợi ích, bao gồm việc là thứ chúng ta có thể truyền dễ dàng hơn từ microservice này sang microservice khác.

Another consideration with using a single sign-on gateway is that if we have decided to offload responsibility for authentication to a gateway, it can be harder to reason about how a microservice behaves when looking at it in isolation. Remember in Chapter 9 when we explored some of the challenges of reproducing production-like environments? If you decide to use a gateway, make sure your developers can launch their services behind one without too much work.

Một cân nhắc khác với việc dùng một cổng single sign-on là nếu chúng ta đã quyết định chuyển trách nhiệm cho xác thực sang một cổng, có thể khó hơn để lý giải về cách một microservice hành xử khi nhìn nó một cách cô lập. Hãy nhớ ở Chương 9 khi chúng ta khám phá một số thách thức của việc tái tạo các môi trường giống production? Nếu bạn quyết định dùng một cổng, hãy chắc chắn rằng các lập trình viên của bạn có thể khởi chạy các dịch vụ của họ phía sau một cổng mà không cần quá nhiều công sức.

One final problem with this approach is that it can lull you into a false sense of security. Again, I like to return to the idea of defense in depth—from network perimeter to subnet, firewall, machine, operating system, and the underlying hardware. You have the ability to implement security measures at all of these points. I have seen some people put all their eggs in one basket, relying on the gateway to handle every step for them. And we all know what happens when we have a single point of failure...

Một vấn đề cuối cùng với cách tiếp cận này là nó có thể ru ngủ bạn vào một cảm giác an toàn giả tạo. Một lần nữa, tôi thích quay lại với ý tưởng phòng thủ nhiều lớp — từ vành đai mạng đến subnet, tường lửa, máy, hệ điều hành, và phần cứng nền tảng. Bạn có khả năng triển khai các biện pháp bảo mật ở tất cả các điểm này. Tôi đã thấy một số người đặt tất cả trứng vào một giỏ, dựa vào cổng để xử lý mọi bước cho họ. Và tất cả chúng ta đều biết điều gì xảy ra khi chúng ta có một điểm lỗi đơn (single point of failure)...

Obviously you could use this gateway to do other things. For example, you could also decide to terminate HTTPS at this level, run intrusion detection, and so on. Do be careful, though. Gateway layers tend to take on more and more functionality, which itself can end up being a giant coupling point. And the more functionality something has, the greater the attack surface.

Hiển nhiên bạn có thể dùng cổng này để làm những việc khác. Chẳng hạn, bạn cũng có thể quyết định kết thúc HTTPS ở mức này, chạy phát hiện xâm nhập, v.v. Tuy nhiên, hãy cẩn thận. Các lớp cổng có xu hướng gánh vác ngày càng nhiều chức năng, mà bản thân điều này rất cuộc có thể trở thành một điểm coupling khổng lồ. Và một thứ càng có nhiều chức năng, bề mặt tấn công càng lớn.

Fine-Grained Authorization — Phân quyền chi tiết

A gateway may be able to provide fairly effective coarse-grained authentication. For example, it could prevent any non-logged-in users from accessing the helpdesk application. Assuming our gateway can extract attributes about the principal as a result of the authentication, it may be able to make more nuanced decisions. For example, it is common to place people in groups or assign them to roles. We can use this information to understand what they can do. So for the helpdesk application, we might allow access only to principals with a specific role (e.g., STAFF). Beyond allowing (or disallowing) access to specific resources or endpoints, though, we need to leave the rest to the microservice itself; it will need to make further decisions about what operations to allow.

Một cổng có thể có khả năng cung cấp xác thực thô (coarse-grained) khá hiệu quả. Chẳng hạn, nó có thể ngăn bất kỳ người dùng chưa đăng nhập nào truy cập ứng dụng helpdesk. Giả sử cổng của chúng ta có thể trích xuất các thuộc tính về principal như là kết quả của việc xác thực, nó có thể có khả năng đưa ra các quyết định tinh tế hơn. Chẳng hạn, việc

đặt người ta vào các nhóm hoặc gán họ vào các vai trò là phổ biến. Chúng ta có thể dùng thông tin này để hiểu họ có thể làm gì. Vậy nên đối với ứng dụng helpdesk, chúng ta có thể chỉ cho phép truy cập đối với các principal có một vai trò cụ thể (ví dụ, STAFF). Tuy nhiên, ngoài việc cho phép (hay không cho phép) truy cập vào các tài nguyên hay endpoint cụ thể, chúng ta cần để phần còn lại cho chính microservice; nó sẽ cần đưa ra các quyết định sâu hơn về việc cho phép những thao tác nào.

Back to our helpdesk application: do we allow any staff member to see any and all details? More likely, we'll have different roles at work. For example, a principal in the CALL_CENTER group might be allowed to view any piece of information about a customer except their payment details. This principal might also be able to issue refunds, but that amount might be capped. Someone who has the CALL_CENTER_TEAM_LEADER role, however, might be able to issue larger refunds.

Quay lại ứng dụng helpdesk của chúng ta: chúng ta có cho phép bất kỳ nhân viên nào xem mọi chi tiết không? Nhiều khả năng hơn, chúng ta sẽ có các vai trò khác nhau hoạt động. Chẳng hạn, một principal trong nhóm CALL_CENTER có thể được phép xem bất kỳ mẫu thông tin nào về một khách hàng ngoại trừ chi tiết thanh toán của họ. Principal này cũng có thể có khả năng phát hành hoàn tiền, nhưng số tiền đó có thể bị giới hạn mức trần. Tuy nhiên, ai đó có vai trò CALL_CENTER_TEAM_LEADER có thể có khả năng phát hành các khoản hoàn tiền lớn hơn.

These decisions need to be local to the microservice in question. I have seen people use the various attributes supplied by identity providers in horrible ways, using really fine-grained roles like CALL_CENTER_50_DOLLAR_REFUND, where they end up putting information specific to one piece of microservice functionality into their directory services. This is a nightmare to maintain and gives very little scope for our services to have their own independent life cycle, as suddenly a chunk of information about how a service behaves lives elsewhere, perhaps in a system managed by a different part of the organization.

Những quyết định này cần cục bộ với microservice đang xét. Tôi đã thấy người ta dùng các thuộc tính khác nhau do nhà cung cấp danh tính cung cấp theo những cách kinh khủng, dùng các vai trò thực sự chi tiết như CALL_CENTER_50_DOLLAR_REFUND, nơi họ rót cuộc đưa thông tin cụ thể cho một mẫu chức năng microservice vào các dịch vụ thư mục của họ. Đây là một cơn ác mộng để bảo trì và cho rất ít không gian để các dịch vụ của chúng ta có vòng đời độc lập của riêng chúng, vì đột nhiên một khối thông tin về cách một dịch vụ hành xử lại sống ở nơi khác, có lẽ trong một hệ thống được quản lý bởi một bộ phận khác của tổ chức.

Making sure the microservice has the information it needs to assess finer-grained authorization requests is worthy of further discussion—we'll revisit this when we look at JWTs in a moment.

Việc đảm bảo rằng microservice có thông tin nó cần để đánh giá các yêu cầu phân quyền chi tiết hơn xứng đáng được thảo luận thêm — chúng ta sẽ quay lại điều này khi chúng ta xem xét JWT trong giây lát.

Instead, favor coarse-grained roles modeled around how your organization works. Going all the way back to the early chapters, remember that we are building software to match how our organization works. So use your roles in this way too.

Thay vào đó, hãy ưu ái các vai trò thô được mô hình hóa quanh cách tổ chức của bạn hoạt động. Quay trở lại tận các chương đầu, hãy nhớ rằng chúng ta đang xây dựng phần mềm để khớp với cách tổ chức của chúng ta hoạt động. Vậy nên hãy dùng các vai trò của bạn theo cách này nữa.

The Confused Deputy Problem — Vấn đề Phó tướng Bối rối

Having a principal authenticate with the system as a whole using something like an SSO gateway is simple enough, and this could be enough to control access to a given microservice. But what happens if that microservice then needs to make additional calls to complete an operation? This can leave us open to a type of vulnerability known as the confused deputy problem. This occurs when an upstream party tricks an intermediate party into doing things it shouldn't be doing. Let's look at a concrete example in Figure 11-8, which illustrates MusicCorp's online shopping site. Our browser-based JavaScript UI talks to the server-side Web Shop microservice, which is a type of **backend for frontend**. We'll explore this in more depth in "Pattern: Backend for **Frontend** (BFF)" on page 480, but for the moment, think of it as a server-side component that performs call aggregation and filtering for a specific external interface (in our case, our browser-based JavaScript UI). Calls made between the browser and the Web Shop can be authenticated using OpenID Connect. So far, so good.

Việc có một principal xác thực với hệ thống nói chung bằng một thứ gì đó như một cổng SSO thì đủ đơn giản, và điều này có thể đủ để kiểm soát truy cập vào một microservice cho trước. Nhưng điều gì xảy ra nếu microservice đó sau đó cần thực hiện các lời gọi bổ sung để hoàn thành một thao tác? Điều này có thể khiến chúng ta dễ bị một loại lỗ hổng được biết đến là vấn đề phó tướng bối rối (confused deputy problem). Điều này xảy ra khi một bên thượng nguồn lừa một bên trung gian làm những việc mà nó không nên làm. Hãy cùng nhìn vào một ví dụ cụ thể trong Hình 11-8, vốn minh họa trang mua sắm trực tuyến của MusicCorp. UI JavaScript chạy trên trình duyệt của chúng ta trò chuyện với microservice Web Shop phía máy chủ, vốn là một dạng backend for frontend. Chúng ta sẽ khám phá điều này sâu hơn trong "Pattern: Backend for Frontend (BFF)" ở trang 480, nhưng tạm thời, hãy nghĩ về nó như một thành phần phía máy chủ thực hiện việc tổng hợp và lọc lời gọi cho một giao diện bên ngoài cụ thể (trong trường hợp của chúng ta, là UI JavaScript chạy trên trình duyệt của chúng ta). Các lời gọi được thực hiện giữa trình duyệt và Web Shop có thể được xác thực bằng OpenID Connect. Cho đến giờ thì ổn.

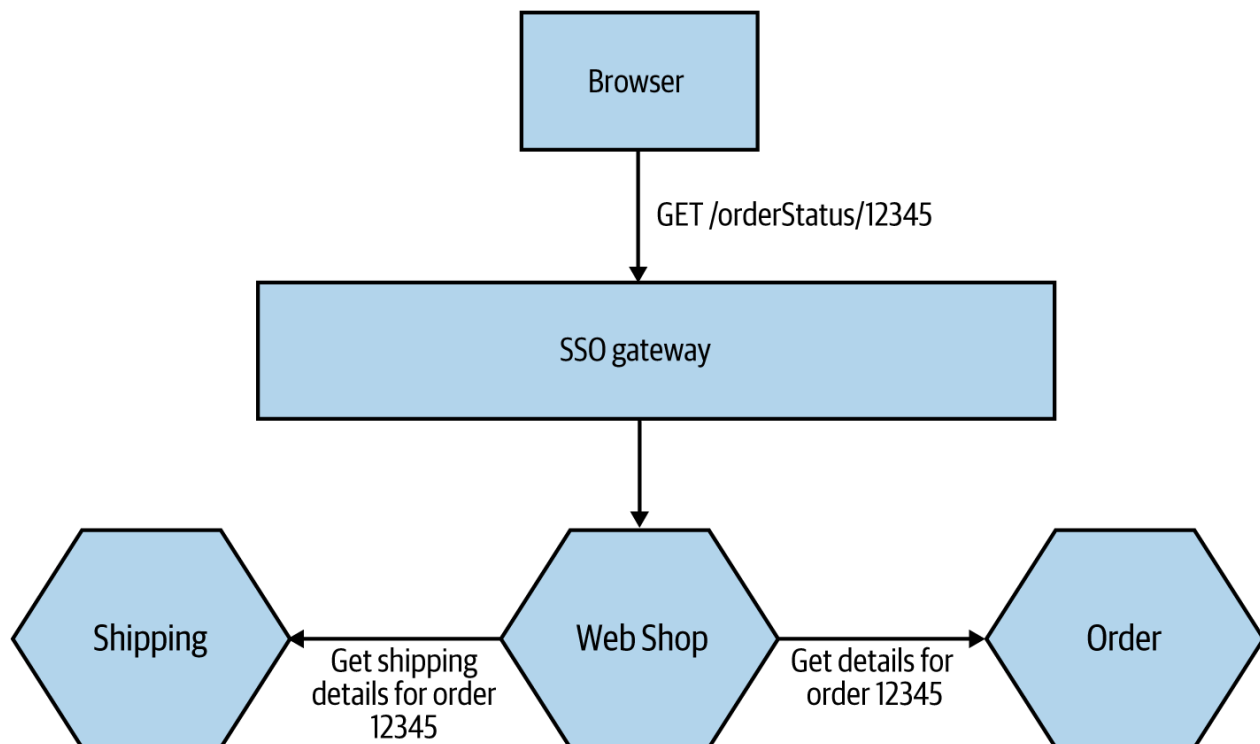


Figure 11-8. An example where a confused deputy could come into play When a user is logged in, they can click on a link to view details of an order. To display the information, we need to pull back the original order from the Order service, but we also want to look up shipping information for the order. So when a logged-in customer clicks a link for /orderStatus/12345 , this request is routed to Web Shop , which then needs to make calls to the downstream Order and Shipping microservices asking for the details for order 12345 .

Hình 11-8. Một ví dụ nơi một phó tướng bối rối có thể xuất hiện Khi một người dùng đã đăng nhập, họ có thể nhấp vào một đường link để xem chi tiết của một đơn hàng. Để hiển thị thông tin, chúng ta cần kéo về đơn hàng gốc từ dịch vụ Order, nhưng chúng ta cũng muốn tra cứu thông tin vận chuyển cho đơn hàng. Vậy nên khi một khách hàng đã đăng nhập nhấp vào một đường link cho /orderStatus/12345, yêu cầu này được định tuyến tới Web Shop, vốn sau đó cần thực hiện các lời gọi tới các microservice Order và Shipping phía hạ nguồn để hỏi chi tiết cho đơn hàng 12345.

But should these downstream services accept the calls from the Web Shop ? We could adopt a stance of implicit trust: because the call came from within our perimeter, it is OK. We could even use certificates or API keys to confirm that it really is the Web Shop asking for this information. But is this enough? For example, a customer who is logged into the online shopping system can see their personal account details. What if the customer could trick the web shop's UI into making a request for someone else's details, maybe by making a call with their own logged-in credentials?

Nhưng liệu các dịch vụ hạ nguồn này có nên chấp nhận các lời gọi từ Web Shop không? Chúng ta có thể đón nhận một lập trường tin cậy ngầm định: vì lời gọi đến từ bên trong vành đai của chúng ta, nó ổn. Chúng ta thậm chí có thể dùng chứng chỉ hoặc API key để xác nhận rằng nó thực sự là Web Shop đang hỏi thông tin này. Nhưng như vậy có đủ không? Chẳng hạn, một khách hàng đã đăng nhập vào hệ thống mua sắm trực tuyến có thể thấy chi tiết tài khoản cá nhân của họ. Nếu khách hàng có thể lừa UI của web shop thực hiện một yêu cầu cho chi tiết của người khác thì sao, có lẽ bằng cách thực hiện một lời gọi với credential đã đăng nhập của chính họ?

In this example, what is to stop the customer from asking for orders that are not theirs? Once logged in, they could start sending requests for other orders that aren't theirs to see if they could get useful information. They could start guessing order IDs to see if they could extract other people's information. Fundamentally, what has happened here is that while we have authenticated the user in question, we aren't providing sufficient authorization . What we want is some part of our system to be able to judge that a request to see User A's details can be granted only if it's User A asking to see them. Where does this logic live, though?

Trong ví dụ này, cái gì ngăn khách hàng hỏi về các đơn hàng không phải của họ? Một khi đã đăng nhập, họ có thể bắt đầu gửi các yêu cầu cho các đơn hàng khác không phải của họ để xem liệu họ có thể lấy được thông tin hữu ích không. Họ có thể bắt đầu đoán các ID đơn hàng để xem liệu họ có thể trích xuất thông tin của người khác không. Về cơ bản, điều đã xảy ra ở đây là trong khi chúng ta đã xác thực người dùng đang xét, chúng ta lại không cung cấp đủ sự phân quyền. Điều chúng ta muốn là một phần nào đó của hệ thống có khả năng phán đoán rằng một yêu cầu xem chi tiết của Người dùng A chỉ có thể được cấp nếu chính Người dùng A đang yêu cầu xem chúng. Tuy nhiên, logic này sống ở đâu?

Centralized, Upstream Authorization — Phân quyền tập trung, thượng nguồn

One option for avoiding the confused deputy problem is to perform all required authorization as soon as the request is received in our system. In Figure 11-8, this would mean that we would aim to authorize the request either in the SSO gateway itself or in the Web Shop. The idea is that by the time the calls are sent to the Order or Shipping microservice, we assume that the requests are allowed.

Một lựa chọn để tránh vấn đề phó tướng bối rối là thực hiện tất cả việc phân quyền cần thiết ngay khi yêu cầu được nhận vào hệ thống của chúng ta. Trong Hình 11-8, điều này sẽ có nghĩa là chúng ta sẽ nhắm tới việc phân quyền cho yêu cầu hoặc trong chính cổng SSO hoặc trong Web Shop. Ý tưởng là vào lúc các lời gọi được gửi tới microservice Order hay Shipping, chúng ta giả định rằng các yêu cầu được cho phép.

This form of upstream authorization effectively implies that we are accepting some form of implicit trust (as opposed to zero trust)—the Shipping and Order microservices have to assume they are only being sent requests that they are allowed to fulfill. The other issue is that an upstream entity—for example, a gateway or something similar—needs to have knowledge of what functionality downstream microservices provide, and it needs to know how to limit access to that functionality.

Dạng phân quyền thượng nguồn này về cơ bản ngụ ý rằng chúng ta đang chấp nhận một dạng tin cậy ngầm định nào đó (trái ngược với zero trust) — các microservice Shipping và Order phải giả định rằng chúng chỉ được gửi các yêu cầu mà chúng được phép thực hiện. Vấn đề khác là một thực thể thượng nguồn — ví dụ, một cổng hay thứ gì đó tương tự — cần có hiểu biết về chức năng nào mà các microservice hạ nguồn cung cấp, và nó cần biết cách giới hạn quyền truy cập vào chức năng đó.

Ideally, though, we want our microservices to be as self-contained as possible, to make it as easy as possible to make changes and roll out new functionality. We want our releases to be as simple as possible—we want **independent deployability**. If the act of deployment now involves both deploying a new microservice and applying some authorization-related configuration to an upstream gateway, then that doesn't seem terribly "independent" to me.

Tuy nhiên, lý tưởng nhất, chúng ta muốn các microservice của mình tự chứa (self-contained) hết mức có thể, để khiến việc thực hiện thay đổi và tung ra chức năng mới dễ dàng nhất có thể. Chúng ta muốn các lần phát hành của mình đơn giản nhất có thể — chúng ta muốn khả năng triển khai độc lập. Nếu hành động triển khai giờ đây liên quan đến cả việc triển khai một microservice mới lẫn việc áp dụng một số cấu hình liên quan đến phân quyền cho một cổng thượng nguồn, thì điều đó với tôi có vẻ chẳng "độc lập" cho lắm.

Therefore, we'd want to push the decision about whether or not the call should be authorized into the same microservice where the functionality being requested lives. This makes the microservice more self-contained and also gives us the option to implement zero trust if we want.

Do đó, chúng ta sẽ muốn đẩy quyết định về việc liệu lời gọi có nên được phân quyền hay không vào chính microservice nơi chức năng đang được yêu cầu sống. Điều này khiến microservice tự chứa hơn và cũng cho chúng ta lựa chọn triển khai zero trust nếu chúng ta muốn.

Decentralizing Authorization — Phi tập trung hóa phân quyền

Given the challenges of centralized authorization in a microservices environment, we'd like to push this logic to the downstream microservice. The Order microservice is where the functionality for accessing order details lives, so it would make logical sense for that service to decide if the call is valid. In this specific case, though, the Order microservice needs information about what human is making the request. So how do we get that information to the Order microservice?

Với những thách thức của việc phân quyền tập trung trong một môi trường microservice, chúng ta muốn đẩy logic này tới microservice hạ nguồn. Microservice Order là nơi chức năng truy cập chi tiết đơn hàng sống, vậy nên sẽ hợp logic khi dịch vụ đó quyết định liệu lời gọi có hợp lệ không. Tuy nhiên, trong trường hợp cụ thể này, microservice Order cần thông tin về con người nào đang thực hiện yêu cầu. Vậy làm sao chúng ta đưa thông tin đó tới microservice Order?

At the simplest level, we could just require that the identifier for the person making the request be sent to the Order microservice. If using HTTP, for example, we could just stick the username in a header. But in such a case, what's to stop a malicious party from inserting just any old name into the request to get the information they need? Ideally, we want a way to make sure that the request is really being made on behalf of an authenticated user and that we can pass along additional information about that user—for example, the groups that user might fall in.

Ở mức đơn giản nhất, chúng ta có thể chỉ yêu cầu rằng định danh của người thực hiện yêu cầu được gửi tới microservice Order. Ví dụ, nếu dùng HTTP, chúng ta có thể chỉ nhét username vào một header. Nhưng trong một trường hợp như vậy, cái gì ngăn một bên độc hại chèn bất kỳ cái tên cũ rích nào vào yêu cầu để lấy thông tin họ cần? Lý tưởng nhất, chúng ta muốn một cách để đảm bảo rằng yêu cầu thực sự đang được thực hiện thay mặt cho một người dùng đã được xác thực và rằng chúng ta có thể truyền kèm thông tin bổ sung về người dùng đó — ví dụ, các nhóm mà người dùng đó có thể thuộc về.

Historically, there are a variety of different ways that this has been handled (including techniques like nested SAML assertions, which, yes, are as painful as they sound), but recently the most common solution to this particular problem has been to use JSON Web Tokens.

Trong lịch sử, có nhiều cách khác nhau mà điều này đã được xử lý (bao gồm các kỹ thuật như các xác nhận SAML lồng nhau, mà, vâng, đúng là đau khổ như cái tên của chúng), nhưng gần đây giải pháp phổ biến nhất cho bài toán cụ thể này là dùng JSON Web Token.

JSON Web Tokens — JSON Web Token

JWTs allow you to store multiple claims about an individual into a string that can be passed around. This token can be signed to ensure that the structure of the token hasn't been manipulated, and it can also optionally be encrypted to provide cryptographic guarantees around who can read the data. Although JWTs can be used for generic exchanges of information where it is important to ensure that the data hasn't been tampered with, they are most commonly used to help transmit information to aid authorization.

JWT cho phép bạn lưu trữ nhiều claim về một cá nhân vào một chuỗi có thể được truyền đi khắp nơi. Token này có thể được ký để đảm bảo rằng cấu trúc của token chưa bị thao túng, và nó cũng có thể tùy chọn được mã hóa để cung cấp các đảm bảo mật mã quanh việc ai có thể đọc dữ liệu. Mặc dù JWT có thể được dùng cho các trao đổi thông tin chung nơi quan trọng là phải đảm bảo rằng dữ liệu chưa bị giả mạo, chúng phổ biến nhất được

dùng để giúp truyền thông tin nhằm hỗ trợ phân quyền.

Once signed, the JWTs can easily be passed via a variety of protocols, and the tokens can optionally be configured to expire after a certain period of time. They are widely supported, with a number of identity providers that support generating JWTs, and a large number of libraries for using JWTs inside your own code.

Một khi đã được ký, JWT có thể dễ dàng được truyền qua nhiều giao thức khác nhau, và các token có thể tùy chọn được cấu hình để hết hạn sau một khoảng thời gian nhất định. Chúng được hỗ trợ rộng rãi, với một số nhà cung cấp danh tính hỗ trợ việc sinh JWT, và một số lượng lớn các thư viện để dùng JWT bên trong mã của riêng bạn.

Format

Định dạng

The main payload of a JWT is a JSON structure, which broadly speaking can contain anything you want. We can see an example token in Example 11-1. The JWT standard does describe some specifically named fields (“public claims”) that you should use if they relate to you. For example, `exp` defines the expiration date of a token. If you use these public claim fields correctly, there is a good chance that the libraries you use will be able to make use of them appropriately—rejecting a token, for example, if the `exp` field states that the token has already expired.¹⁶ Even if you won’t use all of these public claims, it’s worth being aware of what they are to ensure you don’t end up using them for your own application-specific uses, as this could cause some odd behavior in supporting libraries.

Payload chính của một JWT là một cấu trúc JSON, mà nói rộng ra có thể chứa bất cứ thứ gì bạn muốn. Chúng ta có thể thấy một token ví dụ trong Ví dụ 11-1. Chuẩn JWT quả thực mô tả một số trường được đặt tên cụ thể (“public claim”) mà bạn nên dùng nếu chúng liên quan đến bạn. Ví dụ, exp định nghĩa ngày hết hạn của một token. Nếu bạn dùng các trường public claim này một cách đúng đắn, có khả năng cao là các thư viện bạn dùng sẽ có khả năng tận dụng chúng một cách phù hợp — chẳng hạn, từ chối một token nếu trường exp cho biết rằng token đã hết hạn. Ngay cả khi bạn sẽ không dùng tất cả các public claim này, cũng đáng để nhận thức được chúng là gì nhằm đảm bảo rằng bạn không rút cuộc dùng chúng cho các mục đích riêng cụ thể của ứng dụng của mình, vì điều này có thể gây ra một số hành vi kỳ lạ trong các thư viện hỗ trợ.

Example 11-1. An example of the JSON payload of a JWT

Ví dụ 11-1. Một ví dụ về payload JSON của một JWT

```
{ "sub": "123", "name": "Sam Newman", "exp": 1606741736, "groups": "admin, beta" }
```

```
{ "sub": "123", "name": "Sam Newman", "exp": 1606741736, "groups": "admin, beta" }
```

In Example 11-2, we see the token from Example 11-1 encoded. This token is really just a single string, but it is split into three parts delineated with a “.”—the header, payload, and signature.

Trong Ví dụ 11-2, chúng ta thấy token từ Ví dụ 11-1 được mã hóa. Token này thực ra chỉ là một chuỗi đơn, nhưng nó được chia thành ba phần được phân định bằng một dấu “.” — phần header, payload, và signature (chữ ký).

Example 11-2. The result of encoding a JWT payload

Ví dụ 11-2. Kết quả của việc mã hóa một payload JWT

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjMiLCJpYXN0IjoiU2FtIE5ld21hbiIsImV4cCI6MTYwNjc0MTc0LmZ9HMhODGs60I0P5bVVSFixeDxJjGovOEtlNUi iE 0


```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM5ZWRlcjE1IiwiaWF0IjoiYU2FtIE5ld21hbiIsImV4cCI6MTYwNjc0LjZ9MHM0DGs60IOP5bVVSFixeDxJjGovQEtlNUI__iE_0
```

The header

Phần header

The payload (truncated)

Phần payload (đã cắt bớt)

The signature

Phần chữ ký

For the benefit of the example here, I have split the line at each part, but in reality this would be a single string with no line breaks. The header contains information about the signing algorithm being used. This allows the program decoding the token to support different signing schemes. The payload is where we store information about

Để phục vụ cho ví dụ ở đây, tôi đã chia dòng tại mỗi phần, nhưng trong thực tế đây sẽ là một chuỗi đơn không có ngắt dòng. Phần header chứa thông tin về thuật toán ký đang được dùng. Điều này cho phép chương trình giải mã token hỗ trợ các sơ đồ ký khác nhau. Phần payload là nơi chúng ta lưu trữ thông tin về

16 The JWT site has an excellent overview of which libraries support which public claims—it's a great resource in general for all things JWT.

16 Trang JWT có một bản tổng quan tuyệt vời về thư viện nào hỗ trợ public claim nào — nó là một nguồn tài nguyên tuyệt vời nói chung cho mọi thứ về JWT.

the claims that the token is making—this is just the result of encoding the JSON structure in Example 11-1 . The signature is used to ensure that the payload hasn't been manipulated, and it can also be used to ensure that the token was generated by whom you think it was (assuming the token is signed with a private key).

các claim mà token đang đưa ra — đây chỉ là kết quả của việc mã hóa cấu trúc JSON trong Ví dụ 11-1. Phần chữ ký được dùng để đảm bảo rằng payload chưa bị thao túng, và nó cũng có thể được dùng để đảm bảo rằng token được sinh ra bởi người mà bạn nghĩ (giả sử token được ký bằng một khóa riêng tư).

As a simple string, this token can be easily passed along via different communication protocols—as a header in HTTP (in the Authorization header), for example, or perhaps as a piece of metadata in a message. This encoded string can of course be sent over encrypted transport protocol—for example, TLS over HTTP—in which case the token wouldn't be visible to people observing the communication.

Là một chuỗi đơn giản, token này có thể dễ dàng được truyền kèm qua các giao thức giao tiếp khác nhau — như một header trong HTTP (trong header Authorization), chẳng hạn, hoặc có lẽ như một mẫu metadata trong một thông điệp. Chuỗi đã mã hóa này tất nhiên có thể được gửi qua giao thức truyền tải được mã hóa — ví dụ, TLS trên HTTP — trong trường hợp đó token sẽ không nhìn thấy được đối với những người quan sát giao tiếp.

Using tokens

Sử dụng token

Let's take a look at a common way to use JWT tokens in a microservice architecture. In Figure 11-9 , our customer logs in as normal, and once authenticated we generate some sort of token to represent their logged-in session (likely an OAuth token), which is stored on the client device. Subsequent

requests from that client device hit our gateway, which generates a JWT token that will be valid for the duration of that request. It is this JWT token that is then passed to the downstream microservices. They are able to validate the token and extract claims from the payload to determine what sort of authorization is appropriate.

Hãy cùng nhìn vào một cách phổ biến để dùng JWT token trong một kiến trúc microservice. Trong Hình 11-9, khách hàng của chúng ta đăng nhập như bình thường, và một khi đã được xác thực chúng ta sinh ra một dạng token nào đó để đại diện cho phiên đã đăng nhập của họ (nhiều khả năng là một OAuth token), vốn được lưu trên thiết bị client. Các yêu cầu tiếp theo từ thiết bị client đó tới cổng của chúng ta, vốn sinh ra một JWT token sẽ hợp lệ trong suốt khoảng thời gian của yêu cầu đó. Chính JWT token này sau đó được truyền tới các microservice hạ nguồn. Chúng có khả năng xác thực token và trích xuất các claim từ payload để xác định loại phân quyền nào là phù hợp.

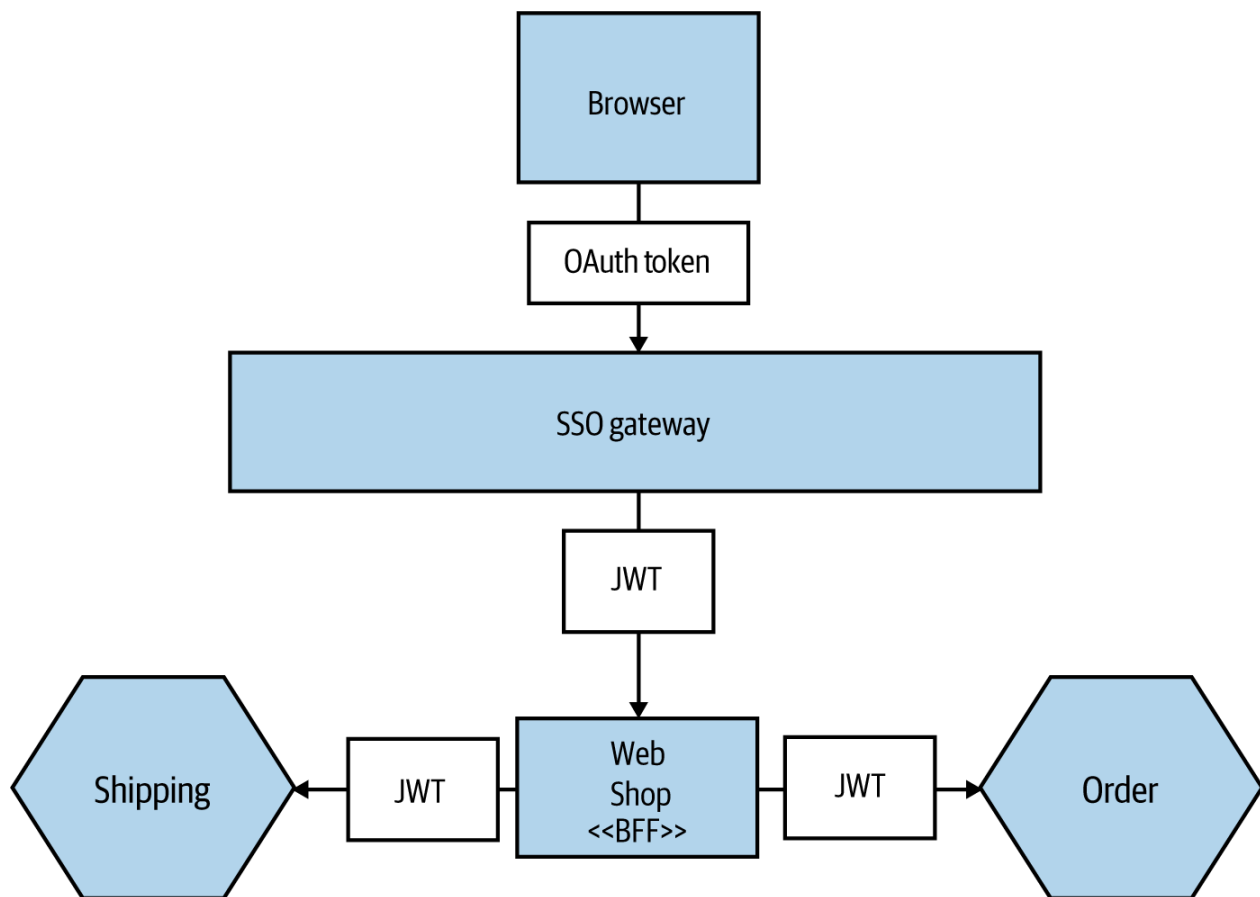


Figure 11-9. A JWT token is generated for a specific request and passed to downstream microservices. A variation to this approach is to generate a JWT token when the user initially authenticates themselves with the system and then have that JWT token stored on the client device. It's worth considering, though, that such a token will have to be valid for the duration of the logged-in session; as we've already discussed, we'd like to limit the period of validity for system-generated credentials to reduce the chances of them being misused, and to reduce the impact if we need to change the keys that are used for generating the encoded token. Generating a JWT token on a per-request basis seems to be the most common solution to this problem, as we show in Figure 11-9. Having some sort of token exchange done in the gateway can also make it much easier to adopt the use of JWT tokens without having to change any part of the authentication flow that involves communication with the client device—if you already have a working SSO solution in place, hiding the fact that JWT tokens

are even used from the main user authentication flow will make such a change less disruptive.

Hình 11-9. Một JWT token được sinh ra cho một yêu cầu cụ thể và được truyền tới các microservice hạ nguồn. Một biến thể của cách tiếp cận này là sinh một JWT token khi người dùng ban đầu tự xác thực với hệ thống rồi sau đó có JWT token đó được lưu trên thiết bị client. Tuy nhiên, đáng để cân nhắc rằng một token như vậy sẽ phải hợp lệ trong suốt khoảng thời gian của phiên đã đăng nhập; như chúng ta đã bàn, chúng ta muốn giới hạn khoảng thời gian hợp lệ cho các credential do hệ thống sinh ra để giảm khả năng chúng bị lạm dụng, và để giảm tác động nếu chúng ta cần thay đổi các khóa được dùng để sinh token đã mã hóa. Việc sinh một JWT token trên cơ sở mỗi yêu cầu có vẻ là giải pháp phổ biến nhất cho bài toán này, như chúng ta thể hiện trong Hình 11-9. Việc có một dạng trao đổi token nào đó được thực hiện trong cổng cũng có thể khiến việc đón nhận sử dụng JWT token dễ dàng hơn nhiều mà không cần thay đổi bất kỳ phần nào của luồng xác thực vốn liên quan đến giao tiếp với thiết bị client — nếu bạn đã có một giải pháp SSO hoạt động được tại chỗ, việc giấu đi sự thật rằng JWT token thậm chí đang được dùng khỏi luồng xác thực người dùng chính sẽ khiến một thay đổi như vậy ít gây gián đoạn hơn.

So with suitable JWT token generation, our downstream microservices are able to get all the information they need to confirm the identity of the user making the request, as well as additional information such as the groups or roles that user is in. The validity of this token can also be checked by the microservice simply by checking the signature of the JWT token as well. Compared to the previous solutions in this space (such as nested SAML assertions), JWT tokens have made the process of decentralizing authorization in a microservice architecture much simpler.

Vậy nên với việc sinh JWT token phù hợp, các microservice hạ nguồn của chúng ta có khả năng lấy tất cả thông tin chúng cần để xác nhận danh tính của người dùng đang thực hiện yêu cầu, cũng như thông tin bổ sung chẳng hạn các nhóm hay vai trò mà người dùng đó thuộc về. Tính hợp lệ của token này cũng có thể được microservice kiểm tra đơn giản bằng cách kiểm tra chữ ký của JWT token nữa. So với các giải pháp trước đây trong lĩnh vực này (chẳng hạn các xác nhận SAML lồng nhau), JWT token đã khiến quá trình phi tập trung hóa phân quyền trong một kiến trúc microservice đơn giản hơn nhiều.

Challenges

Thách thức

There are a few issues around JWT tokens that are worth bearing in mind. The first is the issue of keys. In the case of signed JWT tokens, to verify a signature, the receiver of a JWT token is going to need some information that will need to be communicated out of band—normally a public key. All the issues of key management apply in this case. How does the microservice get the public key? What happens if the public key needs to change? Vault is an example of a tool that could be used by a microservice to retrieve (and handle rotation of) public keys, and it is already designed to work in a highly distributed environment. You could of course just hardcode a public key in a configuration file for the receiving microservice, but you would then have the issue of dealing with the public key changing.

Có một vài vấn đề quanh JWT token đáng để ghi nhớ. Vấn đề đầu tiên là vấn đề về các khóa. Trong trường hợp các JWT token đã ký, để xác minh một chữ ký, bên nhận một JWT token sẽ cần một số thông tin sẽ cần được truyền đạt ngoài luồng (out of band) — thường là một khóa công khai. Tất cả các vấn đề của việc quản lý khóa đều áp dụng trong trường hợp này. Làm sao microservice lấy được khóa công khai? Điều gì xảy ra nếu khóa công khai cần thay đổi? Vault là một ví dụ về một công cụ có thể được dùng bởi một microservice để truy xuất (và xử lý việc xoay vòng) các khóa công khai, và nó vốn đã được thiết kế để hoạt động trong một môi trường phân tán cao. Tất nhiên bạn có thể chỉ hardcode một khóa

công khai trong một tệp cấu hình cho microservice nhận, nhưng khi đó bạn sẽ gặp vấn đề xử lý việc khóa công khai thay đổi.

Secondly, getting the expiration right for a token can be tricky if long processing times are involved. Consider a situation in which a customer has placed an order. This kicks off a set of **asynchronous** processes that could take hours if not days to complete, without any subsequent involvement from the customer (taking payment, sending notification emails, getting the item packaged and shipped, etc.). Do you therefore have to generate a token with a matching period of validity? The question here is, at what point is having a longer-lived token more problematic than having no token? I've spoken to a few teams that have dealt with this issue. Some have generated a special longer-lived token that is scoped to work in only this specific context; others have just stopped using the token at a certain point in the flow. I've not yet looked at enough examples of this problem to determine the right solution here, but it is an issue to be aware of.

Thứ hai, việc đặt thời gian hết hạn đúng cho một token có thể khó khăn nếu thời gian xử lý dài có liên quan. Hãy xét một tình huống trong đó một khách hàng đã đặt một đơn hàng. Điều này khởi động một tập các tiến trình bất đồng bộ có thể mất hàng giờ nếu không phải hàng ngày để hoàn thành, mà không có bất kỳ sự tham gia tiếp theo nào từ khách hàng (thu tiền thanh toán, gửi email thông báo, đóng gói và vận chuyển món hàng, v.v.). Vậy bạn có phải sinh một token với khoảng thời gian hợp lệ tương ứng không? Câu hỏi ở đây là, tại điểm nào thì việc có một token sống lâu hơn lại có vấn đề hơn so với việc không có token nào? Tôi đã trò chuyện với một vài nhóm từng đối phó với vấn đề này. Một số đã sinh ra một token sống lâu hơn đặc biệt vốn được giới hạn phạm vi để chỉ hoạt động trong bối cảnh cụ thể này; số khác chỉ đơn giản ngừng dùng token tại một điểm nào đó trong luồng. Tôi vẫn chưa xem đủ ví dụ về vấn đề này để xác định giải pháp đúng ở đây, nhưng đó là một vấn đề cần nhận thức được.

Finally, in some situations you can end up needing so much information in the JWT token that the token size itself becomes a problem. Although this situation is rare, it does happen. Several years ago I was chatting to one team about using a token to manage authorization for a particular aspect of its system that handled rights management for music. The logic around this was incredibly complex—my client worked out that for any given track it could potentially need up to 10,000 entries in a token to deal with the different scenarios. We realized, though, that at least in that domain, it was only one particular use case that needed this large amount of information, whereas the bulk of the system could make do with a simple token with fewer fields. In such a situation, it made sense to deal with the more complex rights management authorization process in a different way—essentially using the JWT token for the initial “simple” authorization, and then doing a subsequent lookup on a data store to fetch the additional fields as required. This meant the bulk of the system could just work off the tokens.

Cuối cùng, trong một số tình huống bạn có thể rất cuộc cần nhiều thông tin đến mức trong JWT token mà chính kích thước token trở thành một vấn đề. Mặc dù tình huống này hiếm, nó vẫn xảy ra. Vài năm trước tôi trò chuyện với một nhóm về việc dùng một token để quản lý phân quyền cho một khía cạnh cụ thể của hệ thống của họ vốn xử lý việc quản lý quyền (rights management) cho âm nhạc. Logic quanh điều này cực kỳ phức tạp — khách hàng của tôi tính ra rằng đối với bất kỳ track cho trước nào nó có thể tiềm tàng cần đến 10.000 mục trong một token để đối phó với các kịch bản khác nhau. Tuy nhiên, chúng tôi nhận ra rằng, chỉ ít trong miền đó, chỉ có một trường hợp sử dụng cụ thể cần lượng thông tin lớn này, trong khi phần lớn hệ thống có thể xoay xở với một token đơn giản với ít trường hơn. Trong một tình huống như vậy, sẽ hợp lý khi đối phó với quá trình phân quyền quản lý quyền phức tạp hơn theo một cách khác — về cơ bản dùng JWT token cho việc phân quyền “đơn giản” ban đầu, rồi sau đó thực hiện một lượt tra cứu tiếp theo trên một kho

dữ liệu để lấy các trường bổ sung khi cần. Điều này có nghĩa là phần lớn hệ thống có thể chỉ hoạt động dựa trên các token.

Summary — Tóm tắt

As I hope I've articulated in this chapter, building a secure system isn't about doing one thing. It necessitates a holistic view of your system, using some sort of threat modeling exercise, to understand what type of security controls need to be put in place.

Như tôi hy vọng mình đã diễn đạt được trong chương này, việc xây dựng một hệ thống an toàn không phải là việc làm một thứ duy nhất. Nó đòi hỏi một cái nhìn toàn diện về hệ thống của bạn, dùng một dạng buổi mô hình hóa mối đe dọa nào đó, để hiểu loại kiểm soát bảo mật nào cần được đặt ra.

When thinking of these controls, a mix is essential to building a secure system. Defense in depth doesn't just mean having multiple protections in place; it also means you have a multifaceted approach to building a more secure system.

Khi nghĩ về những kiểm soát này, một sự kết hợp là thiết yếu để xây dựng một hệ thống an toàn. Phòng thủ nhiều lớp không chỉ có nghĩa là có nhiều lớp bảo vệ tại chỗ; nó còn có nghĩa là bạn có một cách tiếp cận đa diện để xây dựng một hệ thống an toàn hơn.

We also return again to a core theme of the book: having a system decomposed into finer-grained services gives us many more options for solving problems. Not only can having microservices potentially reduce the impact of any given breach, but it also allows us to consider trade-offs between the overhead of more complex and secure approaches, where data is sensitive, and a more lightweight approach when the risks are lower.

Chúng ta cũng quay lại một lần nữa với một chủ đề cốt lõi của cuốn sách: việc có một hệ thống được phân rã thành các dịch vụ chi tiết hơn cho chúng ta nhiều lựa chọn hơn để giải quyết vấn đề. Việc có các microservice không chỉ có khả năng giảm tác động của bất kỳ vụ xâm phạm cho trước nào, mà nó còn cho phép chúng ta cân nhắc các đánh đổi giữa chi phí phụ trội của các cách tiếp cận phức tạp và an toàn hơn, nơi dữ liệu nhạy cảm, và một cách tiếp cận nhẹ nhàng hơn khi rủi ro thấp hơn.

For a broader look at application security in general, I recommend Agile Application Security by Laura Bell et al. 17

Để có một cái nhìn rộng hơn về bảo mật ứng dụng nói chung, tôi giới thiệu cuốn Agile Application Security của Laura Bell và cộng sự. 17

Next, we'll look at how we can make our systems more reliable, as we turn to the topic of resilience.

Tiếp theo, chúng ta sẽ xem xét cách chúng ta có thể khiến hệ thống của mình đáng tin cậy hơn, khi chúng ta chuyển sang chủ đề về khả năng phục hồi (resilience).

17 Laura Bell et al., Agile Application Security (Sebastopol: O'Reilly, 2017).

17 Laura Bell et al., Agile Application Security (Sebastopol: O'Reilly, 2017).

CHAPTER 12 Resiliency — Chương 12: Khả năng phục hồi (Resiliency)

As software becomes an increasingly vital part of our users' lives, we need to continually improve the quality of **service** we offer. Software failing can have a significant impact on people's lives, even if the software doesn't fall into the category of "safety critical" in the way that things like aircraft control systems do. During the COVID-19 pandemic, which was ongoing at the time of writing, services like online grocery shopping went from being a convenience to becoming a necessity for many people who were unable to leave their homes.

Khi phần mềm ngày càng trở thành một phần thiết yếu trong cuộc sống của người dùng, chúng ta cần liên tục nâng cao chất lượng dịch vụ mà mình cung cấp. Phần mềm gặp sự cố có thể tác động đáng kể đến cuộc sống của con người, ngay cả khi phần mềm đó không thuộc loại "an toàn tối quan trọng" (safety critical) như các hệ thống điều khiển máy bay. Trong đại dịch COVID-19, vốn đang diễn ra vào thời điểm viết cuốn sách này, các dịch vụ như mua sắm tạp hóa trực tuyến đã chuyển từ chỗ là một tiện ích thành một nhu cầu thiết yếu đối với nhiều người không thể rời khỏi nhà.

Against this backdrop, we are often being tasked with creating software that is more and more reliable. The expectations of our users have shifted in terms of what the software can do and when it should be available. The days of only having to support software during office hours are increasingly rare, and there is decreasing tolerance for downtime due to maintenance.

Trong bối cảnh đó, chúng ta thường được giao nhiệm vụ tạo ra phần mềm ngày càng đáng tin cậy hơn. Kỳ vọng của người dùng đã thay đổi về những gì phần mềm có thể làm và thời điểm nó phải sẵn sàng. Những ngày chỉ phải hỗ trợ phần mềm trong giờ hành chính ngày càng hiếm hoi, và mức độ chấp nhận thời gian ngừng hoạt động do bảo trì cũng ngày càng giảm.

As we covered at the start of this book, there is a host of reasons as to why **microservice** architectures are being chosen by organizations around the world. But for many, the prospects of improving the resilience of their service offerings is cited as a major reason.

Như chúng ta đã đề cập ở đầu cuốn sách, có rất nhiều lý do khiến các kiến trúc microservice được những tổ chức trên khắp thế giới lựa chọn. Nhưng với nhiều tổ chức, triển vọng cải thiện khả năng phục hồi (resiliency) của các dịch vụ họ cung cấp được nêu ra như một lý do quan trọng.

Before we get into the details of how a microservice **architecture** can enable **resiliency**, it's important to step back and consider what resiliency actually is. It turns out that when it comes to improving the resiliency of our software, adopting a microservice architecture is only part of the puzzle.

Trước khi đi vào chi tiết về cách một kiến trúc microservice có thể hỗ trợ khả năng phục hồi, điều quan trọng là phải lùi lại một bước và xem xét khả năng phục hồi thực chất là gì. Hóa ra khi nói đến việc cải thiện khả năng phục hồi của phần mềm, việc áp dụng kiến trúc microservice chỉ là một phần của bài toán.

What Is Resiliency? — Khả năng phục hồi là gì?

We use the term resiliency in many different contexts and in many different ways. This can lead to confusion about what the term means and can also result in us thinking too narrowly about the field. Outside the confines of IT, there is a broader area of resilience engineering, which looks at the concept of resiliency as it applies to a host of systems—from firefighting to air traffic control, biological systems, and operating theaters. Drawing on this field, David D. Woods has attempted to categorize the different aspects of resilience to help us think more widely about what resiliency actually means. 1 These four concepts are:

Chúng ta sử dụng thuật ngữ khả năng phục hồi trong rất nhiều ngữ cảnh và theo nhiều cách khác nhau. Điều này có thể dẫn đến sự nhầm lẫn về ý nghĩa của thuật ngữ và cũng có thể khiến chúng ta suy nghĩ quá hạn hẹp về lĩnh vực này. Bên ngoài phạm vi của IT, có một lĩnh vực rộng hơn gọi là kỹ thuật phục hồi (resilience engineering), xem xét khái niệm khả năng phục hồi khi nó được áp dụng cho hàng loạt hệ thống—from cứu hỏa đến kiểm soát không lưu, các hệ thống sinh học, và phòng mổ. Dựa trên lĩnh vực này, David D. Woods đã cố gắng phân loại các khía cạnh khác nhau của khả năng phục hồi để giúp chúng ta suy nghĩ rộng hơn về ý nghĩa thực sự của khả năng phục hồi. 1 Bốn khái niệm này là:

Robustness The ability to absorb expected perturbation

Độ vững chắc (Robustness) Khả năng hấp thụ những nhiễu loạn được dự kiến

Rebound The ability to recover after a traumatic event

Bật lại (Rebound) Khả năng phục hồi sau một sự kiện gây tổn thương

Graceful extensibility How well we deal with a situation that is unexpected

Khả năng mở rộng có kiểm soát (Graceful extensibility) Mức độ chúng ta xử lý tốt một tình huống không lường trước

Sustained adaptability The ability to continually adapt to changing environments, stakeholders, and demands

Khả năng thích ứng bền vững (Sustained adaptability) Khả năng liên tục thích ứng với những môi trường, các bên liên quan, và nhu cầu đang thay đổi

Let's look at each of these concepts in turn and examine how these ideas might (or might not) translate into our world of building microservice architectures.

Hãy lần lượt xem xét từng khái niệm này và khảo sát xem những ý tưởng đó có thể (hay không thể) chuyển dịch vào thế giới xây dựng các kiến trúc microservice của chúng ta như thế nào.

Robustness — Độ vững chắc (Robustness)

Robustness is the concept whereby we **build** mechanisms into our software and processes to accommodate expected problems. We have an advanced understanding of the kinds of perturbations we might face, and we put measures in place so that when these issues arise, our system can deal with them. In the context of our microservice architecture, we have a whole host of perturbations that we might expect: a host can fail, a network connection can time out, a microservice might be unavailable. We can improve the robustness of our architecture in a number of ways to deal with these perturbations, such as automatically spinning up a replacement host, performing retries, or handling failure of a given microservice in a graceful manner.

Độ vững chắc (robustness) là khái niệm theo đó chúng ta xây dựng các cơ chế vào phần mềm và quy trình của mình để thích ứng với những vấn đề được dự kiến. Chúng ta có hiểu biết sâu sắc về những loại nhiễu loạn có thể gặp phải, và chúng ta đặt ra các biện pháp để khi những vấn đề này phát sinh, hệ thống của chúng ta có thể xử lý chúng. Trong ngữ cảnh kiến trúc microservice của chúng ta, có cả một loạt nhiễu loạn mà chúng ta có thể dự kiến: một host có thể hỏng, một kết nối mạng có thể bị time-out, một microservice có thể không sẵn sàng. Chúng ta có thể cải thiện độ vững chắc của kiến trúc theo nhiều cách để xử lý những nhiễu loạn này, chẳng hạn như tự động khởi tạo một host thay thế, thực hiện retry, hoặc xử lý sự cố của một microservice nhất định theo cách có kiểm soát.

Robustness goes beyond software, though. It can apply to people. If you have a single person on call for your software, what happens if that person gets sick or isn't reachable at the time of an incident? This is a fairly easy thing to consider, and the solution might be to have a backup on-call person.

Tuy nhiên, độ vững chắc không chỉ giới hạn ở phần mềm. Nó cũng có thể áp dụng cho con người. Nếu bạn chỉ có một người duy nhất trực (on call) cho phần mềm của mình, điều gì xảy ra nếu người đó bị ốm hoặc không liên lạc được vào thời điểm xảy ra sự cố? Đây là điều khá dễ cân nhắc, và giải pháp có thể là có một người trực dự phòng.

Robustness by definition requires prior knowledge—we are putting measures into place to deal with known perturbations. This knowledge could be based on foresight: we could draw on our understandings of the computer system we are building, its

Theo định nghĩa, độ vững chắc đòi hỏi kiến thức có trước—chúng ta đang đặt ra các biện pháp để xử lý những nhiễu loạn đã biết. Kiến thức này có thể dựa trên sự tiên liệu: chúng ta có thể vận dụng hiểu biết của mình về hệ thống máy tính đang xây dựng, các

1 David D. Woods, “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering,” *Reliability Engineering & System Safety* 141 (September 2015): 5–9, doi.org/10.1016/j.ress.2015.03.018.

1 David D. Woods, “Four Concepts for Resilience and the Implications for the Future of Resilience Engineering,” *Reliability Engineering & System Safety* 141 (September 2015): 5–9, doi.org/10.1016/j.ress.2015.03.018.

supporting services, and our people to consider what might go wrong. But robustness can also come from hindsight—we may improve the robustness of our system after something we didn't expect happens. Perhaps we never considered the fact that our global filesystem could become unavailable, or perhaps we underestimated the impact of our customer service representatives not being available outside working hours.

dịch vụ hỗ trợ của nó, và con người của chúng ta để cân nhắc điều gì có thể xảy ra. Nhưng độ vững chắc cũng có thể đến từ sự hồi cố—chúng ta có thể cải thiện độ vững chắc của hệ thống sau khi một điều ngoài dự kiến xảy ra. Có lẽ chúng ta chưa bao giờ tính đến việc hệ thống tập tin toàn cục của mình có thể trở nên không sẵn sàng, hoặc có lẽ chúng ta đã

đánh giá thấp tác động của việc các đại diện chăm sóc khách hàng không có mặt ngoài giờ làm việc.

One of the challenges around improving the robustness of our system is that as we increase the robustness of our application, we introduce more complexity to our system, which can be the source of new issues. Let's say you're moving your microservice architecture to **Kubernetes**, because you want it to handle desired state management for your microservice workloads. You may have improved some aspects of the robustness of your application as a result, but you've also introduced new potential pain points as well. As such, any attempt to improve the robustness of an application has to be considered, not just in terms of a simple cost/benefit analysis but also in terms of whether or not you're happy with the more complex system you'll have as a result of this.

Một trong những thách thức xoay quanh việc cải thiện độ vững chắc của hệ thống là khi chúng ta tăng độ vững chắc của ứng dụng, chúng ta lại đưa thêm độ phức tạp vào hệ thống, và điều này có thể trở thành nguồn gốc của các vấn đề mới. Giả sử bạn đang chuyển kiến trúc microservice của mình sang Kubernetes, vì bạn muốn nó xử lý việc quản lý trạng thái mong muốn (desired state management) cho các workload microservice của mình. Kết quả là bạn có thể đã cải thiện được một số khía cạnh về độ vững chắc của ứng dụng, nhưng bạn cũng đã đưa vào những điểm gây đau đầu tiềm tàng mới. Như vậy, bất kỳ nỗ lực nào nhằm cải thiện độ vững chắc của một ứng dụng cũng phải được cân nhắc, không chỉ về mặt phân tích chi phí/lợi ích đơn thuần mà còn về việc liệu bạn có hài lòng với hệ thống phức tạp hơn mà bạn sẽ có được do điều này hay không.

Robustness is one area in which **microservices** give you a host of options, and much of what follows in this chapter will focus on what you can do in your software to improve the system's robustness. Just remember that not only is this only one facet of resiliency as a whole, but there is also a host of other nonsoftware-related robustness you might need to consider.

Độ vững chắc là một lĩnh vực mà microservice mang lại cho bạn cả loạt lựa chọn, và phần lớn nội dung tiếp theo trong chương này sẽ tập trung vào những gì bạn có thể làm trong phần mềm của mình để cải thiện độ vững chắc của hệ thống. Chỉ cần nhớ rằng đây không chỉ là một mặt của khả năng phục hồi nói chung, mà còn có cả loạt khía cạnh độ vững chắc khác không liên quan đến phần mềm mà bạn có thể cần cân nhắc.

Rebound — Bật lại (Rebound)

How well we recover—rebound—from disruption is a key part of building a resilient system. All too often I see people focusing their time and energy on trying to eliminate the possibility of an outage, only to be totally unprepared once an outage actually occurs. By all means, do your best to protect against the bad things that you think might happen—improving your system's robustness—but also understand that as your system grows in scale and complexity, eliminating any potential problem becomes unsustainable.

Khả năng phục hồi—bật lại (rebound)—tốt đến đâu sau gián đoạn là một phần then chốt trong việc xây dựng một hệ thống có khả năng phục hồi. Tôi thường thấy mọi người dành quá nhiều thời gian và sức lực để cố loại bỏ khả năng xảy ra sự cố ngừng hoạt động, để rồi hoàn toàn không có sự chuẩn bị một khi sự cố thực sự xảy ra. Bằng mọi cách, hãy cố gắng hết sức để bảo vệ trước những điều tồi tệ mà bạn nghĩ có thể xảy ra—cải thiện độ vững chắc của hệ thống—nhưng cũng hãy hiểu rằng khi hệ thống của bạn tăng về quy mô và độ phức tạp, việc loại bỏ mọi vấn đề tiềm ẩn trở nên không bền vững.

We can improve our ability to rebound from an incident by putting things into place in advance.

For example, having backups in place can allow us to better rebound in the aftermath of data loss (assuming our backups are tested, of course!). Improving our ability to rebound could also include having a playbook we can run through in the wake of a system outage: Do people understand what their role is when an outage occurs? Who will be the point person for handling the situation? How quickly do we need to let our users know what is happening? How will we communicate with our users? Trying to think clearly about how to handle an outage while the outage is going on will be problematic due to the inherent stress and chaos of the situation.

Chúng ta có thể cải thiện khả năng bật lại sau một sự cố bằng cách chuẩn bị sẵn các thứ từ trước. Ví dụ, có sẵn các bản sao lưu (backup) có thể giúp chúng ta bật lại tốt hơn sau khi mất dữ liệu (tất nhiên là với giả định các bản sao lưu của chúng ta đã được kiểm thử!). Cải thiện khả năng bật lại cũng có thể bao gồm việc có một cẩm nang (playbook) mà chúng ta có thể làm theo sau khi hệ thống ngừng hoạt động: Mọi người có hiểu vai trò của mình khi xảy ra sự cố ngừng hoạt động không? Ai sẽ là người chịu trách nhiệm chính để xử lý tình huống? Chúng ta cần thông báo cho người dùng biết chuyện gì đang xảy ra nhanh đến mức nào? Chúng ta sẽ giao tiếp với người dùng ra sao? Cố gắng suy nghĩ rõ ràng về cách xử lý một sự cố ngừng hoạt động trong khi sự cố đang diễn ra sẽ rất rắc rối do sự căng thẳng và hỗn loạn vốn có của tình huống.

Having an agreed plan of action in place in anticipation of this sort of problem can help you better rebound.

Có sẵn một kế hoạch hành động đã được thống nhất để lường trước loại vấn đề này có thể giúp bạn bật lại tốt hơn.

Graceful Extensibility — Khả năng mở rộng có kiểm soát (Graceful Extensibility)

With rebound and robustness, we are primarily dealing with the expected. We are putting mechanisms in place to deal with problems that we can foresee. But what happens when we are surprised? If we aren't prepared for surprise—for the fact that our expected view of the world might be wrong—we end up with a brittle system. As we approach the limits of what we expect our system to be able to handle, things fall apart—we are unable to perform adequately.

Với bật lại và độ vững chắc, chúng ta chủ yếu xử lý những điều đã được dự kiến. Chúng ta đặt ra các cơ chế để xử lý những vấn đề mà chúng ta có thể lường trước. Nhưng điều gì xảy ra khi chúng ta bị bất ngờ? Nếu chúng ta không chuẩn bị cho sự bất ngờ—cho thực tế là quan điểm dự kiến của chúng ta về thế giới có thể sai—chúng ta sẽ kết thúc với một hệ thống dễ vỡ. Khi chúng ta tiến đến giới hạn của những gì chúng ta kỳ vọng hệ thống có thể xử lý, mọi thứ sụp đổ—chúng ta không thể hoạt động đầy đủ được.

Flatter organizations—where responsibility is distributed into the organization rather than held centrally—will often be better prepared to deal with surprise. When the unexpected occurs, if people are restricted in what they have to do, if they have to adhere to a strict set of rules, their ability to deal with surprise will be critically curtailed.

Các tổ chức phẳng hơn—nơi trách nhiệm được phân tán vào trong tổ chức thay vì giữ tập trung—thường sẽ được chuẩn bị tốt hơn để xử lý sự bất ngờ. Khi điều bất ngờ xảy ra, nếu con người bị hạn chế trong những việc họ phải làm, nếu họ phải tuân theo một bộ quy tắc cứng nhắc, thì khả năng xử lý sự bất ngờ của họ sẽ bị thu hẹp một cách nghiêm trọng.

Often, in a drive to optimize our system, we can as an unfortunate side effect increase the brittleness of our system. Take automation as an example. Automation is fantastic—it allows us to do more with

the people we have, but it can also allow us to reduce the people we have, as more can be done with automation. This reduction in staff can be concerning, though. Automation can't handle surprise—our ability to gracefully extend our system, to handle surprise, comes from having people in place with the right skills, experience, and responsibility to handle these situations as they arise.

Thường thì, trong nỗ lực tối ưu hóa hệ thống, một tác dụng phụ đáng tiếc là chúng ta lại làm tăng tính dễ vỡ của hệ thống. Lấy tự động hóa làm ví dụ. Tự động hóa thật tuyệt vời—nó cho phép chúng ta làm được nhiều hơn với số người mình đang có, nhưng nó cũng có thể cho phép chúng ta giảm số người mình có, vì có thể làm được nhiều hơn nhờ tự động hóa. Tuy nhiên, việc cắt giảm nhân sự này có thể gây lo ngại. Tự động hóa không thể xử lý sự bất ngờ—khả năng mở rộng hệ thống một cách có kiểm soát của chúng ta, khả năng xử lý sự bất ngờ, đến từ việc có sẵn những con người với kỹ năng, kinh nghiệm, và trách nhiệm phù hợp để xử lý những tình huống này khi chúng phát sinh.

Sustained Adaptability — Khả năng thích ứng bền vững (Sustained Adaptability)

Having sustained adaptability requires us to not be complacent. As David Woods puts it: “No matter how good we have done before, no matter how successful we’ve been, the future could be different, and we might not be well adapted. We might be precarious and fragile in the face of that new future.”² That we haven’t yet suffered from a catastrophic outage doesn’t mean that it cannot happen. We need to challenge ourselves to make sure we are constantly adapting what we do as an organization to ensure future resiliency. Done right, a concept like **chaos engineering**—which we’ll explore briefly later in this chapter—can be a useful tool in helping build sustained adaptability.

Để có khả năng thích ứng bền vững, chúng ta không được tự mãn. Như David Woods diễn đạt: “Cho dù chúng ta đã làm tốt đến đâu trước đây, cho dù chúng ta đã thành công đến mức nào, tương lai vẫn có thể khác đi, và chúng ta có thể chưa thích ứng tốt. Chúng ta có thể bất bênh và mong manh trước cái tương lai mới đó.”² Việc chúng ta chưa từng hứng chịu một sự cố ngừng hoạt động thảm khốc không có nghĩa là nó không thể xảy ra. Chúng ta cần thách thức chính mình để đảm bảo rằng chúng ta liên tục điều chỉnh những gì mình làm với tư cách một tổ chức nhằm đảm bảo khả năng phục hồi trong tương lai. Nếu làm đúng, một khái niệm như chaos engineering—mà chúng ta sẽ khám phá ngắn gọn ở phần sau của chương này—có thể là một công cụ hữu ích trong việc giúp xây dựng khả năng thích ứng bền vững.

Sustained adaptability often requires a more holistic view of the system. This is, paradoxically, where a drive toward smaller, autonomous teams with increased local, focused responsibility can end with us losing sight of the bigger picture. As we’ll

Khả năng thích ứng bền vững thường đòi hỏi một cái nhìn toàn diện hơn về hệ thống. Nghịch lý thay, đây lại chính là nơi mà xu hướng hướng tới các nhóm nhỏ hơn, tự chủ với trách nhiệm cục bộ, tập trung được tăng cường có thể khiến chúng ta đánh mất tầm nhìn bao quát. Như chúng ta sẽ

² Jason Bloomberg, “Innovation: The Flip Side of Resilience,” *Forbes*, September 23, 2014, <https://oreil.ly/avSmU>.

² Jason Bloomberg, “Innovation: The Flip Side of Resilience,” *Forbes*, September 23, 2014, <https://oreil.ly/avSmU>.

explore in Chapter 15, there is a balancing act between global and local optimization when it comes to organizational dynamics, and that balance isn’t static. In that chapter we’ll look at the

role of focused, stream-aligned teams who own the microservices they need to deliver user-facing functionality and have increased levels of responsibility to make this happen. We'll also look at the role of enabling teams, who support these stream-aligned teams in doing their work, and at how enabling teams can be a big part of helping achieve sustained adaptability at an organizational level.

khám phá ở Chương 15, có một sự cân bằng giữa tối ưu hóa toàn cục và cục bộ khi nói đến động lực của tổ chức, và sự cân bằng đó không tĩnh tại. Trong chương đó, chúng ta sẽ xem xét vai trò của các nhóm stream-aligned tập trung, những nhóm sở hữu các microservice cần thiết để cung cấp chức năng hướng tới người dùng và có mức độ trách nhiệm cao hơn để biến điều này thành hiện thực. Chúng ta cũng sẽ xem xét vai trò của các nhóm enabling, những nhóm hỗ trợ các nhóm stream-aligned này hoàn thành công việc của họ, và xem các nhóm enabling có thể là một phần lớn như thế nào trong việc giúp đạt được khả năng thích ứng bền vững ở cấp độ tổ chức.

Creating a culture that prioritizes creating an environment in which people can share information freely, without fear of retribution, is vital to encourage learning in the wake of an incident. Having the bandwidth to really examine such surprises and extract the key learnings requires time, energy, and people—all things that will reduce the resources available to you to deliver features in the short term. Deciding to embrace sustained adaptability is partly about finding the balancing point between short-term delivery and longer-term adaptability.

Tạo ra một văn hóa ưu tiên việc kiến tạo một môi trường mà mọi người có thể chia sẻ thông tin một cách tự do, không sợ bị trừng phạt, là điều thiết yếu để khuyến khích việc học hỏi sau một sự cố. Có đủ dư địa để thực sự xem xét những bất ngờ như vậy và rút ra những bài học then chốt đòi hỏi thời gian, sức lực, và con người—tất cả những thứ sẽ làm giảm nguồn lực bạn có để cung cấp tính năng trong ngắn hạn. Việc quyết định đón nhận khả năng thích ứng bền vững một phần là việc tìm ra điểm cân bằng giữa việc giao hàng trong ngắn hạn và khả năng thích ứng trong dài hạn.

To work toward sustained adaptability means that you are looking to discover what you don't know. This requires continuing investment, not one-off transactional activities—the term sustained is important here. It's about making sustained adaptability a core part of your organizational strategy and culture.

Hướng tới khả năng thích ứng bền vững có nghĩa là bạn đang tìm cách khám phá những gì mình chưa biết. Điều này đòi hỏi đầu tư liên tục, chứ không phải các hoạt động giao dịch một lần—from “bền vững” ở đây rất quan trọng. Đó là việc biến khả năng thích ứng bền vững thành một phần cốt lõi trong chiến lược và văn hóa của tổ chức bạn.

And Microservice Architecture — Và kiến trúc microservice

As we've discussed, we can see a way in which a microservice architecture can help us achieve the property of robustness, but it is not enough if you want resiliency .

Như chúng ta đã thảo luận, chúng ta có thể thấy một cách mà một kiến trúc microservice có thể giúp chúng ta đạt được thuộc tính độ vững chắc, nhưng như vậy là chưa đủ nếu bạn muốn có khả năng phục hồi.

Taken more broadly, the ability to deliver resiliency is a property not of the software itself but of the people building and running the system. Given the focus of this book, much of what follows in this chapter will focus primarily on what a microservice architecture can help deliver in terms of resiliency—which is almost entirely limited to improving the robustness of applications.

Xét rộng hơn, khả năng đem lại sự phục hồi không phải là thuộc tính của bản thân phần mềm mà là của những con người xây dựng và vận hành hệ thống. Với trọng tâm của cuốn sách này, phần lớn nội dung tiếp theo trong chương này sẽ chủ yếu tập trung vào những gì một kiến trúc microservice có thể giúp đem lại về mặt khả năng phục hồi—điều mà hầu như hoàn toàn giới hạn ở việc cải thiện độ vững chắc của các ứng dụng.

Failure Is Everywhere — Sự cố ở khắp mọi nơi

We understand that things can go wrong. Hard disks can fail. Our software can crash. And as anyone who has read the fallacies of distributed computing can tell you, the network is unreliable. We can do our best to try to limit the causes of failure, but at a certain scale, failure becomes inevitable. Hard drives, for example, are more reliable now than ever before, but they'll break eventually. The more hard drives you have, the higher the likelihood of failure for an individual unit on any given day; failure becomes a statistical certainty at scale.

Chúng ta hiểu rằng mọi thứ có thể trục trặc. Ổ cứng có thể hỏng. Phần mềm của chúng ta có thể sập. Và như bất kỳ ai đã đọc về các ngộ nhận của tính toán phân tán (fallacies of distributed computing) đều có thể nói cho bạn, mạng là thứ không đáng tin cậy. Chúng ta có thể cố gắng hết sức để hạn chế các nguyên nhân gây sự cố, nhưng ở một quy mô nhất định, sự cố trở nên không thể tránh khỏi. Ổ cứng, chẳng hạn, giờ đây đáng tin cậy hơn bao giờ hết, nhưng cuối cùng chúng vẫn sẽ hỏng. Càng có nhiều ổ cứng, khả năng hỏng của một đơn vị riêng lẻ trong bất kỳ ngày nào càng cao; ở quy mô lớn, sự cố trở thành một điều chắc chắn về mặt thống kê.

Even for those of us not thinking at extreme scale, if we can embrace the possibility of failure we will be better off. For example, if we can handle the failure of a microservice gracefully, then it follows that we can also do in-place upgrades of a service, as a planned outage is much easier to deal with than an unplanned one.

Ngay cả với những người trong chúng ta không nghĩ đến quy mô cực lớn, nếu chúng ta có thể đón nhận khả năng xảy ra sự cố thì chúng ta sẽ ở thế tốt hơn. Ví dụ, nếu chúng ta có thể xử lý sự cố của một microservice một cách có kiểm soát, thì suy ra chúng ta cũng có thể nâng cấp tại chỗ một dịch vụ, vì một sự cố ngừng hoạt động có kế hoạch dễ xử lý hơn nhiều so với một sự cố ngoài kế hoạch.

We can also spend a bit less of our time trying to stop the inevitable and a bit more of our time dealing with it gracefully. I'm amazed at how many organizations put processes and controls in place to try to stop failure from occurring but put little to no thought into actually making it easier to recover from failure in the first place. Understanding the things that are likely to fail is key to improving the robustness of our system.

Chúng ta cũng có thể dành ít thời gian hơn để cố ngăn chặn điều không thể tránh khỏi và dành nhiều thời gian hơn để xử lý nó một cách có kiểm soát. Tôi ngạc nhiên trước việc có biết bao tổ chức đặt ra các quy trình và biện pháp kiểm soát để cố ngăn không cho sự cố xảy ra nhưng lại hầu như không suy nghĩ gì đến việc thực sự làm cho việc phục hồi sau sự cố trở nên dễ dàng hơn ngay từ đầu. Hiểu được những thứ có khả năng gây sự cố là chìa khóa để cải thiện độ vững chắc của hệ thống.

Baking in the assumption that everything can and will fail leads you to think differently about how you solve problems. Remember the story of the Google servers we discussed in Chapter 10 ? The Google systems were built in such a way that if a machine failed, it would not lead to interruption of service—improving the robustness of the system as a whole. Google goes further in attempting

to improve the robustness of its servers in other ways—it has discussed how each server contains its own local power supply to ensure it can keep operating if the data center has an outage. ³ As you'll recall from Chapter 10, the hard drives in these servers were attached with Velcro rather than screws to make it easy to replace drives—helping Google get the machine up and running quickly when a drive failed, and in turn helping that component of the system rebound more effectively.

Đưa giả định rằng mọi thứ đều có thể và sẽ gặp sự cố vào trong tư duy sẽ khiến bạn nghĩ khác đi về cách giải quyết vấn đề. Còn nhớ câu chuyện về các máy chủ của Google mà chúng ta đã thảo luận ở Chương 10 không? Các hệ thống của Google được xây dựng theo cách mà nếu một máy bị hỏng, nó sẽ không dẫn đến gián đoạn dịch vụ—cải thiện độ vững chắc của toàn hệ thống. Google còn đi xa hơn trong việc cố gắng cải thiện độ vững chắc của các máy chủ theo những cách khác—họ đã bàn về việc mỗi máy chủ chứa nguồn điện cục bộ riêng để đảm bảo nó có thể tiếp tục hoạt động nếu trung tâm dữ liệu mất điện. ³ Như bạn sẽ nhớ từ Chương 10, các ổ cứng trong những máy chủ này được gắn bằng dây Velcro thay vì bắt vít để dễ thay ổ cứng—giúp Google đưa máy hoạt động trở lại nhanh chóng khi một ổ cứng hỏng, và đến lượt nó giúp thành phần đó của hệ thống bật lại hiệu quả hơn.

So let me repeat: at scale, even if you buy the best kit, the most expensive hardware, you cannot avoid the fact that things can and will fail. Therefore, you need to assume failure can happen. If you build this thinking into everything you do and plan for failure, you can make informed trade-offs. If you know your system can handle the fact that a server can and will fail, there may be diminishing returns from spending more and more money on individual machines. Instead, having a larger number of cheaper machines (perhaps using cheaper components and some Velcro!) like Google did may make a lot more sense.

Vậy nên hãy để tôi nhắc lại: ở quy mô lớn, ngay cả khi bạn mua bộ thiết bị tốt nhất, phần cứng đắt tiền nhất, bạn cũng không thể tránh khỏi thực tế rằng mọi thứ có thể và sẽ gặp sự cố. Do đó, bạn cần giả định rằng sự cố có thể xảy ra. Nếu bạn đưa tư duy này vào mọi việc mình làm và lên kế hoạch cho sự cố, bạn có thể đưa ra những đánh đổi sáng suốt. Nếu bạn biết hệ thống của mình có thể xử lý được thực tế rằng một máy chủ có thể và sẽ hỏng, thì việc chi ngày càng nhiều tiền hơn cho từng cỗ máy riêng lẻ có thể mang lại lợi ích giảm dần. Thay vào đó, việc có một số lượng lớn hơn các cỗ máy rẻ hơn (có lẽ dùng linh kiện rẻ hơn và một ít dây Velcro!) như Google đã làm có thể hợp lý hơn nhiều.

How Much Is Too Much? — Bao nhiêu là quá nhiều?

We touched on the topic of cross-functional requirements in Chapter 9. Understanding cross-functional requirements is all about considering aspects like durability of data, availability of services, throughput, and acceptable latency of operations. Many of the techniques covered in this chapter talk about approaches to implement these

Chúng ta đã chạm đến chủ đề về các yêu cầu liên-chức-năng (cross-functional requirements) ở Chương 9. Hiểu các yêu cầu liên chức năng là toàn bộ việc cân nhắc các khía cạnh như độ bền của dữ liệu, tính sẵn sàng của dịch vụ, thông lượng, và độ trễ chấp nhận được của các thao tác. Nhiều kỹ thuật được đề cập trong chương này bàn về các cách tiếp cận để hiện thực hóa những

³ See “Google Uncloaks Once-Secret Server” by Stephen Shankland for more information on this, including an interesting overview of why Google thinks this approach can be superior to traditional UPS systems.

3 Xem “Google Uncloaks Once-Secret Server” của Stephen Shankland để biết thêm thông tin về điều này, bao gồm cả một cái nhìn tổng quan thú vị về lý do Google cho rằng cách tiếp cận này có thể vượt trội hơn các hệ thống UPS truyền thống.

requirements, but only you know exactly what the requirements themselves might be. So keep your own requirements in mind as you read on.

yêu cầu này, nhưng chỉ bạn mới biết chính xác bản thân các yêu cầu đó có thể là gì. Vậy nên hãy ghi nhớ các yêu cầu của riêng bạn khi đọc tiếp.

Having an **autoscaling** system capable of reacting to increased load or the failure of individual nodes might be fantastic, but it could be overkill for a reporting system that needs to run only twice a month, where being down for a day or two isn't that big of a deal. Likewise, figuring out how to do zero-downtime deployments to eliminate interruption of service might make sense for your online ecommerce system, but for your corporate intranet knowledge base, it's probably a step too far.

Có một hệ thống autoscaling đủ khả năng phản ứng với tải tăng lên hoặc với sự cố của các node riêng lẻ có thể rất tuyệt vời, nhưng nó có thể là thừa thãi đối với một hệ thống báo cáo chỉ cần chạy hai lần một tháng, nơi việc ngừng hoạt động một hai ngày cũng chẳng phải vấn đề lớn. Tương tự, việc tìm ra cách triển khai không-thời-gian-chết (zero-downtime) để loại bỏ gián đoạn dịch vụ có thể hợp lý cho hệ thống thương mại điện tử trực tuyến của bạn, nhưng đối với kho kiến thức nội bộ của công ty thì có lẽ đó là một bước đi quá xa.

How much failure you can tolerate or how fast your system needs to be is driven by the users of your system. That information in turn helps you understand which techniques will make the most sense for you. That said, your users won't always be able to articulate what their exact requirements are. So you need to ask questions to help extract the right information and help them understand the relative costs of providing different levels of service.

Bạn có thể chịu đựng được bao nhiêu sự cố, hay hệ thống của bạn cần nhanh đến mức nào, được quyết định bởi người dùng của hệ thống. Thông tin đó đến lượt nó giúp bạn hiểu kỹ thuật nào sẽ hợp lý nhất với bạn. Dẫu vậy, người dùng của bạn sẽ không phải lúc nào cũng có thể diễn đạt rõ chính xác yêu cầu của họ là gì. Vì thế bạn cần đặt câu hỏi để giúp khai thác đúng thông tin và giúp họ hiểu chi phí tương đối của việc cung cấp các mức dịch vụ khác nhau.

As I mentioned previously, these cross-functional requirements can vary from service to service, but I would suggest defining some general cross-functionals and then overriding them for particular use cases. When it comes to considering if and how to scale out your system to better handle load or failure, start by trying to understand the following requirements:

Như tôi đã đề cập trước đó, các yêu cầu liên chức năng này có thể khác nhau giữa các dịch vụ, nhưng tôi sẽ gợi ý định nghĩa một số yêu cầu liên chức năng chung rồi ghi đề chúng cho những trường hợp sử dụng cụ thể. Khi xét đến việc có nên và làm thế nào để mở rộng hệ thống nhằm xử lý tốt hơn tải hoặc sự cố, hãy bắt đầu bằng cách cố hiểu những yêu cầu sau:

Response time/latency How long should various operations take? It can be useful to measure this with different numbers of users to understand how increasing load will impact the response time. Given the nature of networks, you'll always have outliers, so setting targets for a given percentile of the responses monitored can be useful. The target should also include the number of concurrent connections/users you will expect your software to handle. So you might say, “We expect the website to have a 90th-percentile response time of 2 seconds when handling 200 concurrent connections per second.”

Thời gian phản hồi/độ trễ (Response time/latency) Các thao tác khác nhau nên mất bao lâu? Có thể hữu ích khi đo lường điều này với các số lượng người dùng khác nhau để hiểu việc tăng tải sẽ tác động thế nào đến thời gian phản hồi. Với bản chất của mạng, bạn sẽ luôn có những giá trị ngoại lai, vì vậy việc đặt mục tiêu cho một bách phân vị nhất định của các phản hồi được giám sát có thể hữu ích. Mục tiêu cũng nên bao gồm số lượng kết nối/người dùng đồng thời mà bạn kỳ vọng phần mềm của mình xử lý. Vậy nên bạn có thể nói: “Chúng tôi kỳ vọng trang web có thời gian phản hồi ở bách phân vị thứ 90 là 2 giây khi xử lý 200 kết nối đồng thời mỗi giây.”

Availability Can you expect a service to be down? Is this considered a 24/7 service? Some people like to look at periods of acceptable downtime when measuring availability, but how useful is this to someone calling your service? Either I should be able to rely on your service responding or I shouldn't. Measuring periods of downtime is really more useful from a historical reporting angle.

Tính sẵn sàng (Availability) Bạn có thể chấp nhận một dịch vụ bị ngừng hoạt động không? Đây có được xem là dịch vụ 24/7 không? Một số người thích nhìn vào các khoảng thời gian ngừng hoạt động chấp nhận được khi đo tính sẵn sàng, nhưng điều này hữu ích đến đâu đối với người gọi dịch vụ của bạn? Hoặc là tôi có thể tin cậy rằng dịch vụ của bạn sẽ phản hồi, hoặc là tôi không thể. Việc đo các khoảng thời gian ngừng hoạt động thực ra hữu ích hơn từ góc độ báo cáo lịch sử.

Durability of data How much data loss is acceptable? How long should data be kept for? This is highly likely to change on a case-by-case basis. For example, you might choose to keep user session logs for a year or less to save space, but your financial **transaction** records might need to be kept for many years.

Độ bền của dữ liệu (Durability of data) Mất bao nhiêu dữ liệu là chấp nhận được? Dữ liệu nên được lưu giữ trong bao lâu? Điều này rất có khả năng thay đổi tùy từng trường hợp. Ví dụ, bạn có thể chọn giữ nhật ký phiên người dùng trong một năm hoặc ít hơn để tiết kiệm dung lượng, nhưng hồ sơ giao dịch tài chính của bạn có thể cần được giữ trong nhiều năm.

Taking these ideas and articulating them as service-level objectives (SLOs), which we covered in Chapter 10 , can be a good way to enshrine these requirements as a core part of your software delivery process.

Việc lấy những ý tưởng này và diễn đạt chúng dưới dạng các mục tiêu mức dịch vụ (service-level objectives, SLO), điều mà chúng ta đã đề cập ở Chương 10, có thể là một cách hay để khắc ghi những yêu cầu này như một phần cốt lõi trong quy trình giao hàng phần mềm của bạn.

Degrading Functionality — Suy giảm chức năng

An essential part of building a resilient system, especially when your functionality is spread over a number of different microservices that may be up or down, is the ability to safely degrade functionality. Let's imagine a standard web page on our ecommerce site. To pull together the various parts of that website, we might need several microservices to play a part. One microservice might display the details about the item being offered for sale. Another might show the price and stock level. And we'll probably be showing shopping cart contents too, which may be yet another microservice. If one of those services goes down, and that results in the whole web page being unavailable, then we have arguably made a system that is less resilient than one that requires only one service to be available.

Một phần thiết yếu của việc xây dựng một hệ thống có khả năng phục hồi, đặc biệt khi chức năng của bạn được trải ra trên nhiều microservice khác nhau vốn có thể đang hoạt động hoặc không, là khả năng suy giảm chức năng một cách an toàn. Hãy hình dung một trang web tiêu chuẩn trên trang thương mại điện tử của chúng ta. Để ghép lại các phần khác nhau của trang web đó, chúng ta có thể cần vài microservice cùng đóng góp vai trò. Một microservice có thể hiển thị chi tiết về mặt hàng đang được rao bán. Một microservice khác có thể hiển thị giá và mức tồn kho. Và chúng ta có lẽ cũng sẽ hiển thị nội dung giỏ hàng, vốn có thể lại là một microservice khác nữa. Nếu một trong các dịch vụ đó ngừng hoạt động, và điều đó dẫn đến việc toàn bộ trang web không sẵn sàng, thì có thể nói chúng ta đã tạo ra một hệ thống kém khả năng phục hồi hơn so với một hệ thống chỉ cần một dịch vụ duy nhất sẵn sàng.

What we need to do is understand the impact of each outage and work out how to properly degrade functionality. From a business point of view, we would want our order-taking **workflow** to be as robust as possible, and we might be happy to accept some degradation of functionality to ensure this still works. If the stock levels are unavailable, we might make the decision to still go ahead with the sale and work out the details later. If the shopping cart microservice is unavailable, we're probably in a lot of trouble, but we could still show the web page with the listing. Perhaps we just hide the shopping cart or replace it with an icon saying "Be Back Soon!"

Điều chúng ta cần làm là hiểu tác động của mỗi sự cố ngừng hoạt động và tìm ra cách suy giảm chức năng một cách hợp lý. Từ góc độ nghiệp vụ, chúng ta sẽ muốn luồng nhận đơn hàng của mình vững chắc nhất có thể, và chúng ta có thể sẵn lòng chấp nhận một số suy giảm chức năng để đảm bảo điều này vẫn hoạt động. Nếu mức tồn kho không sẵn sàng, chúng ta có thể đưa ra quyết định vẫn tiến hành bán hàng và xử lý các chi tiết sau. Nếu microservice giỏ hàng không sẵn sàng, chúng ta có lẽ đang gặp khá nhiều rắc rối, nhưng chúng ta vẫn có thể hiển thị trang web với danh sách niêm yết. Có lẽ chúng ta chỉ ẩn giỏ hàng đi hoặc thay nó bằng một biểu tượng ghi "Sẽ quay lại ngay!"

With a single-process monolithic application, we don't have many decisions to make. System health is, to an extent, binary in this context—the process is either up or down. But with a microservice architecture, we need to consider a much more nuanced situation. The right thing to do in any situation is often not a technical decision. We might know what is technically possible when the shopping cart is down, but unless we understand the business context we won't understand what action we should be taking. For example, perhaps we close the entire site, still allow people to browse the catalog of items, or replace the part of the UI containing the cart control with a phone number for placing an order. But for every customer-facing **interface** that uses multiple microservices, or every microservice that depends on multiple downstream collaborators, you need to ask yourself, "What happens if this is down?" and know what to do.

Với một ứng dụng monolith đơn-tiến-trình, chúng ta không có nhiều quyết định phải đưa ra. Tình trạng sức khỏe của hệ thống, ở một mức độ nào đó, là nhị phân trong ngữ cảnh này—tiến trình hoặc đang chạy hoặc đã ngừng. Nhưng với một kiến trúc microservice, chúng ta cần cân nhắc một tình huống nhiều sắc thái hơn nhiều. Điều đúng đắn cần làm trong bất kỳ tình huống nào thường không phải là một quyết định kỹ thuật. Chúng ta có thể biết điều gì là khả thi về mặt kỹ thuật khi giỏ hàng ngừng hoạt động, nhưng trừ khi chúng ta hiểu ngữ cảnh nghiệp vụ, chúng ta sẽ không hiểu hành động mà mình nên thực hiện là gì. Ví dụ, có lẽ chúng ta đóng toàn bộ trang web, vẫn cho phép mọi người duyệt danh mục mặt hàng, hoặc thay phần giao diện chứa điều khiển giỏ hàng bằng một số điện thoại để đặt hàng. Nhưng với mỗi giao diện hướng tới khách hàng sử dụng nhiều microservice, hay mỗi microservice phụ thuộc vào nhiều cộng tác viên hạ nguồn, bạn cần tự hỏi: "Điều gì xảy ra nếu cái này ngừng hoạt động?" và biết phải làm gì.

By thinking about the criticality of each of our capabilities in terms of our cross-functional requirements, we'll be much better positioned to know what we can do. Now let's consider some things we can do from a technical point of view to make sure that when failure occurs, we can handle it gracefully.

Bằng cách suy nghĩ về tính tối quan trọng của mỗi năng lực của chúng ta xét theo các yêu cầu liên chức năng, chúng ta sẽ ở vị thế tốt hơn nhiều để biết mình có thể làm gì. Bây giờ hãy cân nhắc một số việc chúng ta có thể làm từ góc độ kỹ thuật để đảm bảo rằng khi sự cố xảy ra, chúng ta có thể xử lý nó một cách có kiểm soát.

Stability Patterns — Các pattern ổn định

There are a few patterns we can make use of to ensure that if something does go wrong, it doesn't cause nasty ripple-out effects. It is essential you understand these ideas, and you should strongly consider making use of them in your system to ensure that one bad citizen doesn't bring the whole world crashing down around your ears. In a moment, we'll take a look at a few key safety measures you should consider, but before we do, I'd like to share a brief story to outline the sort of thing that can go wrong.

Có một vài pattern mà chúng ta có thể tận dụng để đảm bảo rằng nếu có điều gì đó trục trặc, nó không gây ra các hiệu ứng lan tỏa khó chịu. Việc bạn hiểu những ý tưởng này là điều thiết yếu, và bạn nên cân nhắc nghiêm túc việc tận dụng chúng trong hệ thống của mình để đảm bảo rằng một thành phần xấu không kéo cả thế giới sụp đổ quanh tai bạn. Lát nữa, chúng ta sẽ xem xét một vài biện pháp an toàn then chốt mà bạn nên cân nhắc, nhưng trước khi làm vậy, tôi muốn chia sẻ một câu chuyện ngắn để phác họa loại sự việc có thể trục trặc.

Many years ago, I was a technical lead on a project for AdvertCorp. AdvertCorp (the company name and details are changed to protect the innocent!) provided online classified ads through a very popular website. The website itself handled fairly high volumes and generated a good deal of income for the business. The project I was working on was tasked with consolidating a number of existing services that were used to deliver similar functionality for different types of advertisements. Existing functionality for the different types of ads was slowly being migrated into the new system we were building, with a number of different types of ads still served up from older services. To make this transition transparent to the end customer, we intercepted all calls to the different types of ads in our new system, diverting them to the old systems where required, as outlined in Figure 12-1 . This is actually an example of a strangler fig pattern , which we discussed briefly in “Useful Decompositional Patterns” on page 79 .

Nhiều năm trước, tôi là trưởng nhóm kỹ thuật cho một dự án tại AdvertCorp. AdvertCorp (tên công ty và các chi tiết đã được thay đổi để bảo vệ những người vô can!) cung cấp quảng cáo rao vặt trực tuyến thông qua một trang web rất phổ biến. Bản thân trang web xử lý lượng truy cập khá lớn và tạo ra một khoản thu nhập đáng kể cho doanh nghiệp. Dự án mà tôi đang làm được giao nhiệm vụ hợp nhất một số dịch vụ hiện có vốn được dùng để cung cấp chức năng tương tự cho các loại quảng cáo khác nhau. Chức năng hiện có cho các loại quảng cáo khác nhau đang dần được di trú vào hệ thống mới mà chúng tôi đang xây dựng, với một số loại quảng cáo vẫn được phục vụ từ các dịch vụ cũ hơn. Để làm cho quá trình chuyển đổi này trong suốt đối với khách hàng cuối, chúng tôi chặn tất cả các lời gọi tới các loại quảng cáo khác nhau trong hệ thống mới của mình, chuyển hướng chúng tới các hệ thống cũ khi cần, như được phác họa ở Hình 12-1. Đây thực ra là một ví dụ về strangler fig pattern, mà chúng ta đã thảo luận ngắn gọn ở mục “Useful Decompositional

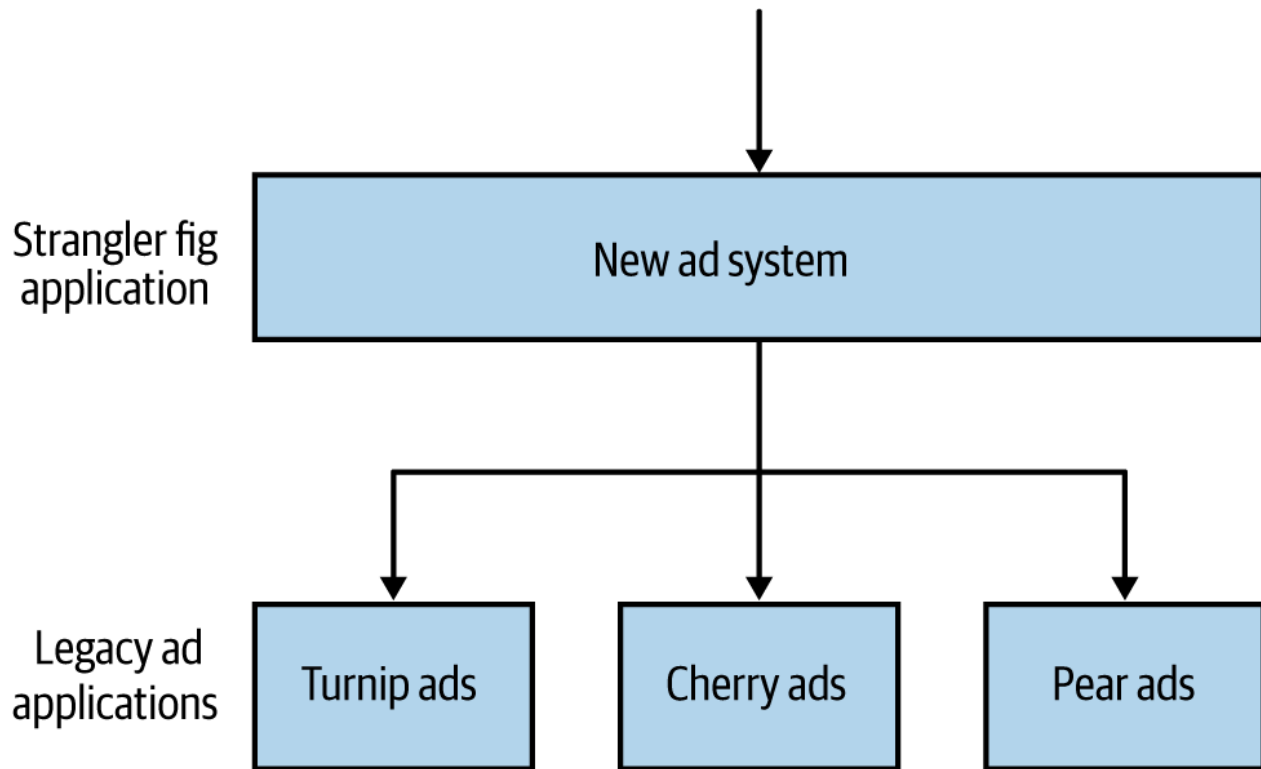


Figure 12-1. A strangler fig pattern being used to direct calls to older legacy systems

Hình 12-1. Một strangler fig pattern được dùng để định hướng các lời gọi tới các hệ thống cũ kế thừa

We had just moved over the highest-volume and biggest-earning product to the new system, but much of the **rest** of the ads were still being served by a number of older applications. In terms of both the number of searches and the money made by these applications, there was a very long tail—many of these older applications received small amounts of traffic and generated small amounts of revenue. The new system had been live for a while and was behaving very well, handling a not insignificant load. At that time we must have been handling around 6,000–7,000 requests per second during peak, and although most of that was very heavily cached by reverse proxies sitting in front of our application servers, the searches for products (the most important aspect of the site) were mostly uncached and required a full server round trip.

Chúng tôi vừa mới chuyển sản phẩm có lượng truy cập cao nhất và đem lại doanh thu lớn nhất sang hệ thống mới, nhưng phần lớn quảng cáo còn lại vẫn đang được phục vụ bởi một số ứng dụng cũ hơn. Xét về cả số lượng tìm kiếm lẫn số tiền kiếm được từ những ứng dụng này, có một cái đuôi rất dài—nhiều ứng dụng cũ này nhận lượng truy cập nhỏ và tạo ra lượng doanh thu nhỏ. Hệ thống mới đã chạy được một thời gian và hoạt động rất tốt, xử lý một mức tải không hề nhỏ. Vào thời điểm đó, chúng tôi hẳn đã xử lý khoảng 6.000–7.000 yêu cầu mỗi giây vào lúc cao điểm, và mặc dù phần lớn lượng đó được cache rất nhiều bởi các reverse proxy đứng trước các máy chủ ứng dụng của chúng tôi, các lượt tìm kiếm sản phẩm (khía cạnh quan trọng nhất của trang web) thì hầu hết không được cache và đòi hỏi một vòng đi-về đầy đủ tới máy chủ.

One morning, just before we hit our daily lunchtime peak, the system started behaving slowly, and then it started failing. We had some level of **monitoring** on our new core application, enough to tell

us that each of our application nodes was hitting a 100% CPU spike, well above the normal levels even at peak. In a short period of time, the entire site went down.

Một buổi sáng, ngay trước khi chúng tôi chạm đỉnh giờ trưa hằng ngày, hệ thống bắt đầu hoạt động chậm chạp, và rồi nó bắt đầu gặp sự cố. Chúng tôi có một mức độ giám sát nào đó trên ứng dụng lõi mới của mình, đủ để cho chúng tôi biết rằng mỗi node ứng dụng của chúng tôi đang lên đỉnh CPU 100%, cao hơn nhiều so với mức bình thường ngay cả vào lúc cao điểm. Trong một khoảng thời gian ngắn, toàn bộ trang web sập.

We managed to track down the culprit and bring the site back up. It turned out to be one of the downstream ad systems, which for the sake of this anonymous case study we'll say was responsible for turnip-related ads. The turnip ad service, one of the oldest and least actively maintained services, had started responding very slowly. Responding very slowly is one of the worst failure modes you can experience. If a system is just not there, you find out pretty quickly. When it's just slow, you end up waiting around for a while before giving up—that process of waiting around can slow the entire system down, cause resource contention, and, as happened in our case, result in a cascading failure. But whatever the cause of the failure, we had created a system that was vulnerable to a downstream issue cascading to cause a system-wide failure. A downstream service, over which we had little control, was able to take down our whole system.

Chúng tôi đã xoay sở truy ra thủ phạm và đưa trang web hoạt động trở lại. Hóa ra đó là một trong các hệ thống quảng cáo hạ nguồn, mà vì mục đích của nghiên cứu trường hợp ẩn danh này, chúng ta sẽ nói rằng nó chịu trách nhiệm cho các quảng cáo liên quan đến củ cải. Dịch vụ quảng cáo củ cải, một trong những dịch vụ cũ nhất và ít được bảo trì tích cực nhất, đã bắt đầu phản hồi rất chậm. Phản hồi rất chậm là một trong những kiểu sự cố tồi tệ nhất bạn có thể gặp. Nếu một hệ thống đơn giản là không có ở đó, bạn sẽ phát hiện ra khá nhanh. Khi nó chỉ chậm thôi, bạn lại phải chờ đợi quanh quẩn một lúc trước khi bỏ cuộc—quá trình chờ đợi đó có thể làm chậm toàn bộ hệ thống, gây ra tranh chấp tài nguyên, và, như đã xảy ra trong trường hợp của chúng tôi, dẫn đến một sự cố theo kiểu xếp tầng (cascading failure). Nhưng dù nguyên nhân sự cố là gì, chúng tôi đã tạo ra một hệ thống dễ tổn thương trước việc một vấn đề hạ nguồn lan tầng để gây ra một sự cố trên toàn hệ thống. Một dịch vụ hạ nguồn, mà chúng tôi có rất ít quyền kiểm soát, đã có thể đánh sập toàn bộ hệ thống của chúng tôi.

While one team looked at the problems with the turnip system, the rest of us started looking at what had gone wrong in our application. We found a few problems, which are outlined in Figure 12-2. We were using an HTTP connection pool to handle our downstream connections. The threads in the pool itself had time-outs configured for how long they would wait when making the downstream HTTP call, which is good. The problem was that the workers were all taking a while to time out due to the slow downstream service. While they were waiting, more requests went to the pool asking for worker threads. With no workers available, these requests themselves hung. It turned out the connection pool library we were using did have a **time-out** for waiting for workers, but this was disabled by default! This led to a huge buildup of blocked threads. Our application normally had 40 concurrent connections at any given time. In the space of five minutes, this situation caused us to peak at around 800 connections, bringing the system down.

Trong khi một nhóm xem xét các vấn đề với hệ thống củ cải, số còn lại trong chúng tôi bắt đầu tìm xem điều gì đã trục trặc trong ứng dụng của mình. Chúng tôi tìm thấy một vài vấn đề, được phác họa ở Hình 12-2. Chúng tôi đang dùng một connection pool HTTP để xử lý các kết nối hạ nguồn của mình. Bản thân các luồng (thread) trong pool đã được cấu hình time-out cho việc chúng sẽ chờ bao lâu khi thực hiện lời gọi HTTP hạ nguồn, điều đó là tốt. Vấn đề là tất cả các worker đều mất một lúc mới time-out do dịch vụ hạ nguồn chậm. Trong khi chúng đang chờ, ngày càng nhiều yêu cầu đến pool xin worker thread.

Không có worker nào sẵn sàng, bản thân những yêu cầu này cũng bị treo. Hóa ra thư viện connection pool mà chúng tôi đang dùng có một time-out cho việc chờ worker, nhưng nó bị tắt theo mặc định! Điều này dẫn đến sự tích tụ khổng lồ các luồng bị chặn. Ứng dụng của chúng tôi bình thường có 40 kết nối đồng thời tại bất kỳ thời điểm nào. Trong khoảng năm phút, tình huống này khiến chúng tôi lên đỉnh khoảng 800 kết nối, đánh sập hệ thống.

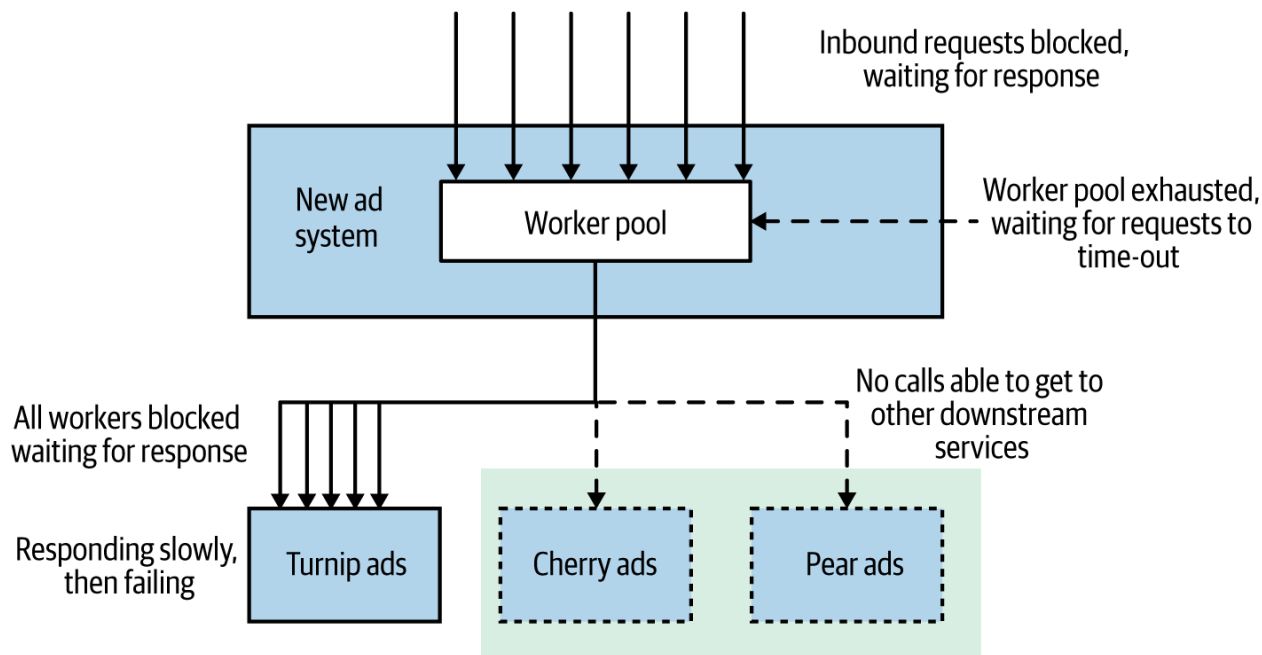


Figure 12-2. An outline of the issues that caused the outage

Hình 12-2. Phác họa các vấn đề đã gây ra sự cố ngừng hoạt động

What was worse was that the downstream service we were talking to represented functionality that less than 5% of our customer base used, and it generated even less revenue than that. When you get down to it, we discovered the hard way that systems that just act slow are much harder to deal with than systems that just fail fast. In a **distributed system**, latency kills.

Tệ hơn nữa là dịch vụ hạ nguồn mà chúng tôi đang giao tiếp đại diện cho chức năng mà dưới 5% cơ sở khách hàng của chúng tôi sử dụng, và nó tạo ra doanh thu còn ít hơn cả con số đó. Nói cho cùng, chúng tôi đã phát hiện ra một cách đau đớn rằng những hệ thống chỉ đơn giản là hoạt động chậm thì khó xử lý hơn nhiều so với những hệ thống đơn giản là gặp sự cố nhanh chóng. Trong một hệ thống phân tán, độ trễ giết người.

Even if we'd had the time-outs on the pool set correctly, we were also sharing a single HTTP connection pool for all outbound requests. This meant that one slow downstream service could exhaust the number of available workers all by itself, even if everything else was healthy. Lastly, it was clear due to the frequent time-outs and errors that the downstream service in question wasn't healthy, but despite this we kept sending traffic its way. In our situation, this meant we were actually making a bad situation worse, as the downstream service had no chance to recover. We ended up implementing three fixes to avoid this happening again: getting our time-outs right, implementing bulkheads to separate out different connection pools, and implementing a **circuit breaker** to avoid sending calls to an unhealthy system in the first place.

Ngay cả khi chúng tôi đã đặt đúng các time-out trên pool, chúng tôi vẫn còn đang dùng chung một connection pool HTTP duy nhất cho tất cả các yêu cầu gửi đi. Điều này có nghĩa là một dịch vụ hạ nguồn chậm có thể tự nó làm cạn kiệt số lượng worker khả dụng, ngay cả

khi mọi thứ khác đều khỏe mạnh. Cuối cùng, do các time-out và lỗi xảy ra thường xuyên, rõ ràng dịch vụ hạ nguồn nói trên không khỏe mạnh, nhưng bất chấp điều này chúng tôi vẫn tiếp tục gửi lưu lượng tới nó. Trong tình huống của chúng tôi, điều này có nghĩa là chúng tôi thực ra đang làm cho một tình huống tồi tệ trở nên tồi tệ hơn, vì dịch vụ hạ nguồn không có cơ hội nào để phục hồi. Cuối cùng chúng tôi đã triển khai ba bản sửa để tránh việc này tái diễn: đặt đúng các time-out, triển khai bulkhead để tách riêng các connection pool khác nhau, và triển khai circuit breaker để ngay từ đầu tránh gửi các lời gọi tới một hệ thống không khỏe mạnh.

Time-Outs — Time-out

Time-outs are easy to overlook, but in a distributed system they are important to get right. How long can I wait before I should give up on a call to a downstream service? If you wait too long to decide that a call has failed, you can slow the whole system down. Time out too quickly, and you'll consider a call that might have worked as failed. Have no time-outs at all, and a downstream service being down could hang your whole system.

Time-out là thứ dễ bị bỏ qua, nhưng trong một hệ thống phân tán, đặt chúng cho đúng là điều quan trọng. Tôi có thể chờ bao lâu trước khi nên bỏ cuộc với một lời gọi tới một dịch vụ hạ nguồn? Nếu bạn chờ quá lâu mới quyết định rằng một lời gọi đã thất bại, bạn có thể làm chậm toàn bộ hệ thống. Time-out quá nhanh, và bạn sẽ coi một lời gọi lẽ ra có thể thành công là đã thất bại. Hoàn toàn không có time-out, và một dịch vụ hạ nguồn ngừng hoạt động có thể làm treo toàn bộ hệ thống của bạn.

In the case of AdvertCorp, we had two time-out related issues. Firstly, we had a missing time-out on the HTTP request pool, meaning that when asking for a worker to make a downstream HTTP request, the request thread would block forever until a worker became available. Secondly, when we finally had an HTTP worker available to make a request to the turnip ad system, we were waiting way too long before giving up on the call. So as Figure 12-3 shows, we needed to add one new time-out and change an existing one.

Trong trường hợp của AdvertCorp, chúng tôi có hai vấn đề liên quan đến time-out. Thứ nhất, chúng tôi thiếu một time-out trên pool yêu cầu HTTP, nghĩa là khi xin một worker để thực hiện một yêu cầu HTTP hạ nguồn, luồng yêu cầu sẽ chặn mãi mãi cho đến khi một worker trở nên khả dụng. Thứ hai, khi cuối cùng chúng tôi đã có một worker HTTP khả dụng để thực hiện một yêu cầu tới hệ thống quảng cáo củ cải, chúng tôi lại chờ quá lâu trước khi bỏ cuộc với lời gọi đó. Vậy nên như Hình 12-3 cho thấy, chúng tôi cần thêm một time-out mới và thay đổi một time-out hiện có.

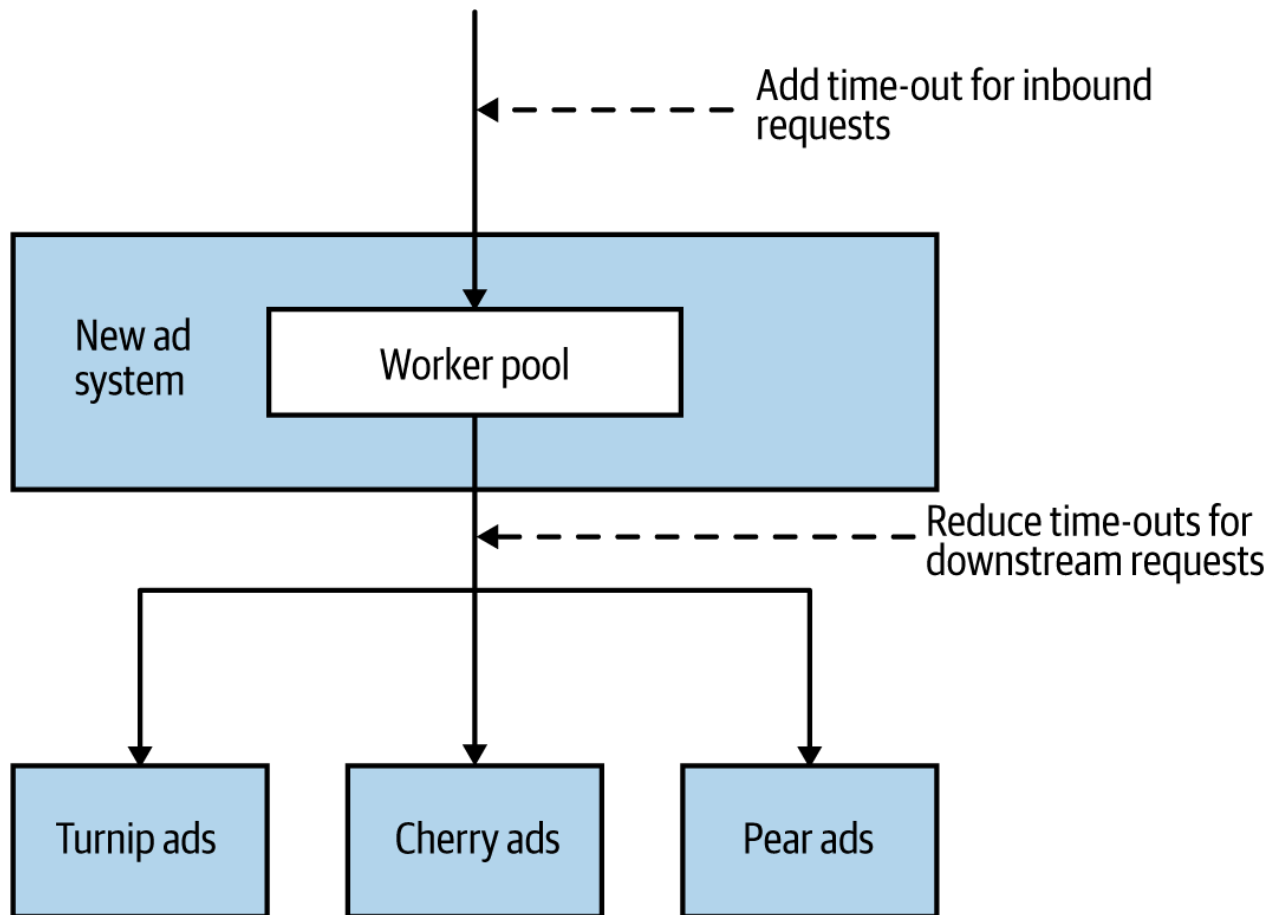


Figure 12-3. Changing time-outs on AdvertCorp system

Hình 12-3. Thay đổi các time-out trên hệ thống AdvertCorp

The time-outs for the downstream HTTP requests had been set to 30 seconds—so we would wait 30 seconds for a response from the turnip system before giving up. The issue is that in the wider context that this call was being made, waiting that long made no sense. The turnip-related ads were being requested as a result of one of our users looking at our website using a browser. Even back when this happened, no one waited 30 seconds for a page to load. Think about what happens if a web page doesn't load after 5, 10, or perhaps 15 seconds. What do you do? You refresh the page. So we were waiting 30 seconds for the turnip ad system to respond, but well before that the original request was no longer valid as the user had just refreshed, causing an additional inbound request. This in turn caused another request to come in for the ad system, and so it went.

Các time-out cho những yêu cầu HTTP hạ nguồn đã được đặt là 30 giây—vậy nên chúng tôi sẽ chờ 30 giây để có phản hồi từ hệ thống củ cải trước khi bỏ cuộc. Vấn đề là trong ngữ cảnh rộng hơn mà lời gọi này được thực hiện, chờ lâu như vậy chẳng có ý nghĩa gì. Các quảng cáo liên quan đến củ cải được yêu cầu là kết quả của việc một người dùng của chúng tôi đang xem trang web bằng trình duyệt. Ngay cả vào thời điểm chuyện này xảy ra, không ai chờ 30 giây để một trang tải xong. Hãy nghĩ xem điều gì xảy ra nếu một trang web không tải xong sau 5, 10, hay có lẽ 15 giây. Bạn làm gì? Bạn tải lại trang. Vậy nên chúng tôi đang chờ 30 giây để hệ thống quảng cáo củ cải phản hồi, nhưng từ rất lâu trước đó yêu cầu ban đầu đã không còn hợp lệ vì người dùng vừa tải lại trang, gây ra một yêu cầu gửi đến bổ sung. Điều này đến lượt nó lại gây ra một yêu cầu khác đến cho hệ thống quảng cáo, và cứ thế tiếp diễn.

When looking at the normal behavior of the turnip ad system, we could see that we would normally expect a response in much less than a second, so waiting for 30 seconds was overkill. Furthermore, we had a target to render a page to the user within 4–6 seconds. Based on this, we made the time-out much more aggressive, setting it to 1 second. We also put a 1 second time-out on waiting for an HTTP worker to be available. This meant that in the worst case, we would expect to wait around 2 seconds for information from the turnip system.

Khi xem xét hành vi bình thường của hệ thống quảng cáo củ cải, chúng tôi có thể thấy rằng bình thường chúng tôi sẽ kỳ vọng một phản hồi trong ít hơn một giây rất nhiều, vậy nên chờ 30 giây là thừa thãi. Hơn nữa, chúng tôi có một mục tiêu là hiển thị một trang cho người dùng trong vòng 4–6 giây. Dựa trên điều này, chúng tôi đã làm cho time-out quyết liệt hơn nhiều, đặt nó là 1 giây. Chúng tôi cũng đặt một time-out 1 giây cho việc chờ một worker HTTP khả dụng. Điều này có nghĩa là trong trường hợp tệ nhất, chúng tôi sẽ kỳ vọng chờ khoảng 2 giây để có thông tin từ hệ thống củ cải.



Time-outs are incredibly useful. Put time-outs on all out-of-process calls, and pick a default time-out for everything. Log when time-outs occur, look at what happens, and change them accordingly. Look at “normal” healthy response times for your downstream services, and use that to guide where you set the time-out threshold.

Time-out cực kỳ hữu ích. Hãy đặt time-out cho tất cả các lời gọi ra-ngoài-tiến-trình, và chọn

một time-out mặc định cho mọi thứ. Hãy ghi log khi time-out xảy ra, xem xét điều gì xảy ra, và thay đổi chúng cho phù hợp. Hãy xem thời gian phản hồi “bình thường” khi khỏe mạnh của các dịch vụ hạ nguồn của bạn, và dùng đó để dẫn lối cho việc bạn đặt ngưỡng time-out ở đâu.

Setting a time-out for a single service call might not be enough. What happens if this time-out is happening as part of a wider set of operations that you might want to give up on even before the time-out occurs? In the case of AdvertCorp, for example, there is no point waiting for the latest turnip prices if there is a good chance the user has already given up asking. In such a situation, it can make sense to have a time-out for the overall operation and to give up if this time-out is exceeded. For that to work, the current time left for the operation would need to be passed downstream. For example, if the overall operation to render a page had to complete within 1,000 ms, and by the time we made the call to the downstream turnip ad service 300 ms had already passed, we would then need to make sure we waited no longer than 700 ms for the rest of the calls to complete.

Việc đặt một time-out cho một lời gọi dịch vụ đơn lẻ có thể là chưa đủ. Điều gì xảy ra nếu time-out này đang diễn ra như một phần của một tập thao tác rộng hơn mà bạn có thể muốn bỏ cuộc ngay cả trước khi time-out xảy ra? Trong trường hợp của AdvertCorp, chẳng hạn, chẳng có ý nghĩa gì khi chờ giá củ cải mới nhất nếu có khả năng cao là người dùng đã bỏ cuộc không hỏi nữa. Trong tình huống như vậy, có thể hợp lý khi có một time-out cho toàn bộ thao tác và bỏ cuộc nếu time-out này bị vượt quá. Để điều đó hoạt động, thời gian còn lại hiện tại cho thao tác sẽ cần được truyền xuống hạ nguồn. Ví dụ, nếu toàn bộ thao tác hiển thị một trang phải hoàn thành trong vòng 1.000 ms, và đến lúc chúng ta thực hiện lời gọi tới dịch vụ quảng cáo củ cải hạ nguồn thì 300 ms đã trôi qua, chúng ta sẽ cần đảm bảo rằng mình không chờ quá 700 ms cho phần còn lại của các lời gọi hoàn thành.



Don't just think about the time-out for a single service call; also think about a time-out for the overall operation, and abort the operation if this overall time-out budget is exceeded.

Đừng chỉ nghĩ về time-out cho một lời gọi dịch vụ đơn lẻ; cũng hãy nghĩ về một time-out cho toàn bộ thao tác, và hủy bỏ thao tác nếu ngân sách time-out tổng thể này bị vượt quá.

Retries — Retry

Some issues with downstream calls are temporary. Packets can get misplaced, or gateways can have an odd spike in load, causing a time-out. Often, retrying the call can make a lot of sense. Coming back to what we just talked about, how often have you refreshed a web page that didn't load, only to find the second attempt worked fine? That's a **retry** in action.

Một số vấn đề với các lời gọi hạ nguồn chỉ là tạm thời. Các gói tin có thể bị thất lạc, hoặc các gateway có thể có một đợt tăng tải bất thường, gây ra một time-out. Thường thì, việc

retry lời gọi có thể rất hợp lý. Quay lại điều chúng ta vừa nói, bạn đã bao nhiêu lần tải lại một trang web không tải được, để rồi thấy lần thử thứ hai hoạt động bình thường? Đó là một retry trong thực tế.

It can be useful to consider what sort of downstream call failures should even be retried. If using a protocol like HTTP, for example, you may get back some useful information in the response codes that can help you determine if a retry is warranted. If you got back a 404 Not Found, a retry is unlikely to be a useful idea. On the other hand, a 503 Service Unavailable or a 504 Gateway Time-out could be considered temporary errors and could justify a retry.

Có thể hữu ích khi cân nhắc loại sự cố lời gọi hạ nguồn nào thì nên được retry. Nếu dùng một giao thức như HTTP, chẳng hạn, bạn có thể nhận lại một số thông tin hữu ích trong các mã phản hồi giúp bạn xác định liệu một retry có chính đáng hay không. Nếu bạn nhận lại một mã 404 Not Found, một retry khó có thể là ý hay. Mặt khác, một mã 503 Service Unavailable hoặc một 504 Gateway Time-out có thể được xem là lỗi tạm thời và có thể biện minh cho một retry.

You will likely need to have a delay before retrying. If the initial time-out or error was caused by the fact that the downstream microservice was under load, then bombarding it with additional requests may well be a bad idea.

Bạn có lẽ sẽ cần có một độ trễ trước khi retry. Nếu time-out hoặc lỗi ban đầu bị gây ra bởi việc microservice hạ nguồn đang chịu tải, thì việc dội bom nó bằng các yêu cầu bổ sung rất có thể là một ý tồi.

If you are going to retry, you need to take this into account when considering your time-out threshold. If the time-out threshold for a downstream call is set to 500 ms, but you allow up to three retries with one second between each retry, then you could end up waiting for up to 3.5 seconds before giving up. As mentioned earlier, having a budget for how long an operation is allowed to take can be a useful idea—you might not decide to do the third (or even second) retry if you’ve already exceeded the overall time-out budget. On the other hand, if this is happening as part of a non-user-facing operation, waiting longer to get something done might be totally acceptable.

Nếu bạn định retry, bạn cần tính đến điều này khi cân nhắc ngưỡng time-out của mình. Nếu ngưỡng time-out cho một lời gọi hạ nguồn được đặt là 500 ms, nhưng bạn cho phép tới ba lần retry với một giây giữa mỗi lần retry, thì bạn có thể kết thúc với việc chờ tới 3,5 giây trước khi bỏ cuộc. Như đã đề cập trước đó, có một ngân sách cho việc một thao tác được phép kéo dài bao lâu có thể là một ý hay—bạn có thể quyết định không thực hiện lần retry thứ ba (hoặc thậm chí thứ hai) nếu bạn đã vượt quá ngân sách time-out tổng thể. Mặt khác, nếu việc này đang diễn ra như một phần của một thao tác không hướng tới người dùng, thì chờ lâu hơn để hoàn thành một việc gì đó có thể là hoàn toàn chấp nhận được.

Bulkheads — Bulkhead

In **Release It!**, 4 Michael Nygard introduces the concept of a **bulkhead** as a way to isolate yourself from failure. In shipping, a bulkhead is a part of the ship that can be sealed off to protect the rest of the ship. So if the ship springs a leak, you can close the bulkhead doors. You lose part of the ship, but the rest of it remains intact.

Trong **Release It!**, 4 Michael Nygard giới thiệu khái niệm bulkhead như một cách để cô lập bản thân khỏi sự cố. Trong ngành tàu thủy, bulkhead là một phần của con tàu có thể được bịt kín để bảo vệ phần còn lại của con tàu. Vậy nên nếu con tàu bị thủng, bạn có thể đóng các cửa bulkhead lại. Bạn mất một phần con tàu, nhưng phần còn lại vẫn nguyên vẹn.

In software architecture terms, there are lots of different bulkheads we can consider. Returning to my own experience with AdvertCorp, we actually missed the chance to implement a bulkhead with regard to the downstream calls. We should have used different connection pools for each downstream connection. That way, if one connection pool got exhausted, the other connections wouldn't be impacted, as we see in Figure 12-4 .

Xét về mặt kiến trúc phần mềm, có rất nhiều loại bulkhead khác nhau mà chúng ta có thể cân nhắc. Quay lại trải nghiệm của chính tôi với AdvertCorp, chúng tôi thực ra đã bỏ lỡ cơ hội triển khai một bulkhead liên quan đến các lời gọi hạ nguồn. Đáng lẽ chúng tôi nên dùng các connection pool khác nhau cho mỗi kết nối hạ nguồn. Theo cách đó, nếu một connection pool bị cạn kiệt, các kết nối khác sẽ không bị ảnh hưởng, như chúng ta thấy ở Hình 12-4.

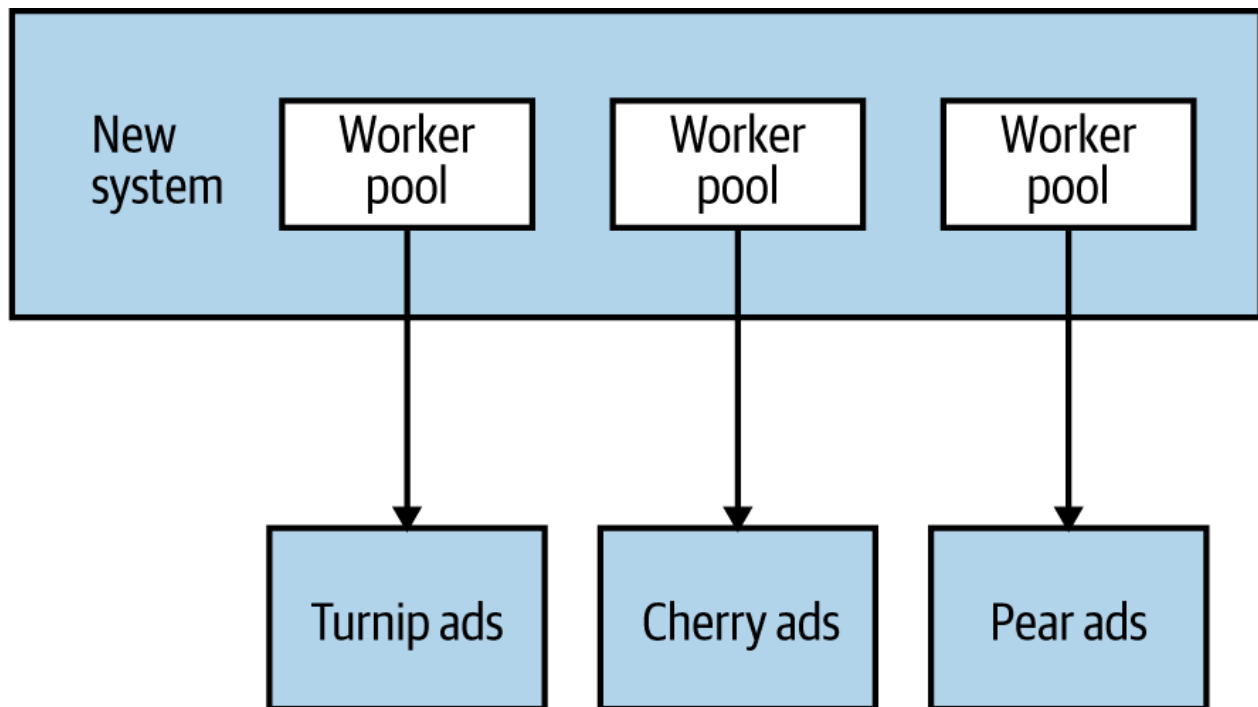


Figure 12-4. Using a connection pool per downstream service to provide bulkheads

Hình 12-4. Dùng một connection pool cho mỗi dịch vụ hạ nguồn để cung cấp các bulkhead

4 Michael T. Nygard, *Release It! Design and **Deploy** Production-Ready Software* , 2nd ed. (Raleigh: Pragmatic Bookshelf, 2018).

4 Michael T. Nygard, *Release It! Design and Deploy Production-Ready Software* , 2nd ed. (Raleigh: Pragmatic Bookshelf, 2018).

Separation of concerns can also be a way to implement bulkheads. By teasing apart functionality into separate microservices, we reduce the chance of an outage in one area affecting another.

Phân tách mối quan tâm (separation of concerns) cũng có thể là một cách để triển khai bulkhead. Bằng cách tách rời chức năng thành các microservice riêng biệt, chúng ta giảm khả năng một sự cố ngừng hoạt động ở một khu vực ảnh hưởng đến khu vực khác.

Look at all the aspects of your system that can go wrong, both inside your microservices and between them. Do you have bulkheads in place? I'd suggest starting with separate connection pools for each downstream connection, at the very least. You may want to go further, however, and consider using

circuit breakers too, which we'll look at in a moment.

Hãy xem xét tất cả các khía cạnh của hệ thống có thể trục trặc, cả bên trong các microservice của bạn lẫn giữa chúng. Bạn đã có sẵn các bulkhead chưa? Tôi sẽ gợi ý bắt đầu với các connection pool riêng cho mỗi kết nối hạ nguồn, ít nhất là vậy. Tuy nhiên, bạn có thể muốn đi xa hơn và cân nhắc dùng cả circuit breaker, mà chúng ta sẽ xem xét trong giây lát.

In many ways, bulkheads are the most important of the patterns we've looked at so far. Time-outs and circuit breakers help you free up resources when they are becoming constrained, but bulkheads can ensure they don't become constrained in the first place. They can also give you the ability to reject requests in certain conditions to ensure that resources don't become even more saturated; this is known as load shedding . Sometimes rejecting a request is the best way to stop an important system from becoming overwhelmed and being a bottleneck for multiple upstream services.

Về nhiều mặt, bulkhead là pattern quan trọng nhất trong số những pattern mà chúng ta đã xem xét cho đến nay. Time-out và circuit breaker giúp bạn giải phóng tài nguyên khi chúng đang trở nên hạn hẹp, nhưng bulkhead có thể đảm bảo rằng ngay từ đầu chúng không trở nên hạn hẹp. Chúng cũng có thể cho bạn khả năng từ chối các yêu cầu trong một số điều kiện nhất định để đảm bảo rằng tài nguyên không trở nên bão hòa hơn nữa; điều này được gọi là load shedding (đổ bớt tải). Đôi khi từ chối một yêu cầu là cách tốt nhất để ngăn một hệ thống quan trọng khỏi bị quá tải và trở thành một nút thắt cổ chai cho nhiều dịch vụ thượng nguồn.

Circuit Breakers — Circuit breaker

In your own home, circuit breakers exist to protect your electrical devices from spikes in power. If a spike occurs, the circuit breaker gets blown, protecting your expensive home appliances. You can also manually disable a circuit breaker to cut the power to part of your home, allowing you to work safely on the electrical system. In another pattern from *Release It!* , Nygard shows how the same idea can work wonders as a protection mechanism for our software.

Trong chính ngôi nhà của bạn, các circuit breaker (cầu dao) tồn tại để bảo vệ các thiết bị điện khỏi các đợt tăng vọt điện áp. Nếu một đợt tăng vọt xảy ra, circuit breaker bị ngắt, bảo vệ các thiết bị gia dụng đắt tiền của bạn. Bạn cũng có thể ngắt một circuit breaker bằng tay để cắt điện một phần ngôi nhà, cho phép bạn làm việc an toàn trên hệ thống điện. Trong một pattern khác từ *Release It!*, Nygard cho thấy cùng ý tưởng đó có thể tạo nên kỳ tích như một cơ chế bảo vệ cho phần mềm của chúng ta.

We can think of our circuit breakers as an automatic mechanism to seal a bulkhead, not only to protect the consumer from the downstream problem but also to potentially protect the downstream service from more calls that may be having an adverse impact. Given the perils of cascading failure, I'd recommend mandating circuit breakers for all your **synchronous** downstream calls. You don't have to write your own, either—in the years since I wrote the first edition of this book, circuit breaker implementations have become widely available.

Chúng ta có thể coi các circuit breaker của mình như một cơ chế tự động để bịt kín một bulkhead, không chỉ để bảo vệ bên tiêu thụ khỏi vấn đề hạ nguồn mà còn để có thể bảo vệ dịch vụ hạ nguồn khỏi nhiều lời gọi hơn vốn có thể đang gây tác động tiêu cực. Với những hiểm họa của sự cố xếp tầng, tôi sẽ khuyến nghị bắt buộc dùng circuit breaker cho tất cả các lời gọi hạ nguồn đồng bộ của bạn. Bạn cũng không phải tự viết lấy—trong những năm kể từ khi tôi viết ấn bản đầu tiên của cuốn sách này, các bản triển khai circuit breaker đã

trở nên có sẵn rộng rãi.

Coming back to AdvertCorp, consider the problem we had with the turnip system responding very slowly before eventually returning an error. Even if we'd gotten the time-outs right, we'd be waiting a long time before we got the error. And then we'd try it again the next time a request came in, and wait. It was bad enough that the downstream service was malfunctioning, but it was slowing down the entire system too.

Quay lại AdvertCorp, hãy cân nhắc vấn đề mà chúng tôi gặp với hệ thống củ cải phản hồi rất chậm trước khi cuối cùng trả về một lỗi. Ngay cả khi chúng tôi đã đặt đúng các time-out, chúng tôi vẫn sẽ chờ một thời gian dài trước khi nhận được lỗi. Và rồi chúng tôi sẽ thử lại nó vào lần sau khi một yêu cầu đến, và chờ. Việc dịch vụ hạ nguồn trực trực đã đủ tệ rồi, nhưng nó còn làm chậm toàn bộ hệ thống nữa.

With a circuit breaker, after a certain number of requests to the downstream resource have failed (due either to error or to a time-out), the circuit breaker is blown. All further requests that go through that circuit breaker fail fast while the breaker is in its blown (open) state, 5 as you can see in Figure 12-5 . After a certain period of time, the client sends a few requests through to see if the downstream service has recovered, and if it gets enough healthy responses it resets the circuit breaker.

Với một circuit breaker, sau khi một số lượng nhất định các yêu cầu tới tài nguyên hạ nguồn đã thất bại (do lỗi hoặc do time-out), circuit breaker bị ngắt. Tất cả các yêu cầu tiếp theo đi qua circuit breaker đó đều gặp sự cố nhanh chóng trong khi cầu dao đang ở trạng thái bị ngắt (mở), 5 như bạn có thể thấy ở Hình 12-5. Sau một khoảng thời gian nhất định, client gửi qua một vài yêu cầu để xem dịch vụ hạ nguồn đã phục hồi chưa, và nếu nó nhận đủ các phản hồi khỏe mạnh thì nó đặt lại circuit breaker.

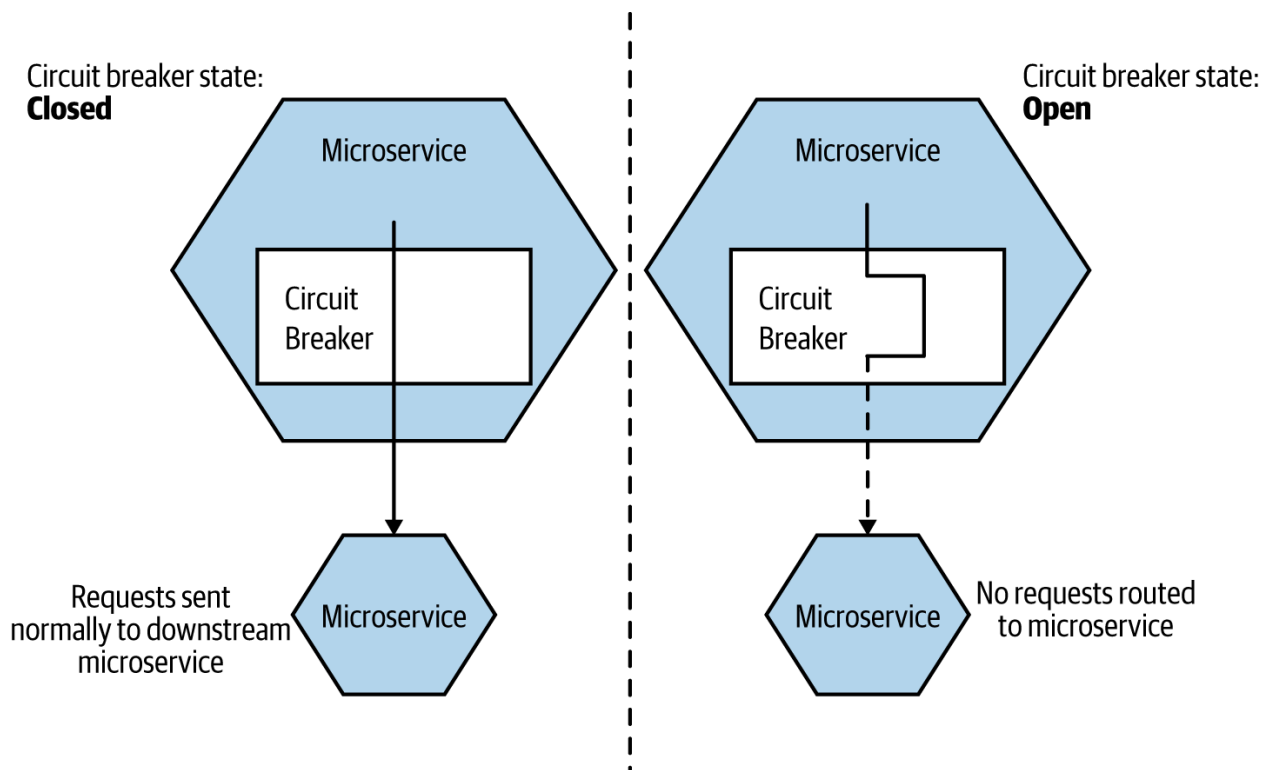


Figure 12-5. An overview of circuit breakers

Hình 12-5. Tổng quan về circuit breaker

How you implement a circuit breaker depends on what a “failed” request means, but when I’ve implemented them for HTTP connections, I’ve commonly taken failure to mean either a time-out or a subset of the 5XX HTTP return codes. In this way, when a downstream resource is timing out or returning errors, after a certain threshold is reached we automatically stop sending traffic and start failing fast. And we can automatically start again when things are healthy.

Cách bạn triển khai một circuit breaker phụ thuộc vào việc một yêu cầu “thất bại” nghĩa là gì, nhưng khi tôi triển khai chúng cho các kết nối HTTP, tôi thường coi thất bại nghĩa là hoặc một time-out hoặc một tập con của các mã trả về HTTP 5XX. Theo cách này, khi một tài nguyên hạ nguồn đang time-out hoặc trả về lỗi, sau khi một ngưỡng nhất định bị đạt tới, chúng ta tự động ngừng gửi lưu lượng và bắt đầu gặp sự cố nhanh chóng. Và chúng ta có thể tự động bắt đầu lại khi mọi thứ khỏe mạnh.

Getting the settings right can be a little tricky. You don’t want to blow the circuit breaker too readily, nor do you want to take too long to blow it. Likewise, you really want to make sure that the downstream service is healthy again before sending traffic. As with time-outs, I’d pick some sensible defaults and stick with them everywhere, and then change them for specific cases.

Việc đặt đúng các thiết lập có thể hơi khó. Bạn không muốn ngắt circuit breaker quá dễ dàng, cũng không muốn mất quá lâu mới ngắt nó. Tương tự, bạn thực sự muốn đảm bảo rằng dịch vụ hạ nguồn đã khỏe mạnh trở lại trước khi gửi lưu lượng. Cũng như với time-out, tôi sẽ chọn một số giá trị mặc định hợp lý và giữ nguyên chúng ở mọi nơi, rồi thay đổi chúng cho những trường hợp cụ thể.

While the circuit breaker is blown, you have some options. One is to queue up the requests and retry them later on. For some use cases, this might be appropriate, especially if you’re carrying out some work as part of an **asynchronous** job. If this call is

Trong khi circuit breaker bị ngắt, bạn có một số lựa chọn. Một là xếp hàng các yêu cầu và retry chúng sau. Với một số trường hợp sử dụng, điều này có thể phù hợp, đặc biệt nếu bạn đang thực hiện công việc nào đó như một phần của một tác vụ bất đồng bộ. Nếu lời gọi này

5 The terminology of an “open” breaker, meaning requests can’t flow, can be confusing, but it comes from electrical circuits. When the breaker is “open,” the circuit is broken and current cannot flow. Closing the breaker allows the circuit to be completed and current to flow once again.

5 Thuật ngữ một cầu dao “mở” (open), nghĩa là các yêu cầu không thể chảy qua, có thể gây nhầm lẫn, nhưng nó bắt nguồn từ các mạch điện. Khi cầu dao “mở”, mạch bị ngắt và dòng điện không thể chảy. Đóng cầu dao cho phép mạch được hoàn thiện và dòng điện chảy trở lại.

being made as part of a synchronous call chain, however, it is probably better to fail fast. This could mean propagating an error up the call chain, or a more subtle degrading of functionality.

đang được thực hiện như một phần của một chuỗi lời gọi đồng bộ, tuy nhiên, thì có lẽ tốt hơn là gặp sự cố nhanh chóng. Điều này có thể nghĩa là lan truyền một lỗi lên trên chuỗi lời gọi, hoặc một sự suy giảm chức năng tinh tế hơn.

In the case of AdvertCorp, we wrapped downstream calls to the legacy systems with circuit breakers, as Figure 12-6 shows. When these circuit breakers blew, we programmatically updated the website to show that we couldn’t currently show ads for, say, turnips. We kept the rest of the website working and clearly communicated to the customers that there was an issue restricted to one part of our product, all in a fully automated way.

Trong trường hợp của AdvertCorp, chúng tôi đã bọc các lời gọi hạ nguồn tới các hệ thống

kế thừa bằng circuit breaker, như Hình 12-6 cho thấy. Khi những circuit breaker này bị ngắt, chúng tôi cập nhật trang web bằng chương trình để hiển thị rằng hiện tại chúng tôi không thể hiển thị quảng cáo cho, chẳng hạn, củ cải. Chúng tôi giữ cho phần còn lại của trang web hoạt động và truyền đạt rõ ràng cho khách hàng rằng có một vấn đề chỉ giới hạn ở một phần sản phẩm của chúng tôi, tất cả theo một cách hoàn toàn tự động.

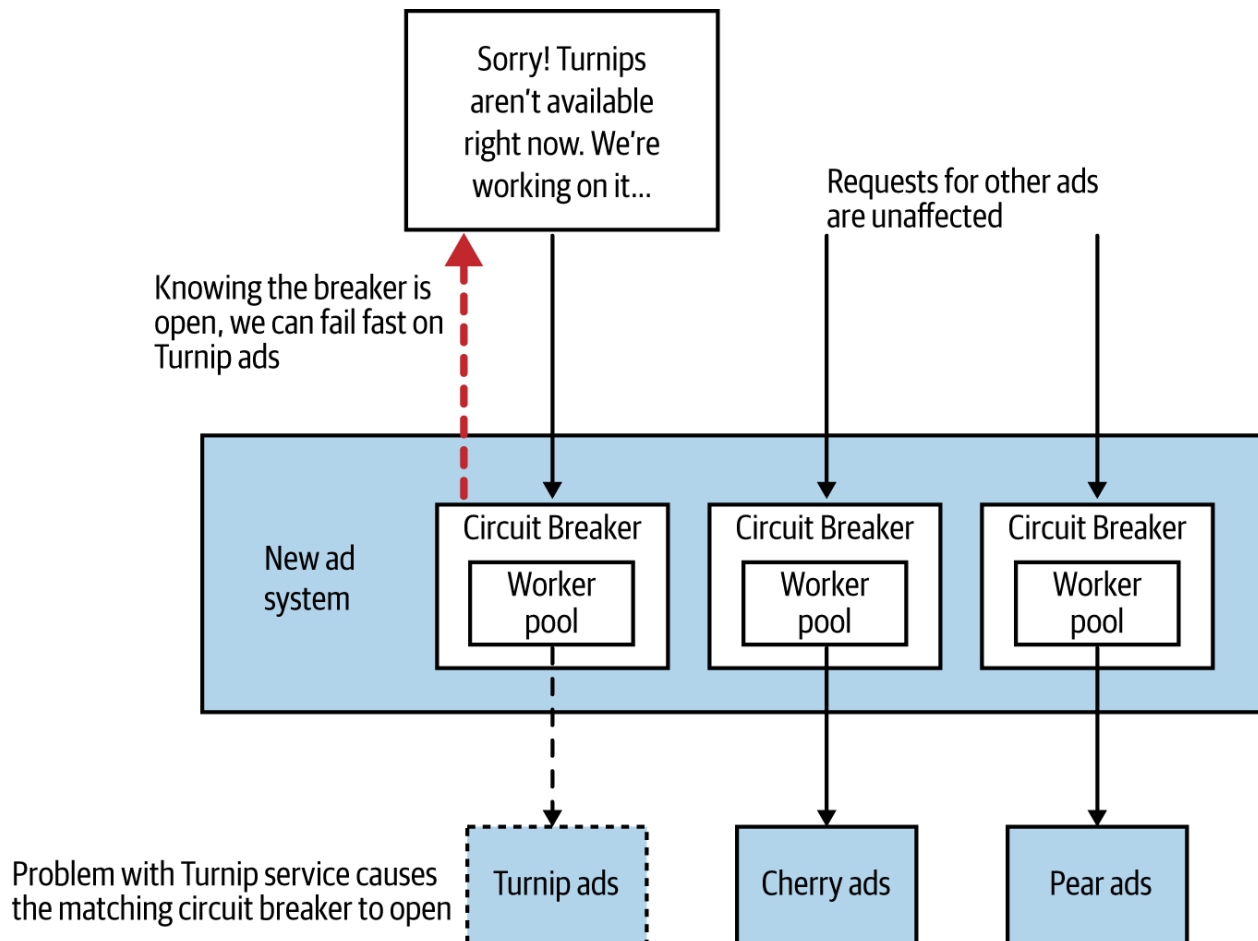


Figure 12-6. Adding circuit breakers to AdvertCorp

Hình 12-6. Thêm circuit breaker vào AdvertCorp

We were able to scope our circuit breakers so that we had one for each of the downstream legacy systems—this lined up well with the fact that we had decided to have different request worker pools for each downstream service.

Chúng tôi đã có thể định phạm vi các circuit breaker của mình sao cho có một cái cho mỗi hệ thống kế thừa hạ nguồn—điều này khớp tốt với việc chúng tôi đã quyết định có các pool worker yêu cầu khác nhau cho mỗi dịch vụ hạ nguồn.

If we have this mechanism in place (as with the circuit breakers in our home), we could use them manually to make it safer to do our work. For example, if we wanted to take a microservice down as part of routine maintenance, we could manually open all the circuit breakers of the upstream consumers so they fail fast while the microservice is offline. Once it's back, we can close the circuit breakers, and everything should go back to normal. Scripting the process to manually open and close a circuit breaker as part of an automated **deployment** process could be a sensible next step.

Nếu chúng ta có cơ chế này sẵn sàng (như với các circuit breaker trong nhà chúng ta), chúng ta có thể dùng chúng bằng tay để làm cho công việc của mình an toàn hơn. Ví dụ, nếu chúng ta muốn hạ một microservice xuống như một phần của bảo trì thường lệ, chúng ta có thể mở bằng tay tất cả các circuit breaker của các bên tiêu thụ thượng nguồn để chúng gặp sự cố nhanh chóng trong khi microservice ở ngoại tuyến. Khi nó trở lại, chúng ta có thể đóng các circuit breaker lại, và mọi thứ sẽ trở lại bình thường. Việc viết kịch bản cho quy trình mở và đóng một circuit breaker bằng tay như một phần của một quy trình triển khai tự động có thể là một bước tiếp theo hợp lý.

Circuit breakers help our application fail fast—and failing fast is always better than failing slow. The circuit breakers allow us to fail before wasting valuable time (and resources) waiting for an unhealthy downstream microservice to respond. Rather than waiting until we try to use the downstream microservice to fail, we could check the status of our circuit breakers earlier. If a microservice we will rely on as part of an operation is currently unavailable, we can abort the operation before we even start.

Circuit breaker giúp ứng dụng của chúng ta gặp sự cố nhanh chóng—và gặp sự cố nhanh luôn tốt hơn gặp sự cố chậm. Circuit breaker cho phép chúng ta gặp sự cố trước khi lãng phí thời gian (và tài nguyên) quý báu để chờ một microservice hạ nguồn không khỏe mạnh phản hồi. Thay vì chờ cho đến khi chúng ta cố sử dụng microservice hạ nguồn để rồi gặp sự cố, chúng ta có thể kiểm tra trạng thái của các circuit breaker sớm hơn. Nếu một microservice mà chúng ta sẽ dựa vào như một phần của một thao tác hiện đang không sẵn sàng, chúng ta có thể hủy bỏ thao tác trước cả khi bắt đầu.

Isolation — Cô lập (Isolation)

The more one microservice depends on another microservice being available, the more the health of one impacts the ability of the other to do its job. If we can use technology that allows a downstream server to be offline, for example, through the use of middleware or some other type of call buffering system, upstream microservices are less likely to be affected by outages, planned or unplanned, of downstream microservices.

Một microservice càng phụ thuộc vào việc một microservice khác sẵn sàng, thì tình trạng sức khỏe của cái này càng tác động đến khả năng làm việc của cái kia. Nếu chúng ta có thể dùng công nghệ cho phép một máy chủ hạ nguồn ở ngoại tuyến, chẳng hạn, thông qua việc dùng middleware hoặc một loại hệ thống đệm lời gọi nào đó khác, thì các microservice thượng nguồn ít có khả năng bị ảnh hưởng bởi các sự cố ngừng hoạt động, có kế hoạch hay ngoài kế hoạch, của các microservice hạ nguồn.

There is another benefit to increasing isolation between services. When services are isolated from each other, much less coordination is needed between service owners. The less coordination needed between teams, the more autonomy those teams have, as they are able to operate and evolve their services more freely.

Có một lợi ích khác của việc tăng cường sự cô lập giữa các dịch vụ. Khi các dịch vụ được cô lập với nhau, cần ít sự phối hợp hơn nhiều giữa những người sở hữu dịch vụ. Càng cần ít sự phối hợp giữa các nhóm, thì các nhóm đó càng có nhiều quyền tự chủ, vì họ có thể vận hành và tiến hóa dịch vụ của mình tự do hơn.

Isolation also applies in terms of how we move from the logical to the physical. Consider two microservices that appear to be entirely isolated from one another. They don't communicate with each other in any way. A problem with one of them shouldn't impact the other, right? But what if

both microservices are running on the same host, and one of the microservices starts using up all the CPU, causing that host to have issues?

Cô lập cũng áp dụng xét về cách chúng ta chuyển từ cấp độ logic sang cấp độ vật lý. Hãy xét hai microservice trông như thể chúng hoàn toàn cô lập với nhau. Chúng không giao tiếp với nhau theo bất kỳ cách nào. Một vấn đề với một trong hai cái không nên ảnh hưởng đến cái kia, đúng không? Nhưng điều gì xảy ra nếu cả hai microservice đang chạy trên cùng một host, và một trong các microservice bắt đầu dùng hết CPU, khiến host đó gặp sự cố?

Consider another example. Two microservices each have their own, logically isolated **database**. But both databases are deployed onto the same database infrastructure. A failure in that database infrastructure would impact both microservices.

Hãy xét một ví dụ khác. Hai microservice mỗi cái có cơ sở dữ liệu riêng, được cô lập về mặt logic. Nhưng cả hai cơ sở dữ liệu được triển khai trên cùng một hạ tầng cơ sở dữ liệu. Một sự cố trong hạ tầng cơ sở dữ liệu đó sẽ ảnh hưởng đến cả hai microservice.

When we consider how we want to deploy our microservices, we also want to strive to ensure a degree of failure isolation to avoid problems like this. For example, ensuring that microservices are running on independent hosts with their own ring-fenced operating system and computing resources is a sensible step—this is what we achieve when we run microservice instances in their own virtual machine or **container**. This kind of isolation, though, can have a cost.

Khi cân nhắc cách chúng ta muốn triển khai các microservice của mình, chúng ta cũng muốn nỗ lực đảm bảo một mức độ cô lập sự cố để tránh các vấn đề như thế này. Ví dụ, đảm bảo rằng các microservice đang chạy trên các host độc lập với hệ điều hành và tài nguyên tính toán được khoanh vùng riêng của chúng là một bước hợp lý—đây là điều chúng ta đạt được khi chạy các thực thể microservice trong máy ảo hoặc container riêng của chúng. Tuy nhiên, kiểu cô lập này có thể đi kèm một cái giá.

We can isolate our microservices from each other more effectively by running them on different machines. This means we need more infrastructure, and tooling to manage that infrastructure. This has a direct cost and can also add to the complexity of our system exposing new avenues of potential failure. Each microservice could have its own totally dedicated database infrastructure, but that is more infrastructure to manage. We could use middleware to provide temporal decoupling between two microservices, but now we have a broker to worry about.

Chúng ta có thể cô lập các microservice với nhau hiệu quả hơn bằng cách chạy chúng trên các máy khác nhau. Điều này có nghĩa là chúng ta cần nhiều hạ tầng hơn, và công cụ để quản lý hạ tầng đó. Điều này có một chi phí trực tiếp và cũng có thể làm tăng độ phức tạp của hệ thống, phơi bày ra những con đường mới dẫn đến sự cố tiềm tàng. Mỗi microservice có thể có hạ tầng cơ sở dữ liệu hoàn toàn riêng, dành riêng cho nó, nhưng đó là nhiều hạ tầng hơn để quản lý. Chúng ta có thể dùng middleware để cung cấp sự tách rời theo thời gian (temporal decoupling) giữa hai microservice, nhưng giờ đây chúng ta lại có một broker phải lo lắng.

Isolation, like so many of the other techniques we have looked at, can help improve the robustness of our applications, but it's rare that it does so for free. Deciding on the acceptable trade-offs around isolation versus cost and increased complexity, like so many other things, can be vital.

Cô lập, cũng như rất nhiều kỹ thuật khác mà chúng ta đã xem xét, có thể giúp cải thiện độ vững chắc của các ứng dụng của chúng ta, nhưng hiếm khi nó làm vậy mà miễn phí. Việc quyết định những đánh đổi chấp nhận được xoay quanh cô lập so với chi phí và độ phức tạp tăng thêm, cũng như rất nhiều thứ khác, có thể là điều tối quan trọng.

Redundancy — Dự phòng (Redundancy)

Having more of something can be a great way to improve the robustness of a component. Having more than one person who knows how the production database works seems sensible, in case someone leaves the company or is away on leave. Having more than one microservice instance makes sense as it allows you to tolerate the failure of one of those instances and still have a chance of delivering the required functionality.

Có nhiều hơn một cái gì đó có thể là một cách tuyệt vời để cải thiện độ vững chắc của một thành phần. Có nhiều hơn một người biết cơ sở dữ liệu sản xuất hoạt động ra sao có vẻ hợp lý, phòng khi ai đó rời công ty hoặc đang nghỉ phép. Có nhiều hơn một thực thể microservice là hợp lý vì nó cho phép bạn chịu đựng được sự cố của một trong những thực thể đó và vẫn có cơ hội cung cấp chức năng cần thiết.

Working out how much redundancy you need, and where, will depend on how well you understand potential failure modes of each component, the impact of that functionality being unavailable, and the cost of adding the redundancy.

Việc tìm ra bạn cần bao nhiêu dự phòng, và ở đâu, sẽ phụ thuộc vào việc bạn hiểu rõ đến mức nào các kiểu sự cố tiềm tàng của mỗi thành phần, tác động của việc chức năng đó không sẵn sàng, và chi phí của việc thêm dự phòng.

On AWS, for example, you do not get an SLA for the uptime of a single EC2 (virtual machine) instance. You have to work on the assumption that it can and will die on you. So it makes sense to have more than one. But going further, EC2 instances are deployed into availability zones (virtual data centers), and you also have no guarantees regarding the availability of a single availability zone, meaning you'd want that second instance to be on a different availability zone to spread the risk.

Trên AWS, chẳng hạn, bạn không có được một SLA cho thời gian hoạt động của một thực thể EC2 (máy ảo) đơn lẻ. Bạn phải làm việc với giả định rằng nó có thể và sẽ chết với bạn. Vậy nên có nhiều hơn một cái là hợp lý. Nhưng đi xa hơn, các thực thể EC2 được triển khai vào các vùng sẵn sàng (availability zone, các trung tâm dữ liệu ảo), và bạn cũng không có gì đảm bảo về tính sẵn sàng của một vùng sẵn sàng đơn lẻ, nghĩa là bạn sẽ muốn thực thể thứ hai đó ở một vùng sẵn sàng khác để phân tán rủi ro.

Having more copies of something can help when it comes to implementing redundancy, but it can also be beneficial when it comes to **scaling** our applications to handle increased load. In the next chapter we'll look at examples of system scaling, and see how scaling for redundancy or scaling for load can differ.

Có nhiều bản sao của một cái gì đó có thể giúp ích khi nói đến việc triển khai dự phòng, nhưng nó cũng có thể có lợi khi nói đến việc mở rộng các ứng dụng của chúng ta để xử lý tải tăng lên. Trong chương tiếp theo, chúng ta sẽ xem các ví dụ về mở rộng hệ thống, và thấy mở rộng để dự phòng hay mở rộng để chịu tải có thể khác nhau như thế nào.

Middleware — Middleware

In “Message Brokers” on page 135, we looked at the role of middleware in the form of message brokers to help with implementing both **request-response** and event-based interactions. One of the useful properties of most message brokers is their ability to provide guaranteed delivery. You send a message to a downstream party, and the broker guarantees to deliver it, with some caveats that we explored earlier. Internally, to provide this guarantee, the **message broker** software will have to implement things like retries and time-outs on your behalf—the same sorts of operations

are being carried out as you would have to do yourself, but they are being done in software written by experts with a deep focus on this sort of thing. Having smart people do work for you is often a good idea.

Trong mục “Message Brokers” trang 135, chúng ta đã xem xét vai trò của middleware dưới dạng message broker để giúp triển khai cả các tương tác request-response lẫn các tương tác dựa trên sự kiện. Một trong những thuộc tính hữu ích của hầu hết các message broker là khả năng cung cấp việc giao nhận được đảm bảo. Bạn gửi một thông điệp tới một bên hạ nguồn, và broker đảm bảo giao nó, với một số điều kiện ràng buộc mà chúng ta đã khám phá trước đó. Bên trong, để cung cấp sự đảm bảo này, phần mềm message broker sẽ phải triển khai những thứ như retry và time-out thay cho bạn—cùng những loại thao tác đang được thực hiện như những gì bạn sẽ phải tự làm, nhưng chúng đang được làm trong phần mềm được viết bởi các chuyên gia tập trung sâu vào loại việc này. Để những người thông minh làm việc cho bạn thường là một ý hay.

Now, in the case of our specific example with AdvertCorp, using middleware to manage request-response communication with the downstream turnip system might not actually have helped much. We would still not be getting responses back to our customers. The one potential benefit would be that we’d be relieving resource contention on our own system, but this would just shift to an increasing number of pending requests being held in the broker. Worse still, many of these requests asking for the latest turnip prices might relate to user requests that are no longer valid.

Bây giờ, trong trường hợp ví dụ cụ thể của chúng tôi với AdvertCorp, việc dùng middleware để quản lý giao tiếp request-response với hệ thống củ cải hạ nguồn thực ra có lẽ đã không giúp được nhiều. Chúng tôi vẫn sẽ không nhận được phản hồi trả về cho khách hàng. Lợi ích tiềm năng duy nhất sẽ là chúng tôi giảm bớt được tranh chấp tài nguyên trên hệ thống của chính mình, nhưng điều này chỉ chuyển sang một số lượng ngày càng tăng các yêu cầu đang chờ được giữ trong broker. Tệ hơn nữa, nhiều yêu cầu trong số này hỏi giá củ cải mới nhất có thể liên quan đến các yêu cầu người dùng vốn đã không còn hợp lệ.

An alternative could be to instead invert the interaction and use middleware to have the turnip system broadcast the last turnip ads, and we could then consume them. But if the downstream turnip system had a problem, we still wouldn’t be able to help the customer looking for the best turnip prices.

Một phương án khác có thể là thay vào đó đảo ngược tương tác và dùng middleware để hệ thống củ cải phát quảng bá các quảng cáo củ cải mới nhất, rồi chúng ta có thể tiêu thụ chúng. Nhưng nếu hệ thống củ cải hạ nguồn gặp vấn đề, chúng ta vẫn sẽ không thể giúp được khách hàng đang tìm giá củ cải tốt nhất.

So using middleware like message brokers to help offload some robustness concerns can be useful, but not in every situation.

Vậy nên dùng middleware như message broker để giúp đẩy bớt một số mối lo về độ vững chắc có thể hữu ích, nhưng không phải trong mọi tình huống.

Idempotency — Tính idempotent (Idempotency)

In idempotent operations, the outcome doesn’t change after the first application, even if the operation is subsequently applied multiple times. If operations are idempotent, we can repeat the call multiple times without adverse impact. This is very useful when we want to replay messages that we aren’t sure have been processed, a common way of recovering from error.

Trong các thao tác idempotent, kết quả không thay đổi sau lần áp dụng đầu tiên, ngay cả khi thao tác sau đó được áp dụng nhiều lần. Nếu các thao tác là idempotent, chúng ta có thể lặp lại lời gọi nhiều lần mà không gây tác động bất lợi. Điều này rất hữu ích khi chúng ta muốn phát lại các thông điệp mà chúng ta không chắc đã được xử lý hay chưa, một cách phổ biến để phục hồi sau lỗi.

Let's consider a simple call to add some points as a result of one of our customers placing an order. We might make a call with the sort of payload shown in Example 12-1 .

Hãy xét một lời gọi đơn giản để thêm một số điểm như kết quả của việc một trong các khách hàng của chúng ta đặt một đơn hàng. Chúng ta có thể thực hiện một lời gọi với loại payload được thể hiện ở Ví dụ 12-1.

Example 12-1. Crediting points to an account

Ví dụ 12-1. Ghi có điểm vào một tài khoản

100 1234

100 1234

If this call is received multiple times, we would add 100 points multiple times. As it stands, therefore, this call is not idempotent. With a bit more information, though, we allow the points bank to make this call idempotent, as shown in Example 12-2 .

Nếu lời gọi này được nhận nhiều lần, chúng ta sẽ thêm 100 điểm nhiều lần. Do đó, ở trạng thái hiện tại, lời gọi này không phải là idempotent. Tuy nhiên, với một chút thông tin nữa, chúng ta cho phép ngân hàng điểm làm cho lời gọi này trở nên idempotent, như được thể hiện ở Ví dụ 12-2.

Example 12-2. Adding more information to the points credit to make it idempotent

Ví dụ 12-2. Thêm thông tin vào lệnh ghi có điểm để làm cho nó idempotent

100 1234 4567

100 1234 4567

We know that this credit relates to a specific order, 4567 . Assuming that we could receive only one credit for a given order, we could apply this credit again without increasing the overall number of points.

Chúng ta biết rằng lần ghi có này liên quan đến một đơn hàng cụ thể, 4567. Với giả định rằng chúng ta chỉ có thể nhận một lần ghi có cho một đơn hàng nhất định, chúng ta có thể áp dụng lần ghi có này lại mà không làm tăng tổng số điểm.

This mechanism works just as well with event-based collaboration and can be especially useful if you have multiple instances of the same type of service subscribing to events. Even if we store which events have been processed, with some forms of asynchronous message delivery there may be small windows in which two workers can see the same message. By processing the events in an idempotent manner, we ensure this won't cause us any issues.

Cơ chế này hoạt động tốt ngang nhau với cộng tác dựa trên sự kiện và có thể đặc biệt hữu ích nếu bạn có nhiều thực thể của cùng một loại dịch vụ đăng ký nhận các sự kiện. Ngay cả khi chúng ta lưu lại những sự kiện nào đã được xử lý, với một số hình thức giao nhận thông điệp bất đồng bộ vẫn có thể có những cửa sổ nhỏ trong đó hai worker có thể thấy cùng một thông điệp. Bằng cách xử lý các sự kiện theo một cách idempotent, chúng ta đảm bảo điều này sẽ không gây ra cho chúng ta vấn đề gì.

Some people get quite caught up with this concept and assume it means that subsequent calls with the same parameters can't have any impact, which then leaves us in an interesting position. We really would still like to record the fact that a call was received in our logs, for example. We want to record the response time of the call and collect this data for monitoring. The key point here is that it is the underlying business operation that we are considering idempotent, not the entire state of the system.

Một số người khá vướng mắc với khái niệm này và cho rằng nó có nghĩa là các lời gọi tiếp theo với cùng tham số không thể có bất kỳ tác động nào, điều này đặt chúng ta vào một vị trí thú vị. Chúng ta thực sự vẫn muốn ghi lại sự kiện rằng một lời gọi đã được nhận trong nhật ký của mình, chẳng hạn. Chúng ta muốn ghi lại thời gian phản hồi của lời gọi và thu thập dữ liệu này để giám sát. Điểm mấu chốt ở đây là chính thao tác nghiệp vụ nền tảng mới là thứ mà chúng ta đang coi là idempotent, chứ không phải toàn bộ trạng thái của hệ thống.

Some of the HTTP verbs, such as GET and PUT, are defined in the HTTP specification to be idempotent, but for that to be the case, they rely on your service handling these calls in an idempotent manner. If you start making these verbs nonidempotent, but callers think they can safely execute them repeatedly, you may get yourself into a mess. Remember, just because you're using HTTP as an underlying protocol doesn't mean you get everything for free!

Một số động từ HTTP, chẳng hạn như GET và PUT, được định nghĩa trong đặc tả HTTP là idempotent, nhưng để được như vậy, chúng dựa vào việc dịch vụ của bạn xử lý những lời gọi này theo một cách idempotent. Nếu bạn bắt đầu làm cho các động từ này không idempotent, nhưng người gọi lại nghĩ rằng họ có thể thực thi chúng lặp lại một cách an toàn, bạn có thể tự đẩy mình vào một mớ hỗn độn. Hãy nhớ, chỉ vì bạn đang dùng HTTP làm giao thức nền tảng không có nghĩa là bạn có được mọi thứ miễn phí!

Spreading Your Risk — Phân tán rủi ro của bạn

One way to scale for resilience is to ensure that you don't put all your eggs in one basket. A simplistic example of this is making sure that you don't have multiple services on one host, where an outage would impact multiple services. But let's consider what host means. In most situations nowadays, a "host" is actually a virtual concept. So what if I have all of my services on different hosts, but all those hosts are actually virtual hosts, running on the same physical box? If that box goes down, I could lose multiple services. Some virtualization platforms enable you to ensure that your hosts are distributed across multiple different physical boxes to reduce the chance of this happening.

Một cách để mở rộng nhằm đạt khả năng phục hồi là đảm bảo rằng bạn không bỏ tất cả trứng vào một giỏ. Một ví dụ đơn giản về điều này là đảm bảo rằng bạn không có nhiều dịch vụ trên một host, nơi mà một sự cố ngừng hoạt động sẽ ảnh hưởng đến nhiều dịch vụ. Nhưng hãy cân nhắc host nghĩa là gì. Trong hầu hết các tình huống ngày nay, một "host" thực ra là một khái niệm ảo. Vậy nếu tôi có tất cả các dịch vụ của mình trên các host khác nhau, nhưng tất cả các host đó thực ra là các host ảo, chạy trên cùng một cỗ máy vật lý thì sao? Nếu cỗ máy đó hỏng, tôi có thể mất nhiều dịch vụ. Một số nền tảng ảo hóa cho phép bạn đảm bảo rằng các host của bạn được phân bố trên nhiều cỗ máy vật lý khác nhau để giảm khả năng điều này xảy ra.

For internal virtualization platforms, it is a common practice to have the virtual machine's root partition mapped to a single SAN (storage area network). If that SAN goes down, it can take down all connected VMs. SANs are big, expensive, and designed not to fail. That said, I have had big, expensive SANs fail on me at least twice in the last 10 years, and each time the results were fairly serious.

Đối với các nền tảng ảo hóa nội bộ, một thực hành phổ biến là ánh xạ phân vùng gốc của máy ảo tới một SAN (storage area network, mạng lưu trữ) duy nhất. Nếu SAN đó hỏng, nó có thể đánh sập tất cả các VM được kết nối. Các SAN thì lớn, đắt tiền, và được thiết kế để không hỏng. Dẫu vậy, tôi đã từng có những SAN lớn, đắt tiền hỏng với tôi ít nhất hai lần trong 10 năm qua, và mỗi lần kết quả đều khá nghiêm trọng.

Another common form of separation to reduce failure is to ensure that not all your services are running in a single rack in the data center, or that your services are distributed across more than one data center. If you're using an underlying service provider, it is important to know if an SLA is offered and to plan accordingly. If you need to ensure your services are down for no more than four hours every quarter, but your hosting provider can only guarantee a maximum downtime of eight hours per quarter, you have to change the SLA or else come up with an alternative solution.

Một hình thức phân tách phổ biến khác để giảm sự cố là đảm bảo rằng không phải tất cả các dịch vụ của bạn đều chạy trong một rack duy nhất trong trung tâm dữ liệu, hoặc rằng các dịch vụ của bạn được phân bố trên nhiều hơn một trung tâm dữ liệu. Nếu bạn đang dùng một nhà cung cấp dịch vụ nền tảng, điều quan trọng là phải biết liệu một SLA có được đưa ra hay không và lập kế hoạch tương ứng. Nếu bạn cần đảm bảo dịch vụ của mình ngừng hoạt động không quá bốn giờ mỗi quý, nhưng nhà cung cấp dịch vụ lưu trữ của bạn chỉ có thể đảm bảo thời gian ngừng hoạt động tối đa tám giờ mỗi quý, thì bạn phải thay đổi SLA hoặc nếu không thì nghĩ ra một giải pháp thay thế.

AWS, for example, is split into regions, which you can think of as distinct clouds. Each region is in turn split into two or more availability zones, as we discussed earlier. These availability zones are AWS's equivalent of a data center. It is essential to have services distributed across multiple availability zones, as AWS does not offer any guarantees about the availability of a single node, or even an entire availability zone. For its compute service, it offers only a 99.95% uptime over a given monthly period of the region as a whole, so you'll want to distribute your workloads across multiple availability zones inside a single region. For some people, this isn't good enough, and they run their services across multiple regions as well.

AWS, chẳng hạn, được chia thành các vùng (region), mà bạn có thể coi như các đám mây riêng biệt. Mỗi vùng đến lượt nó lại được chia thành hai hoặc nhiều vùng sẵn sàng, như chúng ta đã thảo luận trước đó. Các vùng sẵn sàng này là thứ tương đương của AWS với một trung tâm dữ liệu. Việc có các dịch vụ được phân bố trên nhiều vùng sẵn sàng là điều thiết yếu, vì AWS không đưa ra bất kỳ đảm bảo nào về tính sẵn sàng của một node đơn lẻ, hoặc thậm chí của cả một vùng sẵn sàng. Đối với dịch vụ tính toán của mình, AWS chỉ cung cấp 99,95% thời gian hoạt động trong một giai đoạn hằng tháng nhất định của toàn bộ vùng, vậy nên bạn sẽ muốn phân bố các workload của mình trên nhiều vùng sẵn sàng bên trong một vùng duy nhất. Với một số người, điều này vẫn chưa đủ tốt, và họ chạy các dịch vụ của mình trên nhiều vùng nữa.

It should be noted, of course, that because providers give you an SLA “guarantee,” they will tend to limit their liability! If they missing their targets results in you losing customers and a large amount of money, you might find yourself searching through contracts to see if you can claw anything back from them. Therefore, I would strongly suggest you understand the impact of a supplier failing in its obligations to you, and work out if you need to have a plan B (or C) in your pocket. More than one client I've worked with has had a disaster recovery hosting platform with a different supplier, for example, to ensure they weren't too vulnerable to the mistakes of one company.

Tất nhiên, cần lưu ý rằng vì các nhà cung cấp cho bạn một “sự đảm bảo” SLA, họ sẽ có xu hướng giới hạn trách nhiệm pháp lý của mình! Nếu việc họ không đạt mục tiêu khiến bạn mất khách hàng và một khoản tiền lớn, bạn có thể thấy mình đang lục tìm trong các hợp

đồng để xem có thể đòi lại được gì từ họ không. Do đó, tôi sẽ mạnh mẽ khuyên bạn hiểu tác động của việc một nhà cung cấp không thực hiện nghĩa vụ với bạn, và tìm ra xem bạn có cần có một kế hoạch B (hoặc C) trong túi hay không. Hơn một khách hàng mà tôi từng làm việc cùng đã có một nền tảng lưu trữ khắc phục thảm họa với một nhà cung cấp khác, chẳng hạn, để đảm bảo họ không quá dễ tổn thương trước những sai lầm của một công ty.

CAP Theorem — Định lý CAP

We'd like to have it all, but unfortunately we know we can't. And when it comes to distributed systems like those we build using microservice architectures, we even have a mathematical proof that tells us we can't. You may well have heard about the **CAP theorem**, especially in discussions about the merits of various different types of data stores. At its heart it tells us that in a distributed system, we have three things we can trade off against each other: consistency, availability, and partition tolerance. Specifically, the theorem tells us that we get to keep two in a failure mode.

Chúng ta muốn có tất cả, nhưng tiếc thay chúng ta biết là không thể. Và khi nói đến các hệ thống phân tán như những hệ thống mà chúng ta xây dựng bằng các kiến trúc microservice, chúng ta thậm chí còn có một chứng minh toán học nói cho chúng ta biết là không thể. Bạn rất có thể đã nghe nói về định lý CAP, đặc biệt trong các cuộc thảo luận về ưu điểm của các loại kho dữ liệu khác nhau. Ở cốt lõi của nó, định lý nói cho chúng ta rằng trong một hệ thống phân tán, chúng ta có ba thứ có thể đánh đổi lẫn nhau: tính nhất quán (consistency), tính sẵn sàng (availability), và khả năng chịu phân vùng (partition tolerance). Cụ thể, định lý nói cho chúng ta rằng chúng ta chỉ được giữ hai trong số đó ở trạng thái sự cố.

Consistency is the system characteristic by which we will get the same answer if we go to multiple nodes. Availability means that every request receives a response. Partition tolerance is the system's ability to handle the fact that communication between its parts is sometimes impossible.

Tính nhất quán là đặc tính của hệ thống theo đó chúng ta sẽ nhận được cùng một câu trả lời nếu chúng ta truy cập nhiều node. Tính sẵn sàng nghĩa là mọi yêu cầu đều nhận được một phản hồi. Khả năng chịu phân vùng là khả năng của hệ thống xử lý được thực tế rằng việc giao tiếp giữa các phần của nó đôi khi là bất khả thi.

Since Eric Brewer published his original conjecture, the idea has gained a mathematical proof. I'm not going to dive into the math of the proof itself, as not only is this not that sort of book, but I can also guarantee that I would get it wrong. Instead, let's use some worked examples that will help us understand that under it all, the CAP theorem is a distillation of a very logical set of reasoning.

Kể từ khi Eric Brewer công bố giả thuyết ban đầu của mình, ý tưởng này đã có được một chứng minh toán học. Tôi sẽ không lao vào phần toán của bản thân chứng minh, vì không chỉ đây không phải là loại sách đó, mà tôi còn có thể đảm bảo rằng tôi sẽ làm sai. Thay vào đó, hãy dùng một số ví dụ đã được giải để giúp chúng ta hiểu rằng, xét cho cùng, định lý CAP là một sự cô đọng của một tập lập luận rất logic.

Let's imagine that our Inventory microservice is deployed across two separate data centers, as shown in Figure 12-7. Backing our service instance in each data center is a database, and these two databases talk to each other to try to synchronize data between them. Reads and writes are done via the local database node, and replication is used to synchronize the data between the nodes.

Hãy hình dung rằng microservice Inventory của chúng ta được triển khai trên hai trung tâm dữ liệu riêng biệt, như được thể hiện ở Hình 12-7. Đứng sau thực thể dịch vụ của chúng ta trong mỗi trung tâm dữ liệu là một cơ sở dữ liệu, và hai cơ sở dữ liệu này nói chuyện

với nhau để cố đồng bộ dữ liệu giữa chúng. Việc đọc và ghi được thực hiện qua node cơ sở dữ liệu cục bộ, và sao chép (replication) được dùng để đồng bộ dữ liệu giữa các node.

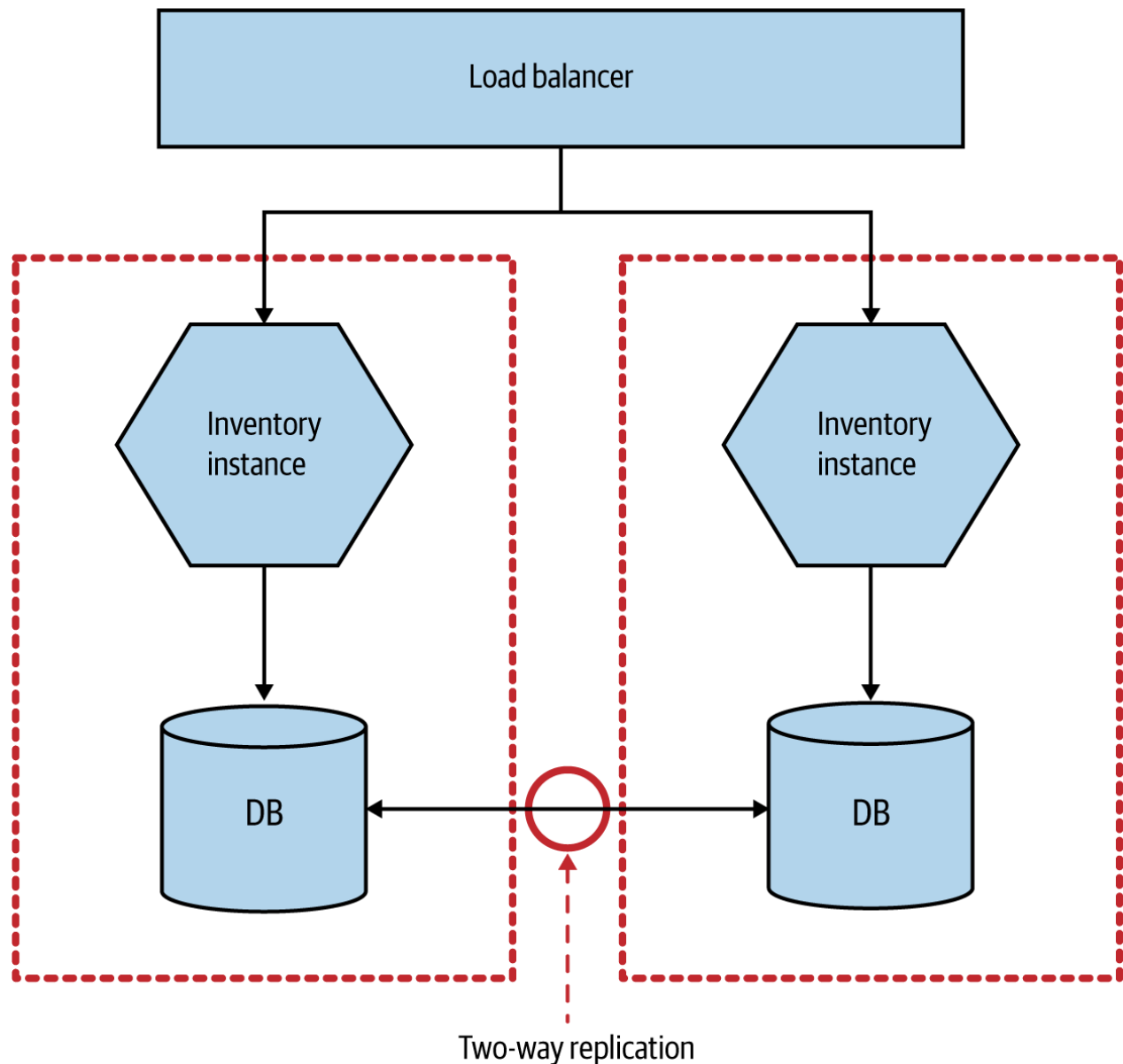


Figure 12-7. Using multiprimary replication to share data between two database nodes Now let's think about what happens when something fails. Imagine that something as simple as the network link between the two data centers stops working. The synchronization at this point fails. Writes made to the primary database in DC1 will not propagate to DC2, and vice versa. Most databases that support these setups also support some sort of queuing technique to ensure that we can recover from this afterward, but what happens in the meantime?

Hình 12-7. Dùng sao chép đa-primary (multiprimary) để chia sẻ dữ liệu giữa hai node cơ sở dữ liệu Bây giờ hãy nghĩ về điều gì xảy ra khi có gì đó hỏng. Hãy hình dung rằng một thứ đơn giản như liên kết mạng giữa hai trung tâm dữ liệu ngừng hoạt động. Việc đồng bộ tại thời điểm này thất bại. Các lần ghi được thực hiện vào cơ sở dữ liệu primary ở DC1 sẽ không lan truyền sang DC2, và ngược lại. Hầu hết các cơ sở dữ liệu hỗ trợ những thiết lập này cũng hỗ trợ một loại kỹ thuật xếp hàng nào đó để đảm bảo rằng chúng ta có thể phục

hồi sau đó, nhưng trong lúc đó điều gì xảy ra?

Sacrificing Consistency — Hy sinh tính nhất quán

Let's assume that we don't shut the Inventory microservice down entirely. If I make a change now to the data in DC1, the database in DC2 doesn't see it. This means any requests made to our inventory node in DC2 see potentially stale data. In other words, our system is still available in that both nodes are able to serve requests, and we have kept the system running despite the partition, but we have lost consistency; we don't get to keep all three traits. This is often called an AP system, because of its availability and partition tolerance.

Hãy giả định rằng chúng ta không hoàn toàn tắt microservice Inventory đi. Nếu bây giờ tôi thực hiện một thay đổi đối với dữ liệu ở DC1, cơ sở dữ liệu ở DC2 không thấy nó. Điều này nghĩa là bất kỳ yêu cầu nào được thực hiện tới node inventory của chúng ta ở DC2 sẽ thấy dữ liệu có khả năng đã cũ. Nói cách khác, hệ thống của chúng ta vẫn sẵn sàng ở chỗ cả hai node đều có thể phục vụ yêu cầu, và chúng ta đã giữ cho hệ thống chạy bất chấp phân vùng, nhưng chúng ta đã mất tính nhất quán; chúng ta không được giữ cả ba đặc tính. Điều này thường được gọi là một hệ thống AP, vì tính sẵn sàng và khả năng chịu phân vùng của nó.

During this partition, if we keep accepting writes, then we accept the fact that at some point in the future they have to be resynchronized. The longer the partition lasts, the more difficult this resynchronization can become.

Trong suốt phân vùng này, nếu chúng ta tiếp tục chấp nhận các lần ghi, thì chúng ta chấp nhận sự thật rằng tại một thời điểm nào đó trong tương lai chúng phải được đồng bộ lại. Phân vùng kéo dài càng lâu, việc đồng bộ lại này càng có thể trở nên khó khăn.

The reality is that even if we don't have a network failure between our database nodes, replication of data is not instantaneous. As touched on earlier, systems that are happy to cede consistency to keep partition tolerance and availability are said to be eventually consistent; that is, we expect at some point in the future that all nodes will see the updated data, but it won't happen at once, so we have to live with the possibility that users see old data.

Thực tế là ngay cả khi chúng ta không có sự cố mạng giữa các node cơ sở dữ liệu của mình, việc sao chép dữ liệu không tức thời. Như đã chạm đến trước đó, các hệ thống sẵn lòng nhường tính nhất quán để giữ khả năng chịu phân vùng và tính sẵn sàng được gọi là nhất quán cuối cùng (eventually consistent); nghĩa là, chúng ta kỳ vọng tại một thời điểm nào đó trong tương lai rằng tất cả các node sẽ thấy dữ liệu đã được cập nhật, nhưng nó sẽ không xảy ra cùng một lúc, vậy nên chúng ta phải sống chung với khả năng người dùng thấy dữ liệu cũ.

Sacrificing Availability — Hy sinh tính sẵn sàng

What happens if we need to keep consistency and want to drop something else instead? Well, to keep consistency, each database node needs to know the copy of the data it has is the same as the other database node. Now in the partition, if the database nodes can't talk to each other, they cannot coordinate to ensure consistency. We are unable to guarantee consistency, so our only option is to refuse to respond to the request. In other words, we have sacrificed availability. Our system is consistent and partition tolerant, or CP. In this mode our service would have to work out how to degrade functionality until the partition is healed and the database nodes can be resynchronized.

Điều gì xảy ra nếu chúng ta cần giữ tính nhất quán và muốn bỏ một thứ khác thay vào đó? Vâng, để giữ tính nhất quán, mỗi node cơ sở dữ liệu cần biết bản sao dữ liệu mà nó có là giống với node cơ sở dữ liệu kia. Bây giờ trong phân vùng, nếu các node cơ sở dữ liệu không thể nói chuyện với nhau, chúng không thể phối hợp để đảm bảo tính nhất quán. Chúng ta không thể đảm bảo tính nhất quán, vậy nên lựa chọn duy nhất của chúng ta là từ chối phản hồi yêu cầu. Nói cách khác, chúng ta đã hy sinh tính sẵn sàng. Hệ thống của chúng ta là nhất quán và chịu phân vùng, hay CP. Ở chế độ này, dịch vụ của chúng ta sẽ phải tìm ra cách suy giảm chức năng cho đến khi phân vùng được hàn gắn và các node cơ sở dữ liệu có thể được đồng bộ lại.

Consistency across multiple nodes is really hard. There are few things (perhaps nothing) harder in distributed systems. Think about it for a moment. Imagine I want to read a record from the local database node. How do I know it is up to date? I have to go and ask the other node. But I also have to ask that database node to not allow it to be updated while the read completes; in other words, I need to initiate a transactional read across multiple database nodes to ensure consistency. But in general people don't do transactional reads, do they? Because transactional reads are slow. They require locks. A read can block an entire system.

Tính nhất quán trên nhiều node thực sự khó. Có rất ít thứ (có lẽ là không có gì) khó hơn trong các hệ thống phân tán. Hãy nghĩ về nó một chút. Hãy hình dung tôi muốn đọc một bản ghi từ node cơ sở dữ liệu cục bộ. Làm sao tôi biết nó được cập nhật? Tôi phải đi hỏi node kia. Nhưng tôi cũng phải yêu cầu node cơ sở dữ liệu đó không cho phép nó được cập nhật trong khi việc đọc hoàn thành; nói cách khác, tôi cần khởi tạo một lần đọc giao dịch trên nhiều node cơ sở dữ liệu để đảm bảo tính nhất quán. Nhưng nói chung mọi người không thực hiện các lần đọc giao dịch, đúng không? Bởi vì các lần đọc giao dịch thì chậm. Chúng đòi hỏi khóa (lock). Một lần đọc có thể chặn cả một hệ thống.

As we've already discussed, distributed systems have to expect failure. Consider our transactional read across a set of consistent nodes. I ask a remote node to lock a given record while the read is initiated. I complete the read and ask the remote node to release its lock, but now I can't talk to it. What happens now? Locks are really hard to get right even in a single process system and are significantly more difficult to implement well in a distributed system.

Như chúng ta đã thảo luận, các hệ thống phân tán phải kỳ vọng sự cố. Hãy xét lần đọc giao dịch của chúng ta trên một tập các node nhất quán. Tôi yêu cầu một node từ xa khóa một bản ghi nhất định trong khi việc đọc được khởi tạo. Tôi hoàn thành việc đọc và yêu cầu node từ xa giải phóng khóa của nó, nhưng giờ tôi không thể nói chuyện với nó. Điều gì xảy ra bây giờ? Khóa thực sự khó làm cho đúng ngay cả trong một hệ thống đơn tiến trình và khó triển khai tốt một cách đáng kể hơn nhiều trong một hệ thống phân tán.

Remember when we talked about distributed transactions in Chapter 6 ? The core reason they are challenging is because of this problem with ensuring consistency across multiple nodes.

Còn nhớ khi chúng ta nói về các giao dịch phân tán ở Chương 6 không? Lý do cốt lõi khiến chúng đầy thách thức là vì chính vấn đề này với việc đảm bảo tính nhất quán trên nhiều node.

Getting multinode consistency right is so hard that I would strongly, strongly suggest that if you need it, don't try to invent it yourself. Instead, pick a data store or lock service that offers these characteristics. Consul, for example, which we discussed in "Dynamic Service Registries" on page 159 , implements a strongly consistent key-value store designed to share configuration between multiple nodes. Along with "Friends don't let friends write their own crypto" should go "Friends don't let friends write their own distributed consistent data store." If you think you need to write your own CP data store, read all the papers on the subject first, then get a PhD, and then look forward to spending

a few years getting it wrong. Meanwhile, I'll be using something off the shelf that does it for me, or more likely trying really hard to build eventually consistent AP systems instead.

Việc làm cho tính nhất quán đa-node đúng khó đến mức tôi sẽ rất, rất mạnh mẽ gợi ý rằng nếu bạn cần nó, đừng cố tự phát minh ra nó. Thay vào đó, hãy chọn một kho dữ liệu hoặc một dịch vụ khóa cung cấp những đặc tính này. Consul, chẳng hạn, mà chúng ta đã thảo luận ở mục “Dynamic Service Registries” trang 159, triển khai một kho khóa-giá trị nhất quán mạnh được thiết kế để chia sẻ cấu hình giữa nhiều node. Bên cạnh “Bạn bè không để bạn bè tự viết crypto của mình” thì nên có thêm “Bạn bè không để bạn bè tự viết kho dữ liệu nhất quán phân tán của mình.” Nếu bạn nghĩ mình cần viết một kho dữ liệu CP của riêng mình, hãy đọc tất cả các bài báo về chủ đề này trước, rồi lấy một bằng tiến sĩ, và sau đó hãy trông đợi việc dành vài năm để làm sai. Trong lúc đó, tôi sẽ dùng một thứ có sẵn làm điều đó thay tôi, hoặc nhiều khả năng hơn là cố gắng thật sự để xây dựng các hệ thống AP nhất quán cuối cùng thay vào đó.

Sacrificing Partition Tolerance? — Hy sinh khả năng chịu phân vùng?

We get to pick two, right? So we've got our eventually consistent AP system. We have our consistent but hard-to-build-and-scale CP system. Why not a CA system? Well, how can we sacrifice partition tolerance? If our system has no partition tolerance, it can't run over a network. In other words, it needs to be a single process operating locally. CA systems don't exist in distributed systems.

Chúng ta được chọn hai, đúng không? Vậy là chúng ta đã có hệ thống AP nhất quán cuối cùng. Chúng ta có hệ thống CP nhất quán nhưng khó-xây-dựng-và-mở-rộng. Tại sao không có một hệ thống CA? Vâng, làm sao chúng ta có thể hy sinh khả năng chịu phân vùng? Nếu hệ thống của chúng ta không có khả năng chịu phân vùng, nó không thể chạy qua một mạng. Nói cách khác, nó cần là một tiến trình đơn hoạt động cục bộ. Các hệ thống CA không tồn tại trong các hệ thống phân tán.

AP or CP? — AP hay CP?

Which is right, AP or CP? Well, the reality is that it depends. As the people building the system, we know the trade-off exists. We know that AP systems scale more easily and are simpler to build, and we know that a CP system will require more work due to the challenges in supporting distributed consistency. But we may not understand the business impact of this trade-off. For our inventory system, if a record is out of date by five minutes, is that OK? If the answer is yes, an AP system might be the answer. But what about the balance held for a customer in a bank? Can that be out of date? Without knowing the context in which the operation is being used, we can't know the right thing to do. Knowing about the CAP theorem helps you understand that this trade-off exists and what questions to ask.

Cái nào đúng, AP hay CP? Vâng, thực tế là còn tùy. Là những người xây dựng hệ thống, chúng ta biết sự đánh đổi tồn tại. Chúng ta biết rằng các hệ thống AP mở rộng dễ dàng hơn và đơn giản hơn để xây dựng, và chúng ta biết rằng một hệ thống CP sẽ đòi hỏi nhiều công sức hơn do những thách thức trong việc hỗ trợ tính nhất quán phân tán. Nhưng chúng ta có thể không hiểu tác động nghiệp vụ của sự đánh đổi này. Đối với hệ thống inventory của chúng ta, nếu một bản ghi cũ đi năm phút, thì có ổn không? Nếu câu trả lời là có, một hệ thống AP có thể là câu trả lời. Nhưng còn số dư được giữ cho một khách hàng trong một ngân hàng thì sao? Cái đó có thể cũ được không? Nếu không biết ngữ cảnh mà thao tác

đang được sử dụng, chúng ta không thể biết điều đúng đắn cần làm. Việc biết về định lý CAP giúp bạn hiểu rằng sự đánh đổi này tồn tại và những câu hỏi nào cần đặt ra.

It's Not All or Nothing — Không phải tất-cả-hoặc-không-có-gì

Our system as a whole doesn't need to be either AP or CP. Our catalog for MusicCorp could be AP, as we don't worry too much about a stale record. But we might decide that our inventory service needs to be CP, as we don't want to sell a customer something we don't have and then have to apologize later.

Toàn bộ hệ thống của chúng ta không cần phải hoặc là AP hoặc là CP. Catalog của MusicCorp có thể là AP, vì chúng ta không lo lắng quá nhiều về một bản ghi cũ. Nhưng chúng ta có thể quyết định rằng dịch vụ inventory của mình cần là CP, vì chúng ta không muốn bán cho một khách hàng thứ mà chúng ta không có rồi sau đó phải xin lỗi.

But individual services don't even need to be CP or AP.

Nhưng các dịch vụ riêng lẻ thậm chí không cần phải là CP hay AP.

Let's think about our Points Balance microservice, where we store records of how many loyalty points our customers have built up. We could decide that we don't care if the balance we show for a customer is stale, but that when it comes to updating a balance we need it to be consistent to ensure that customers don't use more points than they have available. Is this microservice CP, or AP, or both? Really, what we have done is push the trade-offs around the CAP theorem down to individual microservice capabilities.

Hãy nghĩ về microservice Points Balance của chúng ta, nơi chúng ta lưu các bản ghi về số điểm khách hàng thân thiết mà khách hàng của chúng ta đã tích lũy. Chúng ta có thể quyết định rằng chúng ta không quan tâm nếu số dư mà chúng ta hiển thị cho một khách hàng bị cũ, nhưng khi đến việc cập nhật một số dư thì chúng ta cần nó nhất quán để đảm bảo rằng khách hàng không dùng nhiều điểm hơn số điểm họ có sẵn. Microservice này là CP, hay AP, hay cả hai? Thực ra, những gì chúng ta đã làm là đẩy các đánh đổi xoay quanh định lý CAP xuống đến các năng lực microservice riêng lẻ.

Another complexity is that neither consistency nor availability is all or nothing. Many systems allow us a far more nuanced trade-off. For example, with Cassandra I can make different trade-offs for individual calls. So if I need strict consistency, I can perform a read that blocks until all replicas have responded, confirming the value is consistent, or until a specific quorum of replicas have responded, or even just a single node. Obviously, if I block waiting for all replicas to report back and one of them is unavailable, I'll be **blocking** for a long time. On the other hand, if I wanted my read to respond as quickly as possible, I might wait to hear back from only a single node—in which case there is a possibility that this might be an inconsistent view of my data.

Một sự phức tạp khác là tính nhất quán cũng như tính sẵn sàng đều không phải là tất-cả-hoặc-không-có-gì. Nhiều hệ thống cho phép chúng ta một sự đánh đổi nhiều sắc thái hơn nhiều. Ví dụ, với Cassandra tôi có thể thực hiện những đánh đổi khác nhau cho các lời gọi riêng lẻ. Vậy nên nếu tôi cần tính nhất quán nghiêm ngặt, tôi có thể thực hiện một lần đọc chặn cho đến khi tất cả các bản sao đã phản hồi, xác nhận giá trị là nhất quán, hoặc cho đến khi một số quorum cụ thể của các bản sao đã phản hồi, hoặc thậm chí chỉ một node duy nhất. Hiển nhiên, nếu tôi chặn để chờ tất cả các bản sao báo cáo lại và một trong số chúng không sẵn sàng, tôi sẽ bị chặn trong một thời gian dài. Mặt khác, nếu tôi muốn lần đọc của mình phản hồi nhanh nhất có thể, tôi có thể chờ nghe lại chỉ từ một node duy

nhất—trong trường hợp đó có khả năng đây là một góc nhìn không nhất quán về dữ liệu của tôi.

You'll often see posts about people “beating” the CAP theorem. They haven't. What they have done is create a system in which some capabilities are CP, and some are AP. The mathematical proof behind the CAP theorem holds.

Bạn sẽ thường thấy các bài viết về việc người ta “đánh bại” định lý CAP. Họ chưa làm được. Cái họ đã làm là tạo ra một hệ thống trong đó một số năng lực là CP, và một số là AP. Chứng minh toán học đằng sau định lý CAP vẫn đứng vững.

And the Real World — Và thế giới thực

Much of what we've talked about is the electronic world—bits and bytes stored in memory. We talk about consistency in an almost childlike fashion; we imagine that within the scope of the system we have created, we can stop the world and have it all make sense. And yet so much of what we build is just a reflection of the real world, and we don't get to control that, do we?

Phần lớn những gì chúng ta đã nói đến là thế giới điện tử—các bit và byte được lưu trong bộ nhớ. Chúng ta nói về tính nhất quán theo một kiểu gần như trẻ con; chúng ta hình dung rằng trong phạm vi hệ thống mà chúng ta đã tạo ra, chúng ta có thể dừng thế giới lại và làm cho tất cả đều hợp lý. Vậy mà rất nhiều thứ chúng ta xây dựng chỉ là một sự phản ánh của thế giới thực, và chúng ta không được kiểm soát điều đó, đúng không?

Let's revisit our inventory system. This maps to real-world, physical items. We keep a count in our system of how many albums we have in MusicCorp's warehouses. At the start of the day we had 100 copies of Give Blood by the Brakes. We sold one. Now we have 99 copies. Easy, right? But what happens if when the order is being sent out, someone knocks a copy of the album onto the floor and it gets stepped on and broken? What happens now? Our systems say there are 99 copies on the shelf, but there are actually only 98.

Hãy quay lại hệ thống inventory của chúng ta. Cái này ánh xạ tới các mặt hàng vật lý, có thật. Chúng ta giữ trong hệ thống của mình một con số đếm xem chúng ta có bao nhiêu album trong các kho hàng của MusicCorp. Vào đầu ngày, chúng ta có 100 bản của Give Blood của nhóm Brakes. Chúng ta bán một bản. Bây giờ chúng ta có 99 bản. Dễ, đúng không? Nhưng điều gì xảy ra nếu khi đơn hàng đang được gửi đi, ai đó làm rơi một bản của album xuống sàn và nó bị giẫm lên và vỡ? Điều gì xảy ra bây giờ? Hệ thống của chúng ta nói rằng có 99 bản trên kệ, nhưng thực ra chỉ có 98.

What if we made our inventory system AP instead, and we occasionally had to contact a user later on and tell him that one of his items is actually out of stock? Would that be the worst thing in the world? It would certainly be much easier to build and scale it and ensure it is correct.

Sẽ thế nào nếu thay vào đó chúng ta làm cho hệ thống inventory của mình là AP, và thì thoảng chúng ta phải liên hệ với một người dùng sau này và nói với họ rằng một trong các mặt hàng của họ thực ra đã hết hàng? Liệu đó có phải là điều tồi tệ nhất trên đời không? Chắc chắn việc xây dựng và mở rộng nó và đảm bảo nó đúng sẽ dễ hơn nhiều.

We have to recognize that no matter how consistent our systems might be in and of themselves, they cannot know everything that happens, especially when we're keeping records of the real world. This is one of the main reasons that AP systems end up being the right call in many situations. Aside from the complexity of building CP systems, they can't fix all our problems anyway.

Chúng ta phải thừa nhận rằng dù hệ thống của chúng ta có nhất quán đến đâu trong và của chính nó, chúng cũng không thể biết mọi thứ xảy ra, đặc biệt khi chúng ta đang giữ các bản ghi về thế giới thực. Đây là một trong những lý do chính khiến các hệ thống AP rất cuộc lại là lựa chọn đúng trong nhiều tình huống. Ngoài độ phức tạp của việc xây dựng các hệ thống CP, dù sao thì chúng cũng không thể giải quyết được mọi vấn đề của chúng ta.

Antifragility — Tính kháng vỡ (Antifragility)

In the first edition, I talked about the concept of antifragile, as popularized by Nassim Taleb. This concept describes how systems actually benefit from failure and disorder and was highlighted as being inspiration for how some parts of Netflix operated, specifically with respect to concepts like chaos engineering. When looking more broadly at the concept of resiliency, though, we realize that antifragility is just a subset of the concept of resiliency. When we consider the concepts of graceful extensibility and sustained adaptability, which we introduced earlier, this becomes clear.

Trong ấn bản đầu tiên, tôi đã nói về khái niệm kháng vỡ (antifragile), được Nassim Taleb phổ biến. Khái niệm này mô tả cách các hệ thống thực sự hưởng lợi từ sự cố và sự hỗn loạn, và được nêu bật là nguồn cảm hứng cho cách một số bộ phận của Netflix vận hành, cụ thể là về các khái niệm như chaos engineering. Tuy nhiên, khi nhìn rộng hơn vào khái niệm khả năng phục hồi, chúng ta nhận ra rằng tính kháng vỡ chỉ là một tập con của khái niệm khả năng phục hồi. Khi chúng ta xét đến các khái niệm khả năng mở rộng có kiểm soát và khả năng thích ứng bền vững, mà chúng ta đã giới thiệu trước đó, điều này trở nên rõ ràng.

I think that when antifragility became, briefly, a hyped concept in IT, it did so against a backdrop of us thinking narrowly about resiliency—where we thought only about robustness, and maybe about rebound, but ignored the rest. With the field of resilience engineering now gaining more recognition and traction, it seems appropriate to move beyond the term antifragile, while still ensuring that we highlight some of the ideas behind it that are well and truly part of resiliency as a whole.

Tôi nghĩ rằng khi tính kháng vỡ trở thành, trong một thời gian ngắn, một khái niệm được thổi phồng trong IT, nó làm vậy trên một bối cảnh chúng ta đang suy nghĩ hạn hẹp về khả năng phục hồi—nơi chúng ta chỉ nghĩ về độ vững chắc, và có lẽ về bật lại, nhưng bỏ qua phần còn lại. Với lĩnh vực kỹ thuật phục hồi giờ đây đang giành được nhiều sự công nhận và lực kéo hơn, có vẻ thích hợp khi vượt ra ngoài thuật ngữ kháng vỡ, đồng thời vẫn đảm bảo rằng chúng ta nêu bật một số ý tưởng đằng sau nó vốn thực sự và đích thực là một phần của khả năng phục hồi nói chung.

Chaos Engineering — Chaos engineering

Since the first edition of this book, another technique that has gained more attention is chaos engineering. Named for practices used at Netflix, it can be a useful approach to help improve your resiliency—either in terms of ensuring your systems are as robust as you think they are, or else as part of an approach toward the sustained adaptability of your system.

Kể từ ấn bản đầu tiên của cuốn sách này, một kỹ thuật khác đã giành được nhiều sự chú ý hơn là chaos engineering. Được đặt tên theo các thực hành được dùng tại Netflix, nó có thể là một cách tiếp cận hữu ích để giúp cải thiện khả năng phục hồi của bạn—hoặc về mặt đảm bảo rằng các hệ thống của bạn vững chắc như bạn nghĩ, hoặc như một phần của một cách tiếp cận hướng tới khả năng thích ứng bền vững của hệ thống.

Originally inspired by work that Netflix was doing internally, the term chaos engineering struggles somewhat due to confusing explanations of what it means. For many, it means “run a tool on my software and see what it says,” something arguably not helped by the fact that many of the most vocal proponents of chaos engineering are often themselves selling tools to run on your software to do, well, chaos engineering.

Ban đầu được truyền cảm hứng từ công việc mà Netflix đang làm nội bộ, thuật ngữ chaos engineering có phần gặp khó khăn do những lời giải thích gây nhầm lẫn về ý nghĩa của nó. Với nhiều người, nó có nghĩa là “chạy một công cụ trên phần mềm của tôi và xem nó nói gì,” một điều mà có thể nói là không được giúp gì bởi thực tế rằng nhiều người ủng hộ chaos engineering lớn tiếng nhất thường chính họ đang bán các công cụ để chạy trên phần mềm của bạn nhằm làm, ừm, chaos engineering.

Getting a clear definition of what chaos engineering means to its practitioners is difficult. The best definition (at least to my mind) that I’ve come across is this:

Việc có được một định nghĩa rõ ràng về chaos engineering có nghĩa là gì đối với những người thực hành nó là khó. Định nghĩa hay nhất (ít nhất là theo tôi) mà tôi từng gặp là thế này:

Chaos Engineering is the discipline of experimenting on a system in order to build confidence in the system’s capability to withstand turbulent conditions in production. — Principles of Chaos Engineering

Chaos Engineering là kỷ luật thực nghiệm trên một hệ thống nhằm xây dựng sự tự tin vào năng lực của hệ thống chịu đựng được những điều kiện hỗn loạn trong môi trường sản xuất. — Principles of Chaos Engineering

Now the word system here is doing a lot of work. Some will view this narrowly as the software and hardware components. But in the context of resiliency engineering, it’s important that we see the system as being the entirety of the people, the processes, the culture, and yes, the software and infrastructure that go into creating our product. This means we should view chaos engineering more broadly than just “let’s turn off some machines and see what happens.”

Bây giờ từ hệ thống ở đây đang gánh rất nhiều ý nghĩa. Một số người sẽ nhìn nó một cách hạn hẹp như các thành phần phần mềm và phần cứng. Nhưng trong ngữ cảnh kỹ thuật phục hồi, điều quan trọng là chúng ta xem hệ thống là toàn bộ con người, các quy trình, văn hóa, và đúng vậy, cả phần mềm và hạ tầng tạo nên sản phẩm của chúng ta. Điều này có nghĩa là chúng ta nên xem chaos engineering rộng hơn là chỉ “hãy tắt vài cỗ máy đi và xem điều gì xảy ra.”

Game Days — Game Days

Well before chaos engineering had its name, people would run Game Day exercises to test people’s preparedness for certain events. Planned in advance but ideally launched in surprise (for the participants), this gives you the chance to test your people and processes in the wake of a realistic but fictional situation. During my time at Google, this was a fairly common occurrence for various systems, and I certainly think that many organizations could benefit from having these sorts of exercises regularly. Google goes beyond simple tests to mimic server failure, and as part of its DiRT (Disaster Recovery Test) exercises it has simulated large-scale disasters such as earthquakes. 6

Từ rất lâu trước khi chaos engineering có tên gọi của nó, người ta đã chạy các bài tập Game Day để kiểm tra mức độ sẵn sàng của con người trước những sự kiện nhất định. Được lên kế hoạch từ trước nhưng lý tưởng là được khởi động một cách bất ngờ (đối với người tham

gia), điều này cho bạn cơ hội kiểm tra con người và quy trình của mình sau một tình huống thực tế nhưng hư cấu. Trong thời gian tôi ở Google, đây là một việc khá phổ biến đối với nhiều hệ thống khác nhau, và tôi chắc chắn nghĩ rằng nhiều tổ chức có thể hưởng lợi từ việc có những bài tập kiểu này một cách thường xuyên. Google còn đi xa hơn các bài kiểm tra đơn giản để mô phỏng sự cố máy chủ, và như một phần của các bài tập DiRT (Disaster Recovery Test, Kiểm tra Khắc phục Thảm họa), họ đã mô phỏng các thảm họa quy mô lớn như động đất. 6

Game Days can be used to probe suspected sources of weakness in the system. In his book, *Learning Chaos Engineering*, 7 Russ Miles shares the example of a Game Day exercise he facilitated that had been designed to examine, in part, the overreliance on a single staff member named Bob. For the Game Day, Bob was sequestered in a room and unable to help the team during the simulated outage. Bob was observing, though, and ended up having to step in when the team, in its attempt to fix issues with the “fake” system, ended up logging into production by mistake and was in the process of destroying production data. One can only assume that lots of lessons were learned in the aftermath of that exercise.

Game Day có thể được dùng để thăm dò những nguồn yếu kém bị nghi ngờ trong hệ thống. Trong cuốn sách của mình, *Learning Chaos Engineering*, 7 Russ Miles chia sẻ ví dụ về một bài tập Game Day mà ông điều phối, được thiết kế để xem xét, một phần, sự phụ thuộc quá mức vào một nhân viên duy nhất tên là Bob. Trong Game Day đó, Bob bị cách ly trong một căn phòng và không thể giúp đội trong suốt sự cố ngừng hoạt động được mô phỏng. Tuy nhiên, Bob vẫn đang quan sát, và rốt cuộc đã phải nhảy vào khi đội, trong nỗ lực sửa các vấn đề với hệ thống “giả,” cuối cùng đã đăng nhập vào môi trường sản xuất do nhầm lẫn và đang trong quá trình phá hủy dữ liệu sản xuất. Người ta chỉ có thể đoán rằng rất nhiều bài học đã được rút ra sau bài tập đó.

6 Kripa Krishnan, “Weathering the Unexpected,” *acmqueue* 10, no. 9 (2012), <https://oreil.ly/BCSQ7>.

6 Kripa Krishnan, “Weathering the Unexpected,” *acmqueue* 10, no. 9 (2012), <https://oreil.ly/BCSQ7>.

7 Russ Miles, *Learning Chaos Engineering* (Sebastopol: O’Reilly, 2019).

7 Russ Miles, *Learning Chaos Engineering* (Sebastopol: O’Reilly, 2019).

Production Experiments — Thực nghiệm trên môi trường sản xuất

The scale at which Netflix operates is well known, as is the fact that Netflix is based entirely on the AWS infrastructure. These two factors mean that it has to embrace failure well. Netflix realized that planning for failure and actually knowing your software will handle that failure when it occurs are two different things. To that end, Netflix actually incites failure to ensure that its systems are tolerant of failure by running tools on its systems.

Quy mô mà Netflix vận hành là điều nổi tiếng, cũng như thực tế rằng Netflix hoàn toàn dựa trên hạ tầng AWS. Hai yếu tố này có nghĩa là họ phải đón nhận sự cố một cách tốt đẹp. Netflix nhận ra rằng lên kế hoạch cho sự cố và thực sự biết phần mềm của bạn sẽ xử lý được sự cố đó khi nó xảy ra là hai chuyện khác nhau. Để đạt được điều đó, Netflix thực ra kích động sự cố để đảm bảo rằng các hệ thống của họ chịu đựng được sự cố bằng cách chạy các công cụ trên các hệ thống của mình.

The most famous of these tools is Chaos Monkey, which during certain hours of the day turns off random machines in production. Knowing that this can and will happen in production means that the developers who create the systems really have to be prepared for it. Chaos Monkey is just one part of Netflix's Simian Army of failure bots. Chaos Gorilla is used to take out an entire availability zone (the AWS equivalent of a data center), whereas Latency Monkey simulates slow network connectivity between machines. For many, the ultimate test of whether your system really is robust might be unleashing your very own Simian Army on your production infrastructure.

Công cụ nổi tiếng nhất trong số này là Chaos Monkey, công cụ mà trong một số giờ nhất định trong ngày sẽ tắt các cỗ máy ngẫu nhiên trong môi trường sản xuất. Việc biết rằng điều này có thể và sẽ xảy ra trong môi trường sản xuất có nghĩa là các nhà phát triển tạo ra các hệ thống thực sự phải chuẩn bị cho nó. Chaos Monkey chỉ là một phần của Simian Army (Đội quân Khỉ) gồm các bot gây sự cố của Netflix. Chaos Gorilla được dùng để hạ gục cả một vùng sẵn sàng (thứ tương đương của AWS với một trung tâm dữ liệu), trong khi Latency Monkey mô phỏng kết nối mạng chậm giữa các cỗ máy. Với nhiều người, bài kiểm tra tối thượng xem hệ thống của bạn có thực sự vững chắc hay không có thể là tung Simian Army của riêng bạn vào hạ tầng sản xuất của mình.

From Robustness to Beyond — Từ độ vững chắc đến xa hơn

Applied in its narrowest form, chaos engineering could be a useful activity in terms of improving the robustness of our application. Remember, robustness in the context of resilience engineering means the extent to which our system can handle expected problems. Netflix knew it couldn't rely on any given virtual machine being available in its production environment, so it built Chaos Monkey to ensure that its system could survive this expected issue.

Được áp dụng ở hình thức hạn hẹp nhất của nó, chaos engineering có thể là một hoạt động hữu ích về mặt cải thiện độ vững chắc của ứng dụng của chúng ta. Hãy nhớ, độ vững chắc trong ngữ cảnh kỹ thuật phục hồi nghĩa là mức độ mà hệ thống của chúng ta có thể xử lý các vấn đề được dự kiến. Netflix biết rằng họ không thể dựa vào việc một máy ảo bất kỳ nào đó sẽ sẵn sàng trong môi trường sản xuất của mình, vậy nên họ đã xây dựng Chaos Monkey để đảm bảo rằng hệ thống của họ có thể sống sót qua vấn đề được dự kiến này.

If you make use of chaos engineering tooling as part of an approach to continually question the resilience of your system, however, it can have far greater applicability. Using tools in this space to help answer the "what if" questions you might have, continually questioning your understanding, can have a much bigger impact. The Chaos Toolkit is an open source project to help you run experiments on your system, and it has proved very popular. Reliably, the company founded by the creators of the Chaos Toolkit, offers a broader range of tools to help with chaos engineering in general, although perhaps the best-known vendor in this space is Gremlin.

Tuy nhiên, nếu bạn tận dụng công cụ chaos engineering như một phần của một cách tiếp cận để liên tục đặt câu hỏi về khả năng phục hồi của hệ thống mình, thì nó có thể có khả năng áp dụng rộng hơn nhiều. Dùng các công cụ trong không gian này để giúp trả lời các câu hỏi "điều gì sẽ xảy ra nếu" mà bạn có thể có, liên tục đặt câu hỏi về sự hiểu biết của mình, có thể có một tác động lớn hơn nhiều. Chaos Toolkit là một dự án mã nguồn mở để giúp bạn chạy các thực nghiệm trên hệ thống của mình, và nó đã chứng tỏ rất được ưa chuộng. Reliably, công ty được sáng lập bởi những người tạo ra Chaos Toolkit, cung cấp một loạt công cụ rộng hơn để giúp với chaos engineering nói chung, mặc dù có lẽ nhà cung cấp nổi tiếng nhất trong không gian này là Gremlin.

Just remember, running a chaos engineering tool doesn't make you resilient.

Chỉ cần nhớ, việc chạy một công cụ chaos engineering không làm cho bạn có khả năng phục hồi.

Blame — Đổ lỗi

When things go wrong, there are lots of ways we can handle it. Obviously, in the immediate aftermath, our focus is on getting things back up and running, which is sensible. After that, all too often, come the recriminations. There is a default position to look for something or someone to blame. The concept of “root cause analysis” implies there is a root cause. It’s surprising how often we want that root cause to be a human.

Khi mọi thứ trục trặc, có rất nhiều cách chúng ta có thể xử lý nó. Hiển nhiên, trong những giây phút ngay sau đó, trọng tâm của chúng ta là đưa mọi thứ hoạt động trở lại, điều đó là hợp lý. Sau đó, quá thường xuyên, đến phần quy kết tội lỗi. Có một vị thế mặc định là đi tìm một cái gì đó hoặc một ai đó để đổ lỗi. Khái niệm “phân tích nguyên nhân gốc rễ” (root cause analysis) ngụ ý rằng có một nguyên nhân gốc rễ. Thật đáng ngạc nhiên khi chúng ta thường xuyên muốn nguyên nhân gốc rễ đó là một con người đến mức nào.

A few years ago, back when I was working in Australia, Telstra, the main telco (and previous monopoly) had a major outage that impacted both voice and telephony services. This incident was especially problematic due to the scope and duration of the outage. Australia has many very isolated rural communities, and outages like this tend to be especially serious. Almost immediately after the outage, Telstra’s COO put out a statement that made it clear they knew exactly what had caused the problem: 8

Vài năm trước, hồi tôi còn làm việc ở Úc, Telstra, công ty viễn thông chính (và độc quyền trước đây) đã có một sự cố ngừng hoạt động lớn ảnh hưởng đến cả dịch vụ thoại lẫn dịch vụ điện thoại. Sự cố này đặc biệt rắc rối do phạm vi và thời lượng của sự cố ngừng hoạt động. Úc có nhiều cộng đồng nông thôn rất biệt lập, và những sự cố ngừng hoạt động kiểu này có xu hướng đặc biệt nghiêm trọng. Gần như ngay sau sự cố, COO của Telstra đưa ra một tuyên bố làm rõ rằng họ biết chính xác điều gì đã gây ra vấn đề: 8

“We took that node down, unfortunately the individual that was managing that issue did not follow the correct procedure, and he reconnected the customers to the malfunctioning node, rather than transferring them to the nine other redundant nodes that he should have transferred people to,” Ms. McKenzie told reporters on Tuesday afternoon. “We apologise right across our customer base. This is an embarrassing human error.” So firstly, note that this statement came out hours after the outage. And yet Telstra had already unpacked what must be a vastly complex system to know exactly what was to blame—one person. Now, if it’s true that one person making a mistake can really bring an entire telco to its knees, you’d think that would say more about the telco than the individual. In addition, Telstra clearly signaled to its staff at the time that it was very happy to point fingers and apportion blame. 9

“Chúng tôi đã hạ node đó xuống, thật không may cá nhân đang quản lý vấn đề đó đã không tuân theo quy trình đúng, và anh ta đã kết nối lại khách hàng vào node đang trục trặc, thay vì chuyển họ sang chín node dự phòng khác mà lẽ ra anh ta nên chuyển mọi người sang,” bà McKenzie nói với các phóng viên vào chiều thứ Ba. “Chúng tôi xin lỗi trên toàn bộ cơ sở khách hàng của mình. Đây là một lỗi con người đáng xấu hổ.” Vậy nên trước hết, hãy lưu ý rằng tuyên bố này đưa ra nhiều giờ sau sự cố ngừng hoạt động. Vậy mà Telstra đã mổ xẻ một hệ thống chắc hẳn vô cùng phức tạp để biết chính xác cái gì đáng bị đổ lỗi—một con người. Bây giờ, nếu đúng là một con người mắc một sai lầm có thể thực sự khiến cả một công ty viễn thông quỳ gối, thì bạn sẽ nghĩ rằng điều đó nói lên nhiều điều về công

ty viễn thông hơn là về cá nhân đó. Ngoài ra, Telstra rõ ràng đã phát đi tín hiệu cho nhân viên của mình vào thời điểm đó rằng họ rất sẵn lòng chỉ tay và quy kết tội lỗi. 9

The problem with blaming people in the aftermath of incidents like this is that what starts as a short-term passing of the buck ends up creating a culture of fear, where people will be unwilling to come forward to tell you when things go wrong. As a result, you will lose the opportunity to learn from failure, setting yourself up for the same problems happening again. Creating an organization in which people have the safety to admit when mistakes are made is essential in creating a learning culture, and in turn can go a long way toward creating an organization that is able to create more robust software, quite aside from the obvious benefits of creating a happier place to work in.

Vấn đề với việc đổ lỗi cho con người sau những sự cố như thế này là cái khởi đầu như một sự dồn đẩy trách nhiệm trong ngắn hạn rồi cuộc lại tạo ra một văn hóa sợ hãi, nơi mọi người sẽ không sẵn lòng đứng ra để nói cho bạn biết khi có gì đó trục trặc. Kết quả là, bạn sẽ mất đi cơ hội học hỏi từ sự cố, tự đặt mình vào thế những vấn đề tương tự lại tái diễn. Tạo ra một tổ chức nơi mọi người có sự an toàn để thừa nhận khi mắc sai lầm là điều thiết yếu trong việc tạo ra một văn hóa học hỏi, và đến lượt nó có thể tiến xa trong việc tạo ra một tổ chức có khả năng tạo ra phần mềm vững chắc hơn, hoàn toàn tách biệt với những lợi ích hiển nhiên của việc tạo ra một nơi làm việc hạnh phúc hơn.

Coming back to Telstra, with the cause of the blame clearly established in the in-depth investigation that was carried out mere hours after the nationwide outage, we'd clearly expect no subsequent outages, right? Unfortunately, Telstra suffered a string

Quay lại Telstra, với nguyên nhân của sự đổ lỗi đã được xác lập rõ ràng trong cuộc điều tra chuyên sâu được tiến hành chỉ vài giờ sau sự cố ngừng hoạt động toàn quốc, rõ ràng chúng ta sẽ kỳ vọng không có sự cố ngừng hoạt động nào tiếp theo, đúng không? Tiếc thay, Telstra đã hứng chịu một chuỗi

8 Kate Aubusson and Tim Biggs, “Major Telstra Mobile Outage Hits Nationwide, with Calls and Data Affected,” Sydney Morning Herald , February 9, 2016, <https://oreil.ly/4cBcy> .

8 Kate Aubusson and Tim Biggs, “Major Telstra Mobile Outage Hits Nationwide, with Calls and Data Affected,” Sydney Morning Herald , February 9, 2016, <https://oreil.ly/4cBcy> .

9 I wrote more about the Telstra incident at the time—“Telstra, Human Error and Blame Culture,” <https://oreil.ly/OXgUQ> . When I wrote that blog post, I neglected to realize that the company I worked for had Telstra as a client; my then-employer actually handled the situation extremely well, even though it did make for an uncomfortable few hours when some of my comments got picked up by the national press.

9 Tôi đã viết nhiều hơn về sự cố Telstra vào thời điểm đó—“Telstra, Human Error and Blame Culture,” <https://oreil.ly/OXgUQ> . Khi tôi viết bài blog đó, tôi đã sơ suất không nhận ra rằng công ty tôi làm việc có Telstra là một khách hàng; chủ cũ của tôi thực ra đã xử lý tình huống cực kỳ tốt, mặc dù nó quả thực đã tạo nên vài giờ khó chịu khi một số bình luận của tôi được báo chí quốc gia đăng lại.

of subsequent outages. More human error? Perhaps Telstra thought so—in the wake of the series of incidents, the COO resigned.

các sự cố ngừng hoạt động tiếp theo. Lại do lỗi con người? Có lẽ Telstra đã nghĩ vậy—sau loạt sự cố, COO đã từ chức.

For a more informed take on how to create an organization where you can get the best out of mistakes and create a nicer environment for your staff as well, John Allspaw’s “Blameless Post-Mortems and a Just Culture” is a great jumping-off point. 10

Để có một góc nhìn am hiểu hơn về cách tạo ra một tổ chức nơi bạn có thể khai thác được điều tốt nhất từ những sai lầm và đồng thời tạo ra một môi trường tốt đẹp hơn cho nhân viên của mình, bài viết “Blameless Post-Mortems and a Just Culture” của John Allspaw là một điểm khởi đầu tuyệt vời. 10

Ultimately, as I’ve already highlighted numerous times in this chapter, resiliency requires a questioning mind—a drive to constantly examine the weaknesses in our system. This requires a culture of learning, and often the best learning can come in the wake of an incident. It’s therefore vital that you ensure that when the worst happens, you do your best to create an environment in which you can maximize the information you gather in the aftermath to reduce the chances of it happening again.

Cuối cùng, như tôi đã nhấn mạnh nhiều lần trong chương này, khả năng phục hồi đòi hỏi một tư duy hay đặt câu hỏi—một động lực để liên tục xem xét các điểm yếu trong hệ thống của chúng ta. Điều này đòi hỏi một văn hóa học hỏi, và thường thì việc học hỏi tốt nhất có thể đến sau một sự cố. Do đó, điều tối quan trọng là bạn phải đảm bảo rằng khi điều tồi tệ nhất xảy ra, bạn cố gắng hết sức để tạo ra một môi trường trong đó bạn có thể tối đa hóa thông tin mà mình thu thập được sau đó nhằm giảm khả năng nó tái diễn.

Summary — Tóm tắt

As our software becomes more vital to the lives of our users, the drive toward improving the resiliency of the software we create increases. As we’ve seen in this chapter, though, we cannot achieve resiliency just by thinking about our software and infrastructure; we also have to think about our people, processes, and organizations. In this chapter we looked at the four core concepts of resiliency, as described by David Woods:

Khi phần mềm của chúng ta trở nên thiết yếu hơn đối với cuộc sống của người dùng, động lực hướng tới việc cải thiện khả năng phục hồi của phần mềm mà chúng ta tạo ra cũng tăng lên. Tuy nhiên, như chúng ta đã thấy trong chương này, chúng ta không thể đạt được khả năng phục hồi chỉ bằng cách nghĩ về phần mềm và hạ tầng của mình; chúng ta còn phải nghĩ về con người, các quy trình, và các tổ chức của mình. Trong chương này, chúng ta đã xem xét bốn khái niệm cốt lõi của khả năng phục hồi, như được David Woods mô tả:

Robustness The ability to absorb expected perturbation

Độ vững chắc (Robustness) Khả năng hấp thụ những nhiễu loạn được dự kiến

Rebound The ability to recover after a traumatic event

Bật lại (Rebound) Khả năng phục hồi sau một sự kiện gây tổn thương

Graceful extensibility How well we deal with a situation that is unexpected

Khả năng mở rộng có kiểm soát (Graceful extensibility) Mức độ chúng ta xử lý tốt một tình huống không lường trước

Sustained adaptability The ability to continually adapt to changing environments, stakeholders, and demands

Khả năng thích ứng bền vững (Sustained adaptability) Khả năng liên tục thích ứng với những môi trường, các bên liên quan, và nhu cầu đang thay đổi

Looking more narrowly at microservices, they give us a host of ways in which we can improve the robustness of our systems. But this improved robustness is not free—you still have to decide which options to use. Key stability patterns like circuit breakers, time-outs, redundancy, isolation,

idempotency, and the like are all tools at your disposal, but you have to decide when and where to use them. Beyond these narrow

Nhìn hạn hẹp hơn vào microservice, chúng mang lại cho chúng ta cả loạt cách để có thể cải thiện độ vững chắc của các hệ thống của mình. Nhưng độ vững chắc được cải thiện này không miễn phí—bạn vẫn phải quyết định dùng lựa chọn nào. Các pattern ổn định then chốt như circuit breaker, time-out, dự phòng, cô lập, tính idempotent, và những thứ tương tự đều là công cụ trong tầm tay bạn, nhưng bạn phải quyết định khi nào và ở đâu để dùng chúng. Ngoài những

10 John Allspaw, “Blameless Post-Mortems and a Just Culture,” Code as Craft (blog), Etsy, May 22, 2012, [https:// oreil.ly/7LzmL](https://oreil.ly/7LzmL) .

10 John Allspaw, “Blameless Post-Mortems and a Just Culture,” Code as Craft (blog), Etsy, May 22, 2012, [https:// oreil.ly/7LzmL](https://oreil.ly/7LzmL) .

CHAPTER 13 Scaling — Chương 13: Mở rộng

“You’re gonna need a bigger boat.” —Chief Brody, Jaws

“Anh sẽ cần một con thuyền lớn hơn đây.” —Cảnh sát trưởng Brody, Hàm Cá Mập (Jaws)

When we scale our systems, we do so for one of two reasons. Firstly, it allows us to improve the performance of our system, perhaps by allowing us to handle more load or by improving latency. Secondly, we can scale our system to improve its **robustness**. In this chapter, we’ll look at a model to describe the different types of **scaling**, and then we’ll look in detail at how each type of scaling can be implemented using a **microservice architecture**. At the end of this chapter, you should have an array of techniques to handle the scaling issues that may come your way.

Khi mở rộng (scaling) hệ thống của mình, chúng ta làm điều đó vì một trong hai lý do. Thứ nhất, nó cho phép chúng ta cải thiện hiệu năng của hệ thống, có thể bằng cách xử lý được nhiều tải hơn hoặc bằng cách cải thiện độ trễ (latency). Thứ hai, chúng ta có thể mở rộng hệ thống để cải thiện độ vững chắc (robustness) của nó. Trong chương này, chúng ta sẽ xem xét một mô hình để mô tả các kiểu mở rộng khác nhau, rồi sau đó đi sâu vào cách triển khai từng kiểu mở rộng bằng kiến trúc microservice. Kết thúc chương này, bạn sẽ có trong tay một bộ kỹ thuật để xử lý những vấn đề mở rộng có thể gặp phải.

To start with, though, let’s look at the different types of scaling you might want to apply.

Tuy nhiên, để bắt đầu, hãy cùng xem các kiểu mở rộng khác nhau mà bạn có thể muốn áp dụng.

The Four Axes of Scaling — Bốn trục của việc mở rộng

There isn’t one right way to scale a system, as the technique used will depend on the type of constraint you might have. We have a number of different types of scaling we can bring to bear to help with performance, robustness, or perhaps both. A model I’ve often used to describe the different types of scaling is the Scale Cube from The Art of Scalability , 1 which breaks scaling down into three categories that in the context of computer systems cover **functional decomposition**, horizontal duplication, and **data partitioning**. The value of this model is that it helps you understand that you can scale a system along one, two, or all three of these axes, depending on your needs.

Không có một cách đúng duy nhất để mở rộng một hệ thống, vì kỹ thuật được dùng sẽ phụ thuộc vào loại ràng buộc mà bạn gặp phải. Chúng ta có một số kiểu mở rộng khác nhau có thể đưa vào áp dụng để hỗ trợ hiệu năng, độ vững chắc, hoặc có lẽ cả hai. Một mô hình mà tôi thường dùng để mô tả các kiểu mở rộng khác nhau là Scale Cube (Khối

lập phương mở rộng) trong cuốn *The Art of Scalability* ,¹ mô hình này chia việc mở rộng thành ba nhóm mà trong bối cảnh hệ thống máy tính sẽ bao gồm phân rã theo chức năng (functional decomposition), nhân bản theo chiều ngang (horizontal duplication) và phân vùng dữ liệu (data partitioning). Giá trị của mô hình này là nó giúp bạn hiểu rằng bạn có thể mở rộng một hệ thống theo một, hai, hoặc cả ba trục này, tùy theo nhu cầu của bạn.

1 Martin L. Abbott and Michael T. Fisher, *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise* , 2nd ed. (New York: Addison-Wesley, 2015).

1 Martin L. Abbott and Michael T. Fisher, *The Art of Scalability: Scalable Web Architecture, Processes, and Organizations for the Modern Enterprise* , 2nd ed. (New York: Addison-Wesley, 2015).

However, especially in a world of virtualized infrastructure, I've always felt that this model lacked a fourth axis of **vertical scaling**, though that would have the somewhat unfortunate property of making it no longer a cube. Nonetheless, I think it's a useful set of mechanisms for us to determine how best to scale our microservice architectures. Before we look at these types of scaling in detail, along with the relative pros and cons, a brief summary is in order:

Tuy nhiên, đặc biệt là trong một thế giới của hạ tầng ảo hóa, tôi luôn cảm thấy mô hình này thiếu mất một trục thứ tư là mở rộng dọc (vertical scaling), dù việc thêm trục này sẽ dẫn đến một tính chất hơi bất tiện là nó không còn là một khối lập phương nữa. Dù vậy, tôi vẫn nghĩ đây là một tập cơ chế hữu ích để chúng ta xác định cách mở rộng kiến trúc microservice của mình tốt nhất. Trước khi đi vào chi tiết các kiểu mở rộng này, cùng với các ưu nhược điểm tương đối, hãy điểm qua một bản tóm tắt ngắn gọn:

Vertical scaling In a nutshell, this means getting a bigger machine.

Mở rộng dọc (vertical scaling) Nói gọn lại, điều này nghĩa là sắm một cỗ máy lớn hơn.

Horizontal duplication Having multiple things capable of doing the same work.

Nhân bản theo chiều ngang (horizontal duplication) Có nhiều thực thể có khả năng làm cùng một công việc.

Data partitioning Dividing work based on some attribute of the data, e.g., customer group.

Phân vùng dữ liệu (data partitioning) Chia nhỏ công việc dựa trên một thuộc tính nào đó của dữ liệu, ví dụ như nhóm khách hàng.

Functional decomposition Separation of work based on the type, e.g., microservice decomposition.

Phân rã theo chức năng (functional decomposition) Tách công việc dựa trên loại, ví dụ như phân rã thành các microservice.

Understanding what combination of these scaling techniques is most appropriate will fundamentally come down to the nature of the scaling issue you are facing. To explore this in more detail, as well as look at examples of how these concepts could be implemented for MusicCorp, we'll also examine their suitability for a real-world company, FoodCo. 2 FoodCo provides food delivery direct to customers across a number of countries around the world.

Hiểu được tổ hợp các kỹ thuật mở rộng nào là phù hợp nhất về cơ bản sẽ phụ thuộc vào bản chất của vấn đề mở rộng mà bạn đang đối mặt. Để khám phá điều này chi tiết hơn, cũng như xem các ví dụ về cách những khái niệm này có thể được triển khai cho MusicCorp, chúng ta cũng sẽ xem xét tính phù hợp của chúng với một công ty thực tế là FoodCo.² FoodCo cung cấp dịch vụ giao đồ ăn trực tiếp đến khách hàng ở nhiều quốc gia trên khắp thế giới.

Vertical Scaling — Mở rộng dọc

Some operations can just benefit from more grunt. Getting a bigger box with faster CPU and better I/O can often improve latency and throughput, allowing you to process more work in less time. So if your application isn't going fast enough for you or can't handle enough requests, why not get a bigger machine?

Một số thao tác đơn giản là được lợi từ việc có nhiều sức mạnh hơn. Sắm một cỗ máy lớn hơn với CPU nhanh hơn và I/O tốt hơn thường có thể cải thiện độ trễ và thông lượng (throughput), cho phép bạn xử lý nhiều công việc hơn trong thời gian ngắn hơn. Vậy nếu ứng dụng của bạn chạy chưa đủ nhanh hoặc không xử lý nổi đủ lượng request, tại sao không sắm một cỗ máy lớn hơn?

In the case of FoodCo, one of the challenges it is suffering from is increasing write contention on its primary **database**. Normally, vertical scaling is an absolute go-to option to quickly scale writes on a relational database, and indeed FoodCo has already upgraded the database infrastructure multiple times. The problem is that FoodCo has really already pushed this as far as it is comfortable with. Vertical scaling worked for many years, but given the company's growth projections, even if FoodCo could get a bigger machine, that isn't likely to solve the problem in the longer term.

Trong trường hợp của FoodCo, một trong những thách thức mà công ty đang gặp phải là sự tranh chấp ghi (write contention) ngày càng tăng trên cơ sở dữ liệu chính. Thông thường, mở rộng dọc là một lựa chọn mặc định tuyệt đối để nhanh chóng mở rộng khả năng ghi trên một cơ sở dữ liệu quan hệ, và thực tế FoodCo đã nâng cấp hạ tầng cơ sở dữ liệu nhiều lần. Vấn đề là FoodCo gần như đã đẩy việc này đi xa hết mức mà công ty thấy thoải mái. Mở rộng dọc đã hiệu quả trong nhiều năm, nhưng với những dự báo tăng trưởng của công ty, kể cả khi FoodCo có thể sắm một cỗ máy lớn hơn, điều đó cũng khó có thể giải quyết được vấn đề trong dài hạn.

Historically, when vertical scaling required purchasing hardware, this technique was more problematic. The lead time to buying hardware meant that this wasn't

Trong lịch sử, khi mở rộng dọc đòi hỏi phải mua phần cứng, kỹ thuật này gặp nhiều vấn đề hơn. Thời gian chờ để mua được phần cứng đồng nghĩa với việc đây không phải là

2 As before, I have anonymized the company—FoodCo isn't its real name!

2 Như trước đây, tôi đã ẩn danh công ty này—FoodCo không phải là tên thật của nó!

something that could be entered into lightly, and if it turned out that having a bigger machine didn't solve your problems, then you had likely spent a bunch of money you didn't need to. In addition, it was typical to oversize the machine you needed due to the hassle of getting budgetary sign-offs, waiting for the machine to arrive and so on, which in turn led to significant unused capacity in data centers.

điều có thể quyết định một cách dễ dàng, và nếu hóa ra việc có một cỗ máy lớn hơn không giải quyết được vấn đề của bạn, thì rất có thể bạn đã tiêu một đồng tiền không cần thiết. Ngoài ra, người ta thường mua cỗ máy lớn hơn mức cần thiết do sự phiền toái khi phải xin duyệt ngân sách, chờ máy về và vận vận, điều này lại dẫn đến lượng dung lượng không sử dụng đáng kể trong các trung tâm dữ liệu.

The move to virtualization and the emergence of the public cloud have helped immensely with this form of scaling, though.

Tuy nhiên, việc chuyển sang ảo hóa và sự xuất hiện của public cloud đã hỗ trợ rất nhiều cho kiểu mở rộng này.

Implementation

Triển khai

Implementation will vary depending on whose infrastructure you are running on. If running on your own virtualized infrastructure, you may be able to just resize the VM to use more of the underlying hardware—this is something that should be quick and fairly risk-free to implement. If the VM is as big as the underlying hardware can handle, this option is of course a nonstarter—you may have to buy more hardware. Likewise, if you're running on your own bare metal servers and don't have any spare hardware lying around that is bigger than what you are currently running on, then again, you're looking at having to buy more machines.

Việc triển khai sẽ khác nhau tùy thuộc vào việc bạn đang chạy trên hạ tầng của ai. Nếu chạy trên hạ tầng ảo hóa của riêng bạn, bạn có thể chỉ cần thay đổi kích thước VM để dùng nhiều hơn phần cứng bên dưới—đây là việc nên làm nhanh chóng và khá ít rủi ro để triển khai. Nếu VM đã lớn bằng mức phần cứng bên dưới có thể chịu được, thì lựa chọn này dĩ nhiên là bất khả thi—bạn có thể phải mua thêm phần cứng. Tương tự, nếu bạn đang chạy trên các máy chủ bare metal của riêng mình và không có sẵn phần cứng dư thừa nào lớn hơn cái bạn đang chạy, thì một lần nữa, bạn lại đứng trước việc phải mua thêm máy.

In general, if I've gotten to the point where I'd have to purchase new infrastructure to try out vertical scaling, due to the increased cost (and time) for this to have an impact I might well skip this form of scaling for the moment and look instead at horizontal duplication, which we'll come to next.

Nhìn chung, nếu tôi đã tới điểm mà tôi phải mua hạ tầng mới để thử mở rộng dọc, thì do chi phí (và thời gian) tăng thêm để điều này tạo ra tác động, tôi rất có thể sẽ bỏ qua kiểu mở rộng này lúc đó và thay vào đó nhìn sang nhân bản theo chiều ngang, mà chúng ta sẽ bàn tới tiếp theo.

But the emergence of the public cloud has also allowed us to easily rent, on a per-hour basis (and in some cases on an even shorter-term basis), fully managed machines through public cloud vendors. Moreover, the main cloud providers offer a wider variety of machines for different types of problems. Is your workload more memory intensive? Treat yourself to an AWS u-24tb1.metal instance, which provides 24 TB of memory (yes, you read that right). Now, the number of workloads that might actually need this much memory seem pretty rare, but you have that option. You also have machines tailored to high I/O, CPU, or GPU uses as well. If your existing solution is already on the public cloud, this is such a trivial form of scaling to try that if you're after a quick win, it's a bit of a no-brainer.

Nhưng sự xuất hiện của public cloud cũng đã cho phép chúng ta dễ dàng thuê, theo giờ (và trong một số trường hợp theo khoảng thời gian còn ngắn hơn nữa), các cỗ máy được quản lý hoàn toàn thông qua các nhà cung cấp public cloud. Hơn nữa, các nhà cung cấp cloud lớn còn cung cấp một loạt máy đa dạng hơn cho các loại vấn đề khác nhau. Khối lượng công việc của bạn cần nhiều bộ nhớ hơn? Hãy tự thưởng cho mình một instance AWS u-24tb1.metal, cung cấp 24 TB bộ nhớ (vâng, bạn đọc đúng đấy). Giờ thì số lượng khối lượng công việc thực sự cần đến mức bộ nhớ này có vẻ khá hiếm, nhưng bạn vẫn có lựa chọn đó. Bạn cũng có những cỗ máy được tinh chỉnh cho mục đích sử dụng I/O, CPU hoặc GPU cao. Nếu giải pháp hiện tại của bạn đã ở trên public cloud, thì đây là một kiểu mở rộng dễ dàng để thử đến mức nếu bạn đang muốn một thắng lợi nhanh thì việc làm này gần như không cần phải suy nghĩ.

Key benefits

Lợi ích chính

On virtualized infrastructure, especially on a public cloud provider, implementing this form of

scaling will be fast . A lot of the work around scaling applications comes down to experimentation—having an idea about something that can improve your system, making the change, and measuring the impact. Activities that are quick and fairly risk-free to try out are always worth doing early on. And vertical scaling fits the bill here.

Trên hạ tầng ảo hóa, đặc biệt là trên một nhà cung cấp public cloud, việc triển khai kiểu mở rộng này sẽ nhanh . Phần lớn công việc xoay quanh việc mở rộng ứng dụng rất cuộc nằm ở việc thử nghiệm—có một ý tưởng về điều gì đó có thể cải thiện hệ thống, thực hiện thay đổi, và đo lường tác động. Những hoạt động nhanh chóng và khá ít rủi ro để thử nghiệm thì luôn đáng làm sớm. Và mở rộng dọc đáp ứng được tiêu chí này.

It's also worth noting that vertical scaling can make it easier to perform other types of scaling. As a concrete example, moving your database infrastructure to a larger machine may allow it to host the logically isolated databases for newly created **microservices** as part of functional decomposition.

Cũng cần lưu ý rằng mở rộng dọc có thể giúp việc thực hiện các kiểu mở rộng khác dễ dàng hơn. Lấy một ví dụ cụ thể, việc chuyển hạ tầng cơ sở dữ liệu của bạn sang một cỗ máy lớn hơn có thể cho phép nó lưu trữ các cơ sở dữ liệu được cô lập về mặt logic cho các microservice mới được tạo ra như một phần của việc phân rã theo chức năng.

Your code or database is unlikely to need any changes to make use of the larger underlying infrastructure, assuming the operating system and chipsets remain the same. Even if changes are needed to your application to make use of the change of hardware, they might be limited to things like increasing the amount of memory available to your runtime through runtime flags.

Mã hoặc cơ sở dữ liệu của bạn khả năng cao sẽ không cần bất kỳ thay đổi nào để tận dụng hạ tầng bên dưới lớn hơn, miễn là hệ điều hành và chipset vẫn giữ nguyên. Ngay cả khi cần thay đổi ứng dụng của bạn để tận dụng sự thay đổi phần cứng, những thay đổi đó có thể chỉ giới hạn ở những việc như tăng lượng bộ nhớ khả dụng cho runtime của bạn thông qua các cờ runtime.

Limitations

Hạn chế

As we scale up the machines we are running on, our CPUs often don't actually get faster; we just have more cores. This has been a shift over the last 5–10 years. It used to be that each new generation of hardware would deliver big improvements in CPU clock speed, meaning our programs got big jumps in performance. Clock speed improvements have drastically trailed off, though, and instead we get more and more CPU cores to play with. The problem is that often our software has not been written to take advantage of multicore hardware. This could mean that the shift of your application from a 4- to 8-core system may deliver little if any improvement, even if your existing system is CPU-bound. Changing the code to take advantage of multicore hardware can be a significant undertaking and can require a complete shift in programming idioms.

Khi chúng ta nâng cấp các cỗ máy đang chạy, CPU của chúng ta thường không thực sự nhanh hơn; chúng ta chỉ đơn giản là có nhiều nhân (core) hơn. Đây là một sự dịch chuyển trong 5–10 năm qua. Trước đây, mỗi thế hệ phần cứng mới sẽ mang lại những cải thiện lớn về tốc độ xung nhịp CPU, nghĩa là chương trình của chúng ta có những bước nhảy vọt lớn về hiệu năng. Tuy nhiên, sự cải thiện tốc độ xung nhịp đã chững lại đáng kể, và thay vào đó chúng ta nhận được ngày càng nhiều nhân CPU để dùng. Vấn đề là phần mềm của chúng ta thường không được viết để tận dụng phần cứng đa nhân. Điều này có thể đồng nghĩa với việc chuyển ứng dụng của bạn từ hệ thống 4 nhân sang 8 nhân có thể mang lại rất ít cải thiện, nếu có, kể cả khi hệ thống hiện tại của bạn bị giới hạn bởi CPU. Việc thay

đổi mã để tận dụng phần cứng đa nhân có thể là một công việc lớn và có thể đòi hỏi một sự dịch chuyển hoàn toàn về phong cách lập trình.

Having a bigger machine is also likely to do little to improve robustness. A larger, newer server might have improved reliability, but ultimately if that machine is down, then the machine is down. Unlike the other forms of scaling we'll look at, vertical scaling is unlikely to have much impact in improving your system's robustness.

Có một cỗ máy lớn hơn cũng khả năng cao là sẽ ít giúp cải thiện độ vững chắc. Một máy chủ lớn hơn, mới hơn có thể có độ tin cậy được cải thiện, nhưng rồi cuối cùng nếu cỗ máy đó sập thì nó sập. Khác với các kiểu mở rộng khác mà chúng ta sẽ xem xét, mở rộng dọc khả năng cao là không có nhiều tác động trong việc cải thiện độ vững chắc của hệ thống.

Finally, as the machines get larger, they get more expensive—but not always in a way that is matched by the increased resources available to you. Sometimes this means it can be more cost effective to have a larger number of small machines, rather than a smaller number of large machines.

Cuối cùng, khi các cỗ máy trở nên lớn hơn, chúng đắt hơn—nhưng không phải lúc nào cũng theo một cách tương xứng với lượng tài nguyên tăng thêm mà bạn có được. Đôi khi điều này đồng nghĩa với việc có nhiều cỗ máy nhỏ có thể tiết kiệm chi phí hơn so với có ít cỗ máy lớn.

Horizontal Duplication — Nhân bản theo chiều ngang

With horizontal duplication, you duplicate part of your system to handle more workloads. The exact mechanisms vary—we'll look at implementations shortly—but fundamentally horizontal duplication requires you to have a way of distributing the work across these duplicates.

Với nhân bản theo chiều ngang, bạn nhân bản một phần hệ thống của mình để xử lý nhiều khối lượng công việc hơn. Cơ chế chính xác thì khác nhau—chúng ta sẽ xem các cách triển khai ngay sau đây—nhưng về cơ bản nhân bản theo chiều ngang đòi hỏi bạn phải có một cách phân phối công việc qua các bản nhân này.

As with vertical scaling, this type of scaling is on the simpler end of the spectrum and is often one of the things I'll try early on. If your monolithic system can't handle the load, spin up multiple copies of it and see if that helps!

Cũng như mở rộng dọc, kiểu mở rộng này nằm ở phía đơn giản hơn của phổ và thường là một trong những thứ tôi sẽ thử sớm. Nếu hệ thống monolith của bạn không xử lý nổi tải, hãy khởi chạy nhiều bản sao của nó và xem điều đó có giúp ích không!

Implementations

Các cách triển khai

Probably the most obvious form of horizontal duplication that comes to mind is making use of a load balancer to distribute requests across multiple copies of your functionality, as we see in Figure 13-1, where we are load balancing across multiple instances of MusicCorp's Catalog microservice. Load balancer capabilities differ, but you'd expect them all to have some mechanism to distribute load across the nodes, and to detect when a node is unavailable and remove it from the load balancer pool. From the consumer's point of view, the load balancer is an entirely transparent implementation concern—we can view it as part of the logical boundary of the microservice in this regard. Historically, load balancers would be thought of primarily in terms of dedicated hardware, but this has long since ceased to be common— instead, more load balancing is done in software, often running on the client side.

Hình thức nhân bản theo chiều ngang dễ thấy nhất có lẽ là tận dụng một bộ cân bằng tải (load balancer) để phân phối các request qua nhiều bản sao của chức năng, như chúng ta thấy trong Hình 13-1, nơi chúng ta cân bằng tải qua nhiều instance của microservice Catalog của MusicCorp. Khả năng của các bộ cân bằng tải khác nhau, nhưng bạn sẽ kỳ vọng tất cả chúng đều có một cơ chế nào đó để phân phối tải qua các node, và để phát hiện khi một node không khả dụng và loại nó ra khỏi nhóm cân bằng tải. Từ góc nhìn của bên tiêu thụ (consumer), bộ cân bằng tải là một mối quan tâm triển khai hoàn toàn trong suốt—về khía cạnh này, chúng ta có thể xem nó như một phần của ranh giới logic của microservice. Trong lịch sử, các bộ cân bằng tải chủ yếu được nghĩ đến dưới dạng phần cứng chuyên dụng, nhưng điều này từ lâu đã không còn phổ biến nữa—thay vào đó, việc cân bằng tải được thực hiện nhiều hơn bằng phần mềm, thường chạy ở phía client.

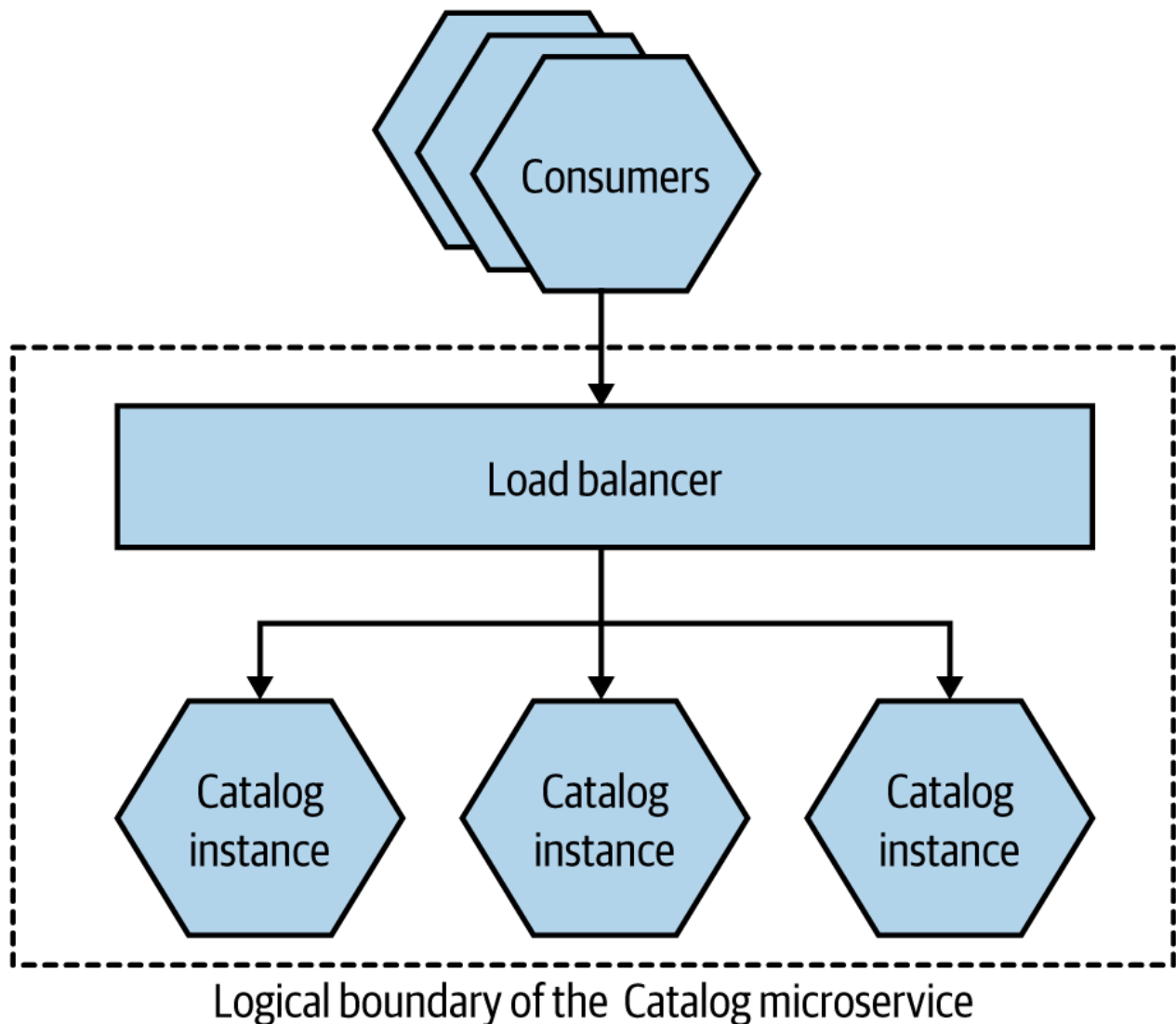


Figure 13-1. The Catalog microservice deployed as multiple instances, with a load balancer to spread requests

Hình 13-1. Microservice Catalog được triển khai dưới dạng nhiều instance, với một bộ cân bằng tải để phân tán các request

Another example of horizontal duplication could be the competing consumer pattern, as detailed in

Enterprise Integration Patterns . 3 In Figure 13-2 , we see new songs being uploaded to MusicCorp. These new songs need to be transcoded into different files to be used as part of MusicCorp's new streaming offering. We have a common queue of work in which these jobs are placed, and a set of Song Transcoder instances all consume from the queue—the different instances are competing over the jobs. To

Một ví dụ khác về nhân bản theo chiều ngang có thể là mẫu competing consumer (người tiêu thụ cạnh tranh), như được trình bày chi tiết trong cuốn Enterprise Integration Patterns .³ Trong Hình 13-2 , chúng ta thấy các bài hát mới được tải lên MusicCorp. Những bài hát mới này cần được chuyển mã (transcode) thành các file khác nhau để dùng làm một phần trong dịch vụ streaming mới của MusicCorp. Chúng ta có một hàng đợi công việc chung trong đó các job này được đặt vào, và một tập các instance Song Transcoder đều tiêu thụ từ hàng đợi—các instance khác nhau cạnh tranh giành lấy các job. Để

3 Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns (Boston: Addison-Wesley, 2003).

3 Gregor Hohpe and Bobby Woolf, Enterprise Integration Patterns (Boston: Addison-Wesley, 2003).

increase the throughput of the system, we can increase the number of Song Trans coder instances.

tăng thông lượng của hệ thống, chúng ta có thể tăng số lượng instance Song Transcoder.

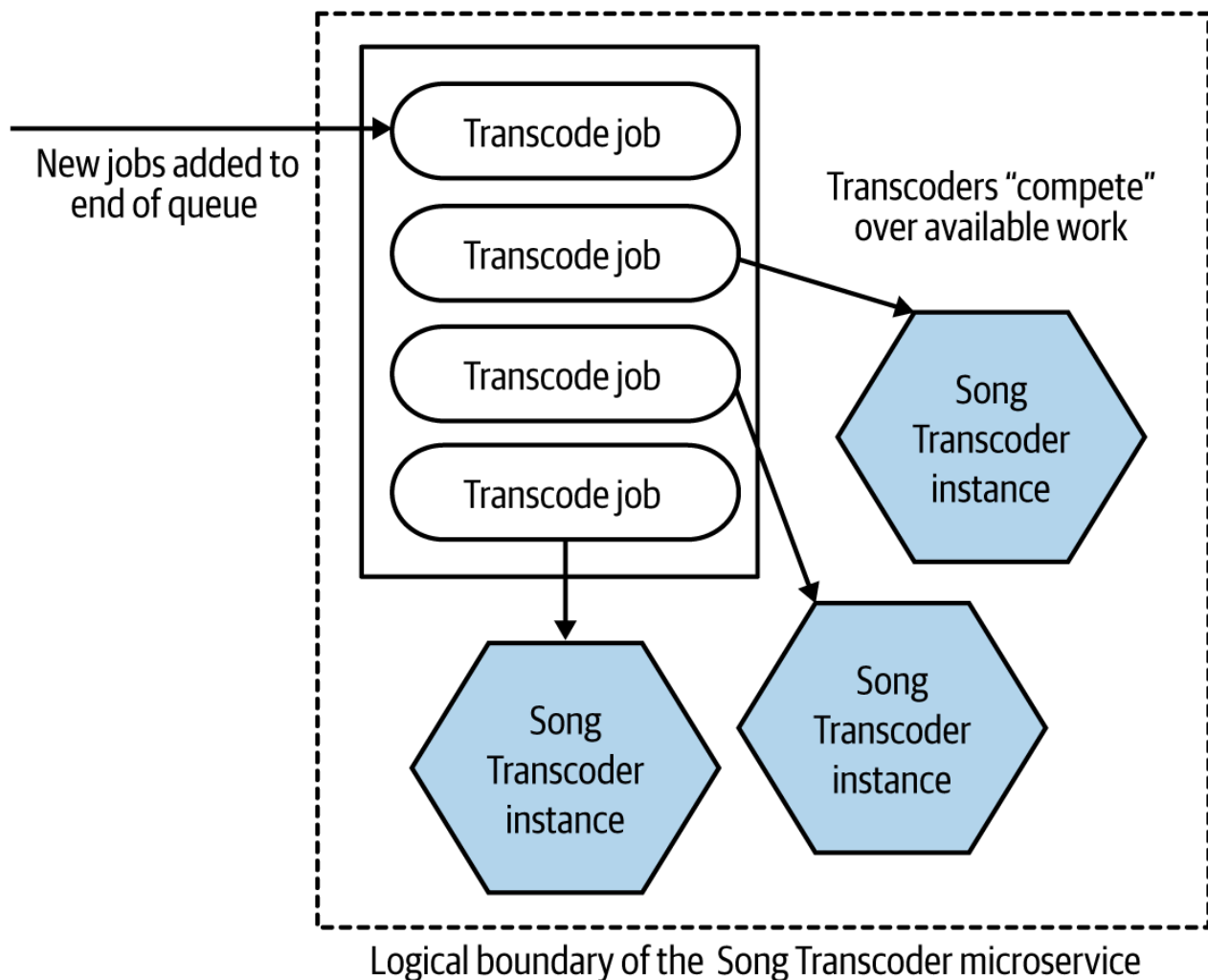


Figure 13-2. Transcoding for streaming being scaled up using the competing consumer pattern

Hình 13-2. Việc chuyển mã cho streaming được mở rộng bằng mẫu competing consumer

In the case of FoodCo, a form of horizontal duplication has been used to reduce the read load on the primary database through the use of read replicas, as we see in Figure 13-3 . This has reduced read load on the primary database node, freeing up resources to handle writes, and has worked very effectively, as a lot of the load on the main system was read-heavy. These reads could easily be redirected to these read replicas, and it's common to use a load balancer over multiple read replicas.

Trong trường hợp của FoodCo, một dạng nhân bản theo chiều ngang đã được dùng để giảm tải đọc trên cơ sở dữ liệu chính thông qua việc sử dụng các bản sao đọc (read replica), như chúng ta thấy trong Hình 13-3 . Việc này đã giảm tải đọc trên node cơ sở dữ liệu chính, giải phóng tài nguyên để xử lý việc ghi, và đã hoạt động rất hiệu quả, vì phần lớn tải trên hệ thống chính nặng về đọc. Những lượt đọc này có thể dễ dàng được chuyển hướng tới các bản sao đọc này, và việc dùng một bộ cân bằng tải đặt trên nhiều bản sao đọc là phổ biến.

The routing to either the primary database or a read replica is handled internally in the microservice. It's transparent to consumers of this microservice as to whether or not a request they send ends up hitting the primary or read replica database.

Việc định tuyến tới cơ sở dữ liệu chính hay một bản sao đọc được xử lý nội bộ trong microservice. Đối với bên tiêu thụ của microservice này thì việc một request mà họ gửi tới cuối cùng đến cơ sở dữ liệu chính hay bản sao đọc là trong suốt.

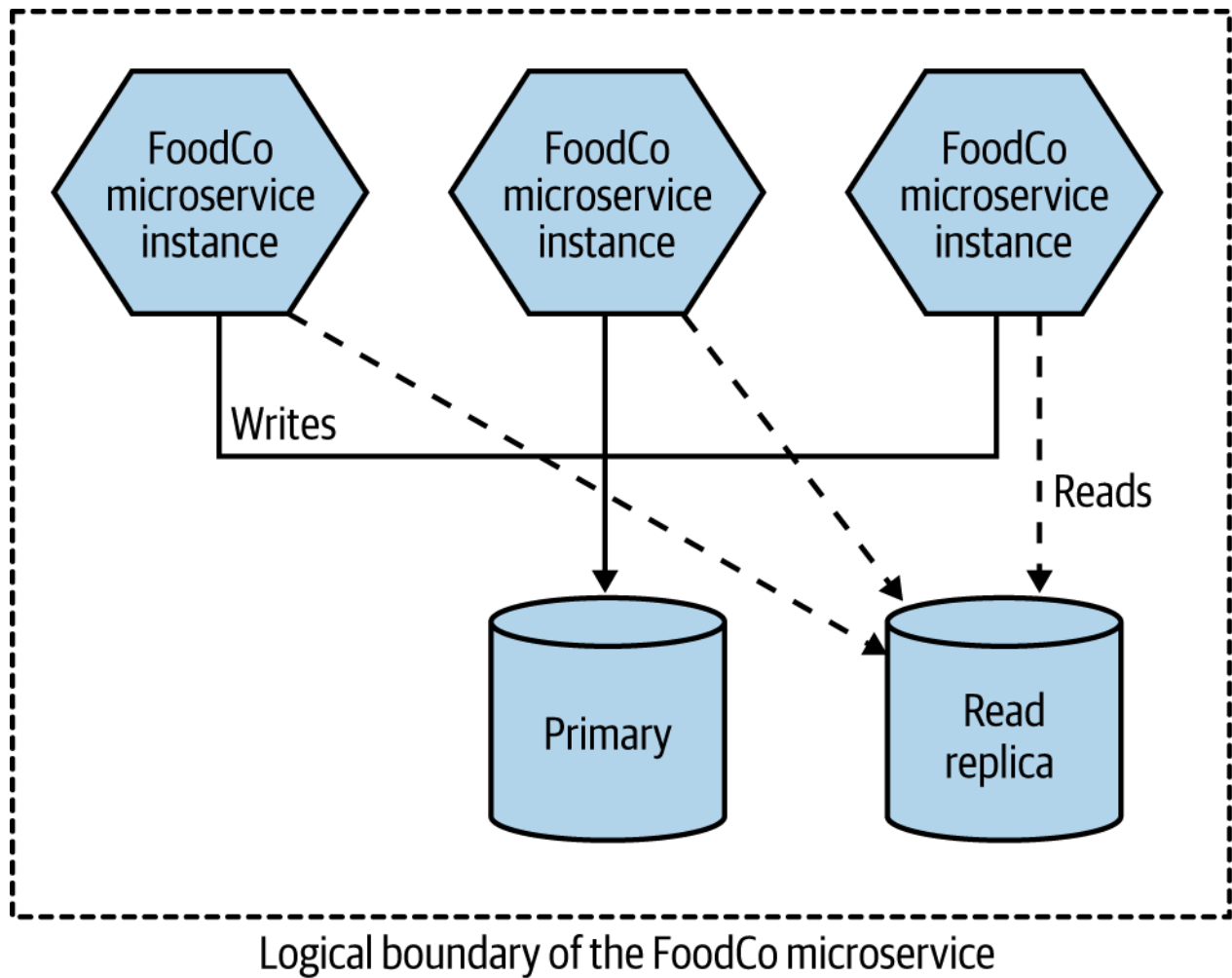


Figure 13-3. FoodCo making use of read replicas to scale read traffic

Hình 13-3. FoodCo sử dụng các bản sao đọc để mở rộng lưu lượng đọc

Key benefits

Lợi ích chính

Horizontal duplication is relatively straightforward. It's rare that the application needs to be updated, as the work to distribute the load can often be done elsewhere—for example, through a queue running on a **message broker** or perhaps in a load balancer. If vertical scaling isn't available to me, this form of scaling is typically the next thing I'd look at.

Nhân bản theo chiều ngang tương đối đơn giản. Hiếm khi ứng dụng cần được cập nhật, vì công việc phân phối tải thường có thể được thực hiện ở nơi khác—ví dụ, thông qua một hàng đợi chạy trên message broker hoặc có lẽ trong một bộ cân bằng tải. Nếu mở rộng dọc không khả dụng với tôi, thì kiểu mở rộng này thường là thứ tiếp theo mà tôi sẽ xem xét.

Assuming the work can be easily spread across the duplicates, it's an elegant way of spreading the load and reducing contention for raw computing resources.

Giả sử công việc có thể được phân tán dễ dàng qua các bản nhân, thì đây là một cách thanh lịch để phân tán tải và giảm sự tranh chấp các tài nguyên tính toán thô.

Limitations

Hạn chế

As with virtually all the scaling options we'll look at, horizontal duplication requires more infrastructure, which can of course cost more money. It can be a bit of a blunt instrument too—you might run multiple full copies of your monolithic application, for example, even if only part of that **monolith** is actually experiencing scaling issues.

Cũng như hầu hết mọi lựa chọn mở rộng mà chúng ta sẽ xem xét, nhân bản theo chiều ngang đòi hỏi nhiều hạ tầng hơn, điều này dĩ nhiên có thể tốn nhiều tiền hơn. Nó cũng có thể là một công cụ hơi thô—ví dụ, bạn có thể chạy nhiều bản sao đầy đủ của ứng dụng monolith, ngay cả khi chỉ một phần của monolith đó thực sự đang gặp vấn đề mở rộng.

Much of the work here is in implementing your load distribution mechanisms. These can range from the simple, such as HTTP load balancing, to the more complex, such as using a message broker or configuring database read replicas. You are relying on this load distribution mechanism to do its job—coming to grips with how it works and with any limitations of your specific choice will be key.

Phần lớn công việc ở đây nằm ở việc triển khai các cơ chế phân phối tải của bạn. Những cơ chế này có thể trải dài từ đơn giản, như cân bằng tải HTTP, đến phức tạp hơn, như dùng một message broker hoặc cấu hình các bản sao đọc cơ sở dữ liệu. Bạn đang phụ thuộc vào cơ chế phân phối tải này để làm tốt công việc của nó—việc nắm vững cách nó hoạt động cùng với bất kỳ hạn chế nào của lựa chọn cụ thể của bạn sẽ là then chốt.

Some systems might place additional requirements on the load distribution mechanism. For example, they might require that each request associated with the same user session gets directed to the same replica. This can be solved through the use of a load balancer that requires sticky session load balancing, but that might in turn limit what load distribution mechanisms you could consider. It's worth noting that systems that require sticky load balancing like this are prone to other problems, and in general I'd avoid building systems that have this requirement.

Một số hệ thống có thể đặt thêm các yêu cầu lên cơ chế phân phối tải. Ví dụ, chúng có thể yêu cầu mỗi request gắn với cùng một phiên người dùng phải được định tuyến tới cùng một bản nhân. Điều này có thể được giải quyết thông qua việc dùng một bộ cân bằng tải yêu cầu cân bằng tải kiểu phiên dính (sticky session), nhưng điều đó lại có thể giới hạn những cơ chế phân phối tải mà bạn có thể cân nhắc. Cũng cần lưu ý rằng các hệ thống yêu cầu cân bằng tải dính kiểu này dễ gặp các vấn đề khác, và nhìn chung tôi sẽ tránh xây dựng các hệ thống có yêu cầu này.

Data Partitioning — Phân vùng dữ liệu

Having begun with the simpler forms of scaling, we're now entering into more difficult territory. Data partitioning requires that we distribute load based on some aspect of data—perhaps distributing load based on the user, for example.

Sau khi bắt đầu với các kiểu mở rộng đơn giản hơn, giờ chúng ta bước vào địa hình khó khăn hơn. Phân vùng dữ liệu đòi hỏi chúng ta phân phối tải dựa trên một khía cạnh nào đó của dữ liệu—ví dụ, có lẽ là phân phối tải dựa trên người dùng.

Implementation

Triển khai

The way data partitioning works is that we take a key associated with the workload and apply a function to it, and the result is the partition (sometimes called a shard) we will distribute the work to. In Figure 13-4, we have two partitions, and our function is quite simple—we send the request

to one database if the family name starts with A to M, or to a different database if the family name starts with N to Z. Now, this is actually a bad example of a partitioning algorithm (we'll get to why that is shortly), but hopefully this is straightforward enough to illustrate the idea.

Cách phân vùng dữ liệu hoạt động là chúng ta lấy một khóa gắn với khối lượng công việc và áp dụng một hàm lên nó, và kết quả là phân vùng (đôi khi gọi là shard) mà chúng ta sẽ phân phối công việc tới. Trong Hình 13-4, chúng ta có hai phân vùng, và hàm của chúng ta khá đơn giản—chúng ta gửi request tới một cơ sở dữ liệu nếu họ của người dùng bắt đầu bằng A đến M, hoặc tới một cơ sở dữ liệu khác nếu họ bắt đầu bằng N đến Z. Giờ thì đây thực ra là một ví dụ tồi về một thuật toán phân vùng (chúng ta sẽ tới lý do vì sao ngay sau đây), nhưng hy vọng nó đủ đơn giản để minh họa ý tưởng.

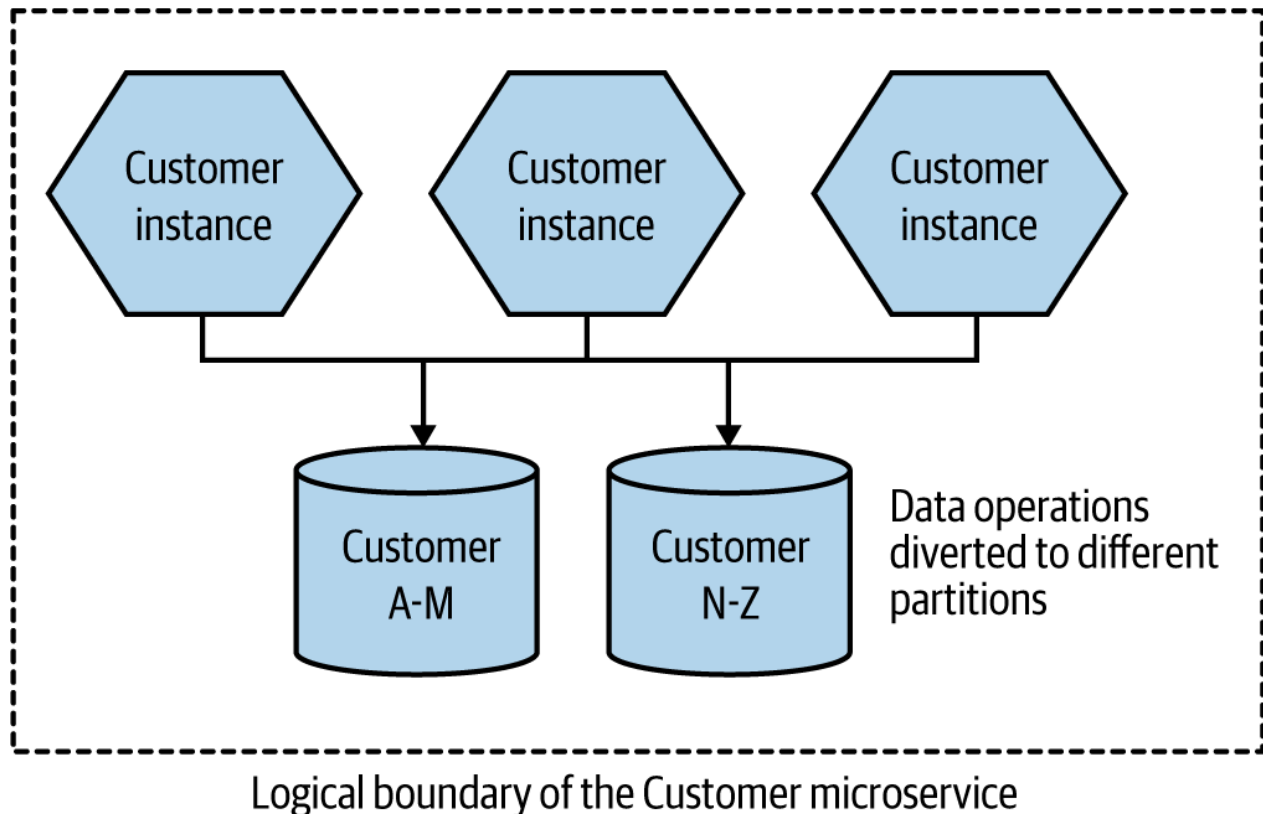


Figure 13-4. Customer data is partitioned across two different databases

Hình 13-4. Dữ liệu khách hàng được phân vùng qua hai cơ sở dữ liệu khác nhau

In this example, we're partitioning at the database level. A request to the Customer microservice can hit any microservice instance. But when we perform an operation that requires the database (reads or writes), that request is directed to the appropriate database node based on the name of the customer. In the case of a relational database, the schema of both database nodes would be identical, but the contents of each would apply only to a subset of the customers.

Trong ví dụ này, chúng ta đang phân vùng ở cấp cơ sở dữ liệu. Một request tới microservice Customer có thể đến bất kỳ instance microservice nào. Nhưng khi chúng ta thực hiện một thao tác cần đến cơ sở dữ liệu (đọc hoặc ghi), request đó được định tuyến tới node cơ sở dữ liệu phù hợp dựa trên tên của khách hàng. Trong trường hợp cơ sở dữ liệu quan hệ, lược đồ (schema) của cả hai node cơ sở dữ liệu sẽ giống hệt nhau, nhưng nội dung của mỗi node chỉ áp dụng cho một tập con khách hàng.

Partitioning at the database level often makes sense if the database technology you are using supports this concept natively, as you can then offload this problem to an existing implementation. We could, however, partition instead at the microservice instance level, as we see in Figure 13-5 . Here, we need to be able to work out from the inbound request what partition the request should be mapped to—in our example, this is being done via some form of proxy. In the case of our customer-based partitioning model, if the name of the customer is in the request headers, then that would be sufficient. This approach makes sense if you want dedicated microservice instances for partitioning, which could be useful if you are making use of in-memory **caching**. It also means you are able to scale each partition at both the database level and the microservice instance level.

Phân vùng ở cấp cơ sở dữ liệu thường hợp lý nếu công nghệ cơ sở dữ liệu bạn đang dùng hỗ trợ khái niệm này một cách nguyên bản, vì khi đó bạn có thể đẩy vấn đề này sang một cách triển khai sẵn có. Tuy nhiên, chúng ta có thể phân vùng ở cấp instance microservice thay vào đó, như chúng ta thấy trong Hình 13-5 . Ở đây, chúng ta cần có khả năng xác định từ request đến vào rằng request đó nên được ánh xạ tới phân vùng nào—trong ví dụ của chúng ta, việc này được thực hiện thông qua một dạng proxy nào đó. Trong trường hợp mô hình phân vùng theo khách hàng của chúng ta, nếu tên khách hàng nằm trong các header của request thì điều đó là đủ. Cách tiếp cận này hợp lý nếu bạn muốn các instance microservice chuyên dụng cho việc phân vùng, điều này có thể hữu ích nếu bạn đang dùng caching trong bộ nhớ (in-memory). Nó cũng đồng nghĩa với việc bạn có thể mở rộng từng phân vùng ở cả cấp cơ sở dữ liệu lẫn cấp instance microservice.

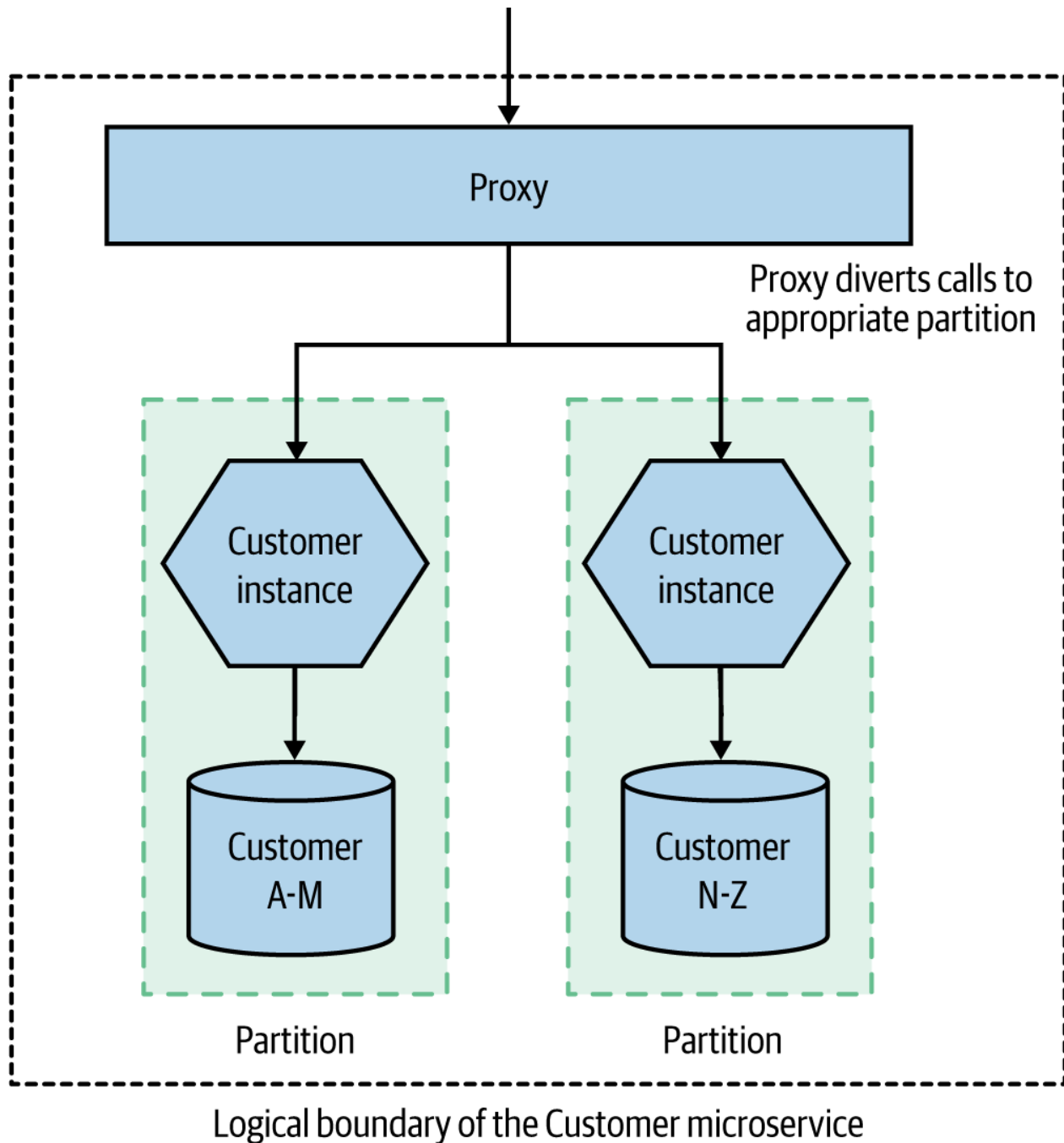


Figure 13-5. Requests are directed to the appropriate microservice instance

Hình 13-5. Các request được định tuyến tới instance microservice phù hợp

As with the example of read replicas, we would want this scaling to be done in such a way that consumers of the microservice are unaware of this implementation detail. When a consumer makes a request to the Customer microservice in Figure 13-5, we would want their request to be dynamically routed to the right partition. The fact that we have implemented data partitioning should be treated as an internal implementation detail of the microservice in question—this gives us freedom to change the partitioning scheme, or perhaps replace partitioning altogether.

Cũng như ví dụ về các bản sao đọc, chúng ta sẽ muốn việc mở rộng này được thực hiện theo

cách mà bên tiêu thụ của microservice không hay biết về chi tiết triển khai này. Khi một bên tiêu thụ gửi request tới microservice Customer trong Hình 13-5, chúng ta sẽ muốn request của họ được định tuyến động tới đúng phân vùng. Việc chúng ta đã triển khai phân vùng dữ liệu nên được coi là một chi tiết triển khai nội bộ của microservice đang xét—điều này cho chúng ta quyền tự do thay đổi sơ đồ phân vùng, hoặc có lẽ thay thế hoàn toàn việc phân vùng.

Another common example of data partitioning is to do it on a geographic basis. You might have one partition per country, or one per region.

Một ví dụ phổ biến khác về phân vùng dữ liệu là thực hiện trên cơ sở địa lý. Bạn có thể có một phân vùng cho mỗi quốc gia, hoặc một phân vùng cho mỗi vùng.

For FoodCo, one option to handle the contention on its primary database is to partition data based on country. So customers in Ghana hit one database, and customers in Jersey hit another. This model wouldn't make sense for FoodCo due to a number of factors. The main problem is that FoodCo has plans to continue geographic expansion and is hoping to drive efficiencies by being able to serve multiple geographical locales from the same system. The idea of having to continually spin up new partitions for each country would drastically increase the cost of moving into new countries.

Đối với FoodCo, một lựa chọn để xử lý sự tranh chấp trên cơ sở dữ liệu chính của công ty là phân vùng dữ liệu dựa trên quốc gia. Vậy thì khách hàng ở Ghana đến một cơ sở dữ liệu, và khách hàng ở Jersey đến một cơ sở dữ liệu khác. Mô hình này sẽ không hợp lý với FoodCo do một số yếu tố. Vấn đề chính là FoodCo có kế hoạch tiếp tục mở rộng địa lý và đang hy vọng thúc đẩy hiệu quả bằng cách phục vụ được nhiều địa phương địa lý từ cùng một hệ thống. Ý tưởng phải liên tục khởi tạo các phân vùng mới cho mỗi quốc gia sẽ làm tăng đáng kể chi phí thâm nhập vào các quốc gia mới.

More often than not, partitioning will be done by the subsystem you rely on. For example, Cassandra uses partitions to distribute both reads and writes across the nodes in a given “ring,” and Kafka supports distributing messages across partitioned topics.

Phần lớn các trường hợp, việc phân vùng sẽ được thực hiện bởi hệ thống con mà bạn dựa vào. Ví dụ, Cassandra dùng các phân vùng để phân phối cả việc đọc lẫn ghi qua các node trong một “ring” nhất định, và Kafka hỗ trợ phân phối các thông điệp qua các topic được phân vùng.

Key benefits

Lợi ích chính

Data partitioning scales really nicely for transactional workloads. If your system is write-constrained, for example, data partitioning can deliver huge improvements.

Phân vùng dữ liệu mở rộng rất tốt cho các khối lượng công việc giao dịch. Ví dụ, nếu hệ thống của bạn bị giới hạn về ghi, phân vùng dữ liệu có thể mang lại những cải thiện rất lớn.

The creation of multiple partitions can also make it easier to reduce the impact and scope of maintenance activities. Rolling out updates can be done on a per-partition basis, and operations that would otherwise require downtime can have reduced impact, as they'll affect only a single partition. For example, if partitioning around geographic regions, operations that might result in interruption of **service** could be done at the least impactful time of day, perhaps in the early hours of the morning. Geographical partitioning can also be very useful if you need to ensure that data cannot leave certain jurisdictions—making sure that data associated with EU citizens remains stored inside the EU, for example.

Việc tạo ra nhiều phân vùng cũng có thể giúp giảm tác động và phạm vi của các hoạt động bảo trì dễ dàng hơn. Việc triển khai các bản cập nhật có thể được thực hiện trên từng phân vùng, và những thao tác mà lẽ ra đòi hỏi downtime có thể có tác động giảm bớt, vì chúng chỉ ảnh hưởng đến một phân vùng duy nhất. Ví dụ, nếu phân vùng theo các vùng địa lý, những thao tác có thể dẫn đến gián đoạn dịch vụ có thể được thực hiện vào thời điểm ít tác động nhất trong ngày, có lẽ vào những giờ sáng sớm. Phân vùng theo địa lý cũng có thể rất hữu ích nếu bạn cần đảm bảo rằng dữ liệu không thể rời khỏi một số khu vực pháp lý nhất định—ví dụ, đảm bảo dữ liệu gắn với công dân EU vẫn được lưu trữ bên trong EU.

Data partitioning can work well with horizontal duplication—each partition could consist of multiple nodes able to handle that work.

Phân vùng dữ liệu có thể kết hợp tốt với nhân bản theo chiều ngang—mỗi phân vùng có thể bao gồm nhiều node có khả năng xử lý công việc đó.

Limitations

Hạn chế

It's worth pointing out that data partitioning has limited utility in terms of improving system robustness. If a partition fails, that portion of your requests will fail. For example, if your load is distributed evenly across four partitions, and one partition fails, then 25% of your requests will end up failing. This isn't as bad as a total failure, but it's still pretty bad. This is why, as outlined earlier, it is common to combine data partitioning with a technique like horizontal duplication to improve the robustness of a given partition.

Cần chỉ ra rằng phân vùng dữ liệu có giá trị hữu dụng hạn chế về mặt cải thiện độ vững chắc của hệ thống. Nếu một phân vùng hỏng, phần request đó của bạn sẽ thất bại. Ví dụ, nếu tải của bạn được phân phối đều qua bốn phân vùng, và một phân vùng hỏng, thì 25% số request của bạn sẽ thất bại. Điều này không tệ bằng một sự cố toàn bộ, nhưng nó vẫn khá tệ. Đây là lý do vì sao, như đã nêu trước đó, người ta thường kết hợp phân vùng dữ liệu với một kỹ thuật như nhân bản theo chiều ngang để cải thiện độ vững chắc của một phân vùng nhất định.

Getting the partition key right can be difficult. In Figure 13-5, we used a fairly simple partitioning scheme, where we partitioned workload based on the family name of the customer. Customers with a family name starting with A–M go to partition 1, and customers with a name that starts with N–Z go to partition 2. As I pointed out when I shared that example, this is not a good partitioning strategy. With data partitioning, we want even distribution of load. But we cannot expect even distribution with the scheme I have outlined. In China, for example, there historically have been a very small number of surnames, and even today there are reckoned to be fewer than 4,000. The most popular 100 surnames, which account for over 80% of the population, skew heavily toward those family names starting with N–Z in Mandarin. This is an example of a scaling scheme that is unlikely to give even distribution of load, and across different countries and cultures could give wildly varying results.

Việc chọn đúng khóa phân vùng có thể khó. Trong Hình 13-5, chúng ta đã dùng một sơ đồ phân vùng khá đơn giản, trong đó chúng ta phân vùng khối lượng công việc dựa trên họ của khách hàng. Khách hàng có họ bắt đầu bằng A–M đến phân vùng 1, và khách hàng có tên bắt đầu bằng N–Z đến phân vùng 2. Như tôi đã chỉ ra khi chia sẻ ví dụ đó, đây không phải là một chiến lược phân vùng tốt. Với phân vùng dữ liệu, chúng ta muốn tải được phân phối đều. Nhưng chúng ta không thể kỳ vọng sự phân phối đều với sơ đồ tôi đã nêu. Ví dụ, ở Trung Quốc, trong lịch sử đã có một số lượng họ rất nhỏ, và ngay cả ngày nay con số ước tính cũng dưới 4.000. 100 họ phổ biến nhất, chiếm hơn 80% dân số, nghiêng nặng

về những họ bắt đầu bằng N–Z trong tiếng Quan thoại. Đây là một ví dụ về một sơ đồ mở rộng khó có khả năng cho phân phối tải đều, và ở các quốc gia và văn hóa khác nhau có thể cho ra những kết quả chênh lệch rất lớn.

A more sensible alternative might be to partition based on a unique ID given to each customer when they signed up. This is much more likely to give us an even distribution of load and would also deal with the situation in which someone changes their name.

Một lựa chọn thay thế hợp lý hơn có thể là phân vùng dựa trên một ID duy nhất được cấp cho mỗi khách hàng khi họ đăng ký. Cách này có khả năng cao hơn nhiều trong việc cho chúng ta sự phân phối tải đều và cũng sẽ xử lý được tình huống ai đó đổi tên của họ.

Adding new partitions to an existing scheme can often be done without too much trouble. For example, adding a new node to a Cassandra ring doesn't require any manual rebalancing of the data; instead, Cassandra has built-in support for dynamically distributing data across the nodes. Kafka also makes it fairly easy to add new partitions after the fact, although messages already in a partition won't move—but producers and consumers can be dynamically notified.

Việc thêm các phân vùng mới vào một sơ đồ sẵn có thường có thể được thực hiện mà không gặp quá nhiều rắc rối. Ví dụ, việc thêm một node mới vào một Cassandra ring không đòi hỏi bất kỳ việc tái cân bằng dữ liệu thủ công nào; thay vào đó, Cassandra có hỗ trợ tích hợp sẵn để phân phối dữ liệu một cách động qua các node. Kafka cũng giúp việc thêm các phân vùng mới sau này khá dễ dàng, mặc dù các thông điệp đã nằm trong một phân vùng sẽ không di chuyển—nhưng các producer và consumer có thể được thông báo một cách động.

Things get more tricky when you realize your partitioning scheme is just not fit for purpose, as in the case of our family name–based scheme outlined earlier. In that situation, you may have a painful road ahead. I remember chatting with a client many years ago who ended up having to take their main production system offline for three days to change the partitioning scheme for their main database.

Mọi thứ trở nên khó khăn hơn khi bạn nhận ra sơ đồ phân vùng của mình đơn giản là không phù hợp với mục đích, như trong trường hợp sơ đồ dựa trên họ đã nêu ở trên. Trong tình huống đó, bạn có thể có một chặng đường gian nan phía trước. Tôi nhớ đã trò chuyện với một khách hàng nhiều năm trước, người rất cuộc đã phải đưa hệ thống production chính của họ offline trong ba ngày để thay đổi sơ đồ phân vùng cho cơ sở dữ liệu chính.

We can also hit an issue with queries. Looking up an individual record is easy, as I can just apply the hashing function to find which instance the data should be on and then retrieve it from the correct shard. But what about queries that span the data in multiple nodes—for example, finding all the customers who are over 18? If you want to query all shards, you need to either query each individual shard and join in memory or else have an alternative read store where both data sets are available. Often querying across shards is handled by an **asynchronous** mechanism, using cached results. Mongo uses map/reduce jobs, for example, to perform these queries.

Chúng ta cũng có thể gặp một vấn đề với các truy vấn. Việc tra cứu một bản ghi riêng lẻ thì dễ, vì tôi chỉ cần áp dụng hàm băm để tìm xem dữ liệu nên nằm trên instance nào rồi truy xuất nó từ đúng shard. Nhưng còn các truy vấn trải rộng trên dữ liệu nằm ở nhiều node thì sao—ví dụ, tìm tất cả khách hàng trên 18 tuổi? Nếu bạn muốn truy vấn tất cả các shard, bạn cần hoặc là truy vấn từng shard riêng lẻ rồi nối (join) trong bộ nhớ, hoặc là có một kho đọc thay thế nơi cả hai tập dữ liệu đều có sẵn. Việc truy vấn xuyên các shard thường được xử lý bằng một cơ chế bất đồng bộ, dùng các kết quả đã cache. Ví dụ, Mongo dùng các job map/reduce để thực hiện những truy vấn này.

As you may have inferred from this brief overview, scaling databases for writes is where things get

very tricky, and where the capabilities of the various databases really start to become differentiated. I often see people changing database technology when they start hitting limits on how easily they can scale their existing write volume. If this happens to you, buying a bigger box is often the quickest way to solve the problem, but in the background you might want to look at other types of databases that might better handle your requirements. With the wealth of different types of databases available, selecting a new database can be a daunting activity, but as a starting point I thoroughly recommend the pleasingly concise *NoSQL Distilled*,⁴ which gives you an overview of the different styles of NoSQL databases available to you— from highly relational stores like graph databases to document stores, column stores, and key-value stores.

Như bạn có thể đã suy ra từ tổng quan ngắn này, việc mở rộng cơ sở dữ liệu cho khả năng ghi là nơi mọi thứ trở nên rất khó khăn, và là nơi mà năng lực của các cơ sở dữ liệu khác nhau thực sự bắt đầu trở nên khác biệt. Tôi thường thấy người ta đổi công nghệ cơ sở dữ liệu khi họ bắt đầu chạm tới giới hạn về việc mở rộng dễ dàng đến đâu khối lượng ghi hiện tại của họ. Nếu điều này xảy ra với bạn, mua một cỗ máy lớn hơn thường là cách nhanh nhất để giải quyết vấn đề, nhưng ở phía sau bạn có thể muốn xem xét các loại cơ sở dữ liệu khác có thể xử lý các yêu cầu của bạn tốt hơn. Với sự phong phú của các loại cơ sở dữ liệu khác nhau hiện có, việc chọn một cơ sở dữ liệu mới có thể là một hoạt động đáng nản, nhưng để khởi đầu tôi hết lòng giới thiệu cuốn *NoSQL Distilled*⁴ ngắn gọn một cách dễ chịu, cuốn này cho bạn một cái nhìn tổng quan về các phong cách cơ sở dữ liệu NoSQL khác nhau hiện có—từ các kho quan hệ cao như graph database đến document store, column store và key-value store.

Fundamentally, data partitioning is more work, especially as it may likely require extensive changes to the data of your existing system. The application code is probably going to be only lightly impacted, though.

Về cơ bản, phân vùng dữ liệu là công việc nhiều hơn, đặc biệt là vì nó có khả năng đòi hỏi những thay đổi sâu rộng đối với dữ liệu của hệ thống hiện có của bạn. Tuy nhiên, mã ứng dụng có lẽ sẽ chỉ bị tác động nhẹ.

Functional Decomposition — Phân rã theo chức năng

With functional decomposition, you extract functionality and allow it to be scaled independently. Extracting functionality from an existing system and creating a new microservice is almost the canonical example of functional decomposition. In Figure 13-6, we see an example from MusicCorp in which the order functionality is extracted from the main system to allow us to scale this functionality separately from the **rest**.

Với phân rã theo chức năng, bạn trích xuất chức năng và cho phép nó được mở rộng độc lập. Việc trích xuất chức năng từ một hệ thống sẵn có và tạo ra một microservice mới gần như là ví dụ kinh điển của phân rã theo chức năng. Trong Hình 13-6, chúng ta thấy một ví dụ từ MusicCorp trong đó chức năng đặt hàng được trích xuất khỏi hệ thống chính để cho phép chúng ta mở rộng chức năng này tách biệt với phần còn lại.

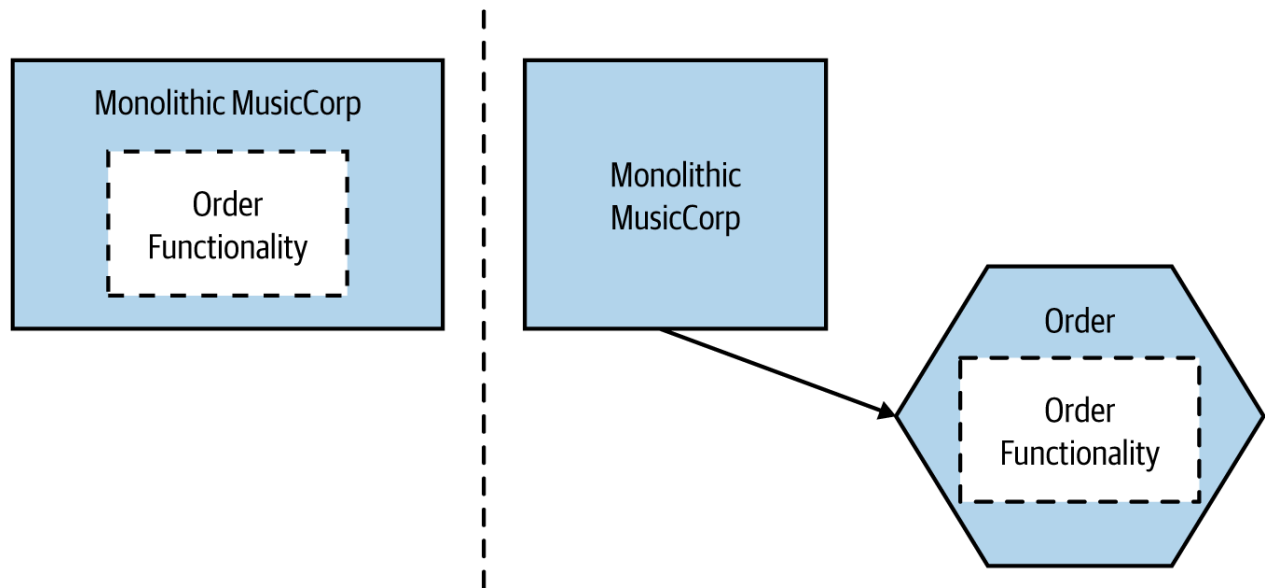


Figure 13-6. The Order microservice is extracted from the existing MusicCorp system

Hình 13-6. Microservice Order được trích xuất khỏi hệ thống MusicCorp hiện có

In the case of FoodCo, this is its forward direction. The company has exhausted vertical scaling, used horizontal duplication as far as it can, discounted data partitioning— what is left is to start to move toward functional decomposition. Key data and workloads are being removed from the core system and core database to make this change

Trong trường hợp của FoodCo, đây là hướng đi tiếp theo của công ty. Công ty đã vắt kiệt mở rộng dọc, đã dùng nhân bản theo chiều ngang tới mức tối đa có thể, đã loại trừ phân vùng dữ liệu—thứ còn lại là bắt đầu chuyển hướng sang phân rã theo chức năng. Dữ liệu và khối lượng công việc then chốt đang được gỡ ra khỏi hệ thống lõi và cơ sở dữ liệu lõi để thực hiện sự thay đổi

4 Pramod J. Sadalage and Martin Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence (Upper Saddle River, NJ: Addison-Wesley 2012).

4 Pramod J. Sadalage and Martin Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence (Upper Saddle River, NJ: Addison-Wesley 2012).

happen. A few quick wins were identified, including data associated with deliveries and menus being moved out of the primary database into dedicated microservices. This has the added benefit that it creates opportunities for FoodCo's growing delivery team to start to organize itself around the **ownership** of these new microservices.

này. Một vài thắng lợi nhanh đã được xác định, bao gồm việc dữ liệu liên quan đến giao hàng và thực đơn được chuyển ra khỏi cơ sở dữ liệu chính sang các microservice chuyên dụng. Việc này có thêm lợi ích là tạo ra cơ hội cho đội giao hàng đang lớn dần của FoodCo bắt đầu tự tổ chức xoay quanh quyền sở hữu của các microservice mới này.

Implementation

Triển khai

I won't dwell too much on this scaling mechanism, as we've already covered the fundamentals of microservices extensively within this book. For a more detailed discussion of how we can make a change like this happen, see Chapter 3 .

Tôi sẽ không bàn quá nhiều về cơ chế mở rộng này, vì chúng ta đã đề cập rất nhiều đến những nền tảng của microservice trong cuốn sách này. Để có một thảo luận chi tiết hơn về cách chúng ta có thể thực hiện một thay đổi như thế này, hãy xem Chương 3 .

Key benefits

Lợi ích chính

The fact that we have split out different types of workloads means we can now rightsize the underlying infrastructure needed for our system. Decomposed functionality that is only occasionally used could be turned off when not needed. Functionality that only has modest load requirements could be deployed onto small machines. On the other hand, functionality that is currently constrained could have more hardware thrown at it, perhaps combining functional decomposition with one of the other scaling axes—such as running multiple copies of our microservice.

Việc chúng ta đã tách riêng các loại khối lượng công việc khác nhau đồng nghĩa với việc giờ đây chúng ta có thể định cỡ phù hợp (rightsized) hạ tầng bên dưới cần thiết cho hệ thống của mình. Chức năng đã được phân rã mà chỉ thỉnh thoảng được dùng có thể được tắt khi không cần. Chức năng chỉ có yêu cầu tải khiêm tốn có thể được triển khai lên các cỗ máy nhỏ. Mặt khác, chức năng hiện đang bị giới hạn có thể được ném thêm nhiều phần cứng vào, có lẽ kết hợp phân rã theo chức năng với một trong các trục mở rộng khác—chẳng hạn như chạy nhiều bản sao của microservice.

This ability to rightsize the infrastructure required to run these workloads gives us more flexibility around optimizing the cost of the infrastructure we need to run the system. This is a key reason why large SaaS providers make such heavy use of microservices, as being able to find the right balance of infrastructure costs can help drive profitability.

Khả năng định cỡ phù hợp hạ tầng cần thiết để chạy các khối lượng công việc này cho chúng ta nhiều linh hoạt hơn trong việc tối ưu chi phí của hạ tầng mà chúng ta cần để chạy hệ thống. Đây là một lý do then chốt vì sao các nhà cung cấp SaaS lớn sử dụng microservice nhiều đến vậy, vì việc có thể tìm được điểm cân bằng đúng đắn cho chi phí hạ tầng có thể giúp thúc đẩy khả năng sinh lời.

By itself, functional decomposition isn't going to make our system more robust, but it at least opens up the opportunity for us to **build** a system that can tolerate a partial failure of functionality, something we explored in more detail in Chapter 12 .

Tự bản thân nó, phân rã theo chức năng sẽ không làm cho hệ thống của chúng ta vững chắc hơn, nhưng ít nhất nó mở ra cho chúng ta cơ hội xây dựng một hệ thống có thể chịu đựng được một sự cố cục bộ của chức năng, điều mà chúng ta đã khám phá chi tiết hơn trong Chương 12 .

Assuming you've taken the microservices route to functional decomposition, you'll have increased opportunity to use different technologies that can scale the decomposed microservice. For example, you could move the functionality to a programming language and runtime that is more efficient for the type of work you are doing, or perhaps you could migrate the data to a database better suited for your read or write traffic.

Giả sử bạn đã đi theo con đường microservice để phân rã theo chức năng, bạn sẽ có thêm cơ hội dùng các công nghệ khác nhau có thể mở rộng microservice đã được phân rã. Ví dụ, bạn có thể chuyển chức năng sang một ngôn ngữ lập trình và runtime hiệu quả hơn cho loại công việc bạn đang làm, hoặc có lẽ bạn có thể di chuyển dữ liệu sang một cơ sở dữ liệu phù hợp hơn cho lưu lượng đọc hoặc ghi của bạn.

Although in this chapter we’re focusing primarily on scale in the context of our software system, functional decomposition also makes it easier to scale our organization as well, a topic we’ll come back to in Chapter 15 .

Mặc dù trong chương này chúng ta đang chủ yếu tập trung vào việc mở rộng trong bối cảnh hệ thống phần mềm của mình, phân rã theo chức năng cũng giúp việc mở rộng tổ chức của chúng ta dễ dàng hơn, một chủ đề mà chúng ta sẽ quay lại trong Chương 15 .

Limitations

Hạn chế

As we explored in detail in Chapter 3 , splitting apart functionality can be a complex activity and is unlikely to deliver benefits in the short term. Of all the forms of scaling we’ve looked at, this is the one that will likely have the biggest impact on your application code—on both the **frontend** and the **backend**. It can also require a significant amount of work in the data tier if you make the choice to move to microservices as well.

Như chúng ta đã khám phá chi tiết trong Chương 3 , việc tách rời chức năng có thể là một hoạt động phức tạp và khó có khả năng mang lại lợi ích trong ngắn hạn. Trong tất cả các kiểu mở rộng mà chúng ta đã xem xét, đây là kiểu khả năng cao sẽ có tác động lớn nhất lên mã ứng dụng của bạn—ở cả frontend lẫn backend. Nó cũng có thể đòi hỏi một lượng công việc đáng kể ở tầng dữ liệu nếu bạn chọn chuyển sang microservice.

You’ll end up increasing the number of microservices you are running, which will increase the overall complexity of the system—potentially leading to more things that need to be maintained, made robust, and scaled. In general, when it comes to scaling a system, I try and exhaust the other possibilities before considering functional decomposition. My view on that may change if the shift to microservices potentially brings with it a host of other things that the organization is looking for. In the case of FoodCo, for example, its drive to grow its development team to both support more countries and deliver more features is key, so a migration toward microservices offers the company a chance to solve not only some of its system scaling issues but also its organizational scaling issues as well.

Bạn sẽ rốt cuộc tăng số lượng microservice mà bạn đang chạy, điều này sẽ làm tăng độ phức tạp tổng thể của hệ thống—có khả năng dẫn đến nhiều thứ hơn cần được bảo trì, làm cho vững chắc và mở rộng. Nhìn chung, khi nói đến việc mở rộng một hệ thống, tôi cố gắng vắt kiệt các khả năng khác trước khi cân nhắc phân rã theo chức năng. Quan điểm của tôi về điều đó có thể thay đổi nếu sự dịch chuyển sang microservice có khả năng mang theo nó một loạt những thứ khác mà tổ chức đang tìm kiếm. Ví dụ, trong trường hợp của FoodCo, nỗ lực phát triển đội ngũ phát triển của họ để vừa hỗ trợ nhiều quốc gia hơn vừa cung cấp nhiều tính năng hơn là then chốt, vậy nên một cuộc di chuyển sang microservice cho công ty cơ hội giải quyết không chỉ một số vấn đề mở rộng hệ thống mà còn cả các vấn đề mở rộng tổ chức của họ nữa.

Combining Models — Kết hợp các mô hình

One of the main drivers behind the original Scale Cube was to stop us from thinking narrowly in terms of one type of scaling, and to help us understand that it often makes sense to scale our application along multiple axes, depending on our need. Let’s come back to the example outlined in Figure 13-6 . We’ve extracted our Order functionality so it can now run on its own infrastructure. As a next step, we could scale the Order microservice in isolation by having multiple copies of it, as we see in Figure 13-7 .

Một trong những động lực chính đằng sau Scale Cube nguyên bản là ngăn chúng ta khỏi việc suy nghĩ hạn hẹp theo một kiểu mở rộng duy nhất, và giúp chúng ta hiểu rằng việc mở rộng ứng dụng của mình theo nhiều trục thường là hợp lý, tùy theo nhu cầu của chúng ta. Hãy quay lại ví dụ được nêu trong Hình 13-6 . Chúng ta đã trích xuất chức năng Order để giờ đây nó có thể chạy trên hạ tầng riêng của mình. Như một bước tiếp theo, chúng ta có thể mở rộng microservice Order một cách cô lập bằng cách có nhiều bản sao của nó, như chúng ta thấy trong Hình 13-7 .

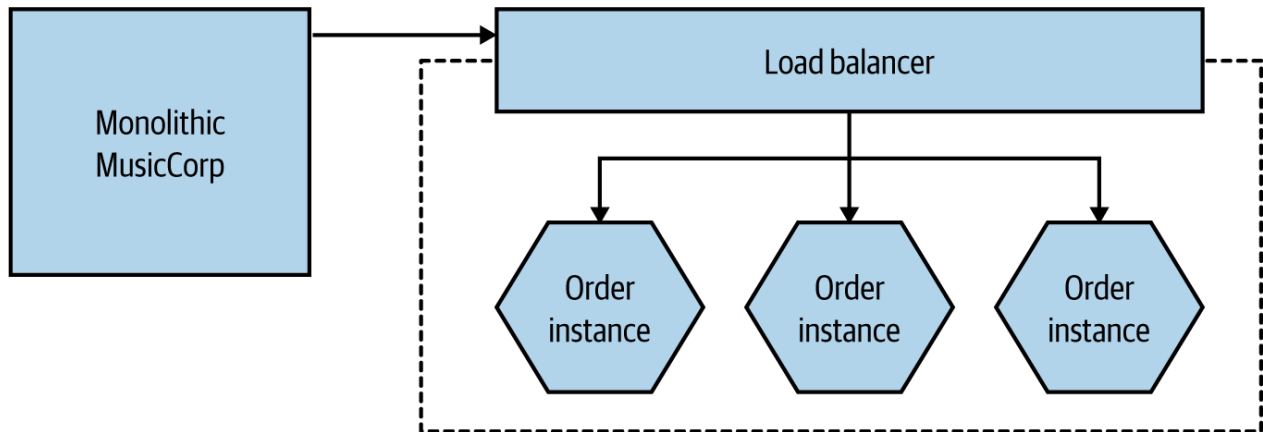


Figure 13-7. The extracted Order microservice is now duplicated for scale Next, we could decide to run different shards of our Order microservice for different geographical locales, as in Figure 13-8 . Horizontal duplication applies within each geographic boundary.

Hình 13-7. Microservice Order đã được trích xuất giờ đây được nhân bản để mở rộng Tiếp theo, chúng ta có thể quyết định chạy các shard khác nhau của microservice Order cho các địa phương địa lý khác nhau, như trong Hình 13-8 . Nhân bản theo chiều ngang áp dụng trong từng ranh giới địa lý.

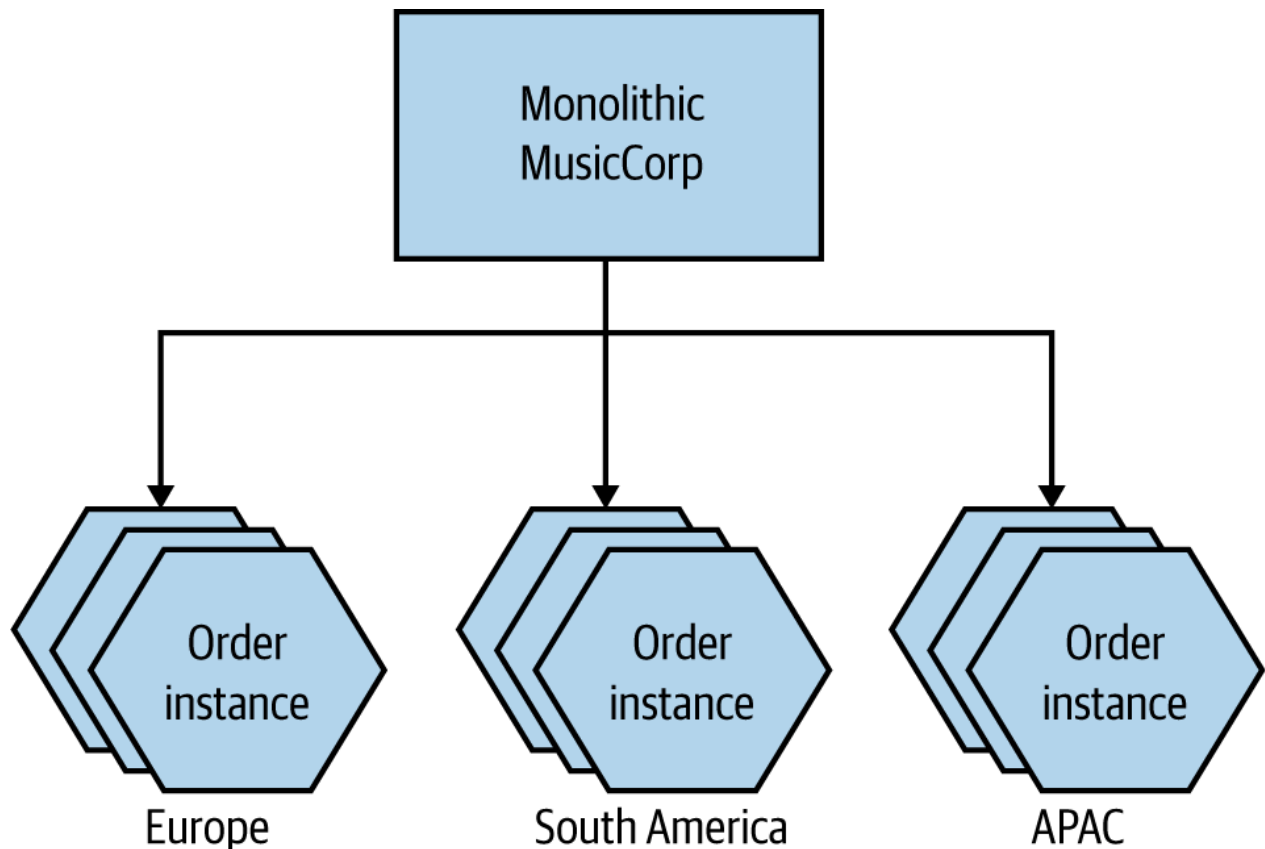


Figure 13-8. MusicCorp's Order microservice is now partitioned across geography, with duplication in each group

Hình 13-8. Microservice Order của MusicCorp giờ đây được phân vùng theo địa lý, với việc nhân bản trong từng nhóm

It's worth noting that by scaling along one axis, other axes might be easier to make use of. For example, the functional decomposition of Order enables us to then spin up multiple duplicates of the Order microservice, and also to partition the load on order processing. Without that initial functional decomposition, we'd be limited to applying those techniques on the monolith as a whole.

Cũng cần lưu ý rằng bằng cách mở rộng theo một trục, các trục khác có thể dễ tận dụng hơn. Ví dụ, việc phân rã theo chức năng của Order cho phép chúng ta sau đó khởi chạy nhiều bản nhân của microservice Order, và cũng phân vùng tải lên việc xử lý đơn hàng. Không có việc phân rã theo chức năng ban đầu đó, chúng ta sẽ bị giới hạn vào việc áp dụng những kỹ thuật đó lên toàn bộ monolith.

The goal when scaling isn't necessarily to scale along all axes, but we should be aware that we have these different mechanisms at our disposal. Given this choice, it's important we understand the pros and cons of each mechanism to work out which ones make the most sense.

Mục tiêu khi mở rộng không nhất thiết là mở rộng theo tất cả các trục, nhưng chúng ta nên ý thức rằng chúng ta có các cơ chế khác nhau này để sử dụng. Với lựa chọn này, điều quan trọng là chúng ta hiểu được ưu nhược điểm của từng cơ chế để xác định những cơ chế nào hợp lý nhất.

Start Small — Bắt đầu từ nhỏ

In The Art of Computer Programming (Addison-Wesley), Donald Knuth famously said:

Trong cuốn The Art of Computer Programming (Addison-Wesley), Donald Knuth đã nói một câu nổi tiếng:

The real problem is that programmers have spent far too much time worrying about efficiency in the wrong places and at the wrong times; premature optimization is the root of all evil (or at least most of it) in programming. Optimizing our system to solve problems we don't have is a great way to waste time that could be better spent on other activities, and also to ensure that we have a system that is needlessly more complex. Any form of optimization should be driven by real need. As we talked about in "Robustness" on page 388, adding new complexity to our system can introduce new sources of fragility as well. By scaling one part of our application, we create a weakness elsewhere. Our Order microservice may now be running on its own infrastructure, helping us better handle the load of the system, but we've got yet another microservice that we need to make sure is available if we want our system to function, and yet more infrastructure that has to be managed and made robust.

Vấn đề thực sự là các lập trình viên đã dành quá nhiều thời gian lo lắng về hiệu năng ở những chỗ sai và vào những thời điểm sai; tối ưu hóa sớm (premature optimization) là gốc rễ của mọi điều xấu xa (hoặc ít nhất là phần lớn) trong lập trình. Việc tối ưu hệ thống của chúng ta để giải quyết những vấn đề mà chúng ta không có là một cách tuyệt vời để lãng phí thời gian mà lẽ ra có thể được dùng tốt hơn vào các hoạt động khác, và cũng để đảm bảo rằng chúng ta có một hệ thống phức tạp hơn mức cần thiết một cách không cần thiết. Bất kỳ hình thức tối ưu nào cũng nên được dẫn dắt bởi nhu cầu thực sự. Như chúng ta đã bàn trong phần "Độ vững chắc (Robustness)" ở trang 388, việc thêm độ phức tạp mới vào hệ thống của chúng ta cũng có thể tạo ra các nguồn dễ vỡ mới. Bằng cách mở rộng một phần của ứng dụng, chúng ta tạo ra một điểm yếu ở nơi khác. Microservice Order của chúng ta giờ đây có thể đang chạy trên hạ tầng riêng của nó, giúp chúng ta xử lý tải của hệ thống tốt hơn, nhưng chúng ta lại có thêm một microservice nữa mà chúng ta cần đảm bảo luôn khả dụng nếu muốn hệ thống của mình hoạt động, và lại thêm hạ tầng phải được quản lý và làm cho vững chắc.

Even if you think you have identified a bottleneck, a process of experimentation is essential to make sure that you are right and that further work is justified. It's amazing to me how many people who would happily describe themselves as computer scientists seem not to have even a basic understanding of the scientific method. 5 If you've identified what you think is a problem, try to identify a small amount of work that can be done to confirm whether or not your proposed solution will work. In the context of scaling systems to handle load, having a suite of automated load tests, for example, can be incredibly useful. Run the tests to get a baseline and to re-create the bottleneck you are experiencing, make a change, and observe the difference. This isn't rocket science, but it is, even in a very small way, attempting to be at least vaguely scientific.

Ngay cả khi bạn nghĩ rằng mình đã xác định được một điểm nghẽn cổ chai, một quá trình thử nghiệm là thiết yếu để chắc chắn rằng bạn đúng và rằng công việc tiếp theo là chính đáng. Thật kinh ngạc với tôi khi có bao nhiêu người sẵn lòng tự mô tả mình là nhà khoa học máy tính lại có vẻ thậm chí không có một hiểu biết cơ bản về phương pháp khoa học.⁵ Nếu bạn đã xác định được thử bạn nghĩ là một vấn đề, hãy cố gắng xác định một lượng công việc nhỏ có thể làm để xác nhận xem giải pháp đề xuất của bạn có hiệu quả hay không. Trong bối cảnh mở rộng hệ thống để xử lý tải, ví dụ, việc có một bộ các bài kiểm thử tải tự động có thể cực kỳ hữu ích. Chạy các bài kiểm thử để có một mốc cơ sở và để tái hiện điểm nghẽn cổ chai mà bạn đang gặp, thực hiện một thay đổi, và quan sát sự khác

biệt. Đây không phải là khoa học tên lửa, nhưng nó, dù chỉ theo một cách rất nhỏ, là sự cố gắng để ít nhất là phần nào mang tính khoa học.

CQRS and Event Sourcing — CQRS và Event Sourcing

The Command Query Responsibility Segregation (CQRS) pattern refers to an alternate model for storing and querying information. Rather than our having a single model for how we both manipulate and retrieve data, as is common, responsibilities for reads and writes are instead handled by separate models. These separate read and write models, implemented in code, could be deployed as separate units, giving us the ability to scale reads and writes independently. CQRS is often, though not always, used in conjunction with event sourcing, where—rather than storing the current state of an entity as a single record—we instead project the state of an entity by looking at the history of events related to that entity.

Mẫu Command Query Responsibility Segregation (CQRS) đề cập đến một mô hình thay thế cho việc lưu trữ và truy vấn thông tin. Thay vì chúng ta có một mô hình duy nhất cho cả việc thao tác lẫn truy xuất dữ liệu, như thường thấy, trách nhiệm cho việc đọc và ghi thay vào đó được xử lý bởi các mô hình riêng biệt. Các mô hình đọc và ghi riêng biệt này, được triển khai trong mã, có thể được triển khai dưới dạng các đơn vị riêng biệt, cho chúng ta khả năng mở rộng việc đọc và ghi một cách độc lập. CQRS thường, dù không phải lúc nào cũng, được dùng kết hợp với event sourcing, trong đó—thay vì lưu trữ trạng thái hiện tại của một thực thể dưới dạng một bản ghi duy nhất—chúng ta thay vào đó suy ra (project) trạng thái của một thực thể bằng cách nhìn vào lịch sử các sự kiện liên quan đến thực thể đó.

Arguably, CQRS is doing something very similar in our application tier to what read replicas can do in the data tier, although due to the large number of different ways CQRS can be implemented, this is a simplification.

Có thể lập luận rằng CQRS đang làm một việc rất tương tự ở tầng ứng dụng của chúng ta với những gì các bản sao đọc có thể làm ở tầng dữ liệu, mặc dù do số lượng lớn các cách khác nhau mà CQRS có thể được triển khai, đây là một sự đơn giản hóa.

Personally, although I see value in the CQRS pattern in some situations, it's a complex pattern to execute well. I've spoken to very smart people who have hit not insignificant issues in making CQRS work. As such, if you are considering CQRS as a way to help scale your application, regard it as one of the harder forms of scaling you'd need to implement, and perhaps try some of the easier stuff first. For example, if you are simply read-constrained, a read replica may well be a significantly less risky and

Cá nhân tôi, mặc dù tôi thấy giá trị trong mẫu CQRS ở một số tình huống, đây là một mẫu phức tạp để thực thi cho tốt. Tôi đã nói chuyện với những người rất thông minh từng gặp những vấn đề không hề nhỏ trong việc làm cho CQRS hoạt động. Như vậy, nếu bạn đang cân nhắc CQRS như một cách để giúp mở rộng ứng dụng của mình, hãy coi nó là một trong những kiểu mở rộng khó hơn mà bạn cần triển khai, và có lẽ hãy thử một số thứ dễ hơn trước. Ví dụ, nếu bạn đơn giản chỉ bị giới hạn về đọc, một bản sao đọc rất có thể là một cách tiếp cận ít rủi ro hơn và

5 Don't get me started on people who start talking about hypotheses and then go about cherry-picking information to confirm their already-held beliefs.

5 Đừng có khiến tôi phải nói về những người bắt đầu nói về giả thuyết rồi sau đó đi lựa chọn thông tin một cách thiên vị để xác nhận những niềm tin sẵn có của họ.

faster approach to start with. My concerns about the complexity of implementation extend to event sourcing—there are some situations it fits really well, but it comes with a host of headaches that need to be accommodated. Both patterns require quite a shift in thinking for developers, which always makes things more challenging. If you decide to use either of these patterns, just make sure this increased cognitive load on your developers is worth it.

nhANH hơn đáng kể để khởi đầu. Những lo ngại của tôi về độ phức tạp của việc triển khai cũng mở rộng tới event sourcing—có một số tình huống nó cực kỳ phù hợp, nhưng nó đi kèm với một loạt cơn đau đầu cần được tính đến. Cả hai mẫu này đều đòi hỏi một sự dịch chuyển khá lớn trong tư duy của các nhà phát triển, điều luôn làm cho mọi thứ thử thách hơn. Nếu bạn quyết định dùng một trong hai mẫu này, hãy chỉ đảm bảo rằng tải nhận thức (cognitive load) tăng thêm này lên các nhà phát triển của bạn là xứng đáng.

One final note on CQRS and event sourcing: from the point of view of a microservice architecture, the decision to use or not use these techniques is an internal implementation detail of a microservice. If you've decided to implement a microservice by splitting responsibility for reads and writes across different processes and models, for example, this should be invisible to consumers of the microservice. If inbound requests need to be redirected to the appropriate model based on the request being made, make this the responsibility of the microservice implementing CQRS. Keeping these implementation details hidden from consumers gives you a lot of flexibility to change your mind later, or to change how you are using these patterns.

Một lưu ý cuối về CQRS và event sourcing: từ góc nhìn của một kiến trúc microservice, quyết định dùng hay không dùng các kỹ thuật này là một chi tiết triển khai nội bộ của một microservice. Ví dụ, nếu bạn đã quyết định triển khai một microservice bằng cách tách trách nhiệm cho đọc và ghi qua các tiến trình và mô hình khác nhau, thì điều này nên là vô hình đối với bên tiêu thụ của microservice. Nếu các request đến vào cần được chuyển hướng tới mô hình phù hợp dựa trên request đang được thực hiện, hãy biến việc này thành trách nhiệm của microservice triển khai CQRS. Việc giữ những chi tiết triển khai này ẩn khỏi bên tiêu thụ cho bạn rất nhiều linh hoạt để đổi ý sau này, hoặc để thay đổi cách bạn đang dùng các mẫu này.

Caching — Caching

Caching is a commonly used performance optimization whereby the previous result of some operation is stored so that subsequent requests can use this stored value rather than spending time and resources recalculating the value.

Caching là một kỹ thuật tối ưu hiệu năng thường được dùng, theo đó kết quả trước đó của một thao tác nào đó được lưu trữ để các request tiếp theo có thể dùng giá trị đã lưu này thay vì tốn thời gian và tài nguyên tính toán lại giá trị.

As an example, consider a Recommendation microservice that needs to check stock levels before recommending an item—there isn't any point in recommending something we don't have in stock! But we've decided to keep a local copy of stock levels in Recommendation (a form of client-side caching) to improve the latency of our operations—we avoid the need to check stock levels whenever we need to recommend something. The source of truth for stock levels is the Inventory microservice, which is considered to be the origin for the client **cache** in the Recommendation microservice. When Recommendation needs to look up a stock level, it can first look in its local cache. If the entry it needs is found, this is considered a cache hit. If the data is not found, it's a cache miss, which results in the need to fetch information from the downstream Inventory microservice. As the data in the origin

can of course change, we need some way to invalidate entries in Recommendation's cache so we know when the locally cached data is so out of date that it can no longer be used.

Lấy một ví dụ, hãy xét một microservice Recommendation cần kiểm tra mức tồn kho trước khi gợi ý một mặt hàng—chẳng có ý nghĩa gì khi gợi ý thứ mà chúng ta không có trong kho! Nhưng chúng ta đã quyết định giữ một bản sao cục bộ của mức tồn kho trong Recommendation (một dạng caching phía client) để cải thiện độ trễ của các thao tác của chúng ta—chúng ta tránh được nhu cầu kiểm tra mức tồn kho mỗi khi chúng ta cần gợi ý thứ gì đó. Nguồn chân lý (source of truth) cho mức tồn kho là microservice Inventory, vốn được coi là gốc (origin) cho client cache trong microservice Recommendation. Khi Recommendation cần tra cứu một mức tồn kho, trước tiên nó có thể tìm trong cache cục bộ của nó. Nếu mục cần tìm được tìm thấy, đây được coi là một lần trúng cache (cache hit). Nếu dữ liệu không được tìm thấy, đó là một lần trượt cache (cache miss), dẫn đến nhu cầu lấy thông tin từ microservice Inventory ở phía dưới. Vì dữ liệu ở gốc dĩ nhiên có thể thay đổi, chúng ta cần một cách nào đó để vô hiệu hóa cache (cache invalidation) các mục trong cache của Recommendation để chúng ta biết khi nào dữ liệu được cache cục bộ quá lỗi thời đến mức không còn dùng được nữa.

Caches can store the results of simple lookups, as in this example, but really they can store any piece of data, such as the result of a complex calculation. We can cache to help improve the performance of our system as part of helping reduce latency, to scale our application, and in some cases even to improve the robustness of our system. Taken together with the fact that there are a number of invalidation mechanisms we could use, and multiple places where we can cache, it means we have a lot of aspects to discuss when it comes to caching in a microservice architecture. Let's start by talking about what sorts of problems caches can help with.

Cache có thể lưu trữ kết quả của các tra cứu đơn giản, như trong ví dụ này, nhưng thực ra chúng có thể lưu trữ bất kỳ mẫu dữ liệu nào, chẳng hạn như kết quả của một phép tính phức tạp. Chúng ta có thể dùng cache để giúp cải thiện hiệu năng của hệ thống như một phần trong việc giúp giảm độ trễ, để mở rộng ứng dụng của mình, và trong một số trường hợp thậm chí để cải thiện độ vững chắc của hệ thống. Gộp lại với thực tế rằng có một số cơ chế vô hiệu hóa mà chúng ta có thể dùng, và nhiều nơi mà chúng ta có thể cache, điều đó đồng nghĩa với việc chúng ta có rất nhiều khía cạnh để bàn khi nói đến caching trong một kiến trúc microservice. Hãy bắt đầu bằng việc nói về những loại vấn đề mà cache có thể giúp giải quyết.

For Performance — Vì hiệu năng

With microservices, we are often concerned about the adverse impact of network latency and about the cost of needing to interact with multiple microservices to get some data. Fetching data from a cache can help greatly here, as we avoid the need for network calls to be made, which also has the impact of reducing load on downstream microservices. Aside from avoiding network hops, it reduces the need to create the data on each request. Consider a situation in which we are asking for the list of the most popular items by genre. This might involve an expensive join query at the database level. We could cache the results of this query, meaning we'll need to regenerate the results only when the cached data becomes invalidated.

Với microservice, chúng ta thường lo ngại về tác động bất lợi của độ trễ mạng và về chi phí của việc cần phải tương tác với nhiều microservice để lấy một mẫu dữ liệu nào đó. Việc lấy dữ liệu từ một cache có thể giúp ích rất nhiều ở đây, vì chúng ta tránh được nhu cầu thực hiện các lời gọi mạng, điều này cũng có tác động làm giảm tải lên các microservice ở

phía dưới. Bên cạnh việc tránh các bước nhảy mạng, nó giảm nhu cầu tạo ra dữ liệu trên mỗi request. Hãy xét một tình huống trong đó chúng ta yêu cầu danh sách các mặt hàng phổ biến nhất theo thể loại. Việc này có thể liên quan đến một truy vấn join tốn kém ở cấp cơ sở dữ liệu. Chúng ta có thể cache kết quả của truy vấn này, nghĩa là chúng ta sẽ chỉ cần tạo lại kết quả khi dữ liệu đã cache trở nên bị vô hiệu hóa.

For Scale — Vì mở rộng

If you can divert reads to caches, you can avoid contention on parts of your system to allow it to better scale. An example of this that we've already covered in this chapter is the use of database read replicas. The read traffic is served by the read replicas, reducing the load on the primary database node and allowing reads to be scaled effectively. The reads on a replica are done against data that might be stale. The read replica will eventually get updated by the replication from primary to replica node—this form of **cache invalidation** is handled automatically by the database technology.

Nếu bạn có thể chuyển hướng các lượt đọc tới các cache, bạn có thể tránh sự tranh chấp ở các phần của hệ thống để cho phép nó mở rộng tốt hơn. Một ví dụ về điều này mà chúng ta đã đề cập trong chương này là việc dùng các bản sao đọc cơ sở dữ liệu. Lưu lượng đọc được phục vụ bởi các bản sao đọc, làm giảm tải lên node cơ sở dữ liệu chính và cho phép việc đọc được mở rộng một cách hiệu quả. Các lượt đọc trên một bản sao được thực hiện trên dữ liệu mà có thể đã cũ. Bản sao đọc tốt cuộc sẽ được cập nhật bằng việc sao chép (replication) từ node chính sang node bản sao—dạng vô hiệu hóa cache này được xử lý tự động bởi công nghệ cơ sở dữ liệu.

More broadly, caching for scale is useful in any situation in which the origin is a point of contention. Putting caches between clients and the origin can reduce the load on the origin, better allowing it to scale.

Rộng hơn, caching để mở rộng hữu ích trong bất kỳ tình huống nào mà gốc là một điểm tranh chấp. Việc đặt các cache giữa client và gốc có thể giảm tải lên gốc, cho phép nó mở rộng tốt hơn.

For Robustness — Vì độ vững chắc

If you have an entire set of data available to you in a local cache, you have the potential to operate even if the origin is unavailable—this in turn could improve the robustness of your system. There are a few things to note about caching for robustness. The main thing is that you'd likely need to configure your cache invalidation mechanism to not automatically evict stale data, and to keep data in the cache until it can be updated. Otherwise, as data gets invalidated, it will be removed from the cache, resulting in a cache miss and a failure to get any data, as the origin is unavailable. This means you need to be prepared to read data that could be quite stale if the origin is offline. In some situations this might be fine, while in others it might be highly problematic.

Nếu bạn có sẵn một tập dữ liệu trọn vẹn trong một cache cục bộ, bạn có tiềm năng vận hành ngay cả khi gốc không khả dụng—điều này lại có thể cải thiện độ vững chắc của hệ thống của bạn. Có một vài điều cần lưu ý về caching để có độ vững chắc. Điều chính là bạn sẽ có khả năng cần cấu hình cơ chế vô hiệu hóa cache của mình để không tự động loại bỏ dữ liệu cũ, và để giữ dữ liệu trong cache cho đến khi nó có thể được cập nhật. Nếu không, khi dữ liệu bị vô hiệu hóa, nó sẽ bị xóa khỏi cache, dẫn đến một lần trượt cache và một thất bại trong việc lấy bất kỳ dữ liệu nào, vì gốc không khả dụng. Điều này có nghĩa là bạn

cần sẵn sàng đọc dữ liệu có thể khá cũ nếu gốc đang offline. Trong một số tình huống điều này có thể ổn, trong khi ở các tình huống khác nó có thể rất rắc rối.

Fundamentally, using a local cache to enable robustness in a situation in which the origin is unavailable means you are favoring availability over consistency.

Về cơ bản, việc dùng một cache cục bộ để cho phép độ vững chắc trong một tình huống mà gốc không khả dụng nghĩa là bạn đang ưu tiên tính khả dụng (availability) hơn tính nhất quán (consistency).

A technique I saw used at the Guardian , and subsequently elsewhere, was to crawl the existing “live” site periodically to generate a static version of the website that could be served in the event of an outage. Although this crawled version wasn’t as fresh as the cached content served from the live system, in a pinch it could ensure that a version of the site would get displayed.

Một kỹ thuật mà tôi từng thấy được dùng tại tờ Guardian , và sau đó ở những nơi khác, là định kỳ thu thập (crawl) trang “trực tiếp” hiện có để tạo ra một phiên bản tĩnh của website có thể được phục vụ trong trường hợp xảy ra sự cố ngừng hoạt động. Mặc dù phiên bản được crawl này không tươi mới bằng nội dung được cache phục vụ từ hệ thống trực tiếp, nhưng trong lúc khẩn cấp nó có thể đảm bảo rằng một phiên bản của trang sẽ được hiển thị.

Where to Cache — Cache ở đâu

As we have covered multiple times, microservices give you options. And this is absolutely the case with caching. We have lots of different places where we could cache. The different cache locations I’ll outline here have different trade-offs, and what sort of optimization you’re trying to make will likely point you toward the cache location that makes the most sense for you.

Như chúng ta đã đề cập nhiều lần, microservice cho bạn các lựa chọn. Và điều này hoàn toàn đúng với caching. Chúng ta có rất nhiều nơi khác nhau để cache. Các vị trí cache khác nhau mà tôi sẽ nêu ở đây có các đánh đổi khác nhau, và loại tối ưu mà bạn đang cố thực hiện sẽ có khả năng chỉ hướng bạn tới vị trí cache hợp lý nhất cho bạn.

To explore our caching options, let’s revisit a situation that we looked at back in “Data Decomposition Concerns” on page 81 , where we were extracting information about sales in MusicCorp. In Figure 13-9 , the Sales microservice maintains a record of items that have been sold. It tracks only the ID of the item sold and the timestamp of the sale. Occasionally, we want to ask the Sales microservice for a list of the top ten best sellers over the previous seven days.

Để khám phá các lựa chọn caching của chúng ta, hãy quay lại một tình huống mà chúng ta đã xem xét trong phần “Các mối quan tâm về phân rã dữ liệu (Data Decomposition Concerns)” ở trang 81 , nơi chúng ta đang trích xuất thông tin về doanh số ở MusicCorp. Trong Hình 13-9 , microservice Sales duy trì một bản ghi về các mặt hàng đã được bán. Nó chỉ theo dõi ID của mặt hàng được bán và dấu thời gian của lần bán. Đôi khi, chúng ta muốn hỏi microservice Sales về danh sách mười mặt hàng bán chạy nhất trong bảy ngày trước đó.

The problem is that the Sales microservice doesn’t know the names of the records, just the IDs. It’s not much use to say, “The best seller this week had the ID 366548, and we sold 35,345 copies!” We want to know the name of the CD with the ID 366548 as well. The Catalog microservice stores that information. This means, as Figure 13-9 shows, that when responding to the request for the top ten best sellers, the Sales microservice needs to request the names of the top ten IDs. Let’s look at how caching might help us and at what types of caches we could use.

Vấn đề là microservice Sales không biết tên của các bản ghi, chỉ biết các ID. Sẽ chẳng mấy hữu ích nếu nói “Mặt hàng bán chạy nhất tuần này có ID 366548, và chúng ta đã bán được 35.345 bản!” Chúng ta cũng muốn biết tên của chiếc CD có ID 366548. Microservice Catalog lưu trữ thông tin đó. Điều này có nghĩa là, như Hình 13-9 cho thấy, khi phản hồi yêu cầu về mười mặt hàng bán chạy nhất, microservice Sales cần yêu cầu tên của mười ID hàng đầu. Hãy xem caching có thể giúp chúng ta thế nào và những loại cache mà chúng ta có thể dùng.

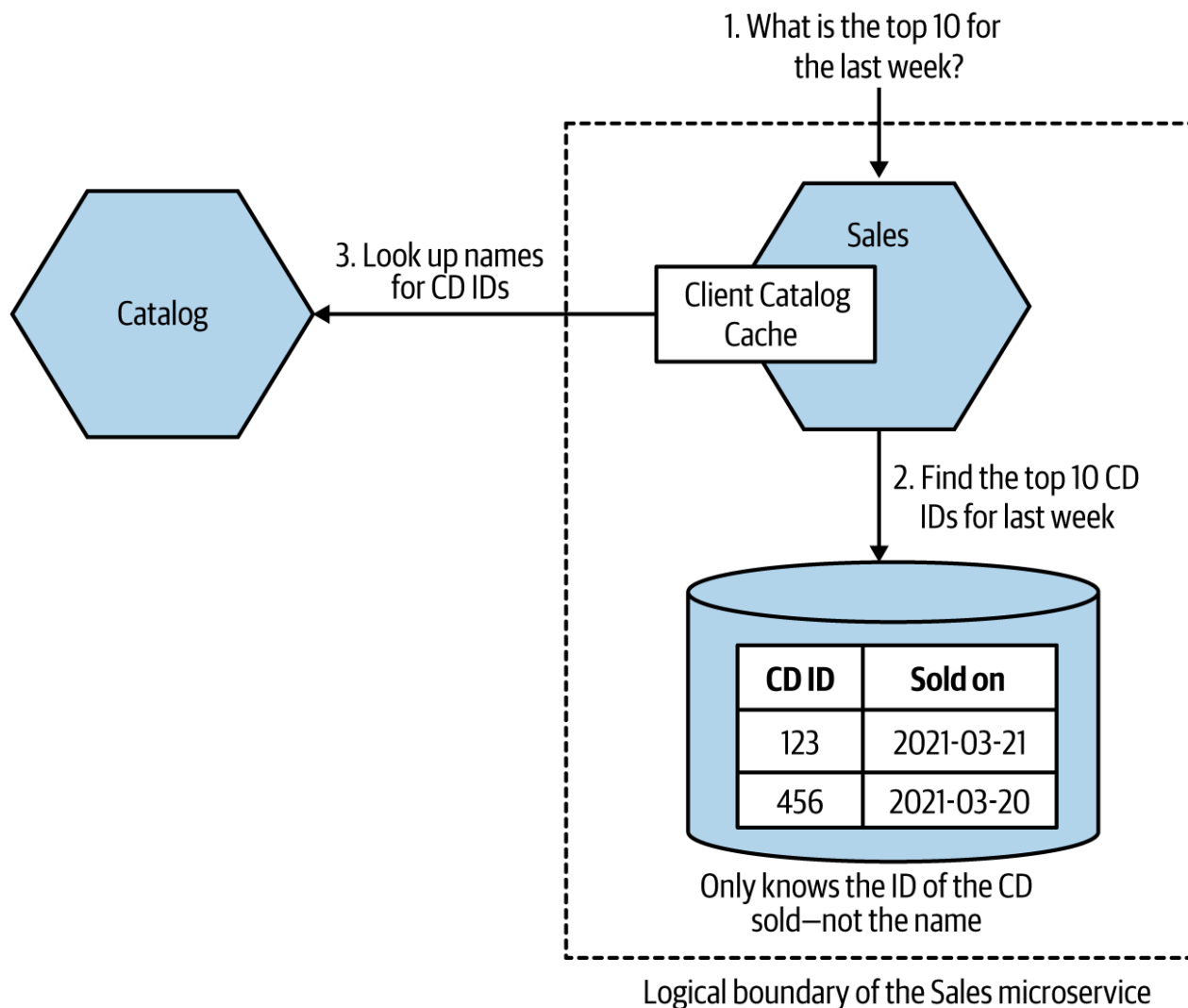


Figure 13-9. An overview of how MusicCorp works out the best sellers

Hình 13-9. Tổng quan về cách MusicCorp tính ra các mặt hàng bán chạy nhất

Client-side

Phía client

With client-side caching, the data is cached outside the scope of the origin. In our example, this could be done as simply as holding an in-memory hashtable with a mapping between ID and name of album inside the running Sales process, as in Figure 13-10. This means that generating our top ten takes any interaction with Catalog out of scope, assuming we get a cache hit for each lookup we need to make. It's important to note that our client cache could decide to cache only some of the

information we get from the microservice. For example, we might get back lots of information about a CD when we ask for information about it, but if all we care about is the name of the album, that's all we have to store in our local cache.

Với caching phía client, dữ liệu được cache bên ngoài phạm vi của gốc. Trong ví dụ của chúng ta, việc này có thể được thực hiện đơn giản như giữ một bảng băm (hashtable) trong bộ nhớ với một ánh xạ giữa ID và tên album bên trong tiến trình Sales đang chạy, như trong Hình 13-10. Điều này có nghĩa là việc tạo ra danh sách mười mặt hàng hàng đầu của chúng ta loại bỏ mọi tương tác với Catalog ra khỏi phạm vi, giả sử chúng ta có một lần trúng cache cho mỗi tra cứu mà chúng ta cần thực hiện. Cần lưu ý rằng client cache của chúng ta có thể quyết định chỉ cache một phần thông tin mà chúng ta nhận được từ microservice. Ví dụ, chúng ta có thể nhận lại rất nhiều thông tin về một chiếc CD khi chúng ta hỏi thông tin về nó, nhưng nếu tất cả những gì chúng ta quan tâm là tên của album, thì đó là tất cả những gì chúng ta phải lưu trữ trong cache cục bộ của mình.

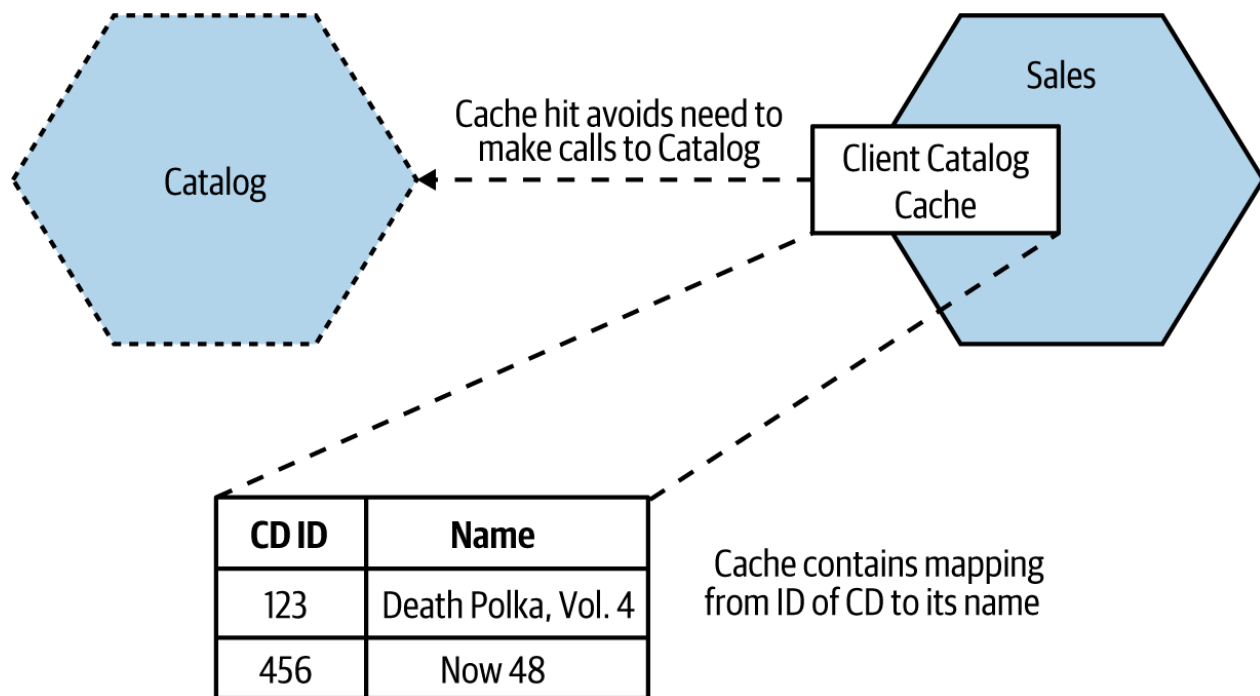


Figure 13-10. Sales holds a local copy of Catalog data

Hình 13-10. Sales giữ một bản sao cục bộ của dữ liệu Catalog

In general, client-side caches tend to be pretty effective, as they avoid the network call to the downstream microservice. This makes them suitable not only for caching for improved latency but also for caching for robustness.

Nhìn chung, các client cache có xu hướng khá hiệu quả, vì chúng tránh được lời gọi mạng tới microservice phía dưới. Điều này khiến chúng phù hợp không chỉ cho caching để cải thiện độ trễ mà còn cho caching để có độ vững chắc.

Client-side caching has a few downsides, though. Firstly, you tend to be more restricted in your options around invalidation mechanisms—something we'll explore more shortly. Secondly, when there's a lot of client-side caching going on, you can see a degree of inconsistency between clients. Consider a situation in which Sales, Recommendation, and Promotions microservices all have a client-side cache of data from Catalog. When the data in Catalog changes, whatever invalidation mechanism we are likely to use will be unable to guarantee that the data is refreshed at the exact

same moment of time in each of those three clients. This means that you could see a different view of the cached data in each of those clients at the same time. The more clients you have, the more problematic this is likely to be. Techniques such as notification-based invalidation, which we'll look at shortly, can help reduce this problem, but they won't eliminate it.

Tuy nhiên, caching phía client có một vài nhược điểm. Thứ nhất, bạn có xu hướng bị hạn chế hơn trong các lựa chọn xoay quanh cơ chế vô hiệu hóa—điều chúng ta sẽ khám phá nhiều hơn ngay sau đây. Thứ hai, khi có nhiều caching phía client đang diễn ra, bạn có thể thấy một mức độ không nhất quán giữa các client. Hãy xét một tình huống trong đó các microservice Sales, Recommendation, và Promotions đều có một client cache của dữ liệu từ Catalog. Khi dữ liệu trong Catalog thay đổi, bất kỳ cơ chế vô hiệu hóa nào mà chúng ta có khả năng dùng cũng sẽ không thể đảm bảo rằng dữ liệu được làm mới ở chính xác cùng một thời điểm trong mỗi client trong ba client đó. Điều này có nghĩa là bạn có thể thấy một góc nhìn khác nhau về dữ liệu đã cache trong mỗi client đó tại cùng một thời điểm. Càng có nhiều client, điều này càng có khả năng rắc rối. Các kỹ thuật như vô hiệu hóa dựa trên thông báo (notification), mà chúng ta sẽ xem ngay sau đây, có thể giúp giảm vấn đề này, nhưng chúng sẽ không loại bỏ được nó.

Another mitigation for this is to have a shared client-side cache, perhaps making use of a dedicated caching tool like Redis or memcached, as we see in Figure 13-11. Here, we avoid the problem of inconsistency between the different clients. This can also be more efficient in terms of resource use, as we are reducing the number of copies of this data we need to manage (caches often end up being in memory, and memory is often one of the biggest infrastructural constraints). The flip side is that our clients now need to make a round trip to the shared cache.

Một biện pháp giảm thiểu khác cho điều này là có một client cache dùng chung, có lẽ tận dụng một công cụ caching chuyên dụng như Redis hoặc memcached, như chúng ta thấy trong Hình 13-11. Ở đây, chúng ta tránh được vấn đề không nhất quán giữa các client khác nhau. Cách này cũng có thể hiệu quả hơn về mặt sử dụng tài nguyên, vì chúng ta đang giảm số lượng bản sao của dữ liệu này mà chúng ta cần quản lý (các cache thường rất cuộc nằm trong bộ nhớ, và bộ nhớ thường là một trong những ràng buộc hạ tầng lớn nhất). Mặt trái là các client của chúng ta giờ đây cần thực hiện một chuyến khứ hồi tới cache dùng chung.

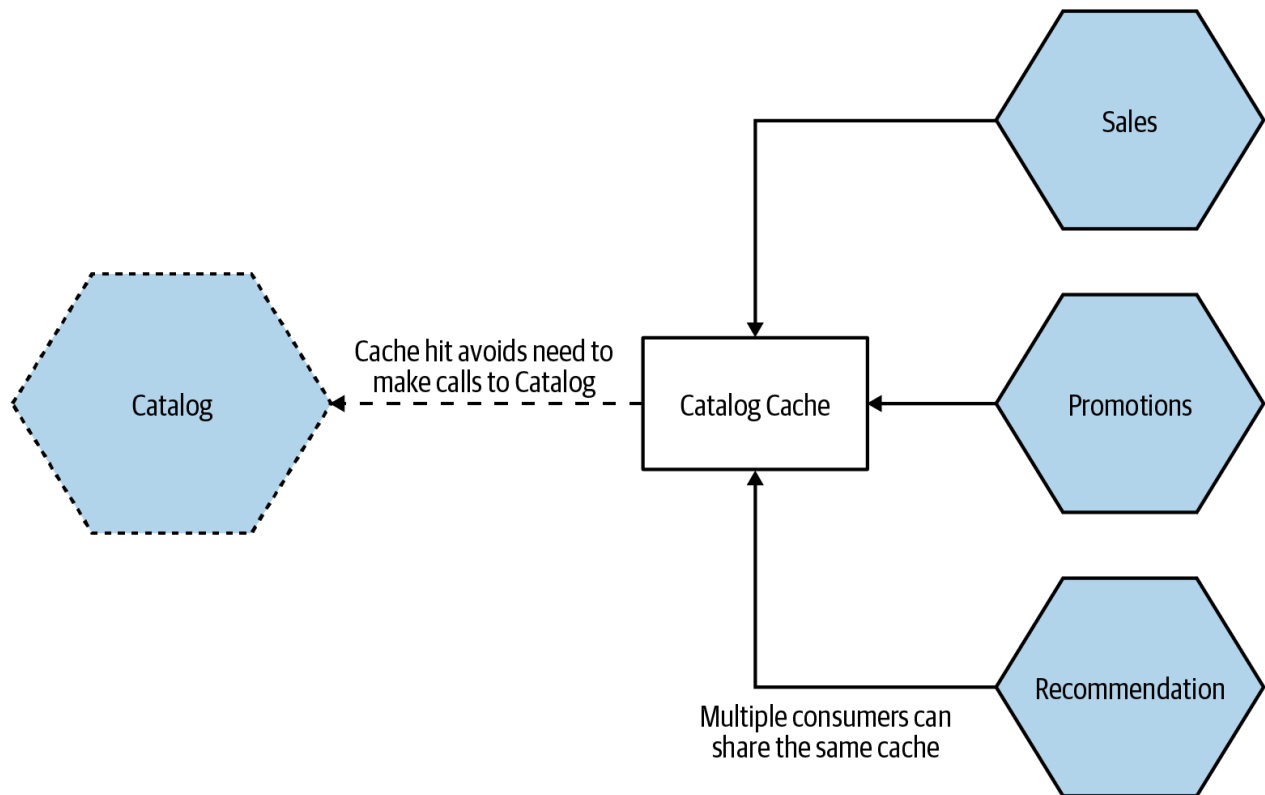


Figure 13-11. Multiple consumers of Catalog making use of a single shared cache

Hình 13-11. Nhiều bên tiêu thụ của Catalog cùng tận dụng một cache dùng chung duy nhất

Another thing to consider here is who is responsible for this shared cache. Depending on who owns it and how it is implemented, a shared cache like this can blur the lines between client-side caching and server-side caching, which we explore next.

Một điều khác cần cân nhắc ở đây là ai chịu trách nhiệm cho cache dùng chung này. Tùy thuộc vào ai sở hữu nó và nó được triển khai thế nào, một cache dùng chung như thế này có thể làm mờ ranh giới giữa caching phía client và caching phía server, mà chúng ta khám phá tiếp theo.

Server-side

Phía server

In Figure 13-12, we see our top ten sales example making use of caching on the server side. Here, the Catalog microservice itself maintains a cache on behalf of its consumers. When the Sales microservice makes its request for the names of the CDs, this information is transparently served up by a cache.

Trong Hình 13-12, chúng ta thấy ví dụ mười mặt hàng bán chạy của mình tận dụng caching ở phía server. Ở đây, chính microservice Catalog duy trì một cache thay mặt cho bên tiêu thụ của nó. Khi microservice Sales thực hiện yêu cầu của nó về tên của các chiếc CD, thông tin này được phục vụ một cách trong suốt bởi một cache.

Here, the Catalog microservice has full responsibility for managing the cache. Due to the nature of how these caches are typically implemented—such as an in-memory data structure, or a local dedicated caching node—it's easier to implement more sophisticated cache invalidation mechanisms. Write-through caches, for example (which we'll look at shortly), would be much

simpler to implement in this situation. Having a server-side cache also makes it easier to avoid the issue with different consumers seeing different cached values that can occur with client-side caching.

Ở đây, microservice Catalog có toàn bộ trách nhiệm quản lý cache. Do bản chất của cách những cache này thường được triển khai—chẳng hạn như một cấu trúc dữ liệu trong bộ nhớ, hoặc một node caching chuyên dụng cục bộ—việc triển khai các cơ chế vô hiệu hóa cache tinh vi hơn dễ dàng hơn. Ví dụ, các cache ghi-xuyên (write-through) (mà chúng ta sẽ xem ngay sau đây) sẽ đơn giản hơn nhiều để triển khai trong tình huống này. Việc có một cache phía server cũng giúp dễ dàng hơn trong việc tránh vấn đề các bên tiêu thụ khác nhau thấy các giá trị đã cache khác nhau, vốn có thể xảy ra với caching phía client.

It's worth noting that, although from the consumer point of view this caching is invisible (it's an internal implementation concern), that doesn't mean we'd have to implement this by caching in code in a microservice instance. We could, for example, maintain a reverse proxy within our microservice's logical boundary, use a hidden Redis node, or divert read queries to read replicas of a database.

Cần lưu ý rằng, mặc dù từ góc nhìn của bên tiêu thụ thì việc caching này là vô hình (đó là một mối quan tâm triển khai nội bộ), điều đó không có nghĩa là chúng ta phải triển khai việc này bằng caching trong mã trong một instance microservice. Ví dụ, chúng ta có thể duy trì một reverse proxy trong ranh giới logic của microservice, dùng một node Redis ẩn, hoặc chuyển hướng các truy vấn đọc tới các bản sao đọc của một cơ sở dữ liệu.

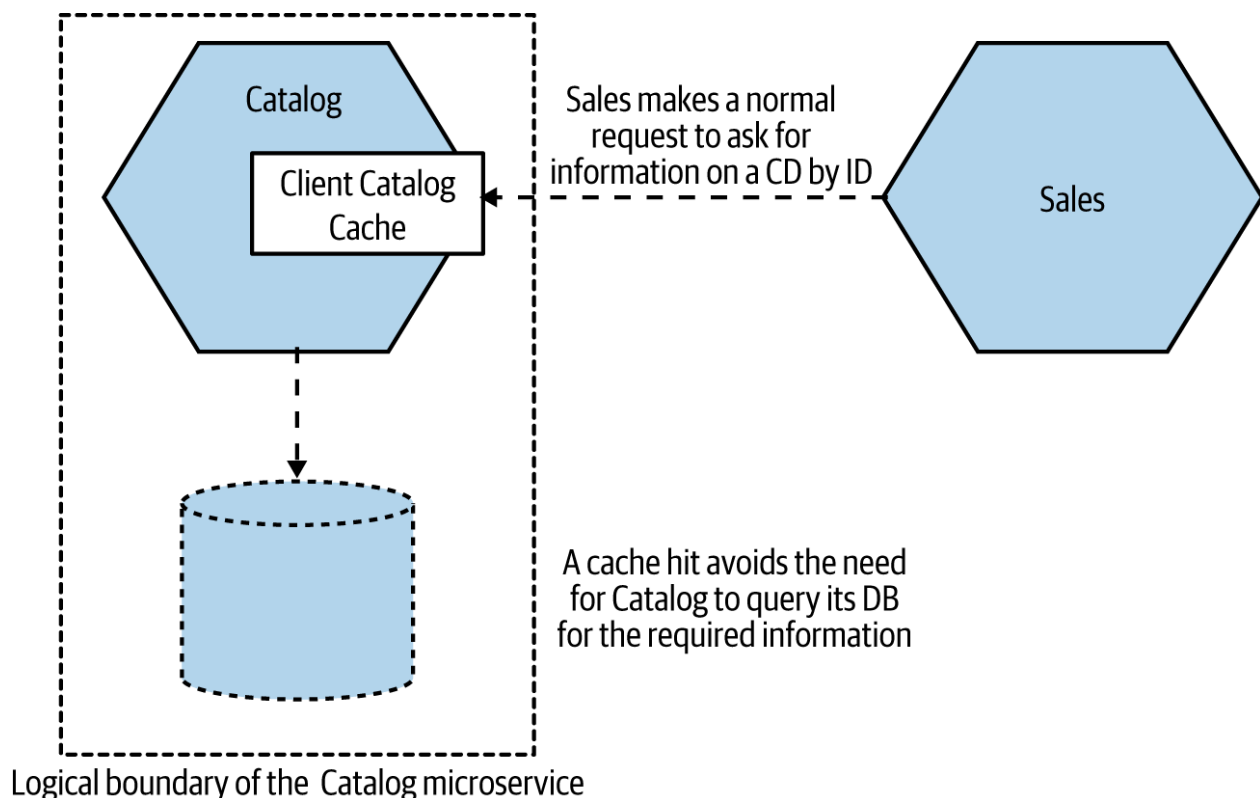


Figure 13-12. The Catalog implements caching internally, making it invisible to consumers

Hình 13-12. Catalog triển khai caching nội bộ, khiến nó vô hình đối với bên tiêu thụ

The major issue with this form of caching is that it has reduced scope for optimizing for latency, as a round trip by consumers to the microservice is still needed. By caching at or near to the perimeter

of a microservice, the cache can ensure that we don't need to carry out further expensive operations (like database queries), but the call needs to be made. This also reduces the effectiveness of this form of caching for any form of robustness.

Vấn đề lớn với hình thức caching này là nó có phạm vi giảm bớt cho việc tối ưu độ trễ, vì một chuyển khứ hồi của bên tiêu thụ tới microservice vẫn cần thiết. Bằng cách cache ở hoặc gần biên (perimeter) của một microservice, cache có thể đảm bảo rằng chúng ta không cần thực hiện các thao tác tốn kém tiếp theo (như các truy vấn cơ sở dữ liệu), nhưng lời gọi vẫn cần được thực hiện. Điều này cũng làm giảm tính hiệu quả của hình thức caching này cho bất kỳ hình thức độ vững chắc nào.

This might make this form of caching seem less useful, but there is huge value to transparently improving performance for all consumers of a microservice just by making a decision to implement caching internally. A microservice that is widely used across an organization may benefit hugely by implementing some form of internal caching, helping perhaps to improve response times for a number of consumers while also allowing the microservice to scale more effectively.

Điều này có thể khiến hình thức caching này có vẻ kém hữu ích hơn, nhưng có giá trị to lớn trong việc cải thiện hiệu năng một cách trong suốt cho tất cả bên tiêu thụ của một microservice chỉ bằng cách đưa ra một quyết định triển khai caching nội bộ. Một microservice được dùng rộng rãi khắp một tổ chức có thể được hưởng lợi rất nhiều bằng cách triển khai một dạng caching nội bộ nào đó, giúp có lẽ là cải thiện thời gian phản hồi cho một số bên tiêu thụ trong khi cũng cho phép microservice mở rộng hiệu quả hơn.

In the case of our top ten scenario, we'd have to consider whether or not this form of caching might help. Our decision would come down to what our main worry is. If it's about the end-to-end latency of the operation, how much time would a server-side cache save? Client-side caching would likely give us a better performance benefit.

Trong trường hợp kịch bản mười mặt hàng hàng đầu của chúng ta, chúng ta sẽ phải cân nhắc xem liệu hình thức caching này có giúp ích hay không. Quyết định của chúng ta sẽ phụ thuộc vào điều gì là mối lo chính của chúng ta. Nếu đó là về độ trễ đầu-cuối (end-to-end) của thao tác, thì một cache phía server sẽ tiết kiệm được bao nhiêu thời gian? Caching phía client có khả năng sẽ cho chúng ta một lợi ích hiệu năng tốt hơn.

Request cache

Request cache

With a request cache, we store a cached answer for the original request. So in Figure 13-13 for example, we store the actual top ten entries. Subsequent requests for the top ten best sellers result in the cached result being returned. No lookups in the Sales data needed, no round trips to Catalog—this is far and away the most effective cache in terms of optimizing for speed.

Với một request cache, chúng ta lưu trữ một câu trả lời đã cache cho request gốc. Vậy ví dụ trong Hình 13-13, chúng ta lưu trữ chính mười mục bán chạy. Các request tiếp theo về mười mặt hàng bán chạy nhất dẫn đến kết quả đã cache được trả về. Không cần tra cứu trong dữ liệu Sales, không cần các chuyển khứ hồi tới Catalog—đây là cache hiệu quả vượt trội nhất về mặt tối ưu cho tốc độ.

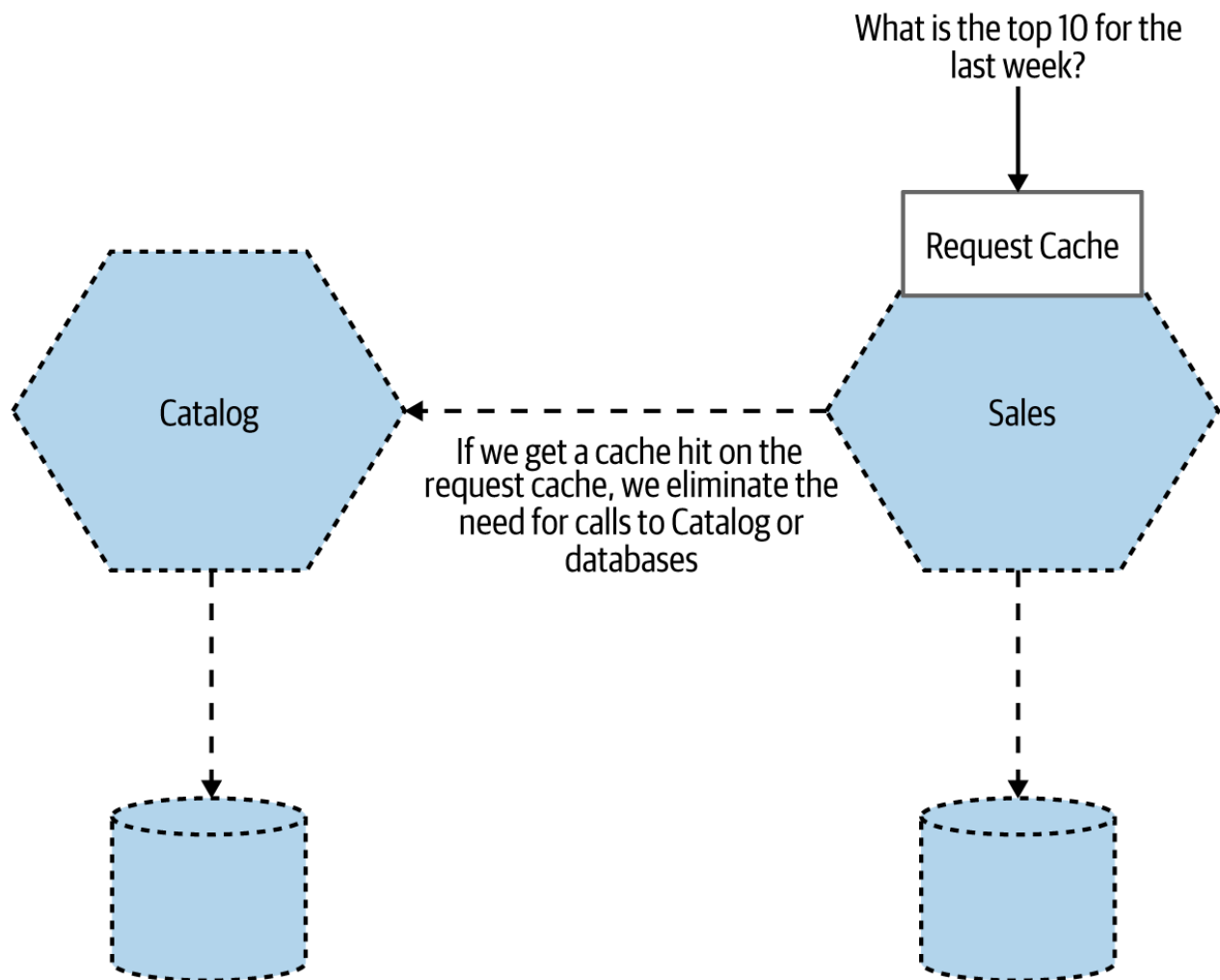


Figure 13-13. Caching the results of the top 10 request

Hình 13-13. Cache kết quả của request mười mặt hàng hàng đầu

The benefits here are obvious. This is super efficient, for one thing. However, we need to recognize that this form of caching is highly specific. We've only cached the result of this specific request. This means that other operations that hit Sales or Catalog won't be hitting a cache and thus won't benefit in any way from this form of optimization.

Các lợi ích ở đây là hiển nhiên. Đầu tiên, cách này cực kỳ hiệu quả. Tuy nhiên, chúng ta cần nhận ra rằng hình thức caching này rất cụ thể. Chúng ta chỉ cache kết quả của request cụ thể này. Điều này có nghĩa là các thao tác khác chạm tới Sales hoặc Catalog sẽ không trúng cache và do đó sẽ không được hưởng lợi theo bất kỳ cách nào từ hình thức tối ưu này.

Invalidation — Vô hiệu hóa

There are only two hard things in Computer Science: cache invalidation and naming things. —Phil Karlton

Trong Khoa học Máy tính chỉ có hai thứ khó: vô hiệu hóa cache và đặt tên cho mọi thứ.

—Phil Karlton

Invalidation is the process by which we evict data from our cache. It's an idea that is simple in concept but complex in execution, if for no other reason than there are a wealth of options in terms of how to implement it, and numerous trade-offs to consider in terms of making use of data that might be out of date. Fundamentally, though, it comes down to deciding in which situations a piece of cached data should be removed from your cache. Sometimes this happens because we are told a new version of a piece of data is available; at other times it might require us to assume our cached copy is stale and fetch a new copy from the origin.

Vô hiệu hóa là quá trình mà qua đó chúng ta loại bỏ dữ liệu khỏi cache của mình. Đây là một ý tưởng đơn giản về mặt khái niệm nhưng phức tạp trong thực thi, nếu không vì lý do nào khác thì vì có vô số lựa chọn về cách triển khai nó, và nhiều đánh đổi cần cân nhắc về việc sử dụng dữ liệu có thể đã lỗi thời. Tuy nhiên, về cơ bản, nó rốt cuộc là việc quyết định trong những tình huống nào thì một mẫu dữ liệu đã cache nên được loại bỏ khỏi cache của bạn. Đôi khi điều này xảy ra vì chúng ta được báo rằng một phiên bản mới của một mẫu dữ liệu đã có sẵn; những lúc khác nó có thể đòi hỏi chúng ta giả định rằng bản sao đã cache của mình là cũ và lấy một bản sao mới từ gốc.

Given the options around invalidation, I think it's a good idea to look at a few of the options you could make use of in a microservice architecture. Please don't consider this to be an exhaustive overview of every option, though!

Với các lựa chọn xoay quanh vô hiệu hóa, tôi nghĩ rằng việc xem xét một vài lựa chọn mà bạn có thể tận dụng trong một kiến trúc microservice là một ý hay. Tuy nhiên, xin đừng coi đây là một tổng quan đầy đủ về mọi lựa chọn!

Time to live (TTL)

Time to live (TTL)

This is one of the simplest mechanisms to use for cache invalidation. Each entry in the cache is assumed to be valid for only a certain duration in time. After that time has passed, the data is invalidated, and we fetch a new copy. We can specify the duration of validity using a simple time to live (TTL) duration—so a TTL of five minutes means our cache would happily provide the cache data for up to five minutes, after which the cached entry is considered to be invalidated and a fresh copy is required. Variations on this theme can include using a timestamp for expiration, which in some situations can be more effective, especially if you are reading through multiple levels of cache.

Đây là một trong những cơ chế đơn giản nhất để dùng cho việc vô hiệu hóa cache. Mỗi mục trong cache được giả định là chỉ hợp lệ trong một khoảng thời gian nhất định. Sau khi thời gian đó đã trôi qua, dữ liệu bị vô hiệu hóa, và chúng ta lấy một bản sao mới. Chúng ta có thể chỉ định thời lượng hợp lệ bằng một khoảng time to live (TTL) đơn giản—vậy một TTL năm phút nghĩa là cache của chúng ta sẽ vui vẻ cung cấp dữ liệu cache trong tối đa năm phút, sau đó mục đã cache được coi là bị vô hiệu hóa và cần một bản sao tươi mới. Các biến thể của chủ đề này có thể bao gồm việc dùng một dấu thời gian cho việc hết hạn, điều mà trong một số tình huống có thể hiệu quả hơn, đặc biệt nếu bạn đang đọc qua nhiều tầng cache.

HTTP supports both a TTL (via the Cache-Control header) and the ability to set a timestamp for expiration through the Expires header on responses, which can be incredibly useful. This means that the origin itself is able to tell downstream clients how long they should assume data is fresh for. Coming back to our Inventory microservice, we could imagine a situation in which the Inventory microservice gives a shorter TTL for stock levels of fast-selling items, or for items for which we are almost out of stock. For items that we don't sell much of, it could provide a longer TTL. This represents

a somewhat advanced use of HTTP cache controls, and tuning cache controls on a per-response basis like this is something I'd do only when tuning the effectiveness of a cache. A simple one-size-fits-all TTL for any given resource type is a sensible starting point.

HTTP hỗ trợ cả một TTL (qua header Cache-Control) lẫn khả năng đặt một dấu thời gian cho việc hết hạn thông qua header Expires trên các phản hồi, điều này có thể cực kỳ hữu ích. Điều này có nghĩa là chính gốc có thể cho các client phía dưới biết chúng nên giả định dữ liệu là tươi mới trong bao lâu. Quay lại microservice Inventory của chúng ta, chúng ta có thể hình dung một tình huống trong đó microservice Inventory cấp một TTL ngắn hơn cho mức tồn kho của các mặt hàng bán chạy, hoặc cho các mặt hàng mà chúng ta gần như hết hàng. Đối với các mặt hàng mà chúng ta không bán được nhiều, nó có thể cấp một TTL dài hơn. Điều này thể hiện một cách dùng các điều khiển cache HTTP có phần nâng cao, và việc tinh chỉnh các điều khiển cache trên từng phản hồi như thế này là điều tôi chỉ làm khi tinh chỉnh tính hiệu quả của một cache. Một TTL đồng nhất đơn giản cho bất kỳ loại tài nguyên nào là một điểm khởi đầu hợp lý.

Even if you're not using HTTP, the idea of the origin giving hints to the client as to how (and if) data should be cached is a really powerful concept. This means you don't have to guess about these things on the client side; you can actually make an informed choice about how to handle a piece of data.

Ngay cả khi bạn không dùng HTTP, ý tưởng gốc đưa ra các gợi ý cho client về việc dữ liệu nên (và liệu có nên) được cache thế nào là một khái niệm thực sự mạnh mẽ. Điều này có nghĩa là bạn không phải đoán về những thứ này ở phía client; bạn thực sự có thể đưa ra một lựa chọn có cơ sở về cách xử lý một mẫu dữ liệu.

HTTP does have more advanced caching capabilities than this, and we'll look at conditional GETs as an example of that in a moment.

HTTP có những khả năng caching nâng cao hơn thế này, và chúng ta sẽ xem các conditional GET như một ví dụ về điều đó trong chốc lát.

One of the challenges of TTL-based invalidation is that although it is simple to implement, it is a pretty blunt instrument. If we request a fresh copy of the data that has a five-minute TTL, and a second later the data at the origin changes, then our cache will be operating on out-of-date data for the remaining four minutes and 59 seconds. So the simplicity of implementation needs to be balanced against how much tolerance you have around operating on out-of-date data.

Một trong những thách thức của việc vô hiệu hóa dựa trên TTL là mặc dù nó đơn giản để triển khai, nó là một công cụ khá thô. Nếu chúng ta yêu cầu một bản sao tươi mới của dữ liệu có TTL năm phút, và một giây sau dữ liệu ở gốc thay đổi, thì cache của chúng ta sẽ vận hành trên dữ liệu lỗi thời trong bốn phút 59 giây còn lại. Vậy nên sự đơn giản của việc triển khai cần được cân bằng với mức độ dung sai mà bạn có xoay quanh việc vận hành trên dữ liệu lỗi thời.

Conditional GETs

Conditional GET

Worth a mention, as this is overlooked, is the ability to issue conditional GET requests with HTTP. As we've just touched on, HTTP provides the ability to specify Cache-Control and Expires headers on responses to enable smarter client-side caching. But if we're working directly with HTTP, we have another option in our arsenal of HTTP goodies: entity tags, or ETags. An ETag is used to determine whether the value of a resource has changed. If I update a customer record, the URI to the resource is the same but the value is different, so I would expect the ETag to change. This becomes powerful when we're using what is called a conditional GET. When making a GET request, we can specify

additional headers, telling the service to send us the resource only if certain criteria are met.

Đáng được nhắc đến, vì điều này thường bị bỏ qua, là khả năng phát các request conditional GET với HTTP. Như chúng ta vừa chạm đến, HTTP cung cấp khả năng chỉ định các header Cache-Control và Expires trên các phản hồi để cho phép caching phía client thông minh hơn. Nhưng nếu chúng ta làm việc trực tiếp với HTTP, chúng ta có một lựa chọn khác trong kho vũ khí những món hay ho của HTTP: entity tag, hay ETag. Một ETag được dùng để xác định xem giá trị của một tài nguyên có thay đổi hay không. Nếu tôi cập nhật một bản ghi khách hàng, URI tới tài nguyên là như nhau nhưng giá trị thì khác, vậy nên tôi sẽ kỳ vọng ETag thay đổi. Điều này trở nên mạnh mẽ khi chúng ta dùng cái gọi là một conditional GET. Khi thực hiện một request GET, chúng ta có thể chỉ định các header bổ sung, bảo service chỉ gửi cho chúng ta tài nguyên nếu một số tiêu chí nhất định được thỏa mãn.

For example, let's imagine we fetch a customer record, and its ETag comes back as o5t6fkd2sa. Later on, perhaps because a Cache-Control directive has told us the resource should be considered stale, we want to make sure we get the latest version. When issuing the subsequent GET request, we can pass in an If-None-Match: o5t6fkd2sa. This tells the server that we want the resource at the specified URI, unless it already matches this ETag value. If we already have the up-to-date version, the service sends us a 304 Not Modified response, telling us we have the latest version. If there is a newer version available, we get a 200 OK with the changed resource and a new ETag for the resource.

Ví dụ, hãy hình dung chúng ta lấy một bản ghi khách hàng, và ETag của nó trả về là o5t6fkd2sa. Sau đó, có lẽ vì một chỉ thị Cache-Control đã cho chúng ta biết tài nguyên nên được coi là cũ, chúng ta muốn chắc chắn rằng mình lấy được phiên bản mới nhất. Khi phát request GET tiếp theo, chúng ta có thể truyền vào một If-None-Match: o5t6fkd2sa. Điều này cho server biết rằng chúng ta muốn tài nguyên tại URI được chỉ định, trừ khi nó đã khớp với giá trị ETag này. Nếu chúng ta đã có phiên bản mới nhất, service gửi cho chúng ta một phản hồi 304 Not Modified, báo cho chúng ta biết rằng chúng ta có phiên bản mới nhất. Nếu có một phiên bản mới hơn sẵn có, chúng ta nhận được một 200 OK với tài nguyên đã thay đổi và một ETag mới cho tài nguyên.

Of course, with a conditional GET, we still make the request from client to server. If you are caching to reduce network round trips, this may not help you much. Where it is useful is in avoiding the cost of needlessly regenerating resources. With TTL-based invalidation, the client asks for a new copy of the resource, even if the resource hasn't changed—the microservice receiving this request then has to regenerate that resource, even if it ends up being exactly the same as what the client already has. If the cost of creating the response is high, perhaps requiring an expensive set of database queries, then conditional GET requests can be an effective mechanism.

Dĩ nhiên, với một conditional GET, chúng ta vẫn thực hiện request từ client tới server. Nếu bạn đang dùng cache để giảm các chuyến khứ hồi mạng, điều này có thể không giúp bạn được nhiều. Nơi nó hữu ích là trong việc tránh chi phí tạo lại tài nguyên một cách không cần thiết. Với vô hiệu hóa dựa trên TTL, client yêu cầu một bản sao mới của tài nguyên, ngay cả khi tài nguyên chưa thay đổi—microservice nhận request này sau đó phải tạo lại tài nguyên đó, ngay cả khi nó rốt cuộc giống hệt với những gì client đã có. Nếu chi phí tạo ra phản hồi cao, có lẽ đòi hỏi một tập các truy vấn cơ sở dữ liệu tốn kém, thì các request conditional GET có thể là một cơ chế hiệu quả.

Notification-based

Dựa trên thông báo

With notification-based invalidation, we use events to help subscribers know if their local cache

entries need to be invalidated. To my mind, this is the most elegant mechanism for invalidation, though that is balanced by its relative complexity with respect to TTL-based invalidation.

Với vô hiệu hóa dựa trên thông báo, chúng ta dùng các sự kiện để giúp các bên đăng ký (subscriber) biết liệu các mục cache cục bộ của họ có cần được vô hiệu hóa hay không. Theo tôi, đây là cơ chế thanh lịch nhất cho việc vô hiệu hóa, mặc dù điều đó được cân bằng bởi độ phức tạp tương đối của nó so với vô hiệu hóa dựa trên TTL.

In Figure 13-14 , our Recommendation microservice is maintaining a client-side cache. Entries in that cache are invalidated when the Inventory microservice fires a Stock Change event, letting Recommendation (or any other subscribers to this event) know that the stock level has increased or decreased for a given item.

Trong Hình 13-14 , microservice Recommendation của chúng ta đang duy trì một client cache. Các mục trong cache đó bị vô hiệu hóa khi microservice Inventory phát ra một sự kiện Stock Change, cho Recommendation (hoặc bất kỳ bên đăng ký nào khác của sự kiện này) biết rằng mức tồn kho đã tăng hoặc giảm đối với một mặt hàng nhất định.

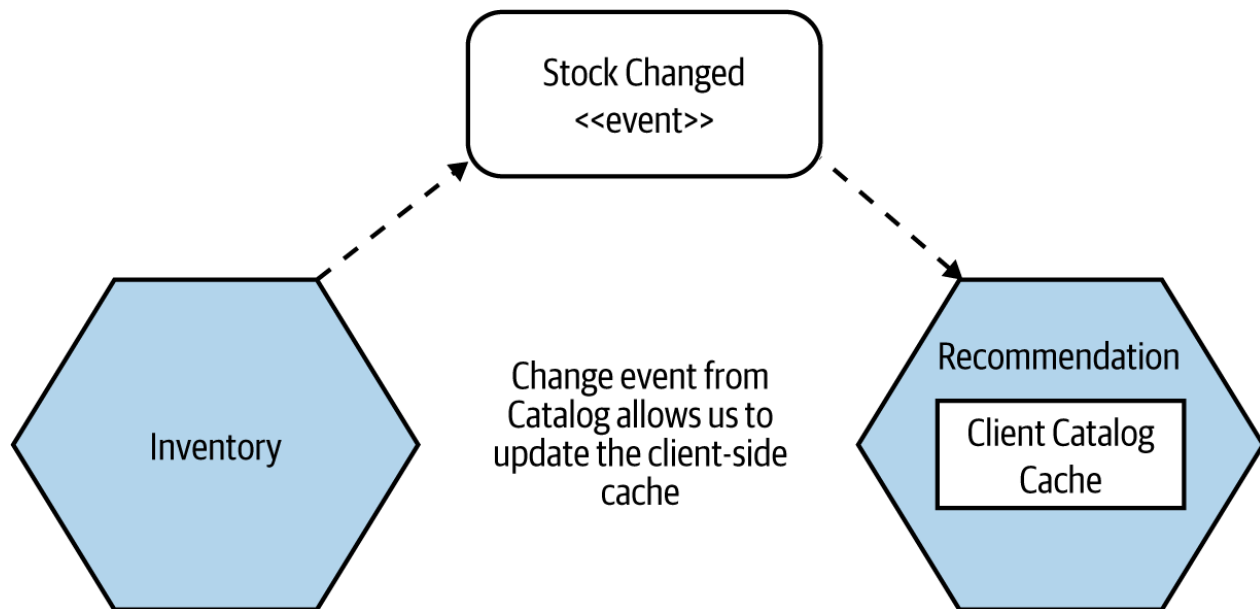


Figure 13-14. Inventory fires Stock Change events, which Recommendation can use to update its local cache

Hình 13-14. Inventory phát ra các sự kiện Stock Change, mà Recommendation có thể dùng để cập nhật cache cục bộ của nó

The main benefit of this mechanism is that it reduces the potential window wherein the cache is serving stale data. The window in which a cache might now be serving stale data is limited to the time taken for the notification to be sent and processed. Depending on the mechanism you use to send the notification, this could be pretty fast.

Lợi ích chính của cơ chế này là nó giảm cửa sổ thời gian tiềm năng mà trong đó cache đang phục vụ dữ liệu cũ. Cửa sổ trong đó một cache giờ đây có thể đang phục vụ dữ liệu cũ bị giới hạn ở thời gian cần để thông báo được gửi đi và được xử lý. Tùy thuộc vào cơ chế bạn dùng để gửi thông báo, điều này có thể khá nhanh.

The downside here is the complexity of implementation. We need the origin to be able to emit notifications, and we need interested parties to be able to respond to these notifications. Now, this

is a natural place to make use of something like a message broker, as this model fits neatly into the typical pub/sub-style interactions that many brokers provide. The additional guarantees that the broker might be able to give us could also be helpful. That said, as we’ve already discussed in “Message Brokers” on page 135 , there is an overhead to managing messaging middleware, and it might be overkill if you were using it only for this purpose. If you were using brokers for other forms of inter-microservice communication, however, it would make sense to make use of technology that you already had on hand.

Nhược điểm ở đây là độ phức tạp của việc triển khai. Chúng ta cần gốc có khả năng phát ra các thông báo, và chúng ta cần các bên quan tâm có khả năng phản hồi các thông báo này. Giờ thì đây là một nơi tự nhiên để tận dụng một thứ như một message broker, vì mô hình này khớp gọn gàng vào các tương tác kiểu pub/sub điển hình mà nhiều broker cung cấp. Các đảm bảo bổ sung mà broker có thể cấp cho chúng ta cũng có thể hữu ích. Dẫu vậy, như chúng ta đã bàn trong phần “Message Broker” ở trang 135 , có một chi phí phụ để quản lý phần mềm trung gian (middleware) nhắn tin, và việc dùng nó có thể là quá mức cần thiết nếu bạn chỉ dùng nó cho mục đích này. Tuy nhiên, nếu bạn đang dùng các broker cho các hình thức giao tiếp giữa các microservice khác, thì việc tận dụng công nghệ mà bạn đã có sẵn sẽ là hợp lý.

One problem to be aware of when using notification-based invalidation is that you may want to know whether the notification mechanism is actually working or not. Consider a situation in which we haven’t received any Stock Changed events from Inventory for a while. Does that mean we haven’t sold items or had items restocked during that time? Perhaps. It could also mean our notification mechanism is down and that we aren’t being sent updates any more. If this is a concern, then we could send a heartbeat event via the same notification mechanism— Recommendation , in our case—to let subscribers know that notifications are still coming, but nothing has actually changed. If a heartbeat event isn’t received, the client can assume an issue and could do whatever is most appropriate—perhaps informing a user that they are seeing stale data, or perhaps just turning off functionality.

Một vấn đề cần lưu ý khi dùng vô hiệu hóa dựa trên thông báo là bạn có thể muốn biết liệu cơ chế thông báo có thực sự đang hoạt động hay không. Hãy xét một tình huống trong đó chúng ta đã không nhận được bất kỳ sự kiện Stock Changed nào từ Inventory trong một thời gian. Điều đó có nghĩa là chúng ta đã không bán mặt hàng nào hoặc không có mặt hàng nào được nhập lại kho trong thời gian đó? Có lẽ vậy. Nó cũng có thể có nghĩa là cơ chế thông báo của chúng ta đang sập và rằng chúng ta không còn được gửi các bản cập nhật nữa. Nếu đây là một mối lo, thì chúng ta có thể gửi một sự kiện heartbeat qua chính cơ chế thông báo đó—tới Recommendation , trong trường hợp của chúng ta—để cho các bên đăng ký biết rằng các thông báo vẫn đang đến, nhưng không có gì thực sự thay đổi. Nếu một sự kiện heartbeat không được nhận, client có thể giả định có một vấn đề và có thể làm bất cứ điều gì phù hợp nhất—có lẽ là thông báo cho người dùng rằng họ đang xem dữ liệu cũ, hoặc có lẽ chỉ đơn giản là tắt chức năng.

You also need to consider what the notification contains. If the notification just says “this thing has changed” without saying what the change is, then when receiving the notification, a consumer would need to go the origin and fetch the new data. On the other hand, if the notification contains the current state of the data, then consumers can load that directly into their local cache. Having a notification containing more data can cause issues regarding size and also carries a risk of potentially exposing sensitive data too broadly. We previously explored this trade-off in more depth when looking at **event-driven** communication in “What’s in an Event?” on page 111 .

Bạn cũng cần cân nhắc thông báo chứa những gì. Nếu thông báo chỉ nói “thứ này đã thay đổi” mà không nói thay đổi đó là gì, thì khi nhận được thông báo, một bên tiêu thụ sẽ cần

đi tới gốc và lấy dữ liệu mới. Mặt khác, nếu thông báo chứa trạng thái hiện tại của dữ liệu, thì các bên tiêu thụ có thể nạp trực tiếp dữ liệu đó vào cache cục bộ của họ. Việc có một thông báo chứa nhiều dữ liệu hơn có thể gây ra các vấn đề liên quan đến kích thước và cũng mang một rủi ro tiềm tàng là phơi bày dữ liệu nhạy cảm ra quá rộng. Trước đây chúng ta đã khám phá sự đánh đổi này sâu hơn khi xem xét giao tiếp hướng sự kiện trong phần “Một Sự kiện gồm những gì? (What’s in an Event?)” ở trang 111 .

Write-through

Ghi-xuyên

With a write-through cache, the cache is updated at the same time as the state in the origin. “At the same time” is where write-through caches get tricky, of course. Implementing a write-through mechanism on a server-side cache is somewhat straightforward, as you could update a database and an in-memory cache within the same **transaction** without too much difficulty. If the cache is elsewhere, it’s more difficult to reason about what “at the same time” means in terms of these entries being updated.

Với một cache ghi-xuyên, cache được cập nhật cùng lúc với trạng thái ở gốc. “Cùng lúc” dĩ nhiên là chỗ mà các cache ghi-xuyên trở nên khó nhằn. Việc triển khai một cơ chế ghi-xuyên trên một cache phía server thì có phần đơn giản, vì bạn có thể cập nhật một cơ sở dữ liệu và một cache trong bộ nhớ trong cùng một giao dịch mà không quá khó khăn. Nếu cache nằm ở nơi khác, việc suy luận về “cùng lúc” nghĩa là gì xét về việc các mục này được cập nhật sẽ khó hơn.

Due to this difficulty, you’d typically see write-through caching being used in a microservice architecture on the server side. The benefits are pretty clear—the window in which a client might see stale data could be practically eliminated. This is balanced against the fact that server-side caches may well be less generally useful, limiting the circumstances in which a write-through cache would be effective in microservices.

Do sự khó khăn này, bạn thường sẽ thấy caching ghi-xuyên được dùng trong một kiến trúc microservice ở phía server. Các lợi ích khá rõ ràng—cửa sổ thời gian trong đó một client có thể thấy dữ liệu cũ có thể gần như được loại bỏ. Điều này được cân bằng với thực tế rằng các cache phía server rất có thể nhìn chung ít hữu ích hơn, giới hạn các tình huống trong đó một cache ghi-xuyên sẽ hiệu quả trong microservice.

Write-behind

Ghi-sau

With a write-behind cache, the cache itself is updated first , and then the origin is updated. Conceptually, you can think of the cache as a buffer. Writing into the cache is faster than updating the origin. So we write the result into the cache, allowing faster subsequent reads, and trust that the origin will be updated afterward.

Với một cache ghi-sau (write-behind), bản thân cache được cập nhật trước , và sau đó gốc được cập nhật. Về mặt khái niệm, bạn có thể nghĩ về cache như một bộ đệm. Việc ghi vào cache nhanh hơn việc cập nhật gốc. Vậy nên chúng ta ghi kết quả vào cache, cho phép các lượt đọc tiếp theo nhanh hơn, và tin rằng gốc sẽ được cập nhật sau đó.

The main concern around write-behind caches is going to be the potential for data loss. If the cache itself isn’t durable, we could lose the data before the data is written to the origin. Additionally, we’re now in an interesting spot—what is the origin in this context? We’d expect the origin to be the microservice where this data is sourced from—but if we update the cache first, is that really the origin? What is our source of truth? When making use of caching, it’s important to separate out what

data is cached (and potentially out of date) and what data can actually be considered to be up to date. Write-behind caches in the context of microservices makes this much less clear.

Mối lo chính xoay quanh các cache ghi-sau sẽ là khả năng mất dữ liệu. Nếu bản thân cache không bền vững (durable), chúng ta có thể mất dữ liệu trước khi dữ liệu được ghi vào gốc. Ngoài ra, giờ chúng ta đang ở một vị trí thú vị—gốc trong bối cảnh này là gì? Chúng ta sẽ kỳ vọng gốc là microservice nơi dữ liệu này bắt nguồn—nhưng nếu chúng ta cập nhật cache trước, thì đó có thực sự là gốc không? Nguồn chân lý của chúng ta là gì? Khi dùng caching, điều quan trọng là phải tách bạch dữ liệu nào được cache (và có khả năng lỗi thời) và dữ liệu nào thực sự có thể được coi là cập nhật mới nhất. Các cache ghi-sau trong bối cảnh microservice khiến điều này kém rõ ràng hơn nhiều.

While write-behind caches are often used for in-process optimization, I've seen them much more rarely used for microservice architectures, partly due to the fact that other, more straightforward forms of caching are good enough, but largely due to the complexity of handling the loss of unwritten cached data.

Mặc dù các cache ghi-sau thường được dùng cho việc tối ưu trong tiến trình (in-process), tôi đã thấy chúng được dùng hiếm hơn nhiều cho các kiến trúc microservice, một phần do thực tế rằng các hình thức caching khác đơn giản hơn đã đủ tốt, nhưng phần lớn là do độ phức tạp của việc xử lý sự mất mát dữ liệu cache chưa được ghi.

The Golden Rule of Caching — Quy tắc vàng của caching

Be careful about caching in too many places! The more caches between you and the source of fresh data, the more stale the data can be, and the harder it can be to determine the freshness of the data that a client eventually sees. It can also be more difficult to reason about where data needs to be invalidated. The trade-off around caching—balancing freshness of data against optimization of your system for load or latency—is a delicate one, and if you cannot easily reason about how fresh (or not) data might be, this becomes difficult.

Hãy cẩn thận với việc cache ở quá nhiều nơi! Càng nhiều cache giữa bạn và nguồn dữ liệu tươi mới, dữ liệu càng có thể cũ, và càng khó để xác định độ tươi mới của dữ liệu mà một client rốt cuộc thấy. Cũng có thể khó hơn để suy luận về nơi dữ liệu cần được vô hiệu hóa. Sự đánh đổi xoay quanh caching—cân bằng độ tươi mới của dữ liệu với việc tối ưu hệ thống của bạn cho tải hoặc độ trễ—là một sự đánh đổi tinh tế, và nếu bạn không thể dễ dàng suy luận về việc dữ liệu có thể tươi mới (hay không) đến mức nào, thì điều này trở nên khó khăn.

Consider a situation in which the Inventory microservice is caching stock levels. Requests to Inventory for stock levels may get served out of this server-side cache, speeding up the request accordingly. Let's also now assume we've set a TTL for this internal cache to be one minute, meaning our server-side cache could be up to one minute behind the actual stock level. Now, it turns out we are also caching on the client side inside Recommendation , where we're also using a TTL of one minute. When an entry in the client-side cache expires, we make a request from Recommendation to Inventory to get an up-to-date stock level, but unbeknownst to us, our request hits the server-side cache, which at this point could also be up to one minute old. So we could end up storing a record in our client-side cache that is already up to one minute old from the start. This means that the stock levels Recommendation is using could potentially be up to two minutes out of date, even though from the point of view of Recommendation , we think they could be only up to one minute out of date.

Hãy xét một tình huống trong đó microservice Inventory đang cache các mức tồn kho. Các request tới Inventory về mức tồn kho có thể được phục vụ ra từ cache phía server này, tăng tốc request một cách tương ứng. Bây giờ hãy cũng giả định rằng chúng ta đã đặt một TTL cho cache nội bộ này là một phút, nghĩa là cache phía server của chúng ta có thể chậm hơn mức tồn kho thực tế tới một phút. Giờ thì hóa ra chúng ta cũng đang cache ở phía client bên trong Recommendation , nơi chúng ta cũng đang dùng một TTL là một phút. Khi một mục trong client cache hết hạn, chúng ta thực hiện một request từ Recommendation tới Inventory để lấy một mức tồn kho cập nhật mới nhất, nhưng mà chúng ta không hề hay biết, request của chúng ta trúng cache phía server, vốn tại thời điểm này cũng có thể đã cũ tới một phút. Vậy nên chúng ta có thể rất cuộc lưu trữ một bản ghi trong client cache của mình mà ngay từ đầu đã cũ tới một phút. Điều này có nghĩa là các mức tồn kho mà Recommendation đang dùng có thể tiềm tàng cũ tới hai phút, mặc dù từ góc nhìn của Recommendation , chúng ta nghĩ rằng chúng chỉ có thể cũ tới một phút.

There are a number of ways to avoid problems like this. Using a timestamp-based expiration for a start would be better than TTLs, but it's also an example of what happens when caching is effectively nested. If you cache the result of an operation that in turn is based on cached inputs, how clear can you be about how up to date the end result is?

Có một số cách để tránh các vấn đề như thế này. Việc dùng một cơ chế hết hạn dựa trên dấu thời gian thì để mở đầu sẽ tốt hơn so với các TTL, nhưng nó cũng là một ví dụ về điều gì xảy ra khi caching bị lồng nhau một cách hữu hiệu. Nếu bạn cache kết quả của một thao tác mà thao tác đó lại dựa trên các đầu vào đã cache, thì bạn có thể rõ ràng đến đâu về việc kết quả cuối cùng cập nhật mới mẻ đến mức nào?

Coming back to the famous quote from Knuth earlier, premature optimization can cause issues. Caching adds complexity, and we want to add as little complexity as possible. The ideal number of places to cache is zero. Anything else should be an optimization you have to make—but be aware of the complexity it can bring.

Quay lại câu nói nổi tiếng của Knuth trước đó, tối ưu hóa sớm có thể gây ra các vấn đề. Caching làm tăng độ phức tạp, và chúng ta muốn thêm càng ít độ phức tạp càng tốt. Số nơi cache lý tưởng là con số không. Bất cứ thứ gì khác đều nên là một sự tối ưu mà bạn buộc phải thực hiện—nhưng hãy ý thức về độ phức tạp mà nó có thể mang lại.



Treat caching primarily as a performance optimization. Cache in as few places as possible to make it easier to reason about the freshness of data.

Hãy coi caching chủ yếu là một sự tối ưu hiệu năng. Hãy cache ở càng ít nơi càng tốt để dễ suy luận hơn về độ tươi mới của dữ liệu.

Freshness Versus Optimization — Độ tươi mới so với Tối ưu

Coming back to our example of TTL-based invalidation, I explained earlier that if we request a fresh copy of the data that has a five-minute TTL, and a second later the data at the origin changes, then our cache will be operating on out-of-date data for the remaining four minutes and 59 seconds. If this is unacceptable, one solution would be to reduce the TTL, thereby reducing the duration in which we could operate on stale data. So perhaps we reduce the TTL to one minute. This means that our window of staleness is reduced to one-fifth of what it was, but we've made five times as many calls to the origin, so we have to consider the associated latency and load impact.

Quay lại ví dụ của chúng ta về vô hiệu hóa dựa trên TTL, tôi đã giải thích trước đó rằng nếu chúng ta yêu cầu một bản sao tươi mới của dữ liệu có TTL năm phút, và một giây sau dữ liệu ở gốc thay đổi, thì cache của chúng ta sẽ vận hành trên dữ liệu lỗi thời trong bốn phút 59 giây còn lại. Nếu điều này không thể chấp nhận được, một giải pháp sẽ là giảm TTL, qua đó giảm thời lượng mà trong đó chúng ta có thể vận hành trên dữ liệu cũ. Vậy nên có lẽ chúng ta giảm TTL xuống một phút. Điều này có nghĩa là cửa sổ dữ liệu cũ của chúng ta được giảm xuống còn một phần năm so với trước, nhưng chúng ta đã thực hiện số lời gọi tới gốc nhiều gấp năm lần, vậy nên chúng ta phải cân nhắc độ trễ và tác động tải đi kèm.

Balancing these forces is going to come down to understanding the requirements of the end user and of the wider system. Users will obviously always want to operate on the freshest data, but not if that means the system falls down under load. Likewise, sometimes the safest thing to do is to turn off features if a cache fails, in order to avoid an overload on the origin causing more serious issues. When it comes to fine-tuning what, where, and how to cache, you'll often find yourself having to balance along a number of axes. This is just another reason to try to keep things as simple as possible—the fewer the caches, the easier it can be to reason about the system.

Việc cân bằng các lực này sẽ rất cuộc phụ thuộc vào việc hiểu các yêu cầu của người dùng cuối và của hệ thống rộng hơn. Người dùng dĩ nhiên sẽ luôn muốn vận hành trên dữ liệu tươi mới nhất, nhưng không phải khi điều đó nghĩa là hệ thống gục ngã dưới tải. Tương tự, đôi khi điều an toàn nhất nên làm là tắt các tính năng nếu một cache hỏng, để tránh việc quá tải lên gốc gây ra các vấn đề nghiêm trọng hơn. Khi nói đến việc tinh chỉnh cái gì, ở đâu, và cách cache, bạn sẽ thường thấy mình phải cân bằng dọc theo một số trục. Đây chỉ là thêm một lý do nữa để cố gắng giữ mọi thứ càng đơn giản càng tốt—càng ít cache, càng dễ suy luận về hệ thống.

Cache Poisoning: A Cautionary Tale — Cache Poisoning: Một câu chuyện cảnh tỉnh

With caching, we often think that if we get it wrong, the worst thing that can happen is we serve stale data for a bit. But what happens if you end up serving stale data forever? Back in Chapter 12 I introduced AdvertCorp, where I was working to help migrate a number of existing legacy applications over to a new platform making use of the strangler fig pattern. This involved intercepting calls to multiple legacy applications and, where these applications had been moved to the new platform, diverting the calls. Our new application operated effectively as a proxy. Traffic for the older legacy applications that we hadn't yet migrated was routed through our new application to the downstream legacy applications. For the calls to legacy applications, we did a few housekeeping things; for example, we made sure that the results from the legacy application had proper HTTP cache headers applied.

Với caching, chúng ta thường nghĩ rằng nếu làm sai, điều tệ nhất có thể xảy ra là chúng ta phục vụ dữ liệu cũ trong một lúc. Nhưng điều gì xảy ra nếu bạn rút cuộc phục vụ dữ liệu cũ mãi mãi? Hồi Chương 12 tôi đã giới thiệu AdvertCorp, nơi tôi đang làm việc để giúp di chuyển một số ứng dụng kế thừa (legacy) hiện có sang một nền tảng mới tận dụng mẫu strangler fig. Việc này liên quan đến việc chặn các lời gọi tới nhiều ứng dụng kế thừa và, ở những nơi mà các ứng dụng này đã được chuyển sang nền tảng mới, chuyển hướng các lời gọi. Ứng dụng mới của chúng tôi vận hành một cách hữu hiệu như một proxy. Lưu lượng cho các ứng dụng kế thừa cũ hơn mà chúng tôi chưa di chuyển được định tuyến qua ứng dụng mới của chúng tôi tới các ứng dụng kế thừa ở phía dưới. Đối với các lời gọi tới các ứng dụng kế thừa, chúng tôi làm một vài việc dọn dẹp; ví dụ, chúng tôi đảm bảo rằng các kết quả từ ứng dụng kế thừa đã được áp dụng các header cache HTTP đúng đắn.

One day, shortly after a normal routine **release**, something odd started happening. A bug had been introduced whereby a small subset of pages were falling through a logic condition in our cache header insertion code, resulting in us not changing the header at all. Unfortunately, this downstream application had also been changed sometime previously to include an Expires: Never HTTP header. This hadn't had any effect earlier, as we were overriding this header. Now we weren't.

Một ngày nọ, ngay sau một lần phát hành thường lệ bình thường, một điều gì đó kỳ lạ bắt đầu xảy ra. Một lỗi đã được đưa vào, theo đó một tập con nhỏ các trang đang rơi qua một điều kiện logic trong mã chèn header cache của chúng tôi, dẫn đến việc chúng tôi không thay đổi header gì cả. Đáng tiếc là ứng dụng phía dưới này cũng đã được thay đổi vào một thời điểm nào đó trước đây để bao gồm một header HTTP Expires: Never. Điều này đã không có tác động gì trước đó, vì chúng tôi đang ghi đè header này. Giờ thì chúng tôi không còn ghi đè nữa.

Our application made heavy use of Squid to cache HTTP traffic, and we noticed the problem quite quickly, as we were seeing more requests bypassing Squid itself to hit our application servers. We fixed the cache header code and pushed out a release, and we also manually cleared the relevant region of the Squid cache. However, that wasn't enough.

Ứng dụng của chúng tôi dùng Squid rất nhiều để cache lưu lượng HTTP, và chúng tôi nhận ra vấn đề khá nhanh, vì chúng tôi thấy nhiều request hơn bỏ qua chính Squid để trúng các máy chủ ứng dụng của chúng tôi. Chúng tôi đã sửa mã header cache và đẩy ra một bản phát hành, và chúng tôi cũng đã xóa thủ công vùng liên quan của cache Squid. Tuy nhiên, như thế là chưa đủ.

As we just discussed, you can cache in multiple places—but sometimes having lots of caches makes your life harder, not easier. When it comes to serving up content to users of a public-facing web application, you could have multiple caches between you and your customer. Not only might you be fronting your website with something like a content delivery network, but some ISPs make use of caching. Can you control those caches? And even if you could, there is one cache that you have little control over: the cache in a user's browser.

Như chúng ta vừa bàn, bạn có thể cache ở nhiều nơi—nhưng đôi khi việc có nhiều cache khiến cuộc sống của bạn khó hơn, chứ không dễ hơn. Khi nói đến việc phục vụ nội dung cho người dùng của một ứng dụng web hướng ra công chúng, bạn có thể có nhiều cache giữa bạn và khách hàng của mình. Bạn không chỉ có thể đang đặt website của mình phía sau một thứ như một content delivery network, mà một số ISP còn dùng caching. Bạn có thể kiểm soát các cache đó không? Và ngay cả khi bạn có thể, thì có một cache mà bạn có rất ít quyền kiểm soát: cache trong trình duyệt của người dùng.

Those pages with Expires: Never stuck in the caches of many of our users and would never be invalidated until the cache became full or the user cleaned them out manually. Clearly we couldn't

make either thing happen; our only option was to change the URLs of these pages so they were refetched.

Những trang với Expires: Never đó đã mắc kẹt trong cache của nhiều người dùng của chúng tôi và sẽ không bao giờ bị vô hiệu hóa cho đến khi cache đầy hoặc người dùng tự xóa chúng thủ công. Rõ ràng chúng tôi không thể khiến điều nào trong hai điều đó xảy ra; lựa chọn duy nhất của chúng tôi là thay đổi các URL của những trang này để chúng được lấy lại.

Caching can be very powerful indeed, but you need to understand the full path of data that is cached from source to destination to really appreciate its complexities and what can go wrong.

Caching quả thực có thể rất mạnh mẽ, nhưng bạn cần hiểu toàn bộ đường đi của dữ liệu được cache từ nguồn tới đích để thực sự thấu hiểu các độ phức tạp của nó và những gì có thể trở nên sai lệch.

Autoscaling — Autoscaling

If you are lucky enough to have fully automatable provisioning of virtual hosts and can fully automate the **deployment** of your microservice instances, then you have the building blocks to allow you to automatically scale your microservices.

Nếu bạn đủ may mắn để có việc cấp phát (provisioning) các host ảo hoàn toàn tự động hóa được và có thể tự động hóa hoàn toàn việc triển khai các instance microservice của mình, thì bạn có các khối xây dựng để cho phép bạn tự động mở rộng các microservice của mình.

For example, you could have the scaling triggered by well-known trends. You might know that your system's peak load is between 9 a.m. and 5 p.m., so you bring up additional instances at 8:45 a.m., and turn them off at 5:15 p.m. If you're using something like AWS (which has very good support for **autoscaling** built in), turning off instances you don't need any longer will help save money. You'll need data to understand how your load changes over time, from day to day, and from week to week. Some businesses have obvious seasonal cycles too, so you may need data going back a fair way to make proper judgment calls.

Ví dụ, bạn có thể có việc mở rộng được kích hoạt bởi các xu hướng đã biết rõ. Bạn có thể biết rằng tải đỉnh của hệ thống của mình là từ 9 giờ sáng đến 5 giờ chiều, vậy nên bạn đưa thêm các instance lên vào lúc 8:45 sáng, và tắt chúng đi vào lúc 5:15 chiều. Nếu bạn đang dùng một thứ như AWS (vốn có hỗ trợ tích hợp rất tốt cho autoscaling), việc tắt các instance mà bạn không còn cần nữa sẽ giúp tiết kiệm tiền. Bạn sẽ cần dữ liệu để hiểu tải của mình thay đổi thế nào theo thời gian, từ ngày này sang ngày khác, và từ tuần này sang tuần khác. Một số doanh nghiệp cũng có các chu kỳ theo mùa rõ ràng, vậy nên bạn có thể cần dữ liệu lùi về khá xa để đưa ra các quyết định đúng đắn.

On the other hand, you could be reactive, bringing up additional instances when you see an increase in load or an instance failure, and removing instances when you no longer need them. Knowing how fast you can scale up once you spot an upward trend is key. If you know you'll get only a couple of minutes' notice about an increase in load, but scaling up will take you at least 10 minutes, then you know you'll need to keep extra capacity around to bridge this gap. Having a good suite of load tests is almost essential here. You can use them to test your autoscaling rules. If you don't have tests that can reproduce different loads that will trigger scaling, then you're only going to find out in production if you got the rules wrong. And the consequences of failure aren't great!

Mặt khác, bạn có thể phản ứng (reactive), đưa thêm các instance lên khi bạn thấy một sự gia tăng tải hoặc một instance hỏng, và loại bỏ các instance khi bạn không còn cần chúng. Việc biết bạn có thể mở rộng nhanh đến đâu một khi bạn phát hiện một xu hướng đi lên là then chốt. Nếu bạn biết rằng bạn sẽ chỉ nhận được vài phút báo trước về một sự gia tăng tải, nhưng việc mở rộng sẽ tốn của bạn ít nhất 10 phút, thì bạn biết rằng bạn sẽ cần giữ thêm dung lượng dự phòng để bắc cầu qua khoảng trống này. Việc có một bộ kiểm thử tải tốt gần như là thiết yếu ở đây. Bạn có thể dùng chúng để kiểm thử các quy tắc autoscaling của mình. Nếu bạn không có các bài kiểm thử có thể tái tạo các mức tải khác nhau sẽ kích hoạt việc mở rộng, thì bạn sẽ chỉ phát hiện ra trong production nếu bạn đã đặt sai các quy tắc. Và hậu quả của thất bại thì không hay ho gì!

A news site is a great example of a type of business in which you may want a mix of predictive and reactive scaling. On the last news site I worked on, we saw very clear daily trends, with views climbing from the morning to lunchtime and then starting to decline. This pattern was repeated day in and day out, with traffic generally lower on the weekend. That gave us a fairly clear trend that could drive proactive scaling of resources, whether up or down. On the other hand, a big news story would cause an unexpected spike, requiring more capacity and often at short notice.

Một trang tin tức là một ví dụ tuyệt vời về một loại doanh nghiệp mà trong đó bạn có thể muốn một sự pha trộn giữa mở rộng dự đoán (predictive) và mở rộng phản ứng (reactive). Trên trang tin tức gần nhất tôi từng làm việc, chúng tôi thấy các xu hướng theo ngày rất rõ ràng, với lượt xem leo lên từ buổi sáng đến giờ ăn trưa rồi sau đó bắt đầu giảm. Mẫu hình này lặp lại ngày này qua ngày khác, với lưu lượng nhìn chung thấp hơn vào cuối tuần. Điều đó cho chúng tôi một xu hướng khá rõ ràng có thể dẫn dắt việc mở rộng chủ động các tài nguyên, dù lên hay xuống. Mặt khác, một tin tức lớn sẽ gây ra một đợt tăng vọt bất ngờ, đòi hỏi nhiều dung lượng hơn và thường là vào thời điểm gấp gáp.

I actually see autoscaling used much more for handling failure of instances than for reacting to load conditions. AWS lets you specify rules like “There should be at least five instances in this group,” so that if one instance goes down a new one is automatically launched. I’ve seen this approach lead to a fun game of whack-a-mole when someone forgets to turn off the rule and then tries to take down the instances for maintenance, only to see them keep spinning up!

Thực tế tôi thấy autoscaling được dùng nhiều hơn cho việc xử lý sự cố của các instance hơn là cho việc phản ứng với các điều kiện tải. AWS cho phép bạn chỉ định các quy tắc như “Nên có ít nhất năm instance trong nhóm này,” để nếu một instance sập thì một instance mới được tự động khởi chạy. Tôi đã thấy cách tiếp cận này dẫn đến một trò chơi đập chuột chũi vui nhộn khi ai đó quên tắt quy tắc rồi sau đó cố gắng tắt các instance để bảo trì, để rồi thấy chúng cứ liên tục khởi động lại!

Both reactive and predictive scaling are very useful and can help you be much more cost effective if you’re using a platform that allows you to pay only for the computing resources you use. But they also require careful observation of the data available to you. I’d suggest using autoscaling for failure conditions first while you collect the data. Once you want to start autoscaling for load, make sure you are very cautious about scaling down too quickly. In most situations, having more computing power at hand than you need is much better than not having enough!

Cả mở rộng phản ứng lẫn mở rộng dự đoán đều rất hữu ích và có thể giúp bạn tiết kiệm chi phí hơn nhiều nếu bạn đang dùng một nền tảng cho phép bạn chỉ trả tiền cho các tài nguyên tính toán mà bạn dùng. Nhưng chúng cũng đòi hỏi sự quan sát cẩn thận đối với dữ liệu có sẵn cho bạn. Tôi sẽ đề xuất dùng autoscaling cho các điều kiện sự cố trước trong khi bạn thu thập dữ liệu. Một khi bạn muốn bắt đầu autoscaling cho tải, hãy chắc chắn rằng bạn rất thận trọng về việc thu nhỏ quá nhanh. Trong hầu hết các tình huống, việc có

nhiều sức mạnh tính toán trong tay hơn mức bạn cần thì tốt hơn nhiều so với việc không có đủ!

Starting Again — Bắt đầu lại từ đầu

The architecture that gets you started may not be the architecture that keeps you going when your system has to handle very different volumes of load. As we’ve already seen, there are some forms of scaling that can have extremely limited impact on the architecture of your system—vertical scaling and horizontal duplication, for example. At certain points, though, you need to do something pretty radical to change the architecture of your system to support the next level of growth.

Kiến trúc đưa bạn khởi đầu có thể không phải là kiến trúc giữ cho bạn tiếp tục đi khi hệ thống của bạn phải xử lý những mức tải rất khác biệt. Như chúng ta đã thấy, có một số hình thức mở rộng có thể có tác động cực kỳ hạn chế lên kiến trúc của hệ thống của bạn—ví dụ như mở rộng dọc và nhân bản theo chiều ngang. Tuy nhiên, ở một số thời điểm nhất định, bạn cần làm một điều gì đó khá triệt để để thay đổi kiến trúc của hệ thống của mình nhằm hỗ trợ cấp độ tăng trưởng tiếp theo.

Recall the story of Gilt, which we touched on in “Isolated Execution” on page 228. A simple monolithic Rails application did well for Gilt for two years. Its business became increasingly successful, which meant more customers and more load. At a certain tipping point, the company had to redesign the application to handle the load it was seeing.

Hãy nhớ lại câu chuyện về Gilt, mà chúng ta đã chạm đến trong phần “Thực thi cô lập (Isolated Execution)” ở trang 228. Một ứng dụng Rails monolith đơn giản đã hoạt động tốt cho Gilt trong hai năm. Hoạt động kinh doanh của họ ngày càng thành công, điều đó nghĩa là nhiều khách hàng hơn và nhiều tải hơn. Tại một điểm tới hạn nào đó, công ty đã phải thiết kế lại ứng dụng để xử lý tải mà nó đang thấy.

A redesign may mean splitting apart an existing monolith, as it did for Gilt. Or it might mean picking new data stores that can handle the load better. It could also mean adopting new techniques, such as moving from **synchronous request-response** to event-based systems, adopting new deployment platforms, changing whole technology stacks, or everything in between.

Một cuộc thiết kế lại có thể có nghĩa là tách rời một monolith hiện có, như nó đã làm với Gilt. Hoặc nó có thể có nghĩa là chọn các kho dữ liệu mới có thể xử lý tải tốt hơn. Nó cũng có thể có nghĩa là áp dụng các kỹ thuật mới, chẳng hạn như chuyển từ request-response đồng bộ sang các hệ thống dựa trên sự kiện, áp dụng các nền tảng triển khai mới, thay đổi toàn bộ tech stack, hoặc mọi thứ ở giữa.

There is a danger that people will see the need to rearchitect when certain scaling thresholds are reached as a reason to build for massive scale from the beginning. This can be disastrous. At the start of a new project, we often don’t know exactly what we want to build, nor do we know if it will be successful. We need to be able to rapidly experiment and understand what capabilities we need to build. If we tried building for massive scale up front, we’d end up front-loading a huge amount of work to prepare for load that may never come, while diverting effort away from more important activities, like understanding if anyone will actually want to use our product. Eric Ries tells the story of spending six months building a product that no one ever downloaded. He reflected that he could have put up a link on a web page that 404’d when people clicked on it to see if there was any demand, spent six months on the beach instead, and learned just as much!

Có một nguy hiểm là người ta sẽ xem nhu cầu tái kiến trúc khi đạt tới một số ngưỡng mở rộng nhất định như một lý do để xây dựng cho quy mô khổng lồ ngay từ đầu. Điều này có

thể là thảm họa. Khi bắt đầu một dự án mới, chúng ta thường không biết chính xác mình muốn xây dựng gì, cũng không biết liệu nó có thành công hay không. Chúng ta cần có khả năng thử nghiệm nhanh chóng và hiểu được những năng lực mà chúng ta cần xây dựng. Nếu chúng ta cố xây dựng cho quy mô khổng lồ ngay từ đầu, chúng ta sẽ rớt cuộc dồn trước một lượng công việc khổng lồ để chuẩn bị cho một mức tải có thể không bao giờ đến, trong khi chuyển hướng nỗ lực ra khỏi các hoạt động quan trọng hơn, như hiểu được liệu có ai thực sự muốn dùng sản phẩm của chúng ta hay không. Eric Ries kể câu chuyện về việc dành sáu tháng xây dựng một sản phẩm mà chẳng ai từng tải về. Ông suy ngẫm rằng ông đã có thể đặt một liên kết lên một trang web mà sẽ báo lỗi 404 khi người ta nhấp vào để xem liệu có nhu cầu nào hay không, thay vào đó dành sáu tháng trên bãi biển, và học được cũng nhiều y như vậy!

The need to change our systems to deal with scale isn't a sign of failure. It is a sign of success.

Nhu cầu thay đổi hệ thống của chúng ta để đối phó với quy mô không phải là một dấu hiệu của thất bại. Nó là một dấu hiệu của thành công.

Summary — Tóm tắt

As we can see, whatever type of scaling you're looking for, microservices give you a lot of different options in terms of how you approach the problem.

Như chúng ta có thể thấy, dù bạn đang tìm kiếm loại mở rộng nào, microservice cũng cho bạn rất nhiều lựa chọn khác nhau về cách bạn tiếp cận vấn đề.

The scaling axes can be a useful model to use when considering what types of scaling are available to you:

Các trục mở rộng có thể là một mô hình hữu ích để dùng khi cân nhắc những loại mở rộng nào có sẵn cho bạn:

Vertical scaling In a nutshell, this means getting a bigger machine.

Mở rộng dọc (vertical scaling) Nói gọn lại, điều này nghĩa là sắm một cỗ máy lớn hơn.

Horizontal duplication Having multiple things capable of doing the same work.

Nhân bản theo chiều ngang (horizontal duplication) Có nhiều thực thể có khả năng làm cùng một công việc.

Data partitioning Dividing work based on some attribute of the data, e.g., customer group.

Phân vùng dữ liệu (data partitioning) Chia nhỏ công việc dựa trên một thuộc tính nào đó của dữ liệu, ví dụ như nhóm khách hàng.

Functional decomposition Separation of work based on the type, e.g., microservice decomposition.

Phân rã theo chức năng (functional decomposition) Tách công việc dựa trên loại, ví dụ như phân rã thành các microservice.

Key to a lot of this is understanding what it is you want—techniques that are effective at scaling to improve latency may not be as effective in helping scale for volume.

Then chốt cho rất nhiều điều này là hiểu được bạn muốn gì—các kỹ thuật hiệu quả trong việc mở rộng để cải thiện độ trễ có thể không hiệu quả bằng trong việc giúp mở rộng cho khối lượng.

CHAPTER 14 User Interfaces —

Chương 14: Giao diện người dùng

So far, we haven't really touched on the world of the **user interface**. A few of you out there might just be providing a cold, hard, clinical **API** to your customers, but many of us find ourselves wanting to create beautiful, functional user interfaces that will delight our customers. The user **interface**, after all, is where we'll be pulling all these **microservices** together into something that makes sense to our customers.

Cho đến giờ, chúng ta vẫn chưa thực sự chạm tới thế giới của giao diện người dùng (UI). Một số bạn ngoài kia có thể chỉ đang cung cấp cho khách hàng một API lạnh lùng, khô khan và thuần kỹ thuật, nhưng nhiều người trong chúng ta lại muốn tạo ra những giao diện người dùng đẹp đẽ, hữu dụng để làm hài lòng khách hàng. Suy cho cùng, giao diện người dùng chính là nơi chúng ta gom tất cả những microservice này lại thành một thứ gì đó có ý nghĩa đối với khách hàng.

When I first started computing, we were mostly talking about big, fat clients that ran on our desktops. I spent many hours with Motif and then Swing trying to make my software as nice to use as possible. Often these systems were just for the creation and manipulation of local files, but many of them had a server-side component. My first job at Thoughtworks involved creating a Swing-based electronic point-of-sale system that was just one of a large number of moving parts, most of which were on the server.

Khi tôi mới bắt đầu làm về máy tính, chúng tôi chủ yếu nói về những ứng dụng client “béo” (fat client) lớn chạy trên máy tính để bàn. Tôi đã dành rất nhiều giờ với Motif rồi sau đó là Swing để cố gắng làm cho phần mềm của mình dễ dùng nhất có thể. Thường thì những hệ thống này chỉ để tạo và thao tác các tệp cục bộ, nhưng nhiều trong số đó cũng có một thành phần phía máy chủ. Công việc đầu tiên của tôi tại Thoughtworks liên quan đến việc xây dựng một hệ thống điểm bán hàng điện tử (electronic point-of-sale) dựa trên Swing, vốn chỉ là một trong số rất nhiều bộ phận cấu thành, mà phần lớn nằm trên máy chủ.

Then came the web. We instead started thinking of our UIs as being “thin,” with more logic on the server side. In the beginning, our server-side programs rendered the entire page and sent it to the client browser, which did very little. Any interactions were handled on the server side via GETs and POSTs triggered by the user clicking on links or filling in forms. Over time, JavaScript became a more popular option to add dynamic behavior to the browser-based UI, and some applications could arguably be as “fat” as the old desktop clients. We subsequently had the rise of the mobile application, and today we have a varied landscape for delivering graphical user interfaces to our users—different platforms, and different technologies for those platforms. This range of technologies gives us a host of options for how we can make effective user interfaces backed by microservices. We'll be exploring all of this and more in this chapter.

Rồi web xuất hiện. Thay vào đó, chúng ta bắt đầu nghĩ về UI của mình như những thứ “mỏng” (thin), với nhiều logic hơn nằm ở phía máy chủ. Ban đầu, các chương trình phía máy chủ kết xuất (render) toàn bộ trang rồi gửi nó tới trình duyệt của client, vốn làm rất ít việc. Mọi tương tác đều được xử lý ở phía máy chủ thông qua các lệnh GET và POST kích hoạt bởi việc người dùng nhấp vào liên kết hoặc điền vào biểu mẫu. Theo thời gian, JavaScript trở thành một lựa chọn phổ biến hơn để thêm hành vi động vào UI chạy trên trình duyệt, và một số ứng dụng thậm chí có thể “béo” chẳng kém gì các ứng dụng desktop client thời xưa. Sau đó là sự trỗi dậy của ứng dụng di động, và ngày nay chúng ta có một bức tranh đa dạng để cung cấp giao diện người dùng đồ họa cho người dùng—nhiều nền tảng khác nhau, và nhiều công nghệ khác nhau cho từng nền tảng đó. Phạm vi công nghệ này cho chúng ta vô số lựa chọn về cách tạo ra những giao diện người dùng hiệu quả được hậu thuẫn bởi microservice. Chúng ta sẽ khám phá tất cả những điều này và hơn thế nữa trong chương này.

Toward Digital — Hướng tới kỹ thuật số (Digital)

Over the last couple of years, organizations have started to move away from thinking that web or mobile should be treated differently; they are instead thinking about digital more holistically. What is the best way for our customers to use the services we offer? And what does that do to our **system architecture**? The understanding that we cannot predict exactly how a customer might end up interacting with our products has driven adoption of more granular APIs, like those delivered by microservices. By combining the capabilities our microservices expose in different ways, we can curate different experiences for our customers on their desktop application, mobile device, and wearable device and even in physical form if they visit our brick-and-mortar store.

Trong vài năm gần đây, các tổ chức đã bắt đầu rời bỏ tư duy rằng web hay di động cần được đối xử khác nhau; thay vào đó họ nghĩ về kỹ thuật số (digital) một cách toàn diện hơn. Đây là cách tốt nhất để khách hàng của chúng ta sử dụng các dịch vụ mà chúng ta cung cấp? Và điều đó tác động thế nào đến kiến trúc hệ thống của chúng ta? Nhận thức rằng chúng ta không thể dự đoán chính xác khách hàng cuối cùng sẽ tương tác với sản phẩm của mình ra sao đã thúc đẩy việc áp dụng các API chi tiết hơn, như những API do microservice cung cấp. Bằng cách kết hợp các năng lực mà microservice của chúng ta phối bày theo những cách khác nhau, chúng ta có thể chăm chút những trải nghiệm khác nhau cho khách hàng trên ứng dụng desktop, thiết bị di động, thiết bị đeo của họ và thậm chí ở dạng vật lý nếu họ ghé thăm cửa hàng truyền thống của chúng ta.

So think of user interfaces as the places where we weave together the various strands of the capabilities we want to offer our users. With that in mind, how do we pull all these strands together? We need to look at this problem from two sides: the who and the how. Firstly, we'll consider the organizational aspects—who owns what responsibilities when it comes to delivering user interfaces? Secondly, we'll look at a set of patterns that can be used to implement these interfaces.

Vậy hãy nghĩ về giao diện người dùng như những nơi mà chúng ta đan kết lại các sợi chỉ năng lực khác nhau mà chúng ta muốn cung cấp cho người dùng. Với ý nghĩ đó, làm thế nào để gom tất cả những sợi chỉ này lại với nhau? Chúng ta cần nhìn vấn đề này từ hai phía: ai và như thế nào. Trước tiên, chúng ta sẽ xét các khía cạnh tổ chức—ai sở hữu trách nhiệm gì khi nói đến việc cung cấp giao diện người dùng? Thứ hai, chúng ta sẽ xem xét một tập các pattern có thể dùng để hiện thực các giao diện này.

Ownership Models — Các mô hình quyền sở hữu (Ownership Models)

As we discussed way back in Chapter 1, the traditional layered **architecture** can cause issues when it comes to delivering software effectively. In Figure 14-1, we see an example in which responsibility for the user interface layer is owned by a single **frontend** team, with the **backend** services work done by another. In this example, adding a simple control involves work being done by three different teams. These sorts of tiered organizational structures can significantly impact our speed of delivery due to the need to constantly coordinate changes and hand off work between teams.

Như chúng ta đã thảo luận từ tận Chương 1, kiến trúc phân tầng (layered architecture) truyền thống có thể gây ra vấn đề khi nói đến việc chuyển giao phần mềm một cách hiệu quả. Trong Hình 14-1, chúng ta thấy một ví dụ trong đó trách nhiệm về tầng giao diện người dùng được sở hữu bởi một đội frontend duy nhất, còn phần việc dịch vụ phía backend lại do một đội khác làm. Trong ví dụ này, việc thêm một control đơn giản đòi hỏi công việc phải được thực hiện bởi ba đội khác nhau. Những kiểu cấu trúc tổ chức theo tầng này có thể tác động đáng kể đến tốc độ chuyển giao của chúng ta do nhu cầu phải liên tục phối hợp các thay đổi và bàn giao công việc giữa các đội.

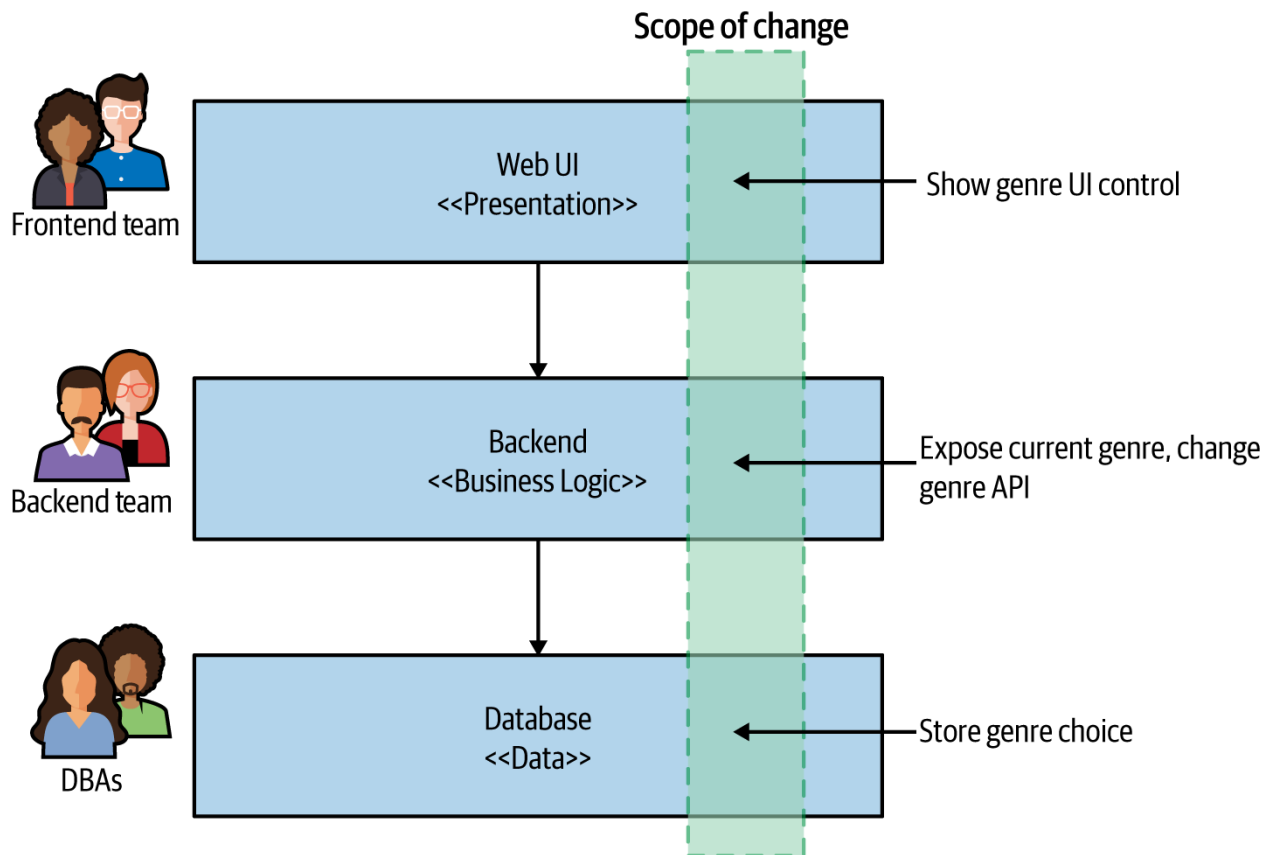


Figure 14-1. Making a change across all three tiers is more involved

Hình 14-1. Thực hiện một thay đổi xuyên suốt cả ba tầng thì phức tạp hơn

The model that I greatly prefer and feel is better aligned to achieving the goal of **independent deployability** is to have the UI broken apart and managed by a team that also manages the server-side components, as we see in Figure 14-2. Here, a single team ends up being responsible for all the

changes we need to make to add our new control.

Mô hình mà tôi ưa thích hơn nhiều và cảm thấy phù hợp hơn với mục tiêu đạt được khả năng triển khai độc lập là tách nhỏ UI ra và để cho một đội cũng quản lý luôn các thành phần phía máy chủ quản lý nó, như chúng ta thấy trong Hình 14-2. Ở đây, một đội duy nhất rất cuộc chịu trách nhiệm cho tất cả những thay đổi mà chúng ta cần thực hiện để thêm control mới.

Teams with full **ownership** of the end-to-end functionality are able to make changes more quickly. Having full ownership encourages each team to have a direct contact point with an end user of the software. With backend teams, it's easy to lose track of who the end user is.

Các đội có toàn quyền sở hữu chức năng đầu-cuối (end-to-end) có thể thực hiện thay đổi nhanh hơn. Việc có toàn quyền sở hữu khuyến khích mỗi đội có một điểm tiếp xúc trực tiếp với người dùng cuối của phần mềm. Với các đội backend, rất dễ mất dấu việc ai mới là người dùng cuối.

Despite the drawbacks, I (unfortunately) still see the dedicated frontend team as being the more common organizational pattern among companies who make use of microservices. Why is this?

Bất chấp những nhược điểm đó, tôi (đáng tiếc) vẫn thấy đội frontend chuyên trách là pattern tổ chức phổ biến hơn trong các công ty sử dụng microservice. Tại sao lại như vậy?

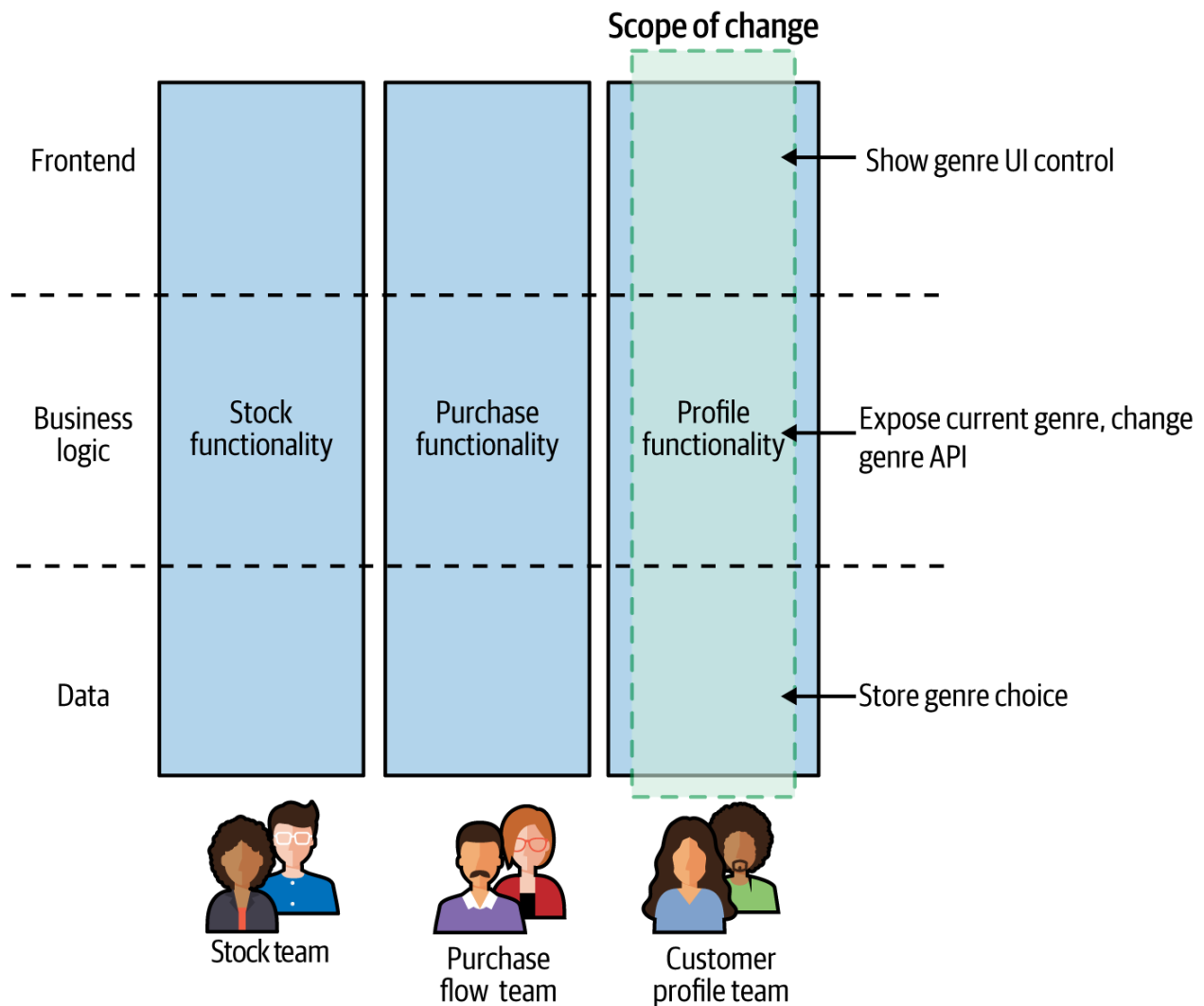


Figure 14-2. The UI is broken apart and is owned by a team that also manages the server-side functionality that supports the UI

Hình 14-2. UI được tách nhỏ và được sở hữu bởi một đội cũng quản lý luôn chức năng phía máy chủ hỗ trợ cho UI

Drivers for Dedicated Frontend Teams — Các động lực dẫn tới đội frontend chuyên trách

The desire for dedicated frontend teams seems to come down to three key factors: scarcity of specialists, a drive for consistency, and technical challenges.

Mong muốn có đội frontend chuyên trách dường như quy về ba yếu tố chính: sự khan hiếm chuyên gia, nhu cầu nhất quán, và các thách thức kỹ thuật.

Firstly, delivering a user interface requires a degree of specialized skills. There are the interaction and graphical design aspects, and then there is the technical know-how required to deliver a great web or native application experience. Specialists with these skills can be hard to come by, and as these folks are such a rare commodity, the temptation is to stick them all together so you can make

sure they are focusing only on their specialty.

Thứ nhất, việc cung cấp một giao diện người dùng đòi hỏi một mức độ kỹ năng chuyên môn nhất định. Có các khía cạnh thiết kế tương tác và thiết kế đồ họa, rồi có cả bí quyết kỹ thuật cần thiết để mang lại một trải nghiệm ứng dụng web hay ứng dụng native tuyệt vời. Các chuyên gia với những kỹ năng này có thể khó kiếm, và vì những người này là một thứ hàng hiếm như vậy, người ta dễ bị cám dỗ gom hết họ lại với nhau để bạn có thể chắc chắn rằng họ chỉ tập trung vào chuyên môn của mình.

The second driver for a separate frontend team is consistency. If you have one team responsible for delivering your customer-facing user interface, you can ensure your UI has a consistent look and feel. You use a consistent set of controls to solve similar problems so that the user interface looks and feels like a single, cohesive entity.

Động lực thứ hai cho một đội frontend riêng là tính nhất quán. Nếu bạn có một đội chịu trách nhiệm cung cấp giao diện người dùng hướng tới khách hàng, bạn có thể đảm bảo UI của mình có một diện mạo và cảm nhận (look and feel) nhất quán. Bạn dùng một tập control nhất quán để giải quyết những vấn đề tương tự nhau, để giao diện người dùng trông và cảm thấy như một thực thể duy nhất, gắn kết.

Finally, some user interface technology can be challenging to work with in a non-monolithic fashion. Here I'm thinking specifically of single-page applications (SPAs), which historically at least haven't been easy to break apart. Traditionally, a web user interface would consist of multiple web pages, and you would navigate from one page to another. With SPAs, the entire application is instead served up inside a single web page. Frameworks like Angular, React, and Vue theoretically allow for the creation of more sophisticated user interfaces than “old-fashioned” websites. We'll be looking at a set of patterns that can give you different options for how to decompose a user interface later in this chapter, and in terms of SPAs I will show how the **micro frontend** concept can allow you to use SPA frameworks while still avoiding the need for a monolithic user interface.

Cuối cùng, một số công nghệ giao diện người dùng có thể khó làm việc theo kiểu phi nguyên khối. Ở đây tôi đang nghĩ cụ thể đến các ứng dụng đơn trang (single-page application, SPA), vốn ít nhất trong quá khứ đã không dễ tách nhỏ. Theo truyền thống, một giao diện người dùng web sẽ gồm nhiều trang web, và bạn điều hướng từ trang này sang trang khác. Với SPA, toàn bộ ứng dụng thay vào đó được phục vụ bên trong một trang web duy nhất. Các framework như Angular, React và Vue về lý thuyết cho phép tạo ra những giao diện người dùng tinh vi hơn so với các website “kiểu cũ”. Chúng ta sẽ xem xét một tập các pattern có thể cho bạn nhiều lựa chọn khác nhau về cách phân rã một giao diện người dùng ở phần sau của chương này, và về SPA, tôi sẽ trình bày cách mà khái niệm micro frontend có thể cho phép bạn sử dụng các framework SPA mà vẫn tránh được nhu cầu phải có một giao diện người dùng nguyên khối.

Toward Stream-Aligned Teams — Hướng tới các đội stream-aligned

I think that having a dedicated frontend team is in general a mistake if you are trying to optimize for good throughput—it creates new handoff points in your organization, slowing things down. Ideally, our teams are aligned around end-to-end slices of functionality, allowing each team to deliver new features to its customers while reducing the amount of coordination needed. My preferred model is a team owning the end-to-end delivery of functionality in a specific part of the domain. This matches what Matthew Skelton and Manuel Pais describe as stream-aligned teams in their book

Team Topologies . 1 As they describe it:

Tôi cho rằng việc có một đội frontend chuyên trách nhìn chung là một sai lầm nếu bạn đang cố tối ưu cho thông lượng (throughput) tốt—nó tạo ra những điểm bàn giao mới trong tổ chức của bạn, làm mọi thứ chậm lại. Lý tưởng nhất, các đội của chúng ta được tổ chức xoay quanh những lát cắt chức năng đầu-cuối, cho phép mỗi đội chuyển giao tính năng mới cho khách hàng của mình trong khi giảm bớt lượng phối hợp cần thiết. Mô hình tôi ưa thích là một đội sở hữu việc chuyển giao đầu-cuối chức năng trong một phần cụ thể của miền nghiệp vụ. Điều này khớp với cái mà Matthew Skelton và Manuel Pais mô tả là stream-aligned team trong cuốn sách Team Topologies của họ. 1 Như họ mô tả:

A **stream-aligned team** is a team aligned to a single, valuable stream of work...[T]he team is empowered to **build** and deliver customer or user value as quickly, safely, and independently as possible, without requiring hand-offs to other teams to perform parts of the work. In a sense, we are talking about full stack teams (rather than full stack developers). 2 A team with end-to-end responsibility for delivery of user-facing functionality is also going to have a more obvious, direct connection to the end user as well. All too often I’ve seen “backend” teams with a hazy idea of what the software does or what the users need, which can cause all sorts of misunderstandings when it comes to implementing new functionality. On the other hand, the end-to-end teams will find it much easier to build a direct connection with the people using the software they create—they can be more focused on making sure that the people they are serving are getting what they need.

Một stream-aligned team là một đội được tổ chức xoay quanh một dòng công việc (stream) đơn lẻ, có giá trị... [Đ]ội được trao quyền để xây dựng và chuyển giao giá trị cho khách hàng hoặc người dùng nhanh nhất, an toàn nhất và độc lập nhất có thể, mà không cần bàn giao cho các đội khác để thực hiện một phần công việc. Theo một nghĩa nào đó, chúng ta đang nói về các đội full stack (chứ không phải lập trình viên full stack). 2 Một đội có trách nhiệm đầu-cuối trong việc chuyển giao chức năng hướng tới người dùng cũng sẽ có một mối liên hệ rõ ràng, trực tiếp hơn với người dùng cuối. Quá thường xuyên tôi đã thấy các đội “backend” chỉ có một ý niệm mơ hồ về việc phần mềm làm gì hoặc người dùng cần gì, điều này có thể gây ra đủ loại hiểu lầm khi đến lúc hiện thực chức năng mới. Mặt khác, các đội đầu-cuối sẽ thấy việc xây dựng một mối liên hệ trực tiếp với những người dùng phần mềm họ tạo ra dễ dàng hơn nhiều—họ có thể tập trung hơn vào việc đảm bảo rằng những người họ đang phục vụ nhận được thứ họ cần.

As a concrete example, I spent some time working with FinanceCo, a successful and growing Fintech firm based in Europe. At FinanceCo, virtually all teams work on

Như một ví dụ cụ thể, tôi đã dành một thời gian làm việc với FinanceCo, một công ty Fintech thành công và đang phát triển có trụ sở tại châu Âu. Tại FinanceCo, hầu như tất cả các đội đều làm việc trên

1 Matthew Skelton and Manuel Pais, Team Topologies (Portland, OR: IT Revolution, 2019).

1 Matthew Skelton and Manuel Pais, Team Topologies (Portland, OR: IT Revolution, 2019).

2 As Charity Majors says, “You’re not a full stack developer unless you build the chips.” software that directly impacts the customer experience and have customer-oriented key performance indicators (KPIs)—the success of a given team is driven less by how many features it has shipped and more by helping to improve the experience of the people using the software. It becomes very clear as to how a change can impact customers. This is only possible due to the fact that the majority of the teams have direct, customer-facing responsibilities in terms of the software they deliver. When you are more removed from the end user, it becomes harder to understand whether your contributions are successful, and you can end up focusing on goals that are a long way removed from things the

people using your software care about.

2 Như Charity Majors nói, “Bạn không phải là một lập trình viên full stack trừ khi bạn tự chế tạo cả con chip.” phần mềm tác động trực tiếp đến trải nghiệm khách hàng và có các chỉ số hiệu suất chính (KPI) hướng tới khách hàng—thành công của một đội bất kỳ ít được đo bằng số lượng tính năng nó đã xuất xưởng mà nhiều hơn bằng việc giúp cải thiện trải nghiệm của những người dùng phần mềm. Trở nên rất rõ ràng về việc một thay đổi có thể tác động đến khách hàng như thế nào. Điều này chỉ khả thi nhờ vào việc đa số các đội có trách nhiệm trực tiếp, hướng tới khách hàng về phần mềm họ chuyển giao. Khi bạn càng xa cách người dùng cuối, càng khó hiểu được liệu những đóng góp của bạn có thành công hay không, và bạn có thể rất cuộc tập trung vào những mục tiêu cách rất xa những thứ mà người dùng phần mềm của bạn quan tâm.

Let’s revisit the reasons why dedicated frontend teams exist—specialists, consistency, and technical challenges—and now let’s look at how these issues can be addressed.

Hãy xem lại những lý do vì sao các đội frontend chuyên trách tồn tại—chuyên gia, tính nhất quán, và các thách thức kỹ thuật—và bây giờ hãy xem những vấn đề này có thể được giải quyết ra sao.

Sharing Specialists — Chia sẻ chuyên gia

Good developers can be hard to find, and finding them is further complicated when you need developers with a certain specialty. In the area of user interfaces, for example, if you are providing native mobile as well as web interfaces, you could find yourself needing people who are experienced with iOS and Android as well as with modern web development. This is quite aside from the fact that you may want dedicated interaction designers, graphic designers, accessibility experts, and the like. People with the right depth of skills for these more “narrow” fields may be thin on the ground, and they may always have more work than there is time for.

Lập trình viên giỏi có thể khó tìm, và việc tìm họ còn phức tạp hơn khi bạn cần lập trình viên có một chuyên môn nhất định. Trong lĩnh vực giao diện người dùng chẳng hạn, nếu bạn đang cung cấp cả giao diện di động native lẫn giao diện web, bạn có thể thấy mình cần những người có kinh nghiệm với iOS và Android cũng như với phát triển web hiện đại. Đó là chưa kể đến việc bạn có thể muốn có các nhà thiết kế tương tác, nhà thiết kế đồ họa, chuyên gia về khả năng tiếp cận (accessibility) chuyên trách, và những vai trò tương tự. Những người có đủ độ sâu kỹ năng cho các lĩnh vực “hẹp” hơn này có thể rất hiếm, và họ có thể luôn có nhiều việc hơn thời gian cho phép.

As we’ve touched on previously, the traditional approach to organizational structures would have you putting all the people who possess the same skill set in the same team, allowing you to tightly control what they work on. But as we’ve also discussed, this leads to siloed organizations.

Như chúng ta đã đề cập trước đây, cách tiếp cận truyền thống đối với cấu trúc tổ chức sẽ khiến bạn đặt tất cả những người sở hữu cùng một bộ kỹ năng vào cùng một đội, cho phép bạn kiểm soát chặt việc họ làm gì. Nhưng như chúng ta cũng đã thảo luận, điều này dẫn đến các tổ chức bị chia silo.

Putting people with specialized skills in their own dedicated team also deprives you of opportunities for other developers to pick up these in-demand skills. You don’t need every developer to learn to become an expert iOS developer, for example, but it can still be useful for some of your developers to learn enough skills in that area to help out with the easy stuff, leaving your specialists free to tackle the really hard tasks. Skill sharing can also be helped by establishing communities of practice—you

might consider having a UI community that cuts across your teams, enabling people to share ideas and challenges with their peers.

Việc đặt những người có kỹ năng chuyên môn vào một đội chuyên trách riêng của họ cũng tương tự cơ hội để các lập trình viên khác tiếp thu những kỹ năng đang được sẵn đón này. Bạn không cần mọi lập trình viên trở thành chuyên gia phát triển iOS chẳng hạn, nhưng vẫn có thể hữu ích nếu một số lập trình viên của bạn học đủ kỹ năng trong lĩnh vực đó để có thể giúp làm những việc dễ, để các chuyên gia của bạn rảnh tay xử lý những tác vụ thực sự khó. Việc chia sẻ kỹ năng cũng có thể được hỗ trợ bằng cách thiết lập các cộng đồng thực hành (community of practice)—bạn có thể cân nhắc lập một cộng đồng UI cắt ngang các đội của mình, cho phép mọi người chia sẻ ý tưởng và thách thức với đồng nghiệp.

I remember when all **database** changes had to be done by a central pool of database administrators (DBAs). Developers had little awareness of how databases worked as a result, and would more frequently create software that used the database poorly. Furthermore, much of the work the experienced DBAs were being asked to do consisted of trivial changes. As more database work got pulled into delivery teams, developers in general got a better appreciation for databases and could start doing the trivial work themselves, freeing up the valuable DBAs to focus on more complex database issues, which made better use of their deeper skills and experience. A similar shift has happened in the space of operations and testers, with more of this work being pulled into teams.

Tôi nhớ thời mà tất cả các thay đổi cơ sở dữ liệu đều phải được thực hiện bởi một nhóm tập trung các quản trị viên cơ sở dữ liệu (database administrator, DBA). Hệ quả là lập trình viên ít hiểu biết về cách cơ sở dữ liệu hoạt động, và sẽ thường xuyên tạo ra phần mềm dùng cơ sở dữ liệu một cách kém cỏi hơn. Hơn nữa, phần lớn công việc mà các DBA giàu kinh nghiệm được yêu cầu làm lại là những thay đổi vật vãnh. Khi ngày càng nhiều công việc cơ sở dữ liệu được kéo vào các đội chuyển giao, lập trình viên nói chung hiểu cơ sở dữ liệu tốt hơn và có thể bắt đầu tự làm những việc vật, giải phóng các DBA quý giá để tập trung vào những vấn đề cơ sở dữ liệu phức tạp hơn, qua đó tận dụng tốt hơn kỹ năng và kinh nghiệm sâu của họ. Một sự chuyển dịch tương tự đã diễn ra trong lĩnh vực vận hành (operations) và kiểm thử viên (tester), với ngày càng nhiều công việc này được kéo vào trong các đội.

So, on the contrary, moving specialists out of dedicated teams won't inhibit the specialists' ability to do their job; in fact, it will likely increase the bandwidth they have to focus on the difficult problems that really need their attention.

Vậy nên, ngược lại, việc đưa các chuyên gia ra khỏi các đội chuyên trách sẽ không cản trở khả năng làm việc của họ; trên thực tế, nó nhiều khả năng còn làm tăng băng thông để họ tập trung vào những vấn đề khó thực sự cần đến sự chú ý của họ.

The trick is to find a more effective way to **deploy** your specialists. Ideally, they would be embedded in the teams. Sometimes, though, there might not be enough work to justify their full-time presence in a given team, in which case they may well split their time between multiple teams. Another model is to have a dedicated team with these skills whose explicit job is to enable other teams. In Team Topologies, Skelton and Pais describe these teams as enabling teams. Their job is to go out and help other teams who are focused on delivering new features to do their job. You can think of these teams as a bit more like an internal consultancy—they can come in and spend targeted time with a stream-aligned team, helping it become more self-sufficient in a particular area or else providing some dedicated time to help roll out a particularly difficult piece of work.

Bí quyết là tìm ra một cách hiệu quả hơn để bố trí các chuyên gia của bạn. Lý tưởng nhất, họ sẽ được nhúng (embed) vào trong các đội. Tuy nhiên, đôi khi có thể không có đủ việc để biện minh cho sự hiện diện toàn thời gian của họ trong một đội nhất định, trong trường

hợp đó họ hoàn toàn có thể chia thời gian giữa nhiều đội. Một mô hình khác là có một đội chuyên trách với những kỹ năng này mà nhiệm vụ rõ ràng của nó là tạo điều kiện cho các đội khác. Trong Team Topologies, Skelton và Pais mô tả những đội này là *enabling team*. Nhiệm vụ của họ là đi ra ngoài và giúp các đội khác đang tập trung vào việc chuyển giao tính năng mới làm tốt công việc của mình. Bạn có thể nghĩ về những đội này hơi giống một đơn vị tư vấn nội bộ—họ có thể đến và dành thời gian có chủ đích với một *stream-aligned team*, giúp đội đó trở nên tự chủ hơn trong một lĩnh vực cụ thể, hoặc dành thời gian chuyên trách để giúp triển khai một mảng công việc đặc biệt khó.

So whether your specialists are embedded full-time in a given team, or work to enable others to do the same work, you can remove organizational silos and also help skill up your colleagues at the same time.

Vậy bất kể chuyên gia của bạn được nhúng toàn thời gian vào một đội nhất định, hay làm việc để giúp người khác làm cùng công việc đó, bạn đều có thể loại bỏ các silo tổ chức và đồng thời cũng giúp nâng cao kỹ năng cho đồng nghiệp của mình.

Ensuring Consistency — Đảm bảo tính nhất quán

The second issue often cited as a reason for dedicated frontend teams is that of consistency. By having a single team responsible for a user interface, you ensure that the UI has a consistent look and feel. This can extend from easy things like using the same colors and fonts to solving the same interface problems in the same way—using a consistent design and interaction language that helps users interact with the system. This consistency doesn’t just help communicate a degree of polish regarding the product itself but also ensures that users will find it easier to use new functionality when it is delivered.

Vấn đề thứ hai thường được viện dẫn làm lý do cho các đội frontend chuyên trách là tính nhất quán. Bằng cách có một đội duy nhất chịu trách nhiệm cho một giao diện người dùng, bạn đảm bảo rằng UI có một diện mạo và cảm nhận nhất quán. Điều này có thể trải rộng từ những thứ dễ như dùng cùng màu sắc và phông chữ cho đến việc giải quyết cùng những vấn đề giao diện theo cùng một cách—dùng một ngôn ngữ thiết kế và tương tác nhất quán giúp người dùng tương tác với hệ thống. Tính nhất quán này không chỉ giúp truyền tải một mức độ tinh tế về chính sản phẩm mà còn đảm bảo rằng người dùng sẽ thấy dễ dùng chức năng mới hơn khi nó được chuyển giao.

There are ways, though, to help ensure a degree of consistency across teams. If you are using the **enabling team** model, with specialists spending time with multiple teams, they can help ensure that the work done by each team is consistent. The creation of shared resources like a living CSS style guide or shared UI components can also help.

Tuy vậy, có những cách để giúp đảm bảo một mức độ nhất quán xuyên suốt các đội. Nếu bạn đang dùng mô hình *enabling team*, với các chuyên gia dành thời gian cho nhiều đội, họ có thể giúp đảm bảo rằng công việc do mỗi đội thực hiện là nhất quán. Việc tạo ra các tài nguyên dùng chung như một bộ hướng dẫn phong cách CSS “sống” (*living CSS style guide*) hay các thành phần UI dùng chung cũng có thể giúp ích.

As a concrete example of an enabling team being used to help with consistency, the Financial Times Origami team builds web components in collaboration with the design team, which encapsulates the brand identity—ensuring a consistent look and feel across the *stream-aligned teams*. This type of enabling team provides two forms of aid—firstly, it shares its expertise in delivering already-built components, and secondly, it helps ensure that the UIs deliver a consistent user experience.

Như một ví dụ cụ thể về việc một enabling team được dùng để giúp đảm bảo tính nhất quán, đội Origami của Financial Times xây dựng các web component cộng tác với đội thiết kế, đóng gói bản sắc thương hiệu—đảm bảo một diện mạo và cảm nhận nhất quán xuyên suốt các stream-aligned team. Kiểu enabling team này mang lại hai dạng trợ giúp—thứ nhất, nó chia sẻ chuyên môn của mình trong việc cung cấp các thành phần đã được dựng sẵn, và thứ hai, nó giúp đảm bảo rằng các UI mang lại một trải nghiệm người dùng nhất quán.

It's worth noting, however, that the driver for consistency shouldn't be considered universally right. Some organizations make seemingly conscious decisions to not require consistency in their user interfaces, because they feel that allowing for more autonomy of teams is preferable. Amazon is one such organization. Earlier versions of its main shopping site had large degrees of inconsistency, with widgets using quite different styles of controls.

Tuy nhiên, đáng lưu ý rằng động lực hướng tới tính nhất quán không nên được coi là luôn đúng trong mọi trường hợp. Một số tổ chức dường như có chủ ý đưa ra quyết định không đòi hỏi tính nhất quán trong giao diện người dùng của họ, bởi họ cảm thấy việc cho phép các đội có nhiều quyền tự chủ hơn là điều đáng mong muốn hơn. Amazon là một tổ chức như vậy. Các phiên bản trước đây của trang mua sắm chính của họ có mức độ thiếu nhất quán lớn, với các widget dùng những kiểu control khá khác nhau.

This is shown to an even greater degree when you look at the web control panel for Amazon Web Services (AWS). Different products in AWS have massively different interaction models, making the user interface quite bewildering. This, however, seems to be a logical extension of Amazon's drive to reduce internal coordination between teams.

Điều này còn thể hiện ở mức độ lớn hơn nữa khi bạn nhìn vào bảng điều khiển web của Amazon Web Services (AWS). Các sản phẩm khác nhau trong AWS có những mô hình tương tác khác nhau một trời một vực, khiến giao diện người dùng khá khó hiểu. Tuy nhiên, điều này dường như là một hệ quả hợp lý của việc Amazon nỗ lực giảm bớt sự phối hợp nội bộ giữa các đội.

The increased autonomy of product teams in AWS appears to manifest itself in other ways as well, not just in terms of an often disjointed user experience. There frequently are multiple different ways to achieve the same task (by running a **container** workload, for example), with different product teams inside AWS often overlapping with one another with similar but incompatible solutions. You might criticize the end result, but AWS has shown that by having these highly autonomous product-oriented teams, it has created a company that has a clear market lead. Speed of delivery trumps a consistency of user experience, at least as far as AWS is concerned.

Quyền tự chủ gia tăng của các đội sản phẩm trong AWS dường như cũng biểu lộ theo những cách khác, không chỉ ở khía cạnh trải nghiệm người dùng thường rời rạc. Thường có nhiều cách khác nhau để hoàn thành cùng một tác vụ (chẳng hạn chạy một workload dạng container), với các đội sản phẩm khác nhau bên trong AWS thường chồng chéo lên nhau với những giải pháp tương tự nhưng không tương thích. Bạn có thể chỉ trích kết quả cuối cùng, nhưng AWS đã cho thấy rằng nhờ có những đội hướng-sản-phẩm tự chủ cao này, nó đã tạo ra một công ty có vị thế dẫn đầu thị trường rõ rệt. Tốc độ chuyển giao thắng thế trước tính nhất quán của trải nghiệm người dùng, ít nhất là theo quan điểm của AWS.

Working Through Technical Challenges — Vượt qua các thách thức kỹ thuật

We've gone through some interesting evolutions when it comes to the development of user interfaces—from green-screen terminal-based textual user interfaces to rich desktop applications, the web, and now native mobile experiences. In many ways, we have gone full circle and then some—our client applications are now built with such complexity and sophistication that they absolutely rival the complexity of the rich desktop applications that were the mainstay of user interface development well into the first decade of the twenty-first century.

Chúng ta đã trải qua một số bước tiến hóa thú vị khi nói đến việc phát triển giao diện người dùng—from các giao diện người dùng dạng văn bản trên terminal màn hình xanh, đến các ứng dụng desktop phong phú, web, và bây giờ là các trải nghiệm di động native. Theo nhiều nghĩa, chúng ta đã đi một vòng tròn tròn vẹn rồi còn hơn thế nữa—các ứng dụng client của chúng ta giờ đây được xây dựng với độ phức tạp và tinh vi đến mức chúng hoàn toàn sánh ngang với độ phức tạp của những ứng dụng desktop phong phú từng là trụ cột của phát triển giao diện người dùng cho đến tận thập niên đầu của thế kỷ hai mươi mốt.

To some degree, the more things change, the more they stay the same. We often are still working with the same UI controls as 20 years ago—buttons, checkboxes, forms, combo boxes, and the like. We've added a few more components to this space, but far fewer than you might think. What has changed is the technology we use to create these graphical user interfaces in the first place.

Ở một mức độ nào đó, mọi thứ càng thay đổi thì lại càng giữ nguyên. Chúng ta thường vẫn đang làm việc với cùng những UI control như 20 năm trước—nút bấm, hộp kiểm (checkbox), biểu mẫu, hộp combo (combo box), và những thứ tương tự. Chúng ta đã thêm một vài thành phần nữa vào không gian này, nhưng ít hơn nhiều so với bạn có thể nghĩ. Cái đã thay đổi là công nghệ mà ngay từ đầu chúng ta dùng để tạo ra những giao diện người dùng đồ họa này.

Some newer technology in this space, specifically single-page applications, cause us issues when it comes to decomposing a user interface. In addition, the wider variety of devices on which we expect the same user interface to be delivered cause other problems that need to be solved.

Một số công nghệ mới hơn trong không gian này, cụ thể là các ứng dụng đơn trang, gây cho chúng ta vấn đề khi đến lúc phân rã một giao diện người dùng. Ngoài ra, sự đa dạng rộng lớn hơn của các thiết bị mà chúng ta kỳ vọng sẽ cung cấp cùng một giao diện người dùng cũng gây ra những vấn đề khác cần được giải quyết.

Fundamentally, our users want to engage with our software in as seamless a way as possible. Be that through a browser on a desktop, or through a native or web mobile app, the result is the same—users interact with our software through a single pane of glass. They shouldn't care whether the user interface is built in a modular or monolithic way. So we have to look at ways to break apart our user interface functionality and bring it all back together again, all while resolving the challenges caused by single-page applications, mobile devices, and more. These issues will occupy us for the **rest** of this chapter.

Về cơ bản, người dùng của chúng ta muốn tương tác với phần mềm của chúng ta theo cách liền mạch nhất có thể. Dù là qua một trình duyệt trên desktop, hay qua một ứng dụng di động native hoặc web, kết quả vẫn như nhau—người dùng tương tác với phần mềm của chúng ta qua một tấm kính duy nhất (single pane of glass). Họ không nên bận tâm xem giao diện người dùng được xây dựng theo kiểu module hóa hay nguyên khối. Vậy nên chúng

ta phải tìm cách tách nhỏ chức năng giao diện người dùng của mình rồi gom tất cả lại với nhau lần nữa, đồng thời giải quyết những thách thức do các ứng dụng đơn trang, thiết bị di động, và hơn thế nữa gây ra. Những vấn đề này sẽ chiếm phần còn lại của chương này.

Pattern: Monolithic Frontend — Pattern: Frontend nguyên khối (Monolithic Frontend)

The monolithic frontend pattern describes an architecture in which all the UI state and behavior is defined in the UI itself, with calls made to backing microservices to get required data or carry out required operations. Figure 14-3 shows an example of this. Our screen wants to display information about an album and its track listing, so the UI makes a request to pull this data from the Album **microservice**. We also display information about the most recent special offers by requesting information from the Promotions microservice. In this example, our microservices return JSON that the UI uses to update the displayed information.

Pattern frontend nguyên khối mô tả một kiến trúc trong đó toàn bộ trạng thái và hành vi của UI được định nghĩa trong chính UI, với các lời gọi đến những microservice hậu thuẫn để lấy dữ liệu cần thiết hoặc thực hiện các thao tác cần thiết. Hình 14-3 cho thấy một ví dụ về điều này. Màn hình của chúng ta muốn hiển thị thông tin về một album và danh sách bài hát của nó, vì vậy UI gửi một yêu cầu để kéo dữ liệu này từ microservice Album. Chúng ta cũng hiển thị thông tin về các ưu đãi đặc biệt mới nhất bằng cách yêu cầu thông tin từ microservice Promotions. Trong ví dụ này, các microservice của chúng ta trả về JSON mà UI dùng để cập nhật thông tin hiển thị.

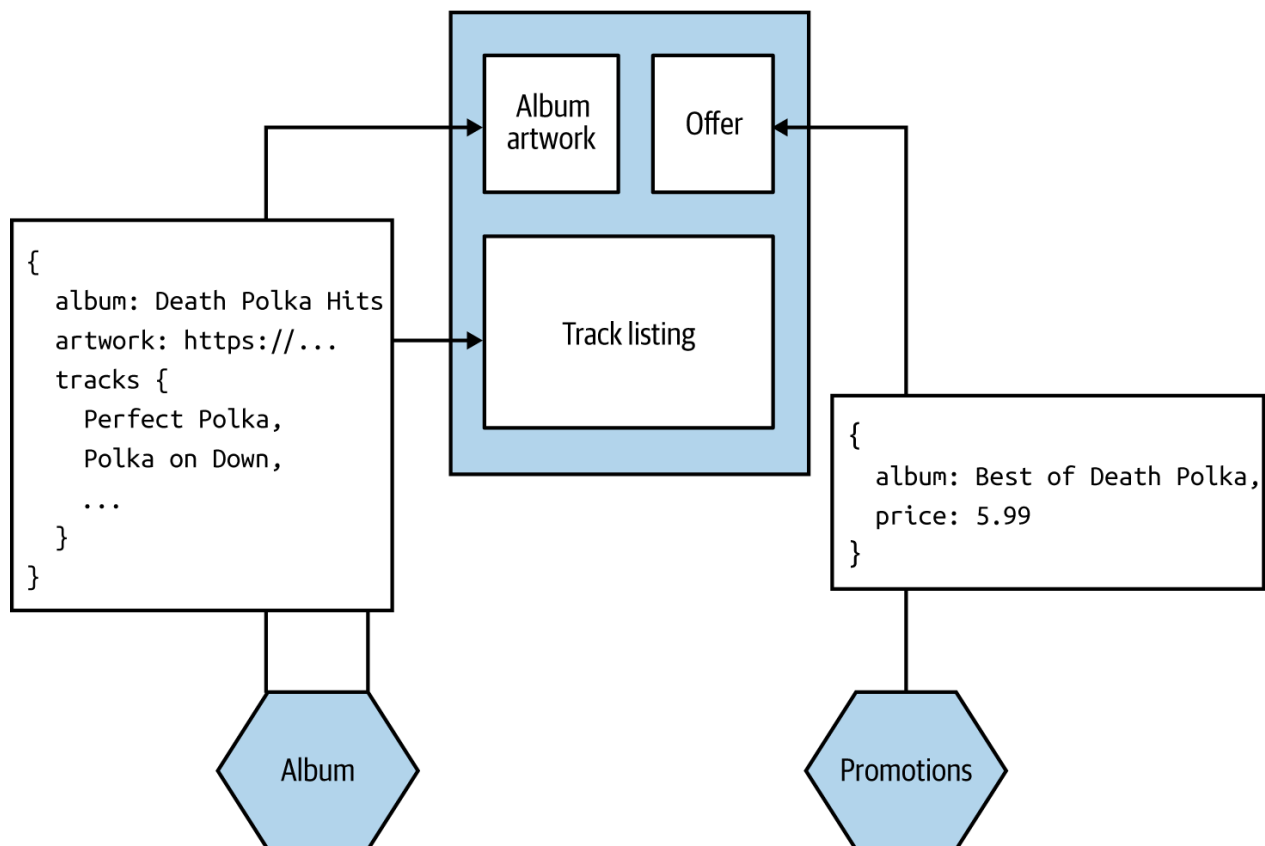


Figure 14-3. Our Album details screen pulls information from downstream microservices to

render the UI This model is the most common one for people building monolithic single-page applications, often with a dedicated frontend team. The requirements of our microservices are pretty straightforward—they just need to share information in a form that can be easily interpreted by the UI. In the case of a web-based UI, this means our microservices would likely need to provide data in a textual format, JSON being the most likely choice. The UI would then need to create the various components that make up the interface, handling synchronization of state and the like with the backend. Using a binary protocol for **service**-to-service communication would be more difficult for web-based clients but could be fine for native mobile devices or “thick” desktop applications.

Hình 14-3. Màn hình chi tiết Album của chúng ta kéo thông tin từ các microservice phía sau (downstream) để kết xuất UI. Mô hình này là mô hình phổ biến nhất đối với những người xây dựng các ứng dụng đơn trang nguyên khối, thường là với một đội frontend chuyên trách. Các yêu cầu đối với microservice của chúng ta khá đơn giản—chúng chỉ cần chia sẻ thông tin ở một dạng mà UI có thể dễ dàng diễn giải. Trong trường hợp một UI dựa trên web, điều này nghĩa là các microservice của chúng ta nhiều khả năng cần cung cấp dữ liệu ở dạng văn bản, với JSON là lựa chọn khả dĩ nhất. Sau đó UI sẽ cần tạo ra các thành phần khác nhau cấu thành giao diện, xử lý việc đồng bộ trạng thái và những thứ tương tự với backend. Việc dùng một giao thức nhị phân cho giao tiếp giữa các dịch vụ sẽ khó hơn đối với các client dựa trên web nhưng có thể ổn đối với thiết bị di động native hoặc các ứng dụng desktop “béo”.

When to Use It — Khi nào nên dùng

There are some downsides to this approach. Firstly, by its nature as a monolithic entity, it can become a driver for (or be caused by) a dedicated frontend team. Having multiple teams share responsibility for this monolithic frontend can be challenging due to the multiple sources of contention. Secondly, we have little ability to tailor the responses for different sorts of devices. If using web technology, we can change the layout of a screen to accommodate different device constraints, but this doesn’t necessarily extend to changing the calls being made to the supporting microservices. My mobile client may only be able to display 10 fields of an order, but if the microservice pulls back all one hundred fields of the order, we end up retrieving unnecessary data. One solution to this approach is for the user interface to specify what fields to pull back when it makes a request, but this assumes that each supporting microservice supports this form of interaction. In “**GraphQL**” on page 488 , we’ll look at how the use of both the **backend for frontend** pattern and GraphQL can help in this case.

Có một số nhược điểm với cách tiếp cận này. Thứ nhất, do bản chất của nó là một thực thể nguyên khối, nó có thể trở thành động lực dẫn tới (hoặc bị gây ra bởi) một đội frontend chuyên trách. Việc để nhiều đội cùng chia sẻ trách nhiệm cho frontend nguyên khối này có thể là thách thức do nhiều nguồn tranh chấp. Thứ hai, chúng ta có rất ít khả năng tùy chỉnh phản hồi cho các loại thiết bị khác nhau. Nếu dùng công nghệ web, chúng ta có thể thay đổi bố cục của một màn hình để phù hợp với những ràng buộc khác nhau của thiết bị, nhưng điều này không nhất thiết mở rộng đến việc thay đổi các lời gọi tới những microservice hỗ trợ. Client di động của tôi có thể chỉ hiển thị được 10 trường của một đơn hàng, nhưng nếu microservice kéo về tất cả một trăm trường của đơn hàng, chúng ta rất cuộc lại lấy về dữ liệu không cần thiết. Một giải pháp cho cách tiếp cận này là để giao diện người dùng chỉ định cần lấy về những trường nào khi nó gửi một yêu cầu, nhưng điều này giả định rằng mỗi microservice hỗ trợ hỗ trợ kiểu tương tác này. Trong phần “**GraphQL**” ở trang 488, chúng ta sẽ xem xét cách dùng cả pattern Backend for Frontend lẫn GraphQL có thể giúp ích trong trường hợp này.

Really, this pattern works best when you want all of the implementation and behavior of your UI in one deployable unit. For a single team developing both the frontend and all supporting microservices, that might be fine. Personally, if you have more than one team working on your software, I think you should fight against that urge, as it can result in you slipping into a layered architecture with the associated organizational silos. If, however, you are unable to avoid a layered architecture and matching organizational structure, this is probably the pattern you'll end up using.

Thực sự thì pattern này hoạt động tốt nhất khi bạn muốn toàn bộ phần hiển thị và hành vi của UI nằm trong một đơn vị triển khai duy nhất. Đối với một đội duy nhất phát triển cả frontend lẫn tất cả các microservice hỗ trợ, điều đó có thể ổn. Cá nhân tôi, nếu bạn có nhiều hơn một đội làm việc trên phần mềm của mình, tôi nghĩ bạn nên chống lại sự thôi thúc đó, vì nó có thể khiến bạn trượt vào một kiến trúc phân tầng với các silo tổ chức đi kèm. Tuy nhiên, nếu bạn không thể tránh được một kiến trúc phân tầng và cấu trúc tổ chức tương ứng, thì đây có lẽ là pattern mà bạn rốt cuộc sẽ dùng.

Pattern: Micro Frontends — Pattern: Micro Frontend

The micro frontend approach is an organizational pattern whereby different parts of a frontend can be worked on and deployed independently. To quote from a highly recommended article by Cam Jackson on the topic, ³ we can define micro frontends as

Cách tiếp cận micro frontend là một pattern tổ chức theo đó các phần khác nhau của một frontend có thể được làm việc và triển khai một cách độc lập. Trích từ một bài viết rất đáng đọc của Cam Jackson về chủ đề này, ³ chúng ta có thể định nghĩa micro frontend như sau:

³ Cam Jackson, “Micro Frontends,” martinfowler.com, June 19, 2019, <https://oreil.ly/U3K40> .

³ Cam Jackson, “Micro Frontends,” martinfowler.com, June 19, 2019, <https://oreil.ly/U3K40>

follows: “An architectural style where independently deliverable frontend applications are composed into a greater whole.”

“Một phong cách kiến trúc trong đó các ứng dụng frontend có thể chuyển giao độc lập được kết hợp lại thành một tổng thể lớn hơn.”

It becomes an essential pattern for stream-aligned teams who want to own delivery of both backend microservices and the supporting UI. Where microservices deliver independent deployability for the backend functionality, micro frontends deliver independent deployability for the frontend.

Nó trở thành một pattern thiết yếu cho các stream-aligned team muốn sở hữu việc chuyển giao cả microservice backend lẫn UI hỗ trợ. Trong khi microservice mang lại khả năng triển khai độc lập cho chức năng backend, thì micro frontend mang lại khả năng triển khai độc lập cho frontend.

The micro frontend concept has gained popularity due to the challenges created by monolithic, JavaScript-heavy web UIs, as typified by single-page applications. With a micro frontend, different teams can work on and make changes to different parts of the frontend. Coming back to Figure 14-2 , the stock team, purchase flow team, and customer profile team are each able to change the frontend functionality associated with their stream of work independent of the other teams.

Khái niệm micro frontend đã trở nên phổ biến do những thách thức mà các UI web nguyên khối, nặng JavaScript gây ra, mà điển hình là các ứng dụng đơn trang. Với micro frontend, các đội khác nhau có thể làm việc và thực hiện thay đổi trên những phần khác nhau của frontend. Quay lại Hình 14-2, đội kho hàng (stock team), đội luồng mua hàng (purchase

flow team), và đội hồ sơ khách hàng (customer profile team) mỗi đội đều có thể thay đổi chức năng frontend gắn với dòng công việc của mình một cách độc lập với các đội khác.

Implementation — Hiện thực

For web-based frontends, we can consider two key decompositional techniques that can aid implementation of the micro frontend pattern. Widget-based decomposition involves splicing different parts of a frontend together into a single screen. Page-based decomposition, on the other hand, has the frontend split apart into independent web pages. Both approaches are worthy of further exploration, which we'll get to shortly.

Đối với các frontend dựa trên web, chúng ta có thể cân nhắc hai kỹ thuật phân rã chủ chốt có thể hỗ trợ việc hiện thực pattern micro frontend. Phân rã theo widget (widget-based decomposition) liên quan đến việc ghép các phần khác nhau của một frontend lại với nhau thành một màn hình duy nhất. Trong khi đó, phân rã theo trang (page-based decomposition) tách frontend ra thành các trang web độc lập. Cả hai cách tiếp cận đều đáng được khám phá thêm, và chúng ta sẽ sớm đến với việc đó.

When to Use It — Khi nào nên dùng

The micro frontend pattern is essential if you want to adopt end-to-end, stream-aligned teams where you are trying to move away from a layered architecture. I could also imagine it being useful in a situation in which you want to retain a layered architecture, but the functionality of the frontend is now so large that multiple dedicated frontend teams are required.

Pattern micro frontend là thiết yếu nếu bạn muốn áp dụng các đội đầu-cuối, stream-aligned khi bạn đang cố rời bỏ một kiến trúc phân tầng. Tôi cũng có thể hình dung nó sẽ hữu ích trong tình huống bạn muốn giữ lại một kiến trúc phân tầng, nhưng chức năng của frontend giờ đây lớn đến mức cần nhiều đội frontend chuyên trách.

There is one key problem with this approach that I'm not sure can be solved. Sometimes the capabilities offered by a microservice do not fit neatly into a widget or a page. Sure, I might want to surface recommendations in a box on a page on our website, but what if I want to weave in dynamic recommendations elsewhere? When I search, I want the type ahead to automatically trigger fresh recommendations, for example. The more cross-cutting a form of interaction is, the less likely this model will fit, and the more likely it is that we'll fall back to just making API calls.

Có một vấn đề chính với cách tiếp cận này mà tôi không chắc có thể giải quyết được. Đôi khi các năng lực do một microservice cung cấp không khớp gọn gàng vào một widget hay một trang. Chắc chắn rồi, tôi có thể muốn hiển thị các gợi ý trong một hộp trên một trang của website, nhưng nếu tôi muốn đan xen các gợi ý động ở chỗ khác thì sao? Ví dụ, khi tôi tìm kiếm, tôi muốn gợi ý tự động (type ahead) tự động kích hoạt những gợi ý mới mẻ. Một dạng tương tác càng cắt ngang (cross-cutting) thì mô hình này càng ít khả năng phù hợp, và càng nhiều khả năng chúng ta sẽ quay lại với việc chỉ thực hiện các lời gọi API.

Self-Contained Systems — Self-Contained Systems

A self-contained system (SCS) is a style of architecture that arose, arguably, out of the lack of focus on UI concerns during the early years of microservices. An SCS could consist of multiple moving parts (potentially microservices) that when taken together constitute a single SCS.

Một self-contained system (SCS) là một phong cách kiến trúc nảy sinh, có thể nói là, từ việc thiếu chú trọng vào các mối quan tâm về UI trong những năm đầu của microservice. Một SCS có thể gồm nhiều bộ phận cấu thành (có khả năng là các microservice) mà khi gộp lại với nhau tạo thành một SCS duy nhất.

As defined, a self-contained system has to comply with some specific criteria that we can see overlap with some of the same things we are trying to achieve with microservices. You can find more information about self-contained systems on the very clear SCS website , but here are some highlights:

Theo định nghĩa, một self-contained system phải tuân thủ một số tiêu chí cụ thể mà chúng ta có thể thấy chồng lấn với một số điều tương tự mà chúng ta đang cố đạt được với microservice. Bạn có thể tìm thêm thông tin về self-contained system trên trang web SCS rất rõ ràng, nhưng đây là một số điểm nổi bật:

- Each SCS is an autonomous web application with no shared UI.
 - Each SCS is owned by one team.
 - Async communication should be used whenever possible.
 - No business code can be shared between SCSs.
- Mỗi SCS là một ứng dụng web tự chủ không có UI dùng chung.
 - Mỗi SCS được sở hữu bởi một đội.
 - Nên dùng giao tiếp bất đồng bộ bất cứ khi nào có thể.
 - Không được chia sẻ mã nghiệp vụ nào giữa các SCS.

The SCS approach hasn't caught on to the same extent as microservices, and it's not a concept I encounter much, despite the fact that I agree with many of the principles it outlines. I especially like calling out that a self-contained system should be owned by one team. I do wonder if this lack of wider use explains why some aspects of the SCS approach seem overly narrow and prescriptive. For example, the insistence on each SCS being an "autonomous web application" implies that many types of user interface couldn't ever be considered an SCS. Does that mean the native iOS application I've built that uses gRPC can be part of an SCS or not?

Cách tiếp cận SCS chưa phổ biến đến mức như microservice, và nó không phải là một khái niệm tôi gặp nhiều, dù tôi đồng ý với nhiều nguyên tắc mà nó vạch ra. Tôi đặc biệt thích việc nó nhấn mạnh rằng một self-contained system nên được sở hữu bởi một đội. Tôi tự hỏi liệu sự thiếu phổ biến rộng rãi này có giải thích vì sao một số khía cạnh của cách tiếp cận SCS dường như quá hẹp và quá cứng nhắc. Ví dụ, sự khẳng định rằng mỗi SCS phải là một "ứng dụng web tự chủ" ngụ ý rằng nhiều loại giao diện người dùng sẽ không bao giờ có thể được coi là một SCS. Vậy điều đó có nghĩa là ứng dụng iOS native mà tôi đã xây dựng có dùng gRPC có thể là một phần của một SCS hay không?

So is the SCS approach in conflict with microservices? Not really. I've worked on many microservices that, when considered in isolation would fit the definition of an SCS all by themselves. There are some interesting ideas in the SCS approach that I agree with, and many of them we've already covered in this book. I just find the approach to be overly prescriptive, to the extent that someone interested in SCSs might find adopting the approach to be incredibly challenging, as it might require wholesale changes to so many aspects of their software delivery.

Vậy cách tiếp cận SCS có mâu thuẫn với microservice không? Không hẳn. Tôi đã làm việc trên nhiều microservice mà, khi xét riêng lẻ, sẽ tự khớp với định nghĩa của một SCS. Có một số ý tưởng thú vị trong cách tiếp cận SCS mà tôi đồng ý, và nhiều trong số đó chúng ta đã đề cập trong cuốn sách này. Tôi chỉ thấy cách tiếp cận này quá cứng nhắc, đến mức một người quan tâm đến SCS có thể thấy việc áp dụng cách tiếp cận này cực kỳ thách thức, vì

nó có thể đòi hỏi những thay đổi toàn diện đối với rất nhiều khía cạnh trong việc chuyển giao phần mềm của họ.

I do worry that manifestos like the SCS concept can guide us down the path of focusing too much on activity, rather than on principles and outcomes. You could follow every SCS characteristic and still potentially miss the point. On reflection, I feel that the SCS approach is a tech-focused approach to promote an organizational concept. As such, I'd rather focus on the importance of stream-aligned teams with reduced coordination, and let the technology and architecture flow from that.

Tôi quả thực lo ngại rằng những bản tuyên ngôn (manifesto) như khái niệm SCS có thể dẫn dắt chúng ta xuống con đường tập trung quá nhiều vào hoạt động, thay vì vào các nguyên tắc và kết quả. Bạn có thể tuân theo từng đặc tính của SCS mà vẫn có khả năng lạc mất trọng tâm. Ngẫm lại, tôi cảm thấy rằng cách tiếp cận SCS là một cách tiếp cận thiên về công nghệ nhằm cổ vũ một khái niệm về tổ chức. Như vậy, tôi thà tập trung vào tầm quan trọng của các stream-aligned team với sự phối hợp được giảm bớt, và để công nghệ cùng kiến trúc tuôn ra từ đó.

Pattern: Page-Based Decomposition — Pattern: Phân rã theo trang (Page-Based Decomposition)

In page-based decomposition, our UI is decomposed into multiple web pages. Different sets of pages can be served from different microservices. In Figure 14-4, we see an example of this pattern for MusicCorp. Requests for pages in /albums/ are routed directly to the Albums microservice, which handles serving up those pages, and we do something similar with /artists/. A common navigation is used to help stitch together these pages. These microservices in turn might fetch information that is needed to construct these pages—for example, fetching stock levels from the Inventory microservice to show on the UI what items are in stock.

Trong phân rã theo trang, UI của chúng ta được phân rã thành nhiều trang web. Các nhóm trang khác nhau có thể được phục vụ từ các microservice khác nhau. Trong Hình 14-4, chúng ta thấy một ví dụ về pattern này cho MusicCorp. Các yêu cầu cho những trang trong /albums/ được định tuyến trực tiếp đến microservice Albums, đảm nhận việc phục vụ những trang đó, và chúng ta làm điều gì đó tương tự với /artists/. Một thanh điều hướng chung được dùng để giúp ghép các trang này lại với nhau. Đến lượt mình, các microservice này có thể nạp thông tin cần thiết để dựng nên các trang đó—ví dụ, lấy mức tồn kho từ microservice Inventory để hiển thị trên UI những mặt hàng nào còn hàng.

<https://musiccorp/albums/...>

<https://musiccorp/artists/...>

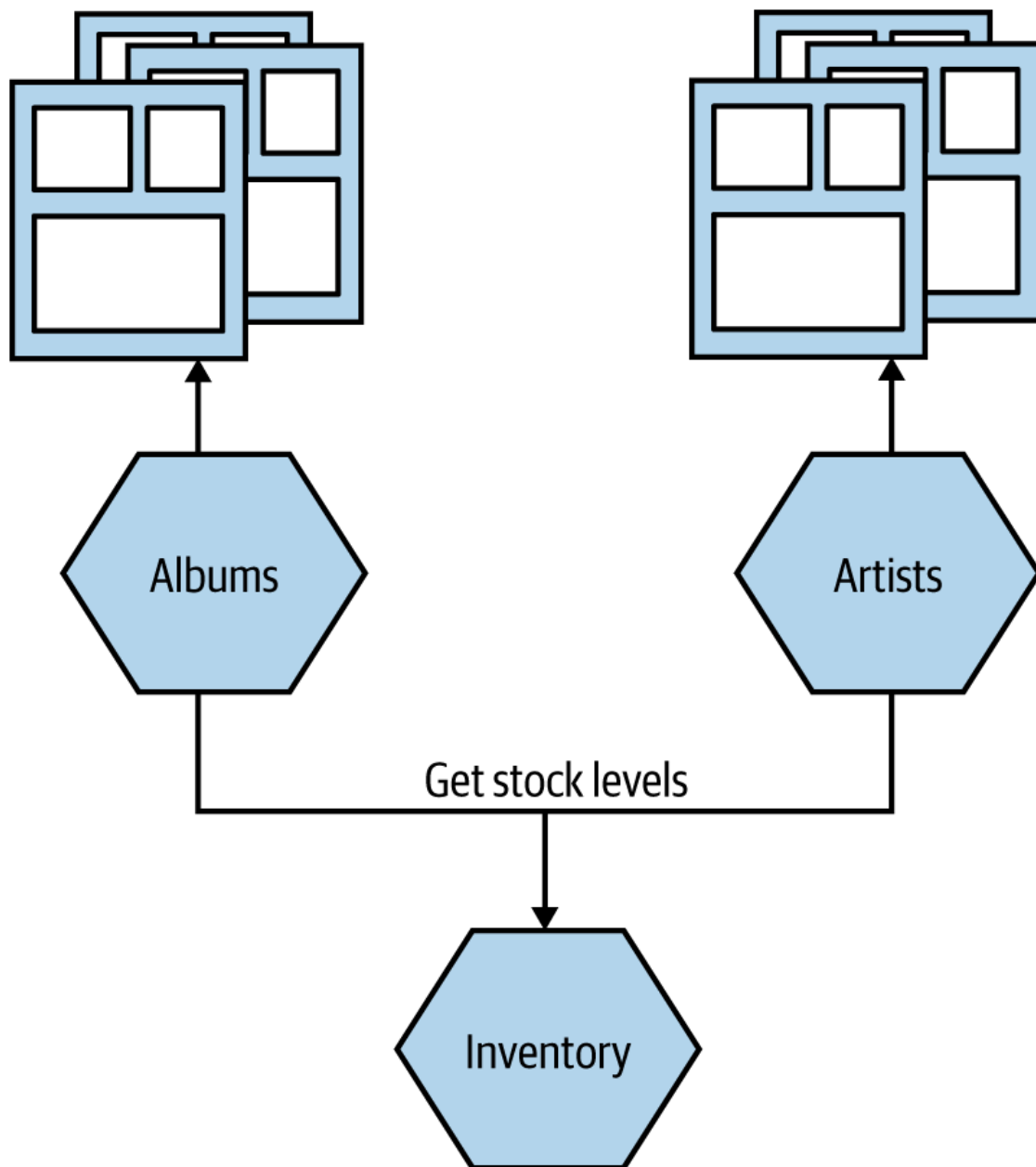


Figure 14-4. The user interface consists of multiple pages, with different groups of pages served up from different microservices

Hình 14-4. Giao diện người dùng gồm nhiều trang, với các nhóm trang khác nhau được phục vụ từ các microservice khác nhau

With this model, a team owning the Albums microservice would be able to render a full UI end to end, making it easy for the team to understand how its changes would impact the user.

Với mô hình này, một đội sở hữu microservice Albums sẽ có thể kết xuất một UI hoàn chỉnh từ đầu đến cuối, giúp đội dễ dàng hiểu được những thay đổi của mình sẽ tác động đến người dùng ra sao.

The Web — Web

Before single-page apps, we had the web. Our interaction with the web was based on visiting URLs and clicking on links that caused new pages to be loaded into our browser. Our browsers were built to allow for navigation around these pages, with bookmarks to mark pages of interest and backward and forward controls to revisit previously accessed pages. You might all be rolling your eyes and thinking, “Of course I know how the web works!”; however, it is a style of user interface that seems to have fallen out of favor. Its simplicity is something I miss when I see our current web-based user interface implementations—we’ve lost a lot by automatically assuming that a web-based UI means single-page apps.

Trước các ứng dụng đơn trang, chúng ta đã có web. Việc tương tác của chúng ta với web dựa trên việc truy cập các URL và nhấp vào các liên kết khiến những trang mới được nạp vào trình duyệt của chúng ta. Các trình duyệt của chúng ta được xây dựng để cho phép điều hướng quanh những trang này, với dấu trang (bookmark) để đánh dấu các trang quan tâm và các nút lùi (backward) và tiến (forward) để xem lại những trang đã truy cập trước đó. Tất cả các bạn có thể đang đảo mắt và nghĩ, “Tất nhiên là tôi biết web hoạt động thế nào!”; tuy nhiên, đó là một phong cách giao diện người dùng dường như đã không còn được ưa chuộng. Sự đơn giản của nó là thứ tôi nhớ khi tôi nhìn vào những bản hiện thực giao diện người dùng dựa trên web hiện tại của chúng ta—chúng ta đã đánh mất rất nhiều khi tự động cho rằng một UI dựa trên web đồng nghĩa với các ứng dụng đơn trang.

In terms of dealing with different types of clients, there is nothing to stop the page adapting what it shows based on the nature of the device requesting the page. The concepts of progressive enhancement (or **graceful degradation**) should be well understood by now.

Về việc xử lý các loại client khác nhau, không có gì ngăn cản trang web điều chỉnh nội dung nó hiển thị dựa trên bản chất của thiết bị đang yêu cầu trang. Các khái niệm về cải tiến tuần tiến (progressive enhancement) (hay suy giảm có kiểm soát) đến giờ chắc đã được hiểu rõ.

The simplicity of page-based decomposition from a technical implementation point of view is a real appeal here. You don’t need any fancy JavaScript running in the browser, nor do you need to use problematic iFrames. The user clicks a link, and a new page is requested.

Sự đơn giản của phân rã theo trang từ góc độ hiện thực kỹ thuật là một điểm hấp dẫn thực sự ở đây. Bạn không cần bất kỳ JavaScript hoa mỹ nào chạy trong trình duyệt, cũng không cần dùng các iFrame phức tạp. Người dùng nhấp vào một liên kết, và một trang mới được yêu cầu.

Where to Use It — Khi nào nên dùng

Useful for either a monolithic frontend or a micro frontend approach, page-based decomposition would be my default choice for user-interface decomposition if my user interface was a website. The web page as a unit of decomposition is such a core concept of the web as a whole that it becomes a simple and obvious technique for breaking down a large web-based user interface.

Hữu ích cho cả cách tiếp cận frontend nguyên khối lẫn micro frontend, phân rã theo trang sẽ là lựa chọn mặc định của tôi cho việc phân rã giao diện người dùng nếu giao diện người dùng của tôi là một website. Trang web như một đơn vị phân rã là một khái niệm cốt lõi của web nói chung đến mức nó trở thành một kỹ thuật đơn giản và hiển nhiên để chia nhỏ một giao diện người dùng web lớn.

I think the problem is that in the rush to use single-page application technology, these user interfaces are becoming increasingly rare, to the extent that user experiences that in my opinion would better fit a website implementation end up being shoehorned into a single-page application. 4 You can of course combine page-based decomposition with some of the other patterns we have covered. I could have a page that contains widgets, for example—something we'll look at next.

Tôi nghĩ vấn đề là trong cơn vội vã dùng công nghệ ứng dụng đơn trang, những giao diện người dùng này đang trở nên ngày càng hiếm, đến mức những trải nghiệm người dùng mà theo ý tôi sẽ phù hợp hơn với một bản hiện thực dạng website lại bị nhồi nhét vào một ứng dụng đơn trang. 4 Tất nhiên bạn có thể kết hợp phân rã theo trang với một số pattern khác mà chúng ta đã đề cập. Ví dụ, tôi có thể có một trang chứa các widget—một điều mà chúng ta sẽ xem xét tiếp theo.

4 I'm looking at you, Sydney Morning Herald !

4 Tôi đang nói về bạn đấy, Sydney Morning Herald!

Pattern: Widget-Based Decomposition — Pattern: Phân rã theo widget (Widget-Based Decomposition)

With widget-based decomposition, a screen in a graphical interface contains widgets that can be independently changed. In Figure 14-5 , we see an example of Music Corp's frontend, with two widgets providing the UI functionality for the shopping basket and the recommendations.

Với phân rã theo widget, một màn hình trong giao diện đồ họa chứa các widget có thể được thay đổi độc lập. Trong Hình 14-5, chúng ta thấy một ví dụ về frontend của MusicCorp, với hai widget cung cấp chức năng UI cho giỏ hàng và các gợi ý.

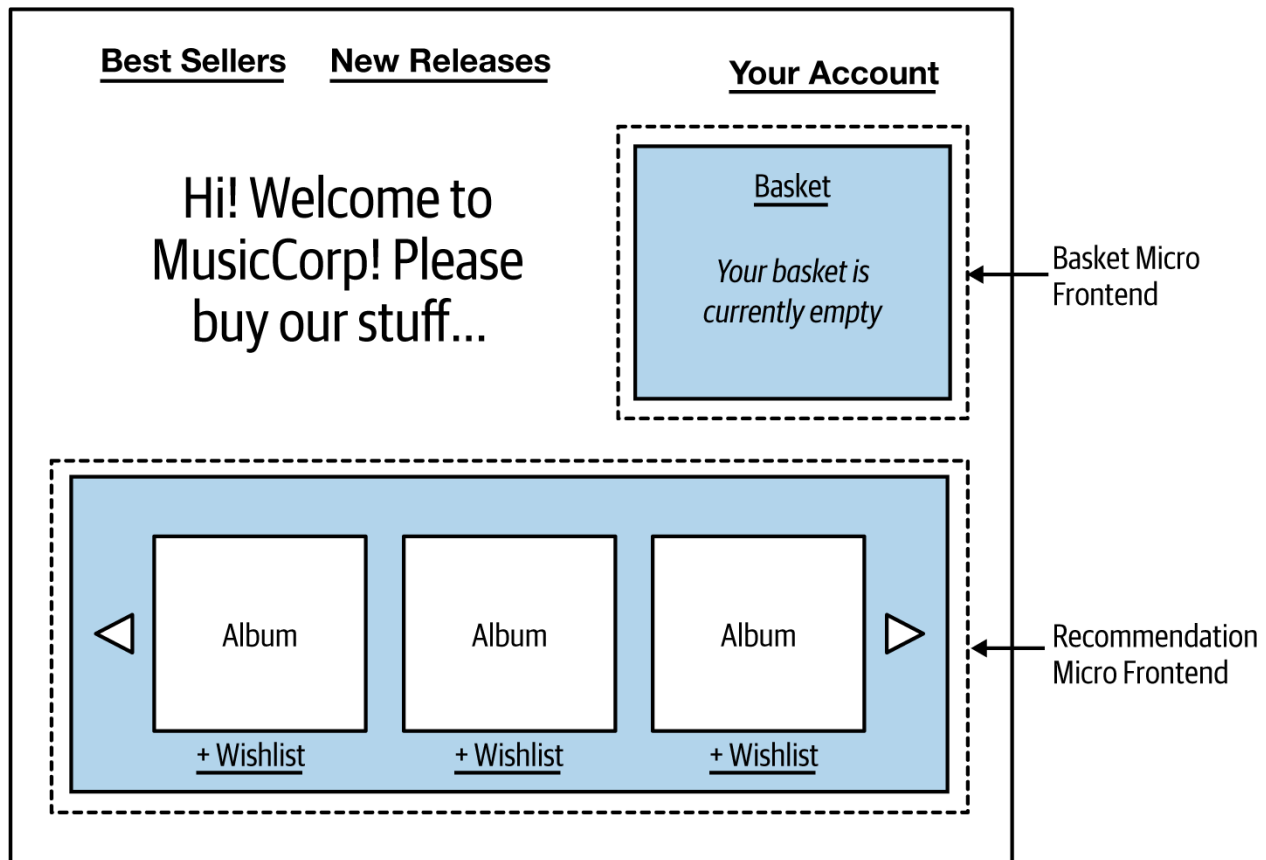


Figure 14-5. Basket and recommendation widgets being used on MusicCorp

Hình 14-5. Các widget giỏ hàng và gợi ý đang được dùng trên MusicCorp

The recommendation widget for MusicCorp pulls back a carousel of recommendations that can be cycled through and filtered. As we see in Figure 14-6, when the user interacts with the recommendation widget—cycling through to the next set of recommendations, for example, or adding items to their wishlist—it may result in calls being made to backing microservices, perhaps in this case the Recommendations and Wishlist microservices. This could align well with a team that owns both those backing microservices and the component itself.

Widget gợi ý cho MusicCorp kéo về một băng chuyền (carousel) các gợi ý có thể được lướt qua và lọc. Như chúng ta thấy trong Hình 14-6, khi người dùng tương tác với widget gợi ý—chẳng hạn lướt sang tập gợi ý tiếp theo, hoặc thêm các mặt hàng vào danh sách yêu thích (wishlist) của họ—nó có thể dẫn đến các lời gọi được thực hiện tới những microservice phía sau, trong trường hợp này có lẽ là các microservice Recommendations và Wishlist. Điều này có thể khớp tốt với một đội sở hữu cả những microservice phía sau đó lần chính thành phần kia.

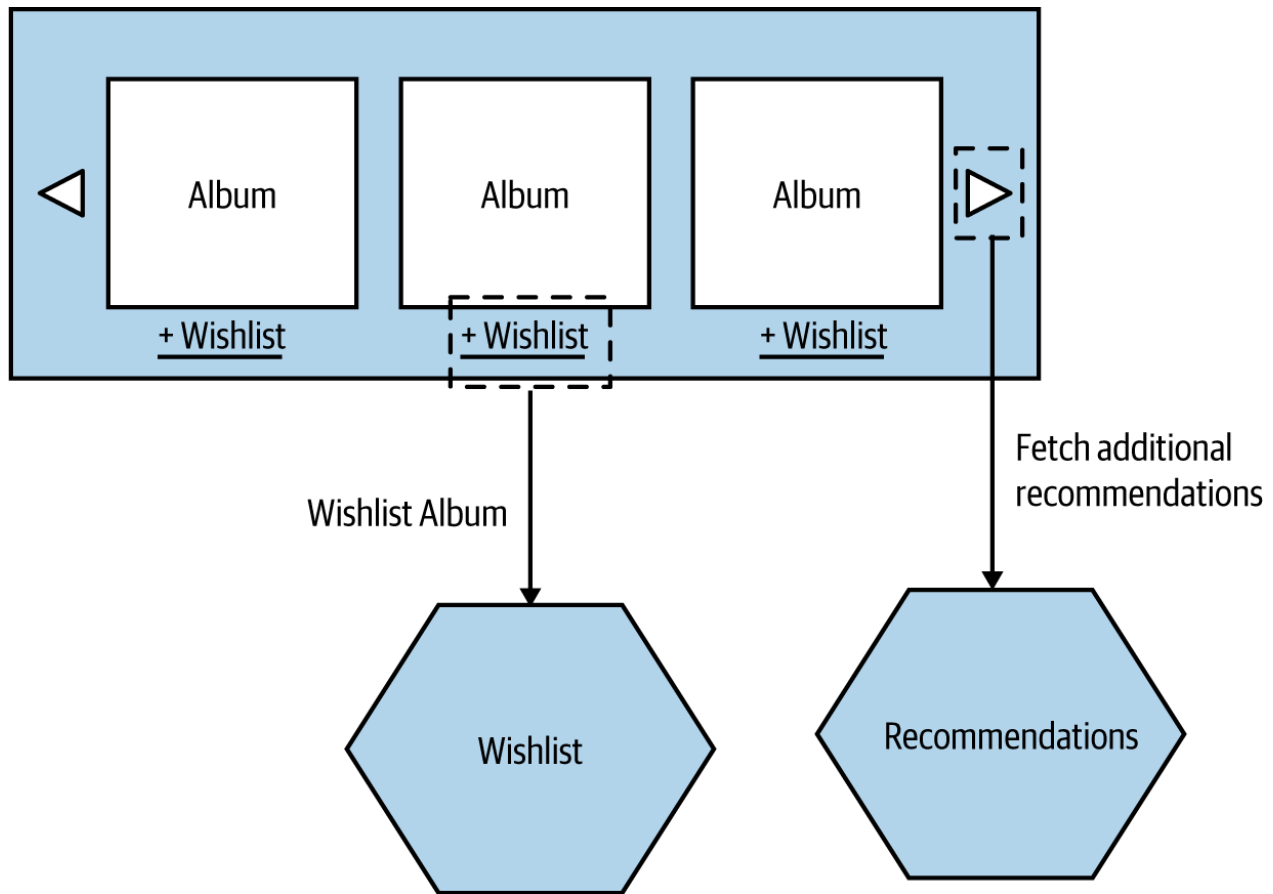


Figure 14-6. Interactions between the recommendation micro frontend and supporting microservices

Hình 14-6. Các tương tác giữa micro frontend gợi ý và các microservice hỗ trợ

Generally speaking, you'll need a "container" application that defines things like the core navigation for the interface and which widgets need to be included. If we were thinking in terms of end-to-end stream-oriented teams, we could imagine a single team providing the recommendation widget and also being responsible for a supporting Recommendations microservice.

Nói chung, bạn sẽ cần một ứng dụng "vật chứa" (container) để định nghĩa những thứ như thanh điều hướng cốt lõi cho giao diện và những widget nào cần được đưa vào. Nếu chúng ta tư duy theo các đội đầu-cuối, hướng-dòng-công-việc, chúng ta có thể hình dung một đội duy nhất cung cấp widget gợi ý đồng thời cũng chịu trách nhiệm cho một microservice Recommendations hỗ trợ.

This pattern is found a lot in the real world. The Spotify user interface, for example, makes heavy use of this pattern. One widget might hold a playlist, another might hold information about an artist, and a third widget might hold information about the artists and other Spotify users that you follow. These widgets are combined in different ways in different situations.

Pattern này được tìm thấy rất nhiều trong thực tế. Ví dụ, giao diện người dùng của Spotify sử dụng rất nhiều pattern này. Một widget có thể chứa một danh sách phát (playlist), một widget khác có thể chứa thông tin về một nghệ sĩ, và widget thứ ba có thể chứa thông tin về các nghệ sĩ và những người dùng Spotify khác mà bạn theo dõi. Các widget này được kết hợp theo những cách khác nhau trong những tình huống khác nhau.

You still need some sort of assembly layer to pull these parts together. This could be as simple as

making use of server-side or client-side templates, though.

Bạn vẫn cần một dạng tầng lắp ráp (assembly layer) nào đó để gom các phần này lại với nhau. Tuy nhiên, điều này có thể đơn giản như việc dùng các template phía máy chủ hoặc phía client.

Implementation — Hiện thực

How to splice a widget into your UI will depend largely on how your UI is created. With a simple website, including widgets as HTML fragments using either client-side or server-side templating can be pretty straightforward, although you can incur problems if the widgets have more complex behavior. As an example, if our recommendation widget contains lots of JavaScript functionality, how would we ensure that this doesn't clash with behavior loaded into the rest of the web page? Ideally, the whole widget could be packaged in such a way that it wouldn't break other aspects of the UI.

Cách ghép một widget vào UI của bạn sẽ phụ thuộc phần lớn vào cách UI của bạn được tạo ra. Với một website đơn giản, việc đưa các widget vào dưới dạng các đoạn HTML (HTML fragment) bằng cách dùng template phía client hoặc phía máy chủ có thể khá đơn giản, mặc dù bạn có thể gặp vấn đề nếu các widget có hành vi phức tạp hơn. Ví dụ, nếu widget gợi ý của chúng ta chứa nhiều chức năng JavaScript, làm thế nào để chúng ta đảm bảo rằng nó không xung đột với hành vi được nạp vào phần còn lại của trang web? Lý tưởng nhất, toàn bộ widget có thể được đóng gói theo cách mà nó sẽ không phá vỡ các khía cạnh khác của UI.

The issue of how to deliver self-contained functionality into a UI without breaking other functionality has historically been especially problematic with single-page apps, partly because the concept of modularization doesn't appear to have been a prime concern for how the supporting SPA frameworks were created. These challenges are worth exploring in more depth.

Vấn đề về cách đưa chức năng tự chứa (self-contained) vào một UI mà không phá vỡ chức năng khác trong lịch sử đã đặc biệt rắc rối với các ứng dụng đơn trang, một phần vì khái niệm module hóa dường như đã không phải là mối quan tâm hàng đầu khi các framework SPA hỗ trợ được tạo ra. Những thách thức này đáng được khám phá sâu hơn.

Dependencies

Phụ thuộc (Dependencies)

Although iFrames have been a heavily used technique in the past, we tend to avoid using them to splice different widgets into a single web page. iFrames have a host of challenges regarding sizing and in terms of making it difficult to communicate between the different parts of the frontend. Instead, widgets are normally either spliced into the UI using server-side templating or else dynamically inserted into the browser on the client side. In both cases, the challenge is that the widget is running in the same browser page with other parts of the frontend, meaning that you need to be careful that the different widgets don't conflict with one another.

Mặc dù iFrame đã là một kỹ thuật được dùng nhiều trong quá khứ, chúng ta có xu hướng tránh dùng chúng để ghép các widget khác nhau vào một trang web duy nhất. iFrame có vô số thách thức liên quan đến việc định kích thước và việc khiến cho giao tiếp giữa các phần khác nhau của frontend trở nên khó khăn. Thay vào đó, các widget thường hoặc được ghép vào UI bằng template phía máy chủ hoặc được chèn động vào trình duyệt ở phía client. Trong cả hai trường hợp, thách thức là widget đang chạy trong cùng một trang

trình duyệt với các phần khác của frontend, nghĩa là bạn cần cẩn thận để các widget khác nhau không xung đột với nhau.

For example, our recommendation widget might make use of React v16, whereas the basket widget is still using React v15. This can be a blessing, of course, as it can help us try out different technology (we could use different SPA frameworks for different widgets), but it can also help when it comes to updating the versions of frameworks being used. I've spoken to a number of teams that have had challenges in moving between Angular or React versions, largely due to the differences in conventions used in the newer framework versions. Upgrading an entire monolithic UI could be daunting, but if you can do it incrementally, updating parts of your frontend piece by piece, you can break the work up and also mitigate the risk of the upgrade introducing new problems.

Ví dụ, widget gợi ý của chúng ta có thể dùng React v16, trong khi widget giỏ hàng vẫn đang dùng React v15. Tất nhiên điều này có thể là một điều may, vì nó có thể giúp chúng ta thử nghiệm công nghệ khác nhau (chúng ta có thể dùng các framework SPA khác nhau cho các widget khác nhau), nhưng nó cũng có thể giúp ích khi đến lúc cập nhật các phiên bản framework đang dùng. Tôi đã nói chuyện với một số đội từng gặp thách thức khi chuyển đổi giữa các phiên bản Angular hay React, phần lớn do sự khác biệt về các quy ước được dùng trong những phiên bản framework mới hơn. Việc nâng cấp toàn bộ một UI nguyên khối có thể đáng sợ, nhưng nếu bạn có thể làm điều đó một cách tăng dần, cập nhật từng phần của frontend từng mảnh một, bạn có thể chia nhỏ công việc và cũng giảm thiểu rủi ro việc nâng cấp gây ra vấn đề mới.

The downside is that you can end up with a lot of duplication between dependencies, which in turn can lead to a large bloat in terms of page load size. I could end up including multiple different versions of the React framework and its associated transitive dependencies, for example. It is unsurprising that many websites now have a page load size several times greater than the size of some operating systems. As a quick unscientific study, I checked the page load of the CNN website at the time of writing, and it was 7.9 MB, which is a great deal larger than the 5 MB for Alpine Linux for example. 7.9 MB is actually at the small end of some of the page load sizes I see.

Mặt trái là bạn có thể rất cuộc gặp rất nhiều sự trùng lặp giữa các phụ thuộc, điều này đến lượt nó có thể dẫn đến một sự phình to lớn về kích thước nạp trang (page load size). Ví dụ, tôi có thể rất cuộc đưa vào nhiều phiên bản khác nhau của framework React và các phụ thuộc bắc cầu (transitive dependency) liên quan của nó. Không có gì ngạc nhiên khi nhiều website giờ đây có kích thước nạp trang lớn gấp nhiều lần kích thước của một số hệ điều hành. Như một nghiên cứu nhanh không chính thống, tôi đã kiểm tra phần nạp trang của website CNN tại thời điểm viết bài, và nó là 7,9 MB, lớn hơn rất nhiều so với 5 MB của Alpine Linux chẳng hạn. 7,9 MB thực ra còn nằm ở mức nhỏ trong số những kích thước nạp trang mà tôi thấy.

Communication between in-page widgets

Giao tiếp giữa các widget trong cùng trang

Although our widgets can be built and deployed independently, we still want them to be able to interact with one another. As an example from MusicCorp, when a user selects one of the albums in the best seller chart, we want other parts of the UI to update based on the selection, as shown in Figure 14-7.

Mặc dù các widget của chúng ta có thể được xây dựng và triển khai độc lập, chúng ta vẫn muốn chúng có thể tương tác với nhau. Như một ví dụ từ MusicCorp, khi một người dùng chọn một trong các album trong bảng xếp hạng bán chạy nhất, chúng ta muốn các phần khác của UI cập nhật dựa trên lựa chọn đó, như thể hiện trong Hình 14-7.

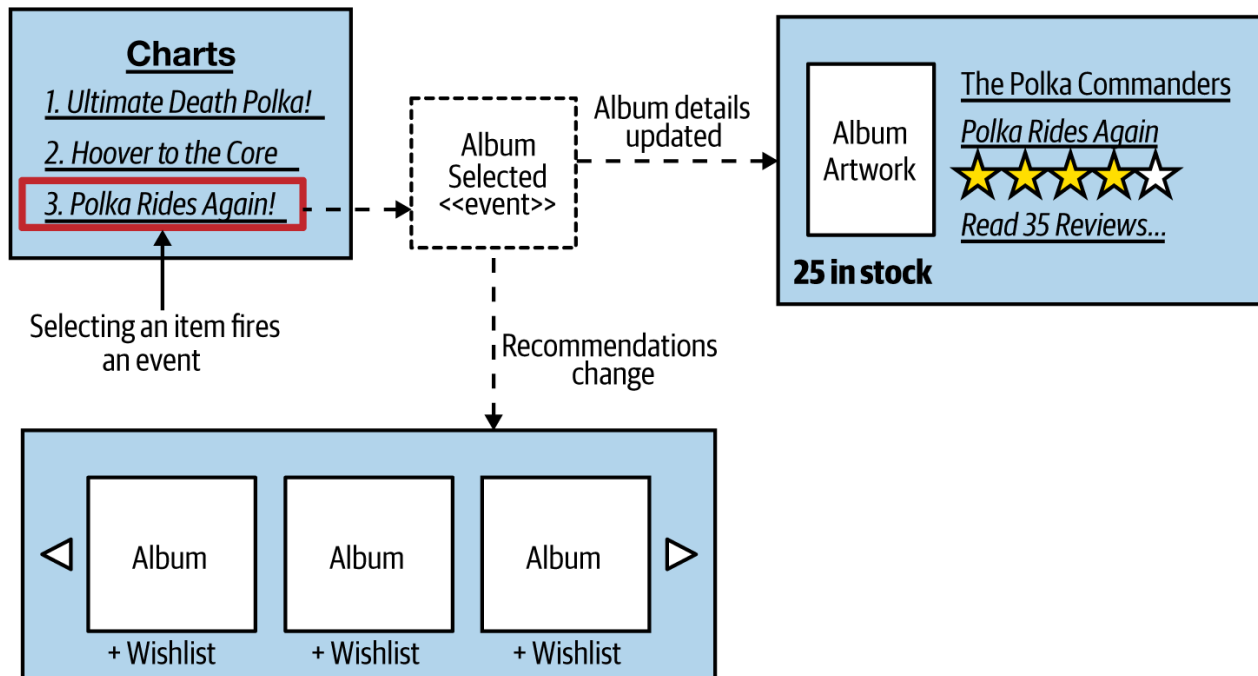


Figure 14-7. The charts widget can emit events that other parts of the UI listen to

Hình 14-7. Widget bảng xếp hạng có thể phát ra các sự kiện mà các phần khác của UI lắng nghe

The way we'd achieve this would be to have the charts widget emit a custom event. Browsers already support a number of standard events that we can use to trigger behavior. These events allow us to react to buttons being pressed, the mouse being scrolled, and the like, and you've probably already made heavy use of such event handling if you've spent any time building JavaScript frontends. It's a simple step to create your own custom events.

Cách chúng ta đạt được điều này là để widget bảng xếp hạng phát ra một sự kiện tùy chỉnh (custom event). Trình duyệt vốn đã hỗ trợ một số sự kiện chuẩn mà chúng ta có thể dùng để kích hoạt hành vi. Những sự kiện này cho phép chúng ta phản ứng với việc nút được nhấn, chuột được cuộn, và những thứ tương tự, và bạn có lẽ đã dùng nhiều cách xử lý sự kiện này nếu bạn từng dành thời gian xây dựng các frontend JavaScript. Tạo sự kiện tùy chỉnh của riêng bạn là một bước đơn giản.

So in our case, when an item is selected in the chart, that widget raises a custom Album Selected event. The recommendation and album details widgets both subscribe to the event and react accordingly, with the recommendations updating based on the selection, and the album details being loaded up. This interaction should of course already be familiar to us, as it mimics the **event-driven** interaction between microservices that we discussed in "Pattern: Event-Driven Communication" on page 108. The only real difference is that these event interactions happen inside the browser.

Vậy trong trường hợp của chúng ta, khi một mục được chọn trong bảng xếp hạng, widget đó phát ra một sự kiện tùy chỉnh Album Selected. Widget gợi ý và widget chi tiết album đều đăng ký nhận sự kiện đó và phản ứng tương ứng, với phân gợi ý cập nhật dựa trên lựa chọn, và phần chi tiết album được nạp lên. Tương tác này tất nhiên đã quen thuộc với chúng ta, vì nó mô phỏng kiểu tương tác hướng sự kiện giữa các microservice mà chúng ta đã thảo luận trong "Pattern: Giao tiếp hướng sự kiện" ở trang 108. Khác biệt thực sự duy nhất là những tương tác sự kiện này diễn ra bên trong trình duyệt.

Web Components — Web Components

At first glance, the Web Component Standard should be an obvious way to implement these widgets. The Web Component Standard describes how you can create a UI component that can sandbox its HTML, CSS, and JavaScript aspects. Unfortunately, the web components standard seems to have taken a long time to settle down, and a longer time to be properly supported by browsers. Much of the initial work around them seems to have stalled, which has obviously impacted adoption. I have yet to meet an organization that uses web components served up from microservices, for example.

Thoạt nhìn, Web Component Standard hẳn là một cách hiển nhiên để hiện thực các widget này. Web Component Standard mô tả cách bạn có thể tạo ra một thành phần UI có thể cô lập (sandbox) các khía cạnh HTML, CSS và JavaScript của nó. Tiếc thay, chuẩn web component dường như đã mất một thời gian dài để ổn định, và một thời gian còn dài hơn để được các trình duyệt hỗ trợ đầy đủ. Phần lớn công việc ban đầu xung quanh chúng dường như đã bị đình trệ, điều này rõ ràng đã tác động đến việc áp dụng. Ví dụ, tôi vẫn chưa gặp một tổ chức nào dùng web component được phục vụ từ microservice.

Given that the Web Component Standard is now fairly well supported, it's possible that we'll see them emerge as a common way to implement sandboxed widgets or larger micro frontends in the future, but after years of waiting for this to come about, I'm not holding my breath.

Với việc Web Component Standard giờ đây đã được hỗ trợ khá tốt, có khả năng chúng ta sẽ thấy chúng nổi lên như một cách phổ biến để hiện thực các widget được cô lập hoặc các micro frontend lớn hơn trong tương lai, nhưng sau nhiều năm chờ đợi điều này thành hiện thực, tôi không nín thở chờ đợi đâu.

When to Use It — Khi nào nên dùng

This pattern makes it easy for multiple stream-aligned teams to contribute to the same UI. It allows for more flexibility than does page-based decomposition, as the widgets delivered by different teams can all coexist in the UI at the same time. It also creates the opportunity for enabling teams to provide reusable widgets that can be used by stream-aligned teams—an example of which I shared earlier when mentioning the role of the Financial Times Origami team.

Pattern này giúp nhiều stream-aligned team dễ dàng cùng đóng góp vào cùng một UI. Nó cho phép linh hoạt hơn so với phân rã theo trang, vì các widget do các đội khác nhau cung cấp đều có thể cùng tồn tại trong UI cùng một lúc. Nó cũng tạo cơ hội cho các enabling team cung cấp những widget tái sử dụng được mà các stream-aligned team có thể dùng—một ví dụ về điều này mà tôi đã chia sẻ trước đó khi đề cập đến vai trò của đội Origami của Financial Times.

The widget decomposition pattern is incredibly useful if you are building a rich web-based user interface, and I would strongly suggest the use of widgets in any situation in which you are making use of a SPA framework and want to break up responsibilities for the frontend, moving toward a micro frontend approach. The techniques and supporting technology around this concept have improved markedly over the last few years, to the extent that, when building an SPA-based web interface, breaking down my UI into micro frontends would be my default approach.

Pattern phân rã theo widget cực kỳ hữu ích nếu bạn đang xây dựng một giao diện người dùng web phong phú, và tôi mạnh mẽ khuyến nghị dùng widget trong bất kỳ tình huống nào mà bạn đang dùng một framework SPA và muốn chia nhỏ trách nhiệm cho frontend, hướng tới một cách tiếp cận micro frontend. Các kỹ thuật và công nghệ hỗ trợ xung quanh

khái niệm này đã cải thiện rõ rệt trong vài năm qua, đến mức, khi xây dựng một giao diện web dựa trên SPA, việc chia nhỏ UI của tôi thành các micro frontend sẽ là cách tiếp cận mặc định của tôi.

My main concerns around widget decomposition in the context of SPAs have to do with the work required to set up the separate bundling of components and the issues around payload size. The former issue is likely a one-time cost and just involves working out what style of packaging best suits your existing toolchain. The latter issue is more problematic. A simple small change in the dependencies of one widget could result in a whole host of new dependencies being included in the application, drastically inflating the page size. If you are building a user interface where page weight is a concern, I'd suggest putting some automated checks in place to alert you if the page weight goes above a certain acceptable threshold.

Mối quan tâm chính của tôi về phân rã theo widget trong bối cảnh SPA liên quan đến công việc cần thiết để thiết lập việc đóng gói (bundling) riêng biệt các thành phần và những vấn đề xung quanh kích thước payload. Vấn đề đầu tiên có lẽ là một chi phí một lần và chỉ liên quan đến việc tìm ra kiểu đóng gói nào phù hợp nhất với chuỗi công cụ (toolchain) hiện có của bạn. Vấn đề thứ hai rắc rối hơn. Một thay đổi nhỏ đơn giản trong các phụ thuộc của một widget có thể dẫn đến một loạt các phụ thuộc mới được đưa vào ứng dụng, làm phình kích thước trang một cách đáng kể. Nếu bạn đang xây dựng một giao diện người dùng mà trọng lượng trang (page weight) là một mối quan tâm, tôi khuyên bạn nên đặt một số kiểm tra tự động để cảnh báo bạn nếu trọng lượng trang vượt quá một ngưỡng chấp nhận được nhất định.

On the other hand, if the widgets are simpler in nature and are largely static components, the ability to include them using something as simple as client-side or server-side templating is very straightforward by comparison.

Mặt khác, nếu các widget có bản chất đơn giản hơn và phần lớn là các thành phần tĩnh, thì khả năng đưa chúng vào bằng một thứ đơn giản như template phía client hoặc phía máy chủ là rất đơn giản khi so sánh.

Constraints — Các ràng buộc (Constraints)

Before moving on to discuss our next pattern, I want to address the topic of constraints. Increasingly, the users of our software interact with it from a variety of different devices. Each of these devices imposes different constraints that our software has to accommodate. On a desktop web application, for example, we consider constraints such as what browser visitors are using, or their screen resolution. People with impaired sight may make use of our software via screen readers, and people with restricted mobility may be more likely to use keyboard-style inputs to navigate the screen.

Trước khi chuyển sang thảo luận pattern tiếp theo của chúng ta, tôi muốn đề cập đến chủ đề ràng buộc (constraint). Ngày càng nhiều, người dùng phần mềm của chúng ta tương tác với nó từ vô số thiết bị khác nhau. Mỗi thiết bị này áp đặt những ràng buộc khác nhau mà phần mềm của chúng ta phải đáp ứng. Trên một ứng dụng web desktop chẳng hạn, chúng ta xét các ràng buộc như khách truy cập đang dùng trình duyệt nào, hoặc độ phân giải màn hình của họ. Người khiếm thị có thể dùng phần mềm của chúng ta thông qua trình đọc màn hình (screen reader), và người hạn chế khả năng vận động có thể có nhiều khả năng dùng các kiểu nhập liệu bằng bàn phím để điều hướng màn hình.

So although our core services—our core offering—might be the same, we need a way to adapt them for the different constraints that exist for each type of interface, and for the different needs of our

users. This can be driven purely from a financial standpoint if you want—more happy customers means more money. But there is also a human, ethical consideration that comes to the fore: when we ignore customers that have specific needs, we deny them the chance to use our services. In some contexts, making it impossible for people to navigate a UI due to design decisions has led to legal action and fines—for example, the UK, with a number of other countries, rightly has legislation in place to ensure access to websites for people with disabilities. 5

Vậy nên mặc dù các dịch vụ cốt lõi của chúng ta—thứ chúng ta cung cấp ở mức cốt lõi—có thể giống nhau, chúng ta cần một cách để điều chỉnh chúng cho những ràng buộc khác nhau tồn tại đối với từng loại giao diện, và cho những nhu cầu khác nhau của người dùng. Điều này có thể được thúc đẩy thuần túy từ góc độ tài chính nếu bạn muốn—nhiều khách hàng hài lòng hơn nghĩa là nhiều tiền hơn. Nhưng cũng có một cân nhắc nhân văn, đạo đức nổi lên: khi chúng ta phớt lờ những khách hàng có nhu cầu đặc biệt, chúng ta tước đi của họ cơ hội sử dụng các dịch vụ của chúng ta. Trong một số bối cảnh, việc khiến người ta không thể điều hướng một UI do các quyết định thiết kế đã dẫn đến kiện tụng và phạt tiền—ví dụ, Vương quốc Anh, cùng với một số quốc gia khác, một cách đúng đắn đã có luật pháp để đảm bảo quyền truy cập website cho người khuyết tật. 5

Mobile devices have brought a whole host of new constraints. The way our mobile applications communicate with a server can have an impact. It isn't just about pure bandwidth concerns, where the limitations of mobile networks can play a part. Different sorts of interactions can drain battery life, leading to some angry customers.

Các thiết bị di động đã mang đến cả một loạt ràng buộc mới. Cách các ứng dụng di động của chúng ta giao tiếp với máy chủ có thể có tác động. Không chỉ đơn thuần là các mối quan tâm về băng thông, nơi các giới hạn của mạng di động có thể đóng một vai trò. Các kiểu tương tác khác nhau có thể làm hao pin, dẫn đến một số khách hàng tức giận.

The nature of interactions changes according to the device, too. I can't easily right-click on a tablet. On a mobile phone, I may want to design my interface for mostly one-handed use, with most operations being controlled by a thumb. Elsewhere, I might allow people to interact with services via SMS in places where bandwidth is at a premium—the use of SMS as an interface is huge in the global south, for example.

Bản chất của các tương tác cũng thay đổi tùy theo thiết bị. Tôi không thể dễ dàng nhấp chuột phải trên một máy tính bảng. Trên một điện thoại di động, tôi có thể muốn thiết kế giao diện của mình cho việc dùng chủ yếu bằng một tay, với hầu hết các thao tác được điều khiển bằng ngón cái. Ở nơi khác, tôi có thể cho phép người ta tương tác với các dịch vụ qua SMS ở những nơi mà băng thông là thứ quý hiếm—ví dụ, việc dùng SMS như một giao diện là rất lớn ở các nước phương Nam toàn cầu (global south).

A wider discussion on the accessibility of user interfaces is outside the scope of this book, but we can at least explore the specific challenges caused by different types of clients, such as mobile devices. A common solution to handling the different needs of client devices is to perform some sort of filtering and call aggregation on the client

Một cuộc thảo luận rộng hơn về khả năng tiếp cận của các giao diện người dùng nằm ngoài phạm vi của cuốn sách này, nhưng ít nhất chúng ta cũng có thể khám phá những thách thức cụ thể do các loại client khác nhau gây ra, chẳng hạn như thiết bị di động. Một giải pháp phổ biến để xử lý các nhu cầu khác nhau của thiết bị client là thực hiện một dạng lọc và gộp lời gọi (call aggregation) nào đó ở phía client

5 I hope it goes without saying that I am not a lawyer, but should you wish to look at the legislation that covers this in the UK, it's the Equality Act 2010, specifically section 20. The W3C also has a good

overview of accessibility guidelines .

5 Tôi hy vọng không cần phải nói rằng tôi không phải là luật sư, nhưng nếu bạn muốn xem luật pháp điều chỉnh vấn đề này ở Anh, đó là Equality Act 2010, cụ thể là mục 20. W3C cũng có một bản tổng quan tốt về các hướng dẫn khả năng tiếp cận.

side. Data that isn't required can be stripped and doesn't need to be sent to the device, and multiple calls can be combined into a single call.

. Dữ liệu không cần thiết có thể được loại bỏ và không cần được gửi tới thiết bị, và nhiều lời gọi có thể được kết hợp lại thành một lời gọi duy nhất.

Next, we'll look at two patterns that can be useful in this space—the central aggregating gateway and the backend for frontend pattern. We'll also look at how GraphQL is being used to help tailor responses for different types of interfaces.

Tiếp theo, chúng ta sẽ xem xét hai pattern có thể hữu ích trong không gian này—gateway gộp trung tâm (central aggregating gateway) và pattern Backend for Frontend. Chúng ta cũng sẽ xem xét cách GraphQL đang được dùng để giúp điều chỉnh phản hồi cho các loại giao diện khác nhau.

Pattern: Central Aggregating Gateway — Pattern: Gateway gộp trung tâm (Central Aggregating Gateway)

A central-purpose aggregating gateway sits between external user interfaces and downstream microservices and performs call filtering and aggregation for all user interfaces. Without aggregation, a user interface may have to make multiple calls to fetch required information, often throwing away data that was retrieved but not needed.

Một gateway gộp đa-mục-đích trung tâm nằm giữa các giao diện người dùng bên ngoài và các microservice phía sau, và thực hiện việc lọc và gộp lời gọi cho tất cả các giao diện người dùng. Không có việc gộp, một giao diện người dùng có thể phải thực hiện nhiều lời gọi để lấy thông tin cần thiết, thường vứt bỏ dữ liệu đã được lấy về nhưng không cần đến.

In Figure 14-8 , we see such a situation. We want to display a screen with information about a customer's recent orders. The screen needs to display some general information about a customer and then list a number of their orders in date order, along with summary information, showing the date and status of each order as well as the price.

Trong Hình 14-8, chúng ta thấy một tình huống như vậy. Chúng ta muốn hiển thị một màn hình với thông tin về các đơn hàng gần đây của một khách hàng. Màn hình cần hiển thị một số thông tin chung về một khách hàng rồi liệt kê một số đơn hàng của họ theo thứ tự ngày, kèm theo thông tin tóm tắt, hiển thị ngày và trạng thái của mỗi đơn hàng cũng như giá.

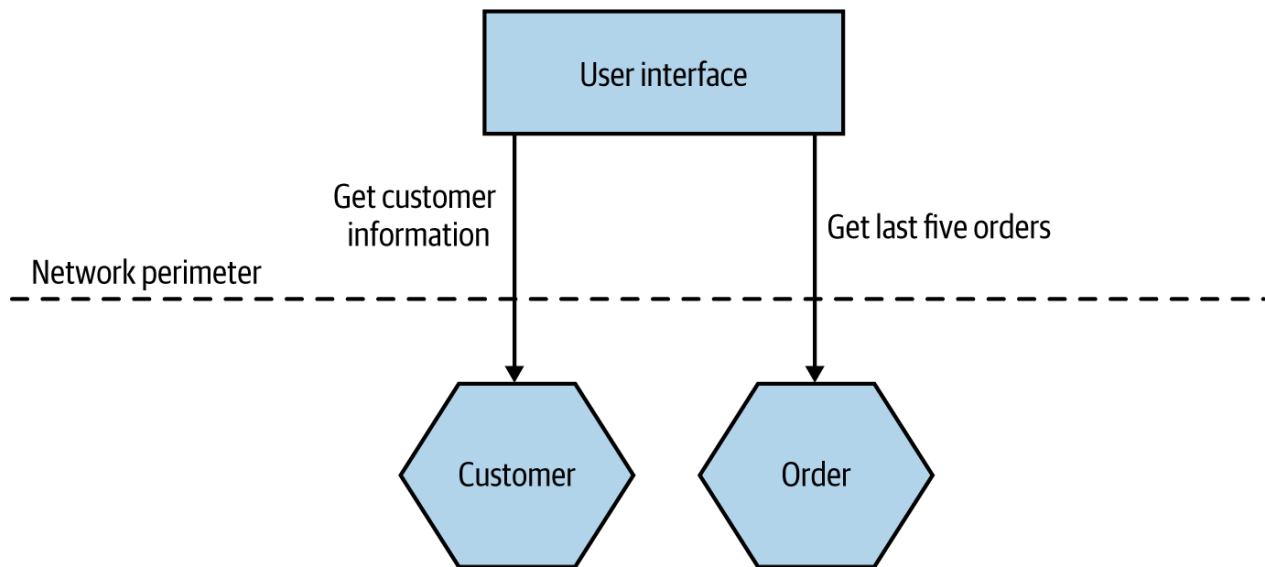


Figure 14-8. Making multiple calls to fetch information for a single screen

Hình 14-8. Thực hiện nhiều lời gọi để lấy thông tin cho một màn hình duy nhất

We make a direct call to the Customer microservice, pulling back the full information about the customer, even though we need only a few fields. We then fetch the order details from the Order microservice. We could improve the situation somewhat, perhaps by changing the Customer or Order microservice to return data that better fits our requirements in this specific case, but that would still require two calls to be made.

Chúng ta thực hiện một lời gọi trực tiếp đến microservice Customer, kéo về toàn bộ thông tin về khách hàng, mặc dù chúng ta chỉ cần một vài trường. Sau đó chúng ta lấy chi tiết đơn hàng từ microservice Order. Chúng ta có thể cải thiện tình hình phần nào, có lẽ bằng cách thay đổi microservice Customer hoặc Order để trả về dữ liệu khớp hơn với yêu cầu của chúng ta trong trường hợp cụ thể này, nhưng điều đó vẫn sẽ đòi hỏi phải thực hiện hai lời gọi.

With an aggregating gateway, we can instead issue a single call from the user interface to the gateway. The aggregating gateway then carries out all the required calls, combines the results into a single response, and throws away any data that the user interface doesn't require (Figure 14-9).

Với một gateway gộp, thay vào đó chúng ta có thể phát một lời gọi duy nhất từ giao diện người dùng tới gateway. Gateway gộp sau đó thực hiện tất cả các lời gọi cần thiết, kết hợp các kết quả thành một phản hồi duy nhất, và vứt bỏ bất kỳ dữ liệu nào mà giao diện người dùng không cần (Hình 14-9).

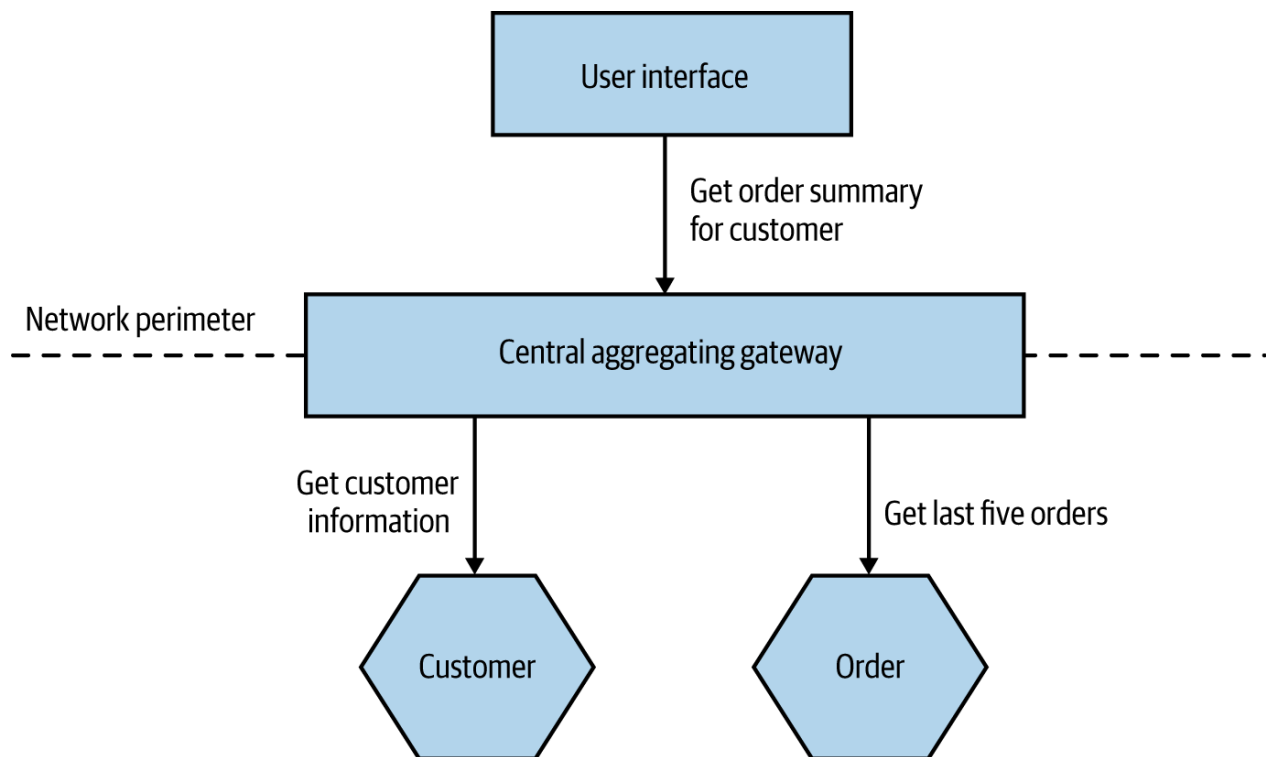


Figure 14-9. The server-side central gateway handles filtering and aggregation of calls to downstream microservices

Hình 14-9. Gateway trung tâm phía máy chủ xử lý việc lọc và gộp các lời gọi tới các microservice phía sau

Such a gateway could also help batch calls as well. For example, rather than needing to look up 10 order IDs via separate calls, I could send one batch request to the aggregating gateway, and it could handle the rest.

Một gateway như vậy cũng có thể giúp gom lời gọi theo lô (batch). Ví dụ, thay vì cần tra cứu 10 ID đơn hàng qua các lời gọi riêng biệt, tôi có thể gửi một yêu cầu theo lô tới gateway gộp, và nó có thể xử lý phần còn lại.

Fundamentally, having some sort of aggregating gateway can reduce the number of calls that the external client needs to make and reduce the amount of data that needs to be sent back. This can lead to significant benefits in terms of reducing bandwidth use and improving the latency of the application.

Về cơ bản, việc có một dạng gateway gộp nào đó có thể giảm số lời gọi mà client bên ngoài cần thực hiện và giảm lượng dữ liệu cần gửi trả về. Điều này có thể mang lại lợi ích đáng kể về việc giảm sử dụng băng thông và cải thiện độ trễ (latency) của ứng dụng.

Ownership — Quyền sở hữu

As more user interfaces make use of the central gateway, and as more microservices need call aggregation and filtering logic for those user interfaces, the gateway becomes a potential source of contention. Who owns the gateway? Is it owned by the people creating the user interfaces, or by the people who own the microservices? Often I find that the central aggregating gateway does so much that it ends up being owned by a dedicated team. Hello, siloed layered architecture!

Khi ngày càng nhiều giao diện người dùng dùng gateway trung tâm, và khi ngày càng nhiều microservice cần logic gộp và lọc lời gọi cho các giao diện người dùng đó, gateway trở thành một nguồn tranh chấp tiềm tàng. Ai sở hữu gateway? Nó được sở hữu bởi những người tạo ra giao diện người dùng, hay bởi những người sở hữu các microservice? Tôi thường thấy rằng gateway gộp trung tâm làm quá nhiều việc đến mức nó rốt cuộc được sở hữu bởi một đội chuyên trách. Xin chào, kiến trúc phân tầng bị chia silo!

Fundamentally, the nature of the call aggregation and filtering is largely driven by the requirements of the external user interfaces. As such, it would make natural sense for the gateway to be owned by the team(s) creating the UI. Unfortunately, especially in an organization in which you have a dedicated frontend team, that team may not have the skills to build such a vital backend component.

Về cơ bản, bản chất của việc gộp và lọc lời gọi phần lớn được dẫn dắt bởi các yêu cầu của những giao diện người dùng bên ngoài. Như vậy, sẽ là điều tự nhiên hợp lý nếu gateway được sở hữu bởi (các) đội tạo ra UI. Tiếc thay, đặc biệt trong một tổ chức nơi bạn có một đội frontend chuyên trách, đội đó có thể không có kỹ năng để xây dựng một thành phần backend trọng yếu như vậy.

Regardless of who ends up owning the central gateway, it has the potential to become a bottleneck for delivery. If multiple teams need to make changes to the gateway, development on it will require coordination between those teams, slowing things down. If one team owns it, that team can become a bottleneck when it comes to delivery. We'll see how the backend for frontend pattern can help resolve these issues shortly.

Bất kể ai rốt cuộc sở hữu gateway trung tâm, nó có khả năng trở thành một nút thắt cổ chai (bottleneck) cho việc chuyển giao. Nếu nhiều đội cần thực hiện thay đổi đối với gateway, việc phát triển trên nó sẽ đòi hỏi sự phối hợp giữa các đội đó, làm mọi thứ chậm lại. Nếu một đội sở hữu nó, đội đó có thể trở thành một nút thắt cổ chai khi đến việc chuyển giao. Chúng ta sẽ sớm thấy pattern Backend for Frontend có thể giúp giải quyết những vấn đề này ra sao.

Different Types of User Interfaces — Các loại giao diện người dùng khác nhau

If the challenges around ownership can be managed, a central aggregating gateway might still work well, until we consider the issue of different devices and their different needs. As we've discussed, the affordances of a mobile device are very different. We have less screen real estate, which means we can display less data. Opening lots of connections to server-side resources can drain battery life and limited data plans. Additionally, the nature of the interactions we want to provide on a mobile device can differ drastically. Think of a typical bricks-and-mortar retailer. On a desktop app, I might allow you to look at the items for sale and order them online or reserve them in a store. On the mobile device, though, I might want to allow you to scan bar codes to do price comparisons or give you context-based offers while in store. As we've built more and more mobile applications, we've come to realize that people use them very differently, and therefore the functionality we need to expose will differ too.

Nếu các thách thức về quyền sở hữu có thể được quản lý, một gateway gộp trung tâm vẫn có thể hoạt động tốt, cho đến khi chúng ta xét đến vấn đề các thiết bị khác nhau và những nhu cầu khác nhau của chúng. Như chúng ta đã thảo luận, các khả năng (affordance) của một thiết bị di động là rất khác. Chúng ta có ít diện tích màn hình hơn, nghĩa là chúng ta có thể hiển thị ít dữ liệu hơn. Việc mở nhiều kết nối tới các tài nguyên phía máy chủ có

thể làm hao pin và các gói cước dữ liệu hạn chế. Ngoài ra, bản chất của các tương tác mà chúng ta muốn cung cấp trên một thiết bị di động có thể khác nhau rất nhiều. Hãy nghĩ về một nhà bán lẻ truyền thống điển hình. Trên một ứng dụng desktop, tôi có thể cho phép bạn xem các mặt hàng đang bán và đặt mua chúng trực tuyến hoặc giữ chúng tại một cửa hàng. Tuy nhiên, trên thiết bị di động, tôi có thể muốn cho phép bạn quét mã vạch để so sánh giá hoặc đưa cho bạn những ưu đãi theo ngữ cảnh khi bạn đang ở trong cửa hàng. Khi chúng ta xây dựng ngày càng nhiều ứng dụng di động, chúng ta dần nhận ra rằng người ta dùng chúng theo những cách rất khác nhau, và do đó chức năng mà chúng ta cần phơi bày cũng sẽ khác.

So in practice, our mobile devices will want to make different and fewer calls and will want to display different (and probably less) data than their desktop counterparts. This means that we need to add additional functionality to our API backend to support different types of user interfaces. In Figure 14-10, we see MusicCorp's web interface and mobile interface both using the same gateway for the customer summary screen, but each client wants a different set of information. The web interface wants more information about the customer and also has a brief summary of the items in each order. This leads us to implement two different aggregating and filtering calls in our backend gateway.

Vậy trên thực tế, các thiết bị di động của chúng ta sẽ muốn thực hiện những lời gọi khác và ít hơn, và sẽ muốn hiển thị dữ liệu khác (và có lẽ là ít hơn) so với các đối tác desktop của chúng. Điều này nghĩa là chúng ta cần thêm chức năng bổ sung vào API backend của mình để hỗ trợ các loại giao diện người dùng khác nhau. Trong Hình 14-10, chúng ta thấy giao diện web và giao diện di động của MusicCorp đều dùng cùng một gateway cho màn hình tóm tắt khách hàng, nhưng mỗi client lại muốn một tập thông tin khác nhau. Giao diện web muốn nhiều thông tin hơn về khách hàng và cũng có một bản tóm tắt ngắn gọn về các mặt hàng trong mỗi đơn hàng. Điều này dẫn chúng ta đến việc hiện thực hai lời gọi gộp và lọc khác nhau trong gateway backend của mình.

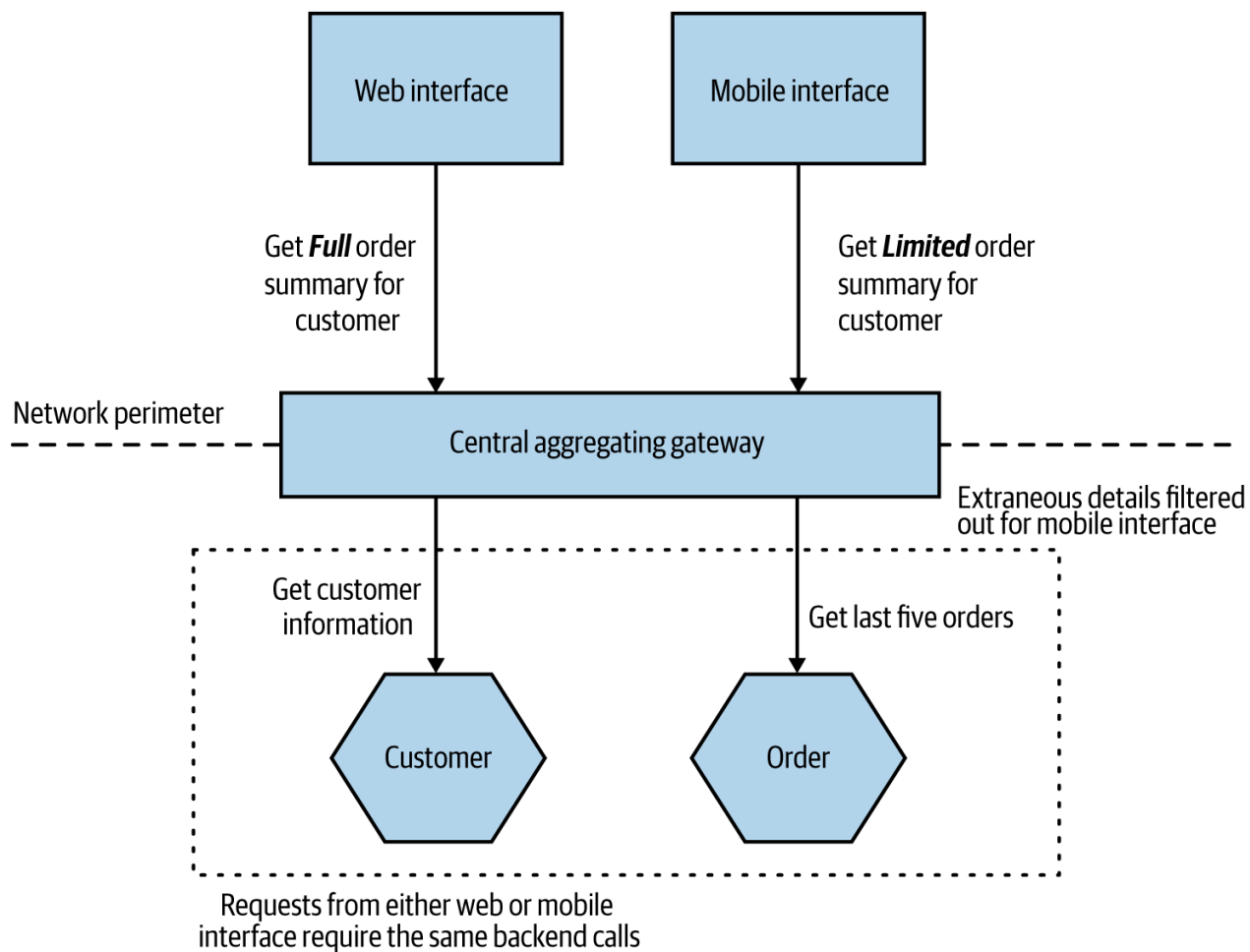


Figure 14-10. Supporting different aggregating calls for different devices

Hình 14-10. Hỗ trợ các lời gọi gộp khác nhau cho các thiết bị khác nhau

This can lead to a lot of bloat in the gateway, especially if we consider different native mobile applications, customer-facing websites, internal administration interfaces, and the like. We also have the problem, of course, that while these different UIs might be owned by different teams, the gateway is a single unit—we have the same old problems of multiple teams having to work on the same deployed unit. Our single aggregating backend can become a bottleneck, as so many changes are trying to be made to the same deployable **artifact**.

Điều này có thể dẫn đến rất nhiều sự phình to trong gateway, đặc biệt nếu chúng ta xét đến các ứng dụng di động native khác nhau, các website hướng tới khách hàng, các giao diện quản trị nội bộ, và những thứ tương tự. Tất nhiên chúng ta cũng có vấn đề là trong khi các UI khác nhau này có thể được sở hữu bởi các đội khác nhau, thì gateway lại là một đơn vị duy nhất—chúng ta lại gặp những vấn đề cũ rích là nhiều đội phải làm việc trên cùng một đơn vị đã triển khai. Backend gộp duy nhất của chúng ta có thể trở thành một nút thắt cổ chai, khi có quá nhiều thay đổi cố được thực hiện trên cùng một artifact triển khai.

Multiple Concerns — Nhiều mối quan tâm (Multiple Concerns)

There are a whole host of concerns that may need to be addressed on the server side when it comes to handling API calls. Aside from the call aggregation and filtering, we can think of more generic concerns like API key management, user **authentication**, or call routing. Often these generic concerns can be handled by **API gateway** products, which are available in many sizes and for many different price points (some of which are eye-wateringly high!). Depending on the sophistication you require, it can make a lot of sense to purchase a product (or license a service) to handle some of these concerns for you. Do you really want to manage API key issuing, tracking, rate limiting, and so on yourself? By all means, look at products in this space to solve these generic concerns, but be wary about also trying to use these products to do your call aggregation and filtering, even if they claim they can.

Có cả một loạt mối quan tâm có thể cần được giải quyết ở phía máy chủ khi đến việc xử lý các lời gọi API. Bên cạnh việc gộp và lọc lời gọi, chúng ta có thể nghĩ đến những mối quan tâm tổng quát hơn như quản lý khóa API (API key), xác thực người dùng, hay định tuyến lời gọi. Thường thì những mối quan tâm tổng quát này có thể được xử lý bởi các sản phẩm API gateway, vốn có sẵn ở nhiều quy mô và nhiều mức giá khác nhau (một số trong đó cao đến mức choáng váng!). Tùy vào độ tinh vi mà bạn cần, có thể rất hợp lý khi mua một sản phẩm (hoặc cấp phép một dịch vụ) để xử lý giúp bạn một số mối quan tâm này. Bạn có thực sự muốn tự mình quản lý việc cấp phát, theo dõi, giới hạn tốc độ (rate limiting) khóa API, v.v. không? Cứ tự nhiên, hãy xem xét các sản phẩm trong không gian này để giải quyết những mối quan tâm tổng quát này, nhưng hãy thận trọng về việc củng cố dùng những sản phẩm này để làm việc gộp và lọc lời gọi của bạn, kể cả khi chúng tuyên bố là chúng có thể.

When customizing a product built by someone else, you often have to work in their world. Your toolchain is restricted because you may not be able to use your programming language and your development practices. Rather than writing Java code, you're configuring routing rules in some odd product-specific DSL (probably using JSON). It can be a frustrating experience, and you are baking some of the smarts of your system into a third-party product. This can reduce your ability to move this behavior later. It's common to realize that a pattern of call aggregation actually relates to some domain functionality that could justify a microservice in its own right (something we'll explore more shortly when we talk about BFFs). If this behavior is in a vendor-specific configuration, moving this functionality can be more problematic, as you'd likely have to reinvent it.

Khi tùy chỉnh một sản phẩm do người khác xây dựng, bạn thường phải làm việc trong thế giới của họ. Chuỗi công cụ của bạn bị hạn chế vì bạn có thể không dùng được ngôn ngữ lập trình và các thực hành phát triển của mình. Thay vì viết mã Java, bạn lại đang cấu hình các quy tắc định tuyến trong một DSL kỳ quặc đặc thù của sản phẩm (có lẽ là dùng JSON). Đó có thể là một trải nghiệm bức bối, và bạn đang nung nấu một phần trí tuệ của hệ thống mình vào trong một sản phẩm bên thứ ba. Điều này có thể làm giảm khả năng của bạn trong việc dịch chuyển hành vi này về sau. Người ta thường nhận ra rằng một pattern gộp lời gọi thực ra liên quan đến một chức năng nghiệp vụ nào đó của miền (domain) đáng để biến mình cho một microservice riêng của nó (một điều chúng ta sẽ khám phá thêm sớm thôi khi nói về các BFF). Nếu hành vi này nằm trong một cấu hình đặc thù của nhà cung cấp, việc dịch chuyển chức năng này có thể rắc rối hơn, vì bạn nhiều khả năng sẽ phải phát minh lại nó.

The situation can become even worse if the aggregating gateway becomes complex enough to require a dedicated team to own and manage it. At worst, adopting more horizontal team ownership can lead to a situation in which to roll out some new functionality you have to get a frontend team to make

changes, the aggregating gateway team to make changes, and the team(s) that owns the microservice to also make its changes. Suddenly everything starts going much more slowly.

Tình hình có thể còn tệ hơn nếu gateway gộp trở nên đủ phức tạp đến mức đòi hỏi một đội chuyên trách để sở hữu và quản lý nó. Tệ nhất, việc áp dụng nhiều quyền sở hữu đội theo chiều ngang hơn có thể dẫn đến một tình huống trong đó để triển khai một chức năng mới nào đó, bạn phải khiến một đội frontend thực hiện thay đổi, đội gateway gộp thực hiện thay đổi, và (các) đội sở hữu microservice cũng phải thực hiện các thay đổi của họ. Đột nhiên mọi thứ bắt đầu chậm đi rất nhiều.

So if you want to make use of a dedicated API gateway, go ahead—but strongly consider having your filtering and aggregation logic elsewhere.

Vậy nếu bạn muốn dùng một API gateway chuyên trách, cứ làm—nhưng hãy nghiêm túc cân nhắc việc đặt logic lọc và gộp của bạn ở nơi khác.

When to Use It — Khi nào nên dùng

For a solution owned by a single team, where one team develops the user interface and the backend microservices, I would be OK with having a single central aggregating gateway. That said, this team sounds like it is doing a LOT of work—in such situations, I tend to see a large degree of conformity across the user interfaces, which often removes the need for these aggregation points in the first place.

Đối với một giải pháp được sở hữu bởi một đội duy nhất, nơi một đội phát triển cả giao diện người dùng lẫn các microservice backend, tôi sẽ chấp nhận việc có một gateway gộp trung tâm duy nhất. Dù vậy, đội này nghe có vẻ đang làm RẤT nhiều việc—trong những tình huống như vậy, tôi có xu hướng thấy một mức độ đồng nhất lớn xuyên suốt các giao diện người dùng, điều mà thường loại bỏ ngay từ đầu nhu cầu phải có những điểm gộp này.

If you do decide to adopt a single central aggregating gateway, please be careful to limit what functionality you put inside it. I'd be extremely wary of pushing this functionality into a more generic API gateway product, for example, for reasons outlined previously.

Nếu bạn quyết định áp dụng một gateway gộp trung tâm duy nhất, hãy cẩn thận giới hạn chức năng mà bạn đặt vào trong nó. Ví dụ, tôi sẽ cực kỳ thận trọng về việc đẩy chức năng này vào một sản phẩm API gateway tổng quát hơn, vì những lý do đã nêu trước đó.

The concept of doing some form of call filtering and aggregation on the backend can be really important, though, in terms of optimizing the user's experience of our user interfaces. The issue is that in a delivery organization with multiple teams, a central gateway can lead to requirements for lots of coordination among those teams.

Tuy nhiên, khái niệm thực hiện một dạng lọc và gộp lời gọi nào đó ở backend có thể thực sự quan trọng, xét về việc tối ưu trải nghiệm của người dùng đối với các giao diện người dùng của chúng ta. Vấn đề là trong một tổ chức chuyển giao có nhiều đội, một gateway trung tâm có thể dẫn đến những yêu cầu về rất nhiều sự phối hợp giữa các đội đó.

So if we still want to do aggregation and filtering on the backend but want to remove the problems associated with the ownership model of a central gateway, what can we do? That's where the backend for frontend pattern comes in.

Vậy nếu chúng ta vẫn muốn làm việc gộp và lọc ở backend nhưng muốn loại bỏ những vấn đề gắn với mô hình quyền sở hữu của một gateway trung tâm, chúng ta có thể làm gì?

Đó là nơi pattern Backend for Frontend xuất hiện.

Pattern: Backend for Frontend (BFF) — Pattern: Backend for Frontend (BFF)

The main distinction between a BFF and a central aggregating gateway is that a BFF is single purpose in nature—it is developed for a specific user interface. This pattern has proved to be very successful in helping handle the differing concerns of user interfaces, and I've seen it work well at a number of organizations, including Sound- Cloud 6 and REA. As we see in Figure 14-11 , where we revisit MusicCorp, the web and mobile shopping interfaces now have their own aggregating backends.

Sự khác biệt chính giữa một BFF và một gateway tập trung là một BFF có bản chất đơn-mục-đích—nó được phát triển cho một giao diện người dùng cụ thể. Pattern này đã chứng tỏ rất thành công trong việc giúp xử lý những mối quan tâm khác nhau của các giao diện người dùng, và tôi đã thấy nó hoạt động tốt ở một số tổ chức, bao gồm SoundCloud 6 và REA. Như chúng ta thấy trong Hình 14-11, nơi chúng ta xem lại MusicCorp, các giao diện mua sắm web và di động giờ đây có những backend riêng của mình.

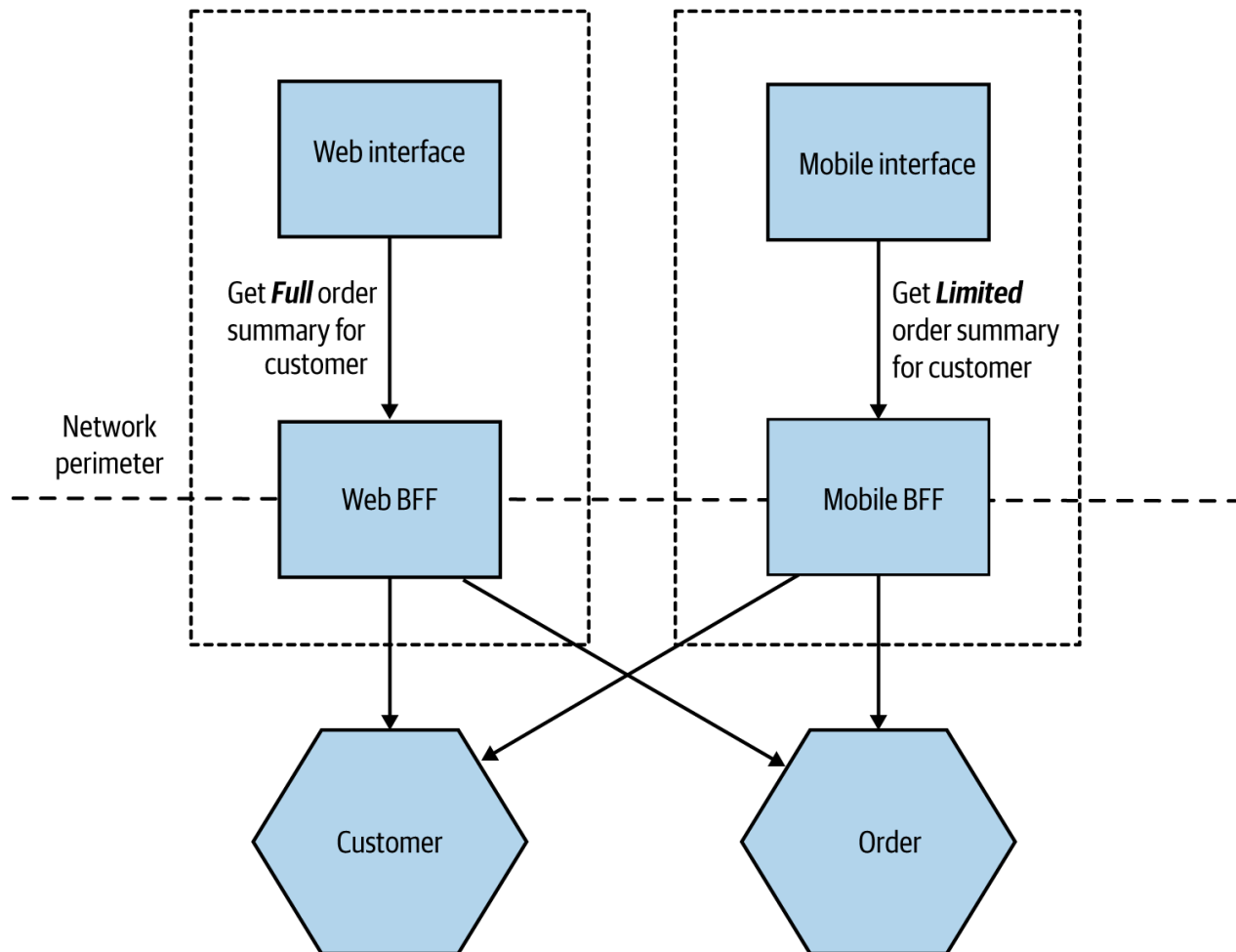


Figure 14-11. Each user interface has its own BFF

Hình 14-11. Mỗi giao diện người dùng có BFF riêng của nó

Due to its specific nature, the BFF sidesteps some of the concerns around the central aggregating gateway. As we aren't trying to be all things to all people, a BFF avoids becoming a bottleneck for development, with multiple teams all trying to share ownership. We are also less worried about **coupling** with the user interface, as coupling is much more acceptable. A given BFF is for a specific user interface—assuming they are owned by the same team, then the inherent coupling is far easier to manage. I

Do bản chất cụ thể của nó, BFF né tránh được một số mối quan tâm xung quanh gateway gộp trung tâm. Vì chúng ta không cố gắng làm mọi thứ cho mọi người, một BFF tránh trở thành một nút thắt cổ chai cho phát triển, với nhiều đội tất cả đều cố chia sẻ quyền sở hữu. Chúng ta cũng ít lo lắng hơn về việc coupling với giao diện người dùng, vì coupling là điều dễ chấp nhận hơn nhiều. Một BFF nhất định là cho một giao diện người dùng cụ thể—giả định chúng được sở hữu bởi cùng một đội, thì coupling vốn có sẽ dễ quản lý hơn nhiều. Tôi

6 See the article “BFF @ SoundCloud” by Lukasz Plotnicki for a great overview of how SoundCloud uses the BFF pattern.

6 Xem bài viết “BFF @ SoundCloud” của Lukasz Plotnicki để có một tổng quan tuyệt vời về cách SoundCloud dùng pattern BFF.

often describe the use of a BFF with a user interface as though the UI is actually split into two parts. One part sits on the client device (the web interface or native mobile application), with the second part, the BFF, being embedded on the server side.

thường mô tả việc dùng một BFF với một giao diện người dùng như thể UI thực ra được chia thành hai phần. Một phần nằm trên thiết bị client (giao diện web hoặc ứng dụng di động native), với phần thứ hai, BFF, được nhúng ở phía máy chủ.

The BFF is tightly coupled to a specific user experience and will typically be maintained by the same team as the user interface, thereby making it easier to define and adapt the API as the UI requires, while also simplifying the process of lining up **release** of both the client and server components.

BFF được coupling chặt với một trải nghiệm người dùng cụ thể và thường sẽ được bảo trì bởi cùng đội với giao diện người dùng, qua đó giúp dễ dàng hơn trong việc định nghĩa và điều chỉnh API theo yêu cầu của UI, đồng thời cũng đơn giản hóa quá trình sắp xếp việc phát hành cả thành phần client lẫn máy chủ trùng khớp với nhau.

How Many BFFs? — Bao nhiêu BFF?

When it comes to delivering the same (or similar) user experience on different platforms, I have seen two different approaches. The model I prefer (and the model I see most often) is to strictly have a single BFF for each different type of client—this is a model I saw used at REA, as outlined in Figure 14-12. The Android and iOS applications, while covering similar functionality, each had their own BFF.

Khi đến việc cung cấp cùng một (hoặc tương tự) trải nghiệm người dùng trên các nền tảng khác nhau, tôi đã thấy hai cách tiếp cận khác nhau. Mô hình tôi ưa thích (và là mô hình tôi thấy thường xuyên nhất) là tuân thủ nghiêm ngặt việc có một BFF duy nhất cho mỗi loại client khác nhau—đây là mô hình tôi đã thấy được dùng tại REA, như được phác họa trong Hình 14-12. Các ứng dụng Android và iOS, dù bao phủ chức năng tương tự nhau, mỗi ứng dụng đều có BFF riêng của nó.

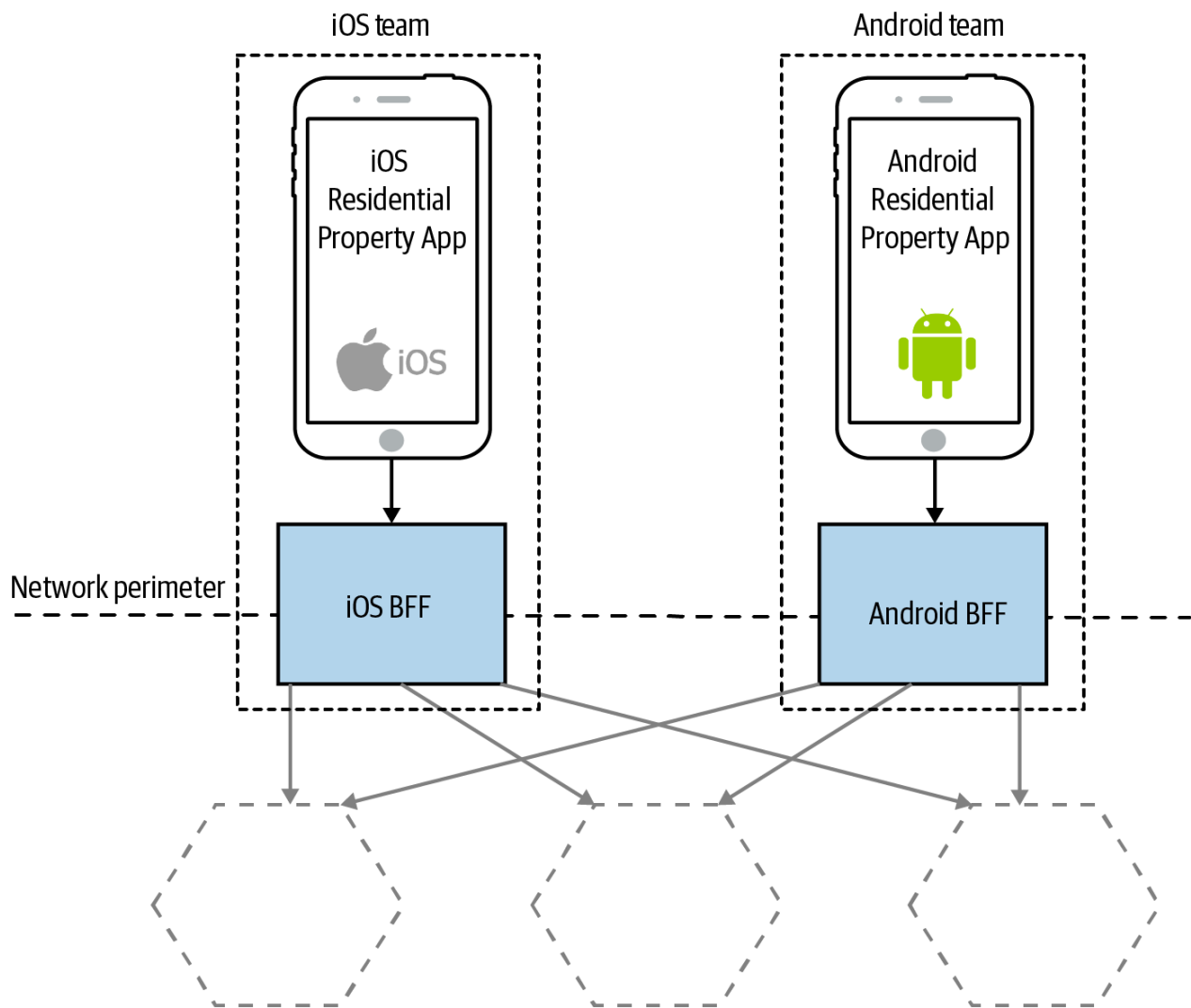


Figure 14-12. The REA iOS and Android applications have different BFFs A variation is to look for opportunities to use the same BFF for more than one type of client, albeit for the same type of user interface. SoundCloud’s Listener application allows people to listen to content on their Android or iOS devices. SoundCloud uses a single BFF for both Android and iOS, as shown in Figure 14-13 .

Hình 14-12. Các ứng dụng iOS và Android của REA có những BFF khác nhau. Một biến thể là tìm kiếm cơ hội để dùng cùng một BFF cho nhiều hơn một loại client, mặc dù là cho cùng một loại giao diện người dùng. Ứng dụng Listener của SoundCloud cho phép người ta nghe nội dung trên các thiết bị Android hoặc iOS của họ. SoundCloud dùng một BFF duy nhất cho cả Android và iOS, như thể hiện trong Hình 14-13.

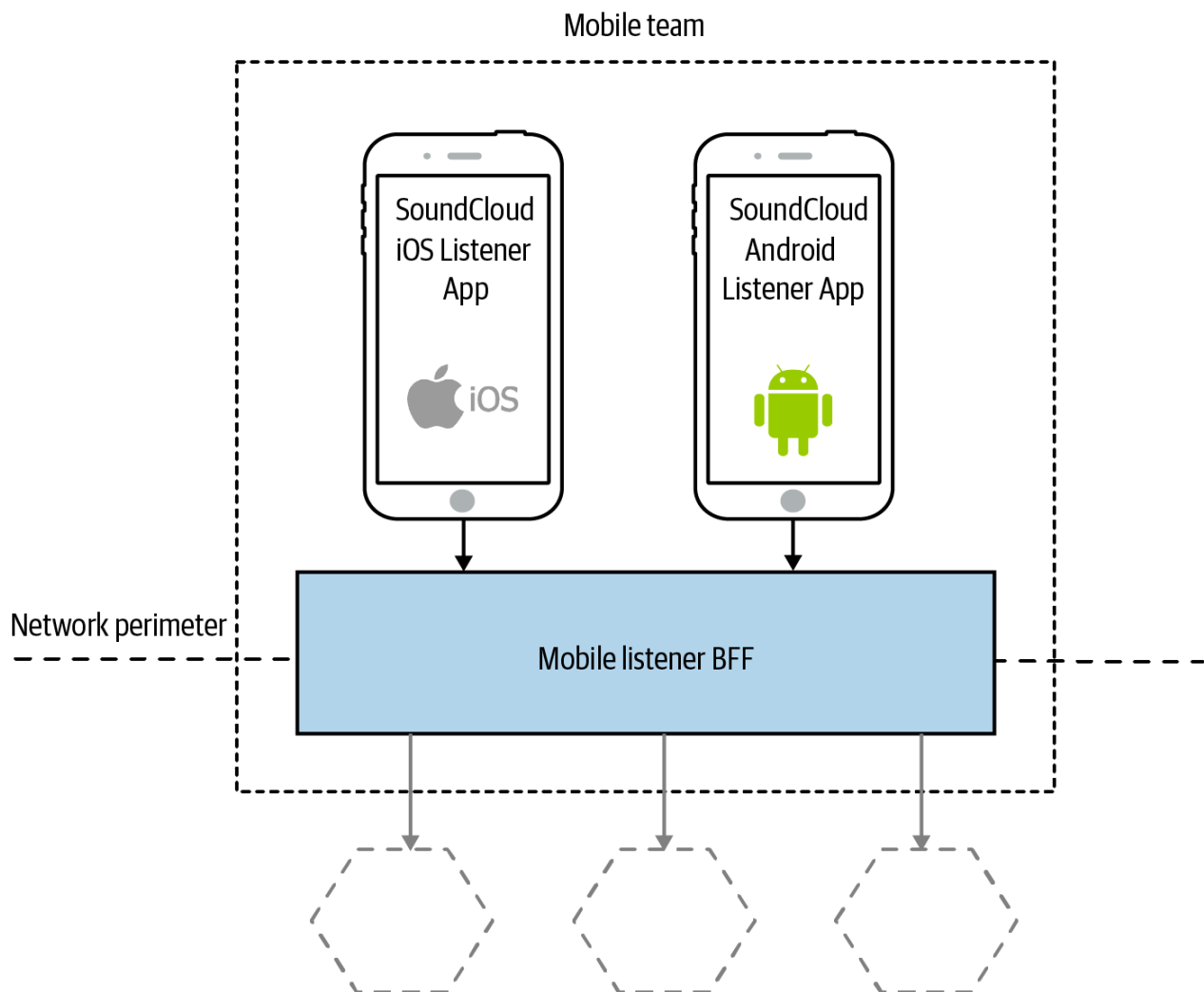


Figure 14-13. SoundCloud sharing a BFF between iOS and Android applications

Hình 14-13. SoundCloud chia sẻ một BFF giữa các ứng dụng iOS và Android

My main concern with this second model is that the more types of clients you have using a single BFF, the more temptation there may be for the BFF to become bloated by handling multiple concerns. The key thing to understand here, though, is that even when clients share a BFF, it is for the same class of user interface—so while SoundCloud’s native Listener applications for iOS and Android use the same BFF, other native applications would use different BFFs. I’m more relaxed about using this model if the same team owns both the Android and iOS applications and owns the BFF too—if these applications are maintained by different teams, I’m more inclined to recommend the more strict model. So you can see your organizational structure as being one of the main drivers for deciding which model makes the most sense (Conway’s law wins again).

Mối quan tâm chính của tôi với mô hình thứ hai này là càng nhiều loại client dùng một BFF duy nhất, thì càng có nhiều cám dỗ để BFF trở nên phình to do phải xử lý nhiều mối quan tâm. Tuy nhiên, điều mấu chốt cần hiểu ở đây là ngay cả khi các client chia sẻ một BFF, thì đó vẫn là cho cùng một lớp giao diện người dùng—vậy nên trong khi các ứng dụng Listener native của SoundCloud cho iOS và Android dùng cùng một BFF, các ứng dụng native khác sẽ dùng những BFF khác. Tôi thoải mái hơn với việc dùng mô hình này nếu cùng một đội sở hữu cả ứng dụng Android lẫn iOS và cũng sở hữu BFF—nếu những ứng dụng này được

bảo trì bởi các đội khác nhau, tôi nghiêng nhiều hơn về việc khuyến nghị mô hình nghiêm ngặt hơn. Vậy bạn có thể coi cấu trúc tổ chức của mình là một trong những động lực chính để quyết định mô hình nào hợp lý nhất (Định luật Conway lại thắng).

Stewart Gleadow from REA has suggested the guideline “one experience, one BFF.” So if the iOS and Android experiences are very similar, then it is easier to justify having a single BFF. 7 If, however, they diverge greatly, then having separate BFFs makes more sense. In the case of REA, although there was overlap between the two experiences, different teams owned them and rolled out similar features in different ways. Sometimes the same feature might be deployed differently on a different mobile device—what amounts to a native experience for an Android application might need to be reworked to feel native on iOS.

Stewart Gleadow từ REA đã đề xuất nguyên tắc “một trải nghiệm, một BFF.” Vậy nên nếu trải nghiệm iOS và Android rất tương tự nhau, thì dễ biện minh hơn cho việc có một BFF duy nhất. 7 Tuy nhiên, nếu chúng phân kỳ rất lớn, thì việc có các BFF riêng biệt hợp lý hơn. Trong trường hợp của REA, mặc dù có sự chồng lấn giữa hai trải nghiệm, các đội khác nhau sở hữu chúng và triển khai các tính năng tương tự theo những cách khác nhau. Đôi khi cùng một tính năng có thể được triển khai khác đi trên một thiết bị di động khác—cái được coi là một trải nghiệm native cho một ứng dụng Android có thể cần được làm lại để cảm thấy native trên iOS.

Another lesson from the REA story (and one we’ve covered many times already) is that software often works best when aligned around team boundaries, and BFFs are no exception. SoundCloud having a single mobile team makes having a single BFF seem sensible at first glance, as does REA having two different BFFs for the two separate teams. It’s worth noting that the SoundCloud engineers I spoke to suggested that having one BFF for the Android and iOS listener applications was something they might reconsider—they had a single mobile team, but in reality they were a mix of Android and iOS specialists, and they found themselves mostly working on one or the other application, which implies they were really two teams in effect.

Một bài học khác từ câu chuyện REA (và là một bài học chúng ta đã đề cập nhiều lần) là phần mềm thường hoạt động tốt nhất khi được tổ chức xoay quanh các ranh giới đội, và BFF cũng không là ngoại lệ. Việc SoundCloud có một đội di động duy nhất khiến việc có một BFF duy nhất thoát nhìn có vẻ hợp lý, cũng như việc REA có hai BFF khác nhau cho hai đội riêng biệt. Đáng lưu ý là các kỹ sư SoundCloud mà tôi trò chuyện đã gợi ý rằng việc có một BFF cho các ứng dụng Listener Android và iOS là điều họ có thể xem xét lại—họ có một đội di động duy nhất, nhưng trên thực tế họ là một hỗn hợp các chuyên gia Android và iOS, và họ thấy mình chủ yếu làm việc trên ứng dụng này hoặc ứng dụng kia, điều này ngụ ý rằng trên thực tế họ thực sự là hai đội.

Often the driver toward having a smaller number of BFFs is a desire to reuse server-side functionality to avoid too much duplication, but there are other ways to handle this, which we’ll cover next.

Thường thì động lực hướng tới việc có số lượng BFF nhỏ hơn là mong muốn tái sử dụng chức năng phía máy chủ để tránh quá nhiều trùng lặp, nhưng có những cách khác để xử lý điều này, mà chúng ta sẽ đề cập tiếp theo.

Reuse and BFFs — Tái sử dụng và BFF

One of the concerns of having a single BFF per user interface is that you can end up with lots of duplication among the BFFs. For example, they may end up performing the same types of aggregation, have the same or similar code for interfacing with downstream services, and so on.

If you are looking to extract common functionality, then often one of the challenges is in finding it. That duplication may occur in the BFFs themselves, but it can also end up being baked into the different clients. Due to the fact that these clients use very different technology stacks, identifying the fact that this duplication is occurring can be difficult. With organizations tending to have a common technology stack for server-side components, having multiple BFFs with duplication may be easier to spot and factor out.

Một trong những mối quan tâm của việc có một BFF cho mỗi giao diện người dùng là bạn có thể rất cuộc gặp rất nhiều trùng lặp giữa các BFF. Ví dụ, chúng có thể rất cuộc thực hiện cùng các kiểu gộp, có cùng hoặc tương tự mã để giao tiếp với các dịch vụ phía sau, v.v. Nếu bạn đang tìm cách trích xuất chức năng chung, thì thường một trong những thách thức là việc tìm ra nó. Sự trùng lặp đó có thể xảy ra trong chính các BFF, nhưng nó cũng có thể rất cuộc bị nướng vào trong các client khác nhau. Do thực tế là các client này dùng những ngăn xếp công nghệ rất khác nhau, việc nhận diện rằng sự trùng lặp này đang xảy ra có thể khó khăn. Với các tổ chức có xu hướng dùng một ngăn xếp công nghệ chung cho các thành phần phía máy chủ, việc có nhiều BFF với sự trùng lặp có thể dễ phát hiện và tách ra hơn.

Some people react to this by wanting to merge the BFFs back together, and they end up with a general-purpose aggregating gateway. My concern around regressing to a single aggregating gateway is that we can end up losing more than we gain, especially as there can be other ways to approach this duplication.

Một số người phản ứng với điều này bằng cách muốn gộp các BFF lại với nhau, và họ rất cuộc có một gateway gộp đa-mục-đích. Mối quan tâm của tôi về việc thoái lui về một gateway gộp duy nhất là chúng ta có thể rất cuộc mất nhiều hơn được, đặc biệt vì có thể có những cách khác để tiếp cận sự trùng lặp này.

7 Stewart has in turn credited Phil Calçado and Mustafa Sezgin for this recommendation.

7 Stewart đến lượt mình đã ghi công cho Phil Calçado và Mustafa Sezgin về khuyến nghị này.

As I have said before, I am fairly relaxed about duplicated code across microservices. Which is to say that, while in a single microservice boundary I will typically do whatever I can to refactor out duplication into suitable abstractions, I don't have the same reaction when confronted by duplication across microservices. This is mostly because I am often more worried about the potential for the extraction of shared code to lead to **tight coupling** between services (a topic we explored in “DRY and the Perils of Code Reuse in a Microservice World” on page 154). That said, there are certainly cases where this is warranted.

Như tôi đã nói trước đây, tôi khá thoải mái với mã bị trùng lặp xuyên suốt các microservice. Tức là, trong khi bên trong một ranh giới microservice đơn lẻ tôi thường sẽ làm bất cứ điều gì có thể để tái cấu trúc (refactor) loại bỏ trùng lặp thành những trừu tượng hóa phù hợp, tôi không có cùng phản ứng đó khi đối mặt với sự trùng lặp xuyên suốt các microservice. Điều này phần lớn là vì tôi thường lo ngại hơn về khả năng việc trích xuất mã dùng chung dẫn đến coupling chặt giữa các dịch vụ (một chủ đề chúng ta đã khám phá trong “DRY và những hiểm họa của việc tái sử dụng mã trong thế giới microservice” ở trang 154). Dù vậy, chắc chắn có những trường hợp mà điều này là chính đáng.

When the time does arise to extract common code to enable reuse between BFFs, there are two obvious options. The first, which is often cheaper but more fraught, is to extract a shared library of some sort. The reason this can be problematic is that shared libraries are a prime source of coupling, especially when used to generate client libraries for calling downstream services. Nonetheless, there

are situations in which this feels right—especially when the code being abstracted is purely a concern inside the service.

Khi đến lúc cần trích xuất mã chung để cho phép tái sử dụng giữa các BFF, có hai lựa chọn hiển nhiên. Lựa chọn thứ nhất, vốn thường rẻ hơn nhưng nhiều rủi ro hơn, là trích xuất ra một thư viện dùng chung nào đó. Lý do điều này có thể rắc rối là vì các thư viện dùng chung là một nguồn coupling hàng đầu, đặc biệt khi được dùng để sinh ra các thư viện client cho việc gọi các dịch vụ phía sau. Tuy vậy, có những tình huống trong đó điều này cảm thấy đúng đắn—đặc biệt khi mã đang được trừu tượng hóa thuần túy là một mối quan tâm bên trong dịch vụ.

The other option is to extract the shared functionality into a new microservice. This can work well if the functionality being extracted represents **business domain** functionality. A variation of this approach might be to push aggregation responsibilities to microservices further downstream. Let's consider a situation in which we want to display a list of the items in a customer's wishlist, along with information about whether or not those items are in stock and the current price, as shown in Table 14-1 .

Lựa chọn còn lại là trích xuất chức năng dùng chung ra thành một microservice mới. Điều này có thể hoạt động tốt nếu chức năng đang được trích xuất đại diện cho chức năng nghiệp vụ của miền. Một biến thể của cách tiếp cận này có thể là đẩy các trách nhiệm gộp tới các microservice xa hơn về phía sau. Hãy xét một tình huống trong đó chúng ta muốn hiển thị một danh sách các mặt hàng trong danh sách yêu thích của một khách hàng, kèm theo thông tin về việc những mặt hàng đó còn hàng hay không và giá hiện tại, như thể hiện trong Bảng 14-1.

Table 14-1. Displaying the wishlist for a customer of MusicCorp

Bảng 14-1. Hiển thị danh sách yêu thích cho một khách hàng của MusicCorp

The Brakes, Give Blood In Stock! \$5.99 Blue Juice, Retrospectable Out of Stock \$7.50 Hot Chip, Why Make Sense? Going Fast! (2 left) \$9.99

The Brakes, Give Blood Còn hàng! \$5.99 Blue Juice, Retrospectable Hết hàng \$7.50 Hot Chip, Why Make Sense? Sắp hết! (còn 2) \$9.99

The Customer microservice stores information about the wishlist and the ID of each item. The Catalog microservice stores the name and price of each item, and the stock levels are stored in our Inventory microservice. To display this same control on both the iOS and Android applications, each BFF would need to make the same three calls to the supporting microservices, as shown in Figure 14-14 .

Microservice Customer lưu trữ thông tin về danh sách yêu thích và ID của mỗi mặt hàng. Microservice Catalog lưu trữ tên và giá của mỗi mặt hàng, còn mức tồn kho được lưu trong microservice Inventory của chúng ta. Để hiển thị cùng control này trên cả ứng dụng iOS lẫn Android, mỗi BFF sẽ cần thực hiện cùng ba lời gọi tới các microservice hỗ trợ, như thể hiện trong Hình 14-14.

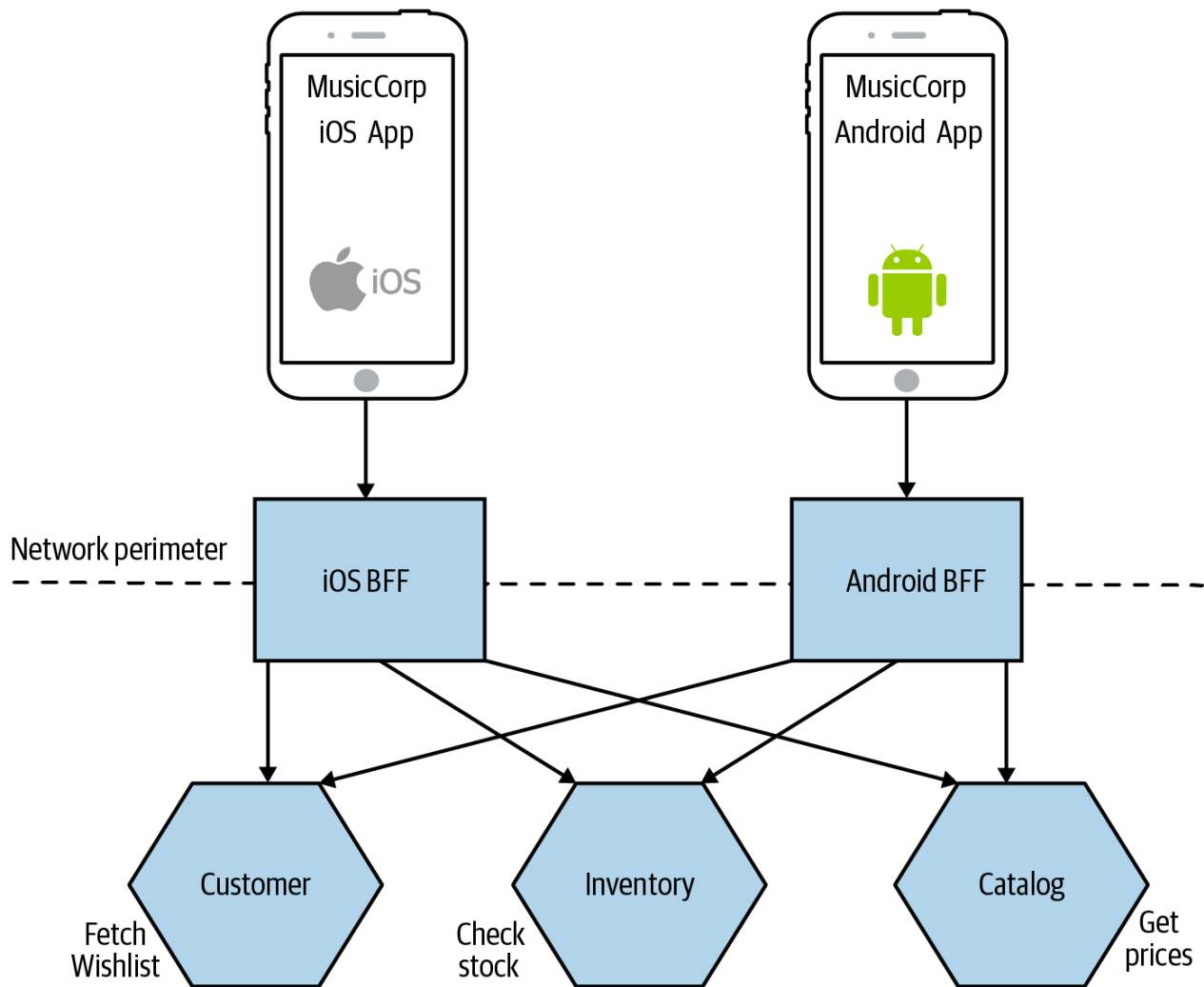


Figure 14-14. Both BFFs are performing the same operations to display a wishlist

Hình 14-14. Cả hai BFF đều đang thực hiện cùng các thao tác để hiển thị một danh sách yêu thích

A way to reduce the duplication in functionality here would be to extract this common behavior into a new microservice. In Figure 14-15, we see our new dedicated Wishlist microservice that our Android and iOS applications can both make use of.

Một cách để giảm sự trùng lặp về chức năng ở đây là trích xuất hành vi chung này ra thành một microservice mới. Trong Hình 14-15, chúng ta thấy microservice Wishlist chuyên trách mới của mình mà các ứng dụng Android và iOS đều có thể dùng.

I have to say that the same code being used in two places wouldn't automatically cause me to want to extract out a service in this way, but I'd certainly be considering it if the **transaction** cost of creating a new service was low enough, or if I was using the code in more than a couple of places—in this specific situation, if we were also showing wishlists on our web interface, for example, a dedicated microservice would start to look even more appealing. I think the old adage of creating an abstraction when you're about to implement something for the third time still feels like a good rule of thumb, even at the service level.

Tôi phải nói rằng cùng một mã được dùng ở hai nơi sẽ không tự động khiến tôi muốn trích

xuất ra một dịch vụ theo cách này, nhưng tôi chắc chắn sẽ cân nhắc điều đó nếu chi phí giao dịch (transaction cost) của việc tạo một dịch vụ mới đủ thấp, hoặc nếu tôi đang dùng mã đó ở nhiều hơn một vài nơi—trong tình huống cụ thể này, ví dụ nếu chúng ta cũng hiển thị danh sách yêu thích trên giao diện web, thì một microservice chuyên trách sẽ bắt đầu trông còn hấp dẫn hơn. Tôi nghĩ câu châm ngôn xưa về việc tạo ra một trừu tượng hóa khi bạn sắp hiện thực một thứ gì đó lần thứ ba vẫn cảm thấy là một quy tắc ngón tay cái tốt, kể cả ở cấp độ dịch vụ.

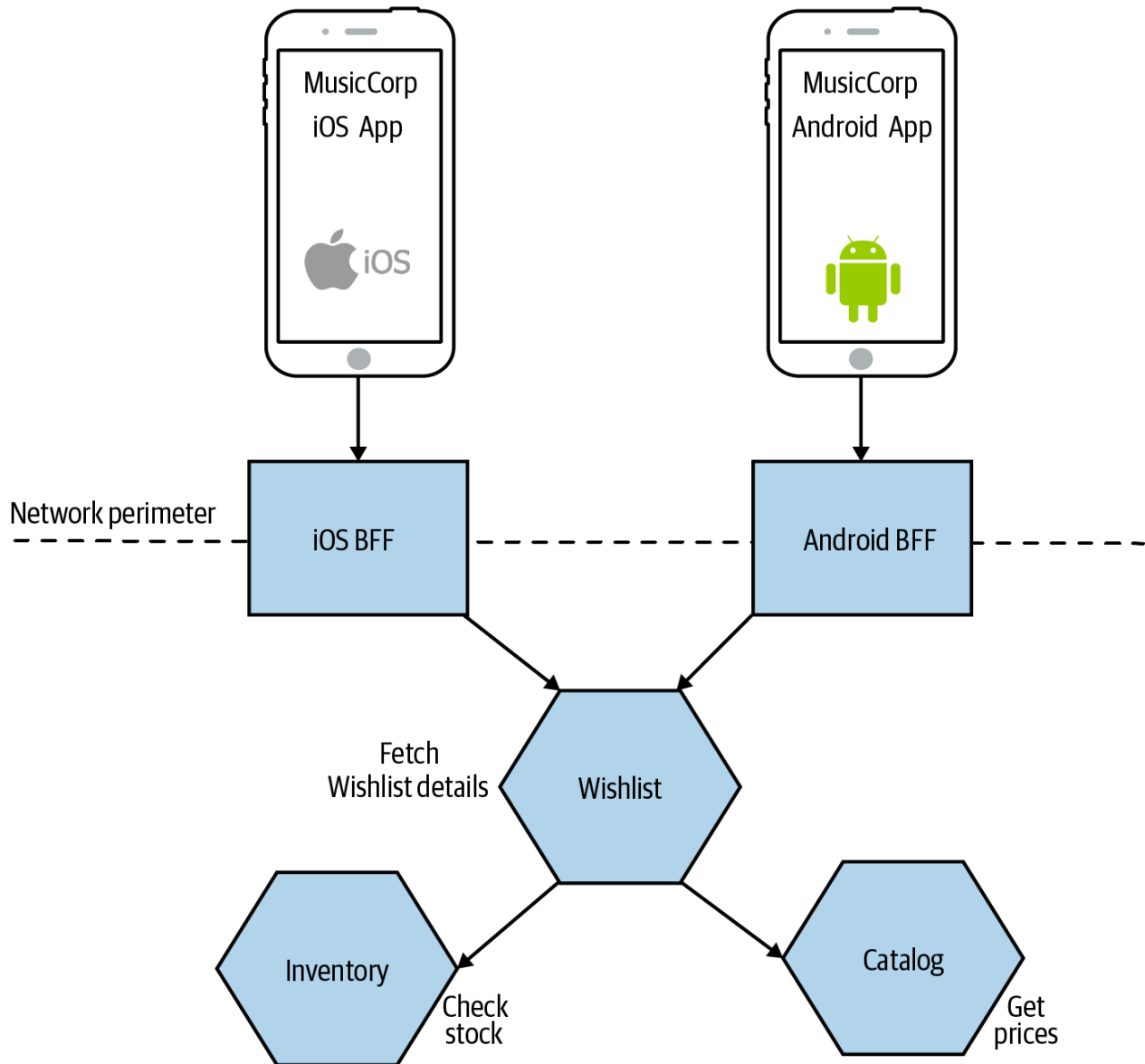


Figure 14-15. Common functionality is extracted into a Wishlist microservice, allowing for reuse across BFFs

Hình 14-15. Chức năng chung được trích xuất ra thành một microservice Wishlist, cho phép tái sử dụng xuyên suốt các BFF

BFFs for Desktop Web and Beyond — BFF cho web desktop và hơn thế nữa

You can think of BFFs as just having a use in solving the constraints of mobile devices. The desktop web experience is typically delivered on more powerful devices with better connectivity, where the cost of making multiple downstream calls is manageable. This can allow your web application to make multiple calls directly to downstream services without the need for a BFF.

Bạn có thể nghĩ về BFF như chỉ có công dụng trong việc giải quyết các ràng buộc của thiết bị di động. Trải nghiệm web desktop thường được cung cấp trên các thiết bị mạnh hơn với kết nối tốt hơn, nơi chi phí của việc thực hiện nhiều lời gọi phía sau là chấp nhận được. Điều này có thể cho phép ứng dụng web của bạn thực hiện nhiều lời gọi trực tiếp tới các dịch vụ phía sau mà không cần một BFF.

I have seen situations, though, in which the use of a BFF for the web too can be useful. When you are generating a larger portion of the web UI on the server side (e.g., using server-side templating), a BFF is the obvious place where this can be done. This approach can also simplify **caching** somewhat, as you can place a reverse proxy in front of the BFF, allowing you to **cache** the results of aggregated calls.

Tuy nhiên, tôi đã thấy những tình huống trong đó việc dùng một BFF cho cả web cũng có thể hữu ích. Khi bạn đang sinh ra một phần lớn hơn của UI web ở phía máy chủ (ví dụ, dùng template phía máy chủ), một BFF là nơi hiển nhiên để có thể làm điều này. Cách tiếp cận này cũng có thể đơn giản hóa việc caching phần nào, vì bạn có thể đặt một reverse proxy phía trước BFF, cho phép bạn cache các kết quả của các lời gọi đã được gộp.

I’ve seen at least one organization use BFFs for other external parties that need to make calls. Coming back to my perennial example of MusicCorp, I might expose a BFF to allow third parties to extract royalty payment information, or to allow streaming to a range of set-top box devices, as we see in Figure 14-16 . These aren’t really BFFs anymore, as the external parties aren’t presenting a “user interface,” but this is an example of the same pattern being used in a different context, so I thought it worth sharing.

Tôi đã thấy ít nhất một tổ chức dùng BFF cho các bên ngoài khác cần thực hiện lời gọi. Quay lại ví dụ muôn thuở của tôi về MusicCorp, tôi có thể phơi bày một BFF để cho phép các bên thứ ba trích xuất thông tin thanh toán tiền bản quyền (royalty), hoặc để cho phép phát trực tuyến (streaming) tới một loạt thiết bị set-top box, như chúng ta thấy trong Hình 14-16. Những thứ này thực ra không còn là BFF nữa, vì các bên ngoài không trình bày một “giao diện người dùng”, nhưng đây là một ví dụ về cùng một pattern được dùng trong một bối cảnh khác, nên tôi nghĩ nó đáng để chia sẻ.

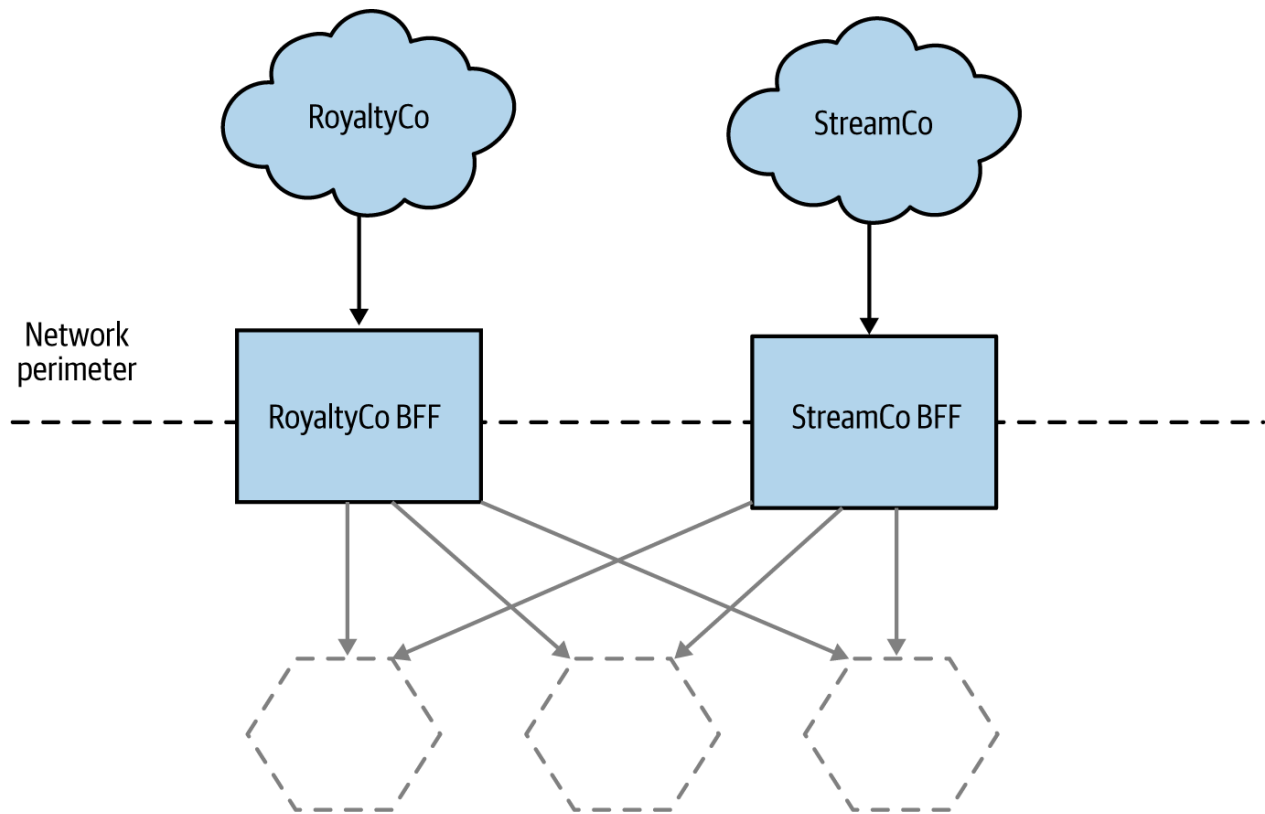


Figure 14-16. Using BFFs to manage external APIs

Hình 14-16. Dùng BFF để quản lý các API bên ngoài

This approach can be especially effective, as third parties often have limited to no ability (or desire) to use or change the API calls they make. With a central API backend, you may have to keep old versions of the API around just to satisfy a small subset of your outside parties unable to make a change; with a BFF, this problem is substantially reduced. It also limits the impact of breaking changes. You could change the API for Facebook in a way that would break compatibility with other parties, but as they use a different BFF, they aren't impacted by this change.

Cách tiếp cận này có thể đặc biệt hiệu quả, vì các bên thứ ba thường có khả năng hạn chế đến không có (hoặc không muốn) dùng hay thay đổi các lời gọi API mà họ thực hiện. Với một backend API trung tâm, bạn có thể phải giữ lại các phiên bản API cũ chỉ để thỏa mãn một nhóm nhỏ các bên ngoài không thể thực hiện thay đổi; với một BFF, vấn đề này được giảm bớt đáng kể. Nó cũng giới hạn tác động của các thay đổi gây vỡ tương thích (breaking change). Bạn có thể thay đổi API cho Facebook theo cách phá vỡ tính tương thích với các bên khác, nhưng vì họ dùng một BFF khác, họ không bị tác động bởi thay đổi này.

When to Use — Khi nào nên dùng

For an application that is only providing a web UI, I suspect a BFF will make sense only if and when you have a significant amount of aggregation required on the server side. Otherwise, I think some of the other UI composition techniques we've already covered can work just as well without requiring an additional server-side component.

Đối với một ứng dụng chỉ cung cấp một UI web, tôi ngờ rằng một BFF sẽ chỉ hợp lý nếu và

khi bạn có một lượng đáng kể việc gộp cần thực hiện ở phía máy chủ. Nếu không, tôi nghĩ một số kỹ thuật bố cục UI khác mà chúng ta đã đề cập có thể hoạt động tốt chẳng kém mà không cần một thành phần phía máy chủ bổ sung.

The moment that you need to provide specific functionality for a mobile UI or third party, though, I would strongly consider using BFFs for each client from the outset. I might reconsider if the cost of deploying additional services is high, but the separation of concerns that a BFF can bring make it a fairly compelling proposition in most cases. I'd be even more inclined to use a BFF if there is significant separation between the people building the UI and downstream services, for reasons I've outlined.

Tuy nhiên, ngay khi bạn cần cung cấp chức năng cụ thể cho một UI di động hoặc bên thứ ba, tôi sẽ mạnh mẽ cân nhắc dùng BFF cho mỗi client ngay từ đầu. Tôi có thể xem xét lại nếu chi phí triển khai các dịch vụ bổ sung là cao, nhưng sự tách bạch mối quan tâm (separation of concerns) mà một BFF có thể mang lại khiến nó là một đề xuất khá thuyết phục trong hầu hết các trường hợp. Tôi sẽ còn nghiêng hơn nữa về việc dùng một BFF nếu có sự tách bạch đáng kể giữa những người xây dựng UI và các dịch vụ phía sau, vì những lý do tôi đã nêu.

We then come to the question of how to implement the BFF—let's look at GraphQL and the role it can play.

Sau đó chúng ta đến với câu hỏi làm thế nào để hiện thực BFF—hãy xem xét GraphQL và vai trò mà nó có thể đóng.

GraphQL — GraphQL

GraphQL is a query language that allows clients to issue queries to access or mutate data. Like SQL, GraphQL allows these queries to be changed dynamically, allowing the client to define exactly what information it wants back. With a standard REST over HTTP call, for example, when issuing a GET request for an Order resource you'd get back all the fields for that order. But what if, in that particular situation, you only wanted the total amount of the order? You could just ignore the other fields, of course, or else provide an alternative resource (an Order Summary , perhaps) that contains just the information you need. With GraphQL, you could issue a request asking for only the fields you need, as we see in Example 14-1 .

GraphQL là một ngôn ngữ truy vấn cho phép client phát các truy vấn để truy cập hoặc thay đổi (mutate) dữ liệu. Giống như SQL, GraphQL cho phép thay đổi các truy vấn này một cách động, cho phép client định nghĩa chính xác thông tin nào nó muốn nhận về. Ví dụ, với một lời gọi REST trên HTTP tiêu chuẩn, khi phát một yêu cầu GET cho một tài nguyên Order bạn sẽ nhận về tất cả các trường cho đơn hàng đó. Nhưng nếu, trong tình huống cụ thể đó, bạn chỉ muốn tổng số tiền của đơn hàng thì sao? Tất nhiên bạn có thể chỉ phớt lờ các trường khác, hoặc cung cấp một tài nguyên thay thế (có lẽ là một Order Summary) chỉ chứa thông tin bạn cần. Với GraphQL, bạn có thể phát một yêu cầu chỉ hỏi những trường bạn cần, như chúng ta thấy trong Ví dụ 14-1.

Example 14-1. A example GraphQL query being used to fetch order information

Ví dụ 14-1. Một truy vấn GraphQL ví dụ được dùng để lấy thông tin đơn hàng

```
{ order(id: 123) { date total status delivery { company driver due date } } }
```

In this query, we've asked for order 123, and we've asked for the total price and the status of the order. We've gone further and asked for information about the delivery of this order, so we can get information on the name of the driver who will deliver our package, the company they work for, and when the package is expected to arrive. With a normal REST API, unless the delivery information was contained inside the Order resource, we may have to make an additional call to fetch this information. So not only is GraphQL helping us ask for exactly the fields we want, but it can also reduce round trips. A query like this requires that we define the various data types we are accessing—explicitly defining types is a key part of GraphQL.

Trong truy vấn này, chúng ta đã hỏi về đơn hàng 123, và chúng ta đã hỏi tổng giá và trạng thái của đơn hàng. Chúng ta đã đi xa hơn và hỏi thông tin về việc giao hàng của đơn này, để chúng ta có thể lấy thông tin về tên của tài xế sẽ giao gói hàng của chúng ta, công ty họ làm việc cho, và khi nào gói hàng dự kiến đến. Với một API REST thông thường, trừ khi thông tin giao hàng được chứa bên trong tài nguyên Order, chúng ta có thể phải thực hiện một lời gọi bổ sung để lấy thông tin này. Vậy nên GraphQL không chỉ giúp chúng ta hỏi chính xác những trường chúng ta muốn, mà nó còn có thể giảm số vòng đi-về (round trip). Một truy vấn như thế này đòi hỏi chúng ta định nghĩa các kiểu dữ liệu khác nhau mà chúng ta đang truy cập—việc định nghĩa kiểu một cách tường minh là một phần chủ chốt của GraphQL.

To implement GraphQL, we need a resolver to handle the queries. A GraphQL resolver sits on the server side and maps the GraphQL queries into calls to actually fetch the information. So in the case of a microservice architecture, we'd need a resolver that would be capable of mapping the request for the order with ID 123 into an equivalent call to a microservice.

Để hiện thực GraphQL, chúng ta cần một resolver để xử lý các truy vấn. Một GraphQL resolver nằm ở phía máy chủ và ánh xạ các truy vấn GraphQL thành các lời gọi để thực sự lấy thông tin. Vậy trong trường hợp một kiến trúc microservice, chúng ta sẽ cần một resolver có khả năng ánh xạ yêu cầu cho đơn hàng có ID 123 thành một lời gọi tương đương tới một microservice.

In this way, we can use GraphQL to implement an aggregating gateway, or even a BFF. The benefit of GraphQL is that we can easily change the aggregation and filtering we want simply by changing the query from the client; no changes are needed at the GraphQL server side of things as long as the GraphQL types support the query we want to make. If we no longer wanted to see the driver name in the example query, we could just omit this from the query itself and it would no longer be sent. On the other hand, if we wanted to see the number of points we were awarded for this order, assuming this information is available in the order type, we could just add this to the query and the information would be returned. This is a significant advantage over BFF implementations that require changes in aggregation logic to also be applied to the BFF itself.

Theo cách này, chúng ta có thể dùng GraphQL để hiện thực một gateway gộp, hoặc thậm chí một BFF. Lợi ích của GraphQL là chúng ta có thể dễ dàng thay đổi việc gộp và lọc mà chúng ta muốn chỉ bằng cách thay đổi truy vấn từ phía client; không cần thay đổi nào ở phía máy chủ GraphQL miễn là các kiểu GraphQL hỗ trợ truy vấn mà chúng ta muốn thực hiện. Nếu chúng ta không còn muốn thấy tên tài xế trong truy vấn ví dụ, chúng ta có thể chỉ cần bỏ nó khỏi chính truy vấn và nó sẽ không còn được gửi nữa. Mặt khác, nếu chúng ta muốn thấy số điểm chúng ta được thưởng cho đơn hàng này, giả định thông tin này có sẵn trong kiểu order, chúng ta có thể chỉ cần thêm nó vào truy vấn và thông tin sẽ được trả về. Đây là một lợi thế đáng kể so với các bản hiện thực BFF vốn đòi hỏi các thay đổi trong logic gộp cũng phải được áp dụng vào chính BFF.

The flexibility that GraphQL gives for the client device to dynamically change the queries being

made without server-side changes means that there is less chance of your GraphQL server becoming a shared, contended resource, as we discussed with the general-purpose aggregating gateway. That said, server-side changes will still be needed if you need to expose new types or add fields to existing types. As such, you may still want multiple GraphQL server backends to align along team boundaries—so GraphQL becomes a way of implementing a BFF.

Sự linh hoạt mà GraphQL mang lại cho thiết bị client để thay đổi động các truy vấn được thực hiện mà không cần thay đổi phía máy chủ nghĩa là có ít khả năng máy chủ GraphQL của bạn trở thành một tài nguyên dùng chung, bị tranh chấp, như chúng ta đã thảo luận với gateway gộp đa-mục-đích. Dù vậy, vẫn sẽ cần các thay đổi phía máy chủ nếu bạn cần phơi bày các kiểu mới hoặc thêm trường vào các kiểu hiện có. Như vậy, bạn vẫn có thể muốn nhiều backend máy chủ GraphQL được tổ chức theo các ranh giới đội—vậy nên GraphQL trở thành một cách để hiện thực một BFF.

I do have concerns around GraphQL, which I outlined in detail in Chapter 5. That said, it is a neat solution allowing for dynamic querying to suit the needs of different types of user interfaces.

Tôi quả thực có những mối quan tâm xung quanh GraphQL, mà tôi đã nêu chi tiết trong Chương 5. Dù vậy, nó là một giải pháp gọn gàng cho phép truy vấn động để phù hợp với nhu cầu của các loại giao diện người dùng khác nhau.

A Hybrid Approach — Một cách tiếp cận lai (Hybrid)

Many of the aforementioned options don't need to be one-size-fits-all. I could see an organization adopting the approach of widget-based decomposition to create a website but using a backend for frontend approach when it comes to its mobile application. The key point is that we need to retain **cohesion** of the underlying capabilities that we offer our users. We need to ensure that logic associated with ordering music or changing customer details lives inside those services that handle those operations and doesn't get smeared all over our system. Avoiding the trap of putting too much behavior into any intermediate layers is a tricky balancing act.

Nhiều trong số các lựa chọn nói trên không cần phải là một-cỡ-vừa-cho-tất-cả. Tôi có thể hình dung một tổ chức áp dụng cách tiếp cận phân rã theo widget để tạo một website nhưng dùng cách tiếp cận Backend for Frontend khi đến ứng dụng di động của nó. Điểm mấu chốt là chúng ta cần giữ tính gắn kết của các năng lực nền tảng mà chúng ta cung cấp cho người dùng. Chúng ta cần đảm bảo rằng logic gắn với việc đặt mua nhạc hay thay đổi chi tiết khách hàng sống bên trong những dịch vụ xử lý các thao tác đó và không bị bôi trét khắp hệ thống của chúng ta. Tránh cái bẫy đặt quá nhiều hành vi vào bất kỳ tầng trung gian nào là một hành động giữ thăng bằng khó khăn.

Summary — Tóm tắt

As I hope I have shown, decomposition of functionality doesn't have to stop at the server side, and having dedicated frontend teams isn't inevitable. I've shared a number of different ways to construct a user interface that can make use of supporting microservices while enabling focused end-to-end delivery.

Như tôi hy vọng mình đã chỉ ra, việc phân rã chức năng không nhất thiết phải dừng lại ở phía máy chủ, và việc có các đội frontend chuyên trách không phải là điều tất yếu. Tôi đã chia sẻ một số cách khác nhau để xây dựng một giao diện người dùng có thể tận dụng các microservice hỗ trợ trong khi vẫn cho phép việc chuyển giao đầu-cuối tập trung.

In our next chapter, we move from the technical to the people side of things when we explore in greater detail the interplay of microservices and organizational structures.

Trong chương tiếp theo của chúng ta, chúng ta chuyển từ khía cạnh kỹ thuật sang khía cạnh con người khi khám phá chi tiết hơn sự tương tác giữa microservice và các cấu trúc tổ chức.

CHAPTER 15 Organizational Structures — Chương 15: Cấu trúc tổ chức

While much of the book so far has focused on the technical challenges in moving toward a fine-grained **architecture**, we’ve also looked at the interplay between our **microservice** architecture and how we organize our teams. In “Toward Stream- Aligned Teams” on page 459 , we looked at the concept of stream-aligned teams, which have end-to-end responsibility for delivery of user-facing functionality, and at how **microservices** can help make such team structures a reality.

Mặc dù phần lớn cuốn sách cho đến nay tập trung vào những thách thức kỹ thuật khi chuyển dịch sang một kiến trúc chi tiết (fine-grained), chúng ta cũng đã xem xét sự tương tác qua lại giữa kiến trúc microservice và cách chúng ta tổ chức các nhóm (team) của mình. Trong phần “Toward Stream-Aligned Teams” ở trang 459, chúng ta đã xem xét khái niệm stream-aligned team, tức nhóm có trách nhiệm đầu-cuối (end-to-end) trong việc chuyển giao chức năng hướng tới người dùng, và cách microservice có thể giúp biến những cấu trúc nhóm như vậy thành hiện thực.

We now need to flesh out these ideas and look at other organizational considerations. As we’ll see, if you want to get the most out of microservices, you ignore your company’s organization chart at your peril!

Giờ đây chúng ta cần khai triển những ý tưởng này và xem xét các cân nhắc tổ chức khác. Như chúng ta sẽ thấy, nếu bạn muốn tận dụng tối đa microservice, thì việc phớt lờ sơ đồ tổ chức của công ty là một điều cực kỳ nguy hiểm!

Loosely Coupled Organizations — Tổ chức có coupling lỏng

Throughout the book, I’ve made the case for a loosely coupled architecture and argued that alignment with more autonomous, loosely coupled, stream-aligned teams is likely going to deliver the best outcomes. A shift to a microservice architecture without a shift in organizational structure will blunt the usefulness of microservices— you may end up paying the (considerable) cost for the architectural change, without getting the return on your investment. I’ve written generally about the need to reduce coordination between teams to help speed up delivery, which in turn enables teams to make more decisions for themselves. These are ideas we’ll explore more fully in this chapter, and we’ll flesh out some of these organizational and behavioral shifts that are required, but before that, I think it’s important to share my vision for what a loosely coupled organization looks like.

Xuyên suốt cuốn sách, tôi đã lập luận cho một kiến trúc có coupling lỏng (loosely coupled)

và cho rằng việc giống hàng với các nhóm tự chủ hơn, có coupling lỏng hơn, và theo kiểu stream-aligned nhiều khả năng sẽ mang lại kết quả tốt nhất. Việc chuyển sang kiến trúc microservice mà không thay đổi cấu trúc tổ chức sẽ làm cùn đi tính hữu dụng của microservice—bạn có thể phải trả cái giá (đáng kể) cho việc thay đổi kiến trúc mà không thu được lợi tức từ khoản đầu tư của mình. Tôi đã viết một cách tổng quát về nhu cầu giảm bớt sự phối hợp giữa các nhóm để giúp tăng tốc chuyển giao, điều này lại cho phép các nhóm tự đưa ra nhiều quyết định hơn. Đây là những ý tưởng mà chúng ta sẽ khám phá đầy đủ hơn trong chương này, và chúng ta sẽ khai triển một số chuyển dịch về tổ chức và hành vi cần thiết, nhưng trước đó, tôi nghĩ điều quan trọng là chia sẻ tầm nhìn của tôi về việc một tổ chức có coupling lỏng trông như thế nào.

In their book *Accelerate*, Nicole Forsgren, Jez Humble, and Gene Kim looked at the characteristics of autonomous, loosely coupled teams to better understand which behaviors are most important to achieving optimum performance. According to the authors, what is key is whether teams can:

Trong cuốn sách *Accelerate* của họ, Nicole Forsgren, Jez Humble, và Gene Kim đã xem xét các đặc điểm của những nhóm tự chủ, có coupling lỏng để hiểu rõ hơn hành vi nào quan trọng nhất trong việc đạt được hiệu suất tối ưu. Theo các tác giả, điều then chốt là liệu các nhóm có thể:

- Make large-scale changes to the design of their system without the permission of somebody outside the team
 - Make large-scale changes to the design of their system without depending on other teams to make changes in their systems or creating significant work for other teams
 - Complete their work without communicating and coordinating with people outside their team
 - **Deploy** and **release** their product or **service** on demand, regardless of other services it depends upon
 - Do most of their **testing** on demand, without requiring an integrated test environment
 - Perform deployments during normal business hours with negligible downtime The **stream-aligned team**, a concept we first encountered in Chapter 1, aligns with this vision for a loosely coupled organization. If you are trying to move toward a stream-aligned team structure, these characteristics make for a fantastic checklist to ensure you are going in the right direction.
- Thực hiện những thay đổi quy mô lớn đối với thiết kế hệ thống của mình mà không cần sự cho phép của một ai đó bên ngoài nhóm
 - Thực hiện những thay đổi quy mô lớn đối với thiết kế hệ thống của mình mà không phụ thuộc vào các nhóm khác để họ thực hiện thay đổi trong hệ thống của họ hoặc tạo ra khối lượng công việc đáng kể cho các nhóm khác
 - Hoàn thành công việc của mình mà không cần giao tiếp và phối hợp với người bên ngoài nhóm
 - Triển khai (deploy) và phát hành (release) sản phẩm hoặc dịch vụ của mình theo nhu cầu, bất kể các dịch vụ khác mà nó phụ thuộc vào
 - Thực hiện hầu hết việc kiểm thử (testing) theo nhu cầu, mà không cần một môi trường kiểm thử tích hợp
 - Thực hiện các đợt triển khai trong giờ làm việc bình thường với thời gian gián đoạn (downtime) không đáng kể

Some of these characteristics look more technical in nature—for example, being able to deploy during normal business hours could be enabled through an architecture that supports zero-downtime deployments. But all of these in fact require a behavioral change. To allow teams to have a fuller sense of **ownership** around their systems, a move away from centralized control is required, including how architectural decision making is done (something we explore in Chapter 16). Fundamentally, achieving loosely coupled organizational structures requires that power and

accountability are decentralized .

Một số đặc điểm này có vẻ mang tính kỹ thuật hơn về bản chất—ví dụ, khả năng triển khai trong giờ làm việc bình thường có thể được kích hoạt thông qua một kiến trúc hỗ trợ triển khai không gián đoạn (zero-downtime). Nhưng tất cả những điều này thực ra đều đòi hỏi một sự thay đổi về hành vi. Để cho phép các nhóm có cảm giác làm chủ trọn vẹn hơn đối với hệ thống của mình, cần phải có một sự dịch chuyển ra khỏi kiểm soát tập trung, bao gồm cả cách ra quyết định kiến trúc được thực hiện (điều mà chúng ta khám phá trong Chương 16). Về cơ bản, để đạt được các cấu trúc tổ chức có coupling lỏng đòi hỏi quyền lực và trách nhiệm giải trình phải được phi tập trung hóa.

The majority of this chapter will discuss how we make this all work, looking at team sizes, types of ownership models, the role of the platform, and more. There are a host of changes you can consider making to move your organization in the right direction.

Phần lớn chương này sẽ thảo luận về cách chúng ta làm cho tất cả những điều này vận hành, xem xét quy mô nhóm, các loại mô hình quyền sở hữu (ownership), vai trò của nền tảng (platform), và hơn thế nữa. Có một loạt các thay đổi mà bạn có thể cân nhắc thực hiện để dịch chuyển tổ chức của mình theo đúng hướng.

Before all that, though, let's explore the interplay between organization and architecture a bit more.

Tuy nhiên, trước tất cả những điều đó, hãy khám phá thêm một chút về sự tương tác qua lại giữa tổ chức và kiến trúc.

1 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

1 Nicole Forsgren, Jez Humble, and Gene Kim, *Accelerate: The Science of Building and Scaling High Performing Technology Organizations* (Portland, OR: IT Revolution, 2018).

Conway's Law — Định luật Conway

Our industry is young and seems to be constantly reinventing itself. And yet a few key “laws” have stood the test of time. Moore's law, for example, states that the density of transistors on integrated circuits doubles every two years, and it has proved to be uncannily accurate (although that trend has been slowing). One law that I have found to be almost universally true, and to be far more useful in my day-to-day work, is Conway's law.

Ngành của chúng ta còn non trẻ và dường như liên tục tự tái phát minh chính mình. Vậy mà một vài “định luật” then chốt vẫn đứng vững trước thử thách của thời gian. Định luật Moore, chẳng hạn, phát biểu rằng mật độ transistor trên các vi mạch tích hợp tăng gấp đôi sau mỗi hai năm, và nó đã được chứng minh là chính xác đến kỳ lạ (mặc dù xu hướng đó đang chậm lại). Một định luật mà tôi thấy gần như luôn đúng, và hữu ích hơn nhiều trong công việc hằng ngày của tôi, là Định luật Conway.

Melvin Conway's paper “How Do Committees Invent?,” published in *Datamation* magazine in April 1968, observed that:

Bài báo của Melvin Conway “How Do Committees Invent?,” đăng trên tạp chí *Datamation* vào tháng 4 năm 1968, đã quan sát rằng:

Any organization that designs a system (defined more broadly here than just information systems) will inevitably produce a design whose structure is a copy of the organization's communication structure. This statement is often quoted, in various forms, as Conway's law. Eric S. Raymond

summarized this phenomenon in The New Hacker's Dictionary (MIT Press) by stating, "If you have four groups working on a compiler, you'll get a 4-pass compiler."

Bất kỳ tổ chức nào thiết kế một hệ thống (được định nghĩa rộng hơn ở đây so với chỉ riêng các hệ thống thông tin) chắc chắn sẽ tạo ra một thiết kế mà cấu trúc của nó là một bản sao của cấu trúc giao tiếp của tổ chức đó. Phát biểu này thường được trích dẫn, dưới nhiều hình thức khác nhau, dưới tên Định luật Conway. Eric S. Raymond đã tóm tắt hiện tượng này trong The New Hacker's Dictionary (MIT Press) bằng câu nói, "Nếu bạn có bốn nhóm làm việc trên một trình biên dịch, bạn sẽ nhận được một trình biên dịch 4 lượt (4-pass)."

Conway's law shows us that a loosely coupled organization results in a loosely coupled architecture (and vice versa), reinforcing the idea that hoping to gain the benefits of a loosely coupled microservice architecture will be problematic without also considering the organization building the software.

Định luật Conway cho chúng ta thấy rằng một tổ chức có coupling lỏng dẫn đến một kiến trúc có coupling lỏng (và ngược lại), củng cố ý tưởng rằng hy vọng đạt được những lợi ích của một kiến trúc microservice có coupling lỏng sẽ là điều nan giải nếu không đồng thời cân nhắc đến tổ chức đang xây dựng phần mềm.

Evidence — Bằng chứng

The story goes that when Melvin Conway submitted his paper on this topic to the Harvard Business Review, the magazine rejected it, claiming he hadn't proved his thesis. I've seen his theory borne out in so many different situations that I've accepted it as true. But you don't have to take my word for it: since Conway's original submission, a lot of work has been done in this area. A number of studies have been carried out to explore the interrelation of organizational structure and the systems they create.

Câu chuyện kể rằng khi Melvin Conway nộp bài báo của mình về chủ đề này cho Harvard Business Review, tạp chí đã từ chối nó, viện cớ rằng ông chưa chứng minh được luận điểm của mình. Tôi đã chứng kiến lý thuyết của ông được kiểm chứng trong rất nhiều tình huống khác nhau đến mức tôi đã chấp nhận nó là đúng. Nhưng bạn không cần phải tin lời tôi: kể từ lần nộp bài ban đầu của Conway, rất nhiều công trình đã được thực hiện trong lĩnh vực này. Một số nghiên cứu đã được tiến hành để khám phá mối tương quan giữa cấu trúc tổ chức và các hệ thống mà chúng tạo ra.

Loosely and tightly coupled organizations

Các tổ chức có coupling lỏng và coupling chặt

In "Exploring the Duality Between Product and Organizational Architectures,"² the authors look at a number of different software systems, loosely categorized as being created either by "loosely coupled organizations" or by "tightly coupled organizations." For tightly coupled organizations, think commercial product firms that are

Trong "Exploring the Duality Between Product and Organizational Architectures,"² các tác giả xem xét một số hệ thống phần mềm khác nhau, được phân loại một cách lỏng lẻo là được tạo ra hoặc bởi "các tổ chức có coupling lỏng" hoặc bởi "các tổ chức có coupling chặt." Đối với các tổ chức có coupling chặt, hãy nghĩ đến các hãng sản phẩm thương mại thường

² Alan MacCormack, Carliss Baldwin, and John Rusnak, "Exploring the Duality Between Product and Organizational Architectures: A Test of the Mirroring Hypothesis," Research Policy 41, no. 8 (October

2012): 1309–24.

2 Alan MacCormack, Carliss Baldwin, and John Rusnak, “Exploring the Duality Between Product and Organizational Architectures: A Test of the Mirroring Hypothesis,” *Research Policy* 41, no. 8 (October 2012): 1309–24.

typically colocated with strongly aligned visions and goals, while loosely coupled organizations are well represented by distributed open source communities.

được đặt cùng một chỗ với tầm nhìn và mục tiêu được giống hệt nhau, trong khi các tổ chức có coupling lỏng được đại diện rõ nét bởi các cộng đồng mã nguồn mở phân tán.

In their study, in which they matched similar product pairs from each type of organization, the authors found that the more loosely coupled organizations actually created more modular, less coupled systems, whereas the more tightly coupled organization’s software was less modularized.

Trong nghiên cứu của mình, nơi họ ghép các cặp sản phẩm tương tự từ mỗi loại tổ chức, các tác giả phát hiện ra rằng các tổ chức có coupling lỏng hơn thực sự tạo ra các hệ thống mô-đun hóa nhiều hơn, ít coupling hơn, trong khi phần mềm của tổ chức có coupling chặt hơn thì ít được mô-đun hóa hơn.

Windows Vista

Windows Vista

Microsoft carried out an empirical study³ in which it looked at how its own organizational structure impacted the quality of a specific software product, Windows Vista. Specifically, the researchers looked at multiple factors to determine how error-prone a component in the system would be. After looking at multiple metrics, including commonly used software quality metrics like code complexity, they found that the metrics associated with organizational structures (such as the number of engineers who have worked on a piece of code) proved to be the most statistically relevant measures.

Microsoft đã tiến hành một nghiên cứu thực nghiệm³ trong đó họ xem xét cách cấu trúc tổ chức của chính mình đã tác động đến chất lượng của một sản phẩm phần mềm cụ thể, Windows Vista. Cụ thể, các nhà nghiên cứu đã xem xét nhiều yếu tố để xác định mức độ dễ phát sinh lỗi của một thành phần trong hệ thống.⁴ Sau khi xem xét nhiều chỉ số (metric), bao gồm các chỉ số chất lượng phần mềm thường dùng như độ phức tạp của mã, họ phát hiện ra rằng các chỉ số liên quan đến cấu trúc tổ chức (chẳng hạn như số lượng kỹ sư đã từng làm việc trên một đoạn mã) hóa ra lại là những thước đo có liên quan về mặt thống kê nhiều nhất.

So here we have another example of an organization’s structure impacting the nature of the system that organization creates.

Vậy ở đây chúng ta có thêm một ví dụ về việc cấu trúc của một tổ chức tác động đến bản chất của hệ thống mà tổ chức đó tạo ra.

Netflix and Amazon

Netflix và Amazon

Probably the two poster children for the idea that organizations and architecture should be aligned are Amazon and Netflix. Early on, Amazon started to understand the benefits of teams owning the whole life cycle of the systems they managed. It wanted teams to own and operate the systems they looked after, managing the entire life cycle. But Amazon also knew that small teams can work faster than large teams. This led to its infamous two-pizza teams, where no team should be so big that it could not be fed by two pizzas. This is, of course, not a totally useful **metric**—we never find out if

we're eating pizza for lunch or dinner (or breakfast!), nor how big the pizzas are—but again, the general idea is that the optimum team size is 8–10 people, and that this team should be customer facing. This driver for small teams owning the whole life cycle of their services is a major reason why Amazon developed Amazon Web Services. It needed to create the tooling to allow its teams to be self-sufficient.

Có lẽ hai gương mặt tiêu biểu nhất cho ý tưởng rằng tổ chức và kiến trúc nên được giống hàng là Amazon và Netflix. Ngay từ sớm, Amazon đã bắt đầu hiểu được những lợi ích của việc các nhóm làm chủ toàn bộ vòng đời của các hệ thống mà họ quản lý. Hãng muốn các nhóm sở hữu và vận hành các hệ thống mà họ chăm sóc, quản lý toàn bộ vòng đời. Nhưng Amazon cũng biết rằng các nhóm nhỏ có thể làm việc nhanh hơn các nhóm lớn. Điều này dẫn đến các two-pizza team khét tiếng của hãng, theo đó không nhóm nào được lớn đến mức không thể nuôi bằng hai chiếc pizza. Tất nhiên, đây không phải là một thước đo hoàn toàn hữu ích—chúng ta không bao giờ biết được liệu mình đang ăn pizza vào bữa trưa hay bữa tối (hay bữa sáng!), cũng như những chiếc pizza đó to cỡ nào—nhưng một lần nữa, ý tưởng chung là quy mô nhóm tối ưu là 8–10 người, và nhóm này nên hướng tới khách hàng. Động lực này hướng tới các nhóm nhỏ làm chủ toàn bộ vòng đời của các dịch vụ của họ là một lý do chính khiến Amazon phát triển Amazon Web Services. Hãng cần tạo ra bộ công cụ để cho phép các nhóm của mình tự cung tự cấp.

3 Nachiappan Nagappan, Brendan Murphy, and Victor Basili, “The Influence of Organizational Structure on Software Quality: An Empirical Case Study” ICSE '08: Proceedings of the 30th International Conference on Software Engineering (New York: ACM, 2008). 4 And we all know Windows Vista was quite error-prone!

3 Nachiappan Nagappan, Brendan Murphy, and Victor Basili, “The Influence of Organizational Structure on Software Quality: An Empirical Case Study” ICSE '08: Proceedings of the 30th International Conference on Software Engineering (New York: ACM, 2008). 4 Và chúng ta đều biết Windows Vista khá dễ phát sinh lỗi!

Netflix learned from this example and ensured that from the beginning it structured itself around small, independent teams, so that the services they created would also be independent from each other. This ensured that the architecture of the system was optimized for speed of change. Effectively, Netflix designed the organizational structure for the **system architecture** it wanted. I've also heard that this extended into the seating plans for teams at Netflix—teams whose services would talk to each other would sit closely together. The idea being that you would want to have more frequent communication with teams that use your services, or whose services you yourself use.

Netflix đã học được từ ví dụ này và bảo đảm rằng ngay từ đầu họ tự tổ chức xoay quanh các nhóm nhỏ, độc lập, để các dịch vụ mà họ tạo ra cũng độc lập với nhau. Điều này bảo đảm rằng kiến trúc của hệ thống được tối ưu cho tốc độ thay đổi. Thực chất, Netflix đã thiết kế cấu trúc tổ chức cho kiến trúc hệ thống mà họ mong muốn. Tôi cũng từng nghe nói rằng điều này mở rộng đến cả sơ đồ chỗ ngồi cho các nhóm tại Netflix—các nhóm có dịch vụ giao tiếp với nhau sẽ ngồi gần nhau. Ý tưởng là bạn sẽ muốn giao tiếp thường xuyên hơn với các nhóm sử dụng dịch vụ của bạn, hoặc những nhóm mà bạn sử dụng dịch vụ của họ.

Team Size — Quy mô nhóm

Ask any developer how big a team should be, and while you'll get varying answers, there will be general consensus that, to a point, smaller is better. If you push them to put a number on the “ideal” team size, you'll largely get answers in the range of 5 to 10 people.

Hãy hỏi bất kỳ lập trình viên nào rằng một nhóm nên lớn cỡ nào, và mặc dù bạn sẽ nhận được những câu trả lời khác nhau, sẽ có sự đồng thuận chung rằng, đến một mức độ nào đó, nhỏ thì tốt hơn. Nếu bạn ép họ đưa ra một con số cho quy mô nhóm “lý tưởng”, bạn sẽ phần lớn nhận được các câu trả lời trong khoảng 5 đến 10 người.

I did some research into the best team size for software development. I found many studies, but a lot of them are flawed in such a way that drawing conclusions for the broader world of software development is too difficult. The best study I found, “Empirical Findings on Team Size and Productivity in Software Development,”⁵ was at least able to draw from a large corpus of data, albeit not one that is necessarily representative of software development as a whole. The researchers’ findings indicated that, “as expected from the literature, productivity is worst for those projects with (an average team size) larger or equal to 9 people.” This research at least appears to back up my own anecdotal experience.

Tôi đã thực hiện một số nghiên cứu về quy mô nhóm tốt nhất cho phát triển phần mềm. Tôi tìm thấy nhiều nghiên cứu, nhưng rất nhiều trong số đó có khiếm khuyết theo cách khiến việc rút ra kết luận cho thế giới phát triển phần mềm rộng lớn hơn trở nên quá khó. Nghiên cứu tốt nhất mà tôi tìm được, “Empirical Findings on Team Size and Productivity in Software Development,”⁵ ít nhất cũng có thể rút ra từ một kho dữ liệu lớn, mặc dù không nhất thiết là một kho dữ liệu đại diện cho phát triển phần mềm nói chung. Phát hiện của các nhà nghiên cứu chỉ ra rằng, “như mong đợi từ tài liệu, năng suất là tệ nhất đối với những dự án có (quy mô nhóm trung bình) lớn hơn hoặc bằng 9 người.” Nghiên cứu này ít nhất cũng có vẻ ủng hộ trải nghiệm giai thoại của chính tôi.

We like working in small teams, and it’s not hard to see why. With a small group of people all focused on the same outcomes, it’s easier to stay aligned, and easier to coordinate on work. I do have an (untested) hypothesis that being geographically dispersed or having large time-zone differences among team members will cause challenges that might further limit the optimum team size, but that thinking is likely better explored by someone other than me.

Chúng ta thích làm việc trong các nhóm nhỏ, và không khó để hiểu tại sao. Với một nhóm nhỏ tất cả đều tập trung vào cùng những kết quả, dễ dàng hơn để giữ sự giống hệt, và dễ dàng hơn để phối hợp công việc. Tôi có một giả thuyết (chưa được kiểm chứng) rằng việc phân tán về mặt địa lý hoặc có sự chênh lệch múi giờ lớn giữa các thành viên nhóm sẽ gây ra những thách thức có thể giới hạn thêm quy mô nhóm tối ưu, nhưng suy nghĩ đó có lẽ tốt hơn nên được khám phá bởi ai đó khác ngoài tôi.

So small teams good, large teams bad. That seems pretty straightforward. Now, if you can do all the work you need to do with a single team, then great! Your world is a simple one, and you could probably skip much of the **rest** of this chapter. But what if you have more work than you have time for? An obvious reaction to that is to add people. But as we know, adding people may not necessarily help you get more done.

Vậy nhóm nhỏ thì tốt, nhóm lớn thì xấu. Điều đó có vẻ khá đơn giản. Giờ đây, nếu bạn có thể làm tất cả công việc cần làm với một nhóm duy nhất, thì tuyệt vời! Thế giới của bạn là một thế giới đơn giản, và bạn có lẽ có thể bỏ qua phần lớn phần còn lại của chương này. Nhưng nếu bạn có nhiều công việc hơn thời gian bạn có thì sao? Một phản ứng hiển nhiên cho điều đó là thêm người. Nhưng như chúng ta đã biết, việc thêm người có thể không nhất thiết giúp bạn hoàn thành được nhiều hơn.

5 Daniel Rodriguez et al., “Empirical Findings on Team Size and Productivity in Software Development,” *Journal of Systems and Software* 85, no. 3 (2012), doi.org/10.1016/j.jss.2011.09.009.

5 Daniel Rodriguez et al., “Empirical Findings on Team Size and Productivity in Software

Understanding Conway’s Law — Hiểu Định luật Conway

Anecdotal and empirical evidence suggests our organizational structure has a strong influence on the nature (and quality) of the systems we create. We also know we want smaller teams too. So how does this understanding help us? Well, fundamentally, if we want a loosely coupled architecture to allow for changes to be made more easily, we want a loosely coupled organization too. Put a different way, the reason we often want a more loosely coupled organization is that we want different parts of the organization to be able to decide and act more quickly and efficiently, and a loosely coupled systems architecture helps hugely with that.

Bằng chứng giai thoại và thực nghiệm gợi ý rằng cấu trúc tổ chức của chúng ta có ảnh hưởng mạnh mẽ đến bản chất (và chất lượng) của các hệ thống mà chúng ta tạo ra. Chúng ta cũng biết rằng chúng ta muốn các nhóm nhỏ hơn. Vậy thì sự hiểu biết này giúp ích gì cho chúng ta? Về cơ bản, nếu chúng ta muốn một kiến trúc có coupling lỏng để cho phép thay đổi dễ dàng hơn, thì chúng ta cũng muốn một tổ chức có coupling lỏng. Nói cách khác, lý do chúng ta thường muốn một tổ chức có coupling lỏng hơn là vì chúng ta muốn các bộ phận khác nhau của tổ chức có thể quyết định và hành động nhanh chóng và hiệu quả hơn, và một kiến trúc hệ thống có coupling lỏng giúp ích rất nhiều cho điều đó.

In Accelerate , the authors found a significant correlation between those organizations that had loosely coupled architecture and their ability to more effectively make use of larger delivery teams:

Trong Accelerate, các tác giả phát hiện ra một mối tương quan đáng kể giữa những tổ chức có kiến trúc coupling lỏng và khả năng của họ trong việc sử dụng hiệu quả hơn các nhóm chuyển giao lớn hơn:

If we achieve a loosely coupled, well-encapsulated architecture with an organizational structure to match, two important things happen. First, we can achieve better delivery performance, increasing both tempo and stability while reducing the burnout and the pain of **deployment**. Second, we can substantially grow the size of our engineering organization and increase productivity linearly—or better than linearly—as we do so. Organizationally, the shift that has been coming for a while now, especially for organizations that operate at scale, is a move away from centralized command and control models. With centralized decision making, the speed at which our organization can react is significantly blunted. This is compounded as the organization grows—the bigger it becomes, the more its centralized nature reduces the efficiency of decision making and the speed of action.

Nếu chúng ta đạt được một kiến trúc có coupling lỏng, được đóng gói (encapsulate) tốt cùng với một cấu trúc tổ chức tương ứng, thì hai điều quan trọng sẽ xảy ra. Thứ nhất, chúng ta có thể đạt được hiệu suất chuyển giao tốt hơn, tăng cả nhịp độ lẫn sự ổn định trong khi giảm tình trạng kiệt sức và nỗi đau khi triển khai. Thứ hai, chúng ta có thể tăng đáng kể quy mô của tổ chức kỹ thuật và tăng năng suất một cách tuyến tính—hoặc tốt hơn tuyến tính—khi chúng ta làm vậy. Về mặt tổ chức, sự dịch chuyển đã diễn ra một thời gian nay, đặc biệt là đối với các tổ chức hoạt động ở quy mô lớn, là một sự dịch chuyển ra khỏi các mô hình chỉ huy và kiểm soát tập trung. Với việc ra quyết định tập trung, tốc độ mà tổ chức của chúng ta có thể phản ứng bị làm cùn đi đáng kể. Điều này càng trầm trọng hơn khi tổ chức phát triển—càng lớn, bản chất tập trung của nó càng làm giảm hiệu quả của việc ra quyết định và tốc độ hành động.

Organizations have increasingly recognized that if you want to scale your organization but still want to move quickly, you need to distribute responsibility more effectively, breaking down central

decision making and pushing decisions into parts of the organization that can operate with increased autonomy.

Các tổ chức ngày càng nhận ra rằng nếu bạn muốn mở rộng tổ chức nhưng vẫn muốn di chuyển nhanh, bạn cần phân bổ trách nhiệm hiệu quả hơn, phá bỏ việc ra quyết định tập trung và đẩy các quyết định vào những bộ phận của tổ chức có thể hoạt động với mức độ tự chủ cao hơn.

The trick, then, is to create large organizations out of smaller, autonomous teams.

Mẹo, do đó, là tạo ra các tổ chức lớn từ những nhóm nhỏ hơn, tự chủ.

Small Teams, Large Organization — Nhóm nhỏ, tổ chức lớn

Adding manpower to a late software project makes it later. —Fred Brooks (Brooks's Law)

Thêm nhân lực vào một dự án phần mềm đang trễ sẽ làm nó càng trễ hơn. —Fred Brooks (Định luật Brooks)

In his famous essay, “The Mythical Man-Month,”⁶ author Fred Brooks attempts to explain why using “a man-month” as an estimation technique is problematic, due to

Trong tiểu luận nổi tiếng của mình, “The Mythical Man-Month,”⁶ tác giả Fred Brooks cố gắng giải thích tại sao việc sử dụng “một người-tháng” làm kỹ thuật ước lượng lại có vấn đề, do

⁶ Frederick P. Brooks Jr, The Mythical Man-Month: Essays on Software Engineering , Anniversary ed. (Boston: Addison-Wesley, 1995).

⁶ Frederick P. Brooks Jr, The Mythical Man-Month: Essays on Software Engineering, Anniversary ed. (Boston: Addison-Wesley, 1995).

the fact it causes us to fall into the trap of thinking we can throw more people at the problem to go faster. The theory goes like this: if a piece of work will take a developer six months, then if we add a second developer, it will only take three months. If we add five developers, so we now have six in total, the work should be done in just one month! Of course, software doesn't work like that.

thực tế là nó khiến chúng ta rơi vào cái bẫy nghĩ rằng có thể ném thêm người vào vấn đề để đi nhanh hơn. Lý thuyết diễn ra như sau: nếu một phần công việc sẽ mất của một lập trình viên sáu tháng, thì nếu chúng ta thêm một lập trình viên thứ hai, nó sẽ chỉ mất ba tháng. Nếu chúng ta thêm năm lập trình viên, để giờ chúng ta có tổng cộng sáu người, công việc sẽ được hoàn thành chỉ trong một tháng! Tất nhiên, phần mềm không hoạt động như vậy.

For you to be able to throw more people (or teams) at a problem to go faster, the work needs to be able to be split apart into tasks that can be worked on, to some degree, in parallel. If one developer is doing some work that another developer is waiting for, the work cannot be done in parallel, it must be done sequentially. Even if the work can be done in parallel, there is often a need to coordinate between the people doing the different streams of work, resulting in additional overhead. The more intertwined the work, the less effective it is to add more people.

Để bạn có thể ném thêm người (hoặc nhóm) vào một vấn đề để đi nhanh hơn, công việc cần phải có khả năng được chia tách thành các tác vụ mà ở một mức độ nào đó có thể được làm song song. Nếu một lập trình viên đang làm một việc nào đó mà một lập trình viên khác đang chờ đợi, thì công việc không thể được làm song song, nó phải được làm tuần tự. Ngay cả khi công việc có thể được làm song song, thường vẫn cần phối hợp giữa những

người đang làm các luồng công việc khác nhau, dẫn đến chi phí phụ trội (overhead). Công việc càng đan xen vào nhau, việc thêm người càng kém hiệu quả.

If you cannot break the work apart into subtasks that can be worked on independently, then you cannot just throw people at the problem. Worse, doing so will likely make you slower—adding new people or spinning up new teams has a cost. Time is needed to help these people be fully productive, and often the developers who have too much work to do are the same developers who will need to spend time helping get people up to speed.

Nếu bạn không thể chia tách công việc thành các tác vụ con có thể được làm độc lập, thì bạn không thể chỉ đơn giản ném người vào vấn đề. Tệ hơn, làm vậy nhiều khả năng sẽ khiến bạn chậm hơn—việc thêm người mới hoặc khởi tạo các nhóm mới đều có chi phí. Cần có thời gian để giúp những người này trở nên đầy đủ năng suất, và thường thì những lập trình viên có quá nhiều việc phải làm lại chính là những lập trình viên sẽ cần dành thời gian giúp người mới bắt nhịp.

The biggest cost to working efficiently at scale in software delivery is the need for coordination. The more coordination between teams working on different tasks, the slower you will be. Amazon, as a company, has recognized this, and structures itself in a way to reduce the need for coordination between its small two-pizza teams. In fact there has been a conscious drive to limit the amount of coordination between teams for this very reason, and restrict this coordination, where possible, to areas where it was absolutely required—between teams that share a boundary between microservices. From Think Like Amazon , 7 by ex-Amazon executive John Rossman:

Chi phí lớn nhất để làm việc hiệu quả ở quy mô lớn trong chuyển giao phần mềm là nhu cầu phối hợp. Càng phối hợp nhiều giữa các nhóm làm việc trên các tác vụ khác nhau, bạn sẽ càng chậm. Amazon, với tư cách là một công ty, đã nhận ra điều này, và tự tổ chức theo cách giảm bớt nhu cầu phối hợp giữa các two-pizza team nhỏ của mình. Trên thực tế đã có một nỗ lực có ý thức nhằm giới hạn lượng phối hợp giữa các nhóm chính vì lý do này, và hạn chế sự phối hợp này, ở nơi có thể, vào những khu vực mà nó tuyệt đối cần thiết—giữa các nhóm chia sẻ một ranh giới giữa các microservice. Trích từ Think Like Amazon, 7 của John Rossman, cựu lãnh đạo của Amazon:

The Two-Pizza Team is autonomous. Interaction with other teams is limited, and when it does occur, it is well documented, and interfaces are clearly defined. It owns and is responsible for every aspect of its systems. One of the primary goals is to lower the communications overhead in organizations, including the number of meetings, coordination points, planning, testing, or releases. Teams that are more independent move faster. Working out how a team fits into the larger organization is essential. Team Topologies defines the concept of a team **API** , which defines broadly how that team interacts with the rest of the organization, not just in terms of microservice interfaces but also in terms of working practices: 8

Two-Pizza Team là tự chủ. Tương tác với các nhóm khác bị giới hạn, và khi nó xảy ra, nó được tài liệu hóa tốt, và các interface được định nghĩa rõ ràng. Nhóm sở hữu và chịu trách nhiệm cho mọi khía cạnh của các hệ thống của mình. Một trong những mục tiêu chính là hạ thấp chi phí phụ trội về giao tiếp trong các tổ chức, bao gồm số lượng cuộc họp, các điểm phối hợp, lập kế hoạch, kiểm thử, hoặc phát hành. Các nhóm độc lập hơn thì di chuyển nhanh hơn. Việc tìm ra cách một nhóm khớp vào tổ chức lớn hơn là điều thiết yếu. Team Topologies định nghĩa khái niệm về một team API, thứ định nghĩa một cách tổng quát cách nhóm đó tương tác với phần còn lại của tổ chức, không chỉ về mặt các interface microservice mà còn về mặt các thực hành làm việc: 8

7 John Rossman, Think Like Amazon: 50 1/2 Ideas to Become a Digital Leader (New York: McGraw-Hill, 2019).

7 John Rossman, *Think Like Amazon: 50 1/2 Ideas to Become a Digital Leader* (New York: McGraw-Hill, 2019).

8 Matthew Skelton and Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).

8 Matthew Skelton and Manuel Pais, *Team Topologies* (Portland, OR: IT Revolution, 2019).

The team API should explicitly consider usability by other teams. Will other teams find it easy and straightforward to interact with us, or will it be difficult and confusing? How easy will it be for a new team to get on board with our code and working practices? How do we respond to pull requests and suggestions from other teams? Is our team backlog and product roadmap easily visible and understandable by other teams?

Team API nên cân nhắc một cách tường minh về khả năng dễ sử dụng bởi các nhóm khác. Liệu các nhóm khác có thấy việc tương tác với chúng ta dễ dàng và đơn giản, hay nó sẽ khó khăn và rối rắm? Một nhóm mới sẽ dễ dàng thế nào để bắt nhịp với mã và các thực hành làm việc của chúng ta? Chúng ta phản hồi các pull request và đề xuất từ các nhóm khác như thế nào? Backlog của nhóm và lộ trình sản phẩm của chúng ta có dễ thấy và dễ hiểu đối với các nhóm khác không?

On Autonomy — Về tính tự chủ

Whatever industry you operate in, it is all about your people, and catching them doing things right, and providing them with the confidence, the motivation, the freedom and desire to achieve their true potential. —John Timpson

Dù bạn hoạt động trong ngành nào, tất cả đều xoay quanh con người của bạn, và việc bắt gặp họ làm những điều đúng đắn, và mang lại cho họ sự tự tin, động lực, sự tự do và khát khao để đạt được tiềm năng thực sự của mình. —John Timpson

Having lots of small teams isn't by itself going to help if these teams just become more silos that are still dependent on other teams to get things done. We need to ensure that each of these small teams has the autonomy to do the job it is responsible for. This means we need to give the teams more power to make decisions, and the tools to ensure they can get as much done as possible without having to constantly coordinate work with other teams. So enabling autonomy is key.

Việc có nhiều nhóm nhỏ tự thân nó sẽ không giúp ích gì nếu những nhóm này chỉ trở thành nhiều silo hơn vẫn còn phụ thuộc vào các nhóm khác để hoàn thành công việc. Chúng ta cần bảo đảm rằng mỗi nhóm nhỏ này có tính tự chủ để làm công việc mà nó chịu trách nhiệm. Điều này có nghĩa là chúng ta cần trao cho các nhóm nhiều quyền hơn để ra quyết định, và các công cụ để bảo đảm họ có thể hoàn thành càng nhiều việc càng tốt mà không phải liên tục phối hợp công việc với các nhóm khác. Vậy nên việc trao tính tự chủ là then chốt.

Many organizations have shown the benefits of creating autonomous teams. Keeping organizational groups small, allowing them to **build** close bonds and work effectively together without bringing in too much bureaucracy, has helped many organizations grow and scale more effectively than some of their peers. W. L. Gore and Associates 9 has found great success by making sure none of its business units ever gets to more than 150 people, to make sure that everyone knows each other. For these smaller business units to work, they have to be given power and responsibility to work as autonomous units.

Nhiều tổ chức đã cho thấy những lợi ích của việc tạo ra các nhóm tự chủ. Việc giữ các nhóm trong tổ chức ở quy mô nhỏ, cho phép họ xây dựng những mối ràng buộc khăng

khít và làm việc với nhau hiệu quả mà không mang vào quá nhiều thủ tục hành chính quan liêu, đã giúp nhiều tổ chức phát triển và mở rộng hiệu quả hơn so với một số đối thủ của họ. W. L. Gore and Associates⁹ đã tìm thấy thành công lớn bằng cách bảo đảm rằng không đơn vị kinh doanh nào của họ vượt quá 150 người, để chắc chắn rằng mọi người đều biết nhau. Để các đơn vị kinh doanh nhỏ hơn này hoạt động, chúng phải được trao quyền lực và trách nhiệm để làm việc như những đơn vị tự chủ.

Many of these organizations seem to have drawn from the work done by anthropologist Robin Dunbar, who looked at humans' ability to form social groupings. His theory was that our cognitive capacity places limits on how effectively we can maintain different forms of social relationships. He estimated 150 people as how big a group could grow to before it would need to split off or else it would collapse under its own weight.

Nhiều tổ chức trong số này dường như đã rút ra từ công trình được thực hiện bởi nhà nhân học Robin Dunbar, người đã nghiên cứu khả năng của con người trong việc hình thành các nhóm xã hội. Lý thuyết của ông là khả năng nhận thức của chúng ta đặt giới hạn lên việc chúng ta có thể duy trì các hình thức quan hệ xã hội khác nhau hiệu quả đến đâu. Ông ước tính 150 người là quy mô mà một nhóm có thể phát triển đến trước khi nó cần phải tách ra nếu không sẽ sụp đổ dưới chính sức nặng của mình.

Timpson, a highly successful UK retailer, has achieved massive scale by empowering its workforce, reducing the need for central functions, and allowing local stores to make decisions for themselves, such as how much to refund unhappy customers. Now chairman of the company, John Timpson is famous for scrapping internal rules and replacing them with just two:

Timpson, một nhà bán lẻ Anh rất thành công, đã đạt được quy mô lớn bằng cách trao quyền cho lực lượng lao động của mình, giảm bớt nhu cầu về các chức năng trung tâm, và cho phép các cửa hàng địa phương tự đưa ra quyết định, chẳng hạn như hoàn tiền bao nhiêu cho những khách hàng không hài lòng. Giờ đây là chủ tịch của công ty, John Timpson nổi tiếng với việc loại bỏ các quy tắc nội bộ và thay chúng chỉ bằng hai quy tắc:

9 Famous for the development of the waterproof material Gore-Tex.

9 Nổi tiếng với việc phát triển vật liệu chống thấm Gore-Tex.

- Look the part.
- Put money in the till.
 - Trông cho ra dáng.
 - Bỏ tiền vào quầy thu ngân.

Autonomy works at the smaller scale too, and most modern companies I work with are looking to create more autonomous teams within their organizations, often trying to copy models from other organizations, such as Amazon's two-pizza team model, or the "Spotify model" that popularized the concept of guilds and chapters. 10 I should sound a note of caution here, of course—by all means, learn from what other organizations do, but understand that copying what someone else does and expecting the same results, without actually understanding why the other organization does the things it does, may not result in the outcome you want.

Tính tự chủ cũng hoạt động ở quy mô nhỏ hơn, và hầu hết các công ty hiện đại mà tôi làm việc cùng đều đang tìm cách tạo ra nhiều nhóm tự chủ hơn trong tổ chức của mình, thường cố gắng sao chép các mô hình từ các tổ chức khác, chẳng hạn như mô hình two-pizza team của Amazon, hay "mô hình Spotify" đã phổ biến hóa khái niệm guild và chapter.¹⁰ Tất nhiên, ở đây tôi cần đưa ra một lời cảnh báo—cứ học hỏi từ những gì các tổ chức khác làm, nhưng hãy hiểu rằng việc sao chép những gì người khác làm và kỳ vọng kết quả giống hệt,

mà không thực sự hiểu tại sao tổ chức kia làm những điều họ làm, có thể không mang lại kết quả mà bạn muốn.

If done right, team autonomy can empower people, help them step up and grow, and get the job done faster. When teams own microservices and have full control over those microservices, they can have greater autonomy within a larger organization.

Nếu được làm đúng, tính tự chủ của nhóm có thể trao quyền cho con người, giúp họ vươn lên và trưởng thành, và hoàn thành công việc nhanh hơn. Khi các nhóm sở hữu microservice và có toàn quyền kiểm soát các microservice đó, họ có thể có tính tự chủ lớn hơn trong một tổ chức lớn hơn.

The concept of autonomy starts to shift our understanding of ownership in a microservice architecture. Let's explore this in more detail.

Khái niệm tự chủ bắt đầu làm dịch chuyển hiểu biết của chúng ta về quyền sở hữu trong một kiến trúc microservice. Hãy khám phá điều này chi tiết hơn.

Strong Versus Collective Ownership — Quyền sở hữu mạnh so với quyền sở hữu tập thể

In “Defining ownership” on page 213, we discussed different types of ownership and explored the implications of these styles of ownership in the context of making code changes. As a brief recap, the two primary forms of code ownership we described are:

Trong “Defining ownership” ở trang 213, chúng ta đã thảo luận về các loại quyền sở hữu khác nhau và khám phá những hàm ý của các phong cách quyền sở hữu này trong bối cảnh thực hiện các thay đổi mã. Tóm tắt nhanh, hai hình thức chính của quyền sở hữu mã mà chúng ta đã mô tả là:

Strong ownership A microservice is owned by a team, and that team decides what changes to make to that microservice. If an external team wants to make changes, it either needs to ask the owning team to make the change on its behalf or else might be required to send a pull request—it would then be entirely up to the owning team to decide under what circumstances the pull request model is accepted. A team may own more than one microservice.

Quyền sở hữu mạnh (strong ownership) Một microservice được sở hữu bởi một nhóm, và nhóm đó quyết định những thay đổi nào sẽ được thực hiện đối với microservice đó. Nếu một nhóm bên ngoài muốn thực hiện thay đổi, nó hoặc cần yêu cầu nhóm sở hữu thực hiện thay đổi thay cho mình hoặc có thể được yêu cầu gửi một pull request—khi đó hoàn toàn tùy thuộc vào nhóm sở hữu quyết định trong những hoàn cảnh nào thì mô hình pull request được chấp nhận. Một nhóm có thể sở hữu nhiều hơn một microservice.

Collective ownership Any team can change any microservice. Careful coordination is needed to ensure that teams don't get in each other's way.

Quyền sở hữu tập thể (collective ownership) Bất kỳ nhóm nào cũng có thể thay đổi bất kỳ microservice nào. Cần phối hợp cẩn thận để bảo đảm rằng các nhóm không cản trở lẫn nhau.

Let's further explore the implications of these ownership models and how they can help (or hinder) a drive toward increased team autonomy.

Hãy khám phá thêm những hàm ý của các mô hình quyền sở hữu này và cách chúng có thể giúp ích (hoặc cản trở) một nỗ lực hướng tới việc tăng tính tự chủ của nhóm.

10 Which famously even Spotify doesn't use anymore.

10 Mà nổi tiếng là ngay cả Spotify cũng không còn dùng nữa.

Strong Ownership — Quyền sở hữu mạnh

With strong ownership, the team owning the microservice calls the shots. At the most basic level, it is in full control of what code changes are made. Taken further, the team may be able to decide on coding standards, programming idioms, when to deploy the software, what technology is used to build the microservice, the deployment platform, and more. By having more responsibility for the changes that happen to the software, teams with strong ownership will have a higher degree of autonomy, with all the benefits that entails.

Với quyền sở hữu mạnh, nhóm sở hữu microservice là người ra quyết định. Ở mức cơ bản nhất, nó toàn quyền kiểm soát những thay đổi mã nào được thực hiện. Đi xa hơn, nhóm có thể quyết định về các tiêu chuẩn lập trình, các thành ngữ lập trình (programming idiom), khi nào triển khai phần mềm, công nghệ nào được dùng để xây dựng microservice, nền tảng triển khai, và hơn thế nữa. Bằng cách có nhiều trách nhiệm hơn đối với những thay đổi xảy ra với phần mềm, các nhóm có quyền sở hữu mạnh sẽ có mức độ tự chủ cao hơn, cùng với tất cả những lợi ích đi kèm.

Strong ownership is ultimately all about optimizing for that team's autonomy. Coming back to the Amazon way of doing things from Think Like Amazon :

Quyền sở hữu mạnh xét cho cùng là về việc tối ưu cho tính tự chủ của nhóm đó. Quay lại với cách làm của Amazon trong Think Like Amazon:

When it comes to Amazon's famous Two-Pizza Teams, most people miss the point. It's not about the team's size. It's about the team's autonomy, accountability, and entrepreneurial mindset. The Two-Pizza Team is about equipping a small team within an organization to operate independently and with agility. Strong ownership models can allow for more local variation. You may be relaxed, for example, about one team deciding to create its microservice in a functional style of Java, as that is a decision that should only impact them. Of course, this variation does need to be tempered somewhat, as some decisions warrant a degree of consistency around them. For example, if everyone else makes use of REST-over-HTTP-based APIs for their microservice endpoints but you decide to use GRPC, you may be causing some issues for other people who want to make use of your microservice. On the other hand, if that GRPC endpoint was only used internally within your team, this might not be problematic. So when making decisions locally that have impact on other teams, coordination may still be needed. Working out when and how to engage a wider organization is something we'll explore shortly when we look at balancing local versus global optimization.

Khi nói đến các Two-Pizza Team nổi tiếng của Amazon, hầu hết mọi người bỏ lỡ trọng điểm. Nó không phải về quy mô của nhóm. Nó là về tính tự chủ, trách nhiệm giải trình, và tư duy khởi nghiệp của nhóm. Two-Pizza Team là về việc trang bị cho một nhóm nhỏ trong một tổ chức khả năng hoạt động độc lập và linh hoạt. Các mô hình quyền sở hữu mạnh có thể cho phép nhiều biến thể cục bộ hơn. Chẳng hạn, bạn có thể thoải mái về việc một nhóm quyết định tạo microservice của mình theo phong cách hàm (functional) của Java, vì đó là một quyết định chỉ nên ảnh hưởng đến họ. Tất nhiên, sự biến thể này cần phải được điều tiết phần nào, vì một số quyết định cần một mức độ nhất quán xung quanh chúng. Ví dụ, nếu mọi người khác đều sử dụng các API dựa trên REST-over-HTTP cho các endpoint microservice của mình nhưng bạn quyết định dùng GRPC, bạn có thể đang gây ra một số vấn đề cho những người khác muốn sử dụng microservice của bạn. Mặt khác,

nếu endpoint GRPC đó chỉ được dùng nội bộ trong nhóm của bạn, điều này có thể không thành vấn đề. Vậy nên khi đưa ra các quyết định cục bộ có ảnh hưởng đến các nhóm khác, sự phối hợp có thể vẫn cần thiết. Việc tìm ra khi nào và làm thế nào để lôi kéo một tổ chức rộng hơn vào cuộc là điều chúng ta sẽ khám phá sớm thôi khi xem xét việc cân bằng giữa tối ưu cục bộ và tối ưu toàn cục.

Fundamentally, the stronger the ownership model a team can adopt, the less coordination is needed, and therefore the more productive the team can be.

Về cơ bản, mô hình quyền sở hữu mà một nhóm có thể áp dụng càng mạnh, thì càng ít cần phối hợp, và do đó nhóm càng có thể làm việc năng suất hơn.

How far does strong ownership go?

Quyền sở hữu mạnh đi xa đến đâu?

Up until this stage, we've talked primarily about aspects like making code changes or choosing technology. But the concept of ownership can go much deeper. Some organizations adopt a model that I describe as full life-cycle ownership. With full life-cycle ownership, a single team comes up with the design, makes the changes, deploys the microservice, manages it in production, and ultimately decommissions the microservice when it is no longer required.

Cho đến giai đoạn này, chúng ta chủ yếu nói về các khía cạnh như thực hiện thay đổi mã hoặc lựa chọn công nghệ. Nhưng khái niệm quyền sở hữu có thể đi sâu hơn nhiều. Một số tổ chức áp dụng một mô hình mà tôi mô tả là quyền sở hữu trọn vòng đời (full life-cycle ownership). Với quyền sở hữu trọn vòng đời, một nhóm duy nhất đưa ra thiết kế, thực hiện các thay đổi, triển khai microservice, quản lý nó trong môi trường production, và cuối cùng ngừng hoạt động (decommission) microservice khi nó không còn cần thiết nữa.

This full life-cycle ownership model further increases the autonomy the team has, as the requirements for external coordination are reduced. Tickets aren't raised with operations teams to get things deployed, no external parties sign off on changes, and the team decides what changes are to be made and when to ship.

Mô hình quyền sở hữu trọn vòng đời này làm tăng thêm tính tự chủ mà nhóm có, vì các yêu cầu về phối hợp bên ngoài được giảm bớt. Không có ticket nào được mở với các nhóm vận hành (operations) để có được thứ gì đó được triển khai, không có bên ngoài nào ký duyệt thay đổi, và nhóm quyết định những thay đổi nào sẽ được thực hiện và khi nào ship.

For many of you, such a model may be fanciful, as you already have a number of existing procedures in place regarding how things must be done. You may also not have the right skills in the team to take on full ownership, or you may require new tooling (for example, self-service deployment mechanisms). It's worth noting, of course, that you don't get to full life-cycle ownership overnight, even if you consider it to be an aspirational goal. Don't be surprised if this change can take years to fully embrace, especially with a larger organization. Many aspects of full life-cycle ownership can require cultural changes and significant adjustments in terms of the expectations you have of some of your people, such as the need to support their software outside office hours. But as you can pull more responsibility for aspects of your microservice into your team, you will further increase the autonomy you have.

Đối với nhiều bạn, một mô hình như vậy có thể là viễn vọng, vì bạn đã có sẵn một số quy trình hiện hữu liên quan đến cách mọi thứ phải được thực hiện. Bạn cũng có thể không có đúng các kỹ năng trong nhóm để đảm nhận quyền sở hữu trọn vẹn, hoặc bạn có thể cần công cụ mới (ví dụ, các cơ chế triển khai tự phục vụ). Tất nhiên, cũng đáng lưu ý rằng bạn không đạt được quyền sở hữu trọn vòng đời chỉ sau một đêm, ngay cả khi bạn coi nó là

một mục tiêu đầy khát vọng. Đừng ngạc nhiên nếu sự thay đổi này có thể mất nhiều năm để đón nhận đầy đủ, đặc biệt là với một tổ chức lớn hơn. Nhiều khía cạnh của quyền sở hữu trọn vòng đời có thể đòi hỏi những thay đổi về văn hóa và những điều chỉnh đáng kể về mặt kỳ vọng mà bạn đặt vào một số người của mình, chẳng hạn như nhu cầu hỗ trợ phần mềm của họ ngoài giờ làm việc. Nhưng khi bạn có thể kéo nhiều trách nhiệm hơn cho các khía cạnh của microservice vào nhóm của mình, bạn sẽ càng làm tăng thêm tính tự chủ mà mình có.

I don't want to suggest that this model is in any way essential to using microservices—I am a firm believer that strong ownership for multiteam organizations is the most sensible model to get the most out of microservices. A strong ownership model around code changes is a good place to start—you can work to move toward full life-cycle ownership over time.

Tôi không muốn gợi ý rằng mô hình này theo bất kỳ cách nào là thiết yếu để sử dụng microservice—tôi tin chắc rằng quyền sở hữu mạnh đối với các tổ chức nhiều nhóm là mô hình hợp lý nhất để tận dụng tối đa microservice. Một mô hình quyền sở hữu mạnh xoay quanh các thay đổi mã là một điểm khởi đầu tốt—bạn có thể làm việc để dần dần hướng tới quyền sở hữu trọn vòng đời theo thời gian.

Collective Ownership — Quyền sở hữu tập thể

With a collective ownership model, a microservice can be changed by any one of a number of teams. One of the main benefits of collective ownership is that you can move people where they are needed. This can be useful if your bottleneck in terms of delivery is caused by a lack of people. For example, several changes require some updates to be made to the Payment microservice to allow for automated monthly billing—you could just assign extra people to implement this change. Of course, throwing people at a problem doesn't always make you go faster, but with collective ownership you certainly do have more flexibility in this regard.

Với một mô hình quyền sở hữu tập thể, một microservice có thể được thay đổi bởi bất kỳ nhóm nào trong một số nhóm. Một trong những lợi ích chính của quyền sở hữu tập thể là bạn có thể di chuyển người đến nơi họ được cần. Điều này có thể hữu ích nếu nút thắt cổ chai của bạn về mặt chuyển giao là do thiếu người. Ví dụ, một số thay đổi đòi hỏi phải thực hiện một số cập nhật đối với microservice Payment để cho phép lập hóa đơn hằng tháng tự động—bạn có thể chỉ cần phân bổ thêm người để triển khai thay đổi này. Tất nhiên, việc ném người vào một vấn đề không phải lúc nào cũng khiến bạn đi nhanh hơn, nhưng với quyền sở hữu tập thể bạn chắc chắn có nhiều linh hoạt hơn về mặt này.

With teams—and people—moving more frequently from microservice to microservice, we require a higher degree of consistency about how things are done. You can't afford a broad range of technology choices or different types of deployment models if you expect a developer to work on a different microservice each week. To get any degree of effectiveness out of a collective ownership model with microservices, you will end up needing to ensure that working on one microservice is much the same as working on any other.

Với các nhóm—và con người—di chuyển thường xuyên hơn từ microservice này sang microservice khác, chúng ta đòi hỏi một mức độ nhất quán cao hơn về cách mọi thứ được thực hiện. Bạn không thể kham nổi một dải rộng các lựa chọn công nghệ hoặc các loại mô hình triển khai khác nhau nếu bạn kỳ vọng một lập trình viên làm việc trên một microservice khác mỗi tuần. Để có được bất kỳ mức độ hiệu quả nào từ một mô hình quyền sở hữu tập thể với microservice, cuối cùng bạn sẽ cần bảo đảm rằng làm việc trên một microservice gần như giống hệt với làm việc trên bất kỳ microservice nào khác.

Inherently, this can undermine one of the key benefits of microservices. Coming back to a quote we used at the start of the book, James Lewis has said that “microservices buy you options.” With a more collective ownership model, you are likely going to need to reduce options in order to introduce a stronger degree of consistency across what teams do and how microservices are implemented.

Về bản chất, điều này có thể làm xói mòn một trong những lợi ích then chốt của microservice. Quay lại một câu trích dẫn mà chúng ta đã dùng ở đầu cuốn sách, James Lewis từng nói rằng “microservice mua cho bạn các lựa chọn (options).” Với một mô hình quyền sở hữu tập thể hơn, bạn nhiều khả năng sẽ cần giảm bớt các lựa chọn để đưa vào một mức độ nhất quán mạnh mẽ hơn xuyên suốt những gì các nhóm làm và cách các microservice được triển khai.

Collective ownership requires a high degree of coordination among individuals and among the teams those individuals are in. This higher degree of coordination results in an increased degree of **coupling** at an organizational level. Coming back to the paper by MacCormack et al., mentioned at the beginning of this chapter, we have this observation:

Quyền sở hữu tập thể đòi hỏi một mức độ phối hợp cao giữa các cá nhân và giữa các nhóm mà những cá nhân đó thuộc về. Mức độ phối hợp cao hơn này dẫn đến mức độ coupling tăng lên ở cấp độ tổ chức. Quay lại với bài báo của MacCormack và cộng sự, được đề cập ở đầu chương này, chúng ta có quan sát sau:

In tightly-coupled organizations...even if not an explicit managerial choice, the design naturally becomes more tightly-coupled. More coordination can lead to more organizational coupling which in turn leads to more coupled system designs. Microservices work best when they can fully embrace the concept of **independent deployability**—and a tightly coupled architecture will be the opposite of what you want.

Trong các tổ chức coupling chặt... ngay cả khi đó không phải là một lựa chọn quản lý tường minh, thiết kế tự nhiên trở nên coupling chặt hơn. Phối hợp nhiều hơn có thể dẫn đến coupling tổ chức nhiều hơn, điều này lại dẫn đến các thiết kế hệ thống coupling hơn. Microservice hoạt động tốt nhất khi chúng có thể đón nhận trọn vẹn khái niệm khả năng triển khai độc lập—và một kiến trúc coupling chặt sẽ là điều trái ngược với những gì bạn muốn.

If you have a small number of developers, and perhaps only a single team, a collective ownership model may be totally fine. But as the number of developers increase, the fine-grained coordination needed to make collective ownership work will end up becoming a significant negative factor in terms of getting the benefits of adopting microservice architectures.

Nếu bạn có một số lượng nhỏ lập trình viên, và có lẽ chỉ một nhóm duy nhất, một mô hình quyền sở hữu tập thể có thể hoàn toàn ổn. Nhưng khi số lượng lập trình viên tăng lên, sự phối hợp chi tiết cần thiết để làm cho quyền sở hữu tập thể vận hành cuối cùng sẽ trở thành một yếu tố tiêu cực đáng kể về mặt thu được những lợi ích của việc áp dụng kiến trúc microservice.

At a Team Level Versus an Organizational Level — Ở cấp độ nhóm so với ở cấp độ tổ chức

The concepts of strong and collective ownership can be applied at different levels of an organization. Within a team, you want people to be on the same page, able to efficiently collaborate with each other, and you want to ensure a high degree of collective ownership as a result. As an example, this would manifest itself in terms of all members of a team being able to directly make changes to the

codebase. A poly-skilled team focused on the end-to-end delivery of customer-facing software will need to be very good at having collective ownership. At an organizational level, if you want teams to have a high degree of autonomy, then it is important that they also have a strong ownership model.

Các khái niệm quyền sở hữu mạnh và tập thể có thể được áp dụng ở các cấp độ khác nhau của một tổ chức. Trong một nhóm, bạn muốn mọi người đồng nhịp với nhau, có thể cộng tác hiệu quả với nhau, và bạn muốn bảo đảm một mức độ quyền sở hữu tập thể cao là kết quả. Lấy ví dụ, điều này sẽ biểu hiện ở việc tất cả các thành viên của một nhóm có thể trực tiếp thực hiện các thay đổi đối với codebase. Một nhóm đa kỹ năng (poly-skilled) tập trung vào việc chuyển giao đầu-cuối phần mềm hướng tới khách hàng sẽ cần phải rất giỏi trong việc có quyền sở hữu tập thể. Ở cấp độ tổ chức, nếu bạn muốn các nhóm có mức độ tự chủ cao, thì điều quan trọng là họ cũng có một mô hình quyền sở hữu mạnh.

Balancing Models — Cân bằng các mô hình

Ultimately, the more you tend toward collective ownership, the more important it becomes to have consistency in how things are done. The more your organization tends toward strong ownership, the more you can allow for local optimization, as we see in Figure 15-1. This balance doesn't need to be fixed—you will likely shift this at different times and around different factors. You may, for example, give teams full freedom around choosing a programming language but still require them to deploy onto the same cloud platform for example.

Xét cho cùng, bạn càng nghiêng về quyền sở hữu tập thể, thì việc có sự nhất quán trong cách mọi thứ được thực hiện càng trở nên quan trọng. Tổ chức của bạn càng nghiêng về quyền sở hữu mạnh, bạn càng có thể cho phép tối ưu cục bộ, như chúng ta thấy trong Hình 15-1. Sự cân bằng này không cần phải cố định—bạn nhiều khả năng sẽ dịch chuyển nó ở những thời điểm khác nhau và xoay quanh các yếu tố khác nhau. Chẳng hạn, bạn có thể cho các nhóm hoàn toàn tự do trong việc chọn một ngôn ngữ lập trình nhưng vẫn yêu cầu họ triển khai lên cùng một nền tảng đám mây (cloud platform).

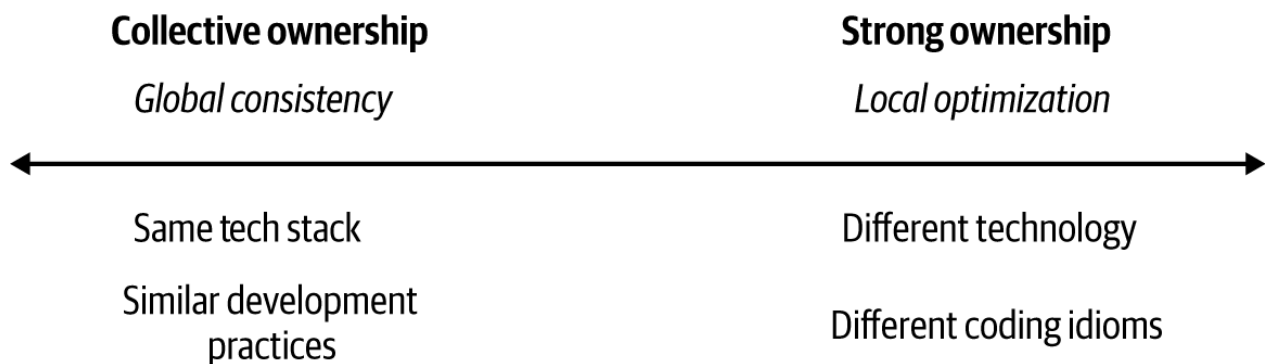


Figure 15-1. The balance between global consistency and local optimization

Hình 15-1. Sự cân bằng giữa nhất quán toàn cục và tối ưu cục bộ

Fundamentally, though, with a collective ownership model you'll nearly always be forced toward the left end of this spectrum—that is, toward requiring a higher degree of global consistency. In my experience, organizations that get the most out of microservices are constantly trying to find ways to shift the balance more to the right. That said, for the best organizations this isn't something that is set in stone but is more something that is constantly being evaluated.

Tuy nhiên, về cơ bản, với một mô hình quyền sở hữu tập thể bạn gần như luôn bị buộc về

phía đầu bên trái của phổ này—tức là, về phía đòi hỏi một mức độ nhất quán toàn cục cao hơn. Theo kinh nghiệm của tôi, các tổ chức tận dụng tối đa microservice liên tục cố gắng tìm cách dịch chuyển sự cân bằng nhiều hơn về phía bên phải. Như đã nói, đối với những tổ chức tốt nhất thì đây không phải là điều được đặt cố định mà là điều liên tục được đánh giá.

The reality is that you cannot perform any balancing unless you are aware, to some extent, of what is going on across your organization. Even if you push a great deal of responsibility into the teams themselves, there can still be value in having a function in your delivery organization that can perform this balancing act.

Thực tế là bạn không thể thực hiện bất kỳ sự cân bằng nào trừ khi bạn nhận thức được, ở một mức độ nào đó, những gì đang diễn ra xuyên suốt tổ chức của mình. Ngay cả khi bạn đẩy một lượng lớn trách nhiệm vào chính các nhóm, vẫn có thể có giá trị trong việc có một chức năng trong tổ chức chuyển giao của bạn có thể thực hiện hành động cân bằng này.

Enabling Teams — Các nhóm hỗ trợ (Enabling Teams)

We last looked at enabling teams in “Sharing Specialists” on page 460 in the context of user interfaces, but they have broader applicability than that. As described in Team Topologies, these are the teams that work to support our stream-aligned teams. Wherever our microservice-owning, end-to-end focused stream-aligned teams are focusing on delivering user-facing functionality, they need help from others to do their job. When discussing the **user interface** we talked about the idea of having an **enabling team** that can help support other teams in terms of creating effective, consistent user experiences. As shown in Figure 15-2, we can envision this as enabling teams working to support multiple stream-aligned teams in some cross-cutting aspect.

Lần cuối cùng chúng ta xem xét các enabling team là trong “Sharing Specialists” ở trang 460 trong bối cảnh giao diện người dùng, nhưng chúng có khả năng áp dụng rộng hơn thế. Như được mô tả trong Team Topologies, đây là những nhóm làm việc để hỗ trợ các stream-aligned team của chúng ta. Bất cứ nơi nào các stream-aligned team sở hữu microservice, tập trung đầu-cuối của chúng ta đang tập trung vào việc chuyển giao chức năng hướng tới người dùng, chúng cần sự giúp đỡ từ những nhóm khác để làm công việc của mình. Khi thảo luận về giao diện người dùng, chúng ta đã nói về ý tưởng có một enabling team có thể giúp hỗ trợ các nhóm khác trong việc tạo ra các trải nghiệm người dùng hiệu quả, nhất quán. Như được thể hiện trong Hình 15-2, chúng ta có thể hình dung điều này như các enabling team làm việc để hỗ trợ nhiều stream-aligned team ở một khía cạnh cắt ngang (cross-cutting) nào đó.

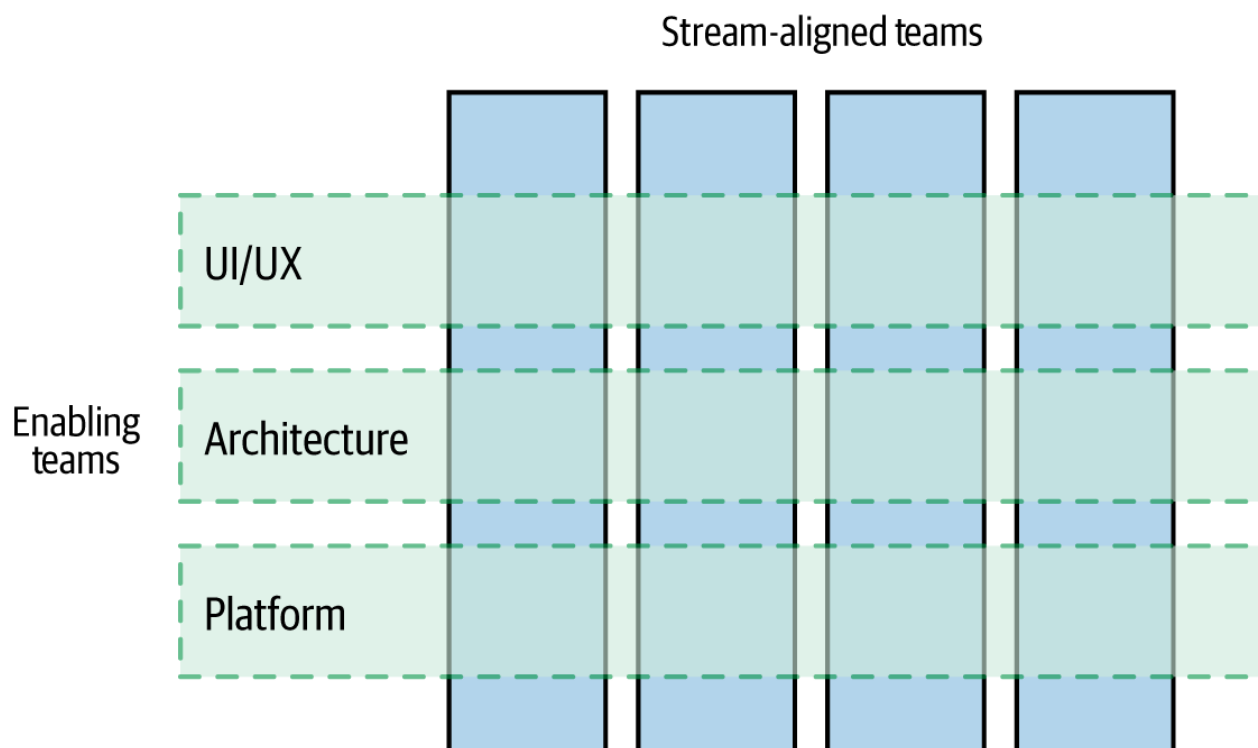


Figure 15-2. Enabling teams support multiple stream-aligned teams

Hình 15-2. Các enabling team hỗ trợ nhiều stream-aligned team

But enabling teams can come in different shapes and sizes.

Nhưng các enabling team có thể đến dưới những hình thù và kích cỡ khác nhau.

Consider Figure 15-3 . Each team has decided to pick a different programming language. When looked at in isolation, each of these decisions seems to make sense— each team has chosen the programming language that it is happiest with. But what about the organization as a whole? Do you want to have to support multiple different programming languages within your organization? How does that complicate rotation between teams, and how does it affect hiring, for that matter?

Hãy xem xét Hình 15-3. Mỗi nhóm đã quyết định chọn một ngôn ngữ lập trình khác nhau. Khi xét riêng lẻ, mỗi quyết định này dường như hợp lý—mỗi nhóm đã chọn ngôn ngữ lập trình mà nó hài lòng nhất. Nhưng còn toàn bộ tổ chức thì sao? Bạn có muốn phải hỗ trợ nhiều ngôn ngữ lập trình khác nhau trong tổ chức của mình không? Điều đó làm phức tạp việc luân chuyển giữa các nhóm như thế nào, và nó ảnh hưởng đến việc tuyển dụng ra sao?

You may decide that you actually don't want this much local optimization—but you need to be aware of these different choices being made and have some ability to discuss these changes if you want some degree of control. This is typically where I see a very small supporting group working across the teams to help connect people so these discussions can happen properly.

Bạn có thể quyết định rằng bạn thực sự không muốn nhiều tối ưu cục bộ đến vậy—nhưng bạn cần nhận thức được những lựa chọn khác nhau đang được đưa ra này và có một khả năng nào đó để thảo luận về những thay đổi này nếu bạn muốn một mức độ kiểm soát nào đó. Đây thường là nơi tôi thấy một nhóm hỗ trợ rất nhỏ làm việc xuyên suốt các nhóm để giúp kết nối mọi người sao cho những cuộc thảo luận này có thể diễn ra một cách đúng đắn.

It's in these cross-cutting supporting groups that I can see the obvious place for architects to be based, at least part of their time. An old-fashioned architect would tell people what to do. In the new, modern, decentralized organization, architects are surveying the landscape, spotting trends, helping connect people, and acting as a sounding board to help other teams get stuff done. They aren't a unit of control in this world; they are yet another enabling function (often with a new name—I've seen terms like principal engineer used for people playing the role of what I would consider an architect). We'll explore the role of architects more in Chapter 16 .

Chính trong những nhóm hỗ trợ cắt ngang này, tôi có thể thấy chỗ đứng hiển nhiên cho các kiến trúc sư, ít nhất là một phần thời gian của họ. Một kiến trúc sư kiểu cũ sẽ bảo người ta phải làm gì. Trong tổ chức mới, hiện đại, phi tập trung, các kiến trúc sư đang khảo sát toàn cảnh, phát hiện các xu hướng, giúp kết nối mọi người, và đóng vai trò như một người để trao đổi giúp các nhóm khác hoàn thành công việc. Họ không phải là một đơn vị kiểm soát trong thế giới này; họ chỉ là một chức năng hỗ trợ khác (thường có một cái tên mới—tôi đã thấy các thuật ngữ như principal engineer được dùng cho những người đóng vai trò mà tôi sẽ coi là một kiến trúc sư). Chúng ta sẽ khám phá vai trò của các kiến trúc sư nhiều hơn trong Chương 16.

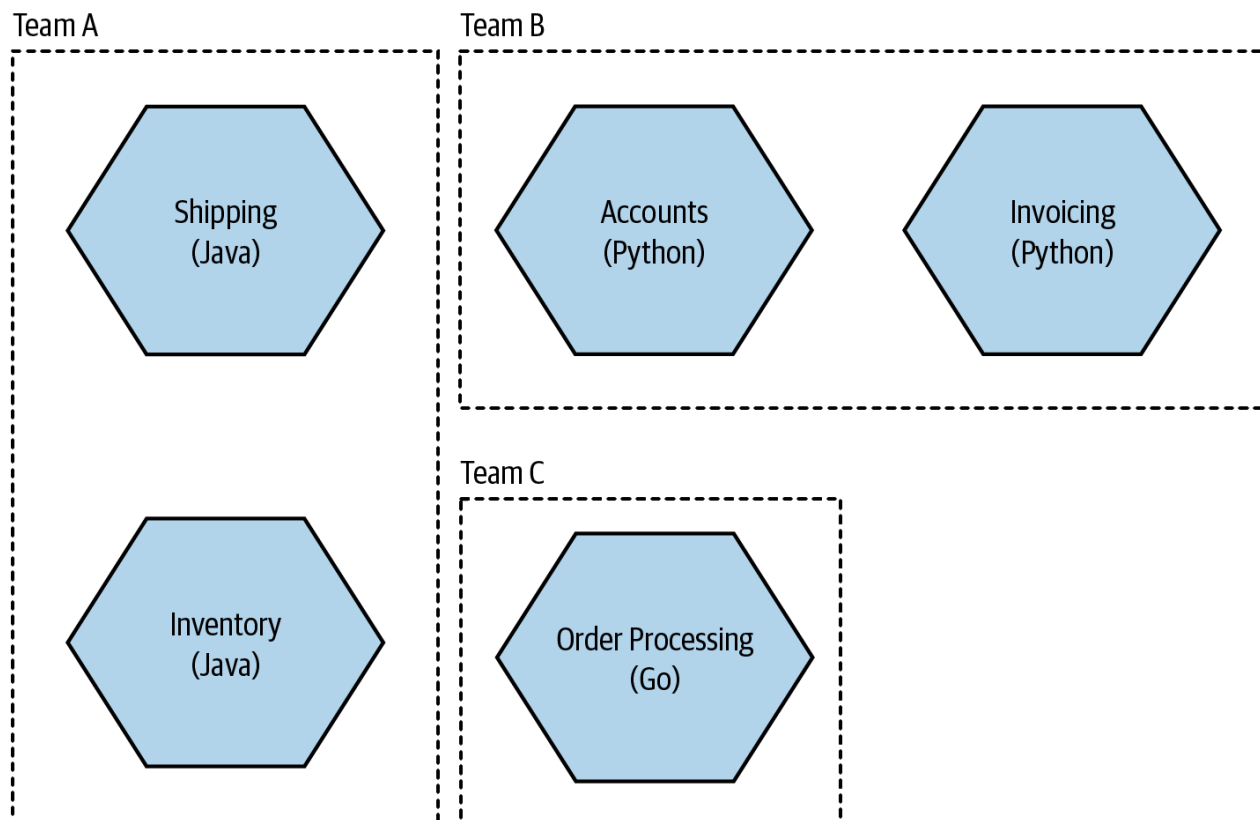


Figure 15-3. Each team has picked a different programming language

Hình 15-3. Mỗi nhóm đã chọn một ngôn ngữ lập trình khác nhau

An enabling team can help identify problems that would be better fixed outside of the teams as well. Consider a scenario in which each team was finding it painful to spin up databases with test data in them. Each team had worked around this issue in different ways, but the issue was never important enough for any team to fix it properly. But then we look across multiple teams and find that many of them might benefit from a proper solution to the problem, and it suddenly becomes obvious that it needs to be addressed.

Một enabling team cũng có thể giúp xác định các vấn đề mà tốt hơn nên được sửa bên ngoài các nhóm. Hãy xem xét một kịch bản trong đó mỗi nhóm thấy việc khởi tạo các cơ sở dữ liệu với dữ liệu kiểm thử trong đó là điều nhức nhối. Mỗi nhóm đã tìm cách xoay xở quanh vấn đề này theo những cách khác nhau, nhưng vấn đề chưa bao giờ đủ quan trọng để bất kỳ nhóm nào sửa nó một cách đúng đắn. Nhưng rồi chúng ta nhìn xuyên suốt nhiều nhóm và phát hiện ra rằng nhiều nhóm trong số đó có thể hưởng lợi từ một giải pháp đúng đắn cho vấn đề, và đột nhiên trở nên hiển nhiên rằng nó cần được giải quyết.

Communities of Practice — Cộng đồng thực hành (Communities of Practice)

A community of practice (CoP) is a cross-cutting group that fosters sharing and learning between peers. When done well, communities of practice are a fantastic way to create an organization in which people are able to continually learn and grow. In her excellent book on the subject, *Building Successful Communities of Practice*, 11 Emily Webber writes:

Một cộng đồng thực hành (community of practice, CoP) là một nhóm cắt ngang nuôi dưỡng việc chia sẻ và học hỏi giữa các đồng nghiệp. Khi được làm tốt, các cộng đồng thực hành là một cách tuyệt vời để tạo ra một tổ chức trong đó mọi người có thể liên tục học hỏi và trưởng thành. Trong cuốn sách xuất sắc của bà về chủ đề này, *Building Successful Communities of Practice*, 11 Emily Webber viết:

Communities of practice create the right environment for social learning, experiential learning and a rounded curriculum, leading to accelerated learning for members...It can encourage a learning culture where people seek better ways to do things, rather than only using existing models.

Các cộng đồng thực hành tạo ra môi trường phù hợp cho việc học hỏi xã hội, học hỏi qua trải nghiệm và một chương trình học toàn diện, dẫn đến việc học tập được đẩy nhanh cho các thành viên... Nó có thể khuyến khích một văn hóa học tập nơi mọi người tìm kiếm những cách làm việc tốt hơn, thay vì chỉ sử dụng các mô hình hiện hữu.

11 Emily Webber, *Building Successful Communities of Practice* (San Francisco: Blurp, 2016).

11 Emily Webber, *Building Successful Communities of Practice* (San Francisco: Blurp, 2016).

Now, I believe that in some cases the same group could be both a CoP and an enabling team, but in my own experience this is rare. There is definite overlap, though. Both enabling teams and communities of practice give you insight into what's happening in different teams across your organization. This insight can help you understand whether you need to rebalance your global versus local optimization, or help you identify the need for some more central help, but the divide here is in the responsibilities and capabilities of the group.

Giờ đây, tôi tin rằng trong một số trường hợp, cùng một nhóm có thể vừa là một CoP vừa là một enabling team, nhưng theo kinh nghiệm của riêng tôi thì điều này hiếm. Tuy vậy, có sự chồng lấn rõ ràng. Cả enabling team và cộng đồng thực hành đều cho bạn cái nhìn sâu vào những gì đang diễn ra trong các nhóm khác nhau xuyên suốt tổ chức của bạn. Cái nhìn sâu này có thể giúp bạn hiểu liệu bạn có cần tái cân bằng giữa tối ưu toàn cục và cục bộ hay không, hoặc giúp bạn xác định nhu cầu về một số sự trợ giúp tập trung hơn, nhưng sự khác biệt ở đây nằm ở các trách nhiệm và năng lực của nhóm.

Members of an enabling team are often working full-time as part of the team, or else they have a significant amount of their time ring-fenced for such a purpose. As such, they have more bandwidth

to put changes into action—to actually work with other teams and help them. Communities of practice are focused more on enabling learning—the individuals in the group are often participating in a forum for a few hours a week at most, and the membership of such a group is often fluid.

Các thành viên của một enabling team thường làm việc toàn thời gian như một phần của nhóm, hoặc nếu không thì họ có một lượng thời gian đáng kể được khoanh vùng cho mục đích đó. Như vậy, họ có nhiều dung lượng hơn để đưa các thay đổi vào hành động—để thực sự làm việc với các nhóm khác và giúp đỡ họ. Các cộng đồng thực hành tập trung nhiều hơn vào việc tạo điều kiện cho việc học—các cá nhân trong nhóm thường tham gia vào một diễn đàn chỉ vài giờ một tuần là nhiều nhất, và thành viên của một nhóm như vậy thường biến động.

CoPs and enabling teams can work very effectively together, of course. Often, a CoP is able to provide valuable insights that can help an enabling team better understand what is needed. Consider a **Kubernetes** CoP sharing its experience of how painful it was to work on its company’s development cluster with the **platform team** that manages the cluster. On the subject of platform teams, that is a topic worth looking into in more detail.

Tất nhiên, các CoP và enabling team có thể làm việc với nhau rất hiệu quả. Thường thì một CoP có thể cung cấp những cái nhìn sâu giá trị có thể giúp một enabling team hiểu rõ hơn những gì được cần. Hãy xem xét một CoP về Kubernetes chia sẻ trải nghiệm của họ về việc làm việc trên cụm phát triển (development cluster) của công ty đã như thế nào với platform team quản lý cụm đó. Về chủ đề các platform team, đó là một chủ đề đáng tìm hiểu chi tiết hơn.

The Platform — Nền tảng (The Platform)

To return to our loosely coupled, stream-aligned teams, we expect them to do their own testing in isolated environments, manage deployments in such a way that they can be done during the day, and make changes to their system architecture when needed. All of this seems to be pushing more and more responsibility—and work—into these teams. Enabling teams as a general concept can help here, but ultimately stream-aligned teams need a self-service set of tools that allows them to do their job—this is the platform.

Quay lại với các stream-aligned team có coupling lỏng của chúng ta, chúng ta kỳ vọng họ tự thực hiện việc kiểm thử của mình trong các môi trường tách biệt, quản lý các đợt triển khai theo cách có thể được thực hiện trong ngày, và thực hiện các thay đổi đối với kiến trúc hệ thống của họ khi cần. Tất cả những điều này dường như đang đẩy ngày càng nhiều trách nhiệm—và công việc—vào các nhóm này. Enabling team như một khái niệm tổng quát có thể giúp ích ở đây, nhưng xét cho cùng các stream-aligned team cần một bộ công cụ tự phục vụ cho phép họ làm công việc của mình—đó chính là nền tảng (platform).

Without a platform, in fact, you may find it difficult to change the organization. In his article “Convergence to Kubernetes,” 12 RVU’s CTO Paul Ingles shares the experience of price comparison website Uswitch moving away from the direct use of low-level AWS services toward a more highly abstracted platform based on Kubernetes. The idea was that this platform allowed RVU’s stream-aligned teams to focus more on delivering new features and spend less time on managing infrastructure. As Paul puts it:

Trên thực tế, nếu không có một nền tảng, bạn có thể thấy khó thay đổi tổ chức. Trong bài viết “Convergence to Kubernetes” của mình, 12 Paul Ingles, CTO của RVU, chia sẻ trải nghiệm của trang web so sánh giá Uswitch khi chuyển ra khỏi việc sử dụng trực tiếp

các dịch vụ AWS cấp thấp sang một nền tảng được trừu tượng hóa cao hơn dựa trên Kubernetes. Ý tưởng là nền tảng này cho phép các stream-aligned team của RVU tập trung nhiều hơn vào việc chuyển giao các tính năng mới và dành ít thời gian hơn cho việc quản lý hạ tầng. Như Paul nói:

We didn't change our organisation because we wanted to use Kubernetes; we used Kubernetes because we wanted to change our organization.

Chúng tôi không thay đổi tổ chức của mình vì chúng tôi muốn dùng Kubernetes; chúng tôi dùng Kubernetes vì chúng tôi muốn thay đổi tổ chức của mình.

12 Paul Ingles, “Convergence to Kubernetes,” Medium, June 18, 2018, <https://oreil.ly/Ho7kY>.

12 Paul Ingles, “Convergence to Kubernetes,” Medium, June 18, 2018, <https://oreil.ly/Ho7kY>.

A platform that can implement common functionality, such as the ability to handle desired state management for microservices, **log aggregation**, and inter-microservice **authorization** and **authentication**, can deliver huge improvements in productivity and enable teams to take on more responsibility without also having to drastically increase the amount of work they do. In fact, a platform should give teams more bandwidth to focus on delivering features.

Một nền tảng có thể triển khai chức năng chung, chẳng hạn như khả năng xử lý quản lý trạng thái mong muốn (desired state management) cho các microservice, gom log (log aggregation), và xác thực và phân quyền giữa các microservice, có thể mang lại những cải thiện to lớn về năng suất và cho phép các nhóm đảm nhận thêm trách nhiệm mà không phải tăng mạnh lượng công việc họ làm. Thực ra, một nền tảng nên cho các nhóm nhiều dung lượng hơn để tập trung vào việc chuyển giao các tính năng.

The platform team

Platform team

A platform needs someone to run and manage it. These technology stacks can be complex enough to warrant some specific expertise. My concern, though, is that on occasion it can be too easy for platform teams to lose sight of why they exist.

Một nền tảng cần ai đó để vận hành và quản lý nó. Các tập công nghệ (technology stack) này có thể đủ phức tạp để cần đến một số chuyên môn cụ thể. Tuy nhiên, mối quan ngại của tôi là đôi khi quá dễ để các platform team đánh mất tầm nhìn về lý do tại sao họ tồn tại.

A platform team has users, in the same way that any other team has users. The users of the platform team are other developers—your job, if you are in a platform team, is to make their lives easier (this of course is the job of any enabling team). That means that the platform you create needs to fit the needs of the teams using it. It also means that you need to work with the teams using your platform not only to help them use it well but also to take on board their feedback and requirements to improve the platform you deliver.

Một platform team có người dùng, theo cách giống như bất kỳ nhóm nào khác có người dùng. Người dùng của platform team là các lập trình viên khác—công việc của bạn, nếu bạn ở trong một platform team, là làm cho cuộc sống của họ dễ dàng hơn (đây tất nhiên là công việc của bất kỳ enabling team nào). Điều đó có nghĩa là nền tảng bạn tạo ra cần khớp với nhu cầu của các nhóm sử dụng nó. Nó cũng có nghĩa là bạn cần làm việc với các nhóm sử dụng nền tảng của bạn không chỉ để giúp họ dùng nó tốt mà còn để tiếp thu phản hồi và yêu cầu của họ nhằm cải thiện nền tảng mà bạn chuyển giao.

In the past, I've preferred calling such a team something like “delivery services” or “delivery support” to better articulate its goal. Really, the job of a platform team isn't to build a platform; it's to make developing and shipping functionality easy. Building a platform is just one way that the members of a platform team can achieve this. I do worry that by calling themselves a platform team they will see all problems as things that can and should be solved by the platform, rather than thinking more widely about other ways to make developers' lives easier.

Trong quá khứ, tôi đã ưa thích gọi một nhóm như vậy là thứ gì đó như “delivery services” hoặc “delivery support” để diễn đạt rõ hơn mục tiêu của nó. Thực sự, công việc của một platform team không phải là xây dựng một nền tảng; mà là làm cho việc phát triển và ship chức năng trở nên dễ dàng. Xây dựng một nền tảng chỉ là một cách mà các thành viên của một platform team có thể đạt được điều này. Tôi lo ngại rằng bằng cách tự gọi mình là một platform team, họ sẽ xem tất cả các vấn đề như những thứ có thể và nên được giải quyết bằng nền tảng, thay vì suy nghĩ rộng hơn về những cách khác để làm cho cuộc sống của các lập trình viên dễ dàng hơn.

Like any good enabling team, a platform team needs to operate almost like an internal consultancy to some extent. If you are in a platform team, you need to be going out to find out what problems people are facing and working with them to help them get these problems fixed. But as you also end up building the platform too, you need to have a heavy dose of product development work in there as well. In fact, taking a product development approach to how you build out your platform is a great idea and could be a great place to help grow new product owners.

Giống như bất kỳ enabling team tốt nào, một platform team cần hoạt động gần như một công ty tư vấn nội bộ ở một mức độ nào đó. Nếu bạn ở trong một platform team, bạn cần ra ngoài để tìm hiểu những vấn đề mọi người đang đối mặt và làm việc với họ để giúp họ sửa những vấn đề này. Nhưng vì bạn cũng cuối cùng xây dựng cả nền tảng nữa, bạn cần có một liều lượng lớn công việc phát triển sản phẩm trong đó. Thực ra, áp dụng một cách tiếp cận phát triển sản phẩm vào cách bạn xây dựng nền tảng là một ý tưởng tuyệt vời và có thể là một nơi tuyệt vời để giúp nuôi dưỡng các product owner mới.

The **paved road**

Con đường lát đá (the paved road)

A concept that has become popular in software development is that of “the paved road.” The idea is that you clearly communicate how you want things to be done and then provide mechanisms whereby these things can be done easily. For example, you might want to ensure that all microservices communicate via mutual TLS. You could then back this up by providing a common framework or deployment platform that would automatically deliver mutual TLS for the microservices running on it. A platform can be a great way to deliver on that paved road.

Một khái niệm đã trở nên phổ biến trong phát triển phần mềm là khái niệm “paved road” (con đường lát đá). Ý tưởng là bạn truyền đạt rõ ràng cách bạn muốn mọi thứ được thực hiện và rồi cung cấp các cơ chế nhờ đó những điều này có thể được thực hiện dễ dàng. Ví dụ, bạn có thể muốn bảo đảm rằng tất cả các microservice giao tiếp qua mutual TLS. Sau đó bạn có thể chống lưng cho điều này bằng cách cung cấp một framework chung hoặc một nền tảng triển khai có thể tự động cung cấp mutual TLS cho các microservice chạy trên đó. Một nền tảng có thể là một cách tuyệt vời để hiện thực hóa con đường lát đá đó.

The key concept behind a paved road is that using the paved road isn't enforced—it just provides an easier way of getting to your destination. So if a team wanted to ensure its microservices communicated via mutual TLS without using the common framework, it would have to find some other way of doing this, but it would still be allowed. The analogy here is that while we might want

everyone to get to the same destination, people are free to find their own paths there—the hope is that the paved road is the easiest way of getting where you need.

Khái niệm then chốt đằng sau một paved road là việc sử dụng paved road không bị bắt buộc—nó chỉ cung cấp một cách dễ dàng hơn để đi đến đích của bạn. Vậy nên nếu một nhóm muốn bảo đảm các microservice của mình giao tiếp qua mutual TLS mà không dùng framework chung, nó sẽ phải tìm một cách khác để làm điều này, nhưng nó vẫn được phép. Ở đây phép loại suy là trong khi chúng ta có thể muốn mọi người đến cùng một đích, mọi người được tự do tìm con đường riêng của mình đến đó—hy vọng là paved road là cách dễ dàng nhất để đến nơi bạn cần.

The paved road concept aims to make the common cases easy, while leaving room for exceptions where warranted.

Khái niệm paved road nhằm tới việc làm cho các trường hợp phổ biến trở nên dễ dàng, trong khi vẫn chừa chỗ cho các ngoại lệ ở những nơi chính đáng.

If we think of the platform as a paved road, by making it optional, you incentivize the platform team to make the platform easy to use. Here is Paul Ingles again, on measuring the effectiveness of the platform team: 13

Nếu chúng ta nghĩ về nền tảng như một paved road, bằng cách làm cho nó tùy chọn, bạn tạo động lực cho platform team làm cho nền tảng dễ sử dụng. Đây lại là Paul Ingles, về việc đo lường tính hiệu quả của platform team:13

We'd set [objectives and key results (OKRs)] around the number of teams we'd want to adopt the platform, the number of applications using the platform **autoscaling** service, the proportion of applications switched to the platform dynamic credentials service, and so on. Some of those we'd track over longer periods of time, and others were helpful to guide progress for a quarter and then we'd drop them in favor of something else. We never mandated the use of the platform, so setting key results for the number of onboarded teams forced us to focus on solving problems that would drive adoption. We also look for natural measures of progress: the proportion of traffic served by the platform, and the proportion of revenue served through platform services are both good examples of that. When you place barriers in people's way that seem arbitrary and capricious, those people will find ways of working around the barriers to get a job done. So in general, I find it much more effective to explain why things should be done in a certain way, and then make it easy to do things in that way, rather than attempting to make the things you don't like impossible.

Chúng tôi đặt [các mục tiêu và kết quả then chốt (OKR)] xoay quanh số lượng nhóm mà chúng tôi muốn áp dụng nền tảng, số lượng ứng dụng dùng dịch vụ autoscaling của nền tảng, tỷ lệ ứng dụng được chuyển sang dịch vụ thông tin xác thực động (dynamic credentials) của nền tảng, và cứ thế. Một số trong đó chúng tôi theo dõi qua những khoảng thời gian dài hơn, và số khác hữu ích để định hướng tiến độ cho một quý rồi chúng tôi bỏ chúng đi để chuyển sang thứ khác. Chúng tôi không bao giờ bắt buộc việc sử dụng nền tảng, nên việc đặt các kết quả then chốt cho số lượng nhóm được onboard buộc chúng tôi tập trung vào việc giải quyết các vấn đề sẽ thúc đẩy việc áp dụng. Chúng tôi cũng tìm các thước đo tiến độ tự nhiên: tỷ lệ lưu lượng được phục vụ bởi nền tảng, và tỷ lệ doanh thu được phục vụ qua các dịch vụ của nền tảng, cả hai đều là những ví dụ tốt cho điều đó. Khi bạn đặt những rào cản vào đường đi của mọi người mà có vẻ tùy tiện và độc đoán, những người đó sẽ tìm cách lách qua các rào cản để hoàn thành công việc. Vậy nên nói chung, tôi thấy hiệu quả hơn nhiều khi giải thích tại sao mọi thứ nên được làm theo một cách nhất định, rồi làm cho việc làm mọi thứ theo cách đó trở nên dễ dàng, thay vì cố làm cho những điều bạn không thích trở nên bất khả thi.

The shift to more autonomous, stream-aligned teams doesn't eliminate the need for having a clear technical vision or for being clear about certain things that all teams are required to do. If there are concrete restrictions (the need to be cloud-vendor agnostic, for example) or specific requirements that all teams need to obey (all PII needs to be encrypted at rest using specific algorithms), then those still need to be clearly communicated, and the reasons for them made clear. The platform can then play a role in making these things easy to do. By using the platform, you are on the

Sự dịch chuyển sang các nhóm tự chủ hơn, theo kiểu stream-aligned không loại bỏ nhu cầu có một tầm nhìn kỹ thuật rõ ràng hoặc nhu cầu rõ ràng về một số điều mà tất cả các nhóm được yêu cầu phải làm. Nếu có những hạn chế cụ thể (chẳng hạn nhu cầu trung lập với nhà cung cấp đám mây) hoặc các yêu cầu cụ thể mà tất cả các nhóm cần tuân theo (mọi PII cần được mã hóa khi lưu trữ bằng các thuật toán cụ thể), thì những điều đó vẫn cần được truyền đạt rõ ràng, và lý do cho chúng phải được làm rõ. Nền tảng sau đó có thể đóng vai trò làm cho những điều này trở nên dễ thực hiện. Bằng cách sử dụng nền tảng, bạn đang đi trên

13 “Organizational Evolution for Accelerating Delivery of Comparison Services at Uswitch,” Team Topologies, June 24, 2020, <https://oreil.ly/zoyvvv>.

13 “Organizational Evolution for Accelerating Delivery of Comparison Services at Uswitch,” Team Topologies, June 24, 2020, <https://oreil.ly/zoyvvv>.

paved road—you'll end up doing a lot of the right things without having to expend much effort.

con đường lát đá—bạn sẽ rất cuộc làm được rất nhiều điều đúng đắn mà không phải bỏ ra nhiều công sức.

On the other hand, I see some organizations try to govern via the platform. Rather than clearly articulate what needs to be done and why, instead they simply say “you must use this platform.” The problem with this is that if the platform isn't easy to use or doesn't suit a particular use case, people will find ways to bypass the platform itself. When teams work outside the platform, they have no clear sense of what restrictions are important to the organization and will find themselves doing the “wrong” thing without realizing it.

Mặt khác, tôi thấy một số tổ chức cố gắng quản trị (govern) thông qua nền tảng. Thay vì diễn đạt rõ ràng những gì cần được làm và tại sao, họ chỉ đơn giản nói “bạn phải dùng nền tảng này.” Vấn đề với điều này là nếu nền tảng không dễ sử dụng hoặc không phù hợp với một trường hợp sử dụng cụ thể, mọi người sẽ tìm cách bỏ qua chính nền tảng đó. Khi các nhóm làm việc bên ngoài nền tảng, họ không có cảm nhận rõ ràng về những hạn chế nào là quan trọng đối với tổ chức và sẽ thấy mình đang làm điều “sai” mà không nhận ra.

Shared Microservices — Microservice dùng chung

As I've already discussed, I'm a big proponent of strong ownership models for microservices. In general, one microservice should be owned by a single team. Despite that, I still find it common for microservices to be owned by multiple teams. Why is this? And what can (or should) you do about it? The drivers that cause people to have microservices shared by multiple teams are important to understand, especially as we may be able to find some compelling alternative models that can address people's underlying concerns.

Như tôi đã thảo luận, tôi là một người ủng hộ mạnh mẽ các mô hình quyền sở hữu mạnh cho microservice. Nói chung, một microservice nên được sở hữu bởi một nhóm duy nhất. Dù vậy, tôi vẫn thấy phổ biến việc các microservice được sở hữu bởi nhiều nhóm. Tại sao lại như vậy? Và bạn có thể (hoặc nên) làm gì về điều đó? Các động lực khiến mọi người có

những microservice được nhiều nhóm chia sẻ là điều quan trọng để hiểu, đặc biệt là vì chúng ta có thể tìm ra một số mô hình thay thế hấp dẫn có thể giải quyết những mối quan ngại sâu xa của mọi người.

Too Hard to Split — Quá khó để tách

Obviously, one of the reasons you may find yourself with a microservice owned by more than one team is that the cost of splitting the microservice into pieces that can be owned by different teams is too high, or perhaps your organization might not see the point of it. This is a common occurrence with large, monolithic systems. If this is the main challenge you face, then I hope some of the advice given in Chapter 3 will be of use. You could also consider merging teams to align more closely with the architecture itself.

Hiển nhiên, một trong những lý do bạn có thể thấy mình có một microservice được sở hữu bởi nhiều hơn một nhóm là vì chi phí tách microservice thành các mảnh có thể được sở hữu bởi các nhóm khác nhau là quá cao, hoặc có lẽ tổ chức của bạn không thấy ý nghĩa của việc đó. Đây là một tình huống thường gặp với các hệ thống lớn, kiểu monolith. Nếu đây là thách thức chính mà bạn đối mặt, thì tôi hy vọng một số lời khuyên được đưa ra trong Chương 3 sẽ hữu ích. Bạn cũng có thể cân nhắc việc hợp nhất các nhóm để giống hàng ngang khít hơn với chính kiến trúc.

FinanceCo, a FinTech company that we previously met in “Toward Stream-Aligned Teams” on page 459, largely operate a strong ownership model with a high degree of team autonomy. However, it still has an existing monolithic system that was slowly being split apart. That monolithic application was to all intents and purposes shared by multiple teams, and the increased cost of working in this shared codebase was obvious.

FinanceCo, một công ty FinTech mà chúng ta đã gặp trước đây trong “Toward Stream-Aligned Teams” ở trang 459, phần lớn vận hành một mô hình quyền sở hữu mạnh với mức độ tự chủ nhóm cao. Tuy nhiên, nó vẫn có một hệ thống kiểu monolith hiện hữu đang dần được tách ra. Ứng dụng monolith đó về mọi ý nghĩa và mục đích đều được chia sẻ bởi nhiều nhóm, và chi phí gia tăng của việc làm việc trong codebase dùng chung này là điều hiển nhiên.

Cross-Cutting Changes — Các thay đổi cắt ngang

Much of what we’ve discussed so far in this book regarding the interplay of organizational structure and architecture is aimed toward reducing the need for coordination between teams. Most of this is about trying to reduce cross-cutting changes as much as we can. We do, though, have to recognize that some cross-cutting changes may be unavoidable.

Phần lớn những gì chúng ta đã thảo luận cho đến nay trong cuốn sách này liên quan đến sự tương tác qua lại giữa cấu trúc tổ chức và kiến trúc đều nhằm tới việc giảm bớt nhu cầu phối hợp giữa các nhóm. Phần lớn điều này là về việc cố gắng giảm các thay đổi cắt ngang (cross-cutting) càng nhiều càng tốt. Tuy vậy, chúng ta phải thừa nhận rằng một số thay đổi cắt ngang có thể là không thể tránh khỏi.

FinanceCo hit one such issue. When it originally started, one account was tied to one user. As the company grew and took on more business users (being previously more focused on consumers), this became a limitation. The company wanted to move to a model in which a single account with

FinanceCo could accommodate multiple users. This was a fundamental change, as up until that point the assumption across the system had been one account = one user.

FinanceCo gặp phải một vấn đề như vậy. Khi nó ban đầu mới khởi đầu, một tài khoản được gắn với một người dùng. Khi công ty phát triển và tiếp nhận thêm nhiều người dùng doanh nghiệp (trước đây tập trung nhiều hơn vào người tiêu dùng), điều này trở thành một giới hạn. Công ty muốn chuyển sang một mô hình trong đó một tài khoản đơn lẻ với FinanceCo có thể chứa nhiều người dùng. Đây là một thay đổi nền tảng, vì cho đến thời điểm đó giả định xuyên suốt hệ thống đã là một tài khoản = một người dùng.

A single team had been formed to make this change happen. The problem was that a vast amount of the work involved making changes to microservices already owned by other teams. This meant that the team's work was partly about making changes and submitting pull requests, or asking other teams to make the changes happen. Coordinating these changes was very painful, as a significant number of microservices needed to be modified to support the new functionality.

Một nhóm duy nhất đã được thành lập để thực hiện thay đổi này. Vấn đề là một lượng lớn công việc liên quan đến việc thực hiện các thay đổi đối với những microservice vốn đã được sở hữu bởi các nhóm khác. Điều này có nghĩa là công việc của nhóm một phần là về việc thực hiện các thay đổi và gửi pull request, hoặc nhờ các nhóm khác thực hiện các thay đổi. Việc phối hợp những thay đổi này rất nhức nhối, vì một số lượng đáng kể microservice cần được sửa đổi để hỗ trợ chức năng mới.

By restructuring our teams and architecture to eliminate one set of cross-cutting changes, we may in fact expose ourselves to a different set of cross-cutting changes that may have a more significant impact. This was the case with FinanceCo—the type of reorganization that would be required to reduce the cost of the multiuser functionality would have increased the cost of making other, more common changes. FinanceCo understood that this particular change was going to be very painful, but that it was such an exceptional type of change that this pain was acceptable.

Bằng cách tái cấu trúc các nhóm và kiến trúc của chúng ta để loại bỏ một tập các thay đổi cắt ngang, chúng ta thực ra có thể tự đặt mình vào một tập các thay đổi cắt ngang khác có thể có tác động đáng kể hơn. Đây là trường hợp của FinanceCo—loại tái tổ chức cần thiết để giảm chi phí của chức năng nhiều người dùng sẽ làm tăng chi phí thực hiện các thay đổi khác, phổ biến hơn. FinanceCo hiểu rằng thay đổi cụ thể này sẽ rất nhức nhối, nhưng rằng nó là một loại thay đổi ngoại lệ đến mức nỗi đau này là chấp nhận được.

Delivery Bottlenecks — Nút thắt cổ chai trong chuyển giao

One key reason people move toward collective ownership, with microservices being shared between teams, is to avoid delivery bottlenecks. What if there is a large backlog of changes that need to be made in a single service? Let's return to MusicCorp, and let's imagine that we are rolling out the ability for a customer to see the genre of a track across our products, as well as adding a brand new type of stock: virtual musical ringtones for the mobile phone. The website team needs to make a change to surface the genre information, with the mobile app team working to allow users to browse, preview, and buy the ringtones. Both changes need to be made to the Catalog microservice, but unfortunately half the team is stuck diagnosing a production failure, with the other half out with food poisoning after a recent team outing to a pop-up food truck being run out of an alley.

Một lý do then chốt khiến mọi người chuyển sang quyền sở hữu tập thể, với việc microservice được chia sẻ giữa các nhóm, là để tránh nút thắt cổ chai trong chuyển giao. Nếu có một backlog lớn các thay đổi cần được thực hiện trong một dịch vụ duy nhất

thì sao? Hãy quay lại với MusicCorp, và hãy tưởng tượng rằng chúng ta đang tung ra khả năng để một khách hàng nhìn thấy thể loại của một bản nhạc trên các sản phẩm của chúng ta, cũng như thêm một loại hàng tồn kho hoàn toàn mới: nhạc chuông ảo cho điện thoại di động. Nhóm website cần thực hiện một thay đổi để hiển thị thông tin thể loại, với nhóm ứng dụng di động làm việc để cho phép người dùng duyệt, nghe thử, và mua nhạc chuông. Cả hai thay đổi đều cần được thực hiện đối với microservice Catalog, nhưng tiếc thay một nửa nhóm bị mắc kẹt trong việc chẩn đoán một sự cố production, với nửa còn lại nghỉ vì ngộ độc thực phẩm sau một chuyến đi chơi nhóm gần đây tới một xe bán đồ ăn pop-up đặt trong một con hẻm.

We have a few options we could consider to avoid the need for both the website and mobile teams to share the Catalog microservice. The first is to just wait. The website and mobile application teams move on to something else. Depending on how important the feature is or how long the delay is likely to be, this may be fine, or it may be a major problem.

Chúng ta có một vài lựa chọn có thể cân nhắc để tránh việc cả nhóm website lẫn nhóm di động phải chia sẻ microservice Catalog. Lựa chọn đầu tiên là chỉ chờ đợi. Các nhóm website và ứng dụng di động chuyển sang làm việc khác. Tùy thuộc vào việc tính năng đó quan trọng đến đâu hoặc khả năng độ trễ kéo dài bao lâu, điều này có thể ổn, hoặc có thể là một vấn đề lớn.

You could instead add people to the catalog team to help them move through their work faster. The more standardized the technology stack and programming idioms in use across your system, the easier it is for other people to make changes in your services. The flip side, of course, as we discussed earlier, is that standardization tends to reduce a team's ability to adopt the right solution for the job and can lead to different sorts of inefficiencies.

Thay vào đó bạn có thể thêm người vào nhóm catalog để giúp họ tiến qua công việc của mình nhanh hơn. Tập công nghệ và các thành ngữ lập trình được sử dụng xuyên suốt hệ thống của bạn càng được chuẩn hóa, thì càng dễ để người khác thực hiện các thay đổi trong các dịch vụ của bạn. Mặt trái, tất nhiên, như chúng ta đã thảo luận trước đó, là việc chuẩn hóa có xu hướng làm giảm khả năng của một nhóm trong việc áp dụng giải pháp đúng cho công việc và có thể dẫn đến những loại kém hiệu quả khác nhau.

Another option that would avoid the need for a shared Catalog microservice could be to split the catalog into a separate general music catalog and a ringtone catalog. If the change being made to support ringtones is fairly small, and the likelihood of this being an area in which we will develop heavily in the future is also quite low, this may well be premature. On the other hand, if there are 10 weeks of ringtone-related features stacked up, splitting out the service could make sense, with the mobile team taking ownership.

Một lựa chọn khác có thể tránh nhu cầu về một microservice Catalog dùng chung có thể là tách catalog thành một catalog âm nhạc tổng quát riêng và một catalog nhạc chuông. Nếu thay đổi đang được thực hiện để hỗ trợ nhạc chuông là khá nhỏ, và khả năng đây là một lĩnh vực mà chúng ta sẽ phát triển mạnh trong tương lai cũng khá thấp, thì điều này rất có thể là quá sớm. Mặt khác, nếu có 10 tuần các tính năng liên quan đến nhạc chuông chất đống lại, thì việc tách dịch vụ ra có thể hợp lý, với nhóm di động nắm quyền sở hữu.

There are a couple of other models we could consider, however. In a moment, we'll look at what can be done in terms of making the shared microservice more "pluggable," allowing other teams to either contribute their code via libraries or else extend from a common framework. First, though, we should explore the potential to bring some ideas from the world of open source development into our company.

Tuy nhiên, có một vài mô hình khác mà chúng ta có thể cân nhắc. Trong giây lát, chúng ta sẽ xem xét những gì có thể được làm về việc khiến microservice dùng chung trở nên “có thể cắm vào” (pluggable) hơn, cho phép các nhóm khác hoặc đóng góp mã của họ qua các thư viện hoặc mở rộng từ một framework chung. Tuy nhiên, trước tiên, chúng ta nên khám phá tiềm năng mang một số ý tưởng từ thế giới phát triển mã nguồn mở vào công ty của mình.

Internal Open Source — Mã nguồn mở nội bộ

Many organizations have decided to implement some form of internal open source, both to help manage the issue of shared codebases, and to make it easier for people outside a team to contribute changes to a microservice they may be making use of.

Nhiều tổ chức đã quyết định triển khai một hình thức mã nguồn mở nội bộ nào đó, vừa để giúp quản lý vấn đề về các codebase dùng chung, vừa để giúp những người bên ngoài một nhóm dễ dàng hơn trong việc đóng góp các thay đổi cho một microservice mà họ có thể đang sử dụng.

With normal open source, a small group of people are considered core committers. They are the custodians of the code. If you want a change to an open source project, either you ask one of the committers to make the change for you or you make the change yourself and send them a pull request. The core committers are still in charge of the codebase; they are the owners.

Với mã nguồn mở thông thường, một nhóm nhỏ người được coi là core committer. Họ là những người gìn giữ mã. Nếu bạn muốn một thay đổi cho một dự án mã nguồn mở, bạn hoặc nhờ một trong các committer thực hiện thay đổi cho bạn hoặc tự mình thực hiện thay đổi và gửi cho họ một pull request. Các core committer vẫn nắm quyền điều hành codebase; họ là chủ sở hữu.

Inside an organization, this pattern can work well too. Perhaps the people who worked on the service originally are no longer on a team together; perhaps they are now scattered across the organization. If they still have commit rights, you can find them and ask for their help, perhaps pairing up with them; or if you have the right tooling, you can send them a pull request.

Bên trong một tổ chức, mẫu hình này cũng có thể hoạt động tốt. Có lẽ những người ban đầu làm việc trên dịch vụ không còn ở cùng một nhóm nữa; có lẽ giờ họ đã rải rác khắp tổ chức. Nếu họ vẫn còn quyền commit, bạn có thể tìm họ và nhờ họ giúp đỡ, có lẽ kết cặp (pair) với họ; hoặc nếu bạn có công cụ phù hợp, bạn có thể gửi cho họ một pull request.

Role of the Core Committers — Vai trò của các core committer

We still want our services to be sensible. We want the code to be of decent quality, and the microservice itself to exhibit some sort of consistency in how it is put together. We also want to make sure that changes being made now don't make future planned changes much harder than they need to be. This means that we need to adopt the same patterns used in normal open source internally too, which means separating out trusted committers (the core team) from untrusted committers (people from outside the team submitting changes).

Chúng ta vẫn muốn các dịch vụ của mình hợp lý. Chúng ta muốn mã có chất lượng tử tế, và bản thân microservice thể hiện một sự nhất quán nào đó trong cách nó được ghép lại với nhau. Chúng ta cũng muốn bảo đảm rằng những thay đổi đang được thực hiện bây giờ không làm cho các thay đổi đã được lên kế hoạch trong tương lai khó hơn nhiều so

với mức cần thiết. Điều này có nghĩa là chúng ta cần áp dụng những mẫu hình giống nhau được dùng trong mã nguồn mở thông thường vào nội bộ nữa, tức là tách những committer được tin cậy (nhóm core) khỏi những committer không được tin cậy (những người từ bên ngoài nhóm gửi các thay đổi).

The core ownership team needs to have some way of vetting and approving changes. It needs to make sure changes are idiomatically consistent—that is, that they follow the general coding guidelines of the rest of the codebase. The people doing the vetting are therefore going to have to spend time working with submitters to make sure each change is of sufficient quality.

Nhóm sở hữu core cần có một cách nào đó để sàng lọc và phê duyệt các thay đổi. Nó cần bảo đảm các thay đổi nhất quán về mặt thành ngữ—tức là, chúng tuân theo các hướng dẫn lập trình chung của phần còn lại của codebase. Do đó những người làm việc sàng lọc sẽ phải dành thời gian làm việc với những người gửi để bảo đảm mỗi thay đổi có chất lượng đủ tốt.

Good gatekeepers put a lot of work into this, communicating clearly with submitters and encouraging good behavior. Bad gatekeepers can use this as an excuse to exert power over others or have religious wars about arbitrary technical decisions. Having seen both sets of behavior, I can tell you one thing is clear: either way takes time. When considering allowing untrusted committers to submit changes to your codebase, you have to decide if the overhead of being a gatekeeper is worth the trouble: could the core team be doing better things with the time it spends vetting patches?

Những người gác cổng (gatekeeper) tốt bỏ rất nhiều công sức vào việc này, giao tiếp rõ ràng với những người gửi và khuyến khích hành vi tốt. Những người gác cổng tồi có thể dùng điều này như một cái cớ để áp đặt quyền lực lên người khác hoặc gây ra các cuộc chiến tranh tôn giáo về những quyết định kỹ thuật tùy tiện. Đã chứng kiến cả hai tập hành vi, tôi có thể nói với bạn một điều rõ ràng: cách nào cũng tốn thời gian. Khi cân nhắc cho phép những committer không được tin cậy gửi các thay đổi vào codebase của bạn, bạn phải quyết định liệu chi phí phụ trội của việc làm người gác cổng có đáng công sức không: liệu nhóm core có thể làm những việc tốt hơn với thời gian mà nó bỏ ra để sàng lọc các bản vá không?

Maturity — Độ trưởng thành

The less stable or mature a service is, the harder it will be to allow people outside the core team to submit patches. Before the key spine of a service is in place, the team may not know what “good” looks like and therefore may struggle to know what a good submission looks like. During this stage, the service itself is undergoing a high degree of change.

Một dịch vụ càng kém ổn định hoặc kém trưởng thành, thì sẽ càng khó để cho phép những người bên ngoài nhóm core gửi các bản vá. Trước khi xương sống then chốt của một dịch vụ đã vào vị trí, nhóm có thể không biết “tốt” trông như thế nào và do đó có thể chật vật để biết một bản gửi tốt trông ra sao. Trong giai đoạn này, bản thân dịch vụ đang trải qua một mức độ thay đổi cao.

Most open source projects tend to not take submissions from a wider group of untrusted committers until the core of the first version is done. Following a similar model for your own organization makes sense. If a service is pretty mature and is rarely changed—for example, our cart service—then perhaps that is the time to open it up for other contributions.

Hầu hết các dự án mã nguồn mở có xu hướng không nhận các bản gửi từ một nhóm rộng hơn các committer không được tin cậy cho đến khi phần core của phiên bản đầu tiên đã

xong. Tuân theo một mô hình tương tự cho tổ chức của riêng bạn là điều hợp lý. Nếu một dịch vụ đã khá trưởng thành và hiếm khi thay đổi—ví dụ, dịch vụ cart của chúng ta—thì có lẽ đó là lúc để mở nó ra cho các đóng góp khác.

Tooling — Công cụ

To best support an internal open source model, you'll need some tooling in place. The use of a distributed version control tool with the ability for people to submit pull requests (or something similar) is important. Depending on the size of the organization, you may also need tooling to allow for a discussion and evolution of patch requests; this may or may not mean a full-blown code review system, but the ability to comment inline on patches is very useful. Finally, you'll need to make it very easy for a committer to build and deploy your software and make it available for others. Typically this involves having well-defined build and deployment pipelines and centralized **artifact** repositories. The more standardized your technology stack is, the easier it will be for people in other teams to make edits and provide patches to a microservice.

Để hỗ trợ tốt nhất một mô hình mã nguồn mở nội bộ, bạn sẽ cần một số công cụ vào vị trí. Việc sử dụng một công cụ quản lý phiên bản phân tán với khả năng cho mọi người gửi pull request (hoặc thứ gì đó tương tự) là quan trọng. Tùy thuộc vào quy mô của tổ chức, bạn cũng có thể cần công cụ để cho phép thảo luận và tiến hóa các yêu cầu vá; điều này có thể có hoặc không có nghĩa là một hệ thống review mã đầy đủ, nhưng khả năng bình luận trực tiếp (inline) trên các bản vá là rất hữu ích. Cuối cùng, bạn sẽ cần làm cho việc một committer build và triển khai phần mềm của bạn và làm cho nó khả dụng cho người khác trở nên rất dễ dàng. Thường thì điều này liên quan đến việc có các build pipeline được định nghĩa rõ ràng và các kho artifact tập trung. Tập công nghệ của bạn càng được chuẩn hóa, thì càng dễ để người ở các nhóm khác thực hiện các chỉnh sửa và cung cấp các bản vá cho một microservice.

Pluggable, Modular Microservices — Microservice có thể cắm vào, mô-đun hóa

At FinanceCo, I saw an interesting challenge associated with a particular microservice that was becoming a bottleneck for many teams. For each country, FinanceCo had dedicated teams focusing on functionality specific to that country. This made a lot of sense, as each country had specific requirements and challenges. But it caused issues for this central service, which needed specific functionality to be updated for each country. The team owning the central microservice was overwhelmed by pull requests being sent to it. The team was doing an excellent job of processing these pull requests quickly, and in fact it focused on this being a core part of its responsibilities, but structurally the situation wasn't really sustainable.

Tại FinanceCo, tôi đã thấy một thách thức thú vị liên quan đến một microservice cụ thể đang trở thành một nút thắt cổ chai cho nhiều nhóm. Đối với mỗi quốc gia, FinanceCo có các nhóm chuyên trách tập trung vào chức năng đặc thù cho quốc gia đó. Điều này rất hợp lý, vì mỗi quốc gia có những yêu cầu và thách thức cụ thể. Nhưng nó gây ra các vấn đề cho dịch vụ trung tâm này, vốn cần chức năng cụ thể được cập nhật cho mỗi quốc gia. Nhóm sở hữu microservice trung tâm bị ngập bởi các pull request được gửi đến cho nó. Nhóm đang làm rất tốt việc xử lý các pull request này một cách nhanh chóng, và thực ra nó tập trung vào việc coi đây là một phần cốt lõi trong các trách nhiệm của mình, nhưng về mặt cấu trúc tình huống thực sự không bền vững.

This is an example of where a team having a large number of pull requests can be a sign of many different potential issues. Are pull requests from other teams being taken seriously? Or are these pull requests a sign that the microservice should potentially change ownership?

Đây là một ví dụ về việc một nhóm có một số lượng lớn pull request có thể là dấu hiệu của nhiều vấn đề tiềm ẩn khác nhau. Liệu các pull request từ các nhóm khác có đang được xem xét nghiêm túc không? Hay những pull request này là dấu hiệu rằng microservice có lẽ nên đổi quyền sở hữu?



If a team has a lot of inbound pull requests, it could be a sign that you really have a microservice that is being shared by multiple teams.

Nếu một nhóm có rất nhiều pull request đến, đó có thể là dấu hiệu rằng bạn thực sự có một microservice đang được nhiều nhóm chia sẻ.

Changing ownership

Đổi quyền sở hữu

Sometimes the right thing to do is to change who owns a microservice. Consider an example in MusicCorp. The Customer Engagement team is having to send a large number of pull requests related to the Recommendation microservice to the Marketing and Promotions team. This is because that a number of changes are happening as far as how customer information is managed, and also because we need to surface these recommendations in different ways.

Đôi khi điều đúng đắn cần làm là đổi người sở hữu một microservice. Hãy xem xét một ví dụ trong MusicCorp. Nhóm Customer Engagement đang phải gửi một số lượng lớn pull request liên quan đến microservice Recommendation cho nhóm Marketing and Promotions. Điều này là vì một số thay đổi đang diễn ra về cách thông tin khách hàng được quản lý, và cũng vì chúng ta cần hiển thị những khuyến nghị này theo các cách khác nhau.

In this situation, it may make sense for the Customer Engagement team to just take ownership of the Recommendation microservice. In the example of FinanceCo, though, no such option existed. The problem was that the sources of pull requests were coming from multiple different teams. So what else could be done?

Trong tình huống này, có thể hợp lý để nhóm Customer Engagement chỉ cần nắm quyền sở hữu microservice Recommendation. Tuy nhiên, trong ví dụ của FinanceCo, không có lựa chọn như vậy tồn tại. Vấn đề là các nguồn của pull request đến từ nhiều nhóm khác nhau. Vậy còn có thể làm gì khác?

Run multiple variations

Chạy nhiều biến thể

One option we explored was that each country team would run its own variation of the shared microservice. So the US team would run its version, the Singapore team its own variation, and so on.

Một lựa chọn mà chúng ta đã khám phá là mỗi nhóm quốc gia sẽ chạy biến thể riêng của microservice dùng chung. Vậy nên nhóm Mỹ sẽ chạy phiên bản của họ, nhóm Singapore biến thể riêng của họ, và cứ thế.

Of course, the issue with this approach is code duplication. The shared microservice implemented a set of standard behaviors and common rules but also wanted some of this functionality to change for each country. We didn't want to duplicate the common functionality. The idea was for the team that currently managed the shared microservice to instead provide a framework, which really just consisted of the existing microservice with only the common functionality. Each country-specific team could launch its own instance of the skeleton microservice, plugging in its own custom functionality, as shown in Figure 15-4 .

Tất nhiên, vấn đề với cách tiếp cận này là sự trùng lặp mã. Microservice dùng chung triển khai một tập các hành vi tiêu chuẩn và các quy tắc chung nhưng cũng muốn một phần chức năng này thay đổi cho mỗi quốc gia. Chúng ta không muốn trùng lặp chức năng chung. Ý tưởng là để nhóm hiện đang quản lý microservice dùng chung thay vào đó cung cấp một framework, mà thực ra chỉ gồm microservice hiện hữu với chỉ phần chức năng chung. Mỗi nhóm đặc thù quốc gia có thể khởi chạy instance riêng của microservice khung này, cắm vào chức năng tùy biến riêng của mình, như được thể hiện trong Hình 15-4.

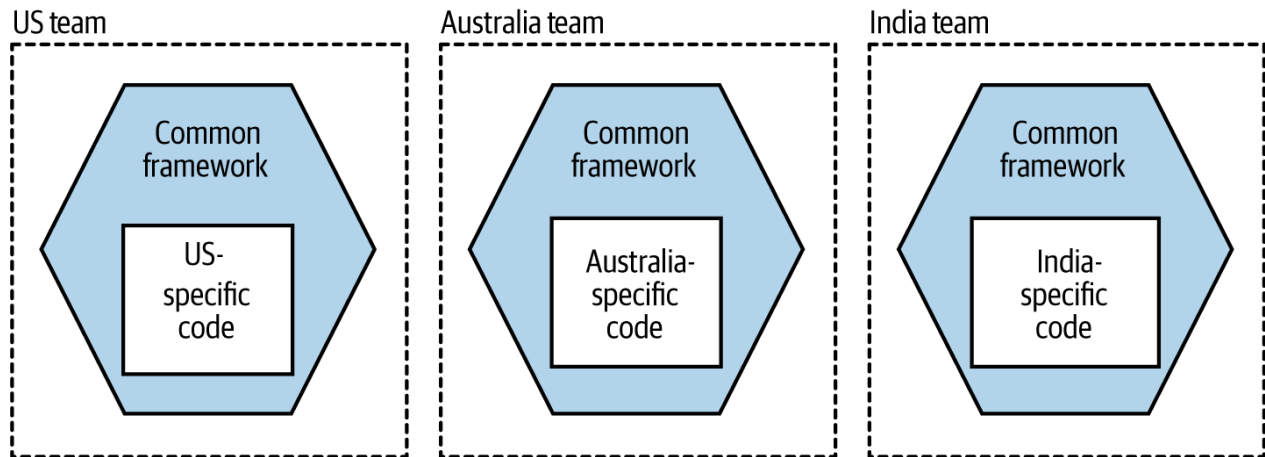


Figure 15-4. A common framework could allow multiple variations of the same microservice to be operated by different teams

Hình 15-4. Một framework chung có thể cho phép nhiều biến thể của cùng một microservice được vận hành bởi các nhóm khác nhau

The important thing to note here is that although we can share the common functionality in this example across each country-specific variation of the microservice, this common functionality cannot be updated in all variations of the microservice at the same time without requiring a large-scale lockstep release. The core team that manages the framework might make a new version available, but it would be up to each team to pull in the latest version of the common code and redeploy it. In this specific situation, FinanceCo was OK with this restriction.

Điều quan trọng cần lưu ý ở đây là mặc dù chúng ta có thể chia sẻ chức năng chung trong ví dụ này xuyên suốt mỗi biến thể đặc thù quốc gia của microservice, chức năng chung này không thể được cập nhật trong tất cả các biến thể của microservice cùng một lúc mà không đòi hỏi một đợt phát hành đồng loạt (lockstep) quy mô lớn. Nhóm core quản lý framework có thể làm cho một phiên bản mới khả dụng, nhưng sẽ tùy thuộc vào mỗi nhóm để kéo phiên bản mới nhất của mã chung vào và triển khai lại nó. Trong tình huống cụ thể này, FinanceCo chấp nhận hạn chế này.

It is worth highlighting that this specific situation was pretty rare, and something I'd encountered only once or twice before. My initial focus was on finding ways to either split apart the responsibilities of this central shared microservice or else reassign ownership. My concern was that creating internal frameworks can be a fraught activity. It's all too easy for the framework to become overly bloated or to constrain the development of the teams using it. This is the sort of problem that doesn't manifest itself on day one. When creating an internal framework, it all starts with the best intentions. Although in FinanceCo's situation I felt it was the right way forward, I'd caution against adopting this approach too readily unless you've exhausted your other options.

Đáng nhấn mạnh rằng tình huống cụ thể này khá hiếm, và là điều mà tôi chỉ gặp một hoặc hai lần trước đây. Trọng tâm ban đầu của tôi là tìm cách hoặc tách rời các trách nhiệm của microservice dùng chung trung tâm này hoặc tái phân bổ quyền sở hữu. Mối quan ngại của tôi là việc tạo các framework nội bộ có thể là một hoạt động đầy rủi ro. Quá dễ để framework trở nên quá cồng kềnh hoặc ràng buộc sự phát triển của các nhóm sử dụng nó. Đây là loại vấn đề không biểu hiện ra ngay từ ngày đầu. Khi tạo một framework nội bộ, tất cả bắt đầu với những ý định tốt đẹp nhất. Mặc dù trong tình huống của FinanceCo tôi cảm thấy đó là con đường đúng để tiến lên, tôi sẽ cảnh báo không nên áp dụng cách tiếp cận này quá dễ dàng trừ khi bạn đã cạn kiệt các lựa chọn khác.

External contribution through libraries

Đóng góp từ bên ngoài thông qua các thư viện

A variation of this approach would be to instead have each country-specific team contribute a library with its country-specific functionality inside it, and then have these libraries packaged together into a single shared microservice, as shown in Figure 15-5 .

Một biến thể của cách tiếp cận này sẽ là thay vào đó để mỗi nhóm đặc thù quốc gia đóng góp một thư viện với chức năng đặc thù quốc gia của mình bên trong nó, rồi đóng gói các thư viện này lại với nhau thành một microservice dùng chung duy nhất, như được thể hiện trong Hình 15-5.

Core service team

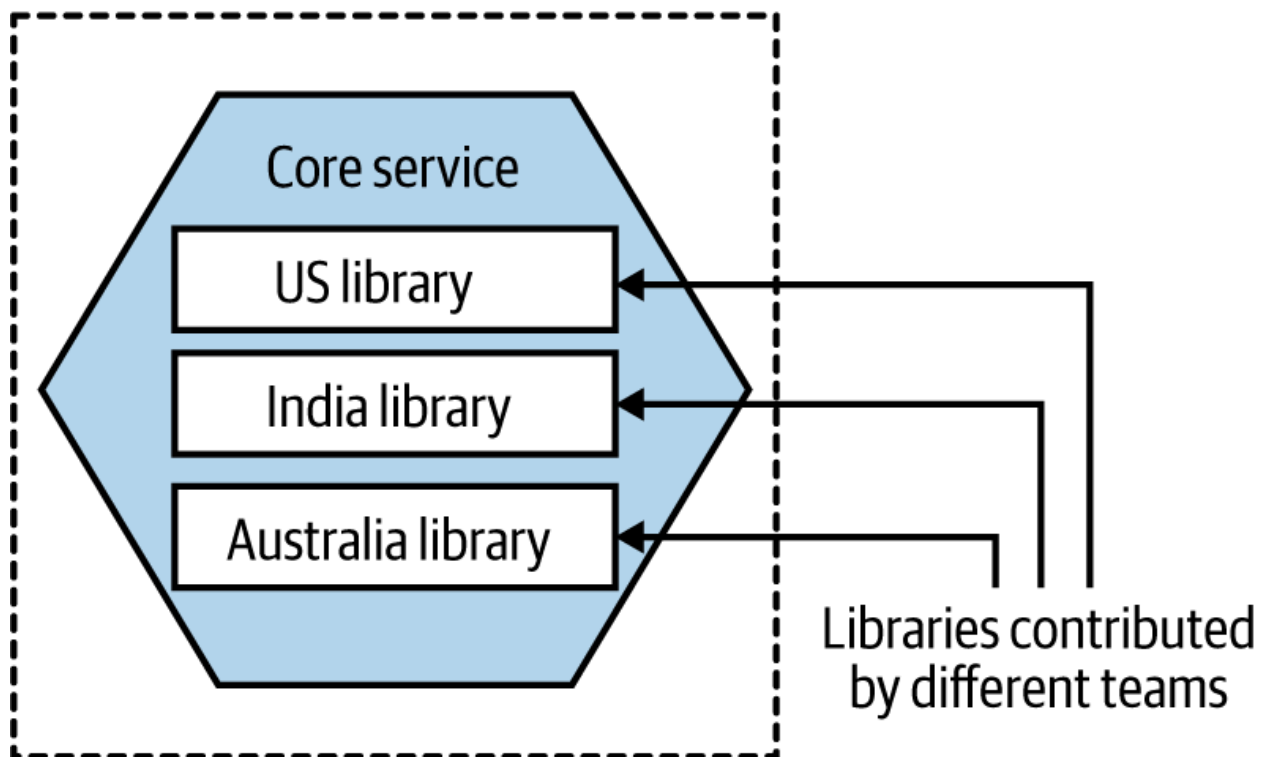


Figure 15-5. Teams contribute libraries with their custom behavior to a central microservice

Hình 15-5. Các nhóm đóng góp các thư viện với hành vi tùy biến của mình vào một microservice trung tâm

The idea here is that if the US team needs to implement US-specific logic, it makes a change in a library that is then included as part of the build of the central microservice.

Ý tưởng ở đây là nếu nhóm Mỹ cần triển khai logic đặc thù cho Mỹ, nó thực hiện một thay đổi trong một thư viện mà sau đó được đưa vào như một phần của bản build của microservice trung tâm.

This approach reduces the need to run additional microservices. We don't need to run one service per country—we can run one central microservice that handles the custom functionality for each country. The challenge here is that the country teams aren't in charge of deciding when their custom functionality goes live. They can make the change and request that this new change be deployed, but the central team would have to schedule this deployment.

Cách tiếp cận này giảm bớt nhu cầu chạy thêm các microservice. Chúng ta không cần chạy một dịch vụ cho mỗi quốc gia—chúng ta có thể chạy một microservice trung tâm xử lý chức năng tùy biến cho mỗi quốc gia. Thách thức ở đây là các nhóm quốc gia không phải là người quyết định khi nào chức năng tùy biến của họ được đưa vào hoạt động. Họ có thể thực hiện thay đổi và yêu cầu rằng thay đổi mới này được triển khai, nhưng nhóm trung tâm sẽ phải lên lịch cho đợt triển khai này.

Furthermore, it's possible that a bug in one of these country-specific libraries could cause a production issue that the central team may then be responsible for resolving. This could make production troubleshooting more complex as a result.

Hơn nữa, có khả năng một lỗi trong một trong những thư viện đặc thù quốc gia này có thể gây ra một sự cố production mà nhóm trung tâm sau đó có thể phải chịu trách nhiệm giải quyết. Điều này có thể làm cho việc gỡ rối (troubleshooting) production phức tạp hơn như một hệ quả.

Nonetheless, this option might be worth considering if it helps you move away from a central microservice being under collective ownership, especially when you cannot justify running multiple variations of the same microservice.

Tuy nhiên, lựa chọn này có thể đáng cân nhắc nếu nó giúp bạn dịch chuyển ra khỏi việc một microservice trung tâm bị đặt dưới quyền sở hữu tập thể, đặc biệt khi bạn không thể biện minh cho việc chạy nhiều biến thể của cùng một microservice.

Change Reviews — Review các thay đổi

When adopting an internal open source approach, the concept of the review is a core principle—the change has to be reviewed before the change can be accepted. But even when working inside a team on a codebase where you have direct commit permissions, there is still value in having your changes reviewed.

Khi áp dụng một cách tiếp cận mã nguồn mở nội bộ, khái niệm review là một nguyên tắc cốt lõi—thay đổi phải được review trước khi thay đổi có thể được chấp nhận. Nhưng ngay cả khi làm việc bên trong một nhóm trên một codebase nơi bạn có quyền commit trực tiếp, vẫn có giá trị trong việc có các thay đổi của bạn được review.

I am a big fan of having my changes reviewed. I've always felt that my code has benefited from a second pair of eyes. By far my most preferred form of review is the type of immediate review you get as part of pair programming. You and another developer write the code together and discuss the changes with each other. It gets reviewed before you check in.

Tôi là một người rất hâm mộ việc có các thay đổi của mình được review. Tôi luôn cảm thấy mã của tôi đã hưởng lợi từ một cặp mắt thứ hai. Cho đến nay hình thức review ưa thích nhất của tôi là kiểu review tức thì mà bạn nhận được như một phần của lập trình kết cặp (pair programming). Bạn và một lập trình viên khác viết mã cùng nhau và thảo luận các thay đổi với nhau. Nó được review trước khi bạn check in.

You don't have to take my word for it. Coming back to Accelerate , a book we've already referenced a number of times now:

Bạn không cần phải tin lời tôi. Quay lại với Accelerate, một cuốn sách mà chúng ta đã tham chiếu một số lần rồi:

We found that approval only for high-risk changes was not correlated with software delivery performance. Teams that reported no approval process or used peer review achieved higher

software delivery performance. Finally, teams that required approval by an external body achieved lower performance. Here we see a distinction drawn between peer-review and external change review. A peer change review is done by someone who is in all likelihood in the same team as you, and who works on the same codebase as you. They are obviously better placed to assess what makes for a good change and are also likely to carry out the review more quickly (more on that shortly). An external review, though, is always more fraught. As the individual is external to your team, they will likely be assessing the change against a list of criteria that may or may not make sense, and as they are in a separate team, they might not get to your changes for a while. As the authors of Accelerate note:

Chúng tôi phát hiện ra rằng việc chỉ phê duyệt cho các thay đổi rủi ro cao không tương quan với hiệu suất chuyển giao phần mềm. Các nhóm báo cáo không có quy trình phê duyệt hoặc sử dụng peer review đạt được hiệu suất chuyển giao phần mềm cao hơn. Cuối cùng, các nhóm đòi hỏi sự phê duyệt bởi một bên ngoài đạt được hiệu suất thấp hơn. Ở đây chúng ta thấy một sự phân biệt được vạch ra giữa peer review và review thay đổi bên ngoài. Một peer change review được thực hiện bởi một người mà nhiều khả năng ở cùng nhóm với bạn, và là người làm việc trên cùng codebase với bạn. Họ rõ ràng ở vị trí tốt hơn để đánh giá điều gì tạo nên một thay đổi tốt và cũng nhiều khả năng thực hiện review nhanh hơn (sẽ nói thêm về điều này sớm thôi). Tuy nhiên, một review bên ngoài luôn đầy rủi ro hơn. Vì cá nhân đó ở bên ngoài nhóm của bạn, họ nhiều khả năng sẽ đánh giá thay đổi dựa trên một danh sách các tiêu chí có thể có hoặc không có ý nghĩa, và vì họ ở trong một nhóm riêng, họ có thể sẽ không xem đến các thay đổi của bạn trong một thời gian. Như các tác giả của Accelerate lưu ý:

What are the chances that an external body, not intimately familiar with the internals of a system, can review tens of thousands of lines of code change by potentially hundreds of engineers and accurately determine the impact on a complex production system? So, in general, we want to make use of peer change reviews and avoid the need for external code reviews.

Khả năng nào để một bên ngoài, vốn không thân thuộc sâu sắc với phần bên trong của một hệ thống, có thể review hàng chục nghìn dòng thay đổi mã bởi có thể hàng trăm kỹ sư và xác định chính xác tác động lên một hệ thống production phức tạp? Vậy nên, nói chung, chúng ta muốn tận dụng các peer change review và tránh nhu cầu về các review mã bên ngoài.

Synchronous versus **asynchronous** code reviews

Review mã đồng bộ so với bất đồng bộ

With pair programming, the code review happens inline at the time the code is being written. In fact, it's more than that. When pairing, you have the driver (the person at the keyboard) and the navigator (who is acting as a second pair of eyes). Both participants are in a constant dialogue about the changes they are making—the act of review and the act of making the changes are happening at the same time. The review becomes an implicit, continual aspect of the pairing relationship. That means that as things are spotted, they are immediately fixed.

Với lập trình kết cặp, việc review mã diễn ra trực tiếp tại thời điểm mã đang được viết. Thực ra, còn hơn thế nữa. Khi kết cặp, bạn có người lái (driver, người ngồi ở bàn phím) và người dẫn đường (navigator, người đóng vai trò là một cặp mắt thứ hai). Cả hai người tham gia đều trong một cuộc đối thoại liên tục về các thay đổi mà họ đang thực hiện—hành động review và hành động thực hiện các thay đổi đang diễn ra cùng một lúc. Việc review trở thành một khía cạnh ngầm định, liên tục của mối quan hệ kết cặp. Điều đó có nghĩa là khi mọi thứ được phát hiện, chúng được sửa ngay lập tức.

If you aren't pairing, the ideal would be for the review to happen very quickly after writing the code. Then you'd like the review itself to be as synchronous as possible. You want to be able to discuss directly with the reviewer any issues they have, agree together on a way forward, make the changes, and move on.

Nếu bạn không kết cặp, lý tưởng nhất sẽ là việc review diễn ra rất nhanh sau khi viết mã. Sau đó bạn sẽ muốn chính việc review càng đồng bộ càng tốt. Bạn muốn có thể thảo luận trực tiếp với người review bất kỳ vấn đề nào họ có, cùng nhau thống nhất một con đường tiến lên, thực hiện các thay đổi, và đi tiếp.

The faster you get feedback on a code change, the faster you can look at the feedback, assess it, ask for clarification, discuss the issue further if needed, and ultimately make any necessary changes. The longer it takes between submitting the code change for review and the review actually happening, the longer and more difficult things become.

Bạn nhận được phản hồi về một thay đổi mã càng nhanh, thì bạn càng nhanh có thể xem xét phản hồi, đánh giá nó, hỏi để làm rõ, thảo luận thêm về vấn đề nếu cần, và cuối cùng thực hiện bất kỳ thay đổi cần thiết nào. Khoảng thời gian giữa việc gửi thay đổi mã để review và việc review thực sự diễn ra càng dài, thì mọi thứ càng trở nên dài hơn và khó khăn hơn.

If you submit a code change for review and don't get feedback about that change until several days later, you've likely moved on to other work. To process the feedback, you'll need to switch context and re-engage with the work you did previously. You might agree with the reviewer's changes (if required), in which case you can make them and resubmit the change for approval. Worst case, you might need to have a further discussion about the points being raised. This asynchronous back and forth between submitter and reviewer can add days to the process of getting changes made.

Nếu bạn gửi một thay đổi mã để review và không nhận được phản hồi về thay đổi đó cho đến vài ngày sau, bạn nhiều khả năng đã chuyển sang công việc khác. Để xử lý phản hồi, bạn sẽ cần chuyển ngữ cảnh và tái gắn kết với công việc bạn đã làm trước đó. Bạn có thể đồng ý với các thay đổi của người review (nếu được yêu cầu), trong trường hợp đó bạn có thể thực hiện chúng và gửi lại thay đổi để phê duyệt. Trường hợp tệ nhất, bạn có thể cần có một cuộc thảo luận thêm về các điểm đang được nêu ra. Sự qua lại bất đồng bộ này giữa người gửi và người review có thể thêm nhiều ngày vào quá trình thực hiện các thay đổi.



Do Code Reviews Promptly!

Hãy review mã một cách kịp thời!

If you want to do code reviews and aren't pair programming, do the review as quickly as possible after the change is submitted, and go through the feedback in as synchronous a way as possible,

ideally face to face with the reviewer.

Nếu bạn muốn review mã và không lập trình kết cặp, hãy thực hiện review càng nhanh càng tốt sau khi thay đổi được gửi, và đi qua phản hồi theo cách càng đồng bộ càng tốt, lý tưởng nhất là mặt đối mặt với người review.

Ensemble programming

Lập trình hợp xướng (Ensemble programming)

Ensemble programming (aka mob programming ¹⁴) is sometimes discussed as a way of doing inline code review. With ensemble programming, a larger group of people (perhaps the whole team) work together on a change. It's primarily about collectively working on a problem and taking input from a large number of individuals.

Lập trình hợp xướng (ensemble programming, hay còn gọi là mob programming¹⁴) đôi khi được thảo luận như một cách thực hiện review mã trực tiếp. Với lập trình hợp xướng, một nhóm lớn hơn (có lẽ là cả nhóm) làm việc với nhau trên một thay đổi. Nó chủ yếu là về việc cùng nhau làm việc trên một vấn đề và tiếp nhận đầu vào từ một số lượng lớn cá nhân.

Of the teams I've spoken to that make use of ensemble programming, most only use it occasionally for specific, tricky problems or important changes, but a lot of development gets done outside the ensemble exercise as well. As such, while the ensemble programming exercise would likely provide sufficient review of the changes being made during the ensemble itself, and in a very synchronous manner at that, you'd still need a way to ensure a review of changes made outside the mob.

Trong số các nhóm mà tôi đã trò chuyện có sử dụng lập trình hợp xướng, hầu hết chỉ dùng nó thỉnh thoảng cho các vấn đề hóc búa cụ thể hoặc các thay đổi quan trọng, nhưng rất nhiều việc phát triển vẫn được hoàn thành bên ngoài hoạt động hợp xướng. Như vậy, trong khi hoạt động lập trình hợp xướng nhiều khả năng sẽ cung cấp đủ sự review cho các thay đổi được thực hiện trong chính buổi hợp xướng, và theo một cách rất đồng bộ nữa, bạn vẫn sẽ cần một cách để bảo đảm việc review các thay đổi được thực hiện bên ngoài buổi hợp xướng.

Some would argue that you just need to do a review of high-risk changes, and therefore doing reviews only as part of an ensemble is sufficient. It's worth noting that the authors of Accelerate surprisingly found no correlation between software delivery performance and reviewing only high-risk changes, compared with a positive correlation when all changes are peer reviewed. So if you do want to ensemble program, go ahead! But you might want to consider reviewing other changes made outside the ensemble too.

Một số người sẽ lập luận rằng bạn chỉ cần thực hiện review cho các thay đổi rủi ro cao, và do đó việc chỉ thực hiện review như một phần của một buổi hợp xướng là đủ. Đáng lưu ý rằng các tác giả của Accelerate, một cách đáng ngạc nhiên, không tìm thấy mối tương quan nào giữa hiệu suất chuyển giao phần mềm và việc chỉ review các thay đổi rủi ro cao, so với một mối tương quan dương khi tất cả các thay đổi đều được peer review. Vậy nên nếu bạn muốn lập trình hợp xướng, cứ làm đi! Nhưng bạn có thể muốn cân nhắc việc review cả các thay đổi khác được thực hiện bên ngoài buổi hợp xướng nữa.

¹⁴ Although the term mob programming is more widespread, I dislike the picture this paints, preferring instead the term ensemble, as it makes clear that we have a collection of people working together, rather than a collection of people throwing Molotov cocktails and breaking windows. I'm not 100% sure who came up with this renaming, but I think it was Maaret Pyhäjärvi.

¹⁴ Mặc dù thuật ngữ mob programming phổ biến hơn, tôi không thích hình ảnh mà nó vẽ

ra, mà thay vào đó thích thuật ngữ ensemble (hợp xướng), vì nó làm rõ rằng chúng ta có một tập hợp những người làm việc với nhau, chứ không phải một tập hợp những người ném bom xăng và đập vỡ cửa sổ. Tôi không hoàn toàn chắc chắn ai là người đã nghĩ ra cách đổi tên này, nhưng tôi nghĩ đó là Maaret Pyhäjärvi.

Personally, I have some deep reservations around some aspects of ensemble programming. You'll find your team is in fact a neurodiverse bunch, and power imbalances in the ensemble can further undermine the goal of collective problem solving. Not everyone is comfortable working in a group, and an ensemble is definitely that. Some people will thrive in such an environment, while others will feel totally unable to contribute. When I've raised this issue with some proponents of ensemble programming, I've gotten a variety of responses, many of which boil down to the belief that if you create the right ensemble environment, anyone will be able to "come out of their shell and contribute." Let's just say that after that particular set of conversations, I rolled my eyes so much I nearly went blind. To be fair, these same concerns can be raised about pair programming as well!

Cá nhân tôi, tôi có một số e dè sâu sắc xoay quanh một số khía cạnh của lập trình hợp xướng. Bạn sẽ thấy nhóm của bạn thực ra là một tập thể đa dạng về thần kinh (neurodiverse), và sự mất cân bằng quyền lực trong buổi hợp xướng có thể làm xói mòn thêm mục tiêu giải quyết vấn đề một cách tập thể. Không phải ai cũng thoải mái khi làm việc trong một nhóm, và một buổi hợp xướng chắc chắn là như vậy. Một số người sẽ phát triển mạnh trong một môi trường như vậy, trong khi những người khác sẽ cảm thấy hoàn toàn không thể đóng góp. Khi tôi nêu vấn đề này với một số người ủng hộ lập trình hợp xướng, tôi đã nhận được nhiều phản hồi khác nhau, nhiều trong số đó tóm lại là niềm tin rằng nếu bạn tạo ra môi trường hợp xướng đúng đắn, bất kỳ ai cũng sẽ có thể "chui ra khỏi vỏ ốc và đóng góp." Cứ cho rằng sau cái loạt cuộc trò chuyện cụ thể đó, tôi đã đảo mắt nhiều đến mức suýt mù. Công bằng mà nói, những mối quan ngại tương tự này cũng có thể được nêu ra về lập trình kết cặp nữa!

While I have no doubt that many ensemble programming proponents won't be as unaware or tone-deaf as this, it's important to remember that creating an inclusive workspace is in part about understanding how to create an environment in which all members of the team are able to fully contribute in a way that is safe and comfortable for them. And don't kid yourself that just because you've got everyone in a room everyone is actually contributing. If you'd like some concrete tips on ensemble programming, then I'd suggest reading Maaret Pyhäjärvi's self-published and concise Ensemble Programming Guidebook . 15

Trong khi tôi không nghi ngờ gì rằng nhiều người ủng hộ lập trình hợp xướng sẽ không vô tâm hoặc điếc trước cảm xúc của người khác đến mức này, điều quan trọng cần nhớ là việc tạo ra một không gian làm việc hòa nhập một phần là về việc hiểu cách tạo ra một môi trường trong đó tất cả các thành viên của nhóm có thể đóng góp trọn vẹn theo một cách an toàn và thoải mái cho họ. Và đừng tự lừa dối mình rằng chỉ vì bạn đã đưa tất cả mọi người vào một căn phòng thì tất cả mọi người đều thực sự đang đóng góp. Nếu bạn muốn một số mẹo cụ thể về lập trình hợp xướng, thì tôi sẽ gợi ý đọc cuốn Ensemble Programming Guidebook tự xuất bản và súc tích của Maaret Pyhäjärvi.¹⁵

The Orphaned Service — Dịch vụ mồ côi (The Orphaned Service)

So what about services that are no longer being actively maintained? As we move toward finer-grained architectures, the microservices themselves become smaller. One of the advantages of smaller microservices, as we have discussed, is the fact that they are simpler. Simpler microservices with less functionality may not need to change for a while. Consider the humble Shopping Basket

microservice, which provides some fairly modest capabilities: Add to Basket, Remove from Basket, and so on. It is quite conceivable that this microservice may not have to change for months after first being written, even if active development is still going on. What happens here? Who owns this microservice?

Vậy còn các dịch vụ không còn được bảo trì tích cực thì sao? Khi chúng ta dịch chuyển sang các kiến trúc chi tiết hơn, bản thân các microservice trở nên nhỏ hơn. Một trong những lợi thế của các microservice nhỏ hơn, như chúng ta đã thảo luận, là việc chúng đơn giản hơn. Các microservice đơn giản hơn với ít chức năng hơn có thể không cần thay đổi trong một thời gian. Hãy xem xét microservice Shopping Basket khiêm tốn, vốn cung cấp một số khả năng khá khiêm tốn: Thêm vào Giỏ, Xóa khỏi Giỏ, và cứ thế. Hoàn toàn có thể hình dung rằng microservice này có thể không phải thay đổi trong nhiều tháng sau khi được viết lần đầu, ngay cả khi việc phát triển tích cực vẫn đang diễn ra. Điều gì xảy ra ở đây? Ai sở hữu microservice này?

If your team structures are aligned along the bounded contexts of your organization, then even services that are not changed frequently still have a de facto owner. Imagine a team that is aligned with the consumer web sales context. It might handle the web-based user **interface** and the Shopping Basket and Recommendation microservices. Even if the cart service hasn't been changed in months, it would naturally fall to this team to make changes if required. One of the benefits of microservices, of course,

Nếu các cấu trúc nhóm của bạn được giống hàng dọc theo các bounded context của tổ chức bạn, thì ngay cả các dịch vụ không được thay đổi thường xuyên vẫn có một chủ sở hữu trên thực tế (de facto). Hãy tưởng tượng một nhóm được giống hàng với bounded context bán hàng web cho người tiêu dùng. Nó có thể xử lý giao diện người dùng trên nền web cùng các microservice Shopping Basket và Recommendation. Ngay cả khi dịch vụ cart không được thay đổi trong nhiều tháng, một cách tự nhiên sẽ thuộc về nhóm này để thực hiện các thay đổi nếu cần. Một trong những lợi ích của microservice, tất nhiên,

15 Maaret Pyhäjärvi, Ensemble Programming Guidebook (self-pub., 2015–2020), <https://ensembleprogramming.xyz>.

15 Maaret Pyhäjärvi, Ensemble Programming Guidebook (self-pub., 2015–2020), <https://ensembleprogramming.xyz>.

is that if the team needs to change the microservice to add a new feature and does not find the feature to its liking, rewriting it shouldn't take too long at all.

là nếu nhóm cần thay đổi microservice để thêm một tính năng mới và không thấy tính năng đó vừa ý, việc viết lại nó sẽ không tốn quá nhiều thời gian.

That said, if you've adopted a truly polyglot approach and are making use of multiple technology stacks, then the challenges of making changes to an orphaned service could be compounded if your team doesn't know the tech stack any longer.

Như đã nói, nếu bạn đã áp dụng một cách tiếp cận thực sự đa ngôn ngữ (polyglot) và đang sử dụng nhiều tập công nghệ, thì những thách thức của việc thực hiện các thay đổi đối với một dịch vụ mồ côi có thể tăng lên nếu nhóm của bạn không còn biết tập công nghệ đó nữa.

Case Study: realestate.com.au — Nghiên cứu tình huống: realestate.com.au

For the first edition of this book, I spent some time chatting with realestate.com.au (REA) about its use of microservices, and a lot of what I learned helped greatly in terms of sharing real-world examples of microservices in action. I also found REA's interplay of organizational structure and architecture to be especially fascinating. This overview of its organizational structure is based on our discussions back in 2014.

Đối với ấn bản đầu tiên của cuốn sách này, tôi đã dành một chút thời gian trò chuyện với realestate.com.au (REA) về việc họ sử dụng microservice, và rất nhiều điều tôi học được đã giúp ích rất nhiều về mặt chia sẻ các ví dụ thực tế về microservice trong thực hành. Tôi cũng thấy sự tương tác qua lại giữa cấu trúc tổ chức và kiến trúc của REA đặc biệt hấp dẫn. Tổng quan này về cấu trúc tổ chức của họ dựa trên các cuộc thảo luận của chúng tôi vào năm 2014.

I am certain that what REA looks like today is quite different. This overview represents a snapshot, a point in time. I'm not suggesting that this is the best way to structure an organization—just that it is what worked best for REA at the time. Learning from other organizations is sensible; copying what they do without understanding why they do it is foolish.

Tôi chắc chắn rằng REA trông như thế nào ngày hôm nay là khá khác. Tổng quan này đại diện cho một ảnh chụp nhanh, một thời điểm. Tôi không gợi ý rằng đây là cách tốt nhất để cấu trúc một tổ chức—chỉ là nó là cái đã hoạt động tốt nhất cho REA vào thời điểm đó. Học hỏi từ các tổ chức khác là điều hợp lý; sao chép những gì họ làm mà không hiểu tại sao họ làm điều đó là điều dại dột.

As it does today, REA's core business of real estate encompassed different facets. In 2014, REA was split into independent lines of business (LOBs). For example, one line of business dealt with residential property in Australia, another with commercial property, and another line handled one of REA's overseas businesses. These lines of business had IT delivery teams (or "squads") associated with them; only some had a single squad, while the biggest line had four squads. So for residential property, there were multiple teams involved with creating the website and listing services to allow people to browse property. People rotated between these teams every now and then but tended to stay within that line of business for extended periods, ensuring that the team members could build up a strong awareness of that part of the domain. This in turn helped the communication between the various business stakeholders and the team delivering features for them.

Như nó vẫn vậy ngày hôm nay, hoạt động kinh doanh cốt lõi của REA về bất động sản bao gồm các khía cạnh khác nhau. Năm 2014, REA được chia thành các mảng kinh doanh (lines of business, LOB) độc lập. Ví dụ, một mảng kinh doanh xử lý bất động sản nhà ở tại Úc, một mảng khác về bất động sản thương mại, và một mảng nữa xử lý một trong các hoạt động kinh doanh ở nước ngoài của REA. Các mảng kinh doanh này có các nhóm chuyên giao IT (hay "squad") gắn với chúng; chỉ một số có một squad duy nhất, trong khi mảng lớn nhất có bốn squad. Vậy nên đối với bất động sản nhà ở, có nhiều nhóm tham gia vào việc tạo ra website và các dịch vụ niêm yết (listing) để cho phép mọi người duyệt bất động sản. Mọi người luân chuyển giữa các nhóm này thỉnh thoảng nhưng có xu hướng ở lại trong mảng kinh doanh đó trong những giai đoạn kéo dài, bảo đảm rằng các thành viên nhóm có thể xây dựng được một sự am hiểu mạnh mẽ về phần đó của miền nghiệp vụ. Điều này lại giúp cho việc giao tiếp giữa các bên liên quan kinh doanh khác nhau và nhóm chuyên giao các tính năng cho họ.

Each squad inside a line of business was expected to own the entire life cycle of each service it

created, including building, testing and releasing, supporting, and even decommissioning. The core delivery services team had the job of providing advice, guidance, and tooling to the squads in the LOBs, helping these squads deliver more effectively. Using our newer terminology, the core delivery services team was playing the role of an enabling team. A strong culture of automation was key, and REA made heavy use of AWS as an important part of enabling the teams to be more autonomous. Figure 15-6 illustrates how this all worked.

Mỗi squad bên trong một mảng kinh doanh được kỳ vọng sở hữu toàn bộ vòng đời của mỗi dịch vụ mà nó tạo ra, bao gồm xây dựng, kiểm thử và phát hành, hỗ trợ, và thậm chí ngừng hoạt động. Nhóm core delivery services có công việc cung cấp lời khuyên, hướng dẫn, và công cụ cho các squad trong các LOB, giúp các squad này chuyển giao hiệu quả hơn. Sử dụng thuật ngữ mới hơn của chúng ta, nhóm core delivery services đang đóng vai trò của một enabling team. Một văn hóa tự động hóa mạnh mẽ là then chốt, và REA sử dụng AWS rất nhiều như một phần quan trọng trong việc cho phép các nhóm trở nên tự chủ hơn. Hình 15-6 minh họa cách tất cả những điều này hoạt động.

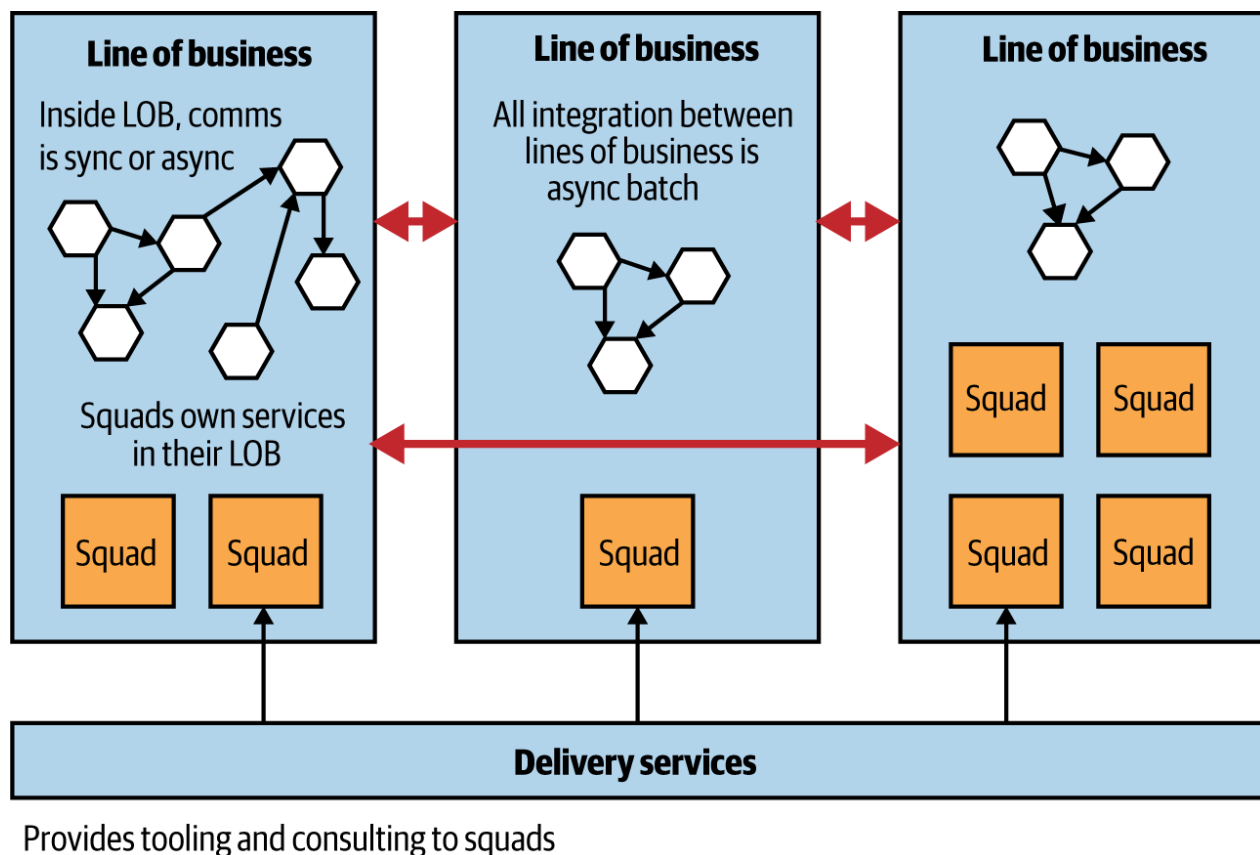


Figure 15-6. An overview of realestate.com.au's organizational and team structure and alignment with architecture

Hình 15-6. Một tổng quan về cấu trúc tổ chức và nhóm của realestate.com.au và sự giống hệt với kiến trúc

It wasn't just the delivery organization that was aligned to how the business operated. This model extended to the architecture too. One example of this was the integration methods. Within an LOB, all services were free to talk to each other in any way they saw fit, as decided by the squads who act as their custodians. But communication between LOBs was mandated to be asynchronous batch, one of the few cast-iron rules of the very small architecture team. This coarse-grained communication

matched that which existed between the different parts of the business too. By insisting on it being batch, each LOB had a lot of freedom in how it acted and how it managed itself. It could afford to take its services down whenever it wanted, knowing that as long as it could satisfy the batch integration with other parts of the business and its own business stakeholders, no one would care.

Không chỉ tổ chức chuyển giao được giống hàng với cách hoạt động kinh doanh. Mô hình này còn mở rộng đến cả kiến trúc. Một ví dụ về điều này là các phương pháp tích hợp. Trong một LOB, tất cả các dịch vụ được tự do giao tiếp với nhau theo bất kỳ cách nào chúng thấy phù hợp, như được quyết định bởi các squad đóng vai trò là người gìn giữ chúng. Nhưng việc giao tiếp giữa các LOB được bắt buộc phải là batch bất đồng bộ, một trong số ít các quy tắc cứng rắn như sắt của nhóm kiến trúc rất nhỏ. Sự giao tiếp ở mức thô (coarse-grained) này khớp với sự giao tiếp tồn tại giữa các bộ phận khác nhau của doanh nghiệp nữa. Bằng cách khẳng định rằng nó là batch, mỗi LOB có rất nhiều tự do trong cách nó hành động và cách nó tự quản lý mình. Nó có thể thoải mái tắt các dịch vụ của mình bất cứ khi nào nó muốn, biết rằng miễn là nó có thể thỏa mãn việc tích hợp batch với các bộ phận khác của doanh nghiệp và các bên liên quan kinh doanh của riêng nó, không ai sẽ bận tâm.

This structure allowed for significant autonomy not only among the teams but also among the different parts of the business, and the ability to deliver change helped the company achieve significant success in the local market. This more autonomous structure also helped grow the company from a handful of services in 2010 to hundreds by 2014, facilitating the ability to deliver change faster.

Cấu trúc này cho phép một sự tự chủ đáng kể không chỉ giữa các nhóm mà còn giữa các bộ phận khác nhau của doanh nghiệp, và khả năng chuyển giao thay đổi đã giúp công ty đạt được thành công đáng kể trên thị trường địa phương. Cấu trúc tự chủ hơn này cũng giúp phát triển công ty từ một dúm dịch vụ vào năm 2010 lên hàng trăm dịch vụ vào năm 2014, tạo điều kiện cho khả năng chuyển giao thay đổi nhanh hơn.

Those organizations that are adaptive enough to change not only their system architecture but also their organizational structure can reap huge benefits in terms of improved autonomy of teams and faster time to market for new features and functionality. REA is just one of a number of organizations that have realized that system architecture doesn't exist in a vacuum.

Những tổ chức đủ thích nghi để thay đổi không chỉ kiến trúc hệ thống của mình mà còn cả cấu trúc tổ chức có thể gặt hái những lợi ích to lớn về mặt cải thiện tính tự chủ của các nhóm và thời gian ra thị trường nhanh hơn cho các tính năng và chức năng mới. REA chỉ là một trong số nhiều tổ chức đã nhận ra rằng kiến trúc hệ thống không tồn tại trong chân không.

Geographical Distribution — Phân tán địa lý

Colocated teams will find synchronous communication to be very simple, especially as they are typically in the same place at the same time. If your team is distributed, synchronous communication can be more difficult, but it is still achievable if the team members are in the same or similar time zones. When communicating with people in various different time zones, the cost of coordination can increase drastically. In one of my previous roles I worked as an architect, helping support teams based in India, the UK, Brazil, and the US, while I myself was based in Australia. Setting up a meeting between myself and the leads of the various teams was incredibly difficult. This meant that we held these meetings infrequently (typically monthly), and we also had to make sure to discuss only the

most important issues during these sessions, as often more than half of the attendees would be working outside their core working hours.

Các nhóm đặt cùng một chỗ sẽ thấy việc giao tiếp đồng bộ rất đơn giản, đặc biệt là vì họ thường ở cùng một nơi vào cùng một thời điểm. Nếu nhóm của bạn phân tán, việc giao tiếp đồng bộ có thể khó khăn hơn, nhưng nó vẫn khả thi nếu các thành viên nhóm ở cùng hoặc gần cùng múi giờ. Khi giao tiếp với những người ở các múi giờ khác nhau, chi phí phối hợp có thể tăng lên đột ngột. Trong một vai trò trước đây của tôi, tôi đã làm việc với tư cách là một kiến trúc sư, giúp hỗ trợ các nhóm đặt tại Ấn Độ, Anh, Brazil, và Mỹ, trong khi bản thân tôi đặt tại Úc. Việc thiết lập một cuộc họp giữa tôi và các trưởng nhóm của các nhóm khác nhau là điều cực kỳ khó khăn. Điều này có nghĩa là chúng tôi tổ chức các cuộc họp này không thường xuyên (thường là hằng tháng), và chúng tôi cũng phải bảo đảm chỉ thảo luận những vấn đề quan trọng nhất trong các buổi này, vì thường thì hơn một nửa số người tham dự sẽ làm việc ngoài giờ làm việc chính của họ.

Outside of these sessions, we would have asynchronous communication, largely via email, about other less time-critical issues. But with me in Australia, the delay in this form of communication was significant. I would wake up on a Monday morning to have a pretty quiet start to the week, as most of the world hadn't woken up—this would give me time to process the emails I had received from teams in the UK, Brazil, and the US on their Friday afternoon, which was my Saturday morning.

Ngoài các buổi này, chúng tôi sẽ có giao tiếp bất đồng bộ, phần lớn qua email, về các vấn đề khác ít cấp bách về thời gian hơn. Nhưng với việc tôi ở Úc, độ trễ trong hình thức giao tiếp này là đáng kể. Tôi sẽ thức dậy vào một sáng thứ Hai để có một khởi đầu khá im ắng cho tuần làm việc, vì hầu hết thế giới chưa thức dậy—điều này sẽ cho tôi thời gian để xử lý các email mà tôi đã nhận được từ các nhóm ở Anh, Brazil, và Mỹ vào chiều thứ Sáu của họ, vốn là sáng thứ Bảy của tôi.

I recall one client project I worked on in which ownership of a single microservice was shared between two geographical locations. Eventually, each site started specializing the work it handled. This allowed it to take ownership of part of the codebase, within which it could have an easier cost of change. The teams then had more coarse-grained communication about how the two parts interrelated; effectively, the communication pathways made possible within the organizational structure matched the coarse-grained API that formed the boundary between the two halves of the codebase.

Tôi nhớ một dự án khách hàng mà tôi đã làm việc trong đó quyền sở hữu của một microservice duy nhất được chia sẻ giữa hai vị trí địa lý. Cuối cùng, mỗi địa điểm bắt đầu chuyên môn hóa công việc mà nó xử lý. Điều này cho phép nó nắm quyền sở hữu một phần của codebase, mà trong đó nó có thể có một chi phí thay đổi dễ dàng hơn. Sau đó các nhóm có sự giao tiếp ở mức thô hơn về cách hai phần liên quan với nhau; thực chất, các con đường giao tiếp được tạo điều kiện trong cấu trúc tổ chức khớp với API ở mức thô tạo thành ranh giới giữa hai nửa của codebase.

So where does this leave us when considering evolving our own service design? Well, I would suggest that geographical boundaries between people involved with the development should be a strong consideration when looking to define both team boundaries and software boundaries. It's much easier for a single team to form when its members are colocated. If colocation isn't possible, and you are looking to form a distributed team, then ensuring that team members are in the same or very similar time zones will aid communication within that team, as it will reduce the need for asynchronous communication.

Vậy điều này để lại chúng ta ở đâu khi cân nhắc việc tiến hóa thiết kế dịch vụ của riêng mình? Tôi sẽ gợi ý rằng các ranh giới địa lý giữa những người tham gia vào việc phát triển

nên là một cân nhắc mạnh mẽ khi tìm cách định nghĩa cả ranh giới nhóm lẫn ranh giới phần mềm. Dễ dàng hơn nhiều để một nhóm duy nhất hình thành khi các thành viên của nó được đặt cùng một chỗ. Nếu việc đặt cùng một chỗ là không thể, và bạn đang tìm cách hình thành một nhóm phân tán, thì việc bảo đảm rằng các thành viên nhóm ở cùng hoặc rất gần cùng múi giờ sẽ hỗ trợ việc giao tiếp trong nhóm đó, vì nó sẽ giảm bớt nhu cầu về giao tiếp bất đồng bộ.

Perhaps your organization decides that it wants to increase the number of people working on your project by opening up an office in another country. At this point, you should think actively about what parts of your system can be moved over. Perhaps this is what drives your decisions about what functionality to split out next.

Có lẽ tổ chức của bạn quyết định rằng nó muốn tăng số lượng người làm việc trên dự án của bạn bằng cách mở một văn phòng ở một quốc gia khác. Tại thời điểm này, bạn nên tích cực suy nghĩ về những bộ phận nào của hệ thống có thể được chuyển sang. Có lẽ đây là điều thúc đẩy các quyết định của bạn về việc tách ra chức năng nào tiếp theo.

It is also worth noting at this point that, at least based on the observations of the authors of the “Exploring the Duality Between Product and Organizational Architectures” report I referenced earlier, if the organization building the system is more loosely coupled (e.g., it consists of geographically distributed teams), the systems being built will tend to be more modular, and therefore hopefully less coupled. The tendency of a single team that owns many services to lean toward tighter integration is very hard to maintain in a more distributed organization.

Cũng đáng lưu ý tại thời điểm này rằng, ít nhất dựa trên các quan sát của các tác giả báo cáo “Exploring the Duality Between Product and Organizational Architectures” mà tôi đã tham chiếu trước đó, nếu tổ chức xây dựng hệ thống có coupling lỏng hơn (ví dụ, nó gồm các nhóm phân tán về mặt địa lý), các hệ thống được xây dựng sẽ có xu hướng mô-đun hóa nhiều hơn, và do đó hy vọng là ít coupling hơn. Xu hướng của một nhóm duy nhất sở hữu nhiều dịch vụ nghiêng về tích hợp chặt chẽ hơn là rất khó để duy trì trong một tổ chức phân tán hơn.

Conway’s Law in Reverse — Định luật Conway theo chiều ngược lại

So far, we’ve spoken about how the organization impacts the system design. But what about the reverse? Namely, can a system design change the organization? While I haven’t been able to find the same quality of evidence to support the idea that Conway’s law works in reverse, I’ve seen it anecdotally.

Cho đến nay, chúng ta đã nói về cách tổ chức tác động đến thiết kế hệ thống. Nhưng còn chiều ngược lại thì sao? Cụ thể, liệu một thiết kế hệ thống có thể thay đổi tổ chức không? Trong khi tôi chưa thể tìm thấy cùng chất lượng bằng chứng để ủng hộ ý tưởng rằng Định luật Conway hoạt động theo chiều ngược lại, tôi đã thấy nó một cách giai thoại.

Probably the best example was a client I worked with many years ago. Back in the days when the web was fairly nascent, and the internet was seen as something that arrived on an AOL floppy disk through the door, this company was a large print firm with a small, modest website. It had a website because that was the thing to do, but in the grand scheme of things the website was fairly unimportant to how the business operated. When the original system was created, a fairly arbitrary technical decision was made as to how the system would work.

Có lẽ ví dụ tốt nhất là một khách hàng mà tôi làm việc cùng nhiều năm trước. Trở lại những ngày khi web còn khá non trẻ, và internet được xem như thứ gì đó đến qua một chiếc đĩa mềm AOL gửi qua cửa, công ty này là một hãng in ấn lớn với một website nhỏ, khiêm tốn. Nó có một website vì đó là điều nên làm, nhưng trong bức tranh tổng thể website khá không quan trọng đối với cách doanh nghiệp hoạt động. Khi hệ thống ban đầu được tạo ra, một quyết định kỹ thuật khá tùy tiện đã được đưa ra về cách hệ thống sẽ hoạt động.

The content for this system was sourced in multiple ways, but most of it came from third parties that were placing ads for viewing by the general public. There was an input system that allowed content to be created by the paying third parties, a central system that took that data and enriched it in various ways, and an output system that created the final website that the general public could browse.

Nội dung cho hệ thống này được lấy theo nhiều cách, nhưng phần lớn đến từ các bên thứ ba đang đặt quảng cáo để công chúng xem. Có một hệ thống đầu vào cho phép nội dung được tạo ra bởi các bên thứ ba trả tiền, một hệ thống trung tâm lấy dữ liệu đó và làm giàu nó theo nhiều cách, và một hệ thống đầu ra tạo ra website cuối cùng mà công chúng có thể duyệt.

Whether the original design decisions were right at the time is a conversation for historians, but many years on the company had changed quite a bit, and I and many of my colleagues were starting to wonder if the system design was fit for the company's present state. Its physical print business had diminished significantly, and the revenues and therefore business operations of the organization were now dominated by its online presence.

Liệu các quyết định thiết kế ban đầu có đúng vào thời điểm đó hay không là một cuộc trò chuyện dành cho các nhà sử học, nhưng nhiều năm sau công ty đã thay đổi khá nhiều, và tôi cùng nhiều đồng nghiệp của mình bắt đầu tự hỏi liệu thiết kế hệ thống có còn phù hợp với trạng thái hiện tại của công ty không. Hoạt động kinh doanh in ấn vật lý của nó đã suy giảm đáng kể, và doanh thu và do đó các hoạt động kinh doanh của tổ chức giờ đây bị chi phối bởi sự hiện diện trực tuyến của nó.

What we saw at that time was an organization tightly aligned to this three-part system. Three channels or divisions in the IT side of the business aligned with each of the input, core, and output parts of the business. Within those channels, there were separate delivery teams. What I didn't realize at the time was that these organizational structures didn't predate the system design but actually grew up around it. As the print side of the business diminished and the digital side of the business grew, the system design inadvertently laid the path for how the organization grew.

Những gì chúng tôi thấy vào thời điểm đó là một tổ chức được giống hàng chặt chẽ với hệ thống ba phần này. Ba kênh hoặc bộ phận trong phía IT của doanh nghiệp giống hàng với mỗi phần đầu vào, lõi, và đầu ra của doanh nghiệp. Trong những kênh đó, có các nhóm chuyển giao riêng biệt. Điều mà tôi không nhận ra vào thời điểm đó là những cấu trúc tổ chức này không có trước thiết kế hệ thống mà thực ra mọc lên xung quanh nó. Khi phía in ấn của doanh nghiệp suy giảm và phía số hóa của doanh nghiệp phát triển, thiết kế hệ thống đã vô tình vạch ra con đường cho cách tổ chức phát triển.

In the end we realized that whatever the shortcomings of the system design were, we would have to make changes to the organizational structure to make a shift. The company is now much changed, but it happened over a period of many years.

Cuối cùng chúng tôi nhận ra rằng dù những thiếu sót của thiết kế hệ thống là gì, chúng tôi sẽ phải thực hiện các thay đổi đối với cấu trúc tổ chức để tạo ra một sự dịch chuyển. Công ty giờ đây đã thay đổi rất nhiều, nhưng nó diễn ra qua một giai đoạn nhiều năm.

People — Con người

No matter how it looks at first, it's always a people problem. —Gerry Weinberg, The Second Law of Consulting

Bất kể nó trông như thế nào ban đầu, nó luôn là một vấn đề về con người. —Gerry Weinberg, Định luật Thứ hai của Tư vấn

We have to accept that in a microservice environment, it is harder for a developer to think about writing code in their own little world. They have to be more aware of the implications of things like calls across network boundaries or the implications of failure. We've also talked about the ability of microservices to make it easier to try out new technologies, from data stores to languages. But if you're moving from a world in which you have a monolithic system, where the majority of your developers have had to use only one language and have remained completely oblivious to the operational concerns, throwing them into the world of microservices may be a rude awakening for them.

Chúng ta phải chấp nhận rằng trong một môi trường microservice, một lập trình viên sẽ khó hơn để nghĩ về việc viết mã trong thế giới nhỏ bé của riêng mình. Họ phải nhận thức hơn về những hàm ý của những thứ như các lời gọi vượt qua ranh giới mạng hoặc những hàm ý của sự thất bại. Chúng ta cũng đã nói về khả năng của microservice trong việc làm cho việc thử nghiệm các công nghệ mới trở nên dễ dàng hơn, từ các kho dữ liệu (data store) đến các ngôn ngữ. Nhưng nếu bạn đang chuyển từ một thế giới trong đó bạn có một hệ thống kiểu monolith, nơi phần lớn các lập trình viên của bạn chỉ phải dùng một ngôn ngữ và đã hoàn toàn không biết gì về các mối quan tâm vận hành, thì việc ném họ vào thế giới của microservice có thể là một cú thức tỉnh thô bạo đối với họ.

Likewise, pushing power into development teams to increase autonomy can be fraught. People who in the past have thrown work over the wall to someone else are accustomed to having someone else to blame and may not feel comfortable being fully accountable for their work. You may even find contractual barriers to having your developers carry pagers for the systems they support! These changes can be made in a gradual way, and initially it will make sense to look to change responsibilities for people who are most willing and able to make the change.

Tương tự, việc đẩy quyền lực vào các nhóm phát triển để tăng tính tự chủ có thể đầy rủi ro. Những người trong quá khứ đã ném công việc qua tường cho ai đó khác đã quen với việc có ai đó khác để đổ lỗi và có thể không cảm thấy thoải mái khi hoàn toàn chịu trách nhiệm giải trình cho công việc của mình. Bạn thậm chí có thể thấy các rào cản về mặt hợp đồng đối với việc để các lập trình viên của bạn đeo máy nhắn tin (pager) cho các hệ thống mà họ hỗ trợ! Những thay đổi này có thể được thực hiện theo một cách dần dần, và ban đầu sẽ hợp lý để tìm cách thay đổi trách nhiệm cho những người sẵn lòng và có khả năng nhất để thực hiện thay đổi.

Although this book has mostly been about technology, people are not just a side consideration; they are the people who built what you have now and will build what happens next. Coming up with a vision for how things should be done without considering how your current staff will feel about it and without considering what capabilities they have is likely to lead to a bad place.

Mặc dù cuốn sách này phần lớn nói về công nghệ, con người không chỉ là một cân nhắc phụ; họ là những người đã xây dựng những gì bạn có bây giờ và sẽ xây dựng những gì xảy ra tiếp theo. Việc đưa ra một tầm nhìn về cách mọi thứ nên được thực hiện mà không cân nhắc việc nhân viên hiện tại của bạn sẽ cảm thấy thế nào về nó và không cân nhắc những năng lực mà họ có nhiều khả năng sẽ dẫn đến một nơi tồi tệ.

Each organization has its own set of dynamics around this topic. Understand your staff’s appetite for change. Don’t push them too fast! Maybe you can have a separate team handle frontline support or deployment for a short period, giving your developers time to adjust to new practices. You may, however, have to accept that you need different sorts of people in your organization to make this all work. You will likely need to change how you hire—in fact, it might be easier to show what is possible by bringing in new people from outside who already have experience in working the way you want. The right new hires may well go a long way toward showing others what is possible.

Mỗi tổ chức có tập động lực riêng xoay quanh chủ đề này. Hãy hiểu sự sẵn lòng thay đổi của nhân viên của bạn. Đừng thúc ép họ quá nhanh! Có lẽ bạn có thể có một nhóm riêng xử lý hỗ trợ tuyến đầu hoặc triển khai trong một giai đoạn ngắn, cho các lập trình viên của bạn thời gian để điều chỉnh với các thực hành mới. Tuy nhiên, bạn có thể phải chấp nhận rằng bạn cần những loại người khác nhau trong tổ chức của mình để làm cho tất cả những điều này vận hành. Bạn nhiều khả năng sẽ cần thay đổi cách bạn tuyển dụng—thực ra, có thể dễ dàng hơn để cho thấy điều gì là khả thi bằng cách mang vào những người mới từ bên ngoài đã có kinh nghiệm làm việc theo cách bạn muốn. Những người mới được tuyển dụng phù hợp rất có thể đi một chặng đường dài hướng tới việc cho người khác thấy điều gì là khả thi.

Whatever your approach, understand that you need to be clear in articulating the responsibilities of your people in a microservices world, and also be clear about why those responsibilities are important to you. This can help you see what your skill gaps might be and think about how to close them. For many, this will be a pretty scary journey. Just remember that without people on board, any change you might want to make could be doomed from the start.

Dù cách tiếp cận của bạn là gì, hãy hiểu rằng bạn cần phải rõ ràng trong việc diễn đạt các trách nhiệm của con người của bạn trong một thế giới microservice, và cũng rõ ràng về lý do tại sao những trách nhiệm đó quan trọng đối với bạn. Điều này có thể giúp bạn thấy những lỗ hổng kỹ năng của mình có thể là gì và suy nghĩ về cách lấp đầy chúng. Đối với nhiều người, đây sẽ là một hành trình khá đáng sợ. Chỉ cần nhớ rằng nếu không có con người tham gia, bất kỳ thay đổi nào bạn có thể muốn thực hiện đều có thể bị tiêu vong ngay từ đầu.

Summary — Tóm tắt

Conway’s law highlights the perils of trying to enforce a system design that doesn’t match the organization. This points us, for microservices at least, toward a model where strong ownership of microservices is the norm. Sharing microservices or attempting to practice collective ownership at scale can often result in us undermining the benefits of microservices.

Định luật Conway làm nổi bật những hiểm họa của việc cố gắng áp đặt một thiết kế hệ thống không khớp với tổ chức. Điều này chỉ cho chúng ta, ít nhất là đối với microservice, hướng tới một mô hình trong đó quyền sở hữu mạnh đối với microservice là chuẩn mực. Việc chia sẻ microservice hoặc cố gắng thực hành quyền sở hữu tập thể ở quy mô lớn thường có thể dẫn đến việc chúng ta làm xói mòn những lợi ích của microservice.

When organization and architecture are not in alignment, we get tension points, as outlined throughout this chapter. By recognizing the link between the two, we’ll make sure the system we are trying to build makes sense for the organization we’re building it for.

Khi tổ chức và kiến trúc không giống hệt nhau, chúng ta nhận được các điểm căng thẳng, như được phác họa xuyên suốt chương này. Bằng cách nhận ra mối liên kết giữa hai thứ, chúng

ta sẽ bảo đảm rằng hệ thống mà chúng ta đang cố gắng xây dựng có ý nghĩa đối với tổ chức mà chúng ta đang xây dựng nó cho.

If you would like to explore this topic further, in addition to the previously mentioned Team Topologies , I can also thoroughly recommend the talk “Scale, Microservices and Flow,” by James Lewis, 16 from which I gained many insights that helped shape this chapter. It is well worth a watch if you are interested in delving more deeply into some of the ideas I have covered in this chapter.

Nếu bạn muốn khám phá chủ đề này sâu hơn, bên cạnh cuốn Team Topologies đã được đề cập trước đó, tôi cũng có thể nhiệt thành giới thiệu bài nói “Scale, Microservices and Flow,” của James Lewis,¹⁶ từ đó tôi đã thu được nhiều cái nhìn sâu giúp định hình chương này. Nó rất đáng xem nếu bạn quan tâm đến việc đào sâu hơn vào một số ý tưởng mà tôi đã đề cập trong chương này.

In our next chapter, we will explore in more depth a topic I’ve already touched on— the role of the architect.

Trong chương tiếp theo của chúng ta, chúng ta sẽ khám phá sâu hơn một chủ đề mà tôi đã đề cập—vai trò của kiến trúc sư.

16 James Lewis, “Scale, Microservices and Flow,” YOW! Conferences, February 10, 2020, YouTube video, 51:03, <https://oreil.ly/ON81J> .

16 James Lewis, “Scale, Microservices and Flow,” YOW! Conferences, February 10, 2020, YouTube video, 51:03, <https://oreil.ly/ON81J>.

CHAPTER 16 The Evolutionary Architect — Chương 16: Kiến trúc sư tiến hóa

As we have seen so far, **microservices** give us a lot of choices, and accordingly a lot of decisions to make. For example, how many different technologies should we use, should we let different teams use different programming idioms, and should we split or merge a **microservice**? How do we go about making these decisions? With the faster pace of change and the more fluid environment that these architectures allow, the role of the architect also has to change. In this chapter, I'll take a fairly opinionated view of what the role of an architect is and hopefully launch one final assault on the ivory tower.

Như chúng ta đã thấy từ trước đến giờ, microservices mang lại cho ta rất nhiều lựa chọn, và do đó cũng có rất nhiều quyết định phải đưa ra. Chẳng hạn, chúng ta nên dùng bao nhiêu công nghệ khác nhau, có nên để các nhóm khác nhau dùng những lối hành văn lập trình (programming idiom) khác nhau hay không, và có nên tách hay gộp một microservice hay không? Chúng ta làm thế nào để đưa ra những quyết định ấy? Với nhịp độ thay đổi nhanh hơn và môi trường linh hoạt hơn mà các kiến trúc này cho phép, vai trò của kiến trúc sư cũng phải thay đổi theo. Trong chương này, tôi sẽ đưa ra một góc nhìn khá thiên kiến về vai trò của kiến trúc sư là gì, và hy vọng tung ra một đợt công kích cuối cùng nhằm vào tháp ngà.

What's in a Name? — Tên gọi có ý nghĩa gì?

You keep using that word. I do not think it means what you think it means. —Inigo Montoya, from *The Princess Bride*

Anh cứ dùng mãi cái từ đó. Tôi không nghĩ nó có nghĩa như anh tưởng đâu. —Inigo Montoya, trong phim *The Princess Bride*

Architects have an important job. They are in charge of making sure the system has a joined-up technical vision, one that should help deliver the software that customers need. In some places, they may have to work with only one team, in which case the role of the architect and that of the technical lead are often one and the same. In other places, they may be defining the vision for an entire program of work, coordinating with multiple teams across the world, or perhaps even an entire organization. At whatever level architects operate, their role is a tricky one to pin down, and despite it often being the obvious career progression for developers in enterprise organizations, it is also a role that gets more criticism than virtually any other in our field. More than any other role,

architects can have a direct impact on the quality of the systems built, on the working conditions of their colleagues, and on their organization's ability to respond to change, and yet their role seems very poorly understood. Why is that?

Kiến trúc sư có một công việc quan trọng. Họ phụ trách việc đảm bảo hệ thống có một tầm nhìn kỹ thuật mạch lạc, một tầm nhìn giúp tạo ra phần mềm mà khách hàng cần. Ở một số nơi, họ có thể chỉ phải làm việc với một nhóm duy nhất, và trong trường hợp đó vai trò của kiến trúc sư và của trưởng nhóm kỹ thuật (technical lead) thường là một. Ở những nơi khác, họ có thể đang định hình tầm nhìn cho cả một chương trình công việc lớn, phối hợp với nhiều nhóm trên khắp thế giới, hoặc thậm chí cho cả một tổ chức. Dù kiến trúc sư hoạt động ở cấp độ nào, vai trò của họ cũng rất khó định nghĩa cho rõ ràng, và mặc dù nó thường là con đường thẳng tiến hiển nhiên cho các lập trình viên trong các tổ chức doanh nghiệp lớn, đây cũng là một vai trò nhận nhiều chỉ trích hơn gần như bất kỳ vai trò nào khác trong lĩnh vực của chúng ta. Hơn bất kỳ vai trò nào khác, kiến trúc sư có thể tác động trực tiếp đến chất lượng của các hệ thống được xây dựng, đến điều kiện làm việc của đồng nghiệp họ, và đến khả năng phản ứng với thay đổi của tổ chức họ, vậy mà vai trò của họ lại có vẻ rất ít được hiểu đúng. Vì sao lại như vậy?

Our industry is a young one. We seem to forget sometimes that we have been creating programs that run on what we recognize as computers for only 75 years or so. Our profession doesn't fit into a nice neat box that society as a whole understands. We aren't like electricians, plumbers, medical doctors, or engineers. How many times have you told someone what you do at a party, only for the conversation to stop? The world as a whole struggles to understand software development—as I've outlined multiple times throughout this book, we frequently don't seem to understand it ourselves.

Ngành của chúng ta còn non trẻ. Đôi khi chúng ta dường như quên rằng mới chỉ khoảng 75 năm nay chúng ta mới tạo ra những chương trình chạy trên thứ mà ta nhận ra là máy tính. Nghề của chúng ta không vừa vặn vào một cái hộp gọn gàng, ngăn nắp mà toàn xã hội hiểu được. Chúng ta không giống thợ điện, thợ ống nước, bác sĩ hay kỹ sư. Đã bao nhiêu lần bạn nói với ai đó tại một bữa tiệc rằng mình làm nghề gì, để rồi cuộc trò chuyện rơi vào im lặng? Thế giới nói chung khó lòng hiểu được việc phát triển phần mềm—như tôi đã nhiều lần chỉ ra xuyên suốt cuốn sách này, chính chúng ta cũng thường có vẻ không hiểu được nó.

So we borrow from other professions. We call ourselves software “engineers,” or “architects.” But we aren't architects or engineers in the way that society understands those professions. Architects and engineers have a rigor and discipline we could only dream of, and their importance in society is well understood. I remember talking to a friend of mine the day before he became a qualified architect. “Tomorrow,” he said, “if I give you advice down at the pub about how to **build** something and it's wrong, I get held to account. I could get sued, as in the eyes of the law I am now a qualified architect and I should be held responsible if I get it wrong.” The importance of these jobs to society means that there are required qualifications people have to meet. In the UK, for example, a minimum of seven years' study is required before you can be called an architect. But these jobs are also based on a body of knowledge going back thousands of years. And software architects? Not quite. Which is partly why I view many forms of IT certification as worthless, as we know so little about what “good” looks like.

Thế nên chúng ta mượn từ các nghề khác. Chúng ta tự gọi mình là “kỹ sư” phần mềm, hay “kiến trúc sư”. Nhưng chúng ta không phải là kiến trúc sư hay kỹ sư theo cách mà xã hội hiểu về những nghề ấy. Kiến trúc sư và kỹ sư có một sự nghiêm cẩn và kỷ luật mà chúng ta chỉ có thể mơ tới, và tầm quan trọng của họ trong xã hội được hiểu rõ. Tôi nhớ có lần nói chuyện với một người bạn vào ngày trước khi anh ấy trở thành kiến trúc sư có chứng chỉ hành nghề. Anh ấy nói: “Ngày mai, nếu tôi cho cậu lời khuyên ở quán rượu về cách

xây một thứ gì đó mà nó sai, tôi sẽ phải chịu trách nhiệm. Tôi có thể bị kiện, vì trong mắt pháp luật giờ tôi là một kiến trúc sư có chứng chỉ và tôi phải chịu trách nhiệm nếu làm sai.” Tầm quan trọng của những nghề này đối với xã hội nghĩa là có những bằng cấp bắt buộc mà người ta phải đáp ứng. Ở Anh chẳng hạn, cần tối thiểu bảy năm học trước khi bạn có thể được gọi là kiến trúc sư. Nhưng những nghề này còn dựa trên một kho tri thức có từ hàng ngàn năm trước. Còn kiến trúc sư phần mềm thì sao? Chưa hẳn như vậy. Đó cũng là một phần lý do tôi xem nhiều dạng chứng chỉ IT là vô giá trị, bởi chúng ta biết quá ít về việc “tốt” trông như thế nào.

I don't say this to belittle the term software engineering , 1 coined back in the 1960s by Margaret Hamilton, but it was as much aspirational as it was about the current reality. The term emerged as a call to improve the quality of the software being created, and in recognition of the fact that software projects often failed and yet were increasingly being used in vital mission- and safety-critical fields. Much work has been done to improve the situation since then, but my own take after 20 years in the industry is that we've still got a lot to learn about doing a good (or at least a better) job.

Tôi nói điều này không phải để hạ thấp thuật ngữ kỹ thuật phần mềm (software engineering), 1 vốn được Margaret Hamilton đặt ra hồi thập niên 1960, nhưng nó mang tính khát vọng nhiều ngang với thực tế lúc bấy giờ. Thuật ngữ này xuất hiện như một lời kêu gọi nâng cao chất lượng phần mềm đang được tạo ra, và như một sự thừa nhận rằng các dự án phần mềm thường thất bại trong khi lại ngày càng được dùng trong những lĩnh vực trọng yếu về sứ mệnh (mission-critical) và an toàn (safety-critical). Đã có nhiều công sức bỏ ra để cải thiện tình hình kể từ đó, nhưng quan điểm riêng của tôi sau 20 năm trong ngành là chúng ta vẫn còn rất nhiều điều phải học về việc làm tốt (hay ít ra là tốt hơn) công việc của mình.

Part of us wants recognition, so we borrow names from other professions that already have the recognition we crave. But this can prove problematic, if we borrow working practices from those professions without understanding the mindset behind them or taking into account how software development is different from, say, civil engineering. None of this should be taken as an argument that we shouldn't aim to have more rigor in our work—just that we cannot simply borrow ideas from elsewhere and assume they will work for us. Our industry is very young, and the challenge is that we have far fewer absolutes around which we agree as an industry.

Một phần trong chúng ta muốn được công nhận, nên chúng ta mượn tên gọi từ các nghề khác vốn đã có được sự công nhận mà ta khao khát. Nhưng điều này có thể gây rắc rối, nếu ta mượn các tập quán làm việc từ những nghề ấy mà không hiểu lối tư duy đằng sau chúng hoặc không tính đến chuyện phát triển phần mềm khác với, chẳng hạn, kỹ thuật xây dựng (civil engineering) ra sao. Không nên hiểu bất cứ điều gì trong số này như một lập luận rằng chúng ta không nên hướng tới sự nghiêm cẩn hơn trong công việc—chỉ là chúng ta không thể đơn giản mượn ý tưởng từ nơi khác rồi mặc nhiên cho rằng chúng sẽ phù hợp với mình. Ngành của chúng ta rất non trẻ, và thử thách là chúng ta có ít hơn nhiều những điều tuyệt đối mà cả ngành cùng đồng thuận.

1 For several reasons, not the least of which is that I have a degree in software engineering... Perhaps the term architect , or at least the common understanding of what architects do, has done the most harm in this way: the idea of someone who draws up detailed plans for others to interpret and expects them to be carried out; the balance of part artist, part engineer, overseeing the creation of what is often a singular vision, with all other viewpoints being subservient, except for the occasional objection from the structural engineer regarding the laws of physics. In our industry, this view of the architect leads to some terrible practices, with architects creating diagram after diagram, page after page of documentation, with a view to inform the construction of the perfect system, while failing to take into account the fundamentally unknowable future, and utterly devoid of any understanding

of how hard their plans will be to implement, or whether or not they will actually work, let alone having any ability to change as we learn more.

1 Vì vài lý do, mà không phải nhỏ nhất là vì tôi có bằng về kỹ thuật phần mềm... Có lẽ thuật ngữ kiến trúc sư, hay ít nhất là cách hiểu thông thường về việc kiến trúc sư làm gì, đã gây tổn hại nhiều nhất theo kiểu này: hình ảnh một người vẽ ra các bản thiết kế chi tiết để người khác diễn giải và mong họ thực hiện theo; sự pha trộn giữa một phần nghệ sĩ, một phần kỹ sư, giám sát việc tạo ra cái thường là một tầm nhìn đơn nhất, với mọi quan điểm khác đều phải phục tùng, ngoại trừ thỉnh thoảng có phản đối từ kỹ sư kết cấu về các định luật vật lý. Trong ngành của chúng ta, cách nhìn này về kiến trúc sư dẫn đến một số tập quán tồi tệ, với việc các kiến trúc sư tạo ra hết sơ đồ này đến sơ đồ khác, hết trang tài liệu này đến trang tài liệu khác, với mục đích định hướng việc xây dựng một hệ thống hoàn hảo, trong khi không tính đến tương lai về cơ bản là bất khả tri, và hoàn toàn thiếu vắng bất kỳ hiểu biết nào về việc các kế hoạch của họ khó triển khai đến đâu, hay liệu chúng có thực sự hoạt động hay không, chứ chưa nói đến việc có khả năng thay đổi khi chúng ta học hỏi thêm.

But architects of the built environment are operating in a different realm from that of software architects. Their constraints are different, the end product different. The cost of change is so much higher in construction than it is in software development. You can't unpour concrete, but you can change code, and even the infrastructure we run our code on is much more malleable than before, thanks to virtualization. Buildings are fairly fixed once built—they can be changed, or expanded, or torn down, but the associated costs are very high. But we expect our software to continually change to suit our needs.

Nhưng kiến trúc sư của môi trường xây dựng hoạt động trong một cõi khác hẳn với kiến trúc sư phần mềm. Các ràng buộc của họ khác nhau, sản phẩm cuối cũng khác. Chi phí thay đổi trong xây dựng cao hơn nhiều so với trong phát triển phần mềm. Bạn không thể “rút” bê tông đã đổ ra, nhưng bạn có thể thay đổi mã, và ngay cả hạ tầng mà ta chạy mã trên đó cũng dễ uốn nắn hơn nhiều so với trước đây, nhờ ảo hóa (virtualization). Các tòa nhà khá cố định một khi đã xây xong—chúng có thể được thay đổi, mở rộng hay phá dỡ, nhưng chi phí kèm theo rất cao. Trong khi đó chúng ta lại kỳ vọng phần mềm của mình liên tục thay đổi để phù hợp với nhu cầu.

So if software **architecture** is different from the architecture of the built environment, perhaps we should be a bit clearer in terms of what software architecture actually is.

Vậy nếu kiến trúc phần mềm khác với kiến trúc của môi trường xây dựng, thì có lẽ chúng ta nên rõ ràng hơn một chút về việc kiến trúc phần mềm thực ra là gì.

What Is Software Architecture? — Kiến trúc phần mềm là gì?

One of the most famous definitions of software architecture comes via an email from Ralph Johnson: “Architecture is about the important stuff. Whatever that is.”² So does this mean that anything important is done by the architect? Does that mean that all other work being done is not important? The issue with this oft-quoted statement is that it's often used in isolation, without any understanding of the wider response in which Ralph shared it. Firstly, it's clear that he is talking from the perspective of a software developer. He goes on to say:

Một trong những định nghĩa nổi tiếng nhất về kiến trúc phần mềm đến từ một email của Ralph Johnson: “Kiến trúc là về những thứ quan trọng. Dù những thứ đó là gì đi nữa.”² Vậy điều này có nghĩa là mọi thứ quan trọng đều do kiến trúc sư làm? Nó có nghĩa là tất

cả những công việc khác đang được thực hiện đều không quan trọng? Vấn đề với câu nói thường được trích dẫn này là nó thường bị dùng một cách tách rời, không kèm theo bất kỳ hiểu biết nào về phản hồi rộng hơn mà Ralph đã chia sẻ nó trong đó. Trước hết, rõ ràng là ông đang nói từ góc nhìn của một lập trình viên phần mềm. Ông nói tiếp:

2 This is from an email exchange on the extreme programming mailing list, which Martin Fowler then shared in his article “Who Needs an Architect?” .

2 Đây là trích từ một trao đổi email trên mailing list về extreme programming, mà Martin Fowler sau đó đã chia sẻ trong bài viết “Who Needs an Architect?”.

So, a better definition would be “In most successful software projects, the expert developers working on that project have a shared understanding of the system design. This shared understanding is called ‘architecture.’ This understanding includes how the system is divided into components and how the components interact through interfaces. These components are usually composed of smaller components, but the architecture only includes the components and interfaces that are understood by all the developers.” This would be a better definition because it makes clear that architecture is a social construct (well, software is too, but architecture is even more so) because it doesn’t just depend on the software, but on what part of the software is considered important by group consensus. Here Ralph is using the term components in its most general sense. In the context of this book, we can think of the components as our microservices, and perhaps the modules inside those microservices.

Vậy nên một định nghĩa tốt hơn sẽ là “Trong hầu hết các dự án phần mềm thành công, các lập trình viên kỹ cựu làm việc trên dự án đó có một hiểu biết chung về thiết kế hệ thống. Hiểu biết chung này được gọi là ‘kiến trúc’. Hiểu biết này bao gồm cách hệ thống được chia thành các thành phần (component) và cách các thành phần tương tác với nhau qua các interface. Những thành phần này thường được cấu thành từ các thành phần nhỏ hơn, nhưng kiến trúc chỉ bao gồm những thành phần và interface được tất cả lập trình viên hiểu.” Đây sẽ là một định nghĩa tốt hơn bởi nó làm rõ rằng kiến trúc là một kiến tạo xã hội (social construct)—(thực ra phần mềm cũng vậy, nhưng kiến trúc còn hơn thế)—bởi nó không chỉ phụ thuộc vào phần mềm, mà còn vào phần nào của phần mềm được nhóm đồng thuận xem là quan trọng. Ở đây Ralph dùng thuật ngữ thành phần theo nghĩa chung nhất. Trong bối cảnh cuốn sách này, ta có thể nghĩ về các thành phần như những microservice của mình, và có lẽ là các module bên trong những microservice ấy.

Software architecture is the shape of the system. Architecture happens, by design or accident. We make a series of ad hoc decisions, and we end up with the results— without thinking about things in terms of architecture, we end up with architecture nonetheless. Architecture can sometimes be what happens while we’re busy making other plans.

Kiến trúc phần mềm là hình dạng của hệ thống. Kiến trúc xảy ra, dù có chủ đích hay tình cờ. Chúng ta đưa ra một loạt quyết định ngẫu hứng, rồi rồi cuộc nhận về các kết quả—mà không hề nghĩ đến mọi thứ theo nghĩa kiến trúc, ta vẫn cứ có kiến trúc. Đôi khi kiến trúc là cái xảy ra trong lúc chúng ta đang bận rộn lên những kế hoạch khác.

A dedicated architect is someone who should see and understand that whole system, understand the forces acting on it. They need to ensure there is a vision for the architecture that is fit for purpose and is clearly understood—an architectural vision that satisfies the needs of the system and its users, as well as those of the people who work on the system itself. Looking at only one facet—e.g., logical but not physical, shape but not developer experience—limits an architect’s effectiveness. If you accept that architecture is about understanding the system, then limiting the scope of what you care about limits your ability to reason and make changes.

Một kiến trúc sư chuyên trách là người nên nhìn thấy và hiểu được toàn bộ hệ thống ấy,

hiểu được các lực đang tác động lên nó. Họ cần đảm bảo có một tầm nhìn cho kiến trúc vừa phù hợp với mục đích vừa được hiểu rõ ràng—một tầm nhìn kiến trúc thỏa mãn nhu cầu của hệ thống và người dùng nó, cũng như nhu cầu của những người làm việc trên chính hệ thống đó. Chỉ nhìn vào một khía cạnh duy nhất—ví dụ, lý luận mà không vật lý, hình dạng mà không phải trải nghiệm của lập trình viên—sẽ giới hạn hiệu quả của kiến trúc sư. Nếu bạn chấp nhận rằng kiến trúc là về việc hiểu hệ thống, thì giới hạn phạm vi những gì bạn quan tâm sẽ giới hạn khả năng lập luận và tạo ra thay đổi của bạn.

Architecture can be invisible to the people living with it. It can be so slight as to not really be there. It can be something that guides and helps achieve the right outcome. It can be suffocating and overbearing. It can delight without you realizing it is even a thing, and crush the spirit from you without any malice being intended. So whether or not architecture is “about the important stuff,” it’s certainly important .

Kiến trúc có thể vô hình đối với những người đang sống cùng nó. Nó có thể mờ nhạt đến mức như thể không thực sự hiện diện. Nó có thể là thứ dẫn dắt và giúp đạt được kết quả đúng đắn. Nó có thể ngọt ngào và áp đặt. Nó có thể làm bạn vui thích mà bạn không nhận ra nó là một thứ gì đó, và bóp nghẹt tinh thần bạn mà chẳng có ác ý nào được nhắm tới. Vậy nên dù kiến trúc có “về những thứ quan trọng” hay không, thì chắc chắn nó cũng quan trọng.

Another pithy quote that is often used to define software architecture comes from the same article where Martin shares Ralph’s views: “So you might end up defining architecture as things that people perceive as hard to change.” Martin’s idea that architecture is the stuff that’s hard to change makes sense at some level and brings us back to the concept of architecture in the built environment. Where things are harder to change, they need a bit more up-front thought to really make sure we are going in the right direction. But there is a problem with taking a simple definition of a complex idea and running with that as a working definition—if this statement were entirely how you thought about software architecture, you’d miss out on a lot. Yes, a lot of software architecture is about thinking about the things that will be hard to change, but it is also about creating space to allow change in the design.

Một câu trích dẫn súc tích khác thường được dùng để định nghĩa kiến trúc phần mềm đến từ chính bài viết mà Martin chia sẻ quan điểm của Ralph: “Vậy nên bạn có thể đi đến chỗ định nghĩa kiến trúc là những thứ mà người ta cảm nhận là khó thay đổi.” Ý tưởng của Martin rằng kiến trúc là những thứ khó thay đổi có lý ở một mức độ nào đó và đưa chúng ta trở lại với khái niệm kiến trúc trong môi trường xây dựng. Ở những chỗ mà mọi thứ khó thay đổi hơn, chúng cần được suy nghĩ trước nhiều hơn một chút để thực sự chắc chắn rằng chúng ta đang đi đúng hướng. Nhưng có một vấn đề với việc lấy một định nghĩa đơn giản cho một ý tưởng phức tạp rồi chạy theo nó như một định nghĩa thực hành—nếu câu nói này hoàn toàn là cách bạn nghĩ về kiến trúc phần mềm, bạn sẽ bỏ lỡ rất nhiều thứ. Đúng, phần lớn kiến trúc phần mềm là về việc nghĩ tới những thứ sẽ khó thay đổi, nhưng nó cũng là về việc tạo ra không gian để cho phép thay đổi trong thiết kế.

Making Change Possible — Làm cho thay đổi trở nên khả thi

Coming back to the world of buildings rather than software systems: architect Mies van der Rohe arguably did more to pioneer what we now think of as the modern skyscraper than any other architect—his famous Seagram Building became the blueprint for much of what followed. The Seagram Building differs from a lot of what came before. The outer walls of the building are nonstructural—they wrap a steel outer frame. The main building services—lifts, 3 stairways, air

conditioning, water and waste, and the electrical system—run through a central concrete core. Watch a modern high-rise being constructed today, and it's this central concrete core that is built first, a giant crane often seen perching on top. Each floor of the Seagram Building has no interior structural walls—this means that you have total flexibility in terms of how the space is used. You can reconfigure the space as you see fit, routing electrical wiring and air conditioning to different parts of each floor via suspended ceilings and ducts in the floor itself.

Trở lại với thế giới các tòa nhà thay vì hệ thống phần mềm: kiến trúc sư Mies van der Rohe được cho là người đã có công tiên phong cho cái mà ngày nay ta gọi là tòa nhà chọc trời hiện đại hơn bất kỳ kiến trúc sư nào khác—tòa nhà Seagram nổi tiếng của ông đã trở thành khuôn mẫu cho phần lớn những gì theo sau. Tòa nhà Seagram khác với rất nhiều thứ trước đó. Các bức tường ngoài của tòa nhà không chịu lực—chúng bao bọc một khung thép bên ngoài. Các hệ thống dịch vụ chính của tòa nhà—thang máy, 3 cầu thang bộ, điều hòa không khí, cấp thoát nước, và hệ thống điện—chạy xuyên qua một lõi bê tông trung tâm. Hãy xem một tòa cao ốc hiện đại được xây dựng ngày nay, và chính cái lõi bê tông trung tâm này là thứ được xây trước, với một cần cầu khổng lồ thường được thấy đậu trên đỉnh. Mỗi tầng của tòa nhà Seagram không có tường kết cấu bên trong—điều này nghĩa là bạn có sự linh hoạt hoàn toàn về cách sử dụng không gian. Bạn có thể tái cấu hình không gian theo ý mình thấy phù hợp, dẫn dây điện và điều hòa không khí đến các phần khác nhau của mỗi tầng qua trần giả và các hộp kỹ thuật trong chính sàn nhà.

It's interesting to note that the Seagram Building was developed using a process in which the design of the building evolved while the construction was carried out. Now where have we seen that idea before?

Điều thú vị đáng lưu ý là tòa nhà Seagram được phát triển bằng một quy trình trong đó thiết kế của tòa nhà tiến hóa trong lúc việc xây dựng đang diễn ra. Vậy chúng ta đã thấy ý tưởng đó ở đâu rồi nhỉ?

The idea with this design was to deliver what Mies van der Rohe called “universal space”—a large, single-span volume that could be reconfigured to suit different needs. The use of buildings changes, so the idea was to deliver space that is as flexible as possible in terms of how it can be used. In this way, Mies van der Rohe not only had to focus on the fundamental aesthetics of the building, finding a space for core services that would be difficult if not impossible to change later, but he also had to ensure that the building could be used in different ways than originally envisaged. Shortly, we'll look at how we allow for change in the space of a microservice architecture.

Ý tưởng của thiết kế này là tạo ra cái mà Mies van der Rohe gọi là “không gian phổ quát” (universal space)—một khối tích lớn, nhịp đơn, có thể được tái cấu hình để phù hợp với các nhu cầu khác nhau. Cách sử dụng các tòa nhà thay đổi, nên ý tưởng là tạo ra không gian linh hoạt nhất có thể về cách nó được sử dụng. Theo cách này, Mies van der Rohe không chỉ phải tập trung vào tính thẩm mỹ cơ bản của tòa nhà, tìm một chỗ cho các dịch vụ lõi vốn sẽ khó nếu không muốn nói là bất khả thi để thay đổi về sau, mà ông còn phải đảm bảo rằng tòa nhà có thể được dùng theo những cách khác với hình dung ban đầu. Lát nữa, chúng ta sẽ xem cách chúng ta tạo điều kiện cho thay đổi trong không gian của một kiến trúc microservice.

An Evolutionary Vision for the Architect — Một tầm nhìn tiến hóa cho kiến trúc sư

Our requirements as architects of software shift more rapidly than they do for people who design and build buildings—as do the tools and techniques at our disposal. The things we create are not fixed points in time. Once launched into production, our software will continue to evolve as the way it is used changes. For most things we

Các yêu cầu đặt ra cho chúng ta với tư cách là kiến trúc sư phần mềm thay đổi nhanh hơn so với những người thiết kế và xây dựng các tòa nhà—cũng như các công cụ và kỹ thuật mà chúng ta có trong tay. Những thứ chúng ta tạo ra không phải là các điểm cố định trong thời gian. Một khi được đưa vào sản xuất (production), phần mềm của chúng ta sẽ tiếp tục tiến hóa khi cách nó được sử dụng thay đổi. Đối với hầu hết những thứ chúng ta

3 Elevators, for my North American readers.

3 Tức là elevators, dành cho độc giả Bắc Mỹ của tôi.

create, we have to accept that once the software gets into the hands of our users, we will have to react and adapt rather than expect a never-changing **artifact**. Thus, software architects need to shift their thinking away from creating the perfect end product and focus instead on helping create a framework in which the right systems can emerge and continue to grow as we learn more.

tạo ra, chúng ta phải chấp nhận rằng một khi phần mềm đến tay người dùng, chúng ta sẽ phải phản ứng và thích nghi thay vì kỳ vọng vào một sản phẩm không bao giờ thay đổi. Do đó, kiến trúc sư phần mềm cần chuyển dịch tư duy của mình khỏi việc tạo ra sản phẩm cuối hoàn hảo và thay vào đó tập trung vào việc giúp tạo ra một khuôn khổ trong đó những hệ thống đúng đắn có thể nảy sinh và tiếp tục phát triển khi chúng ta học hỏi thêm.

Although I have spent much of the chapter discouraging comparisons to other professions, there is an analogy that I like when it comes to the role of the IT architect and that I think encapsulates this aspect of the role well. Erik Doernenburg at Thoughtworks first shared with me the idea that we should think of the architect's role more as town planner than architect of the built environment. The role of the town planner should be familiar to any of you who have played SimCity or Cities: Skylines before. A town planner's role is to look at a multitude of sources of information and then attempt to optimize the layout of a city to best suit the needs of present-day citizens, while also taking into account future use. The way they influence how the city evolves, though, is interesting. They do not say, "Build this specific building there"; instead, they define zones that allow for local decision making within certain constraints. So, as in SimCity, you might designate part of your city as an industrial zone, and another part as a residential zone. It is then up to other people to decide what buildings get created, but there are restrictions: if you want to build a factory, it will need to be in an industrial zone. Rather than worrying too much about what happens in one zone, the town planner will instead spend far more time working out how people and utilities move from one zone to another.

Mặc dù tôi đã dành phần lớn chương này để ngăn cản việc so sánh với các nghề khác, vẫn có một phép loại suy mà tôi thích khi nói về vai trò của kiến trúc sư IT và mà tôi nghĩ gói gọn khía cạnh này của vai trò khá tốt. Erik Doernenburg ở Thoughtworks là người đầu tiên chia sẻ với tôi ý tưởng rằng chúng ta nên nghĩ về vai trò của kiến trúc sư giống như một nhà quy hoạch đô thị (town planner) hơn là kiến trúc sư của môi trường xây dựng. Vai trò của nhà quy hoạch đô thị hẳn quen thuộc với bất kỳ ai trong số các bạn từng chơi SimCity hay Cities: Skylines. Vai trò của một nhà quy hoạch đô thị là xem xét vô số nguồn thông tin rồi cố gắng tối ưu bố cục của một thành phố sao cho phù hợp nhất với nhu cầu

của những công dân hiện tại, đồng thời cũng tính đến cách sử dụng trong tương lai. Tuy vậy, cách họ tác động đến việc thành phố tiến hóa ra sao thì lại thú vị. Họ không nói: “Hãy xây tòa nhà cụ thể này ở chỗ đó”; thay vào đó, họ định nghĩa các vùng (zone) cho phép việc ra quyết định tại chỗ trong một số ràng buộc nhất định. Vậy nên, như trong SimCity, bạn có thể chỉ định một phần thành phố là vùng công nghiệp, và một phần khác là vùng dân cư. Việc tòa nhà nào được tạo ra thì sau đó tùy thuộc vào người khác quyết định, nhưng có các hạn chế: nếu bạn muốn xây một nhà máy, nó sẽ cần nằm trong vùng công nghiệp. Thay vì lo lắng quá nhiều về việc gì xảy ra trong một vùng, nhà quy hoạch đô thị sẽ dành nhiều thời gian hơn để tính toán cách con người và các tiện ích di chuyển từ vùng này sang vùng khác.

More than one person has likened a city to a living creature. The city changes over time. It shifts and evolves as its occupants use it in different ways, or as external forces shape it. The town planner does their best to anticipate these changes but accepts that trying to exert direct control over all aspects of what happens is futile. So our architects as town planners need to set direction in broad strokes and get involved in being highly specific about implementation detail only in limited cases. They need to ensure that the system is fit for purpose now but also a platform for the future.

Hơn một người đã ví thành phố với một sinh thể sống. Thành phố thay đổi theo thời gian. Nó dịch chuyển và tiến hóa khi cư dân của nó sử dụng nó theo những cách khác nhau, hoặc khi các lực bên ngoài định hình nó. Nhà quy hoạch đô thị cố gắng hết sức để dự đoán những thay đổi này nhưng chấp nhận rằng việc cố gắng kiểm soát trực tiếp mọi khía cạnh của những gì xảy ra là vô ích. Vậy nên các kiến trúc sư đóng vai nhà quy hoạch đô thị của chúng ta cần định hướng theo những nét lớn và chỉ tham gia rất cụ thể vào chi tiết triển khai trong một số trường hợp hạn chế. Họ cần đảm bảo rằng hệ thống vừa phù hợp với mục đích hiện tại vừa là nền tảng cho tương lai.

The comparison with software should be obvious. As our users use our software, we need to react and change. We cannot foresee everything that will happen, and so rather than plan for each and every eventuality, we should plan to allow for change by avoiding the urge to overspecify every last thing. Our city—the system—needs to be a good, happy place for everyone who uses it.

Sự so sánh với phần mềm hẳn là hiển nhiên. Khi người dùng sử dụng phần mềm của chúng ta, chúng ta cần phản ứng và thay đổi. Chúng ta không thể tiên liệu mọi thứ sẽ xảy ra, và do đó thay vì lên kế hoạch cho từng tình huống có thể có, chúng ta nên lên kế hoạch để cho phép thay đổi bằng cách tránh thúc đẩy đặc tả quá mức từng thứ nhỏ nhất cuối cùng. Thành phố của chúng ta—hệ thống—cần là một nơi tốt đẹp, hạnh phúc cho tất cả những ai sử dụng nó.

Defining System Boundaries — Định nghĩa ranh giới hệ thống

To continue with the metaphor of the architect as town planner for a moment, what are our zones? These are our microservice boundaries, or perhaps coarse-grained groups of microservices. As architects, we need to worry much less about what happens inside a zone and more about what happens between the zones. That means we need to spend time thinking about how our microservices talk to each other and ensuring that we can properly monitor the overall health of our system. From an architecture space, this is how we create our own universal space—by defining some specific boundaries, we highlight to our colleagues building the system those areas where changes can be made more freely without breaking some fundamental aspect of our architecture.

Vẫn dùng phép ẩn dụ kiến trúc sư như nhà quy hoạch đô thị thêm một lúc nữa, vậy các vùng của chúng ta là gì? Đó là các ranh giới microservice của chúng ta, hoặc có lẽ là các

nhóm microservice ở mức thô (coarse-grained). Với tư cách kiến trúc sư, chúng ta cần lo lắng ít hơn nhiều về việc gì xảy ra bên trong một vùng và nhiều hơn về việc gì xảy ra giữa các vùng. Điều đó nghĩa là chúng ta cần dành thời gian suy nghĩ về cách các microservice nói chuyện với nhau và đảm bảo rằng chúng ta có thể giám sát đúng cách sức khỏe tổng thể của hệ thống. Từ góc độ không gian kiến trúc, đây là cách chúng ta tạo ra không gian phổ quát của riêng mình—bằng cách định nghĩa một số ranh giới cụ thể, chúng ta làm nổi bật cho các đồng nghiệp đang xây dựng hệ thống những khu vực nơi có thể tạo ra thay đổi một cách tự do hơn mà không phá vỡ một khía cạnh cơ bản nào đó của kiến trúc.

To look at a very simple example, in Figure 16-1 we see the Recommendations microservice accessing information from the Promotions and Sales microservices. As we've already covered at length, we are free to change the functionality hidden inside these three microservices without worrying about breaking the overall system—I can change whatever I want in Sales or Promotions as long as I continue to maintain the expectations that Recommendations has about how it will interact with these downstream microservices.

Để xem một ví dụ rất đơn giản, trong Hình 16-1 chúng ta thấy microservice Recommendations đang truy cập thông tin từ các microservice Promotions và Sales. Như chúng ta đã đề cập kỹ lưỡng, chúng ta được tự do thay đổi chức năng ẩn bên trong ba microservice này mà không phải lo lắng về việc phá vỡ hệ thống tổng thể—tôi có thể thay đổi bất cứ thứ gì mình muốn trong Sales hoặc Promotions miễn là tôi tiếp tục duy trì những kỳ vọng mà Recommendations có về cách nó sẽ tương tác với các microservice phía hạ nguồn (downstream) này.

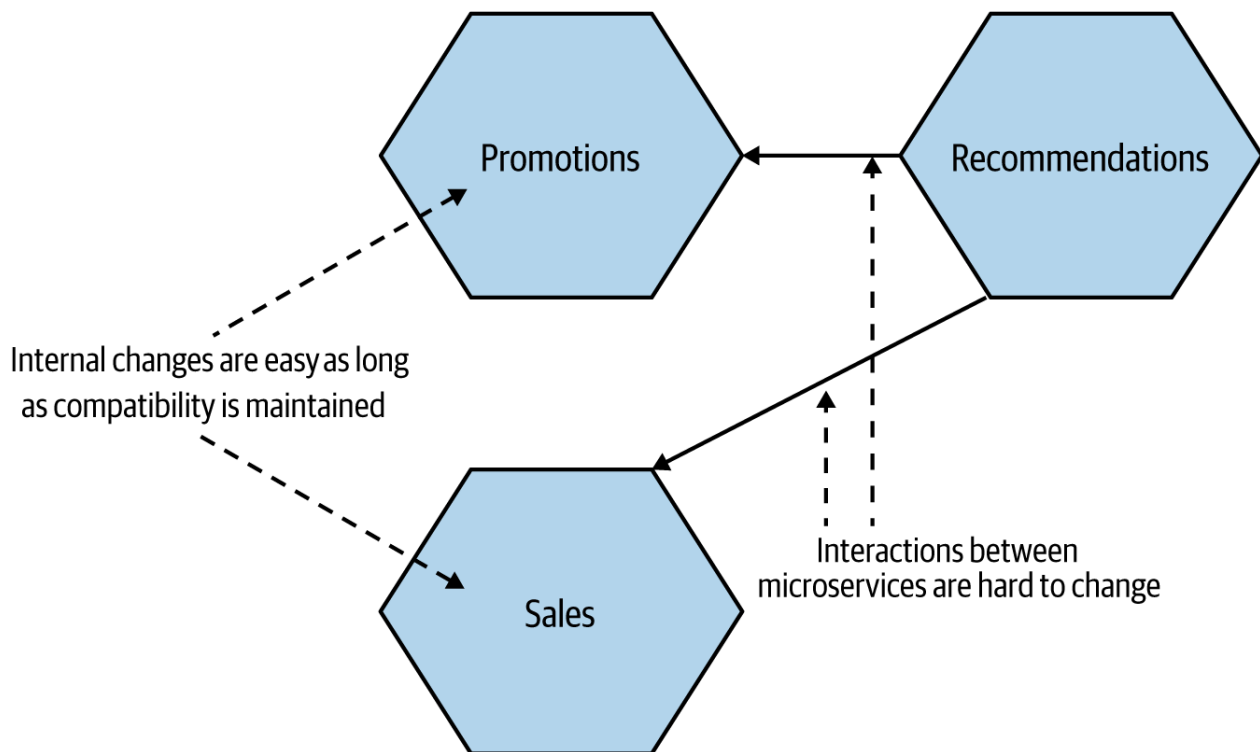


Figure 16-1. Changes inside a microservice boundary are easy to make, as long as the interactions between microservices don't change

Hình 16-1. Các thay đổi bên trong ranh giới một microservice rất dễ thực hiện, miễn là các tương tác giữa các microservice không thay đổi

We can create space for change at larger-scope levels as well. In Figure 16-2, we see the microservices from Figure 16-1 actually exist in a marketing zone that maps to a specific team's responsibility. We've defined an expected behavior in terms of how the marketing functionality interacts with the larger system. Inside the marketing zone, we can make any changes we like, as long as compatibility with the larger system is maintained. Coming back to the idea of understanding what things are hard to change, organizational structures often fall into this category, and as such existing team structures can help define these zones for you. Coordinating changes within a team across microservices owned by that team will be easier than changing the interactions that are exposed to other teams.

Chúng ta cũng có thể tạo ra không gian cho thay đổi ở các cấp độ phạm vi lớn hơn. Trong Hình 16-2, chúng ta thấy các microservice từ Hình 16-1 thực ra tồn tại trong một vùng marketing tương ứng với trách nhiệm của một nhóm cụ thể. Chúng ta đã định nghĩa một hành vi kỳ vọng về cách chức năng marketing tương tác với hệ thống lớn hơn. Bên trong vùng marketing, chúng ta có thể tạo ra bất kỳ thay đổi nào mình thích, miễn là duy trì được khả năng tương thích với hệ thống lớn hơn. Trở lại với ý tưởng về việc hiểu những thứ nào khó thay đổi, các cấu trúc tổ chức thường rơi vào loại này, và do đó các cấu trúc nhóm hiện có có thể giúp định nghĩa các vùng này cho bạn. Việc điều phối các thay đổi trong nội bộ một nhóm trên các microservice do nhóm đó sở hữu sẽ dễ hơn việc thay đổi các tương tác được phơi bày ra cho các nhóm khác.

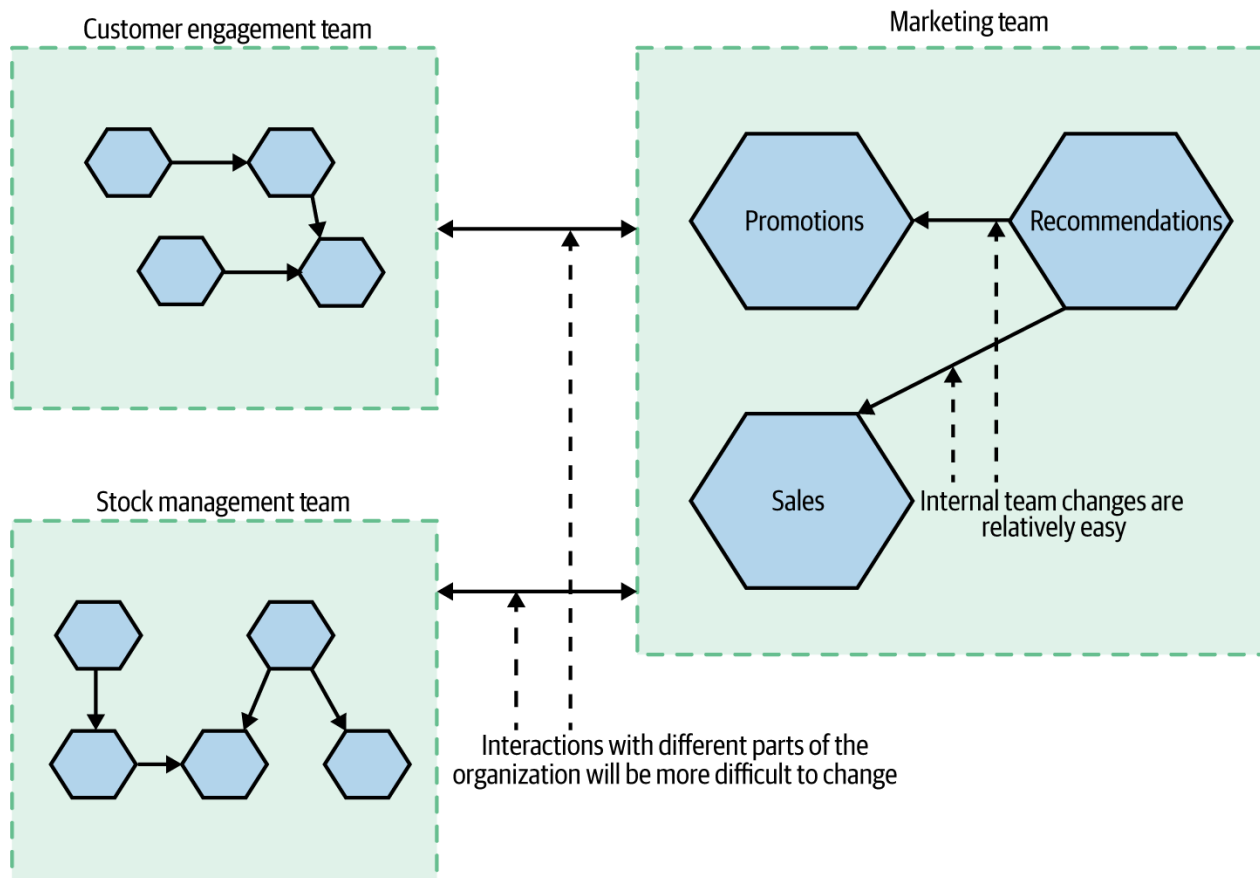


Figure 16-2. Changes within a zone are easier to make than changes between zones

Hình 16-2. Các thay đổi trong nội bộ một vùng dễ thực hiện hơn các thay đổi giữa các vùng

This ties in nicely with the concept of a team **API**, which we discussed in “Small Teams, Large Organization” on page 496. An architect can help facilitate creation of a team API, making sure the

team's microservices and working practices fit in with the wider organization.

Điều này khớp rất đẹp với khái niệm team API, mà chúng ta đã thảo luận trong “Small Teams, Large Organization” ở trang 496. Một kiến trúc sư có thể giúp tạo điều kiện cho việc tạo ra một team API, đảm bảo rằng các microservice và các tập quán làm việc của nhóm khớp với tổ chức rộng hơn.

By defining spaces in which these changes can be made without compromising the system as a whole, we make developers' lives easier and also focus our attention on parts of the system that are harder to change. Remember the concept of **information hiding** that we explored in Chapter 2 ? As we explored there, hiding information inside a microservice boundary makes it much easier to create a stable **interface** for consumers. When we make changes to the microservice, it is easier to ensure we haven't broken compatibility with external consumers. Here, we can define an architecture to provide information hiding at the team level, rather than just at the microservice level. This gives us another level of information hiding and creates a larger safe space in which a team can make local changes without breaking the wider system.

Bằng cách định nghĩa những không gian trong đó có thể tạo ra các thay đổi này mà không làm tổn hại hệ thống như một tổng thể, chúng ta khiến cuộc sống của các lập trình viên dễ dàng hơn và cũng tập trung sự chú ý của mình vào những phần của hệ thống khó thay đổi hơn. Còn nhớ khái niệm che giấu thông tin (information hiding) mà chúng ta đã khám phá ở Chương 2 không? Như chúng ta đã khám phá ở đó, việc che giấu thông tin bên trong một ranh giới microservice khiến việc tạo ra một interface ổn định cho các bên tiêu thụ (consumer) dễ hơn nhiều. Khi chúng ta thực hiện các thay đổi đối với microservice, việc đảm bảo rằng chúng ta không phá vỡ khả năng tương thích với các bên tiêu thụ bên ngoài sẽ dễ hơn. Ở đây, chúng ta có thể định nghĩa một kiến trúc để cung cấp việc che giấu thông tin ở cấp độ nhóm, thay vì chỉ ở cấp độ microservice. Điều này mang lại cho chúng ta thêm một tầng che giấu thông tin và tạo ra một không gian an toàn lớn hơn trong đó một nhóm có thể tạo ra các thay đổi cục bộ mà không phá vỡ hệ thống rộng hơn.

Within each microservice or larger zone, you may be OK with the team that owns that zone picking a different technology stack or data store. Other concerns may kick in here, of course. Your inclination to let teams pick the right tool for the job may be tempered by the fact that it becomes harder to hire people or move them between teams if you have 10 different technology stacks to support. Similarly, if each team picks a completely different data store, you may find yourself lacking enough experience to run any of them at scale. Netflix, for example, has mostly standardized on Cassandra as a data-store technology. Although it may not be the best fit for all of its cases, Netflix feels that the value gained by building tooling and expertise around Cassandra is more important than having to support and operate at scale multiple other platforms that may be a better fit for certain tasks. Netflix is an extreme example, where scale is likely the strongest overriding factor, but you get the idea.

Trong mỗi microservice hay vùng lớn hơn, bạn có thể chấp nhận để nhóm sở hữu vùng đó chọn một stack công nghệ hay kho dữ liệu (data store) khác. Tất nhiên ở đây có thể nảy sinh những mối bận tâm khác. Khuynh hướng để các nhóm tự chọn công cụ phù hợp cho công việc của bạn có thể bị kìm hãm bởi thực tế là việc tuyển người hay luân chuyển họ giữa các nhóm trở nên khó hơn nếu bạn có 10 stack công nghệ khác nhau phải hỗ trợ. Tương tự, nếu mỗi nhóm chọn một kho dữ liệu hoàn toàn khác nhau, bạn có thể thấy mình thiếu đủ kinh nghiệm để vận hành bất kỳ kho nào trong số chúng ở quy mô lớn. Netflix chẳng hạn đã phần lớn chuẩn hóa trên Cassandra làm công nghệ kho dữ liệu. Mặc dù nó có thể không phải là lựa chọn phù hợp nhất cho tất cả các trường hợp của mình, Netflix cảm thấy rằng giá trị thu được từ việc xây dựng công cụ và chuyên môn quanh Cassandra quan trọng hơn việc phải hỗ trợ và vận hành ở quy mô lớn nhiều nền tảng khác vốn có thể phù hợp hơn cho một số tác vụ nhất định. Netflix là một ví dụ cực đoan, nơi quy mô

có lẽ là yếu tố chi phối mạnh nhất, nhưng bạn hiểu ý tôi muốn nói.

Between microservices is where things can get messy, however. If one microservice decides to expose **REST** over HTTP, another makes use of gRPC, and a third uses Java RMI, then integration can become a nightmare, as consuming microservices have to understand and support multiple styles of interchange. This is why I try to stick to the guideline that we should “be worried about what happens between the boxes, and be liberal in what happens inside.”

Tuy nhiên, giữa các microservice mới là chỗ mọi thứ có thể trở nên lộn xộn. Nếu một microservice quyết định phơi bày REST trên HTTP, một cái khác dùng gRPC, và cái thứ ba dùng Java RMI, thì việc tích hợp có thể trở thành cơn ác mộng, vì các microservice tiêu thụ phải hiểu và hỗ trợ nhiều kiểu trao đổi khác nhau. Đây là lý do tôi cố bám theo nguyên tắc rằng chúng ta nên “lo lắng về những gì xảy ra giữa các hộp, và rộng lượng về những gì xảy ra bên trong.”

So a successful architecture is as much about allowing for change to suit the needs of our users as anything. But one thing people often forget is that our system doesn’t just accommodate users; it also accommodates the people that actually build the software themselves. A successful architecture also helps create a nice environment in which to do our work.

Vậy nên một kiến trúc thành công cũng nhiều phần là về việc cho phép thay đổi để phù hợp với nhu cầu của người dùng như bất kỳ điều gì khác. Nhưng một điều mà người ta thường quên là hệ thống của chúng ta không chỉ phục vụ người dùng; nó còn phục vụ chính những người thực sự xây dựng phần mềm. Một kiến trúc thành công cũng giúp tạo ra một môi trường dễ chịu để chúng ta làm việc.

A Social Construct — Một kiến tạo xã hội

No plan survives contact with the enemy. —Helmuth von Moltke (heavily paraphrased)

Không kế hoạch nào sống sót sau khi chạm mặt kẻ thù. —Helmuth von Moltke (diễn giải khá thoáng)

So you’ve thought about the vision, about the constraints, and about what you need to accomplish. You think you understand what will be hard to change, and the spaces where you need to make change possible. Now what? Well, the architecture is what happens, not what you think should happen—this is the difference between vision and reality. Architects of the built environment need to work with the people constructing the building to help them understand what the vision is, but also to change the plan when reality challenges that vision. It’s possible that what you think is possible fundamentally isn’t. If an architect isn’t embedded to some extent with the people creating the system, then they will be unable to help communicate the vision to the people doing the work, nor will the architect understand where that vision is no longer fit for purpose. The construction crew may encounter things on the ground that weren’t foreseen, or perhaps a supply shortage might cause a rethink in terms of the design.

Vậy là bạn đã nghĩ về tầm nhìn, về các ràng buộc, và về những gì bạn cần hoàn thành. Bạn nghĩ mình hiểu cái gì sẽ khó thay đổi, và những không gian nơi bạn cần làm cho thay đổi trở nên khả thi. Giờ thì sao? À, kiến trúc là cái xảy ra, chứ không phải cái bạn nghĩ rằng nên xảy ra—đây là sự khác biệt giữa tầm nhìn và thực tế. Kiến trúc sư của môi trường xây dựng cần làm việc với những người xây dựng tòa nhà để giúp họ hiểu tầm nhìn là gì, nhưng cũng để thay đổi kế hoạch khi thực tế thách thức tầm nhìn ấy. Có khả năng cái mà bạn nghĩ là khả thi thì về cơ bản lại không khả thi. Nếu một kiến trúc sư không hòa mình ở một mức độ nào đó với những người tạo ra hệ thống, thì họ sẽ không thể giúp truyền đạt

tầm nhìn cho những người thực hiện công việc, cũng như sẽ không hiểu được ở đâu tầm nhìn ấy không còn phù hợp với mục đích nữa. Đội thi công có thể gặp những thứ ngoài thực địa mà không ai lường trước, hoặc có lẽ một sự thiếu hụt nguồn cung có thể buộc phải nghĩ lại về thiết kế.

Architecture is what happens, not what is planned. If as an architect you remove yourself from the process of putting this vision into place, then you're not an architect—you're a dreamer. The architecture that will emerge may or may not bear any relationship to what you want. It will happen with or without you. Implementing an architecture requires the work of many people and a host of decisions, large and small.

Kiến trúc là cái xảy ra, chứ không phải cái được lập kế hoạch. Nếu với tư cách kiến trúc sư bạn tự tách mình khỏi quá trình đưa tầm nhìn này vào hiện thực, thì bạn không phải là kiến trúc sư—bạn là kẻ mơ mộng. Kiến trúc rồi sẽ nảy sinh có thể có hoặc không có quan hệ gì với cái bạn muốn. Nó sẽ xảy ra dù có hay không có bạn. Triển khai một kiến trúc đòi hỏi công sức của nhiều người và một loạt quyết định, lớn và nhỏ.

As Grady Booch put it: 4

Như Grady Booch đã nói: 4

In the beginning, the architecture of a software-intensive system is a statement of vision. In the end, the [a]rchitecture of every such system is a reflection of the billions upon billions of small and large, intentional and accidental design decisions made along the way. This means that even if you have a dedicated individual who is ultimately accountable for the architecture, there are many people responsible for putting this vision into practice. Implementing a successful architecture is going to be a team effort. Coming back to Ralph Johnson's quote from earlier, "architecture is a social construct." A great example of this comes from Comcast, which has shared its experiences of how it decentralized decision making through the use of an architecture guild. 5 Given its scale, Comcast decided to leverage experiences from industry steering groups, where collective decision making is key:

Ở khởi đầu, kiến trúc của một hệ thống đậm tính phần mềm là một tuyên ngôn về tầm nhìn. Đến cuối cùng, kiến trúc của mọi hệ thống như vậy là sự phản chiếu của hàng tỷ tỷ quyết định thiết kế lớn nhỏ, có chủ đích và tình cờ, được đưa ra dọc đường. Điều này nghĩa là ngay cả khi bạn có một cá nhân chuyên trách rất cuộc chịu trách nhiệm về kiến trúc, vẫn có nhiều người chịu trách nhiệm đưa tầm nhìn này vào thực tiễn. Triển khai một kiến trúc thành công sẽ là nỗ lực của cả nhóm. Trở lại câu nói của Ralph Johnson ở trên, "kiến trúc là một kiến tạo xã hội." Một ví dụ tuyệt vời đến từ Comcast, công ty đã chia sẻ kinh nghiệm về cách họ phi tập trung hóa việc ra quyết định thông qua việc sử dụng một hội đồng kiến trúc (architecture guild). 5 Với quy mô của mình, Comcast quyết định tận dụng kinh nghiệm từ các nhóm điều phối trong ngành, nơi việc ra quyết định tập thể là then chốt:

At Comcast we realized this problem looked very similar to the way open standards bodies work: getting multiple autonomous groups to agree on technical approaches. We designed an internal Architecture Guild explicitly modeled after a very successful standards body, the Internet Engineering Task Force (IETF) that defines many important Internet protocols. —Jon Moore, chief software architect at Comcast Cable

Tại Comcast chúng tôi nhận ra rằng vấn đề này trông rất giống cách các tổ chức tiêu chuẩn mở vận hành: làm cho nhiều nhóm tự trị đồng thuận về các cách tiếp cận kỹ thuật. Chúng tôi đã thiết kế một Architecture Guild nội bộ được mô phỏng rõ rệt theo một tổ chức tiêu chuẩn rất thành công, Internet Engineering Task Force (IETF), tổ chức định nghĩa nhiều

giao thức Internet quan trọng. —Jon Moore, kiến trúc sư phần mềm trưởng tại Comcast Cable

Comcast’s approach has a level of formality that some organizations might find onerous, but it seems to work well for the company, given its size and distribution.

Cách tiếp cận của Comcast có một mức độ trang trọng mà một số tổ chức có thể thấy nặng nề, nhưng có vẻ nó hoạt động tốt với công ty, xét đến quy mô và sự phân tán của nó.

Habitability — Khả năng cư ngụ

Yet another concept that comes from the built environment and has resonance in the field of software development is habitability . I first learned of this term from Frank Buschmann—he explained that an architect has responsibility for ensuring that the environment they create is nice to work in. If the architecture is the framing of the system, which describes how the hard-to-change things fit together, then there are

Lại thêm một khái niệm nữa đến từ môi trường xây dựng và có sự cộng hưởng trong lĩnh vực phát triển phần mềm là khả năng cư ngụ (habitability). Tôi lần đầu biết đến thuật ngữ này từ Frank Buschmann—ông giải thích rằng một kiến trúc sư có trách nhiệm đảm bảo rằng môi trường họ tạo ra là một nơi dễ chịu để làm việc. Nếu kiến trúc là khung sườn của hệ thống, thứ mô tả cách những thứ khó thay đổi khớp với nhau, thì cũng có

4 Grady Booch (@Grady_Booch), Twitter, September 4, 2020, 5:12 a.m., <https://oreil.ly/ZgPRZ> .

4 Grady Booch (@Grady_Booch), Twitter, ngày 4 tháng 9 năm 2020, 5:12 sáng, <https://oreil.ly/ZgPRZ>.

5 Jon Moore, “Architecture with 800 of My Closest Friends: The Evolution of Comcast’s Architecture Guild,” InfoQ, May 14, 2019, <https://oreil.ly/aIvbi> .

5 Jon Moore, “Architecture with 800 of My Closest Friends: The Evolution of Comcast’s Architecture Guild,” InfoQ, ngày 14 tháng 5 năm 2019, <https://oreil.ly/aIvbi>.

also times when constraints may need to be put in place. Get this wrong, though, and working in the system can become painful and error-prone.

những lúc cần phải đặt ra các ràng buộc. Tuy nhiên, làm sai điều này, thì việc làm việc trong hệ thống có thể trở nên đau đớn và dễ sai sót.

As Richard Gabriel, author of *Patterns of Software* , 6 explains:

Như Richard Gabriel, tác giả của *Patterns of Software*, 6 giải thích:

Habitability is the characteristic of source code that enables programmers coming to the code later in its life to understand its construction and intentions and to change it comfortably and confidently. A modern software development ecosystem consists of more than just code, however—it extends beyond this to the technologies we use and the working practices we adopt. All too frequently I’ve seen developers cursing the technology they are told to use—often technology selected by people who don’t ever have to make use of it. The more you make the evolution of your architecture and the selection of the tools and technology you use a collaborative process, the easier it will be for you to ensure that the end result is a habitable environment in which the people building the system feel happy and productive in their work.

Khả năng cư ngụ là đặc tính của mã nguồn cho phép các lập trình viên đến với mã ở giai đoạn về sau trong vòng đời của nó hiểu được cấu trúc và ý đồ của nó và thay đổi nó một

cách thoải mái và tự tin. Tuy nhiên, một hệ sinh thái phát triển phần mềm hiện đại bao gồm nhiều hơn chỉ mã—nó mở rộng ra ngoài cái này đến cả các công nghệ chúng ta dùng và các tập quán làm việc chúng ta áp dụng. Quá thường xuyên tôi đã thấy các lập trình viên nguyên rủa công nghệ mà họ bị bảo phải dùng—thường là công nghệ được chọn bởi những người chẳng bao giờ phải đụng đến nó. Bạn càng làm cho sự tiến hóa của kiến trúc và việc lựa chọn các công cụ, công nghệ mà bạn dùng trở thành một quá trình cộng tác, thì bạn sẽ càng dễ đảm bảo rằng kết quả cuối cùng là một môi trường có thể cư ngụ trong đó những người xây dựng hệ thống cảm thấy hạnh phúc và năng suất trong công việc.

If we are to ensure that the systems we create are habitable for our developers, then our architects and other decision makers need to understand the impact of their decisions. At the very least, this means spending time with the team, and ideally actually spending time coding with the team. For those of you who practice pair programming, it becomes a simple matter for an architect to join a team for a short period as one member of a pair. Participating in ensemble programming exercises could also yield significant benefits, although an architect taking part in such a group activity needs to be aware how their presence may change the dynamic of the ensemble.

Nếu chúng ta muốn đảm bảo rằng các hệ thống mình tạo ra là có thể cư ngụ được cho các lập trình viên của mình, thì các kiến trúc sư và những người ra quyết định khác cần hiểu tác động của các quyết định của họ. Ít nhất, điều này nghĩa là dành thời gian với nhóm, và lý tưởng là thực sự dành thời gian viết mã cùng nhóm. Đối với những ai trong các bạn thực hành lập trình cặp (pair programming), việc một kiến trúc sư gia nhập một nhóm trong một khoảng thời gian ngắn với tư cách là một thành viên của cặp trở thành chuyện đơn giản. Việc tham gia các bài tập lập trình theo nhóm (ensemble programming) cũng có thể mang lại lợi ích đáng kể, mặc dù một kiến trúc sư tham gia vào một hoạt động tập thể như vậy cần ý thức được rằng sự hiện diện của mình có thể thay đổi động lực của cả nhóm như thế nào.

Ideally, you should work on normal tasks to really understand what “normal” work is like. I cannot emphasize how important it is for the architect to actually spend time with the teams building the system! This is significantly more effective than having a call or just looking at their code. As for how often you should do this, that depends greatly on the size of the team(s) you are working with. But the key is that it should be a routine activity. If you are working with four teams, for example, perhaps make sure you spend half a day with each team every four weeks, working with them on their delivery tasks to ensure you build awareness and improved communications with the teams you are working with.

Lý tưởng nhất, bạn nên làm các tác vụ bình thường để thực sự hiểu công việc “bình thường” là như thế nào. Tôi không thể nhấn mạnh đủ tầm quan trọng của việc kiến trúc sư thực sự dành thời gian với các nhóm đang xây dựng hệ thống! Điều này hiệu quả hơn đáng kể so với việc gọi điện hay chỉ nhìn vào mã của họ. Còn về việc nên làm điều này thường xuyên đến mức nào, thì điều đó phụ thuộc rất nhiều vào quy mô của (các) nhóm bạn đang làm việc cùng. Nhưng điều mấu chốt là nó nên là một hoạt động thường lệ. Chẳng hạn nếu bạn đang làm việc với bốn nhóm, có lẽ hãy đảm bảo rằng bạn dành nửa ngày với mỗi nhóm sau mỗi bốn tuần, làm việc với họ trên các tác vụ chuyển giao của họ để đảm bảo bạn xây dựng nhận thức và cải thiện giao tiếp với các nhóm bạn đang làm việc cùng.

6 Richard Gabriel, *Patterns of Software: Tales from the Software Community* (New York: Oxford University Press, 1996).

6 Richard Gabriel, *Patterns of Software: Tales from the Software Community* (New York: Oxford University Press, 1996).

A Principled Approach — Một cách tiếp cận dựa trên nguyên tắc

Rules are for the obedience of fools and the guidance of wise men. —Generally attributed to Douglas Bader

Các luật lệ là để cho kẻ ngu tuân theo và để dẫn dắt người khôn ngoan. —Thường được gán cho Douglas Bader

Making decisions in system design is all about trade-offs, and microservice architectures give us lots of trade-offs to make! When picking a data store, do we pick a platform that we have less experience with but that gives us better **scaling**? Is it OK for us to have two different technology stacks in our system? What about three? Some decisions can be made completely on the spot with the information available to us, and these are the easiest to make. But what about those decisions that might have to be made on incomplete information?

Việc ra quyết định trong thiết kế hệ thống tất cả đều là về sự đánh đổi (trade-off), và các kiến trúc microservice cho chúng ta rất nhiều sự đánh đổi phải thực hiện! Khi chọn một kho dữ liệu, chúng ta có chọn một nền tảng mà ta ít kinh nghiệm hơn nhưng cho ta khả năng mở rộng tốt hơn không? Có ổn không khi chúng ta có hai stack công nghệ khác nhau trong hệ thống của mình? Còn ba thì sao? Một số quyết định có thể được đưa ra hoàn toàn ngay tại chỗ với thông tin sẵn có, và những quyết định này là dễ nhất. Nhưng còn những quyết định có thể phải được đưa ra dựa trên thông tin chưa đầy đủ thì sao?

Framing can help here, and a great way to help frame our decision making is to define a set of principles and practices that guide it, based on goals that we are trying to achieve. Let's look at each of these aspects of framing in turn.

Việc đóng khung (framing) có thể giúp ích ở đây, và một cách tuyệt vời để giúp đóng khung việc ra quyết định của chúng ta là định nghĩa một bộ các nguyên tắc và tập quán dẫn dắt nó, dựa trên các mục tiêu mà chúng ta đang cố gắng đạt được. Hãy lần lượt xem xét từng khía cạnh này của việc đóng khung.

Strategic Goals — Mục tiêu chiến lược

The role of the architect is already daunting enough, so luckily we usually don't have to also define strategic goals! Strategic goals should speak to where your company is going and to how it sees itself as best making its customers happy. These will be high-level goals and may not include technology at all. They could be defined at a company level or a division level. They might be things like “Expand into Southeast Asia to unlock new markets,” or “Let the customer achieve as much as possible using self-**service**.” The key is that they define where your organization is headed, so you need to make sure the technology is aligned to that.

Vai trò của kiến trúc sư vốn đã đủ đáng sợ rồi, nên may thay chúng ta thường không phải đồng thời định nghĩa các mục tiêu chiến lược! Các mục tiêu chiến lược nên nói lên công ty của bạn đang đi đâu và cách nó tự thấy mình làm khách hàng hài lòng tốt nhất ra sao. Đây sẽ là các mục tiêu ở cấp cao và có thể chẳng hề bao gồm công nghệ. Chúng có thể được định nghĩa ở cấp công ty hay cấp bộ phận. Chúng có thể là những thứ như “Mở rộng sang Đông Nam Á để khai mở các thị trường mới,” hay “Cho phép khách hàng tự làm được càng nhiều càng tốt thông qua tự phục vụ.” Điều mấu chốt là chúng định nghĩa tổ chức của bạn đang hướng tới đâu, nên bạn cần đảm bảo rằng công nghệ được căn chỉnh với điều đó.

If you're the person defining the company's technical vision, this may mean you'll need to spend more time with the nontechnical parts of your organization (or “the business,” as they are often

called). What is the driving vision for the business? And how does it change?

Nếu bạn là người định nghĩa tầm nhìn kỹ thuật của công ty, điều này có thể nghĩa là bạn sẽ cần dành nhiều thời gian hơn với các bộ phận phi kỹ thuật của tổ chức (hay “phía nghiệp vụ,” như chúng thường được gọi). Tầm nhìn dẫn dắt cho nghiệp vụ là gì? Và nó thay đổi ra sao?

Principles — Nguyên tắc

Principles are rules you have made in order to align what you are doing to some larger goal, and they will sometimes change. For example, if one of your strategic goals as an organization is to decrease the time to market for new features, you may define a principle that says that delivery teams have full control throughout the life cycle of their software to ship whenever they are ready, independently of any other team. If another goal is that your organization is moving to aggressively grow its offerings in other countries, you may decide to implement a principle that the entire system must be portable to allow for it to be deployed locally in order to respect sovereignty of data.

Nguyên tắc là các luật lệ mà bạn đặt ra nhằm căn chỉnh những gì bạn đang làm với một mục tiêu lớn hơn nào đó, và đôi khi chúng sẽ thay đổi. Chẳng hạn, nếu một trong các mục tiêu chiến lược của tổ chức bạn là giảm thời gian đưa ra thị trường (time to market) cho các tính năng mới, bạn có thể định nghĩa một nguyên tắc rằng các nhóm chuyển giao có toàn quyền kiểm soát xuyên suốt vòng đời phần mềm của họ để phát hành bất cứ khi nào họ sẵn sàng, độc lập với bất kỳ nhóm nào khác. Nếu một mục tiêu khác là tổ chức của bạn đang quyết liệt phát triển các sản phẩm của mình ở các quốc gia khác, bạn có thể quyết định triển khai một nguyên tắc rằng toàn bộ hệ thống phải có khả năng chuyển dời (portable) để cho phép nó được triển khai cục bộ nhằm tôn trọng chủ quyền dữ liệu (sovereignty of data).

You probably don’t want loads of these. Fewer than 10 principles is a good number— small enough for people to remember them, or to fit on small posters. The more principles you have, the greater the chance that they overlap or contradict each other.

Bạn có lẽ không muốn có quá nhiều thứ kiểu này. Dưới 10 nguyên tắc là một con số hợp lý—đủ nhỏ để mọi người nhớ được, hoặc để vừa trên các tấm áp phích nhỏ. Càng có nhiều nguyên tắc, khả năng chúng chồng chéo hay mâu thuẫn nhau càng lớn.

Heroku’s Twelve Factors is a set of design principles structured around the goal of helping you create applications that work well on the Heroku platform. These principles may also make sense in other contexts. Some of them are actually constraints based on behaviors your application needs to exhibit in order to work on Heroku. A constraint is really something that is very hard (or virtually impossible) to change, whereas principles are things we decide to choose. You may decide to explicitly call out those things that are principles versus those that are constraints to help highlight those things you really can’t change. Personally, I think there can be some value in keeping them in the same list, to encourage challenging constraints every now and then and see if they really are immovable!

Twelve Factors của Heroku là một bộ các nguyên tắc thiết kế được cấu trúc xoay quanh mục tiêu giúp bạn tạo ra các ứng dụng hoạt động tốt trên nền tảng Heroku. Các nguyên tắc này cũng có thể hợp lý trong các bối cảnh khác. Một số trong chúng thực ra là các ràng buộc dựa trên những hành vi mà ứng dụng của bạn cần thể hiện để hoạt động trên Heroku. Một ràng buộc thực ra là thứ rất khó (hoặc gần như bất khả thi) để thay đổi, trong khi nguyên tắc là những thứ ta quyết định chọn. Bạn có thể quyết định nêu rõ ra những thứ là nguyên tắc so với những thứ là ràng buộc để giúp làm nổi bật những thứ bạn thực

sự không thể thay đổi. Cá nhân tôi nghĩ có thể có chút giá trị trong việc giữ chúng trong cùng một danh sách, để thỉnh thoảng khuyến khích việc thách thức các ràng buộc và xem liệu chúng có thực sự bất di bất dịch hay không!

Practices — Tập quán

Our practices are how we ensure our principles are being carried out. They are a set of detailed, practical guidelines for performing tasks. They will often be technology specific and should be low level enough that any developer can understand them. Practices could include coding guidelines, the fact that all log data needs to be captured centrally, or the fact that HTTP/REST is the standard integration style. Due to their technical nature, practices will typically change more often than principles.

Tập quán của chúng ta là cách chúng ta đảm bảo các nguyên tắc của mình đang được thực thi. Chúng là một bộ các hướng dẫn chi tiết, thực tiễn để thực hiện các tác vụ. Chúng thường mang tính đặc thù về công nghệ và nên đủ ở mức thấp để bất kỳ lập trình viên nào cũng có thể hiểu được. Tập quán có thể bao gồm các hướng dẫn viết mã, việc tất cả dữ liệu log cần được thu thập tập trung, hay việc HTTP/REST là kiểu tích hợp chuẩn. Do bản chất kỹ thuật của chúng, tập quán thường sẽ thay đổi nhiều hơn so với nguyên tắc.

As with principles, sometimes practices reflect constraints within your organization. For example, if you have decided to pick Azure as your cloud platform, this will need to be reflected in your practices.

Cũng như với nguyên tắc, đôi khi tập quán phản ánh các ràng buộc trong tổ chức của bạn. Chẳng hạn, nếu bạn đã quyết định chọn Azure làm nền tảng đám mây của mình, điều này sẽ cần được phản ánh trong các tập quán của bạn.

Practices should underpin your principles. A principle stating that delivery teams control the full life cycle of their systems may mean you have a practice stating that all microservices are deployed into isolated AWS accounts, providing self-service management of the resources and isolation from other teams.

Tập quán nên làm nền tảng cho các nguyên tắc của bạn. Một nguyên tắc nói rằng các nhóm chuyển giao kiểm soát toàn bộ vòng đời của hệ thống của họ có thể nghĩa là bạn có một tập quán nói rằng tất cả microservice được triển khai vào các tài khoản AWS biệt lập, cung cấp khả năng tự quản lý tài nguyên và sự cô lập khỏi các nhóm khác.

Combining Principles and Practices — Kết hợp nguyên tắc và tập quán

One person's principles are another's practices. You might decide to call the use of HTTP/REST a principle rather than a practice, for example. And that would be fine. The key point is that there is value in having overarching ideas that guide how the system evolves, and in having enough detail so that people know how to implement those ideas. For a small enough group, perhaps a single team, combining principles and practices might be OK. However, for larger organizations, where the technology and working practices may differ from place to place, you may want a different set of practices in different places, as long as they all map to a common set of principles. A .NET team, for example, might have one set of practices, and a Java team another. The principles, though, could be the same for both.

Nguyên tắc của người này là tập quán của người khác. Chẳng hạn, bạn có thể quyết định gọi việc dùng HTTP/REST là một nguyên tắc thay vì một tập quán. Và điều đó cũng ổn. Điểm mấu chốt là có giá trị trong việc có những ý tưởng bao trùm dẫn dắt cách hệ thống tiến hóa, và trong việc có đủ chi tiết để mọi người biết cách triển khai những ý tưởng ấy. Đối với một nhóm đủ nhỏ, có lẽ là một nhóm đơn lẻ, việc kết hợp nguyên tắc và tập quán có thể ổn. Tuy nhiên, đối với các tổ chức lớn hơn, nơi công nghệ và các tập quán làm việc có thể khác nhau giữa nơi này với nơi khác, bạn có thể muốn có một bộ tập quán khác nhau ở những nơi khác nhau, miễn là tất cả chúng đều ánh xạ tới một bộ nguyên tắc chung. Chẳng hạn, một nhóm .NET có thể có một bộ tập quán, và một nhóm Java một bộ khác. Tuy nhiên, các nguyên tắc có thể giống nhau cho cả hai.

A Real-World Example — Một ví dụ thực tế

An old colleague of mine, Evan Bottcher, developed the diagram shown in Figure 16-3 in the course of working with one of his clients. The figure shows the interplay of goals, principles, and practices in a very clear format. Over the course of a couple of years, the practices on the far right will change fairly regularly, whereas the principles remain fairly static. A diagram such as this can be printed nicely on a single sheet of paper and shared, and each idea is simple enough for the average developer to remember. There is, of course, more detail behind each point here, but being able to articulate this in summary form is very useful.

Một đồng nghiệp cũ của tôi, Evan Bottcher, đã phát triển sơ đồ trong Hình 16-3 trong quá trình làm việc với một trong các khách hàng của anh ấy. Hình này cho thấy sự tương tác giữa các mục tiêu, nguyên tắc và tập quán theo một định dạng rất rõ ràng. Qua một vài năm, các tập quán ở phía cực phải sẽ thay đổi khá thường xuyên, trong khi các nguyên tắc vẫn khá tĩnh tại. Một sơ đồ như thế này có thể được in gọn gàng trên một tờ giấy đơn và chia sẻ, và mỗi ý tưởng đủ đơn giản để lập trình viên trung bình nhớ được. Tất nhiên, đằng sau mỗi điểm ở đây còn có nhiều chi tiết hơn, nhưng việc có thể diễn đạt điều này dưới dạng tóm lược thì rất hữu ích.

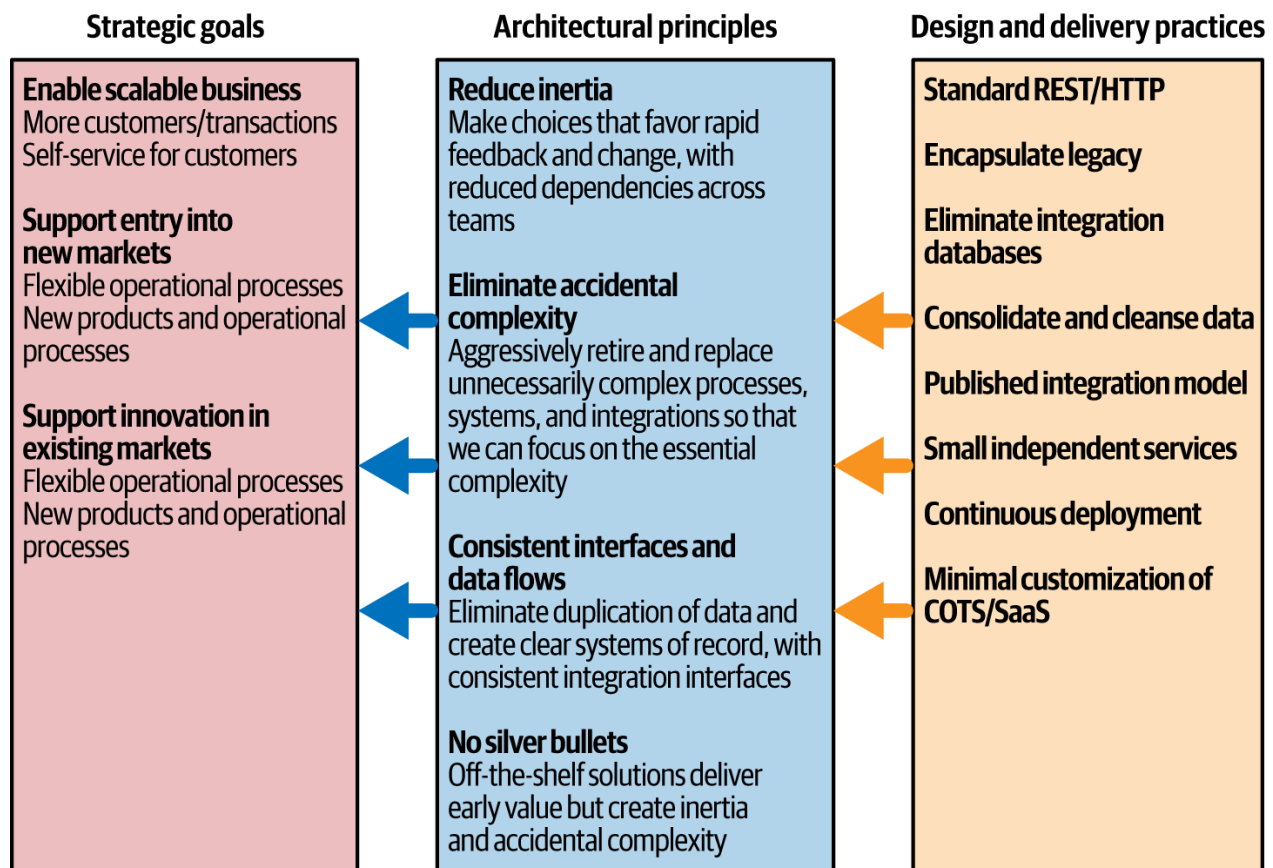


Figure 16-3. A real-world example of principles and practices

Hình 16-3. Một ví dụ thực tế về các nguyên tắc và tập quán

It makes sense to have documentation supporting some of these items, and even better is having working code that shows how these practices can be implemented. In “The Platform” on page 506, we looked at how the creation of a common set of tools can make it easy for developers to do the right thing—ideally, the platform should make following these practices as easy as possible, and as the practices change, the platform should change accordingly.

Việc có tài liệu hỗ trợ cho một số trong các mục này là hợp lý, và tốt hơn nữa là có mã chạy được cho thấy cách các tập quán này có thể được triển khai. Trong “The Platform” ở trang 506, chúng ta đã xem xét cách việc tạo ra một bộ công cụ chung có thể khiến lập trình viên dễ dàng làm điều đúng đắn—lý tưởng nhất, nền tảng nên làm cho việc tuân theo các tập quán này dễ nhất có thể, và khi các tập quán thay đổi, nền tảng cũng nên thay đổi theo.

Guiding an Evolutionary Architecture — Dẫn dắt một kiến trúc tiến hóa

So if our architecture is not static but is ever-changing and evolving, how do we make sure it is growing and changing in the way we want, rather than just mutating into some unmanageable giant blob of pain, suffering, and recriminations? In Building Evolutionary Architectures, 7 the authors outline fitness functions to help collect information about the relative “fitness” of the architecture in order to help architects decide if they need to take action. From the book:

Vậy nếu kiến trúc của chúng ta không tĩnh mà luôn thay đổi và tiến hóa, thì làm thế nào chúng ta đảm bảo nó đang lớn lên và thay đổi theo cách chúng ta muốn, thay vì chỉ đột biến thành một khối khổng lồ không thể quản lý gồm đau đớn, khổ sở và những lời quy trách? Trong cuốn *Building Evolutionary Architectures*, 7 các tác giả phác họa các hàm thích nghi (fitness function) để giúp thu thập thông tin về “độ thích nghi” tương đối của kiến trúc nhằm giúp kiến trúc sư quyết định liệu họ có cần hành động hay không. Trích từ cuốn sách:

Evolutionary computing includes a number of mechanisms that allow a solution to gradually emerge via small changes in each generation of the software. At each generation of the solution, the engineer assesses the current state: Is it closer to or further away from the ultimate goal? For example, when using a genetic algorithm to optimize wing design, the fitness function assess[es] wind resistance, weight, air flow, and other characteristics desirable to good wing design. Architects define a fitness function to explain what better is and to help measure when the goal is met. In software, fitness functions check that developers preserve important architectural characteristics. The idea of a fitness function is that it is used to understand the current state of some important property, such that if that property changes outside of some allowable bounds, then the change needs to be looked into. Typically, fitness functions will be used to ensure that the architecture is being built to follow the principles and constraints that have been laid down.

Tính toán tiến hóa bao gồm một số cơ chế cho phép một lời giải dần dần nảy sinh thông qua những thay đổi nhỏ ở mỗi thế hệ của phần mềm. Ở mỗi thế hệ của lời giải, kỹ sư đánh giá trạng thái hiện tại: Nó đang gần hơn hay xa hơn mục tiêu tối hậu? Chẳng hạn, khi dùng một thuật toán di truyền để tối ưu thiết kế cánh máy bay, hàm thích nghi đánh giá lực cản gió, trọng lượng, luồng không khí, và các đặc tính khác mong muốn cho một thiết kế cánh tốt. Kiến trúc sư định nghĩa một hàm thích nghi để giải thích cái “tốt hơn” là gì và để giúp đo lường khi nào mục tiêu được đáp ứng. Trong phần mềm, các hàm thích nghi kiểm tra xem các lập trình viên có giữ gìn các đặc tính kiến trúc quan trọng hay không. Ý tưởng của một hàm thích nghi là nó được dùng để hiểu trạng thái hiện tại của một thuộc tính quan trọng nào đó, sao cho nếu thuộc tính đó thay đổi ra ngoài một số giới hạn cho phép, thì sự thay đổi cần được xem xét kỹ. Thông thường, các hàm thích nghi sẽ được dùng để đảm bảo rằng kiến trúc đang được xây dựng để tuân theo các nguyên tắc và ràng buộc đã được đặt ra.

To borrow an example from *Building Evolutionary Architectures*, consider the requirement that the response from a given service must be received in 100 ms or less. You could implement a fitness function to collect performance data from this service, perhaps either in a performance test environment or from a real-world running system to ensure that the actual behavior of the system meets the requirements. *Building Evolutionary Architectures* goes into a lot more detail on this topic, and I thoroughly recommend it if you want to explore this concept further.

Để mượn một ví dụ từ *Building Evolutionary Architectures*, hãy xét yêu cầu rằng phản hồi từ một dịch vụ nhất định phải được nhận trong 100 ms hoặc ít hơn. Bạn có thể triển khai một hàm thích nghi để thu thập dữ liệu hiệu năng từ dịch vụ này, có lẽ trong một môi trường kiểm thử hiệu năng hoặc từ một hệ thống đang chạy thực tế để đảm bảo rằng hành vi thực tế của hệ thống đáp ứng các yêu cầu. *Building Evolutionary Architectures* đi sâu hơn nhiều vào chủ đề này, và tôi hết lòng khuyên bạn đọc nó nếu bạn muốn khám phá khái niệm này xa hơn.

Fitness functions for architecture can come in many shapes and forms. The fundamental concept, though, is that you collect real-world data to understand whether or not your architecture is achieving “fitness” against that criteria. This could relate to system performance, code **coupling**, cycle time, or a host of other aspects. These fitness functions act as another source of information to

help an architect understand where they might need to get involved. Please note, however, that for me, fitness functions work best when combined with close collaboration with the people building the system. Fitness functions should be a useful way to help you understand if the architecture is moving in the right direction, but they don't replace the need to

Các hàm thích nghi cho kiến trúc có thể đến dưới nhiều hình hài và dạng thức. Tuy nhiên, khái niệm cơ bản là bạn thu thập dữ liệu thực tế để hiểu liệu kiến trúc của bạn có đang đạt “độ thích nghi” so với tiêu chí đó hay không. Điều này có thể liên quan đến hiệu năng hệ thống, mức độ coupling của mã, thời gian chu kỳ (cycle time), hoặc một loạt các khía cạnh khác. Các hàm thích nghi này đóng vai trò như một nguồn thông tin khác để giúp kiến trúc sư hiểu ở đâu họ có thể cần can dự vào. Tuy nhiên, xin lưu ý rằng đối với tôi, các hàm thích nghi hoạt động tốt nhất khi được kết hợp với sự cộng tác chặt chẽ với những người đang xây dựng hệ thống. Các hàm thích nghi nên là một cách hữu ích để giúp bạn hiểu liệu kiến trúc có đang đi đúng hướng hay không, nhưng chúng không thay thế nhu cầu

7 Neal Ford, Rebecca Parsons, and Patrick Kua, *Building Evolutionary Architectures* (Sebastopol: O'Reilly, 2017).

7 Neal Ford, Rebecca Parsons, và Patrick Kua, *Building Evolutionary Architectures* (Sebastopol: O'Reilly, 2017).

actually speak with people on the ground. In fact, I'd suggest that defining the right fitness functions will require close collaboration.

thực sự trò chuyện với những người ở thực địa. Thực ra, tôi sẽ gợi ý rằng việc định nghĩa các hàm thích nghi đúng đắn sẽ đòi hỏi sự cộng tác chặt chẽ.

Architecture in a Stream-Aligned Organization — Kiến trúc trong một tổ chức theo hướng dòng chảy

In Chapter 15, we looked at how modern software delivery organizations are shifting toward a more stream-aligned model in which autonomous independent teams focus on the end-to-end delivery of functionality, with their priorities being product driven. We also talked about cross-cutting teams—enabling teams that support stream-aligned teams. Where does the architect fit into this world? Well, sometimes the scope of a **stream-aligned team** is complex enough to require a dedicated architect (here again we often see a blurring of the lines between the traditional technical lead and architect roles). In many cases, though, architects are asked to work across multiple teams.

Trong Chương 15, chúng ta đã xem xét cách các tổ chức chuyển giao phần mềm hiện đại đang chuyển dịch sang một mô hình theo hướng dòng chảy (stream-aligned) hơn, trong đó các nhóm tự trị độc lập tập trung vào việc chuyển giao chức năng từ đầu đến cuối (end-to-end), với các ưu tiên của họ được dẫn dắt bởi sản phẩm. Chúng ta cũng đã nói về các nhóm xuyên suốt (cross-cutting)—các enabling team hỗ trợ các stream-aligned team. Kiến trúc sư nằm ở đâu trong thế giới này? À, đôi khi phạm vi của một stream-aligned team đủ phức tạp để đòi hỏi một kiến trúc sư chuyên trách (ở đây một lần nữa chúng ta thường thấy sự mờ nhòe ranh giới giữa các vai trò trưởng nhóm kỹ thuật và kiến trúc sư truyền thống). Tuy nhiên, trong nhiều trường hợp, các kiến trúc sư được yêu cầu làm việc xuyên qua nhiều nhóm.

Many of the responsibilities of the architect can be seen as enabling responsibilities—clearly communicating technical vision, understanding challenges as they emerge, and helping adapt the technical vision accordingly. The architect helps connect people, keeping an eye on the bigger

picture and helping teams understand how what they are doing fits into the greater whole. This fits neatly into the idea of an architect being part of an **enabling team**, as we see in Figure 16-4 . Such an enabling team could consist of a mix of people—perhaps folks who are dedicated to the team full-time, and others who pitch in to help from time to time.

Nhiều trách nhiệm của kiến trúc sư có thể được xem là các trách nhiệm mang tính hỗ trợ (enabling)—truyền đạt rõ ràng tầm nhìn kỹ thuật, hiểu các thách thức khi chúng nảy sinh, và giúp điều chỉnh tầm nhìn kỹ thuật cho phù hợp. Kiến trúc sư giúp kết nối con người, để mắt đến bức tranh lớn hơn và giúp các nhóm hiểu cách những gì họ đang làm khớp vào tổng thể lớn hơn. Điều này khớp gọn gàng với ý tưởng về một kiến trúc sư là một phần của một enabling team, như chúng ta thấy trong Hình 16-4. Một enabling team như vậy có thể gồm một hỗn hợp con người—có lẽ những người chuyên trách cho nhóm toàn thời gian, và những người khác thỉnh thoảng góp sức giúp đỡ.

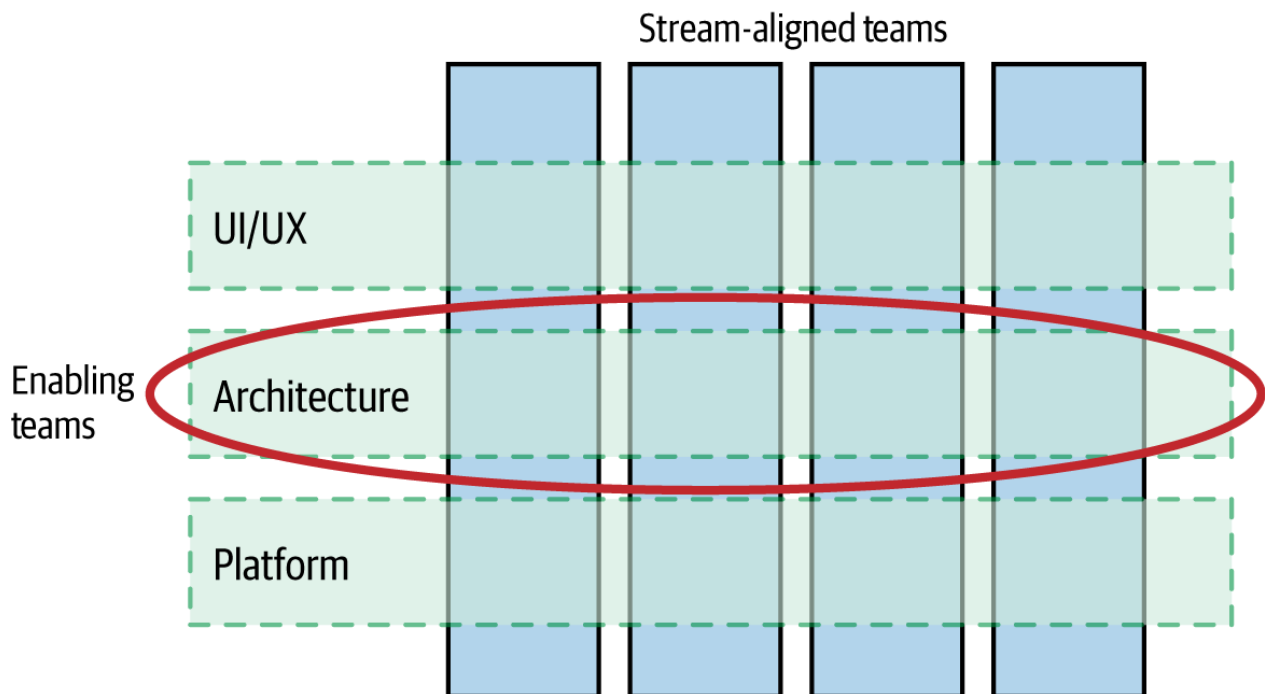


Figure 16-4. An architecture function as an enabling team

Hình 16-4. Một chức năng kiến trúc dưới dạng một enabling team

A model I greatly favor is to have a small number of dedicated architects in this team (perhaps just one or two people in many cases), but having this team augmented over time with technologists from each delivery team—the technical leads of each team at a minimum. The architect is responsible for making sure the group works. This distributes the work and ensures that there is a higher level of buy-in. It also ensures that information flows freely from the teams into the group, and as a result, the decision making is much more sensible and informed.

Một mô hình mà tôi rất ưa thích là có một số ít kiến trúc sư chuyên trách trong nhóm này (có lẽ chỉ một hoặc hai người trong nhiều trường hợp), nhưng theo thời gian bổ sung cho nhóm này các nhà công nghệ từ mỗi nhóm chuyển giao—tối thiểu là các trưởng nhóm kỹ thuật của từng nhóm. Kiến trúc sư chịu trách nhiệm đảm bảo rằng cả nhóm vận hành được. Điều này phân tán công việc và đảm bảo có mức độ đồng thuận cao hơn. Nó cũng đảm bảo rằng thông tin chảy tự do từ các nhóm vào nhóm này, và kết quả là việc ra quyết định trở nên hợp lý và am hiểu hơn nhiều.

Sometimes the group may make decisions that the architect disagrees with. At this point, what is the architect to do? Having been in this position before, I can tell you this is one of the most challenging situations to face. Often I take the approach that I should go with the group decision. I take the view that I've done my best to convince people, but ultimately I wasn't convincing enough. The group is often much wiser than the individual, and I've been proven wrong more than once! And imagine how disempowering it can be for a group to have been given space to come up with a decision and then ultimately be ignored. But sometimes I have overruled the group. But why, and when? How do you draw the lines?

Đôi khi nhóm có thể đưa ra những quyết định mà kiến trúc sư không đồng tình. Đến lúc này, kiến trúc sư phải làm gì? Từng ở trong vị trí này trước đây, tôi có thể nói với bạn rằng đây là một trong những tình huống khó đối mặt nhất. Thường thì tôi chọn cách đi theo quyết định của nhóm. Tôi giữ quan điểm rằng tôi đã cố hết sức để thuyết phục mọi người, nhưng rốt cuộc tôi đã không đủ sức thuyết phục. Nhóm thường khôn ngoan hơn cá nhân nhiều, và tôi đã hơn một lần được chứng minh là sai! Và thử tưởng tượng việc một nhóm được trao không gian để đi đến một quyết định rồi rốt cuộc bị phớt lờ thì có thể làm họ mất quyền lực đến mức nào. Nhưng đôi khi tôi đã bác bỏ quyết định của nhóm. Nhưng tại sao, và khi nào? Bạn vạch ranh giới ở đâu?

Think about teaching a child to ride a bike. You can't ride it for them. You watch them wobble, but if you step in every time it looks like they might fall off, then they'll never learn, and in any case they fall off far less than you think they will! But if you see them about to veer into traffic or into a nearby duck pond, then you have to step in. Of course, I've frequently been proven wrong in such situations—I've let the team go off and do something that I felt was wrong, and what they did worked! Likewise, as an architect, you need to have a firm grasp of when, figuratively, your team is steering into a duck pond. You also need to be aware that even if you know you are right and overrule the team, this can undermine your position and also make the team feel that they don't have a say. Sometimes the right thing is to go along with a decision you don't agree with. Knowing when to do this and when not to is tough but sometimes vital.

Hãy nghĩ về việc dạy một đứa trẻ đi xe đạp. Bạn không thể đạp xe thay nó. Bạn nhìn nó loạng choạng, nhưng nếu bạn ra tay mỗi khi có vẻ như nó sắp ngã, thì nó sẽ chẳng bao giờ học được, và dù sao đi nữa nó ngã ít hơn nhiều so với bạn tưởng! Nhưng nếu bạn thấy nó sắp lao vào dòng xe cộ hay vào một cái ao vẹt gần đó, thì bạn phải ra tay. Tất nhiên, tôi thường xuyên được chứng minh là sai trong những tình huống như vậy—tôi đã để nhóm đi làm một thứ mà tôi cảm thấy là sai, và cái họ làm lại có tác dụng! Tương tự, với tư cách kiến trúc sư, bạn cần nắm chắc khi nào, theo nghĩa bóng, nhóm của bạn đang lái vào một cái ao vẹt. Bạn cũng cần ý thức rằng ngay cả khi bạn biết mình đúng và bác bỏ quyết định của nhóm, điều này có thể làm suy yếu vị thế của bạn và cũng khiến nhóm cảm thấy họ không có tiếng nói. Đôi khi điều đúng đắn là đi theo một quyết định mà bạn không đồng tình. Biết khi nào nên làm điều này và khi nào không thì khó nhưng đôi khi tối quan trọng.

Where things get interesting, as we'll discuss shortly, is when an architect also has to get involved in **governance** activities. This can cause some confusion about the role of any cross-cutting architecture team. What happens when one team diverges from the technical strategy? Is that OK? Perhaps it's a sensible exception, but it might also cause more fundamental issues. A short-term decision made in the name of expediency might compromise bigger changes that are trying to be made. Imagine that the architecture group is trying to help shift the organization away from the use of centralized data due to the coupling and operational issues it causes, but one of the teams decides to just throw some new data into a **shared database**, as it is under pressure to deliver quickly. What happens then?

Chỗ mọi thứ trở nên thú vị, như chúng ta sẽ thảo luận ngay sau đây, là khi một kiến trúc

sự cũng phải dính líu vào các hoạt động governance. Điều này có thể gây ra một số lúng túng về vai trò của bất kỳ nhóm kiến trúc xuyên suốt nào. Điều gì xảy ra khi một nhóm đi chệch khỏi chiến lược kỹ thuật? Điều đó có ổn không? Có lẽ đó là một ngoại lệ hợp lý, nhưng nó cũng có thể gây ra những vấn đề căn bản hơn. Một quyết định ngắn hạn được đưa ra nhân danh sự tiện lợi có thể làm tổn hại những thay đổi lớn hơn đang được cố gắng thực hiện. Hãy tưởng tượng rằng nhóm kiến trúc đang cố giúp tổ chức rời xa việc sử dụng dữ liệu tập trung do sự coupling và các vấn đề vận hành mà nó gây ra, nhưng một trong các nhóm quyết định cứ ném một số dữ liệu mới vào một cơ sở dữ liệu dùng chung, vì nó đang chịu áp lực phải chuyển giao nhanh. Khi đó điều gì xảy ra?

In my experience, this all comes down to good, clear communication and an understanding of responsibilities. If I saw a product owner making decisions that I felt were going to undermine some sort of cross-cutting activities I was working toward, I'd go and have a chat with them. Perhaps the answer is that the short-term decision is right (and arguably this ends up being some sort of **technical debt** that we have consciously taken onboard). In other cases, perhaps the product owner is able to change what they are planning to help work with the overall strategy. In the worst cases, the issue might need to be escalated.

Theo kinh nghiệm của tôi, tất cả điều này quy về sự giao tiếp tốt, rõ ràng và một sự hiểu biết về các trách nhiệm. Nếu tôi thấy một product owner đang đưa ra những quyết định mà tôi cảm thấy sẽ làm tổn hại một loại hoạt động xuyên suốt nào đó mà tôi đang hướng tới, tôi sẽ đến trò chuyện với họ. Có lẽ câu trả lời là quyết định ngắn hạn đó đúng (và có thể nói cái này rốt cuộc trở thành một loại nợ kỹ thuật mà chúng ta đã chủ tâm gánh lấy). Trong các trường hợp khác, có lẽ product owner có thể thay đổi cái họ đang dự định để giúp ăn khớp với chiến lược tổng thể. Trong trường hợp tệ nhất, vấn đề có thể cần được đẩy lên cấp cao hơn.

At REA, the online real-estate company I've talked about in a few earlier chapters, product owners would occasionally make decisions to prioritize work in such a way that it caused technical debt to build up, leading to subsequent problems. The issue was that the product owners were primarily held to account in regard to their ability to deliver features and make customers happy—whereas often the issues around technical debt were laid at the feet of the technical leaders. A shift was made to make the product owners also responsible for aspects of the software that were technical in nature—this meant that they had to take a more active role in understanding the more technical aspects of the system (**security** or performance, for example) and work more collaboratively with the technical experts in terms of prioritizing work to be done. The act of making nontechnical product owners more accountable for prioritization around technical activities is nontrivial, but it is absolutely worth it in my experience.

Tại REA, công ty bất động sản trực tuyến mà tôi đã nói đến trong vài chương trước, các product owner thỉnh thoảng sẽ đưa ra những quyết định ưu tiên công việc theo cách làm cho nợ kỹ thuật tích tụ, dẫn đến những vấn đề về sau. Vấn đề là các product owner chủ yếu bị quy trách nhiệm về khả năng chuyển giao các tính năng và làm khách hàng hài lòng của họ—trong khi thường thì các vấn đề liên quan đến nợ kỹ thuật lại bị đổ lên đầu các lãnh đạo kỹ thuật. Một sự chuyển dịch đã được thực hiện để làm cho các product owner cũng chịu trách nhiệm về các khía cạnh của phần mềm mang tính kỹ thuật—điều này nghĩa là họ phải có vai trò chủ động hơn trong việc hiểu các khía cạnh kỹ thuật hơn của hệ thống (ví dụ bảo mật hay hiệu năng) và làm việc cộng tác hơn với các chuyên gia kỹ thuật trong việc ưu tiên công việc cần làm. Hành động làm cho các product owner phi kỹ thuật chịu trách nhiệm hơn về việc ưu tiên quanh các hoạt động kỹ thuật là không tầm thường, nhưng theo kinh nghiệm của tôi nó hoàn toàn xứng đáng.

Building a Team — Xây dựng một đội ngũ

Being the main point person for the technical vision of your system and ensuring that you're executing on this vision isn't just about making sure the right technology decisions get made. It's the people you work with who will be doing the work. It is the role of any technical leader to help these people grow—to help them be part of creating that vision—and to ensure that they can be active participants in shaping and implementing the vision too.

Việc là đầu mối chính cho tầm nhìn kỹ thuật của hệ thống và đảm bảo bạn đang thực thi được tầm nhìn này không chỉ là về việc đảm bảo các quyết định công nghệ đúng đắn được đưa ra. Chính những con người bạn làm việc cùng mới là những người sẽ thực hiện công việc. Đó là vai trò của bất kỳ lãnh đạo kỹ thuật nào trong việc giúp những con người này phát triển—giúp họ trở thành một phần của việc tạo ra tầm nhìn ấy—và đảm bảo rằng họ cũng có thể là những người tham gia chủ động vào việc định hình và triển khai tầm nhìn.

Helping the people around you with their own career growth can take many forms, most of which are outside the scope of this book. There is one aspect, though, in which a microservice architecture is especially relevant. With larger, monolithic systems, there are fewer opportunities for people to step up and “own” something. With microservices, on the other hand, we have multiple autonomous codebases that will have their own independent life cycles. Helping people step up by having them take **ownership** of individual microservices before accepting more responsibility can be a great way to help them achieve their own career goals, and at the same time it lightens the load on whomever is in charge!

Việc giúp đỡ những người quanh bạn với sự phát triển sự nghiệp của chính họ có thể có nhiều hình thức, hầu hết trong số đó nằm ngoài phạm vi của cuốn sách này. Tuy nhiên, có một khía cạnh trong đó một kiến trúc microservice đặc biệt liên quan. Với các hệ thống lớn hơn, kiểu monolith, có ít cơ hội hơn để người ta bước lên và “sở hữu” một thứ gì đó. Ngược lại, với microservices, chúng ta có nhiều cơ sở mã tự trị mà sẽ có vòng đời độc lập của riêng chúng. Việc giúp mọi người bước lên bằng cách để họ nhận quyền sở hữu các microservice riêng lẻ trước khi nhận thêm trách nhiệm có thể là một cách tuyệt vời để giúp họ đạt được các mục tiêu sự nghiệp của chính họ, và đồng thời nó cũng giảm tải cho bất kỳ ai đang phụ trách!

I am a strong believer that great software comes from great people. If you worry only about the technology side of the equation, you're missing way more than half the picture.

Tôi là người tin tưởng mạnh mẽ rằng phần mềm tuyệt vời đến từ những con người tuyệt vời. Nếu bạn chỉ lo lắng về phía công nghệ của phương trình, bạn đang bỏ lỡ hơn nửa bức tranh.

The Required Standard — Tiêu chuẩn bắt buộc

When you're working through your practices and thinking about the trade-offs you need to make, one of the most important balances to find is how much variability to allow in your system. One of the key ways to identify what should be constant from microservice to microservice is to define what a well-behaved, good microservice looks like. What is a “good citizen” microservice in your system? What capabilities does it need to have to ensure that your system is manageable, and that one bad microservice doesn't bring down the whole system? As with people, what a “good citizen” microservice is in one context does not reflect what it looks like somewhere else. Nonetheless, there are some common characteristics of well-behaved microservices that I think are fairly important to observe. These are the few key areas in which allowing too much divergence can result in a

pretty horrid time. As Ben Christensen from Facebook puts it, when you think about the bigger picture, “it needs to be a cohesive system made of many small parts with autonomous life cycles but all coming together.” So you need to find a balance in which you optimize for the autonomy of individual microservices without losing sight of the bigger picture. Defining clear attributes that each microservice should have is one way of being clear as to where that balance sits. Let’s touch on some of those attributes.

Khi bạn làm việc qua các tập quán của mình và suy nghĩ về những sự đánh đổi mà bạn cần thực hiện, một trong những cân bằng quan trọng nhất cần tìm là cho phép bao nhiêu sự biến thiên trong hệ thống của bạn. Một trong những cách then chốt để xác định cái gì nên là hằng định từ microservice này sang microservice khác là định nghĩa xem một microservice tốt, hành xử chuẩn mực trông như thế nào. Một microservice “công dân tốt” trong hệ thống của bạn là gì? Nó cần có những năng lực nào để đảm bảo rằng hệ thống của bạn có thể quản lý được, và rằng một microservice tồi không kéo sập cả hệ thống? Cũng như với con người, một microservice “công dân tốt” trong một bối cảnh không phản ánh việc nó trông như thế nào ở một nơi khác. Tuy nhiên, có một số đặc điểm chung của các microservice hành xử chuẩn mực mà tôi nghĩ là khá quan trọng cần tuân thủ. Đây là một vài lĩnh vực then chốt trong đó việc cho phép quá nhiều sự khác biệt có thể dẫn đến một quãng thời gian khá kinh hoàng. Như Ben Christensen từ Facebook nói, khi bạn nghĩ về bức tranh lớn hơn, “nó cần là một hệ thống mạch lạc được tạo nên từ nhiều phần nhỏ với các vòng đời tự trị nhưng tất cả cùng hợp lại.” Vậy nên bạn cần tìm một sự cân bằng trong đó bạn tối ưu cho tính tự trị của các microservice riêng lẻ mà không đánh mất bức tranh lớn hơn. Định nghĩa các thuộc tính rõ ràng mà mỗi microservice nên có là một cách để làm rõ điểm cân bằng ấy nằm ở đâu. Hãy chạm vào một số trong những thuộc tính đó.

Monitoring — Giám sát

It is essential that we are able to draw up coherent, cross-service views of our system health. This has to be a system-wide view, not a microservice-specific view. As we discussed in Chapter 10, knowing the health of an individual microservice is useful, but often only when you’re trying to diagnose a wider problem or understand a larger trend. To make this as easy as possible, I would suggest ensuring that all microservices emit health-related and general-**monitoring**-related metrics in the same way.

Việc chúng ta có thể dựng nên những góc nhìn mạch lạc, xuyên dịch vụ về sức khỏe hệ thống của mình là thiết yếu. Đây phải là một góc nhìn cấp toàn hệ thống, không phải một góc nhìn đặc thù theo một microservice. Như chúng ta đã thảo luận ở Chương 10, biết được sức khỏe của một microservice riêng lẻ thì hữu ích, nhưng thường chỉ khi bạn đang cố chẩn đoán một vấn đề rộng hơn hoặc hiểu một xu hướng lớn hơn. Để làm điều này dễ nhất có thể, tôi sẽ gợi ý đảm bảo rằng tất cả microservice phát ra các chỉ số liên quan đến sức khỏe và giám sát nói chung theo cùng một cách.

You might choose to adopt a push mechanism, where each microservice needs to push this data into a central location. Whatever you pick, try to keep it standardized. Make the technology inside the box opaque, and don’t require that your monitoring systems change in order to support it. Logging falls into the same category here: we need it in one place.

Bạn có thể chọn áp dụng một cơ chế đẩy (push), trong đó mỗi microservice cần đẩy dữ liệu này vào một vị trí trung tâm. Dù bạn chọn cái gì, hãy cố giữ cho nó được chuẩn hóa. Hãy làm cho công nghệ bên trong cái hộp trở nên mờ đục, và đừng đòi hỏi các hệ thống giám sát của bạn phải thay đổi để hỗ trợ nó. Logging cũng rơi vào cùng loại này ở đây: chúng ta

cần nó ở một chỗ.

Interfaces — Interface

Picking a small number of defined interface technologies helps integrate new consumers. Having one standard is good. Two isn't too bad, either. Having twenty different styles of integration is not good. This isn't just about picking the technology and the protocol. If you pick HTTP/REST, for example, will you use verbs or nouns? How will you handle pagination of resources? How will you handle versioning of endpoints?

Việc chọn một số ít các công nghệ interface đã được định nghĩa giúp tích hợp các bên tiêu thụ mới. Có một chuẩn là tốt. Hai cũng không tệ lắm. Có hai mươi kiểu tích hợp khác nhau thì không tốt. Đây không chỉ là về việc chọn công nghệ và giao thức. Chẳng hạn nếu bạn chọn HTTP/REST, bạn sẽ dùng động từ hay danh từ? Bạn sẽ xử lý phân trang tài nguyên (pagination) như thế nào? Bạn sẽ xử lý việc phiên bản hóa (versioning) các endpoint như thế nào?

Architectural Safety — An toàn kiến trúc

We cannot afford for one badly behaved microservice to ruin the party for everyone. We have to ensure that our microservices shield themselves accordingly from unhealthy, downstream calls. The more microservices we have that do not properly handle the potential failure of downstream calls, the more fragile our systems will be. This might mean, for example, that you want to mandate certain practices around inter-service communication, such as requiring the use of circuit breakers (a topic we explored in “Stability Patterns” on page 395).

Chúng ta không thể chịu nổi việc một microservice hành xử tồi tệ phá hỏng cuộc vui của tất cả mọi người. Chúng ta phải đảm bảo rằng các microservice của mình tự che chắn cho phù hợp khỏi các lời gọi hạ nguồn không lành mạnh. Càng có nhiều microservice không xử lý đúng cách khả năng thất bại tiềm tàng của các lời gọi hạ nguồn, thì các hệ thống của chúng ta sẽ càng mong manh. Chẳng hạn, điều này có thể nghĩa là bạn muốn bắt buộc một số tập quán nhất định quanh việc giao tiếp giữa các dịch vụ, chẳng hạn như yêu cầu sử dụng circuit breaker (một chủ đề chúng ta đã khám phá trong “Stability Patterns” ở trang 395).

Playing by the rules is important when it comes to response codes, too. If your circuit breakers rely on HTTP codes, and one microservice decides to send back 2XX codes for errors or confuses 4XX codes with 5XX codes, then these safety measures can fall apart. Similar concerns would apply even if you're not using HTTP; we need to know the difference between a request that was OK and processed correctly, a request that was bad and thus prevented the microservice from doing anything with it, and a request that might be OK but we can't tell because the server was down. Knowing this is key to ensuring we can fail fast and track down issues. If our microservices play fast and loose with these rules, we end up with a more vulnerable system.

Việc chơi đúng luật cũng quan trọng khi nói đến các mã phản hồi (response code). Nếu các circuit breaker của bạn dựa vào các mã HTTP, và một microservice quyết định gửi trả về các mã 2XX cho lỗi hoặc lẫn lộn các mã 4XX với các mã 5XX, thì các biện pháp an toàn này có thể sụp đổ. Những mối lo tương tự sẽ áp dụng ngay cả khi bạn không dùng HTTP; chúng ta cần biết sự khác biệt giữa một yêu cầu vốn ổn và được xử lý đúng, một yêu cầu vốn tồi và do đó ngăn microservice làm bất cứ điều gì với nó, và một yêu cầu có thể ổn nhưng chúng ta không thể biết được vì máy chủ đã sụp. Biết được điều này là then

chốt để đảm bảo chúng ta có thể thất bại nhanh (fail fast) và truy tìm các vấn đề. Nếu các microservice của chúng ta chơi cầu thả và tùy tiện với những luật này, chúng ta rất cuộc có một hệ thống dễ tổn thương hơn.

Governance and the Paved Road — Governance và con đường đã lát

Part of what architects need to handle is governance. What do I mean by governance ? It turns out the COBIT (Control Objectives for Information Technologies) framework has a pretty good definition: 8

Một phần trong những gì kiến trúc sư cần xử lý là governance. Tôi muốn nói gì khi nói governance? Hóa ra framework COBIT (Control Objectives for Information Technologies) có một định nghĩa khá hay: 8

Governance ensures that enterprise objectives are achieved by evaluating stakeholder needs, conditions and options; setting direction through prioritization and decision making; and monitoring performance, compliance and progress against agreed-on direction and objectives. In a nutshell, we can consider governance as agreeing how things should be done, making sure people know how things should be done, and making sure things are done that way. In some environments, governance just happens informally, as part of normal software development activities. In other environments, especially within larger organizations, this might need to be a more concrete function.

Governance đảm bảo rằng các mục tiêu của doanh nghiệp được đạt tới bằng cách đánh giá nhu cầu, điều kiện và lựa chọn của các bên liên quan; định hướng thông qua việc ưu tiên hóa và ra quyết định; và giám sát hiệu suất, mức tuân thủ và tiến độ so với định hướng và các mục tiêu đã được đồng thuận. Nói gọn lại, chúng ta có thể coi governance là việc đồng thuận về cách mọi thứ nên được làm, đảm bảo mọi người biết cách mọi thứ nên được làm, và đảm bảo mọi thứ được làm theo cách đó. Trong một số môi trường, governance chỉ diễn ra một cách phi chính thức, như một phần của các hoạt động phát triển phần mềm bình thường. Trong các môi trường khác, đặc biệt là trong các tổ chức lớn hơn, điều này có thể cần là một chức năng cụ thể hơn.

Governance can apply to multiple things in the forum of IT. We want to focus on the aspect of technical governance, something I feel is the job of the architect. If one of the architect's jobs is ensuring there is a technical vision, then governance is about

Governance có thể áp dụng cho nhiều thứ trong diễn đàn của IT. Chúng ta muốn tập trung vào khía cạnh governance kỹ thuật, một thứ tôi cảm thấy là công việc của kiến trúc sư. Nếu một trong các công việc của kiến trúc sư là đảm bảo có một tầm nhìn kỹ thuật, thì governance là về

8 COBIT 5: A Business Framework for the Governance and Management of Enterprise IT (Rolling Meadows, IL: ISACA, 2012).

8 COBIT 5: A Business Framework for the Governance and Management of Enterprise IT (Rolling Meadows, IL: ISACA, 2012).

ensuring that what we are building matches this vision, and evolving the vision if needed.

việc đảm bảo rằng những gì chúng ta đang xây dựng khớp với tầm nhìn này, và tiến hóa tầm nhìn nếu cần.

Fundamentally, governance should be a group activity. A properly functioning governance group can work together to share the work and shape the vision. It could be an informal chat with a small enough team, or a more structured regular meeting with formal group membership for a larger scope. This is where I think the principles we covered earlier should be discussed and changed as required. If a formal group is required, this group needs to consist predominantly of people who are executing the work being governed. This group should also be responsible for tracking and managing technical risks.

Về cơ bản, governance nên là một hoạt động tập thể. Một nhóm governance vận hành đúng cách có thể cùng nhau làm việc để chia sẻ công việc và định hình tầm nhìn. Nó có thể là một cuộc trò chuyện phi chính thức với một nhóm đủ nhỏ, hoặc một cuộc họp định kỳ có cấu trúc hơn với tư cách thành viên nhóm chính thức cho một phạm vi lớn hơn. Đây là chỗ tôi nghĩ các nguyên tắc chúng ta đã đề cập ở trước nên được thảo luận và thay đổi khi cần. Nếu cần một nhóm chính thức, nhóm này cần chủ yếu gồm những người đang thực hiện công việc đang được governance. Nhóm này cũng nên chịu trách nhiệm theo dõi và quản lý các rủi ro kỹ thuật.

Getting together and agreeing on how things can be done is a good idea. But spending time making sure people are following these guidelines is less fun, as is placing a burden on developers to implement all these standard things you expect each microservice to do. I am a great believer in making it easy to do the right thing—and as we discussed in Chapter 15, the **paved road** is a really useful concept here. The architect has a role to clearly articulate the vision—where you are going—and to make it easy to get there. As such, they should be involved in helping shape the requirements of whatever paved road you build. For many, the platform will be the biggest example of this—the architect ends up being an important stakeholder for the **platform team**.

Việc tụ họp lại và đồng thuận về cách mọi thứ có thể được làm là một ý tưởng hay. Nhưng việc dành thời gian đảm bảo mọi người đang tuân theo các hướng dẫn này thì kém vui hơn, cũng như việc đặt một gánh nặng lên các lập trình viên buộc họ triển khai tất cả những thứ chuẩn mực mà bạn kỳ vọng mỗi microservice phải làm. Tôi là một người rất tin tưởng vào việc làm cho việc làm điều đúng đắn trở nên dễ dàng—và như chúng ta đã thảo luận ở Chương 15, paved road là một khái niệm thực sự hữu ích ở đây. Kiến trúc sư có một vai trò trong việc diễn đạt rõ ràng tầm nhìn—nơi bạn đang đi tới—và làm cho việc đến được đó trở nên dễ dàng. Như vậy, họ nên tham gia vào việc giúp định hình các yêu cầu của bất kỳ paved road nào bạn xây dựng. Đối với nhiều người, nền tảng sẽ là ví dụ lớn nhất của điều này—kiến trúc sư tốt cuộc trở thành một bên liên quan quan trọng đối với nhóm nền tảng.

We’ve already looked at the role of the platform in some depth, so let’s look at a couple of other techniques we can use to make it as easy as possible for people to do the right thing.

Chúng ta đã xem xét vai trò của nền tảng khá sâu rồi, nên hãy xem một vài kỹ thuật khác mà chúng ta có thể dùng để làm cho việc làm điều đúng đắn trở nên dễ nhất có thể với mọi người.

Exemplars — Mẫu hình tham chiếu

Written documentation is good and useful. I clearly see the value in it; after all, I’ve written this book. But developers also like code—code they can run and explore. If you have a set of standards or best practices you would like to encourage, then having exemplars you can point people to is useful. The idea is that people can’t go far wrong just by imitating some of the better parts of your system.

Tài liệu viết là tốt và hữu ích. Tôi rõ ràng thấy giá trị trong nó; xét cho cùng, tôi đã viết cuốn sách này. Nhưng các lập trình viên cũng thích mã—mã mà họ có thể chạy và khám phá. Nếu bạn có một bộ tiêu chuẩn hay tập quán tốt nhất mà bạn muốn khuyến khích, thì việc có các mẫu hình tham chiếu (exemplar) mà bạn có thể chỉ cho mọi người thì hữu ích. Ý tưởng là người ta không thể đi sai quá xa chỉ bằng cách bắt chước một số phần tốt hơn trong hệ thống của bạn.

Ideally, these should be real-world microservices running in your system that get things right, rather than isolated microservices that are implemented merely as “perfect examples.” By ensuring your exemplars are actually being used, you ensure that all the principles you have actually make sense.

Lý tưởng nhất, đây nên là các microservice thực tế đang chạy trong hệ thống của bạn làm mọi thứ đúng đắn, thay vì các microservice biệt lập được triển khai chỉ như những “ví dụ hoàn hảo.” Bằng cách đảm bảo rằng các mẫu hình tham chiếu của bạn thực sự đang được sử dụng, bạn đảm bảo rằng tất cả các nguyên tắc bạn có thực sự có ý nghĩa.

Tailored Microservice Template — Mẫu microservice may đo

Wouldn't it be great if you could make it really easy for all developers to follow most of the guidelines you have with very little work? What if, out of the box, the developers had most of the code in place to implement the core attributes that each microservice needs?

Sẽ thật tuyệt nếu bạn có thể khiến tất cả lập trình viên dễ dàng tuân theo hầu hết các hướng dẫn bạn có với rất ít công sức? Nếu, ngay khi vừa lấy ra dùng, các lập trình viên đã có sẵn hầu hết mã cần thiết để triển khai các thuộc tính cốt lõi mà mỗi microservice cần thì sao?

Many frameworks exist for different programming languages that attempt to give you the building blocks for your own microservice template. Spring Boot is probably the most successful example of such a framework for the JVM. The core Spring Boot framework is fairly light, but you can then decide to pull together a set of libraries to provide features like checking health, serving HTTP, or exposing metrics. So right out of the box, you have a simple “Hello World” microservice that can be launched from the command line.

Nhiều framework tồn tại cho các ngôn ngữ lập trình khác nhau cố gắng cung cấp cho bạn các khối xây dựng cho mẫu microservice của riêng bạn. Spring Boot có lẽ là ví dụ thành công nhất của một framework như vậy cho JVM. Framework Spring Boot lõi khá nhẹ, nhưng sau đó bạn có thể quyết định gom lại một bộ thư viện để cung cấp các tính năng như kiểm tra sức khỏe, phục vụ HTTP, hay phơi bày các metric. Vậy nên ngay khi vừa lấy ra dùng, bạn có một microservice “Hello World” đơn giản có thể được khởi chạy từ dòng lệnh.

Many people then take these frameworks and standardize this setup for their company. For example, when spinning up a new microservice, they may script things so that they get a Spring Boot template with the core libraries their organization uses already wired in; it might already pull in the libraries to handle circuit breakers and be configured to handle JWT **authentication** for inbound calls. Normally, such an automated template creation would create a matching **build pipeline** as well.

Sau đó nhiều người lấy các framework này và chuẩn hóa thiết lập này cho công ty của họ. Chẳng hạn, khi dựng một microservice mới, họ có thể viết script cho mọi thứ để họ nhận được một mẫu Spring Boot với các thư viện lõi mà tổ chức của họ dùng đã được đấu nối sẵn; nó có thể đã kéo sẵn các thư viện để xử lý circuit breaker và được cấu hình để xử lý

xác thực JWT cho các lời gọi đến. Thông thường, một việc tạo mẫu tự động như vậy cũng sẽ tạo ra một build pipeline tương ứng.

Caution warranted

Cần thận trọng

The selection and configuration of these tailored microservices templates is commonly a task for the platform team. They might, for example, provide a template for each supported language, ensuring that when using the template the resulting microservices work well with the platform itself. This can cause challenges, however.

Việc lựa chọn và cấu hình các mẫu microservice may đo này thường là một tác vụ cho nhóm nền tảng. Chẳng hạn, họ có thể cung cấp một mẫu cho mỗi ngôn ngữ được hỗ trợ, đảm bảo rằng khi dùng mẫu thì các microservice kết quả hoạt động tốt với chính nền tảng. Tuy nhiên, điều này có thể gây ra các thách thức.

I have seen many a team's morale and productivity destroyed by having a mandated framework thrust upon it. In a drive to improve code reuse, more and more work is placed into a centralized framework until it becomes an overwhelming monstrosity. If you decide to use a tailored microservice template, think very carefully about what its job is. Ideally, its use should be purely optional, but if you are going to be more forceful in its adoption, you need to understand that ease of use for the developers has to be a prime guiding force. Allowing the developers who use the template to recommend and even contribute changes to the framework, perhaps as part of an internal open source model, can help greatly here.

Tôi đã thấy tinh thần và năng suất của không ít nhóm bị hủy hoại bởi việc bị áp đặt một framework bắt buộc. Trong nỗ lực cải thiện việc tái sử dụng mã, ngày càng nhiều công việc được dồn vào một framework tập trung cho đến khi nó trở thành một con quái vật áp đảo. Nếu bạn quyết định dùng một mẫu microservice may đo, hãy suy nghĩ thật kỹ về công việc của nó là gì. Lý tưởng nhất, việc sử dụng nó nên là hoàn toàn tùy chọn, nhưng nếu bạn định ép buộc hơn trong việc áp dụng nó, bạn cần hiểu rằng sự dễ sử dụng cho các lập trình viên phải là một lực dẫn dắt hàng đầu. Việc cho phép các lập trình viên dùng mẫu đề xuất và thậm chí đóng góp các thay đổi cho framework, có lẽ như một phần của một mô hình mã nguồn mở nội bộ, có thể giúp ích rất nhiều ở đây.

As we discussed in “DRY and the Perils of Code Reuse in a Microservice World” on page 154 , we have to be aware of the perils of shared code. In our desire to create reusable code, we can introduce sources of coupling between microservices. At least one organization I spoke to is so worried about this that it actually copies its microservice template code manually into each microservice. This means that an upgrade to the core microservice template takes longer to be applied across its system, but this is less concerning to the organization than the danger of coupling. Other teams I've spoken to have simply treated the microservice template as a shared binary dependency, although they have to be very diligent in not letting the tendency for DRY (don't repeat yourself) result in an overly coupled system!

Như chúng ta đã thảo luận trong “DRY and the Perils of Code Reuse in a Microservice World” ở trang 154, chúng ta phải ý thức được các hiểm họa của mã dùng chung. Trong khao khát tạo ra mã tái sử dụng được, chúng ta có thể đưa vào các nguồn coupling giữa các microservice. Ít nhất một tổ chức mà tôi đã trò chuyện lo lắng về điều này đến mức nó thực sự sao chép mã mẫu microservice của mình một cách thủ công vào từng microservice. Điều này nghĩa là một bản nâng cấp cho mẫu microservice lỗi mất nhiều thời gian hơn để được áp dụng trên toàn hệ thống của nó, nhưng điều này ít đáng lo hơn đối với tổ chức so với mối nguy của coupling. Các nhóm khác mà tôi đã trò chuyện chỉ đơn giản coi mẫu

microservice như một phụ thuộc nhị phân (binary dependency) dùng chung, mặc dù họ phải rất cẩn trọng trong việc không để khuynh hướng DRY (don't repeat yourself) dẫn đến một hệ thống bị coupling quá mức!

The Paved Road at Scale — Con đường đã lát ở quy mô lớn

The use of in-house internal microservice templates and frameworks is often found in organizations that have large numbers of microservices. Netflix and Monzo are two such organizations. Each has decided to standardize on its technology stack to some degree (the JVM in the case of Netflix, Go in terms of Monzo), allowing it to speed up the creation of a new microservice with standard, expected behavior by using a common set of tools. With a more divergent technology stack, having a standard microservice template for your own needs becomes more difficult.

Việc sử dụng các mẫu và framework microservice nội bộ tự xây dựng thường được thấy ở các tổ chức có số lượng microservice lớn. Netflix và Monzo là hai tổ chức như vậy. Mỗi tổ chức đã quyết định chuẩn hóa trên stack công nghệ của mình ở một mức độ nào đó (JVM trong trường hợp Netflix, Go trong trường hợp Monzo), cho phép nó tăng tốc việc tạo ra một microservice mới với hành vi chuẩn, được kỳ vọng bằng cách dùng một bộ công cụ chung. Với một stack công nghệ phân kỳ hơn, việc có một mẫu microservice chuẩn cho nhu cầu của riêng bạn trở nên khó hơn.

If you were to embrace multiple disparate technology stacks, you'd need a matching microservice template for each. This could be a way you subtly constrain language choices in your teams, though. If the in-house microservice template supports only the JVM, then people may be discouraged from picking alternative stacks if they have to do lots more work themselves. Netflix, for example, is especially concerned with aspects like fault tolerance to ensure that the outage of one part of its system cannot take everything down. To handle this, a large amount of work has been done to ensure that there are client libraries on the JVM to provide teams with the tools they need to keep their microservice well behaved. Introducing a new technology stack would mean having to reproduce all this effort. The main concern for Netflix is less about the duplicated effort and more about the fact that it is so easy to get this wrong. The risk of a microservice getting newly implemented fault tolerance wrong is high if it could impact more of the system. Netflix mitigates this by using “sidecar services,” which communicate locally with a JVM that is using the appropriate libraries.

Nếu bạn muốn đón nhận nhiều stack công nghệ rời rạc khác nhau, bạn sẽ cần một mẫu microservice tương ứng cho mỗi cái. Tuy nhiên, đây có thể là một cách bạn ngầm ràng buộc các lựa chọn ngôn ngữ trong các nhóm của mình. Nếu mẫu microservice nội bộ chỉ hỗ trợ JVM, thì người ta có thể bị nản lòng khi chọn các stack thay thế nếu họ phải tự làm thêm nhiều việc hơn. Chẳng hạn, Netflix đặc biệt bận tâm về các khía cạnh như khả năng chịu lỗi (fault tolerance) để đảm bảo rằng sự cố của một phần hệ thống của nó không thể kéo sập mọi thứ. Để xử lý điều này, một lượng công việc lớn đã được thực hiện để đảm bảo rằng có các thư viện client trên JVM để cung cấp cho các nhóm những công cụ họ cần nhằm giữ cho microservice của họ hành xử chuẩn mực. Việc đưa vào một stack công nghệ mới sẽ nghĩa là phải tái tạo lại toàn bộ nỗ lực này. Một bận tâm chính của Netflix không phải về công sức bị trùng lặp mà về việc rất dễ làm sai điều này. Rủi ro của việc một microservice triển khai khả năng chịu lỗi mới sai là cao nếu nó có thể tác động đến nhiều phần hệ thống hơn. Netflix giảm thiểu điều này bằng cách dùng “sidecar service,” vốn giao tiếp cục bộ với một JVM đang sử dụng các thư viện thích hợp.

Service meshes have given us another potential way to offload common behavior. Some functionality that was commonly seen as an internal microservice's responsibility can now be pushed to a

microservice mesh. This can ensure more consistency of behavior across microservices written in different programming languages and also reduce the responsibilities of these microservice templates.

Các service mesh đã mang lại cho chúng ta một cách tiềm năng khác để chuyển bớt hành vi chung ra ngoài. Một số chức năng vốn thường được xem là trách nhiệm của một microservice nội bộ giờ có thể được đẩy sang một microservice mesh. Điều này có thể đảm bảo sự nhất quán hơn về hành vi giữa các microservice được viết bằng các ngôn ngữ lập trình khác nhau và cũng giảm bớt các trách nhiệm của các mẫu microservice này.

Technical Debt — Nợ kỹ thuật

We are often put in situations in which we cannot follow through to the letter on our technical vision. Often, we need to make a choice to cut a few corners to get some urgent features out. This is just one more trade-off that we'll find ourselves having to make. Our technical vision exists for a reason. If we deviate from this reason, it might have a short-term benefit but a long-term cost. A concept that helps us understand this trade-off is technical debt. When we accrue technical debt, just like debt in the real world it has an ongoing cost and is something we want to pay down.

Chúng ta thường bị đặt vào những tình huống trong đó ta không thể theo đúng từng câu chữ tầm nhìn kỹ thuật của mình. Thường thì chúng ta cần lựa chọn cắt bớt vài góc để đưa ra một số tính năng cấp bách. Đây chỉ là thêm một sự đánh đổi nữa mà chúng ta sẽ thấy mình phải thực hiện. Tầm nhìn kỹ thuật của chúng ta tồn tại vì một lý do. Nếu chúng ta đi chệch khỏi lý do này, nó có thể có lợi ích ngắn hạn nhưng cái giá dài hạn. Một khái niệm giúp chúng ta hiểu sự đánh đổi này là nợ kỹ thuật (technical debt). Khi chúng ta tích lũy nợ kỹ thuật, cũng như nợ trong thế giới thực, nó có một cái giá tiếp diễn và là thứ chúng ta muốn trả dần.

Sometimes technical debt isn't just something we cause by taking shortcuts. What happens if our vision for the system changes, but not all of our system matches? In this situation, too, we have created new sources of technical debt.

Đôi khi nợ kỹ thuật không chỉ là thứ chúng ta gây ra bằng cách đi đường tắt. Điều gì xảy ra nếu tầm nhìn của chúng ta về hệ thống thay đổi, nhưng không phải toàn bộ hệ thống của chúng ta khớp với nó? Trong tình huống này, chúng ta cũng đã tạo ra các nguồn nợ kỹ thuật mới.

The architect's job is to look at the bigger picture and understand this balance. Having some view as to the level of debt and where to get involved is important. Depending on your organization, you might be able to provide gentle guidance, but have the teams themselves decide how to track and pay down the debt. For other organizations, you may need to be more structured, perhaps maintaining a debt log that is reviewed regularly.

Công việc của kiến trúc sư là nhìn vào bức tranh lớn hơn và hiểu sự cân bằng này. Việc có một góc nhìn nào đó về mức độ nợ và nơi cần can dự vào là quan trọng. Tùy thuộc vào tổ chức của bạn, bạn có thể đưa ra hướng dẫn nhẹ nhàng, nhưng để chính các nhóm tự quyết định cách theo dõi và trả dần nợ. Đối với các tổ chức khác, bạn có thể cần có cấu trúc hơn, có lẽ duy trì một nhật ký nợ (debt log) được rà soát định kỳ.

Exception Handling — Xử lý ngoại lệ

So our principles and practices guide how our systems should be built. But what happens when our system deviates from that? Sometimes we make a decision that is just an exception to the rule. In these cases, it might be worth capturing such a decision in a log somewhere for future reference. If enough exceptions are found, it may eventually make sense to change the applicable principle or practice to reflect a new understanding of the world. For example, we might have a practice that states that we will always use MySQL for data storage. But then we see compelling reasons to use Cassandra for highly scalable storage, at which point we change our practice to say, “Use MySQL for most storage requirements, unless you expect large growth in volumes, in which case use Cassandra.”

Vậy các nguyên tắc và tập quán của chúng ta dẫn dắt cách các hệ thống của chúng ta nên được xây dựng. Nhưng điều gì xảy ra khi hệ thống của chúng ta đi chệch khỏi điều đó? Đôi khi chúng ta đưa ra một quyết định chỉ là một ngoại lệ so với quy tắc. Trong các trường hợp này, có thể đáng để ghi lại một quyết định như vậy trong một nhật ký nào đó để tham chiếu trong tương lai. Nếu tìm thấy đủ nhiều ngoại lệ, rốt cuộc có thể hợp lý khi thay đổi nguyên tắc hay tập quán liên quan để phản ánh một hiểu biết mới về thế giới. Chẳng hạn, chúng ta có thể có một tập quán nói rằng chúng ta sẽ luôn dùng MySQL cho việc lưu trữ dữ liệu. Nhưng rồi chúng ta thấy những lý do thuyết phục để dùng Cassandra cho việc lưu trữ có khả năng mở rộng cao, đến lúc đó chúng ta thay đổi tập quán của mình thành, “Dùng MySQL cho hầu hết các nhu cầu lưu trữ, trừ khi bạn dự kiến khối lượng sẽ tăng lớn, trong trường hợp đó hãy dùng Cassandra.”

It’s worth reiterating, though, that every organization is different. I’ve worked with some companies at which the development teams have a high degree of trust and autonomy, and the principles are lightweight (and the need for overt exception handling is greatly reduced, if not eliminated). In more structured organizations in which developers have less freedom, tracking exceptions may be vital to ensuring that the rules in place properly reflect the challenges people are facing. With all that said, I am a fan of microservices as a way of optimizing for autonomy of teams, giving them as much freedom as possible to solve the problem at hand. If you are working in an organization that places lots of restrictions on how developers can do their work, then microservices may not be for you.

Tuy nhiên, đáng nhắc lại rằng mỗi tổ chức đều khác nhau. Tôi đã làm việc với một số công ty mà tại đó các nhóm phát triển có mức độ tin tưởng và tự trị cao, và các nguyên tắc thì nhẹ nhàng (và nhu cầu xử lý ngoại lệ công khai giảm đi rất nhiều, nếu không muốn nói là bị loại bỏ). Trong các tổ chức có cấu trúc hơn nơi các lập trình viên có ít tự do hơn, việc theo dõi các ngoại lệ có thể là tối quan trọng để đảm bảo rằng các quy tắc hiện hành phản ánh đúng những thách thức mà mọi người đang đối mặt. Với tất cả những điều đã nói, tôi là một người hâm mộ microservices như một cách tối ưu cho tính tự trị của các nhóm, trao cho họ càng nhiều tự do càng tốt để giải quyết vấn đề trước mắt. Nếu bạn đang làm việc trong một tổ chức đặt ra rất nhiều hạn chế về cách các lập trình viên có thể làm công việc của họ, thì microservices có lẽ không dành cho bạn.

Summary — Tóm tắt

To summarize this chapter, here are what I see as the core responsibilities of the **evolutionary architect**:

Để tóm tắt chương này, đây là những gì tôi thấy là các trách nhiệm cốt lõi của kiến trúc sư tiến hóa:

Vision Ensure there is a clearly communicated technical vision for the system that will help it meet the requirements of your customers and organization.

Tầm nhìn Đảm bảo có một tầm nhìn kỹ thuật được truyền đạt rõ ràng cho hệ thống mà sẽ giúp nó đáp ứng các yêu cầu của khách hàng và tổ chức của bạn.

Empathy Understand the impact of your decisions on your customers and colleagues.

Đồng cảm Hiểu tác động của các quyết định của bạn đối với khách hàng và đồng nghiệp của bạn.

Afterword: Bringing It All Together

— Lời kết: Kết nối tất cả lại với nhau

This book has covered a lot of ground, and I've shared a lot of advice along the way. Given the breadth of coverage, I thought it sensible to summarize some of my key advice regarding **microservice** architectures. For those of you who have read the whole book, this should be a great refresher. For those of you who are impatient and jumped to the end, be aware that there is a lot of detail behind this advice, and I'd urge you read up on the detail behind some of these ideas rather than just adopting these ideas blindly.

Cuốn sách này đã đề cập đến rất nhiều nội dung, và tôi đã chia sẻ rất nhiều lời khuyên trên hành trình đó. Với phạm vi bao quát rộng như vậy, tôi nghĩ sẽ hợp lý khi tóm tắt lại một số lời khuyên cốt lõi của mình về kiến trúc microservice. Với những bạn đã đọc trọn cuốn sách, đây sẽ là một phần ôn tập tuyệt vời. Còn với những bạn nóng vội nhảy thẳng đến cuối, hãy lưu ý rằng đằng sau những lời khuyên này là rất nhiều chi tiết, và tôi tha thiết khuyên bạn nên đọc kỹ phần chi tiết đằng sau một số ý tưởng này thay vì áp dụng chúng một cách mù quáng.

With all that said, I'm aiming to keep this last chapter as brief as possible, so let's get started.

Với tất cả những điều đã nói, tôi đặt mục tiêu giữ cho chương cuối này càng ngắn gọn càng tốt, vậy nên hãy bắt đầu thôi.

What Are Microservices? — Microservice là gì?

As introduced in Chapter 1, **microservices** are a type of **service-oriented architecture** that focuses on **independent deployability**. Independent deployability means that you can make a change to a microservice, **deploy** that microservice, and **release** its functionality to the end users without requiring other microservices to change. Getting the most out of a microservice architecture means embracing this concept. Normally, each microservice is deployed as a process, with communication with other microservices being done over some form of network protocol. It's common to deploy multiple instances of a microservice, perhaps so that you can provide more scale, or else to improve **robustness** by having redundancy.

Như đã giới thiệu ở Chương 1, microservice là một dạng kiến trúc hướng dịch vụ (service-oriented architecture) tập trung vào khả năng triển khai độc lập (independent deployability). Khả năng triển khai độc lập nghĩa là bạn có thể thực hiện một thay đổi trên một microservice, triển khai microservice đó, và phát hành (release) chức năng của nó đến người dùng cuối mà không đòi hỏi các microservice khác phải thay đổi theo. Tận dụng tối đa một kiến trúc microservice nghĩa là đón nhận trọn vẹn khái niệm này. Thông thường, mỗi microservice được triển khai dưới dạng một tiến trình, với việc giao tiếp với

các microservice khác được thực hiện qua một dạng giao thức mạng nào đó. Việc triển khai nhiều phiên bản (instance) của một microservice là chuyện phổ biến, có thể để cung cấp khả năng mở rộng (scale) lớn hơn, hoặc để cải thiện độ vững chắc (robustness) thông qua dự thừa (redundancy).

To deliver independent deployability, we need to make sure when changing one microservice that we don't break interactions with other microservices. This requires that our interfaces with other microservices are stable, and that changes be made in a backward-compatible way. **Information hiding**, which I expanded on in Chapter 2, describes an approach in which as much information as possible (code, data) is hidden behind an **interface**. You should expose only the bare minimum over your service interfaces to satisfy your consumers. The less you expose, the easier it is to ensure the changes you make are going to be backward compatible. Information hiding also allows us to make technological changes within a microservice boundary in a way that won't impact consumers.

Để mang lại khả năng triển khai độc lập, chúng ta cần đảm bảo rằng khi thay đổi một microservice, ta không phá vỡ các tương tác với những microservice khác. Điều này đòi hỏi các interface của chúng ta với những microservice khác phải ổn định, và các thay đổi phải được thực hiện theo cách tương thích ngược (backward-compatible). Che giấu thông tin (information hiding), điều mà tôi đã mở rộng ở Chương 2, mô tả một cách tiếp cận trong đó càng nhiều thông tin (mã, dữ liệu) càng tốt được giấu sau một interface. Bạn chỉ nên phơi bày tối thiểu những gì cần thiết qua các interface dịch vụ để đáp ứng những consumer của mình. Càng phơi bày ít, càng dễ đảm bảo rằng những thay đổi bạn thực hiện sẽ tương thích ngược. Che giấu thông tin cũng cho phép chúng ta thực hiện các thay đổi công nghệ bên trong ranh giới của một microservice theo cách không ảnh hưởng đến consumer.

One of the key ways we implement independent deployability is by hiding the **database**. If a microservice needs to store state in a database, this should be entirely hidden from the outside world. Internal databases should not be directly exposed to external consumers, as this causes too much **coupling** between the two, which undermines independent deployability. In general, avoid situations in which multiple microservices all access the same database.

Một trong những cách then chốt để chúng ta hiện thực khả năng triển khai độc lập là che giấu cơ sở dữ liệu (database). Nếu một microservice cần lưu trạng thái trong một cơ sở dữ liệu, điều này nên được giấu hoàn toàn khỏi thế giới bên ngoài. Các cơ sở dữ liệu nội bộ không nên bị phơi bày trực tiếp cho các consumer bên ngoài, vì điều này gây ra quá nhiều coupling giữa hai bên, làm xói mòn khả năng triển khai độc lập. Nhìn chung, hãy tránh những tình huống trong đó nhiều microservice cùng truy cập một cơ sở dữ liệu giống nhau.

Microservices work very well with **domain-driven design** (DDD). DDD gives us concepts that help us find our microservice boundaries, with the resulting architecture being aligned around the **business domain**. This is extremely helpful in situations where organizations are creating more business-centric IT teams. With a team focused on one part of the business domain, it can now take **ownership** of the microservices that match this part of the business.

Microservice hoạt động rất tốt với domain-driven design (DDD). DDD mang lại cho chúng ta những khái niệm giúp tìm ra ranh giới microservice, với kiến trúc thu được được căn chỉnh theo miền nghiệp vụ (business domain). Điều này cực kỳ hữu ích trong những tình huống mà các tổ chức đang xây dựng những đội ngũ IT lấy nghiệp vụ làm trung tâm hơn. Với một đội tập trung vào một phần của miền nghiệp vụ, giờ đây đội đó có thể nhận quyền sở hữu (ownership) các microservice tương ứng với phần nghiệp vụ này.

Moving to Microservices — Chuyển sang Microservice

Microservices bring a lot of complexity—enough complexity that the reasons for using them need to be seriously considered. I remain convinced that a simple single-process **monolith** is a totally sensible starting point for a new system. Over time, though, we learn things, and we start to see the ways in which our current **system architecture** is no longer fit for purpose. At that point, looking to change is appropriate.

Microservice mang lại rất nhiều phức tạp—đủ phức tạp đến mức cần cân nhắc nghiêm túc các lý do để sử dụng chúng. Tôi vẫn tin chắc rằng một monolith đơn tiến trình đơn giản là điểm khởi đầu hoàn toàn hợp lý cho một hệ thống mới. Tuy nhiên, theo thời gian, chúng ta học được nhiều điều, và bắt đầu nhận ra những cách mà kiến trúc hệ thống hiện tại của mình không còn phù hợp với mục đích nữa. Đến lúc đó, tìm đến sự thay đổi là điều thích hợp.

It is important to understand what you are trying to get out of a microservice architecture. What is the goal? What positive outcome do you expect a shift to microservices to bring? The outcome you are aiming for will directly impact how you break your monolith apart. If you are trying to change your system architecture to better handle scale, you'll end up making different changes than if your main driver is to improve organizational autonomy. I cover this more in Chapter 3, and in even more detail in my book *Monolith to Microservices*.

Điều quan trọng là phải hiểu bạn đang cố đạt được gì từ một kiến trúc microservice. Mục tiêu là gì? Bạn kỳ vọng việc chuyển sang microservice sẽ mang lại kết quả tích cực nào? Kết quả bạn nhắm tới sẽ tác động trực tiếp đến cách bạn tách monolith ra. Nếu bạn đang cố thay đổi kiến trúc hệ thống để xử lý khả năng mở rộng tốt hơn, bạn sẽ thực hiện những thay đổi khác so với khi động lực chính của bạn là cải thiện quyền tự chủ của tổ chức. Tôi đề cập đến điều này nhiều hơn ở Chương 3, và thậm chí chi tiết hơn nữa trong cuốn sách *Monolith to Microservices* của mình.

Many of the problems with microservices are evident only after you hit production. Therefore, I strongly recommend an incremental, evolutionary decomposition of an existing monolith rather than a “big bang” rewrite. Identify a microservice you want to create, extract the appropriate functionality from the monolith, deploy the new microservice into production, and start using it in anger. Based on that, you'll see if you are helping move toward your goal, but you'll also learn a lot that will make the next microservice extraction easier—or perhaps it will suggest to you that microservices might not be the way forward after all!

Nhiều vấn đề với microservice chỉ lộ rõ sau khi bạn đưa vào production. Vì vậy, tôi mạnh mẽ khuyến nghị phân rã một monolith hiện hữu theo hướng tăng dần, tiến hóa, thay vì viết lại theo kiểu “big bang”. Hãy xác định một microservice bạn muốn tạo, trích xuất chức năng phù hợp ra khỏi monolith, triển khai microservice mới vào production, và bắt đầu thực sự sử dụng nó. Dựa trên đó, bạn sẽ thấy liệu mình có đang tiến gần đến mục tiêu hay không, nhưng bạn cũng sẽ học được rất nhiều điều giúp lần trích xuất microservice tiếp theo dễ dàng hơn—hoặc có lẽ nó sẽ gợi ý cho bạn rằng microservice rốt cuộc có thể không phải là hướng đi phù hợp!

Communication Styles — Các Phong cách Giao tiếp

We summarized the main forms of inter-microservice communication in Chapter 4, shared again in Figure A-1. This isn't meant to be a universal model but is intended to just give an overview of the different types of communication that are most common.

Chúng ta đã tóm tắt các hình thức chính của giao tiếp giữa các microservice ở Chương 4, được chia sẻ lại trong Hình A-1. Đây không phải là một mô hình phổ quát mà chỉ nhằm đưa ra một cái nhìn tổng quan về các kiểu giao tiếp phổ biến nhất.

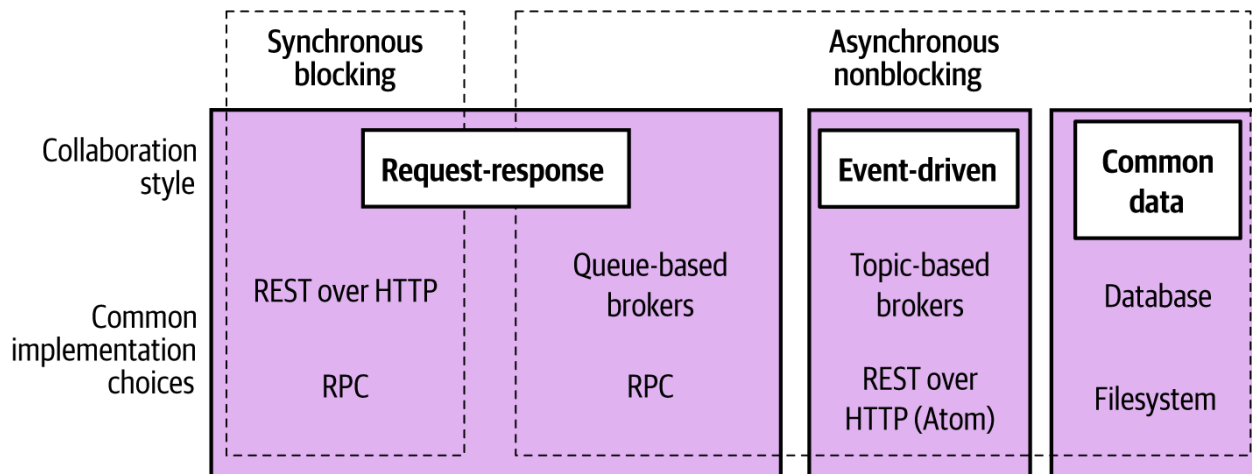


Figure A-1. Different styles of inter-microservice communication along with example implementing technologies

Hình A-1. Các phong cách giao tiếp giữa các microservice khác nhau cùng với ví dụ về công nghệ hiện thực

With **request-response** communication, a microservice sends a request to a downstream microservice and expects a response. With **synchronous** request-response, we would expect the response to come back to the microservice instance that sent the request. With **asynchronous** request-response, it's possible for the response to come back to a different instance of the upstream microservices.

Với giao tiếp request-response, một microservice gửi một yêu cầu đến một microservice phía dưới (downstream) và mong đợi một phản hồi. Với request-response đồng bộ (synchronous), chúng ta sẽ kỳ vọng phản hồi quay về đúng phiên bản microservice đã gửi yêu cầu. Với request-response bất đồng bộ (asynchronous), phản hồi có thể quay về một phiên bản khác của các microservice phía trên (upstream).

With **event-driven** communication, a microservice emits an event, and other microservices, if they are interested in that event, can react to it. Events are just statements of fact—information that is shared about something that has happened. With event-driven communication, a microservice doesn't tell another microservice what to do; it just shares events. It's up to downstream microservices to make a judgment call as to what they do with that information. Event-driven communication is by definition asynchronous in nature.

Với giao tiếp hướng sự kiện (event-driven), một microservice phát ra một sự kiện, và các microservice khác, nếu quan tâm đến sự kiện đó, có thể phản ứng với nó. Sự kiện chỉ đơn thuần là những phát biểu về sự thật—thông tin được chia sẻ về một điều gì đó đã xảy ra. Với giao tiếp hướng sự kiện, một microservice không bảo một microservice khác phải làm gì; nó chỉ chia sẻ các sự kiện. Việc quyết định làm gì với thông tin đó là tùy thuộc vào các microservice phía dưới. Giao tiếp hướng sự kiện về bản chất là bất đồng bộ.

One microservice may communicate over more than one protocol. For example, in Figure A-2 we see a Shipping microservice providing a **REST** interface for request-response interaction, which also fires events when changes are made.

Một microservice có thể giao tiếp qua nhiều giao thức. Ví dụ, trong Hình A-2 chúng ta thấy một microservice Shipping cung cấp một interface REST cho tương tác request-response, đồng thời cũng phát ra các sự kiện khi có thay đổi.

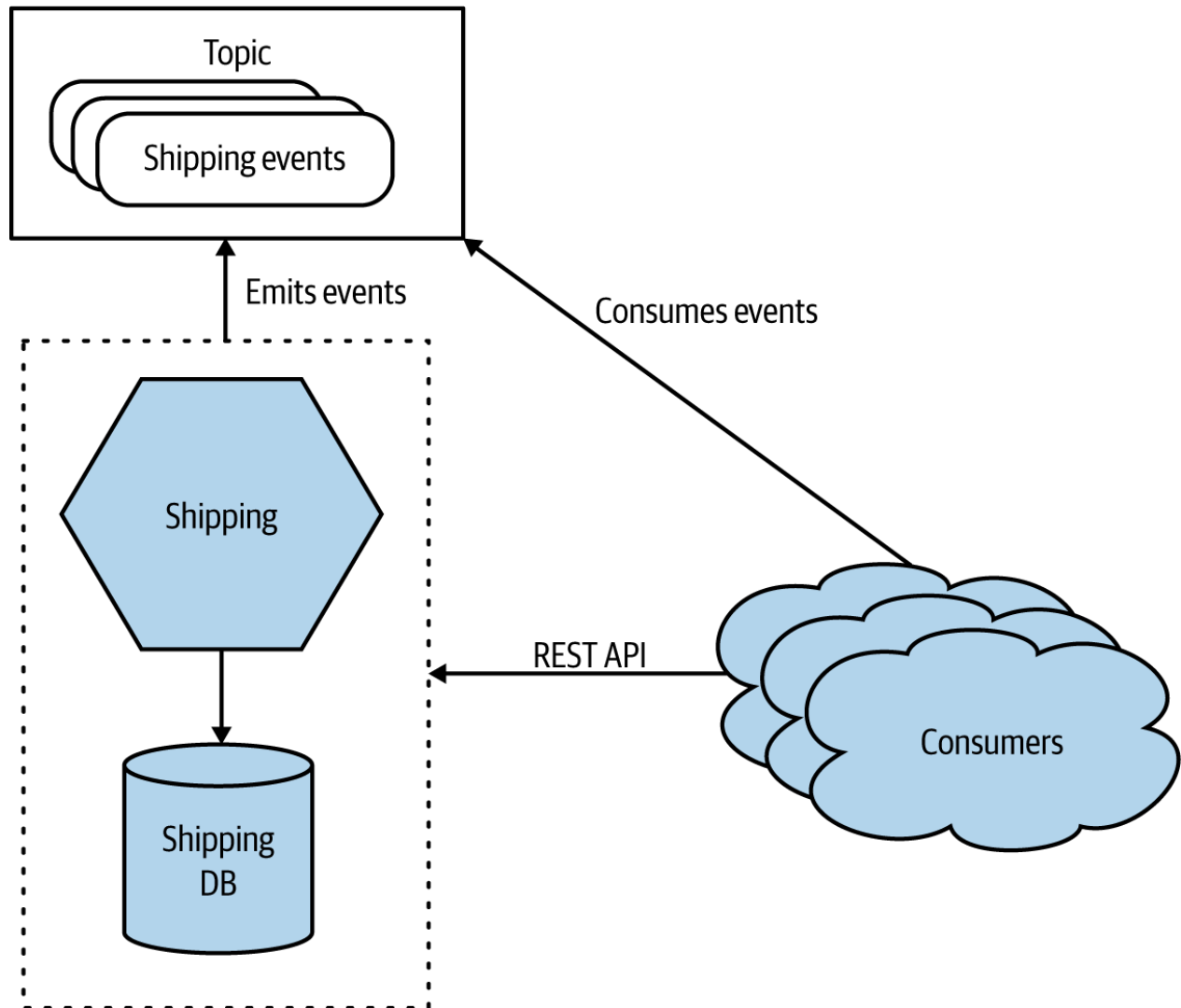


Figure A-2. A microservice exposing its functionality over a REST **API** and a topic

Hình A-2. Một microservice phơi bày chức năng của mình qua một REST API và một topic

Event-driven collaboration can make it easier to **build** more loosely coupled architectures, but it can require more work to understand how the system is behaving. This type of communication also often requires the use of specialist technology such as message brokers, which can further complicate matters. If you can use a fully managed **message broker**, that can help lower the cost of these types of systems.

Cộng tác hướng sự kiện có thể giúp dễ dàng hơn trong việc xây dựng các kiến trúc với coupling lỏng hơn, nhưng nó có thể đòi hỏi nhiều công sức hơn để hiểu hệ thống đang hành xử ra sao. Kiểu giao tiếp này cũng thường đòi hỏi sử dụng công nghệ chuyên dụng như message broker, vốn có thể khiến mọi thứ phức tạp hơn nữa. Nếu bạn có thể dùng một message broker được quản lý hoàn toàn (fully managed), điều đó có thể giúp giảm chi phí cho những kiểu hệ thống này.

Request-response and event-driven interaction models both have their place, and often which one you use will be a personal preference. Some problems just fit one model more than another, and it's common for a microservice architecture to have a mix of styles.

Cả mô hình tương tác request-response lẫn hướng sự kiện đều có chỗ đứng riêng, và thường thì việc bạn dùng cái nào sẽ là sở thích cá nhân. Một số bài toán đơn giản là hợp với mô hình này hơn mô hình kia, và việc một kiến trúc microservice có sự pha trộn các phong cách là chuyện phổ biến.

Workflow — Workflow

When looking to get multiple microservices collaborating to perform some overarching operation, look to explicitly model the process using sagas, a topic we explored in Chapter 6.

Khi muốn nhiều microservice cộng tác để thực hiện một thao tác bao trùm nào đó, hãy tìm cách mô hình hóa quy trình một cách tường minh bằng cách dùng saga, một chủ đề mà chúng ta đã khám phá ở Chương 6.

In general, distributed transactions should be avoided in situations where you can use a **saga** instead. Distributed transactions add significant complexity to systems, have problematic failure modes, and often don't deliver what you expect even when they work. Sagas are in virtually all cases a better fit for implementing business processes that span multiple microservices.

Nhìn chung, nên tránh các giao dịch phân tán (distributed transaction) trong những tình huống mà bạn có thể dùng saga thay thế. Giao dịch phân tán làm tăng đáng kể độ phức tạp cho hệ thống, có những chế độ thất bại đầy rắc rối, và thường không mang lại điều bạn mong đợi ngay cả khi chúng hoạt động. Trong hầu hết mọi trường hợp, saga là lựa chọn phù hợp hơn để hiện thực các quy trình nghiệp vụ trải dài qua nhiều microservice.

There are two different styles of sagas to consider: orchestrated sagas and choreographed sagas. Orchestrated sagas use a centralized orchestrator to coordinate with other microservices and ensure that things are done. In general, this is a simple and straightforward approach, but the central orchestrator can end up doing too much if you aren't careful, and it can become a source of contention when multiple teams are working on the same business process. With choreographed sagas, there is no central coordinator; instead, the responsibility for the business process is distributed into a number of collaborating microservices. This can be a more complex architecture to implement, and it requires more work to ensure that the right things are happening, but on the flip side it is much less prone to coupling and works well for multiple teams.

Có hai phong cách saga khác nhau cần cân nhắc: saga điều phối tập trung (orchestrated saga) và saga điều phối phân tán (choreographed saga). Saga điều phối tập trung dùng một bộ điều phối (orchestrator) trung tâm để phối hợp với các microservice khác và đảm bảo mọi việc được hoàn thành. Nhìn chung, đây là một cách tiếp cận đơn giản và trực diện, nhưng bộ điều phối trung tâm có thể rất cuộc làm quá nhiều việc nếu bạn không cẩn thận, và nó có thể trở thành nguồn tranh chấp khi nhiều đội cùng làm việc trên cùng một quy trình nghiệp vụ. Với saga điều phối phân tán, không có bộ điều phối trung tâm; thay vào đó, trách nhiệm với quy trình nghiệp vụ được phân tán vào một số microservice cộng tác với nhau. Đây có thể là một kiến trúc phức tạp hơn để hiện thực, và đòi hỏi nhiều công sức hơn để đảm bảo những điều đúng đắn đang diễn ra, nhưng đổi lại nó ít bị coupling hơn nhiều và hoạt động tốt với nhiều đội.

Personally, I love choreographed sagas, but then I've used them a lot and made a lot of mistakes implementing them. My general advice is that orchestrated sagas work fine when a single team

owns responsibility for the whole process, but they become more problematic with multiple teams. Choreographed sagas can justify their increased complexity in situations in which multiple teams are expected to collaborate on a process.

Cá nhân tôi yêu thích saga điều phối phân tán, nhưng đó là vì tôi đã dùng chúng rất nhiều và cũng mắc rất nhiều sai lầm khi hiện thực chúng. Lời khuyên chung của tôi là saga điều phối tập trung hoạt động tốt khi một đội duy nhất nắm trách nhiệm cho toàn bộ quy trình, nhưng chúng trở nên rắc rối hơn với nhiều đội. Saga điều phối phân tán có thể biến mình cho độ phức tạp tăng thêm của chúng trong những tình huống mà nhiều đội được kỳ vọng cộng tác trên một quy trình.

Build — Build

Each microservice should have its own build, its own CI pipeline. When I make a change to a microservice, I expect to be able to build that microservice by itself. Avoid situations in which you have to build all your microservices together, as this makes independent deployability much harder.

Mỗi microservice nên có build riêng, có CI pipeline riêng. Khi tôi thực hiện một thay đổi trên một microservice, tôi mong đợi có thể build chỉ riêng microservice đó. Hãy tránh những tình huống mà bạn phải build tất cả các microservice cùng nhau, vì điều này khiến khả năng triển khai độc lập khó hơn nhiều.

For reasons outlined in Chapter 7, I am not a fan of monorepos. If you really want to use them, then please understand the challenges they cause around clear lines of ownership and potential complexity of builds. But definitely make sure that, whether you use a **monorepo** or a **multirepo** approach, each microservice has its own CI build process that can be triggered independently of any other builds.

Vì những lý do đã nêu ở Chương 7, tôi không phải là người hâm mộ monorepo. Nếu bạn thực sự muốn dùng chúng, thì xin hãy hiểu những thách thức mà chúng gây ra xoay quanh việc phân định rõ ràng quyền sở hữu và độ phức tạp tiềm tàng của các build. Nhưng dứt khoát phải đảm bảo rằng, dù bạn dùng cách tiếp cận monorepo hay multirepo, mỗi microservice đều có quy trình CI build riêng có thể được kích hoạt độc lập với bất kỳ build nào khác.

Deployment — Triển khai

Microservices are normally deployed as a process. This process can be deployed onto a physical machine, a virtual machine, a **container**, or an FaaS platform. Ideally, we want microservices to be as isolated from one another as possible in a deployed environment. We don't want a situation in which one microservice using up lots of computing resources can impact a different microservice. In general, this means we want to have each microservice using its own ring-fenced operating system and set of computing resources. Containers are especially effective at giving each microservice instance its own ring-fenced set of resources, making them a great choice for microservice deployments.

Microservice thường được triển khai dưới dạng một tiến trình. Tiến trình này có thể được triển khai lên một máy vật lý, một máy ảo, một container, hoặc một nền tảng FaaS. Lý tưởng nhất là chúng ta muốn các microservice được cô lập với nhau càng nhiều càng tốt trong một môi trường đã triển khai. Chúng ta không muốn tình huống mà một microservice tốn nhiều tài nguyên tính toán có thể ảnh hưởng đến một microservice khác. Nhìn chung,

điều này nghĩa là chúng ta muốn mỗi microservice dùng hệ điều hành và bộ tài nguyên tính toán được khoanh vùng riêng của nó. Container đặc biệt hiệu quả trong việc cấp cho mỗi phiên bản microservice một bộ tài nguyên được khoanh vùng riêng, khiến chúng trở thành lựa chọn tuyệt vời cho việc triển khai microservice.

Kubernetes can be very useful if you are looking to run container workloads across multiple machines. It's not something I'd recommend for just a few microservices, as it brings with it its own sources of complexity. Where possible, use a managed Kubernetes cluster, as this allows you to avoid some of this complexity.

Kubernetes có thể rất hữu ích nếu bạn muốn chạy các tải công việc container trên nhiều máy. Đây không phải thứ tôi khuyên dùng chỉ cho một vài microservice, vì nó mang theo những nguồn phức tạp riêng. Khi có thể, hãy dùng một cụm Kubernetes được quản lý (managed), vì điều này cho phép bạn tránh được một phần độ phức tạp này.

FaaS is an interesting emerging pattern in deploying code. Rather than having to specify how many copies of something you want, you just give your code to the FaaS platform and say, “When this happens, run this code.” This is really nice from a developer point of view, and I think an abstraction like this is likely the future for a large amount of server-side development. The current implementations aren't without problems, though. In terms of microservices, deploying a whole microservice as a single “function” on a FaaS platform is a totally fine way to start.

FaaS là một mẫu hình đang nổi lên thú vị trong việc triển khai mã. Thay vì phải chỉ định bạn muốn bao nhiêu bản sao của một thứ gì đó, bạn chỉ cần đưa mã của mình cho nền tảng FaaS và nói: “Khi điều này xảy ra, hãy chạy đoạn mã này.” Điều này thực sự rất hay từ góc nhìn của lập trình viên, và tôi nghĩ một mức trừu tượng như thế này nhiều khả năng là tương lai cho một lượng lớn việc phát triển phía máy chủ. Tuy nhiên, các hiện thực hiện tại không phải không có vấn đề. Về mặt microservice, việc triển khai cả một microservice dưới dạng một “hàm” (function) duy nhất trên một nền tảng FaaS là một cách khởi đầu hoàn toàn ổn.

One final note: separate in your mind the concepts of **deployment** and release. Just because you've deployed something to production doesn't mean it has to be released to your users. By separating these concepts, you open up the opportunity to roll out your software in different ways—for example, by using canary releases or parallel runs. All of this and more is covered in depth in Chapter 8 .

Một lưu ý cuối: hãy tách bạch trong đầu bạn các khái niệm triển khai (deployment) và phát hành (release). Việc bạn đã triển khai một thứ gì đó vào production không có nghĩa là nó phải được phát hành cho người dùng của bạn. Bằng cách tách bạch các khái niệm này, bạn mở ra cơ hội tung phần mềm ra theo những cách khác nhau—ví dụ, bằng cách dùng canary release hoặc parallel run. Tất cả những điều này và nhiều hơn nữa được đề cập chi tiết ở Chương 8.

Testing — Kiểm thử

It makes a lot of sense to have a suite of automated functional tests to give you fast feedback on the quality of your software before the users see it, and this is absolutely something you should do. Microservices give you a lot of options in terms of the different types of tests you can write, as we explored in Chapter 9 .

Việc có một bộ kiểm thử (testing) chức năng tự động để cung cấp phản hồi nhanh về chất lượng phần mềm trước khi người dùng nhìn thấy là điều rất hợp lý, và đây tuyệt đối là

điều bạn nên làm. Microservice cho bạn rất nhiều lựa chọn về các kiểu kiểm thử khác nhau mà bạn có thể viết, như chúng ta đã khám phá ở Chương 9.

When compared to other types of architectures, though, end-to-end tests can be especially problematic for microservice architectures. They can end up being more expensive to write and maintain for microservice architectures than for simpler nondistributed architectures, and the tests themselves can end up having a lot more failures that don't necessarily point to a problem with your code. End-to-end tests that span multiple teams are particularly challenging.

Tuy nhiên, khi so với các kiểu kiến trúc khác, end-to-end test có thể đặc biệt rắc rối với kiến trúc microservice. Chúng có thể trở nên tốn kém hơn để viết và bảo trì cho kiến trúc microservice so với các kiến trúc không phân tán đơn giản hơn, và bản thân các test có thể rất cuộc có nhiều thất bại hơn mà không nhất thiết chỉ ra vấn đề trong mã của bạn. Các end-to-end test trải dài qua nhiều đội đặc biệt đầy thách thức.

Over time, look to reduce your reliance on end-to-end tests; consider replacing some of the effort put into this form of **testing** with consumer-driven contracts, schema compatibility checking, and testing in production. These activities can deliver much more effectively than end-to-end tests on quickly catching issues on more distributed systems.

Theo thời gian, hãy tìm cách giảm sự phụ thuộc vào end-to-end test; cân nhắc thay thế một phần công sức bỏ ra cho hình thức kiểm thử này bằng consumer-driven contract, kiểm tra tương thích schema, và kiểm thử trong production. Những hoạt động này có thể mang lại hiệu quả hơn nhiều so với end-to-end test trong việc nhanh chóng bắt được các vấn đề trên các hệ thống phân tán hơn.

Monitoring and Observability — Giám sát và Khả năng quan sát (Monitoring and Observability)

In Chapter 10, I explained how **monitoring** is an activity, something we do to a system, but that focusing on an activity rather than an outcome is problematic, a thread that has run through this book. Instead, we should focus on the **observability** of our systems. Observability is the extent to which we can understand what a system is doing by examining external outputs. Making a system that has good observability requires that we build this thinking into our software and ensure that the right types of external outputs are available.

Ở Chương 10, tôi đã giải thích rằng monitoring là một hoạt động, một việc chúng ta làm với hệ thống, nhưng việc tập trung vào một hoạt động thay vì một kết quả là điều rắc rối, một mạch tư tưởng xuyên suốt cuốn sách này. Thay vào đó, chúng ta nên tập trung vào observability của hệ thống. Observability là mức độ mà chúng ta có thể hiểu một hệ thống đang làm gì bằng cách xem xét các đầu ra bên ngoài. Tạo ra một hệ thống có observability tốt đòi hỏi chúng ta xây dựng tư duy này vào trong phần mềm và đảm bảo rằng những kiểu đầu ra bên ngoài đúng đắn luôn sẵn có.

Distributed systems can fail in strange ways, and microservices are no different. We cannot predict all causes of system failure, so it can be hard to know what information we'll need ahead of time to diagnose and fix issues. Using tooling that can help you interrogate these external outputs in ways that you cannot expect becomes increasingly important. I suggest that you look at tools like Lightstep and Honeycomb that were built with this thinking in mind.

Các hệ thống phân tán có thể thất bại theo những cách lạ lùng, và microservice cũng không ngoại lệ. Chúng ta không thể dự đoán mọi nguyên nhân gây thất bại hệ thống, nên có thể khó biết trước ta sẽ cần thông tin gì để chẩn đoán và sửa các vấn đề. Việc dùng công cụ

có thể giúp bạn truy vấn các đầu ra bên ngoài này theo những cách bạn không thể lường trước trở nên ngày càng quan trọng. Tôi gợi ý bạn xem xét các công cụ như Lightstep và Honeycomb vốn được xây dựng với tư duy này.

Finally, as your system grows in scale, it becomes more and more likely that there will always be an error somewhere. But in a large-scale system, one machine having a problem isn't necessarily cause for everyone to jump into action, nor should this necessarily result in a rude awakening for anyone at 3 a.m. Using “test in production” techniques like parallel runs and synthetic transactions can be much more effective for picking up problems that might actually be impacting the end users.

Cuối cùng, khi hệ thống của bạn tăng quy mô, ngày càng có nhiều khả năng là luôn có một lỗi ở đâu đó. Nhưng trong một hệ thống quy mô lớn, việc một máy gặp vấn đề không nhất thiết là có để mọi người nhảy vào hành động, cũng không nhất thiết phải dẫn đến việc đánh thức ai đó một cách phũ phàng lúc 3 giờ sáng. Việc dùng các kỹ thuật “kiểm thử trong production” như parallel run và giao dịch tổng hợp (synthetic transaction) có thể hiệu quả hơn nhiều trong việc phát hiện các vấn đề thực sự đang ảnh hưởng đến người dùng cuối.

Security — Bảo mật

Microservices give us more opportunity to defend our application in depth, which in turn can lead to more secure systems. On the other hand, they often have a larger attack surface area, which can leave us more exposed to attack! This balancing act is why it's so important to have a holistic understanding of **security**, something I shared in Chapter 11 .

Microservice cho chúng ta nhiều cơ hội hơn để phòng thủ ứng dụng theo chiều sâu, từ đó có thể dẫn đến những hệ thống an toàn hơn. Mặt khác, chúng thường có bề mặt tấn công (attack surface) lớn hơn, khiến chúng ta phơi mình trước nguy cơ tấn công nhiều hơn! Việc cân bằng này là lý do tại sao có một sự hiểu biết toàn diện về bảo mật (security) lại quan trọng đến vậy, điều mà tôi đã chia sẻ ở Chương 11.

With more information flowing over networks, it becomes more important to consider the protection of data in transit. The increased number of moving parts also means that automation is a vital part of microservice security. Managing patching, certificates, and secrets using manual, error-prone processes can leave you vulnerable to attack. So use tools that allow for ease of automation.

Với ngày càng nhiều thông tin chảy qua mạng, việc cân nhắc bảo vệ dữ liệu khi truyền (data in transit) trở nên quan trọng hơn. Số lượng thành phần chuyển động tăng lên cũng có nghĩa là tự động hóa là một phần thiết yếu của bảo mật microservice. Quản lý việc vá lỗi, chứng chỉ, và bí mật (secret) bằng các quy trình thủ công, dễ sai sót có thể khiến bạn dễ bị tấn công. Vì vậy hãy dùng các công cụ cho phép tự động hóa dễ dàng.

JWTs can be used to decentralize **authorization** logic in a way that also avoids the need for additional round trips. This can help protect you from issues like the confused deputy problem, while at the same time ensuring your microservice can run in a more independent fashion.

JWT có thể được dùng để phi tập trung hóa logic phân quyền theo cách cũng tránh được nhu cầu phải có thêm các vòng đi-về (round trip). Điều này có thể giúp bảo vệ bạn khỏi các vấn đề như confused deputy, đồng thời đảm bảo microservice của bạn có thể chạy một cách độc lập hơn.

Finally, increasing numbers of people are adopting a zero-trust mindset. With **zero trust**, you operate as though your system has already been compromised and you need to build your

microservices accordingly. It may seem like a paranoid stance, but I’m increasingly of the opinion that embracing this principle can actually simplify how you view the security of your system.

Cuối cùng, ngày càng nhiều người đang đón nhận tư duy zero trust. Với zero trust, bạn vận hành như thể hệ thống của bạn đã bị xâm phạm và bạn cần xây dựng các microservice của mình tương ứng. Điều này nghe có vẻ như một lập trường hoang tưởng, nhưng tôi ngày càng có quan điểm rằng đón nhận nguyên tắc này thực ra có thể đơn giản hóa cách bạn nhìn nhận tính bảo mật của hệ thống.

Resiliency — Khả năng phục hồi (Resiliency)

In Chapter 12, we looked at **resiliency** as a whole, and I shared with you the four key concepts that need to be considered when thinking about resiliency:

Ở Chương 12, chúng ta đã xem xét khả năng phục hồi (resiliency) một cách tổng thể, và tôi đã chia sẻ với bạn bốn khái niệm then chốt cần cân nhắc khi suy nghĩ về khả năng phục hồi:

Robustness The ability to absorb expected perturbation

Độ vững chắc (Robustness) Khả năng hấp thụ các nhiễu loạn đã được dự kiến

Rebound The ability to recover after a traumatic event

Phục hồi nảy lại (Rebound) Khả năng hồi phục sau một sự kiện gây tổn thương

Graceful extensibility How well we deal with a situation that is unexpected

Khả năng mở rộng có kiểm soát (Graceful extensibility) Mức độ chúng ta xử lý tốt một tình huống bất ngờ

Sustained adaptability The ability to continually adapt to changing environments, stakeholders, and demands

Khả năng thích ứng bền bỉ (Sustained adaptability) Khả năng liên tục thích ứng với các môi trường, bên liên quan, và nhu cầu thay đổi

Taken as a whole, microservice architectures can help with some of these things (namely robustness and rebound), but as we see from this list, that by itself doesn’t make you resilient. Much of being resilient is about team and organizational behavior and culture.

Xét tổng thể, kiến trúc microservice có thể giúp ích với một số điều này (cụ thể là độ vững chắc và phục hồi nảy lại), nhưng như chúng ta thấy từ danh sách này, bản thân điều đó không khiến bạn có khả năng phục hồi. Phần lớn của việc có khả năng phục hồi nằm ở hành vi và văn hóa của đội ngũ và tổ chức.

Fundamentally, you have to explicitly do things to make your application more robust. Robustness isn’t free—microservices give us the option to improve the resiliency of our systems, but we have to make that choice. For example, we have to understand that any call we make to another microservice might fail, that machines might die, and that bad things happen to good network packets. Stability patterns like bulkheads, circuit breakers, and properly configured time-outs can help greatly.

Về cơ bản, bạn phải làm những việc một cách tường minh để khiến ứng dụng của mình vững chắc hơn. Độ vững chắc không tự nhiên mà có—microservice cho chúng ta lựa chọn cải thiện khả năng phục hồi của hệ thống, nhưng chúng ta phải đưa ra lựa chọn đó. Ví dụ, chúng ta phải hiểu rằng bất kỳ lời gọi nào ta thực hiện đến một microservice khác đều có thể thất bại, rằng các máy có thể chết, và rằng những điều tồi tệ xảy ra với cả những gói

tin mạng vô tội. Các pattern về ổn định như bulkhead, circuit breaker, và time-out được cấu hình đúng cách có thể giúp ích rất nhiều.

Scaling — Mở rộng (Scaling)

Microservices give us a number of different ways to scale an application. In Chapter 13, I explore the four axes of **scaling**, which I share below:

Microservice cho chúng ta một số cách khác nhau để mở rộng (scale) một ứng dụng. Ở Chương 13, tôi khám phá bốn trục của việc mở rộng, mà tôi chia sẻ dưới đây:

Vertical scaling In a nutshell, this means getting a bigger machine.

Mở rộng dọc (Vertical scaling) Nói ngắn gọn, điều này nghĩa là dùng một cỗ máy lớn hơn.

Horizontal duplication Having multiple things capable of doing the same work.

Nhân bản theo chiều ngang (Horizontal duplication) Có nhiều thứ có khả năng làm cùng một công việc.

Data partitioning Dividing work based on some attribute of the data, e.g., customer group.

Phân vùng dữ liệu (Data partitioning) Chia công việc dựa trên một thuộc tính nào đó của dữ liệu, ví dụ: nhóm khách hàng.

Functional decomposition Separation of work based on the type, e.g., microservice decomposition.

Phân rã theo chức năng (Functional decomposition) Tách công việc dựa trên loại, ví dụ: phân rã microservice.

With scaling, do the easy stuff first. Vertical scaling and horizontal duplication are quick and easy compared with the other two axes presented here. If they work, great! If not, you can look at the other mechanisms. It's common as well to mix the different types of scaling—partitioning your traffic based on customers, for example, and then having each partition scaled horizontally.

Với việc mở rộng, hãy làm những thứ dễ trước. Mở rộng dọc và nhân bản theo chiều ngang nhanh và dễ so với hai trục còn lại được trình bày ở đây. Nếu chúng có tác dụng, tuyệt vời! Nếu không, bạn có thể xem xét các cơ chế khác. Việc trộn các kiểu mở rộng khác nhau cũng là chuyện phổ biến—chẳng hạn, phân vùng lưu lượng của bạn dựa trên khách hàng, rồi cho mỗi phân vùng được mở rộng theo chiều ngang.

User Interfaces — Giao diện người dùng

All too often, the **user interface** is an afterthought when it comes to system decomposition—we break apart our microservices but leave a monolithic user interface. This in turn leads to the problems of having separate **frontend** and **backend** teams. Instead, we want stream-aligned teams, where one team owns all the functionality associated with an end-to-end slice of user functionality. To make that change happen and get rid of siloed frontend and backend teams, we need to break apart our user interfaces.

Quá thường xuyên, giao diện người dùng (UI) bị xem là chuyện tính sau khi nói đến phân rã hệ thống—chúng ta tách các microservice ra nhưng để lại một giao diện người dùng nguyên khối. Điều này lại dẫn đến những vấn đề khi có các đội frontend và backend tách biệt. Thay vào đó, chúng ta muốn các đội stream-aligned, trong đó một đội sở hữu toàn bộ chức năng gắn với một lát cắt chức năng người dùng từ đầu đến cuối (end-to-end). Để biến

sự thay đổi đó thành hiện thực và xóa bỏ các đội frontend và backend bị tách rời thành các silo, chúng ta cần tách các giao diện người dùng của mình ra.

In Chapter 14, I share how we can use micro frontends to deliver decomposed user interfaces using single-page app frameworks like React. User interfaces often face problems in terms of the number of calls they need to make, or because they need to perform call aggregation and filtering to suit mobile devices. The **backend for frontend** (BFF) pattern can help provide server-side aggregation and filtering in these situations, although if you are able to use **GraphQL**, you may be able to sidestep the use of BFFs.

Ở Chương 14, tôi chia sẻ cách chúng ta có thể dùng micro frontend để mang lại các giao diện người dùng đã được phân rã bằng các framework single-page app như React. Giao diện người dùng thường gặp vấn đề về số lượng lời gọi mà chúng cần thực hiện, hoặc vì chúng cần thực hiện tổng hợp và lọc lời gọi để phù hợp với thiết bị di động. Pattern backend for frontend (BFF) có thể giúp cung cấp việc tổng hợp và lọc phía máy chủ trong những tình huống này, mặc dù nếu bạn có thể dùng GraphQL, bạn có thể né được việc dùng BFF.

Organization — Tổ chức

In Chapter 15, we looked at the shift away from horizontally aligned, siloed teams toward team structures that are organized around end-to-end slices of functionality. These stream-aligned teams, as the authors of *Team Topologies* describe them, are supported by enabling teams, as Figure A-3 shows. Enabling teams will often have a specific cross-cutting focus, such as focusing on security or usability, and support the stream-aligned teams in these aspects.

Ở Chương 15, chúng ta đã xem xét sự dịch chuyển từ các đội bị tách thành silo, căn chỉnh theo chiều ngang, sang các cấu trúc đội được tổ chức xoay quanh các lát cắt chức năng từ đầu đến cuối. Các đội stream-aligned này, như cách các tác giả của *Team Topologies* mô tả, được hỗ trợ bởi các enabling team, như Hình A-3 cho thấy. Các enabling team thường có một trọng tâm cắt ngang cụ thể, chẳng hạn như tập trung vào bảo mật hoặc tính khả dụng (usability), và hỗ trợ các đội stream-aligned ở những khía cạnh này.

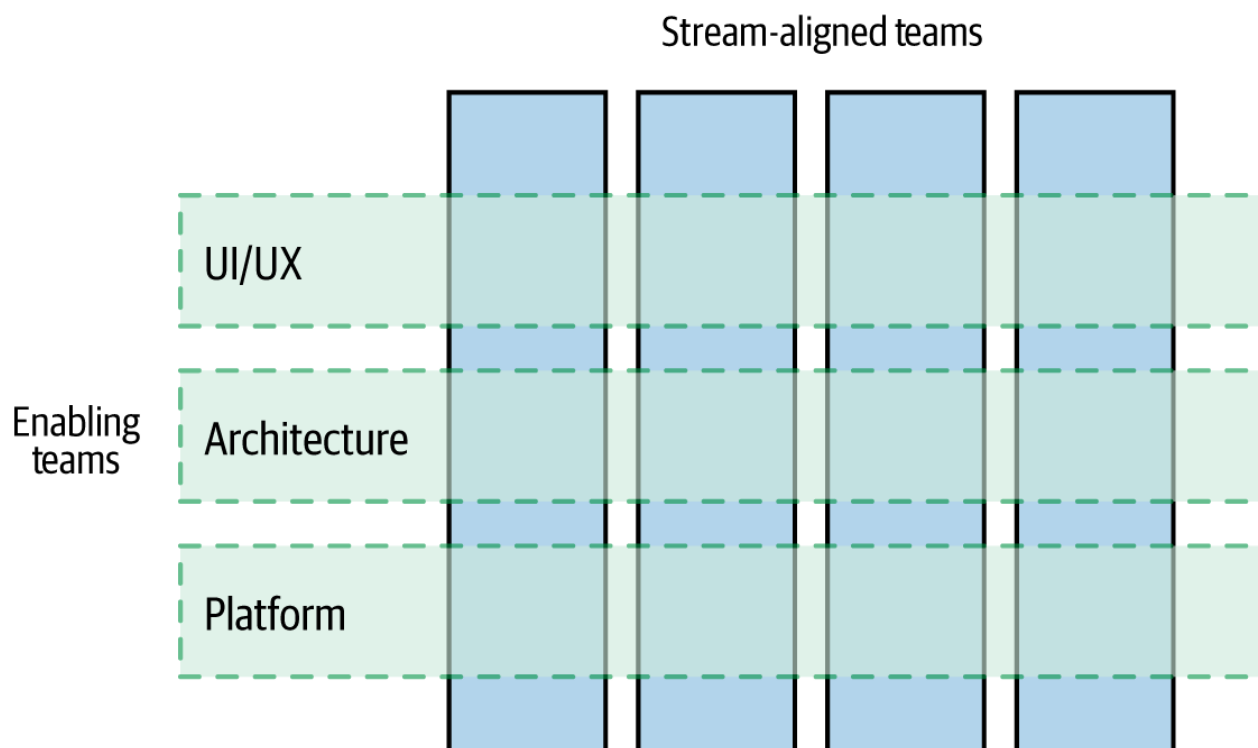


Figure A-3. Enabling teams support multiple stream-aligned teams

Hình A-3. Các enabling team hỗ trợ nhiều đội stream-aligned

Making these stream-aligned teams as autonomous as possible means that they need self-service tools to avoid having to constantly ask other teams to do things for them. As part of this, a platform can be incredibly useful. It's important, though, that we see a platform as a type of **paved road** — that is, as something that makes it easy to do the right thing, without requiring that it must be used. Making a platform optional ensures that making the platform easy to use remains a key focus of the team owning it, while also allowing teams to make a different choice when warranted.

Làm cho các đội stream-aligned này tự chủ nhất có thể nghĩa là chúng cần các công cụ tự phục vụ (self-service) để tránh phải liên tục nhờ các đội khác làm việc giúp mình. Như một phần của điều này, một platform có thể cực kỳ hữu ích. Tuy nhiên, điều quan trọng là chúng ta phải xem một platform như một dạng paved road—tức là một thứ giúp dễ dàng làm điều đúng đắn, mà không bắt buộc phải dùng nó. Làm cho một platform là tùy chọn đảm bảo rằng việc khiến platform dễ dùng vẫn là một trọng tâm then chốt của đội sở hữu nó, đồng thời cũng cho phép các đội đưa ra lựa chọn khác khi cần thiết.

Architecture — Kiến trúc

It's important that we don't see the architecture of our system as fixed and unchanging. Instead, we should view our system architecture as something that should be able to continually change as circumstances require. For you to get the most out of microservice architectures, moving to an organization where more autonomy is pushed into teams means that responsibility for the technical vision needs to become a more collaborative process. The architect sitting in an ivory tower will either be a significant blocker to a microservice architecture or else will become an ignored irrelevance.

Điều quan trọng là chúng ta đừng xem kiến trúc của hệ thống mình là cố định và bất biến. Thay vào đó, chúng ta nên nhìn kiến trúc hệ thống của mình như một thứ có thể liên tục thay đổi khi hoàn cảnh đòi hỏi. Để bạn tận dụng tối đa các kiến trúc microservice, việc chuyển sang một tổ chức nơi nhiều quyền tự chủ được đẩy vào các đội nghĩa là trách nhiệm với tầm nhìn kỹ thuật cần trở thành một quy trình mang tính cộng tác hơn. Vị kiến trúc sư ngồi trong tháp ngà sẽ hoặc là một vật cản đáng kể đối với một kiến trúc microservice, hoặc sẽ trở thành một sự hiện diện vô nghĩa bị phớt lờ.

The role of shepherding the architecture of a system can be entirely distributed into the teams, and at a certain level of scale this can work well. However, as the organization grows, having people with dedicated time to look across the system as a whole becomes essential. Call them principal engineers, technical product owners, or architects, it doesn't really matter—the role that they need to play is the same. As I showed in Chapter 16, architects in a microservice organization need to support teams, connect people, spot patterns emerging, and spend enough time embedded with teams to see how the big picture stuff plays out in reality.

Vai trò dẫn dắt kiến trúc của một hệ thống có thể được phân tán hoàn toàn vào các đội, và ở một mức quy mô nhất định điều này có thể hoạt động tốt. Tuy nhiên, khi tổ chức lớn lên, việc có những người dành thời gian chuyên trách để nhìn xuyên suốt toàn bộ hệ thống trở nên thiết yếu. Gọi họ là *principal engineer*, *technical product owner*, hay kiến trúc sư cũng không thực sự quan trọng—vai trò mà họ cần đảm nhận là như nhau. Như tôi đã trình bày ở Chương 16, kiến trúc sư trong một tổ chức microservice cần hỗ trợ các đội, kết nối mọi người, phát hiện các pattern đang nổi lên, và dành đủ thời gian gắn bó với các đội để thấy được những thứ thuộc về bức tranh toàn cảnh diễn ra thế nào trong thực tế.

Further Reading

Throughout this book, I have referenced many papers, presentations, and books that I have learned a great deal from, and I have made sure to list them in Bibliography. Since the first edition, though, the two books that have had the biggest impact on my thinking, and as a result have been referenced extensively in this new edition, are worth calling out here as “must reads.” The first is *Accelerate* by Nicole Forsgren, Jez Humble, and Gene Kim. The second is *Team Topologies* by Matthew Skelton and Manuel Pais. These two books are, in my opinion, the two most useful books on software development written in the last ten years, and I consider them essential reading whether you are into microservices or not.

As a companion to this book, my own *Monolith to Microservices* goes into more depth on how to break apart existing system architectures.

Looking Forward

In the future, I suspect the technology that makes microservices easier to build and run will continue to improve, and I am especially keen to see what the second- (and third-) generation FaaS products look like. Whether or not FaaS takes off, Kubernetes will become even more widespread, even if it will increasingly be hidden behind more developer-friendly abstraction layers. Kubernetes has won, but in a way that I think most application developers shouldn't have to worry about. I remain very interested in seeing how Wasm changes how we think about deployments, and I still have a suspicion that unikernels may have a second coming as well.

Since this book's first edition, microservices have well and truly gone mainstream in a way that has surprised me, and that has concerned me too. It seems that a lot of people adopting microservices are

doing so more because everyone else is doing it, rather than microservices being right for them. As such, I fully expect us to hear more horror stories about failed microservice implementations, which I will digest with relish to see what can be learned. I also fully expect a wider industry backlash against microservices at some point when the microservice disaster case studies reach critical mass. Applying critical thinking to work out what approach makes the most sense in any given situation isn't very sexy or marketable, and I don't expect that will change in a world where selling technology is more profitable than selling ideas.

I don't mean to sound pessimistic! We are, as an industry, still very young, and we are still finding our place in the world. The amount of energy and ingenuity that gets put into software development continues to keep me interested, and I can't wait to see what the next decade brings.

Final Words

Microservice architectures give you more options, and more decisions to make. Making decisions in this world is a far more common activity than in simpler, monolithic systems. You won't get all of these decisions right, I can guarantee that. So, knowing you are going to get some things wrong, what are your options? Well, I would suggest finding ways to make each decision small in scope; that way, if you get one wrong, you impact only a small part of your system. Learn to embrace the concept of evolutionary architecture, in which your system bends and flexes and changes over time as you learn new things. Think not of big-bang rewrites, but instead of a series of changes made to your system over time to keep it supple.

I hope by now I've shared with you enough information and experiences to help you decide whether microservices are for you. If they are, I hope you think of this as a journey, not a destination. Go incrementally. Break your system apart piece by piece, learning as you go. And get used to it: in many ways, the discipline to continually change and evolve our systems is a far more important lesson to learn than any other I have shared with you through this book. Change is inevitable. Embrace it.